RT-Thread RTOS

Generated on Sat Nov 7 2026 20:26:19 for RT-Thread RTOS by Doxygen 1.18.0

Sat Nov 7 2026 20:26:19

1 RT-Thread User Guide 1

1.1 Introduction . . . 1

1.1.1 RT-Thread Overview . . . 1

1.1.2 License Agreement . . . 2

1.1.3 RT-Thread Frame . . . 2

1.2 Start Guide . . . 3

1.2.1 Preparation . . . 3

1.2.2 First acquaintance with RT-Thread . . . 4

1.2.3 User Entry Code . . . 8

1.2.4 Example of a Marquee . . . 8

1.2.5 Other Examples . . . 9

1.2.6 Frequently Asked Question . . . 10

1.2.7 Keil MDK Installation . . . 10

1.3 Kernel . . . 16

1.3.1 Kernel Basics . . . 16

1.3.1.1 Introduction to RT-Thread Kernel . . . 17

1.3.1.2 RT-Thread Startup Process . . . 20

1.3.1.3 RT-Thread Program Memory Distribution . . . 21

1.3.1.4 RT-Thread Automatic Initialization Mechanism . . . 23

1.3.1.5 RT-Thread Kernel Configuration Example . . . 24

1.3.1.6 Common Macro Definition Description . . . 26

1.3.2 Object Management . . . 27

1.3.2.1 Static and Dynamic Objects . . . 27

1.3.2.2 Object Management Structure . . . 28

1.3.2.3 Object Control Block . . . 29

1.3.2.4 Object Management APIs . . . 30

1.3.3 Thread Management . . . 31

1.3.3.1 Thread Management Features . . . 32

1.3.3.2 Working Mechanism of Thread . . . 32

1.3.3.3 Thread Management . . . 38

1.3.3.4 Thread Application Sample . . . 42

1.3.4 Clock & Timer Management . . . 46

1.3.4.1 Clock Tick(OS Tick) . . . 46

1.3.4.2 Timer Management . . . 47

1.3.4.3 Timer Application Sample . . . 55

1.3.4.4 High Precision Delay . . . 57

1.3.5 Inter-thread Synchronization . . . 57

1.3.5.1 Semaphores . . . 58

1.3.5.2 Mutex . . . 69

1.3.5.3 Event . . . 78

1.3.6 Inter-thread Communication . . . 85

1.3.6.1 Mailbox . . . 85

1.3.6.2 Message Queue . . . 92
1.3.6.3 Signal . . . 101
1.3.7 Memory Management . . . 106
1.3.7.1 Memory Management Functional Features . . . 106
1.3.7.2 Memory Heap Management . . . 107
1.3.7.3 Memory Pool . . . 116
1.3.8 Interrupt Management . . . 123
1.3.8.1 Cortex-M CPU Architecture Foundation . . . 124
1.3.8.2 RT-Thread Interruption Mechanism . . . 127
1.3.8.3 RT-Thread Interrupt Management Interface . . . 133
1.3.8.4 Interrupt and Polling . . . 138
1.3.8.5 Global Interrupt Switch Usage Example . . . 139
1.3.9 Kernel Porting . . . 140
1.3.9.1 CPU Architecture Porting . . . 140
1.3.9.2 BSP Porting . . . 150
1.3.10 QEMU virt64 AArch64 SMP Boot Flow . . . 150
1.3.10.1 QEMU virt64 AArch64 SMP Boot Flow . . . 150
1.4 Components . . . 153
1.4.1 Virtual File System . . . 154
1.4.1.1 An Introduction to DFS . . . 154
1.4.1.2 Mount Management . . . 156
1.4.1.3 Document Management . . . 159
1.4.1.4 Directory management . . . 163
1.4.1.5 DFS Configuration Options . . . 166
1.4.1.6 DFS Application Example . . . 169
1.4.1.7 FAQ . . . 174
1.4.2 Device Driver . . . 176
1.4.2.1 I/O Device Framework . . . 179
1.4.2.2 PIN Device . . . 192
1.4.2.3 Pin control (pinctrl) . . . 203
1.4.2.4 UART . . . 206
1.4.2.5 UART device model (DM) . . . 217
1.4.2.6 UART early console . . . 218
1.4.2.7 ADC Device . . . 220
1.4.2.8 I2C Bus Device . . . 224
1.4.2.9 I2C device model (DM) . . . 229
1.4.2.10 SPI device . . . 234
1.4.2.11 SPI device model (DM) . . . 247
1.4.2.12 PWM Device . . . 252
1.4.2.13 RTC Device . . . 257
1.4.2.14 Clock Time Subsystem . . . 262
1.4.2.15 HWTIMER Device . . . 280

1.4.2.16 PTP clock subsystem . . . 286
1.4.2.17 TEE subsystem . . . 291
1.4.2.18 WATCHDOG Device . . . 297
1.4.2.19 WLAN Device . . . 300
1.4.2.20 Sensor Device . . . 308
1.4.2.21 Audio Device . . . 315
1.4.2.22 Open Firmware (OFW) . . . 322
1.4.2.23 OFW nodes and properties . . . 324
1.4.2.24 OFW boot and FDT . . . 326
1.4.2.25 OFW addressing and MMIO . . . 328
1.4.2.26 OFW interrupts . . . 329
1.4.2.27 OFW raw FDT helpers . . . 331
1.4.2.28 OFW platform enumeration . . . 332
1.4.2.29 Devicetree Compiler . . . 333
1.4.2.30 ATA / AHCI . . . 336
1.4.2.31 Block layer (`blk`) . . . 339
1.4.2.32 Physical disk (`rt_blk_disk`) . . . 344
1.4.2.33 Disk partition probing . . . 348
1.4.2.34 Hardware cache (hwcache) . . . 351
1.4.2.35 DVFS . . . 355
1.4.2.36 CAN device . . . 358
1.4.2.37 CAN FD DLC helpers (`can_dm`) . . . 363
1.4.2.38 Clock framework (CLK) . . . 365
1.4.2.39 Fixed clock provider . . . 370
1.4.2.40 Bus and driver binding (core) . . . 371
1.4.2.41 Device model helpers (dm.h) . . . 374
1.4.2.42 System power off and reboot (core) . . . 377
1.4.2.43 DMA engine . . . 379
1.4.2.44 DMA memory pools . . . 380
1.4.2.45 Graphics (DM) . . . 382
1.4.2.46 Backlight devices . . . 387
1.4.2.47 Simple framebuffer (`simple-framebuffer`) . . . 388
1.4.2.48 Boot Logo . . . 389
1.4.2.49 Hardware spinlock . . . 392
1.4.2.50 Industrial I/O (IIO) . . . 396
1.4.2.51 IIO device model (DM) . . . 397
1.4.2.52 Input subsystem . . . 401
1.4.2.53 Input device model (DM) . . . 402
1.4.2.54 Input userspace API (UAPI) . . . 407
1.4.2.55 Input touch bridge . . . 409
1.4.2.56 Input power keys . . . 412
1.4.2.57 LED subsystem . . . 414

1.4.2.58 LED device model (DM) . . . 416
1.4.2.59 Mailbox subsystem . . . 419
1.4.2.60 Mailbox device model (DM) . . . 422
1.4.2.61 NUMA helpers . . . 425
1.4.2.62 NVMe subsystem . . . 428
1.4.2.63 NVMe device model (DM) . . . 431
1.4.2.64 NVMEM subsystem . . . 433
1.4.2.65 NVMEM device model (DM) . . . 435
1.4.2.66 PCI / PCIe . . . 436
1.4.2.67 PCI enumeration and drivers . . . 438
1.4.2.68 PCI config space access . . . 442
1.4.2.69 PCI host controllers . . . 443
1.4.2.70 PCI INTx interrupts . . . 445
1.4.2.71 PCI power management . . . 446
1.4.2.72 PCI MSI and MSI-X . . . 447
1.4.2.73 PCI and device tree . . . 449
1.4.2.74 PCI bridge function driver . . . 450
1.4.2.75 PCI endpoint (EP) mode . . . 452
1.4.2.76 Generic PHY (Phye) . . . 453
1.4.2.77 Phye core API . . . 454
1.4.2.78 Phye and device tree . . . 456
1.4.2.79 Phye provider driver . . . 457
1.4.2.80 Phye consumer integration . . . 458
1.4.2.81 Generic USB PHY . . . 460
1.4.2.82 PIC overview . . . 461
1.4.2.83 PIC virtual IRQ domain . . . 463
1.4.2.84 PIC core framework . . . 464
1.4.2.85 PIC cascade and routing . . . 466
1.4.2.86 PIC and device tree . . . 468
1.4.2.87 PIC and PCI MSI . . . 470
1.4.2.88 PIC platform examples . . . 472
1.4.2.89 Platform bus . . . 473
1.4.2.90 Power subsystem overview . . . 479
1.4.2.91 Power domain . . . 492
1.4.2.92 Regulator . . . 496
1.4.2.93 Reset controller . . . 500
1.4.2.94 RPMsg . . . 504
1.4.2.95 SCMI firmware protocol . . . 513
1.4.2.96 SCMI agent transports . . . 517
1.4.2.97 SCSI host and commands . . . 519
1.4.2.98 SDIO / MMC core . . . 523
1.4.2.99 SDIO device model . . . 525

1.4.2.100 SDIO supplies (regulator) . . . 527
1.4.2.101 SDHCI host controller . . . 528
1.4.2.102 System controller (Syscon) . . . 531
1.4.2.103 Thermal management . . . 535
1.4.2.104 Thermal cooling devices . . . 539
1.4.2.105 UFS host . . . 541
1.4.3 FinSH Console . . . 544
1.4.3.1 Introduction of FinSH . . . 544
1.4.3.2 FinSH Built-in Commands . . . 545
1.4.3.3 Custom FinSH Command . . . 549
1.4.3.4 FinSH Function Configuration . . . 550
1.4.3.5 FinSH Application Examples . . . 551
1.4.3.6 FinSH Porting . . . 552
1.4.4 utest Framework . . . 552
1.4.4.1 utest Introduction . . . 552
1.4.4.2 utest API . . . 553
1.4.4.3 Configuration Enable . . . 555
1.4.4.4 Application Paradigm . . . 555
1.4.4.5 Requirements for running test cases . . . 556
1.4.4.6 Running Test Cases . . . 557
1.4.4.7 Test Case Run Process . . . 558
1.4.4.8 NOTE . . . 559
1.4.4.9 How-to add utest cases into RT-Thread for your module . . . 559
1.4.5 Dynamic Module: dlmodule . . . 560
1.4.5.1 Introduction . . . 560
1.4.5.2 Using Dynamic Module . . . 560
1.4.5.3 APIs of Dynamic Module . . . 562
1.4.5.4 Libdl API of POSIX Standard . . . 564
1.4.5.5 FAQs . . . 565
1.4.6 Socket Abstraction Layer: SAL . . . 565
1.4.6.1 SAL Introduction . . . 565
1.4.6.2 Initialization . . . 570
1.4.6.3 BSD Socket API Introduction . . . 570
1.4.6.4 Network Protocol Stack Access . . . 576
1.4.7 AT Commands . . . 579
1.4.7.1 Introduction to AT Commands . . . 579
1.4.7.2 Introduction to AT Components . . . 580
1.4.7.3 AT Server . . . 580
1.4.7.4 AT Client . . . 585
1.4.7.5 FAQs . . . 594
1.4.8 POSIX Interface . . . 595
1.4.8.1 Introduction to Pthreads . . . 595

1.4.8.2 Thread . . . 596
1.4.8.3 Mutex . . . 601
1.4.8.4 Conditional Variable . . . 605
1.4.8.5 Read-write Lock . . . 609
1.4.8.6 Barrier . . . 614
1.4.8.7 Semaphore . . . 616
1.4.8.8 Message Queue . . . 622
1.4.8.9 Thread Advanced Programming . . . 626
1.4.9 Ulog Log . . . 637
1.4.9.1 Ulog Introduction . . . 637
1.4.9.2 Log Initialization . . . 640
1.4.9.3 Log Output API . . . 640
1.4.9.4 ULog Usage Example . . . 641
1.4.9.5 Log Advanced Features . . . 644
1.4.9.6 Common Problems . . . 657
1.4.10 Power Management: PM . . . 658
1.4.10.1 Introduction of PM Components . . . 658
1.4.10.2 PM Framework . . . 660
1.4.10.3 Low Power State and Mode . . . 661
1.4.10.4 The calling process . . . 662
1.4.10.5 Introduction to APIs . . . 663
1.4.10.6 Instruction for Use . . . 664
1.4.10.7 Migration instructions . . . 664
1.4.10.8 MSH Commands . . . 666
1.4.10.9 Common problems and debugging methods . . . 667
1.4.11 Network Framework . . . 667
1.4.11.1 TCP/IP Introduction to Network Protocols . . . 667
1.4.11.2 Introduction to the Network Framework of RT-Thread . . . 669
1.4.11.3 Network Framework Workflow . . . 671
1.4.11.4 Network Socket Programming . . . 672
1.4.11.5 Network Function Configuration . . . 678
1.4.11.6 Network Application Example . . . 679
1.5 Tool . . . 686
1.5.1 Env . . . 686
1.5.1.1 Main Features . . . 686
1.5.1.2 User Guide . . . 687
1.5.1.3 Archive Information . . . 687
1.5.2 SCons . . . 697
1.5.2.1 Introduction to SCons . . . 697
1.5.2.2 Basic Functions of SCons . . . 697
1.5.2.3 SCons Advanced . . . 699
1.5.2.4 SConscript Examples . . . 701

1.5.2.5 Manage Projects with SCons . . . 704
1.6 Contribution . . . 709
1.6.1 Contribution Guide . . . 709
1.6.2 RT-Thread Coding Style . . . 709
1.6.2.1 1. Directory Naming . . . 710
1.6.2.2 2. File Naming . . . 710
1.6.2.3 3. Header Files . . . 710
1.6.2.4 4. Header File Comments . . . 710
1.6.2.5 5. Structure Defines . . . 710
1.6.2.6 6. Macros . . . 710
1.6.2.7 7. Function Naming and Declaration . . . 710
1.6.2.8 8. Commenting . . . 711
1.6.2.9 9. Indent . . . 711
1.6.2.10 10. Braces and Spaces . . . 711
1.6.2.11 11. trace, log Information . . . 712
1.6.2.12 12. Functions . . . 712
1.6.2.13 13. Objects . . . 712
1.6.2.14 14. Format Code . . . 713
1.6.3 BSP Contribution Guide . . . 713
1.6.3.1 BSP Contribution Guide . . . 713
1.6.4 How to write doxygen documentation for RT-Thread . . . 715
1.6.4.1 General Rules on writing API documentation . . . 715
1.6.4.2 Detailed Rules on writing API documentation . . . 715
1.6.4.3 Build & Run . . . 716
1.6.4.4 How to use groups in doxygen documentation. . . . 717
1.6.4.5 How to write doxygen documentation for macro. . . . 717
1.6.4.6 How to write doxygen documentation for structure. . . . 717
1.6.4.7 How to write doxygen documentation for union. . . . 718
1.6.4.8 How to write doxygen documentation for enumeration. . . . 718
1.6.4.9 How to write doxygen documentation for typedef. . . . 718
1.6.4.10 How to write doxygen documentation for function. . . . 718
1.6.5 RT-Thread Versioning Policy and Implementation Rules . . . 719
1.6.5.1 RT-Thread Versioning Policy and Implementation Rules . . . 719
1.6.6 RT-Thread Release Process . . . 724
1.6.6.1 RT-Thread Release Process . . . 724

2 libcpu Kconfig ARM **733**

2.1 . . . 733
2.2 . . . 733
2.3 . . . 733
2.4 . . . 733
2.5 . . . 734

2.6 . . . 734

3 Topic Index **735**

3.1 RT-Thread API Guide . . . 735

4 Data Structure Index **737**

4.1 Data Structures . . . 737

5 File Index **743**

5.1 File List . . . 743

6 Topic Documentation **745**

6.1 Basic Definitions . . . 745

6.1.1 Detailed Description . . . 745

6.1.2 Macro Definition Documentation . . . 745

6.1.2.1 RT_VERSION_MAJOR . . . 745

6.1.2.2 RT_VERSION_MINOR . . . 745

6.1.2.3 RT_VERSION_PATCH . . . 745

6.1.2.4 RT_IS_ALIGN . . . 745

6.1.2.5 RT_ALIGN . . . 746

6.1.2.6 RT_ALIGN_DOWN . . . 746

6.2 Kernel . . . 746

6.2.1 Detailed Description . . . 746

6.2.2 Function Documentation . . . 747

6.2.2.1 rt_interrupt_enter() . . . 747

6.2.2.2 rt_interrupt_leave() . . . 747

6.2.2.3 rt_interrupt_get_nest() . . . 747

6.2.3 Object Management . . . 747

6.2.3.1 Detailed Description . . . 748

6.2.3.2 Macro Definition Documentation . . . 748

6.2.3.3 Typedef Documentation . . . 749

6.2.3.4 Enumeration Type Documentation . . . 749

6.2.3.5 Function Documentation . . . 750

6.2.4 Thread Management . . . 755

6.2.4.1 Detailed Description . . . 756

6.2.4.2 Macro Definition Documentation . . . 756

6.2.4.3 Typedef Documentation . . . 758

6.2.4.4 Function Documentation . . . 758

6.2.5 Clock and Timer Management . . . 774

6.2.5.1 Detailed Description . . . 775

6.2.5.2 Macro Definition Documentation . . . 775

6.2.5.3 Typedef Documentation . . . 776

6.2.5.4 Function Documentation . . . 776

6.2.6 Inter-Thread Communication . . . 781

6.2.6.1 Detailed Description . . . 782
6.2.6.2 Macro Definition Documentation . . . 782
6.2.6.3 Function Documentation . . . 783
6.2.6.4 Signal . . . 789
6.2.6.5 Semaphore . . . 791
6.2.6.6 Mutex . . . 795
6.2.6.7 Event . . . 800
6.2.6.8 MailBox . . . 804
6.2.6.9 Message Queue . . . 809
6.2.7 Memory Management . . . 814
6.2.7.1 Detailed Description . . . 815
6.2.7.2 Function Documentation . . . 815
6.2.8 Runtime Trace and Record . . . 821
6.2.8.1 Detailed Description . . . 821
6.2.8.2 Function Documentation . . . 822
6.2.9 Other useful kernel service . . . 822
6.2.9.1 Detailed Description . . . 823
6.2.9.2 Macro Definition Documentation . . . 823
6.2.9.3 Function Documentation . . . 827
6.2.10 Error Code . . . 835
6.2.11 System Initialization . . . 835
6.2.11.1 Detailed Description . . . 835
6.2.11.2 Function Documentation . . . 836
6.3 Components . . . 836
6.3.1 Detailed Description . . . 836
6.3.2 Device Virtual File System . . . 836
6.3.2.1 Detailed Description . . . 837
6.3.2.2 Function Documentation . . . 837
6.3.2.3 File Descriptor . . . 843
6.3.2.4 File System API . . . 844
6.3.2.5 File API . . . 844
6.3.2.6 File POSIX API . . . 844
6.3.3 Device Driver . . . 854
6.3.3.1 Detailed Description . . . 856
6.3.3.2 Macro Definition Documentation . . . 856
6.3.3.3 Enumeration Type Documentation . . . 859
6.3.3.4 ADC . . . 860
6.3.3.5 Clock . . . 862
6.3.3.6 DAC . . . 875
6.3.3.7 Audio . . . 877
6.3.3.8 CAN Driver . . . 886
6.3.3.9 I2C . . . 891

6.3.3.10 Pin . . . 896
6.3.3.11 PWM . . . 902
6.3.3.12 RTC . . . 906
6.3.3.13 Serial . . . 909
6.3.3.14 Serial v2 . . . 912
6.3.3.15 SPI . . . 915
6.3.3.16 Touch . . . 927
6.3.3.17 blk . . . 929
6.3.4 Finsh Shell . . . 931
6.3.4.1 Detailed Description . . . 931
6.3.4.2 Macro Definition Documentation . . . 931
6.3.4.3 Function Documentation . . . 932
6.3.5 Application Module . . . 933
6.4 Hardware Related Package . . . 933
6.4.1 Detailed Description . . . 934
6.4.2 Function Documentation . . . 934
6.4.2.1 rt_hw_interrupt_disable() . . . 934
6.4.2.2 rt_hw_interrupt_enable() . . . 934
6.4.2.3 rt_hw_interrupt_init() . . . 934
6.4.2.4 rt_hw_interrupt_mask() . . . 934
6.4.2.5 rt_hw_interrupt_umask() . . . 935
6.4.2.6 rt_hw_interrupt_install() . . . 935
6.5 Doxygen Example . . . 935
6.5.1 Detailed Description . . . 935
6.5.2 Doxygen Example of Enumeration . . . 935
6.5.2.1 Detailed Description . . . 936
6.5.2.2 Enumeration Type Documentation . . . 936
6.5.3 Doxygen Example of Groups . . . 936
6.5.3.1 Detailed Description . . . 936
6.5.3.2 Macro Definition Documentation . . . 936
6.5.3.3 Enumeration Type Documentation . . . 936
6.5.4 Doxygen Example of Macro . . . 937
6.5.4.1 Detailed Description . . . 937
6.5.4.2 Macro Definition Documentation . . . 937
6.5.4.3 Group A of Macros . . . 937
6.5.4.4 Group B of Macros . . . 938
6.5.5 Doxygen Example of Structure . . . 938
6.5.5.1 Detailed Description . . . 938
6.5.6 Doxygen Example of Typedef . . . 938
6.5.6.1 Detailed Description . . . 939
6.5.6.2 Typedef Documentation . . . 939
6.5.7 Doxygen Example of Union . . . 939

6.5.7.1 Detailed Description . . . 939
6.5.8 Doxygen Example of Function . . . 939
6.5.8.1 Detailed Description . . . 939
6.5.8.2 Function Documentation . . . 939

7 Data Structure Documentation **941**
7.1 can_configure Struct Reference . . . 941
7.1.1 Detailed Description . . . 941
7.1.2 Field Documentation . . . 941
7.1.2.1 baud_rate . . . 941
7.1.2.2 msgboxsz . . . 941
7.1.2.3 sndboxnumber . . . 941
7.1.2.4 mode . . . 941
7.1.2.5 privmode . . . 941
7.1.2.6 reserved . . . 942
7.1.2.7 ticks . . . 942
7.2 dfs_seq_file Struct Reference . . . 942
7.2.1 Detailed Description . . . 942
7.3 dfs_seq_ops Struct Reference . . . 942
7.3.1 Detailed Description . . . 942
7.4 dogygen_example_struct Struct Reference . . . 942
7.4.1 Detailed Description . . . 942
7.4.2 Field Documentation . . . 942
7.4.2.1 m1 . . . 942
7.4.2.2 m2 . . . 943
7.4.2.3 m3 . . . 943
7.4.2.4 m4 . . . 943
7.4.2.5 m5 . . . 943
7.5 dogygen_example_struct_another Struct Reference . . . 943
7.5.1 Detailed Description . . . 943
7.5.2 Field Documentation . . . 943
7.5.2.1 m1 . . . 943
7.5.2.2 m2 . . . 943
7.6 dogygen_example_struct_another2 Struct Reference . . . 943
7.6.1 Detailed Description . . . 944
7.6.2 Field Documentation . . . 944
7.6.2.1 m1 . . . 944
7.6.2.2 m2 . . . 944
7.7 doxygen_example_union Union Reference . . . 944
7.7.1 Detailed Description . . . 944
7.7.2 Field Documentation . . . 945
7.7.2.1 v1 . . . 945

7.7.2.2 v2 . . . 945
7.8 rt_adc_core_ops Struct Reference . . . 945
7.8.1 Detailed Description . . . 945
7.8.2 Field Documentation . . . 945
7.8.2.1 open . . . 945
7.8.2.2 close . . . 945
7.8.2.3 session_config . . . 946
7.8.2.4 control . . . 946
7.9 rt_adc_device Struct Reference . . . 946
7.9.1 Detailed Description . . . 946
7.9.2 Field Documentation . . . 946
7.9.2.1 parent . . . 946
7.9.2.2 ops . . . 947
7.9.2.3 state . . . 947
7.9.2.4 spinlock . . . 947
7.9.2.5 default_vref_mv . . . 947
7.9.2.6 session_ctrl . . . 947
7.10 rt_adc_ops Struct Reference . . . 947
7.10.1 Detailed Description . . . 947
7.10.2 Field Documentation . . . 947
7.10.2.1 core . . . 947
7.10.2.2 sequence . . . 947
7.11 rt_adc_sequence_cfg Struct Reference . . . 947
7.11.1 Detailed Description . . . 948
7.11.2 Field Documentation . . . 948
7.11.2.1 buffer . . . 948
7.11.2.2 buffer_length . . . 948
7.11.2.3 timeout_ms . . . 948
7.12 rt_adc_sequence_ops Struct Reference . . . 948
7.12.1 Detailed Description . . . 948
7.12.2 Field Documentation . . . 948
7.12.2.1 start . . . 948
7.12.2.2 read . . . 949
7.12.2.3 stop . . . 949
7.13 rt_adc_session_ctrl Struct Reference . . . 949
7.13.1 Detailed Description . . . 949
7.13.2 Field Documentation . . . 949
7.13.2.1 channels . . . 949
7.13.2.2 channel_count . . . 950
7.13.2.3 configured . . . 950
7.14 rt_audio_buf_info Struct Reference . . . 950
7.14.1 Detailed Description . . . 950

7.14.2 Field Documentation . . . 950
7.14.2.1 buffer . . . 950
7.14.2.2 block_size . . . 950
7.14.2.3 block_count . . . 950
7.14.2.4 total_size . . . 950
7.15 rt_audio_caps Struct Reference . . . 950
7.15.1 Detailed Description . . . 951
7.15.2 Field Documentation . . . 951
7.15.2.1 main_type . . . 951
7.15.2.2 sub_type . . . 951
7.15.2.3 udata . . . 951
7.16 rt_audio_configure Struct Reference . . . 951
7.16.1 Detailed Description . . . 951
7.16.2 Field Documentation . . . 951
7.16.2.1 samplerate . . . 951
7.16.2.2 channels . . . 951
7.16.2.3 samplebits . . . 951
7.17 rt_audio_device Struct Reference . . . 951
7.17.1 Detailed Description . . . 952
7.17.2 Field Documentation . . . 952
7.17.2.1 parent . . . 952
7.17.2.2 ops . . . 952
7.17.2.3 replay . . . 952
7.17.2.4 record . . . 952
7.18 rt_audio_ops Struct Reference . . . 952
7.18.1 Detailed Description . . . 952
7.18.2 Field Documentation . . . 952
7.18.2.1 getcaps . . . 952
7.18.2.2 configure . . . 953
7.18.2.3 init . . . 953
7.18.2.4 start . . . 953
7.18.2.5 stop . . . 953
7.18.2.6 transmit . . . 953
7.18.2.7 buffer_info . . . 953
7.19 rt_audio_pipe Struct Reference . . . 953
7.19.1 Detailed Description . . . 953
7.19.2 Field Documentation . . . 953
7.19.2.1 ringbuffer . . . 953
7.19.2.2 suspended_read_list . . . 953
7.19.2.3 suspended_write_list . . . 954
7.20 rt_audio_portal_device Struct Reference . . . 954
7.20.1 Detailed Description . . . 954

7.21 rt_audio_record Struct Reference . . . 954
7.21.1 Detailed Description . . . 954
7.22 rt_audio_replay Struct Reference . . . 954
7.22.1 Detailed Description . . . 954
7.22.2 Field Documentation . . . 954
7.22.2.1 mp . . . 954
7.22.2.2 queue . . . 954
7.22.2.3 lock . . . 955
7.22.2.4 cmp . . . 955
7.22.2.5 buf_info . . . 955
7.22.2.6 write_data . . . 955
7.22.2.7 write_index . . . 955
7.22.2.8 read_index . . . 955
7.22.2.9 pos . . . 955
7.22.2.10 event . . . 955
7.22.2.11 activated . . . 955
7.23 rt_blk_device Struct Reference . . . 955
7.23.1 Detailed Description . . . 955
7.24 rt_blk_disk Struct Reference . . . 955
7.24.1 Detailed Description . . . 956
7.25 rt_blk_disk_ops Struct Reference . . . 956
7.25.1 Detailed Description . . . 956
7.26 rt_can_bit_timing Struct Reference . . . 956
7.26.1 Detailed Description . . . 956
7.26.2 Field Documentation . . . 956
7.26.2.1 prescaler . . . 956
7.26.2.2 num_seg1 . . . 956
7.26.2.3 num_seg2 . . . 956
7.26.2.4 num_sjw . . . 956
7.26.2.5 num_sspoff . . . 956
7.27 rt_can_bit_timing_config Struct Reference . . . 956
7.27.1 Detailed Description . . . 957
7.27.2 Field Documentation . . . 957
7.27.2.1 count . . . 957
7.27.2.2 items . . . 957
7.28 rt_can_device Struct Reference . . . 957
7.28.1 Detailed Description . . . 957
7.28.2 Field Documentation . . . 957
7.28.2.1 parent . . . 957
7.28.2.2 ops . . . 957
7.28.2.3 config . . . 958
7.28.2.4 status . . . 958

7.28.2.5 timerinitflag . . . 958
7.28.2.6 timer . . . 958
7.28.2.7 status_indicate . . . 958
7.28.2.8 lock . . . 958
7.28.2.9 can_rx . . . 958
7.28.2.10 can_tx . . . 958
7.28.2.11 nb_tx_rb . . . 958
7.28.2.12 nb_tx_rb_pool . . . 958
7.29 rt_can_filter_config Struct Reference . . . 958
7.29.1 Detailed Description . . . 959
7.29.2 Field Documentation . . . 959
7.29.2.1 count . . . 959
7.29.2.2 actived . . . 959
7.29.2.3 items . . . 959
7.30 rt_can_filter_item Struct Reference . . . 959
7.30.1 Detailed Description . . . 959
7.30.2 Field Documentation . . . 959
7.30.2.1 id . . . 959
7.30.2.2 ide . . . 959
7.30.2.3 rtr . . . 959
7.30.2.4 mode . . . 960
7.30.2.5 mask . . . 960
7.30.2.6 hdr_bank . . . 960
7.30.2.7 rxfifo . . . 960
7.31 rt_can_msg Struct Reference . . . 960
7.31.1 Detailed Description . . . 960
7.31.2 Field Documentation . . . 960
7.31.2.1 id . . . 960
7.31.2.2 ide . . . 960
7.31.2.3 rtr . . . 960
7.31.2.4 rsv . . . 961
7.31.2.5 len . . . 961
7.31.2.6 priv . . . 961
7.31.2.7 hdr_index . . . 961
7.31.2.8 rxfifo . . . 961
7.31.2.9 nonblocking . . . 961
7.31.2.10 data . . . 961
7.32 rt_can_ops Struct Reference . . . 961
7.32.1 Detailed Description . . . 961
7.32.2 Field Documentation . . . 962
7.32.2.1 configure . . . 962
7.32.2.2 control . . . 962

7.32.2.3 sendmsg . . . 962
7.32.2.4 recvmsg . . . 962
7.32.2.5 sendmsg_nonblocking . . . 963
7.33 rt_can_status Struct Reference . . . 963
7.33.1 Detailed Description . . . 963
7.33.2 Field Documentation . . . 964
7.33.2.1 rcverrcnt . . . 964
7.33.2.2 snderrcnt . . . 964
7.33.2.3 errcode . . . 964
7.33.2.4 rcvpkg . . . 964
7.33.2.5 dropedrcvpkg . . . 964
7.33.2.6 sndpkg . . . 964
7.33.2.7 dropedsndpkg . . . 964
7.33.2.8 bitpaderrcnt . . . 964
7.33.2.9 formaterrcnt . . . 964
7.33.2.10 ackerrcnt . . . 964
7.33.2.11 biterrcnt . . . 964
7.33.2.12 crcerrcnt . . . 964
7.33.2.13 rcvchange . . . 965
7.33.2.14 sndchange . . . 965
7.33.2.15 lasterrtype . . . 965
7.34 rt_can_status_ind_type Struct Reference . . . 965
7.34.1 Detailed Description . . . 965
7.34.2 Field Documentation . . . 965
7.34.2.1 ind . . . 965
7.34.2.2 args . . . 965
7.35 rt_clk Struct Reference . . . 965
7.35.1 Detailed Description . . . 965
7.35.2 Field Documentation . . . 966
7.35.2.1 dev_id . . . 966
7.35.2.2 con_id . . . 966
7.36 rt_clk_array Struct Reference . . . 966
7.36.1 Detailed Description . . . 966
7.37 rt_clk_cell Struct Reference . . . 966
7.37.1 Detailed Description . . . 966
7.37.2 Field Documentation . . . 966
7.37.2.1 rate . . . 966
7.38 rt_clk_fixed_rate Struct Reference . . . 966
7.38.1 Detailed Description . . . 966
7.39 rt_clk_node Struct Reference . . . 966
7.39.1 Detailed Description . . . 967
7.40 rt_clk_notifier Struct Reference . . . 967

7.40.1 Detailed Description . . . 967
7.41 rt_cpu_usage_stats Struct Reference . . . 967
7.41.1 Detailed Description . . . 967
7.42 rt_dac_device Struct Reference . . . 967
7.42.1 Detailed Description . . . 967
7.43 rt_dac_ops Struct Reference . . . 967
7.43.1 Detailed Description . . . 968
7.44 rt_device Struct Reference . . . 968
7.44.1 Detailed Description . . . 968
7.44.2 Field Documentation . . . 968
7.44.2.1 parent . . . 968
7.44.2.2 type . . . 968
7.44.2.3 flag . . . 968
7.44.2.4 open_flag . . . 968
7.44.2.5 ref_count . . . 968
7.44.2.6 device_id . . . 968
7.44.2.7 readlink . . . 968
7.44.2.8 user_data . . . 969
7.45 rt_device_blk_geometry Struct Reference . . . 969
7.45.1 Detailed Description . . . 969
7.45.2 Field Documentation . . . 969
7.45.2.1 sector_count . . . 969
7.45.2.2 bytes_per_sector . . . 969
7.45.2.3 block_size . . . 969
7.46 rt_device_blk_sectors Struct Reference . . . 969
7.46.1 Detailed Description . . . 969
7.46.2 Field Documentation . . . 969
7.46.2.1 sector_begin . . . 969
7.46.2.2 sector_end . . . 970
7.47 rt_device_graphic_info Struct Reference . . . 970
7.47.1 Detailed Description . . . 970
7.47.2 Field Documentation . . . 970
7.47.2.1 pixel_format . . . 970
7.47.2.2 bits_per_pixel . . . 970
7.47.2.3 pitch . . . 970
7.47.2.4 width . . . 970
7.47.2.5 height . . . 970
7.47.2.6 framebuffer . . . 970
7.47.2.7 smem_len . . . 970
7.48 rt_device_graphic_ops Struct Reference . . . 971
7.48.1 Detailed Description . . . 971
7.49 rt_device_notify Struct Reference . . . 971

7.49.1 Detailed Description . . . 971
7.50 rt_device_pin Struct Reference . . . 971
7.50.1 Detailed Description . . . 971
7.51 rt_device_pin_mode Struct Reference . . . 971
7.51.1 Detailed Description . . . 971
7.52 rt_device_pin_value Struct Reference . . . 971
7.52.1 Detailed Description . . . 971
7.53 rt_device_pwm Struct Reference . . . 971
7.53.1 Detailed Description . . . 971
7.54 rt_device_rect_info Struct Reference . . . 971
7.54.1 Detailed Description . . . 972
7.54.2 Field Documentation . . . 972
7.54.2.1 x . . . 972
7.54.2.2 y . . . 972
7.54.2.3 width . . . 972
7.54.2.4 height . . . 972
7.55 rt_dma_chan Struct Reference . . . 972
7.55.1 Detailed Description . . . 972
7.55.2 Field Documentation . . . 973
7.55.2.1 name . . . 973
7.55.2.2 ctrl . . . 973
7.55.2.3 slave . . . 973
7.55.2.4 list . . . 973
7.55.2.5 conf_err . . . 973
7.55.2.6 prep_err . . . 973
7.55.2.7 conf . . . 973
7.55.2.8 transfer . . . 973
7.55.2.9 callback . . . 973
7.55.2.10 priv . . . 973
7.56 rt_dma_controller Struct Reference . . . 973
7.56.1 Detailed Description . . . 974
7.56.2 Member Function Documentation . . . 974
7.56.2.1 RT_BITMAP_DECLARE() . . . 974
7.56.3 Field Documentation . . . 974
7.56.3.1 list . . . 974
7.56.3.2 dev . . . 974
7.56.3.3 addr_mask . . . 974
7.56.3.4 ops . . . 974
7.56.3.5 channels_nodes . . . 974
7.56.3.6 mutex . . . 974
7.57 rt_dma_controller_ops Struct Reference . . . 974
7.57.1 Detailed Description . . . 975

7.57.2 Field Documentation . . . 975
7.57.2.1 request_chan . . . 975
7.57.2.2 release_chan . . . 975
7.57.2.3 start . . . 976
7.57.2.4 pause . . . 976
7.57.2.5 stop . . . 976
7.57.2.6 config . . . 976
7.57.2.7 prep_memcpy . . . 977
7.57.2.8 prep_cyclic . . . 977
7.57.2.9 prep_single . . . 977
7.58 rt_dma_map_ops Struct Reference . . . 978
7.58.1 Detailed Description . . . 978
7.58.2 Field Documentation . . . 978
7.58.2.1 alloc . . . 978
7.58.2.2 free . . . 978
7.58.2.3 sync_out_data . . . 979
7.58.2.4 sync_in_data . . . 979
7.59 rt_dma_pool Struct Reference . . . 979
7.59.1 Detailed Description . . . 980
7.59.2 Field Documentation . . . 980
7.59.2.1 region . . . 980
7.59.2.2 list . . . 980
7.59.2.3 flags . . . 980
7.59.2.4 map . . . 980
7.59.2.5 bits . . . 980
7.59.2.6 start . . . 980
7.59.2.7 dev . . . 980
7.60 rt_dma_slave_config Struct Reference . . . 980
7.60.1 Detailed Description . . . 981
7.60.2 Field Documentation . . . 981
7.60.2.1 direction . . . 981
7.60.2.2 src_addr_width . . . 981
7.60.2.3 dst_addr_width . . . 981
7.60.2.4 src_addr . . . 981
7.60.2.5 dst_addr . . . 981
7.60.2.6 src_maxburst . . . 981
7.60.2.7 dst_maxburst . . . 981
7.60.2.8 src_port_window_size . . . 981
7.60.2.9 dst_port_window_size . . . 981
7.61 rt_dma_slave_transfer Struct Reference . . . 981
7.61.1 Detailed Description . . . 982
7.61.2 Field Documentation . . . 982

7.61.2.1 src_addr . . . 982
7.61.2.2 dst_addr . . . 982
7.61.2.3 buffer . . . 982
7.61.2.4 dma_handle . . . 982
7.61.2.5 buffer_len . . . 982
7.61.2.6 period_len . . . 982
7.62 rt_hw_backtrace_frame Struct Reference . . . 982
7.62.1 Detailed Description . . . 982
7.63 rt_i2c_bus_device Struct Reference . . . 982
7.63.1 Detailed Description . . . 983
7.64 rt_i2c_bus_device_ops Struct Reference . . . 983
7.64.1 Detailed Description . . . 983
7.65 rt_i2c_client Struct Reference . . . 983
7.65.1 Detailed Description . . . 983
7.66 rt_i2c_configuration Struct Reference . . . 983
7.66.1 Detailed Description . . . 983
7.67 rt_i2c_msg Struct Reference . . . 983
7.67.1 Detailed Description . . . 983
7.68 rt_i2c_priv_data Struct Reference . . . 983
7.68.1 Detailed Description . . . 983
7.69 rt_inputcapture_ops Struct Reference . . . 983
7.69.1 Detailed Description . . . 983
7.70 rt_interrupt_context Struct Reference . . . 984
7.70.1 Detailed Description . . . 984
7.70.2 Field Documentation . . . 984
7.70.2.1 context . . . 984
7.70.2.2 node . . . 984
7.71 rt_ipc_object Struct Reference . . . 984
7.71.1 Detailed Description . . . 984
7.71.2 Field Documentation . . . 984
7.71.2.1 parent . . . 984
7.71.2.2 suspend_thread . . . 984
7.72 rt_list_node Struct Reference . . . 984
7.72.1 Detailed Description . . . 985
7.72.2 Field Documentation . . . 985
7.72.2.1 next . . . 985
7.72.2.2 prev . . . 985
7.73 rt_lockless_slist_node Struct Reference . . . 985
7.73.1 Detailed Description . . . 985
7.73.2 Field Documentation . . . 985
7.73.2.1 next . . . 985
7.74 rt_mailbox Struct Reference . . . 985

7.74.1 Detailed Description . . . 985
7.74.2 Field Documentation . . . 985
7.74.2.1 parent . . . 985
7.74.2.2 msg_pool . . . 986
7.74.2.3 size . . . 986
7.74.2.4 entry . . . 986
7.74.2.5 in_offset . . . 986
7.74.2.6 out_offset . . . 986
7.74.2.7 suspend_sender_thread . . . 986
7.75 rt_mempool Struct Reference . . . 986
7.75.1 Detailed Description . . . 986
7.75.2 Field Documentation . . . 986
7.75.2.1 parent . . . 986
7.75.2.2 start_address . . . 986
7.75.2.3 size . . . 987
7.75.2.4 block_size . . . 987
7.75.2.5 block_list . . . 987
7.75.2.6 block_total_count . . . 987
7.75.2.7 block_free_count . . . 987
7.75.2.8 suspend_thread . . . 987
7.76 rt_messagequeue Struct Reference . . . 987
7.76.1 Detailed Description . . . 987
7.76.2 Field Documentation . . . 987
7.76.2.1 parent . . . 987
7.76.2.2 msg_pool . . . 988
7.76.2.3 msg_size . . . 988
7.76.2.4 max_msgs . . . 988
7.76.2.5 entry . . . 988
7.76.2.6 msg_queue_head . . . 988
7.76.2.7 msg_queue_tail . . . 988
7.76.2.8 msg_queue_free . . . 988
7.76.2.9 suspend_sender_thread . . . 988
7.77 rt_mutex Struct Reference . . . 988
7.77.1 Detailed Description . . . 988
7.77.2 Field Documentation . . . 989
7.77.2.1 parent . . . 989
7.77.2.2 ceiling_priority . . . 989
7.77.2.3 priority . . . 989
7.77.2.4 hold . . . 989
7.77.2.5 reserved . . . 989
7.77.2.6 owner . . . 989
7.77.2.7 taken_list . . . 989

7.78 rt_object Struct Reference . . . 989
7.78.1 Detailed Description . . . 989
7.78.2 Field Documentation . . . 989
7.78.2.1 name . . . 989
7.78.2.2 type . . . 990
7.78.2.3 flag . . . 990
7.78.2.4 module_id . . . 990
7.78.2.5 list . . . 990
7.79 rt_object_information Struct Reference . . . 990
7.79.1 Detailed Description . . . 990
7.79.2 Field Documentation . . . 990
7.79.2.1 type . . . 990
7.79.2.2 object_list . . . 990
7.79.2.3 object_size . . . 990
7.80 rt_pin_irq_hdr Struct Reference . . . 990
7.80.1 Detailed Description . . . 990
7.81 rt_pin_ops Struct Reference . . . 991
7.81.1 Detailed Description . . . 991
7.82 rt_pm Struct Reference . . . 991
7.82.1 Detailed Description . . . 991
7.83 rt_pm_ops Struct Reference . . . 991
7.83.1 Detailed Description . . . 991
7.84 rt_pwm_configuration Struct Reference . . . 991
7.84.1 Detailed Description . . . 991
7.85 rt_pwm_ops Struct Reference . . . 991
7.85.1 Detailed Description . . . 991
7.86 rt_qspi_configuration Struct Reference . . . 991
7.86.1 Detailed Description . . . 991
7.87 rt_qspi_device Struct Reference . . . 991
7.87.1 Detailed Description . . . 992
7.88 rt_qspi_message Struct Reference . . . 992
7.88.1 Detailed Description . . . 992
7.89 rt_rtc_device Struct Reference . . . 992
7.89.1 Detailed Description . . . 992
7.90 rt_rtc_ops Struct Reference . . . 992
7.90.1 Detailed Description . . . 992
7.91 rt_rtc_wkalarm Struct Reference . . . 992
7.91.1 Detailed Description . . . 992
7.92 rt_sched_thread_ctx Struct Reference . . . 992
7.92.1 Detailed Description . . . 992
7.92.2 Field Documentation . . . 993
7.92.2.1 thread_list_node . . . 993

7.92.2.2 stat . . . 993
7.92.2.3 sched_flag_locked . . . 993
7.92.2.4 sched_flag_ttmr_set . . . 993
7.92.2.5 sched_thread_priv . . . 993
7.93 rt_sched_thread_priv Struct Reference . . . 993
7.93.1 Detailed Description . . . 993
7.93.2 Field Documentation . . . 993
7.93.2.1 init_tick . . . 993
7.93.2.2 remaining_tick . . . 993
7.93.2.3 current_priority . . . 993
7.93.2.4 init_priority . . . 994
7.93.2.5 number_mask . . . 994
7.94 rt_semaphore Struct Reference . . . 994
7.94.1 Detailed Description . . . 994
7.94.2 Field Documentation . . . 994
7.94.2.1 parent . . . 994
7.94.2.2 value . . . 994
7.95 rt_serial_device Struct Reference . . . 994
7.95.1 Detailed Description . . . 994
7.96 rt_serial_rx_fifo Struct Reference . . . 994
7.96.1 Detailed Description . . . 994
7.97 rt_serial_tx_fifo Struct Reference . . . 994
7.97.1 Detailed Description . . . 995
7.98 rt_slist_node Struct Reference . . . 995
7.98.1 Detailed Description . . . 995
7.98.2 Field Documentation . . . 995
7.98.2.1 next . . . 995
7.99 rt_spi_bus Struct Reference . . . 995
7.99.1 Detailed Description . . . 995
7.100 rt_spi_configuration Struct Reference . . . 995
7.100.1 Detailed Description . . . 995
7.101 rt_spi_device Struct Reference . . . 995
7.101.1 Detailed Description . . . 995
7.102 rt_spi_message Struct Reference . . . 995
7.102.1 Detailed Description . . . 995
7.103 rt_spi_ops Struct Reference . . . 996
7.103.1 Detailed Description . . . 996
7.104 rt_spinlock Struct Reference . . . 996
7.104.1 Detailed Description . . . 996
7.105 rt_thread Struct Reference . . . 996
7.105.1 Detailed Description . . . 996
7.105.2 Field Documentation . . . 996

7.105.2.1 sp . . . 996
7.105.2.2 entry . . . 996
7.105.2.3 parameter . . . 996
7.105.2.4 stack_addr . . . 997
7.105.2.5 stack_size . . . 997
7.105.2.6 error . . . 997
7.105.2.7 thread_timer . . . 997
7.105.2.8 cleanup . . . 997
7.105.2.9 sig_pending . . . 997
7.105.2.10 sig_mask . . . 997
7.105.2.11 sig_ret . . . 997
7.105.2.12 sig_vectors . . . 997
7.105.2.13 si_list . . . 997
7.105.2.14 user_data . . . 997
7.106 rt_timer Struct Reference . . . 997
7.106.1 Detailed Description . . . 998
7.106.2 Field Documentation . . . 998
7.106.2.1 parent . . . 998
7.106.2.2 timeout_func . . . 998
7.106.2.3 parameter . . . 998
7.106.2.4 init_tick . . . 998
7.106.2.5 timeout_tick . . . 998
7.107 rt_touch_config Struct Reference . . . 998
7.107.1 Detailed Description . . . 998
7.108 rt_touch_data Struct Reference . . . 998
7.108.1 Detailed Description . . . 998
7.109 rt_touch_device Struct Reference . . . 998
7.109.1 Detailed Description . . . 999
7.110 rt_touch_info Struct Reference . . . 999
7.110.1 Detailed Description . . . 999
7.111 rt_touch_ops Struct Reference . . . 999
7.111.1 Detailed Description . . . 999
7.112 rt_uart_ops Struct Reference . . . 999
7.112.1 Detailed Description . . . 999
7.113 rt_wqueue Struct Reference . . . 999
7.113.1 Detailed Description . . . 999
7.114 scmi_clk_attributes Struct Reference . . . 999
7.114.1 Detailed Description . . . 999
7.115 scmi_clk_describe_attributes_in Struct Reference . . . 999
7.115.1 Detailed Description . . . 999
7.116 scmi_clk_describe_attributes_out Struct Reference . . . 1000
7.116.1 Detailed Description . . . 1000

7.117 scmi_clk_describe_rates_in Struct Reference . . . 1000
7.117.1 Detailed Description . . . 1000
7.118 scmi_clk_describe_rates_out Struct Reference . . . 1000
7.118.1 Detailed Description . . . 1000
7.119 scmi_clk_name_in Struct Reference . . . 1000
7.119.1 Detailed Description . . . 1000
7.120 scmi_clk_name_out Struct Reference . . . 1000
7.120.1 Detailed Description . . . 1000
7.121 scmi_clk_rate_get_in Struct Reference . . . 1000
7.121.1 Detailed Description . . . 1000
7.122 scmi_clk_rate_get_out Struct Reference . . . 1000
7.122.1 Detailed Description . . . 1001
7.123 scmi_clk_rate_set_in Struct Reference . . . 1001
7.123.1 Detailed Description . . . 1001
7.124 scmi_clk_rate_set_out Struct Reference . . . 1001
7.124.1 Detailed Description . . . 1001
7.125 scmi_clk_state_in Struct Reference . . . 1001
7.125.1 Detailed Description . . . 1001
7.126 scmi_clk_state_out Struct Reference . . . 1001
7.126.1 Detailed Description . . . 1001
7.127 scmi_pinctrl_attributes_in Struct Reference . . . 1001
7.127.1 Detailed Description . . . 1001
7.128 scmi_pinctrl_attributes_out Struct Reference . . . 1001
7.128.1 Detailed Description . . . 1001
7.129 scmi_pinctrl_list_assoc_in Struct Reference . . . 1002
7.129.1 Detailed Description . . . 1002
7.130 scmi_pinctrl_list_assoc_out Struct Reference . . . 1002
7.130.1 Detailed Description . . . 1002
7.131 scmi_pinctrl_name_get_in Struct Reference . . . 1002
7.131.1 Detailed Description . . . 1002
7.132 scmi_pinctrl_name_get_out Struct Reference . . . 1002
7.132.1 Detailed Description . . . 1002
7.133 scmi_pinctrl_protocol_attributes Struct Reference . . . 1002
7.133.1 Detailed Description . . . 1002
7.134 scmi_pinctrl_request_in Struct Reference . . . 1002
7.134.1 Detailed Description . . . 1002
7.135 scmi_pinctrl_request_out Struct Reference . . . 1002
7.135.1 Detailed Description . . . 1003
7.136 scmi_pinctrl_settings_conf_in Struct Reference . . . 1003
7.136.1 Detailed Description . . . 1003
7.137 scmi_pinctrl_settings_conf_out Struct Reference . . . 1003
7.137.1 Detailed Description . . . 1003

7.138 scmi_pinctrl_settings_get_in Struct Reference . . . 1003
7.138.1 Detailed Description . . . 1003
7.139 scmi_pinctrl_settings_get_out Struct Reference . . . 1003
7.139.1 Detailed Description . . . 1003
7.140 scmi_power_attributes Struct Reference . . . 1003
7.140.1 Detailed Description . . . 1003
7.141 scmi_power_state_set_in Struct Reference . . . 1003
7.141.1 Detailed Description . . . 1003
7.142 scmi_power_state_set_out Struct Reference . . . 1004
7.142.1 Detailed Description . . . 1004
7.143 scmi_reset_attr_in Struct Reference . . . 1004
7.143.1 Detailed Description . . . 1004
7.144 scmi_reset_attr_out Struct Reference . . . 1004
7.144.1 Detailed Description . . . 1004
7.145 scmi_reset_in Struct Reference . . . 1004
7.145.1 Detailed Description . . . 1004
7.146 scmi_reset_out Struct Reference . . . 1004
7.146.1 Detailed Description . . . 1004
7.147 scmi_scmi_sensor_axis_description_get_in Struct Reference . . . 1004
7.147.1 Detailed Description . . . 1004
7.148 scmi_scmi_sensor_axis_description_get_out Struct Reference . . . 1004
7.148.1 Detailed Description . . . 1005
7.149 scmi_sensor_attributes Struct Reference . . . 1005
7.149.1 Detailed Description . . . 1005
7.150 scmi_sensor_axis_name_in Struct Reference . . . 1005
7.150.1 Detailed Description . . . 1005
7.151 scmi_sensor_axis_name_out Struct Reference . . . 1005
7.151.1 Detailed Description . . . 1005
7.152 scmi_sensor_config_get_in Struct Reference . . . 1005
7.152.1 Detailed Description . . . 1005
7.153 scmi_sensor_config_get_out Struct Reference . . . 1005
7.153.1 Detailed Description . . . 1005
7.154 scmi_sensor_config_set_in Struct Reference . . . 1005
7.154.1 Detailed Description . . . 1005
7.155 scmi_sensor_config_set_out Struct Reference . . . 1006
7.155.1 Detailed Description . . . 1006
7.156 scmi_sensor_continuous_update_notify_in Struct Reference . . . 1006
7.156.1 Detailed Description . . . 1006
7.157 scmi_sensor_continuous_update_notify_out Struct Reference . . . 1006
7.157.1 Detailed Description . . . 1006
7.158 scmi_sensor_description_get_in Struct Reference . . . 1006
7.158.1 Detailed Description . . . 1006

7.159 scmi_sensor_description_get_out Struct Reference . . . 1006
7.159.1 Detailed Description . . . 1006
7.160 scmi_sensor_list_update_intervals_in Struct Reference . . . 1006
7.160.1 Detailed Description . . . 1006
7.161 scmi_sensor_list_update_intervals_out Struct Reference . . . 1006
7.161.1 Detailed Description . . . 1007
7.162 scmi_sensor_name_in Struct Reference . . . 1007
7.162.1 Detailed Description . . . 1007
7.163 scmi_sensor_name_out Struct Reference . . . 1007
7.163.1 Detailed Description . . . 1007
7.164 scmi_sensor_reading_in Struct Reference . . . 1007
7.164.1 Detailed Description . . . 1007
7.165 scmi_sensor_reading_out Struct Reference . . . 1007
7.165.1 Detailed Description . . . 1007
7.166 scmi_sensor_trip_point_config_in Struct Reference . . . 1007
7.166.1 Detailed Description . . . 1007
7.167 scmi_sensor_trip_point_config_out Struct Reference . . . 1007
7.167.1 Detailed Description . . . 1007
7.168 scmi_sensor_trip_point_notify_in Struct Reference . . . 1008
7.168.1 Detailed Description . . . 1008
7.169 scmi_sensor_trip_point_notify_out Struct Reference . . . 1008
7.169.1 Detailed Description . . . 1008
7.170 scmi_voltage_attr_in Struct Reference . . . 1008
7.170.1 Detailed Description . . . 1008
7.171 scmi_voltage_attr_out Struct Reference . . . 1008
7.171.1 Detailed Description . . . 1008
7.172 scmi_voltage_attributes Struct Reference . . . 1008
7.172.1 Detailed Description . . . 1008
7.173 scmi_voltage_config_get_in Struct Reference . . . 1008
7.173.1 Detailed Description . . . 1008
7.174 scmi_voltage_config_get_out Struct Reference . . . 1008
7.174.1 Detailed Description . . . 1009
7.175 scmi_voltage_config_set_in Struct Reference . . . 1009
7.175.1 Detailed Description . . . 1009
7.176 scmi_voltage_config_set_out Struct Reference . . . 1009
7.176.1 Detailed Description . . . 1009
7.177 scmi_voltage_level_get_in Struct Reference . . . 1009
7.177.1 Detailed Description . . . 1009
7.178 scmi_voltage_level_get_out Struct Reference . . . 1009
7.178.1 Detailed Description . . . 1009
7.179 scmi_voltage_level_set_in Struct Reference . . . 1009
7.179.1 Detailed Description . . . 1009

7.180 scmi_voltage_level_set_out Struct Reference . . . 1009
7.180.1 Detailed Description . . . 1009

8 File Documentation **1011**

8.1 C:/Renesas/rt-thread/components/drivers/include/drivers/adc_v2.h File Reference . . . 1011
8.1.1 Detailed Description . . . 1012
8.1.2 Macro Definition Documentation . . . 1012
8.1.2.1 RT_ADC_CHANNEL_MASK . . . 1012
8.1.3 Enumeration Type Documentation . . . 1012
8.1.3.1 rt_adc_control_cmd . . . 1012
8.1.3.2 rt_adc_input_mode . . . 1012
8.1.3.3 rt_adc_state . . . 1013
8.1.4 Function Documentation . . . 1013
8.1.4.1 rt_adc_channel_mask_count() . . . 1013
8.1.4.2 rt_adc_channel_mask_take_lsb() . . . 1013
8.1.4.3 rt_hw_adc_register() . . . 1013
8.1.4.4 rt_adc_session_channel_index() . . . 1014
8.1.4.5 rt_adc_read_sequence() . . . 1014
8.1.4.6 rt_adc_raw_to_voltage_mv() . . . 1015
8.1.4.7 rt_adc_raw_to_scaled_voltage_mv() . . . 1015
8.1.4.8 rt_adc_voltage() . . . 1016
8.2 C:/Renesas/rt-thread/components/drivers/include/drivers/adc_v2_trigger.h File Reference . . . 1016
8.2.1 Detailed Description . . . 1016
8.3 C:/Renesas/rt-thread/components/drivers/include/drivers/dma.h File Reference . . . 1016
8.3.1 Detailed Description . . . 1018
8.3.2 Macro Definition Documentation . . . 1018
8.3.2.1 RT_DMA_F_LINEAR . . . 1018
8.3.2.2 RT_DMA_F_32BITS . . . 1018
8.3.2.3 RT_DMA_F_NOCACHE . . . 1018
8.3.2.4 RT_DMA_F_DEVICE . . . 1018
8.3.2.5 RT_DMA_F_NOMAP . . . 1018
8.3.2.6 RT_DMA_F_WT . . . 1018
8.3.2.7 RT_DMA_PAGE_SIZE . . . 1018
8.3.3 Enumeration Type Documentation . . . 1019
8.3.3.1 rt_dma_transfer_direction . . . 1019
8.3.3.2 rt_dma_slave_buswidth . . . 1019
8.3.4 Function Documentation . . . 1019
8.3.4.1 rt_dma_controller_add_direction() . . . 1019
8.3.4.2 rt_dma_controller_set_addr_mask() . . . 1020
8.3.4.3 rt_dma_controller_register() . . . 1020
8.3.4.4 rt_dma_controller_unregister() . . . 1020
8.3.4.5 rt_dma_chan_start() . . . 1020

8.3.4.6 rt_dma_chan_pause() . . . 1020
8.3.4.7 rt_dma_chan_stop() . . . 1020
8.3.4.8 rt_dma_chan_config() . . . 1020
8.3.4.9 rt_dma_chan_done() . . . 1021
8.3.4.10 rt_dma_prep_memcpy() . . . 1021
8.3.4.11 rt_dma_prep_cyclic() . . . 1021
8.3.4.12 rt_dma_prep_single() . . . 1021
8.3.4.13 rt_dma_chan_request() . . . 1021
8.3.4.14 rt_dma_chan_release() . . . 1021
8.3.4.15 rt_dma_alloc() . . . 1021
8.3.4.16 rt_dma_free() . . . 1021
8.3.4.17 rt_dma_alloc_coherent() . . . 1022
8.3.4.18 rt_dma_free_coherent() . . . 1022
8.3.4.19 rt_dma_sync_out_data() . . . 1022
8.3.4.20 rt_dma_sync_in_data() . . . 1022
8.3.4.21 rt_dma_device_is_coherent() . . . 1023
8.3.4.22 rt_dma_device_set_ops() . . . 1023
8.3.4.23 rt_dma_pool_install() . . . 1023
8.3.4.24 rt_dma_pool_extract() . . . 1023

Chapter 1

RT-Thread User Guide

Introduction

Start Guide

Kernel

Components

Tool

Contribution

1.1 Introduction

As a beginner of Real-time Operating Systems (RTOS), you might be new to RT-Thread. However, with a better understanding of it over time, you will gradually discover the charm of RT-Thread and its advantages over other RTOSs of the same type. RT-Thread is an RTOS.

RT-Thread was born in 2006, with years of experience accumulated till now, along with the rise of the Internet of Things (IoT), it is evolving into a powerful, component-rich IoT operating system.

More info. about RT-Thread, see "RT-Thread on Wikipedia".

1.1.1 RT-Thread Overview

RT-Thread, short for Real Time-Thread, is an embedded real-time multi-threaded operating system. One of its main purposes is to support multi-tasking. Allowing multiple tasks to run simultaneously does not mean that the processor actually performs multiple tasks at the same time - a processor core can only run one task at a time. Every task is executed quickly, and through the task scheduler which determines the sequence according to priority, the tasks are switched rapidly, giving the illusion that multiple tasks are running at the same time. In the RT-Thread system, tasks are implemented by threads, and scheduled by the task scheduler.

RT-Thread is mainly written in the C programming language, making it easy to understand and port. It applies object-oriented programming methods to real-time system design, making the code elegant, structured, modular, and very customizable. RT-Thread comes in a few varieties - the NANO version is a minimal kernel that requires only 3KB of flash and 1.2KB of RAM, which can be tailored with easy-to-use tools. For resource-rich IoT devices, RT-↩ Thread can use an online software package management tool, together with system configuration tools, to achieve an intuitive and rapid modular design, while being able to seamlessly import rich software featurepacks. Through this, complex functions like Android's graphical interface, touch sliding effects, smart voice control and more can be easily created.

Compared with the Linux operating system, RT-Thread is small in size, low in cost, low in power consumption and fast in startup. In addition, RT-Thread is highly responsible, with low resource usage, which is ideally suitable for various resource constraints such as cost, power consumption, etc. Although the 32-bit MCU is its main operating platform, other CPUs, such as ones with MMU, ones based on ARM9, ARM11 and even the Cortex-A series CPUs are suitable for RT-Thread in specific applications.

1.1.2 License Agreement

The RT-Thread system is a completely open source system, which follows the Apache License 2.0 open source license agreement. The RT-Thread system can be used free of charge in commercial products and does not require opening private code up to the public.

1.1.3 RT-Thread Frame

In recent years, the concept of the Internet of Things has become widely known, and the IoT market has developed rapidly. The networking of embedded devices is the trend of the times. Terminal networking has greatly increased the complexity of software, and traditional RTOS kernels can hardly meet the needs of the market. For this reason, the concept of the Internet of Things Operating System (IoT OS) came into being. **IoT operating system refers to the software platform that is based on operating system kernel (like RTOS, Linux, etc.) and includes relatively complete middleware components such as a file system, graphics library, etc. It has low overhead and high security, abides by the Communication Protocol and is capable of connecting to the cloud.** RT-Thread is an IoT OS.

One of the main differences between RT-Thread and many other RTOSs such as FreeRTOS and uC/OS is that it is not only a real-time kernel, but also has a rich middle-tier component, as shown in the following figure.

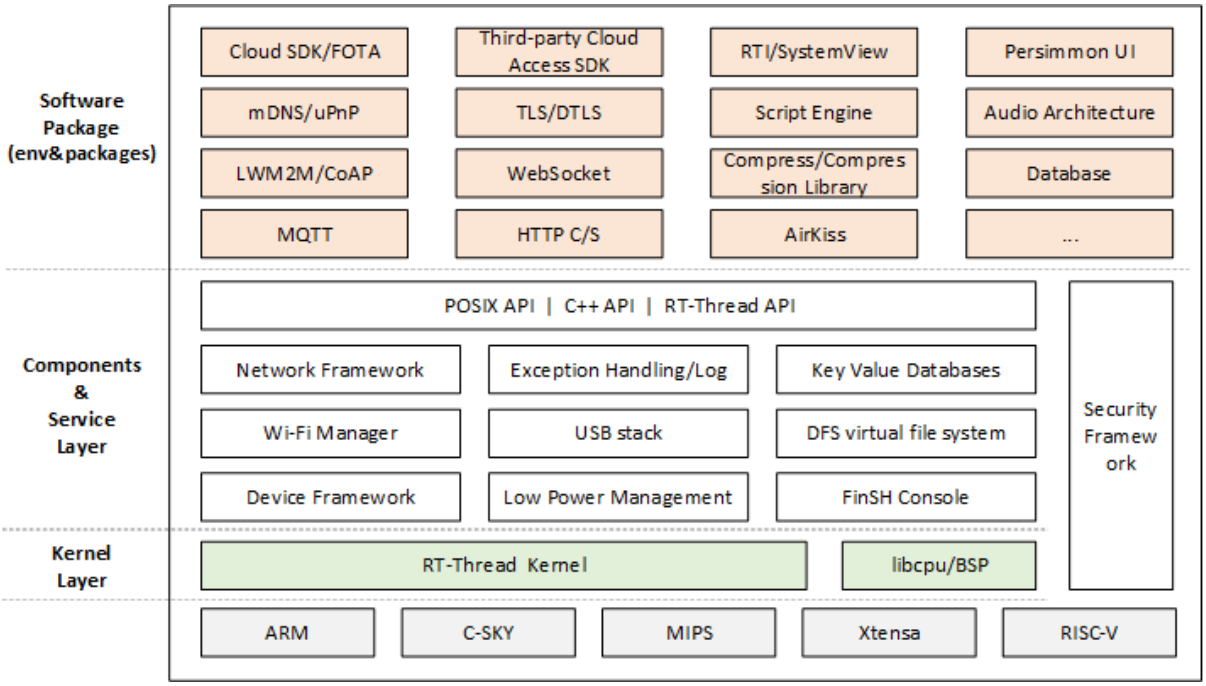


Figure 1.1 RT-Thread Software Framework

It includes:

- Kernel layer: RT-Thread kernel, the core part of RT-Thread, includes the implementation of objects in the kernel system, such as multi-threading and its scheduling, semaphore, mailbox, message queue, memory management, timer, etc.; libcpu/BSP (Chip Migration Related Files/Board Support Package) is closely related to hardware and consists of peripheral drivers and CPU transport.
- Components and Service Layer: Components are software modules on top of the RT-Thread kernel, such as virtual file systems, FinSH command-line interfaces, network frameworks, device frameworks, and more. Its modular design allows for high internal cohesion within the assembly and low coupling between components.

- RT-Thread software package: A general-purpose software component running on the RT-Thread IoT operating system platform for different application areas, consisting of description information, source code or library files. RT-Thread provides an open package platform with officially available or developer-supplied packages that provide developers with a choice of reusable packages that are an important part of the RT-Thread ecosystem. The package ecosystem is critical to the choice of an operating system because these packages are highly reusable and modular, making it easy for application developers to build the system they want in the shortest amount of time. RT-Thread supports more than 60 software packages, listed below:

  1. Internet of Things related software packages: Paho MQTT, WebClient, mongoose, WebTerminal, etc.
  2. Scripting language related software packages: JerryScript and MicroPython are currently supported.
  3. Multimedia related software packages: Openmv, mupdf.
  4. Tools packages: CmBacktrace, EasyFlash, EasyLogger, SystemView.
  5. System related software packages: RTGUI, Persimmon UI, lwext4, partition, SQLite, etc.
  6. Peripheral library and driver software packages: RealTek RTL8710BN SDK.
  7. Others.

1.2 Start Guide

Because of its particularity, the embedded operating system is often closely related to the hardware platform, and specific embedded operating systems can only run on specific hardware. For those who might not have an RT-Thread compatible hardware module, or want to test out their ideas, a complete RT-Thread system can be developed in the simulation environment MDK-ARM.

MDK-ARM (Microcontroller Development Kit - ARM) is a complete integrated development environment (IDE) from ARM. It includes an efficient C/C++ compiler for ARM chips (ARM7, ARM9, Cortex-M series, Cortex-R series, etc.), a project wizard and project management for various ARM devices and evaluation boards and a simulator for simulating hardware platforms in software. It supports debuggers connected to simulators debugging the target board, such as the commonly available ST-Link, J-Link, etc. The simulator software in MDK-ARM uses a complete software simulation to interpret and execute machine instructions from ARM and implement some peripheral logic to form a complete virtual hardware environment, enabling users to execute the corresponding target program on the computer without using a real hardware platform.

Because of its full STM32F103 software simulation environment, the MDK-ARM integrated development environment gives us the opportunity to run object code directly on the computer without using a real hardware environment. This simulator platform can completely virtualize the various operating modes and peripherals of the ARM Cortex-M3, such as exceptions, interrupts, clock timers, serial ports, etc., which is almost identical to the real hardware environment. RT-Thread can run on both the simulated environment and on real hardware.

What will follow is a demonstration of RT-Thread running on a simulated STM32F103 microcontroller through MDK-ARM.

1.2.1 Preparation

MDK development environment: MDK-ARM 5.24 (official or evaluation version, version 5.14 and above) needs to be installed. This version is a relatively new version, which can provide relatively complete debugging functions. An installation guide can be found here: Keil MDK Installation.

1.2.2 First acquaintance with RT-Thread

To see the code size of RT-Thread we first need to get an example of RT-Thread that is suited for this environment, which can be obtained from the following link:

RT-Thread Simulator Sample

This example is a zip file, unzip it. The directory structure after decompression is as shown below:

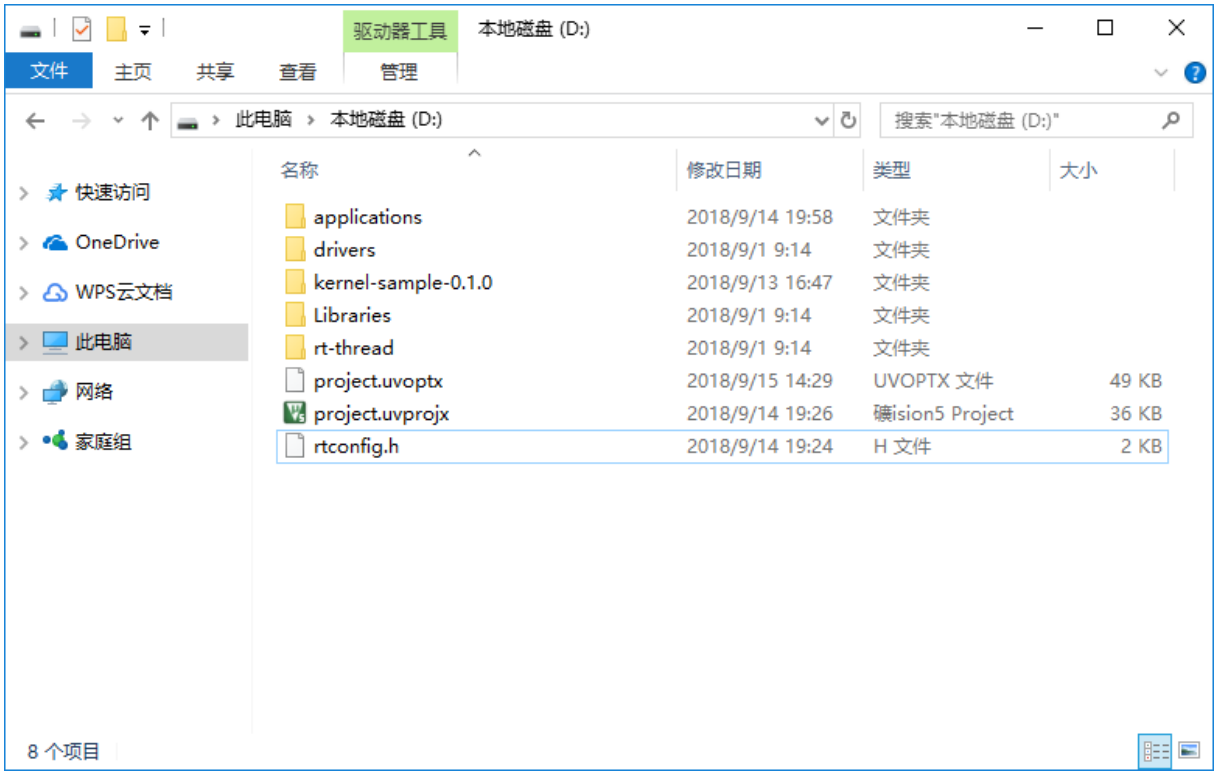


Figure 1.2 rtthread_simulator_v0.1.0 Code Directory

Descriptions of the file types contained in each directory are shown in the following table:

Directory Name	Description
applications	RT-Thread application.
rt-thread	Source file for RT-Thread.
- components	Respective component directories of RT-Thread.
- include	Header file for RT-Thread kernel.
- libcpu	Porting code for various types of chips, including porting files of STM32.
- src	Source file for RT-Thread kernel.
- tools	Script file of RT-Thread commanding building tool.
drivers	Driver of RT-Thread, implementations of bottom driver of different platforms.
Libraries	ST's STM32 firmware library file.
kernel-sample-0.↩ 1.0	Kernel sample for RT-Thread.

In the directory, there is a file with the name "project.uvprojx", which is an MDK5 project file in the sample referenced in this manual. Double-click "project.uvprojx" icon to open the project file:

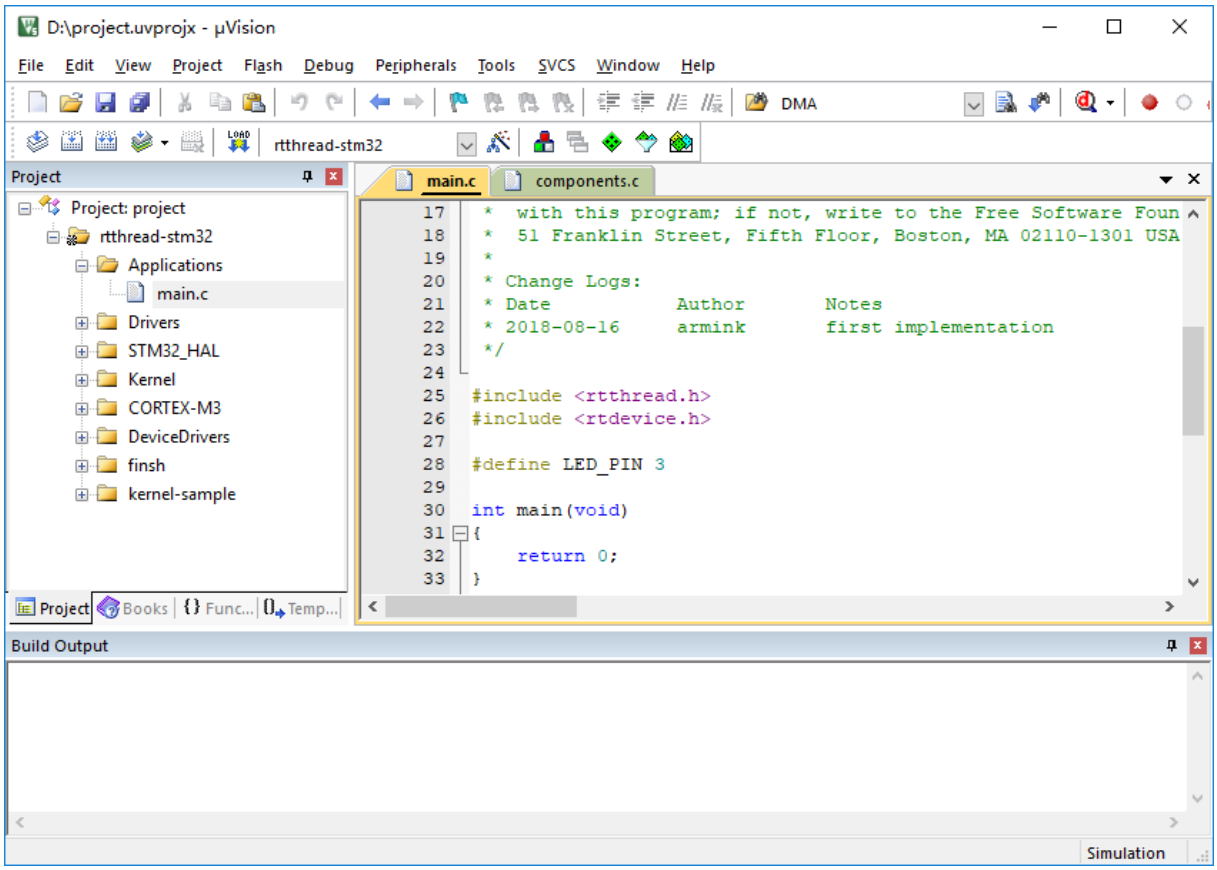


Figure 1.3 Open the project

Under the "Project" column on the left side of the main window of the project, you can see the file list of the project. These files are stored in the following groups, respectively:

Directory Group	Description
Applications	The corresponding directory is rtthread_simulator_v0.1.0/applications, used to store user application code.
Drivers	The corresponding directory is rtthread_simulator_v0.1.0/drivers, used to store the bottom driver code for RT-Thread.
STM32_HAL	The corresponding directory is rtthread_simulator_v0.1.0/Libraries/CMSIS/Device/ST/↩ STM32F1xx, used to store the firmware library files of STM32.
kernel-sample	The corresponding directory is rtthread_simulator_v0.1.0/kernel-sample-0.1.0, used to store kernel samples of RT-Thread.
Kernel	The corresponding directory is rtthread_simulator_v0.1.0/src, used to store RT-Thread kernel core code.
CORTEX-M3	The corresponding directory is rtthread_simulator_v0.1.0/rt-thread/libcpu, used to store ARM Cortex-M3 porting code.
DeviceDrivers	The corresponding directory is rtthread_simulator_v0.1.0/rt-thread/components/drivers, used to store driver framework source code of RT-Thread.
finsh	The corresponding directory is rtthread_simulator_v0.1.0/rt-thread/components/finsh, used to store command line of RT-Thread finsh command line component.

Now click the button from the toolbar on the top the window, , to compile the project as shown:

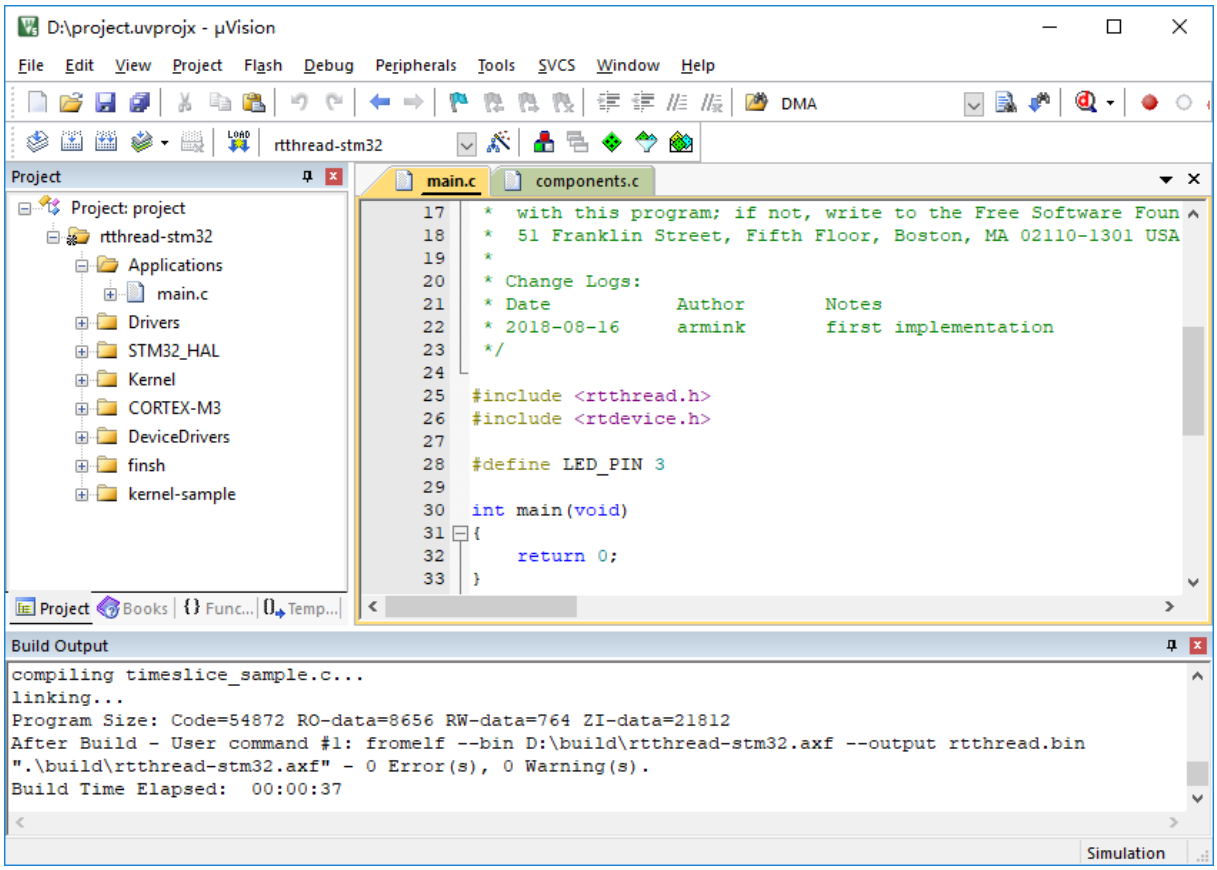


Figure 1.4 compiling

The result of the compilation is displayed in the "Build Output" bar at the bottom of the window. If nothing else, it will say "0 Error(s), ∗ Warning(s)." on the last line, that is, there are no errors or warnings.

After compiling RT-Thread/STM32, we can simulate running RT-Thread through the MDK-ARM simulator. Click  at the top right of the window or directly hit Ctrl+F5 to enter the simulation interface and hit F5 to start, then click the button in the toolbar shown in the screen shot or select “View→Serial Windows→UART#1” in the menu bar to open the serial port 1 window. You can see that the output of the serial port only shows the logo of RT-Thread. This is because the user code is empty and the result of its simulation is as shown:

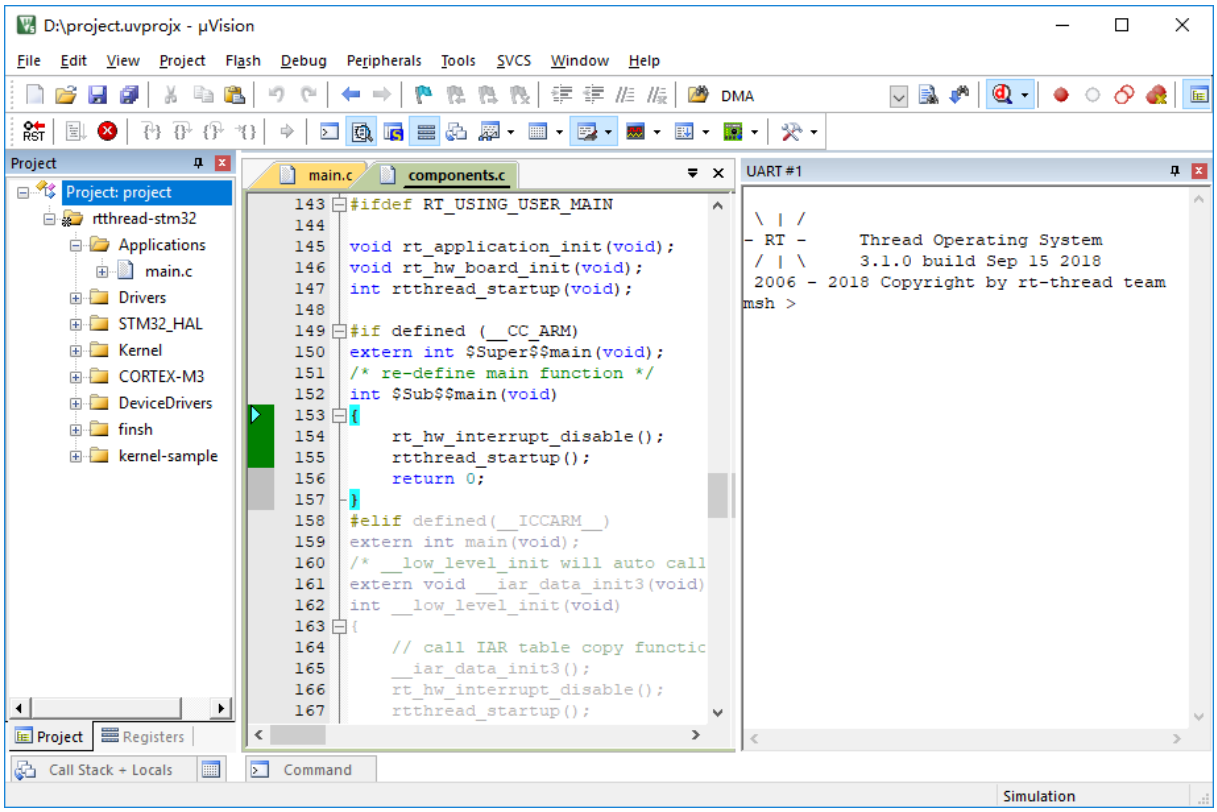


Figure 1.5 simulate RT-Thread1

>We can output all the commands supported by the current system by inputting the Tab key or `help + enter`, as shown in the following figure.

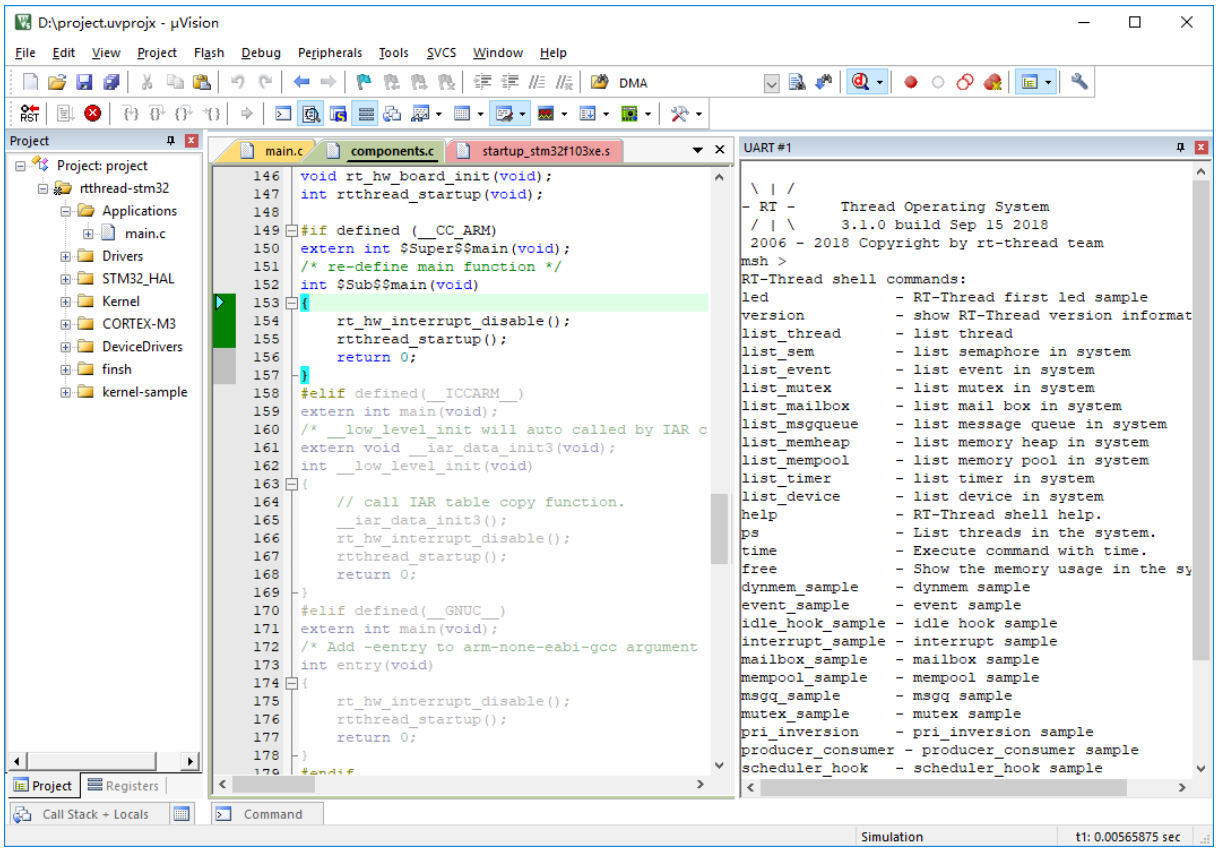


Figure 1.6 simulate RT-Thread2

1.2.3 User Entry Code

The above startup code is related to the RT-Thread system, so how do users add initialization code for their own applications? RT-Thread uses main function as the user code entry, all you need to do is just add your own code to the main function.

```
int main(void)
{
  /* user app entry */
  return 0;
}
```

>Note: In order to complete the initialization for the system functions before entering the main program, you can use the `$sub$$` and `$super$$` function identifiers to call another sample before entering the main program, this was, users can ignore the initialization operations before the main() function. See ARM® Compiler v5.06 for µVision® armlink User Guide for details.

1.2.4 Example of a Marquee

For technical engineers working on electronics, marquee is probably the simplest example, the equivalent of Hello World in every programming language programmers learn. So we will start with a marquee in the following example, to make it periodically update (turn on or off) the LED.

Under UART#1, input msh command: led and then click Enter to run it, as shown:

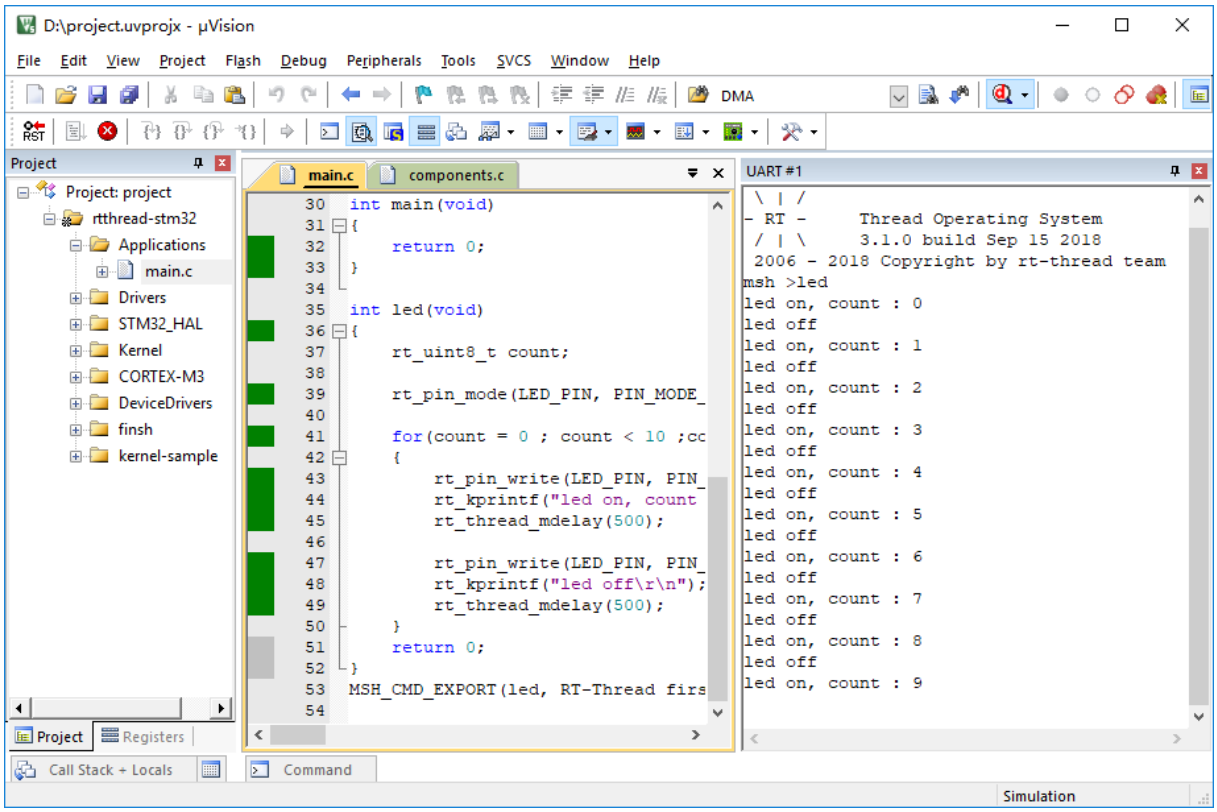


Figure 1.7 run led

Example of a Marquee

```
/*
 * Manifest of programs: Marquee sample
 *
 * marquee is probably the simplest example, it is like the first program
 * Hello World in every programming language that programmers learned. So we will start with a marquee in
     the following example, start a thread to make it periodically
 * update (turn on or off) the LED.
 */

int led(void)
{
    rt_uint8_t count;

    rt_pin_mode(LED_PIN, PIN_MODE_OUTPUT);

    for(count = 0 ; count < 10 ;count++)
    {
        rt_pin_write(LED_PIN, PIN_HIGH);
        rt_kprintf("led on, count : %d\r\n", count);
        rt_thread_mdelay(500);

        rt_pin_write(LED_PIN, PIN_LOW);
        rt_kprintf("led off\r\n");
        rt_thread_mdelay(500);
    }
    return 0;
}
MSH_CMD_EXPORT(led, RT-Thread first led sample);
```

1.2.5 Other Examples

Additional kernel examples can be found in the kernel-sample-0.1.0 directory.

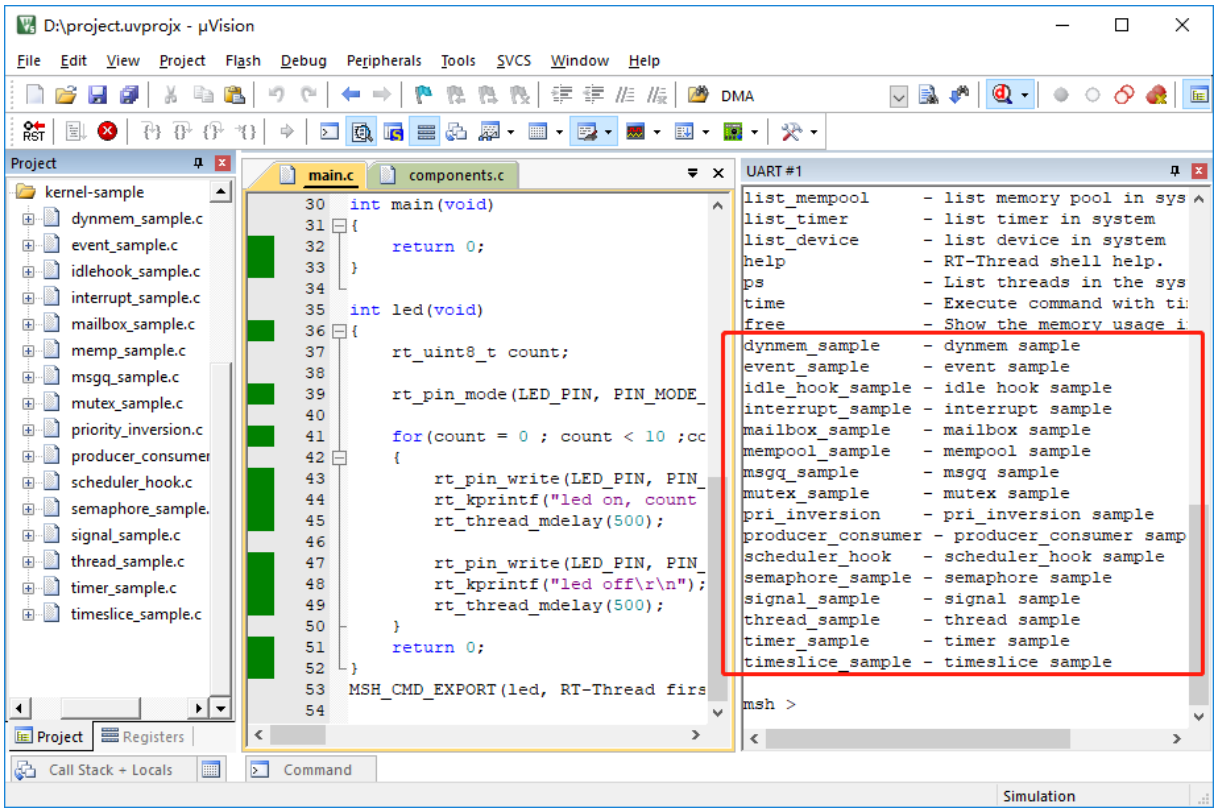


Figure 1.8 more kernel samples

1.2.6 Frequently Asked Question

- Compilation error occurred as following:

```
rt-thread\src\kservice.c(823): error: #929: incorrect use of vaarg fieldwidth = aarg(args, int);
rt-thread\src\kservice.c(842): error: #929: incorrect use of vaarg precision = aarg(args, int);
.........
```

Cause: This type of problem is usually caused by installation of ADS, when ADS and keil coexist, the header file of va_start points to the ADS folder.

Solution:

- Delete ADS environment variables
- Uninstall ADS and Keil, restart the computer, reload Keil

1.2.7 Keil MDK Installation

Before running the RT-Thread operating system, we need to install MDK-ARM 5.24 (either official or evaluation version, version 5.14 and above), this version is also a relatively new version. This version can provide relatively complete debugging functions. Here, we are using evaluation version 5.24 of 16k compiled code limit. If you want to remove the 16k compiled code limit, please purchase the official MDK-ARM.

Firstly, download the MDK-ARM evaluation version from the official website of www.keil.com: http://www.↩keil.com/download/

When downloading, you need to fill in some basic information, please fill in the corresponding complete information, and then start downloading. After it is downloaded, double-click the mouse to start the installation, you will see the software installation as shown:

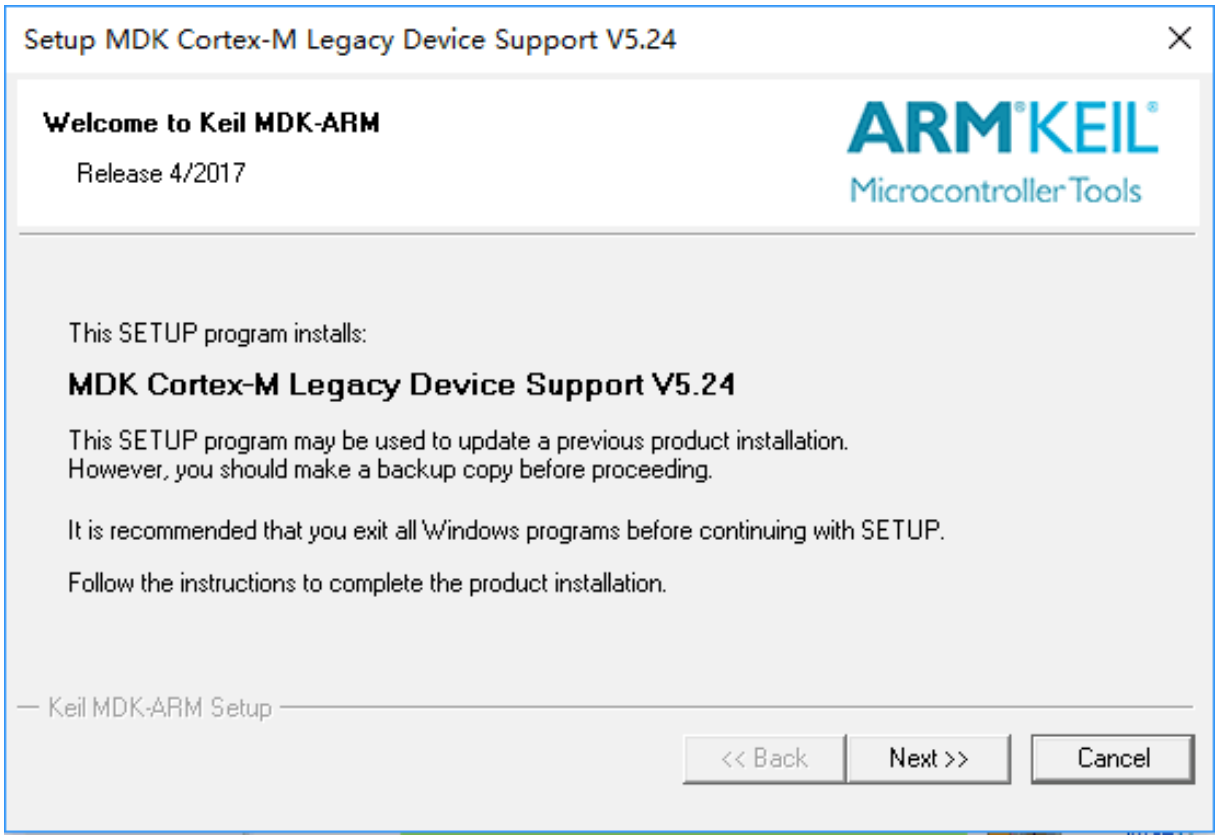


Figure 1.9 First Step

This is the MDK-ARM installation instructions, click “Next>>” to enter the next step, as shown.

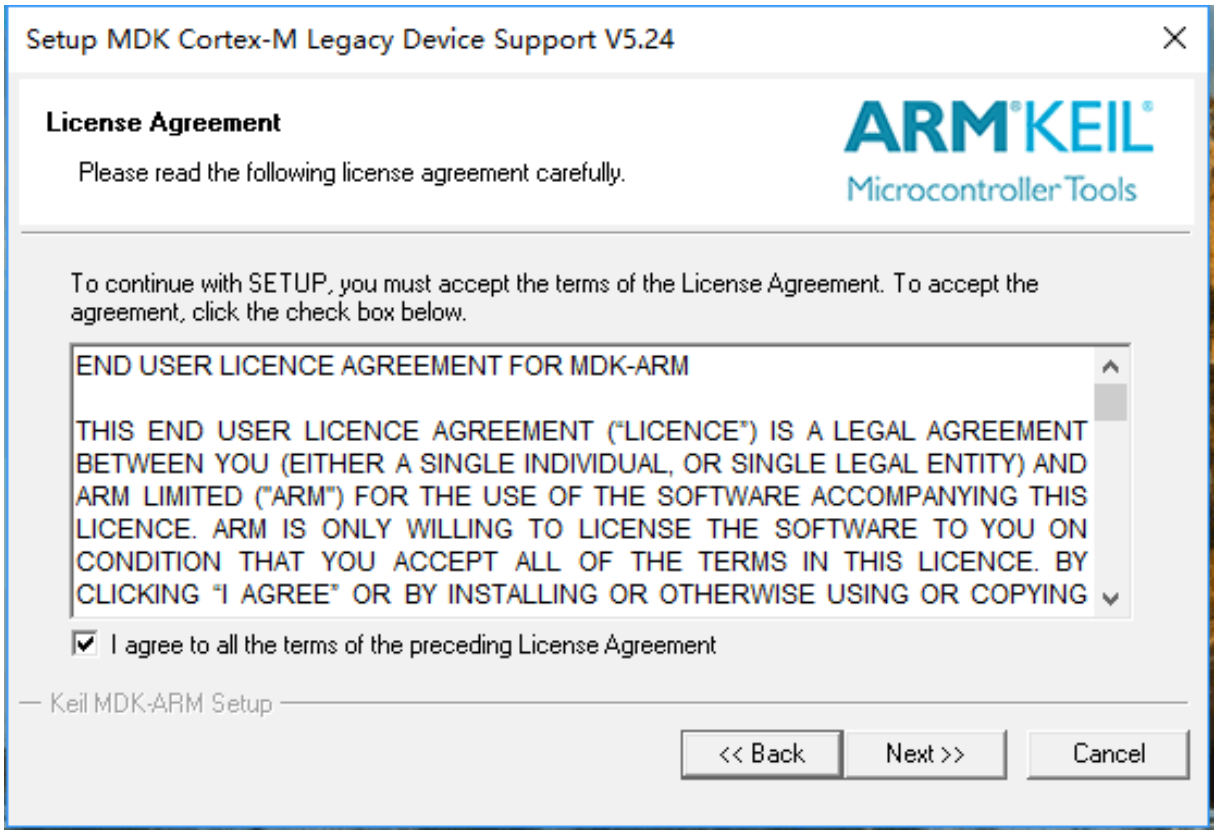


Figure 1.10 Second Step

Click "" in the box next to "I agree to all the terms of the preceding License Agreement" and click "Next >>" to proceed to the next step of installation, as shown:

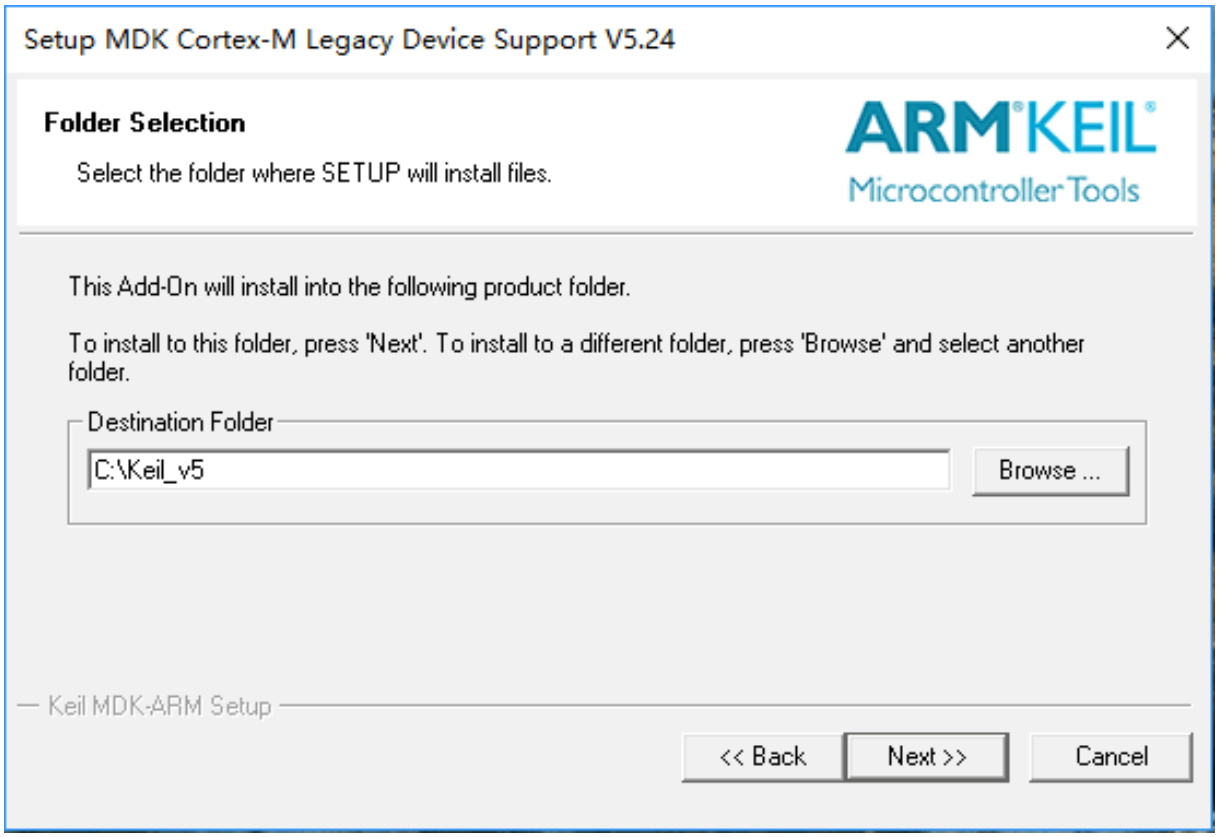


Figure 1.11 Third Step

Click "Browse..." to select the installation directory of MDK-ARM or directly input installation path in the "Destination Folder" box. Here, we default to "C:/Keil_v5", then click "Next>>" to proceed to the next step of installation, as shown:

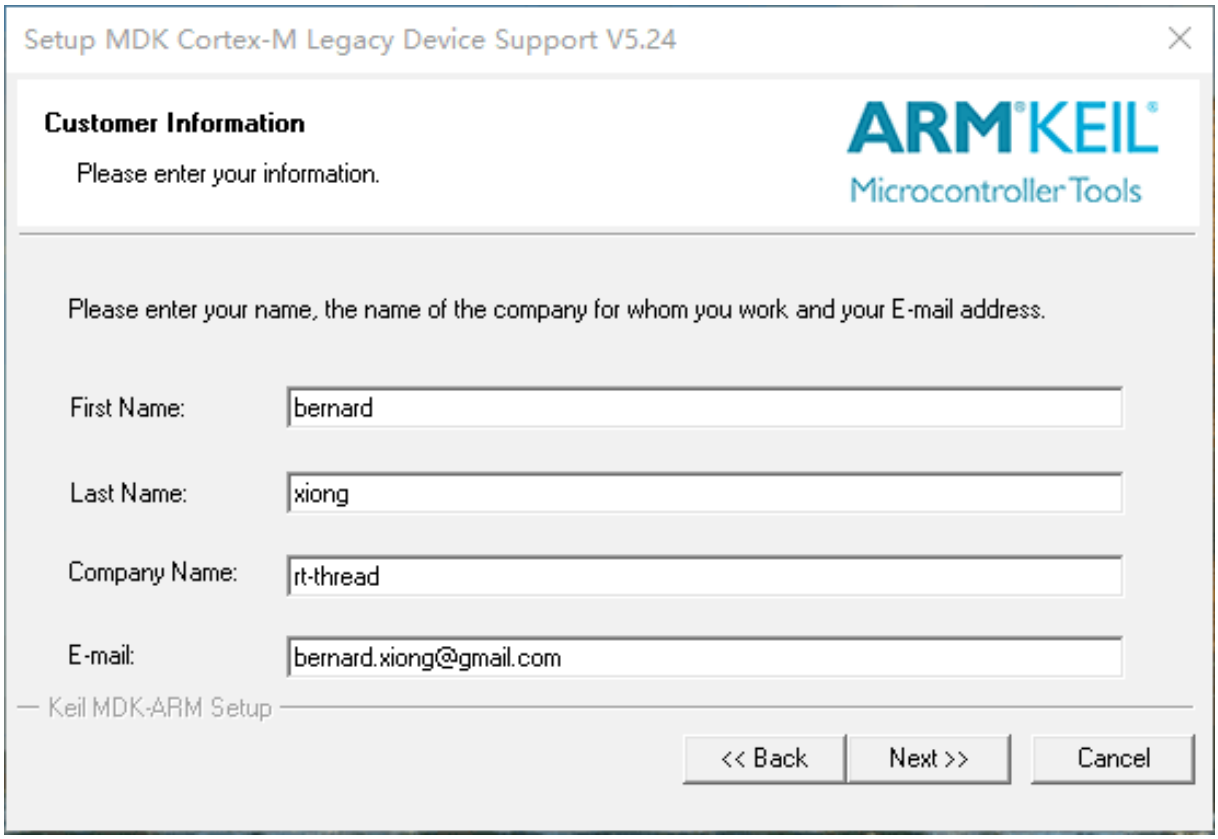


Figure 1.12 Fourth Step

Input your name after "First Name", input your last name after "Last Name", input your company name after "↩ Company Name", input your email address after "E-mail", and then click "Next>> " for the installation. Wait for a while for the installation to finish and you will see the following:

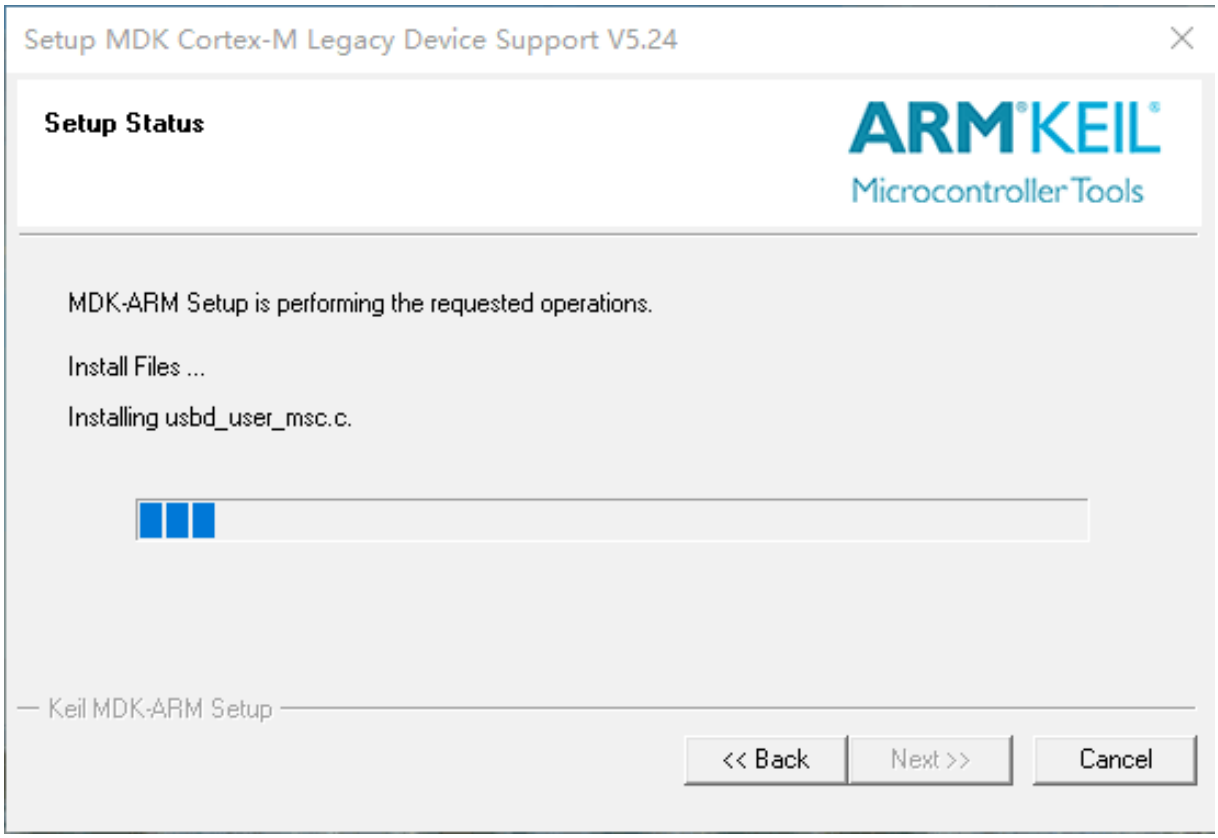


Figure 1.13 Fifth Step

The default selection does not need to be changed, just click “Next” to enter the next step as shown.

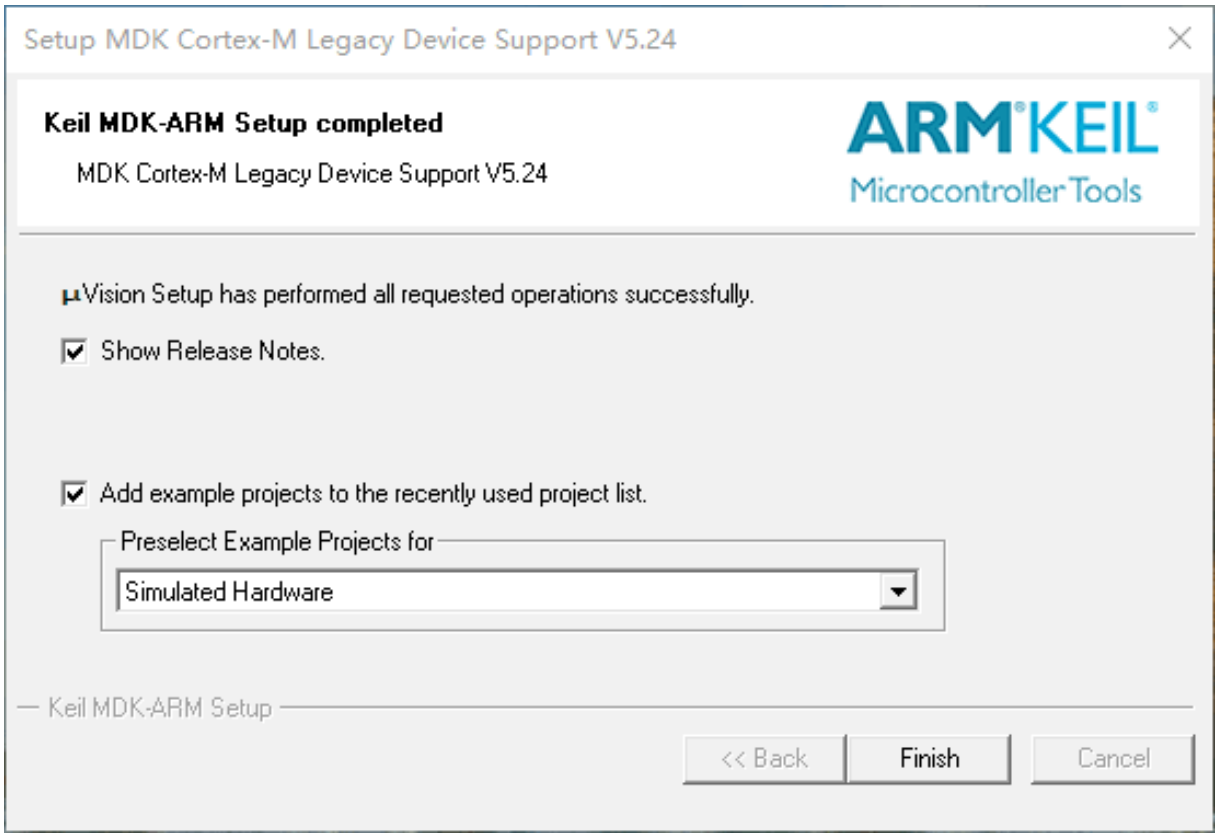


Figure 1.14 MDK-ARM Installment Complete

Here, you can click "Finish" to complete the installation of the entire MDK-ARM software.

With a useful took like MDK-ARM, you can start the RT-Thread operating system easily and explore real-time operating systems.

>Note: There is a charge for Official version of MDK-ARM. If you want to be able to compile larger binaries, please purchase the official version of MDK-ARM. RT-Thread operating system also supports GNU GCC compiler by Free Software Foundation which is an open source compiler. For more information on how to use GNU related tools, please refer to the related documentation on RT-Thread website.

1.3 Kernel

- Kernel Basics
- Object Management
- Thread Management
- Clock & Timer Management
- Inter-thread Synchronization
- Inter-thread Communication
- Memory Management
- Interrupt Management
- Kernel Porting
- QEMU virt64 AArch64 SMP Boot Flow

1.3.1 Kernel Basics

This chapter gives a brief introduction to the software architecture of the RT-Thread kernel, beginning with its composition and implementation. While also introducing RT-Thread kernel-related concepts for beginners.

After understanding this chapter, readers will have an elementary understanding of the RT Thread kernel and will be able to answer questions such as:

- What are the constituents of the kernel?
- How does the system startup?
- How is the memory distributed?
- What are the methods of kernel configuration?

In the nutshell, this is only a brief introduction to software architecture decomposition and implementation of the real-time kernel. This will give understanding and concepts of how RT-Thread kernel works togther. After learning from this chapter, readers will have basic knowledge of each kernel components, system booting up proccesses, memory allocation and distrubtion, and methods of kernel configuration.

1.3.1.1 Introduction to RT-Thread Kernel

Kernel is the most basic and fundenmental part of an Operating System. Kernel service library and RT-Thread kernel libraries are interfacing between hardware and components/service layer. This includes the implementation of real-time kernel service library (`rtservice.h/kservice.c`) and other RT-Thread kernel libraries such as object management, thread management and scheduler, inter-thread communication management, clock management and memory management respectively. Below diagram is the core architecture diagram of the core kernel.

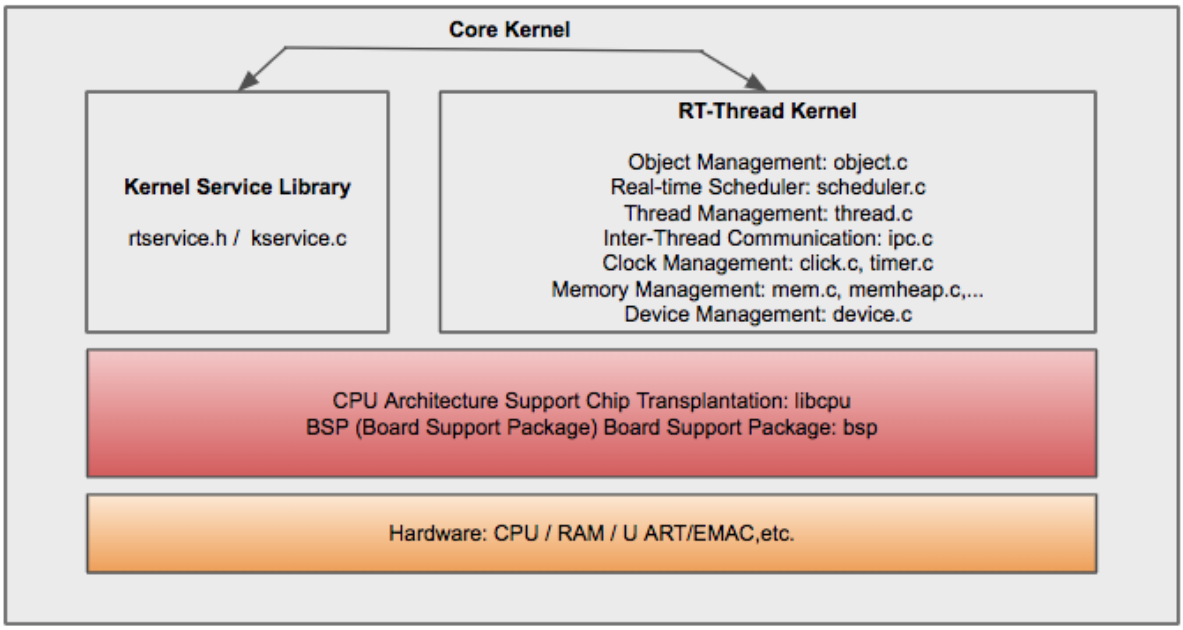


Figure 1.15 RT-Thread Kernel and its Substructure

Implementation of core kernel libraries are similar to a small set of standard C runtime library and it can run independently. For example, Standard C library (C runtime library) provides "strcpy", "memcpy", "printf", "scanf" and other function implementations. RT-Thread kernel libraries also provide the function implementations which are mainly used by Core Kernel. However, to avoid name conflicts, specifically functions' names are preceded with rt_.

The built of the Kernel will be vary depending on the complier. For example, using GNU GCC compiler, it will use more implementation from the standard C library. Last but not least, the minimum resource requirements of the Kernel is 3KB ROM and 1.2KB RAM.

1.3.1.1.1 Object Management

The Kernel object system can access and manage all of the kernel objects, such as:

- thread
- semaphore and mutex
- event/fast event, mailbox, messagequeue
- memory pool
- timer
- .....

Object management will be covered in detail in the chapter Object Management.

1.3.1.1.2 Thread Scheduling

Thread is the smallest scheduling unit in the RT-Thread operating system. The thread scheduling algorithm is a **Priority-based Full Preemptive Multi-Thread** scheduling algorithm.

The system can support up to 256(0 - 255) thread priorities. For systems with tight resources, configurations with 8 or 32 thread priorities can be chosen(For example, STM32 has 32 thread priorities as per the default configuration). Lower numbers have a higher priority where 0 represents the highest priority furthermore the lowest priority(highest number) is reserved for idle threads.

RT-Thread supports the creation of multiple threads with the same priority. Threads having the same priority are scheduled with a Time Slice Rotation Scheduling algorithm so that each thread runs for the same amount of time.

The number of threads is bounded by the memory of the hardware platform and not the system.

Thread management will be covered in detail in the chapter Thread Management.

1.3.1.1.3 Clock & Timer Management

RT-Thread's Clock management is based upon a **clock beat**, which is the smallest clock unit in the RT-Thread operating system.

The RT-Thread timer provides two types of timer mechanisms:

- **One-Shot Timer** - Triggers only one timer event after startup and then stops automatically.
- **Periodic Trigger Timer** - Periodically triggers timer events until the user manually stops the timer or it will continue to operate.

The RT-Thread timer can be set to the `HARD_TIMER` or the `SOFT_TIMER` mode depending on the context in which the timeout function is executed.

The timer service is concluded using a timer timing callback i.e. a timeout function. The user can select the appropriate type of timer according to their real-time requirements for timing processing.

Timer will be explained further in the chapter Clock & Timer Management.

1.3.1.1.4 Synchronization between Threads

RT-Thread uses thread semaphores, mutexes, and event sets to achieve inter-thread synchronization.

Thread synchronizations happen through the acquisition and release of semaphore and mutexes.

The mutex uses priority inheritance to solve the common priority inversion problem in the real-time system. The thread synchronization mechanism allows threads to wait according to priorities or to acquire semaphores/mutexes following the First In First Out(FIFO) method.

Event sets are primarily used for synchronization between threads, they can achieve one-to-many and many-to-many synchronization. It allows **OR trigger** (*independent synchronization*) and **AND trigger**(*associative synchronization*) suitable for situations where threads are waiting for multiple events.

The concepts of semaphores, mutexes, and event sets are detailed in the chapter Inter-thread Synchronization.

1.3.1.1.5 Inter-Thread Communication

RT-Thread supports communication mechanisms such as mailbox, message queue, etc. The mailbox's message length is fixed to 4 bytes. Whereas, message queue can receive messages in variable size and cache the messages in its own memory space.

Compared to a message queue, a mailbox is more efficient. The sending action of the mailbox and message queue can be safely used in an ISR (Interrupt Service Routine). The communication mechanism allows threads to wait by priority or to acquire by the First In First Out (FIFO) method.

The concept of mailbox and message queue will be explained in detail in the chapter Inter-thread Communication.

1.3.1.1.6 Memory Management

RT-Thread allows:

1. Static Memory Pool
2. Dynamic Memory Pool

When the static memory pool has available memory, the time allocated to the memory block will be constant.

When the static memory pool is empty, the system will then request for suspending or blocking the thread of the memory block. The thread will abandon the request and return if the memory block is not obtained after waiting for a while, or the thread will abandon and return immediately. The waiting time depends on the waiting time parameter set when the memory block is applied. When other threads release the memory block to the memory pool, the system will wake up the thread if there are suspended threads waiting to be allocated of memory blocks.

Under circumstances of different system resources, the dynamic memory heap management module respectively provides memory management algorithms for small memory systems and SLAB memory management algorithms for large memory systems.

There is also a dynamic memory heap management called memheap, suitable for memory heaps in systems with multiple addresses that can be discontinuous. Using memheap, the user can "paste" multiple memory heaps together, letting them operate as if operating a memory heap.

The concept of memory management will be explained in the chapter Memory Management.

1.3.1.1.7 I/O Device Management

RT-Thread uses I2C, SPI, USB, UART, etc., as peripheral devices and is uniformly registered through the device. It realized a device management subsystem accessed by the name, and it can access hardware devices according to a unified API interface. On the device driver interface, depending on the characteristics of the embedded system, corresponding events can be attached to different devices. The driver notifies the upper application program when the device event is triggered.

The concept of I/O device management will be explained in the chapter I/O Device Framework.

1.3.1.2 RT-Thread Startup Process

The understanding of most codes usually starts from learning the startup process. We will firstly look for the source of the startup. Taking MDK-ARM as an example, the user program entry for MDK-ARM is the `main()` function located in the `main.c` file. The launching of the system starts from the assembly code `startup_stm32f103xe.s`, jumps to the C code, initializes the RT-Thread system function, and finally enters the user program entry `main()`.

To complete the RT-Thread system function initialization before entering `main()`, we used the MDK extensions `$Sub$$` and `$Super$$`. Users can add the prefix of `$Sub$$` to main to make it a new function `$Sub$$main`.

`$Sub$$main` can call some functions to be added before main (here, RT-Thread system initialization function is added). Then, call `$Super$$main` to the `main()` function so that the user does not have to manage the system initialization before main().

For more information on the use of the `$Sub$$` and `$Super$$` extensions, see the ARM® Compiler v5.06 for Vision®armlink User Guide.

Let's take a look at this code defined in components.c:

```
/* $Sub$$main Function */
int $Sub$$main(void)
{
  rtthread_startup();
  return 0;
}
```

Here, the `$Sub$$main` function simply calls the `rtthread_startup()` function. RT-Thread allows multiple platforms and multiple compilers, and the `rtthread_startup()` function is a uniform entry point specified by RT-Thread, so the `$Sub$$main` function only needs to call the `rtthread_startup()` function (RT-Thread compiled using compiler GNU GCC is an example where it jumps directly from the assembly startup code section to the `rtthread_startup()` function and starts the execution of the first C code).

The `rtthread_startup()` function can be found in the code of `components.c`, the startup process of RT-Thread is as shown below:

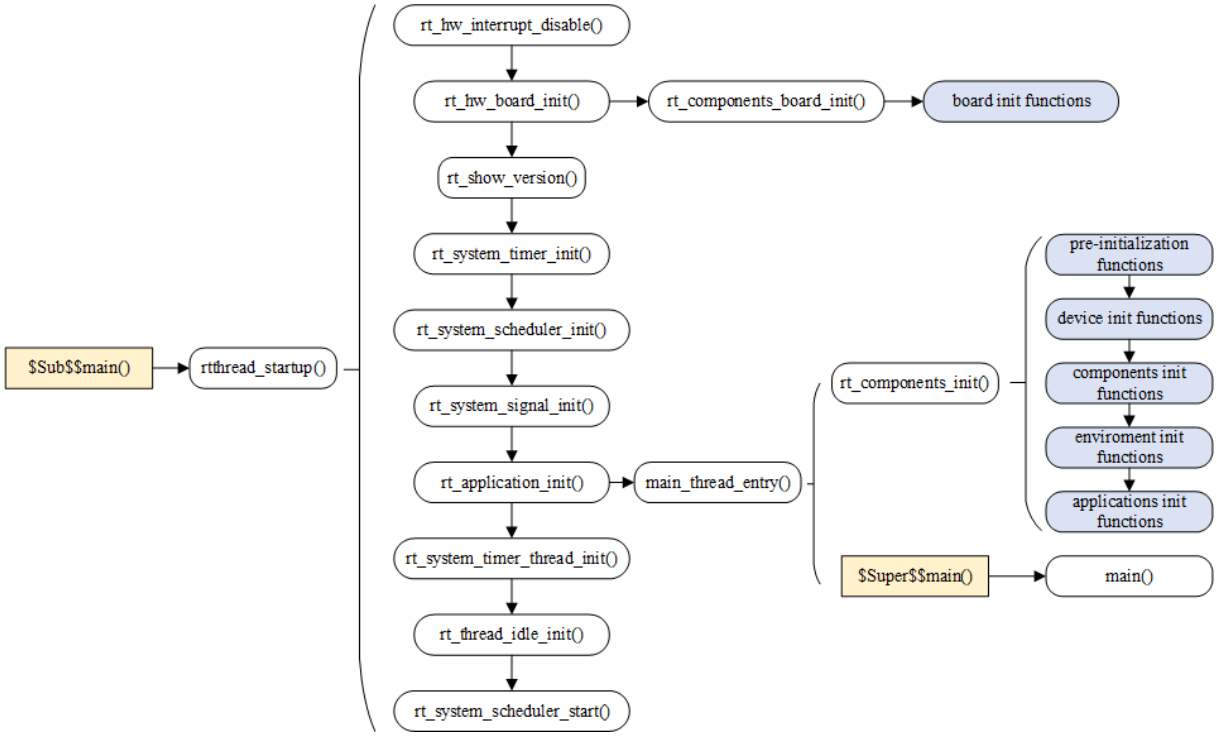


Figure 1.16 System startup process

Code for the `rtthread_startup()` function is as follows:

```
int rtthread_startup(void)
{
    rt_hw_interrupt_disable();

    /* Board level initialization: system heap initialization is required inside the function */
    rt_hw_board_init();

    /* Print RT-Thread version information */
    rt_show_version();

    /* Timer initialization */
    rt_system_timer_init();

    /* Scheduler initialization */
    rt_system_scheduler_init();

#ifdef RT_USING_SIGNALS
    /* Signal initialization */
    rt_system_signal_init();
#endif

    /* Create a user main() thread here */
    rt_application_init();

    /* Timer thread initialization */
    rt_system_timer_thread_init();

    /* Idle thread initialization */
    rt_thread_idle_init();

    /* Start scheduler */
    rt_system_scheduler_start();

    /* Will not execute till here */
    return 0;
}
```

This part of the startup code can be roughly divided into four parts:

1. Initialize hardware related to the system.
2. Initialize system kernel objects, such as timers, schedulers, and signals.
3. Create the main thread initialize various modules in the main thread one by one.
4. Initialize the timer thread, idle thread, and start the scheduler.

Set the system clock in `rt_hw_board_init()` to provide heartbeat and serial port initialization for the system, bound to the system's input and output terminals to this serial port. Subsequent system operation information will be printed out from the serial port later.

The `main()` function is the user code entry for RT-Thread, and users can add their own applications to the `main()` function.

```
int main(void)
{
  /* user app entry */
  return 0;
}
```

1.3.1.3 RT-Thread Program Memory Distribution

The general MCU contains storage space that includes the on-chip Flash and the on-chip RAM. RAM is equivalent to memory, and Flash is comparable to a hard disk. The compiler classifies a program into several parts stored in different memory areas of the MCU.

After the Keil project is compiled, there will be a prompt stating the occupied space by the corresponding program, for example:

```
linking...
Program Size: Code=48008 RO-data=5660 RW-data=604 ZI-data=2124
After Build - User command \#1: fromelf --bin.\\build\\rtthread-stm32.axf--output rtthread.bin
".\\build\\rtthread-stm32.axf" - 0 Error(s), 0 Warning(s).
Build Time Elapsed: 00:00:07
```

The Program Size mentioned above contains the following sections:

1. **Code**: Code Segment, section of code that stores the program
2. **RO-data**: Read-Only data segment, stores the constants defined in the program
3. **RW-data**: Read and Write data segment, stores global variables initialized to non-zero values
4. **ZI-data**: 0 data segment, stores uninitialized global variables and initialized-to-0 variables

After compiling the project, there will be a generated .map file that describes the size and address of each function. The last few lines of the file also illustrate the relationship between the above fields:

```
Total RO Size (Code + RO Data) 53668 ( 52.41kB)
Total RW Size (RW Data + ZI Data) 2728 ( 2.66kB)
Total ROM Size (Code + RO Data + RW Data) 53780 ( 52.52kB)
```

1. RO Size contains Code and RO-data, indicating the size of the Flash occupied by the program
2. RW Size contains RW-data and ZI-data, indicating the size of the RAM occupied when operating
3. ROM Size contains Code, RO Data, and RW Data. Indicating the size of the Flash occupied by the programming system;

Before the program runs, the file entity needs to be flashed into the STM32 Flash, usually a bin or hex file. The burned file is called an executable image file. The left part in the figure below shows the memory distribution after the executable image file is flashed into STM32 which includes the RO and RW segments. The RO segment stores data of Code and RO-data, and the RW segment holds the data of RW-data. Since ZI-data is 0, it is not included in the image file.

STM32 is launched from Flash by default after power-on. After launching, RW-data (initialized global variable) the RW segment is transferred to RAM, but the RO segment is not transferred. Meaning the execution code of the CPU is read from the Flash. The ZI segment is allocated according to the ZI address and size is given by the compiler, and the RAM area is cleared.

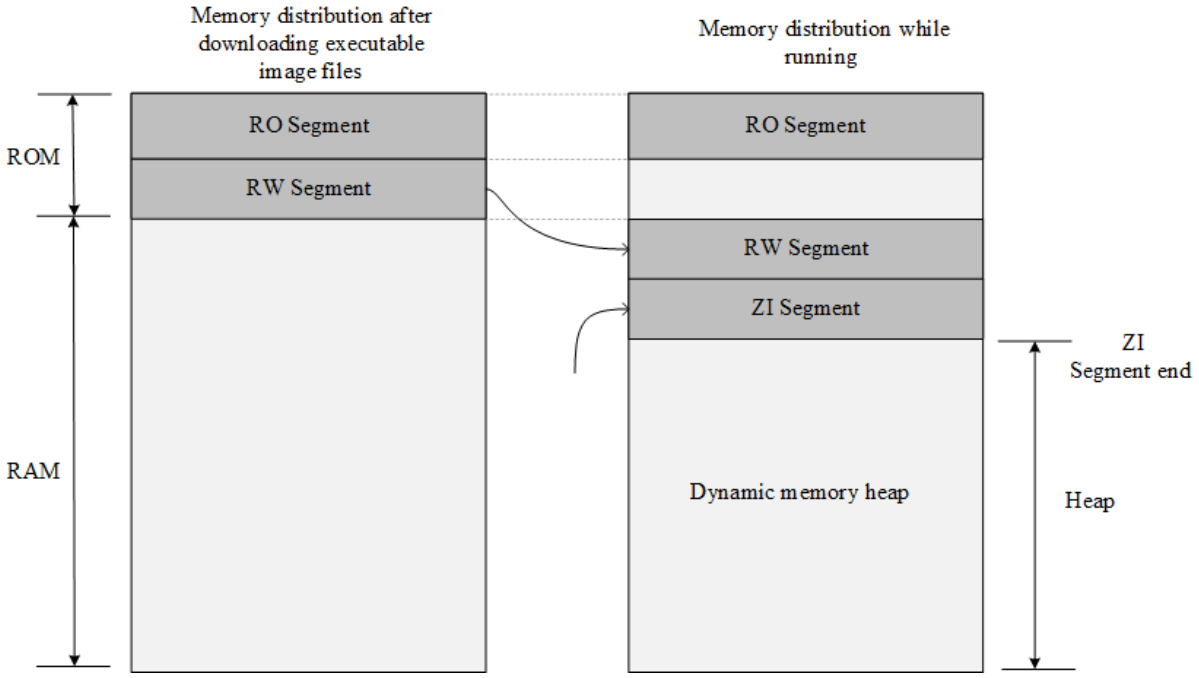


Figure 1.17 RT-Thread Memory Distribution

The dynamic memory heap is unused RAM space, and the memory blocks requested and released by the application come from this space.

Like the following example:

```
rt_uint8_t* msg_ptr;
msg_ptr = (rt_uint8_t*) rt_malloc (128);
rt_memset(msg_ptr, 0, 128);
```

The 128-byte memory space pointed by the `msg_ptr` pointer in the code is in the dynamic memory heap space.

Some global variables are stored in the RW segment and the ZI segment. The RW segment holds the global variable with the initial value (the global variable in the constant form is placed in the RO segment, which is a read-only property), and uninitialized global variable is stored in the ZI segment, as in the following example:

```
#include <rtthread.h>

const static rt_uint32_t sensor_enable = 0x000000FE;
rt_uint32_t sensor_value;
rt_bool_t sensor_inited = RT_FALSE;

void sensor_init()
{
    /* ... */
}
```

The `sensor_value` is stored in the ZI segment and is automatically initialized to zero after system startup (some library functions provided by the user program or compiler are initialized to zero). The sensor_inited variable is stored in the RW segment, and the sensor_enable is stored in the RO segment.

1.3.1.4 RT-Thread Automatic Initialization Mechanism

The automatic initialization mechanism means that the initialization function does not need to be called by explicit function. It only needs to be declared by macro definition at the function definition, and it will be executed during system startup.

For example, calling a macro definition in the serial port driver to inform the function that needs to be called to initialize the system. The code is as follows:

```
int rt_hw_usart_init(void)  /* Serial port initialization function */
{
    ... ...
    /* Register serial port 1 device */
    rt_hw_serial_register(&serial1, "uart1",
                          RT_DEVICE_FLAG_RDWR | RT_DEVICE_FLAG_INT_RX,
                          uart);
    return 0;
}
INIT_BOARD_EXPORT(rt_hw_usart_init);    /* Use component auto-initialization mechanism */
```

The last part of the sample code `INIT_BOARD_EXPORT(rt_hw_usart_init)` indicates that the automatic initialization function is used. In this way, `rt_hw_usart_init()` function is automatically called by the system, so where is it called?

In the system startup flowchart, there are two functions: `rt_components_board_init()` and `rt_components_init()`, subsequent functions inside the box with the underlying color represent functions that are automatically initialized, where:

1. “board init functions” are all initialization functions declared by `INIT_BOARD_EXPORT(fn)`.
2. “pre-initialization functions” are all initialization functions declared by `INIT_PREV_EXPORT(fn)`.
3. “device init functions” are all initialization functions declared by `INIT_DEVICE_EXPORT(fn)`.
4. “components init functions” are all initialization functions declared by `INIT_COMPONENT_EXPORT(fn)`.
5. “enviroment init functions” are all initialization functions declared by `INIT_ENV_EXPORT(fn)`.
6. “application init functions” are all initialization functions declared by `INIT_APP_EXPORT(fn)`.

The `rt_components_board_init()` function executes earlier, mainly to initialize the relevant hardware environment. When this function is executed, it will traverse the initialization function table declared by `INIT_BOARD_EXPORT(fn)` and call each function.

The `rt_components_init()` function is called and executed in the main thread created after the operating system is running. At this time, the hardware environment and the operating system have been initialized and the application-related code can be executed. The `rt_components_init()` function will transverse through the remaining few initialization function tables declared by macros.

RT-Thread's automatic initialization mechanism uses a custom RTI symbol segment, it puts the function pointer that needs to be initialized at startup into this segment and forms an initialization function table which will be traversed during system startup. It calls the functions in the table to achieve the purpose of automatic initialization.

The macro interface definitions used to implement the automatic initialization function are described in the following table:

Initialization sequence	Macro Interface	Description
1	INIT_BOARD_EXPORT(fn)	Very early initialization, the scheduler has not started yet.
2	INIT_PREV_EXPORT(fn)	Mainly used for pure software initialization, functions without too many dependencies
3	INIT_DEVICE_EXPORT(fn)	Peripheral driver initialization related, such as network card devices
4	INIT_COMPONENT_EXPORT(fn)	Component initialization, such as file system or LWIP
5	INIT_ENV_EXPORT(fn)	System environment initialization, such as mounting file systems
6	INIT_APP_EXPORT(fn)	Application initialization, such as application GUI

Initialization function actively declares through these macro interfaces, such as `INIT_BOARD_EXPORT(rt_hw_usart_init)`, the linker will automatically collect all the declared initialization functions, placed in the RTI symbol segment, the symbol segment is located in the RO segment of the memory distribution. All functions in this RTI symbol segment are automatically called when the system is initialized.

1.3.1.5 RT-Thread Kernel Configuration Example

An important feature of RT-Thread is its high degree of tailorability, which allows for fine-tuning of the kernel and flexible removal of components.

Configuration is mainly done by modifying the file under project directory - `rtconfig.h`. User can conditionally compile the code by opening/closing the macro definition in the file, and finally achieve the purpose of system configuration and cropping, as follows:

1RT-Thread Kernel part

```
/* Indicates the maximum length of the name of the kernel object. If the maximum length of the name of the
      object in the code is greater than the length of the macro definition,
 * the extra part will be cut off. */
#define RT_NAME_MAX 12

/* Set the number of aligned bytes when bytes are aligned. Usually use ALIGN(RT_ALIGN_SIZE) for byte
      alignment.*/
#define RT_ALIGN_SIZE 8

/* Define the number of system thread priorities; usually define the priority of idle threads with
      RT_THREAD_PRIORITY_MAX-1 */
```

```
#define RT_THREAD_PRIORITY_MAX 32

/* Define the clock beat. When it is 100, it means 100 tick per second, and a tick is 10ms. */
#define RT_TICK_PER_SECOND 100

/* Check if the stack overflows, if not defined, close. */
#define RT_USING_OVERFLOW_CHECK

/* Define this macro to enable debug mode, if not defined, close.  */
#define RT_USING_DEBUG
/* When debug mode is enabled: Define this macro to enable the print component initialization information,
      if not defined, close.  */
#define RT_DEBUGING_AUTO_INIT

/* Defining this macro means the use of the hook function is started, if not defined, close. */
#define RT_USING_HOOK

/* Defines the stack size of idle threads. */
#define IDLE_THREAD_STACK_SIZE 256
```

2Inter-thread synchronization and communication part, the objects that will be used in this part are semaphores, mutexes, events, mailboxes, message queues, signals, and so on.

```
/* Define this macro to enable the use of semaphores, if not defined, close. */
#define RT_USING_SEMAPHORE

/* Define this macro to enable the use of mutexes, if not defined, close. */
#define RT_USING_MUTEX

/* Define this macro to enable the use of events, if not defined, close. */
#define RT_USING_EVENT

/* Define this macro to enable the use of mailboxes, if not defined, close. */
#define RT_USING_MAILBOX

/* Define this macro to enable the use of message queues, if not defined, close. */
#define RT_USING_MESSAGEQUEUE

/* Define this macro to enable the use of signals, if not defined, close. */
#define RT_USING_SIGNALS
```

3Memory Management Part

```
/* Start the use of static memory pool */
#define RT_USING_MEMPOOL

/* Define this macro to start the concatenation of two or more memory heap , if not defined, close. */
#define RT_USING_MEMHEAP

/* Start algorithm for small memory management */
#define RT_USING_SMALL_MEM

/* Turn off SLAB memory management algorithm */
/* #define RT_USING_SLAB */

/* Start the use of heap */
#define RT_USING_HEAP
```

4Kernel Device Object

```
/* Indicates the start of useing system devices */
#define RT_USING_DEVICE

/* Define this macro to start the use of system console devices, if not defined, close. */
#define RT_USING_CONSOLE
/* Define the buffer size of the console device. */
#define RT_CONSOLEBUF_SIZE 128
/* Name of the console device. */
#define RT_CONSOLE_DEVICE_NAME "uart1"
```

5Automatic Initialization Method

```
/* Define this macro to enable automatic initialization mechanism, if not defined, close. */
#define RT_USING_COMPONENTS_INIT

/* Define this macro to set application entry as main function */
#define RT_USING_USER_MAIN
/* Define the stack size of the main thread */
#define RT_MAIN_THREAD_STACK_SIZE 2048
```

6FinSH

```
/* Define this macro to start the use of the system FinSH debugging tool, if not defined, close. */
```

```
#define RT_USING_FINSH

/* While starting the system FinSH: the thread name is defined as tshell */
#define FINSH_THREAD_NAME "tshell"

/* While turning the system FinSH: use history commands. */
#define FINSH_USING_HISTORY
/* While turning the system FinSH: define the number of historical command lines. */
#define FINSH_HISTORY_LINES 5

/* While turning the system FinSH: define this macro to use symbol table in Finsh, if not defined, close. */
#define FINSH_USING_SYMTAB

/* While turning the system FinSH: define the priority of the thread. */
#define FINSH_THREAD_PRIORITY 20
/* While turning the system FinSHdefine the stack size of the thread. */
#define FINSH_THREAD_STACK_SIZE 4096
/* While turning the system FinSHdefine the length of command character. */
#define FINSH_CMD_SIZE 80

/* While turning the system FinSH: define this macro to enable the MSH function. */
#define FINSH_USING_MSH
/* While turning the system FinSHwhen MSH function is enabled, macro is defined to use the MSH function by
    default. */
#define FINSH_USING_MSH_DEFAULT
/* While turning the system FinSHdefine this macro to use only the MSH function. */
#define FINSH_USING_MSH_ONLY
```

7About MCU

```
/* Define the MCU used in this project is STM32F103ZE; the system defines the chip pins by defining the chip
    type. */
#define STM32F103ZE

/* Define the clock source frequency. */
#define RT_HSE_VALUE 8000000

/* Define this macro to enable the use of UART1. */
#define RT_USING_UART1
```

In practice, the system configuration file `rtconfig.h` is automatically generated by configuration tools and does not need to be changed manually.

1.3.1.6 Common Macro Definition Description

Macro definitions are often used in RT-Thread. For example, some common macro definitions in the Keil compilation environment:

1`rt_inline`, definition is as follows, static keyword is to make the function only available for use in the current file; inline means inline, after modification using static, the compiler is recommended to perform inline expansion when calling the function.

```
#define rt_inline                   static __inline
```

2`rt_used`definition is as follows, the purpose of this macro is to explain to the compiler that this code is useful, compilation needs to be saved even if it is not called in the function. For example, RT-Thread auto-initialization uses custom segments, using `rt_used` will retain custom code snippets.

```
#define rt_used                     __attribute__((used))
```

3`RT_UNUSED`definition is as follows, indicates that a function or variable may not be used. This attribute prevents the compiler from generating warnings.

```
#define RT_UNUSED                   __attribute__((unused))
```

4`rt_weak`definition is as follows, often used to define functions, when linking the function, the compiler will link the function without the keyword prefix first and link the function modified by weak if it can't find those functions.

```
#define rt_weak                     __weak
```

5`ALIGN(n)` definition is as follows, is used to align its stored address with n bytes when allocating an address space to an object. Here, `n` can be the power of 2. Byte alignment not only facilitates quick CPU access, but also save memory space if byte alignment is properly used.

```
#define ALIGN(n)                    __attribute__((aligned(n)))
```

6`RT_ALIGN(size,align)` definition is as follows, to increase size to a multiple of an integer defined by align. For example, `RT_ALIGN(13,4)` will return to 16.

```
#define RT_ALIGN(size, align)       (((size) + (align) - 1) & ~((align) - 1))
```

1.3.2 Object Management

1.3.2.1 Static and Dynamic Objects

The RT-Thread kernel is designed with object-oriented method. The system-level infrastructures are all kernel objects such as threads, semaphores, mutexes, timers, and more. Kernel objects fall into two categories: static kernel objects and dynamic kernel objects. Static kernel objects are usually placed in RW and ZI segments, initialized in the program after system startup; dynamic kernel objects are created from the memory heap and then manually initialized.

The following code is an example of static threads and dynamic threads:

```
/* Thread 1 object and stack used while running */
static struct rt_thread thread1;
static rt_uint8_t thread1_stack[512];

/* Thread 1 entry */
void thread1_entry(void* parameter)
{
    int i;

    while (1)
    {
        for (i = 0; i < 10; i ++)
        {
            rt_kprintf("%d\n", i);

            /* Delay 100ms */
            rt_thread_mdelay(100);
        }
    }
}

/* Thread 2 entry */
void thread2_entry(void* parameter)
{
     int count = 0;
     while (1)
     {
        rt_kprintf("Thread2 count:%d\n", ++count);

        /* Delay 50ms */
        rt_thread_mdelay(50);
    }
}

/* Thread routine initialization */
int thread_sample_init()
{
     rt_thread_t thread2_ptr;
     rt_err_t result;

    /* Initialize thread 1 */
    /* The thread entry is thread1_entry and the parameter is RT_NULL
     * Thread stack is thread1_stack
     * Priority is 200 and time slice is 10 OS Tick
     */
    result = rt_thread_init(&thread1,
                            "thread1",
                            thread1_entry, RT_NULL,
                            &thread1_stack[0], sizeof(thread1_stack),
                            200, 10);

    /* Start thread */
    if (result == RT_EOK) rt_thread_startup(&thread1);

    /* Create thread 2 */
    /* The thread entry is thread2_entry and the parameter is RT_NULL
     *  Stack space is 512, priority is 250, and time slice is 25 OS Tick
     */
    thread2_ptr = rt_thread_create("thread2",
                                   thread2_entry, RT_NULL,
                                   512, 250, 25);

    /* Start thread */
    if (thread2_ptr != RT_NULL) rt_thread_startup(thread2_ptr);

    return 0;
}
```

In this example, `thread1` is a static thread object and `thread2` is a dynamic thread object. The memory space of the `thread1` object, including the thread control block `thread1` and the stack space `thread1_stack` are all determined while compiling, because there is no initial value in the code and they are uniformly placed in the uninitialized data segment. The space used by `thread2` is dynamically allocated includes the thread control block (the content pointed to by `thread2_ptr`) and the stack space.

Static objects take up RAM space and is not depend on the memory heap manager. When allocating static objects, the time needed is determined. Dynamic objects depend on the memory heap manager. It requests RAM space while running. When the object is deleted, the occupied RAM space is released. These two methods have their own advantages and disadvantages, and can be selected according to actual needs.

1.3.2.2 Object Management Structure

RT-Thread uses the kernel object management system to access/manage all kernel objects. Kernel objects contain most of the facilities in the kernel. These kernel objects can be statically allocated static objects and dynamic objects allocated from the system memory heap. .

Because of this design for kernel object, RT-Thread is able to not depend on the specific memory allocation method, and the flexibility of the system is greatly improved.

RT-Thread kernel objects include: threads, semaphores, mutexes, events, mailboxes, message queues and timers, memory pools, device drivers, and more. The object container contains information about each type of kernel object, including object type, size, and so on. The object container assigns a linked list to each type of kernel object. All kernel objects are linked to the linked list. The kernel object container and linked list of RT-Thread are shown in the following figure:

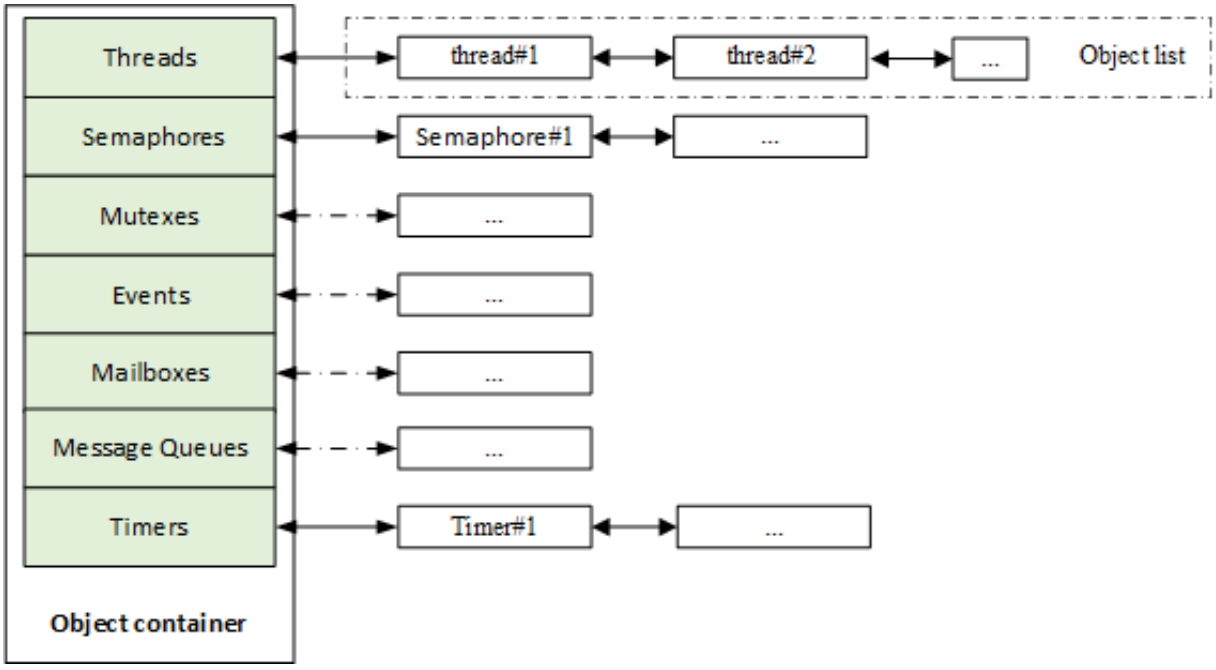


Figure 1.18 RT-Thread Kernel Object Container and Linked List

The following figure shows the derivation and inheritance relationships of various kernel objects in RT-Thread. For each specific kernel object and object control block, in addition to the basic structure, they have their own extended attributes (private attributes). Take thread control block for an example, the base object is extended, attributes like thread state, precedence and so on are added. These attributes are not used in the operation of the base class object and are only used in operations related to a specific thread. Therefore, from the object-oriented point of view, each concrete object can be considered as a derivative of an abstract object, inheriting the attributes of the base object and extending the attributes related to itself.

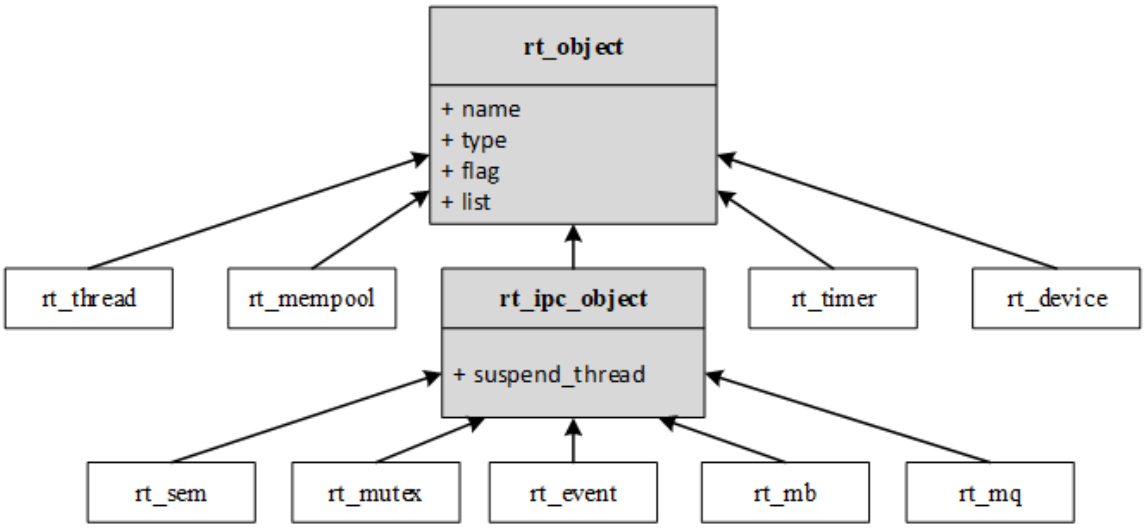


Figure 1.19 RT-Thread Kernel Object Inheritance Relationship

In the object management module, a common data structure is defined to store the common attributes of various objects. Each specific object only needs to add some special attributes of its own and its own feature will be clearly expressed.

The advantages of this design approach are:

(1) Improve the reusability and scalability of the system. It is easy to add new object categories. It only needs to inherit the attributes of the general object and add a small amount of extension.

(2) Provide a unified object operation mode, simplify the operation of various specific objects, and improve the reliability of the system.

Derivations from object control block rt_object in the above figure includes: thread object, memory pool object, timer object, device object and IPC object (IPC: Inter-Process Communication. In RT-Thread real-time operating system, IPC objects is used for synchronization and communicate between threads); derivations from IPC objects includes: semaphores, mutexes, events, mailboxes, message queues, signals, etc.

1.3.2.3 Object Control Block

A simple code example snippet for data structure of kernel object control block:

```
struct rt_object
{
    /* Kernel object name     */
    char      name[RT_NAME_MAX];
    /* Kernel object type     */
    rt_uint8_t  type;
    /* Parameters to the kernel object   */
    rt_uint8_t  flag;
    /* Kernel object management linked list */
    rt_list_t   list;
};
```

More details about this structure, see `struct rt_object`.

Regarding types currently supported by kernel objects, see:

```
enum rt_object_class_type
```

From the above type specification, we can see that if it is a static object, the highest bit of the object type will be 1 (which is the OR operation of `RT_Object_Class_Static` and other object types and operations). Otherwise it will be dynamic object, and the maximum number of object classes that the system can accommodate is 127.

1.3.2.4 Object Management APIs

Data structure of kernel object container:

```
struct rt_object_information
{
    /* Object type */
    enum rt_object_class_type type;
    /* Object linked list */
    rt_list_t object_list;
    /* Object size */
    rt_size_t object_size;
};
```

A class of objects is managed by an `rt_object_information` structure, and each practical instance of such type of object is mounted to the `object_list` in the form of a linked list. The memory block size of this type of object is identified by `object_size` (the memory block each practical instance of each type of object is the same size).

1.3.2.4.1 Initialization Object

An uninitialized static object must be initialized before it can be used. The initialization object uses the following interfaces:

```
void rt_object_init(struct  rt_object*  object ,
                    enum rt_object_class_type  type ,
                    const char* name)
```

When this function is called to initialize the object, the system will place the object into the object container for management, that is, initialize some parameters of the object, and then insert the object node into the object linked list of the object container.

More details about this function, see `rt_object_init()`.

1.3.2.4.2 Detach Object

Detach an object from the kernel object manager. The following interfaces are used to detach objects:

```
void rt_object_detach(rt_object_t object);
```

Calling this interface makes a static kernel object to be detached from the kernel object container, meaning the corresponding object node is deleted from the kernel object container linked list. After the object is detached, the memory occupied by the object will not be released.

More details about this function, see `rt_object_detach()`.

1.3.2.4.3 Allocate object

The above descriptions are interfaces of objects initialization and detachment, both of which are under circumstances that object-oriented memory blocks already exist. But dynamic objects can be requested when needed. The memory space is freed for other applications when not needed. To request assigning new objects, you can use the following interfaces:

```
rt_object_t rt_object_allocate(enum rt_object_class_type type, const char* name)
```

When calling the above interface, the system first needs to obtain object information according to the object type (especially the size information of the object type for the system to allocate the correct size of the memory data block), and then allocate memory space corresponding to the size of the object from the memory heap. Next, to start necessary initialization for the object, and finally insert it into the object container linked list in which it is located.

More details about this function, see `rt_object_allocate()`.

1.3.2.4.4 Delete Object

For a dynamic object, when it is no longer used, you can call the following interface to delete the object and release the corresponding system resources:

```
void rt_object_delete(rt_object_t object);
```

When the above interface is called, the object is first detached from the object container linked list, and then the memory occupied by the object is released.

More details about this function, see `rt_object_delete()`.

1.3.2.4.5 Identify objects

Identify whether the specified object is a system object (static kernel object). The following interface is used to identify the object:

```
rt_err_t rt_object_is_systemobject(rt_object_t object);
```

Calling the `rt_object_is_systemobject` interface can help to identify whether an object is a system object. In RT-Thread operating system, a system object is also a static object, `RT_Object_Class_Static` bit is set to 1 on the object type identifier. Usually, objects that are initialized using the `rt_object_init()` method are system objects.

More details about this function, see `rt_object_is_systemobject()`.

1.3.3 Thread Management

When we are facing a big task in our daily life, we usually break it down into a number of simple, easy-to-manage smaller tasks. Then, we would deal with these smaller tasks one by one, gradually, the big task is worked out. In a multi-threaded operating system, developers also need to break down a complex application into multiple small, schedulable, and serialized program units. When tasks are reasonably divided and properly executed, this design allows the system to meet the capacity and time requirements of the real-time system. For example, to have the embedded system to perform such tasks, the system would collect data through sensors and display the data on the screen. In a multi-threaded real-time system, the task can be decomposed into two subtasks. The subtask, as shown in the following figure, reads the sensor data continuously and writes the data into the shared memory. The other subtask periodically reads the data from the shared memory and outputs the sensor data onto the screen.

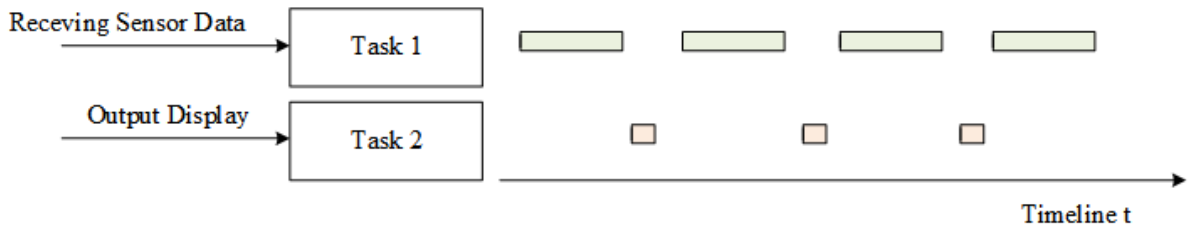


Figure 1.20 Switching Execution of Sensor's Data Receiving Task and Display Task

In RT-Thread, the application entity corresponding to the above subtask is the thread. The thread is the carrier of the task. It is the most basic scheduling unit in RT-Thread. It describes the running environment of a task execution. It also describes the priority level of the task - an important task can be set a relatively high priority, while a non-important task can be set a lower priority, and different tasks can also be set to the same priority and take turns while running.

When a thread runs, it thinks it is has sole access to the CPU as it is running. The runtime environment when the thread executes is called the context, specifically each variables and data, including all register variables, stacks, memory information, and so on.

This chapter will be divided into five sections to introduce thread management in RT-Thread. After reading this chapter, readers will have a deeper understanding of the thread management mechanism of RT-Thread. They will have clear answers to questions like what states does a thread have, how to create a thread, why do idle threads exist, etc.

1.3.3.1 Thread Management Features

The main function of RT-Thread's thread management is to manage and schedule threads. There are two types of threads in the system, namely system threads and user threads. System threads are threads created by the RT-Thread kernel. User threads are the thread created by the application. Both types of threads will allocate thread objects from the kernel object container. When a thread is deleted, it will also be deleted from the object container. As shown in the following figure, each thread has important attributes, such as a thread control block, thread stack, entry function, and so on.

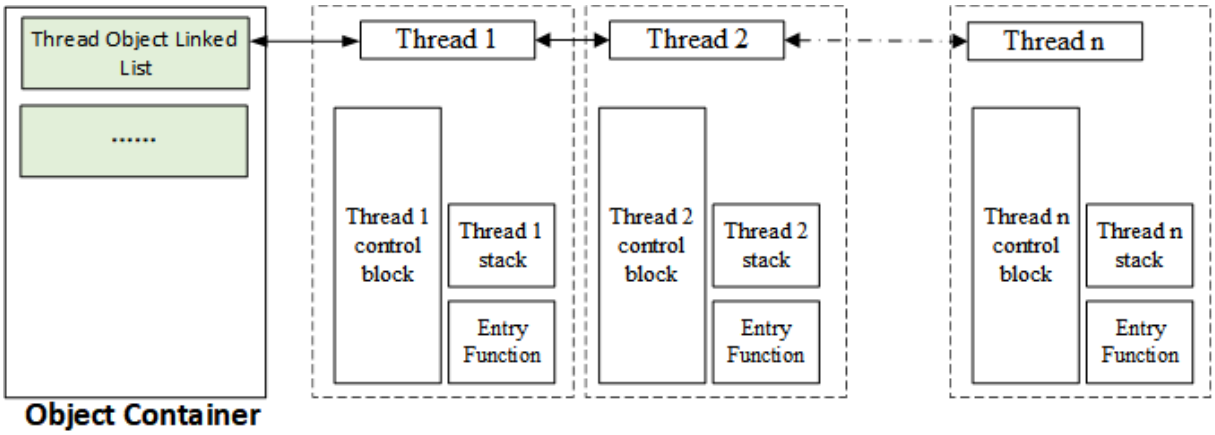


Figure 1.21 Object Container and Thread Object

The thread scheduler of RT-Thread is preemptive and its main job is to find the highest priority thread from the list of ready threads so as to ensure that the highest priority thread can be run. Once the highest priority task is ready, it can always get the CPU usage it needs.

When a running thread makes the running condition ready for another thread with a higher priority, then the current running thread's CPU usage right is deprived, or in other words, released, and the high-priority thread immediately gets CPU access.

If it is the interrupt service routine that makes the running condition ready for the thread with a higher priority, when the interrupt is completed, the interrupted thread is suspended, the thread with the higher priority starts running.

When the scheduler schedules threads and switches them, the current thread context is first saved. When it is switched back to this thread, the scheduler restores the context information of the thread.

1.3.3.2 Working Mechanism of Thread

1.3.3.2.1 Thread Control Block

In RT-Thread, the thread control block is represented by structure `struct rt_thread`, which is a data structure used by the operating system to manage threads. It stores information about the thread, such as priority, thread name, thread status, etc. It also includes a linked list structure for connecting threads, event collection of thread waiting, etc.

A simple code example snippet for data structure of thread control block:

```
struct rt_thread
{
    ......

    /* Stack Pointer and Entry pointer */
    void       *sp;                     /* Stack Pointer */
    void       *entry;                  /* Entry Function Pointer */
```

```
    void       *parameter;              /* Parameter */
    void       *stack_addr;             /* Stack Address Pointer */
    rt_uint32_t stack_size;             /* Stack Size */

    /* Error Code */
    rt_err_t    error;                  /* Thread Error Code */
    rt_uint8_t  stat;                   /* Thread State */

    /* Priority */
    rt_uint8_t  current_priority;       /* Current Priority */

    ......
};
```

More details about this structure, see `struct rt_thread`.

1.3.3.2.2 Thread Important Attributes

1.3.3.2.2.1 Thread Stack

RT-Thread's threads have a dependent stack. When the thread is switched, the context of the current thread is stored in the stack. When the thread is about to resume operation, the context information is read from the stack and recovered.

The thread stack is also used to store local variables in functions: local variables in functions are applied from the thread stack space; local variables in functions are initially allocated from registers (ARM architecture), when this function calls another function, these local variables will be placed on the stack.

For the first run of thread, the context can be constructed manually to set initial environment like: entry function (PC register), entry parameter (R0 register), return position (LR register), current machine operating status (CPSR register).

The growth direction of the thread stack is closely related to the CPU architecture. Versions before RT-Thread 3.1.0 only allow the stack to grow from high address to low address. For the ARM Cortex-M architecture, the thread stack can be constructed as shown below.

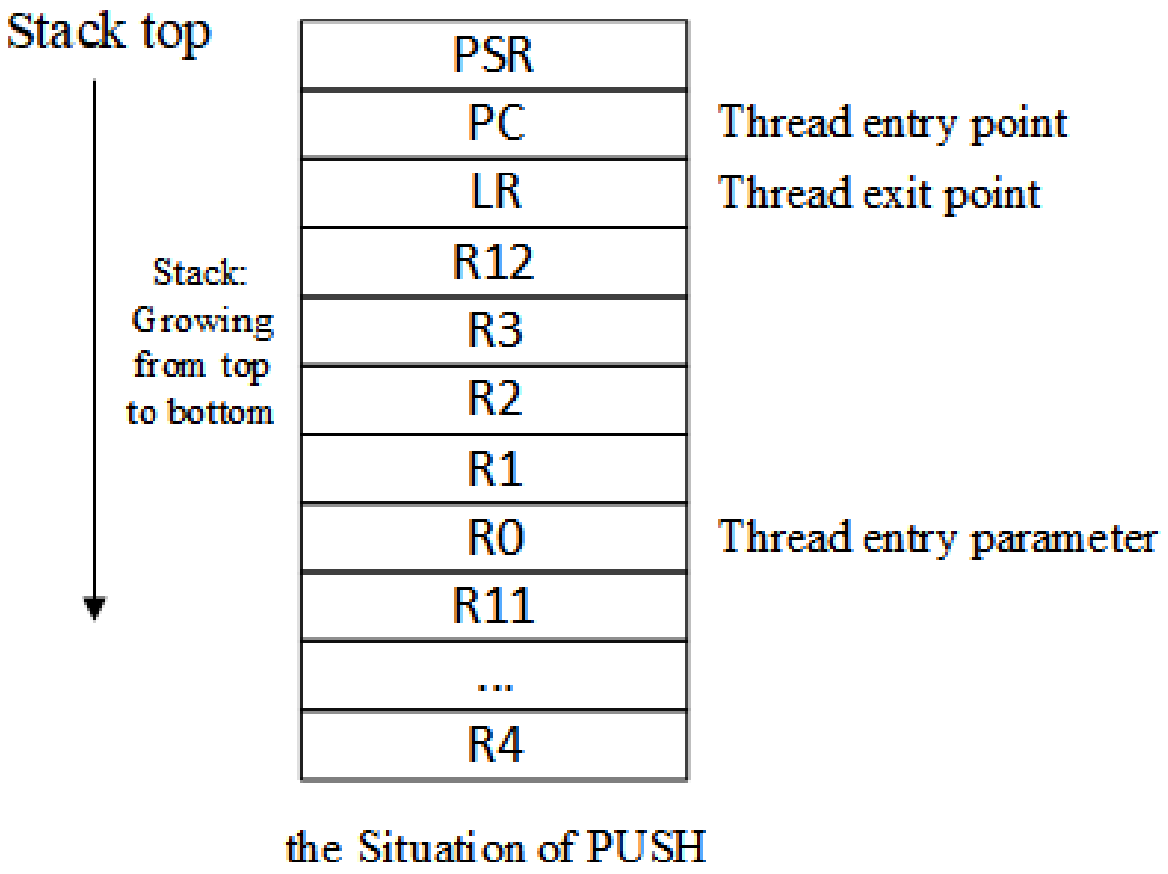


Figure 1.22 Thread Stack

When setting the size for thread stack, a larger thread stack can be designed for an MCU with a relatively large resource pool; or a larger stack can be set initially, for example, a size of 1K or 2K bytes, then in FinSH, use `list_thread` command to check the size of the stack used by the thread during the running of the thread. With this command, you can see the maximum stack depth used by the thread from the start of the thread to the current point in time, and then add the appropriate margin to form the final thread stack size, and finally modify the size of the stack space.

1.3.3.2.2.2 Thread State

Only one thread is allowed to run on a processor at any time. Apart from the running process, threads have various different operating states, such as initial state, suspended state, ready state, etc. In RT-Thread, a thread can have five states, and the operating system would automatically adjust its state based on its running condition.

The five states of a thread in RT-Thread are shown in the following table:

States	Description
Initial State	Thread is in initial state when it has just been created but has not started running; in initial state, the thread does not participate in scheduling. In RT-Thread, the macro definition of this state is `RT_THREAD_INIT`

States	Description
Ready State	In ready state, the thread is queued according to priority, waiting to be executed; the processor is available again once the current thread is finished, and the operating system will then immediately find the ready thread with the highest priority to run. In RT-Thread, the macro definition of this state is RT_THREAD_READY
Running State	Thread is currently running. In a single-core system, only the thread returned from the rt_thread_self() function is running; in a multi-core system, more than one thread may be running. In RT-Thread, the macro definition of this state is RT_THREAD_RUNNING
Suspended State	Also known as the blocking state. It may be suspended and paused because the resource is unavailable, or the thread is suspended because it is voluntarily delayed. In suspended state, threads do not participate in scheduling. In RT-Thread, the macro definition of this state is RT_THREAD_SUSPEND
Closed State	It will be turned to closed state when the thread finishes running. The thread in closed state does not participate in the thread's scheduling. In RT-Thread, the macro definition of this state is RT_THREAD_CLOSE

1.3.3.2.2.3 Thread Priority

The priority of the RT-Thread thread indicates the thread's priority of being scheduled. Each thread has its priority. The more important the thread, the higher priority it should be given, resulting in a higher chance of being scheduled.

RT-Thread supports a maximum of 256 thread priorities (0∼255). The lower the number, the higher the priority, with 0 being the highest priority. In some systems with tight resources, you can choose system configurations that only support 8 or 32 priorities according to the actual situation; for the ARM Cortex-M series, 32 priorities are commonly used. The lowest priority is assigned to idle threads by default and is not used by users. In the system, when a thread with a higher priority is ready, the current thread with the lower priority will be swapped out immediately, and the high-priority thread will preempt the processor.

1.3.3.2.2.4 Time Slice

Each thread has a time slice parameter, but time slice is only valid for ready-state threads of the same priority. The system schedules the ready-state threads with the same priority using time slice rotation scheduling method. In this case, time slice plays the role of constraining thread's single running time and the unit is a system tick (OS Tick). Suppose there are 2 ready-state threads, A and B with the same priority. Time slice of A thread is set to 10, and time slice of B thread is set to 5. When there is no ready-state thread with higher priority than A in the system, the system will switch back and forth between thread A and B. Each time the system performs 10 OS Ticks on thread A, and 5 OS Ticks on thread B, as shown below.

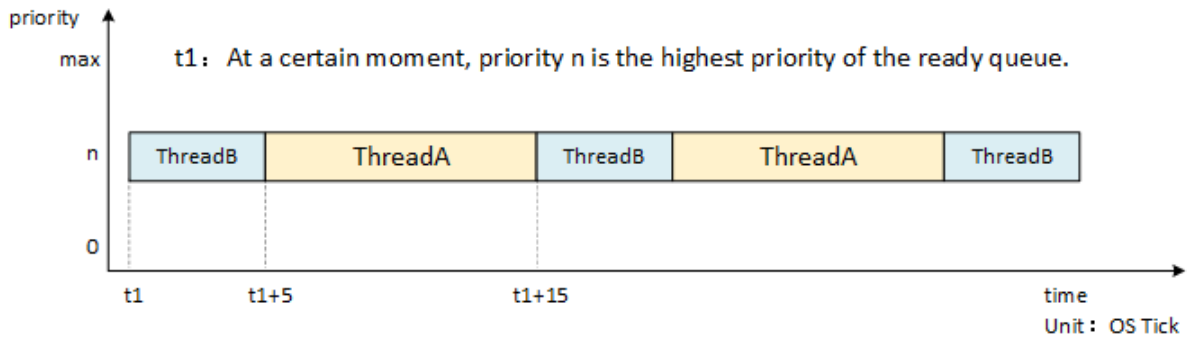


Figure 1.23 Same Priority Time Slice Round Robin

1.3.3.2.2.5 Thread Entry Function

"Entry" in Thread Control Block is the thread's entry function, which is the thread's main function. The thread's entry function is created by the user. There are generally two forms of code:

∗∗Infinite Loop Mode∗∗

In real-time systems, threads are usually passive: this is determined by the characteristics of real-time systems, which usually wait for external events to occur, and then perform the appropriate services:

```
void thread_entry(void* parameter)
{
    while (1)
    {
    /* Wait for an event to occur */

    /* Serve and process events */
    }
}
```

It may seem that threads have no restrictions on program execution and that all operations can be performed. But as a real-time system, with clear priorities, if a program in a thread is stuck in an infinite loop, then threads with a lower priorities will not be executed. Therefore, one thing that must be noted in the real-time operating system is that the thread cannot be stuck in an infinite loop operation, and there must be an action to relinquish the use of the CPU, such as calling a delay function in the loop or actively suspending. The purpose of user designing a infinite loop thread is to let this thread be continuously scheduled and run by the system and never be deleted.

Sequential Execution or Finite-Cycle Mode:

With simple sequential statements, such as `while()` or `for()` loop, etc., these threads will not cycle or permanently loop. They can be described as a "one-off" threads and will surely be executed. After it finished executing, the thread will be automatically deleted by the system.

```
static void thread_entry(void* parameter)
{
    /* Processing Transaction #1 */
    ...
    /* Processing Transaction #2 */
    ...
    /* Processing Transaction #3 */
}
```

1.3.3.2.2.6 Thread Error Code

Each thread is one execution scenario. Error codes are closely related to the execution environment, so each thread is equipped with a variable to store the error code. Some error codes are listed below as example, more definition see `include/klibc/kerrno.h`:

```
#define RT_EOK            0 /* No error     */
#define RT_ERROR          1 /* Regular error     */
#define RT_ETIMEOUT       2 /* Timeout error     */
#define RT_EFULL          3 /* Resource is full     */
#define RT_EEMPTY         4 /* No resource     */
#define RT_ENOMEM         5 /* No memory     */
#define RT_ENOSYS         6 /* System does not support     */
#define RT_EBUSY          7 /* System busy     */
#define RT_EIO            8 /* IO error       */
#define RT_EINTR          9 /* Interrupt system call   */
#define RT_EINVAL        10 /* Invalid Parameter      */
```

1.3.3.2.3 Switching Thread State

RT-Thread provides a set of operating system call interfaces that make the state of a thread to switch back and forth between these five states. The conversion relationship between these states is shown in the following figure:

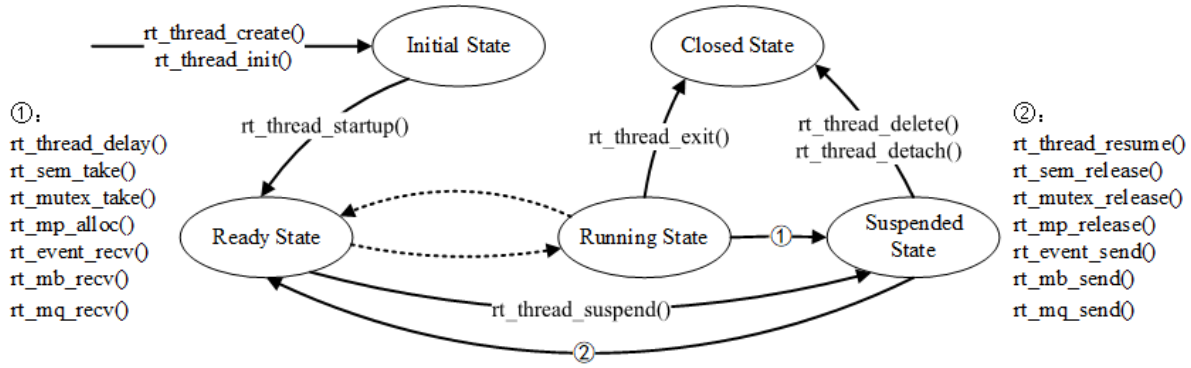


Figure 1.24 Thread State Switching Diagram

Threads enter the initial state (RT_THREAD_INIT) by calling the functions `rt_thread_create()` or `rt_thread_init()`; threads in the initial state enter the ready state (RT_THREAD_READY) by calling the function `rt_thread_startup()`; threads in the ready state are scheduled by the scheduler and enter the running state (RT_THREAD_RUNNING). When a running thread calls a function such as `rt_thread_delay()`, `rt_sem_take()`, `rt_mutex_take()`, `rt_mb_recv()` or fails to get resources, it will enter the suspended state (RT_THREAD_SUSPEND); if a thread in suspended state waits until timeout and still doesn't acquire the resources, or other threads release the resources, it will return to the ready state.

If a thread in suspended state calls the functions `rt_thread_delete()` or `rt_thread_detach()`, it will switch to the closed state (RT_THREAD_CLOSE); as for thread in the running state, if the operation is completed, the function `rt_thread_exit()` will be executed at the last part of the thread to change its state into the closed state.

>In RT-Thread, a thread does not actually have a running state; the ready state and the running state are equivalent.

1.3.3.2.4 System thread

As mentioned previously, system threads refers to threads created by the system and user threads are threads created by the user program calling the thread management interface. System threads In RT-Thread kernel include the idle thread and the main thread.

1.3.3.2.4.1 Idle Thread

An idle thread is the lowest priority thread created by the system, and its thread state is always ready. When no other ready thread exists in the system, the scheduler will schedule the idle thread, which is usually an infinite loop and can never be suspended. In addition, idle threads have special functions in RT-Thread:

If a thread finishes running, the system will automatically delete the thread by executing the function `rt_thread_exit()`, first removing the thread from the system ready queue, then changing the state of the thread to the closed state which means it no longer participates in system scheduling, and then suspend it into the `rt_thread_defunct` queue (a thread queue that is not reclaimed and in a closed state). Lastly, the idle thread reclaims the resources of the deleted thread.

The idle thread also provides an interface to run a hook function set by the user. The hook function is called when idle thread is running, which is suitable for operations like hooking into power management, watchdog feeding, etc.

1.3.3.2.4.2 Main Thread

When the system starts, it will create the main thread. Its entry function is `main_thread_entry()`. User's application entry function `main()` starts from here. After the system scheduler starts, the main thread starts running. The process is as follows. Users can add their own application initialization code to the `main()` function.

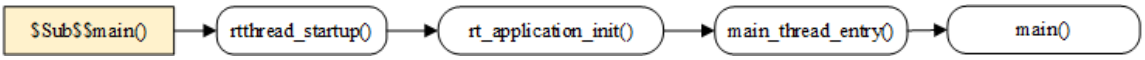


Figure 1.25 Main Thread Calling Process

1.3.3.3 Thread Management

The first two sections of this chapter conceptually explain the function and working mechanism of threads. This section will delve into the various interfaces of threads and give some source code to help the reader understand threads.

The following figure depicts related operations to threads, including creating / initializing threads, starting threads, running threads, deleting / detaching threads. You can use `rt_thread_create()` to create a dynamic thread and `rt_thread_init()` to initialize a static thread. The difference between a dynamic thread and a static thread is that for a dynamic thread, the system automatically allocates stack space and thread handles from the dynamic memory heap (only after initializing the heap can the system create a dynamic thread). As for static threads, it is the user that allocates the stack space and the thread handle.

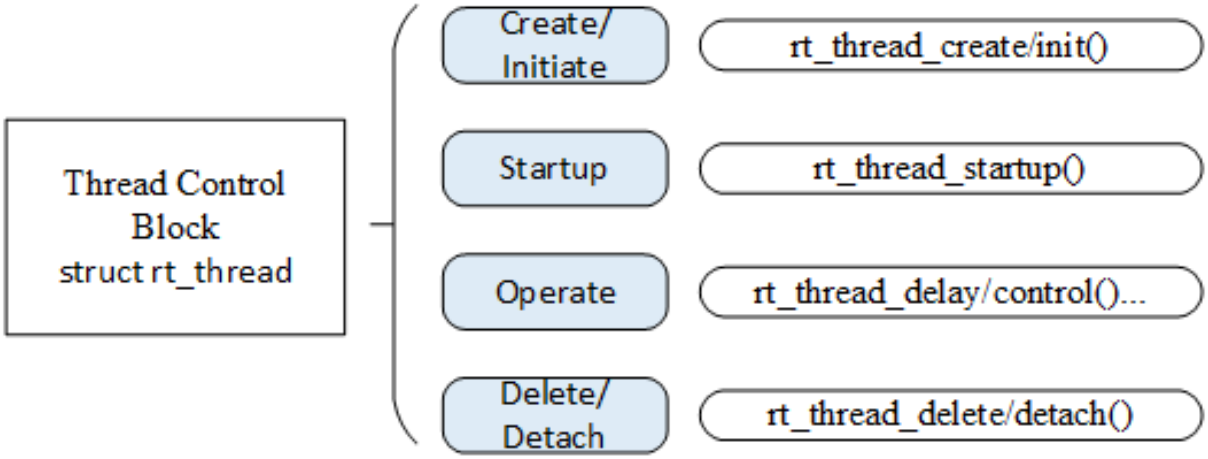


Figure 1.26 Thread Related Operations

1.3.3.3.1 Create and Delete Thread

To become an executable object, a thread must be created by the kernel of the operating system. You can create a dynamic thread through the following interface:

```
rt_thread_t rt_thread_create(const char*    name,
                             void           (*entry)(void* parameter),
                             void*          parameter,
                             rt_uint32_t    stack_size,
                             rt_uint8_t     priority,
                             rt_uint32_t    tick);
```

When this function is called, the system will allocate a thread handle from the dynamic heap memory and allocate the corresponding space from the dynamic heap memory according to the stack size specified in the parameter. The allocated stack space is aligned in `RT_ALIGN_SIZE` mode configured in `rtconfig.h`.

More details about this function, see rt_thread_create().

For some threads created with rt_thread_create(), when not needed or when an error occurs, one can use the following function interface to completely remove the thread from the system:

```
rt_err_t rt_thread_delete(rt_thread_t thread);
```

After calling this function, the thread object will be moved out of the thread list and removed from the kernel object manager. Consequently, the stack space occupied by the thread will also be freed, and the reclaimed space will be reused for other memory allocations. In fact, using the rt_thread_delete() function to delete the thread interface is just changing the corresponding thread state to RT_THREAD_CLOSE state and then putting it into rt_thread_defunct queue; the actual delete action (releasing the thread control block and releasing the thread stack) needs to be completed later by an idle thread when it is being executed.

This function is only valid when the system dynamic heap is enabled (meaning RT_USING_HEAP macro definition is already defined).

More details about this function, see rt_thread_delete().

1.3.3.3.2 Initialize and Detach Thread

The initialization of a thread can be done using the following function interface, to initialize a static thread object:

```
rt_err_t rt_thread_init(struct rt_thread    *thread,
                        const char          *name,
                        void                (*entry)(void* parameter),
                        void                *parameter,
                        void                *stack_start,
                        rt_uint32_t         stack_size,
                        rt_uint8_t          priority,
                        rt_uint32_t         tick);
```

The thread handle of the static thread (in other words, the thread control block pointer) and the thread stack are provided by the user. A static thread means that the thread control block and the thread running stack are generally set to global variables, which are determined and allocated when compiling. The kernel is not responsible for dynamically allocating memory space. It should be noted that the user-provided stack starting address needs to be system aligned (for example, 4-byte alignment is required on ARM).

More details about this function, see rt_thread_init().

For threads initialized with rt_thread_init(), using rt_thread_detach() will cause the thread object to be detached from the thread queue and kernel object manager. The detach thread function is as follows:

```
rt_err_t rt_thread_detach(rt_thread_t thread);
```

This function interface corresponds to the rt_thread_delete() function. The object operated by the rt_thread_delete() function is the handle created by rt_thread_create(), and the object operated by the rt_thread_detach() function is the thread control block initialized with the rt_thread_init() function. Again, the thread itself should not call this interface to detach thread itself.

More details about this function, see rt_thread_detach().

1.3.3.3.3 Start Thread

The thread created (initialized) is in initial state and does not enter the scheduling queue of the ready thread. We can call the following function interface to make the thread enter the ready state after the thread is initialized/created successfully:

```
rt_err_t rt_thread_startup(rt_thread_t thread);
```

When this function is called, the state of the thread is changed to the ready state and placed in the corresponding priority queue for scheduling. If the newly started thread has a higher priority than the current thread, it will immediately switch to the new thread.

More details about this function, see rt_thread_startup().

1.3.3.3.4 Obtaining Current Thread

During the running of the program, the same piece of code may be executed by multiple threads. At the time of execution, the currently executed thread handle can be obtained through the following function interface:

```
rt_thread_t rt_thread_self(void);
```

More details about this function, see `rt_thread_self()`.

1.3.3.3.5 Making Thread Release Processor Resources

When the current thread's time slice runs out or the thread actively requests to release the processor resource, it will no longer occupy the processor, and the scheduler will select the next thread of the same priority to execute. After the thread calls this interface, the thread is still in the ready queue. The thread gives up the processor using the following function interface:

```
rt_err_t rt_thread_yield(void);
```

After calling this function, the current thread first removes itself from its ready priority thread queue, then suspends itself to the end of the priority queue list, and then activates the scheduler for thread context switching (if there is no other thread with the same priority, then this thread continues to execute without context switching action).

The `rt_thread_yield()` function is similar to the `rt_schedule()` function, but the behavior of the system is completely different when other ready-state threads of the same priority exist. After executing the `rt_thread_yield()` function, the current thread is swapped out and the next ready thread of the same priority will be executed. After the `rt_schedule()` function is executed, the current thread is not necessarily swapped out. Even if it is swapped out, it will not be placed at the end of the ready thread list. Instead, the thread with the highest priority is selected in the system and executed. If there is no thread in the system with a higher priority than the current thread, the system will continue to execute the current thread after the `rt_schedule()` function is executed.

More details about this function, see `rt_thread_yield()`.

1.3.3.3.6 Thread Sleep

In practical applications, we sometimes need to delay the current thread running for a period of time and re-run at a specified time. This is called "thread sleep". Thread sleep can use the following three function interfaces:

```
rt_err_t rt_thread_delay(rt_tick_t tick);
rt_err_t rt_thread_mdelay(rt_int32_t ms);
```

These function interfaces have the same effect. Calling them can cause the current thread to suspend for a specified period of time. After, the thread will wake up and enter the ready state again. This function accepts a parameter that specifies the sleep time of the thread.

More details about this function, see `rt_thread_delay()` & `rt_thread_mdelay()`.

1.3.3.3.7 Suspend and Resume Thread

When a thread calls `rt_thread_delay()`, the thread will voluntarily suspend; when a function such as `rt_sem_take()`, `rt_mb_recv()` is called, the resource is not available for use and will cause the thread to suspend. A thread in a suspended state, if it waits for resources longer than the set time, then the thread will no longer wait for these resources and will return to the ready state; or, when other threads release the resource the thread is waiting for, the thread will also return to the ready state.

The thread suspends using the following function interface:

```
rt_err_t rt_thread_suspend(rt_thread_t thread);
```

More details about this function, see `rt_thread_suspend()`.

> Generally, you should not use this function to suspend the thread itself, if you really need to use rt_thread_suspend() to suspend the current task, immediately after calling function rt_thread_suspend(), rt_schedule() needs to be called.

Functions' context switch is achieved manually. User only needs to understand the role of the interface, which is not recommended.

To resume a thread is to let the suspended thread re-enter the ready state and put the thread into the system's ready queue. Thread resuming uses the following function interface:

```
rt_err_t rt_thread_resume(rt_thread_t thread);
```

More details about this function, see `rt_thread_resume()`.

1.3.3.3.8 Control Thread

When you need other control over a thread, such as dynamically changing the priority of a thread, you can call the following function interface:

```
rt_err_t rt_thread_control(rt_thread_t thread, rt_uint8_t cmd, void* arg);
```

More details about this function, see `rt_thread_control()`.

1.3.3.3.9 Set and Delete Idle Hooks

The idle hook function is a hook function of the idle thread. If the idle hook function is set, the idle hook function can be automatically executed to perform other things, such as the LED of system indicator , when the system executes the idle thread. The interface for setting/deleting idle hooks is as follows:

```
rt_err_t rt_thread_idle_sethook(void (*hook)(void));
rt_err_t rt_thread_idle_delhook(void (*hook)(void));
```

More details about this function, see `rt_thread_idle_sethook()` & `rt_thread_idle_delhook()`.

> An idle thread is a thread whose state is always ready. Therefore, hook function must ensure that idle threads will not be suspended at any time. Functions like `rt_thread_delay()`, `rt_sem_take()`, etc can't be used because they may cause the thread to suspend.

1.3.3.3.10 Set the Scheduler Hook

During the time when the system is running, it is in the process of thread running, interrupt triggering, responding to interrupts, switching to other threads, and switching between threads. In other words, context switching is the most common event in the system. Sometimes the user may want to know what kind of thread switch has occurred at times, you can set a corresponding hook function by calling the following function interface. This hook function will be called when the system thread switches:

```
void rt_scheduler_sethook(void (*hook)(struct rt_thread* from, struct rt_thread* to));
```

More details about this function, see `rt_scheduler_sethook()`.

Hook function `hook()` is declared as follows:

```
void hook(struct rt_thread* from, struct rt_thread* to);
```

Input parameters for the scheduler hook function hook() are shown in the following table:

Function Parameters	Description
from	Indicates the thread control block pointer that the system wants to switch out
to	Indicates the thread control block pointer that the system wants to switch out

> Please carefully compile your hook function, any carelessness is likely to cause the entire system to run abnormally (in this hook function, it is basically not allowed to call the system API, and should not cause the current running context to suspend).

1.3.3.4 Thread Application Sample

An application example in Keil simulator environment is given below.

1.3.3.4.1 Create Thread Sample

This sample is creating a dynamic thread and initializing a static thread. A thread is automatically deleted by the system after it has finished running. The other thread is always printing the counts, as follows:

```
#include <rtthread.h>

#define THREAD_PRIORITY         25
#define THREAD_STACK_SIZE       512
#define THREAD_TIMESLICE        5

static rt_thread_t tid1 = RT_NULL;

/* Entry Function for Thread 1 */
static void thread1_entry(void *parameter)
{
    rt_uint32_t count = 0;

    while (1)
    {
        /* Thread 1 runs with low priority and prints the count value all the time */
        rt_kprintf("thread1 count: %d\n", count ++);
        rt_thread_mdelay(500);
    }
}

ALIGN(RT_ALIGN_SIZE)
static char thread2_stack[1024];
static struct rt_thread thread2;
/* Entry for Thread 2 */
static void thread2_entry(void *param)
{
    rt_uint32_t count = 0;

    /* Thread 2 has a higher priority to preempt thread 1 and get executed */
    for (count = 0; count < 10 ; count++)
```

```
    {
        /* Thread 2 prints count value */
        rt_kprintf("thread2 count: %d\n", count);
    }
    rt_kprintf("thread2 exit\n");
    /* Thread 2 will also be automatically detached from the system after it finishes running. */
}

/* Thread Sample */
int thread_sample(void)
{
    /* Creat thread 1, Name is thread1Entry is thread1_entry */
    tid1 = rt_thread_create("thread1",
                            thread1_entry, RT_NULL,
                            THREAD_STACK_SIZE,
                            THREAD_PRIORITY, THREAD_TIMESLICE);

    /* Start this thread if you get the thread control block */
    if (tid1 != RT_NULL)
        rt_thread_startup(tid1);

    /* Creat thread 2, Name is thread2Entry is thread2_entry */
    rt_thread_init(&thread2,
                   "thread2",
                   thread2_entry,
                   RT_NULL,
                   &thread2_stack[0],
                   sizeof(thread2_stack),
                   THREAD_PRIORITY - 1, THREAD_TIMESLICE);
    rt_thread_startup(&thread2);

    return 0;
}

/* Export to msh command list */
MSH_CMD_EXPORT(thread_sample, thread sample);
```

The simulation results are as follows:

```
\ | /
- RT -     Thread Operating System
 / | \     3.1.0 build Aug 24 2018
 2006 - 2018 Copyright by rt-thread team
msh >thread_sample
msh >thread2 count: 0
thread2 count: 1
thread2 count: 2
thread2 count: 3
thread2 count: 4
thread2 count: 5
thread2 count: 6
thread2 count: 7
thread2 count: 8
thread2 count: 9
thread2 exit
thread1 count: 0
thread1 count: 1
thread1 count: 2
thread1 count: 3
...
```

When thread 2 counts to a certain value, it will stop running. Then thread 2 is automatically deleted by the system, and therefore the counting stops. Thread 1 prints the count all the time.

> About deleting threads: Most threads are executed cyclically without needing to be deleted. For thread that can finish running, RT-Thread automatically deletes the thread after the thread finishes running, and deletes it in `rt_thread_exit()`. User only needs to understand the role of the interface. It is not recommended to use this interface (this interface can be called by other threads or call this interface in the timer timeout function to delete a thread which is not used very often).

1.3.3.4.2 Thread Time Slice Round-Robin Scheduling Sample

This sample is creating two threads that will always print counts when executing, as follows:

```
#include <rtthread.h>

#define THREAD_STACK_SIZE   1024
```

```c
#define THREAD_PRIORITY     20
#define THREAD_TIMESLICE    10

/* Thread Entry */
static void thread_entry(void* parameter)
{
    rt_uint32_t value;
    rt_uint32_t count = 0;

    value = (rt_uint32_t)parameter;
    while (1)
    {
        if(0 == (count % 5))
        {
            rt_kprintf("thread %d is running ,thread %d count = %d\n", value , value , count);

            if(count> 200)
                return;
        }
         count++;
     }
}

int timeslice_sample(void)
{
    rt_thread_t tid = RT_NULL;
    /* Create Thread 1 */
    tid = rt_thread_create("thread1",
                           thread_entry, (void*)1,
                           THREAD_STACK_SIZE,
                           THREAD_PRIORITY, THREAD_TIMESLICE);
    if (tid != RT_NULL)
        rt_thread_startup(tid);


    /* Create Thread 2 */
    tid = rt_thread_create("thread2",
                           thread_entry, (void*)2,
                           THREAD_STACK_SIZE,
                           THREAD_PRIORITY, THREAD_TIMESLICE-5);
    if (tid != RT_NULL)
        rt_thread_startup(tid);
    return 0;
}

/* Export to msh command list */
MSH_CMD_EXPORT(timeslice_sample, timeslice sample);
```

The simulation results are as follows:

```
 \ | /
- RT -     Thread Operating System
 / | \     3.1.0 build Aug 27 2018
 2006 - 2018 Copyright by rt-thread team
msh >timeslice_sample
msh >thread 1 is running ,thread 1 count = 0
thread 1 is running ,thread 1 count = 5
thread 1 is running ,thread 1 count = 10
thread 1 is running ,thread 1 count = 15
...
thread 1 is running ,thread 1 count = 125
thread 1 is rthread 2 is running ,thread 2 count = 0
thread 2 is running ,thread 2 count = 5
thread 2 is running ,thread 2 count = 10
thread 2 is running ,thread 2 count = 15
thread 2 is running ,thread 2 count = 20
thread 2 is running ,thread 2 count = 25
thread 2 is running ,thread 2 count = 30
thread 2 is running ,thread 2 count = 35
thread 2 is running ,thread 2 count = 40
thread 2 is running ,thread 2 count = 45
thread 2 is running ,thread 2 count = 50
thread 2 is running ,thread 2 count = 55
thread 2 is running ,thread 2 count = 60
thread 2 is running ,thread 2 cunning ,thread 2 count = 65
thread 1 is running ,thread 1 count = 135
...
thread 2 is running ,thread 2 count = 205
```

As can be seen from the running count results, thread 2 runs half the time of thread 1.

1.3.3.4.3 Thread Scheduler Hook Sample

When thread is scheduling switch, it executes the schedule. We can set a scheduler hook so that we can do other things when the thread is being switched. This sample is printing switch information between the threads in the scheduler hook function, as shown in the following code.

```
#include <rtthread.h>

#define THREAD_STACK_SIZE   1024
#define THREAD_PRIORITY     20
#define THREAD_TIMESLICE    10

/* Counter for each thread */
volatile rt_uint32_t count[2];

/* Threads 1, 2 share an entry, but the entry parameters are different */
static void thread_entry(void* parameter)
{
    rt_uint32_t value;

    value = (rt_uint32_t)parameter;
    while (1)
    {
        rt_kprintf("thread %d is running\n", value);
        rt_thread_mdelay(1000); // Delay for a while
    }
}

static rt_thread_t tid1 = RT_NULL;
static rt_thread_t tid2 = RT_NULL;

static void hook_of_scheduler(struct rt_thread* from, struct rt_thread* to)
{
    rt_kprintf("from: %s -->  to: %s \n", from->name , to->name);
}

int scheduler_hook(void)
{
    /* Set the scheduler hook */
    rt_scheduler_sethook(hook_of_scheduler);

    /* Create Thread 1 */
    tid1 = rt_thread_create("thread1",
                            thread_entry, (void*)1,
                            THREAD_STACK_SIZE,
                            THREAD_PRIORITY, THREAD_TIMESLICE);
    if (tid1 != RT_NULL)
        rt_thread_startup(tid1);

    /* Create Thread 2 */
    tid2 = rt_thread_create("thread2",
                            thread_entry, (void*)2,
                            THREAD_STACK_SIZE,
                            THREAD_PRIORITY,THREAD_TIMESLICE - 5);
    if (tid2 != RT_NULL)
        rt_thread_startup(tid2);
    return 0;
}

/* Export to msh command list */
MSH_CMD_EXPORT(scheduler_hook, scheduler_hook sample);
```

The simulation results are as follows:

```
 \ | /
- RT -     Thread Operating System
 / | \     3.1.0 build Aug 27 2018
 2006 - 2018 Copyright by rt-thread team
msh > scheduler_hook
msh >from: tshell -->  to: thread1
thread 1 is running
from: thread1 -->  to: thread2
thread 2 is running
from: thread2 -->  to: tidle
from: tidle -->  to: thread1
thread 1 is running
from: thread1 -->  to: tidle
from: tidle -->  to: thread2
thread 2 is running
from: thread2 -->  to: tidle
...
```

It can be seen from the simulation results that when the threads are being switched, the set scheduler hook function is working normally and printing the information of the thread switching, including switching to the idle thread.

1.3.4 Clock & Timer Management

The concept of time is very important. You need to set a time to go out with friends and it takes time to complete tasks. Life is inseparable from time. The same is true for operating systems, which require time to regulate the execution of their tasks. The smallest time unit in operating system is clock tick (OS Tick). This chapter focuses on introduction of clock ticks and clock-based timers. After reading this chapter, we will learn how clock ticks are generated and how to use RT-Thread timers.

1.3.4.1 Clock Tick(OS Tick)

Any operating system needs to provide a clock tick for the system to handle all time-related events, such as thread latency, thread time slice rotation scheduling, timer timeout, etc. Clock tick is a specific periodic interrupt. This interrupt can be regarded as the system heartbeat. The time interval between interrupts depends on different applications, generally it is 1ms–100ms. The faster the clock tick rate, the greater the overhead of the system. The number of clock ticks counted from the start of the system is called the system time.

In RT-Thread, the length of clock tick can be adjusted according to the value of RT_TICK_PER_SECOND, which is equal to 1/RT_TICK_PER_SECOND second.

1.3.4.1.1 Clock Tick Implementation

Clock tick is generated by a hardware timer configured in interrupt trigger mode. `void rt_tick_increase(void)` will be called when an interrupt occurs,notifying the operating system that a system clock has passed; different hardware driver have different timer interrupt implementations. Here is an example of using STM32 `SysTick_Handler` to achieve clock Tick.

```
void SysTick_Handler(void)
{
    /* Entry Interrupt*/
    rt_interrupt_enter();
    ......
    rt_tick_increase();
    /* Leave Interrupt */
    rt_interrupt_leave();
}
```

Call `rt_tick_increase()` in interrupt function to self-add global variable rt_tick. The code is as follows:

```
void rt_tick_increase(void)
{
    struct rt_thread *thread;

    /* Global variable rt_tick self-add */
    ++ rt_tick;

    /* Check time slice */
    thread = rt_thread_self();

    -- thread->remaining_tick;
    if (thread->remaining_tick == 0)
    {
        /* Re-assign initial value */
        thread->remaining_tick = thread->init_tick;

        /* Thread suspends */
        rt_thread_yield();
    }

    /* Check time slice */
    rt_timer_check();
}
```

You can see that global variable rt_tick is incremented by one on every clock tick. The value of rt_tick indicates the total number of clock ticks that the system has elapsed since it started, that is, system time. In addition, on every clock tick, whether the current thread's time slice is exhausted and whether there is a timer timeout will be checked.

>In interrupt, rt_timer_check() is used to check the system hardware timer linked list. If there is a timer timeout, the corresponding timeout function will be called. All timers are removed from the timer linked list if it timed out, and periodic timer is added to the timer linked list when it is started again.

1.3.4.1.2 Obtain Clock Tick

Since global variable rt_tick is incremented by one on every clock tick, the value of the current rt_tick will be returned by calling `rt_tick_get`, which is the current clock tick value. This interface can be used to record the length of time a system is running, or to measure the time it takes for a task to run. The interface function is as follows:

```
rt_tick_t rt_tick_get(void);
```

The following table describes the return values of `rt_tick_get()` function:

Return	Description
rt_tick	Current clock tick value

1.3.4.2 Timer Management

Timer refers to triggering an event after a certain specified time from a specified moment, for example, setting a timer to wake up yourself the next morning. Timer includes hardware timer and software timer:

1) **Hardware timer** is the timing function provided by the chip itself. The hardware timer can be used by configuring the timer module into a timer mode and setting the time. Hardware timer is accurate to nanosecond precision, and is interrupt trigger mode.

2) **Software timer** is a type of system interface provided by the operating system. It is built on the basis of the hardware timer to enable the system to provide a timer service with no constraint on numbers.

RT-Thread operating system provides software-implemented timers in units of clock tick (OS Tick), that is, the timing value must be an integer multiple of OS Tick. For example, an OS Tick is 10ms, then the software timer can only be timed 10ms, 20ms, 100ms, etc., but not 15ms. RT-Thread timer is also based on the clock tick, providing timing capabilities based on integral multiples of the clock tick.

1.3.4.2.1 RT-Thread Timer Introduction

RT-Thread timer provides two types of timer mechanisms: the first type is a one-shot timer, which only triggers a timer event for onetime after startup, and then the timer stops automatically. The second type is a periodic trigger timer, which periodically triggers a timer event until the user manually stops it, otherwise it will continue to execute forever.

In addition, according to the context in which the timeout function is executed, RT-Thread timer can be divided into HARD_TIMER mode and SOFT_TIMER mode, as shown in the following figure.

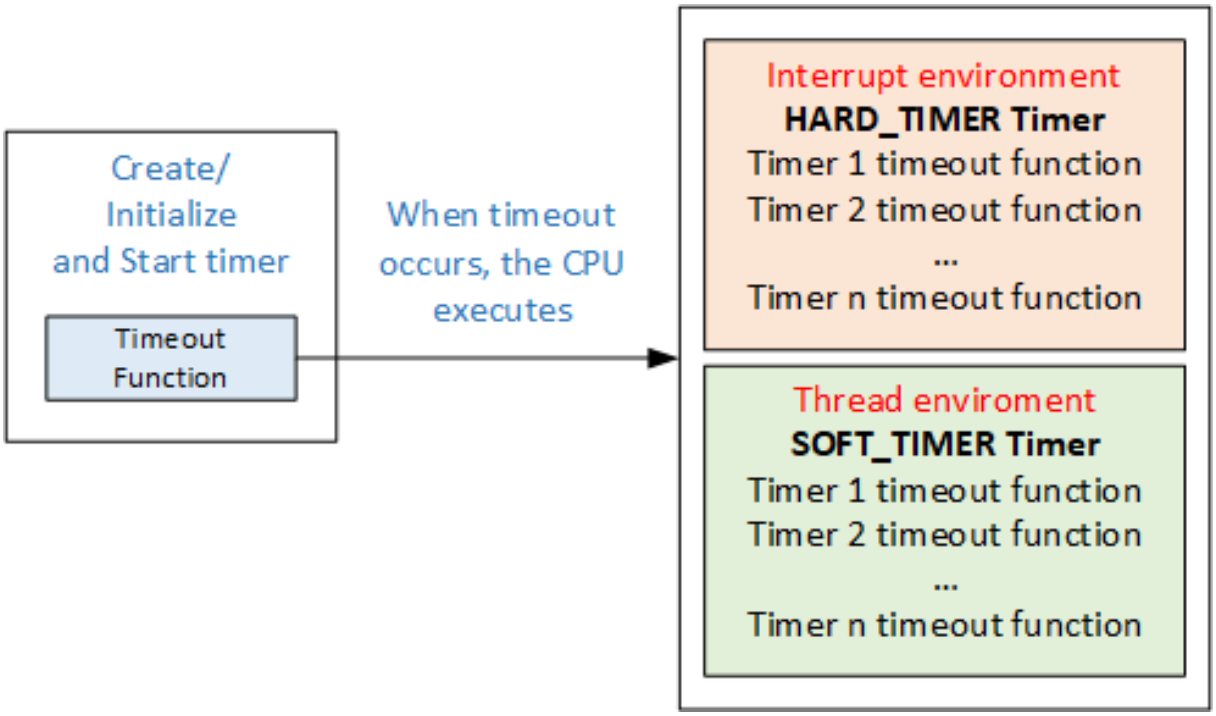


Figure 1.27 Timer Context

1.3.4.2.1.1 HARD_TIMER Mode

The timer timeout function of HARD_TIMER mode is executed in the interrupt context and can be specified with the parameter RT_TIMER_FLAG_HARD_TIMER when initializing/creating the timer.

When executed in interrupt context, the requirements for timeout function are the same as those for the interrupt service routine: execution time should be as short as possible, and the execution should not cause the current context to suspend and wait. For example, a timeout function executed in an interrupt context should not attempt to apply for dynamic memory, free dynamic memory, etc.

The default mode of RT-Thread timer is HARD_TIMER mode which means after the timer timeout, the timeout function runs in the context of the system clock interrupt. The execution mode in the interrupt context determines that the timer's timeout function should not call any system function that will cause the current context to suspend; nor can it be executing for a very long time, otherwise the response time of other interrupts will be lengthened or the running time of other threads will be preempted.

1.3.4.2.1.2 SOFT_TIMER Mode

The SOFT_TIMER mode is configurable and macro definition RT_USING_TIMER_SOFT is used to determine whether the mode should be enabled. When this mode is enabled, the system will create a timer thread at initialization, and then the timer timeout function of SOFT_TIMER mode will be executed in the context of the timer thread. SOFT_TIMER mode can be specified using the parameter RT_TIMER_FLAG_SOFT_TIMER when initializing/↩ creating the timer.

1.3.4.2.2 Timer Working Mechanism

The following is an example to illustrate the working mechanism of RT-Thread timer. Two important global variables are maintained in the RT-Thread timer module:

(1) Elapsed tick time of the current system rt_tick (when hardware timer interrupt comes, it will add 1);

(2) Timer linked list `rt_timer_list`. Newly created and activated timers of the system are inputted into the `rt_timer_list` linked list in a timeout-ordered manner.

As shown in the figure below, the current tick value of the system is 20. In the current system, three timers have been created and started, which are Timer1 with 50 ticks set time, Timer2 with 100 ticks, and Timer3 with 500 ticks. Current time of the system rt_tick=20 is added respectively on these three timers and they are linked from small to large in the rt_timer_list linked list, forming a timer linked list structure as shown in the figure.

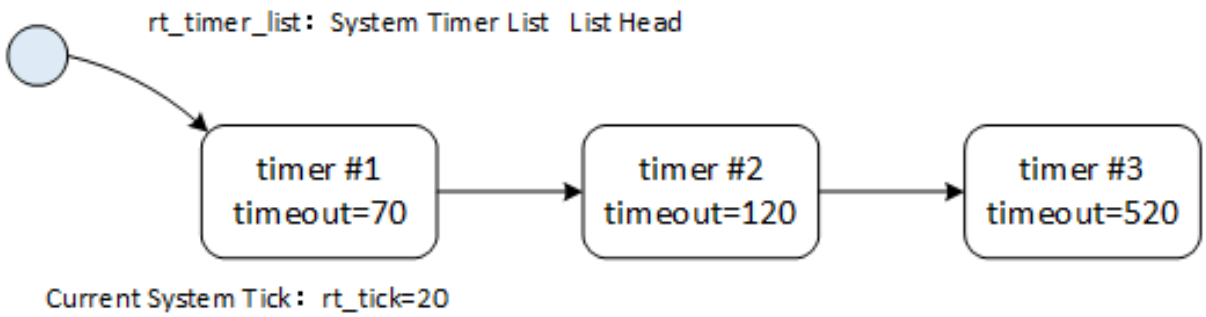


Figure 1.28 Timer Linked List Diagram

Along with the trigger of the hardware timer, `rt_tick` has been increasing (rt_tick variable is incremented by 1 every time the hardware timer is interrupted). After 50 tick, rt_tick is increased from 20 to 70, which is equal to the timeout value of Timer1. Then, timeout functions associated with the Timer1 timer will be triggered and Timer1 from the rt_timer_list linked list will be removed. Similarly, after 100 ticks and 500 ticks, timeout functions associated with Timer2 and Timer3 are triggered, and timers of Time2 and Timer3 will be removed from the rt_timer_list linked list.

If after system's 10 ticks (current rt_tick=30), a new task has created Timer4 with 300 ticks, so the Timer4's timeout is equal to rt_tick add 300, that is 330. Timer4 will be inserted in between Timer2 and Timer3, forming a linked list structure shown below:

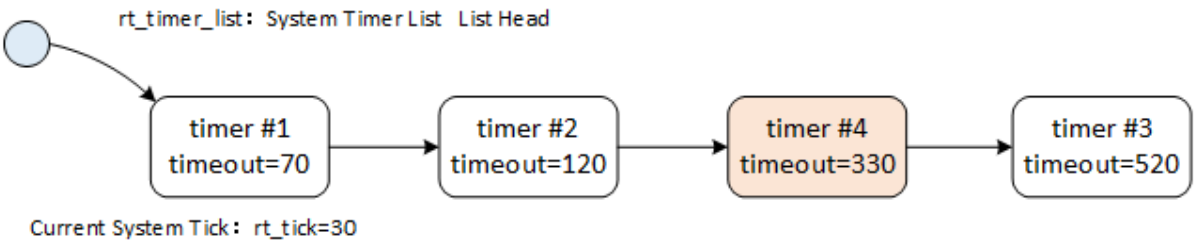


Figure 1.29 Timer linked List Insertion Diagram

1.3.4.2.2.1 Timer Control Block

In RT-Thread operating system, timer control block is defined by structure `struct rt_timer` and forms a timer kernel object, which is then linked to the kernel object container for management. It is a data structure used by the operating system to manage timers. It stores information about timers, such as the initial number of ticks, the number of timeout ticks, the linked list structure used to connect timers, timeout callback functions, etc.

```
struct rt_timer
{
    struct rt_object parent;
    rt_list_t row[RT_TIMER_SKIP_LIST_LEVEL];  /* Timer Linked List Node */

    void (*timeout_func)(void *parameter);      /* Timeout Function */
    void      *parameter;                       /* Parameters of Timeout Function  */
    rt_tick_t init_tick;                        /* Timer Initial Timeout Ticks */
    rt_tick_t timeout_tick;                     /* Number of ticks when the timer actually times out */
};
typedef struct rt_timer *rt_timer_t;
```

Timer control block is defined by structure `struct rt_timer` and forms a timer kernel object, which is then linked to the kernel object container for management. The `list` member is used to link an active (already started) timer to the `rt_timer_list` linked list.

1.3.4.2.2.2 Timer Skip List Algorithm

In the introduction of working mechanics of the timer above, we have talked about that the newly created and activated timers are inserted into the rt_timer_list linked list in the order of the timeout, that is, rt_timer_list linked list is an ordered list. RT-Thread uses a skip list algorithm to speed up the search for linked list elements.

Skip list is a data structure based on a parallel linked list, which is simple to implement, and the time complexity of insertion, deletion, and search is O(log n). Skip list is a kind of linked list, but it adds a "skip" function to the linked list. It is this function that enables the skip list to have the time complexity of O(log n) when looking for elements, for example:

An ordered list, as shown in the following figure, searches for elements {13, 39} from the ordered list. The number of comparisons is {3, 5} and the total number of comparisons is 3 + 5 = 8 times.

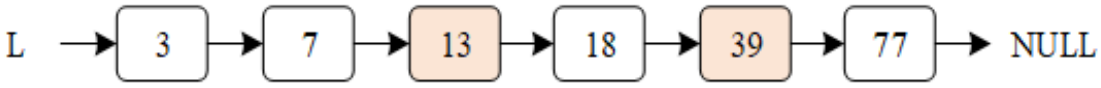


Figure 1.30 Ordered Linked List

After using the skip list algorithm, a method similar to binary search tree can be used to extract some nodes as indexes, and then the structure shown in the following figure is obtained:

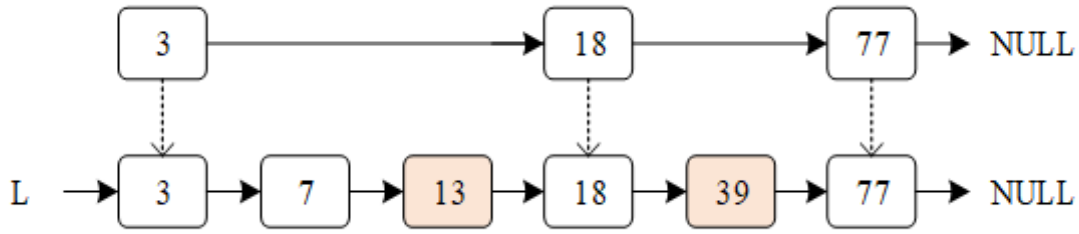


Figure 1.31 Ordered Linked List Index

In this structure, {3, 18, 77} is extracted as first-level index, so that the number of comparisons can be reduced when searching. For example, there are only 3 times of comparisons when searching 39 (by comparing 3, 18, 39). Of course, we can also extract some elements from the first-level index, as a secondary index, which can speed up the element search.

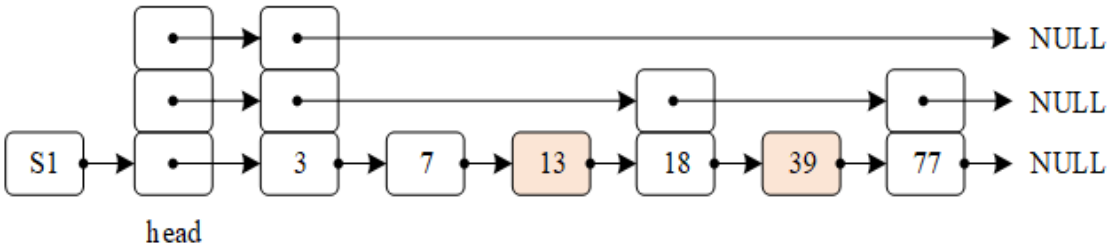


Figure 1.32 Three Layer Skip List

Therefore, the timer skip list can pass the index of the upper layer, reducing the number of comparisons during the search and improving the efficiency of the search. This is an algorithm of "space in exchange of time", macro definition RT_TIMER_SKIP_LIST_LEVEL is used to configure the number of layers in skip list. The default value is 1, which means that ordered linked list algorithm for first-order ordered list graph is used. Each additional one means that another level of index is added to the original list.

1.3.4.2.3 Timer Management

RT-Thread timer is introduced in the previous sections and the working mechanism of the timer is conceptually explained. This section will go deep into the various interfaces of the timer to help the reader understand the RT-Thread timer at the code level.

Timer management system needs to be initialized at system startup. This can be done through the following function interface:

```
void rt_system_timer_init(void);
```

If you need to use SOFT_TIMER, the following function interface should be called when the system is initialized:

```
void rt_system_timer_thread_init(void);
```

Timer control block contains important parameters related to the timer and acts as a link between various states of the timer. Relevant operations of the timer are as shown in the following figure. Relevant operations of the timer includes: creating/initializing the timer, starting the timer, running the timer, and deleting/detaching the timer. All the timers will be moved from the timer linked list after their timings expire. However, periodic timer is added back to the timer linked list when it is started again, which is related to timer parameter settings. Each time an operating system clock interrupt occurs, a change is made to the timer status parameter that has timed out.

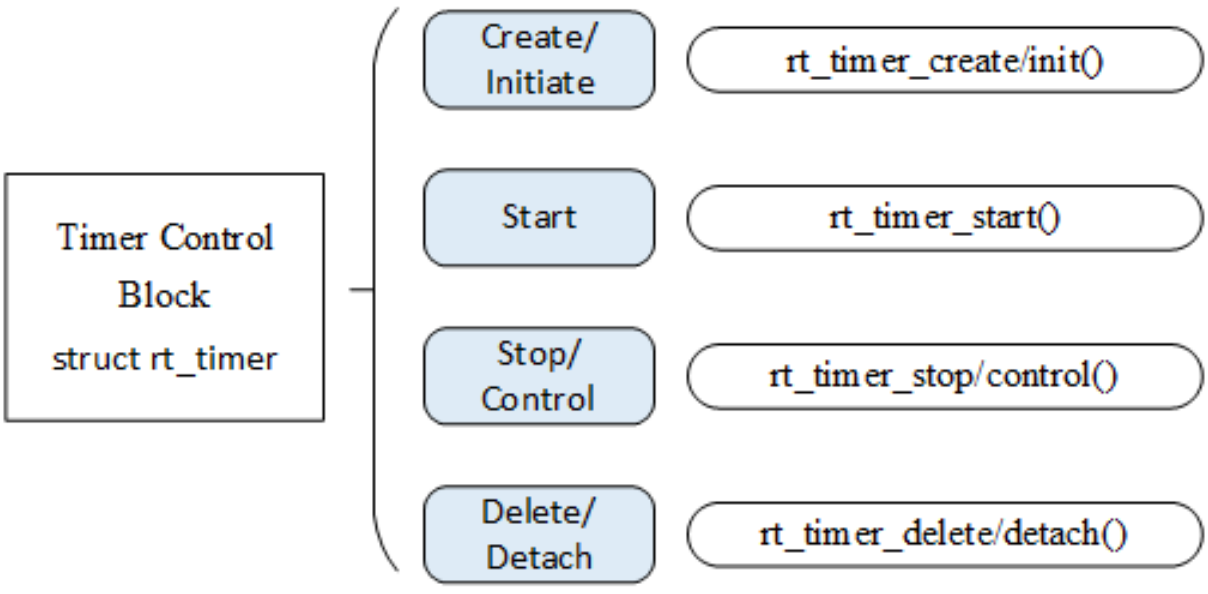


Figure 1.33 Timer Related Operations

1.3.4.2.3.1 Create and Delete Timer

When dynamically creating a timer, the following function interface can be used:

```
rt_timer_t rt_timer_create(const char* name,
                           void (*timeout)(void* parameter),
                           void* parameter,
                           rt_tick_t time,
                           rt_uint8_t flag);
```

After calling the function interface, kernel first allocates a timer control block from the dynamic memory heap and then performs basic initialization on the control block. The description of each parameter and return value is detailed in the following table:

Input parameters and return values of `rt_timer_create()`

Parameters	Description
name	Name of the timer
void (timeout) (void parameter)	Timer timeout function pointer (this function is called when the timer expires)
parameter	Entry parameter of the timer timeout function (when the timer expires, calling timeout callback function will pass this parameters as the entry parameter to the timeout function)
time	Timeout of the timer, the unit is the clock tick
flag	Parameters when the timer is created. The supported values include one-shot timing, periodic timing, hardware timer, software timer, etc. (You can use multiple values with "OR")
Return	——
RT_NULL	Creation failed (usually returning RT_NULL due to insufficient system memory)
Timer Handle	Timer was created successfully.

In include/rtdef.h, some timer related macros are defined, as follows:

```
#define RT_TIMER_FLAG_ONE_SHOT      0x0     /* One shot timing     */
#define RT_TIMER_FLAG_PERIODIC      0x2     /* Periodic timing     */

#define RT_TIMER_FLAG_HARD_TIMER    0x0     /* Hardware timer   */
#define RT_TIMER_FLAG_SOFT_TIMER    0x4     /* Software timer  */
```

The above two sets of values can be assigned to the flag in an "or" logical manner. When the specified flag is RT_TIMER_FLAG_HARD_TIMER, if the timer expires, the timer's callback function will be called in the context of the service routine of the clock interrupt; when the specified flag is RT_TIMER_FLAG_SOFT_TIMER, if the timer expires, the timer's callback function will be called in the context of the system clock timer thread.

When the system no longer uses dynamic timers, the following function interface can be used:

```
rt_err_t rt_timer_delete(rt_timer_t timer);
```

After calling this function interface, the system will remove this timer from the rt_timer_list linked list, and then release the memory occupied by the corresponding timer control block. The parameters and return values are detailed in the following table:

Input parameters and return values of rt_timer_delete()

Parameters	Description
timer	Timer handle, pointing at timer needs to be deleted

Parameters	Description
Return	——
RT_EOK	Deletion is successful (if the parameter timer handle is RT_NULL, it will result in an ASSERT assertion)

1.3.4.2.3.2 Initialize and Detach Timer

When creating a timer statically , the timer can be initialized by using `rt_timer_init` interface. The function interface is as follows:

```
void rt_timer_init(rt_timer_t timer,
                   const char* name,
                   void (*timeout)(void* parameter),
                   void* parameter,
                   rt_tick_t time, rt_uint8_t flag);
```

When using this function interface, the corresponding timer control block, the corresponding timer name, timer timeout function, etc will be initialized. The description of each parameter and return value is shown in the following table:

Input parameters and return values of `rt_timer_init()`

Parameters	Description
timer	Timer handle, pointing to the to-be-initialized timer control block
name	Name of the timer
void (timeout) (void parameter)	Timer timeout function pointer (this function is called when the timer expires)
parameter	Entry parameter of the timer timeout function (when the timer expires, the call timeout callback function will pass this parameter as the entry parameter to the timeout function)
time	Timeout of the timer, the unit is clock tick
flag	Parameters of when the timer is created. The supported values include one shot timing, periodic timing, hardware timer, and software timer (multiple values can be taken with OR). For details, see Creating a Timer.

When a static timer does not need to be used again, you can use the following function interface:

```
rt_err_t rt_timer_detach(rt_timer_t timer);
```

When detaching a timer, the system will detach the timer object from the kernel object container, but the memory occupied by the timer object will not be released. The parameters and return values are detailed in the following table:

Input parameters and return values for `rt_timer_detach()`

Parameters	Description
timer	Timer handle, pointing to the to-be-detached timer control block
**Return **	——
RT_EOK	Successfully detached

1.3.4.2.3.3 Start and Stop Timer

When the timer is created or initialized, it will not be started immediately. It will start after timer function interface is called. The timer function interface is started as follows:

```
rt_err_t rt_timer_start(rt_timer_t timer);
```

After the timer start function interface is called, the state of the timer is changed to activated state (RT_TIMER_↩FLAG_ACTIVATED) and inserted into rt_timer_list queue linked list, the parameters and return values are detailed in the following table:

Input parameters and return values of rt_timer_start()

Parameters	Description
timer	Timer handle, pointing to the to-be-initialized timer control block
Return	——
RT_EOK	Successful startup

For an example of starting the timer, please refer to the sample code below.

After starting the timer, if you want to stop it, you can use the following function interface:

```
rt_err_t rt_timer_stop(rt_timer_t timer);
```

After the timer stop function interface is called, the timer state will change to the stop state and will be detached from the rt_timer_list linked list without participating in the timer timeout check. When a (periodic) timer expires, this function interface can also be called to stop the (periodic) timer itself. The parameters and return values are detailed in the following table:

Input parameters and return values of rt_timer_stop()

Parameters	Description
timer	Timer handle, pointing to the to-be-stopped timer control block
Return	——
RT_EOK	Timer successfully stopped
- RT_ERROR	timer is in stopped state

1.3.4.2.3.4 Control Timer

In addition to some of the programming interfaces provided above, RT-Thread additionally provides a timer control function interface to obtain or set more timer information. The control timer function interface is as follows:

```
rt_err_t rt_timer_control(rt_timer_t timer, rt_uint8_t cmd, void* arg);
```

The control timer function interface can view or change the setting of the timer according to the parameters of the command type. The description of each parameter and return value is as follows:

Input parameters and return values of rt_timer_control()

Parameters	Description
timer	Timer handle, pointing to the to-be-stopped timer control block

Parameters	Description
cmd	The command for controlling the timer currently supports four command interfaces, which are setting timing, viewing the timing time, setting a one shot trigger, and setting the periodic trigger.
arg	Control command parameters corresponding to cmd. For example, when cmd is the set timeout time, the timeout time parameter can be set by arg.
Return	——
RT_EOK	Successful

Commands supported by function parameters cmd:

```
#define RT_TIMER_CTRL_SET_TIME      0x0     /* Set Timeout value      */
#define RT_TIMER_CTRL_GET_TIME      0x1     /* Obtain Timer Timeout Time      */
#define RT_TIMER_CTRL_SET_ONESHOT   0x2     /* Set the timer as a oneshot timer.    */
#define RT_TIMER_CTRL_SET_PERIODIC  0x3     /* Set the timer as a periodic timer */
```

See "dynamic timer routine" for code that uses the timer control interface.

1.3.4.3 Timer Application Sample

This is an example of creating a timer that creates two dynamic timers, one for one shot timing, another one for periodic timing and for the periodic timer to run for a while and then stop running, as shown below:

```
#include <rtthread.h>

/* Timer Control Block */
static rt_timer_t timer1;
static rt_timer_t timer2;
static int cnt = 0;

/* Timer 1 Timeout Function */
static void timeout1(void *parameter)
{
    rt_kprintf("periodic timer is timeout %d\n", cnt);

    /* On the 10th time, stops perodic timer */
    if (cnt++>= 9)
    {
        rt_timer_stop(timer1);
        rt_kprintf("periodic timer was stopped! \n");
    }
}

/* Timer 2 Timeout Function */
static void timeout2(void *parameter)
{
    rt_kprintf("one shot timer is timeout\n");
}

int timer_sample(void)
{
    /* Create Timer 1 Periodic Timer */
    timer1 = rt_timer_create("timer1", timeout1,
                             RT_NULL, 10,
                             RT_TIMER_FLAG_PERIODIC);

    /* Start Timer 1*/
    if (timer1 != RT_NULL) rt_timer_start(timer1);

    /* Create Timer 2 One Shot Timer */
    timer2 = rt_timer_create("timer2", timeout2,
                             RT_NULL,  30,
                             RT_TIMER_FLAG_ONE_SHOT);

    /* Start Timer 2 */
    if (timer2 != RT_NULL) rt_timer_start(timer2);
    return 0;
}

/* Export to msh command list */
MSH_CMD_EXPORT(timer_sample, timer sample);
```

The simulation results are as follows:

```
 \ | /
- RT -     Thread Operating System
 / | \     3.1.0 build Aug 24 2018
 2006 - 2018 Copyright by rt-thread team
msh >timer_sample
msh >periodic timer is timeout 0
periodic timer is timeout 1
one shot timer is timeout
periodic timer is timeout 2
periodic timer is timeout 3
periodic timer is timeout 4
periodic timer is timeout 5
periodic timer is timeout 6
periodic timer is timeout 7
periodic timer is timeout 8
periodic timer is timeout 9
periodic timer was stopped!
```

Timeout function of periodic timer 1 runs once every 10 OS Ticks for 10 times (after 10 times, rt_timer_stop is called to stop timer 1); timeout function of one-shot timer 2 runs once on the 30th OS Tick.

The example of initializing timer is similar to the example of creating a timer. This program initializes two static timers, one is one-shot timing and one is periodic timing, as shown in the following code:

```
#include <rtthread.h>

/* Timer Control Block */
static struct rt_timer timer1;
static struct rt_timer timer2;
static int cnt = 0;

/* Timer 1 Timeout Function */
static void timeout1(void* parameter)
{
    rt_kprintf("periodic timer is timeout\n");
    /* Run for 10 times */
    if (cnt++>= 9)
    {
        rt_timer_stop(&timer1);
    }
}

/* Timer 2 Timeout Function */
static void timeout2(void* parameter)
{
    rt_kprintf("one shot timer is timeout\n");
}

int timer_static_sample(void)
{
    /* Initialize Timer */
    rt_timer_init(&timer1, "timer1",  /* Timer name is timer1 */
                  timeout1, /* Callback handler for timeout */
                  RT_NULL, /* Entry parameter of the timeout function */
                  10, /* Timing length in OS Tick, 10 OS Tick */
                  RT_TIMER_FLAG_PERIODIC); /* Periodic timer */
    rt_timer_init(&timer2, "timer2",   /* Timer name is timer2 */
                  timeout2, /* Callback handler for timeout */
                    RT_NULL, /* Entry parameter of the timeout function */
                    30, /* Timing length is 30 OS Tick */
                  RT_TIMER_FLAG_ONE_SHOT); /* One-shot timer */

    /* Start Timer */
    rt_timer_start(&timer1);
    rt_timer_start(&timer2);
    return 0;
}
/* Export to msh command list */
MSH_CMD_EXPORT(timer_static_sample, timer_static sample);
```

The simulation results are as follows:

```
\ | /
- RT -     Thread Operating System
 / | \     3.1.0 build Aug 24 2018
 2006 - 2018 Copyright by rt-thread team
msh >timer_static_sample
msh >periodic timer is timeout
periodic timer is timeout
one shot timer is timeout
periodic timer is timeout
periodic timer is timeout
periodic timer is timeout
periodic timer is timeout
```

```
periodic timer is timeout
periodic timer is timeout
periodic timer is timeout
periodic timer is timeout
```

The timeout function of periodic timer1 runs once every 10 OS Ticks for 10 times (After 10 times rt_timer_stop is called to stop timer1); the timeout function of one-shot timer2 runs once on the 30th OS Tick.

1.3.4.4 High Precision Delay

The minimum precision of the RT-Thread timer is determined by the system clock tick (1 OS Tick = 1/RT_TICK_↩ PER_SECOND second, RT_TICK_PER_SECOND value is defined in the rtconfig.h file), and the timer must be set to an integer multiple of the OS Tick. When it is necessary to implement system timing for a shorter time length. For example, the OS Tick is 10ms but the program needs to implement a timing or delay of 1ms. In this case, the operating system timer can't meet the requirements. This problem can be solved by reading the counter of a hardware timer of the system or using hardware timer directly.

In Cortex-M series, SysTick has been used by RT-Thread as an OS Tick. It is configured to trigger an interrupt after 1/RT_TICK_PER_SECOND seconds. The interrupt handler uses the Cortex-M3 default name `SysTick↩ _Handler`. In Cortex-M3 CMSIS (Cortex Microcontroller Software Interface Standard) specification, System↩ CoreClock represents the dominant frequency of the chip, so based on SysTick and SystemCoreClock, we can use SysTick to obtain an accurate delay function, as shown in the following example, Cortex-M3 SysTick-based precision delay (requires the system to enable SysTick):

The high-precision delay routine is as follows:

```
#include <board.h>
void rt_hw_us_delay(rt_uint32_t us)
{
    rt_uint32_t delta;
    /* Obtain the number of ticks of the delay */
    us = us * (SysTick->LOAD/(1000000/RT_TICK_PER_SECOND));
    /* Obtain current time */
    delta = SysTick->VAL;
    /* Loop to obtain the current time until the specified time elapses and exits the loop */
    while (delta - SysTick->VAL< us);
}
```

The entry parameter us indicates the number of microseconds that need to be delayed. This function can only support delays shorter than 1 OS Tick, otherwise the SysTick will overflow and not be able to obtain the specified delay time.

1.3.5 Inter-thread Synchronization

In a multi-threaded real-time system, the completion of a task can often be accomplished through coordination of multiple threads, so how do these multiple threads collaborate well with each other to perform without errors? Here is an example.

For example, two threads in one task: one thread receives data from the sensor and writes the data to shared memory, while another thread periodically reads data from the shared memory and sends it to display. The following figure depicts data transfer between two threads:

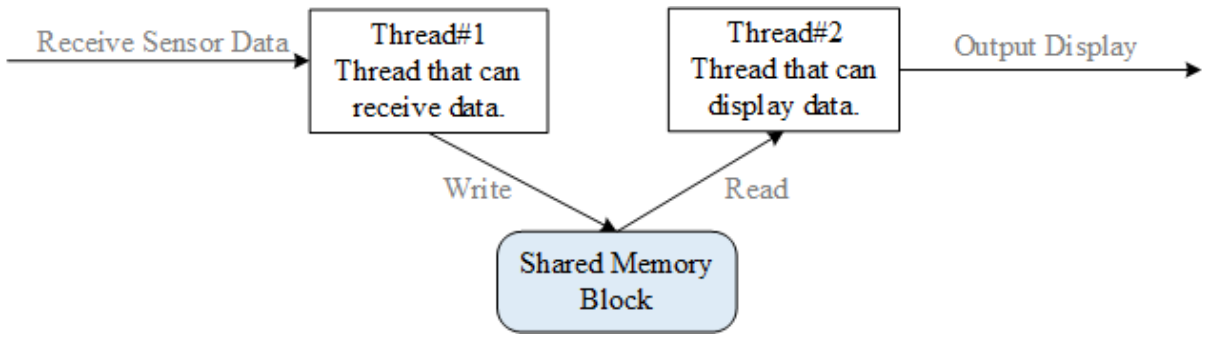


Figure 1.34 Diagram of Data Transfer between Threads

If access to shared memory is not exclusive, then it may be accessed simultaneously by each thread, which causes data consistency issues. For example, before thread #2 (thread that can display data) attempts to display data, thread #1 (thread that can receive data) has not yet completed the writing in of data, then the display will contain data sampled at different times, causing the display data to be disordered.

Thread #1 that writes the sensor data to the shared memory block and thread #2 that reads the sensor data from the shared memory block access the same memory block. In order to prevent data errors, the actions of the two threads must be mutually exclusive. One of threads should only be allowed after another thread completes its operation on the shared memory block. This way, thread #1 and thread #2 can work properly to execute this task correctly.

Synchronization refers to running in a predetermined order. Thread synchronization refers to multiple threads controlling the execution order between threads through specific mechanisms (such as mutex, event object, critical section). In other words, establish a relationship of execution order by synchronization between threads and if there is no synchronization, the threads will be out-of-order.

Multiple threads operate / access the same area (code), this block of code is called the critical section, the shared memory block in the above example is the critical section. Thread mutual exclusion refers to the exclusiveness of access to critical section resources. When multiple threads use critical section resources, only one thread is allowed each time. Other threads that want to use the resource must wait until the resource occupant releases the resource. Thread mutex can be seen as a special kind of thread synchronization.

There are many ways to synchronize threads. The core idea is that **only one (or one kind of) thread is allowed to run when accessing the critical section.** There are many ways to enter/exit the critical section:

1) Call rt_hw_interrupt_disable() to enter the critical section and call rt_hw_interrupt_enable() to exit the critical section; see the *Global Interrupt Switch* in *Interrupt Management* for details.

2)Call rt_enter_critical() to enter the critical section and call rt_exit_critical() to exit the critical section.

This chapter introduces several synchronization methods: **semaphores, mutex**, and **event**. After learning this chapter, you will learn how to use semaphores, mutex, and event to synchronize threads.

1.3.5.1 Semaphores

Take parking lot as an example to understand the concept of semaphore:

When the parking lot is empty, the administrator of the parking lot finds that there are a lot of empty parking spaces. And then, cars outside will enter the parking lot and get parking spaces.

When the parking space of the parking lot is full, the administrator finds that there is no empty parking space. As a result, cars outside will be prohibited from entering the parking lot, and they will be waiting in line;

When cars are leaving the parking lot, the administrator finds that there are empty parking spaces for cars outside to enter the parking lot; after the empty parking spaces are taken, cars outside are prohibited from entering.

In this example, the administrator is equivalent to the semaphore. The number of empty parking spaces that the administrator is in charge of is the value of the semaphore (non-negative, dynamic change); the parking space is equivalent to the common resource (critical section), and the cars are equivalent to the threads. Cars access the parking spaces by obtaining permission from the administrator, which is similar to thread accessing public resource by obtaining the semaphore.

1.3.5.1.1 Semaphore Working Mechanism

Semaphore is a light-duty kernel object that can solve the problems of synchronization between threads. By obtaining or releasing semaphore, a thread can achieve synchronization or mutual exclusion.

Schematic diagram of semaphore is shown in the figure below. Each semaphore object has a semaphore value and a thread waiting queue. Semaphore value corresponds to the actual number of instances of semaphore object and the number of resources. If the semaphore value is 5, it means that there are 5 semaphore instances (resources) that can be used. If the number of semaphore instances is zero, the thread that is applying for the semaphore will be suspended on the waiting queue of the semaphore, waiting for available semaphore instances (resources).

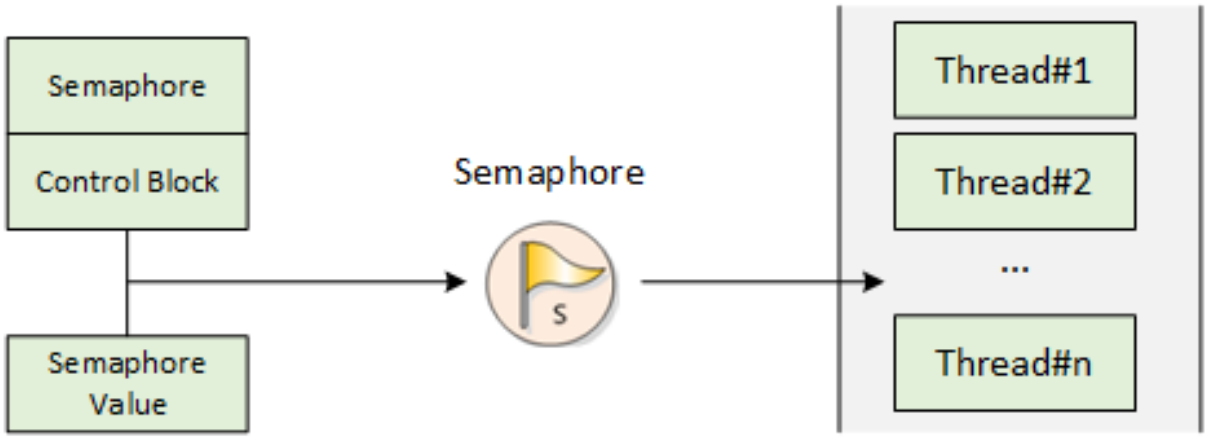


Figure 1.35 Schematic Diagram of Semaphore

1.3.5.1.2 Semaphore Control Block

In RT-Thread, semaphore control block is a data structure used by the operating system to manage semaphores, represented by struct rt rt_semaphore. Another C expression is rt_sem_t, which represents the handle of the semaphore, and the implementation in C language is a pointer to the semaphore control block. The detailed definition of semaphore control block structure is as follows:

```
struct rt_semaphore
{
   struct rt_ipc_object parent;  /* Inherited from the ipc_object class */
   rt_uint16_t value;              /* Semaphore Value */
};
/* rt_sem_t is the type of pointer pointing to semaphore structure */
typedef struct rt_semaphore* rt_sem_t;
```

rt_semaphore object is derived from rt_ipc_object and is managed by the IPC container. The maximum semaphore is 65535.

1.3.5.1.3 Semaphore Management

Semaphore control block contains important parameters related to the semaphore and acts as a link between various states of the semaphore. Interfaces related to semaphore are shown in the figure below. Operations on a semaphore includes: creating/initializing the semaphore, obtaining the semaphore, releasing the semaphore, and deleting/detaching the semaphore.

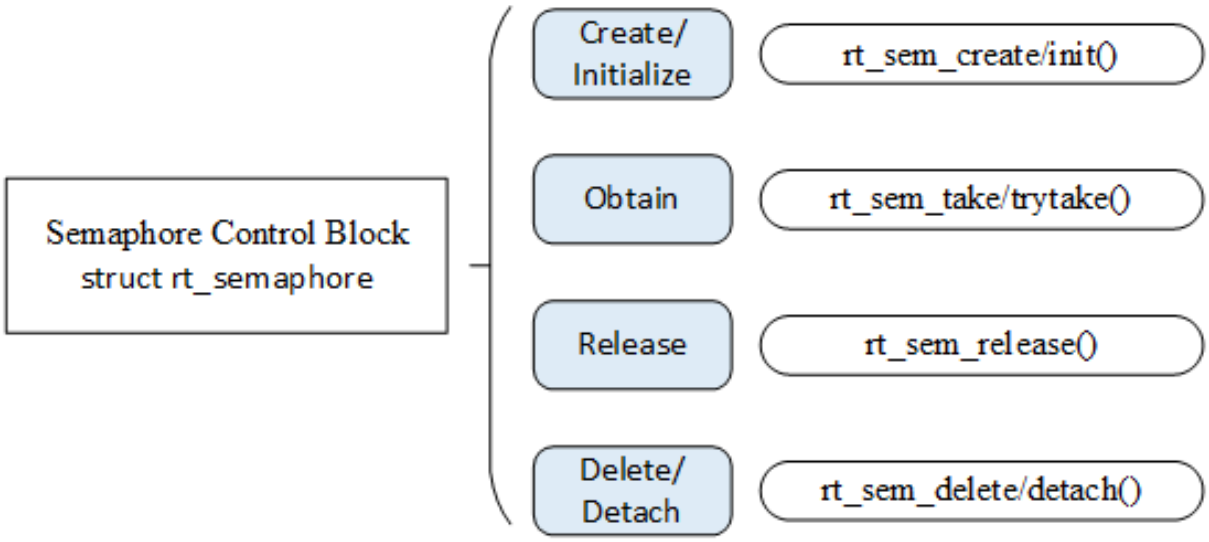


Figure 1.36 Interfaces Related to Semaphore

1.3.5.1.3.1 Create and Delete Semaphore

When creating a semaphore, the kernel first creates a semaphore control block, then performs basic initialization on the control block. The following function interface is used to create a semaphore:

```
rt_sem_t rt_sem_create(const char *name,
                       rt_uint32_t value,
                       rt_uint8_t flag);
```

When this function is called, the system will first allocate a semaphore object from the object manager and initialize the object, and then initialize the parent class IPC object and semaphore-related parts. Among parameters specified in the creation of semaphore, semaphore flag parameter determines the queuing way of how multiple threads wait when the semaphore is not available. When the RT_IPC_FLAG_FIFO (first in, first out) mode is selected, the waiting thread queue will be queued in a first in first out manner. The first thread that goes in will firstly obtain the waiting semaphore. When the RT_IPC_FLAG_PRIO (priority waiting) mode is selected, the waiting threads will be queued in order of priority. Threads waiting with the highest priority will get the wait semaphore first. The following table describes the input parameters and return values for this function:

Input parameters and return values of rt_sem_create()

Parameters	Description
name	Semaphore Name
value	Semaphore Initial Value
flag	Semaphore flag, which can be the following values: RT_IPC_FLAG_FIFO or RT_IPC_FLAG_PRIO
Return	——
RT_NULL	Creation failed
semaphore control block pointer	Creation successful

For dynamically created semaphores, when the system no longer uses semaphore, they can be removed to release system resources. To delete a semaphore, use the following function interface:

```
rt_err_t rt_sem_delete(rt_sem_t sem);
```

When this function is called, the system will delete this semaphore. If there is a thread waiting for this semaphore when it is being deleted, the delete operation will first wake up the thread waiting on the semaphore (return value of

the waiting thread is - RT_ERROR), then release the semaphore's memory resources. The following table describes the input parameters and return values for this function:

Input parameters and return values of rt_sem_delete()

Parameters	Description
sem	Semaphore object created by rt_sem_create()
Return	——
RT_EOK	Successfully deleted

1.3.5.1.3.2 Initialize and Detach Semaphore

For a static semaphore object, its memory space is allocated by the compiler during compiling and placed on the read-write data segment or on the uninitialized data segment. In this case,rt_sem_create interface is no longer needed to create the semaphore to use it, just initialize it before using it. To initialize the semaphore object, use the following function interface:

```
rt_err_t rt_sem_init(rt_sem_t       sem,
                     const char     *name,
                     rt_uint32_t    value,
                     rt_uint8_t     flag)
```

When this function is called, the system will initialize the semaphore object, then initialize the IPC object and parts related to the semaphore. The flag mentioned above in semaphore function creation can be used as the semaphore flag here. The following table describes the input parameters and return values for this function:

Input parameters and return values of rt_sem_init()

Parameters	Description
sem	Semaphore object handle
name	Semaphore name
value	Semaphore initial value
flag	Semaphore flag, which can be the following values: RT_IPC_FLAG_FIFO or RT_IPC_FLAG_↩ PRIO
Return	——
RT_EOK	Initialization successful

For statically initialized semaphore, detaching the semaphore is letting the semaphore object detach from the kernel object manager. To detach the semaphore, use the following function interface:

```
rt_err_t rt_sem_detach(rt_sem_t sem);
```

After using this function, the kernel wakes up all threads suspended in the semaphore wait queue and then detaches the semaphore from the kernel object manager. The waiting thread that was originally suspended on the semaphore will get the return value of -RT_ERROR. The following table describes the input parameters and return values for this function:

Input parameters and return values of rt_sem_detach()

Parameters	Description
sem	Semaphore object handle

Parameters	Description
Return	——
RT_EOK	Successfully detached

1.3.5.1.3.3 Obtain Semaphore

Thread obtains semaphore resource instances by obtaining semaphores. When the semaphore value is greater than zero, the thread will get the semaphore, and the corresponding semaphore value will be reduced by 1. The semaphore is obtained using the following function interface:

```
rt_err_t rt_sem_take (rt_sem_t sem, rt_int32_t time);
```

When calling this function, if the value of the semaphore is zero, it means the current semaphore resource instance is not available, and the thread applying for the semaphore will choose according to the time parameters to either return directly, or suspend for a period of time, or wait forever. While waiting, if other threads or ISR released the semaphore, then the thread will stop the waiting. If the semaphore is still not available after parameter specified time, the thread will time out and return, return value is -RT_ETIMEOUT. The following table describes the input parameters and return values for this function:

Input parameters and return values of rt_sem_take()

Parameters	Description
sem	Semaphore object handle
time	Specified wait time, unit is operating system clock tick (OS Tick)
Return	——
RT_EOK	Semaphore obtained successfully
-RT_ETIMEOUT	Did not received semaphore after timeout
-RT_ERROR	Other errors

1.3.5.1.3.4 Obtain Semaphore without Waiting

When user does not want to suspend thread on the applied semaphore and wait, the semaphore can be obtained using the wait-free mode , and the following function interface is used for obtaining the semaphore without waiting:

```
rt_err_t rt_sem_trytake(rt_sem_t sem);
```

This function has the same effect as rt_sem_take(sem, 0), which means when the semaphore resource instance requested by the thread is not available, it will not wait on the semaphore, instead it returns to -RT_ETIMEOUT directly. The following table describes the input parameters and return values for this function:

Input parameters and return values for rt_sem_trytake()

Parameter	Description
sem	Semaphore object handle
Return	——
RT_EOK	Semaphore successfully obtained
-RT_ETIMEOUT	Semaphore obtainment failed

1.3.5.1.3.5 Semaphore Release

Semaphore is released to wake up the thread that suspends on the semaphore. To release the semaphore, use the following function interface:

```
rt_err_t rt_sem_release(rt_sem_t sem);
```

For example, when semaphore value is zero and a thread is waiting for this semaphore, releasing the semaphore will wake up the first thread waiting in the thread queue of the semaphore, and this thread will obtain the semaphore; otherwise value of the semaphore will plus 1. The following table describes the input parameters and return values of the function:

Input parameters and return values of rt_sem_release()

Parameters	Description
sem	Semaphore object handle
Return	——
RT_EOK	Semaphore successfully released

1.3.5.1.4 Semaphore Application Sample

This is a sample of semaphore usage routine. This routine creates a dynamic semaphore and initializes two threads, one thread sends the semaphore, and one thread receives the semaphore and performs the corresponding operations. As shown in the following code:

Use of semaphore

```
#include <rtthread.h>

#define THREAD_PRIORITY         25
#define THREAD_TIMESLICE        5

/* pointer to semaphore */
static rt_sem_t dynamic_sem = RT_NULL;

ALIGN(RT_ALIGN_SIZE)
static char thread1_stack[1024];
static struct rt_thread thread1;
static void rt_thread1_entry(void *parameter)
{
    static rt_uint8_t count = 0;

    while(1)
    {
        if(count <= 100)
        {
            count++;
        }
        else
            return;

        /* count release semaphore every 10 counts */
         if(0 == (count % 10))
        {
            rt_kprintf("t1 release a dynamic semaphore.\n");
            rt_sem_release(dynamic_sem);
        }
    }
}

ALIGN(RT_ALIGN_SIZE)
static char thread2_stack[1024];
static struct rt_thread thread2;
static void rt_thread2_entry(void *parameter)
{
    static rt_err_t result;
    static rt_uint8_t number = 0;
    while(1)
    {
```

```
        /* permanently wait for the semaphore; once obtain the semaphore, perform the number self-add
      operation */
        result = rt_sem_take(dynamic_sem, RT_WAITING_FOREVER);
        if (result != RT_EOK)
        {
            rt_kprintf("t2 take a dynamic semaphore, failed.\n");
            rt_sem_delete(dynamic_sem);
            return;
        }
        else
        {
            number++;
            rt_kprintf("t2 take a dynamic semaphore. number = %d\n" ,number);
        }
    }
}

/* initialization of the semaphore sample */
int semaphore_sample(void)
{
    /* create a dynamic semaphore with an initial value of 0 */
    dynamic_sem = rt_sem_create("dsem", 0, RT_IPC_FLAG_FIFO);
    if (dynamic_sem == RT_NULL)
    {
        rt_kprintf("create dynamic semaphore failed.\n");
        return -1;
    }
    else
    {
        rt_kprintf("create done. dynamic semaphore value = 0.\n");
    }

    rt_thread_init(&thread1,
                   "thread1",
                   rt_thread1_entry,
                   RT_NULL,
                   &thread1_stack[0],
                   sizeof(thread1_stack),
                   THREAD_PRIORITY, THREAD_TIMESLICE);
    rt_thread_startup(&thread1);

    rt_thread_init(&thread2,
                   "thread2",
                   rt_thread2_entry,
                   RT_NULL,
                   &thread2_stack[0],
                   sizeof(thread2_stack),
                   THREAD_PRIORITY-1, THREAD_TIMESLICE);
    rt_thread_startup(&thread2);

    return 0;
}
/* export to msh command list */
MSH_CMD_EXPORT(semaphore_sample, semaphore sample);
```

Simulation results:

```
 \ | /
- RT -     Thread Operating System
 / | \     3.1.0 build Aug 27 2018
 2006 - 2018 Copyright by rt-thread team
msh >semaphore_sample
create done. dynamic semaphore value = 0.
msh >t1 release a dynamic semaphore.
t2 take a dynamic semaphore. number = 1
t1 release a dynamic semaphore.
t2 take a dynamic semaphore. number = 2
t1 release a dynamic semaphore.
t2 take a dynamic semaphore. number = 3
t1 release a dynamic semaphore.
t2 take a dynamic semaphore. number = 4
t1 release a dynamic semaphore.
t2 take a dynamic semaphore. number = 5
t1 release a dynamic semaphore.
t2 take a dynamic semaphore. number = 6
t1 release a dynamic semaphore.
t2 take a dynamic semaphore. number = 7
t1 release a dynamic semaphore.
t2 take a dynamic semaphore. number = 8
t1 release a dynamic semaphore.
t2 take a dynamic semaphore. number = 9
t1 release a dynamic semaphore.
t2 take a dynamic semaphore. number = 10
```

As the result of the operation above: Thread 1 sends a semaphore when the count is multiple of 10 (the thread exits after the count reaches 100), and thread 2 adds one on top of the number after receiving the semaphore.

Another semaphore application routine is shown below. This sample will use two threads and three semaphores to implement an example of producers and consumers. Among them:

Another application routine for semaphores is as follows:

To be more accurate, the producer consumer model is actually a "producer-consumer-warehouse" model. We call the available spots in the warehouse "empty seats", and once an available spot ("empty seat") is taken, we then call it a "full seat". For this model, the following points should be clarified: 1The producer only produces when the warehouse is not full; the producer will stop production when the warehouse is full. 2The consumer can consume only when there are products in the warehouse; consumers will wait if the warehouse is empty. 3When the consumer consumes, the warehouse is not full anymore, and then the producer will be notified to produce again. 4The producer should notify the consumer to consumer after it produces consumable products.

This routine will use two threads and three semaphores to implement examples of producers and consumers. Among them The three semaphores in the example are: sem_lock: This semaphore acts as a lock because both threads operate on the same array array which means this array is a shared resource and sem_lock is used to protect this shared resource. sem_empty: Its value is used to indicate the number of "warehouse" available seats , and the value of sem_empty is initialized to 5, indicating that there are 5 "empty seats" . sem_fullIts value is used to indicate the number of "full seats" in the "warehouse", and the value of sem_full is initialized to 0, indicating that there are 0 "full seats".

The 2 threads in the example are: Producer thread: After obtaining the available seat (sem_empty value minus 1), generate a number, loop into the array, and then release a "full seat"(sem_full value plus 1). Consumer thread: After getting the "full seat" (the value of sem_full is decremented by 1), the contents of the array are read and added, and then an "empty seat" is released (the value of sem_empty is increased by 1).

Producer consumer routine

```
#include <rtthread.h>

#define THREAD_PRIORITY       6
#define THREAD_STACK_SIZE     512
#define THREAD_TIMESLICE      5

/* Define a maximum of 5 elements to be generated */
#define MAXSEM 5

/* An array of integers used to place production */
rt_uint32_t array[MAXSEM];

/* Point to the producer and consumer's read-write position in the array  */
static rt_uint32_t set, get;

/* Pointer to the thread control block */
static rt_thread_t producer_tid = RT_NULL;
static rt_thread_t consumer_tid = RT_NULL;

struct rt_semaphore sem_lock;
struct rt_semaphore sem_empty, sem_full;

/* Pointer to the thread control block */
void producer_thread_entry(void *parameter)
{
    int cnt = 0;

    /* Run for 10 times*/
    while (cnt < 10)
    {
        /* Obtain one vacancy */
        rt_sem_take(&sem_empty, RT_WAITING_FOREVER);

        /* Modify array content, lock */
        rt_sem_take(&sem_lock, RT_WAITING_FOREVER);
        array[set % MAXSEM] = cnt + 1;
        rt_kprintf("the producer generates a number: %d\n", array[set % MAXSEM]);
        set++;
        rt_sem_release(&sem_lock);

        /*  */
        rt_sem_release(&sem_full);
        cnt++;

        /* Pause for a while */
        rt_thread_mdelay(20);
```

```
    }

    rt_kprintf("the producer exit!\n");
}

/* Consumer thread entry */
void consumer_thread_entry(void *parameter)
{
    rt_uint32_t sum = 0;

    while (1)
    {
        /* obtain a "full seat" */
        rt_sem_take(&sem_full, RT_WAITING_FOREVER);

        /* Critical region, locked for operation */
        rt_sem_take(&sem_lock, RT_WAITING_FOREVER);
        sum += array[get % MAXSEM];
        rt_kprintf("the consumer[%d] get a number: %d\n", (get % MAXSEM), array[get % MAXSEM]);
        get++;
        rt_sem_release(&sem_lock);

        /* Release one vacancy */
        rt_sem_release(&sem_empty);

        /* The producer produces up to 10 numbers, stops, and the consumer thread stops accordingly */
        if (get == 10) break;

        /* Pause for a while */
        rt_thread_mdelay(50);
    }

    rt_kprintf("the consumer sum is: %d\n", sum);
    rt_kprintf("the consumer exit!\n");
}

int producer_consumer(void)
{
    set = 0;
    get = 0;

    /* Initialize 3 semaphores */
    rt_sem_init(&sem_lock, "lock",     1,      RT_IPC_FLAG_FIFO);
    rt_sem_init(&sem_empty, "empty",   MAXSEM, RT_IPC_FLAG_FIFO);
    rt_sem_init(&sem_full, "full",     0,      RT_IPC_FLAG_FIFO);

    /* Create producer thread */
    producer_tid = rt_thread_create("producer",
                                    producer_thread_entry, RT_NULL,
                                    THREAD_STACK_SIZE,
                                    THREAD_PRIORITY - 1,
                                    THREAD_TIMESLICE);
    if (producer_tid != RT_NULL)
    {
        rt_thread_startup(producer_tid);
    }
    else
    {
        rt_kprintf("create thread producer failed");
        return -1;
    }

    /* Create consumer thread */
    consumer_tid = rt_thread_create("consumer",
                                    consumer_thread_entry, RT_NULL,
                                    THREAD_STACK_SIZE,
                                    THREAD_PRIORITY + 1,
                                    THREAD_TIMESLICE);
    if (consumer_tid != RT_NULL)
    {
        rt_thread_startup(consumer_tid);
    }
    else
    {
        rt_kprintf("create thread consumer failed");
        return -1;
    }

    return 0;
}

/* Export to msh command list */
MSH_CMD_EXPORT(producer_consumer, producer_consumer sample);
```

The simulation results for this routine are as follows:

```
\ | /
```

```
- RT -     Thread Operating System
 / | \     3.1.0 build Aug 27 2018
 2006 - 2018 Copyright by rt-thread team
msh >producer_consumer
the producer generates a number: 1
the consumer[0] get a number: 1
msh >the producer generates a number: 2
the producer generates a number: 3
the consumer[1] get a number: 2
the producer generates a number: 4
the producer generates a number: 5
the producer generates a number: 6
the consumer[2] get a number: 3
the producer generates a number: 7
the producer generates a number: 8
the consumer[3] get a number: 4
the producer generates a number: 9
the consumer[4] get a number: 5
the producer generates a number: 10
the producer exit!
the consumer[0] get a number: 6
the consumer[1] get a number: 7
the consumer[2] get a number: 8
the consumer[3] get a number: 9
the consumer[4] get a number: 10
the consumer sum is: 55
the consumer exit!
```

This routine can be understood as the process of producers producing products and putting them into the warehouse and the consumer taking the products from the warehouse.

(1) Producer thread:

1) Obtain 1 "empty seat" (put product number), now the number of "empty seats" is decremented by 1;

2) Lock protection; the generated number value is cnt+1, and the value is looped into the array; then unlocked;

3) Release 1 "full seat" (put one product into the warehouse, there will be one more "full seat" in the warehouse), add 1 to the number "full seats".

(2) Consumer thread

1Obtain 1 "full seat" (take product number), then the the number of "full seats" is decremented by 1;

2Lock protection; read the number produced by the producer from array and add it to the last number; then unlock it;

3Release 1 "empty seat" (take one product from the warehouse, then there is one more "empty seat" in the warehouse), add 1 to the number of "empty seats".

The producer generates 10 numbers in turn, and the consumers take them away in turn and sum the values of the 10 numbers. Semaphore lock protects array critical region resources, ensuring the exclusivity of number taking for the consumers each time and achieving inter-thread synchronization.

1.3.5.1.5 Semaphore Usage Occasion

Semaphores is a very flexible way to synchronize and can be used in a variety of situations, like forming locks, synchronization, resource counts, etc. It can also be conveniently used for synchronization between threads and threads, interrupts and threads.

1.3.5.1.5.1 Thread Synchronization

Thread synchronization is one of the simplest types of semaphore applications. For example, using semaphores to synchronize between two threads, the value of the semaphore is initialized to 0, indicating that there are 0 semaphore resource instances; and the thread attempting to acquire the semaphore will wait directly on this semaphore.

When the thread holding the semaphore completes the work it is processing, it will release this semaphore. Thread waiting on this semaphore can be awaken, and it can then perform the next part of the work. This occasion can also be seen as using the semaphore for the work completion flag: the thread holding the semaphore completes its own work, and then notifies the thread waiting for the semaphore to continue the next part of the work.

1.3.5.1.5.2 Lock

Locks, a single lock is often applied to multiple threads accessing the same shared resource (in other words, critical region). When semaphore is used as a lock, the semaphore resource instance should normally be initialized to 1, indicating that the system has one resource available by default. Because the semaphore value always varies between 1 and 0, so this type of lock is also called binary semaphore. As shown in the following figure, when a thread needs to access shared resource, it needs to obtain the resource lock first. When this thread successfully obtains the resource lock, other threads that intend to access the shared resource will suspend because they cannot obtain the resource. This is because it is already locked (semaphore value is 0) when other threads are trying to obtain the lock. When thread holding the semaphore is processed and exiting the critical region, it will release the semaphore and unlock the lock, and the first waiting thread that is suspending on the lock will be awaken to gain access to the critical region.

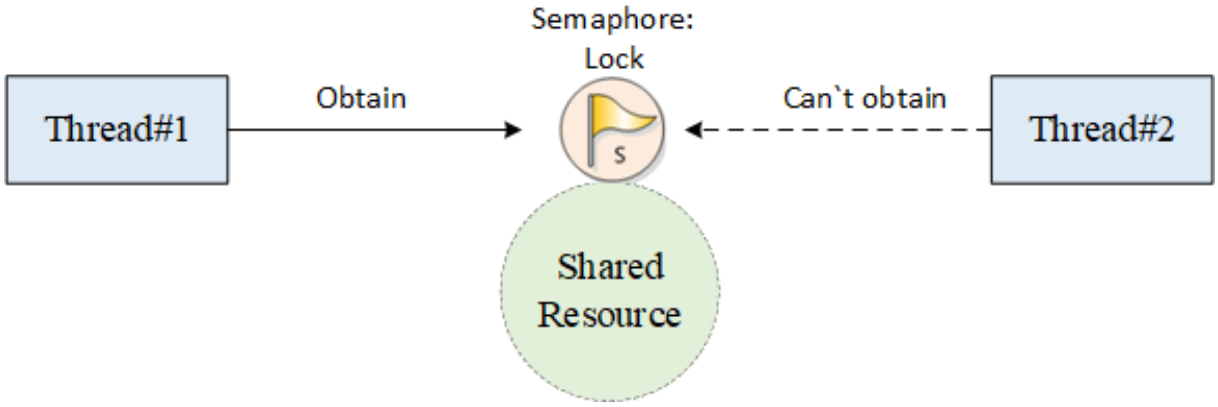


Figure 1.37 Lock

1.3.5.1.5.3 Interrupt Synchronization between Threads

Semaphore can also be easily applied to interrupting synchronization between threads, such as an interrupt trigger. When interrupting service routine, thread needs to be notifies to perform corresponding data processing. At this time, the initial value of the semaphore can be set to 0. When the thread tries to hold this semaphore, since the initial value of the semaphore is 0, the thread will then suspends on this semaphore until the semaphore is released. When the interrupt is triggered, hardware-related actions are firstly performed, such as reading corresponding data from the hardware I/O port, and confirming the interrupt to clear interrupt source, and then releasing a semaphore to wake up the corresponding thread for subsequent data processing. For example, the processing of FinSH threads is as shown in the following figure.

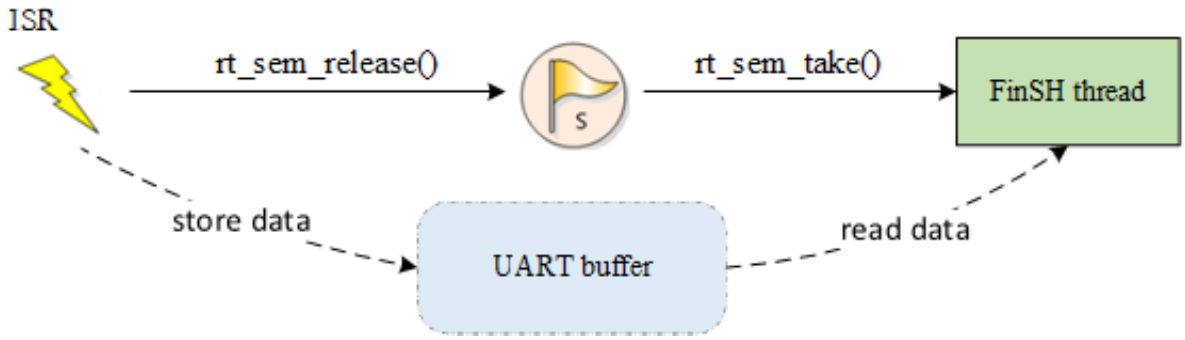


Figure 1.38 sync between ISR and FinSH thread

The value of the semaphore is initially 0. When the FinSH thread attempts to obtain the semaphore, it will be suspended because the semaphore value is 0. When the console device has data input, an interrupt is generated to enter the interrupt service routine. In the interrupt service routine, it reads the data of the console device, puts the read data into the UART buffer for buffering, and then releases the semaphore. The semaphore release will wake up the shell thread. After the interrupt service routine has finished, if there are no ready threads with higher priority than the shell thread in the system, the shell thread will hold the semaphore and run, obtaining the input data from the UART buffer.

>The mutual exclusion between interrupts and threads cannot be done by means of semaphores (locks), but by means of switch interrupts.

1.3.5.1.5.4 Resource Count

Semaphore can also be considered as an incrementing or decrementing counter. It should be noted that the semaphore value is non-negative. For example, if the value of a semaphore is initialized to 5, then the semaphore can be reduced by a maximum of 5 consecutive times until the counter is reduced to zero. Resource count is suitable for occasions where the processing speeds between threads do not match. At this time, the semaphore can be counted as the number of completed tasks of the previous thread, and when dispatched to the next thread, it can also be used in a continuous manner handling multiple events each time. For example, in the producer and consumer problem, the producer can release the semaphore multiple times, and then the consumer can process multiple semaphore resources each time when dispatched.

>Generally, resource count is mostly inter-thread synchronization in a hybrid mode, because there are still multiple accesses from threads for a single resource processing, which requires accessing and processing for a single resource, and operate lock mutex operation.

1.3.5.2 Mutex

Mutexes, also known as mutually exclusive semaphores, are a special binary semaphore. Mutex is similar to a parking lot with only one parking space: when one car enters, the parking lot gate is locked and other vehicles are waiting outside. When the car inside comes out, parking lot gate will open and the next car can enter.

1.3.5.2.1 Mutex Working Mechanism

The difference between a mutex and a semaphore is that the thread with a mutex has ownership of the mutex, mutex supports recursive access and prevents thread priority from reversing; and mutex can only be released by the thread holding it, whereas semaphore can be released by any thread.

There are only two states of mutex, unlocked and locked (two state values). When a thread holds it, then the mutex is locked and its ownership is obtained by this thread. Conversely, when this thread releases it, it unlocks the mutex and loses its ownership. When a thread is holding a mutex, other threads will not be able to unlock this mutex or hold it. The thread holding the mutex can also acquire the lock again without being suspended, as shown in the following figure. This feature is quite different from the general binary semaphore: in semaphore, because there is no instance, the thread will suspend if the thread recursively holds the semaphore (which eventually leads to a deadlock).

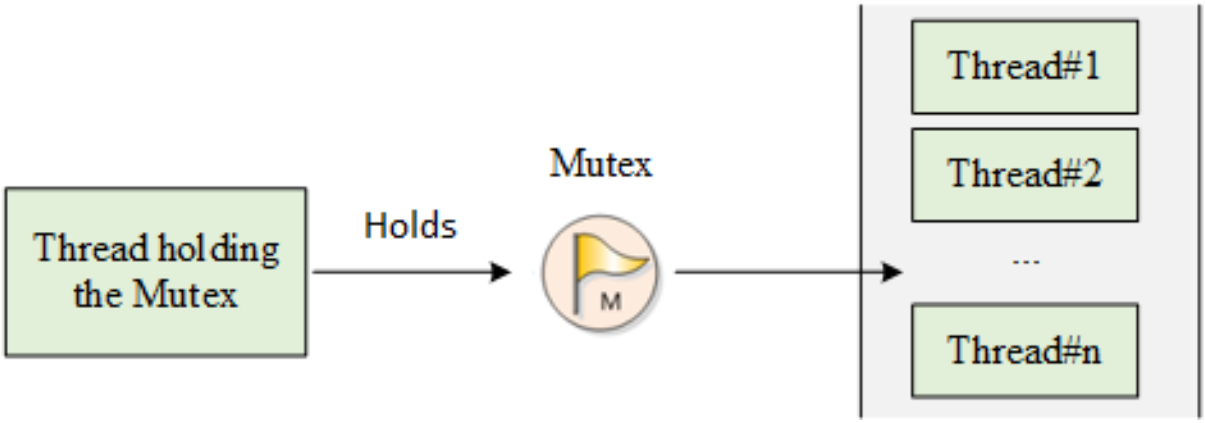


Figure 1.39 Mutex Working Mechanism Diagram

Another potential problem with using semaphores is thread priority inversion. The so-called priority inversion is when a high-priority thread attempts to access shared resource through the semaphore mechanism, if the semaphore is already held by a low-priority thread which may happen to be preempted by other medium-priority threads while running, this leads to high-priority threads being blocked by many lower-priority threads which means instantaneity is difficult to guarantee. As shown in the following figure: There are three threads with the priority levels A, B, and C, priority A$>$ B $>$ C. Threads A and B are in suspended state, waiting for an event to trigger; thread C is running, and thread C starts using a shared resource M. While using the resource, the event thread A is waiting for occurs and thread A switches to ready state because it has higher priority than thread C, so it executes immediately. But when thread A wants to use shared resource M, because it is being used by thread C, thread A is suspended and thread C is running. If the event thread B is waiting for occurs, thread B switches to ready state. Since thread B has a higher priority than thread C, thread B starts running, thread C won't run until thread B finishes. Thread A is only executed when thread C releases the shared resource M. In this case, the priority has been reversed: thread B runs before thread A. This does not guarantee the response time for high priority threads.

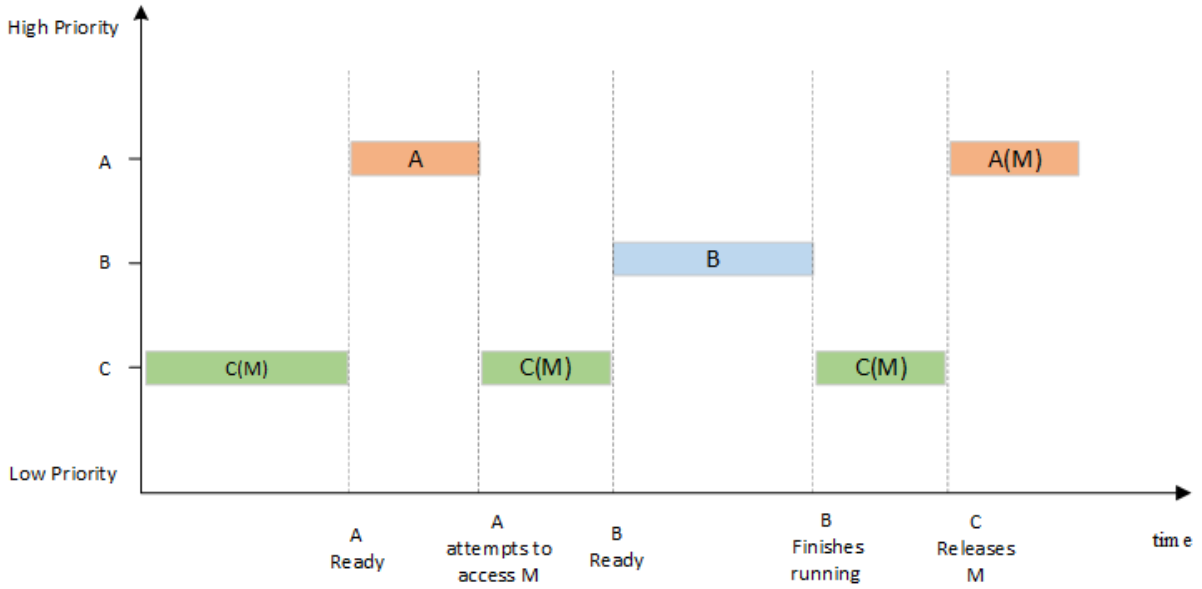


Figure 1.40 Priority Inversion (M is the semaphore)

In the RT-Thread operating system, mutex can solve the priority inversion problem and implement the priority inheritance algorithm. Priority inheritance solves the problems caused by priority inversion by raising the priority of thread C to the priority of thread A during the period during when thread A is suspended trying to access the shared resource. This prevents C (indirectly preventing A) from being preempted by B, as shown in the following figure. Priority inheritance refers to raising the priority of a low-priority thread that occupies a certain resource, making the priority level of the low-priority thread to be equal to the priority of the thread with the highest priority level among all threads waiting for the resource, and then executes. When this is low-priority releases the resource, the priority level returns to the initial setting. Therefore, threads that inherit priority help to prevent the system resources from being preempted by any intermediate-priority thread.

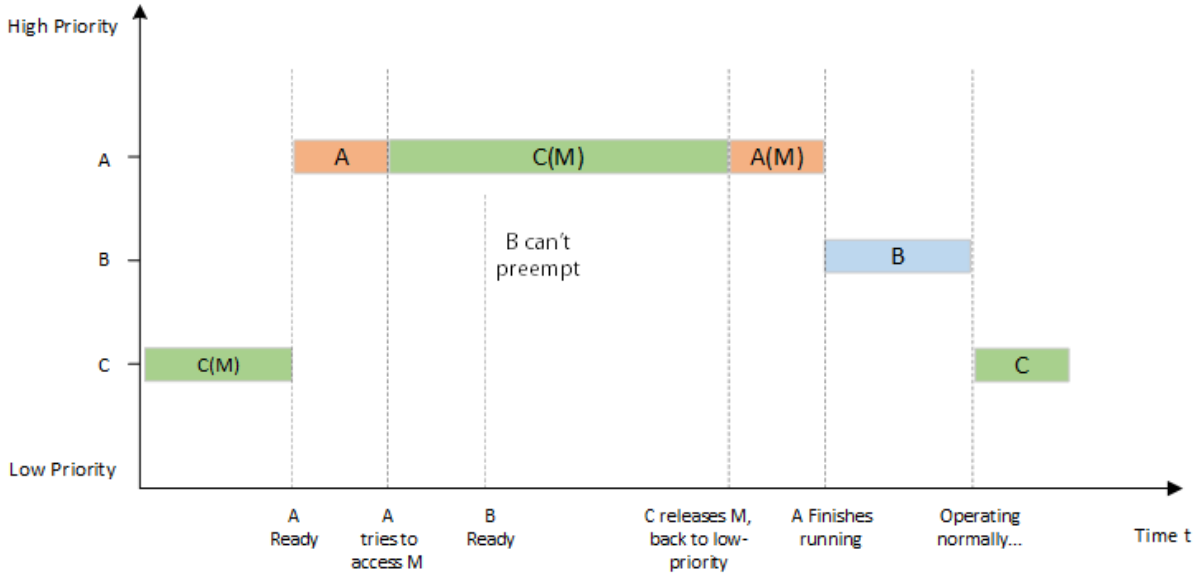


Figure 1.41 Priority Inheritance (M is a mutex)

>After the mutex is obtained, release the mutex as soon as possible. During the time when holding the mutex, you must not change the priority of the thread holding the mutex.

1.3.5.2.2 Mutex Control Block

In RT-Thread, mutex control block is a data structure used by the operating system to manage mutexes, represented by the struct rt rt_mutex. Another C expression, rt_mutex_t, represents the handle of the mutex, and the implementation in C language refers to the pointer of the mutex control block. See the following code for a detailed definition of the mutex control block structure:

```
struct rt_mutex
    {
        struct rt_ipc_object parent;                /* inherited from the ipc_object class */

        rt_uint16_t          value;                   /* mutex value */
        rt_uint8_t           original_priority;     /* hold the original priority of the thread */
        rt_uint8_t           hold;                    /* number of times holding the threads */
        struct rt_thread    *owner;                 /* thread that currently owns the mutex */
    };
    /* rt_mutext_t pointer type of the one poniter pointing to the mutex structure */
    typedef struct rt_mutex* rt_mutex_t;
```

The rt_mutex object is derived from rt_ipc_object and is managed by the IPC container.

1.3.5.2.3 Mutex Management

The mutex control block contains important parameters related to mutex and it plays an important role in the implementation of the mutex function. The mutex-related interface is as shown in the following figure. The operation of a mutex includes: creating/initiating a mutex, obtaining a mutex, releasing a mutex, and deleting/detaching a mutex.

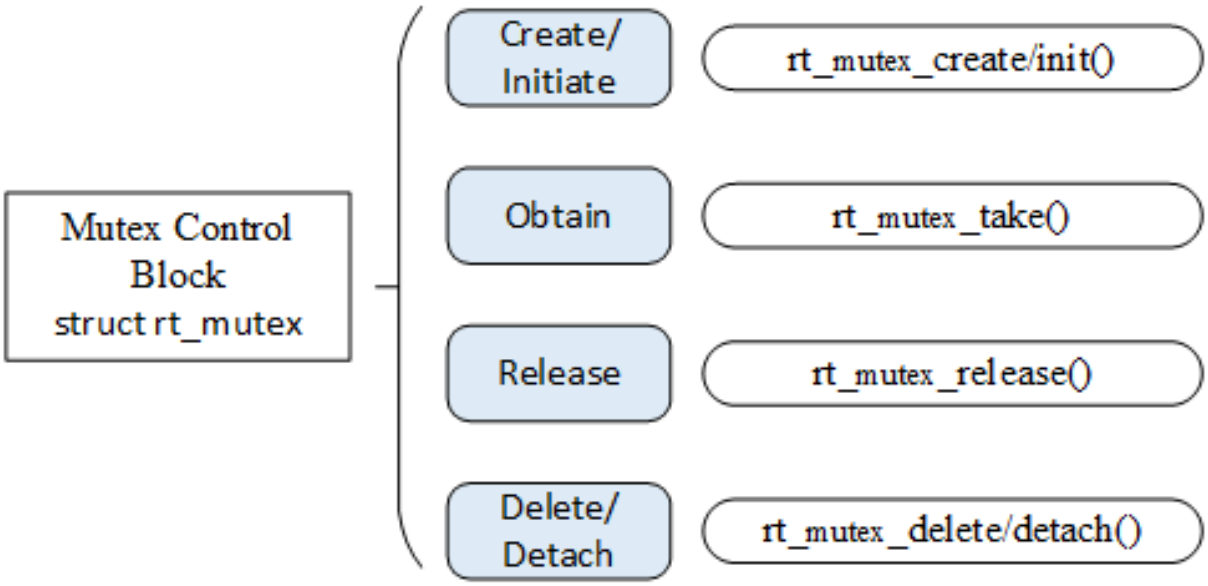


Figure 1.42 Mutex Related Interface

1.3.5.2.3.1 Create and Delete Mutex

When creating a mutex, the kernel first creates a mutex control block and then completes the initialization of the control block. Create a mutex using the following function interface:

```
rt_mutex_t rt_mutex_create (const char* name, rt_uint8_t flag);
```

You can call the rt_mutex_create function to create a mutex whose name is designated by name. When this function is called, the system will first allocate a mutex object from the object manager, initialize the object, and then initialize the parent class IPC object and the mutex-related part. The flag of the mutex is set to RT_IPC_FLAG_PRIO, which means that when multiple threads are waiting for resources, the resources will be accessed by the thread with higher priority. The flag is set to RT_IPC_FLAG_FIFO, which means that when multiple threads are waiting for resources, resources are being accesses in a first-come-first-served order. The following table describes the input parameters and return values for this function:

Input parameters and return values of rt_mutex_create()

Parameters	Description
name	Mutex name
flag	Mutex flag, which can take the following values: RT_IPC_FLAG_FIFO or RT_IPC_FLAG_PRIO
Return	——
Mutex handle	Created successfully
RT_NULL	Creation failed

For dynamically created mutex, when the mutex is no longer used, the system resource is released by removing the mutex. To remove a mutex, use the following function interface:

```
rt_err_t rt_mutex_delete (rt_mutex_t mutex);
```

When a mutex is deleted, all threads waiting for this mutex will be woken up, return value for the waiting threads is - RT_ERROR. The system then removes the mutex from the kernel object manager linked list and releases the memory space occupied by the mutex. The following table describes the input parameters and return values for this function:

Input parameters and return values of rt_mutex_delete()

Parameters	Description
mutex	The handle of the mutex object
Return	——
RT_EOK	Deleted successfully

1.3.5.2.3.2 Initialize and Detach Mutex

The memory of a static mutex object is allocated by the compiler during system compilation, and is usually placed in a read-write data segment or an uninitialized data segment. Before using such static mutex objects, you need to initialize them first. To initialize the mutex, use the following function interface:

```
rt_err_t rt_mutex_init (rt_mutex_t mutex, const char* name, rt_uint8_t flag);
```

When using this function interface, you need to specify the handle of the mutex object (that is, the pointer to the mutex control block), the mutex name, and the mutex flag. The mutex flag can be the flags mentioned in the creation of mutex function above. The following table describes the input parameters and return values for this function:

Input parameters and return values of rt_mutex_init()

Parameters	Description
mutex	The handle of the mutex object, which is provided by the user and points to the memory block of the mutex object
name	Mutex name
flag	Mutex flag, which can take the following values: RT_IPC_FLAG_FIFO or RT_IPC_FLAG_PRIO
Return	——
RT_EOK	Initialization successful

For statically initialized muex, detaching mutex means to remove the mutex object from the kernel object manager. To detach the mutex, use the following function interface:

```
rt_err_t rt_mutex_detach (rt_mutex_t mutex);
```

After using this function interface, the kernel wakes up all threads suspended on the mutex (the return value of the thread is -RT_ERROR), and then the system detaches the mutex from the kernel object manager. The following table describes the input parameters and return values for this function:

Input parameters and return values for rt_mutex_detach()

Parameters	Description
mutex	The handle of the mutex object
Return	——
RT_EOK	Successful

1.3.5.2.3.3 Obtain Mutex

Once the thread obtains the mutex, the thread has ownership of the mutex, that is, a mutex can only be held by one thread at a time. To obtain the mutex, use the following function interface:

```
rt_err_t rt_mutex_take (rt_mutex_t mutex, rt_int32_t time);
```

If the mutex is not controlled by another thread, the thread requesting the mutex will successfully obtain the mutex. If the mutex is already controlled by the current thread, then add one 1 to the number of holds for the mutex, and the current thread will not be suspended to wait. If the mutex is already occupied by another thread, the current thread suspends and waits on the mutex until another thread releases it or until the specified timeout elapses. The following table describes the input parameters and return values for this function:

Input parameters and return values of rt_mutex_take()

Parameters	Description
mutex	The handle of the mutex object
time	Specified waiting time
Return	——
RT_EOK	Successfully obtained mutex
-RT_ETIMEOUT	Timeout
-RT_ERROR	Failed to obtain

1.3.5.2.3.4 Release Mutex

When a thread completes the access to a mutually exclusive resource, it should release the mutex it occupies as soon as possible, so that other threads can obtain the mutex in time. To release the mutex, use the following function interface:

```
rt_err_t rt_mutex_release(rt_mutex_t mutex);
```

When using this function interface, only threads that already have control of the mutex can release it. Each time the mutex is released, its holding count is reduced by one. When the mutex's holding count is zero (that is, the holding thread has released all holding operations), it becomes available, threads waiting on the semaphore is awaken. If the thread's priority is increased by the mutex, then when the mutex is released, the thread reverts to the priority level before holding the mutex. The following table describes the input parameters and return values for this function:

Input parameters and return values of rt_mutex_release()

Parameters	Description
mutex	The handle of the mutex object
Return	——
RT_EOK	Success

1.3.5.2.4 Mutex Application Sample

This is a mutex application routine, and a mutex lock is a way to protect shared resources. When a thread has the mutex lock, it can protect shared resources from being destroyed by other threads. The following example can be used to illustrate. There are two threads: thread 1 and thread 2, thread 1 adds 1 to each of the two numbers; thread 2 also adds 1 to each of the two numbers. mutex is used to ensure that the operation of the thread changing the values of the 2 numbers is not interrupted. As shown in the following code:

Mutex routine

```
#include <rtthread.h>

#define THREAD_PRIORITY         8
#define THREAD_TIMESLICE        5

/* Pointer to the mutex */
static rt_mutex_t dynamic_mutex = RT_NULL;
static rt_uint8_t number1,number2 = 0;

ALIGN(RT_ALIGN_SIZE)
static char thread1_stack[1024];
static struct rt_thread thread1;
static void rt_thread_entry1(void *parameter)
{
      while(1)
      {
          /* After thread 1 obtains the mutex, it adds 1 to number1 and number2, and then releases the
      mutex. */
          rt_mutex_take(dynamic_mutex, RT_WAITING_FOREVER);
          number1++;
          rt_thread_mdelay(10);
          number2++;
          rt_mutex_release(dynamic_mutex);
       }
}

ALIGN(RT_ALIGN_SIZE)
static char thread2_stack[1024];
static struct rt_thread thread2;
static void rt_thread_entry2(void *parameter)
{
      while(1)
      {
          /* After thread 2 obtains the mutex, check whether the values of number1 and number2 are the same.
      If they are the same, it means the mutex succeesfully played the role of a lock. */
          rt_mutex_take(dynamic_mutex, RT_WAITING_FOREVER);
          if(number1 != number2)
          {
            rt_kprintf("not protect.number1 = %d, number2 = %d \n",number1 ,number2);
          }
          else
          {
            rt_kprintf("mutex protect ,number1 = number2 is %d\n",number1);
          }

           number1++;
           number2++;
           rt_mutex_release(dynamic_mutex);

          if(number1>=50)
              return;
      }
}

/* Initialization of the mutex sample */
int mutex_sample(void)
{
    /* Create a dynamic mutex */
    dynamic_mutex = rt_mutex_create("dmutex", RT_IPC_FLAG_FIFO);
    if (dynamic_mutex == RT_NULL)
    {
        rt_kprintf("create dynamic mutex failed.\n");
        return -1;
    }

    rt_thread_init(&thread1,
                   "thread1",
                   rt_thread_entry1,
                   RT_NULL,
                   &thread1_stack[0],
                   sizeof(thread1_stack),
                   THREAD_PRIORITY, THREAD_TIMESLICE);
    rt_thread_startup(&thread1);

    rt_thread_init(&thread2,
                   "thread2",
                   rt_thread_entry2,
                   RT_NULL,
                   &thread2_stack[0],
                   sizeof(thread2_stack),
                   THREAD_PRIORITY-1, THREAD_TIMESLICE);
    rt_thread_startup(&thread2);
    return 0;
}

/* Export to the MSH command list */
```

```
MSH_CMD_EXPORT(mutex_sample, mutex sample);
```

Both thread 1 and thread 2 use mutexes to protect the operation on the 2 numbers (if the obtain and release mutex statements in thread 1 are commented out, thread 1 will no longer protect number), the simulation results are as follows :

```
\ | /
- RT -     Thread Operating System
 / | \     3.1.0 build Aug 24 2018
 2006 - 2018 Copyright by rt-thread team
msh >mutex_sample
msh >mutex protect ,number1 = number2 is 1
mutex protect ,number1 = number2 is 2
mutex protect ,number1 = number2 is 3
mutex protect ,number1 = number2 is 4
...
mutex protect ,number1 = number2 is 48
mutex protect ,number1 = number2 is 49
```

Threads use mutexes to protect the operation on the two numbers, keeping the number values consistent.

Another example of a mutex is shown in the following code. This example creates three dynamic threads to check if the priority level of the thread holding the mutex is adjusted to the highest priority level among the waiting threads.

Prevent priority inversion routine

```
#include <rtthread.h>

/* Pointer to the thread control block */
static rt_thread_t tid1 = RT_NULL;
static rt_thread_t tid2 = RT_NULL;
static rt_thread_t tid3 = RT_NULL;
static rt_mutex_t mutex = RT_NULL;


#define THREAD_PRIORITY       10
#define THREAD_STACK_SIZE     512
#define THREAD_TIMESLICE    5

/* Thread 1 Entry */
static void thread1_entry(void *parameter)
{
    /* Let the low priority thread run first */
    rt_thread_mdelay(100);

    /* At this point, thread3 holds the mutex and thread2 is waiting to hold the mutex */

    /* Check the priority level of thread2 and thread3 */
    if (tid2->current_priority != tid3->current_priority)
    {
        /* The priority is different, the test fails */
        rt_kprintf("the priority of thread2 is: %d\n", tid2->current_priority);
        rt_kprintf("the priority of thread3 is: %d\n", tid3->current_priority);
        rt_kprintf("test failed.\n");
        return;
    }
    else
    {
        rt_kprintf("the priority of thread2 is: %d\n", tid2->current_priority);
        rt_kprintf("the priority of thread3 is: %d\n", tid3->current_priority);
        rt_kprintf("test OK.\n");
    }
}

/* Thread 2 Entry */
static void thread2_entry(void *parameter)
{
    rt_err_t result;

    rt_kprintf("the priority of thread2 is: %d\n", tid2->current_priority);

    /* Let the low-priority thread run first */
    rt_thread_mdelay(50);

    /*
     * Trying to hold a mutex lock. At this point, thread 3 has the mutex lock, so the priority level of
      thread 3 should be raised
     * to the same level of priority as thread 2
     */
    result = rt_mutex_take(mutex, RT_WAITING_FOREVER);

    if (result == RT_EOK)
    {
```

```
        /* Release mutex lock */
        rt_mutex_release(mutex);
    }
}

/* Thread 3 Entry */
static void thread3_entry(void *parameter)
{
    rt_tick_t tick;
    rt_err_t result;

    rt_kprintf("the priority of thread3 is: %d\n", tid3->current_priority);

    result = rt_mutex_take(mutex, RT_WAITING_FOREVER);
    if (result != RT_EOK)
    {
        rt_kprintf("thread3 take a mutex, failed.\n");
    }

    /* Operate a long cycle, 500ms */
    tick = rt_tick_get();
    while (rt_tick_get() - tick < (RT_TICK_PER_SECOND / 2)) ;

    rt_mutex_release(mutex);
}

int pri_inversion(void)
{
    /* Created a mutex lock */
    mutex = rt_mutex_create("mutex", RT_IPC_FLAG_FIFO);
    if (mutex == RT_NULL)
    {
        rt_kprintf("create dynamic mutex failed.\n");
        return -1;
    }

    /* Create thread 1*/
    tid1 = rt_thread_create("thread1",
                            thread1_entry,
                            RT_NULL,
                            THREAD_STACK_SIZE,
                            THREAD_PRIORITY - 1, THREAD_TIMESLICE);
    if (tid1 != RT_NULL)
         rt_thread_startup(tid1);

    /* Create thread 2 */
    tid2 = rt_thread_create("thread2",
                            thread2_entry,
                            RT_NULL,
                            THREAD_STACK_SIZE,
                            THREAD_PRIORITY, THREAD_TIMESLICE);
    if (tid2 != RT_NULL)
        rt_thread_startup(tid2);

    /* Create thread 3 */
    tid3 = rt_thread_create("thread3",
                            thread3_entry,
                            RT_NULL,
                            THREAD_STACK_SIZE,
                            THREAD_PRIORITY + 1, THREAD_TIMESLICE);
    if (tid3 != RT_NULL)
        rt_thread_startup(tid3);

    return 0;
}

/* Export to the msh command list */
MSH_CMD_EXPORT(pri_inversion, prio_inversion sample);
```

The simulation results are as follows:

```
\ | /
- RT -     Thread Operating System
 / | \     3.1.0 build Aug 27 2018
 2006 - 2018 Copyright by rt-thread team
msh >pri_inversion
the priority of thread2 is: 10
the priority of thread3 is: 11
the priority of thread2 is: 10
the priority of thread3 is: 10
test OK.
```

The routine demonstrates how to use the mutex. Thread 3 holds the mutex first, and then thread 2 tries to hold the mutex, at which point thread 3's priority is raised to the same level as thread 2.

>It is important to remember that mutexes cannot be used in interrupt service routines.

1.3.5.2.5 Occasions to Use Mutex

The use of a mutex is relatively simple because it is a type of semaphore and it exists in the form of a lock. At the time of initialization, the mutex is always unlocked, and when it is held by the thread, it immediately becomes locked. Mutex is more suitable for:

(1) When a thread holds a mutex multiple times. This can avoid the problem of deadlock caused by multiple recursive holdings of the same thread.

(2) A situation in which priority inversion may occur due to multi-thread synchronization.

1.3.5.3 Event

Event set is also one of the mechanisms for synchronization between threads. An event set can contain multiple events. Event set can be used to complete one-to-many, many-to-many thread synchronization. Let's take taking bus as an example to illustrate event. There may be the following situations when waiting for a bus at a bus stop:

P1 is taking a bus to a certain place, only one type of bus can reach the destination. P1 can leave for the destination once that bus arrives.

P1 is taking a bus to a certain place, 3 types of buses can reach the destination. P1 can leave for the destination once any 1 of the 3 types of bus arrives.

P1 is traveling with P2 to a certain place together, P1 can't leave for the destination unless two conditions are met. These two conditions are “P2 arrives at the bus stop” and “bus arrives at the bus stop”.

Here, P1 leaving for a certain place can be regarded as a thread, and “bus arrives at the bus stop” and “P2 arrives at the bus stop” are regarded as the occurrence of events. Situation is a specific event to wakes up the thread; situation is any single event to wake up the thread; situation is when multiple events must occur simultaneously to wake up the thread.

1.3.5.3.1 Event Set Working Mechanism

The event set is mainly used for synchronization between threads. Unlike the semaphore, it can achieve one-to-many, many-to-many synchronization. That is, the relationship between a thread and multiple events can be set as follows: any one of the events wakes up the thread, or several events wake up the thread for subsequent processing; likewise, the event can be multiple threads to synchronize multiple events. This collection of multiple events can be represented by a 32-bit unsigned integer variable, each bit of the variable representing an event, and the thread associates one or more events by "logical AND" or "logical OR" to form event combination. The "logical OR" of an event is also called independent synchronization, which means that the thread is synchronized with one of the events; the event "logical AND" is also called associative synchronization, which means that the thread is synchronized with several events.

The event set defined by RT-Thread has the following characteristics:

1) Events are related to threads only, and events are independent of each other: each thread can have 32 event flags, recorded with a 32-bit unsigned integer, each bit representing an event;

2) Event is only used for synchronization and does not provide data transfer functionality;

3) Events are not queuing, that is, sending the same event to the thread multiple times (if the thread has not had time read it), the effect is equivalent to sending only once.

In RT-Thread, each thread has an event information tag with three attributes. They are RT_EVENT_FLAG_AND (logical AND), RT_EVENT_FLAG_OR (logical OR), and RT_EVENT_FLAG_CLEAR (clear flag). When the thread

waits for event synchronization, it can determine whether the currently received event satisfies the synchronization condition by 32 event flags and this event information flag.

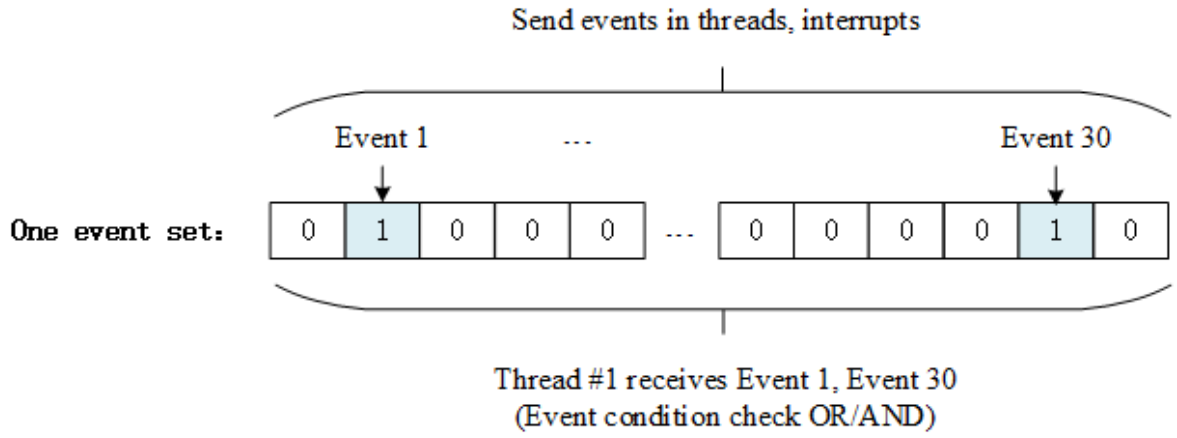


Figure 1.43 Event Set Work Diagram

As shown in the figure above, the first and 30th bits of the event flag of thread #1 are set. If the event information flag is set to logical AND, it means that thread #1 will be triggered to wake up only after both event 1 and event 30 occur. If the event information flag is set to logical OR, the occurrence of either event 1 or event 30 will trigger to wake up thread #1. If the message flag also sets the clear flag bit, this means event 1 and event 30 will be automatically cleared to zero when thread #1 wakes up, otherwise the event flag will still be present (set to 1).

1.3.5.3.2 Event Set Control Block

In RT-Thread, event set control block is a data structure used by the operating system to manage events, represented by the structure struct rt_event. Another C expression, rt_event_t, represents the handle of the event set, and the implementation in C language is a pointer to the event set control block. See the following code for a detailed definition of the event set control block structure:

```
struct rt_event
{
    struct rt_ipc_object parent;    /* Inherited from the ipc_object class */

    /* The set of events, each bit represents 1 event, the value of the bit can mark whether an event occurs
      */
    rt_uint32_t set;
};
/* rt_event_t is the pointer type poniting to the event structure */
typedef struct rt_event* rt_event_t;
```

rt_event object is derived from rt_ipc_object and is managed by the IPC container.

1.3.5.3.3 Management of Event Sets

Event set control block contains important parameters related to the event set and plays an important role in the implementation of the event set function. The event set related interfaces are as shown in the following figure. The operations on an event set include: create/initiate event sets, send events, receive events, and delete/detach event sets.

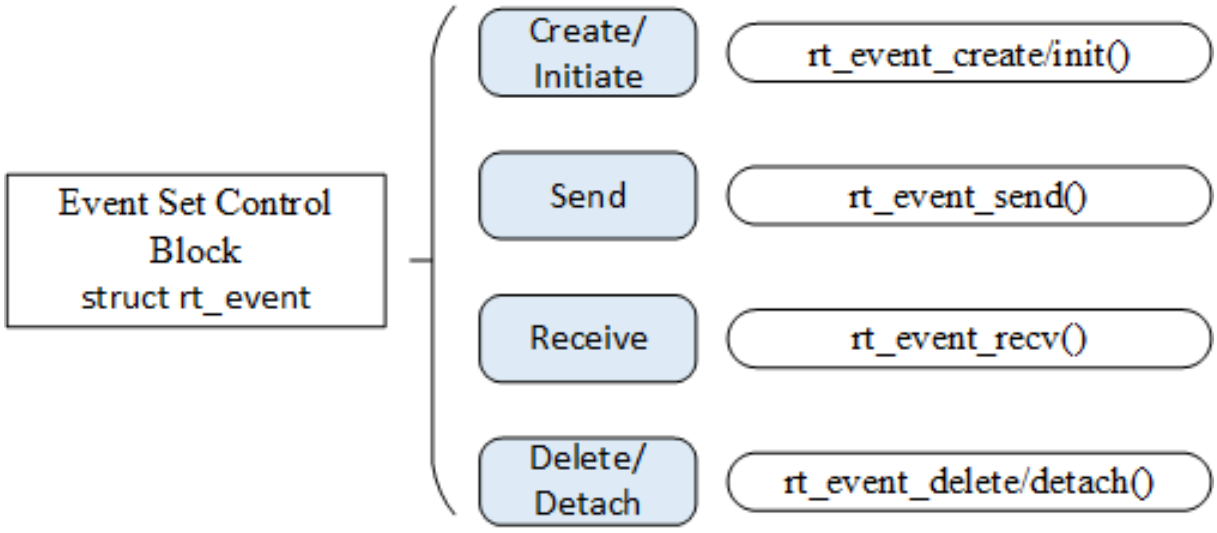


Figure 1.44 Event Related Interface

1.3.5.3.3.1 Create and Delete Event Set

When creating an event set, the kernel first creates an event set control block, and then performs basic initialization on the event set control block. The event set is created using the following function interface:

```
rt_event_t rt_event_create(const char* name, rt_uint8_t flag);
```

When the function interface is called, the system allocates the event set object from the object manager, initializes the object, and then initializes the parent class IPC object. The following table describes the input parameters and return values for this function:

Input parameters and return values for rt_event_create()

Parameters	Description
name	Name of the event set.
flag	The flag of the event set, which can take the following values: RT_IPC_FLAG_FIFO or RT_IPC_FLAG_PRIO
Return	——
RT_NULL	Creation failed.
Handle of the event object	Creation successful

When the system no longer uses the event set object created by rt_event_create(), the system resource is released by deleting the event set object control block. To delete an event set you can use the following function interface:

```
rt_err_t rt_event_delete(rt_event_t event);
```

When you call rt_event_delete function to delete an event set object, you should ensure that the event set is no longer used. All threads that are suspended on the event set will be awaken before the deletion (the return value of the thread is -RT_ERROR), and then the memory block occupied by the event set object is released. The following table describes the input parameters and return values for this function:

Input parameters and return values of rt_event_delete()

Parameters	Description
event	The handle of the event set object
Return	——
RT_EOK	Success

1.3.5.3.3.2 Initialize and Detach Event Set

The memory of a static event set object is allocated by the compiler during system compilation, and is usually placed in a read-write data segment or an uninitialized data segment. Before using a static event set object, you need to initialize it first. The initialization event set uses the following function interface:

```
rt_err_t rt_event_init(rt_event_t event, const char* name, rt_uint8_t flag);
```

When the interface is called, you need to specify the handle of the static event set object (that is, the pointer pointing to the event set control block), and then the system will initialize the event set object and add it to the system object container for management. The following table describes the input parameters and return values for this function:

Input parameters and return values of rt_event_init()

Parameters	Description
event	The handle of the event set object
name	The name of the event set
flag	The flag of the event set, which can take the following values: RT_IPC_FLAG_FIFO or RT_IPC_FLAG_PRIO
Return	——
RT_EOK	Success

When the system no longer uses the event set object initialized by rt_event_init(), the system resources are released by detaching the event set object control block. Detaching event set means to detach the event set object from the kernel object manager. To detach an event set, use the following function interface:

```
rt_err_t rt_event_detach(rt_event_t event);
```

When the user calls this function, the system first wakes up all the threads suspended on the event set wait queue (the return value of the thread is -RT_ERROR), and then detaches the event set from the kernel object manager. The following table describes the input parameters and return values for this function:

Input parameters and return values for rt_event_detach()

Parameters	Description
event	The handle of the event set object
Return	——
RT_EOK	Success

1.3.5.3.3.3 Send Event

The send event function can send one or more events in the event set as follows:

```
rt_err_t rt_event_send(rt_event_t event, rt_uint32_t set);
```

When using the function interface, the event flag value of the event set object is set by the event flag specified by the parameter set, and then the waiting thread linked list transversely waiting for the event set objects is used to determine whether there is a thread has the event activation requirement that matches the current event flag value. If there is, wake up the thread. The following table describes the input parameters and return values for this function:

Input parameters and return values of rt_event_send()

Parameters	Description
event	The handle of the event set object
set	The flag value of one or more events sent
Return	——
RT_EOK	Success

1.3.5.3.3.4 Receive Event

The kernel uses a 32-bit unsigned integer to identify the event set, each bit represents an event, so an event set object can wait to receive 32 events at the same time, and the kernel can decide how to activate the thread by specifying the parameter "logical AND" or "logical OR". Using the "logical AND" parameter indicates that the thread is only activated when all waiting events occur, and using the "logical OR" parameter means that the thread is activated as soon as one waiting event occurs. To receive events, use the following function interface:

```
rt_err_t rt_event_recv(rt_event_t event,
                       rt_uint32_t set,
                       rt_uint8_t option,
                       rt_int32_t timeout,
                       rt_uint32_t* recved);
```

If it has already occurred, it determines whether to reset the corresponding flag of the event according to whether RT_EVENT_FLAG_CLEAR is set on the parameter option. If it has already occurred, it determines whether to reset the corresponding flag of the event according to whether RT_EVENT_FLAG_CLEAR is set on the parameter option, then return (where the recved parameter returns to the received event); if it has not occurred, fill the waiting set and option parameters into the structure of the thread itself, then suspend the thread on this event until its waiting event satisfies the condition or until the specified timeout elapses. If the timeout is set to zero, it means that when the event to be accepted by the thread does not meet the requirements, it does not wait, but returns directly-RT_ETIMEOUT. The following table describes the input parameters and return values for this function:

Input parameters and return values of rt_event_recv()

Parameters	Description
event	The handle of the event set object
set	Receive events of interest to the thread
option	Receive options
timeout	Timeout
recved	Point to the received event
Return	——
RT_EOK	Successful
-RT_ETIMEOUT	Timeout
-RT_ERROR	Error

The value of option can be:

```
/* Select AND or OR to receive events */
RT_EVENT_FLAG_OR
RT_EVENT_FLAG_AND

/* Choose to clear reset event flag */
RT_EVENT_FLAG_CLEAR
```

1.3.5.3.4 Event Set Application Sample

This is the application routine for the event set, which initializes an event set, two threads. One thread waits for an event of interest to it, and another thread sends an event, as shown in code listing 6-5:

Event set usage routine

```
#include <rtthread.h>

#define THREAD_PRIORITY      9
#define THREAD_TIMESLICE     5

#define EVENT_FLAG3 (1 << 3)
#define EVENT_FLAG5 (1 << 5)

/* Event control block */
static struct rt_event event;

ALIGN(RT_ALIGN_SIZE)
static char thread1_stack[1024];
static struct rt_thread thread1;

/* Thread 1 entry function*/
static void thread1_recv_event(void *param)
{
    rt_uint32_t e;

    /* The first time the event is received, either event 3 or event 5 can trigger thread 1, clearing the
      event flag after receiving */
    if (rt_event_recv(&event, (EVENT_FLAG3 | EVENT_FLAG5),
                      RT_EVENT_FLAG_OR | RT_EVENT_FLAG_CLEAR,
                      RT_WAITING_FOREVER, &e) == RT_EOK)
    {
        rt_kprintf("thread1: OR recv event 0x%x\n", e);
    }

    rt_kprintf("thread1: delay 1s to prepare the second event\n");
    rt_thread_mdelay(1000);

    /* The second time the event is received, both event 3 and event 5 can trigger thread 1, clearing the
      event flag after receiving */
    if (rt_event_recv(&event, (EVENT_FLAG3 | EVENT_FLAG5),
                      RT_EVENT_FLAG_AND | RT_EVENT_FLAG_CLEAR,
                      RT_WAITING_FOREVER, &e) == RT_EOK)
    {
        rt_kprintf("thread1: AND recv event 0x%x\n", e);
    }
    rt_kprintf("thread1 leave.\n");
}


ALIGN(RT_ALIGN_SIZE)
static char thread2_stack[1024];
static struct rt_thread thread2;

/* Thread 2 Entry */
static void thread2_send_event(void *param)
{
    rt_kprintf("thread2: send event3\n");
    rt_event_send(&event, EVENT_FLAG3);
    rt_thread_mdelay(200);

    rt_kprintf("thread2: send event5\n");
    rt_event_send(&event, EVENT_FLAG5);
    rt_thread_mdelay(200);

    rt_kprintf("thread2: send event3\n");
    rt_event_send(&event, EVENT_FLAG3);
    rt_kprintf("thread2 leave.\n");
}

int event_sample(void)
{
    rt_err_t result;

    /* Initialize event object */
    result = rt_event_init(&event, "event", RT_IPC_FLAG_FIFO);
    if (result != RT_EOK)
    {
        rt_kprintf("init event failed.\n");
        return -1;
    }

    rt_thread_init(&thread1,
                   "thread1",
```

```
                    thread1_recv_event,
                    RT_NULL,
                    &thread1_stack[0],
                    sizeof(thread1_stack),
                    THREAD_PRIORITY - 1, THREAD_TIMESLICE);
    rt_thread_startup(&thread1);

    rt_thread_init(&thread2,
                    "thread2",
                    thread2_send_event,
                    RT_NULL,
                    &thread2_stack[0],
                    sizeof(thread2_stack),
                    THREAD_PRIORITY, THREAD_TIMESLICE);
    rt_thread_startup(&thread2);

    return 0;
}

/* Export to the msh command list */
MSH_CMD_EXPORT(event_sample, event sample);
```

The simulation results are as follows:

```
 \ | /
- RT -     Thread Operating System
 / | \     3.1.0 build Aug 24 2018
 2006 - 2018 Copyright by rt-thread team
msh >event_sample
thread2: send event3
thread1: OR recv event 0x8
thread1: delay 1s to prepare the second event
msh >thread2: send event5
thread2: send event3
thread2 leave.
thread1: AND recv event 0x28
thread1 leave.
```

The routine demonstrates how to use the event set. Thread 1 receives events twice before and after, using the "logical OR" and "logical AND" respectively.

1.3.5.3.5 Occasions to Use Event Set

Event sets can be used in a variety of situations, and it can replace semaphores to some extent for inter-thread synchronization. A thread or interrupt service routine sends an event to the event set object, and the waiting thread is awaken and the corresponding event is processed. However, unlike semaphore, the event transmission operation is not cumulative until the event is cleared, and the release actions of semaphore are cumulative. Another feature of the event is that the receiving thread can wait for multiple events, meaning multiple events correspond to one thread or multiple threads. At the same time, according to thread waiting parameters, you can choose between a "logical OR" trigger or a "logical AND" trigger. This feature is not available for semaphores, etc. The semaphore can only recognize a single release action, and cannot wait for multiple types of release at the same time. The following figure shows the multi-event receiving diagram:

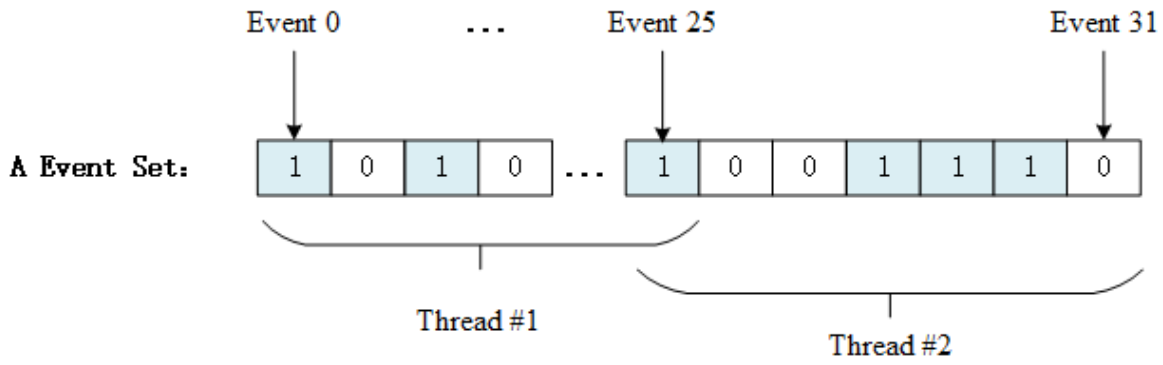


Figure 1.45 Multi-event Receiving Schematic

An event set contains 32 events, and a particular thread only waits for and receives events it is interested in. It can be a thread waiting for the arrival of multiple events (threads 1, 2 are waiting for multiple events, logical "and" or logical "or" can be used to trigger the thread in events), or multiple threads waiting for an event to arrive (event 25). When there are events of interest to them occur, the thread will be awaken and subsequent processing actions will be taken.

1.3.6 Inter-thread Communication

In the last chapter, we talked about inter-thread synchronization, concepts such as semaphores, mutexes, and event sets were mentioned. Following the last chapter, this chapter is going to explain inter-thread communication. In bare-metal programming, global variables are often used for communication between functions. For example, some functions may change the value of a global variable due to some operations. Another function reads the global variable and will perform corresponding actions to achieve communication and collaboration according to the global variable values it read. More tools are available in RT-Thread to help pass information between different threads. These tools are covered in more detail in this chapter. After reading this chapter, you will learn how to use mailboxes, message queues, and signals for communication between threads.

1.3.6.1 Mailbox

Mailbox service is a typical inter-thread communication method in real-time operating systems.For example, there are two threads, thread 1 detects the state of the button and sends it's state, and thread 2 reads the state of the button and turns on or off the LED according to the state of the button. Here, a mailbox can be used to communicate. Thread 1 sends the status of the button as an email to the mailbox. Thread 2 reads the message in the mailbox to get the button status and turn on or off the LED accordingly.

Thread 1 here can also be extended to multiple threads. For example, there are three threads, thread 1 detects and sends the button state, thread 2 detects and sends the ADC information, and thread 3 performs different operations according to the type of information received.

1.3.6.1.1 Mailbox Working Mechanism

The mailbox of the RT-Thread operating system is used for inter-thread communication, which is characterized by low overhead and high efficiency. Each message in the mailbox can only hold a fixed 4 bytes(for a 32-bit processing system, the pointer is 4 bytes in size, so an email can hold only one pointer). A typical mailbox is also called message exchange. As shown in the following figure, a thread or an interrupt service routine sends a 4-byte message to a mailbox, and one or more threads can receive and process the message from the mailbox.

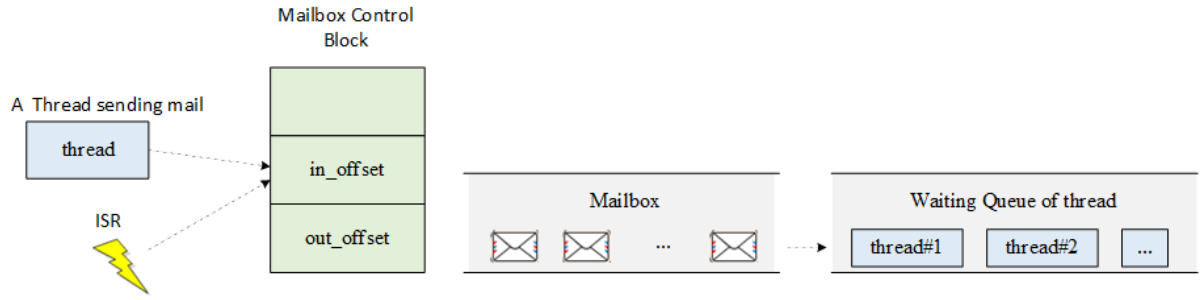


Figure 1.46 Mailbox Working Mechanism Diagram

The sending process of the non-blocking mode mails can be safely used in ISR. It is an effective way for the thread, the interrupt service, and the timer to send a message to the thread. In general, the receiving of the mails can be a blocked process, depending on whether there is a message in the mailbox and the timeout set when the message

was received. When there is no mail in the mailbox and the set timeout is not 0, the receive of the mails will become blocked. In such cases, the mails can only be received by threads.

When a thread sends a message to a mailbox, if the mailbox is not full, the message will be copied to the mailbox. If the mailbox is full, the thread sending the message can set a timeout, and choose to wait and suspend or return directly - RT_EFULL. If the thread sending the message chooses to suspend and wait, then when the mails in the mailbox are received and space is left open again, the thread sending the message will be awaken and will continue to send.

When a thread receives a message from a mailbox, if the mailbox is empty, the thread receiving the message can choose whether to set a timeout or wait and suspend until a new message is received to be awaken. When the set timeout is up and the mailbox still hasn't received the message, the thread that chose to wait till timeout will be awaken and return -RT_ETIMEOUT. If there are messages in the mailbox, then the thread receiving the message will copy the 4-byte message in the mailbox to the receiving cache.

1.3.6.1.2 Mailbox Control Block

In RT-Thread, the mailbox control block is a data structure used by the operating system to manage mailboxes, represented by the structure `struct rt_mailbox`. Another C expression, `rt_mailbox_t`, represents the handle of the mailbox, and the implementation in C language is a pointer to the mailbox control block. See the following code for a detailed definition of the mailbox control block structure:

```
struct rt_mailbox
{
    struct rt_ipc_object parent;

    rt_uint32_t* msg_pool;            /* the start address of the mailbox buffer */
    rt_uint16_t size;                 /* the size of the mailbox buffer     */

    rt_uint16_t entry;                 /* the number of messages in the mailbox     */
    rt_uint16_t in_offset, out_offset;    /* the entry and exit pointer of the mailbox buffer   */
    rt_list_t suspend_sender_thread;      /* send the suspend and wait queue of the thread */
};
typedef struct rt_mailbox* rt_mailbox_t;
```

The `rt_mailbox` object is derived from `rt_ipc_object` and is managed by the IPC container.

1.3.6.1.3 Management of Mailbox

The mailbox control block is a structure that contains important parameters related to mailbox and it plays an important role in the function implementation of the mailbox. The relevant interfaces of the mailbox are as shown in the following figure. The operation on a mailbox includes: create/initiate a mailbox, send a mail, receive a mail, and delete/detach a mailbox.

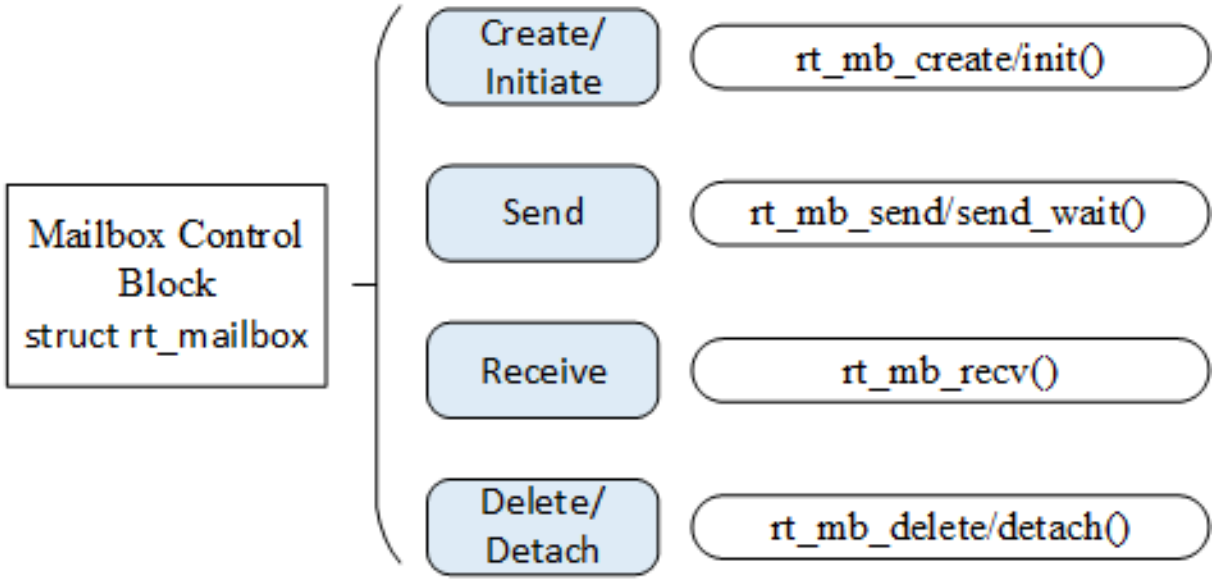


Figure 1.47 Mailbox Related Interface

1.3.6.1.3.1 Create and Delete Mailbox

To dynamically create a mailbox object, call the following function interface:

```
rt_mailbox_t rt_mb_create (const char* name, rt_size_t size, rt_uint8_t flag);
```

When a mailbox object is created, a mailbox object is first allocated from the object manager, and then a memory space is dynamically allocated to the mailbox for storing the mail. The size of the memory is equal to the product of the message size (4 bytes) and the mailbox capacity. Then initialize the number of incoming messages and the offset of the outgoing message in the mailbox. The following table describes the input parameters and return values for this function:

Input parameters and return values of rt_mb_create()

Parameters	Description
name	The name of the mailbox
size	Mailbox capacity
flag	The mailbox flag, which can take the following values: RT_IPC_FLAG_FIFO or RT_IPC_FLAG_PRIO
Return	——
RT_NULL	Creation failed
The handle of the mailbox object	Creation successful

When a mailbox created with rt_mb_create() is no longer used, it should be deleted to release the corresponding system resources. Once the operation is completed, the mailbox will be permanently deleted. The function interface for deleting a mailbox is as follows:

```
rt_err_t rt_mb_delete (rt_mailbox_t mb);
```

When deleting a mailbox, if a thread is suspended on the mailbox object, the kernel first wakes up all threads suspended on the mailbox (the thread return value is -RT_ERROR), then releases the memory used by the mailbox, and finally deletes the mailbox object. The following table describes the input parameters and return values for this function:

Input parameters and return values of rt_mb_delete()

Parameters	Description
mb	The handle of the mailbox object
Return	——
RT_EOK	Successful

1.3.6.1.3.2 Initialize and Detach Mailbox

Initializing a mailbox is similar to creating a mailbox, except that the mailbox initialized is for static mailbox objects. Different from creating a mailbox, the memory of a static mailbox object is allocated by the compiler during system compilation which is usually placed in a read-write data segment or an uninitialized data segment. The rest of the initialization is the same as the creation of a mailbox. The function interface is as follows:

```
rt_err_t rt_mb_init(rt_mailbox_t mb,
                    const char* name,
                    void* msgpool,
                    rt_size_t size,
                    rt_uint8_t flag)
```

When the mailbox is initialized, the function interface needs to obtain the mailbox object control block that the user has applied for, the pointer of the buffer, the mailbox name and mailbox capacity (the number of messages that can be stored). The following table describes the input parameters and return values for this function:

Input parameters and return values of rt_mb_init()

Parameters	Description
mb	The handle of the mailbox object
name	Mailbox name
msgpool	Buffer pointer
size	Mailbox capacity
flag	The mailbox flag, which can take the following values: RT_IPC_FLAG_FIFO or RT_IPC_FLAG↩ _PRIO
Return	——
RT_EOK	Successful

The size parameter here specifies the capacity of the mailbox, if the number of bytes in the buffer pointed to by msgpool is N, then the mailbox capacity should be N/4.

Detaching the mailbox means to detach the statically initialized mailbox objects from the kernel object manager. Use the following interface to detach the mailbox:

```
rt_err_t rt_mb_detach(rt_mailbox_t mb);
```

After using this function interface, the kernel wakes up all the threads suspended on the mailbox (the threads return -RT_ERROR), and then detaches the mailbox objects from the kernel object manager. The following table describes the input parameters and return values for this function:

Input parameters and return values for rt_mb_detach()

Parameters	Description
mb	The handle of the mailbox object

Parameters	Description
Return	——
RT_EOK	Successful

1.3.6.1.3.3 Send Mail

The thread or ISR can send mail to other threads through the mailbox. The function interface of sending mails is as follows:

```
rt_err_t rt_mb_send (rt_mailbox_t mb, rt_uint32_t value);
```

The message sent can be any data 32-bit formatted, an integer value or a pointer pointing to the buffer. When the mailbox is fully filled with mails, the thread or ISR that sent the mail will receive a return value of -RT_EFULL. The following table describes the input parameters and return values for this function:

Input parameters and return values of rt_mb_send()

Parameters	Description
mb	The handle of the mailbox object
value	Content of email
Return	——
RT_EOK	Sent successfully
-RT_EFULL	The mailbox is filled

1.3.6.1.3.4 Send Mails with Waiting

Users can also send mails to specified mailbox through the following function interface:

```
rt_err_t rt_mb_send_wait (rt_mailbox_t mb,
                         rt_uint32_t value,
                         rt_int32_t timeout);
```

The difference between rt_mb_send_wait() and rt_mb_send() is that there is waiting time. If the mailbox is full, the thread sending the message will wait for the mailbox to release space as mails are received according to the set timeout parameter. If by the set timeout, there is still no available space, then the thread sending the message will wake up and return an error code. The following table describes the input parameters and return values for this function:

Input parameters and return values of rt_mb_send_wait()

Parameters	Description
mb	The handle of the mailbox object
value	Mail content
timeout	Timeout
Return	——
RT_EOK	Sent successfully
-RT_ETIMEOUT	Timeout
-RT_ERROR	Failed, return error

1.3.6.1.3.5 Receive Mails

Only when there is a mail in the mailbox, the recipient can receive the mail immediately and return RT_EOK. Otherwise, the thread receiving the message will suspend on the waiting thread queue of the mailbox or return directly according to the set timeout. The receiving mail function interface is as follows:

```
rt_err_t rt_mb_recv (rt_mailbox_t mb, rt_uint32_t* value, rt_int32_t timeout);
```

When receiving a mail, the recipient needs to specify the mailbox handle, and specify the location to store the received message and the maximum timeout that it can wait. If a timeout is set at the time of receiving, -RT_↩ ETIMEOUT will be returned when the message has not been received within the specified time. The following table describes the input parameters and return values for this function:

Input parameters and return values of rt_mb_recv()

Parameters	Description
mb	The handle of the mailbox object
value	Mail content
timeout	Timeout
Return	——
RT_EOK	Sent successfully
-RT_ETIMEOUT	Timeout
-RT_ERROR	Failed, return error

1.3.6.1.4 Mail Usage Sample

This is a mailbox application routine that initializes 2 static threads, 1 static mailbox object, one of the threads sends mail to the mailbox, and one thread receives mail from the mailbox. As shown in the following code:

Mailbox usage routine

```
#include <rtthread.h>

#define THREAD_PRIORITY      10
#define THREAD_TIMESLICE     5

/* Mailbox control block */
static struct rt_mailbox mb;
/* Memory pool for mails storage */
static char mb_pool[128];

static char mb_str1[] = "I'm a mail!";
static char mb_str2[] = "this is another mail!";
static char mb_str3[] = "over";

ALIGN(RT_ALIGN_SIZE)
static char thread1_stack[1024];
static struct rt_thread thread1;

/* Thread 1 entry */
static void thread1_entry(void *parameter)
{
    char *str;

    while (1)
    {
        rt_kprintf("thread1: try to recv a mail\n");

        /* Receive mail from the mailbox */
        if (rt_mb_recv(&mb, (rt_uint32_t *)&str, RT_WAITING_FOREVER) == RT_EOK)
        {
            rt_kprintf("thread1: get a mail from mailbox, the content:%s\n", str);
            if(!strcmp(str, mb_str3))
                break;

            /* Delay 100ms */
```

```
            rt_thread_mdelay(100);
        }
    }
    /* Executing the mailbox object detachment */
    rt_mb_detach(&mb);
}

ALIGN(RT_ALIGN_SIZE)
static char thread2_stack[1024];
static struct rt_thread thread2;

/* Thread 2 entry*/
static void thread2_entry(void *parameter)
{
    rt_uint8_t count;

    count = 0;
    while (count < 10)
    {
        count ++;
        if (count & 0x1)
        {
            /* Send the mb_str1 address to the mailbox */
            rt_mb_send(&mb, (rt_uint32_t)&mb_str1);
        }
        else
        {
            /* Send the mb_str2 address to the mailbox */
            rt_mb_send(&mb, (rt_uint32_t)&mb_str2);
        }

        /* Delay 200ms */
        rt_thread_mdelay(200);
    }

    /* Send mail to inform thread 1 that thread 2 has finished running */
    rt_mb_send(&mb, (rt_uint32_t)&mb_str3);
}

int mailbox_sample(void)
{
    rt_err_t result;

    /* Initialize a mailbox */
    result = rt_mb_init(&mb,
                        "mbt",                      /* Name is mbt */
                        &mb_pool[0],                /* The memory pool used by the mailbox is mb_pool */
                        sizeof(mb_pool) / 4,        /* The number of messages in the mailbox because a
      message occupies 4 bytes */
                        RT_IPC_FLAG_FIFO);          /* Thread waiting in FIFO approach */
    if (result != RT_EOK)
    {
        rt_kprintf("init mailbox failed.\n");
        return -1;
    }

    rt_thread_init(&thread1,
                   "thread1",
                   thread1_entry,
                   RT_NULL,
                   &thread1_stack[0],
                   sizeof(thread1_stack),
                   THREAD_PRIORITY, THREAD_TIMESLICE);
    rt_thread_startup(&thread1);

    rt_thread_init(&thread2,
                   "thread2",
                   thread2_entry,
                   RT_NULL,
                   &thread2_stack[0],
                   sizeof(thread2_stack),
                   THREAD_PRIORITY, THREAD_TIMESLICE);
    rt_thread_startup(&thread2);
    return 0;
}

/* Export to the msh command list */
MSH_CMD_EXPORT(mailbox_sample, mailbox sample);
```

The simulation results are as follows:

```
 \ | /
- RT -     Thread Operating System
 / | \     3.1.0 build Aug 27 2018
 2006 - 2018 Copyright by rt-thread team
msh >mailbox_sample
thread1: try to recv a mail
```

```
thread1: get a mail from mailbox, the content:I'm a mail!
msh >thread1: try to recv a mail
thread1: get a mail from mailbox, the content:this is another mail!
...
thread1: try to recv a mail
thread1: get a mail from mailbox, the content:this is another mail!
thread1: try to recv a mail
thread1: get a mail from mailbox, the content:over
```

The routine demonstrates how to use the mailbox. Thread 2 sends the mails, for a total of 11 times; thread 1 receives the mails, for a total of 11 mails, prints the contents of the mail, and determines end.

1.3.6.1.5 Occasions to Use Mails

Mailbox is a simple inter-thread messaging method, which is characterized by low overhead and high efficiency. In the implementation of the RT-Thread operating system, a 4-byte message can be delivered at a time, and the mailbox has certain storage capabilities, which can cache a certain number of messages (the number of messages is determined by the capacity specified when creating and initializing the mailbox). The maximum length of a message in a mailbox is 4 bytes, so the mailbox can be used for messages less than 4 bytes. Since on the 32 system, 4 bytes can be placed right on a pointer, when a larger message needs to be transferred between threads, a pointer pointing to a buffer can be sent as an mail to the mailbox, which means the mailbox can also delivery a pointer. For example,

```
struct msg
{
    rt_uint8_t *data_ptr;
    rt_uint32_t data_size;
};
```

For such a message structure, it contains a pointer pointing to data `data_ptr` and a variable `data_size` of the length of the data block. When a thread needs to send this message to another thread, the following operations can be used:

```
struct msg* msg_ptr;

msg_ptr = (struct msg*)rt_malloc(sizeof(struct msg));
msg_ptr->data_ptr = ...;  /* Point to the corresponding data block address */
msg_ptr->data_size = len; /* Length of data block */
/* Send this message pointer to the mb mailbox */
rt_mb_send(mb, (rt_uint32_t)msg_ptr);
```

When receiving the thread, because the pointer is what's being received, and msg_ptr is a newly allocated memory block, so after the thread receiving the message finishes processing, the corresponding memory block needs to be released:

```
struct msg* msg_ptr;
if (rt_mb_recv(mb, (rt_uint32_t*)&msg_ptr) == RT_EOK)
{
    /* After the thread receiving the message finishes processing, the corresponding memory block needs to
      be released: */
    rt_free(msg_ptr);
}
```

1.3.6.2 Message Queue

Message Queuing is another commonly used inter-thread communication method, which is an extension of the mailbox. Can be used in a variety of occasions, like message exchange between threads, use the serial port to receive variable length data, etc.

1.3.6.2.1 Message Queue Working Mechanism

Message queue can receive messages with unfixed length from threads or ISR and cache messages in their own memory space. Other threads can also read the corresponding message from the message queue, and when the message queue is empty, the thread reading the messages can be suspended. When a new message arrives, the suspended thread will be awaken to receive and process the message. Message queue is an asynchronous way of communication.

As shown in the following figure, a thread or interrupt service routine can put one or more messages into the message queue. Similarly, one or more threads can also get messages from the message queue. When multiple messages are sent to the message queue, the message that first enters the message queue is first passed to the thread. That is, the thread first gets the message that first enters the message queue, in other words, it follows the first in first out principle (FIFO).

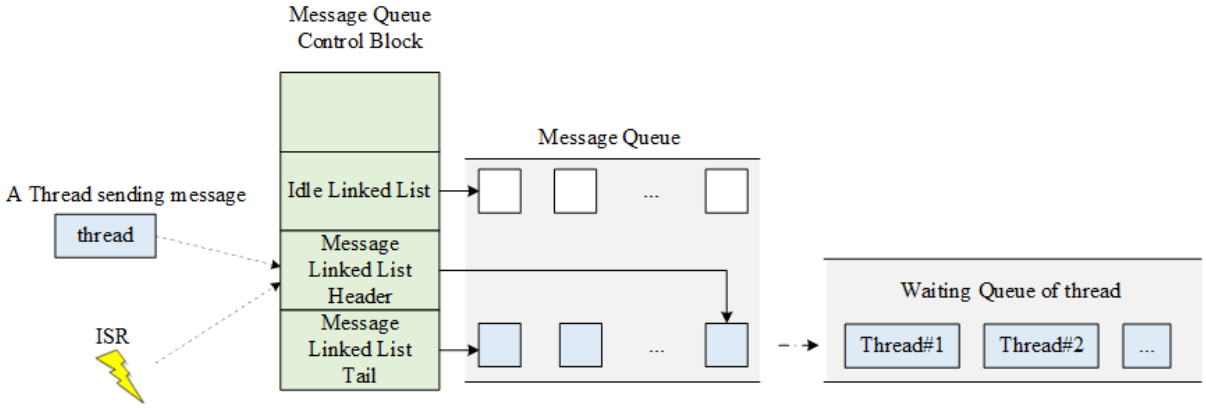


Figure 1.48 Message Queue Working Mechanism Diagram

The message queue object of the RT-Thread operating system consists of multiple elements. When a message queue is created, it is assigned a message queue control block: name, memory buffer, message size, and queue length. At the same time, each message queue object contains multiple message boxes, and each message box can store one message. The first and last message boxes in the message queue are respectively called the message linked list header and the message linked list tail, corresponding to `msg_queue_head` and `msg_queue_tail` in the queue control block. Some message boxes may be empty, they form a linked list of idle message boxes via `msg_queue_free`. The total number of message boxes in all message queues is the length of the message queue, which can be specified when creating the message queue.

1.3.6.2.2 Message Queue Control Block

In RT-Thread, a message queue control block is a data structure used by the operating system to manage message queues, represented by the structure `struct rt_messagequeue`. Another C expression, `rt_mq_t`, represents the handle of the message queue. The implementation in C language is a pointer to the message queue control block. See the following code for a detailed definition of the message queue control block structure:

```
struct rt_messagequeue
{
    struct rt_ipc_object parent;

    void* msg_pool;          /* Pointer pointing to the buffer storing the messages */

    rt_uint16_t msg_size;    /* The length of each message */
    rt_uint16_t max_msgs;    /* Maximum number of messages that can be stored */

    rt_uint16_t entry;        /* Number of messages already in the queue */

    void* msg_queue_head;    /* Message linked list header  */
    void* msg_queue_tail;    /* Message linked list tail  */
    void* msg_queue_free;    /* Idle message linked list */
};
typedef struct rt_messagequeue* rt_mq_t;
```

rt_messagequeue object is derived from rt_ipc_object and is managed by the IPC container.

1.3.6.2.3 Management of Message Queue

The message queue control block is a structure that contains important parameters related to the message queue and plays an important role in the implementation of functions of the message queue. The relevant interfaces of the message queue are shown in the figure below. Operations on a message queue include: create a message queue - send a message - receive a message - delete a message queue.

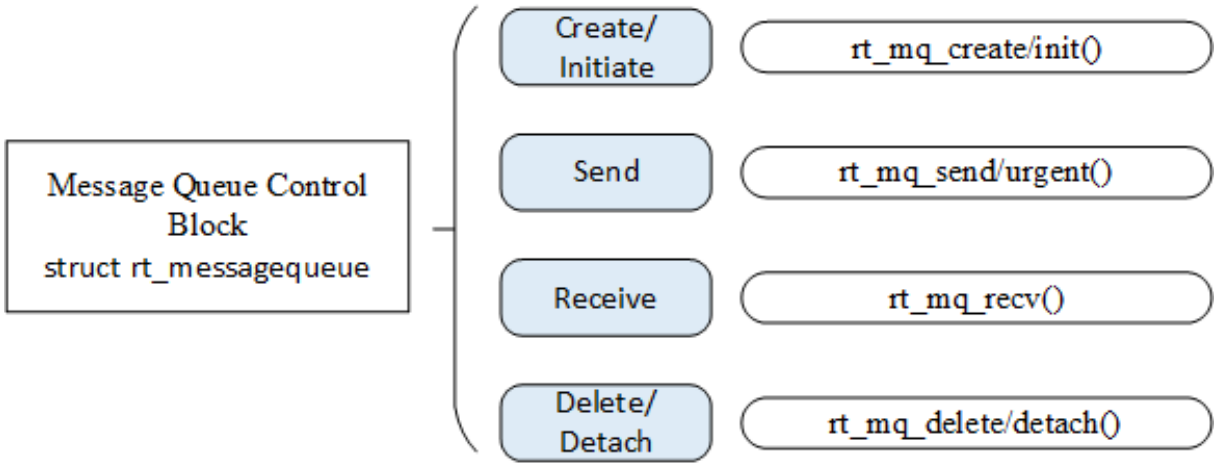


Figure 1.49 Message Queue Related Interfaces

1.3.6.2.3.1 Create and Delete Message Queue

The message queue should be created before it is used, or the existing static message queue objects should be initialized. The function interface for creating the message queue is as follows:

```
rt_mq_t rt_mq_create(const char* name, rt_size_t msg_size,
                     rt_size_t max_msgs, rt_uint8_t flag);
```

When creating a message queue, first allocate a message queue object from the object manager, then allocate a memory space to the message queue object forming an idle message linked list. The size of this memory = ↩ [message size + message header (for linked list connection) Size] X the number of messages in the message queue. Then initialize the message queue, at which point the message queue is empty. The following table describes the input parameters and return values for this function:

Input parameters and return values for rt_mq_create()

Parameters	Description
name	The name of the message queue
msg_size	The maximum length of a message in the message queue, in bytes
max_msgs	The number of messages in the message queue
flag	The waiting method took by the message queue, which can take the following values: RT_IPC_FLAG_FIFO or RT_IPC_FLAG_PRIO
Return	——
RT_EOK	Sent successfully
The handle of the message queue object	Successful
RT_NULL	Fail

When the message queue is no longer in use, it should be deleted to free up system resources, and once the operation is complete, the message queue will be permanently deleted. The function interface for deleting the message queue is as follows:

```
rt_err_t rt_mq_delete(rt_mq_t mq);
```

When deleting a message queue, if a thread is suspended on the message queue's waiting queue, the kernel wakes up all the threads suspended on the message waiting queue (the thread returns the value is -RT_ERROR), and then releases the memory used by the message queue. Finally, delete the message queue object. The following table describes the input parameters and return values for this function:

Input parameters and return values of rt_mq_delete()

Parameters	Description
mq	The handle of the message queue object
Return	——
RT_EOK	Successful

1.3.6.2.3.2 Initialize and Detach Message Queue

Initializing a static message queue object is similar to creating a message queue object, except that the memory of a static message queue object is allocated by the compiler during system compilation and is typically placed in a read data segment or an uninitialized data segment. Initialization is required before using such static message queue objects. The function interface for initializing a message queue object is as follows:

```
rt_err_t rt_mq_init(rt_mq_t mq, const char* name,
                    void *msgpool, rt_size_t msg_size,
                    rt_size_t pool_size, rt_uint8_t flag);
```

When the message queue is initialized, the interface requires the handle to the message queue object that the user has requested (that is, a pointer pointing to the message queue object control block), the message queue name, the message buffer pointer, the message size, and the message queue buffer size. As shown in the following figure, all messages after the message queue is initialized are suspended on the idle message list, and the message queue is empty. The following table describes the input parameters and return values for this function:

Input parameters and return values of rt_mq_init()

Parameters	Description
mq	The handle of the message queue object
name	The name of the message queue
msgpool	Pointer pointing to the buffer storing the messages
msg_size	The maximum length of a message in the message queue, in bytes
pool_size	The buffer size for storing messages
flag	The waiting method took by the message queue, which can take the following values: RT_IPC↩ _FLAG_FIFO or RT_IPC_FLAG_PRIO
Return	——
RT_EOK	Successful

Detaching the message queue will cause the message queue object to be detached from the kernel object manager. The following interface is used to detach from the message queue:

```
rt_err_t rt_mq_detach(rt_mq_t mq);
```

After using this function interface, the kernel wakes up all threads suspended on the message waiting queue object (the thread return value is -RT_ERROR) and then detaches the message queue object from the kernel object manager. The following table describes the input parameters and return values for this function:

Input parameters and return values for rt_mq_detach()

Parameters	Description
mq	The handle of the message queue object
Return	——
RT_EOK	Successful

1.3.6.2.3.3 Send Message

A thread or ISR can send a message to the message queue. When sending a message, the message queue object first takes an idle message block from the idle message list, then copies the content of the message sent by the thread or the interrupt service program to the message block, and then suspends the message block to the end of the message queue. The sender can successfully send a message if and only if there is an idle message block available on the idle message list; when there is no message block available on the idle message list, it means that the message queue is full, at this time, the thread or interrupt program that sent the message will receive an error code (-RT_EFULL). The function interface for sending messages is as follows:

```
rt_err_t rt_mq_send (rt_mq_t mq, void* buffer, rt_size_t size);
```

When sending a message, the sender specifies the object handle of the sent message queue (that is, a pointer pointing to the message queue control block) and specifies the content of the message being sent and the size of the message. As shown in the figure below, after sending a normal message, the first message on the idle message list is transferred to the end of the message queue. The following table describes the input parameters and return values for this function:

Input parameters and return values of rt_mq_send()

Parameter	Description
mq	The handle of the message queue object
buffer	Message content
size	Message size
Return	——
RT_EOK	Successful
-RT_EFULL	Message queue is full
-RT_ERROR	Failed, indicating that the length of the sent message is greater than the maximum length of the message in the message queue.

1.3.6.2.3.4 Send an Emergency Message

The process of sending an emergency message is almost the same as sending a message. The only difference is that when an emergency message is sent, the message block taken from the idle message list is not put in the end of the message queue, but the head of the queue. The recipient can receive the emergency message preferentially, so that the message can be processed in time. The function interface for sending an emergency message is as follows:

```
rt_err_t rt_mq_urgent(rt_mq_t mq, void* buffer, rt_size_t size);
```

The following table describes the input parameters and return values for this function:

Input parameters and return values of rt_mq_urgent()

Parameters	Description
mq	The handle of the message queue object
buffer	Message content
size	Message size
Return	——
RT_EOK	Successful
-RT_EFULL	Message queue is full
-RT_ERROR	Fail

1.3.6.2.3.5 Receive Message

Only when there is a message in the message queue, can the receiver receive message, otherwise the receiver will set according to the timeout, or suspend on the waiting queue of the message queue, or return directly. The function interface to receive message is as follows:

```
rt_err_t rt_mq_recv (rt_mq_t mq, void* buffer,
                     rt_size_t size, rt_int32_t timeout);
```

When receiving a message, the receiver needs to specify the message queue object handle that stores the message and specify a memory buffer into which the contents of the received message will be copied. In addition, the timeout when the message is not received in time needs to be set. As shown in the figure below, the message on the front of the message queue is transferred to the end of the idle message linked list after receiving a message. The following table describes the input parameters and return values for this function:

Input parameters and return values of rt_mq_recv()

Parameters	Description
mq	The handle of the message queue object
buffer	Message content
size	Message size
timeout	Specified timeout
Return	——
RT_EOK	Received successfully
-RT_ETIMEOUT	Timeout
-RT_ERROR	Fail, return error

1.3.6.2.4 Message Queue Application Example

This is a message queue application routine. Two static threads are initialized in the routine, one thread will receive messages from the message queue; and another thread will periodically send regular messages and emergency messages to the message queue, as shown in the following code:

Message queue usage routine

```
#include <rtthread.h>

/* Message queue control block */
static struct rt_messagequeue mq;
/* The memory pool used to place messages in the message queue */
```

```
static rt_uint8_t msg_pool[2048];

ALIGN(RT_ALIGN_SIZE)
static char thread1_stack[1024];
static struct rt_thread thread1;
/* Thread 1 entry function */
static void thread1_entry(void *parameter)
{
    char buf = 0;
    rt_uint8_t cnt = 0;

    while (1)
    {
        /* Receive messages from the message queue */
        if (rt_mq_recv(&mq, &buf, sizeof(buf), RT_WAITING_FOREVER) >= 0)
        {
            rt_kprintf("thread1: recv msg from msg queue, the content:%c\n", buf);
            if (cnt == 19)
            {
                break;
            }
        }
        /* Delay 50ms */
        cnt++;
        rt_thread_mdelay(50);
    }
    rt_kprintf("thread1: detach mq \n");
    rt_mq_detach(&mq);
}

ALIGN(RT_ALIGN_SIZE)
static char thread2_stack[1024];
static struct rt_thread thread2;
/* Thread 2 entry */
static void thread2_entry(void *parameter)
{
    int result;
    char buf = 'A';
    rt_uint8_t cnt = 0;

    while (1)
    {
        if (cnt == 8)
        {
            /* Send emergency message to the message queue */
            result = rt_mq_urgent(&mq, &buf, 1);
            if (result != RT_EOK)
            {
                rt_kprintf("rt_mq_urgent ERR\n");
            }
            else
            {
                rt_kprintf("thread2: send urgent message - %c\n", buf);
            }
        }
        else if (cnt>= 20)/* Exit after sending 20 messages */
        {
            rt_kprintf("message queue stop send, thread2 quit\n");
            break;
        }
        else
        {
            /* Send a message to the message queue */
            result = rt_mq_send(&mq, &buf, 1);
            if (result != RT_EOK)
            {
                rt_kprintf("rt_mq_send ERR\n");
            }

            rt_kprintf("thread2: send message - %c\n", buf);
        }
        buf++;
        cnt++;
        /* Delay 5ms */
        rt_thread_mdelay(5);
    }
}

/* Initialization of the message queue example */
int msgq_sample(void)
{
    rt_err_t result;

    /* Initialize the message queue */
    result = rt_mq_init(&mq,
                        "mqt",
                        &msg_pool[0],          /* Memory pool points to msg_pool */
```

```
                        1,                      /* The size of each message is 1 byte */
                        sizeof(msg_pool),       /* The size of the memory pool is the size of msg_pool */
                        RT_IPC_FLAG_FIFO);      /* If there are multiple threads waiting, assign messages in
      first come first get mode. */

    if (result != RT_EOK)
    {
        rt_kprintf("init message queue failed.\n");
        return -1;
    }

    rt_thread_init(&thread1,
                   "thread1",
                   thread1_entry,
                   RT_NULL,
                   &thread1_stack[0],
                   sizeof(thread1_stack), 25, 5);
    rt_thread_startup(&thread1);

    rt_thread_init(&thread2,
                   "thread2",
                   thread2_entry,
                   RT_NULL,
                   &thread2_stack[0],
                   sizeof(thread2_stack), 25, 5);
    rt_thread_startup(&thread2);

    return 0;
}

/* Export to the msh command list */
MSH_CMD_EXPORT(msgq_sample, msgq sample);
```

The simulation results are as follows:

```
\ | /
- RT -     Thread Operating System
 / | \     3.1.0 build Aug 24 2018
 2006 - 2018 Copyright by rt-thread team
msh > msgq_sample
msh >thread2: send message - A
thread1: recv msg from msg queue, the content:A
thread2: send message - B
thread2: send message - C
thread2: send message - D
thread2: send message - E
thread1: recv msg from msg queue, the content:B
thread2: send message - F
thread2: send message - G
thread2: send message - H
thread2: send urgent message - I
thread2: send message - J
thread1: recv msg from msg queue, the content:I
thread2: send message - K
thread2: send message - L
thread2: send message - M
thread2: send message - N
thread2: send message - O
thread1: recv msg from msg queue, the content:C
thread2: send message - P
thread2: send message - Q
thread2: send message - R
thread2: send message - S
thread2: send message - T
thread1: recv msg from msg queue, the content:D
message queue stop send, thread2 quit
thread1: recv msg from msg queue, the content:E
thread1: recv msg from msg queue, the content:F
thread1: recv msg from msg queue, the content:G
...
thread1: recv msg from msg queue, the content:T
thread1: detach mq
```

The routine demonstrates how to use message queue. Thread 1 receives messages from the message queue; thread 2 periodically sends regular and emergency messages to the message queue. Since the message "I" that thread 2 sent is an emergency message, it will be inserted directly into the front of the message queue. So after receiving the message "B", thread 1 receives the emergency message and then receives the message "C".

1.3.6.2.5 Occasions to Use Message Queue

Message Queue can be used where occasional long messages are sent, including exchanging message between threads and threads and sending message to threads in interrupt service routines (interrupt service routines cannot

receive messages). The following sections describe the use of message queues from two perspectives, sending messages and synchronizing messages.

1.3.6.2.5.1 Sending Messages

The obvious difference between message queue and mailbox is that the length of the message is not limited to 4 bytes. In addition, message queue also includes a function interface for sending emergency messages. But when you create a message queue with a maximum length of 4 bytes for all messages, the message queue object will be reduced to a mailbox. This unrestricted message is also reflected in the case of code writing which is also a mailbox-like code:

```
struct msg
{
    rt_uint8_t *data_ptr;    /* Data block starting address */
    rt_uint32_t data_size;   /* Data block size   */
};
```

Similar to the mailbox example is the message structure definition. Now, let's assume that such a message needs to be sent to thread receiving messages. In the mailbox example, this structure can only send pointers pointing to this structure (after the function pointer is sent, the thread receiving messages can access the content pointing to this address, usually this piece of data needs to be left to the thread receiving the messages to release). How message queues is used is quite different:

```
void send_op(void *data, rt_size_t length)
{
    struct msg msg_ptr;

    msg_ptr.data_ptr = data;  /* Point to the corresponding data block address */
    msg_ptr.data_size = length; /* Datablock length */

    /* Send this message pointer to the mq message queue */
    rt_mq_send(mq, (void*)&msg_ptr, sizeof(struct msg));
}
```

Note that in the above code, the data content of a local variable is sent to the message queue. In the thread that receives message, the same structure that receives messages using local variables is used:

```
void message_handler()
{
    struct msg msg_ptr; /* Local variable used to place the message */

    /* Receive messages from the message queue into msg_ptr */
    if (rt_mq_recv(mq, (void*)&msg_ptr, sizeof(struct msg)) >= 0)
    {
        /* Successfully received the message, corresponding data processing is performed */
    }
}
```

Because message queue is a direct copy of data content, so in the above example, local structure is used to save the message structure, which eliminates the trouble of dynamic memory allocation (and no need to worry, because message memory space has already been released when the thread receives the message).

1.3.6.2.5.2 Synchronizing Messages

In general system designs, the problem of sending synchronous messages is often encountered. At this time, corresponding implementations can be selected according to the state of the time: two threads can be implemented in the form of [message queue + semaphore or mailbox]. The thread sending messages sends the corresponding message to the message queue in the form of message sending. After the message is sent, it is hoping to receive the confirmation from the threads receiving messages. The working diagram is as shown below:

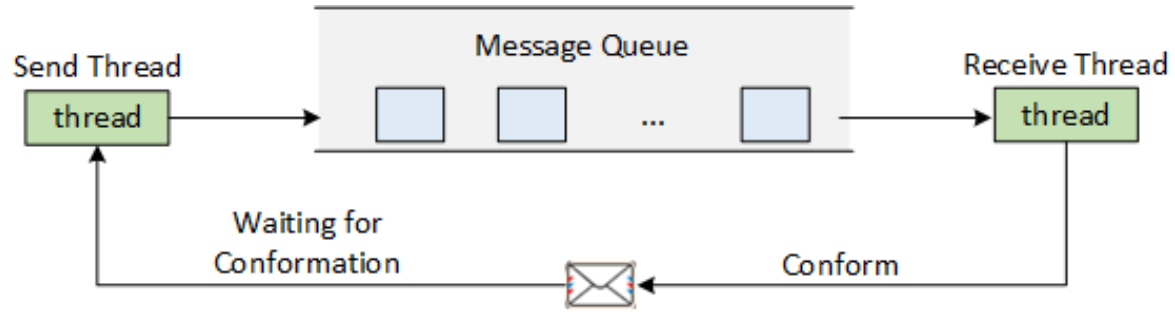


Figure 1.50 Synchronizing Messages Diagram

Depending on the message confirmation, the message structure can be defined as:

```
struct msg
{
    /* Other members of the message structure */
    struct rt_mailbox ack;
};
/* or */
struct msg
{
    /* Other members of the message structure */
    struct rt_semaphore ack;
};
```

The first type of message uses a mailbox as a confirmation flag, while the second type of message uses a semaphore as a confirmation flag. The mailbox is used as a confirmation flag, which means that the receiving thread can notify some status values to the thread sending messages; and the semaphore is used as a confirmation flag that can only notify the thread sending messages in a single way, and the message has been confirmed to be received.

1.3.6.3 Signal

A signal (also known as a soft interrupt signal), from a software perspective, is a simulation of interrupt mechanism. When it comes to its principle, thread receiving a signal is similar to processor receiving an interrupt request.

1.3.6.3.1 Signal Working Mechanism

Signals are used for asynchronous communication in RT-Thread. The POSIX standard defines that sigset_t type defines a signal set. However, the sigset_t type may be defined differently in different systems. In RT-Thread, sigset_t is defined as unsigned long and named rt_sigset_t, the signals that the application can use are SIGUSR1(10) and SIGUSR2(12).

The essence of signal is soft interrupt, which is used to notify the thread that an asynchronous event has occurred. It is used to notify abnormality and handle emergency between threads. No operation is needed for a thread to wait for the signal's arrival. In fact, the thread does not know when the signal will arrive. Threads can send soft interrupt signals by calling `rt_thread_kill()` between each other.

The thread that receives the signals has different processing methods for various signals, and these methods can be divided into three categories:

The first one is an interrupt-like processing program. For signals that need to be processed, the thread can specify a function to process.

The second one is to ignore a signal and do nothing about it, as if it had not happened.

The third one is to reserve the default value of the system for processing the signal.

As shown in the following figure, suppose that thread 1 needs to process the signal. First, thread 1 installs a signal and unmasks it. At the same time, it sets the approach of how to process abnormality of signals. Then other threads can send a signal to thread 1, triggering thread 1 to process the signal.

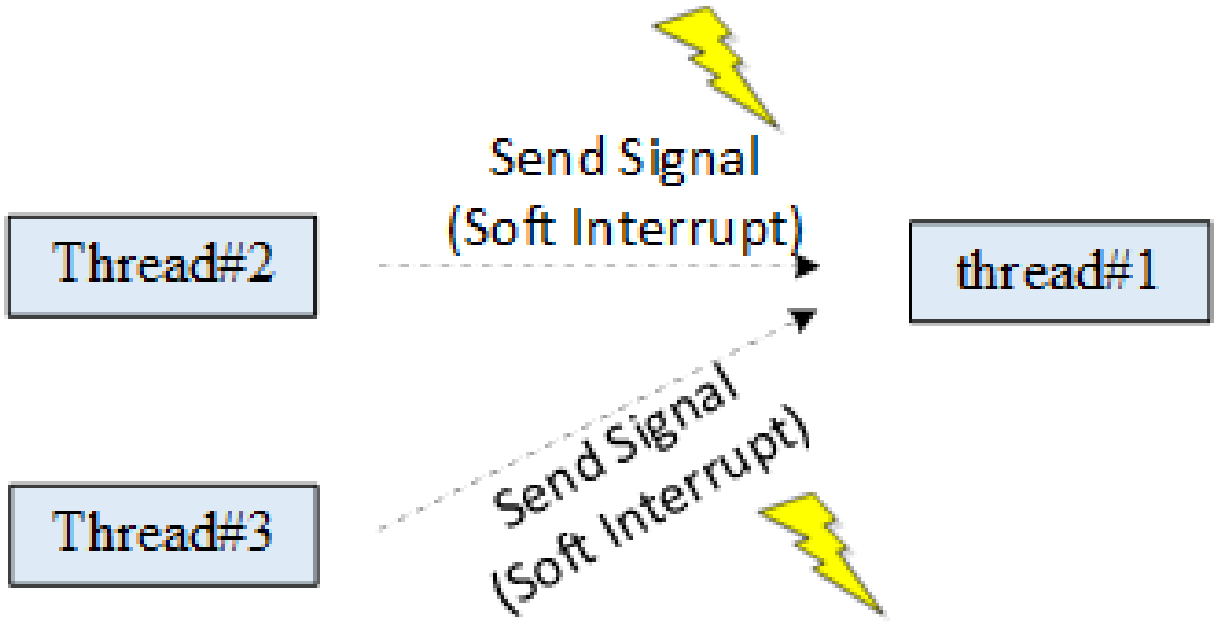


Figure 1.51 Signal Working Mechanism

When the signal is passed to thread 1, if it is suspended, it will be switched to ready to process the corresponding signal. If it is running, it will create a new stack frame space on its current thread stack to process the corresponding signal. It should be noted that the thread stack size used will also increase accordingly.

1.3.6.3.2 Management of Signals

Operations on signals include: install signal, block signal, unblock signal, send signal, and wait for signal. The interfaces of the signal are shown in the following figure:

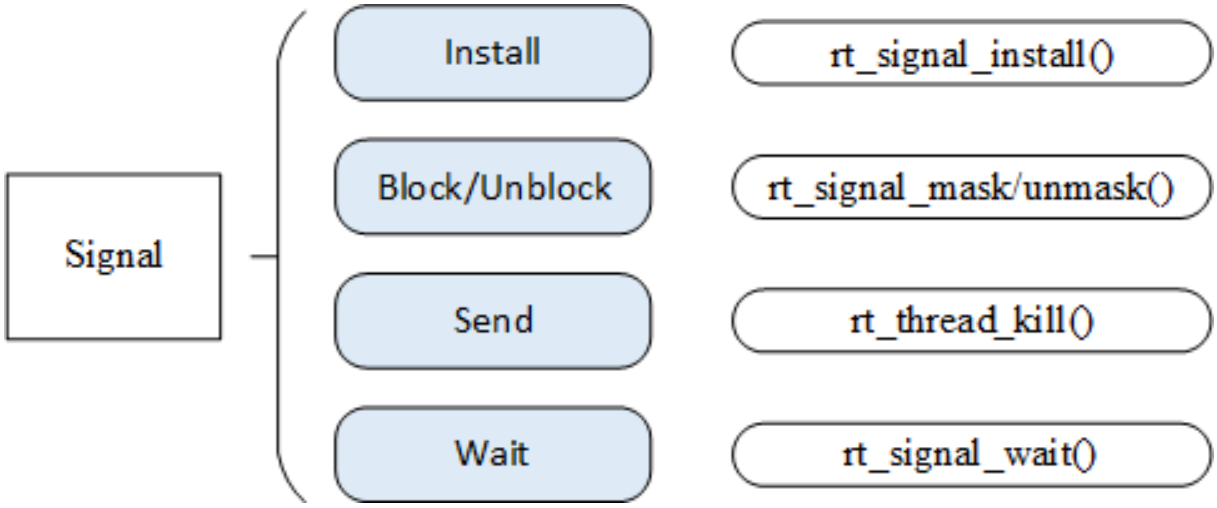


Figure 1.52 Signal Related Interface

1.3.6.3.2.1 Install Signal

If the thread is to process a signal, then the signal needs to be installed in the thread. Signal installation is primarily used to determine the mapping relation between the signal value and the actions of the thread on that signal value, that is what signal is to be processed, what actions will be taken when the signal is passed to the thread. See the following code for a detailed definition:

```
rt_sighandler_t rt_signal_install(int signo, rt_sighandler_t[] handler);
```

`rt_sighandler_t` is the type of function pointer that defines the signal process function. The following table describes the input parameters and return values for this function:

Input parameters and return values for rt_signal_install()

Parameters	Description
signo	Signal value (only SIGUSR1 and SIGUSR2 are open to the user, the same applies below)
handler	Set the approach to process signal values
Return	——
SIG_ERR	Wrong signal
The handler value before the signal is installed	Successful

Setting the handler parameter during signal installation determines the different processing methods for this signal. Processing methods can be divided into three types:

1) Similar to the interrupt processing method, the parameter points to the user-defined processing function when the signal occurs, and is processed by the function.

2) The parameter is set to SIG_IGN. Ignore a signal, and do nothing on the signal, as if it had not happened.

3) The parameter is set to SIG_DFL, and the system will call the default function _signal_default_handler() to process.

1.3.6.3.2.2 Block Signal

Blocking signal can also be understood as shielding signals. If the signal is blocked, the signal will not be delivered to the thread that installed the signal and will not cause soft interrupt processing. Call rt_signal_mask() to block the signal:

```
void rt_signal_mask(int signo);
```

The following table describes the input parameters for this function:

rt_signal_mask() function parameters

Parameters	Description
signo	Signal value

1.3.6.3.2.3 Unblock Signal

Several signals can be installed in the thread. Using this function can give some "attention" to some of the signals, then sending these signals will cause a soft interrupt for the thread. Calling rt_signal_unmask() to unblock the signal:

```
void rt_signal_unmask(int signo);
```

The following table describes the input parameters for this function:

rt_signal_unmask() function parameters

Parameters	Description
signo	Signal value

1.3.6.3.2.4 Send Signals

When we need to process abnormality, a signal can be sent to the thread that has been set to process abnormality. Call rt_thread_kill() to send a signal to any thread:

```
int rt_thread_kill(rt_thread_t tid, int sig);
```

The following table describes the input parameters and return values for this function:

Input parameters and return values for rt_thread_kill()

Parameters	Description
tid	The thread receiving signal
sig	Signal value
Return	——
RT_EOK	Sent successfully
-RT_EINVAL	Parameter error

1.3.6.3.2.5 Wait for Signal

Wait for the arrival of the set signal. If this signal did not arrive, suspend the thread until the signal arrival or the wait until time exceeds the specified timeout. If the signal arrived, the pointer pointing to the signal body is stored in si, as follows is the function to wait for signal.

```
int rt_signal_wait(const rt_sigset_t *set,
                       rt_siginfo_t[] *si, rt_int32_t timeout);
```

rt_siginfo_t is the data type that defines the signal information, and the following table describes the input parameters and return values of the function:

Input parameters and return values for rt_signal_wait()

Parameters	Description
set	Specify the signal to wait
si	Pointer pointing to signal information
timeout	Waiting time

Parameters	Description
Return	——
RT_EOK	Signal arrives
-RT_ETIMEOUT	Timeout
-RT_EINVAL	Parameter error

1.3.6.3.3 Signal Application Example

This is a signal application routine, as shown in the following code. This routine creates one thread. When the signal is installed, the signal processing mode is set to custom processing. The processing function of the signal is defined to be thread1_signal_handler(). After the thread is running and the signal is installed, a signal is sent to this thread. This thread will receive the signal and print the message.

Signal usage routine

```
#include <rtthread.h>

#define THREAD_PRIORITY         25
#define THREAD_STACK_SIZE       512
#define THREAD_TIMESLICE        5

static rt_thread_t tid1 = RT_NULL;

/* Signal process function for thread 1 signal handler */
void thread1_signal_handler(int sig)
{
    rt_kprintf("thread1 received signal %d\n", sig);
}

/* Entry function for thread 1 */
static void thread1_entry(void *parameter)
{
    int cnt = 0;

    /* Install signal */
    rt_signal_install(SIGUSR1, thread1_signal_handler);
    rt_signal_unmask(SIGUSR1);

    /* Run for 10 times */
    while (cnt < 10)
    {
        /* Thread 1 runs with low-priority and prints the count value all through*/
        rt_kprintf("thread1 count : %d\n", cnt);

        cnt++;
        rt_thread_mdelay(100);
    }
}

/* Initialization of the signal example */
int signal_sample(void)
{
    /* Create thread 1 */
    tid1 = rt_thread_create("thread1",
                            thread1_entry, RT_NULL,
                            THREAD_STACK_SIZE,
                            THREAD_PRIORITY, THREAD_TIMESLICE);

    if (tid1 != RT_NULL)
        rt_thread_startup(tid1);

    rt_thread_mdelay(300);

    /* Send signal SIGUSR1 to thread 1 */
    rt_thread_kill(tid1, SIGUSR1);

    return 0;
}

/* Export to the msh command list */
MSH_CMD_EXPORT(signal_sample, signal sample);
```

The simulation results are as follows:

```
 \ | /
```

```
- RT -     Thread Operating System
 / | \     3.1.0 build Aug 24 2018
 2006 - 2018 Copyright by rt-thread team
msh >signal_sample
thread1 count : 0
thread1 count : 1
thread1 count : 2
msh >thread1 received signal 10
thread1 count : 3
thread1 count : 4
thread1 count : 5
thread1 count : 6
thread1 count : 7
thread1 count : 8
thread1 count : 9
```

In the routine, the thread first installs the signal and unblocks it, then sends a signal to the thread. The thread receives the signal and prints out the signal received: SIGUSR1 (10).

1.3.7 Memory Management

In a computing system, there are usually two types of memory space: internal memory space and external memory space. Internal memory can be quickly accessed, its contents can be read to and changed and only an address is required. The contents of internal memory are deleted after each power off. It is what would usually be called RAM (Random Access Memory) and is analogous to the RAM in a desktop computer. On the other hand, external memory has relatively fixed contents, retains data even after power off. It is usually called ROM (Read-Only Memory) and is analogous to the hard disk in a desktop computer.

In a computer system, variables and intermediate data are generally stored in RAM, and they are only transferred from RAM to CPU for calculation when actually used. The memory size required by some data needs to be determined according to the actual situation during the running of the program, which requires the system to have the ability to dynamically manage the memory space.The user applies to the system when he needs a block of memory space, then the system selects a suitable memory space to allocate to the user. After the user finishes using it, the memory space is released back to the system which enables the system to recycle the memory space.

This chapter introduces two kinds of memory management methods in RT-Thread, namely dynamic memory heap management and static memory pool management. After studying this chapter, readers will understand the memory management principle and usage of RT-Thread.

1.3.7.1 Memory Management Functional Features

Because time requirements are very strict in real-time systems, memory management is often much more demanding than in general-purpose operating systems:

1) Time for allocating memory must be deterministic. The general memory management algorithm needs to find a free memory block that is compatible with the data according to the length of the data to be stored, and then store the data therein. The time it takes to find such a free block of memory is uncertain, but for real-time systems, this is unacceptable. Real-time systems require that the allocation process of the memory block is completed within a predictable certain time, otherwise the response of a real-time task to an external event will become indeterminable.

2) As memory is constantly being allocated and released, the entire memory area will fragment. This means that instead of one large continuous block, the memory area will have small memory blocks taken up inbetween, limiting the largest possible contiguous block which the code can ask for. For general-purpose systems, this issue can be solved by rebooting the system (once every month or once per few months). However, this solution is unacceptable for embedded systems that need to work continuously in the field work all the year round.

3) The resource environment of embedded system is also different. Some systems have relatively tight resources, only tens of kilobytes of memory are available for allocation, while some systems have several megabytes of memory. This makes choosing an efficient memory allocation algorithm for these different systems more complicated.

The RT-Thread operating system provides different memory allocation management algorithms for memory management according to different upper layer applications and system resources. Generally, it can be divided into two categories: memory heap management and memory pool management. Memory heap management is divided into three cases according to specific memory devices:

The first is allocation management for small memory blocks (small memory management algorithm);

The second is allocation management for large memory blocks (slab management algorithm);

The third is allocation management for multiple memory heaps (memheap management algorithm)

1.3.7.2 Memory Heap Management

Memory heap management is used to manage a contiguous memory space. We introduced the memory distribution of RT-Thread in chapter "Kernel Basics". As shown in the following figure, RT-Thread uses the space at "the end of the ZI segment" to the end of the memory as the memory heap.

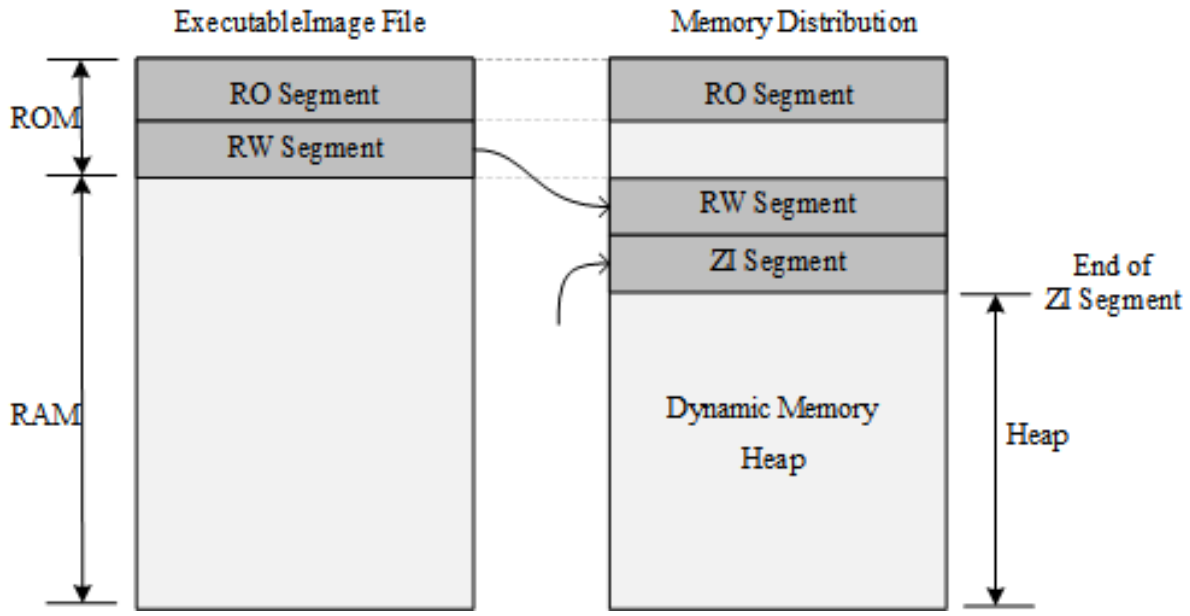


Figure 1.53 RT-Thread Memory Distribution

If current resources allow it, the memory heap can allocate memory blocks of any size according to the needs of the user. When the user does not need to use these memory blocks, they can be released back to the heap for other applications to allocate and use. In order to meet different needs, the RT-Thread system provides different memory management algorithms, namely small memory management algorithm, slab management algorithm and memheap management algorithm.

The small memory management algorithm is mainly for system with less resources and with less than 2MB of memory. The slab memory management algorithm mainly provides a fast algorithm similar to multiple memory pool management algorithms when the system resources are rich. In addition to the above, RT-Thread also has a management algorithm for a multi-memory heap, namely the memheap management algorithm. The memheap management algorithm is suitable for where there are multiple memory heaps in the system. It can “paste” multiple memories together to form a large memory heap, which is very easy to use for users.

Any or none of these memory heap management algorithms can be chosen when the system is running and all algorithms provide the same API interface to the application.

>Because the memory heap manager needs to meet the security allocation in multi-threaded conditions, which means mutual exclusion between multiple threads needs to be taken into consideration, please do not allocate or release dynamic memory blocks in interrupt service routines, as that may result in the current context being suspended.

1.3.7.2.1 Small Memory Management Algorithm

The small memory management algorithm is a simple memory allocation algorithm. Initially, it is a large piece of memory. When a memory block needs to be allocated, the matching memory block is segmented from the large memory block, and then this matching free memory block is returned to the heap management system. Each memory block contains data head for management use through which the used block and the free block are linked by a doubly linked list.

Use this implementation before 4.1.0

As shown in the following figure:

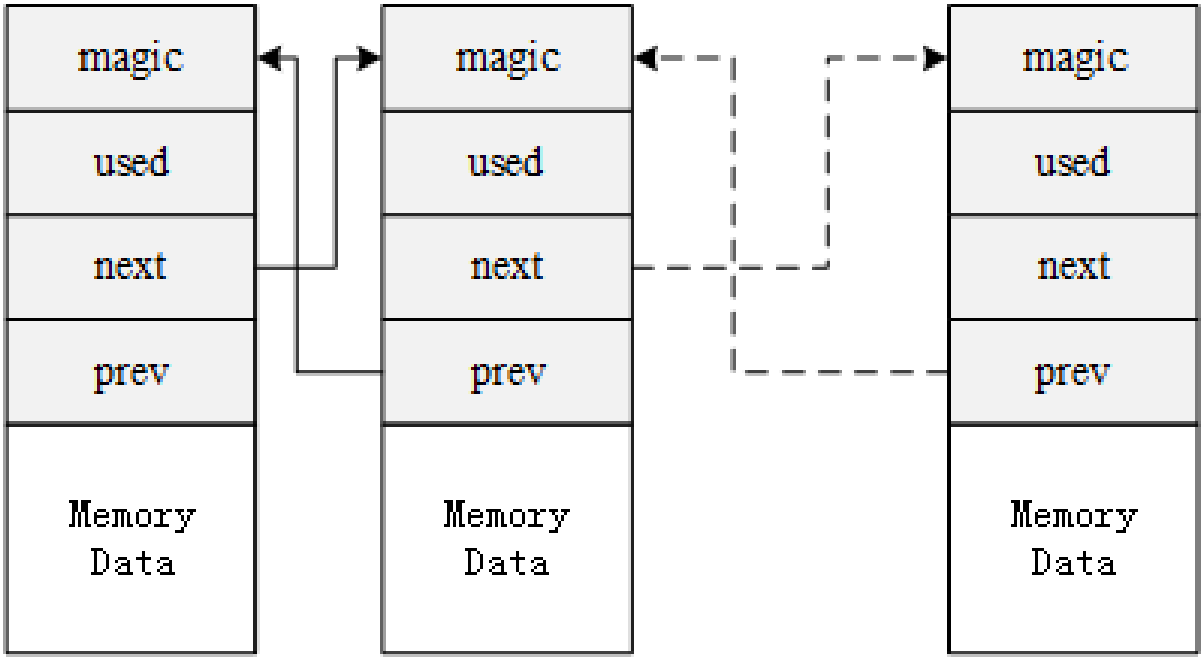


Figure 1.54 Small Memory Management Working Mechanism Diagram

Each memory block (whether it is an allocated memory block or a free memory block) contains a data head, including:

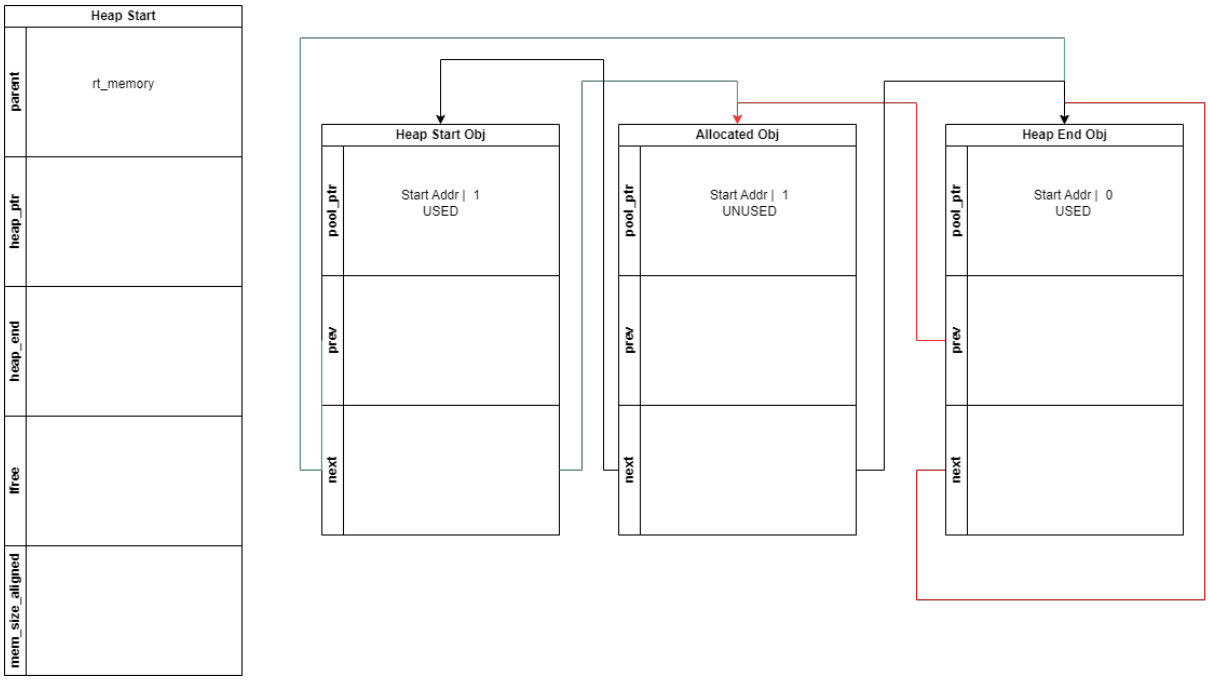
1) magic: Variable (also called magic number). It will be initialized to 0x1ea0 (that is, the English word heap) which is used to mark this memory block as a memory data block for memory management. The variable is not only used to identify that the data block is a memory data block for memory management. It is also a memory protection word: if this area is overridden, it means that the memory block is illegally overridden (normally only the memory manager will operate on this memory).

2)used: Indicates whether the current memory block has been allocated.

The performance of memory management is mainly reflected in the allocation and release of memory. The small memory management algorithm can be embodied by the following examples.

4.1.0 and later use this implementation

As shown in the following figure:



Heap Start: The heap head address stores memory usage information, the heap head address pointer, the heap end address pointer, the minimum heap free address pointer, and the memory size.

Each memory block (whether it is an allocated memory block or a free memory block) contains a data head, including:

pool_ptr: Small memory object address. If the last bit of the memory block is 1, it is used. If the last bit of the memory block is 0, it is not used. When used, the structure members of the small memory algorithm can be quickly obtained by calc.

As shown in the following figure, the free list pointer lfree initially points to a 32-byte block of memory. When the user thread wants to allocate a 64-byte memory block, since the memory block pointed to by this lfree pointer is only 32 bytes and does not meet the requirements, the memory manager will continue to search for the next memory block. When the next memory block with 128 bytes is found, it meets the requirements of the allocation. Because this memory block is large, the allocator will split the memory block, and the remaining memory block (52 bytes) will remain in the lfree linked list, as shown in the following table which is after 64 bytes is allocated.

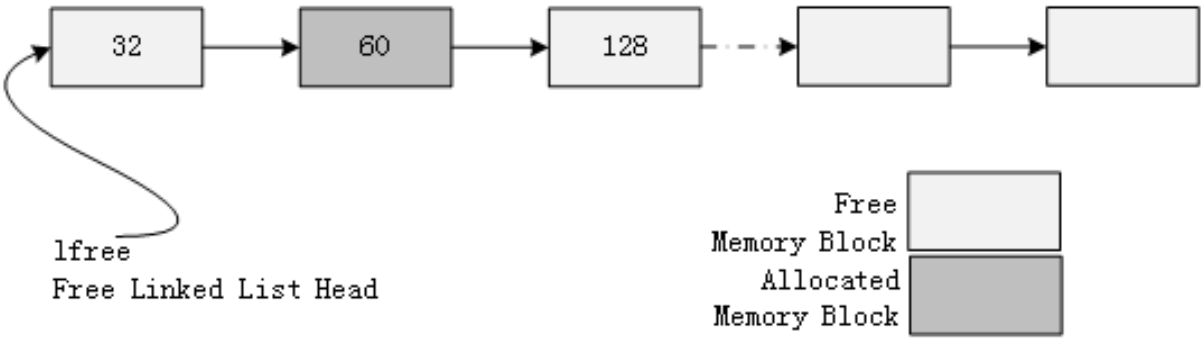


Figure 1.55 Small Memory Management Algorithm Linked List Structure Diagram 1

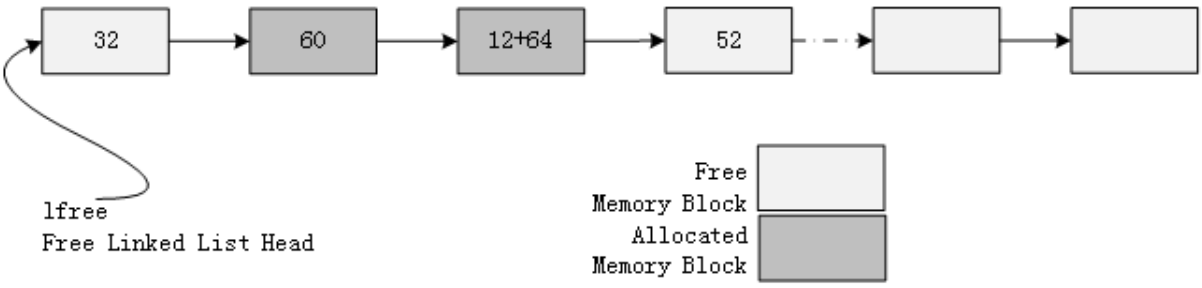


Figure 1.56 Small Memory Management Algorithm Linked List Structure Diagram 2

In addition, a 12-byte data head is reserved for `magic, used` information, and linked list nodes before each memory block are allocated. The address returned to the application is actually the address after 12 bytes of this memory block. The 12-byte data head is the part that the user should never use. (Note: The length of the 12-byte data head will be different depending on the alignment of the system).

As for releasing, it is the reverse process, but the allocator will check if the adjacent memory blocks are free, and if they are free, the allocator will merge them into one large free memory block.

1.3.7.2.2 Slab Management Algorithm

RT-Thread's slab allocator is an optimized memory allocation algorithm for embedded systems based on the slab allocator implemented by DragonFly BSD founder Matthew Dillon. The most primitive slab algorithm is Jeff Bonwick's efficient kernel memory allocation algorithm introduced for the Solaris operating system.

RT-Thread's slab allocator implementation mainly removes the object construction and destruction process, and only retains the pure buffered memory pool algorithm. The slab allocator is divided into multiple zones according to the size of the object which can also be seen as having a memory pool for each type of object, as shown in the following figure:

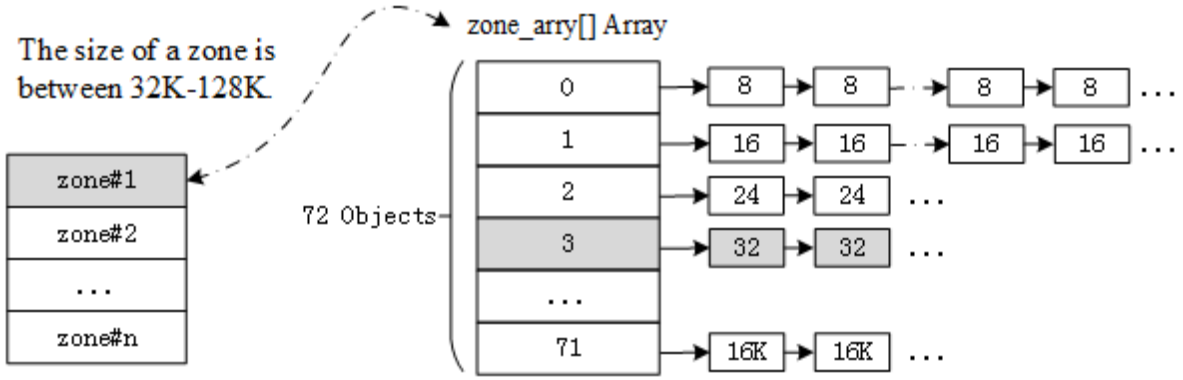


Figure 1.57 slab Memory Allocation Structure

A zone is between 32K and 128K bytes in size, and the allocator will automatically adjust based on the heap size when the heap is initialized. The zone in the system includes up to 72 objects, which can allocate up to 16K of memory at a time. It will directly allocate from the page allocator if it exceeds 16K. The size of the memory block allocated on each zone is fixed. Zones that can allocate blocks of the same size are linked in a linked list. The zone linked lists of the 72 objects are managed in an array. (zone_array[]).

Here are the two main operations for the memory allocator:

(1) Memory Allocation

Assuming a 32-byte memory is allocated, the slab memory allocator first finds the corresponding zone linked list from the linked list head of zone array in accordance with the 32-byte value. If the linked list is empty, assign a new zone to the page allocator and return the first free block of memory from the zone. If the linked list is not empty, a free block must exist in the first zone node in the zone linked list (otherwise it would not have been placed in the linked list), so then take the corresponding free block. If all free memory blocks in the zone are used after the allocation, the allocator needs to remove this zone node from the linked list.

(2)Memory Release

The allocator needs to find the zone node where the memory block is located, and then link the memory block to the zone's free memory block linked list. If the free linked list of the zone indicates that all the memory blocks of the zone have been released, it means that the zone is completely free. The system will release the fully free zone to the page allocator when the number of free zones in the zone linked list reaches a certain number.

1.3.7.2.3 memheap Management Algorithm

The memheap management algorithm is suitable for systems with multiple memory heaps that are not contiguous. Using memheap memory management can simplify the use of multiple memory heaps in the system: when there are multiple memory heaps in the system, the user only needs to initialize multiple needed memheaps during system initialization and turn on the memheap function to attach multiple memheaps (addresses can be discontinuous) for the system's heap allocation.

>The original heap function will be turned off after memheap is turned on. Both can only be selected by turning RT_USING_MEMHEAP_AS_HEAP on or off.

The working mechanism of memheap is shown in the figure below. First, add multiple blocks of memory to the memheap_item linked list to attach. The allocation of a memory block starts with allocating memory from default memory heap. When it can not be allocated, memheap_item linked list is looked up, and an attempt is made to allocate a memory block from another memory heap. This process is opaque to the user, who only sees one memory heap.

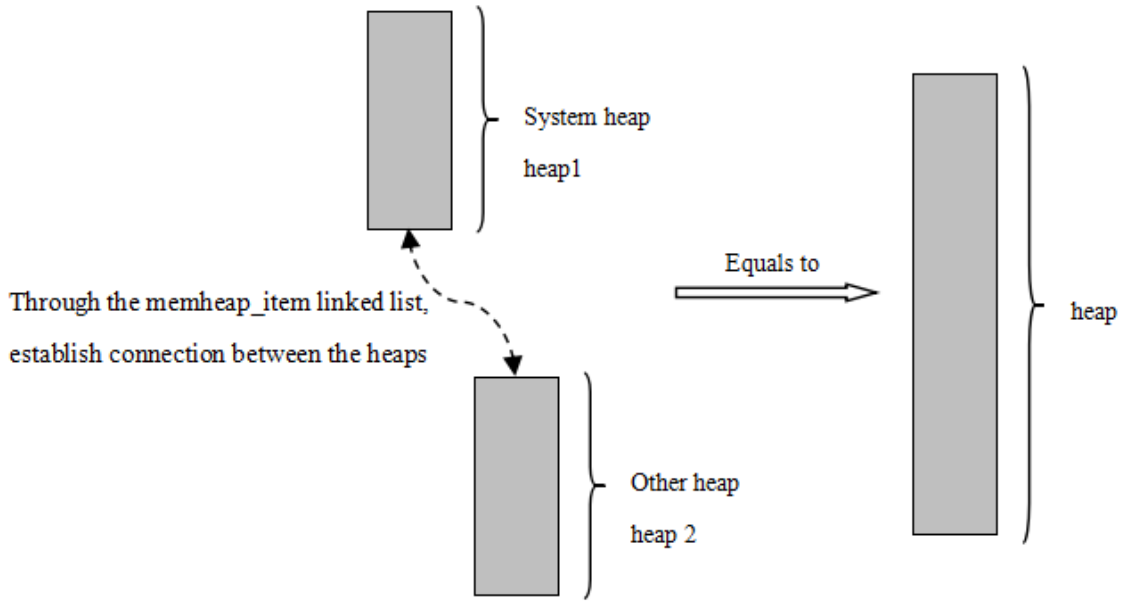


Figure 1.58 memheap Handling Multiple Memory Heaps

1.3.7.2.4 Memory Heap Configuration and Initialization

When using the memory heap, heap initialization must be done at system initialization, which can be done through the following function interface:

```
void rt_system_heap_init(void* begin_addr, void* end_addr);
```

This function will use the memory space of the parameters begin_addr, end_addr as a memory heap. The following table describes the input parameters for this function:

Input parameter for rt_system_heap_init()

Parameters	Description
begin_addr	Start address for heap memory area
end_addr	End address for heap memory area

When using memheap heap memory, you must initialize the heap memory at system initialization, which can be done through the following function interface:

```
rt_err_t rt_memheap_init(struct rt_memheap  *memheap,
                         const char  *name,
                         void        *start_addr,
                         rt_uint32_t size)
```

If there are multiple non-contiguous memheaps, the function can be called multiple times to initialize it and join the memheap_item linked list. The following table describes the input parameters and return values for this function:

Input parameters and return values of rt_memheap_init()

Parameters	Description
memheap	memheap control block
name	The name of the memory heap
start_addr	Heap memory area start address
size	Heap memory size
Return	——
RT_EOK	Successful

1.3.7.2.5 Memory Heap Management

Operations of the memory heap are as shown in the following figure, including: initialization, application for memory blocks, release of memory. After use, all dynamic memory should be released for future use by other programs.

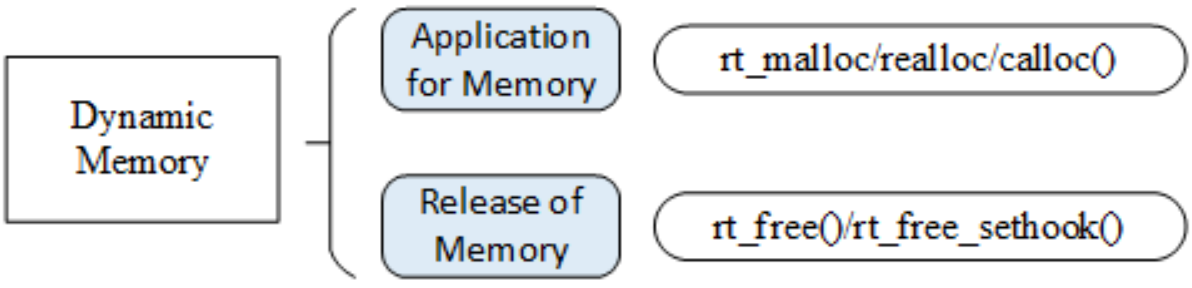


Figure 1.59 Operations of the Memory Heap

1.3.7.2.5.1 Allocate and Release Memory Block

Allocate a memory block of user-specified size from the memory heap. The function interface is as follows:

```
void *rt_malloc(rt_size_t nbytes);
```

The rt_malloc function finds a memory block of the appropriate size from the system heap space and returns the available address of the memory block to the user. The following table describes the input parameters and return values for this function:

Input parameters and return values of rt_malloc()

Parameters	Description
nbytes	The size of the memory block to be allocated, in bytes
Return	——
Allocated memory block address	Successful
RT_NULL	Fail

After the application uses the memory applied from the memory allocator, it must be released in time, otherwise it will cause a memory leak. The function interface for releasing the memory block is as follows:

```
void rt_free (void *ptr);
```

The rt_free function will return the to-be-released memory back to the heap manager. When calling this function, user needs to pass the to-be-released pointer of the memory block. If it is a null pointer, it returns directly. The following table describes the input parameters for this function:

Input parameters of rt_free()

Parameters	Description
ptr	to-be-released memory block pointer

1.3.7.2.5.2 Re-allocate Memory Block

Re-allocating the size of the memory block (increase or decrease) based on the allocated memory block can be done through the following function interface:

```
void *rt_realloc(void *rmem, rt_size_t newsize);
```

When the memory block is re-allocated, the original memory block data remains the same (in the case of reduction, the subsequent data is automatically truncated). The following table describes the input parameters and return values for this function:

Input parameters and return values of rt_realloc()

Parameters	Description
rmem	Point to the allocated memory block
newsize	Re-allocated memory size
Return	——
Re-allocated memory block address	Successful

1.3.7.2.5.3 Allocate Multiple Memory Blocks

Allocating multiple memory blocks with contiguous memory addresses from the memory heap can be done through the following function interface:

```
void *rt_calloc(rt_size_t count, rt_size_t size);
```

The following table describes the input parameters and return values for this function:

Input parameters and return values of rt_calloc()

Parameters	Description
count	Number of memory block
size	Size of memory block
Return	——
Pointer pointing to the first memory block address	Successful, all allocated memory blocks are initialized to zero.
RT_NULL	Allocation failed

1.3.7.2.5.4 Set Memory Hook Function

When allocating memory blocks, user can set a hook function. The function interface called is as follows:

```
void rt_malloc_sethook(void (*hook)(void *ptr, rt_size_t size));
```

The hook function set will callback after the memory allocation. During the callback, the allocated memory block address and size are passed as input parameters. The following table describes the input parameters for this function:

Input parameters for rt_malloc_sethook()

Parameters	Description
hook	Hook function pointer

The hook function interface is as follows:

```
void hook(void *ptr, rt_size_t size);
```

The following table describes the input parameters for the hook function:

Allocate hook function interface parameters

Parameters	Description
ptr	The allocated memory block pointer
size	The size of the allocated memory block

When releasing memory, user can set a hook function, the function interface called is as follows:

```
void rt_free_sethook(void (*hook)(void *ptr));
```

The hook function set will callback before the memory release is completed. During the callback, the released memory block address is passed in as the entry parameter (the memory block is not released at this time). The following table describes the input parameters for this function:

Input parameters for rt_free_sethook()

Parameters	Description
hook	Hook function pointer

The hook function interface is as follows:

```
void hook(void *ptr);
```

The following table describes the input parameters for the hook function:

Input parameters of the hook function

Parameters	Description
ptr	Memory block pointer to be released

1.3.7.2.6 Memory Heap Management Application Example

This is an example of a memory heap application. This program creates a dynamic thread that dynamically requests memory and releases it. Each time it apples for more memory, it ends when it can't apply for it, as shown in the following code:

Memory heap management

```
#include <rtthread.h>

#define THREAD_PRIORITY      25
#define THREAD_STACK_SIZE    512
#define THREAD_TIMESLICE     5

/* thread entry */
void thread1_entry(void *parameter)
{
    int i;
    char *ptr = RT_NULL; /* memory block pointer */

    for (i = 0; ; i++)
    {
        /* memory space for allocating (1 << i) bytes each time */
        ptr = rt_malloc(1 << i);

        /* if allocated successfully */
        if (ptr != RT_NULL)
        {
            rt_kprintf("get memory :%d byte\n", (1 << i));
            /* release memory block */
            rt_free(ptr);
            rt_kprintf("free memory :%d byte\n", (1 << i));
            ptr = RT_NULL;
        }
        else
        {
            rt_kprintf("try to get %d byte memory failed!\n", (1 << i));
            return;
        }
    }
}

int dynmem_sample(void)
{
    rt_thread_t tid = RT_NULL;

    /* create thread 1 */
    tid = rt_thread_create("thread1",
                           thread1_entry, RT_NULL,
                           THREAD_STACK_SIZE,
                           THREAD_PRIORITY,
                           THREAD_TIMESLICE);
    if (tid != RT_NULL)
        rt_thread_startup(tid);

    return 0;
```

```
}
/* Export to the msh command list */
MSH_CMD_EXPORT(dynmem_sample, dynmem sample);
```

The simulation results are as follows:

```
\ | /
- RT -     Thread Operating System
 / | \     3.1.0 build Aug 24 2018
 2006 - 2018 Copyright by rt-thread team
msh >dynmem_sample
msh >get memory :1 byte
free memory :1 byte
get memory :2 byte
free memory :2 byte
...
get memory :16384 byte
free memory :16384 byte
get memory :32768 byte
free memory :32768 byte
try to get 65536 byte memory failed!
```

The memory is successfully allocated in the routine and the information is printed; when trying to apply 65536 byte, 64KB, of memory, the allocation fails because the total RAM size is only 64K and the available RAM is less than 64K.

1.3.7.3 Memory Pool

The memory heap manager can allocate blocks of any size, which is very flexible and convenient. However, it also has obvious shortcomings. Firstly, the allocation efficiency is not high because free memory blocks need to be looked up for each allocation. Secondly, it is easy to generate memory fragmentation. In order to improve the memory allocation efficiency and avoid memory fragmentation, RT-Thread provides another method of memory management: Memory Pool.

Memory pool is a memory allocation method for allocating a large number of small memory blocks of the same size. It can greatly speed up memory allocation and release, and can avoid memory fragmentation as much as possible. In addition, RT-Thread's memory pool allows thread suspend function. When there is no free memory block in the memory pool, the application thread will be suspended until there is a new available memory block in the memory pool, and then the suspended application thread will be awakened.

The thread suspend function of the memory pool is very suitable for scenes that need to be synchronized by memory resources. For example, when playing music, the player thread decodes the music file and then sends it to the sound card driver to drive the hardware to play music.

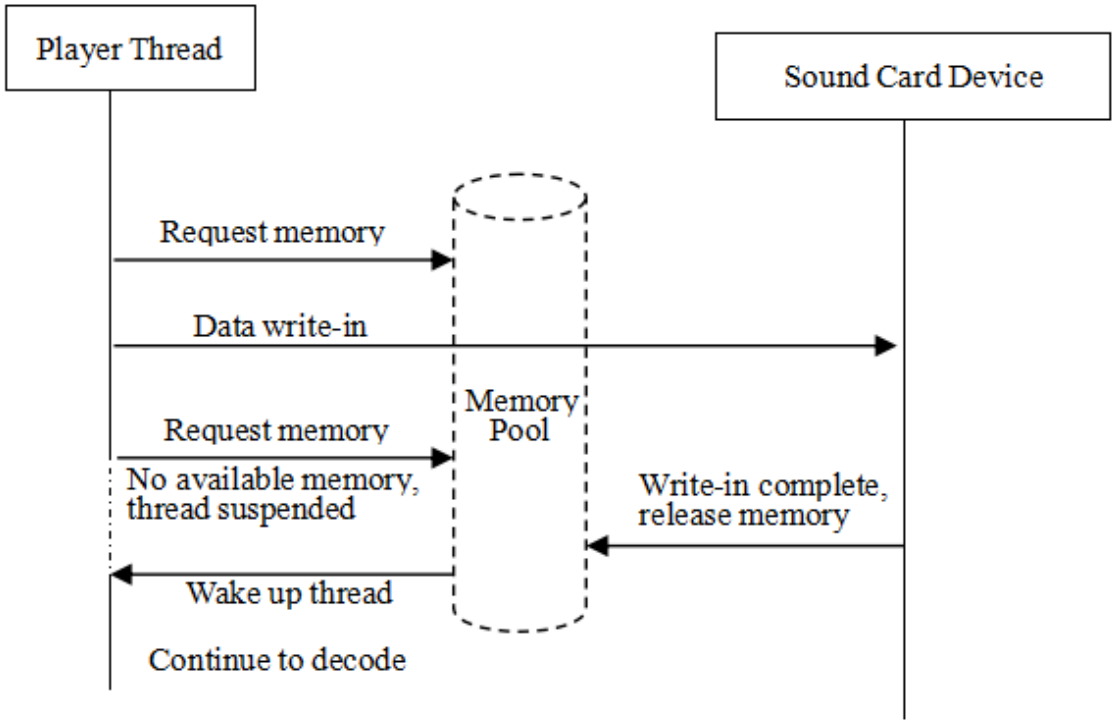


Figure 1.60 Player Thread & Sound Card Driver Relationship

As shown in the figure above, when the player thread needs to decode the data, it will request the memory block from the memory pool. If there is no memory block available, the thread will be suspended, otherwise it will obtain the memory block to place the decoded data.

The player thread then writes the memory block containing the decoded data to the sound card abstraction device (the thread will return immediately and continue to decode more data).

After the sound card device is written, the callback function set by the player thread is called to release the written memory block. If the player thread is suspended because there is no memory block in the memory pool available, then it will be awakened to continue to decode.

1.3.7.3.1 Memory Pool Working Mechanism

1.3.7.3.1.1 Memory Pool Control Block

The memory pool control block is a data structure used by the operating system to manage the memory pool. It stores some information about the memory pool, such as the start address of the data area in the memory pool, the memory block size and the memory block list. It also includes memory blocks, a linked list structure used for the connection between memory blocks, event set of the thread suspended due to the memory block being unavailable, and so on.

In the RT-Thread real-time operating system, the memory pool control block is represented by the structure `struct rt_mempool`. Another C expression, `rt_mp_t`, represents the memory block handle. The implementation in C language is a pointer pointing to the memory pool control block. For details, see the following code:

```
struct rt_mempool
{
    struct rt_object parent;

    void        *start_address;  /* start address of memory pool data area */
```

```
    rt_size_t     size;           /* size of memory pool data area */

    rt_size_t     block_size;    /* size of memory block  */
    rt_uint8_t    *block_list;   /* list of memory block  */

    /* maximum number of memory blocks that can be accommodated in the memory pool data area  */
    rt_size_t     block_total_count;
    /* number of free memory blocks in the memory pool  */
    rt_size_t     block_free_count;
    /* list of threads suspended because memory blocks are unavailable */
    rt_list_t     suspend_thread;
    /* number of threads suspended because memory blocks are unavailable */
    rt_size_t     suspend_thread_count;
};
typedef struct rt_mempool* rt_mp_t;
```

1.3.7.3.1.2 Memory Block Allocation Mechanism

When the memory pool is created, it first asks for a large amount of memory from the system. Then it divides the memory into multiple small memory blocks of the same size. The small memory blocks are directly connected by a linked list (this linked list is also called a free linked list). At each allocation, the first memory block is taken from the head of the free linked list and provided to the applicant. As you can see from the figure below, there are multiple memory pools of different sizes allowed in physical memory. Each memory pool is composed of multiple free memory blocks, which are used by the kernel for memory management. When a memory pool object is created, the memory pool object is assigned to a memory pool control block. The parameters of the memory control block include the memory pool name, memory buffer, memory block size, number of blocks, and a queue of threads waiting.

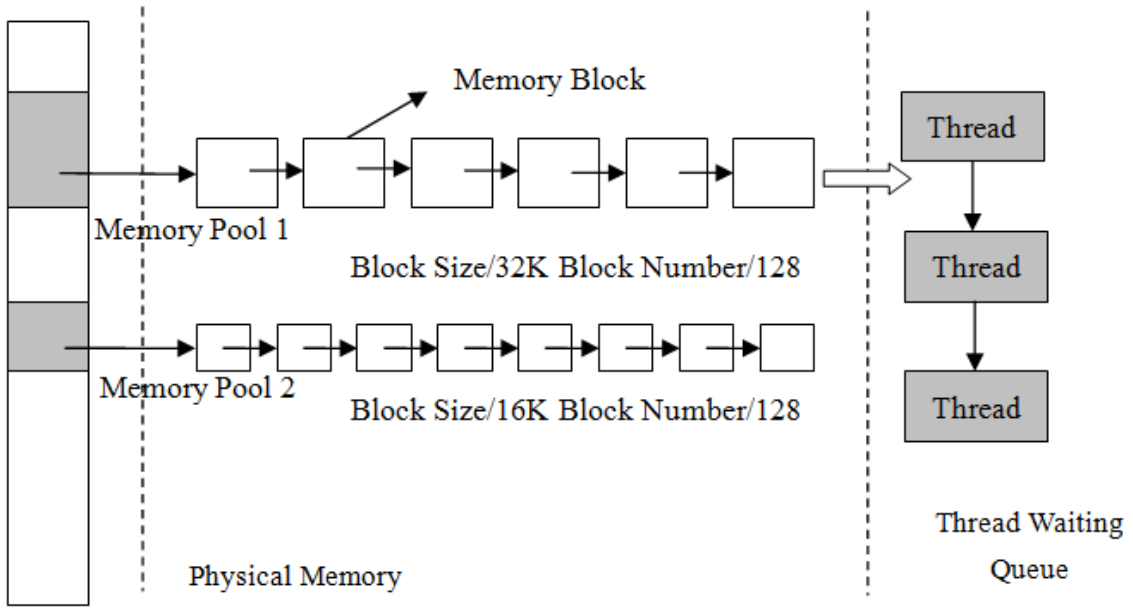


Figure 1.61 Memory Pool Working Mechanism Diagram

The kernel is responsible for allocating memory pool control blocks to the memory pool. It also receives the request for allocation of memory blocks from the user thread. When this information is obtained, the kernel can allocate memory for the memory pool from available memory. Once the memory pool is initialized, the size of the memory blocks inside will no longer be available for adjustment.

Each memory pool object consists of the above structure, where suspend_thread forms a list for thread waiting for memory blocks, that is, when there is no memory block available in the memory pool, and the request thread allows waiting, the thread applying for the memory block will suspend on the suspend_thread linked list.

1.3.7.3.2 Memory Pool Management

The memory pool control block is a structure that contains important parameters related to the memory pool and acts as a link between various states of the memory pool. The related interfaces of the memory pool are as shown in the following figure. The operation of the memory pool includes: creating/initializing the memory pool, appling for memory blocks, releasing memory blocks and deleting/detaching memory pools. It needs to noted that not all memory pools will be deleted. The deletion is relegated to the user, but the used memory blocks should be released.

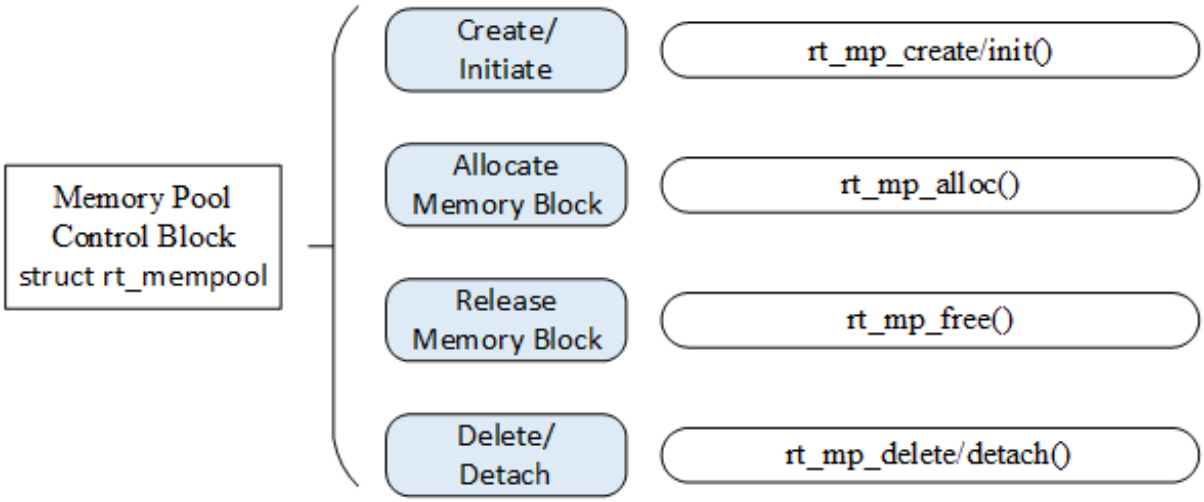


Figure 1.62 Related Interfaces of Memory Pool

1.3.7.3.2.1 Create and Delete Memory Pool

To create a memory pool, a memory pool object is created first and then a memory heap is allocated from the heap. Creating a memory pool is a prerequisite for allocating and releasing memory blocks from the corresponding memory pool. After the memory pool is created, a thread can perform operations like application, release and so on. To create a memory pool, use the following function interface. This function returns a created memory pool object.

```
rt_mp_t rt_mp_create(const char* name,
                     rt_size_t block_count,
                     rt_size_t block_size);
```

Using this function interface can create a memory pool that matches the size and number of memory blocks required. The creation will be successful if system resources allow it (most importantly memory heap resources). When you create a memory pool, you need to give the memory pool a name. The kernel then requests a memory pool object from the system. Next, a memory buffer calculated from the number and sizes of blocks will be allocated from the memory heap. Then the memory pool object is initialized. Afterwards, the successfully applied memory block buffer is organized into the idle linked list used for allocation. The following table describes the input parameters and return values for this function:

Input parameters and return values for rt_mp_create()

Parameters	Description
name	Name of the memory pool
block_count	Number of memory blocks
block_size	Size of memory block
Return	——

Parameters	Description
Handle of memory pool	Creation of memory pool object successful
RT_NULL	Creation of memory pool failed

Deleting the memory pool will delete the memory pool object and release the applied memory. Use the following function interface:

```
rt_err_t rt_mp_delete(rt_mp_t mp);
```

When a memory pool is deleted, all threads waiting on the memory pool object will be awakened (return -RT_↩ ERROR). Then the memory pool data storage area allocated from the memory heap is released, and the memory pool object is deleted. The following table describes the input parameters and return values for this function:

Input parameters and return values of rt_mp_delete()

Parameters	Description
mp	memory pool object handle
Return	——
RT_EOK	Deletion successful

1.3.7.3.2.2 Initialize and Detach Memory Pool

Memory pool initialization is similar to memory pool creation, except that the memory pool initialization is used for static memory management, and the memory pool control block is derived from static objects that the user applies in the system. In addition, unlike memory pool creation, the memory space used by the memory pool object here is a buffer space specified by user. The user passes the pointer of the buffer to the memory pool control block, the rest of the initialization is the same as the creation of the memory pool. The function interface is as follows:

```
rt_err_t rt_mp_init(rt_mp_t mp,
                    const char* name,
                    void *start,
                    rt_size_t size,
                    rt_size_t block size);
```

When initializing the memory pool, the following arguments need to be passed: the memory pool object that needs to be initialized, the memory space used by the memory pool, the number and sizes of memory blocks managed by the memory pool and a name to the memory pool. This way, the kernel can initialize the memory pool and organize the memory space used by the memory pool into a free block linked list for allocation. The following table describes the input parameters and return values for this function:

Input parameters and return values of rt_mp_init()

Parameters	Description
mp	memory pool object
name	memory pool name
start	starting address of memory pool
size	memory pool data area size
block_size	memory pool size
Return	——
RT_EOK	initialization successful
- RT_ERROR	Fail

The number of memory pool blocks = size / (block_size + 4-byte, linked list pointer size), the calculation result needs to be rounded (an integer).

For example, if the size of the memory pool data area is set to 4096 bytes, and the memory block size block_size is set to 80 bytes, then the number of memory blocks applied is 4096/(80+4)=48.

Detaching the memory pool means the memory pool object will be detached from the kernel object manager. Use the following function interface to detach the memory pool:

```
rt_err_t rt_mp_detach(rt_mp_t mp);
```

After using this function interface, the kernel wakes up all threads waiting on the memory pool object and then detaches the memory pool object from the kernel object manager. The following table describes the input parameters and return values for this function:

Input parameters and return values for rt_mp_detach()

Parameters	Description
mp	memory pool object
Return	——
RT_EOK	Successful

1.3.7.3.2.3 Allocate and Release Memory Block

To allocate a memory block from the specified memory pool, use the following interface:

```
void *rt_mp_alloc (rt_mp_t mp, rt_int32_t time);
```

The time parameter means the timeout period for applying for allocation of memory blocks. If there is a memory block available in the memory pool, remove a memory block from the free linked list of the memory pool, reduce the number of free blocks and return this memory block; if there are no free memory blocks in the memory pool, determine the timeout time setting: if the timeout period is set to zero, the empty memory block is immediately returned; if the waiting time is greater than zero, the current thread is suspended on the memory pool object until there is free memory block available in the memory pool, or the waiting time elapses. The following table describes the input parameters and return values for this function:

Input parameters and return values of rt_mp_alloc()

Parameters	Description
mp	Memory pool object
time	Timeout
Return	——
Allocated memory block address	Successful
RT_NULL	Fail

Any memory block must be released after it has been used. Otherwise, memory leaks will occur. The memory block is released using the following interface:

```
void rt_mp_free (void *block);
```

When using the function interface, firstly, the memory pool object of (or belongs to) the memory block will be calculated by the pointer of the memory block that needs to be released. Secondly, the number of available memory blocks of the memory pool object will be increased. Thirdly, the released memory block to the linked list of free memory blocks will be added. Then, it will be determined whether there is a suspended thread on the memory pool

object, if so, the first thread on the suspended thread linked list will be awakened. The following table describes the input parameters for this function:

Input parameters of rt_mp_free()

Parameters	Description
block	memory block pointer

1.3.7.3.3 Memory Pool Application Example

This is a static internal memory pool application routine that creates a static memory pool object and 2 dynamic threads. One thread will try to get the memory block from the memory pool, and the other thread will release the memory block, as shown in the following code:

Memory pool usage example

```
#include <rtthread.h>

static rt_uint8_t *ptr[50];
static rt_uint8_t mempool[4096];
static struct rt_mempool mp;

#define THREAD_PRIORITY      25
#define THREAD_STACK_SIZE    512
#define THREAD_TIMESLICE     5

/* pointer pointing to the thread control block */
static rt_thread_t tid1 = RT_NULL;
static rt_thread_t tid2 = RT_NULL;

/* thread 1 entry */
static void thread1_mp_alloc(void *parameter)
{
    int i;
    for (i = 0 ; i < 50 ; i++)
    {
        if (ptr[i] == RT_NULL)
        {
            /* Trying to apply for a memory block 50 times, when no memory block is available,
                thread 1 suspends, thread 2 runs */
            ptr[i] = rt_mp_alloc(&mp, RT_WAITING_FOREVER);
            if (ptr[i] != RT_NULL)
                rt_kprintf("allocate No.%d\n", i);
        }
    }
}

/* thread 2 entry, thread 2 has a lower priority than Thread 1, so thread 1 should be executed first. */
static void thread2_mp_release(void *parameter)
{
    int i;

    rt_kprintf("thread2 try to release block\n");
    for (i = 0; i < 50 ; i++)
    {
        /* release all successfully allocated memory blocks */
        if (ptr[i] != RT_NULL)
        {
            rt_kprintf("release block %d\n", i);
            rt_mp_free(ptr[i]);
            ptr[i] = RT_NULL;
        }
    }
}

int mempool_sample(void)
{
    int i;
    for (i = 0; i < 50; i ++) ptr[i] = RT_NULL;

    /* initialize the memory pool object */
    rt_mp_init(&mp, "mp1", &mempool[0], sizeof(mempool), 80);

    /* create thread 1: applying for memory pool */
    tid1 = rt_thread_create("thread1", thread1_mp_alloc, RT_NULL,
                            THREAD_STACK_SIZE,
                            THREAD_PRIORITY, THREAD_TIMESLICE);
```

```
    if (tid1 != RT_NULL)
        rt_thread_startup(tid1);


    /* create thread 2: release memory pool */
    tid2 = rt_thread_create("thread2", thread2_mp_release, RT_NULL,
                            THREAD_STACK_SIZE,
                            THREAD_PRIORITY + 1, THREAD_TIMESLICE);
    if (tid2 != RT_NULL)
        rt_thread_startup(tid2);

    return 0;
}

/* export to the msh command list */
MSH_CMD_EXPORT(mempool_sample, mempool sample);
```

The simulation results are as follows:

```
 \ | /
- RT -     Thread Operating System
 / | \     3.1.0 build Aug 24 2018
 2006 - 2018 Copyright by rt-thread team
msh >mempool_sample
msh >allocate No.0
allocate No.1
allocate No.2
allocate No.3
allocate No.4
...
allocate No.46
allocate No.47
thread2 try to release block
release block 0
allocate No.48
release block 1
allocate No.49
release block 2
release block 3
release block 4
release block 5
...
release block 47
release block 48
release block 49
```

This routine initializes 4096/(80+4) = 48 memory blocks when initializing the memory pool object.

1) After thread 1 applies for 48 memory blocks, the memory block has been used up and needs to be released elsewhere to be applied again; but at this time, thread 1 has applied for another one in the same way, because it cannot be allocated, so thread 1 suspends;

2) Thread 2 starts to execute the operation of releasing the memory. When thread 2 releases a memory block, it means there is a memory block that is free. Wake up thread 1 to apply for memory, and then apply again after the application is successful, thread 1 suspends again, and repeats this process;

3) Thread 2 continues to release the remaining memory blocks, release is complete.

1.3.8 Interrupt Management

Interrupts often occur in embedded operating systems. When the CPU is processing a normal task, an external urgent event has occurred, requiring the CPU to suspend the current task to handle the asynchronous event. After the external event has been handled, CPU then returns to the interrupted address to continue working on the previous task. The system that implements this function is called the interrupt system, and the source of the request requesting for the CPU interrupt is called the interrupt source. An interrupt is an exception. An exception is any event that causes the processor to move away from normal operation and execute special code. If it is not processed in time, the system will either encounter an error or face a complete breakdown. So appropriately handling exceptions to avoid errors is a very important part of improving software robustness (stability). The following picture is a simple interrupt diagram.

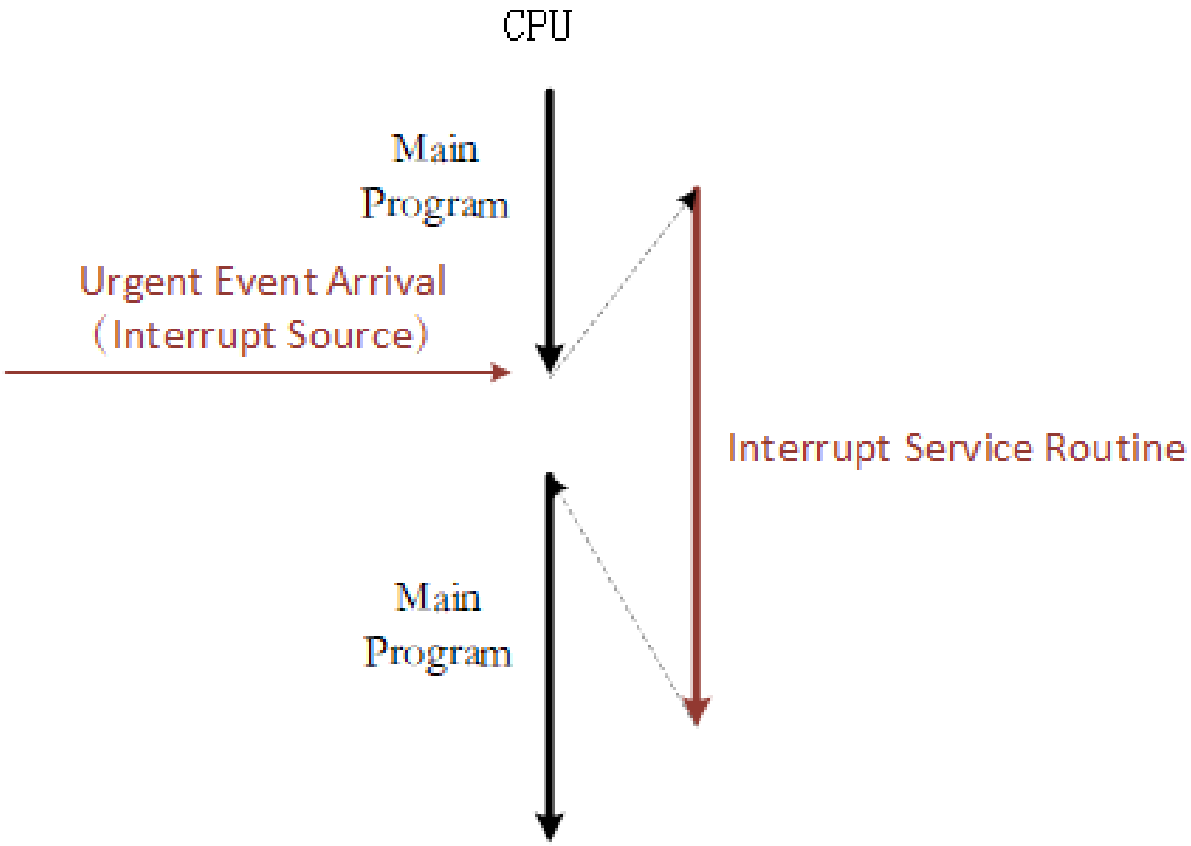


Figure 1.63 Interrupt Diagram

Interrupt processing is closely related to the CPU architecture. Therefore, this chapter first introduces the ARM Cortex-M CPU architecture, and then introduces the RT-Thread interrupt management mechanism in conjunction with the Cortex-M CPU architecture. After reading this chapter, you will learn more about the interrupt handling process of RT-Thread, how to add an interrupt service routine (ISR) and other matters related.

1.3.8.1 Cortex-M CPU Architecture Foundation

Unlike older classic ARM processors (like ARM7, ARM9), the ARM Cortex-M processor has a very different architecture. Cortex-M is serious which Cortex M0/M3/M4/M7 models. There will be some differences between each model. For example, the Cortex-M4 has more floating point calculation functions than the Cortex-M3, but their programming models are basically the same, so the parts of the book that describe interrupt management and porting are not going to be too finely differentiated for the Cortex M0/M3/M4/M7. This section focuses on the architectural aspects related to RT-Thread interrupt management.

1.3.8.1.1 Introduction to Register

The register set of Cortex-M series CPU has 16 general register sets and several special function registers from R0∼R15, as shown in the figure below.

R13 in the general register set is used as the stack pointer register (SP); R14 is used as the link register (LR), which is used to store the return address when the subroutine is called; R15 is used as the program counter (PC) , where the stack pointer register can be either the main stack pointer (MSP) or the process stack pointer (PSP).

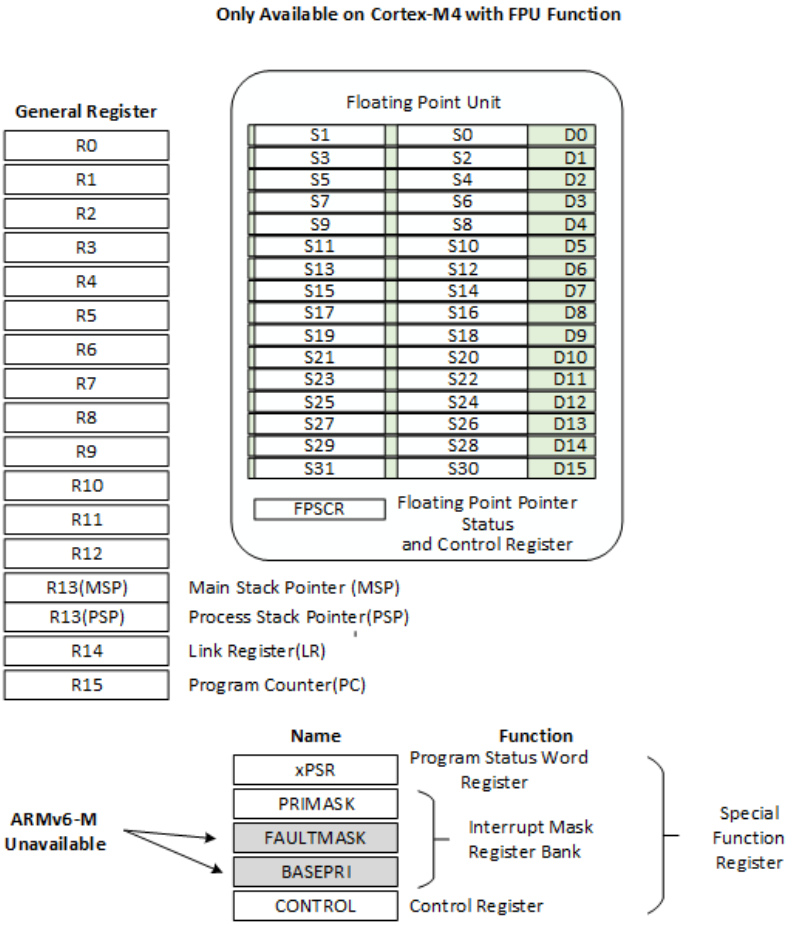


Figure 1.64 Register Schematic

Special function registers include program status word register bank (PSRs), interrupt mask register banks (PRIMASK, FAULTMASK, BASEPRI), and control registers (CONTROL). Special function registers can be accessed through MSR/MRS instructions, such as:

```
MRS R0, CONTROL ; Read CONTROL to R0
MSR CONTROL, R0 ; Write R0 to the CONTROL register
```

The program status word registers stores arithmetic and logic flags, such as negative flags, null result flags, overflow flags, and so on. The interrupt mask register bank controls the Cortex-M interrupt disable. The control registers are used to define the privilege level and decide which stack pointer is to be used.

In the case of a Cortex-M4 or Cortex-M7 with a floating point unit, the control register is also used to indicate whether the floating point unit is currently in use. The floating point unit contains 32 floating point general-purpose registers S0∼S31 and a special FPSCR register (Floating point status and control register).

1.3.8.1.2 Operating Scheme and Privilege Level

Cortex-M introduces the concept of operation scheme and privilege level, which are thread mode and processing mode respectively. If it enters exception or interrupt processing, it enters processing mode, otherwise it is thread mode.

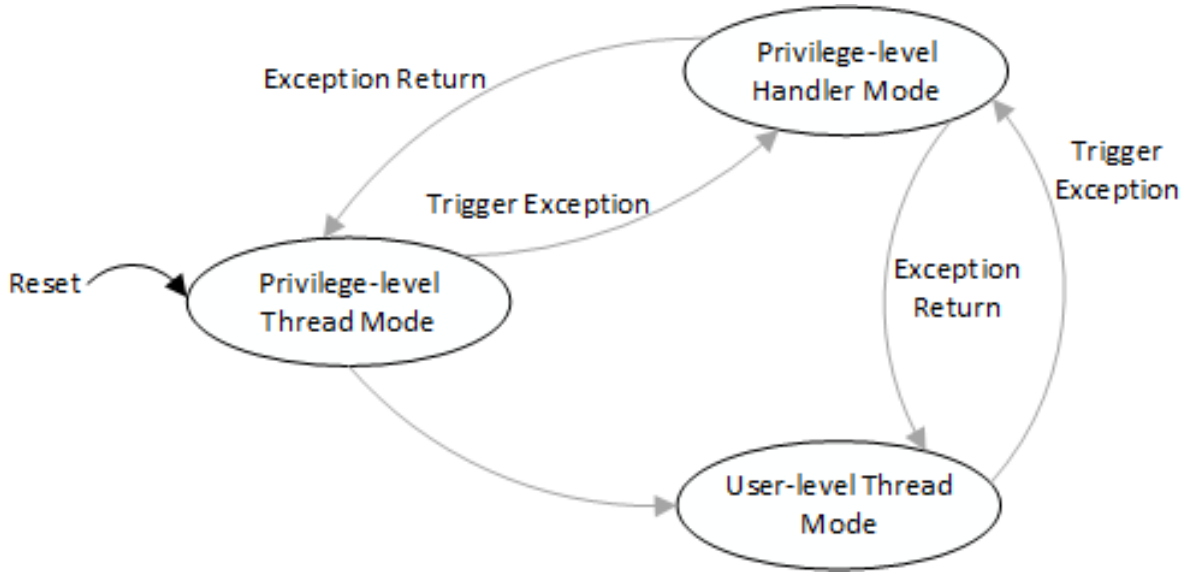


Figure 1.65 Cortex-M Working Mode Switching Diagram

Cortex-M has two running levels, privilege-level and user-level. Thread mode can work at both privilege-level and user-level, while processing mode always works at the privilege-level and can be controlled by the CONTROL special register. The switching of different working modes is as shown in the figure above.

Cortex-M's stack register SP corresponds to two physical registers MSP and PSP, MSP is the main stack, PSP is the process stack. Processing mode always uses MSP as the stack; thread mode can choose to use MSP or PSP as the stack, also controlled through special register CONTROL. After reset, Cortex-M enters thread mode, privilege-level, and uses the MSP stack by default.

1.3.8.1.3 Nested Vector Interrupt Controller

The Cortex-M interrupt controller is called NVIC (nested vectored interrupt controller) and supports interrupt nesting. When an interrupt is triggered and the system responds, the processor hardware automatically pushes the context register of the current location of running into the interrupt stack. The registers in this section include the PSR, PC, LR, R12, and R3-R0 registers.

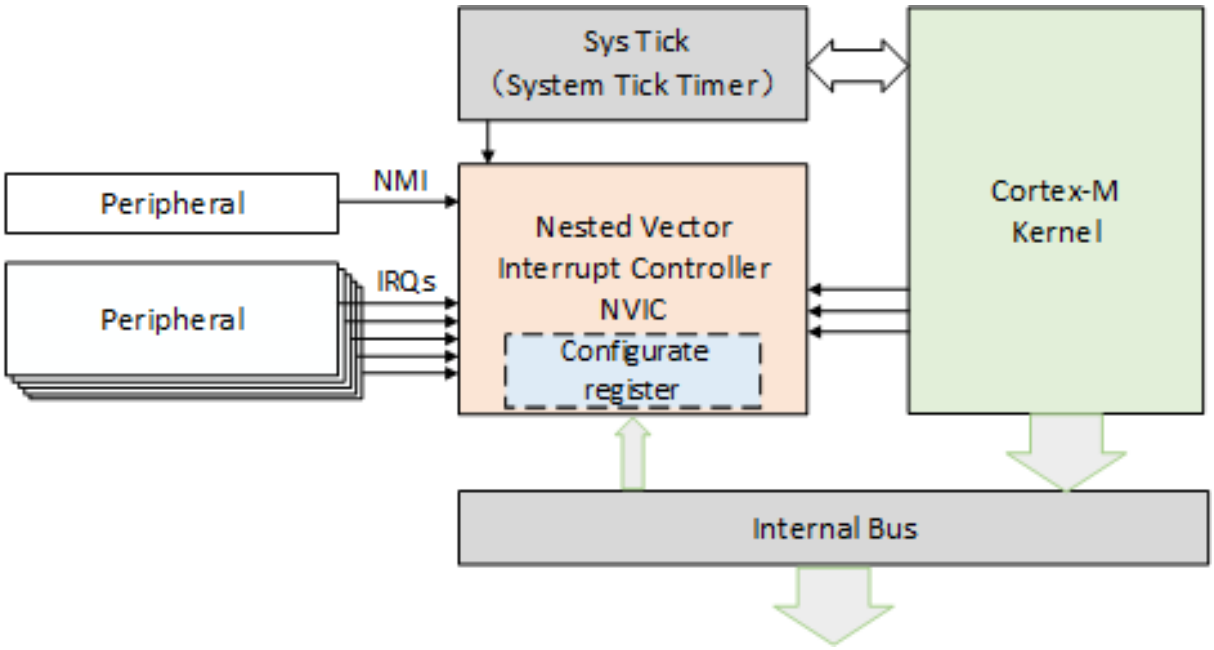


Figure 1.66 Relationship between Cortex-M Kernel and NVIC Diagram

When the system is servicing an interrupt, if a higher priority interrupt is triggered, then the processor will also interrupt the currently running interrupt service routine, and then save the context of the interrupt service program register PSR, PC, LR, R12, R3-R0 to the interrupt stack.

1.3.8.1.4 PendSV System Call

PendSV, also known as a suspendable system call, is an exception that can be suspended like a normal interrupt. It is specifically designed to assist the operating system in context switching. PendSV exceptions are initialized as lowest priority exceptions. Each time a context switch is required, the PendSV exception is triggered manually, and the context switch is performed in the PendSV exception handler. The detailed process of operating system context switching using the PendSV mechanism will be illustrated in the next chapter, *Kernel Porting*.

1.3.8.2 RT-Thread Interruption Mechanism

1.3.8.2.1 Interrupt Vector Table

The interrupt vector table is the entry point for all interrupt handlers. The following figure shows the Cortex-M serious of interrupt handlers: linking a function (user interrupt service routine) to the interrupt vector in a virtual interrupt vector table. When the interrupt vector corresponds to an interrupt, the hooked user interrupt service routine is called.

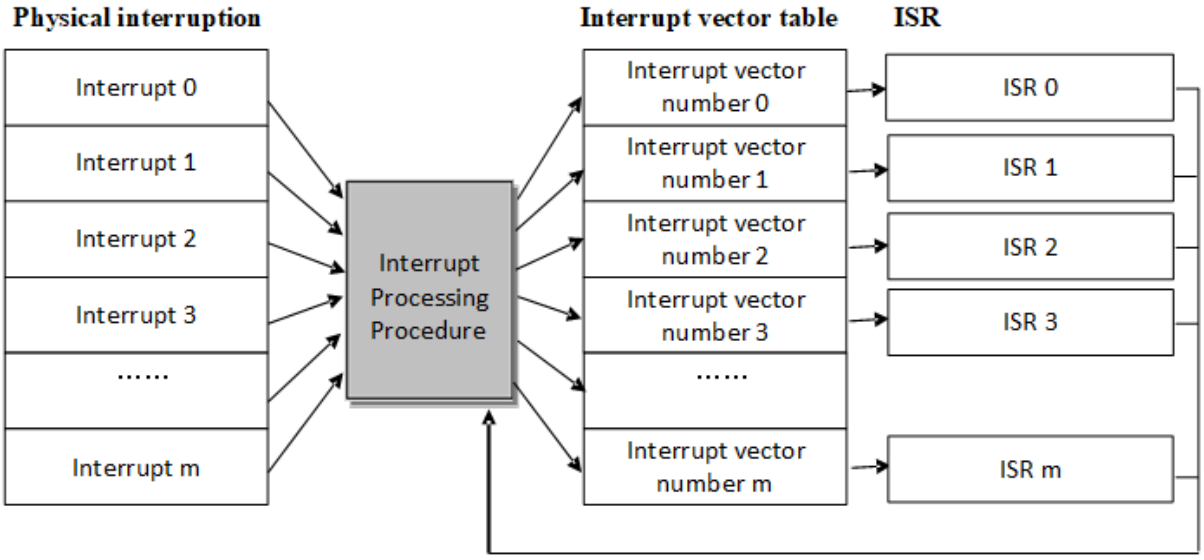


Figure 1.67 Interrupt Processing

On the Cortex-M core, all interrupts are processed using the interrupt vector table which means when an interrupt is triggered, the processor will directly determine which interrupt source it is, and then jump directly to the corresponding fixed location for processing. The interrupt service routines must be placed together at a uniform address (this address must be set to the NVIC interrupt vector offset register). The interrupt vector table is generally defined by an array or given in the start code. Given by the start code is applied by default:

```
__Vectors       DCD     __initial_sp              ; Top of Stack
                DCD     Reset_Handler             ; Reset processing function
                DCD     NMI_Handler               ; NMI processing function
                DCD     HardFault_Handler         ; Hard Fault processing function
                DCD     MemManage_Handler         ; MPU Fault processing function
                DCD     BusFault_Handler          ; Bus Fault processing function
                DCD     UsageFault_Handler        ; Usage Fault processing function
                DCD     0                         ; reserve
                DCD     0                         ; reserve
```

```
                DCD     0                       ; reserve
                DCD     0                       ; reserved
                DCD     SVC_Handler             ; SVCall processing function
                DCD     DebugMon_Handler        ; Debug Monitor processing function
                DCD     0                       ; reserve
                DCD     PendSV_Handler          ; PendSV processing function
                DCD     SysTick_Handler         ; SysTick processing function

... ...

NMI_Handler             PROC
                EXPORT NMI_Handler              [WEAK]
                B       .
                ENDP
HardFault_Handler       PROC
                EXPORT HardFault_Handler        [WEAK]
                B       .
                ENDP
... ...
```

Note the [WEAK] after the code, which is the symbol weakening identifier. The symbols before [WEAK] (such as NMI_Handler, HardFault_Handler) will be weakened if the entire code encounters symbols with the same names (for example, NMI_Handler function with the same name), then the code will use symbols that are not weakened (functions with the same name as NMI_Handler), and the code associated with the weakened symbols will be automatically discarded.

Take the SysTick interrupt as an example. In the system startup code, you need to fill in the SysTick_Handler interrupt entry function, and then implement the function to respond to the SysTick interrupt. The interrupt handler function sample program is as follows:

```
void SysTick_Handler(void)
{
    /* enter interrupt */
    rt_interrupt_enter();

    rt_tick_increase();

    /* leave interrupt */
    rt_interrupt_leave();
}
```

1.3.8.2.2 Interrupt Processing

In RT-Thread interrupt management, interrupt handler is divided into three parts: interrupt preamble, user interrupt service routine, and interrupt follow-up procedure, as shown in the following figure:

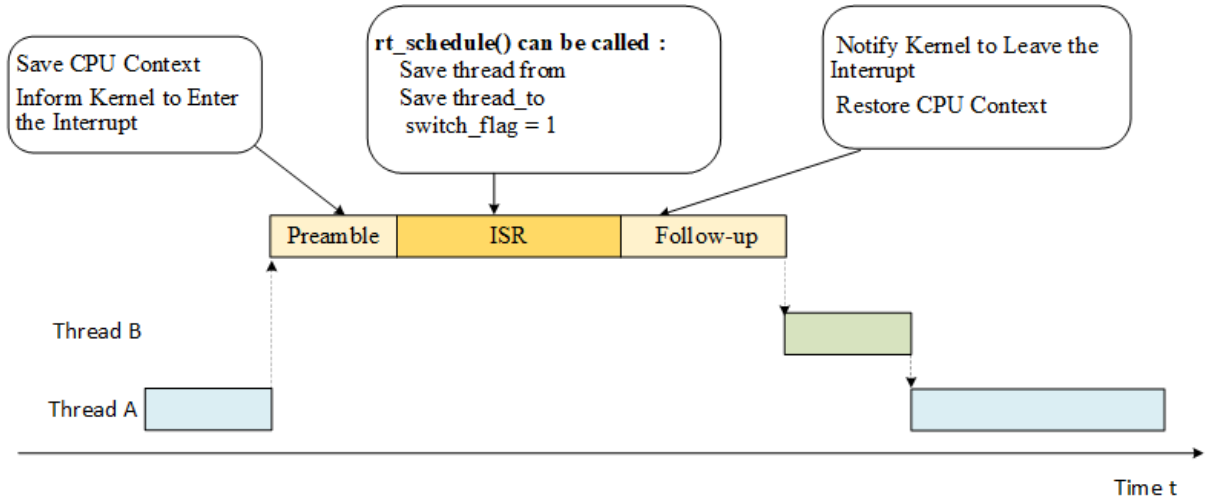


Figure 1.68 3 Parts of the Interrupt Handler

1.3.8.2.2.1 Interrupt Preamble

The main job of interrupt preamble is as follows:

1) Save the CPU interrupt context. This part is related to the CPU architecture. Different CPU architectures are implemented differently.

For Cortex-M, this part of work is done automatically by hardware. When an interrupt is triggered and the system responds, the processor hardware automatically pushes the context register of the currently running portion into the interrupt stack. The registers in this section include the PSR, PC, LR, R12, and R3-R0 registers.

2) Inform the kernel to enter the interrupt state, call the rt_interrupt_enter() function, and add 1 to the global variable rt_interrupt_nest to record the number of levels of interrupt nesting. The code is as follows.

```
void rt_interrupt_enter(void)
{
    rt_base_t level;

    level = rt_hw_interrupt_disable();
    rt_interrupt_nest ++;
    rt_hw_interrupt_enable(level);
}
```

1.3.8.2.2.2 User Interrupt Service Routine

In the user interrupt service routine (ISR), there are two cases. The first case is that no thread switching is performed. In this case, after user interrupt service routine and interrupt subsequent program finished running, it exits and return to the interrupted thread. .

In another case, thread switching is required during interrupt processing. In this case, the rt_hw_context_switch_↩ interrupt() function is called for context switching. This function is related to the CPU architecture, and different CPU architectures are implemented differently.

In Cortex-M architecture, the function implementation of rt_hw_context_switch_interrupt() is shown in the following figure. It sets the thread rt_interrupt_to_thread variable that needs to be switched, and then triggers the PendSV exception (PendSV exception is specifically used to assist context switching and is initialized to the lowest level). After the PendSV exception is triggered, the PendSV exception interrupt handler will not be executed immediately as the interrupt processing is still in progress, the PendSV exception interrupt handler will be entered only after the interrupt subsequent program finishes running and exited the interrupt processing.

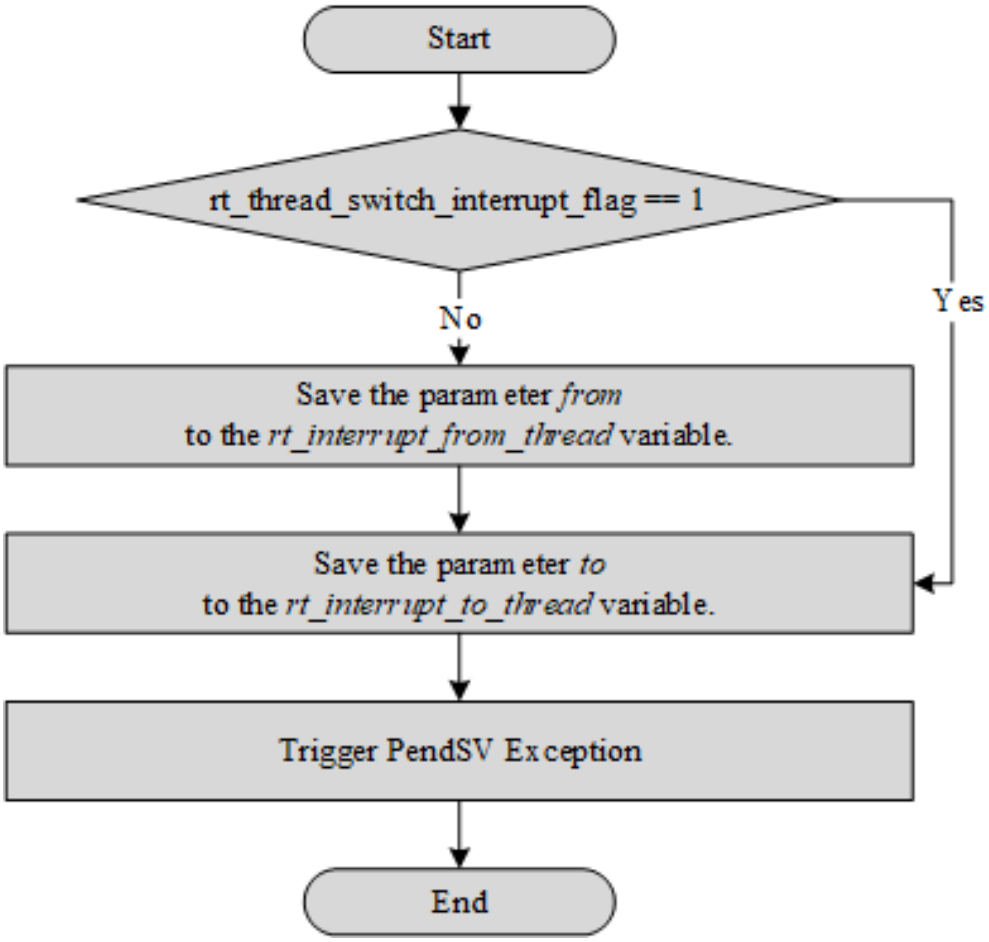


Figure 1.69 Function rt_hw_context_switch_interrupt() Implementation Process

1.3.8.2.2.3 Interrupt Follow-up Procedure

The main work done by interrupt follow-up procedure is:

1 Inform the kernel to leave the interrupt state and reduce the global variable rt_interrupt_nest by 1 through calling the rt_interrupt_leave() function. The code is as follows.

```
void rt_interrupt_leave(void)
{
    rt_base_t level;

    level = rt_hw_interrupt_disable();
    rt_interrupt_nest --;
    rt_hw_interrupt_enable(level);
}
```

2 Restore the CPU context before the interrupt. If thread is not switched during the interrupt processing, the CPU context of the *from* thread is restored. If the thread is switched during the interrupt, the CPU context of the *to* thread is restored. This part of the implementation is related to the CPU architecture. Different CPU architectures are implemented differently. The implementation process in the Cortex-M architecture is shown in the following figure.

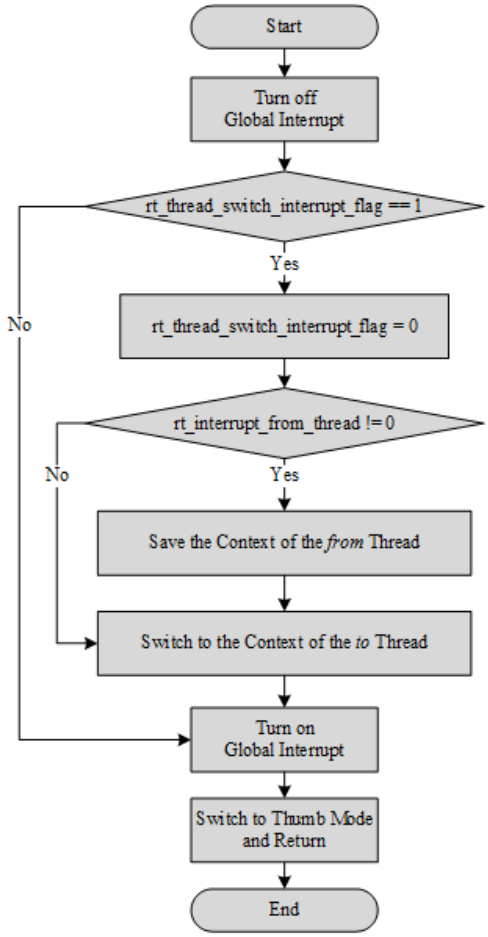


Figure 1.70 Function rt_hw_context_switch_interrupt() Implementation Process

1.3.8.2.3 Interrupt Nesting

In the case of interrupt nesting, in the process of executing the interrupt service routine, if a high priority interrupt occurs, the execution of the current interrupt service routine will be interrupted to execute the interrupt service routine of the high priority interrupt. After the processing of the high priority interrupt is completed, the interrupted interrupt service routine is resumed. If thread scheduling is required, the thread context switch will occur when all interrupt handlers finish running, as shown in the following figure.

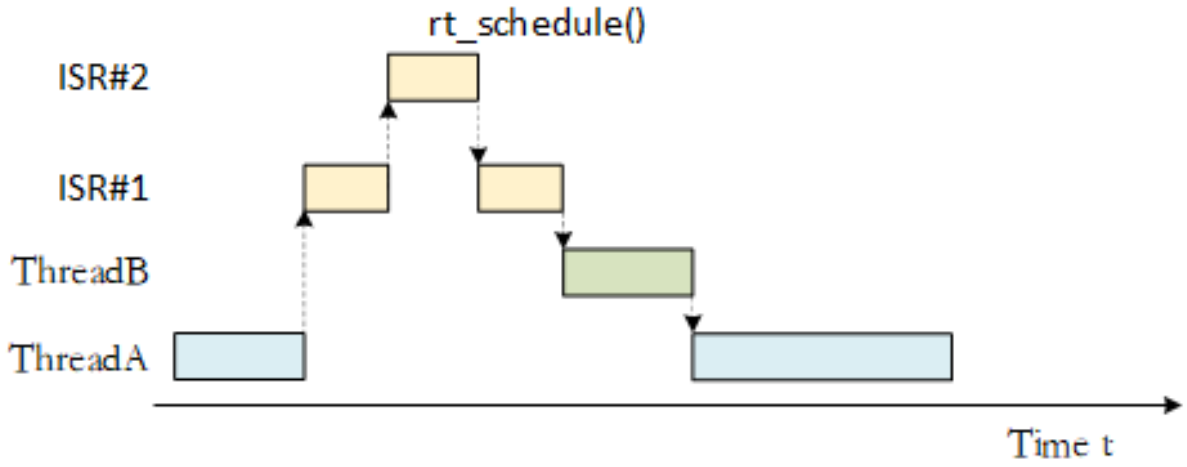


Figure 1.71 Thread Switching during Interrupt

1.3.8.2.4 Interrupt Stack

During the interrupt processing, before the system responds to the interrupt, the software code (or processor) needs to save the context of the current thread (usually stored in the thread stack of the current thread), and then call the interrupt service routine for interrupt response and processing. During interrupt processing (essentially calling the user's interrupt service routine function), the interrupt handler function is likely to have its own local variables, which require the corresponding stack space to save, so the interrupt response still needs a stack space as the context to run the interrupt handler. The interrupt stack can be saved in the stack of the interrupted thread. When exiting from the interrupt, the corresponding thread is resumed to be executed.

The interrupt stack can also be completely separated from the thread stack, that is, when entering the interrupt each time, after the interrupt thread context is saved, it switches to the new interrupt stack and runs independently. When the interrupt exits, the corresponding context is resumed. Using an independent interrupt stack is relatively easy to implement, and it is easier to understand and grasp the thread stack usage (otherwise it must reserve space for the interrupt stack. If the system supports interrupt nesting, you should also consider how much space should be reserved for nested interrupt?).

RT-Thread adopts interrupt stack that provides independence. When an interrupt occurs, the preprocessor of the interrupt will replace the user's stack pointer into the interrupt stack space reserved by the system in advance, and then restore the user's stack when the interrupt exits. This way, the interrupt does not occupy the stack space of the thread, thereby improving the utilization of the memory space, and as the number of threads increases, the effect of reducing the memory footprint is more obvious.

There are two stack pointers in the Cortex-M processor core. One is the main stack pointer (MSP) which is the stack pointer by default. It is used before the first thread and in the interrupt and exception handlers. The other is the thread stack pointer (PSP), used in threads. When the interrupt and exception service routine exits, modify the value of the second bit of LR register as 1, and the SP of the thread is switched from MSP to PSP.

1.3.8.2.5 Processing of the Bottom Half of the Interruption

RT-Thread does not make any assumptions or restrictions on the processing time required by the interrupt service routine, but like other real-time operating systems or non-real-time operating systems, users need to ensure that all interrupt service routines are completed in the shortest possible time (the interrupt service routine is equivalent to having the highest priority in the system and will preempt all threads to execute first). In the process of interrupt nesting or masking the corresponding interrupt source, the other nested interrupt processing and the next interrupt signal of its own interrupt source will not delayed.

When an interrupt occurs, the interrupt service routine needs to obtain the corresponding hardware state or data. If the interrupt service routine is to perform simple processing on the state or data, such as a CPU clock interrupt, the interrupt service routine only needs to add one to the system clock variable and then terminate the interrupt service routine. Such interrupts often require relatively short running time. However, for other interrupts, the interrupt service routine needs to perform a series of more time-consuming processing after obtaining the hardware state or data. Usually, the interrupt is divided into two parts, the **top half** and the **bottom half**. In the top half, after getting the hardware state and data, open the blocked interrupt, send a notification to the relevant thread (which can be the semaphore, event, mailbox or message queue provided by RT-Thread), and then end the interrupt service program. Then, the relevant thread, after receiving the notification, further processes the state or data, this part of the processing is called bottom half processing.

In order to illustrate the implementation of the bottom half processing in RT-Thread, we take a virtual network device receiving network data packets as an example, as shown in the following code. Assume that after receiving the data message, the system analyzes and processes the message is a relatively time consuming process that is much less important than an external interrupt source signal. It can also be processed without masking the interrupt source signal.

The program in this example creates an nwt thread that will block on the nw_bh_sem signal after it starts to run. Once this semaphore is released, the next nw_packet_parser process will be executed to begin the *Bottom Half* event processing.

```
/*
 * program list: interrupt bottom half processing example
 */

/* semaphore used to wake up threads */
rt_sem_t nw_bh_sem;

/* thread for data reading and analysis */
void demo_nw_thread(void *param)
{
    /*First, perform the necessary initialization work on the device. */
    device_init_setting();

    /*.. other operations..*/

    /* create a semaphore to respond to Bottom Half events */
    nw_bh_sem = rt_sem_create("bh_sem", 0, RT_IPC_FLAG_FIFO);

    while(1)
    {
        /* Finally, let demo_nw_thread wait on nw_bh_sem. */
        rt_sem_take(nw_bh_sem, RT_WAITING_FOREVER);

        /* After receiving the semaphore signal, start the real Bottom Half processing. */
        nw_packet_parser (packet_buffer);
        nw_packet_process(packet_buffer);
    }
}

int main(void)
{
    rt_thread_t thread;

    /* create processing thread */
    thread = rt_thread_create("nwt",demo_nw_thread, RT_NULL, 1024, 20, 5);

    if (thread != RT_NULL)
        rt_thread_startup(thread);
}
```

Let's take a look at how Top Half is handled in demo_nw_isr and how Bottom Half is opened, as in the following example.

```
void demo_nw_isr(int vector, void *param)
{
    /* When the network device receives the data, it is met with an interrupt exception and starts executing
      this ISR. */
    /* Start the processing of the Top Half, such as reading the status of the hardware device to determine
      what kind of interruption occurred. */
    nw_device_status_read();

    /*.. Some other data operations, etc. ..*/

    /* Release nw_bh_sem, send a signal to demo_nw_thread, ready to start Bottom Half */
    rt_sem_release(nw_bh_sem);

    /* Then exit the interrupted Top Half section and end the device's ISR */
}
```

As can be seen from the two code snippets of the above example, the interrupt service routine completes the start and end of the interrupt Bottom Half by waiting and releasing a semaphore object. Since the interrupt processing is divided into two parts, Top and Bottom, the interrupt processing becomes an asynchronous process. This part of the system overhead requires the user to seriously consider whether the interrupt service processing time is greater than the time to send notifications to Bottom Half and process when using RT-Thread.

1.3.8.3 RT-Thread Interrupt Management Interface

In order to isolate the operating system from the underlying exceptions and interrupt hardware, RT-Thread encapsulates interrupts and exceptions into a set of abstract interfaces, as shown in the following figure:

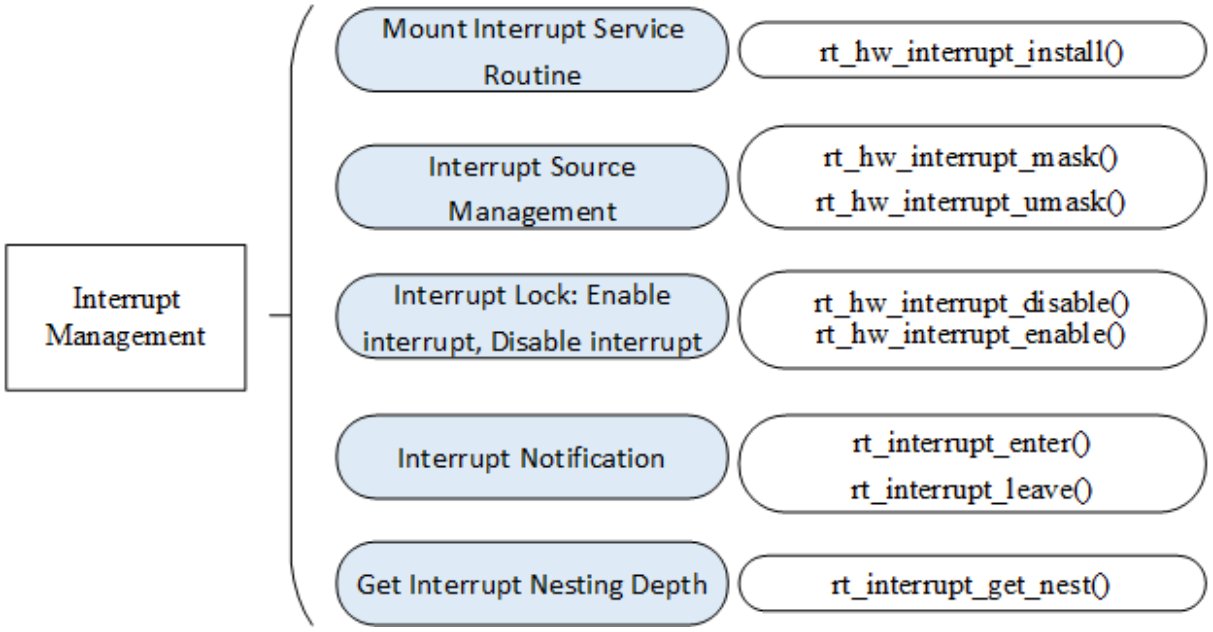


Figure 1.72 Interrupt Related Interfaces

1.3.8.3.1 Mount Interrupt Service Routine

The system associates the user's interrupt handler with the specified interrupt number. You can call the following interface to mount a new interrupt service routine:

```
rt_isr_handler_t rt_hw_interrupt_install(int vector,
                                         rt_isr_handler_t  handler,
                                         void *param,
                                         char *name);
```

After calling rt_hw_interrupt_install(), when the interrupt source generates an interrupt, the system will automatically call the mounted interrupt service routine. The following table describes the input parameters and return values for this function:

Input parameters and return values of rt_hw_interrupt_install()

Parameters	Description
vector	vector is the mounted interrupt number
handler	newly mounted interrupt service routine
param	param is passed as a parameter to the interrupt service routine
name	name of the interrupt
Return	——
return	the handle of the interrupt service routine mounted before the interrupt service routine was mounted

>This API does not appear in every migration branch. For example, there is usually no such API in the migration branch of Cortex-M0/M3/M4.

The interrupt service routine is a kind of runtime environment that requires special attention. It runs in a non-threaded execution environment (generally a special operating mode of the chip (privileged mode)). In this runtime environment, the current thread cannot be suspended because the current thread does not exist. During the execution of related operations, information similar to print prompt information will appear, "Function [abc_func] shall not used in ISR", meaning a function that should not be called in the interrupt service routine.

1.3.8.3.2 Interrupt Source Management

Usually before the ISR is ready to process an interrupt signal, we need to mask the interrupt source and open the previously blocked interrupt source in time after the ISR finishes processing the status or data.

Masking the interrupt source ensures that the hardware state or data will not be disturbed during the following processing. The following function interface can be called:

```
void rt_hw_interrupt_mask(int vector);
```

After the rt_hw_interrupt_mask function interface is called, the corresponding interrupt will be masked (usually when this interrupt is triggered, the interrupt status register will change accordingly, but will not be sent to the processor for processing). The following table describes the input parameters for this function:

Input parameters of rt_hw_interrupt_mask()

Parameters	Description
vector	interrupt number to be masked

>This API does not appear in every migration branch. For example, there is usually no such API in the migration branch of Cortex-M0/M3/M4.

In order to avoid losing the hardware interrupt signal as much as possible, the following function interface can be called to enable the blocked interrupt source:

```
void rt_hw_interrupt_umask(int vector);
```

After the rt_hw_interrupt_umask function interface is called, if the interrupt (and corresponding peripheral) is configured correctly, after the interrupt is triggered, it will be sent to the processor for processing. The following table describes the input parameters for this function:

Input parameters of rt_hw_interrupt_umask()

Parameters	Description
vector	enable the blocked interrupt number

>This API does not appear in every migration branch. For example, there is usually no such API in the migration branch of Cortex-M0/M3/M4.

1.3.8.3.3 Global Interrupt Switch

The global interrupt switch, also known as the interrupt lock, is the easiest way to disable multi-threaded access to critical sections by shutting down the interrupts to ensure that the current thread is not interrupted by other events (because the entire system no longer responds to those external events that could trigger a thread rescheduling), that is, the current thread will not be preempted unless the thread voluntarily gives up control of the processor. When you need to shut off the interrupt of the entire system , you can call the following function interface:

```
rt_base_t rt_hw_interrupt_disable(void);
```

The following table describes the return values for this function:

Return value of rt_hw_interrupt_disable()

Return	Description
Interrupt Status	interrupt status before the function rt_hw_interrupt_disable runs

To resume interrupt can also be understood as turn on an interrupt. The rt_hw_interrupt_enable() function is used to "enable" interrupts, which resumes the interrupt state before the rt_hw_interrupt_disable() function is called. If the interrupt state is turned off before the rt_hw_interrupt_disable() function is called, then the interrupt state is still turned off after calling this function. Resuming interrupts are often used in pairs with turning off interrupts. The function interface called is as follows:

```
void rt_hw_interrupt_enable(rt_base_t level);
```

The following table describes the input parameters for this function:

Input parameters for rt_hw_interrupt_enable()

Parameters	Description
level	the interrupt status returned by the previous rt_hw_interrupt_disable

1) The method of using the interrupt lock to operate the critical section can be applied to any occasion, and other types of synchronization methods are implemented relying on the interrupt lock. It can be said that the interrupt lock is the most powerful and efficient synchronization method. The main problem with using interrupt locks is that the system will no longer respond to any interrupts during the interrupt shutdown and will not be able to respond to external events. Therefore, the impact of the interrupt lock on the real-time system is very large. When used improperly, the system will be completely non-real-time (may cause the system to completely deviate from the required time requirement); when used properly, it will become a fast, efficient synchronization.

For example, to ensure that a line of code (such as assignments) is running mutually exclusively , the quickest way is to use interrupt locks instead of semaphores or mutexes:

```
/* turn off the interrupt */
level = rt_hw_interrupt_disable();
a = a + value;
/* resume interrupt */
rt_hw_interrupt_enable(level);
```

When using an interrupt lock, you need to ensure that the interrupt is turned off for a very short time, such as a = a + value in the above code; you can also switch to another method, such as using semaphores:

```
/* get semaphore lock */
rt_sem_take(sem_lock, RT_WAITING_FOREVER);
a = a + value;
/* release the semaphore lock */
rt_sem_release(sem_lock);
```

In the implementation of rt_sem_take and rt_sem_release, this code already has the behavior of using interrupt locks to protect semaphore internal variables, so for operations such as a = a + value;, it is more concise and fast to use interrupt locks.

2) The function rt_base_t rt_hw_interrupt_disable(void) and the function void rt_hw_interrupt_enable(rt_base_t level) generally need to be used in pairs to ensure correct interrupt status.

In RT-Thread, the API for switching global interrupts supports multi-level nesting. The code for simple nested interrupts is shown in the following code:

Simple nested interrupt use

```
#include <rthw.h>

void global_interrupt_demo(void)
{
    rt_base_t level0;
```

```
    rt_base_t level1;

    /* The global interrupt is turned off for the first time. The global interrupt status before being
      turned off may be turned on or off. */
    level0 = rt_hw_interrupt_disable();
    /* The global interrupt is turned off for the second time. The global interrupt status before being
      turned off may be turned on or off. */
    level1 = rt_hw_interrupt_disable();

    do_something();

    /* Resume the global interrupt to the state before the second turn-off, so the global interrupt is still
      turned off after this enable. */
    rt_hw_interrupt_enable(level1);
    /* Resume the global interrupt to the state before the first turn-off, so the global interrupt status
      can be on or off. */
    rt_hw_interrupt_enable(level0);
}
```

This feature can bring great convenience to the development of the code. For example, if interrupt is turned off in a function, call some sub-functions and then turn on the interrupt. There may also be code for interrupt switch in these subfunctions. Since the API for global interrupts allows the use of nest, users do not need to do special processing for this code.

1.3.8.3.4 Interrupt Notification

When the entire system is interrupted by an interrupt and enters the interrupt handler function, it needs to inform the kernel that it has entered the interrupt state. In this case, the following interfaces can be used:

```
void rt_interrupt_enter(void);
void rt_interrupt_leave(void);
```

These two interfaces are used respectively in the interrupt preamble and interrupt follow-up procedures, and will both modify the values of rt_interrupt_nest (interrupt nesting depth):

Whenever an interrupt is entered, the rt_interrupt_enter() function can be called to notify the kernel that it has entered the interrupt state and increased the interrupt nesting depth (execute rt_interrupt_nest++);

Whenever an interrupt is exited, the rt_interrupt_leave() function can be called to notify the kernel that it has exited the interrupt state and reduced the interrupt nesting depth (execute rt_interrupt_nest–). Be careful not to call these two interface functions in applications.

The role of using rt_interrupt_enter/leave() is that in the interrupt service routine, if a kernel-related function (such as releasing a semaphore) is called, the kernel can be adjusted in time according to the current interrupt status. For example, if a semaphore is released in the interrupt and a thread is awakened, but the current system is found to be in the interrupt context, then Interrupt during thread switching should be implemented during the thread switching, instead of switching immediately .

However, if the interrupt service routine does not call kernel-related functions (release semaphores, etc.), you may not call the rt_interrupt_enter/leave() function at this time.

In the upper application, the rt_interrupt_get_nest() interface is called when the kernel needs to know that it has entered the interrupt state or the currently nested interrupt depth. It will return rt_interrupt_nest. as follows:

```
rt_uint8_t rt_interrupt_get_nest(void);
```

The following table describes the return value of rt_interrupt_get_nest()

Return	Description
0	the current system is not in an interrupt context
1	the current system is in an interrupt context

Return	Description
Bigger Than 1	current interrupt nesting level

1.3.8.4 Interrupt and Polling

When the drive peripheral is working, whether the programming mode is triggered by interrupt mode or polling mode is often the first problem to be considered by the driver developer, and there is a difference between the real-time operating system and the time-sharing operating system when it comes to this problem. Because the polling mode itself adopts the sequential execution mode: corresponding processing is done after finding corresponding event. Therefore, the polling mode is relatively simple and clear in terms of implementation. For example, to write data to the serial port, the program code writes the next data only when the serial controller finishes writing a data (otherwise the data is discarded). The corresponding code can look like this:

```
/* polling mode writes data to the serial port */
    while (size)
    {
        /* Determine if the data in the UART peripheral is sent. */
        while (!(uart->uart_device->SR & USART_FLAG_TXE));
        /* Send the next data when all data has been sent. */
        uart->uart_device->DR = (*ptr & 0x1FF);

        ++ptr; --size;
    }
```

In the real-time system, the polling mode may be very problematic, because in a real-time operating system, when a program is continuously executed (when polling), the thread it is running will always run, and the thread with lower priority will not be running. In a time-sharing system, it will be the opposite. There is almost no difference in priority. You can run this program in one time slice and then run another program on another slice.

So generally, in real-time systems, interrupt mode is mostly used to drive peripherals. When the data has arrived, the relevant processing threads are awoken by the interrupt, and then the subsequent actions are executed . For example, some serial peripherals that carry FIFO (FIFO queue with a certain amount of data) can be written as shown below:

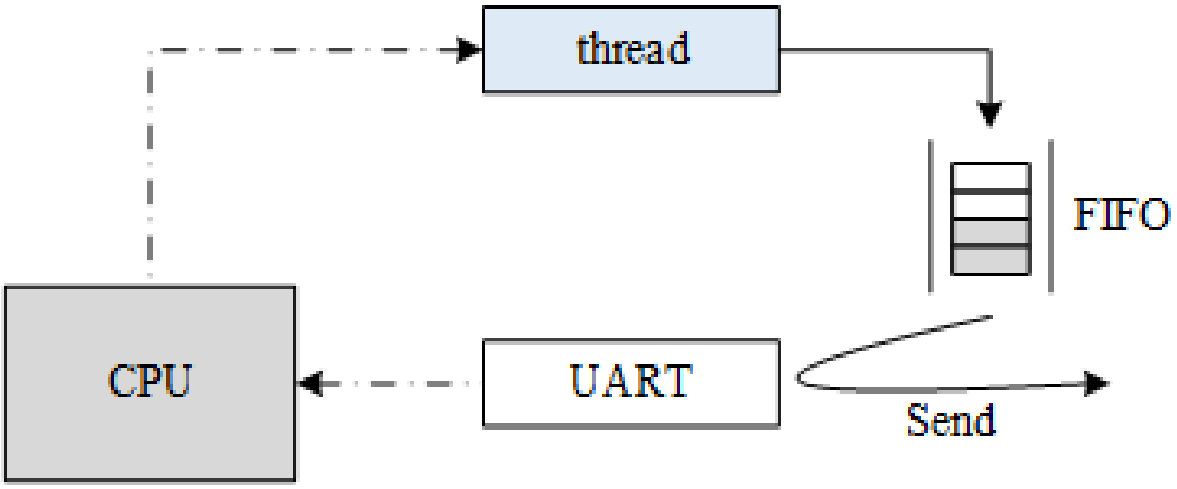


Figure 1.73 Interrupt Mode Drive Peripheral

The thread first writes data to the serial port's FIFO. When the FIFO is full, the thread actively suspends. The serial controller continuously fetches data from the FIFO and sends it out at a configured baud rate (for example, 115200 bps). When all data in the FIFO is sent, an interrupt is triggered to the processor; when the interrupt service routine is executed, the thread can be awaken. Here is an example of a FIFO type device. In reality, there are also DMA type devices with similar principles.

For low-speed devices, this mode is very good because the processor can run other threads before the serial peripheral sends the data in the FIFO which improves the overall operating efficiency of the system. (Even for time-sharing

systems, such a mode is very necessary.) But for some high-speed devices, such as when the transmission speed reaches 10Mbps, assuming that the amount of data sent at one time is 32 bytes, we can calculate the time required to send such a piece of data: (32 X 8) X 1/10Mbps = 25us. When data needs to be transmitted continuously, the system will trigger an interrupt after 25us to wake up the upper thread to continue the next transmission. Suppose the system's thread switching time is 8us, (usually the real-time operating system's thread context switching only takes a few us) then when the entire system is running, the data bandwidth utilization will be only 25/(25+8) = 75.8%. However, with polling mode, the data bandwidth utilization rate may reach 100%. This is also why people generally think that the data throughput in the real-time system is insufficient. The system overhead is consumed in the thread switching. (some real-time systems may even use the bottom half processing and hierarchical interrupt processing as described earlier in this chapter which means the time overhead of interrupting to the sending thread is lengthened and the efficiency will be further reduced).

Through the above calculation process, we can see some of the key factors: the smaller the amount of transmitted data, the faster the transmission speed, and the greater the impact on data throughput. Ultimately, it depends on how often the system generates interrupts. When a real-time system wants to increase data throughput, there are several ways that can be considered:

1) Increase the length of each data volume for each transmission, and try to let the peripherals send as much data as possible every time;

2) Change the interrupt mode to polling mode if necessary. At the same time, in order to solve the problem that the processor is always preempted with polling mode and other low-priority threads cannot be operated, the priority of the polling thread can be lowered accordingly.

1.3.8.5 Global Interrupt Switch Usage Example

This is an interrupted application routine: when multiple threads access the same variable, use the switch global interrupt to protect the variable, as shown in the following code:

Use switch interrupts to access global variables

```
#include <rthw.h>
#include <rtthread.h>

#define THREAD_PRIORITY      20
#define THREAD_STACK_SIZE    512
#define THREAD_TIMESLICE     5

/* global variables accessed simultaneously */
static rt_uint32_t cnt;
void thread_entry(void *parameter)
{
    rt_uint32_t no;
    rt_uint32_t level;

    no = (rt_uint32_t) parameter;
    while (1)
    {
        /* turn off glocal interrupt */
        level = rt_hw_interrupt_disable();
        cnt += no;
        /* resume glocal interrupt */
        rt_hw_interrupt_enable(level);

        rt_kprintf("protect thread[%d]'s counter is %d\n", no, cnt);
        rt_thread_mdelay(no * 10);
    }
}

/* user application entry */
int interrupt_sample(void)
{
    rt_thread_t thread;

    /* create t1 thread */
    thread = rt_thread_create("thread1", thread_entry, (void *)10,
                              THREAD_STACK_SIZE,
                              THREAD_PRIORITY, THREAD_TIMESLICE);
    if (thread != RT_NULL)
```

```
        rt_thread_startup(thread);

    /* create t2 thread */
    thread = rt_thread_create("thread2", thread_entry, (void *)20,
                              THREAD_STACK_SIZE,
                              THREAD_PRIORITY, THREAD_TIMESLICE);
    if (thread != RT_NULL)
        rt_thread_startup(thread);

    return 0;
}

/* export to the msh command list */
MSH_CMD_EXPORT(interrupt_sample, interrupt sample);
```

The simulation results are as follows:

```
 \ | /
- RT -     Thread Operating System
 / | \     3.1.0 build Aug 27 2018
 2006 - 2018 Copyright by rt-thread team
msh >interrupt_sample
msh >protect thread[10]'s counter is 10
protect thread[20]'s counter is 30
protect thread[10]'s counter is 40
protect thread[20]'s counter is 60
protect thread[10]'s counter is 70
protect thread[10]'s counter is 80
protect thread[20]'s counter is 100
protect thread[10]'s counter is 110
protect thread[10]'s counter is 120
protect thread[20]'s counter is 140
...
```

>Since shutting down the global interrupt will cause the entire system to fail to respond to the interrupt, when using the global interrupt as a means of exclusive access to the critical section, it is necessary to ensure that the global interrupt is very short, such as the time to run several machine instructions.

1.3.9 Kernel Porting

After learning the previous chapters, everyone has a better understanding of RT-Thread, but many people are not familiar with how to port the RT-Thread kernel to different hardware platforms. Kernel porting refers to the RT-Thread kernel running on different chip architectures and different boards. It can have functions such as thread management and scheduling, memory management, inter-thread synchronization and communication, and timer management. Porting can be divided into two parts: CPU architecture porting and BSP (Board support package) porting .

This chapter will introduce CPU architecture porting and BSP porting. The CPU architecture porting part will be introduced in conjunction with the Cortex-M CPU architecture. Therefore, it is necessary to review "Cortex-M CPU Architecture Foundation" in the previous chapter Interrupt Management. After reading this chapter, how to complete the RT-Thread kernel porting will be learned.

1.3.9.1 CPU Architecture Porting

There are many different CPU architectures in the embedded world, for example, Cortex-M, ARM920T, MIPS32, RISC-V, etc. In order to enable RT-Thread to run on different CPU architecture chips, RT-Thread provides a libcpu abstraction layer to adapt to different CPU architectures. The libcpu layer provides unified interfaces to the kernel, including global interrupt switches, thread stack initialization, context switching, and more.

RT-Thread's libcpu abstraction layer provides a unified set of CPU architecture porting interfaces downwards. This part of the interface includes global interrupt switch functions, thread context switch functions, clock beat configuration and interrupt functions, Cache, and so on. The following table shows the interfaces and variables that the CPU architecture migration needs to implement.

libcpu porting related API

Functions and Variables	Description
rt_base_t rt_hw_interrupt_disable(void);	disable global interrupt
void rt_hw_interrupt_enable(rt_base_t level);	enable global interrupt
rt_uint8_t *rt_hw_stack_init(void *tentry, void *parameter, rt_uint8_t *stack_addr, void *texit);	The initialization of the thread stack, the kernel will call this function during thread creation and thread initialization.
void rt_hw_context_switch_to(rt_uint32 to);	Context switch without source thread, which is called when the scheduler starts the first thread, and is called in the signal.
void rt_hw_context_switch(rt_uint32 from, rt_uint32 to);	Switch from *from* thread to *to* thread, used for switch between threads.
void rt_hw_context_switch_interrupt(rt_uint32 from, rt_uint32 to);	Switch from *from* thread to *to* thread, used for switch in interrupt.
rt_uint32_t rt_thread_switch_interrupt_flag;	A flag indicating that a switch is needed in the interrupt.
rt_uint32_t rt_interrupt_from_thread, rt_interrupt_to_thread;	Used to save *from* and *to* threads when the thread is context switching.

1.3.9.1.1 Implement Global Interrupt Enable/Disable

Regardless of kernel code or user code, there may be some variables that need to be used in multiple threads or interrupts. If there is no corresponding protection mechanism, it may lead to critical section problems. In order to solve this problem, RT-Thread provides a series of inter-thread synchronization and communication mechanism. But these mechanisms require the global interrupt enable/disable function provided in libcpu. They are, respectively:

```
/* disable global interrupt */
rt_base_t rt_hw_interrupt_disable(void);

/* enable global interrupt */
void rt_hw_interrupt_enable(rt_base_t level);
```

The following describes how to implement these two functions on the Cortex-M architecture. As mentioned earlier, the Cortex-M implements the CPS instruction in order to achieve fast switch interrupts, which can be used here.

```
CPSID I ;PRIMASK=1 ; disable global interrupt
CPSIE I ;PRIMASK=0 ; enable global interrupt
```

1.3.9.1.1.1 Disable Global Interrupt

The functions that need to be done in order in the rt_hw_interrupt_disable() function are:

1). Save the current global interrupt status and use the status as the return value of the function.

2). Disable the global interrupt.

Based on MDK, the global interrupt disabled function on the Cortex-M core, is shown in the following code:

Disable global interrupt

```
;/*
; * rt_base_t rt_hw_interrupt_disable(void);
; */
rt_hw_interrupt_disable    PROC      ;PROC pseudoinstruction definition function
    EXPORT  rt_hw_interrupt_disable  ;EXPORT output defined function, similar to C language extern
    MRS     r0, PRIMASK              ;read the value of the PRIMASK register to the r0 register
    CPSID   I                        ;disable global interrupt
    BX      LR                       ;function renturn
    ENDP                             ;ENDP end of function
```

The above code first uses the MRS instruction to save the value of the PRIMASK register to the r0 register, then disable the global interrupt with the "CPSID I" instruction, and finally returns with the BX instruction. The data stored by r0 is the return value of the function. Interrupts can occur between the “MRS r0, PRIMASK” instruction and “CPSID I”, which does not cause a global interrupt status disorder.

There are different conventions for different CPU architectures regarding how registers are managed during function calls and in interrupt handlers. A more detailed introduction to the use of registers for Cortex-M can be found in the official ARM manual, "∗Procedure Call Standard for the ARM ® Architecture∗."

1.3.9.1.1.2 Enable Global Interrupt

In `rt_hw_interrupt_enable(rt_base_t level)`, the variable *level* is used as the state to be restored, overriding the global interrupt status of the chip.

Based on MDK, implementation on the Cortex-M core enables a global interrupt, as shown in the following code:

Enable global interrupt

```
;/*
; * void rt_hw_interrupt_enable(rt_base_t level);
; */
rt_hw_interrupt_enable    PROC      ; PROC pseudoinstruction definition function
    EXPORT  rt_hw_interrupt_enable  ; EXPORT output defined function, similar to "extern" in C language
    MSR     PRIMASK, r0             ; write the value of the r0 register to the PRIMASK register
    BX      LR                      ; function renturn
    ENDP                            ; ENDP end of function
```

The above code first uses the MSR instruction to write the value register of r0 to the PRIMASK register, thus restoring the previous interrupt status.

1.3.9.1.2 Implement Thread Stack Initialization

When dynamically creating threads and initializing threads, the internal thread initialization function *_rt_thread_init()* is used. The _rt_thread_init() function calls the stack initialization function *rt_hw_stack_init()*, which manually constructs a context in the stack initialization function. The context will be used as the initial value for each thread's first execution. The layout of the context on the stack is shown below:

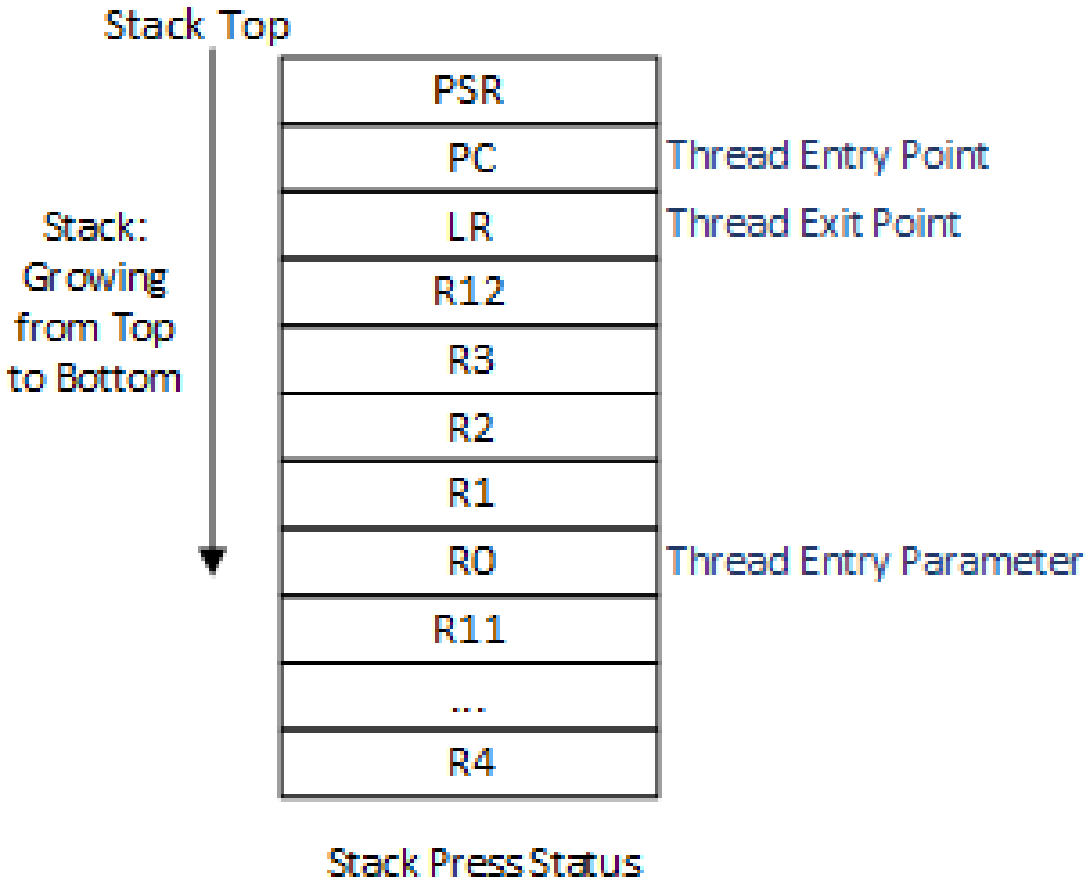


Figure 1.74 Context Information in the Stack

The following code is the stack initialization code:

Build a context on the stack

```
rt_uint8_t *rt_hw_stack_init(void       *tentry,
                             void       *parameter,
                             rt_uint8_t *stack_addr,
                             void       *texit)
{
    struct stack_frame *stack_frame;
    rt_uint8_t         *stk;
    unsigned long       i;

    /* align the incoming stack pointer */
    stk  = stack_addr + sizeof(rt_uint32_t);
    stk  = (rt_uint8_t *)RT_ALIGN_DOWN((rt_uint32_t)stk, 8);
    stk -= sizeof(struct stack_frame);

    /* obtain the pointer to the stack frame of the context */
    stack_frame = (struct stack_frame *)stk;

    /* set the default value of all registers to 0xdeadbeef */
    for (i = 0; i < sizeof(struct stack_frame) / sizeof(rt_uint32_t); i ++)
    {
        ((rt_uint32_t *)stack_frame)[i] = 0xdeadbeef;
    }

    /* save the first parameter in the r0 register according to the ARM APCS calling standard */
    stack_frame->exception_stack_frame.r0  = (unsigned long)parameter;
    /* set the remaining parameter registers to 0 */
    stack_frame->exception_stack_frame.r1  = 0;                 /* r1 register */
    stack_frame->exception_stack_frame.r2  = 0;                 /* r2 register */
    stack_frame->exception_stack_frame.r3  = 0;                 /* r3 register */
    /* set IP (Intra-Procedure-call scratch register.) to 0 */
    stack_frame->exception_stack_frame.r12 = 0;                 /* r12 register */
    /* save the address of the thread exit function in the lr register */
    stack_frame->exception_stack_frame.lr  = (unsigned long)texit;
```

```
    /* save the address of the thread entry function in the pc register */
    stack_frame->exception_stack_frame.pc  = (unsigned long)tentry;
    /* Set the value of psr to 0x01000000L, which means that the default switch is Thumb mode. */
    stack_frame->exception_stack_frame.psr = 0x01000000L;

    /* return the stack address of the current thread       */
    return stk;
}
```

1.3.9.1.3 Implement Context Switching

In different CPU architectures, context switches between threads and context switches from interrupts to context, the register portion of the context may be different or the same. In Cortex-M, context switching is done uniformly using PendSV exceptions, and there is no difference in the switching parts. However, in order to adapt to different CPU architectures, RT-Thread's libcpu abstraction layer still needs to implement three thread switching related functions:

1 rt_hw_context_switch_to(): no source thread, switching to the target thread, which is called when the scheduler starts the first thread.

2 rt_hw_context_switch()In a threaded environment, switch from the current thread to the target thread.

3 rt_hw_context_switch_interrupt ()In the interrupt environment, switch from the current thread to the target thread.

There are differences between switching in a threaded environment and switching in an interrupt environment. In the threaded environment, if the rt_hw_context_switch() function is called, the context switch can be performed immediately; in the interrupt environment, it needs to wait for the interrupt handler to complete processing the functions before switching.

Due to this difference, the implementation of rt_hw_context_switch() and rt_hw_context_switch_interrupt() is not the same on platforms such as ARM9. If the thread's schedule is triggered in the interrupt handler, rt_hw_context↩ _switch_interrupt() is called in the dispatch function to trigger the context switch. After the interrupt handler has processed the interrupt, check the rt_thread_switch_interrupt_flag variable before the schedule exits. If the value of the variable is 1, the context switch of the thread is completed according to the rt_interrupt_from_thread variable and the rt_interrupt_to_thread variable.

In the Cortex-M processor architecture, context switching can be made more compact based on the features of automatic partial push and PendSV.

Context switching between threads, as shown in the following figure:

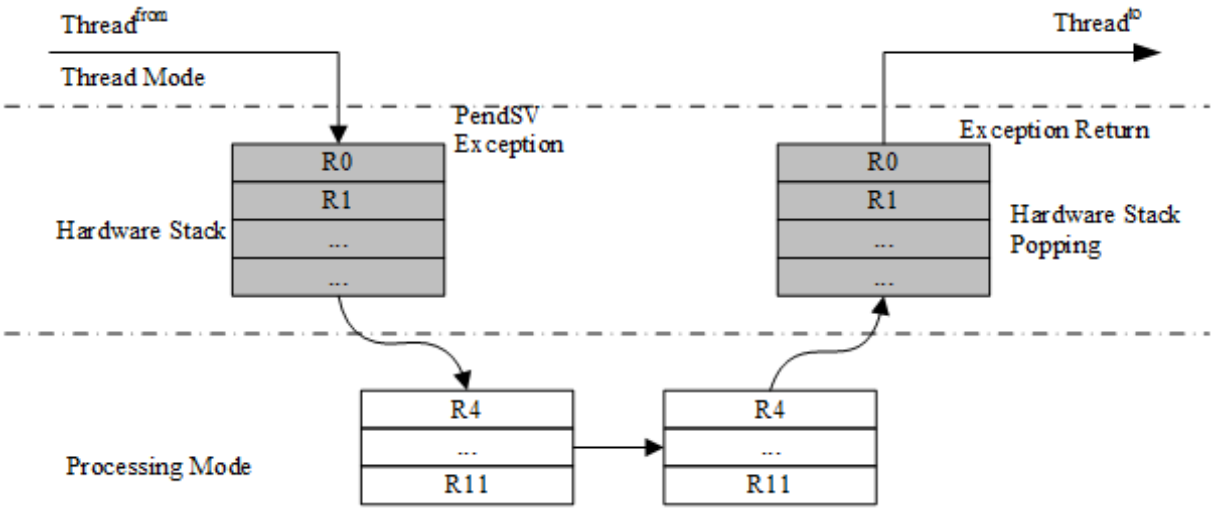


Figure 1.75 Context Switch between Threads

The hardware automatically saves the PSR, PC, LR, R12, R3-R0 registers of the *from* thread before entering the PendSV interrupt, then saves the R11∼R4 registers of the *from* thread in PendSV, and restores the R4∼R11 registers of the *to* thread, and finally the hardware automatically restores the R0∼R3, R12, LR, PC, PSR registers of the *to* thread after exiting the PendSV interrupt.

The context switch from interrupt to thread can be represented by the following figure:

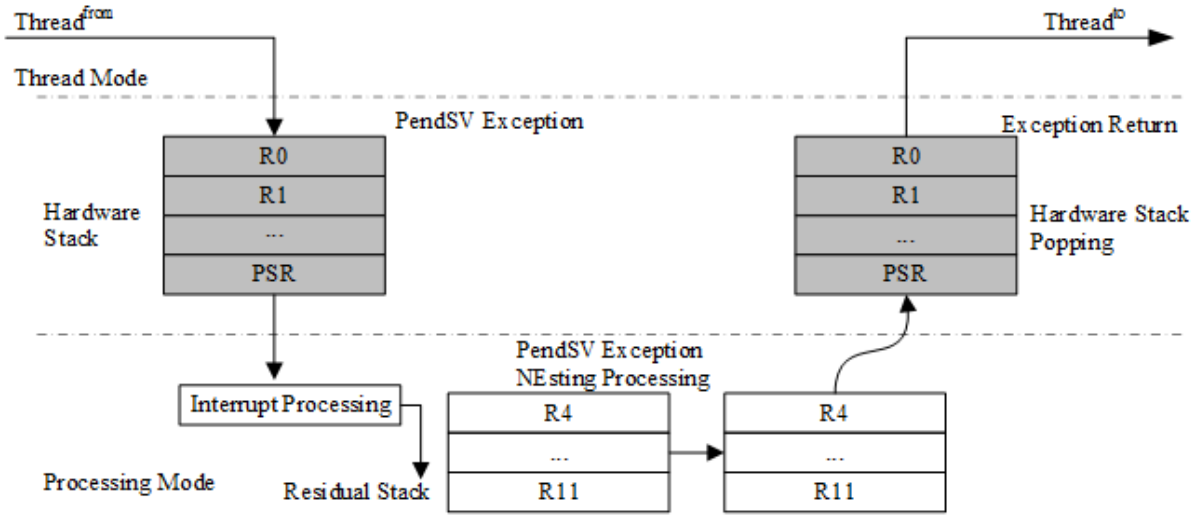


Figure 1.76 Interrupt to Thread Switching

The hardware automatically saves the PSR, PC, LR, R12, R3-R0 registers of the *from* thread before entering the interrupt, and then triggers a PendSV exception. R11∼R4 registers of the *from* thread are saved and R4∼R11 registers of the *to* thread are restored in the PendSV exception handler. Finally, the hardware automatically restores the R0∼R3, R12, PSR, PC, LR registers of the *to* thread after exiting the PendSV interrupt.

Obviously, in the Cortex-M kernel, the rt_hw_context_switch() and rt_hw_context_switch_interrupt() have same functions, which is finishing saving and replying the remaining contexts in PendSV. So we just need to implement a piece of code to simplify the porting.

1.3.9.1.3.1 Implement rt_hw_context_switch_to()

rt_hw_context_switch_to() has only the target thread and no source thread. This function implements the function of switching to the specified thread. The following figure is a flowchart:

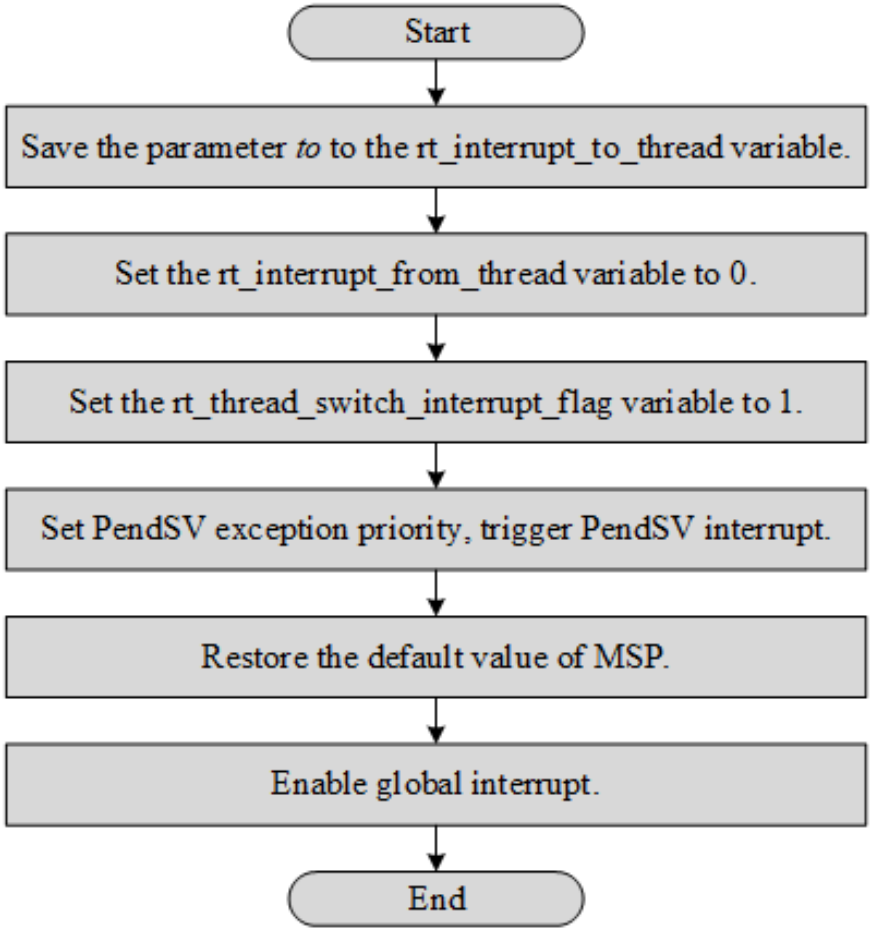


Figure 1.77 rt_hw_context_switch_to() Flowchart

The rt_hw_context_switch_to() implementation on the Cortex-M3 kernel (based on MDK), as shown in the following code:

MDK version rt_hw_context_switch_to() implementation

```
;/*
; * void rt_hw_context_switch_to(rt_uint32 to);
; * r0 --> to
; * this fucntion is used to perform the first thread switch
; */
rt_hw_context_switch_to    PROC
    EXPORT rt_hw_context_switch_to
    ; r0 is a pointer pointing to the SP member of the thread control block of the to thread
    ; save the value of the r0 register to the rt_interrupt_to_thread variable
    LDR     r1, =rt_interrupt_to_thread
    STR     r0, [r1]

    ; set the from thread to empty, indicating that no context is needed to save from
    LDR     r1, =rt_interrupt_from_thread
    MOV     r0, #0x0
    STR     r0, [r1]

    ; set the flag to 1, indicating that switching is required, this variable will be cleared when switching
      in the PendSV exception handler
    LDR     r1, =rt_thread_switch_interrupt_flag
    MOV     r0, #1
    STR     r0, [r1]

    ; set PendSV exception priority to lowest priority
    LDR     r0, =NVIC_SYSPRI2
    LDR     r1, =NVIC_PENDSV_PRI
    LDR.W   r2, [r0,#0x00]       ; read
    ORR     r1,r1,r2             ; modify
    STR     r1, [r0]             ; write-back

    ; trigger PendSV exception (PendSV exception handler will be executed)
    LDR     r0, =NVIC_INT_CTRL
```

```
LDR     r1, =NVIC_PENDSVSET
STR     r1, [r0]

; abandon the stack from chip startup to before the first context switch, set the value of the MSP as
  when it is started
LDR     r0, =SCB_VTOR
LDR     r0, [r0]
LDR     r0, [r0]
MSR     msp, r0

; enable global interrupts and global exceptions. After enabling, the PendSV exception handler will be
  entered.
CPSIE   F
CPSIE   I

; will not execute to here
ENDP
```

1.3.9.1.3.2 Implement rt_hw_context_switch()/ rt_hw_context_switch_interrupt()

The function rt_hw_context_switch() and the function rt_hw_context_switch_interrupt() have two parameters, the *from* thread and the *to* thread. They implement the function to switch from the *from* thread to the *to* thread. The following figure is a specific flow chart:

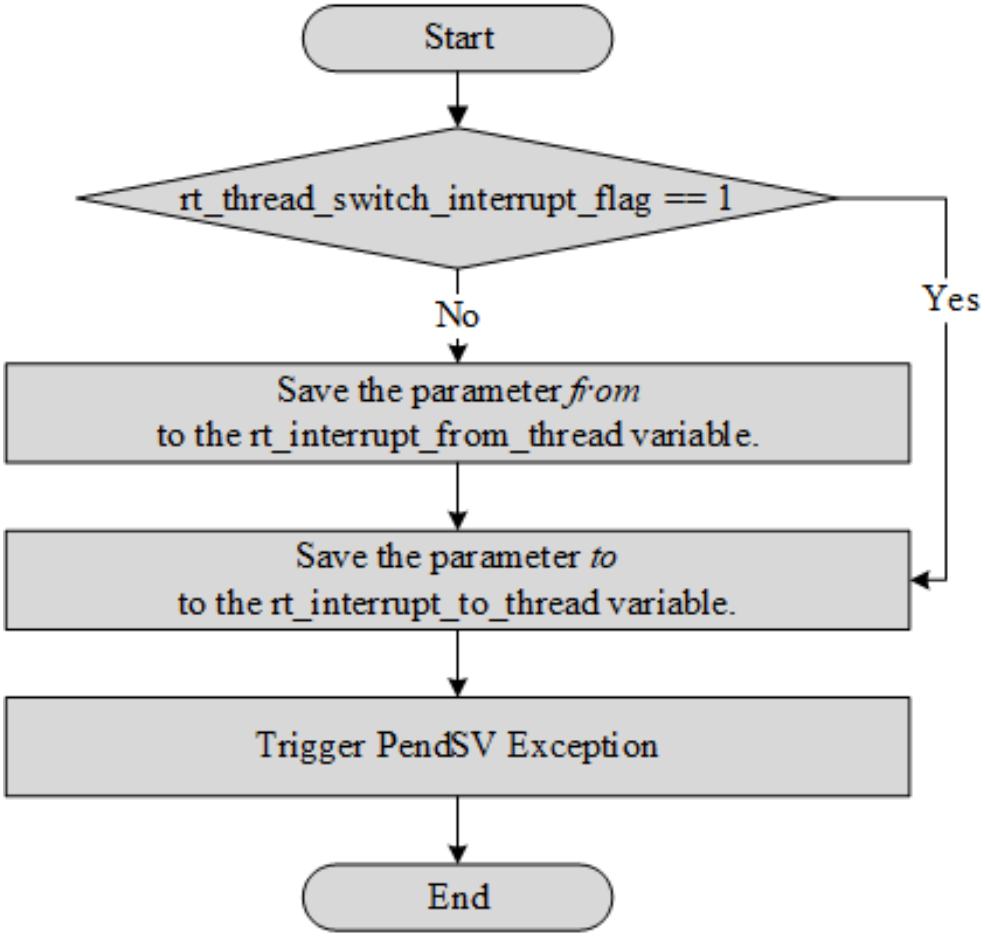


Figure 1.78 Rt_hw_context_switch()/ rt_hw_context_switch_interrupt() Flowchart

The rt_hw_context_switch() and rt_hw_context_switch_interrupt() implementations on the Cortex-M3 kernel (based on MDK) are shown in the following code:

Implement rt_hw_context_switch()/rt_hw_context_switch_interrupt()

```
;/*
```

```
; * void rt_hw_context_switch(rt_uint32 from, rt_uint32 to);
; * r0 --> from
; * r1 --> to
; */
rt_hw_context_switch_interrupt
    EXPORT rt_hw_context_switch_interrupt
rt_hw_context_switch    PROC
    EXPORT rt_hw_context_switch

    ; check if the rt_thread_switch_interrupt_flag variable is 1
    ; skip updating the contents of the thread from if the variable is 1
    LDR     r2, =rt_thread_switch_interrupt_flag
    LDR     r3, [r2]
    CMP     r3, #1
    BEQ     _reswitch
    ; set the rt_thread_switch_interrupt_flag variable to 1
    MOV     r3, #1
    STR     r3, [r2]

    ; update the rt_interrupt_from_thread variable from parameter r0
    LDR     r2, =rt_interrupt_from_thread
    STR     r0, [r2]

_reswitch
    ; update the rt_interrupt_to_thread variable from parameter r1
    LDR     r2, =rt_interrupt_to_thread
    STR     r1, [r2]

    ; trigger PendSV exception, will enter the PendSV exception handler to complete the context switch
    LDR     r0, =NVIC_INT_CTRL
    LDR     r1, =NVIC_PENDSVSET
    STR     r1, [r0]
    BX      LR
```

1.3.9.1.3.3 Implement PendSV Interrupt

In Cortex-M3, the PendSV interrupt handler is PendSV_Handler(). The actual thread switching is done in Pend↩ SV_Handler(). The following figure is a specific flow chart:

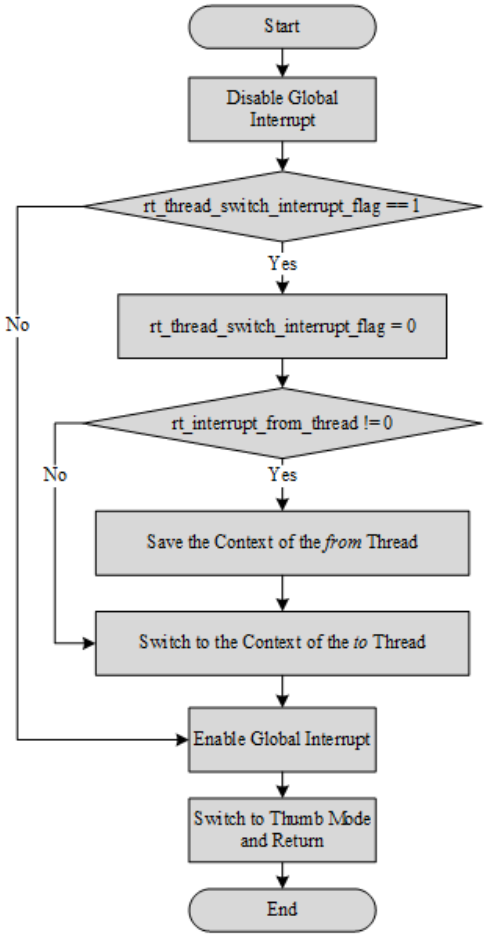


Figure 1.79 PendSV Interrupt Handling

The following code is a PendSV_Handler implementation:

```
; r0 --> switch from thread stack
; r1 --> switch to thread stack
; psr, pc, lr, r12, r3, r2, r1, r0 are pushed into [from] stack
PendSV_Handler   PROC
    EXPORT PendSV_Handler

    ; disable global interrupt
    MRS     r2, PRIMASK
    CPSID   I

    ; check if the rt_thread_switch_interrupt_flag variable is 0
    ; if it is zero, jump to pendsv_exit
    LDR     r0, =rt_thread_switch_interrupt_flag
    LDR     r1, [r0]
    CBZ     r1, pendsv_exit         ; pendsv already handled

    ; clear the rt_thread_switch_interrupt_flag variable
    MOV     r1, #0x00
    STR     r1, [r0]

    ; check the rt_thread_switch_interrupt_flag variable
    ; if it is 0, the context save of the from thread is not performed.
    LDR     r0, =rt_interrupt_from_thread
    LDR     r1, [r0]
    CBZ     r1, switch_to_thread

    ; save the context of the from thread
    MRS     r1, psp                 ; obtain the stack pointer of the thread from
    STMFD   r1!, {r4 - r11}       ; save r4~r11 to the thread's stack
    LDR     r0, [r0]
    STR     r1, [r0]                ; update the SP pointer of the thread's control block

switch_to_thread
    LDR     r1, =rt_interrupt_to_thread
    LDR     r1, [r1]
```

```
    LDR     r1, [r1]                ; obtain the stack pointer of the thread to

    LDMFD   r1!, {r4 - r11}       ; restore the register value of the thread to in the stack of the thread
    MSR     psp, r1                 ; update the value of r1 to psp

pendsv_exit
    ; restore global interrupt status
    MSR     PRIMASK, r2

    ; modify bit 2 of the lr register to ensure that the process uses the PSP stack pointer
    ORR     lr, lr, #0x04
    ; exit interrupt function
    BX      lr
    ENDP
```

1.3.9.1.4 Impalement OS Tick

With the basics of switching global interrupts and context switching, RTOS can perform functions such as creating, running, and scheduling threads. With OS Tick, RT-Thread can schedule time-to-roll rotations for threads of the same priority, implementing timer functions, implementing rt_thread_delay() delay functions, and so on.

In oder to finish the porting of libcpu, we need to ensure that the rt_tick_increase() function is called periodically during the clock tick interrupt. The call cycle depends on the value of the RT_TICK_PER_SECOND macro of rtconfig.h.

In Cortex M, the OS tick can be implemented by SysTick's interrupt handler.

```
void SysTick_Handler(void)
{
    /* enter interrupt */
    rt_interrupt_enter();

    rt_tick_increase();

    /* leave interrupt */
    rt_interrupt_leave();
}
```

1.3.9.2 BSP Porting

In a practical project, for the same CPU architecture, different boards may use the same CPU architecture, carry different peripheral resources, and complete different products, so we also need to adapt to the board. RT-Thread provides a BSP abstraction layer to fit boards commonly seen. If you want to use the RT-Thread kernel on a board, in addition to the need to have the corresponding chip architecture porting, you need to port the corresponding board, that is, implement a basic BSP. The job is to establish a basic environment for the operating system to run. The main tasks that need to be completed are:

1) Initialize the CPU internal registers and set the RAM operation timing.

2) Implement clock driver and interrupt controller driver and interrupt management.

3) Implement UART and GPIO driver.

4) Initialize the dynamic memory heap to implement dynamic heap memory management.

1.3.10 QEMU virt64 AArch64 SMP Boot Flow

1.3.10.1 QEMU virt64 AArch64 SMP Boot Flow

This guide walks through the multi-core boot path of RT-Thread on AArch64 using `bsp/qemu-virt64-aarch64` as the concrete reference. It is written to be beginner-friendly and mirrors the current BSP implementation: from `_start` assembly, early MMU bring-up, `rtthread_startup()`, PSCI wakeup of secondary cores, to all CPUs entering the scheduler. The key transitions are paired with static SVG diagrams so the flow stays readable on GitHub and in generated documentation.

- Target setup: QEMU `-machine virt`, `-cpu cortex-a57`, `-smp >=2`, `RT_USING_SMP` enabled, device tree contains `enable-method = "psci"`.
- Goal: Know who does what, where the code lives, and what to check when SMP does not come up.

1.3.10.1.1 Big Picture First

Figure 1.80 QEMU virt64 AArch64 SMP boot overview

The overview highlights the two key boundaries in this BSP: CPU0 completes the early assembly and common board bring-up, then `rt_hw_secondary_cpu_up()` hands secondary cores over to their own ASM plus C initialization path before everyone meets in the scheduler.

1.3.10.1.2 Boot CPU: from `_start` to MMU on

Input registers: QEMU firmware loads the image and jumps to `_start` at `libcpu/aarch64/cortex-a/entry_point.S`, passing the DTB physical address in `x0` (with `x1`∼`x3` reserved).

What `_start` does (short version)

1. Clear thread pointers: zero `tpidr_el1/tpidrro_el0` to avoid stale per-cpu state.
2. Unify exception level: `init_cpu_el` drops to EL1h, enables timer access, masks unwanted traps.
3. Clear BSS: `init_kernel_bss` fills `__bss` with zeros so globals start clean.
4. Prepare stack: `init_cpu_stack_early` switches to SP_EL1 and uses `.boot_cpu_stack_top` as the early stack.
5. Remember the FDT: `rt_hw_fdt_install_early(x0)` stores DTB address/size before MMU is enabled.
6. Early MMU mapping: `init_mmu_early/enable_mmu_early` build a 0∼1G identity map, set TTBR0/TTBR1 and SCTLR_EL1, flush I/D cache and TLB, then branch to `rtthread_startup()` (address in x8).

   Tip: the early page table only covers minimal kernel space; the C phase will remap a fuller layout.

1.3.10.1.3 C-side startup backbone

`rtthread_startup()` (in `src/components.c`) is the spine of the sequence:

- **Interrupts off + spinlock ready**: `rt_hw_local_irq_disable()` followed by `_cpus_lock` init to keep early steps non-preemptible.
- **Board init**: `rt_hw_board_init()` directly calls the BSP hook `rt_hw_common_setup()` (`libcpu/aarch64/common/setup.c`) to:
  - set VBAR, build kernel address space, copy DTB to a safe region and pre-parse it;
  - configure MMU mappings; init memblock/page allocator/system heap;
  - parse DT for console, memory, initrd;
  - init GIC (and GICv3 Redistributor if enabled), UART, global GTIMER;
  - install SMP IPIs (`RT_SCHEDULE_IPI`, `RT_STOP_IPI`, `RT_SMP_CALL_IPI`) and unmask them;
  - set idle hook `rt_hw_idle_wfi` so idle CPUs enter low-power wait.
- **Kernel subsystems**: init system timer, scheduler, signals, and create main/timer/idle threads.
- **Start scheduling**: `rt_system_scheduler_start()` runs `main_thread_entry()` first.

1.3.10.1.4 How secondary cores are brought up

`main_thread_entry()` calls `rt_hw_secondary_cpu_up()` before invoking user `main()`, so all CPUs join scheduling.

1.3.10.1.4.1 What `rt_hw_secondary_cpu_up()` does

1. Convert `_secondary_cpu_entry` to a physical address via `rt_kmem_v2p()`—the real entry the firmware jumps to.
2. Walk CPU nodes recorded at boot (`cpu_info_init()` stored DTB info in `cpu_np[]` and `rt_cpu_↩ mpidr_table[]`).
3. Read `enable-method`:
   - QEMU virt64: `"psci"` → use `cpu_psci_ops.cpu_boot()` to issue `CPU_ON(target, entry)` to firmware.
   - Legacy compatibility: `"spin-table"` → write `cpu-release-addr` and `sev` to wake.
4. Any failure prints a warning but does not halt the boot flow, making diagnosis easier.

1.3.10.1.4.2 What happens on a secondary core

- **Assembly entry `_secondary_cpu_entry`**:
  - Read `mpidr_el1`, compare with `rt_cpu_mpidr_table` to find the logical CPU id, store it back, and write it into `TPIDR` for per-cpu access.
  - Allocate its own stack by offsetting `ARCH_SECONDARY_CPU_STACK_SIZE` per core.
  - Re-run `init_cpu_el`/`init_cpu_stack_early`, reuse the same early MMU path, then branch to `rt_hw_secondary_cpu_bsp_start()`.
- **C-side handoff `rt_hw_secondary_cpu_bsp_start()`** (`libcpu/aarch64/common/↩ setup.c`):
  - Reset VBAR and synchronize with the boot CPU via `_cpus_lock`.
  - Update this core's MPIDR entry and bind the shared `MMUTable`.
  - Init local vector table, GIC CPU interface (and GICv3 Redistributor if present), enable the local GTIMER.
  - Unmask the three SMP IPIs; re-calibrate `loops_per_tick` for microsecond delay if needed.
  - Call `rt_dm_secondary_cpu_init()` to register the CPU device, then enter the scheduler via `rt_system_scheduler_start()`.

1.3.10.1.4.3 Bring-up timeline

Figure 1.81 QEMU virt64 AArch64 SMP bring-up timeline

Read this figure from top to bottom: CPU0 reaches `rt_system_scheduler_start()` first, `main_↩ thread_entry()` triggers `CPU_ON`, and each secondary CPU repeats the minimal EL plus MMU path before entering `rt_hw_secondary_cpu_bsp_start()`.

1.3.10.1.5 Source map (where to read the code)

Stage	File	Role
Boot assembly	`libcpu/aarch64/cortex-a/entry_point.S`	`_start`, `_secondary_cpu_entry`, early MMU enable
BSP hook	`bsp/qemu-virt64-aarch64/drivers/board.c`	Wires `rt_hw_board_init()` to `rt_hw_common_setup()`
Memory/GIC/IPI init	`libcpu/aarch64/common/setup.c`	`rt_hw_common_setup()`, `rt_hw_secondary_cpu_up()`, `rt_hw_secondary_cpu_bsp_start()`
C entry skeleton	`src/components.c`	`rtthread_startup()`, `main_thread_entry()`

1.3.10.1.6 Quick checks when SMP fails to come up

- Device tree: contains `enable-method = "psci"` and QEMU is started with `-machine virt` (PSCI firmware included).
- `_secondary_cpu_entry` physical address: `rt_kmem_v2p()` must not return 0, otherwise a check fails.
- Init order: GIC/Timer must be ready before calling `rt_hw_secondary_cpu_up()`; if you fork a custom BSP, do these first.
- UART logs: look for `Call cpu X on success/failed`; add extra prints in `_secondary_cpu_entry` if needed, and use QEMU `-d cpu_reset -smp N` to debug.

1.3.10.1.7 AArch64 pocket notes (just enough)

- **Exception levels**: startup may be at EL3/EL2; `init_cpu_el` descends to EL1h where the kernel runs.
- **Two stack pointers**: `spsel #1` selects `SP_EL1` so user mode cannot touch the kernel stack.
- **MMU bring-up order**: build page tables → configure TCR/TTBR → flush cache/TLB → set `SCTLR_EL1.M/C/I` → `isb`.
- **MPIDR**: unique core affinity; stored in `rt_cpu_mpidr_table[]` to map logical CPU ids and IPI targets.

With these in place, the QEMU virt64 AArch64 BSP SMP path is clear: the boot CPU prepares memory and shared peripherals, `main_thread_entry()` issues PSCI wakeups, secondary cores land with the same MMU/EL setup, and all CPUs join the scheduler.

1.4 Components

- Virtual File System
- Device Driver
- FinSH Console
- utest Framework
- Dynamic Module: dlmodule

- Socket Abstraction Layer: SAL
- AT Commands
- POSIX Interface
- Ulog Log
- Power Management: PM
- Network Framework

1.4.1 Virtual File System

In early days, the amount of data to be stored in embedded systems was relatively small and data types were relatively simple. The data were stored by directly writing to a specific address in storage devices. However, with today modern technology, embedded device's functions are getting complicated and required more data storage. Therefore, we need new data management methods to simplify and organize the data storage.

A file system is made up of abstract data types and also a mechanism for providing data access, retrieve, implements, and store them in hierarchical structure. A folder contains multiple files and a file contains multiple organized data on the file system. This chapter explains about the RT-Thread file system, architecture, features and usage of virtual file system in RT-Thread OS.

1.4.1.1 An Introduction to DFS

Device File System (DFS) is a virtual file system component and name structure is similar to UNIX files and folders. Following is the files and folders structure:

The root directory is represented by "/". For example, if users want to access to f1.bin file under root directory, it can be accessed by "/f1.bin". If users want to access to f1.bin file under /2019 folder, it can be accessed by "/data/2019/f1.bin" according to their folder paths as in UNIX/Linux unlike Windows System.

1.4.1.1.1 The Architecture of DFS

The main features of the RT-Thread DFS component are:

- Provides a unified POSIX file and directory operations interface for applications: read, write, poll/select, and more.
- Supports multiple types of file systems, such as FatFS, RomFS, DevFS, etc., and provides management of common files, device files, and network file descriptors.
- Supports multiple types of storage devices such as SD Card, SPI Flash, Nand Flash, etc.

The hierarchical structure of DFS is shown in the following figure, which is mainly divided into POSIX interface layer, virtual file system layer and device abstraction layer.

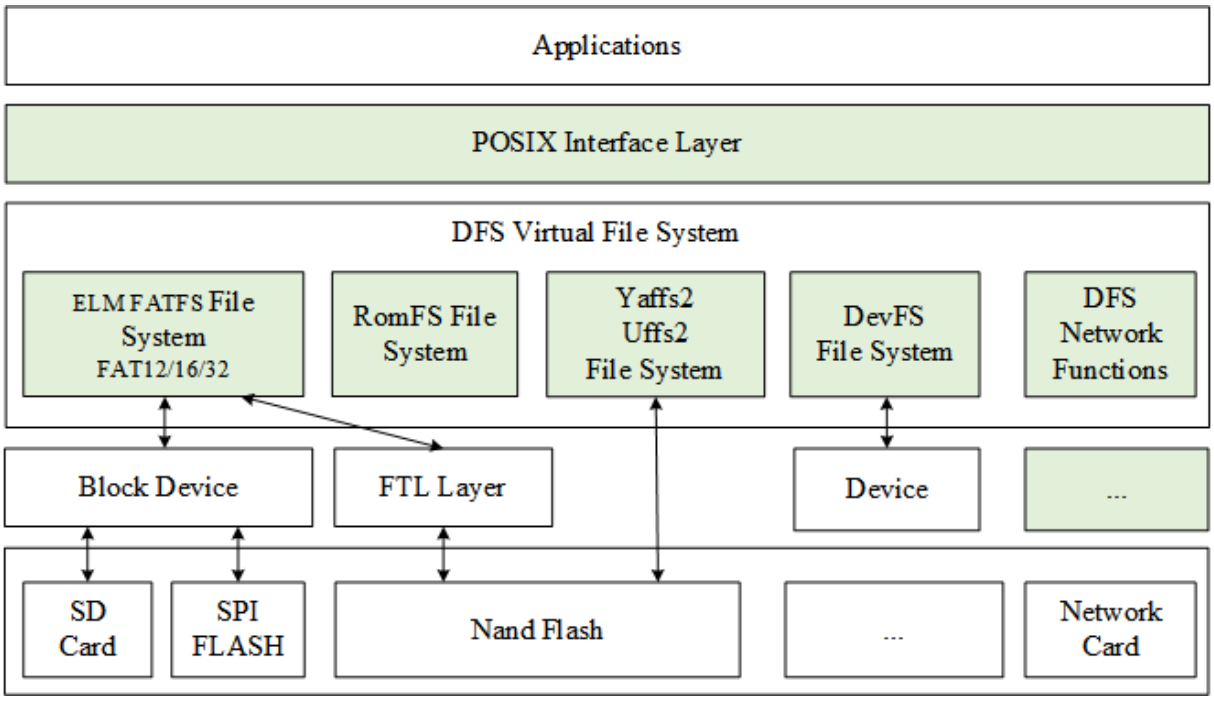


Figure 1.82 The hierarchical structure of DFS

1.4.1.1.2 POSIX Interface Layer

POSIX stands for Portable Operating System Interface of UNIX (POSIX). The POSIX standard defines the interface standard that the operating system should provide for applications. It is a general term for a series of API standards defined by IEEE for software to run on various UNIX operating systems.

The POSIX standard is intended to achieve software portability at the source code level. In other words, a program written for a POSIX-compatible operating system should be able to compile and execute on any other POSIX operating system (even from another vendor). RT-Thread supports the POSIX standard interface, so it is easy to port Linux/Unix programs to the RT-Thread operating system.

On UNIX-like systems, normal files, device files, and network file descriptors are the same. In the RT-Thread operating system, DFS is used to achieve this uniformity. With the uniformity of such file descriptors, we can use the `poll/select` interface to uniformly poll these descriptors and bring convenience to the implement of the program functions.

Using the `poll/select` interface to block and simultaneously detect whether a group of I/O devices which support non-blocking have events (such as readable, writable, high-priority error output, errors, etc.) until a device trigger the event was or exceed the specified wait time. This mechanism can help callers find devices that are currently ready, reducing the complexity of programming.

1.4.1.1.3 Virtual File System Layer

Users can register specific file systems to DFS, such as FatFS, RomFS, DevFS, etc. Here are some common file system types:

- FatFS is a Microsoft FAT format compatible file system developed for small embedded devices. It is written in ANSI C and has good hardware independence and portability. It is the most commonly used file system type in RT-Thread.

- The traditional RomFS file system is a simple, compact, read-only file system that does not support dynamic erasing and saving or storing data in order, thus it supports applications to run in XIP (execute In Place) method and save RAM space while the system is running.
- The Jffs2 file system is a log flash file system. It is mainly used for NOR flash memory, based on MTD driver layer, featuring: readable and writable, supporting data compression, Hash table based log file system, and providing crash/power failure security protection, write balance support, etc..
- DevFS is the device file system. After the function is enabled in the RT-Thread operating system, the devices in the system can be virtualized into files in the `/dev` folder, so that the device can use the interfaces such as `read` and `write` according to the operation mode of the file to operate.
- NFS (Network File System) is a technology for sharing files over a network between different machines and different operating systems. In the development and debugging phase of the operating system, this technology can be used to build an NFS-based root file system on the host and mount it on the embedded device, which can easily modify the contents of the root file system.
- UFFS is short for Ultra-low-cost Flash File System. It is an open source file system developed by Chinese people and used for running Nand Flash in small memory environments such as embedded devices. Compared with the Yaffs file system which often used in embedded devices, it has the advantages of less resource consumption, faster startup speed and free.

1.4.1.1.4 Device Abstraction Layer

The device abstraction layer abstracts physical devices such as SD Card, SPI Flash, and Nand Flash into devices that are accessible to the file system. For example, the FAT file system requires that the storage device be a block device type.

Different file system types are implemented independently of the storage device driver, so the file system function can be correctly used after the drive interface of the underlying storage device is docked with the file system.

1.4.1.2 Mount Management

The initialization process of the file system is generally divided into the following steps:

1. Initialize the DFS component.
2. Initialize a specific type of file system.
3. Create a block device on the memory.
4. Format the block device.
5. Mount the block device to the DFS directory.
6. When the file system is no longer in use, you can unmount it.

1.4.1.2.1 Initialize the DFS Component

The initialization of the DFS component is done by the dfs_init() function. The dfs_init() function initializes the relevant resources required by DFS and creates key data structures that allow DFS to find a specific file system in the system and get a way to manipulate files within a particular storage device. This function will be called automatically if auto-initialization is turned on (enabled by default).

1.4.1.2.2 Registered File System

After the DFS component is initialized, you also need to initialize the specific type of file system used, that is, register a specific type of file system into DFS. The interface to register the file system is as follows:

```
int dfs_register(const struct dfs_filesystem_ops *ops);
```

Parameter	Description
ops	a collection of operation functions of the file system
return	——
0	file registered successfully
-1	file fail to register

This function does not require user calls, it will be called by the initialization function of different file systems, such as the elm-FAT file system's initialization function `elm_init()`. After the corresponding file system is enabled, if automatic initialization is enabled (enabled by default), the file system initialization function will also be called automatically.

The `elm_init()` function initializes the elm-FAT file system, which calls the `dfs_register()` function to register the elm-FAT file system with DFS. The file system registration process is shown below:

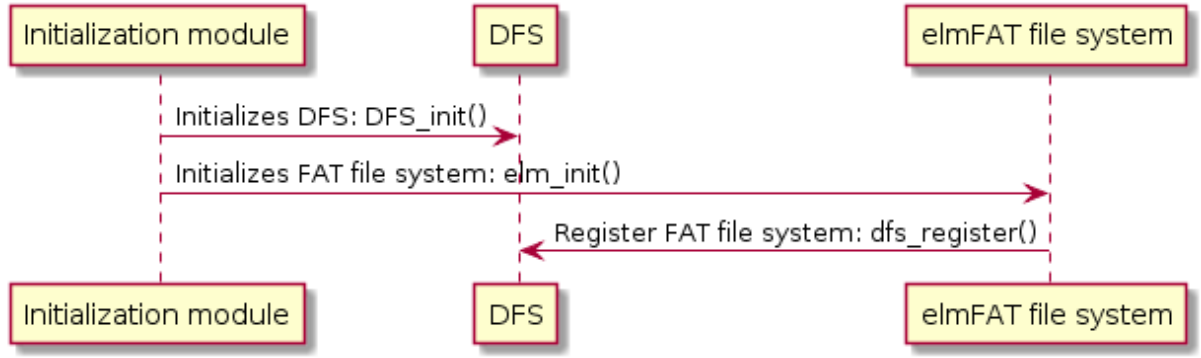


Figure 1.83 Register file system

1.4.1.2.3 Register a Storage Device as a Block Device

Only block devices can be mounted to the file system, so you need to create the required block devices on the storage device. If the storage device is SPI Flash, you can use the "Serial Flash Universal Driver Library SFUD" component, which provides various SPI Flash drivers, and abstracts the SPI Flash into a block device for mounting. The process of registering block device is shown as follows:

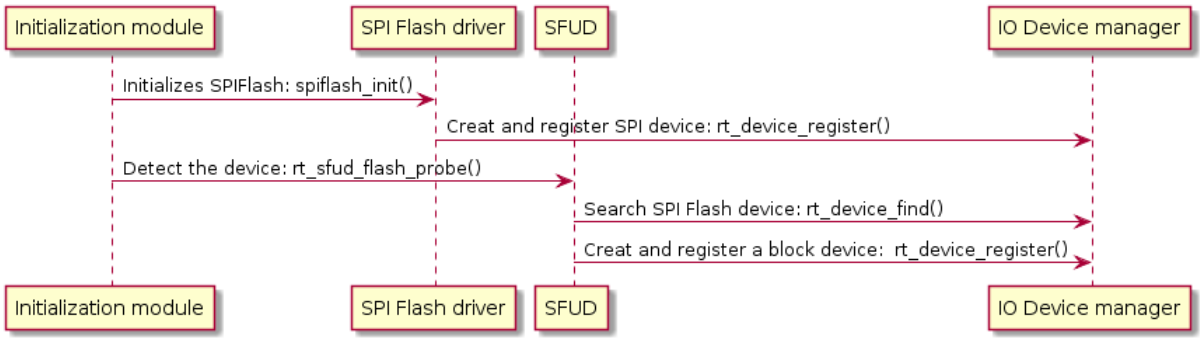


Figure 1.84 The timing diagram of registering block device

1.4.1.2.4 Format the file system

After registering a block device, you also need to create a file system of the specified type on the block device, that is, format the file system. You can use the `dfs_mkfs()` function to format the specified storage device and create a file system. The interface to format the file system is as follows

```
int dfs_mkfs(const char * fs_name, const char * device_name);
```

Parameter	Description
fs_name	type of the file system
device_name	name of the block device
return	——
0	file system formatted successfully
-1	fail to format the file system

The file system type (fs_name) possible values and the corresponding file system is shown in the following table:

Value	File System Type
elm	elm-FAT file system
jffs2	jffs2 journaling flash file system
nfs	NFS network file system
ram	RamFS file system
rom	RomFS read-only file system
uffs	uffs file system

Take the elm-FAT file system format block device as an example. The formatting process is as follows:

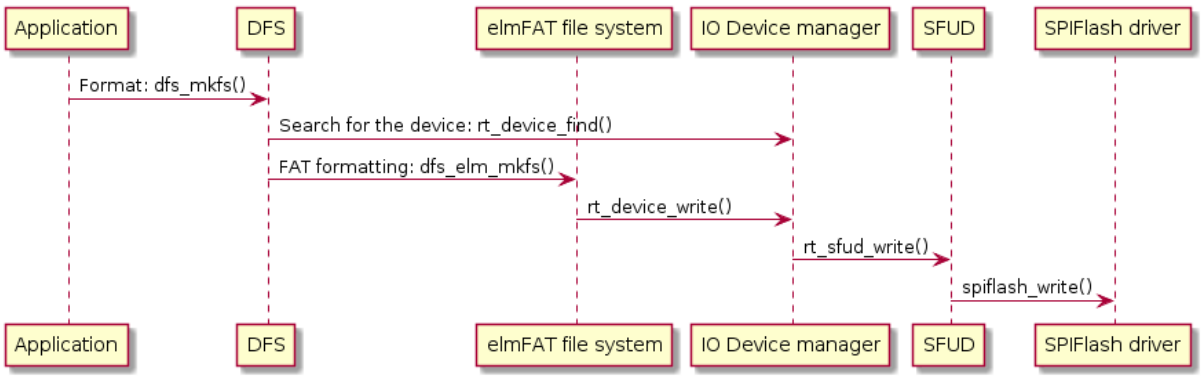


Figure 1.85 Formatted file system

You can also format the file system using the `mkfs` command. The result of formatting the block device sd0 is as follows:

```
msh />mkfs sd0                  # Sd0 is the name of the block device, the command will format by default
sd0 is elm-FAT file system
msh />
msh />mkfs -t elm sd0           # Use the -t parameter to specify the file system type as elm-FAT file
      system
```

1.4.1.2.5 Mount file system

In RT-Thread, mounting refers to attaching a storage device to an existing path. To access a file on a storage device, we must mount the partition where the file is located to an existing path and then access the storage device through this path. The interface to mount the file system is as follows:

```
int dfs_mount(const char   *device_name,
              const char   *path,
              const char   *filesystemtype,
              unsigned long rwflag,
              const void   *data);
```

Parameter	Description
device_name	the name of the block device that has been formatted
path	the mount path
filesystemtype	The type of the mounted file system. Possible values can refer to the dfs_mkfs() function description.
rwflag	read and write flag bit
data	private data for a specific file system
return	——
0	file system mounted successfully
-1	file system mount fail to be mounted

If there is only one storage device, it can be mounted directly to the root directory `/`.

1.4.1.2.6 Unmount a file system

When a file system does not need to be used anymore, it can be unmounted. The interface to unmount the file system is as follows:

```
int dfs_unmount(const char *specialfile);
```

Parameter	Description
specialfile	mount path
return	——
0	unmount the file system successfully
-1	fail to unmount the file system

1.4.1.3 Document Management

This section introduces the functions that are related to the operation of the file. The operation of the file is generally based on the file descriptor fd, as shown in the following figure:

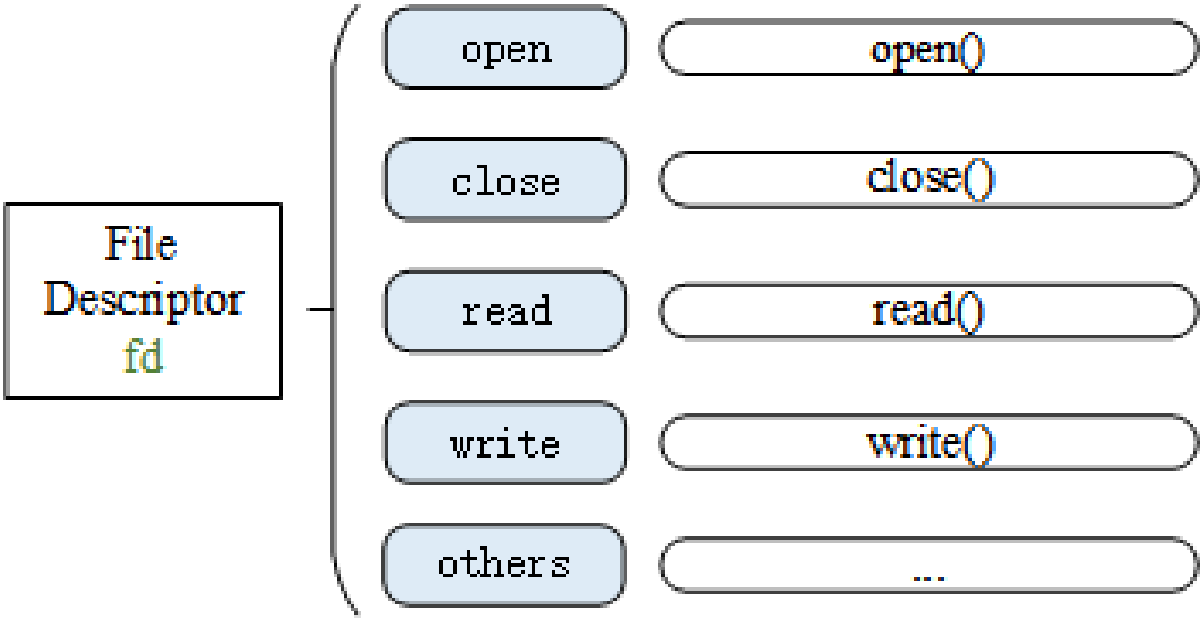


Figure 1.86 common function of file management

1.4.1.3.1 Open and Close Files

To open or create a file, you can call the following open() function:

```
int open(const char *file, int flags, ...);
```

Parameter	Description
file	file names that are opened or created
flags	Specify the way to open the file, and values can refer to the following table.
return	——
file descriptor	file opened successfully
-1	fail to open the file

A file can be opened in a variety of ways, and multiple open methods can be specified at the same time. For example, if a file is opened by O_WRONLY and O_CREAT, then when the specified file which need to be open does not exist, it will create the file first and then open it as write-only. The file opening method is as follows:

Parameter	Description
O_RDONLY	open file in read-only mode
O_WRONLY	open file in write-only mode
O_RDWR	open file in read-write mode
O_CREAT	if the file to be open does not exist, then you can create the file
O_APPEND	When the file is read or written, it will start from the end of the file, that is, the data written will be added to the end of the file in an additional way.
O_TRUNC	empty the contents of the file if it already exists

If you no longer need to use the file, you can use the `close()` function to close the file, and `close()` will write the data back to disk and release the resources occupied by the file.

```
int close(int fd);
```

Parameter	Description
fd	file descriptor
return	——
0	file closed successfully
-1	fail to close the file

1.4.1.3.2 Read and Write Data

To read the contents of a file, use the `read()` function:

```
int read(int fd, void *buf, size_t len);
```

Parameter	Description
fd	file descriptor
buf	buffer pointer
len	read number of bytes of the files
return	——
int	the number of bytes actually read
0	read data has reached the end of the file or there is no readable data
-1	read error, error code to view the current thread's errno

This function reads the `len` bytes of the file pointed to by the parameter `fd` into the memory pointed to by the `buf pointer`. In addition, the read/write position pointer of the file moves with the byte read.

To write data into a file, use the `write()` function:

```
int write(int fd, const void *buf, size_t len);
```

Parameter	Description
fd	file descriptor
buf	buffer pointer
len	the number of bytes written to the file
return	——
int	the number of bytes actually written
-1	write error, error code to view the current thread's errno

This function writes `len` bytes in the memory pointed out by the `buf pointer` into the file pointed out by the parameter `fd`. In addition, the read and write location pointer of the file moves with the bytes written.

1.4.1.3.3 Rename

To rename a file, use the `rename()` function:

```
int rename(const char *old, const char *new);
```

Parameter	Description
old	file's old name
new	new name
return	——
0	change the name successfully
-1	fail to change the name

This function changes the file name specified by the parameter `old` to the file name pointed to by the parameter `new`. If the file specified by `new` already exists, the file will be overwritten.

1.4.1.3.4 Get Status

To get the file status, use the following `stat()` function:

```
int stat(const char *file, struct stat *buf);
```

Parameter	Description
file	file name
buf	structure pointer to a structure that stores file status information
return	——
0	access status successfully
-1	fail to access to status

1.4.1.3.5 Delete Files

Delete a file in the specified directory using the `unlink()` function:

```
int unlink(const char *pathname);
```

Parameter	Description
pathname	specify the absolute path to delete the file
return	——
0	deleted the file successfully
-1	fail to deleted the file

1.4.1.3.6 Synchronize File Data to Storage Devices

Synchronize all modified file data in memory to the storage device using the `fsync()` function:

```
int fsync(int fildes);
```

Parameter	Description
fildes	file descriptor
Return	——
0	synchronize files successfully

Parameter	Description
-1	fail to synchronize files

1.4.1.3.7 Query file system related information

Use the `statfs()` function to query file system related information.

```
int statfs(const char *path, struct statfs *buf);
```

Parameter	Description
path	file system mount path
buf	structure pointer for storing file system information
Return	——
0	query file system information successfully
-1	fail to query file system information

1.4.1.3.8 Monitor I/O device status

To monitor the I/O device for events, use the `select()` function:

```
int select( int nfds,
            fd_set *readfds,
            fd_set *writefds,
            fd_set *exceptfds,
            struct timeval *timeout);
```

Parameter	Description
nfds	The range of all file descriptors in the collection, that is, the maximum value of all file descriptors plus 1
readfds	Collection of file descriptors that need to monitor read changes
writefds	Collection of file descriptors that need to monitor write changes
exceptfds	Collection of file descriptors that need to be monitored for exceptions
timeout	timeout of **select**
return	——
positive value	a read/write event or error occurred in the monitored file collection
0	waiting timeout, no readable or writable or erroneous files
negative value	error

Use the `select()` interface to block and simultaneously detect whether a group of non-blocking I/O devices have events (such as readable, writable, high-priority error output, errors, etc.) until a device triggered an event or exceeded a specified wait time.

1.4.1.4 Directory management

This section describes functions that directory management often uses, and operations on directories are generally based on directory addresses, as shown in the following image:

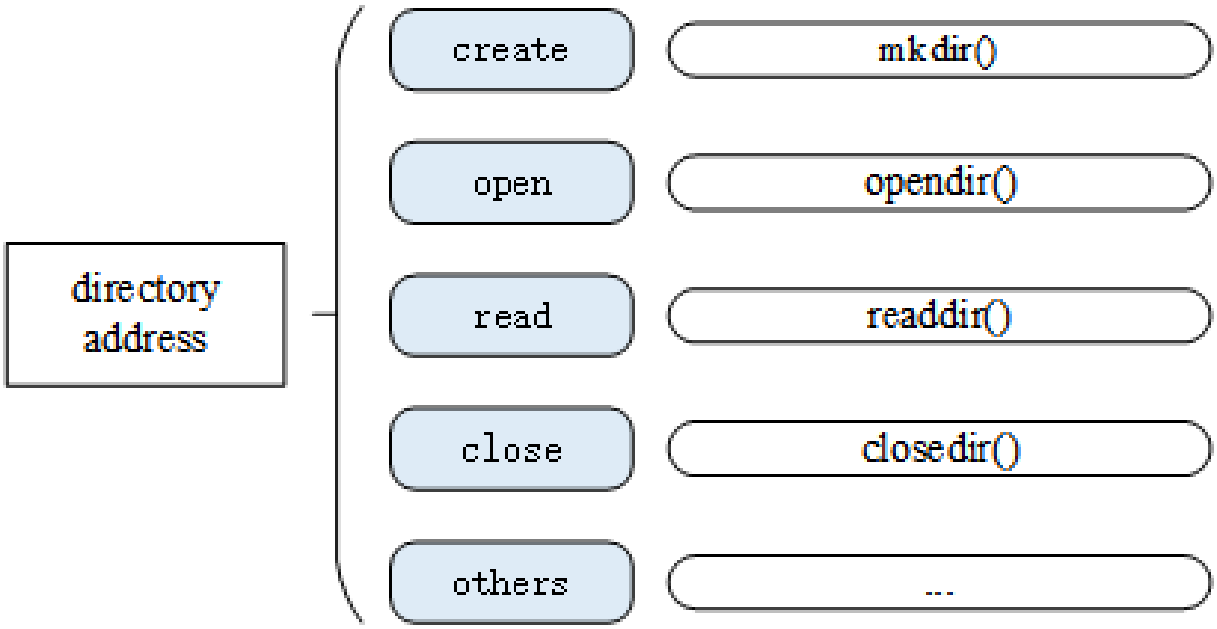


Figure 1.87 functions that directory management often uses

1.4.1.4.1 Create and Delete Directories

To create a directory, you can use the mkdir() function:

```
int mkdir(const char *path, mode_t mode);
```

Parameter	Description
path	the absolute address of the directory
mode	create a pattern
Return	——
0	create directory successfully
-1	fail to create directory

This function is used to create a directory as a folder, the parameter path is the absolute path of the directory, the parameter mode is not enabled in the current version, so just fill in the default parameter 0x777.

Delete a directory using the rmdir() function:

```
int rmdir(const char *pathname);
```

Parameter	Description
pathname	absolute path to delete the directory
Return	——
0	delete the directory successfully
-1	fail to delete the directory

1.4.1.4.2 Open and Close the Directory

Open the directory to use the `opendir()` function:

```
DIR* opendir(const char* name);
```

Parameter	Description
name	absolute address of the directory
Return	——
DIR	open the directory successfully, and return to a pointer to the directory stream
NULL	fail to open

To close the directory, use the `closedir()` function:

```
int closedir(DIR* d);
```

Parameter	Description
d	directory stream pointer
Return	——
0	directory closed successfully
-1	directory closing error

This function is used to close a directory and must be used with the `opendir()` function.

1.4.1.4.3 Read Directory

To read the directory, use the `readdir()` function:

```
struct dirent* readdir(DIR *d);
```

Parameter	Description
d	directory stream pointer
Return	——
dirent	read successfully and return to a structure pointer to a directory entry
NULL	read to the end of the directory

This function is used to read the directory, and the parameter d is the directory stream pointer. In addition, each time a directory is read, the pointer position of the directory stream is automatically recursed by 1 position backward.

1.4.1.4.4 Get the Read Position of the Directory Stream

To get the read location of the directory stream, use the `telldir()` function:

```
long telldir(DIR *d);
```

Parameter	Description
d	directory stream pointer
Return	——
long	read the offset of the position

The return value of this function records the current position of a directory stream. This return value represents the offset from the beginning of the directory file. You can use this value in the following `seekdir()` to reset the directory to the current position. In other words, the `telldir()` function can be used with the `seekdir()` function to reset the read position of the directory stream to the specified offset.

1.4.1.4.5 Set the Location to Read the Directory Next Time

Set the location to read the directory next time using the `seekdir()` function:

```
void seekdir(DIR *d, off_t offset);
```

Parameter	Description
d	directory stream pointer
offset	the offset value, displacement from this directory

This is used to set the read position of the parameter d directory stream, and starts reading from this new position when readdir() is called.

1.4.1.4.6 Reset the Position of Reading Directory to the Beginning

To reset the directory stream's read position to the beginning, use the `rewinddir()` function:

```
void rewinddir(DIR *d);
```

Parameter	Description
d	directory stream pointer

This function can be used to set the current read position of the `d` directory stream to the initial position of the directory stream.

1.4.1.5 DFS Configuration Options

The specific configuration path of the file system in menuconfig is as follows:

```
RT-Thread Components  --->
    Device virtual file system  --->
```

The configuration menu description and corresponding macro definitions are shown in the following table:

Configuration Options	Corresponding Macro Definition	Description
[*] Using device virtual file system	RT_USING_DFS	Open DFS virtual file system
[*] Using working directory	DFS_USING_WORKDIR	open a relative path
(2) The maximal number of mounted file system	DFS_FILESYSTEMS_MAX	maximum number of mounted file systems
(2) The maximal number of file system type	DFS_FILESYSTEM_TYPES_MAX	maximum number of supported file systems
(4) The maximal number of opened files	DFS_FD_MAX	maximum number of open files
[ ] Using mount table for file system	RT_USING_DFS_MNTTABLE	open the automatic mount table
[*] Enable elm-chan fatfs	RT_USING_DFS_ELMFAT	open the elm-FatFs file system
[*] Using devfs for device objects	RT_USING_DFS_DEVFS	open the DevFS device file system

Configuration Options	Corresponding Macro Definition	Description
[ ] Enable ReadOnly file system on flash	RT_USING_DFS_ROMFS	open the RomFS file system
[ ] Enable RAM file system	RT_USING_DFS_RAMFS	open the RamFS file system
[ ] Enable UFFS file system: Ultra-low-cost Flash File System	RT_USING_DFS_UFFS	open the UFFS file system
[ ] Enable JFFS2 file system	RT_USING_DFS_JFFS2	open the JFFS2 file system
[ ] Using NFS v3 client file system	RT_USING_DFS_NFS	open the NFS file system

By default, the RT-Thread operating system does not turn on the relative path function in order to obtain a small memory footprint. When the Support Relative Paths option is not turned on, you should use an absolute directory when working with files and directory interfaces (because there is no currently working directory in the system). If you need to use the current working directory and the relative directory, you can enable the relative path function in the configuration item of the file system.

When the option `[*] Use mount table for file system` is selected, the corresponding macro `RT_USING_DFS_MNTTABLE` will be enabled to turn on the automatic mount table function. The automatic `mount_table[]` is provided by the user in the application code. The user needs to specify the device name, mount path, file system type, read and write flag and private data in the table. After that, the system will traverse the mount table to execute the mount. It should be noted that the mount table must end with `{0}` to judge the end of the table.

The automatic mount table `mount_table []` is shown below, where the five members of `mount_table [0]` are the five parameters of function `dfs_mount ()`. This means that the elm file system is mounted / path on the flash 0 device, rwflag is 0, data is 0, `mount_table [1]` is `{0}` as the end to judge the end of the table.

```
const struct dfs_mount_tbl mount_table[] =
{
    {"flash0", "/", "elm", 0, 0},
    {0}
};
```

1.4.1.5.1 elm-FatFs File System Configuration Option

Elm-FatFs can be further configured after opening the elm-FatFs file system in menuconfig. The configuration menu description and corresponding macro definitions are as follows:

Configuration Options	Corresponding Macro Definition	Description
(437) OEM code page	RT_DFS_ELM_CODE_PAGE	encoding mode
[*] Using RT_DFS_ELM_WORD_ACCESS	RT_DFS_ELM_WORD_ACCESS	
Support long file name (0: LFN disable) --->	RT_DFS_ELM_USE_LFN	open long file name submenu
(255) Maximal size of file name length	RT_DFS_ELM_MAX_LFN	maximum file name length
(2) Number of volumes (logical drives) to be used.	RT_DFS_ELM_DRIVES	number of devices mounting FatFs
(4096) Maximum sector size to be handled.	RT_DFS_ELM_MAX_SECTOR_SIZE	the sector size of the file system
[ ] Enable sector erase feature	RT_DFS_ELM_USE_ERASE	
[*] Enable the reentrancy (thread safe) of the FatFs module	RT_DFS_ELM_REENTRANT	open reentrant

1.4.1.5.1.1 Long File Name

By default, FatFs file naming has the following disadvantages:

- The file name (without suffix) can be up to 8 characters long and the suffix can be up to 3 characters long. The file name and suffix will be truncated when the limit is exceeded.
- File name does not support case sensitivity (displayed in uppercase).

If you need to support long filenames, you need to turn on the option to support long filenames. The submenu of the long file name is described as follows:

Configuration Options	Corresponding Macro Definition	Description
( ) 0: LFN disable	RT_DFS_ELM_USE_LFN_0	close the long file name
( ) 1: LFN with static LFN working buffer	RT_DFS_ELM_USE_LFN_1	use static buffers to support long file names, and multi-threaded operation of file names will bring re-entry problems
( ) 2: LFN with dynamic LFN working buffer on the stack	RT_DFS_ELM_USE_LFN_2	long file names are supported by temporary buffers in the stack. Larger demand for stack space.
(X) 3: LFN with dynamic LFN working buffer on the heap	RT_DFS_ELM_USE_LFN_3	use the heap (malloc request) buffer to store long filenames, it is the safest (default)

1.4.1.5.1.2 Encoding Mode

When long file name support is turned on, you can set the encoding mode for the file name. RT-Thread/FatFs uses 437 encoding (American English) by default. If you need to store the Chinese file name, you can use 936 encoding (GBK encoding). The 936 encoding requires a font library of approximately 180KB. If you only use English characters as a file, we recommend using 437 encoding (American English), this will save this 180KB of Flash space.

The file encodings supported by FatFs are as follows:

```
/* This option specifies the OEM code page to be used on the target system.
/  Incorrect setting of the code page can cause a file open failure.
/
/   1   - ASCII (No extended character. Non-LFN cfg. only)
/   437 - U.S.
/   720 - Arabic
/   737 - Greek
/   771 - KBL
/   775 - Baltic
/   850 - Latin 1
/   852 - Latin 2
/   855 - Cyrillic
/   857 - Turkish
/   860 - Portuguese
/   861 - Icelandic
/   862 - Hebrew
/   863 - Canadian French
/   864 - Arabic
/   865 - Nordic
/   866 - Russian
/   869 - Greek 2
/   932 - Japanese (DBCS)
/   936 - Simplified Chinese (DBCS)
/   949 - Korean (DBCS)
/   950 - Traditional Chinese (DBCS)
*/
```

1.4.1.5.1.3 File System Sector Size

Specify the internal sector size of FatFs, which needs to be greater than or equal to the sector size of the actual hardware driver. For example, if a spi flash chip sector is 4096 bytes, the above macro needs to be changed to 4096. Otherwise, when the FatFs reads data from the driver, the array will be out of bounds and the system will crash (the new version gives a warning message when the system is executed) .

Usually Flash device can be set to 4096, and the common TF card and SD card have a sector size of 512.

1.4.1.5.1.4 Reentrant

FatFs fully considers the situation of multi-threaded safe read and write security. When reading and writing FafFs in multi-threading, in order to avoid the problems caused by re-entry, you need to open the macro above. If the system has only one thread to operate the file system and there is no reentrancy problem, you can turn it off to save resources.

1.4.1.5.1.5 More Configuration

FatFs itself supports a lot of configuration options and the configuration is very flexible. The following file is a FatFs configuration file that can be modified to customize FatFs.

`components/dfs/filesystems/elmfat/ffconf.h`

1.4.1.6 DFS Application Example

1.4.1.6.1 FinSH Command

After the file system is successfully mounted, the files and directories can be operated. The commonly used FinSH commands for file system operations are shown in the following table:

FinSH Command	Description
ls	display information about files and directories
cd	enter the specified directory
cp	copy file
rm	delete the file or the directory
mv	move the file or rename it
echo	write the specified content to the specified file, write the file when it exists, and create a new file and write when the file does not exist.
cat	display the contents of the file
pwd	print out the current directory address
mkdir	create a folder
mkfs	formatted the file system

Use the `ls` command to view the current directory information, and the results are as follows:

```
msh />ls                        # use the `ls` command to view the current directory information
Directory /:                    # you can see that the root directory already exists /
```

Use the `mkdir` command to create a folder, and the results are as follows:

```
msh />mkdir rt-thread             # create an rt-thread folder
msh />ls                          # view directory information as follows
Directory /:
rt-thread           <DIR>
```

Use the `echo` command to output the input string to the specified output location. The result is as follows:

```
msh />echo "hello rt-thread!!!"               # outputs the string to standard output
hello rt-thread!!!
msh />echo "hello rt-thread!!!" hello.txt     # output the string output to the hello.txt file
msh />ls
Directory /:
rt-thread           <DIR>
hello.txt           18
msh />
```

Use the `cat` command to view the contents of the file. The result is as follows:

```
msh />cat hello.txt                     # view the contents of the hello.txt file and output
hello rt-thread!!!
```

Use the `rm` command to delete a folder or file. The result is as follows:

```
msh />ls                                # view the information of current directory
Directory /:
rt-thread           <DIR>
hello.txt           18
msh />rm rt-thread                      # delete the rt-thread folder
msh />ls
Directory /:
hello.txt           18
msh />rm hello.txt                      # delete the hello.txt file
msh />ls
Directory /:
msh />
```

1.4.1.6.2 Read and Write File Examples

Once the file system is working, you can run the application example. In the sample code, you first create a file `text.txt` using the `open()` function and write the string `"RT -Thread Programmer!\n"` in the file using the `write()` function, and then close the file. Use the `open()` function again to open the `text.txt` file, read the contents and print it out, and close the file finally.

The sample code is as follows:

```
#include <rtthread.h>
#include <dfs_posix.h> /* this header file need to be included when you need to operate the file */

static void readwrite_sample(void)
{
    int fd, size;
    char s[] = "RT-Thread Programmer!", buffer[80];

    rt_kprintf("Write string %s to test.txt.\n", s);

    /* open the '/text.txt' file in create and read-write mode and create the file if it does not exist*/
    fd = open("/text.txt", O_WRONLY | O_CREAT);
    if (fd>= 0)
    {
        write(fd, s, sizeof(s));
        close(fd);
        rt_kprintf("Write done.\n");
    }

      /* open the '/text.txt' file in read-only mode */
    fd = open("/text.txt", O_RDONLY);
    if (fd>= 0)
    {
        size = read(fd, buffer, sizeof(buffer));
        close(fd);
        rt_kprintf("Read from file test.txt : %s \n", buffer);
        if (size < 0)
            return ;
    }
  }
/* export to the msh command list */
MSH_CMD_EXPORT(readwrite_sample, readwrite sample);
```

1.4.1.6.3 An Example of Changing the File Name

The sample code in this section shows how to modify the file name. The program creates a function `rename_sample()` that manipulates the file and exports it to the msh command list. This function calls the `rename()` function to rename the file named `text.txt` to `text1.txt`. The sample code is as follows:

```
#include <rtthread.h>
#include <dfs_posix.h> /* this header file need to be included when you need to operate the file */

static void rename_sample(void)
{
    rt_kprintf("%s => %s", "/text.txt", "/text1.txt");

    if (rename("/text.txt", "/text1.txt") < 0)
        rt_kprintf("[error!]\n");
    else
        rt_kprintf("[ok!]\n");
}
/* export to the msh command list */
MSH_CMD_EXPORT(rename_sample, rename sample);
```

Run the example in the FinSH console and the results are as follows:

```
msh />echo "hello" text.txt
msh />ls
Directory /:
text.txt           5
msh />rename_sample
/text.txt => /text1.txt [ok!]
msh />ls
Directory /:
text1.txt           5
```

In the example demonstration, we first create a file named `text.txt` using the echo command, and then run the sample code to change the file name of the file `text.txt` to `text1.txt`.

1.4.1.6.4 Get File Status Example

The sample code shows how to get the file status. The program creates a function `stat_sample()` that manipulates the file and exports it to the msh command list. This function calls the `stat()` function to get the file size information of the text.txt file. The sample code is as follows:

```
#include <rtthread.h>
#include <dfs_posix.h> /* this header file need to be included when you need to operate the file */

static void stat_sample(void)
{
    int ret;
     struct stat buf;
     ret = stat("/text.txt", &buf);
    if(ret == 0)
    rt_kprintf("text.txt file size = %d\n", buf.st_size);
    else
    rt_kprintf("text.txt file not fonud\n");
}
/* export to the msh command list */
MSH_CMD_EXPORT(stat_sample, show text.txt stat sample);
```

Run the example in the FinSH console and the results are as follows:

```
msh />echo "hello" text.txt
msh />stat_sample
text.txt file size = 5
```

During the example run, the file `text.txt` is first created with the `echo` command, then the sample code is run, and the file size information for the file `text.txt` is printed.

1.4.1.6.5 Create a Directory Example

The sample code in this section shows how to create a directory. The program creates a function file `mkdir_sample()` that manipulates the file and exports it to the msh command list, which calls the `mkdir()` function to create a folder called `dir_test`. The sample code is as follows:

```
#include <rtthread.h>
#include <dfs_posix.h> /* this header file need to be included when you need to operate the file */

static void mkdir_sample(void)
{
    int ret;

    /* create a directory */
    ret = mkdir("/dir_test", 0x777);
    if (ret < 0)
    {
        /* fail to create a directory */
        rt_kprintf("dir error!\n");
    }
    else
    {
        /* create a directory successfully */
        rt_kprintf("mkdir ok!\n");
    }
}
/* export to the msh command list */
MSH_CMD_EXPORT(mkdir_sample, mkdir sample);
```

Run the example in the FinSH console and the result is as follows:

```
msh />mkdir_sample
mkdir ok!
msh />ls
Directory /:
dir_test                <DIR>    # <DIR> it indicates that the type of the directory is a folder
```

This example demonstrates creating a folder named `dir_test` in the root directory.

1.4.1.6.6 Read directory Example

The sample code shows how to read the directory. The program creates a function `readdir_sample()` that manipulates the file and exports it to the msh command list. This function calls the `readdir()` function to get the contents of the `dir_test` folder and print it out. The sample code is as follows:

```
#include <rtthread.h>
#include <dfs_posix.h> /* this header file need to be included when you need to operate the file */

static void readdir_sample(void)
{
    DIR *dirp;
    struct dirent *d;

    /* open the / dir_test directory */
    dirp = opendir("/dir_test");
    if (dirp == RT_NULL)
    {
        rt_kprintf("open directory error!\n");
    }
    else
    {
        /* read the directory */
        while ((d = readdir(dirp)) != RT_NULL)
        {
            rt_kprintf("found %s\n", d->d_name);
        }

        /* close the directory */
        closedir(dirp);
    }
}
/* exports to the msh command list */
MSH_CMD_EXPORT(readdir_sample, readdir sample);
```

Run the example in the FinSH console and the result is as follows:

```
msh />ls
Directory /:
```

```
dir_test                <DIR>
msh />cd dir_test
msh /dir_test>echo "hello" hello.txt       # create a hello.txt file
msh /dir_test>cd ..                        # switch to the parent folder
msh />readdir_sample
found hello.txt
```

In this example, first create a hello.txt file under the dir_test folder and exit the dir_test folder. At this point, run the sample program to print out the contents of the dir_test folder.

1.4.1.6.7 An Example of Setting the location of the read directory

The sample code in this section shows how to set the location to read the directory next time. The program creates a function `telldir_sample()` that manipulates the file and exports it to the msh command list. This function first opens the root directory, then reads all the directory information in the root directory and prints the directory information. Meanwhile, use the `telldir()` function to record the location information of the third directory entry. Before reading the directory information in the root directory for the second time, use the `seekdir()` function to set the read location to the address of the third directory entry previously recorded. At this point, read the information in the root directory again, and the directory information is printed out. The sample code is as follows:

```c
#include <rtthread.h>
#include <dfs_posix.h> /* this header file need to be included when you need to operate the file */

/* assume that the file operation is done in one thread */
static void telldir_sample(void)
{
    DIR *dirp;
    int save3 = 0;
    int cur;
    int i = 0;
    struct dirent *dp;

    /* open the root directory */
    rt_kprintf("the directory is:\n");
    dirp = opendir("/");

    for (dp = readdir(dirp); dp != RT_NULL; dp = readdir(dirp))
    {
        /* save the directory pointer for the third directory entry */
        i++;
        if (i == 3)
            save3 = telldir(dirp);

        rt_kprintf("%s\n", dp->d_name);
    }

    /* go back to the directory pointer of the third directory entry you just saved */
    seekdir(dirp, save3);

    /* Check if the current directory pointer is equal to the pointer to the third directory entry that was
      saved. */
    cur = telldir(dirp);
    if (cur != save3)
    {
        rt_kprintf("seekdir (d, %ld); telldir (d) == %ld\n", save3, cur);
    }

    /* start printing from the third directory entry */
    rt_kprintf("the result of tell_seek_dir is:\n");
    for (dp = readdir(dirp); dp != NULL; dp = readdir(dirp))
    {
         rt_kprintf("%s\n", dp->d_name);
    }

    /* close the directory */
    closedir(dirp);
}
/* exports to the msh command list */
MSH_CMD_EXPORT(telldir_sample, telldir sample);
```

In this demo example, you need to manually create the five folders from `hello_1` to `hello_5` in the root directory with the `mkdir` command, making sure that there is a folder directory under the root directory for running the sample.

Run the example in the FinSH console and the results are as follows:

```
msh />ls
Directory /:
hello_1             <DIR>
hello_2             <DIR>
hello_3             <DIR>
hello_4             <DIR>
hello_5             <DIR>
msh />telldir_sample
the directory is:
hello_1
hello_2
hello_3
hello_4
hello_5
the result of tell_seek_dir is:
hello_3
hello_4
hello_5
```

After running the sample, you can see that the first time you read the root directory information, it starts from the first folder and prints out all the directory information in the root directory. When the directory information is printed for the second time, since the starting position of the reading is set to the position of the third folder by using the `seekdir()` function, the second time when reading the root directory is from the third folder. Start reading until the last folder, only the directory information from `hello_3` to `hello_5` is printed.

1.4.1.7 FAQ

1.4.1.7.1 Q: What should I do if I find that the file name or folder name is not displayed properly?

A: Check if long file name support is enabled, DFS feature configuration section.

1.4.1.7.2 Q: What should I do if the file system fails to initialize?

A: Check if the type and number of file systems allowed to be mounted in the file system configuration project are sufficient.

1.4.1.7.3 Q: What should I do if the file system *mkfs* command fails?

A: Check if the storage device exists. If it exists, check to see if the device driver can pass the function test, if it fails, check the driver error. Check if the libc function is enabled.

1.4.1.7.4 Q: What should I do if the file system fails to mount?

A:

- Check if the specified mount path exists. The file system can be mounted directly to the root directory ("/"), but if you want to mount it on another path, such as ("/sdcard"). You need to make sure that the ("/sdcard") path exists. Otherwise, you need to create the `sdcard` folder in the root directory before the mount is successful.
- Check if the file system is created on the storage device. If there is no file system on the storage device, you need to create a file system on the storage using the `mkfs` command.

1.4.1.7.5 Q: What should I do if SFUD cannot detect the Flash specific model?

A:

- Check if the hardware pin settings are wrong.
- Whether the SPI device is already registered.
- Whether the SPI device is mounted to the bus.
- Check the `Using auto probe flash JEDEC SFDP parameter` and the `Using defined supported flash chip information table' under the 'RT-Thread Components → Device Drivers -> Using SPI Bus/Device device drivers -> Using Serial Flash Universal Driver` menu, to see whether the configuration item is selected, if it is not selected then you need to enable these two options.
- If the storage device is still not recognized with the above option turned on, then issues can be raised in the `SFUD` project.

1.4.1.7.6 Q: Why does the benchmark test of the storage device take too long?

A:

- Compare the `benchmark test data` when the `system tick` is 1000 and the length of time required for this test. If the time lag is too large, you can think that the test work is not working properly.
- Check the settings of the system tick, because some delay operations will be determined according to the tick time, so you need to set the appropriate `system tick` value according to the system conditions. If the system's `system tick` value is no less than 1000, you will need to use a logic analyzer to check the waveform to determine that the communication rate is normal.

1.4.1.7.7 Q: SPI Flash implements elmfat file system, and how to keep some sectors not used by file system?

A: You can create multiple block devices for the entire storage device using the `partition` tool package provided by RT-Thread. And block devices can be assigned different functions.

1.4.1.7.8 Q: What should I do if the program gets stuck during the test file system?

A: Try using the debugger or print some necessary debugging information to determine where the program is stuck and ask questions.

1.4.1.7.9 Q: How can I check the problem of the file system step by step?

A: You can step through the problem from the bottom to the top.

- First check if the storage device is successfully registered and the function is normal.
- Check if a file system is created in the storage device.
- Check if the specified file system type is registered to the DFS framework, and often check if the allowed file system types and quantities are sufficient.
- Check if DFS is successfully initialized. The initialization of this step is pure software, so the possibility of error is not high. It should be noted that if component auto-initialization is turned on, there is no need to manually initialize it again.

1.4.2 Device Driver

- I/O Device Framework
- PIN Device
- GPIO device model (DM)
- Pin control (pinctrl)
- UART
- UART device model (DM)
- UART early console
- ADC Device
- I2C Bus Device
- I2C device model (DM)
- SPI device
- SPI device model (DM)
- PWM Device
- RTC Device
- RTC DM helpers (`rtc_dm.h`)
- Clock Time Subsystem
- HWTIMER Device
- PTP clock subsystem
- TEE subsystem
- WATCHDOG Device
- WLAN Device
- Sensor Device
- Audio Device
- Open Firmware (OFW)
- OFW nodes and properties
- OFW boot and FDT
- OFW addressing and MMIO
- OFW interrupts
- OFW raw FDT helpers
- OFW platform enumeration
- Devicetree Compiler
- ATA / AHCI
- Block layer (`blk`)
- Physical disk (`rt_blk_disk`)

- Disk partition probing
- Hardware cache (hwcache)
- DVFS
- CAN device
- CAN FD DLC helpers (`can_dm`)
- Clock framework (CLK)
- Fixed clock provider
- Bus and driver binding (core)
- Device model helpers (dm.h)
- System power off and reboot (core)
- DMA engine
- DMA memory pools
- Graphics (DM)
- Backlight devices
- Simple framebuffer (`simple-framebuffer`)
- Boot Logo
- Hardware spinlock
- Industrial I/O (IIO)
- IIO device model (DM)
- Input subsystem
- Input device model (DM)
- Input userspace API (UAPI)
- Input touch bridge
- Input power keys
- LED subsystem
- LED device model (DM)
- Mailbox subsystem
- Mailbox device model (DM)
- NUMA helpers
- NVMe subsystem
- NVMe device model (DM)
- NVMEM subsystem
- NVMEM device model (DM)
- PCI / PCIe
- PCI enumeration and drivers
- PCI config space access

- PCI host controllers
- PCI INTx interrupts
- PCI power management
- PCI MSI and MSI-X
- PCI and device tree
- PCI bridge function driver
- PCI endpoint (EP) mode
- Generic PHY (Phye)
- Phye core API
- Phye and device tree
- Phye provider driver
- Phye consumer integration
- Generic USB PHY
- PIC overview
- PIC virtual IRQ domain
- PIC core framework
- PIC cascade and routing
- PIC and device tree
- PIC and PCI MSI
- PIC platform examples
- Platform bus
- Power subsystem overview
- Power supply
- Charger (gpio-charger)
- Board poweroff and restart
- Power domain
- Regulator
- Reset controller
- RPMsg
- RPMsg character device
- RPMsg name service
- SCMI firmware protocol
- SCMI agent transports
- SCSI host and commands
- SDIO / MMC core
- SDIO device model

- SDIO supplies (regulator)
- SDHCI host controller
- System controller (Syscon)
- Thermal management
- Thermal cooling devices
- UFS host

1.4.2.1 I/O Device Framework

1.4.2.1.0.1 For driver authors (short guide)

Layer	Responsibility
Driver	Map hardware, implement `init/read/write/control`, register `rt_device` (directly or via a sub-framework).
Framework	Share code across vendors (`serial`, `spi`, `pwm`, ...)—override only deltas.
App	Never touch registers—use `rt_device_find` + standard APIs so swapping BSP does not break apps.

Lifecycle: `rt_device_register` → app `open` → `oflag` decides exclusive vs shared access → `close` should drain TX and disable IRQ/DMA.

Thread safety: assume concurrent `read`/`write` unless your driver serializes with a mutex; document if not thread-safe.

Most embedded systems include some I/O (Input/Output) devices, data displays on instruments, serial communication on industrial devices, Flash or SD cards for saving data on data acquisition devices,as well as Ethernet interfaces for network devices, are examples of I/O devices that are commonly seen in embedded systems.

This chapter describes how RT-Thread manages different I/O devices.

1.4.2.1.1 I/O Device Introduction

1.4.2.1.1.1 I/O Device Framework

RT-Thread provides a set of I/O device framework, as shown in the following figure. It is located between the hardware and the application. It is divided into three layers. From top to bottom, they are I/O device management layer, device driver framework layer, and device driver layer.

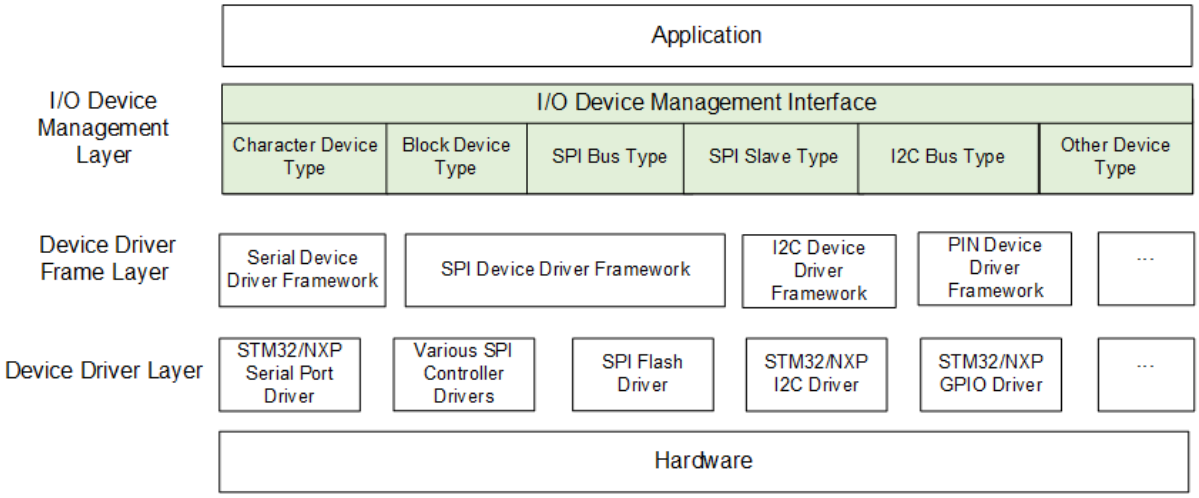


Figure 1.88 I/O Device Framework

The application obtains the correct device driver through the I/O device management interface, and then uses this device driver to perform data (or control) interaction with the bottom I/O hardware device.

The I/O device management layer implements the encapsulation of device drivers. The application accesses the bottom devices through the standard interface provided by the I/O device layer. The upgrade and replacement of the device driver will not affect the upper layer application. In this way, the hardware-related code of the device can exist independently of the application, and both parties only need to pay attention to the respective function implementation, thereby reducing the coupling and complexity of the code and improving the reliability of the system.

The device driver framework layer is an abstraction of the same kind of hardware device driver. The same part of the same hardware device driver of different manufacturers is extracted, and the different parts are left out of interface, implemented by the driver.

The device driver layer is a set of programs that drive the hardware devices to work, enabling access to hardware devices. It is responsible for creating and registering I/O devices. For devices with simple operation logic, you can register devices directly into the I/O Device Manager without going through the device driver framework layer. The sequence diagram is as shown below. There are mainly two points:

- The device driver creates a device instance with hardware access capabilities based on the device model definition and registers the device with the `rt_device_register()` interface in the I/O Device Manager.
- The application finds the device through the `rt_device_find()` interface and then uses the I/O device management interface to access the hardware.

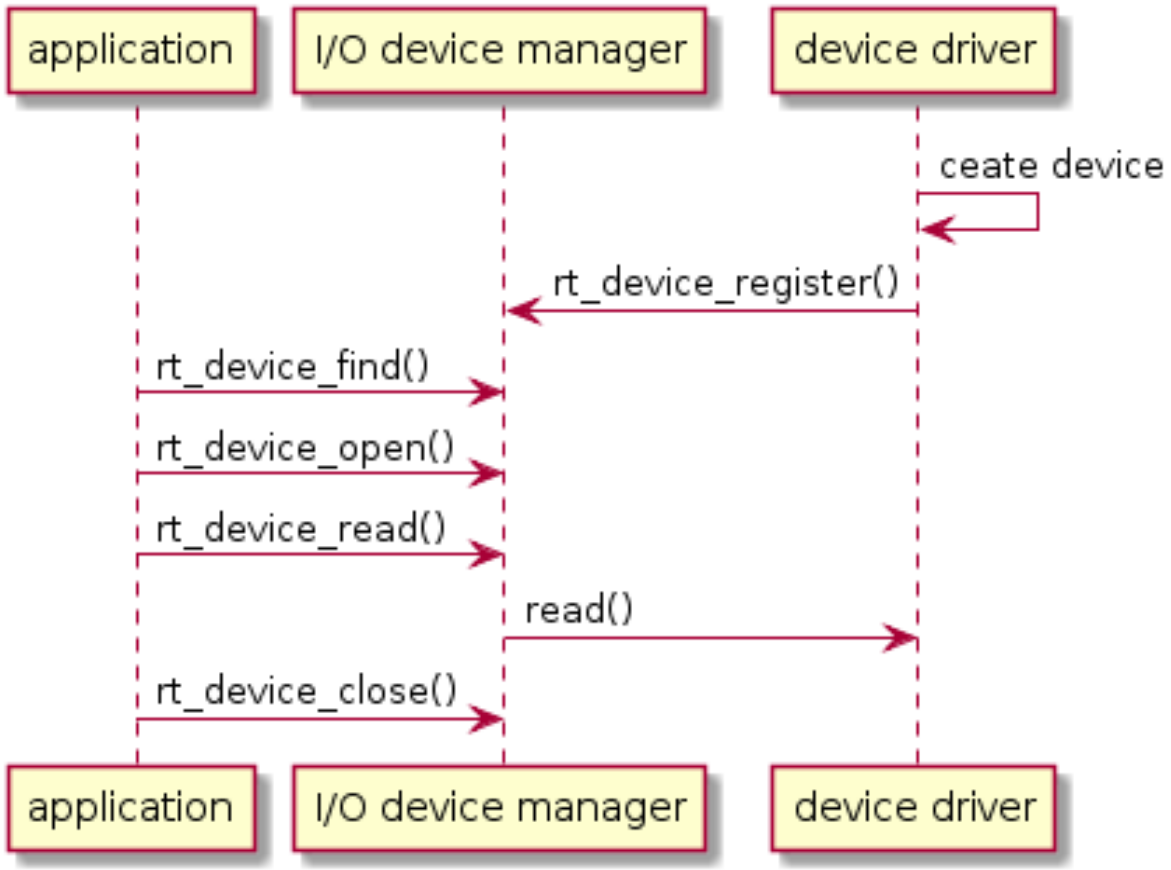


Figure 1.89 Simple I/O Device Using Sequence Diagram

For other devices, such as watchdog, the created device instance will be registered to the corresponding device driver framework, and then the device driver framework will register with the I/O device manager. The main points are as follows:

- The watchdog device driver creates a watchdog device instance with hardware access capability based on the watchdog device model definition and registers the watchdog device through the `rt_hw_watchdog↩_register()` interface into the watchdog device driver framework.
- The watchdog device driver framework registers the watchdog device to the I/O Device Manager via the `rt_device_register()` interface.
- The application accesses the watchdog device hardware through the I/O device management interface.

Usage of Watchdog device:

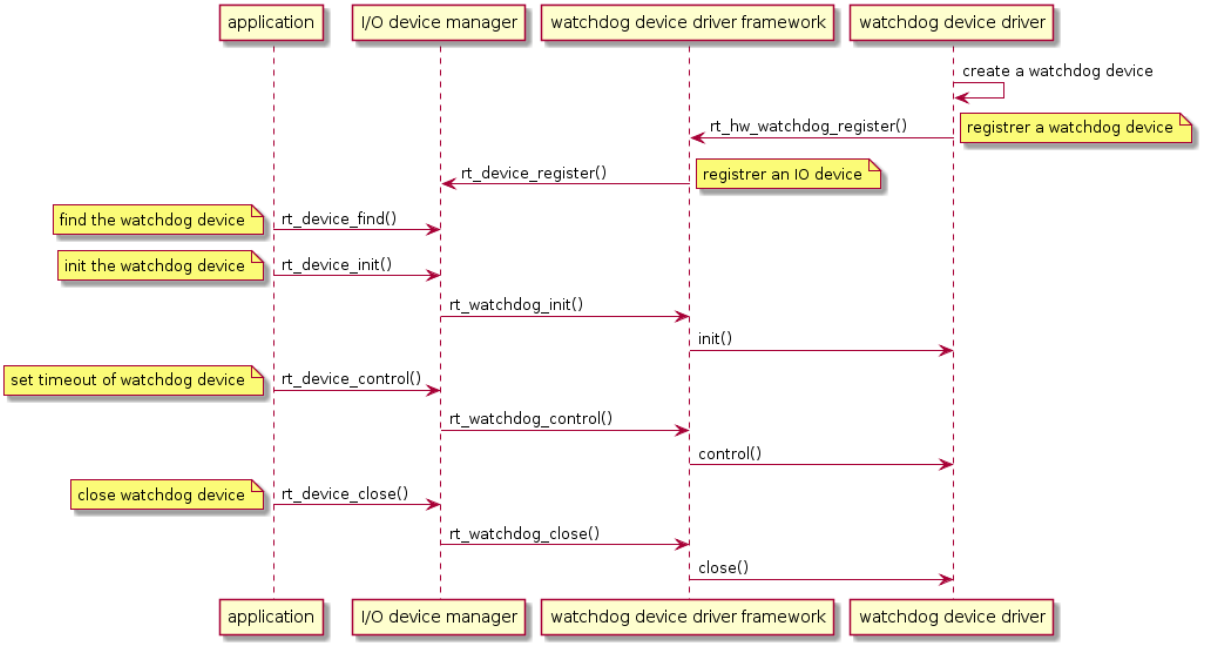


Figure 1.90 Watchdog Device Use Sequence Diagram

1.4.2.1.1.2 I/O Device Model

The device model of RT-Thread is based on the kernel object model, which is considered a kind of objects and is included in the scope of the object manager. Each device object is derived from the base object. Each concrete device can inherit the properties of its parent class object and derive its own properties. The following figure is a schematic diagram of the inheritance and derivation relationship of device object.

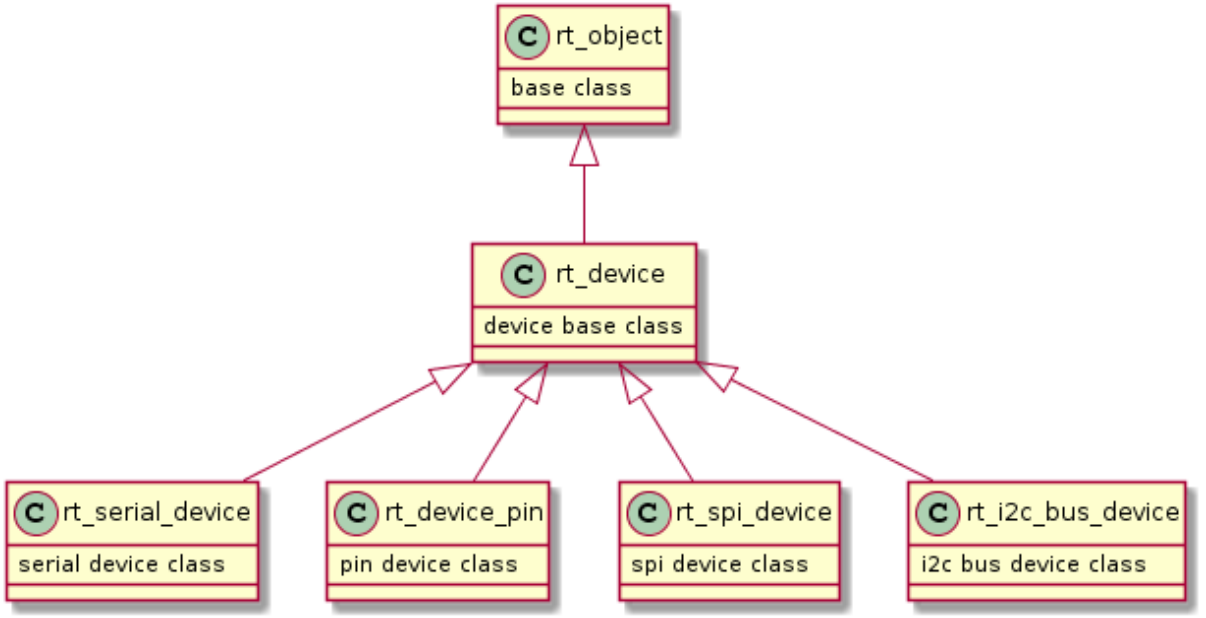


Figure 1.91 Device Inheritance Diagram

The specific definitions of device objects are as follows:

```
struct rt_device
{
    struct rt_object          parent;        /* kernel object base class */
```

```
    enum rt_device_class_type type;          /* device type */
    rt_uint16_t               flag;          /* device parameter */
    rt_uint16_t               open_flag;     /* device open flag */
    rt_uint8_t                ref_count;     /* number of times the device was cited */
    rt_uint8_t                device_id;     /* device ID,0 - 255 */

    /* data transceiving callback function */
    rt_err_t (*rx_indicate)(rt_device_t dev, rt_size_t size);
    rt_err_t (*tx_complete)(rt_device_t dev, void *buffer);

    const struct rt_device_ops *ops;    /* device operate methods */

    /* device's private data */
    void *user_data;
};
typedef struct rt_device *rt_device_t;
```

1.4.2.1.1.3 I/O Device Type

RT-Thread supports multiple I/O device types, the main device types are as follows:

```
RT_Device_Class_Char             /* character device       */
RT_Device_Class_Block            /* block device         */
RT_Device_Class_NetIf            /* network interface device    */
RT_Device_Class_MTD              /* memory device       */
RT_Device_Class_RTC              /* RTC device        */
RT_Device_Class_Sound            /* sound device        */
RT_Device_Class_Graphic          /* graphic device        */
RT_Device_Class_I2CBUS           /* I2C bus device     */
RT_Device_Class_USBDevice        /* USB device */
RT_Device_Class_USBHost          /* USB host device   */
RT_Device_Class_SPIBUS           /* SPI bus device     */
RT_Device_Class_SPIDevice        /* SPI device        */
RT_Device_Class_SDIO             /* SDIO device       */
RT_Device_Class_Miscellaneous    /* miscellaneous devices       */
```

Character devices and block devices are commonly used device types, and their classification is based on the transmission processing between device data and the system. Character mode devices allow for unstructured data transfers, that is, data usually transfers in the form of serial, one byte at a time. Character devices are usually simple devices such as serial ports and buttons.

A block device transfers one data block at a time, for example 512 bytes data at a time. This data block is enforced by the hardware. Data blocks may use some type of data interface or some mandatory transport protocol, otherwise an error may occur. Therefore, sometimes the block device driver must perform additional work on read or write operations, as shown in the following figure:

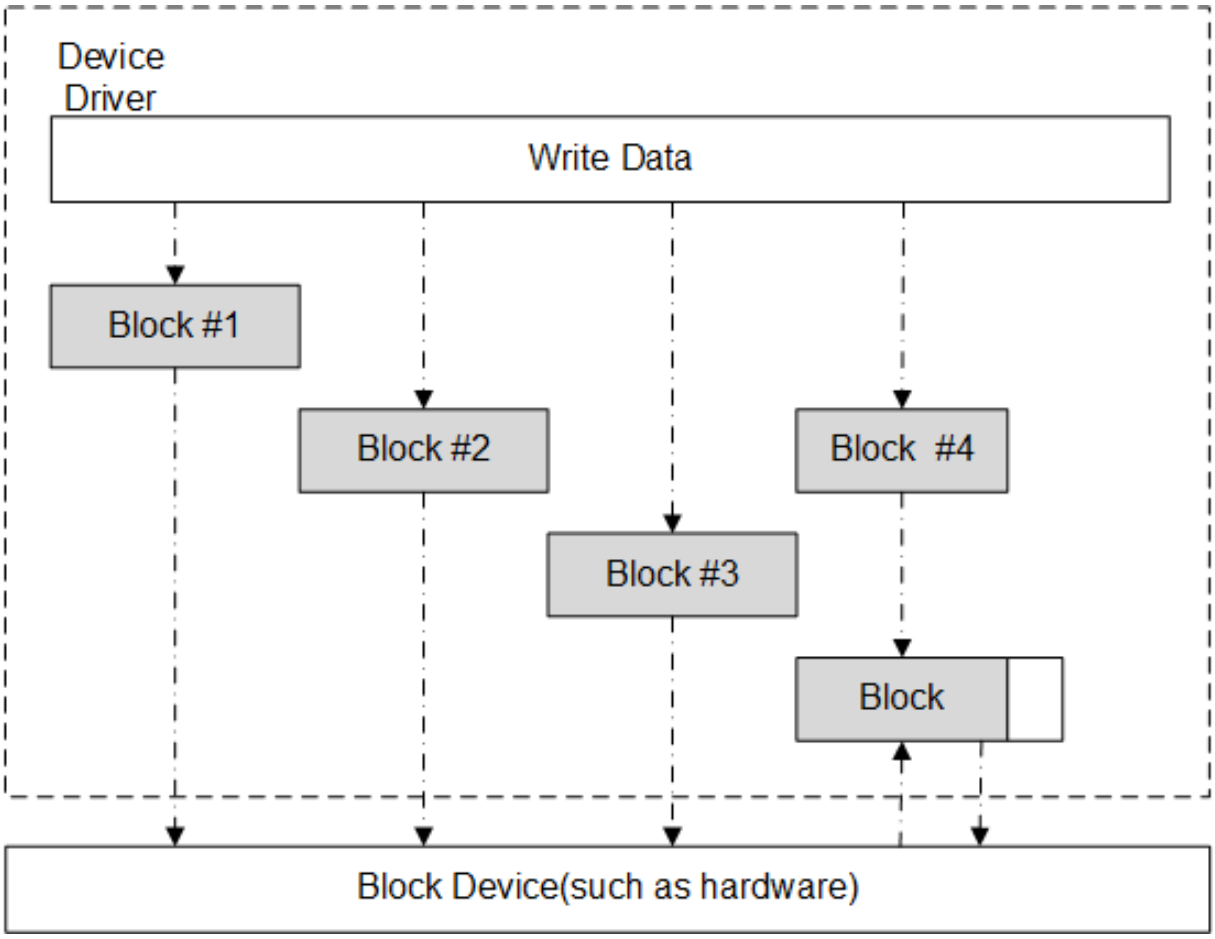


Figure 1.92 Block Device

When the system serves a write operation with a large amount of data, the device driver must first divide the data into multiple packets, each with the data size specified by the device. In the actual process, the last part of the data size may be smaller than the normal device block size. Each block in the above figure is written to the device using a separate write request, and the first three are directly written. However, the last data block size is smaller than the device block size, and the device driver must process the last data block differently than the first 3 blocks. Normally, the device driver needs to first perform a read operation of the corresponding device block, then overwrite the write data onto the read data, and then write the "composited" data block back to the device as a whole block. . For example, for block 4 in the above figure, the driver needs to read out the device block corresponding to block 4, and then overwrite the data to be written to the data read from the device block, and merge them into a new block. Finally write back to the block device.

1.4.2.1.2 Create and Register I/O Device

The driver layer is responsible for creating device instances and registering them in the I/O Device Manager. You can create device instances in a statically declared manner or dynamically create them with the following interfaces:

```
rt_device_t rt_device_create(int type, int attach_size);
```

Parameters	Description
type	device type, the device type values listed in "I/O Device Type" section can be used here
attach_size	user data size
Return	–

Parameters	Description
Device Handle	Create successfully
RT_NULL	Creation failed, dynamic memory allocation failed

When this interface is called, the system allocates a device control block from the dynamic heap memory, the size of which is the sum of `struct rt_device` and `attach_size`, and the type of the device is set by the parameter type. After the device is created, implementing its access to the hardware is needed.

```
struct rt_device_ops
{
    /* common device interface */
    rt_err_t  (*init)    (rt_device_t dev);
    rt_err_t  (*open)    (rt_device_t dev, rt_uint16_t oflag);
    rt_err_t  (*close)   (rt_device_t dev);
    rt_ssize_t (*read)   (rt_device_t dev, rt_off_t pos, void *buffer, rt_size_t size);
    rt_ssize_t (*write)  (rt_device_t dev, rt_off_t pos, const void *buffer, rt_size_t size);
    rt_err_t  (*control)(rt_device_t dev, int cmd, void *args);
};
```

A description of each method of operation is shown in the following table:

Method Name	Method Description
init	Initialize the device. After the device is initialized, the flag of the device control block is set to the active state(RT_DEVICE_FLAG_ACTIVATED). If the flag in the device control block has been set to the active state, then the initialization interface will be returned immediately when running again, and will not be re-initialized.
open	Open the device. Some devices are not started when the system is started, or the device needs to send and receive data. However, if the upper application is not ready, the device should not be enabled by default and start receiving data. Therefore, it is recommended to enable the device when calling the open interface when writing the bottom driver.
close	Close the device. When the device is open, the device control block maintains an open count, the count will add 1 when the device is opended, and the count will subtract 1 when the device is closed, and a real shutdown operation is operated when the counter turns to 0.
read	Read data from the device. The parameter pos is the offset of the read data, but some devices do not necessarily need to specify the offset, such as serial devices, the device driver should ignore this parameter. But for block devices, pos and size are measured in the block size of the block device. For example, the block size of the block device is 512 byte, and in the parameter pos = 10, size = 2, then the driver should return the 10th block in the device (starting from the 0th block) for a total of 2 blocks of data. The type returned by this interface is rt_size_t, which is the number of bytes read or the number of blocks. Normally, the value of size in the parameter should be returned. If it returns zero, set the corresponding errno value.
write	Write data to the device. The parameter pos is the offset of the write data. Similar to read operations, for block devices, pos and size are measured in the block size of the block device. The type returned by this interface is rt_size_t, which is the number of bytes or blocks of data actually written. Normally, the value of size in the parameter should be returned. If it returns zero, set the corresponding errno value.
control	Control the device according to the cmd command. Commands are often implemented by the bottom device drivers. For example, the parameter RT_DEVICE_CTRL_BLK_GETGEOME means to get the size information of the block device.

When a dynamically created device is no longer needed, it can be destroyed using the following function:

```
void rt_device_destroy(rt_device_t device);
```

Parameters	Description
device	device handle

After the device is created, it needs to be registered to the I/O Device Manager for the application to access. The functions for registering the device are as follows:

```
rt_err_t rt_device_register(rt_device_t dev, const char* name, rt_uint8_t flags);
```

Parameters	Description
dev	device handle
name	device name, the maximum length of the device name is specified by the macro RT_NAME_↩ MAX defined in rtconfig.h, and the extra part is automatically truncated
flags	device mode flag
Return	–
RT_EOK	registration success
-RT_ERROR	registration failed, dev is empty or name already exists

>It should be avoided to repeatedly register registered devices and to register devices with the same name.

flags parameters support the following parameters (multiple parameters can be supported in OR logic):

```
#define RT_DEVICE_FLAG_RDONLY       0x001 /* read only */
#define RT_DEVICE_FLAG_WRONLY       0x002 /* write only  */
#define RT_DEVICE_FLAG_RDWR         0x003 /* read and write  */
#define RT_DEVICE_FLAG_REMOVABLE    0x004 /* can be removed  */
#define RT_DEVICE_FLAG_STANDALONE   0x008 /* stand alone   */
#define RT_DEVICE_FLAG_SUSPENDED    0x020 /* suspended  */
#define RT_DEVICE_FLAG_STREAM       0x040 /* stream mode  */
#define RT_DEVICE_FLAG_INT_RX       0x100 /* interrupt reception */
#define RT_DEVICE_FLAG_DMA_RX       0x200 /* DMA reception */
#define RT_DEVICE_FLAG_INT_TX       0x400 /* interrupt sending */
#define RT_DEVICE_FLAG_DMA_TX       0x800 /* DMA sending */
```

Device Stream Mode The RT_DEVICE_FLAG_STREAM parameter is used to output a character string to the serial terminal: when the output character is `\n` , it automatically fills in a `\r` to make a branch.

Successfully registered devices can use the `list_device` command on the FinSH command line to view all device information in the system, including the device name, device type, and number of times the device is opened:

```
msh />list_device
device           type         ref count
-------- -------------------- ----------
e0       Network Interface    0
sd0      Block Device         1
rtc      RTC                  0
uart1    Character Device     0
uart0    Character Device     2
msh />
```

When the device is logged off, the device will be removed from the device manager and the device will no longer be found through the device. Logging out of the device does not release the memory occupied by the device control block. The function to log off of the device is as follows:

```
rt_err_t rt_device_unregister(rt_device_t dev);
```

Parameters	Description
dev	device handle
Return	–
RT_EOK	successful

The following code is an example of registering a watchdog device. After calling the `rt_hw_watchdog_register()` interface, the device is registered to the I/O Device Manager via the `rt_device_register()` interface.

```
const static struct rt_device_ops wdt_ops =
{
    rt_watchdog_init,
    rt_watchdog_open,
    rt_watchdog_close,
    RT_NULL,
    RT_NULL,
    rt_watchdog_control,
};

rt_err_t rt_hw_watchdog_register(struct rt_watchdog_device *wtd,
                                 const char               *name,
                                 rt_uint32_t               flag,
                                 void                     *data)
{
    struct rt_device *device;
    RT_ASSERT(wtd != RT_NULL);

    device = &(wtd->parent);

    device->type        = RT_Device_Class_Miscellaneous;
    device->rx_indicate = RT_NULL;
    device->tx_complete = RT_NULL;

    device->ops         = &wdt_ops;
    device->user_data   = data;

    /* register a character device */
    return rt_device_register(device, name, flag);
}
```

1.4.2.1.3 Access I/O Devices

The application accesses the hardware device through the I/O device management interface, which is accessible to the application when the device driver is implemented. The mapping relationship between the I/O device management interface and the operations on the I/O device is as follows:

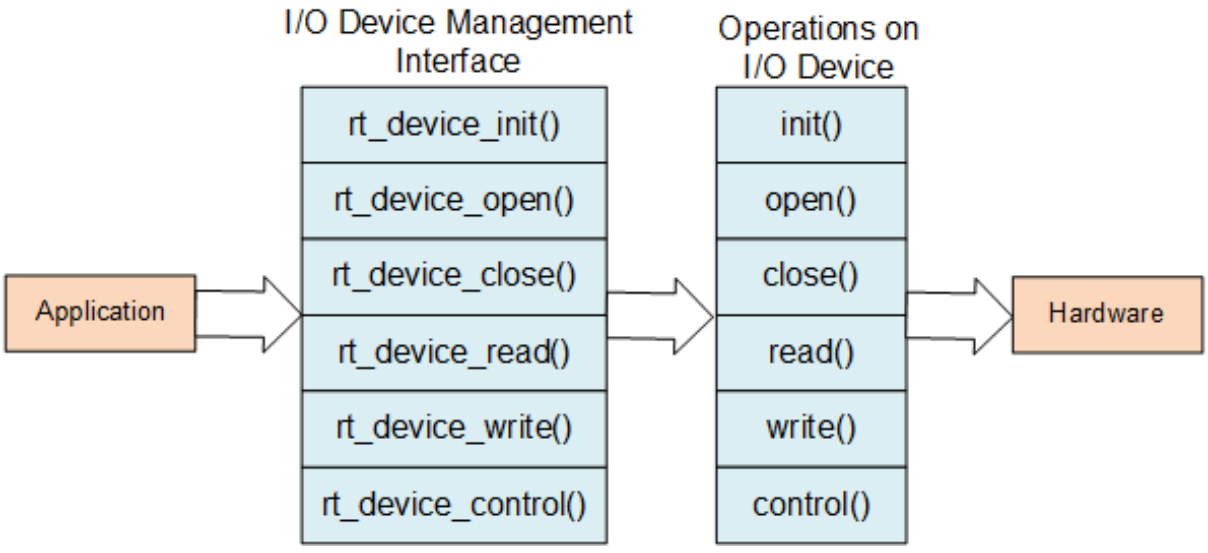


Figure 1.93 Mapping between the I/O Device Management Interface and the Operations on the I/O Device

1.4.2.1.3.1 Find Device

The application obtains the device handle based on the device name, which in turn allows the device to operate. To find device, use function below:

```
rt_device_t rt_device_find(const char* name);
```

Parameters	Description
name	device name
Return	–
device handle	finding the corresponding device will return the corresponding device handle
RT_NULL	no corresponding device object found

1.4.2.1.3.2 Initialize Device

Once the device handle is obtained, the application can initialize the device using the following functions:

```
rt_err_t rt_device_init(rt_device_t dev);
```

Parameters	Description
dev	device handle
Return	–
RT_EOK	device initialization succeeded
Error Code	device initialization failed

>When a device has been successfully initialized, calling this interface will not repeat initialization.

1.4.2.1.3.3 Open and Close Device

Through the device handle, the application can open and close the device. When the device is opened, it will detect whether the device has been initialized. If it is not initialized, it will call the initialization interface to initialize the device by default. Open the device with the following function:

```
rt_err_t rt_device_open(rt_device_t dev, rt_uint16_t oflags);
```

Parameters	Description
dev	device handle
oflags	open device in oflag mode
Return	–
RT_EOK	device successfully turned on
-RT_EBUSY	device will not allow being repeated opened if the RT_DEVICE_FLAG_STANDALONE parameter is included in the parameters specified when the device is registered.
Other Error Code	device failed to be turned on

oflags supports the following parameters:

```
#define RT_DEVICE_OFLAG_CLOSE 0x000   /* device was already closed(internal use)*/
#define RT_DEVICE_OFLAG_RDONLY 0x001  /* open the device in read-only mode */
#define RT_DEVICE_OFLAG_WRONLY 0x002  /* open the device in write-only mode */
#define RT_DEVICE_OFLAG_RDWR 0x003    /* open the device in read-and_write mode */
#define RT_DEVICE_OFLAG_OPEN 0x008    /* device was already closed(internal use) */
#define RT_DEVICE_FLAG_STREAM 0x040   /* open the device in stream mode */
#define RT_DEVICE_FLAG_INT_RX 0x100   /* open the device in interrupt reception mode */
#define RT_DEVICE_FLAG_DMA_RX 0x200   /* open the device in DMA mode */
#define RT_DEVICE_FLAG_INT_TX 0x400   /* open the device in interrupt sending mode */
#define RT_DEVICE_FLAG_DMA_TX 0x800   /* open the device in DMA mode */
```

>If the upper application needs to set the device's receive callback function, it must open the device as RT_↩ DEVICE_FLAG_INT_RX or RT_DEVICE_FLAG_DMA_RX, otherwise the callback function will not be called.

After the application opens the device to complete reading and writing, if no further operations are needed, you can close the device using the following functions:

```
rt_err_t rt_device_close(rt_device_t dev);
```

Parameters	Description
dev	device handle
Return	–
RT_EOK	device successfully closed
-RT_ERROR	device has been completely closed and cannot be closed repeatedly
Other Error Code	failed to close device

>Device interfaces `rt_device_open()` and `rt_device_close()` need to used in pairs. Open a device requires close the device, so that the device will be completely closed, otherwise the device will remain on.

1.4.2.1.3.4 Control Device

By commanding the control word, the application can also control the device with the following function:

```
rt_err_t rt_device_control(rt_device_t dev, rt_uint8_t cmd, void* arg);
```

Parameters	Description
dev	device handle
cmd	command control word, this parameter is usually related to the device driver
arg	controlled parameter
Return	–
RT_EOK	function executed successfully
-RT_ENOSYS	execution failed, dev is empty
Other Error Code	execution failed

The generic device command for the parameter `cmd` can be defined as follows:

```
#define RT_DEVICE_CTRL_RESUME           0x01   /* resume device */
#define RT_DEVICE_CTRL_SUSPEND          0x02   /* suspend device */
#define RT_DEVICE_CTRL_CONFIG           0x03   /* configure device */
#define RT_DEVICE_CTRL_SET_INT          0x10   /* set interrupt */
#define RT_DEVICE_CTRL_CLR_INT          0x11   /* clear interrupt */
#define RT_DEVICE_CTRL_GET_INT          0x12   /* obtain interrupt status */
```

1.4.2.1.3.5 Read and Write Device

Application can read data from the device by the following function:

```
rt_size_t rt_device_read(rt_device_t dev, rt_off_t pos,void* buffer, rt_size_t size);
```

Parameters	Description
dev	device handle

Parameters	Description
pos	read data offset
buffer	memory buffer pointer, the data read will be saved in the buffer
size	size of the data read
Return	–
Actual Size of the Data Read	If it is a character device, the return size is in bytes. If it is a block device, the returned size is in block units.
0	need to read the current thread's errno to determine the error status

Calling this function will read the data from the dev device and store it in the buffer. The maximum length of this buffer is *size*, and *pos* has different meanings depending on the device class.

Writing data to the device can be done by the following function:

```
rt_size_t rt_device_write(rt_device_t dev, rt_off_t pos,const void* buffer, rt_size_t size);
```

Parameters	Description
dev	device handle
pos	write data offset
buffer	memory buffer pointer, placing the data to be written in
size	size of the written data
Return	–
Actual Size of the Data Written	If it is a character device, the return size is in bytes. If it is a block device, the returned size is in block units.
0	need to read the current thread's errno to determine the error status

Calling this function will write the data in the buffer to the *dev* device . The maximum length of the written data is *size*, and *pos* has different meanings depending on the device class.

1.4.2.1.3.6 Data Transceiving and Call-back

When the hardware device receives the data, the following function can be used to call back another function to set the data receiving indication to notify the upper application thread that the data arrives:

```
rt_err_t rt_device_set_rx_indicate(rt_device_t dev, rt_err_t (*rx_ind)(rt_device_t dev,rt_size_t size));
```

Parameters	Description
dev	device handle
rx_ind	callback function pointer
Return	–
RT_EOK	set successfully

The callback of this function will be provided by the user. When the hardware device receives the data, it will perform the callback function and pass the received data length to the upper layer application in the *size* parameter. The upper application thread should read the data from the device as soon as it receives the indication.

When the application calls `rt_device_write()` to write data, if the bottom hardware can support automatic sending, the upper application can set a callback function. This callback function is called after the bottom hardware

data has been sent (for example, when the DMA transfer is complete or the FIFO has been written to complete, triggered interrupt). Use the following function to set the device with a send completion indication. The function parameters and return values are as follows:

```
rt_err_t rt_device_set_tx_complete(rt_device_t dev, rt_err_t (*tx_done)(rt_device_t dev,void *buffer));
```

Parameters	Description
dev	device handle
tx_done	callback function pointer
Return	–
RT_EOK	set successfully

When this function is called, the callback function is provided by the user. When the hardware device sends the data, the driver calls back the function and passes the sent data block address buffer as a parameter to the upper application. When the upper layer application (thread) receives the indication, it will release the buffer memory block or use it as the buffer for the next write data according to the condition of sending the buffer.

1.4.2.1.3.7 Access Device Sample

The following code is an example of accessing a device. First, find the watchdog device through the `rt_device_find()` port, obtain the device handle, then initialize the device through the `rt_device_init()` port, and set the watchdog device timeout through the `rt_device_control()` port.

```
#include <rtthread.h>
#include <rtdevice.h>

#define IWDG_DEVICE_NAME    "iwg"

static rt_device_t wdg_dev;

static void idle_hook(void)
{
    /* feed the dog in the callback function of the idle thread */
    rt_device_control(wdg_dev, RT_DEVICE_CTRL_WDT_KEEPALIVE, NULL);
    rt_kprintf("feed the dog!\n ");
}

int main(void)
{
    rt_err_t res = RT_EOK;
    rt_uint32_t timeout = 1000;    /* timeout */

    /* find the watchdog device based on the device name, and obtain the device handle */
    wdg_dev = rt_device_find(IWDG_DEVICE_NAME);
    if (!wdg_dev)
    {
        rt_kprintf("find %s failed!\n", IWDG_DEVICE_NAME);
        return -RT_ERROR;
    }
    /* initialize device */
    res = rt_device_init(wdg_dev);
    if (res != RT_EOK)
    {
        rt_kprintf("initialize %s failed!\n", IWDG_DEVICE_NAME);
        return res;
    }
    /* set watchdog timeout */
    res = rt_device_control(wdg_dev, RT_DEVICE_CTRL_WDT_SET_TIMEOUT, &timeout);
    if (res != RT_EOK)
    {
        rt_kprintf("set %s timeout failed!\n", IWDG_DEVICE_NAME);
        return res;
    }
    /* set idle thread callback function */
    rt_thread_idle_sethook(idle_hook);

    return res;
}
```

1.4.2.2 PIN Device

GPIO device model (DM)

1.4.2.2.1 Introduction of Pin

With **RT_USING_DM**, GPIO controllers use a global virtual pin namespace and device-tree **gpios** properties — see GPIO device model (DM). This page documents the **rt_pin_**∗ application API (BSP **GET_PIN** macros apply to legacy non-DM setups).

The pins on the chip are generally divided into four categories: power supply, clock, control, and I/O. The I/O pins are further divided into General Purpose Input Output (GPIO) and function-multiplexed I/O (such as SPI/I2C/UART, etc.) pins, referring to their usage mode.

Most MCU pins have more than one function. Their internal structure is different and their supported functionality are different. The actual function of the pin can be switched through different configurations. The main features of the General Purpose Input Output (GPIO) port are as follows:

- Programmable Interrupt: The interrupt trigger mode is configurable. Generally, there are five interrupt trigger modes as shown in the following figure:

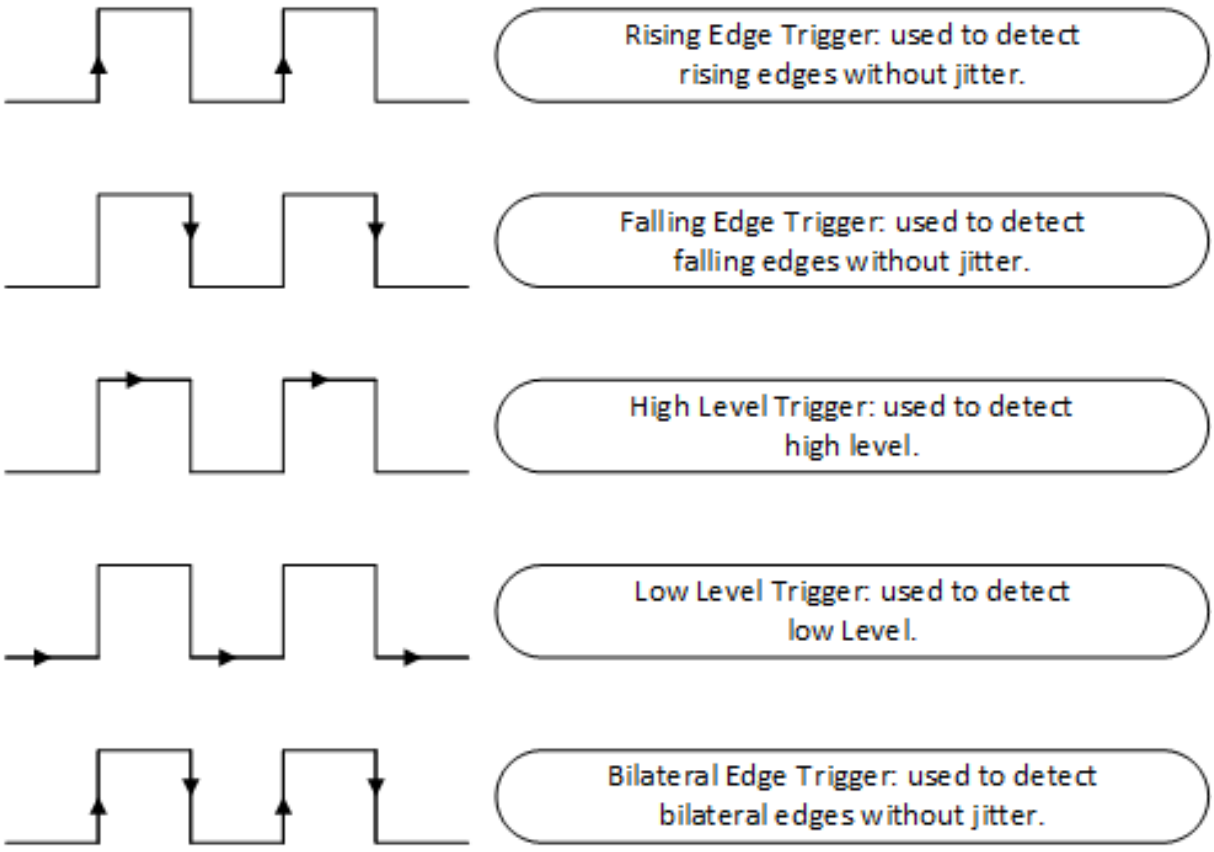


Figure 1.94 5 Interrupt Trigger Modes

- Input and output modes can be controlled.
  - Output modes generally include Output push-pull, Output open-drain, Output pull-up, and Output pull-down. When the pin is in the output mode, the connected peripherals can be controlled by configuring the level of the pin output to be high or low.
  - Input modes generally include: Input floating, Input pull-up, Input pull-down, and Analog. When the pin is in the input mode, the level state of the pin can be read, that is, high level or low level.

1.4.2.2.2 Access PIN Device

The application accesses the GPIO through the PIN device management interface provided by RT-Thread. The related interfaces are as follows:

Function	Description
rt_pin_mode()	Set pin mode
rt_pin_write()	Set the pin level
rt_pin_read()	Read pin level
rt_pin_attach_irq()	Bind pin interrupt callback function
rt_pin_irq_enable()	Enable pin interrupt
rt_pin_detach_irq()	Detach pin interrupt callback function

1.4.2.2.2.1 Obtain Pin Number

The pin numbers provided by RT-Thread need to be distinguished from the chip pin numbers, which not the same. The pin numbers are defined by the PIN device driver and are related to the specific chip used. There are two ways to obtain the pin number: use the macro definition or view the PIN driver file.

Use Macro Definition

If you use the BSP in the `rt-thread/bsp/stm32` directory, you can use the following macro to obtain the pin number:

```
GET_PIN(port, pin)
```

The sample code for the pin number corresponding to LED0 with pin number PF9 is as follows:

```
#define LED0_PIN        GET_PIN(F, 9)
```

View Driver Files

If you use a different BSP, you will need to check the PIN driver code `drv_gpio.c` file to confirm the pin number. There is an array in this file that holds the number information for each PIN pin, as shown below:

```
static const rt_uint16_t pins[] =
{
    __STM32_PIN_DEFAULT,
    __STM32_PIN_DEFAULT,
    __STM32_PIN(2, A, 15),
    __STM32_PIN(3, B, 5),
    __STM32_PIN(4, B, 8),
    __STM32_PIN_DEFAULT,
    __STM32_PIN_DEFAULT,
    __STM32_PIN_DEFAULT,
    __STM32_PIN(8, A, 14),
    __STM32_PIN(9, B, 6),
    ... ...
}
```

Take `__STM32_PIN(2, A, 15)` as an example, 2 is the pin number used by RT-Thread, A is the port number, and 15 is the pin number, so the pin number corresponding to PA15 is 2.

1.4.2.2.2.2 Set Pin Mode

Before the pin is used, you need to set the input or output mode first, and the following functions are used:

```
void rt_pin_mode(rt_base_t pin, rt_base_t mode);
```

Parameter	Discription
pin	Pin number
mode	Pin operation mode

At present, the pin working mode supported by RT-Thread can take one of the five macro definition values as shown. The mode supported by the chip corresponding to each mode needs to refer to the specific implementation of the PIN device driver:

```
#define PIN_MODE_OUTPUT 0x00            /* Output */
#define PIN_MODE_INPUT 0x01             /* Input */
#define PIN_MODE_INPUT_PULLUP 0x02      /* input Pull up  */
#define PIN_MODE_INPUT_PULLDOWN 0x03    /* input Pull down  */
#define PIN_MODE_OUTPUT_OD 0x04         /* output Open drain  */
```

An example of use is as follows:

```
#define BEEP_PIN_NUM            35  /* PB0 */

/* Buzzer pin is in output mode */
rt_pin_mode(BEEP_PIN_NUM, PIN_MODE_OUTPUT);
```

1.4.2.2.2.3 Set The Pin Level

The function to set the pin output level is as follows:

```
void rt_pin_write(rt_base_t pin, rt_base_t value);
```

Parameter	Discription
pin	Pin number
value	Level logic value, which can take one of two macro definition values: PIN_LOW means low level, or PIN_HIGH means high level

Examples of use are as follows:

```
#define BEEP_PIN_NUM            35  /* PB0 */

/* Beep's pin is in output mode */
rt_pin_mode(BEEP_PIN_NUM, PIN_MODE_OUTPUT);
/* Set low level */
rt_pin_write(BEEP_PIN_NUM, PIN_LOW);
```

1.4.2.2.2.4 Read Pin Level

The functions to read the pin level are as follows:

```
int rt_pin_read(rt_base_t pin);
```

Parameter	Description
pin	Pin number
return	——

Parameter	Description
PIN_LOW	Low level
PIN_HIGH	High level

Examples of use are as follows:

```
#define BEEP_PIN_NUM            35  /* PB0 */
int status;

/* Buzzer pin is in output mode */
rt_pin_mode(BEEP_PIN_NUM, PIN_MODE_OUTPUT);
/* Set low level */
rt_pin_write(BEEP_PIN_NUM, PIN_LOW);

status = rt_pin_read(BEEP_PIN_NUM);
```

1.4.2.2.2.5 Bind Pin Interrupt Callback Function

To use the interrupt functionality of a pin, you can use the following function to configure the pin to some interrupt trigger mode and bind an interrupt callback function to the corresponding pin. When the pin interrupt occurs, the callback function will be executed.:

```
rt_err_t rt_pin_attach_irq(rt_int32_t pin, rt_uint32_t mode,
                           void (*hdr)(void *args), void *args);
```

Parameter	Description
pin	Pin number
mode	Interrupt trigger mode
hdr	Interrupt callback function. Users need to define this function
args	Interrupt the parameters of the callback function, set to RT_NULL when not needed
return	——
RT_EOK	Binding succeeded
error code	Binding failed

Interrupt trigger mode mode can take one of the following five macro definition values:

```
#define PIN_IRQ_MODE_RISING 0x00         /* Rising edge trigger */
#define PIN_IRQ_MODE_FALLING 0x01        /* Falling edge trigger */
#define PIN_IRQ_MODE_RISING_FALLING 0x02 /* Edge trigger (triggered on both rising and falling edges)*/
#define PIN_IRQ_MODE_HIGH_LEVEL 0x03     /* High level trigger */
#define PIN_IRQ_MODE_LOW_LEVEL 0x04      /* Low level trigger */
```

Examples of use are as follows:

```
#define KEY0_PIN_NUM            55  /* PD8 */
/* Interrupt callback function */
void beep_on(void *args)
{
    rt_kprintf("turn on beep!\n");

    rt_pin_write(BEEP_PIN_NUM, PIN_HIGH);
}
static void pin_beep_sample(void)
{
    /* Button 0 pin is the input mode */
    rt_pin_mode(KEY0_PIN_NUM, PIN_MODE_INPUT_PULLUP);
    /* Bind interrupt, rising edge mode, callback function named beep_on */
    rt_pin_attach_irq(KEY0_PIN_NUM, PIN_IRQ_MODE_FALLING, beep_on, RT_NULL);
}
```

1.4.2.2.2.6 Enable Pin Interrupt

After binding the pin interrupt callback function, use the following function to enable pin interrupt:

```
rt_err_t rt_pin_irq_enable(rt_base_t pin, rt_uint32_t enabled);
```

Parameter	Description
pin	Pin number
enabled	Status, one of two values: PIN_IRQ_ENABLE, and PIN_IRQ_DISABLE
return	——
RT_EOK	Enablement succeeded
error code	Enablement failed

Examples of use are as follows:

```
#define KEY0_PIN_NUM            55  /* PD8 */
/* Interrupt callback function */
void beep_on(void *args)
{
    rt_kprintf("turn on beep!\n");

    rt_pin_write(BEEP_PIN_NUM, PIN_HIGH);
}
static void pin_beep_sample(void)
{
    /* Key 0 pin is the input mode */
    rt_pin_mode(KEY0_PIN_NUM, PIN_MODE_INPUT_PULLUP);
    /* Bind interrupt, rising edge mode, callback function named beep_on */
    rt_pin_attach_irq(KEY0_PIN_NUM, PIN_IRQ_MODE_FALLING, beep_on, RT_NULL);
    /* Enable interrupt */
    rt_pin_irq_enable(KEY0_PIN_NUM, PIN_IRQ_ENABLE);
}
```

1.4.2.2.2.7 Detach Pin Interrupt Callback Function

You can use the following function to detach the pin interrupt callback function:

```
rt_err_t rt_pin_detach_irq(rt_int32_t pin);
```

Parameter	Description
pin	Pin number
return	——
RT_EOK	Detachment succeeded
error code	Detachment failed

After the pin detaches the interrupt callback function, the interrupt is not closed. You can also call the bind interrupt callback function to bind the other callback functions again.

```
#define KEY0_PIN_NUM            55  /* PD8 */
/* Interrupt callback function */
void beep_on(void *args)
{
    rt_kprintf("turn on beep!\n");

    rt_pin_write(BEEP_PIN_NUM, PIN_HIGH);
}
static void pin_beep_sample(void)
{
    /* Key 0 pin is the input mode */
    rt_pin_mode(KEY0_PIN_NUM, PIN_MODE_INPUT_PULLUP);
    /* Bind interrupt, rising edge mode, callback function named beep_on */
    rt_pin_attach_irq(KEY0_PIN_NUM, PIN_IRQ_MODE_FALLING, beep_on, RT_NULL);
    /* Enable interrupt */
    rt_pin_irq_enable(KEY0_PIN_NUM, PIN_IRQ_ENABLE);
    /* Detach interrupt callback function */
    rt_pin_detach_irq(KEY0_PIN_NUM);
}
```

1.4.2.2.3 PIN Device Usage Example

The following sample code is the pin device usage example. The main steps of the sample code are as follows:

1. Set the corresponding pin of the beep to the output mode and give a default low state.
2. Set the key 0 and button 1 corresponding to the input mode, then bind the interrupt callback function and enable the interrupt.
3. When the key 0 is pressed, the beep starts to sound, and when the key 1 is pressed, the beep stops.

```
/*
 * Program listing: This is a PIN device usage routine
 * The routine exports the pin_beep_sample command to the control terminal
 * Command call formatpin_beep_sample
 * Program function: control the buzzer by controlling the level state of the corresponding pin of the
     buzzer by pressing the button
*/

#include <rtthread.h>
#include <rtdevice.h>

/* Pin number, determined by looking at the device driver file drv_gpio.c */
#ifndef BEEP_PIN_NUM
    #define BEEP_PIN_NUM            35  /* PB0 */
#endif
#ifndef KEY0_PIN_NUM
    #define KEY0_PIN_NUM            55  /* PD8 */
#endif
#ifndef KEY1_PIN_NUM
    #define KEY1_PIN_NUM            56  /* PD9 */
#endif

void beep_on(void *args)
{
    rt_kprintf("turn on beep!\n");

    rt_pin_write(BEEP_PIN_NUM, PIN_HIGH);
}

void beep_off(void *args)
{
    rt_kprintf("turn off beep!\n");

    rt_pin_write(BEEP_PIN_NUM, PIN_LOW);
}

static void pin_beep_sample(void)
{
    /* Beep pin is in output mode */
    rt_pin_mode(BEEP_PIN_NUM, PIN_MODE_OUTPUT);
    /* Default low level */
    rt_pin_write(BEEP_PIN_NUM, PIN_LOW);

    /* KEY 0 pin is the input mode */
    rt_pin_mode(KEY0_PIN_NUM, PIN_MODE_INPUT_PULLUP);
    /* Bind interrupt, falling edge mode, callback function named beep_on */
    rt_pin_attach_irq(KEY0_PIN_NUM, PIN_IRQ_MODE_FALLING, beep_on, RT_NULL);
    /* Enable interrupt */
    rt_pin_irq_enable(KEY0_PIN_NUM, PIN_IRQ_ENABLE);

    /* KEY 1 pin is input mode */
    rt_pin_mode(KEY1_PIN_NUM, PIN_MODE_INPUT_PULLUP);
    /* Binding interrupt, falling edge mode, callback function named beep_off */
    rt_pin_attach_irq(KEY1_PIN_NUM, PIN_IRQ_MODE_FALLING, beep_off, RT_NULL);
    /* Enable interrupt */
    rt_pin_irq_enable(KEY1_PIN_NUM, PIN_IRQ_ENABLE);
}
/* Export to the msh command list */
MSH_CMD_EXPORT(pin_beep_sample, pin beep sample);
```

1.4.2.2.4 GPIO device model (DM)

1.4.2.2.4.1 GPIO / PIN under `RT_USING_DM`

`RT_USING_PIN` with **`RT_USING_DM`** adds a **global GPIO namespace**: each platform GPIO controller registers a contiguous **virtual pin number** range. Application code still uses **`rt_pin_mode`** / **`rt_pin_write`** / **`rt_pin_read`** (PIN Device); drivers use **`rt_pin_get_named_pin`** and optional **GPIO IRQ domains**.

Sources:

File	Role
dev_pin_↩ dm.c	Global pin table, **pin_api_init**, **pin_pic_init**, DM **rt_pin_ops** shim
dev_pin_↩ ofw.c	**rt_ofw_get_named_pin** — `gpios` / `gpio` properties
dev_↩ pin.c	Legacy single **gpio** device when no DM controllers
pin-pl061.↩ c	Reference platform driver (`arm,pl061`)

Kconfig

Option	Role
RT_USING_PIN	GPIO framework (default **y**)
RT_USING_DM	Required for DM path described here
RT_PIN_PL061	In-tree PL061 driver
SOC_DM_PIN_DIR	BSP adds SoC GPIO drivers via **osource**

Architecture

```
DT: gpio-controller node (#gpio-cells, interrupt-controller optional)
      |
      |  platform probe
      v
struct rt_device_pin + rt_pin_ops (per controller)
      |
      +-- pin_api_init(gpio, pin_nr)     → global pin_start..pin_start+nr-1
      +-- pin_pic_init(gpio, irq)        → optional per-line PIC domain
      +-- rt_dm_dev_bind_fwdata → rt_ofw_data(np) = gpio
      |
      v
First controller also: rt_device_pin_register("gpio", pin_api_dm_ops)
      |
      v
Consumers:
      rt_pin_get_named_pin(dev, "foo", index, &mode, &val)
      rt_pin_mode(pin, mode); rt_pin_write(pin, val);
```

Concept	Meaning
Local pin	Index `0 ... pin_nr-1` passed to **rt_pin_ops**
Virtual pin	**pin_start + local** — value for **rt_pin_*** and **rt_pin_get_named_pin**
pin_device_find	Maps virtual pin → owning **struct rt_device_pin**

Controllers are linked on **pin_nodes**; **pin_total_nr** only grows (ranges are not recycled).

Platform driver checklist

1. **struct rt_device_pin** (or embed as first member of private struct).
2. Implement **struct rt_pin_ops**: at minimum **pin_mode**, **pin_write**, **pin_read**.
3. **probe**: iomap, clock, **rt_dm_dev_bind_fwdata(dev, NULL, &gpio->parent)**.
4. **pin_api_init(&gpio->parent, gpio_count)** — assigns **pin_start**.

5. If GPIO has a shared IRQ line:
   - **irq = rt_dm_dev_get_irq(dev, 0)**
   - **pin_pic_init(&gpio->parent, irq)**
   - Hardware ISR: loop pending bits → **pin_pic_handle_isr(&gpio->parent, local_pin)**
   - Install **rt_hw_interrupt_install(irq, ...)** + **umask**
6. Optional **pin_parse** for non-default **#gpio-cells** (flags from **dt-bindings/pin/pin.h**).

Reference: **pin-pl061.c** (pl061_probe).

struct rt_pin_ops (DM-relevant)

Callback	Role
pin_mode	Input / output / pull / open-drain
pin_write / **pin_read**	Level
pin_attach_irq	Direct per-pin ISR (optional)
pin_detach_irq	Remove handler
pin_irq_enable	Enable/disable line interrupt
pin_irq_mode	Edge/level when using legacy IRQ path
pin_parse	Decode **#gpio-cells** → local pin + **flags**
pin_get	Name → virtual pin (optional)
pin_debounce	Debounce time

If **pin_attach_irq** is **NULL**, **dev_pin_dm.c** stores handlers in **legacy_isr[]** and relies on **pin_irq_mode** + **pin_pic_handle_isr**.

Device tree: gpios properties

rt_ofw_get_named_pin(np, propname, index, out_mode, out_value) tries:

- **{propname}-gpios**, then **{propname}-gpio**
- If **propname** is **NULL**, tries **gpios** / **gpio**

Flow:

1. **rt_ofw_parse_phandle_cells(..., "#gpio-cells", ...)**
2. **rt_platform_ofw_request** on GPIO provider if needed
3. **pin_parse** or default: **args[0]** = local pin
4. Convert **flags** → **PIN_MODE_***, active level
5. Return **local + pin_dev->pin_start** (virtual pin)

Wrapper on device:

```
rt_ssize_t rt_pin_get_named_pin(struct rt_device *dev, const char *propname,
        int index, rt_uint8_t *out_mode, rt_uint8_t *out_value);
rt_ssize_t rt_pin_get_named_pin_count(struct rt_device *dev, const char *propname);
```

Typical **probe**:

```
pin = rt_pin_get_named_pin(dev, "reset", 0, &mode, &active);
if (pin < 0)
    return pin;
rt_pin_mode(pin, mode);
rt_pin_write(pin, active);   /* or inactive per DT flags */
```

Examples in-tree: **phye-generic-usb.c** (reset), **gpio-restart.c**, **led-gpio.c** (child node), **backlight-gpio.c**.

GPIO IRQ + PIC

struct rt_pin_irqchip is embedded in **struct rt_device_pin** (after **parent**).

API	Role
pin_pic_init(gpio, irq)	**rt_pic_linear_irq**, **legacy_isr** array, **pin_dm_ops**
pin_pic_handle_isr(gpio, local_pin)	Dispatch **PIC** path and/or **legacy_isr** callback

Per-line DT interrupt (2 cells: pin + flags) uses **pin_dm_ops.irq_parse** / **irq_map**:

- Maps hwirq → virtual IRQ
- **rt_pic_cascade(pirq, gpio->irqchip.irq)** — GPIO block IRQ cascades to controller's **rt_dm_dev_get_irq**

For **rt_pin_attach_irq(virtual_pin, ...)** without DT interrupt on that line, legacy path + **pin_irq_enable** on hardware still applies.

Application / driver usage

DM systems should prefer:

```
/* Not GET_PIN(port, n) from old BSP tables */
pin = rt_pin_get_named_pin(&pdev->parent, "led", 0, &mode, NULL);
rt_pin_mode(pin, mode);
```

Legacy **GET_PIN** / **drv_gpio.c** pin tables apply to non-DM BSP GPIO only.

Pitfalls

Issue	Mitigation
Skip **pin_api_init**	**rt_ofw_get_named_pin** adds wrong **pin_start**
pin_parse missing	Only **args[0]** used; wrong if `#gpio-cells` > 1 with flags
IRQ without **pin_pic_init**	Shared IRQ controllers need **pin_pic_handle_isr**
pin_attach_irq vs legacy	If **pin_attach_irq** **NULL**, must enable via **pin_irq_enable** after attach
Provider not probed	Ensure **rt_platform_ofw_request** or platform driver registered

---

See also

- PIN Device — **rt_pin_*** application API
- PIC cascade and routing — GPIO PIC cascade
- `components/drivers/include/drivers/dev_pin.h`
- `components/drivers/pin/dev_pin_dm.c`, `dev_pin_ofw.c`

1.4.2.2.5 GPIO device model (DM)

1.4.2.2.5.1 GPIO / PIN under RT_USING_DM

RT_USING_PIN with **RT_USING_DM** adds a **global GPIO namespace**: each platform GPIO controller registers a contiguous **virtual pin number** range. Application code still uses **rt_pin_mode** / **rt_pin_write** / **rt_pin_read** (PIN Device); drivers use **rt_pin_get_named_pin** and optional **GPIO IRQ domains**.
Sources:

File	Role
dev_pin_↩ dm.c	Global pin table, **pin_api_init**, **pin_pic_init**, DM **rt_pin_ops** shim
dev_pin_↩ ofw.c	**rt_ofw_get_named_pin** — `gpios` / `gpio` properties
dev_↩ pin.c	Legacy single **gpio** device when no DM controllers
pin-pl061.↩ c	Reference platform driver (`arm,pl061`)

Kconfig

Option	Role
RT_USING_PIN	GPIO framework (default **y**)
RT_USING_DM	Required for DM path described here
RT_PIN_PL061	In-tree PL061 driver
SOC_DM_PIN_DIR	BSP adds SoC GPIO drivers via **osource**

Architecture

```
DT: gpio-controller node (#gpio-cells, interrupt-controller optional)
      |
      |  platform probe
      v
struct rt_device_pin + rt_pin_ops (per controller)
      |
      +-- pin_api_init(gpio, pin_nr)     → global pin_start..pin_start+nr-1
      +-- pin_pic_init(gpio, irq)        → optional per-line PIC domain
      +-- rt_dm_dev_bind_fwdata → rt_ofw_data(np) = gpio
      |
      v
First controller also: rt_device_pin_register("gpio", pin_api_dm_ops)
      |
      v
Consumers:
      rt_pin_get_named_pin(dev, "foo", index, &mode, &val)
      rt_pin_mode(pin, mode); rt_pin_write(pin, val);
```

Concept	Meaning
Local pin	Index `0 ... pin_nr-1` passed to **rt_pin_ops**
Virtual pin	**pin_start + local** — value for **rt_pin_*** and **rt_pin_get_named_pin**
pin_device_find	Maps virtual pin → owning **struct rt_device_pin**

Controllers are linked on **pin_nodes**; **pin_total_nr** only grows (ranges are not recycled).

Platform driver checklist

1. **struct rt_device_pin** (or embed as first member of private struct).
2. Implement **struct rt_pin_ops**: at minimum **pin_mode**, **pin_write**, **pin_read**.
3. **probe**: iomap, clock, **rt_dm_dev_bind_fwdata(dev, NULL, &gpio->parent)**.
4. **pin_api_init(&gpio->parent, gpio_count)** — assigns **pin_start**.
5. If GPIO has a shared IRQ line:

- **irq = rt_dm_dev_get_irq(dev, 0)**
- **pin_pic_init(&gpio->parent, irq)**
- Hardware ISR: loop pending bits → **pin_pic_handle_isr(&gpio->parent, local_pin)**
- Install **rt_hw_interrupt_install(irq, ...)** + **umask**

6. Optional **pin_parse** for non-default **#gpio-cells** (flags from **dt-bindings/pin/pin.h**).

Reference: **pin-pl061.c** (pl061_probe).

struct rt_pin_ops (DM-relevant)

Callback	Role
pin_mode	Input / output / pull / open-drain
pin_write / **pin_read**	Level
pin_attach_irq	Direct per-pin ISR (optional)
pin_detach_irq	Remove handler
pin_irq_enable	Enable/disable line interrupt
pin_irq_mode	Edge/level when using legacy IRQ path
pin_parse	Decode **#gpio-cells** → local pin + **flags**
pin_get	Name → virtual pin (optional)
pin_debounce	Debounce time

If **pin_attach_irq** is **NULL**, **dev_pin_dm.c** stores handlers in **legacy_isr[]** and relies on **pin_irq_mode** + **pin_pic_handle_isr**.

Device tree: gpios properties

rt_ofw_get_named_pin(np, propname, index, out_mode, out_value) tries:

- **{propname}-gpios**, then **{propname}-gpio**
- If **propname** is **NULL**, tries **gpios** / **gpio**

Flow:

1. **rt_ofw_parse_phandle_cells(..., "#gpio-cells", ...)**
2. **rt_platform_ofw_request** on GPIO provider if needed
3. **pin_parse** or default: **args[0]** = local pin
4. Convert **flags** → **PIN_MODE_***, active level
5. Return **local + pin_dev->pin_start** (virtual pin)

Wrapper on device:

```
rt_ssize_t rt_pin_get_named_pin(struct rt_device *dev, const char *propname,
        int index, rt_uint8_t *out_mode, rt_uint8_t *out_value);
rt_ssize_t rt_pin_get_named_pin_count(struct rt_device *dev, const char *propname);
```

Typical **probe**:

```
pin = rt_pin_get_named_pin(dev, "reset", 0, &mode, &active);
if (pin < 0)
    return pin;
rt_pin_mode(pin, mode);
rt_pin_write(pin, active);   /* or inactive per DT flags */
```

Examples in-tree: **phye-generic-usb.c** (reset), **gpio-restart.c**, **led-gpio.c** (child node), **backlight-gpio.c**.

GPIO IRQ + PIC

struct rt_pin_irqchip is embedded in **struct rt_device_pin** (after **parent**).

API	Role
pin_pic_init(gpio, irq)	**rt_pic_linear_irq**, **legacy_isr** array, **pin_dm_ops**
pin_pic_handle_isr(gpio, local_pin)	Dispatch **PIC** path and/or **legacy_isr** callback

Per-line DT interrupt (2 cells: pin + flags) uses **pin_dm_ops.irq_parse** / **irq_map**:

- Maps hwirq → virtual IRQ
- **rt_pic_cascade(pirq, gpio->irqchip.irq)** — GPIO block IRQ cascades to controller's **rt_dm_dev_get_irq**

For **rt_pin_attach_irq(virtual_pin, ...)** without DT interrupt on that line, legacy path + **pin_irq_enable** on hardware still applies.

Application / driver usage

DM systems should prefer:

```
/* Not GET_PIN(port, n) from old BSP tables */
pin = rt_pin_get_named_pin(&pdev->parent, "led", 0, &mode, NULL);
rt_pin_mode(pin, mode);
```

Legacy **GET_PIN** / **drv_gpio.c** pin tables apply to non-DM BSP GPIO only.

Pitfalls

Issue	Mitigation
Skip **pin_api_init**	**rt_ofw_get_named_pin** adds wrong **pin_start**
pin_parse missing	Only **args[0]** used; wrong if `#gpio-cells` > 1 with flags
IRQ without **pin_pic_init**	Shared IRQ controllers need **pin_pic_handle_isr**
pin_attach_irq vs legacy	If **pin_attach_irq** **NULL**, must enable via **pin_irq_enable** after attach
Provider not probed	Ensure **rt_platform_ofw_request** or platform driver registered

See also

- PIN Device — **rt_pin_*** application API
- PIC cascade and routing — GPIO PIC cascade
- `components/drivers/include/drivers/dev_pin.h`
- `components/drivers/pin/dev_pin_dm.c`, `dev_pin_ofw.c`

1.4.2.3 Pin control (pinctrl)

1.4.2.3.1 Pinctrl subsystem

Pinctrl applies **device-tree pin configurations** (mux, pull, drive strength, and related pad settings) before or during driver **probe**.

- Core: **components/drivers/pinctrl/pinctrl.c**
- API / **PIN_CONFIG_***: **components/drivers/include/drivers/dev_pin.h**

Kconfig: **RT_USING_PINCTRL** (requires **RT_USING_DM** and **RT_USING_PIN**). BSP may add more providers via **SOC_DM_PINCTRL_DIR**.

1.4.2.3.1.1 End-to-end flow

1. Device tree (consumer)

The **consumer** node (UART, I2C, SPI, ...) references **config subnodes** under a pinctrl controller:

Property	Role
`pinctrl-0`, **`pinctrl-1`**, ...	Phandles to config subnodes (one **state** per index)
`pinctrl-names`	Names for each index: `"default"`, `"sleep"`, `"active"`,...

```
uart0: serial@40000000 {
    compatible = "vendor,uart";
    reg = <0x40000000 0x1000>;
    pinctrl-names = "default", "sleep";
    pinctrl-0 = <&uart0_pins>;
    pinctrl-1 = <&uart0_sleep_pins>;
};

pinctrl: pinctrl@20000000 {
    compatible = "vendor,pinctrl";
    reg = <0x20000000 0x1000>;

    uart0_pins: uart0-pins {
        /* properties interpreted by the provider driver */
    };
};
```

`pinctrl-0` may contain **several** phandles (e.g. separate SCL/SDA nodes). They are applied **in order**; the first error stops the rest.
Phandles must target a **config child** of the controller, not the controller root nor the consumer itself (see comment in **`ofw_pin_ctrl_confs_apply()`**).

2. Core apply (`pinctrl.c`)

For **`rt_pin_ctrl_confs_apply(dev, index)`** (or after name lookup):

```
foreach phandle in consumer's "pinctrl-<index>":
    conf_np  = node from phandle
    walk parents until node has "compatible"  →  provider_np
    if !rt_ofw_data(provider_np):
        rt_platform_ofw_request(provider_np)
    pinctrl = rt_ofw_data(provider_np)   /* struct rt_device_pin * */
    pinctrl->ops->pin_ctrl_confs_apply(&pinctrl->parent, conf_np)
```

3. Provider

The **provider** is a platform driver for the pinctrl controller. At **`probe`** it typically:

1. Maps **`reg`**, enables clock/reset.
2. Sets **`struct rt_pin_ops.pin_ctrl_confs_apply`** to parse **`fw_conf_np`** and program hardware.
3. Sets **`rt_ofw_data(controller_ofw_node) = &struct rt_device_pin.parent`**.

Optional: **`pin_ctrl_gpio_request`** — pad setup when the GPIO layer claims a line (**`pin_gpio_request`** in **`dev_pin_dm.c`**).

4. Platform bus (automatic)

`platform_probe()` runs **before** the driver **`probe`**:

```
#ifdef RT_USING_PINCTRL
    if (rt_pin_ctrl_confs_apply_by_name(dev, RT_NULL))   /* "default" */
    {
        rt_pin_ctrl_confs_apply(dev, 0);                 /* fallback pinctrl-0 */
    }
#endif
```

So most drivers only need correct DT; they do **not** need to call pinctrl in **`probe`** unless switching states at runtime.
Full platform order: Platform bus.

1.4.2.3.1.2 Usage

Consumer driver (typical)

Boot-time mux — no code; define **pinctrl-names** + **pinctrl-0** (and extra states if needed).
Runtime state change (sleep, high-speed, etc.):

```
/* By name */
rt_pin_ctrl_confs_apply_by_name(&pdev->parent, "active");

/* By index */
rt_ssize_t idx = rt_pin_ctrl_confs_lookup(&pdev->parent, "high-speed");
if (idx >= 0)
    rt_pin_ctrl_confs_apply(&pdev->parent, (int)idx);
```

name == NULL in **rt_pin_ctrl_confs_apply_by_name** means **"default"**.
Some drivers (**regulator-fixed**, **led-gpio**, ...) call apply again before using GPIO-related pads so DT and hardware stay aligned.

Public API

API	Role
rt_pin_ctrl_confs_lookup(dev, name)	Index in **pinctrl-names**; **-RT_EEMPTY** if missing
rt_pin_ctrl_confs_apply(dev, index)	Apply all phandles in **pinctrl-<index>**
rt_pin_ctrl_confs_apply_by_name(dev, name)	Lookup + apply

Without **RT_USING_OFW**: **-RT_ENOSYS**.

Return	Meaning
RT_EOK	State applied
-RT_EEMPTY	No property / no name
-RT_ERROR	Provider missing or no **pin_ctrl_confs_apply**
-RT_EIO	Bad phandle

Provider driver (BSP)

1. Embed **struct rt_device_pin** (usually first member of private struct).
2. Implement **pin_ctrl_confs_apply(device, fw_conf_np)** — read provider-specific properties on **fw_conf_np** and write pad registers (or firmware, etc.).
3. **rt_ofw_data(dev->ofw_node) = &pin_dev->parent** at end of probe.
4. Document your binding (property names and cell layout) in the BSP; the core does not parse SoC-specific tuples.

PIN_CONFIG_* in **dev_pin.h** are generic semantic types; providers map DT properties to hardware as needed.

1.4.2.3.1.3 DT layout example (multi-phandle)

From **pinctrl.c** comments — one consumer, two phandles, two config nodes:

```
i2c@4700 {
    pinctrl-names = "default";
    pinctrl-0 = <&i2c_pin_scl, &i2c_pin_sda>;
};

&pinctrl {
    i2c_pin_scl { /* provider-specific */ };
    i2c_pin_sda { /* provider-specific */ };
};
```

1.4.2.3.1.4 Pitfalls

Issue	Mitigation
Provider not probed	Register platform driver; core calls **`rt_platform_ofw_request`**
Wrong phandle	Must point to **config subnode** under controller
Name / index mismatch	**`rt_pin_ctrl_confs_lookup`** string must match **`pinctrl-names`**
Partial apply	First failing phandle aborts the list
No **`rt_ofw_data`**	Provider **`probe`** must set it on controller node
Runtime PM	Add **`pinctrl-1`** + **`"sleep"`**; **`apply_by_name`** on suspend/resume

1.4.2.3.1.5 See also

- Platform bus — probe order
- Open Firmware (OFW) — phandles
- `components/drivers/pinctrl/pinctrl.c`
- `components/drivers/core/platform.c`

1.4.2.4 UART

1.4.2.4.1 UART (serial) device

UART (Universal Asynchronous Receiver/Transmitter) is the usual **character-device** path for console, AT commands, and board bring-up. In RT-Thread the **driver stack** lives under **`components/drivers/serial/`** (Kconfig **`RT_USING_SERIAL`**), while applications see devices named **`uart0`**, **`uart1`**, ...

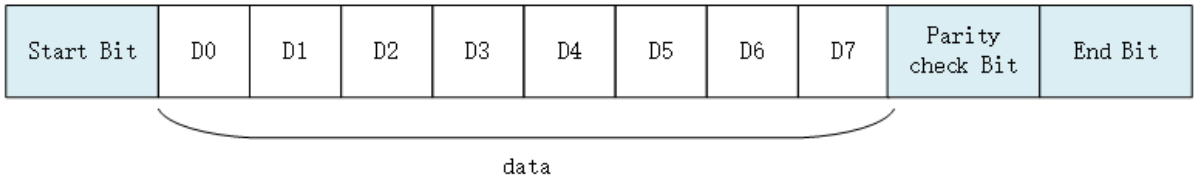


Figure 1.95 Serial transmission data format

Layer	Doc / source
Application (`rt_device_*`)	This page — **Application access**
Framework (`rt_serial_device`, V1/V2)	`dev_serial.c` / `dev_serial_v2.c`, `dev_serial.h`
DM naming, bootargs	UART device model (DM) — `serial_dm.c`
Early printk	UART early console
Controllers (8250, PL011, ...)	`serial/device/*`, platform **`RT_PLATFORM_DRIVER_EXPORT`**

Kconfig: **`RT_USING_SERIAL`** (default on). **`RT_USING_SERIAL_V1`** vs **`RT_USING_SERIAL_V2`**; optional **`RT_SERIAL_USING_DMA`**, **`RT_USING_SERIAL_BYPASS`**. SoC drivers: **`RT_USING_DM && RT_USING_SERIAL`** → `serial/device/Kconfig`.

1.4.2.4.1.1 Driver writer (quick path)

1. Implement **`struct rt_uart_ops`** / serial ops: **`configure`**, **`control`**, **`putc`**, **`getc`**, optional **`dma_transmit`**.

2. With DM + OFW: map MMIO/IRQ, clocks/resets, **serial_dev_set_name**, then **rt_hw_serial_register** (see UART device model (DM)).
3. Report RX/TX completion via **rt_hw_serial_isr(serial, RT_SERIAL_EVENT_*)**.
4. Apply **pinctrl** and regulators (if any) before enabling TX.

1.4.2.4.1.2 Application access

The application uses the **generic device interface** (rtdevice.h). The UART is a **character device**; **struct serial_configure** and **RT_DEVICE_CTRL_CONFIG** are in **drivers/dev_serial.h** (V1) or **dev_serial_v2.h** (V2).
Main APIs:

Funtion	Description
rt_device_find()	find device
rt_device_open()	open device
rt_device_read()	read device
rt_device_write()	write device
rt_device_control()	control device
rt_device_set_rx_indicate()	set receive callback function
rt_device_set_tx_complete()	set send complete callback function
rt_device_close()	close device

1.4.2.4.1.3 Find UART Device

The application obtains the device handle according to the uart device name, and then can operate the uart device.The device find function is shown below

```
rt_device_t rt_device_find(const char* name);
```

Parameter	Description
name	device's name
back	——
device handle	finding the corresponding device will return to the corresponding device handle
RT_NULL	corresponding device object was not found

Generally, the name of the uart device registered to the system is uart0, uart1, etc. samples are as follows:

```
#define SAMPLE_UART_NAME       "uart2"  /* uart device name */
static rt_device_t serial;              /* uart device handle */
/* Find uart device*/
serial = rt_device_find(SAMPLE_UART_NAME);
```

1.4.2.4.1.4 Open UART Device

Through the device handle, the application can open and close the device. When the device is opened, it will detect whether the device has been initialized. If it is not initialized, it will call the initialization interface to initialize the device by default. Open the device through the following functions:

```
rt_err_t rt_device_open(rt_device_t dev, rt_uint16_t oflags);
```

Parameter	Description
dev	device handle
oflags	device mode flags

Parameter	Description
back	——
RT_EOK	device opened successfully
-RT_EBUSY	If the standalone parameter RT_DEVICE_FLAG_STANDALONE is included in the functions specified when the device is registered, the device will not be allowed to be opened repeatedly
Other error codes	device failed to open

oflags parameters support the following values (Use OR logic to support multiple values):

```
#define RT_DEVICE_FLAG_STREAM       0x040     /* Stream mode      */
/* Receive mode function */
#define RT_DEVICE_FLAG_INT_RX       0x100     /* Interrupt receive mode */
#define RT_DEVICE_FLAG_DMA_RX       0x200     /* DMA receiving mode */
/* Transmit mode flags */
#define RT_DEVICE_FLAG_INT_TX       0x400     /* Interrupt transmit mode */
#define RT_DEVICE_FLAG_DMA_TX       0x800     /* DMA transmit mode */
```

There are three modes of uart data receiving and sending: interrupt mode, polling mode and DMA mode. When used, only one of the three modes can be selected. If the open parameter oflag of the serial port does not specify the use of interrupt mode or DMA mode, the polling mode is used by default.
The DMA (Direct Memory Access) transfer mode does not require the CPU to directly control the transfer, and does not have the process of reserving the scene and restoring the scene as they have in the interrupt processing mode. The DMA controller opens a path for directly transferring data to the RAM and the I/O device, which saves CPU resources to do other things. Using DMA transfer can continuously acquire or send a piece of information without taking up interrupts or delays, which is useful when communication is frequent or when large pieces of information are to be transmitted.

> RT_DEVICE_FLAG_STREAM: Stream mode is used to output a string to the serial terminal: when the output character is "\n" (corresponding to the hexadecimal value 0x0A), a "\r" is automatically output in front (corresponding to hexadecimal value is 0x0D).

The stream mode RT_DEVICE_FLAG_STREAM can be used with the receive and send mode parameter with the "|" logic.
An example of using a uart device in **interrupt receive mode and polling mode** as follows:

```
#define SAMPLE_UART_NAME       "uart2"  /* uart device name */
static rt_device_t serial;              /* uart device handle */
/* find uart device */
serial = rt_device_find(SAMPLE_UART_NAME);

/* Open the uart device in interrupt receive mode and polling mode*/
rt_device_open(serial, RT_DEVICE_FLAG_INT_RX);
```

If the uart is to use the DMA receive mode, the oflags takes the value RT_DEVICE_FLAG_DMA_RX. An example of using a uart device in the **DMA receive and polling send mode** is as follows:

```
#define SAMPLE_UART_NAME       "uart2"  /* uart device's name */
static rt_device_t serial;              /* uart device handle */
/* find uart device */
serial = rt_device_find(SAMPLE_UART_NAME);

/* Open the uart device in DMA receive and polling send mode*/
rt_device_open(serial, RT_DEVICE_FLAG_DMA_RX);
```

1.4.2.4.1.5 Control UART Device

Through command control word, the application can configure the uart device by the following function:

```
rt_err_t rt_device_control(rt_device_t dev, rt_uint8_t cmd, void* arg);
```

Parameter	Description
dev	device handle
cmd	command control word can be valued asRT_DEVICE_CTRL_CONFIG
arg	controlled parameter struct serial_configure
Back	——

Parameter	Description
RT_EOK	function executed successfully
-RT_ENOSYS	execution failed, dev is empty
Other error codes	execution failed

- The prototype of control parameter structure: struct serial_configure is as follows

```
struct serial_configure
{
    rt_uint32_t baud_rate;          /* Baudrate */
    rt_uint32_t data_bits    :4;    /* Data bit */
    rt_uint32_t stop_bits    :2;    /* Stop bit */
    rt_uint32_t parity       :2;    /* Parity bit */
    rt_uint32_t bit_order    :1;    /* Prioritized by order */
    rt_uint32_t invert       :1;    /* Mode */
    rt_uint32_t bufsz        :16;   /* Receive data buffer size */
    rt_uint32_t reserved     :4;    /* Reserved bit */
};
```

- The default macro configuration provided by RT-Thread is as follows

```
#define RT_SERIAL_CONFIG_DEFAULT           \
{                                          \
    BAUD_RATE_115200, /* 115200 bps */     \
    DATA_BITS_8,      /* 8 databits */     \
    STOP_BITS_1,      /* 1 stopbit */      \
    PARITY_NONE,      /* No parity  */     \
    BIT_ORDER_LSB,    /* LSB first sent */ \
    NRZ_NORMAL,       /* Normal mode */    \
    RT_SERIAL_RB_BUFSZ, /* Buffer size */  \
    0                                      \
}
```

The configuration parameters provided by RT-Thread can be defined as the following macro definitions:

```
/* The baudrate can be defined as*/
#define BAUD_RATE_2400                  2400
#define BAUD_RATE_4800                  4800
#define BAUD_RATE_9600                  9600
#define BAUD_RATE_19200                 19200
#define BAUD_RATE_38400                 38400
#define BAUD_RATE_57600                 57600
#define BAUD_RATE_115200                115200
#define BAUD_RATE_230400                230400
#define BAUD_RATE_460800                460800
#define BAUD_RATE_921600                921600
#define BAUD_RATE_2000000               2000000
#define BAUD_RATE_3000000               3000000
/* Data bits can be defined as*/
#define DATA_BITS_5                     5
#define DATA_BITS_6                     6
#define DATA_BITS_7                     7
#define DATA_BITS_8                     8
#define DATA_BITS_9                     9
/* Stop bits can be defined as */
#define STOP_BITS_1                     0
#define STOP_BITS_2                     1
#define STOP_BITS_3                     2
#define STOP_BITS_4                     3
/* Parity bits can be defined as */
#define PARITY_NONE                     0
#define PARITY_ODD                      1
#define PARITY_EVEN                     2
/* Bit order can be defined as */
#define BIT_ORDER_LSB                   0
#define BIT_ORDER_MSB                   1
/* Mode canbe defined as */
#define NRZ_NORMAL                      0   /* normal mode */
#define NRZ_INVERTED                    1   /* inverted mode */
/* Default size of the receive data buffer */
#define RT_SERIAL_RB_BUFSZ              64
```

Receive Buffer

When the uart device is opened using interrupt receive mode, the uart driver framework will open a buffer according to the size of RT_SERIAL_RB_BUFSZ to save the received data. When the underlying driver receives a data, it will put the data into the buffer in the interrupt service program.

>The default size of the receive data buffer is 64 bytes. If the number of received data in one-time is too large and the data is not read in time, the data of the buffer will be overwritten by the newly received data, resulting in data loss. It is recommended to increase the buffer.

A sample for configuring uart hardware parameters such as data bits, check bits, stop bits, and so on are shown below

```
#define SAMPLE_UART_NAME       "uart2"  /* uart device's name */
static rt_device_t serial;              /* uart device handle */
struct serial_configure config = RT_SERIAL_CONFIG_DEFAULT; /* Configuration parameters */
/* Find uart devices */
serial = rt_device_find(SAMPLE_UART_NAME);

/* Open the uart device in interrupt receive and polling send mode */
rt_device_open(serial, RT_DEVICE_FLAG_INT_RX);

config.baud_rate = BAUD_RATE_115200;
config.data_bits = DATA_BITS_8;
config.stop_bits = STOP_BITS_2;
config.parity = PARITY_NONE;
/* The serial port configuration parameters can only be modified after opening the device */
rt_device_control(serial, RT_DEVICE_CTRL_CONFIG, &config);
```

1.4.2.4.1.6 Send Data

To write data to the serial port, the following functions can be used:

```
rt_size_t rt_device_write(rt_device_t dev, rt_off_t pos, const void* buffer, rt_size_t size);
```

Parameter	Description
dev	device handle
pos	Write data offset, this parameter is not used in uart device
buffer	Memory buffer pointer, place the data to be written
size	The size of the written data
back	——
The actual size of the written data	If it is a character device, the return size is in bytes;
0	It needs to read the current thread's errno to determine the error status

Calling this function will write the data in the `buffer` to the `dev` device, the size of the write data is: size.
The sample program for writing data to the serial port is as follows:

```
#define SAMPLE_UART_NAME       "uart2"  /* uart device's name */
static rt_device_t serial;              /* uart device handle */
char str[] = "hello RT-Thread!\r\n";
struct serial_configure config = RT_SERIAL_CONFIG_DEFAULT; /* Configuration parameter */
/*find uart device */
serial = rt_device_find(SAMPLE_UART_NAME);

/* Open the uart device in interrupt reception and polling mode */
rt_device_open(serial, RT_DEVICE_FLAG_INT_RX);
/* Send string */
rt_device_write(serial, 0, str, (sizeof(str) - 1));
```

1.4.2.4.1.7 Set The Send completion Callback Function

When the application calls `rt_device_write()` to write data, if the underlying hardware can support automatic transmission, the upper application can set a callback function. This callback function is called after the underlying hardware data has been sent (for example, when the DMA transfer is complete or the FIFO has been written to complete the completion interrupt). You can set the device to send completion instructions by the following function:

```
rt_err_t rt_device_set_tx_complete(rt_device_t dev, rt_err_t (*tx_done)(rt_device_t dev,void *buffer));
```

Parameter	Description
dev	device handle
tx_done	callback function pointer
back	——
RT_EOK	set up successfully

When this function is called, the callback function is provided by the user. When the hardware device sends the

data, the device driver calls back this function and passes the sent data block address buffer as a parameter to the upper application. When the application (thread) receives the indication, it will release the buffer memory block or use it as the buffer for the next write data according to the condition of sending the buffer.

1.4.2.4.1.8 Set The Receive Callback Function

The data receiving instruction can be set by the following function. When the serial port receives the data, it will inform the upper application thread that the data has arrived:

```
rt_err_t rt_device_set_rx_indicate(rt_device_t dev, rt_err_t (*rx_ind)(rt_device_t dev,rt_size_t size));
```

Parameter	Description
dev	device handle
rx_ind	callback function pointer
dev	device handle (callback function parameter)
size	buffer data size (callback function parameter)
back	——
RT_EOK	set up successfully

The callback function for this function is provided by the user. If the uart device is opened in interrupt receive mode, the callback function will be called when the serial port receives a data, and the data size of the buffer will be placed in the `size` parameter, and the uart device handle will be placed in the `dev` parameter.
If the uart is opened in DMA receive mode, the callback function is called when the DMA completes receiving a batch of data.
Normally the receiving callback function can send a semaphore or event to notify the serial port data processing thread that data has arrived. The example is as follows:

```
#define SAMPLE_UART_NAME       "uart2"  /* uart device name */
static rt_device_t serial;              /* uart device handle */
static struct rt_semaphore rx_sem;      /* The semaphore used to receive the message */

/* Receive data callback function */
static rt_err_t uart_input(rt_device_t dev, rt_size_t size)
{
    /* When the serial port receives the data, it triggers interrupts, calls this callback function, and
      sends the received semaphore */
    rt_sem_release(&rx_sem);

    return RT_EOK;
}

static int uart_sample(int argc, char *argv[])
{
    serial = rt_device_find(SAMPLE_UART_NAME);

    /* Open the uart device in interrupting receive mode */
    rt_device_open(serial, RT_DEVICE_FLAG_INT_RX);

    /* Initialization semaphore */
    rt_sem_init(&rx_sem, "rx_sem", 0, RT_IPC_FLAG_FIFO);

    /* Set the receive callback function */
    rt_device_set_rx_indicate(serial, uart_input);
}
```

1.4.2.4.1.9 Receive Data

You can call the following function to read the data received by the uart:

```
rt_size_t rt_device_read(rt_device_t dev, rt_off_t pos, void* buffer, rt_size_t size);
```

Parameter	Description
dev	device handle
pos	Read data offset, uart device dose not use this parameter
buffer	Buffer pointer, the data read will be saved in the buffer

Parameter	Description
size	Read the size of the data
back	——
Read the actual size of the data	If it is a character device, the return size is in bytes.
0	It needs to read the current thread's errno to determine the error status

Read data offset: pos is not valid for character devices. This parameter is mainly used for block devices.
An example of using the interrupt receive mode with the receive callback function is as follows:

```
static rt_device_t serial;            /* uart device handle */
static struct rt_semaphore rx_sem;    /* Semaphore used to receive messages */

/* Thread receiving data */
static void serial_thread_entry(void *parameter)
{
    char ch;

    while (1)
    {
        /* Reads a byte of data from the serial port and waits for the receiving semaphore if it is not read
      */
        while (rt_device_read(serial, -1, &ch, 1) != 1)
        {
            /* Blocking waiting to receive semaphore, waiting for the semaphore to read the data again*/
            rt_sem_take(&rx_sem, RT_WAITING_FOREVER);
        }
        /* Read the data through the serial port dislocation output*/
        ch = ch + 1;
        rt_device_write(serial, 0, &ch, 1);
    }
}
```

1.4.2.4.1.10 Close The UART Device

After the application completes the serial port operation, the uart device can be closed by the following functions:

```
rt_err_t rt_device_close(rt_device_t dev);
```

Parameter	Description
dev	device handle
back	——
RT_EOK	device closed successfully
-RT_ERROR	The device has been completely shut down and cannot be shut down repeatedly
other error codes	fail to close the device

Use the `rt_device_close()` interface and `rt_device_open()` interface in pair. When you open the device, you need to close the device once, so that the device will be completely shut down, otherwise the device will remain open.

1.4.2.4.2 Examples Of Using UART Device

1.4.2.4.2.1 Interrupt Receiving And Polling Send

The main steps of the sample code are as follows:

1. First find the uart device to get the device handle.
2. Initialize the semaphore that the callback function sends, and then open the uart device in read/write and interrupt receive mode.
3. Set the receive callback function of the uart device, then send the string and create a read data thread.
4. The read data thread will try to read a character data. If there is no data, it will hang and wait for the semaphore. When the uart device receives a data, it will trigger an interrupt and call the receive callback

function. This function will send a semaphore to wake up the thread. At this point, the thread will immediately read the received data.

5. This sample code is not limited to a specific BSP. According to the uart device registered by BSP, modify the uart device's name corresponding to the sample code's macro definition SAMPLE_UART_NAME to run.

The running sequence diagram is shown as follows:

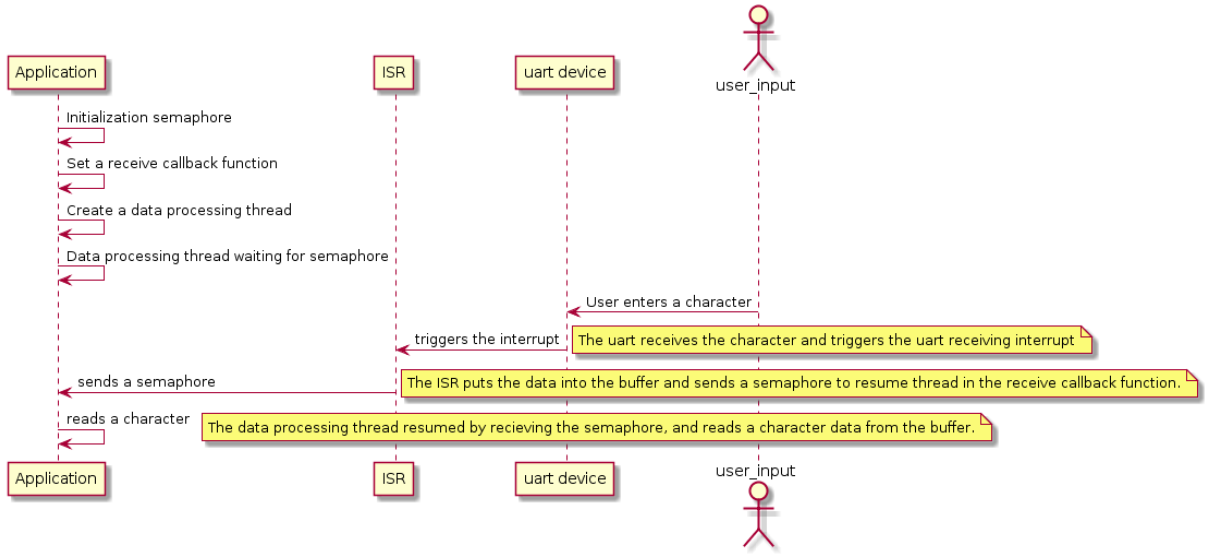


Figure 1.96 Serial Port Interrupt Reception and Polling Transmission Sequence Diagram

```
/*
 * Program list: This is a uart device usage routine
 * The routine exports the uart_sample command to the control terminal
 * Format of command: uart_sample uart2
 * Command explanation: the second parameter of the command is the name of the uart device. If it is null,
     the default uart device wil be used
 * Program function: output the string "hello RT-Thread!" through the serial port, and then malposition the
     input character
*/

#include <rtthread.h>

#define SAMPLE_UART_NAME       "uart2"

/* Semaphore used to receive messages */
static struct rt_semaphore rx_sem;
static rt_device_t serial;

/* Receive data callback function */
static rt_err_t uart_input(rt_device_t dev, rt_size_t size)
{
    /* After the uart device receives the data, it generates an interrupt, calls this callback function, and
      then sends the received semaphore. */
    rt_sem_release(&rx_sem);

    return RT_EOK;
}

static void serial_thread_entry(void *parameter)
{
    char ch;

    while (1)
    {
        /* Read a byte of data from the serial port and wait for the receiving semaphore if it is not read
       */
        while (rt_device_read(serial, -1, &ch, 1) != 1)
        {
            /* Being Suspended and waiting for the semaphore */
            rt_sem_take(&rx_sem, RT_WAITING_FOREVER);
        }
        /* Read the data from the serial port and output through dislocation */
        ch = ch + 1;
        rt_device_write(serial, 0, &ch, 1);
    }
```

```
}

static int uart_sample(int argc, char *argv[])
{
    rt_err_t ret = RT_EOK;
    char uart_name[RT_NAME_MAX];
    char str[] = "hello RT-Thread!\r\n";

    if (argc == 2)
    {
        rt_strncpy(uart_name, argv[1], RT_NAME_MAX);
    }
    else
    {
        rt_strncpy(uart_name, SAMPLE_UART_NAME, RT_NAME_MAX);
    }

    /* Find uart devices in the system */
    serial = rt_device_find(uart_name);
    if (!serial)
    {
        rt_kprintf("find %s failed!\n", uart_name);
        return -RT_ERROR;
    }

    /* Initialize the semaphore */
    rt_sem_init(&rx_sem, "rx_sem", 0, RT_IPC_FLAG_FIFO);
    /* Open the uart device in interrupt receive and polling send mode */
    rt_device_open(serial, RT_DEVICE_FLAG_INT_RX);
    /* Set the receive callback function */
    rt_device_set_rx_indicate(serial, uart_input);
    /* Send string */
    rt_device_write(serial, 0, str, (sizeof(str) - 1));

    /* Create a serial thread */
    rt_thread_t thread = rt_thread_create("serial", serial_thread_entry, RT_NULL, 1024, 25, 10);
    /* Start the thread successfully */
    if (thread != RT_NULL)
    {
        rt_thread_startup(thread);
    }
    else
    {
        ret = -RT_ERROR;
    }

    return ret;
}
/* Export to the msh command list */
MSH_CMD_EXPORT(uart_sample, uart device sample);
```

1.4.2.4.2.2 DMA Reception And Polling Transmission

When the serial port receives a batch of data, it will call the receive callback function. The receive callback function will send the data size of the buffer at this time to the waiting data processing thread through the message queue. After the thread gets the message, it is activated and reads the data. In general, the DMA receive mode completes data reception in conjunction with the DMA receive completion interrupt and the serial port idle interrupt.

- This sample code is not limited to a specific BSP. According to the uart device registered by BSP, modify the sample code macro to define the uart device name corresponding to SAMPLE_UART_NAME to run.

The running sequence diagram is shown below:

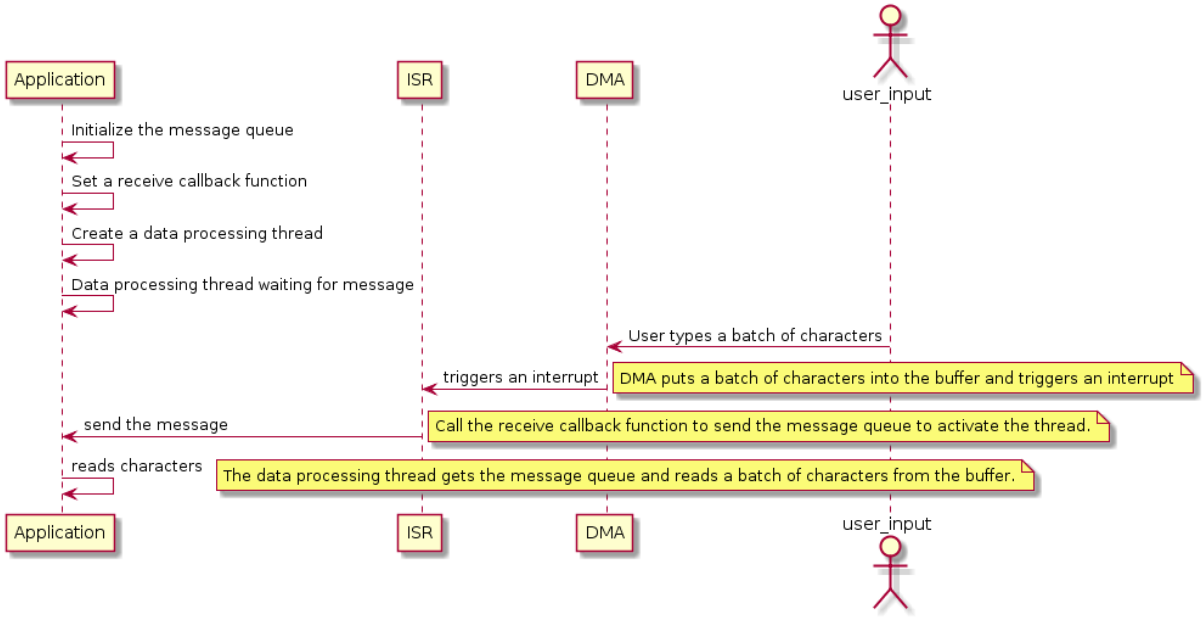


Figure 1.97 Serial DMA Receiving and Polling Transmission Sequence Diagram

```c
/*
 * Program list: This is a uart device DMA receive usage routine
 * The routine exports the uart_dma_sample command to the control terminal
 * Command format: uart_dma_sample uart3
 * Command explanation: The second parameter of the command is the name of the uart device to be used. If it
      is empty, the default uart device will be used.
 * Program function: output the string "hello RT-Thread!" through the serial port, and output the received
      data through the serial port, and then print the received data.
*/

#include <rtthread.h>

#define SAMPLE_UART_NAME       "uart3"      /* uart device name */

/* Serial port receiving message structure*/
struct rx_msg
{
    rt_device_t dev;
    rt_size_t size;
};
/* uart device handle */
static rt_device_t serial;
/* Message queue control block*/
static struct rt_messagequeue rx_mq;

/* Receive data callback function */
static rt_err_t uart_input(rt_device_t dev, rt_size_t size)
{
    struct rx_msg msg;
    rt_err_t result;
    msg.dev = dev;
    msg.size = size;

    result = rt_mq_send(&rx_mq, &msg, sizeof(msg));
    if ( result == -RT_EFULL)
    {
        /* message queue full */
        rt_kprintf("message queue full\n");
    }
    return result;
}

static void serial_thread_entry(void *parameter)
{
    struct rx_msg msg;
    rt_err_t result;
    rt_uint32_t rx_length;
    static char rx_buffer[RT_SERIAL_RB_BUFSZ + 1];

    while (1)
    {
        rt_memset(&msg, 0, sizeof(msg));
        /* Read messages from the message queue*/
        result = rt_mq_recv(&rx_mq, &msg, sizeof(msg), RT_WAITING_FOREVER);
```

```
        if (result >= 0)
        {
            /*Read data from the serial port*/
            rx_length = rt_device_read(msg.dev, 0, rx_buffer, msg.size);
            rx_buffer[rx_length] = '\0';
            /* Output the read message through the uart device: serial */
            rt_device_write(serial, 0, rx_buffer, rx_length);
            /* Print data */
            rt_kprintf("%s\n",rx_buffer);
        }
    }
}

static int uart_dma_sample(int argc, char *argv[])
{
    rt_err_t ret = RT_EOK;
    char uart_name[RT_NAME_MAX];
    static char msg_pool[256];
    char str[] = "hello RT-Thread!\r\n";

    if (argc == 2)
    {
        rt_strncpy(uart_name, argv[1], RT_NAME_MAX);
    }
    else
    {
        rt_strncpy(uart_name, SAMPLE_UART_NAME, RT_NAME_MAX);
    }

    /* find uart device */
    serial = rt_device_find(uart_name);
    if (!serial)
    {
        rt_kprintf("find %s failed!\n", uart_name);
        return -RT_ERROR;
    }

    /* Initialize message queue */
    rt_mq_init(&rx_mq, "rx_mq",
               msg_pool,                 /* a pool for storing messages */
               sizeof(struct rx_msg),    /* The maximum length of a message*/
               sizeof(msg_pool),         /* The size of the message pool */
               RT_IPC_FLAG_FIFO);        /* If there are multiple threads waiting, assign messages according
      to the order. */

    /* Open the uart device in DMA receive and polling send mode */
    rt_device_open(serial, RT_DEVICE_FLAG_DMA_RX);
    /* Set the receive callback function */
    rt_device_set_rx_indicate(serial, uart_input);
    /* Send string */
    rt_device_write(serial, 0, str, (sizeof(str) - 1));

    /* Create a thread */
    rt_thread_t thread = rt_thread_create("serial", serial_thread_entry, RT_NULL, 1024, 25, 10);
    /* Start the thread if it is created successfully*/
    if (thread != RT_NULL)
    {
        rt_thread_startup(thread);
    }
    else
    {
        ret = -RT_ERROR;
    }

    return ret;
}
/* Export to the msh command list */
MSH_CMD_EXPORT(uart_dma_sample, uart device dma sample);
```

1.4.2.4.2.3 See also (driver)

- UART device model (DM) — naming, bootargs, platform probe
- UART early console — earlycon, handoff to uartN
- Pin control (pinctrl) — pin mux before UART enable
- DMA engine — RX/TX DMA
- components/drivers/serial/SConscript — build matrix

1.4.2.5 UART device model (DM)

1.4.2.5.1 UART / serial DM helpers

Kconfig uses **RT_USING_SERIAL**; registered device names are **uart0**, **uart1**, ... Implementation directory: **components/drivers/serial/** (serial core + controller drivers).

Piece	Built when	Source
Serial framework	**RT_USING_SERIAL**	dev_serial.c or dev_serial_v2.c
DM naming / bootargs parse	**RT_USING_DM**	serial_dm.c
OFW platform drivers (8250, PL011, ...)	**RT_USING_DM** + **RT_USING_SERIAL**	serial/device/* (host/Kconfig)

Header: **components/drivers/include/drivers/serial_dm.h**.
Application **rt_device_find("uart2")** usage: UART. Early TX before probe: UART early console.

1.4.2.5.1.1 serial_dev_set_name

```
int serial_dev_set_name(struct rt_serial_device *sdev);
```

- With **RT_USING_OFW**, tries DTS alias id for **serial** then **uart** on **sdev->parent.ofw_node**.
- Otherwise uses an atomic counter seeded from **rt_ofw_get_alias_last_id("serial")** / **uart** so extra ports do not collide with aliased indices.
- Sets the parent **rt_device** name to **uartu** via **rt_dm_dev_set_name**.

INIT_PLATFORM_EXPORT(serial_dm_naming_framework_init) runs before platform UART probes to initialize **uid_min**.
Prefer **aliases { serial0 = &uart0 }** or **uart0 = &...** in DTS so names match board docs and FinSH/console scripts.

1.4.2.5.2 Boot argument parsing

serial_cfg_from_args

```
struct serial_configure serial_cfg_from_args(char *str);
```

Parses the **earlycon** / **console** option tail (after the MMIO base in **earlycon=** strings):

Segment	Meaning
Leading digits	Baud rate (default from **RT_SERIAL_CONFIG_DEFAULT** if missing)
n / o / e	Parity none / odd / even
One digit	Data bits
r	RTS/CTS (RT_SERIAL_FLOWCONTROL_CTSRTS)

Example: **115200n8** → 115200, no parity, 8 data bits.
If the string ends with the four-byte suffix **OFW**, that marker is cleared—firmware earlycon handoff so the full driver does not re-parse stale bootargs text. **Pass a writable buffer** when this path is used.

serial_base_from_args

```
void *serial_base_from_args(char *str);
```

Finds **0x** in the option string and parses a hexadecimal **physical base** for early bring-up (before OFW **iomap**).

serial_for_each_args

Macro to iterate comma-separated sub-arguments in a boot string (multi-earlycon lists).

1.4.2.5.1.3 Typical platform `probe` (8250 / PL011)

1. Map registers: **`rt_ofw_iomap(np, 0)`** or **`rt_dm_dev_iomap(dev, 0)`** (driver-specific).
2. **`rt_ofw_get_irq`** / **`rt_dm_dev_get_irq`** for the interrupt line.
3. Optional **`rt_clk_get`**, **`rt_reset_control_get`**, **`current-speed`**, **`reg-shift`** from DT.
4. Fill **`struct rt_serial_device`** / SoC wrapper, **`serial_dev_set_name`** (often inside **`serial8250_setup`** / driver `setup`).
5. **`rt_hw_serial_register(&serial->parent, name, flags, priv)`** — exposes **`uartN`** to applications.

Example OFW 8250: **`components/drivers/serial/device/8250/8250-ofw.c`**, **`core.c`** (`serial8250_setup`).
RX DMA: request a channel from DMA engine; completion feeds the serial ISR (**`rt_hw_serial_isr`**, events **`RT_SERIAL_EVENT_RX_DMADONE`**).

1.4.2.5.1.4 Pitfalls

- **Alias vs probe order**: ports without aliases get **`uart0`**, **`uart1`** in registration order—not always DTS label order.
- **Pinctrl**: apply **`pinctrl-0`** (or board init) **before** first TX—see Pin control (pinctrl).
- **Console vs app**: reopening the shell UART can conflict—document exclusive use or use Serial V2 policy if enabled (**`RT_USING_SERIAL_V2`**).
- **`RT_DEVICE_CTRL_CONFIG`**: must reach hardware in the driver **`control`** / **`configure`** path or users see wrong baud after open.

1.4.2.5.1.5 See also

- UART — application API
- UART early console — FDT **`earlycon`**, **`stdout-path`**
- Platform bus — platform driver registration
- DMA engine
- `components/drivers/include/drivers/dev_serial.h`, `dev_serial_v2.h`
- `components/drivers/serial/serial_dm.c`

1.4.2.6 UART early console

1.4.2.6.1 Early console (before full UART driver)

Early output runs **before** the normal **`rt_serial_device`** / **`rt_hw_serial_register`** path: a minimal **putc** hook is installed from FDT **`/chosen stdout-path`** and/or **`earlycon=`** in **`bootargs`**.
Sources:

- **8250 / NS16550 / DW APB**: `components/drivers/serial/device/8250/early.c`
- **Hypervisor console**: `components/drivers/serial/device/serial-early-hvc.c`

Baud/format/base parsing: **`serial_cfg_from_args`**, **`serial_base_from_args`** in **`serial_dm.c`** — see UART device model (DM).

1.4.2.6.1.1 8250 earlycon (`early.c`)

Registration

`RT_FDT_EARLYCON_EXPORT` entries (examples):

Symbol	`earlycon` name	Example compatible
`ns16550`	`uart8250`	`ns16550, ns16550a`
`dw8250`	`uart8250`	`snps,dw-apb-uart`
`tegra20`	`uart8250`	`nvidia,tegra20-uart`
`bcm2835aux`	`uart8250`	`brcm,bcm2835-aux-uart`

Bootargs example:

```
earlycon=uart8250,mmio32,0x10009000,115200n8
```

The FDT earlycon core passes **`options`** into **`serial8250_early_fdt_setup`**, which calls **`serial8250_early_config`** then maps MMIO with **`rt_ioremap_early`**.

`serial8250_early_putc`

Polls **UART LSR** (**TEMT** | **THRE**) after writing **UART_TX**∗∗**—safe with interrupts masked. Installed as** ∗∗**`con->console_putc`**.

Lifecycle: `serial8250_early_kick`

why	Action
`FDT_EARLYCON_KICK_UPDATE`	**`rt_ioremap`** full mapping when switching from early to runtime map
`FDT_EARLYCON_KICK_COMPLETED`	**`rt_iounmap`** early mapping after the platform driver owns the port

If **`config.baud_rate`** is zero at setup, firmware already programmed the UART; the code only clears **IER**.

Divisor

When **`serial->freq`** is set, **`init_serial`** programs **DLL**/**DLM** from **`freq / (16 * baud_rate)`**.

1.4.2.6.1.2 HVC earlycon (`serial-early-hvc.c`)

```
RT_FDT_EARLYCON_EXPORT(hvc, "hvc", "vmrt-thread,hvc-console", hvc_early_setup);
```

- **`hvc_early_setup`** calls **`rt_hv_version`**; on success **`console_putc`** → **`rt_hv_console(c)`**.
- No MMIO ∗∗reg∗∗—hypervisor console path.

1.4.2.6.1.3 Handoff to full driver

1. Earlycon prints via **`rt_kprintf`** / **`console_putc`**.
2. Platform probe (e.g. **`8250-ofw`**) maps the same UART, **`serial_dev_set_name`**, **`rt_hw_serial_register`** → **`uartN`** for apps.
3. Early mapping is released on **`FDT_EARLYCON_KICK_COMPLETED`** when the serial core finishes transition.

Do not leave early **putc** and the full driver writing the same UART without the kick handshake.

1.4.2.6.1.4 Pitfalls

- **`reg-shift`**: early defaults may differ from DT **`reg-shift`** in probe—keep them consistent for your IP.
- **Options buffer**: **`serial_cfg_from_args`** may modify the string (**`OFW`** suffix).
- **RX**: early path is **TX-only**; input needs the full UART driver or another console backend.

1.4.2.6.1.5 See also

- UART device model (DM)
- UART
- Open Firmware (OFW) — **`chosen`**, **`stdout-path`**
- `components/drivers/serial/device/8250/core.c`, `8250-ofw.c`

1.4.2.7 ADC Device

1.4.2.7.1 An Introduction to ADC

An ADC (analog-to-digital converter) is a hardware device that converts continuously changing analog signals to discrete digital signals. Usually, these analog signals include temperature, pressure, sound, video and many other types of signals. Converting them is important, as digital signals are easier to store, process, and transmit. This conversion can be achieved by using an ADC device which is commonly integrated in various platforms. Historically, ADCs were first used to convert received wireless signals to digital signals, for example, television signals, or signals from long-short broadcast stations.

1.4.2.7.1.1 Conversion Process

As shown in the figure below, the analog-to-digital conversion generally involves steps of sampling, holding, quantifying, and encoding. In actual circuits, some processes are combined, such as sampling and holding, while quantization and encoding are implemented simultaneously in the conversion process.

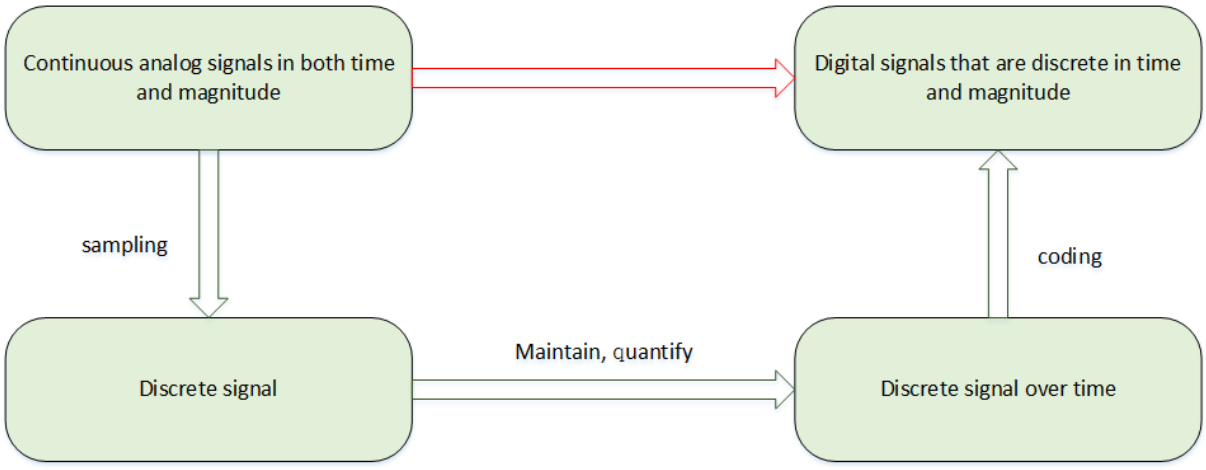


Figure 1.98 ADC Conversion Process

Sampling is the conversion of analog signals that changes continuously over time into time-discrete analog signals. It takes a certain amount of time for the analog signals obtained by sampling to be converted into digital signals. In order to provide a stable value for the subsequent quantization coding process, it is required to keep the sampling analog signals for a period of time after the sampling circuit.
The process of converting a numerically continuous analog quantity into a digital quantity is called quantization. Digital signals are discrete numerically. The output voltage of the sample-and-hold circuit also needs to be naturalized to a corresponding discrete level in a similar way, and any digital quantity can only be an integer multiple of a certain minimum quantity unit. The quantized value also requires the encoding process, which is the digital output of the A/D converter.

1.4.2.7.1.2 Resolution

Resolution is represented as binary (or decimal) numbers. Generally, it comes in 8 bits, 10 bits, 12 bits, 16 bits, etc. A larger resolution, in bits, means more accuracy in the conversion of analog to digital signals.

1.4.2.7.1.3 Precision

Precision is the maximum error value between analog signals and real ADC device numerical points' values.An ADC with a high resolution might have a low precision, meaning that factors like noise can affect the numerical ADC reading more than small changes in the input signal.

1.4.2.7.1.4 Conversion Rate

The conversion rate is the reciprocal of time taken for an ADC device to complete conversion from an analog to a digital signal. For example, an ADC device with a conversion rate of 1MHz means that the ADC conversion time is 1 microsecond.

1.4.2.7.2 Access ADC Device

The application accesses the ADC hardware through the ADC device management interface provided by RT-Thread. The relevant interfaces are as follows:

Function	Description
rt_device_find()	Find device handles based on ADC device name
rt_adc_enable()	Enable ADC devices
rt_adc_read()	Read ADC device data
rt_adc_disable()	Close the ADC device

1.4.2.7.2.1 Find ADC Devices

The application gets the device handler based on the ADC device name to operate the ADC device. Following is the interface function to find the devices:

```
rt_device_t rt_device_find(const char* name);
```

Parameter	Description
name	The name of the ADC device
Return	——
Device handle	Finding the corresponding device will return to the corresponding device handle
RT_NULL	No device found

In a nutshell, the names of the ADC devices are registered as adc0, adc1, and so on. What follows is an example on how to use it

```
#define ADC_DEV_NAME        "adc1"  /* ADC device name */
rt_adc_device_t adc_dev;            /* ADC device handle */
/* find the device */
adc_dev = (rt_adc_device_t)rt_device_find(ADC_DEV_NAME);
```

1.4.2.7.2.2 Enable ADC Channel

It is required to enable the ADC device with the following interface function before reading and operating the ADC device.

```
rt_err_t rt_adc_enable(rt_adc_device_t dev, rt_uint32_t channel);
```

Parameter	Description
dev	ADC device handle
channel	ADC channel
Return	——
RT_EOK	Succeed
-RT_ENOSYS	Failed, the device operation method is empty
Other error code	Failed

An usage example is as follows:

```
#define ADC_DEV_NAME        "adc1"  /* ADC device name */
#define ADC_DEV_CHANNEL     5       /* ADC channel */
rt_adc_device_t adc_dev;            /* ADC device handle */
/* find the device */
adc_dev = (rt_adc_device_t)rt_device_find(ADC_DEV_NAME);
/* enable the device */
rt_adc_enable(adc_dev, ADC_DEV_CHANNEL);
```

1.4.2.7.2.3 Read ADC Channel Sample Values

Reading the ADC channel sample values can be done by the following function:

```
rt_uint32_t rt_adc_read(rt_adc_device_t dev, rt_uint32_t channel);
```

Parameter	Description
dev	ADC device handle
channel	ADC channel
Return	——
Read values	

An example of using the ADC sampled voltage value is as follows:

```
#define ADC_DEV_NAME        "adc1"      /* ADC device name */
#define ADC_DEV_CHANNEL     5           /* ADC channel */
#define REFER_VOLTAGE       330         /* Reference voltage 3.3V, data accuracy multiplied by 100 and
      reserve 2 decimal places*/
#define CONVERT_BITS        (1 « 12)    /* The number of conversion bits is 12  */

rt_adc_device_t adc_dev;                /* ADC device handle */
rt_uint32_t value
/* find the device */
adc_dev = (rt_adc_device_t)rt_device_find(ADC_DEV_NAME);
/*  enable the device  */
rt_adc_enable(adc_dev, ADC_DEV_CHANNEL);
/* Read sampling values */
value = rt_adc_read(adc_dev, ADC_DEV_CHANNEL);
/*  Convert to the corresponding voltage value  */
vol = value * REFER_VOLTAGE / CONVERT_BITS;
rt_kprintf("the voltage is :%d.%02d \n", vol / 100, vol % 100);
```

The calculation formula of the actual voltage value is: `sampling value * reference voltage/(1 << resolution digit)`. In the above example, variable *vol* was enlarged 100 times, so finally the integer part of voltage is obtained through *vol / 100*, and the decimal part of voltage is obtained through *vol % 100*.

1.4.2.7.2.4 Disabling the ADC Channel

An ADC channel can be disabled through the following function:

```
rt_err_t rt_adc_disable(rt_adc_device_t dev, rt_uint32_t channel);
```

Parameter	Description
dev	ADC device handle
channel	ADC channel

Parameter	Description
Return	——
RT_EOK	Succeed
-RT_ENOSYS	Failed, the device operation method is empty
Other error code	Failed

An example:

```
#define ADC_DEV_NAME        "adc1"  /* ADC device name */
#define ADC_DEV_CHANNEL     5       /* ADC channel */
rt_adc_device_t adc_dev;            /* ADC device handle */
rt_uint32_t value
/* find the device */
adc_dev = (rt_adc_device_t)rt_device_find(ADC_DEV_NAME);
/* enable the device */
rt_adc_enable(adc_dev, ADC_DEV_CHANNEL);
/* read sampling values */
value = rt_adc_read(adc_dev, ADC_DEV_CHANNEL);
/*  convert to the corresponding voltage value */
vol = value * REFER_VOLTAGE / CONVERT_BITS;
rt_kprintf("the voltage is :%d.%02d \n", vol / 100, vol % 100);
/* close the channel */
rt_adc_disable(adc_dev, ADC_DEV_CHANNEL);
```

1.4.2.7.2.5 FinSH Command

To find out the registered device, you can use the command adc probe followed by the registered ADC device name as following,

```
msh >adc probe adc1
probe adc1 success
```

A channel of the enabled device can use the command `adc enable` followed by the channel number.

```
msh >adc enable 5
adc1 channel 5 enables success
```

To read data from a channel of an ADC device, you can use the command `adc read` followed by the channel number.

```
msh >adc read 5
adc1 channel 5  read value is 0x00000FFF
msh >
```

To close a channel of an ADC device, you can use the command `adc disable` followed by the channel number.

```
msh >adc disable 5
adc1 channel 5 disable success
msh >
```

1.4.2.7.3 ADC Device Usage Example

The specific usage of the ADC device can refer to the following sample code. The main steps of the sample code are as follows:

1. First find the device handle based on the ADC device name “adc1”.
2. After the device is enabled, read the sample value of the corresponding channel 5 of the adc1 device, and then calculate the actual voltage value with the resolution of 12 bits and the reference voltage of 3.3V.
3. Finally close the corresponding channel of the ADC device.

Running result: Print the raw and converted data which actually read , and print the calculated actual voltage value.

```
/*
 * Program Listing ADC Device Usage Routines
 * The routine exports the adc_sample command to the control terminal
 * adc_sample Command call format: adc_sample
 * Program function: The voltage value is sampled by the ADC device and converted to a numerical value.
 *                   The sample code reference voltage is 3.3V and the number of conversion bits is 12 bits.
*/

#include <rtthread.h>
#include <rtdevice.h>

#define ADC_DEV_NAME        "adc1"      /* ADC device name */
#define ADC_DEV_CHANNEL     5           /* ADC channel */
```

```
#define REFER_VOLTAGE       330         /* Reference voltage 3.3V, data accuracy multiplied by 100 and
      reserve 2 decimal places*/
#define CONVERT_BITS        (1 « 12)    /* The number of conversion bits is 12 */

static int adc_vol_sample(int argc, char *argv[])
{
    rt_adc_device_t adc_dev;
    rt_uint32_t value, vol;
    rt_err_t ret = RT_EOK;

    /* find the device */
    adc_dev = (rt_adc_device_t)rt_device_find(ADC_DEV_NAME);
    if (adc_dev == RT_NULL)
    {
        rt_kprintf("adc sample run failed! can't find %s device!\n", ADC_DEV_NAME);
        return -RT_ERROR;
    }

    /* enable the device */
    ret = rt_adc_enable(adc_dev, ADC_DEV_CHANNEL);

    /* read sampling values */
    value = rt_adc_read(adc_dev, ADC_DEV_CHANNEL);
    rt_kprintf("the value is :%d \n", value);

    /* convert to the corresponding voltage value */
    vol = value * REFER_VOLTAGE / CONVERT_BITS;
    rt_kprintf("the voltage is :%d.%02d \n", vol / 100, vol % 100);

    /* close the channel */
    ret = rt_adc_disable(adc_dev, ADC_DEV_CHANNEL);

    return ret;
}
/* export to the msh command list */
MSH_CMD_EXPORT(adc_vol_sample, adc voltage convert sample);
```

1.4.2.8 I2C Bus Device

1.4.2.8.1 Introduction of I2C

The I2C (Inter Integrated Circuit) bus is a half-duplex, bidirectional two-wire synchronous serial bus developed by Philips. The I2C bus has only two signal lines, one is the bidirectional data line SDA (serial data), and the other is the bidirectional clock line SCL (serial clock). Compared to the SPI bus, which has two lines for receiving data and transmitting data between the master and slave devices, the I2C bus uses only one line for data transmission and reception.
Like SPI, I2C works in a master-slave manner. Unlike SPI-master-multi-slave architecture, it allows multiple master devices to exist at the same time. Each device connected to the bus has a unique address, and the master device initiates data transfer, and generates a clock signal. The slave device is addressed by the master device, and only one master device is allowed to communicate at a time. As shown below:

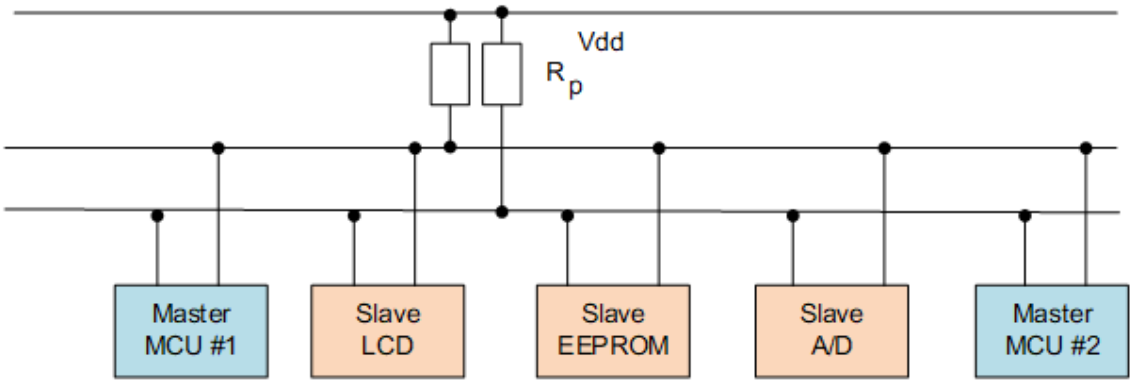


Figure 1.99 I2C Bus master-slave device connection mode

The main data transmission format of the I2C bus is shown in the following figure:

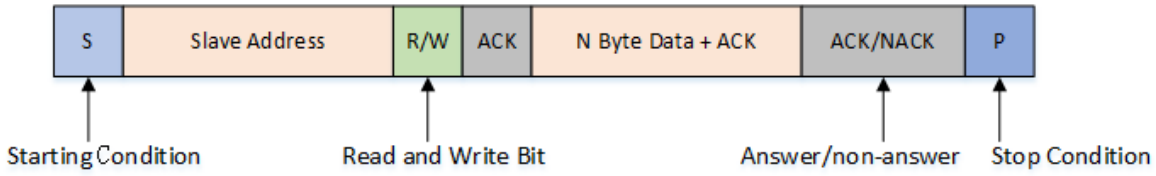


Figure 1.100 I2C Bus Data Transmission Format

When the bus is idle, both SDA and SCL are in a high state. When the host wants to communicate with a slave, it will send a start condition first, then send the slave address and read and write control bits, and then transfer the data (the host can send or receive data). The host will send a stop condition when the data transfer ends. Each byte transmitted is 8 bits, with the high bit first and the low bit last. The different terms in the data transmission process are as follows:

- **Starting Condition** When SCL is high, the host pulls SDA low, indicating that data transfer is about to begin.
- **Slave Address** The first byte sent by the master is the slave address, the upper 7 bits are the address, the lowest bit is the R/W read/write control bit, R/W bit equals to 1 means the read operation, and 0 means the write operation. The general slave address has 7-bit address mode and 10-bit address mode. In the 10-bit address mode, the first 7 bits of the first byte are a combination of 11110XX, where the last two bits (XX) are two highest 10-bit addresses. The second byte is the remaining 8 bits of the 10-bit slave address, as shown in the following figure:



Figure 1.101 7-bit address and 10-bit address format

- **Answer Signal** Each time a byte of data is transmitted, the receiver needs to reply with an ACK (acknowledge). The slave sends an ACK when writing data and the ACK by the host when reading data. When the host reads the last byte of data, it can send NACK (Not acknowledge) and then stop the condition.
- **Data** After the slave address is sent, some commands may be sent, depending on the slave, and then the data transmission starts, and is sent by the master or the slave. Each data is 8 bits, and the number of bytes of data is not limited.
- **Repeat Start Condition** In a communication process, when the host may need to transfer data with different slaves or need to switch read and write operations, the host can send another start condition.
- **Stop Condition** When SDA is low, the master pulls SCL high and stays high, then pulls SDA high to indicate the end of the transfer.

1.4.2.8.2 Access to I2C Bus Devices

In general, the MCU's I2C device communicates as a master and slave. In the RT-Thread, the I2C master is virtualized as an I2C bus device. The I2C slave communicates with the I2C bus through the I2C device interface. The related interfaces are as follows:

Function	Description
rt_device_find()	Find device handles based on I2C bus device name
rt_i2c_transfer()	transfer data

1.4.2.8.2.1 Finding I2C Bus Device

Before using the I2C bus device, you need to obtain the device handle according to the I2C bus device name, so that you can operate the I2C bus device. The device function is as follows.

```
rt_device_t rt_device_find(const char* name);
```

Parameter	Description
name	I2C bus device name
Return Value	——
device handle	Finding the corresponding device will return the corresponding device handle
RT_NULL	No corresponding device object found

In general, the name of the I2C device registered to the system is i2c0, i2c1, etc. The usage examples are as follows:

```
#define AHT10_I2C_BUS_NAME      "i2c1"  /* Sensor connected I2C bus device name */
struct rt_i2c_bus_device *i2c_bus;      /* I2C bus device handle */

/* Find the I2C bus device and get the I2C bus device handle */
i2c_bus = (struct rt_i2c_bus_device *)rt_device_find(name);
```

1.4.2.8.2.2 Data Transmission

You can use `rt_i2c_transfer()` for data transfer by getting the I2C bus device handle. The function prototype is as follows:

```
rt_size_t rt_i2c_transfer(struct rt_i2c_bus_device *bus,
                          struct rt_i2c_msg         msgs[],
                          rt_uint32_t               num);
```

Parameter	Description
bus	I2C bus device handle
msgs↩ []	Message array pointer to be transmitted
num	The number of elements in the message array
Return Value	——
the number of elements in the message array	succeeded
error code	failed

Like the custom transport interface of the SPI bus, the data transmitted by the custom transport interface of the I2C bus is also in units of one message. The parameter msgs[] points to the array of messages to be transmitted. The user can customize the content of each message to implement two different data transmission modes supported by the I2C bus. If the master needs to send a repeat start condition, it will need to send 2 messages.
>This function will call rt_mutex_take(), which cannot be called inside the interrupt service routine, which will cause assertion to report an error.
The prototypes of the I2C message data structure are as follows:

```
struct rt_i2c_msg
{
    rt_uint16_t addr;    /* Slave address */
    rt_uint16_t flags;   /* Reading, writing signs, etc. */
    rt_uint16_t len;     /* Read and write data bytes */
    rt_uint8_t  *buf;    /* Read and write data buffer pointer*/
}
```

Slave address (addr): Supports 7-bit and 10-bit binary addresses. You need to view the data sheets of different devices.
>The slave address used by the RT-Thread I2C device interface does not contain read/write bits. The read/write bit control needs to modify the flag `flags`.
The flags `flags` can be defined as macros that can be combined with other macros using the bitwise operation "|" as needed.

```
#define RT_I2C_WR              0x0000        /* Write flag */
#define RT_I2C_RD              (1u << 0)     /* Read flag */
#define RT_I2C_ADDR_10BIT      (1u << 2)     /* 10-bit address mode */
#define RT_I2C_NO_START        (1u << 4)     /* No start condition */
#define RT_I2C_IGNORE_NACK     (1u << 5)     /* Ignore NACK */
#define RT_I2C_NO_READ_ACK     (1u << 6)     /* Do not send ACK when reading */
```

Examples of use are as follows:

```
#define AHT10_I2C_BUS_NAME      "i2c1"  /* Sensor connected I2C bus device name */
#define AHT10_ADDR               0x38   /* Slave address */
struct rt_i2c_bus_device *i2c_bus;      /* I2C bus device handle */

/* Find the I2C bus device and get the I2C bus device handle */
i2c_bus = (struct rt_i2c_bus_device *)rt_device_find(name);

/* Read sensor register data */
static rt_err_t read_regs(struct rt_i2c_bus_device *bus, rt_uint8_t len, rt_uint8_t *buf)
{
    struct rt_i2c_msg msgs;

    msgs.addr = AHT10_ADDR;     /* Slave address */
    msgs.flags = RT_I2C_RD;     /* Read flag */
    msgs.buf = buf;             /* Read and write data buffer pointer*/
    msgs.len = len;             /* Read and write data bytes */

    /* Call the I2C device interface to transfer data */
    if (rt_i2c_transfer(bus, &msgs, 1) == 1)
    {
        return RT_EOK;
    }
    else
    {
        return -RT_ERROR;
    }
}
```

1.4.2.8.3 I2C Bus Device Usage Example

The specific usage of the I2C device can be referred to the following sample code. The main steps of the sample code are as follows:

1. First find the I2C name based on the I2C device name, get the device handle, and then initialize the aht10 sensor.
2. The two functions that control the sensor are the write sensor register `write_reg()` and the read sensor register `read_regs()`, both called `rt_i2c_transfer()` to transfer the data. The function `read_temp_humi()` calls the above two functions to read the temperature and humidity information.

```
/*
 * Program listing: This is an I2C device usage routine
 * The routine exports the i2c_aht10_sample command to the control terminal
 * Command call format: i2c_aht10_sample i2c1
 * Command explanation: The second parameter of the command is the name of the I2C bus device to be used. If
     it is empty, the default I2C bus device is used.
 * Program function: read the temperature and humidity data of the aht10 sensor and print.
*/

#include <rtthread.h>
#include <rtdevice.h>

#define AHT10_I2C_BUS_NAME          "i2c1"  /* Sensor connected I2C bus device name */
#define AHT10_ADDR                  0x38    /* Slave address */
#define AHT10_CALIBRATION_CMD       0xE1    /* Calibration command */
#define AHT10_NORMAL_CMD            0xA8    /* General command */
#define AHT10_GET_DATA              0xAC    /* Get data command */

static struct rt_i2c_bus_device *i2c_bus = RT_NULL;     /* I2C bus device handle */
static rt_bool_t initialized = RT_FALSE;                /* Sensor initialization status */

/* Write sensor register */
static rt_err_t write_reg(struct rt_i2c_bus_device *bus, rt_uint8_t reg, rt_uint8_t *data)
{
    rt_uint8_t buf[3];
    struct rt_i2c_msg msgs;

    buf[0] = reg; //cmd
    buf[1] = data[0];
    buf[2] = data[1];

    msgs.addr = AHT10_ADDR;
    msgs.flags = RT_I2C_WR;
    msgs.buf = buf;
```

```
    msgs.len = 3;

    /* Call the I2C device interface to transfer data */
    if (rt_i2c_transfer(bus, &msgs, 1) == 1)
    {
        return RT_EOK;
    }
    else
    {
        return -RT_ERROR;
    }
}

/* Read sensor register data */
static rt_err_t read_regs(struct rt_i2c_bus_device *bus, rt_uint8_t len, rt_uint8_t *buf)
{
    struct rt_i2c_msg msgs;

    msgs.addr = AHT10_ADDR;
    msgs.flags = RT_I2C_RD;
    msgs.buf = buf;
    msgs.len = len;

    /* Call the I2C device interface to transfer data */
    if (rt_i2c_transfer(bus, &msgs, 1) == 1)
    {
        return RT_EOK;
    }
    else
    {
        return -RT_ERROR;
    }
}

static void read_temp_humi(float *cur_temp, float *cur_humi)
{
    rt_uint8_t temp[6];

    write_reg(i2c_bus, AHT10_GET_DATA, 0);      /* send command */
    rt_thread_mdelay(400);
    read_regs(i2c_bus, 6, temp);                /* obtian sensor data */

    /* Humidity data conversion */
    *cur_humi = (temp[1] << 12 | temp[2] << 4 | (temp[3] & 0xf0) >> 4) * 100.0 / (1 << 20);
    /* Temperature data conversion */
    *cur_temp = ((temp[3] & 0xf) << 16 | temp[4] << 8 | temp[5]) * 200.0 / (1 << 20) - 50;
}

static void aht10_init(const char *name)
{
    rt_uint8_t temp[2] = {0, 0};

    /* Find the I2C bus device and get the I2C bus device handle */
    i2c_bus = (struct rt_i2c_bus_device *)rt_device_find(name);

    if (i2c_bus == RT_NULL)
    {
        rt_kprintf("can't find %s device!\n", name);
    }
    else
    {
        write_reg(i2c_bus, AHT10_NORMAL_CMD, temp);
        rt_thread_mdelay(400);

        temp[0] = 0x08;
        temp[1] = 0x00;
        write_reg(i2c_bus, AHT10_CALIBRATION_CMD, temp);
        rt_thread_mdelay(400);
        initialized = RT_TRUE;
    }
}

static void i2c_aht10_sample(int argc, char *argv[])
{
    float humidity, temperature;
    char name[RT_NAME_MAX];

    humidity = 0.0;
    temperature = 0.0;

    if (argc == 2)
    {
        rt_strncpy(name, argv[1], RT_NAME_MAX);
    }
    else
    {
        rt_strncpy(name, AHT10_I2C_BUS_NAME, RT_NAME_MAX);
```

```
    }

    if (!initialized)
    {
        /* Sensor initialization */
        aht10_init(name);
    }
    if (initialized)
    {
        /* Read temperature and humidity data */
        read_temp_humi(&temperature, &humidity);

        rt_kprintf("read aht10 sensor humidity   : %d.%d %%\n", (int)humidity, (int)(humidity * 10) % 10);
        rt_kprintf("read aht10 sensor temperature: %d.%d \n", (int)temperature, (int)(temperature * 10) %
      10);
    }
    else
    {
        rt_kprintf("initialize sensor failed!\n");
    }
}
/* Export to the msh command list */
MSH_CMD_EXPORT(i2c_aht10_sample, i2c aht10 sample);
```

1.4.2.9 I2C device model (DM)

1.4.2.9.1 I2C bus framework under RT_USING_DM

When **RT_USING_DM** and **RT_USING_I2C** are enabled, RT-Thread adds an **I2C-specific rt_bus** (`name = "i2c"`) on top of the core DM bus layer (Bus and driver binding (core)). **Bus controllers** still register as **RT_Device_Class_I2CBUS** devices (`i2c0`, `i2c1`, ...); **slave drivers** bind as **struct rt_i2c_client** devices discovered from device tree children.
Legacy **manual** usage (`rt_device_find("i2c1")` + `rt_i2c_transfer`) remains valid — see I2C Bus Device. This page covers **automatic DT binding** and **RT_I2C_DRIVER_EXPORT**.
Sources:

File	Role
dev_i2c_↩ bus.c	**struct rt_bus i2c_bus**, match/probe, **i2c_bus_scan_clients**
dev_i2c_↩ dm.c	**i2c_timings_ofw_parse**
dev_i2c_↩ core.c	Calls **i2c_bus_scan_clients** after **rt_i2c_bus_device_register**
dev_↩ i2c.h	**rt_i2c_client**, **rt_i2c_driver**, **RT_I2C_DRIVER_EXPORT**
dev_i2c_↩ dm.h	**struct i2c_timings**, scan/timings declarations

---

1.4.2.9.1.1 Kconfig

Option	Role
RT_USING_I2C	Core I2C stack (`dev_i2c_core.c`, `dev_i2c_dev.c`, ...)
RT_USING_DM	Adds **dev_i2c_bus.c** + **dev_i2c_dm.c** (I2C bus + OFW scan)
RT_USING_OFW	Required for **i2c_bus_scan_clients** and **i2c_timings_ofw_parse**
SOC_DM_I2C_DIR	BSP SoC adapters (e.g. `i2c-rk3x.c`)

Without **RT_USING_DM**, **struct rt_i2c_client** has no embedded **rt_device** — only **bus** + **client_↩ addr**.

1.4.2.9.1.2 Architecture

```
Platform I2C controller (e.g. rk3x, bit-bang)
      |
      |  probe: i2c_timings_ofw_parse, rt_dm_dev_iomap, clocks
      |  bus->parent.ofw_node = controller np
      |  rt_i2c_bus_device_register(&bus->parent, "i2cN")
      v
rt_device "i2cN"  (RT_Device_Class_I2CBUS, user_data -> rt_i2c_bus_device)
      |
      |  i2c_bus_scan_clients(bus)   /* dev_i2c_core.c */
      v
For each available child in DT:
      alloc rt_i2c_client, reg -> client_addr, ofw_node
      rt_i2c_device_register(client)  -> rt_bus_add_device("i2c", ...)
      v
i2c_match: driver ids[] name OR ofw_ids compatible
      |
      v
i2c_probe -> driver->probe(client)
      |
      v
Client driver uses client->bus + rt_i2c_transfer / master_send
```

Two bus layers:

Layer	Object	Example
I2C bus device	**`struct rt_i2c_bus_device`**	Hardware adapter; **`master_xfer`**, mutex **`lock`**
DM I2C bus	**`struct rt_bus i2c_bus`**	Binds **`rt_i2c_driver`** **`rt_i2c_client`**

Core DM **`platform`** bus probes the **controller**; subsystem **`i2c`** bus probes **clients** under that controller's OFW node.

1.4.2.9.1.3 `struct rt_bus i2c` (dev_i2c_bus.c)

Registered at boot: **`INIT_CORE_EXPORT(i2c_bus_init)`** → **`rt_bus_register(&i2c_bus)`**.

Callback	Behavior
`i2c_match`	1) Match **`driver->ids[].name`** to **`client->name`** (from DT node name). 2) Else **`rt_ofw_node_match(client->ofw_node, driver->ofw_ids)`**. Sets **`client->id`** or **`client->ofw_id`**.
`i2c_probe`	Requires **`client->bus`**; calls **`driver->probe(client)`**.
`i2c_remove` / **`i2c_shutdown`**	Forward to driver hooks.

Driver registration:

```
rt_err_t rt_i2c_driver_register(struct rt_i2c_driver *driver);
/* RT_I2C_DRIVER_EXPORT(drv) -> RT_DRIVER_EXPORT(drv, i2c, BUILIN) */
```

Sets **`driver->parent.bus = &i2c_bus`** then **`rt_driver_register`**.

Client registration:

```
rt_err_t rt_i2c_device_register(struct rt_i2c_client *client);
/* internally: rt_bus_add_device(&i2c_bus, &client->parent) */
```

1.4.2.9.1.4 Device tree: controller + clients

Controller node

```
i2c1: i2c@ff160000 {
    compatible = "rockchip,rk3588-i2c", "rockchip,rk3066-i2c";
    reg = <0x0 0xff160000 0x0 0x1000>;
    interrupts = <GIC_SPI 32 IRQ_TYPE_LEVEL_HIGH>;
    clocks = <&cru CLK_I2C1>, <&cru PCLK_I2C1>;
    clock-names = "i2c", "pclk";
    #address-cells = <1>;
    #size-cells = <0>;

    rtc@68 {
        compatible = "dallas,ds1307";
        reg = <0x68>;
```

```
    };
};
```

Property	Role
clock-frequency	Parsed into **struct i2c_timings.bus_freq_hz**
i2c-scl-rising-time-ns, **i2c-scl-falling-time-ns**, ...	Optional timing cells — **i2c_timings_ofw_parse**
#address-cells = <1>, **#size-cells = <0>**	Standard I2C child addressing
Child reg	7-bit (or 10-bit) slave address → **client->client_addr**

Client discovery (i2c_bus_scan_clients)

Called automatically when **rt_i2c_bus_device_register** succeeds (**RT_USING_DM** only).
For each **available** child of **bus->parent.ofw_node**:

1. If child has **compatible**, use it as client node; else look one level deeper (**i2c-mux** style).
2. **rt_calloc** **struct rt_i2c_client**.
3. Read **reg** → **client_addr**.
4. **client->parent.ofw_node**, **client->name = rt_ofw_node_name**, **client->bus = bus**.
5. **rt_dm_dev_set_name(&client->parent, "s", name)**.
6. **rt_i2c_device_register(client)** → triggers **i2c_match** + **probe** if a driver is already registered.

Order note: If the **client driver** registers before the **bus**, clients are not scanned yet. Typically **controller probe runs first** (platform init), then scan instantiates clients while **I2C drivers** are already linked via **RT_I2C_DRIVER_EXPORT**. If a driver loads late, you may need manual **rt_i2c_device_register** or re-scan (not provided in-tree — ensure init order).

1.4.2.9.1.5 Bus controller driver (adapter)

Pattern from SoC drivers (e.g. **bsp/rockchip/dm/i2c/i2c-rk3x.c**):

```
static rt_err_t my_i2c_probe(struct rt_platform_device *pdev)
{
    struct rt_device *dev = &pdev->parent;
    struct my_i2c *i2c = rt_calloc(1, sizeof(*i2c));

    i2c_timings_ofw_parse(dev->ofw_node, &i2c->timings, RT_TRUE);

    i2c->regs = rt_dm_dev_iomap(dev, 0);
    i2c->irq  = rt_dm_dev_get_irq(dev, 0);
    /* clocks: rt_clk_get_by_name, prepare_enable */

    rt_dm_dev_set_name_auto(&i2c->parent.parent, "i2c");

    i2c->parent.ops = &my_i2c_ops;
    i2c->parent.parent.ofw_node = dev->ofw_node;
    dev->user_data = i2c;

    return rt_i2c_bus_device_register(&i2c->parent, rt_dm_dev_get_name(dev));
}
```

Step	API
Timings	**i2c_timings_ofw_parse(np, &timings, use_defaults)**
Resources	**rt_dm_dev_iomap**, **rt_dm_dev_get_irq**, **rt_clk_*** (Clock framework (CLK))

Step	API
Naming	**rt_dm_dev_set_name** / **rt_dm_dev_set_name_auto** → **i2c0**, **i2c1**, …
Publish bus	**rt_i2c_bus_device_register(&i2c->parent, name)** — triggers **client scan**

rt_i2c_bus_device embeds **struct rt_device parent**; **user_data** on that device points at the full adapter struct (controller private data).
Bit-bang buses (**RT_USING_I2C_BITOPS**) call **rt_i2c_bit_add_bus** → same **rt_i2c_bus_device_register** path.

1.4.2.9.1.6 Client driver (I2C slave)

```
static const struct rt_ofw_node_id my_ids[] = {
    { .compatible = "vendor,my-sensor", .data = (void *)&chip_info },
    { /* sentinel */ },
};

static const struct rt_i2c_device_id my_tbl[] = {
    { .name = "my-sensor", .data = (void *)&chip_info },
    { /* sentinel */ },
};

static struct rt_i2c_driver my_driver = {
    .ids     = my_tbl,      /* optional: match by DT node name */
    .ofw_ids = my_ids,      /* match by compatible */
    .probe   = my_probe,
    .remove  = my_remove,
};
RT_I2C_DRIVER_EXPORT(my_driver);

static rt_err_t my_probe(struct rt_i2c_client *client)
{
    struct rt_i2c_bus_device *bus = client->bus;
    rt_uint16_t addr = client->client_addr;
    const void *cfg = rt_i2c_client_id_data(client);

    /* Use rt_i2c_master_send/recv or rt_i2c_transfer on client->bus */
    return RT_EOK;
}
```

Field / API	Role
client->bus	Parent adapter — **must** be set (scan sets it; manual clients must assign).
client->client_addr	From DT **reg** (scan) or driver setup.
client->parent.ofw_node	Child DT node for **rt_dm_dev_get_irq** etc.
rt_i2c_client_id_data(client)	**id->data** or **ofw_id->data** from match table

After **probe**, many drivers register a functional device (RTC, sensor, regulator) on top of the client, using **client->parent** only as the DM binding handle.

1.4.2.9.1.7 struct i2c_timings (OFW)

Defined in **dev_i2c_dm.h**. **i2c_timings_ofw_parse(dev_np, timings, use_defaults)** reads:

DT property	Field
clock-frequency	**bus_freq_hz** (default 100 kHz if missing and **use_defaults**)
i2c-scl-rising-time-ns	**scl_rise_ns**
i2c-scl-falling-time-ns	**scl_fall_ns**
i2c-scl-internal-delay-ns	**scl_int_delay_ns**
i2c-sda-falling-time-ns	**sda_fall_ns**

DT property	Field
i2c-sda-hold-time-ns	**sda_hold_ns**
i2c-digital-filter-width-ns	**digital_filter_width_ns**
i2c-analog-filter-cutoff-frequency	**analog_filter_cutoff_freq_hz**

Controller **probe** passes parsed timings into IP-specific **calc_timings** (SoC-specific).

1.4.2.9.1.8 Transfer path (unchanged by DM)

DM only changes **how clients appear**. Data path is still:

```
rt_i2c_bus_lock(bus, timeout);
rt_i2c_transfer(bus, msgs, num);   /* -> ops->master_xfer */
rt_i2c_bus_unlock(bus);
```

Or **rt_i2c_master_send** / **rt_i2c_master_recv**. Bus mutex serializes masters on one adapter.

1.4.2.9.1.9 DM vs non-DM summary

Topic	With RT_USING_DM	Without DM
Client struct	**rt_i2c_client** embeds **rt_device parent**	**bus + client_addr only
Binding	**RT_I2C_DRIVER_EXPORT** + DT scan	App finds bus, bit-bangs address manually
Bus register side effect	**i2c_bus_scan_clients**	No scan
Timing helpers	**i2c_timings_ofw_parse**	Driver parses DT itself or fixed Hz

1.4.2.9.1.10 Engineer checklist

1. **Controller**: platform **probe** → timings + resources → **rt_i2c_bus_device_register** with **ofw_node** on **bus->parent**.
2. **Client driver**: **ofw_ids** + optional **ids**, **RT_I2C_DRIVER_EXPORT**, **probe** uses **client->bus** and **client_addr**.
3. **DT**: controller **#address-cells** / **#size-cells**, child **reg** + **compatible**.
4. **Transfers**: always through **client->bus**, respect **rt_i2c_bus_lock** if sharing with other code.
5. **Remove**: controller **remove** should **rt_device_unregister** bus; clients removed via **i2c_remove** when bus tears down (verify BSP order).

1.4.2.9.1.11 Pitfalls

- **client->bus == NULL** in **i2c_probe** → **-RT_EINVAL** — scan did not run or client registered manually without **bus**.
- **Wrong reg address** — probe succeeds but **rt_i2c_transfer** NACKs on wire.
- **Driver before bus** — no automatic re-scan; child never **probe** until bus registers.
- **i2c-mux depth** — scan only handles **one** level of non-compatible wrapper; deeper mux trees may need explicit client creation.
- **Name vs compatible match** — **ids[].name** must match DT **node name**, not **compatible** string; prefer **ofw_ids** for compatible-based binding.
- **Mixing DM client with legacy app** — app can still **rt_i2c_bus_device_find("i2c1")**; ensure mutex discipline if both access the same bus.

1.4.2.9.1.12 See also

- I2C Bus Device — protocol, **`rt_i2c_transfer`**, application usage
- Bus and driver binding (core) — core **`rt_bus`** / **`rt_driver`**
- Device model helpers (dm.h) — **`rt_dm_dev_iomap`**, naming
- Platform bus — controller **`RT_PLATFORM_DRIVER_EXPORT`**
- Open Firmware (OFW) — DT properties
- `components/drivers/include/drivers/dev_i2c.h`
- `components/drivers/i2c/dev_i2c_bus.c`, `dev_i2c_core.c`, `dev_i2c_dm.c`

1.4.2.10 SPI device

1.4.2.10.1 SPI (Serial Peripheral Interface)

SPI is a synchronous full-duplex bus (MOSI, MISO, SCLK, CS). RT-Thread exposes a **bus device** (`spi0`, ...) and **SPI slave devices** (`spi10`, `spi1_0`, or driver-specific names) for transfers.

Topic	Page
Application API (`rt_spi_transfer`, QSPI, ...)	This page
DM bus, DT scan, **`RT_SPI_DRIVER_EXPORT`**	SPI device model (DM)

Kconfig: **`RT_USING_SPI`**. With **`RT_USING_DM`**, **`dev_spi_bus.c`** / **`dev_spi_dm.c`** add automatic child binding from device tree (SPI device model (DM)). Without DM, use manual mount below.
Implementation: **`components/drivers/spi/`**, headers **`drivers/dev_spi.h`**.

1.4.2.10.2 Introduction to SPI

SPI (Serial Peripheral Interface) is a high-speed, full-duplex, synchronous communication bus commonly used for short-range communication. It is mainly used in EEPROM, FLASH, real-time clock, AD converter, and digital signal processing and between the device and the digital signal decoder. SPI generally uses 4 lines of communication, as shown in the following figure:

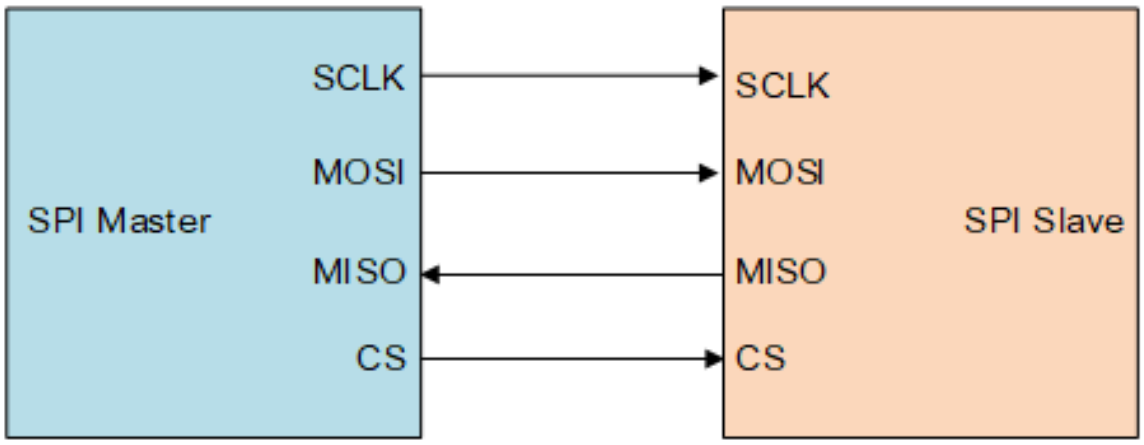


Figure 1.102 Ways of communication from SPI Master to SPI Slave

- MOSI :SPI Bus Master Output/Slave Input.
- MISO :SPI Bus Master Input/Slave Output.
- SCLK :Serial Clock, Master device outputs clock signal to slave device.

- CS : select the slave device, also called SS, CSB, CSN, EN, etc., the master device outputs a chip select signal to the slave device.

The SPI works in master-slave mode and usually has one master and one or more slaves. The communication is initiated by the master device. The master device selects the slave device to communicate through CS, and then provides a clock signal to the slave device through SCLK. The data is output to the slave device through the MOSI, and the data sent by the slave device is received through the MISO.
As shown in the figure below, the chip has two SPI controllers. The SPI controller corresponds to the SPI master. Each SPI controller can connect multiple SPI slaves. The slave devices mounted on the same SPI controller share three signal pins: SCK, MISO, MOSI, but the CS pins of each slave device are independent.

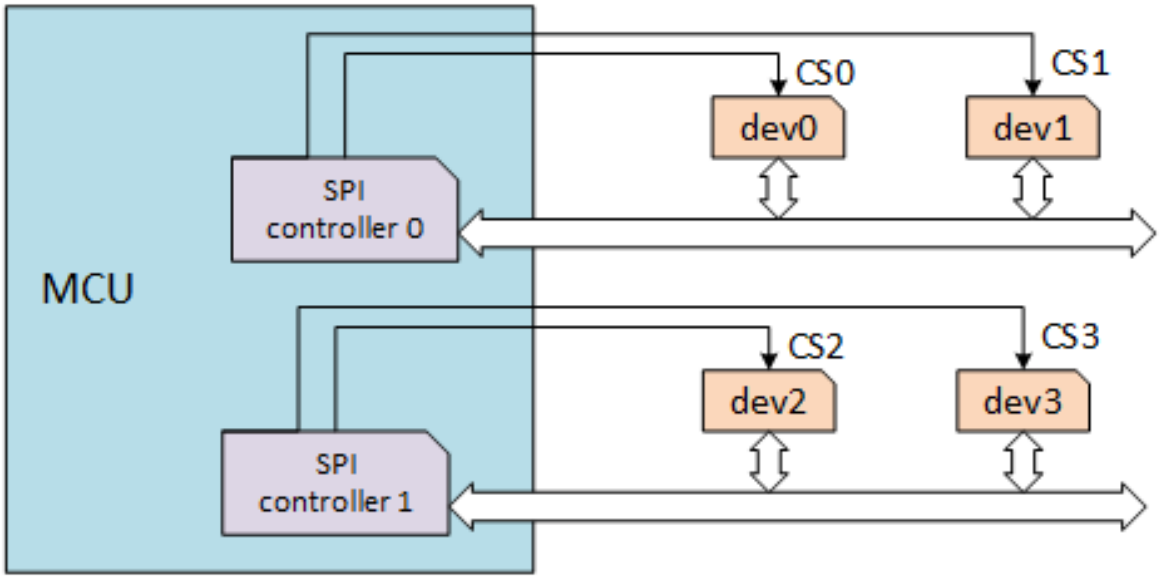


Figure 1.103 Connect from one SPI controller to multiple SPI slaves

The master device selects the slave device by controlling the CS pin, typically active low. Only one CS pin is active on an SPI master, and the slave connected to the active CS pin can now communicate with the master.
The slave's clock is provided by the master through SCLK, and MOSI and MISO complete the data transfer based on SCLK. The working timing mode of the SPI is determined by the phase relationship between CPOL (Clock Polarity) and CPHA (Clock Phase). CPOL represents the state of the initial level of the clock signal. A value of 0 indicates that the initial state of the clock signal is low, and a value of 1 indicates that the initial level of the clock signal is high. CPHA indicates on which clock edge the data is sampled. A value of 0 indicates that the data is sampled on the first clock change edge, and a value of 1 indicates that the data is sampled on the second clock change edge. There are 4 working timing modes according to different combinations of CPOL and CPHA: CPOL=0, CPHA=0; CPOL=0, CPHA=1; CPOL=1, CPHA=0; CPOL=1, CPHA=1. As shown below:

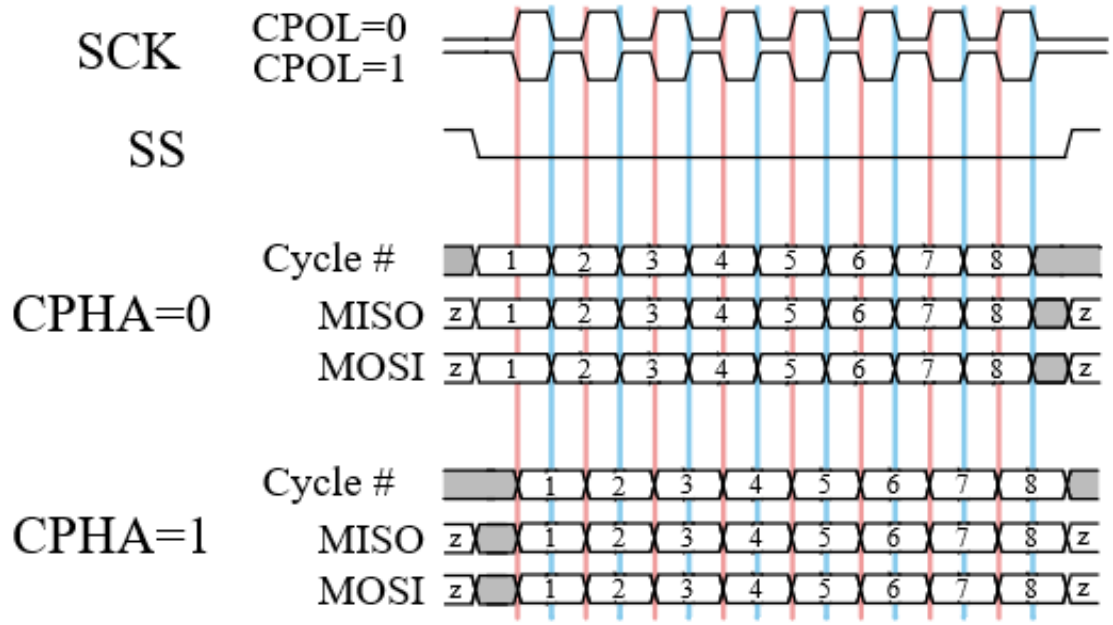


Figure 1.104 4 working timing modes of SPI

QSPI: QSPI is short for Queued SPI and is an extension of the SPI interface from Motorola, which is more extensive than SPI applications. Based on the SPI protocol, Motorola has enhanced its functionality, added a queue transfer mechanism, and introduced a queue serial peripheral interface protocol (QSPI protocol). Using this interface, users can transfer transmission queues containing up to 16 8-bit or 16-bit data at one time. Once the transfer is initiated, CPU is not required until the end of the transfer, greatly improving the transfer efficiency. Compared to SPI, the biggest structural feature of QSPI is the replacement of the transmit and receive data registers of the SPI with 80 bytes of RAM.
Dual SPI Flash: For SPI Flash, full-duplex is not commonly used. You can send a command byte into Dual mode and let it work in half-duplex mode to double data transfer. Thus, MOSI becomes SIO0 (serial io 0), and MISO becomes SIO1 (serial io 1), so that 2 bit data can be transmitted in one clock cycle, which doubles the data transmission.
Quad SPI Flash: Similar to the Dual SPI, Quad SPI Flash adds two I/O lines (SIO2, SIO3) to transfer 4 bits of data in one clock.
So for SPI Flash, there are three types of standard SPI Flash, Dual SPI Flash, Quad SPI Flash. At the same clock, the higher the number of lines, the higher the transmission rate.

1.4.2.10.3 Mount SPI Device

> **Device model (`RT_USING_DM`)**: After **`rt_spi_bus_register`**, children with **`compatible`** under the controller node are scanned automatically; use **`RT_SPI_DRIVER_EXPORT`** for chip drivers (SPI device model (DM)). The APIs below are the **legacy manual attach** path (still supported for BSP init and boards without OFW).

The SPI driver registers the SPI bus and the SPI device needs to be mounted to the SPI bus that has already been registered.

```
rt_err_t rt_spi_bus_attach_device(struct rt_spi_device *device,
                                  const char          *name,
                                  const char          *bus_name,
                                  void                *user_data)
```

Parameter	Description
device	SPI device handle
name	SPI device name
bus_name	SPI bus name

Parameter	Description
user_data	user data pointer
Return	——
RT_EOK	Success
Other Errors	Failure

This function is used to mount an SPI device to the specified SPI bus, register the SPI device with the kernel, and save user_data to the control block of the SPI device.
The general SPI bus naming principle is spix, and the SPI device naming principle is spixy. For example, spi10 means device 0 mounted on the spi1 bus. User_data is generally the CS pin pointer of the SPI device. When data is transferred, the SPI controller will operate this pin for chip select.
If you use the BSP in the `rt-thread/bsp/stm32` directory, you can use the following function to mount the SPI device to the bus:

```
rt_err_t rt_hw_spi_device_attach(const char *bus_name, const char *device_name, GPIO_TypeDef* cs_gpiox,
      uint16_t cs_gpio_pin);
```

The following sample code mounts the SPI FLASH W25Q128 to the SPI bus:

```
static int rt_hw_spi_flash_init(void)
{
    __HAL_RCC_GPIOB_CLK_ENABLE();
    rt_hw_spi_device_attach("spi1", "spi10", GPIOB, GPIO_PIN_14);

    if (RT_NULL == rt_sfud_flash_probe("W25Q128", "spi10"))
    {
        return -RT_ERROR;
    };

    return RT_EOK;
}
/* Export to automatic initialization */
INIT_COMPONENT_EXPORT(rt_hw_spi_flash_init);
```

1.4.2.10.4 Configuring SPI Device

The SPI device's transmission parameters need to be configured after the SPI device is mounted to the SPI bus.

```
rt_err_t rt_spi_configure(struct rt_spi_device *device,
                          struct rt_spi_configuration *cfg)
```

Parameter	Description
device	SPI device handle
cfg	SPI configuration parameter pointer
Return	——
RT_EOK	Success

This function saves the configuration parameters pointed to by `cfg` to the control block of the SPI device device, which is used when transferring data.
The `struct rt_spi_configuration` prototype is as follows:

```
struct rt_spi_configuration
{
    rt_uint8_t mode;        /* mode */
    rt_uint8_t data_width;  /* data width, 8 bits, 16 bits, 32 bits */
    rt_uint16_t reserved;   /* reserved */
    rt_uint32_t max_hz;     /* maximum frequency */
};
```

**Mode: **Contains MSB/LSB, master-slave mode, timing mode, etc. The available macro combinations are as follows:

```
/* Set the data transmission order whether the MSB bit is first or the LSB bit is before */
#define RT_SPI_LSB      (0<<2)                      /* bit[2]: 0-LSB */
#define RT_SPI_MSB      (1<<2)                      /* bit[2]: 1-MSB */

/* Set the master-slave mode of the SPI */
#define RT_SPI_MASTER   (0<<3)                      /* SPI master device */
```

```
#define RT_SPI_SLAVE    (1<<3)                          /* SPI slave device */

/* Set clock polarity and clock phase */
#define RT_SPI_MODE_0   (0 | 0)                          /* CPOL = 0, CPHA = 0 */
#define RT_SPI_MODE_1   (0 | RT_SPI_CPHA)                /* CPOL = 0, CPHA = 1 */
#define RT_SPI_MODE_2   (RT_SPI_CPOL | 0)                /* CPOL = 1, CPHA = 0 */
#define RT_SPI_MODE_3   (RT_SPI_CPOL | RT_SPI_CPHA)      /* CPOL = 1, CPHA = 1 */

#define RT_SPI_CS_HIGH  (1<<4)                          /* Chipselect active high */
#define RT_SPI_NO_CS    (1<<5)                          /* No chipselect */
#define RT_SPI_3WIRE    (1<<6)                          /* SI/SO pin shared */
#define RT_SPI_READY    (1<<7)                          /* Slave pulls low to pause */
```

Data width: The data width format that can be sent and received by the SPI master and SPI slaves is set to 8-bit, 16-bit or 32-bit.

Maximum Frequency Set the baud rate for data transfer, also based on the baud rate range at which the SPI master and SPI slaves operate.

The example for configuration is as follows:

```
struct rt_spi_configuration cfg;
cfg.data_width = 8;
cfg.mode = RT_SPI_MASTER | RT_SPI_MODE_0 | RT_SPI_MSB;
cfg.max_hz = 20 * 1000 *1000;                           /* 20M */

rt_spi_configure(spi_dev, &cfg);
```

1.4.2.10.5 QSPI Configuration

To configure the transmission parameters of a QSPI device, use the following function:

```
rt_err_t rt_qspi_configure(struct rt_qspi_device *device, struct rt_qspi_configuration *cfg);
```

Parameter	Description
device	QSPI device handle
cfg	QSPI configuration parameter pointer
Return	——
RT_EOK	Success

This function saves the configuration parameters pointed to by `cfg` to the control block of the QSPI device, which is used when transferring data.

The `struct rt_qspi_configuration` prototype is as follows:

```
struct rt_qspi_configuration
{
    struct rt_spi_configuration parent;     /* SPI device configuration parent */
    rt_uint32_t medium_size;                /* medium size */
    rt_uint8_t ddr_mode;                    /* double rate mode */
    rt_uint8_t qspi_dl_width ;              /* QSPI bus width, single line mode 1 bit, 2 line mode 2 bits, 4
      line mode 4 bits */
};
```

1.4.2.10.6 Access SPI Device

In general, the MCU's SPI device communicates as a master and slave. In the RT-Thread, the SPI master is virtualized as an SPI bus device. The application uses the SPI device management interface to access the SPI slave device. The main interfaces are as follows:

Function	Description
rt_device_find()	Find device handles based on SPI device name
rt_spi_transfer_message()	Custom transfer data
rt_spi_transfer()	Transfer data once
rt_spi_send()	Send data once
rt_spi_recv()	Receive data one
rt_spi_send_then_send()	Send data twice

Function	Description
rt_spi_send_then_recv()	Send then Receive

>The SPI data transfer related interface will call rt_mutex_take(). This function cannot be called in the interrupt service routine, which will cause the assertion to report an error.

1.4.2.10.6.1 Find SPI Device

Before using the SPI device, you need to find and obtain the device handle according to the SPI device name, so that you can operate the SPI device. The device function is as follows.

```
rt_device_t rt_device_find(const char* name);
```

Parameter	Description
name	Device name
Return	——
device handle	Finding the corresponding device will return the corresponding device handle
RT_NULL	Corresponding device object unfound

In general, the name of the SPI device registered to the system is spi10, qspi10, etc. The usage examples are as follows:

```
#define W25Q_SPI_DEVICE_NAME     "qspi10"   /* SPI device name */
struct rt_spi_device *spi_dev_w25q;     /* SPI device handle */

/* Find the spi device to get the device handle */
spi_dev_w25q = (struct rt_spi_device *)rt_device_find(W25Q_SPI_DEVICE_NAME);
```

1.4.2.10.6.2 Transfer Custom Data

By obtaining the SPI device handle, the SPI device management interface can be used to access the SPI device device for data transmission and reception. You can transfer messages by the following function:

```
struct rt_spi_message *rt_spi_transfer_message(struct rt_spi_device  *device,struct rt_spi_message *message)
```

Parameter	Description
device	SPI device handle
message	message pointer
Return	——
RT_NULL	Send successful
Non-null pointer	Send failed, return a pointer to the remaining unsent message

This function can transmit a series of messages, the user can customize the value of each parameter of the message structure to be transmitted, so that the data transmission mode can be conveniently controlled. The `struct rt_spi_message` prototype is as follows:

```
struct rt_spi_message
{
    const void *send_buf;           /* Send buffer pointer */
    void *recv_buf;                 /* Receive buffer pointer */
    rt_size_t length;               /* Send/receive data bytes */
    struct rt_spi_message *next;    /* Pointer to the next message to continue sending */
    unsigned cs_take    : 1;        /* Take chip selection*/
    unsigned cs_release : 1;        /* Release chip selection */
};
```

send_buf :sendbuf is the send buffer pointer. When the value is RT_NULL, it means that the current transmission is only receiving state, and no data needs to be sent.
recv_buf :recvbuf is the receive buffer pointer. When the value is RT_NULL, it means that the current transmission is in the transmit-only state. It does not need to save the received data, so the received data is directly discarded.
length :The unit of length is word, that is, when the data length is 8 bits, each length occupies 1 byte; when the data

length is 16 bits, each length occupies 2 bytes.
next :The parameter next is a pointer to the next message to continue to send. If only one message is sent, the value of this pointer is RT_NULL. Multiple messages to be transmitted are connected together in a singly linked list by the next pointer.
cs_take :A cs_take value of 1 means that the corresponding CS is set to a valid state before data is transferred.
cs_release :A cs_release value of 1 indicates that the corresponding CS is released after the data transfer ends.
>When send_buf or recv_buf is not empty, the available size for both cannot be less than length. If you use this function to transfer messages, the first message sent by cs_take needs to be set to 1. Set the chip to be valid, and the cs_release of the last message needs to be set to 1. Release the chip select.
An example of use is as follows:

```
#define W25Q_SPI_DEVICE_NAME     "qspi10"    /* SPI device name */
struct rt_spi_device *spi_dev_w25q;          /* SPI device handle */
struct rt_spi_message msg1, msg2;
rt_uint8_t w25x_read_id = 0x90;              /* command */
rt_uint8_t id[5] = {0};

/* Find the spi device to get the device handle */
spi_dev_w25q = (struct rt_spi_device *)rt_device_find(W25Q_SPI_DEVICE_NAME);
/* Send command to read ID */
struct rt_spi_message msg1, msg2;

msg1.send_buf   = &w25x_read_id;
msg1.recv_buf   = RT_NULL;
msg1.length     = 1;
msg1.cs_take    = 1;
msg1.cs_release = 0;
msg1.next       = &msg2;

msg2.send_buf   = RT_NULL;
msg2.recv_buf   = id;
msg2.length     = 5;
msg2.cs_take    = 0;
msg2.cs_release = 1;
msg2.next       = RT_NULL;

rt_spi_transfer_message(spi_dev_w25q, &msg1);
rt_kprintf("use rt_spi_transfer_message() read w25q ID is:%x%x\n", id[3], id[4]);
```

1.4.2.10.6.3 Transfer Data Once

If only transfer data for once, use the following function:

```
rt_size_t rt_spi_transfer(struct rt_spi_device *device,
                          const void           *send_buf,
                          void                 *recv_buf,
                          rt_size_t             length);
```

Parameter	Description
device	SPI device handle
send_buf	Send data buffer pointer
recv_buf	Receive data buffer pointer
length	Length of data send/received
Return	——
0	Transmission failed
Non-0 Value	Length of data successfully transferred

This function is equivalent to calling `rt_spi_transfer_message()` to transfer a message. When starting to send data, the chip is selected. When the function returns, the chip is released. The message parameter is configured as follows:

```
struct rt_spi_message msg

msg.send_buf   = send_buf;
msg.recv_buf   = recv_buf;
msg.length     = length;
msg.cs_take    = 1;
msg.cs_release = 1;
msg.next        = RT_NULL;
```

1.4.2.10.6.4 Send Data Once

If only send data once and ignore the received data, use the following function:

```
rt_size_t rt_spi_send(struct rt_spi_device *device,
                      const void           *send_buf,
                      rt_size_t             length)
```

Parameter	Description
device	SPI device handle
send_buf	Send data buffer pointer
length	Length of data sent
Return	——
0	Transmission failed
Non-0 Value	Length of data successfully transferred

Call this function to send the data of the buffer pointed to by send_buf, ignoring the received data. This function is a wrapper of the `rt_spi_transfer()` function.
This function is equivalent to calling `rt_spi_transfer_message()` to transfer a message. When the data starts to be sent, the chip is selected. When the function returns, the chip is released. The message parameter is configured as follows:

```
struct rt_spi_message msg

msg.send_buf   = send_buf;
msg.recv_buf   = RT_NULL;
msg.length     = length;
msg.cs_take    = 1;
msg.cs_release = 1;
msg.next       = RT_NULL;
```

1.4.2.10.6.5 Receive Data Once

If only receive data once, use the following function:

```
rt_size_t rt_spi_recv(struct rt_spi_device *device,
                      void                 *recv_buf,
                      rt_size_t             length);
```

Parameter	Description
device	SPI device handle
recv_buf	Send data buffer pointer
length	Length of data sent
Return	——
0	Transmission failed
Non-0 Value	Length of data successfully transferred

Call this function to receive the data and save it to the buffer pointed to by recv_buf. This function is a wrapper of the `rt_spi_transfer()` function. The SPI bus protocol stipulates that the master can only generate a clock, so when receiving data, the master will send the data 0XFF.
This function is equivalent to calling `rt_spi_transfer_message()` to transfer a message. When receiving data, the chip is selected. When the function returns, the chip is released. The message parameter is configured as follows:

```
struct rt_spi_message msg

msg.send_buf   = RT_NULL;
msg.recv_buf   = recv_buf;
msg.length     = length;
msg.cs_take    = 1;
msg.cs_release = 1;
msg.next       = RT_NULL;
```

1.4.2.10.6.6 Send Data Twice in Succession

If need to send data of 2 buffers in succession and the CS is not released within the process, you can call the following function:

```
rt_err_t rt_spi_send_then_send(struct rt_spi_device *device,
                               const void          *send_buf1,
                               rt_size_t            send_length1,
                               const void          *send_buf2,
                               rt_size_t            send_length2);
```

Parameter	Description
device	SPI device handle
send_buf1	Send data buffer pointer 1
send_length1	Send data buffer length 1
send_buf2	Send data buffer pointer 2
send_length2	Send data buffer length 2
Return	——
RT_EOK	Send Successful
-RT_EIO	Send Failed

This function can continuously send data of 2 buffers, ignore the received data, select the CS when send_buf1 is sent, and release the CS after sending send_buf2.
This function is suitable for writing a piece of data to the SPI device, sending data such as commands and addresses for the first time, and sending data of the specified length for the second time. The reason is that it is sent twice instead of being merged into one data block, or `rt_spi_send()` is called twice, because in most data write operations, commands and addresses need to be sent first, and the length is usually only a few bytes. If send it in conjunction with the data that follows, it will need a memory space request and a lot of data handling. If `rt_spi_send()` is called twice, the chip select will be released after the command and address are sent. Most SPI devices rely on setting the chip select once to be the start of the command, so the chip selects the command or address after sending. After the data is released, the operation is discarded.
This function is equivalent to calling `rt_spi_transfer_message()` to transfer 2 messages. The message parameter is configured as follows:

```
struct rt_spi_message msg1,msg2

msg1.send_buf   = send_buf1;
msg1.recv_buf   = RT_NULL;
msg1.length     = send_length1;
msg1.cs_take    = 1;
msg1.cs_release = 0;
msg1.next       = &msg2;

msg2.send_buf   = send_buf2;
msg2.recv_buf   = RT_NULL;
msg2.length     = send_length2;
msg2.cs_take    = 0;
msg2.cs_release = 1;
msg2.next       = RT_NULL;
```

1.4.2.10.6.7 Receive Data After Sending Data

If need to send data to the slave device first, then receive the data sent from the slave device, and the CS is not released within the process, call the following function to implement:

```
rt_err_t rt_spi_send_then_recv(struct rt_spi_device *device,
                               const void          *send_buf,
                               rt_size_t            send_length,
                               void                *recv_buf,
                               rt_size_t            recv_length);
```

Parameter	Description
device	SPI slave device handle

Parameter	Description
send_buf	Send data buffer pointer
send_length	Send data buffer length
recv_buf	Receive data buffer pointer
recv_length	Receive data buffer length
Return	——
RT_EOK	Successful
-RT_EIO	Failed

This function select CS when sending the first data send_buf when the received data is ignored, and the second data is sent. At this time, the master device will send the data 0XFF, and the received data will be saved in recv_buf, and CS will be released when the function returns.
This function is suitable for reading a piece of data from the SPI slave device. The first time it will send some command and address data, and then receive the data of the specified length.
This function is equivalent to calling `rt_spi_transfer_message()` to transfer 2 messages. The message parameter is configured as follows:

```
struct rt_spi_message msg1,msg2

msg1.send_buf   = send_buf;
msg1.recv_buf   = RT_NULL;
msg1.length     = send_length;
msg1.cs_take    = 1;
msg1.cs_release = 0;
msg1.next       = &msg2;

msg2.send_buf   = RT_NULL;
msg2.recv_buf   = recv_buf;
msg2.length     = recv_length;
msg2.cs_take    = 0;
msg2.cs_release = 1;
msg2.next       = RT_NULL;
```

The SPI device management module also provides `rt_spi_sendrecv8()` and `rt_spi_sendrecv16()` functions, both are wrapper of the `rt_spi_send_then_recv()`. `rt_spi_sendrecv8()` sends a byte data and receives one byte data, and`rt_spi_sendrecv16()` sends 2 bytes. The section data receives 2 bytes of data at the same time.

1.4.2.10.7 Access QSPI Device

The data transfer interface of QSPI is as follows:

Parameter	Description
rt_qspi_transfer_message()	Transfer message
rt_qspi_send_then_recv()	Send then receive
rt_qspi_send()	Send data once

>The QSPI data transfer related interface will call rt_mutex_take(). This function cannot be called in the interrupt service routine, which will cause the assertion to report an error.

1.4.2.10.7.1 Transfer Data

Transfer messages by the following function:

```
rt_size_t rt_qspi_transfer_message(struct rt_qspi_device  *device, struct rt_qspi_message *message);
```

Parameter	Description
device	QSPI device handle
message	Message pointer

Parameter	Description
Return	——
Actual transmitted message size	

The message `structure struct rt_qspi_message` prototype is as follows:

```
struct rt_qspi_message
{
    struct rt_spi_message parent;   /* inhert from struct rt_spi_message */

    struct
    {
        rt_uint8_t content;         /* Instruction content */
        rt_uint8_t qspi_lines;      /* Instruction mode, single line mode 1 bit, 2 line mode 2 bits, 4 line
      mode 4 bits */
    } instruction;                  /* Instruction phase */

     struct
    {
        rt_uint32_t content;        /* Address/alternate byte content */
        rt_uint8_t size;            /* Address/alternate byte size */
        rt_uint8_t qspi_lines;      /* Address/alternate byte mode, single line mode 1 bit, 2 line mode 2
      bits, 4 line mode 4 bits */
    } address, alternate_bytes;     /* Address/alternate byte stage */

    rt_uint32_t dummy_cycles;       /* Dummy cycle */
    rt_uint8_t qspi_data_lines;     /* QSPI data line */
};
```

1.4.2.10.7.2 Receive Data

Use the following function to receive data:

```
rt_err_t rt_qspi_send_then_recv(struct rt_qspi_device *device,
                                const void *send_buf,
                                rt_size_t send_length,
                                void *recv_buf,
                                rt_size_t recv_length);
```

Parameter	Description
device	QSPI device handle
send_buf	Send data buffer pointer
send_length	Send data length
recv_buf	Receive data buffer pointer
recv_length	Receive data length
Return	——
RT_EOK	Successful
Other Errors	Failed

The send_buf parameter contains the sequence of commands that will be sent.

1.4.2.10.7.3 Send Data

```
rt_err_t rt_qspi_send(struct rt_qspi_device *device, const void *send_buf, rt_size_t length)
```

Parameter	Description
device	QSPI device handle
send_buf	Send data buffer pointer
length	Send data length
Return	——
RT_EOK	Successful

Parameter	Description
Other Errors	Failed

The send_buf parameter contains the sequence of commands and data to be sent.

1.4.2.10.8 Special Usage Scenarios

In some special usage scenarios, a device wants to monopolize the bus for a period of time, and the CS is always valid during the period, during which the data transmission may be intermittent, then the relevant interface can be used as shown. The transfer data function must use `rt_spi_transfer_message()`, and this function must set the cs_take and cs_release of the message to be transmitted to 0 value, because the CS has already used other interface control, and does not need to control during data transmission.

1.4.2.10.8.1 Acquire the SPI bus

In the case of multi-threading, the same SPI bus may be used in different threads. In order to prevent the data being transmitted by the SPI bus from being lost, the slave device needs to acquire the right to use the SPI bus before starting to transfer data. To transfer data using the bus, use the following function to acquire the SPI bus:

```
rt_err_t rt_spi_take_bus(struct rt_spi_device *device);
```

Parameter	Description
device	SPI device handle
Return	——
RT_EOK	Successful
Other Errors	Failed

1.4.2.10.8.2 Select CS

After obtaining the usage right of the bus from the device, you need to set the corresponding chip selection signal to be valid. You can use the following function to select the CS:

```
rt_err_t rt_spi_take(struct rt_spi_device *device)
```

Parameter	Description
device	SPI device handle
Return	——
0	Successful
Other Errors	Failed

1.4.2.10.8.3 Add a New Message

When using `rt_spi_transfer_message()` to transfer messages, all messages to be transmitted are connected in the form of a singly linked list. Use the following function to add a new message to be sent to the message list:

```
void rt_spi_message_append(struct rt_spi_message *list,
                           struct rt_spi_message *message);
```

Parameter	Description
list	Message link node to be transmitted
message	New message pointer

1.4.2.10.8.4 Release CS

After the device data transfer is completed, CS need to be released. Use the following function to release the CS:

```
rt_err_t rt_spi_release(struct rt_spi_device *device)
```

Parameter	**D**escription
device	SPI device handle
Return	——
0	Successful
Other Errors	Failed

1.4.2.10.8.5 Release Data Bus

The slave device does not use the SPI bus to transfer data. The bus must be released as soon as possible so that other slave devices can use the SPI bus to transfer data. The following function can be used to release the bus:

```
rt_err_t rt_spi_release_bus(struct rt_spi_device *device);
```

Parameter	Description
device	SPI device handle
Return	——
RT_EOK	Successful

1.4.2.10.9 SPI Device Usage Example

The specific use of the SPI device can be referred to the following sample code. The sample code first finds the SPI device to get the device handle, and then uses the rt_spi_transfer_message() send command to read the ID information.

```
/*
 * Program listing: This is a SPI device usage routine
 * The routine exports the spi_w25q_sample command to the control terminal
 * Command call format: spi_w25q_sample spi10
 * Command explanation: The second parameter of the command is the name of the SPI device to be used. If it
     is empty, the default SPI device is used.
 * Program function: read w25q ID data through SPI device
*/

#include <rtthread.h>
#include <rtdevice.h>

#define W25Q_SPI_DEVICE_NAME     "qspi10"

static void spi_w25q_sample(int argc, char *argv[])
{
    struct rt_spi_device *spi_dev_w25q;
    char name[RT_NAME_MAX];
    rt_uint8_t w25x_read_id = 0x90;
    rt_uint8_t id[5] = {0};

    if (argc == 2)
    {
        rt_strncpy(name, argv[1], RT_NAME_MAX);
    }
    else
    {
        rt_strncpy(name, W25Q_SPI_DEVICE_NAME, RT_NAME_MAX);
    }

    /* Find the spi device to get the device handle */
    spi_dev_w25q = (struct rt_spi_device *)rt_device_find(name);
    if (!spi_dev_w25q)
    {
        rt_kprintf("spi sample run failed! can't find %s device!\n", name);
    }
    else
    {
        /* Method 1: Send the command to read the ID using rt_spi_send_then_recv() */
        rt_spi_send_then_recv(spi_dev_w25q, &w25x_read_id, 1, id, 5);
```

```
        rt_kprintf("use rt_spi_send_then_recv() read w25q ID is:%x%x\n", id[3], id[4]);

        /* Method 2: Send the command to read the ID using rt_spi_transfer_message() */
        struct rt_spi_message msg1, msg2;

        msg1.send_buf   = &w25x_read_id;
        msg1.recv_buf   = RT_NULL;
        msg1.length     = 1;
        msg1.cs_take    = 1;
        msg1.cs_release = 0;
        msg1.next       = &msg2;

        msg2.send_buf   = RT_NULL;
        msg2.recv_buf   = id;
        msg2.length     = 5;
        msg2.cs_take    = 0;
        msg2.cs_release = 1;
        msg2.next       = RT_NULL;

        rt_spi_transfer_message(spi_dev_w25q, &msg1);
        rt_kprintf("use rt_spi_transfer_message() read w25q ID is:%x%x\n", id[3], id[4]);

    }
}
/* Export to the msh command list */
MSH_CMD_EXPORT(spi_w25q_sample, spi w25q sample);
```

1.4.2.10.9.1 See also (driver)

- SPI device model (DM) — DT scan, **spi_device_ofw_parse**, **RT_SPI_DRIVER_EXPORT**
- Platform bus — SPI controller platform drivers
- PIN Device — **cs-gpios**
- components/drivers/spi/SConscript

1.4.2.10.9.2 See also (driver)

- SPI device model (DM)
- Platform bus
- PIN Device
- `components/drivers/spi/SConscript`

1.4.2.11 SPI device model (DM)

1.4.2.11.1 SPI bus framework under `RT_USING_DM`

When **`RT_USING_DM`** and **`RT_USING_SPI`** are enabled, RT-Thread adds an **SPI-specific `rt_bus`** (`name = "spi"`) on top of the core DM layer (Bus and driver binding (core)). **Bus controllers** register as **`RT_Device_Class_SPIBUS`** devices (`spi0`, `spi1`, ...); **slave drivers** bind as **`struct rt_spi_device`** instances discovered from device-tree children.
Legacy **manual** attach (`rt_spi_bus_attach_device`, `rt_hw_spi_device_attach`, GPIO **`user_data`**) remains valid — see SPI device. This page covers **DT scan**, **`spi_device_ofw_parse`**, and **`RT_SPI_DRIVER_EXPORT`**.
Sources:

File	Role
`dev_spi_bus.c`	**`struct rt_bus spi_bus`**, match/probe, **`spi_bus_scan_devices`**
`dev_spi_dm.c`	**`spi_device_ofw_parse`**
`dev_spi_core.c`	**`spi_bus_register`** → scan + CS GPIO from DT

File	Role
dev_↩ spi.h	**rt_spi_driver**, **RT_SPI_DRIVER_EXPORT**
dev_spi_↩ dm.h	Parse / scan declarations

1.4.2.11.1.1 Kconfig

Option	Role
RT_USING_SPI	Core (dev_spi_core.c, dev_spi.c, ...)
RT_USING_DM	Adds **dev_spi_bus.c** + **dev_spi_dm.c**
RT_USING_OFW	Required for **spi_bus_scan_devices** and **spi_device_ofw_parse**
RT_USING_QSPI	QSPI bus mode **RT_SPI_BUS_MODE_QSPI**, alloc **rt_qspi_device** in scan
SOC_DM_SPI_DIR	BSP SoC SPI/QSPI platform drivers

Without **RT_USING_DM**, **struct rt_spi_device** has no **name** / **id** / **ofw_id** / **chip_select[]** — use manual attach only.

1.4.2.11.1.2 Architecture

```
Platform SPI/QSPI controller
      |
      |  probe: iomap, clk, pinctrl; bus->parent.ofw_node = controller np
      |  rt_spi_bus_register / rt_qspi_bus_register
      v
rt_device "spiN"  (RT_Device_Class_SPIBUS)
      |
      |  spi_bus_scan_devices(bus)   /* dev_spi_core.c, after CS pins */
      v
For each DT child with "compatible":
      alloc rt_spi_device or rt_qspi_device
      spi_device_ofw_parse → config, reg → chip_select[]
      rt_spi_device_register → rt_bus_add_device("spi", ...)
      v
spi_match: driver ids[].name OR ofw_ids compatible
      |
      v
spi_probe → driver->probe(device)
      |     map cs_pin from bus->cs_pins[chip_select[0]]
      |     if parent.type == Unknown → rt_spidev_device_init (user-visible dev)
      v
App: rt_spi_transfer* on device, or rt_device_find for spidev-style node
```

Two layers:

Layer	Object	Example
SPI bus	**struct rt_spi_bus**	Hardware adapter; **ops->configure**, **ops->xfer**, mutex **lock**
DM SPI bus	**struct rt_bus spi_bus**	Binds **rt_spi_driver** **rt_spi_device**

The **platform** bus probes the **controller**; subsystem **spi** bus probes **children** under that controller's OFW node.

1.4.2.11.1.3 struct rt_bus spi (dev_spi_bus.c)

INIT_CORE_EXPORT(spi_bus_init) → **rt_bus_register(&spi_bus)**.

Callback	Behavior
spi_match	1) **driver->ids[].name** vs **device->name** (DT node name). 2) Else **rt_ofw_node_match(ofw_node, driver->ofw_ids)**.
spi_probe	Requires **device->bus**; **driver->probe(device)**; then **cs_pin = bus->cs_pins[chip_select↩ [0]]** if controller listed **cs GPIOs**; if **parent.type == RT_Device_Class_Unknown**, **rt_spidev_device_init** registers a **RT_Device_Class_SPIDevice** for **rt_device_↩ find**.
spi_remove / **spi_shutdown**	Driver hook, **rt_free(device)**.

Driver registration:

```
rt_err_t rt_spi_driver_register(struct rt_spi_driver *driver);
/* RT_SPI_DRIVER_EXPORT(drv) -> RT_DRIVER_EXPORT(drv, spi, BUILIN) */
```

Client registration:

```
rt_err_t rt_spi_device_register(struct rt_spi_device *device);
/* rt_bus_add_device(&spi_bus, &device->parent) */
```

1.4.2.11.1.4 Device tree

Controller

```
spi1: spi@ff640000 {
    compatible = "rockchip,rk3588-spi", "rockchip,rk3066-spi";
    reg = <0x0 0xff640000 0x0 0x1000>;
    interrupts = <GIC_SPI 106 IRQ_TYPE_LEVEL_HIGH>;
    clocks = <&cru CLK_SPI1>, <&cru PCLK_SPI1>;
    clock-names = "spiclk", "apb_pclk";
    #address-cells = <1>;
    #size-cells = <0>;
    cs-gpios = <&gpio3 RK_PB3 GPIO_ACTIVE_LOW>;

    flash@0 {
        compatible = "jedec,spi-nor";
        reg = <0>;
        spi-max-frequency = <50000000>;
    };
};
```

Property (controller)	Role
#address-cells / **#size-cells**	Standard SPI child addressing
cs-gpios	Optional; parsed via **rt_pin_get_named_↩ pin(&bus->parent, "cs", i, ...)** into **bus->cs_↩ pins[]**
num-cs (driver-specific)	Some BSPs set **bus->num_chipselect**

Child (slave)

Property	Role
compatible	Required for scan (children without it are skipped)
reg	Chip select index(es) → **chip_select[]** (up to **RT_SPI_CS_CNT_MAX**)
spi-max-frequency	**config.max_hz**
spi-cpol, **spi-cpha**, **spi-cs-high**, **spi-lsb-first**, **spi-3wire**	**config.mode** bits
spi-tx-bus-width, **spi-rx-bus-width**	Dual/quad line width hints (DM **data_width_tx/↩ rx**)

Property	Role
spi-cs-setup-delay-ns, **spi-cs-hold-delay-ns**, **spi-cs-inactive-delay-ns**	**rt_spi_delay** on device

spi_device_ofw_parse (dev_spi_dm.c) fills **struct rt_spi_configuration** and CS indices before **rt_spi_device_register**.

1.4.2.11.1.5 spi_bus_scan_devices

Called from **spi_bus_register** when **RT_USING_DM** and **bus->parent.ofw_node** is set.
For each **available** child with **compatible**:

1. Allocate **struct rt_spi_device** or **struct rt_qspi_device** from **bus->mode**.
2. **spi_dev->bus = bus**, **name = rt_ofw_node_name**, **parent.ofw_node = child**.
3. **rt_dm_dev_set_name(..., rt_ofw_node_full_name)** (full path string).
4. **spi_device_ofw_parse(spi_dev)** — on error, skip device.
5. **rt_spi_device_register(spi_dev)** → match + probe.

Order: Controller **probe** should run before client drivers need the bus; **RT_SPI_DRIVER_EXPORT** drivers link at **INIT_DEVICE_EXPORT**. If a driver registers before the bus exists, no auto re-scan — ensure bus registers first (typical platform init order).

1.4.2.11.1.6 Bus controller driver (adapter)

Pattern (SoC-specific sources under **SOC_DM_SPI_DIR** or BSP):

```
static rt_err_t my_spi_probe(struct rt_platform_device *pdev)
{
    struct rt_device *dev = &pdev->parent;
    struct my_spi *spi = rt_calloc(1, sizeof(*spi));

    spi->regs = rt_dm_dev_iomap(dev, 0);
    spi->irq  = rt_dm_dev_get_irq(dev, 0);
    /* rt_clk_get_by_name, prepare_enable; pinctrl from DT */

    spi->parent.parent.ofw_node = dev->ofw_node;
    spi->parent.ops = &my_spi_ops;

    rt_dm_dev_set_name_auto(&spi->parent.parent, "spi");

    return rt_spi_bus_register(&spi->parent, rt_dm_dev_get_name(dev), &my_spi_ops);
}
```

Step	API
Resources	**rt_dm_dev_iomap**, **rt_dm_dev_get_irq**, **rt_clk_*** (Clock framework (CLK))
Pins	**cs-gpios** on controller → automatic in **spi_bus_register**; **pinctrl-*** on controller (Pin control (pinctrl))
Publish	**rt_spi_bus_register** or **rt_qspi_bus_register** — triggers **scan**

Bit-bang: **rt_spi_bit_add_bus** → same **rt_spi_bus_register** path.

1.4.2.11.1.7 Client driver (SPI slave)

```
static const struct rt_ofw_node_id my_ids[] = {
    { .compatible = "vendor,my-flash" },
    { /* sentinel */ },
};

static struct rt_spi_driver my_driver = {
    .ofw_ids = my_ids,
    .probe   = my_probe,
    .remove  = my_remove,
```

```
};
RT_SPI_DRIVER_EXPORT(my_driver);

static rt_err_t my_probe(struct rt_spi_device *device)
{
    /* device->bus, device->chip_select[], device->config already parsed */
    /* Optional: rt_spi_configure(device, &device->config); */
    /* Register MTD/sensor or use transfers directly */
    return RT_EOK;
}
```

Field / API	Role
device->bus	Parent adapter — required (**-RT_EINVAL** in **spi_probe** if NULL)
device->chip_select[0]	From DT **reg**; used with **bus->cs_pins[]**
device->cs_pin	Set in **spi_probe** from GPIO table or **PIN_NONE**
rt_spi_device_id_data(device)	**id->data** or **ofw_id->data**

Generic spidev path (dev_spi.c)

Built-in **spidev_driver** matches known **compatible** strings and DT node **names** (table in **dev_spi.c**). After **spidev_probe**, **spi_probe** may call **rt_spidev_device_init** so users get **rt_device_find("spi1_0")**-style names (**s_d**, bus name + CS).
Note: Nodes whose **compatible** is exactly **"spidev"** are **rejected** in **spidev_probe** — use a vendor entry from the table (e.g. **rockchip,spidev** is listed but contains **spidev** substring — the check uses **rt_dm_dev_prop_index_of_string(..., "spidev")** which may match **rockchip,spidev** — that would fail! Line 175-179 checks index of "spidev" in compatible - rockchip,spidev would match and return error. That might be intentional to block linux spidev nodes.

1.4.2.11.1.8 Transfer path (unchanged by DM)

DM only affects **how devices appear**. Data path is still:

```
rt_spi_configure(device, &cfg);   /* or use pre-parsed device->config */
rt_spi_transfer(device, tx, rx, len);
/* or rt_spi_transfer_message, send_then_recv, take_bus / release */
```

Bus mutex: **rt_spi_take_bus** / **rt_spi_release_bus**. Do not call transfer APIs from ISR (they take mutex).

1.4.2.11.1.9 DM vs non-DM

Topic	With **RT_USING_DM**	Without DM
Device struct	**rt_spi_device** + **rt_device** **parent**, OFW fields	**bus**, **config**, **cs_pin**, **user_data**
Binding	**RT_SPI_DRIVER_EXPORT** + DT scan	**rt_spi_bus_attach_device**, manual **cs_pin**
Bus register	**spi_bus_scan_devices**, **cs-gpios** parse	No scan
Config from DT	**spi_device_ofw_parse**	**rt_spi_configure** in init code

1.4.2.11.1.10 Engineer checklist

1. **Controller**: platform **probe** → resources → **bus->parent.ofw_node** → **rt_spi_bus_register**.
2. **DT**: controller **#address-cells** / **#size-cells**, **cs-gpios** if needed; child **reg** + **compatible** + **spi-max-frequency**.
3. **Client driver**: **ofw_ids** (+ optional **ids** by node name), **RT_SPI_DRIVER_EXPORT**, **probe** uses **device->bus** and **rt_spi_*** APIs.

4. **CS**: Prefer controller **cs-gpios** + child **reg**; legacy boards can still pass GPIO in **rt_spi_bus_attach_device** **user_data** without DM.
5. **Remove**: Tear down controller bus; **spi_remove** frees scanned devices when bus unregisters (verify BSP order).

1.4.2.11.1.11 Pitfalls

- **device->bus == NULL** → probe fails — scan did not run or manual device missing **bus**.
- **No compatible on child** — skipped by scan silently.
- **Wrong reg (CS index)** — probe OK, wrong chip selected on wire.
- **Driver before bus** — no in-tree re-scan; child never probes.
- **ids[].name** must match DT **node name**, not **compatible** — prefer **ofw_ids**.
- **QSPI child on SPI bus** — scan picks type from **bus->mode**; mismatch causes alloc/assert issues.
- **Mixing DM scan with manual attach** on same bus — possible but avoid duplicate CS / duplicate device names.

1.4.2.11.1.12 See also

- SPI device — **rt_spi_transfer**, attach API, QSPI, examples
- Bus and driver binding (core) — core **rt_bus**
- Platform bus — controller **RT_PLATFORM_DRIVER_EXPORT**
- PIN Device — **cs-gpios**
- Open Firmware (OFW)
- components/drivers/include/drivers/dev_spi.h
- components/drivers/spi/dev_spi_bus.c, dev_spi_core.c, dev_spi_dm.c

1.4.2.12 PWM Device

1.4.2.12.1 Introduction to PWM

PWM (Pulse Width Modulation) is a method of digitally encoding the level of an analog signal. The frequency of the square wave is used to encode the level of a specific analog signal by pulses of different frequencies. The output receives a series of pulses of equal magnitude and uses these pulses to replace the device with the desired waveform.

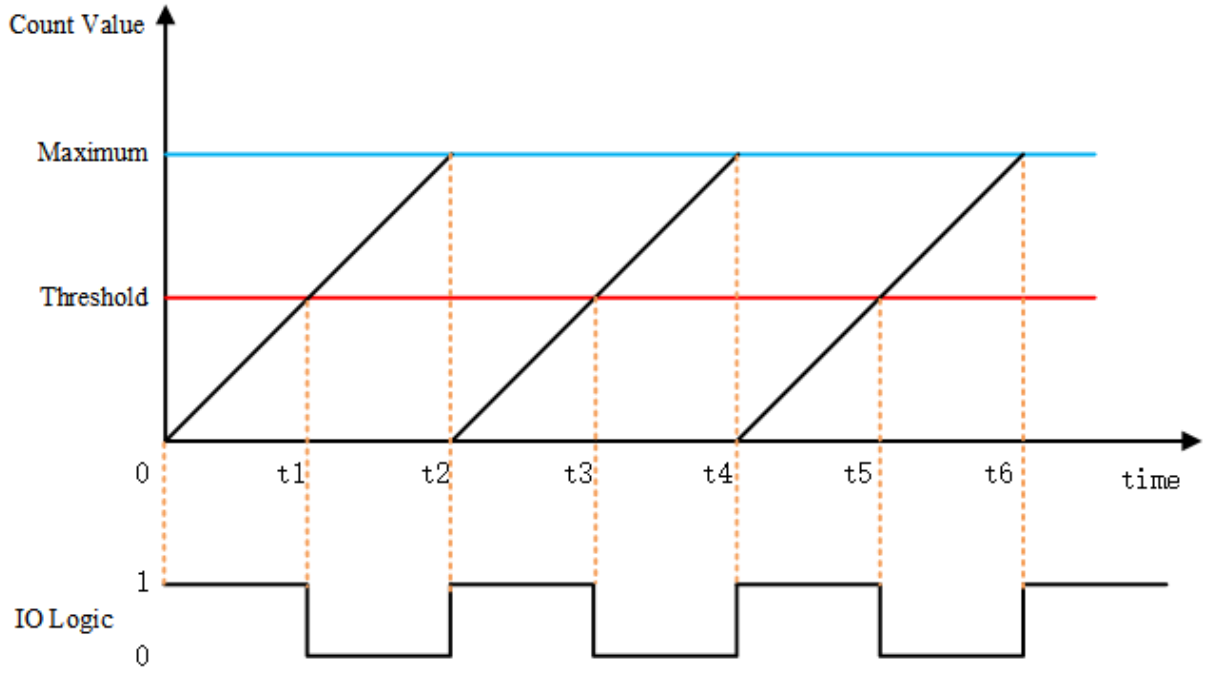


Figure 1.105 PWM Schematic Diagram

Above is a simple schematic diagram of PWM. Assuming that the timer works in a up-counter mode. When the count value is less than the threshold, it outputs a level state, such as a high level. When the count value is greater than the threshold, it outputs the opposite, such as a low level. When the count value reaches the maximum value, the counter recounts from 0 and returns to the original level state. The ratio of the high-level duration (pulse width) to the cycle time is the duty cycle, ranging from 0 to 100%. The high level of the above picture is just half of the cycle time, so the duty cycle is 50%.

One of the common PWM control scenarios is to adjust the brightness of a light or screen. The brightness can be adjusted through changing the duty cycle. The PWM does not adjust the light continuously, rather it constantly turns the screen on and off. When the light is turned on and off fast enough, the naked eye will always think that it is always bright. In the process of switching it on and off, the longer the light is off, the lower the brightness of the screen to the naked eye. Conversely, the longer the light is on, the brighter the screen will appear.

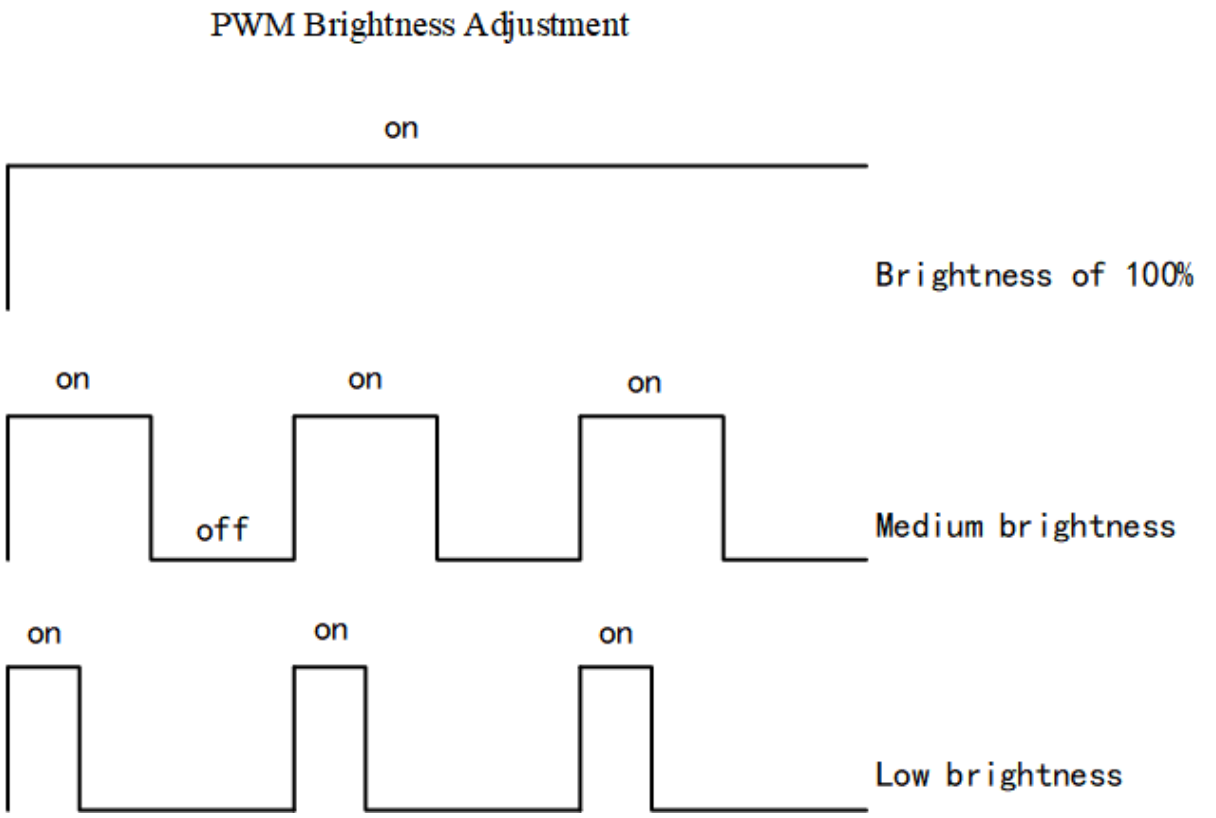


Figure 1.106 PWM Brightness Adjustment

1.4.2.12.2 Access to PWM Devices

The application accesses the PWM device hardware through the PWM device management interface provided by RT-Thread. The related interfaces are as follows:

Function	Description
rt_device_find()	Find device handles based on the name of PWM device
rt_pwm_set()	Set PWM period and pulse width
rt_pwm_enable()	Enable PWM device
rt_pwm_disable()	Disable the PWM device

1.4.2.12.2.1 Find the PWM Device

The application obtains the device handle based on the name of PWM device, which in turn can operate the PWM device. The function is as follows:

```
rt_device_t rt_device_find(const char* name);
```

Parameter	Description
name	Device
Return	——
Device handle	Found the corresponding device, will return the corresponding device handle
RT_NULL	Device not found

In general, the name of the PWM device registered to the system is pwm0, pwm1, etc. The usage examples are as follows:

```
#define PWM_DEV_NAME        "pwm3"  /* name of PWM device */
struct rt_device_pwm *pwm_dev;      /* PWM device handle */
/* Search the device */
pwm_dev = (struct rt_device_pwm *)rt_device_find(PWM_DEV_NAME);
```

1.4.2.12.2.2 Set PWM Period and Pulse Width

Set the PWM period and duty cycle by using the following function:

```
rt_err_t rt_pwm_set(struct rt_device_pwm *device,
                    int channel,
                    rt_uint32_t period,
                    rt_uint32_t pulse);
```

Parameter	Description
device	PWM device handle
channel	PWM channel
period	PWM period (ns)
pulse	PWM pulse width time (ns)
Return	——
RT_EOK	successful
-RT_EIO	device is null
-RT_ENOSYS	Device operation method is null
Other Errors	Execute failed

The output frequency of the PWM is determined by the period. For example, the time of a period is 0.5ms (milliseconds), the period value is 500000ns (nanoseconds), the output frequency is 2KHz, the duty cycle is `pulse / period`, and the pulse value cannot exceed period.
An example of use is as follows:

```
#define PWM_DEV_NAME        "pwm3"  /* name of PWM device */
#define PWM_DEV_CHANNEL      4      /* PWM channel */
struct rt_device_pwm *pwm_dev;      /* PWM device handle */
rt_uint32_t period, pulse;

period = 500000;    /* The period is 0.5ms, the unit is nanoseconds */
pulse = 0;          /* PWM pulse width value, the unit is nanoseconds */
/* Search the device */
pwm_dev = (struct rt_device_pwm *)rt_device_find(PWM_DEV_NAME);
/* Set the PWM period and pulse width */
rt_pwm_set(pwm_dev, PWM_DEV_CHANNEL, period, pulse);
```

1.4.2.12.2.3 Enable the PWM Device

After setting the PWM period and pulse width, you can enable the PWM device by the following function:

```
rt_err_t rt_pwm_enable(struct rt_device_pwm *device, int channel);
```

Parameter	Description
device	PWM device handle
channel	PWM channel
Return	——
RT_EOK	Enable device successful
-RT_ENOSYS	Device operation method is null
Other Errors	Enable device failed

An example of use is as follows:

```
#define PWM_DEV_NAME        "pwm3"  /* name of PWM device */
#define PWM_DEV_CHANNEL      4      /* PWM channel */
struct rt_device_pwm *pwm_dev;      /* PWM device handle */
```

```
rt_uint32_t period, pulse;

period = 500000;    /* The period is 0.5ms, the unit is nanoseconds */
pulse = 0;          /* PWM pulse width value, the unit is nanoseconds */
/* Search the device */
pwm_dev = (struct rt_device_pwm *)rt_device_find(PWM_DEV_NAME);
/* Set the PWM period and pulse width */
rt_pwm_set(pwm_dev, PWM_DEV_CHANNEL, period, pulse);
/* Enable the device */
rt_pwm_enable(pwm_dev, PWM_DEV_CHANNEL);
```

1.4.2.12.2.4 Disable the PWM device Channel

Use the following function to turn off the corresponding channel of the PWM device.

```
rt_err_t rt_pwm_disable(struct rt_device_pwm *device, int channel);
```

Parameter	Description
device	PWM device handle
channel	PWM channel
Return	——
RT_EOK	Turn off device successful
-RT_EIO	Device handle is null
Other Errors	Turn off device failed

An example of use is as follows:

```
#define PWM_DEV_NAME        "pwm3"  /* name of PWM device */
#define PWM_DEV_CHANNEL      4      /* PWM channel */
struct rt_device_pwm *pwm_dev;      /* PWM device handle */
rt_uint32_t period, pulse;

period = 500000;    /* The period is 0.5ms, the unit is nanoseconds */
pulse = 0;          /* PWM pulse width value, the unit is nanoseconds */
/* Search the device */
pwm_dev = (struct rt_device_pwm *)rt_device_find(PWM_DEV_NAME);
/* Set the PWM period and pulse width */
rt_pwm_set(pwm_dev, PWM_DEV_CHANNEL, period, pulse);
/* Enable the device */
rt_pwm_enable(pwm_dev, PWM_DEV_CHANNEL);
/* Turn off the device channel */
rt_pwm_disable(pwm_dev,PWM_DEV_CHANNEL);
```

1.4.2.12.3 FinSH Command

To set the period and duty cycle of a channel of a PWM device, use the command `pwm_set pwm1 1 500000 5000`. The first parameter is the command, the second parameter is the PWM device name, the third parameter is the PWM channel, and the fourth parameter is PWM period (ns), the fifth parameter is the pulse width (ns).

```
msh />pwm_set pwm1 1 500000 5000
msh />
```

To enable a channel of the PWM device, use the command `pwm_enable pwm1 1`. The first parameter is the command, the second parameter is the PWM device name, and the third parameter is the PWM channel.

```
msh />pwm_enable pwm1 1
msh />
```

To disable a channel of the PWM device, use the command `pwm_disable pwm1 1`. The first parameter is the command, the second parameter is the PWM device name, and the third parameter is the PWM channel.

```
msh />pwm_disable pwm1 1
msh />
```

1.4.2.12.4 PWM Device Usage Example

The following sample code is a PWM device usage sample . The main steps of the sample code are as follows:

1. Find the PWM device to get the device handle.
2. Set the PWM period and pulse width.

3. Enable the PWM device.
4. The pulse width is modified every 50 milliseconds in the while loop.
5. Connect the PWM channel to a LED, and you can see that the LED changes from dark to bright gradually, and then from bright to dark.

```
/*
 * Program list: This is PWM device usage example
 * The routine exports the pwm_led_sample command to the control terminal
 * Format for Command: pwm_led_sample
 * Program function: By controlling the brightness of the LED light through the PWM device,
 * you can see that the LED changes from dark to bright gradually, then from bright to dark.
 */

#include <rtthread.h>
#include <rtdevice.h>

#define PWM_DEV_NAME        "pwm3"  /* PWM device name */
#define PWM_DEV_CHANNEL     4       /* PWM channel */

struct rt_device_pwm *pwm_dev;      /* PWM device handle */

static int pwm_led_sample(int argc, char *argv[])
{
    rt_uint32_t period, pulse, dir;

    period = 500000;    /* The period is 0.5ms, the unit is nanoseconds */
    dir = 1;            /* Increase or decrease direction of PWM pulse width value */
    pulse = 0;          /* PWM pulse width value, the unit is nanoseconds*/

    /* Search the Device */
    pwm_dev = (struct rt_device_pwm *)rt_device_find(PWM_DEV_NAME);
    if (pwm_dev == RT_NULL)
    {
        rt_kprintf("pwm sample run failed! can't find %s device!\n", PWM_DEV_NAME);
        return -RT_ERROR;
    }

    /* Set PWM period and pulse width defaults */
    rt_pwm_set(pwm_dev, PWM_DEV_CHANNEL, period, pulse);
    /* Enable device */
    rt_pwm_enable(pwm_dev, PWM_DEV_CHANNEL);

    while (1)
    {
        rt_thread_mdelay(50);
        if (dir)
        {
            pulse += 5000;      /* Increase 5000ns each time from 0 */
        }
        else
        {
            pulse -= 5000;      /* 5000ns reduction from the maximum */
        }
        if (pulse >= period)
        {
            dir = 0;
        }
        if (0 == pulse)
        {
            dir = 1;
        }

        /* Set the PWM period and pulse width */
        rt_pwm_set(pwm_dev, PWM_DEV_CHANNEL, period, pulse);
    }
}
/* Export to the msh command list */
MSH_CMD_EXPORT(pwm_led_sample, pwm sample);
```

1.4.2.13 RTC Device

RTC DM helpers (`rtc_dm.h`)

1.4.2.13.1 Introduction of RTC

With **RT_USING_DM**, hardware RTC drivers register via platform / I2C / SPI **probe** — see RTC DM helpers (`rtc_dm.h`). This page documents the **set_date** / **set_time** / **time()** application API.
The RTC (Real-Time Clock) provides accurate real-time clock time, which can be used to generate information such as year, month, day, hour, minute, and second. At present, most real-time clock chips use a higher precision crystal

oscillator as a clock source. In order to work when the main power supply is powered down, some clock chips will be powered by a battery to keep the time information valid.
The RT-Thread RTC device provides the basic services for the operating system's time system. RTCs find many uses in IoT scenarios, and even in secure transmission processes such as SSL, RTC has become an indispensable part.

1.4.2.13.2 Access RTC Devices

The application accesses the RTC hardware through the RTC device management interface, and the relevant interfaces are as follows:

Function	Description
set_date()	Set date, year, month, day
set_time()	Set time, hour, minute, second
time()	Obtain current time

1.4.2.13.2.1 Set Date

Set the current date value of the RTC device by the following functions:

```
rt_err_t set_date(rt_uint32_t year, rt_uint32_t month, rt_uint32_t day)
```

Parameter	Description
year	The year to be set to take effect
month	The month to be set to take effect
day	The date to be set to take effect
return	——
RT_EOK	Set-up succeeded
-RT_ERROR	Set-up failed, no rtc device found
other error code	Set-up failed

An example of use is as follows:

```
/* Set the date to December 3, 2018 */
set_date(2018, 12, 3);
```

1.4.2.13.2.2 Set Time

Set the current time value of the RTC device by the following function:

```
rt_err_t set_time(rt_uint32_t hour, rt_uint32_t minute, rt_uint32_t second)
```

Parameter	Description
hour	The hour to be set to take effect
minute	The minute to be set to take effect
second	The second to be set to take effect
return	——
RT_EOK	Set-up succeeded
-RT_ERROR	Set-up failed, no rtc device found
other error code	Set-up failed

An example of use is as follows:

```
/* Set the time to 11:15:50 */
```

```
set_time(11, 15, 50);
```

1.4.2.13.2.3 Obtain Current Time

Obtain time using the time API in the C standard library:

```
time_t time(time_t *t)
```

Parameter	Description
t	Time data pointer
return	——
Current time value	

Examples of use are as follows:

```
time_t now;     /* Save the current time value obtained */
/* Obtain Time */
now = time(RT_NULL);
/* Printout time information */
rt_kprintf("%s\n", ctime(&now));
```

>Currently only one RTC device is allowed in the system and the name is "rtc".

1.4.2.13.3 Functional Configuration

1.4.2.13.3.1 Enable Soft RTC (Software Emulation RTC)

You can use the function of enabling RTC software emulation, which is ideal for products that do not require high time precision and have no hardware RTC. The configuration options of menuconfig are as follows:

```
RT-Thread Components →
    Device Drivers:
        -*- Using RTC device drivers                                    /* Use RTC device driver */
        [ ]   Using software simulation RTC device                      /* Use software simulation RTC device
      */
```

1.4.2.13.3.2 Enable NTP Time Automatic Synchronization

If the RT-Thread is connected to the Internet, you can enable automatic NTP time synchronization to synchronize local time periodically.
First open the NTP function in menuconfig as follows:

```
RT-Thread online packages →
    IoT - internet of things →
        netutils: Networking utilities for RT-Thread:
             [*]   Enable NTP(Network Time Protocol) client
```

After NTP is turned on, the RTC's automatic synchronization function will be automatically turned on, and the synchronization period and the delay time of the first synchronization can also be set:

```
RT-Thread Components →
    Device Drivers:
        -*- Using RTC device drivers                                    /* Use RTC device driver */
        [ ]   Using software simulation RTC device                      /* Use software simulation RTC device
      */
        [*]   Using NTP auto sync RTC time                              /* Automatically synchronize RTC time
      with NTP */
         (30)    NTP first sync delay time(second) for network connect /* The delay for performing NTP time
      synchronization for the first time. The purpose of the delay is to reserve a certain amount of time
      for the network connection and try to increase the success rate of the first NTP time synchronization.
      The default time is 30S */
         (3600)  NTP auto sync period(second)                           /* NTP The synchronization period is
      automatically synchronized in seconds, and the default period is one hour (ie 3600S). */
```

1.4.2.13.4 FinSH Command

Enter `date` to view the current time.

```
msh />date
Fri Feb 16 01:11:56 2018
msh />
```

Also use the `date` command, after the command, enter `year month date hour minute second` (between spaces, 24H system), and set the current time to 2018-02-16 01:15:30. The approximate effect is as follows:

```
msh />date 2018 02 16 01 15 30
```

```
msh />
```

1.4.2.13.5 RTC Device Usage Examples

For the specific usage of the RTC device, refer to the following example code. First, set the year, month, date, hour, minute and second information, and then delay the data for 3 seconds to get the current time information.

```
/*
 * Program listing: This is an RTC device usage routine
 * The routine exports the rtc_sample command to the control terminal
 * Command call formatrtc_sample
 * Program function: Set the date and time of the RTC device. After a delay, obtain the current time and
     print the display.
*/

#include <rtthread.h>
#include <rtdevice.h>

static int rtc_sample(int argc, char *argv[])
{
    rt_err_t ret = RT_EOK;
    time_t now;

    /* Set date */
    ret = set_date(2018, 12, 3);
    if (ret != RT_EOK)
    {
        rt_kprintf("set RTC date failed\n");
        return ret;
    }

    /* Set time */
    ret = set_time(11, 15, 50);
    if (ret != RT_EOK)
    {
        rt_kprintf("set RTC time failed\n");
        return ret;
    }

    /* Delay 3 seconds */
    rt_thread_mdelay(3000);

    /* Obtain Time */
    now = time(RT_NULL);
    rt_kprintf("%s\n", ctime(&now));

    return ret;
}
/* Export to the msh command list */
MSH_CMD_EXPORT(rtc_sample, rtc sample);
```

1.4.2.13.6 RTC DM helpers (`rtc_dm.h`)

1.4.2.13.6.1 `rtc_dm.h` API

Header: **`components/drivers/rtc/rtc_dm.h`**. Implementation: **`components/drivers/rtc/rtc_dm.c`**.
DM RTC chip drivers include this header at **`probe`** to set the registered device name and to convert alarm fields. Chip **`control()`** / **`RT_DEVICE_CTRL_RTC_`**∗ and application **`set_date`** / **`set_time`** are documented in RTC Device.

`rtc_dev_set_name`

```
int rtc_dev_set_name(struct rt_device *rtc_dev);
```

Assigns the RTC **`rt_device`** name before **`rt_device_register`**:

Condition	Name set
No device named **`"rtc"`** yet	**`rtc`**
`"rtc"` already exists	**`rtc1`**, **`rtc2`**, … (incrementing id)

Returns the result of **`rt_dm_dev_set_name`** (0 on success).
Note: **`set_date()`** / **`set_time()`** in **`dev_rtc.c`** only look up **`"rtc"`**, not **`rtc1`**.

rtc_wkalarm_to_timestamp

```
time_t rtc_wkalarm_to_timestamp(struct rt_rtc_wkalarm *alarm);
```

Builds a **time_t** for **today's calendar date** (from current **time()**) with hour/minute/second taken from **alarm** (tm_hour, tm_min, tm_sec). Other **rt_rtc_wkalarm** fields are ignored.
Uses **localtime_r** + **timegm**.

rtc_timestamp_to_wkalarm

```
void rtc_timestamp_to_wkalarm(time_t timestamp, struct rt_rtc_wkalarm *alarm);
```

Fills **alarm->tm_hour**, **tm_min**, **tm_sec** from **timestamp** (local time). Does not set **enable** or date fields.

See also

- RTC Device
- components/drivers/include/drivers/dev_rtc.h — **struct rt_rtc_wkalarm**, **RT_DEVICE_CTRL_RTC_***

1.4.2.13.7 RTC DM helpers (`rtc_dm.h`)

1.4.2.13.7.1 rtc_dm.h API

Header: **components/drivers/rtc/rtc_dm.h**. Implementation: **components/drivers/rtc/rtc_dm.c**.
DM RTC chip drivers include this header at **probe** to set the registered device name and to convert alarm fields. Chip **control()** / **RT_DEVICE_CTRL_RTC_*** and application **set_date** / **set_time** are documented in RTC Device.

rtc_dev_set_name

```
int rtc_dev_set_name(struct rt_device *rtc_dev);
```

Assigns the RTC **rt_device** name before **rt_device_register**:

Condition	Name set
No device named **"rtc"** yet	**rtc**
"rtc" already exists	**rtc1**, **rtc2**, … (incrementing id)

Returns the result of **rt_dm_dev_set_name** (0 on success).
Note: **set_date()** / **set_time()** in **dev_rtc.c** only look up **"rtc"**, not **rtc1**.

rtc_wkalarm_to_timestamp

```
time_t rtc_wkalarm_to_timestamp(struct rt_rtc_wkalarm *alarm);
```

Builds a **time_t** for **today's calendar date** (from current **time()**) with hour/minute/second taken from **alarm** (tm_hour, tm_min, tm_sec). Other **rt_rtc_wkalarm** fields are ignored.
Uses **localtime_r** + **timegm**.

rtc_timestamp_to_wkalarm

```
void rtc_timestamp_to_wkalarm(time_t timestamp, struct rt_rtc_wkalarm *alarm);
```

Fills **alarm->tm_hour**, **tm_min**, **tm_sec** from **timestamp** (local time). Does not set **enable** or date fields.

See also

- RTC Device
- components/drivers/include/drivers/dev_rtc.h — **struct rt_rtc_wkalarm**, **RT_DEVICE_CTRL_RTC_***

1.4.2.14 Clock Time Subsystem

1.4.2.14.1 Overview

The clock_time subsystem provides a unified, high-resolution time base and programmable event scheduling for RT-Thread. It decouples a monotonic counter (clock source) from deadline delivery (clock event), so platforms can mix hardware counters and timers while the kernel and libc see consistent behavior.

1.4.2.14.2 Architecture

Figure 1.107 Clock time architecture

1.4.2.14.2.1 Layering and Responsibilities

- Upper layers:
  - POSIX/libc time APIs (clock_gettime, nanosleep) use boottime and hrtimer APIs.
  - Soft RTC uses boottime as its monotonic time base for timekeeping.
  - Device drivers (input event timestamps, vDSO, PIC statistics) use boottime for timestamping.
- clock_time subsystem: core APIs, clock source/event devices, the hrtimer scheduler, boottime helpers, and the clock_timer adapter.
- Lower layers: BSP drivers provide hardware counters and timers, which are wrapped as clock_time devices or clock_timer devices.

1.4.2.14.2.2 Internal Components

- Core API (clock_time_core.c)
  - Registers clock_time devices, manages default source/event selection, and provides counter <-> nanosecond conversion with fixed-point scaling.
- Clock source device (rt_clock_time_device + RT_CLOCK_TIME_CAP_SOURCE)
  - Supplies a free-running counter and frequency for monotonic time reads.
- Clock event device (rt_clock_time_device + RT_CLOCK_TIME_CAP_EVENT)
  - Programs the next deadline and calls rt_clock_time_event_isr() on expiry.
- Clock hrtimer (clock_hrtimer.c)
  - Schedules high-resolution timeouts, programs the next event, and dispatches callbacks. Falls back to software timer when no hardware event is available.
- Boottime helpers (clock_boottime.c)
  - Converts the monotonic counter into timeval/timespec/seconds for upper layers.
- Clock timer adapter (clock_timer.c)
  - Exposes a unified hardware timer device (rt_clock_timer) and can register itself as a clock_time event device.
- Architecture sources (arch/∗ and clock_time_arm_arch.c)
  - Provide fast CPU counters or architectural timers and register them as the default clock source when available.

1.4.2.14.2.3 Data Flow

- Read path
  - Clock source counter -> scaled resolution -> nanoseconds -> boottime or clock_gettime.
- Timeout path
  - HRTimer queue -> next expiry -> set_timeout on event device -> event ISR -> hrtimer processing -> callbacks.

1.4.2.14.3 Configuration

Enable the subsystem in menuconfig:

```
RT-Thread Components ->
    Device Drivers ->
        [*] Clock time subsystem (RT_USING_CLOCK_TIME)
```

Optional settings:

- CLOCK_TIMER_FREQ (RISC-V): base counter frequency used by the clock source.
- RT_CLOCK_TIME_ARM_ARCH: enable ARM architected timer integration (DM/OFW).

1.4.2.14.4 BSP Integration Checklist

- Provide a clock source:
  - Register a rt_clock_time_device with CAP_SOURCE, or use the provided architecture source (AArch64/RISC-V) via rt_clock_time_source_init().
- Provide a clock event:
  - Register a rt_clock_time_device with CAP_EVENT and call rt_clock_time_event_isr() in its interrupt handler.
  - Or register a rt_clock_timer device; it can become the default event device automatically.
- Keep event ISRs short; heavy work should run in thread context if needed.

1.4.2.14.5 Detailed Documents

- Clock Time Core
- Clock HRTimer
- Clock Boottime Helpers
- CLOCK_TIMER Device

1.4.2.14.6 Clock Time Core

1.4.2.14.6.1 Overview

The clock_time core defines the device model and conversion helpers used by the subsystem. It manages the default clock source and clock event devices and provides fixed-point conversion between counters and nanoseconds.

1.4.2.14.6.2 Device Model

Clock Time Operations

```
struct rt_clock_time_ops
{
    rt_uint64_t (*get_freq)(struct rt_clock_time_device *dev);
    rt_uint64_t (*get_counter)(struct rt_clock_time_device *dev);
    rt_err_t    (*set_timeout)(struct rt_clock_time_device *dev, rt_uint64_t delta);
};
```

get_freq

- Purpose: return the counter frequency in Hz.
- Requirements:
  - Must be stable and match the source used by get_counter.
  - Should be non-zero; zero indicates the device is not ready.

get_counter

- Purpose: return a monotonically increasing counter value.
- Requirements:
  - Must not go backwards.
  - Wrap behavior is allowed but should be consistent with the width of the counter used by the device.

set_timeout

- Purpose: program the next timeout relative to the current counter value.
- Parameters: `delta` is in the same counter units returned by get_counter.
- Behavior:
  - If `delta` is 0 and the hardware supports it, the event can be canceled.
  - Return -RT_ENOSYS when the device does not support event programming.

Clock Time Device

```
struct rt_clock_time_device
{
    struct rt_device parent;
    const struct rt_clock_time_ops *ops;
    rt_uint64_t res_scale;
    rt_uint8_t caps; /* RT_CLOCK_TIME_CAP_SOURCE / RT_CLOCK_TIME_CAP_EVENT */
};
```

- res_scale provides extra precision in the conversion pipeline. If set to 0, RT_CLOCK_TIME_RESMUL is used by default.
- caps advertises whether the device can be used as a clock source, a clock event, or both.

Register and Select Devices

```
rt_err_t rt_clock_time_device_register(struct rt_clock_time_device *dev,
                                       const char *name,
                                       rt_uint8_t caps);
void rt_clock_time_set_default_source(struct rt_clock_time_device *dev);
void rt_clock_time_set_default_event(struct rt_clock_time_device *dev);
struct rt_clock_time_device *rt_clock_time_get_default_source(void);
struct rt_clock_time_device *rt_clock_time_get_default_event(void);
```

rt_clock_time_device_register

- Purpose: register a clock_time device and its capabilities.
- Parameters:
  - `dev`: device object with ops and res_scale initialized.
  - `name`: device name; if NULL, only capability registration is performed.
  - `caps`: RT_CLOCK_TIME_CAP_SOURCE and/or RT_CLOCK_TIME_CAP_EVENT.
- Behavior:

  - Registers the device under RT_Device_Class_Timer when `name` is provided.
  - If this is the first source, it becomes the default source.
  - If this is the first event, it becomes the default event.
- Return values:
  - RT_EOK on success.
  - Negative error code if device registration fails.

rt_clock_time_set_default_source / rt_clock_time_set_default_event

- Purpose: explicitly select the default source or event device.
- Behavior: overrides the auto-selected default.

rt_clock_time_get_default_source / rt_clock_time_get_default_event

- Purpose: query the current default devices.
- Behavior:
  - Source defaults to the internal tick-based device if none was registered.
  - Event returns NULL if no event device is available.

rt_clock_time_source_init

- Purpose: BSP hook to register an architecture counter or board source.
- Notes: this function is weak and may be overridden by BSP code.

1.4.2.14.6.3 Clock Time Query APIs

```
rt_uint64_t rt_clock_time_get_freq(void);
rt_uint64_t rt_clock_time_get_counter(void);
rt_uint64_t rt_clock_time_get_event_freq(void);
rt_uint64_t rt_clock_time_get_event_res_scaled(void);
```

rt_clock_time_get_freq

- Purpose: return the frequency of the default source in Hz.
- Return values:
  - Non-zero frequency on success.
  - 0 if no valid source is available.

rt_clock_time_get_counter

- Purpose: return the current counter from the default source.
- Return values:
  - Monotonic counter value on success.
  - 0 if no valid source is available.

rt_clock_time_get_event_freq

- Purpose: return the event device frequency in Hz.
- Behavior: if no event device exists, falls back to the default source.

rt_clock_time_get_event_res_scaled

- Purpose: return the scaled resolution for the event device.
- Behavior: if no event device exists, falls back to the default source.

1.4.2.14.6.4 Conversion Helpers

```
rt_uint64_t rt_clock_time_get_res_scaled(void);
rt_uint64_t rt_clock_time_counter_to_ns(rt_uint64_t cnt);
rt_uint64_t rt_clock_time_ns_to_counter(rt_uint64_t ns);
```

rt_clock_time_get_res_scaled

- Purpose: return the scaled resolution for the default source.
- Return values:
  - Non-zero scaled resolution when the source is ready.
  - 0 when the source is missing or frequency is invalid.

rt_clock_time_counter_to_ns

- Purpose: convert a counter value to nanoseconds based on the default source.
- Notes: returns 0 when resolution is unavailable.

rt_clock_time_ns_to_counter

- Purpose: convert nanoseconds to counter units for the default source.
- Notes: returns 0 when resolution is unavailable.

Internally, the core computes a scaled resolution:

```
res_scaled = (1e9 * res_scale) / freq
```

Nanoseconds are then derived using the scale factor RT_CLOCK_TIME_RESMUL to avoid floating-point math. This keeps precision stable even when freq is not a power of ten.

1.4.2.14.6.5 Event API

```
rt_err_t rt_clock_time_set_timeout(rt_uint64_t delta);
void rt_clock_time_event_isr(void);
```

rt_clock_time_set_timeout

- Purpose: program the next event using the default event device.
- Parameters: `delta` in event counter units (same as event get_counter).
- Return values:
  - RT_EOK on success.
  - -RT_ENOSYS if no event device exists or set_timeout is unsupported.

rt_clock_time_event_isr

- Purpose: drive the hrtimer scheduler when an event expires.
- Usage:
  - Call from the hardware timer ISR after acknowledging the interrupt.
  - This function processes expired hrtimers and programs the next event.

1.4.2.14.6.6 BSP Integration Example

Register a Clock Source

```
static rt_uint64_t demo_get_freq(struct rt_clock_time_device *dev)
{
    RT_UNUSED(dev);
    return 10000000ULL; /* 10 MHz */
}

static rt_uint64_t demo_get_counter(struct rt_clock_time_device *dev)
{
    RT_UNUSED(dev);
    return demo_hw_counter_read();
}

static const struct rt_clock_time_ops demo_src_ops =
{
    demo_get_freq,
    demo_get_counter,
    RT_NULL,
};

static struct rt_clock_time_device demo_src_dev =
{
    .ops = &demo_src_ops,
    .res_scale = RT_CLOCK_TIME_RESMUL,
};

void rt_clock_time_source_init(void)
{
    rt_clock_time_device_register(&demo_src_dev, "clock_time_demo_src",
                                  RT_CLOCK_TIME_CAP_SOURCE);
    rt_clock_time_set_default_source(&demo_src_dev);
}
```

Register a Clock Event

```
static rt_err_t demo_set_timeout(struct rt_clock_time_device *dev, rt_uint64_t delta)
{
    RT_UNUSED(dev);
    return demo_hw_timer_program(delta);
}

static const struct rt_clock_time_ops demo_evt_ops =
{
    demo_get_freq,
    demo_get_counter,
    demo_set_timeout,
};

static struct rt_clock_time_device demo_evt_dev =
{
    .ops = &demo_evt_ops,
    .res_scale = RT_CLOCK_TIME_RESMUL,
};

static void demo_timer_isr(void)
{
    demo_hw_timer_ack();
    rt_clock_time_event_isr();
}

void demo_register_event(void)
{
    rt_clock_time_device_register(&demo_evt_dev, "clock_time_demo_evt",
                                  RT_CLOCK_TIME_CAP_EVENT);
    rt_clock_time_set_default_event(&demo_evt_dev);
}
```

1.4.2.14.6.7 Notes

- Event callbacks may run in interrupt context when backed by a hardware event.
- If no event device is available, the hrtimer subsystem falls back to a software timer.

1.4.2.14.7 Clock HRTimer

1.4.2.14.7.1 Overview

The clock hrtimer layer provides high-resolution timeout scheduling on top of clock_time. It keeps a sorted timeout list, programs the next deadline, and runs callbacks when the deadline expires.

1.4.2.14.7.2 Key Concepts

- Time base: delay counts are expressed in the clock source counter units.
- Event programming: the next expiry is converted to event units and sent to the clock event device.
- Fallback: if no hardware event exists, a software timer triggers processing.

1.4.2.14.7.3 API

```
void rt_clock_hrtimer_init(rt_clock_hrtimer_t timer,
                           const char *name,
                           rt_uint8_t flag,
                           void (*timeout)(void *parameter),
                           void *parameter);
rt_err_t rt_clock_hrtimer_start(rt_clock_hrtimer_t timer, unsigned long cnt);
rt_err_t rt_clock_hrtimer_stop(rt_clock_hrtimer_t timer);
rt_err_t rt_clock_hrtimer_control(rt_clock_hrtimer_t timer, int cmd, void *arg);
rt_err_t rt_clock_hrtimer_detach(rt_clock_hrtimer_t timer);

void rt_clock_hrtimer_delay_init(struct rt_clock_hrtimer *timer);
void rt_clock_hrtimer_delay_detach(struct rt_clock_hrtimer *timer);
rt_err_t rt_clock_hrtimer_sleep(struct rt_clock_hrtimer *timer, unsigned long cnt);
rt_err_t rt_clock_hrtimer_ndelay(struct rt_clock_hrtimer *timer, unsigned long ns);
rt_err_t rt_clock_hrtimer_udelay(struct rt_clock_hrtimer *timer, unsigned long us);
rt_err_t rt_clock_hrtimer_mdelay(struct rt_clock_hrtimer *timer, unsigned long ms);
```

Flags reuse RT_TIMER_FLAG_* definitions (one-shot/periodic/hard timer). `cnt` is a counter delta based on the default clock source.

rt_clock_hrtimer_init

```
void rt_clock_hrtimer_init(rt_clock_hrtimer_t timer,
                           const char *name,
                           rt_uint8_t flag,
                           void (*timeout)(void *parameter),
                           void *parameter);
```

- Purpose: initialize a high-resolution timer object.
- Parameters:
  - `timer`: the hrtimer object to initialize.
  - `name`: timer name (truncated to RT_NAME_MAX-1).
  - `flag`: RT_TIMER_FLAG_* (one-shot/periodic/hard timer).
  - `timeout`: callback function on expiry.
  - `parameter`: user parameter passed to the callback.
- Behavior:
  - Clears internal state, initializes list node and completion.
  - Does not start the timer; call `rt_clock_hrtimer_start()` to arm it.
- Context: thread context.

rt_clock_hrtimer_start

```
rt_err_t rt_clock_hrtimer_start(rt_clock_hrtimer_t timer, unsigned long cnt);
```

- Purpose: arm a timer to expire after `cnt` counter ticks.
- Parameters:

  - `cnt`: relative delay in clock source counter units.

- Return values:
  - RT_EOK: timer armed.
  - -RT_ERROR: timer already active or invalid `cnt`.

- Notes:
  - `cnt` must be less than half of the maximum counter range to avoid wrap ambiguity.
  - Starting a timer programs the next clock event if this timer becomes the earliest deadline.

rt_clock_hrtimer_stop

```
rt_err_t rt_clock_hrtimer_stop(rt_clock_hrtimer_t timer);
```

- Purpose: cancel an active timer.
- Return values:
  - RT_EOK: timer stopped.
  - -RT_ERROR: timer was not active.
- Notes: stopping a timer may reprogram the next event.

rt_clock_hrtimer_control

```
rt_err_t rt_clock_hrtimer_control(rt_clock_hrtimer_t timer, int cmd, void *arg);
```

- Purpose: query or modify a timer after initialization.
- Common commands (same as RT-Thread timers):
  - RT_TIMER_CTRL_GET_TIME: get `delay_cnt` into `*(unsigned long *)arg`.
  - RT_TIMER_CTRL_SET_TIME: set `delay_cnt` from `*(unsigned long *)arg`.
  - RT_TIMER_CTRL_SET_ONESHOT / RT_TIMER_CTRL_SET_PERIODIC: set mode.
  - RT_TIMER_CTRL_GET_STATE: return activated/deactivated state.
  - RT_TIMER_CTRL_GET_REMAIN_TIME: get absolute timeout counter.
  - RT_TIMER_CTRL_GET_FUNC / RT_TIMER_CTRL_SET_FUNC: get/set callback.
  - RT_TIMER_CTRL_GET_PARM / RT_TIMER_CTRL_SET_PARM: get/set parameter.
- Notes:
  - Changing time or mode while active updates internal deadline but does not implicitly restart a stopped timer.

rt_clock_hrtimer_detach

```
rt_err_t rt_clock_hrtimer_detach(rt_clock_hrtimer_t timer);
```

- Purpose: detach a timer and wake any waiters.
- Behavior:
  - Marks the timer inactive and wakes `rt_clock_hrtimer_sleep()` waiters with an error code.
  - Removes the timer from the list if needed.
- Use case: cleanup when a timer is no longer valid.

rt_clock_hrtimer_delay_init

```
void rt_clock_hrtimer_delay_init(struct rt_clock_hrtimer *timer);
```

- Purpose: initialize a one-shot hrtimer for blocking delays.
- Behavior: sets a built-in timeout callback that signals a completion.

rt_clock_hrtimer_delay_detach

```
void rt_clock_hrtimer_delay_detach(struct rt_clock_hrtimer *timer);
```

- Purpose: detach a delay timer created by `rt_clock_hrtimer_delay_init()`.
- Notes: safe to call even if the timer has already expired.

rt_clock_hrtimer_sleep

```
rt_err_t rt_clock_hrtimer_sleep(struct rt_clock_hrtimer *timer, unsigned long cnt);
```

- Purpose: block the current thread until the delay expires.
- Parameters: `timer` must be initialized (typically by delay_init).
- Return values:
  - RT_EOK: timeout expired normally.
  - -RT_EINTR: interrupted by signal or detach.
  - -RT_EINVAL: `cnt` is zero.
- Context: thread context only.

rt_clock_hrtimer_ndelay / udelay / mdelay

```
rt_err_t rt_clock_hrtimer_ndelay(struct rt_clock_hrtimer *timer, unsigned long ns);
rt_err_t rt_clock_hrtimer_udelay(struct rt_clock_hrtimer *timer, unsigned long us);
rt_err_t rt_clock_hrtimer_mdelay(struct rt_clock_hrtimer *timer, unsigned long ms);
```

- Purpose: sleep for a duration expressed in nanoseconds, microseconds, or milliseconds.
- Behavior:
  - Converts the time into counter units using the current clock_time resolution and calls `rt_clock_↩hrtimer_sleep()`.
- Notes:
  - The actual delay is limited by the clock source resolution and event programming granularity.

1.4.2.14.7.4 Typical Flow

1. Initialize a hrtimer with a callback.
2. Convert a time interval to counter units, or use the ndelay/udelay/mdelay helpers.
3. Start the timer; the subsystem will program the next event.
4. When the event ISR fires, rt_clock_time_event_isr() runs hrtimer processing and dispatches callbacks.

1.4.2.14.7.5 Example: One-shot Timeout

```
#include <drivers/clock_time.h>

static struct rt_clock_hrtimer demo_timer;

static void demo_timeout(void *parameter)
{
    RT_UNUSED(parameter);
    rt_kprintf("hrtimer timeout\n");
}

static void demo_hrtimer_start(void)
{
    rt_uint64_t ns = 5ULL * 1000 * 1000; /* 5 ms */
    unsigned long cnt = (unsigned long)rt_clock_time_ns_to_counter(ns);

    rt_clock_hrtimer_init(&demo_timer, "demo", RT_TIMER_FLAG_ONE_SHOT,
                          demo_timeout, RT_NULL);
    rt_clock_hrtimer_start(&demo_timer, cnt);
}
```

1.4.2.14.7.6 Example: Sleep Helper

```
static void demo_hrtimer_sleep(void)
{
    struct rt_clock_hrtimer timer;

    rt_clock_hrtimer_delay_init(&timer);
    rt_clock_hrtimer_mdelay(&timer, 10);
    rt_clock_hrtimer_delay_detach(&timer);
}
```

1.4.2.14.7.7 Notes

- Callbacks may run in interrupt context when using a hardware event device.
- rt_clock_hrtimer_sleep() waits on a completion and may return -RT_EINTR if interrupted.

1.4.2.14.8 Clock Boottime Helpers

1.4.2.14.8.1 Overview

Boottime helpers convert the clock_time monotonic counter into standard time formats. The resulting values represent time since boot and do not depend on RTC or wall-clock settings.

1.4.2.14.8.2 API

```
rt_err_t rt_clock_boottime_get_us(struct timeval *tv);
rt_err_t rt_clock_boottime_get_s(time_t *t);
rt_err_t rt_clock_boottime_get_ns(struct timespec *ts);
```

All functions return RT_EOK on success or -RT_ERROR if the clock source is unavailable. The returned values are monotonic and suitable for measuring elapsed time.

rt_clock_boottime_get_us

```
rt_err_t rt_clock_boottime_get_us(struct timeval *tv);
```

- Purpose: get time since boot as seconds + microseconds.
- Parameters: `tv` must be a valid pointer to `struct timeval`.
- Return values:
  - RT_EOK: data written to `tv`.
  - -RT_ERROR: no valid clock_time source or conversion failed.
- Notes:
  - `tv_usec` is derived from the clock_time resolution and may not be exact microseconds if the underlying counter does not align to 1 us.

rt_clock_boottime_get_s

```
rt_err_t rt_clock_boottime_get_s(time_t *t);
```

- Purpose: get time since boot in whole seconds.
- Parameters: `t` must be a valid pointer to `time_t`.
- Return values:
  - RT_EOK: `*t` updated.
  - -RT_ERROR: no valid clock_time source or conversion failed.
- Notes:
  - Sub-second precision is discarded; use `rt_clock_boottime_get_us()` or `rt_clock_boottime_get_ns()` if needed.

rt_clock_boottime_get_ns

```
rt_err_t rt_clock_boottime_get_ns(struct timespec *ts);
```

- Purpose: get time since boot as seconds + nanoseconds.
- Parameters: `ts` must be a valid pointer to `struct timespec`.
- Return values:
  - RT_EOK: data written to `ts`.
  - -RT_ERROR: no valid clock_time source or conversion failed.
- Notes:
  - `tv_nsec` reflects the clock_time resolution; it may not be 1 ns granularity if the counter frequency is lower.

1.4.2.14.8.3 Example

```
#include <drivers/clock_time.h>

static void demo_boottime(void)
{
    struct timespec ts;

    if (rt_clock_boottime_get_ns(&ts) == RT_EOK)
    {
        rt_kprintf("boottime: %ld.%09ld\n", (long)ts.tv_sec, ts.tv_nsec);
    }
}
```

1.4.2.14.8.4 Notes

- The boottime helpers are used by the soft RTC implementation to build a stable base time.
- If no clock_time source is registered, the subsystem falls back to the tick counter and the resolution matches RT_TICK_PER_SECOND.

1.4.2.14.9 CLOCK_TIMER Device

Note: CLOCK_TIMER is the unified hardware timer device in the `clock_time` subsystem.

1.4.2.14.9.1 Introduction to the Timer

Hardware timers generally have two modes of operation, timer mode and counter mode. No matter which mode is operated, it works by counting the pulse signal counted by the internal counter module. Here are some important concepts of timers.
Timer mode: Counts the internal pulse. Timers are often used as timing clocks for timing detection, timing response, and timing control.
Counter mode: The counter can count up or down. The maximum count value of a 16-bit counter is 65535, and the maximum value of a 32-bit counter is 4 294 967 295.
∗∗Counting frequency∗∗Since the input frequency is usually fixed, the time it takes for the counter to reach its desired count number can be calculated from just the given frequency - `time = count value / count frequency`. For example, if the counting frequency is 1 MHz, the counter counts once every 1 / 1000000 seconds. That is, every 1 microsecond, the counter is incremented by one (or subtracted by one), at this time, the maximum timing capability of the 16-bit counter is 65535 microseconds, or 65.535 milliseconds.

1.4.2.14.9.2 Relationship to clock_time

CLOCK_TIMER devices can be used directly by applications through the RT-Thread device API, and can also act as a clock_time event device. When a clock_timer is registered with `rt_clock_timer_register()` and selected as the default event, its ISR calls `rt_clock_time_event_isr()` internally, which drives the high-resolution timer scheduler.

1.4.2.14.9.3 Access Hardware Timer Device

The application accesses the hardware timer device through the I/O device management interface provided by RT-Thread. The related interfaces are as follows:

Function	Description
rt_device_find()	to look up the timer device
rt_device_open()	to open the timer device in read-write mode
rt_device_set_rx_indicate()	to set the timeout callback function
rt_device_control()	to control the timer device, you can set the timing mode (single time /↩ cycle),counting frequency, or stop the timer
rt_device_write()	to set the timeout value of the timer. The timer then starts
rt_device_read()	to get the current value of the timer
rt_device_close()	to turn off the timer device.

Find Timer Device

The application obtains the device handle based on the hardware timer device name, and thus can operate the hardware timer device. The device function is as follows:

```
rt_device_t rt_device_find(const char* name);
```

Parameter	Description
name	hardware timer device name
return	——
timer device handle	will return to the corresponding device handle if the corresponding device is found
RT_NULL	No device found

In general, the hardware timer device name registered to the system is timer0, timer1, etc. The usage examples are as follows:

```
#define CLOCK_TIMER_DEV_NAME   "timer0"     /* timer name */
rt_device_t hw_dev;                    /* timer device handle */
/* find timer device */
```

```
hw_dev = rt_device_find(CLOCK_TIMER_DEV_NAME);
```

Details:

- The device name is assigned by the BSP when calling `rt_clock_timer_register()`.
- If multiple timers are registered, each is exposed as a separate device.

Open Timer Device

With the device handle, the application can open the device. When the device is open, it will detect whether the device has been initialized. If it is not initialized, it will call the initialization interface to initialize the device by default. Open the device with the following function:

```
rt_err_t rt_device_open(rt_device_t dev, rt_uint16_t oflags);
```

Parameter	Description
dev	hardware timer device handle
oflags	device open mode, is generally opened in read and write mode, which is to take the value↩ RT_DEVICE_OFLAG_RDWR
return	——
RT_EOK	device opened successfully
other error code	device fail to open

An example of use is as follows:

```
#define CLOCK_TIMER_DEV_NAME   "timer0"     /* timer name */
rt_device_t hw_dev;                     /* timer device handle */
/* find timer device */
hw_dev = rt_device_find(CLOCK_TIMER_DEV_NAME);
/* to open the timer device in read-write mode */
rt_device_open(hw_dev, RT_DEVICE_OFLAG_RDWR);
```

Details:

- Opening the device ensures the driver is initialized and the working frequency is applied.
- Call open before using control/read/write APIs; otherwise behavior depends on the BSP driver and may fail with -RT_ENOSYS or other errors.

Set the Timeout Callback Function

Set the timer timeout callback function with the following function - this is the function that will be called when the timer reaches its set count value:

```
rt_err_t rt_device_set_rx_indicate(rt_device_t dev, rt_err_t (*rx_ind)(rt_device_t dev, rt_size_t size))
```

Parameter	Description
dev	device handle
rx_ind	timeout callback function, provided by the caller
return	——
RT_EOK	success

An example of use is as follows

```
#define CLOCK_TIMER_DEV_NAME   "timer0"     /* timer name */
rt_device_t hw_dev;                     /* timer device handle */

/* timer timeout callback function */
static rt_err_t timeout_cb(rt_device_t dev, rt_size_t size)
{
    rt_kprintf("this is clock_timer timeout callback function!\n");
    rt_kprintf("tick is :%d !\n", rt_tick_get());

    return 0;
```

```
}

static int clock_timer_sample(int argc, char *argv[])
{
    /* find timer device */
    hw_dev = rt_device_find(CLOCK_TIMER_DEV_NAME);
    /* open the device in read and write mode */
    rt_device_open(hw_dev, RT_DEVICE_OFLAG_RDWR);
    /* set the timeout callback function */
    rt_device_set_rx_indicate(hw_dev, timeout_cb);
}
```

Details:

- The callback runs in the timer interrupt context, so it must be fast and non-blocking.
- `size` is the size of the timeout data (sizeof(rt_clock_timerval_t)) passed by the driver; most users can ignore it.

Control the Timer Device

By sending control words, the application can configure the hardware timer device with the following function:

```
rt_err_t rt_device_control(rt_device_t dev, rt_uint8_t cmd, void* arg);
```

Parameter	Description
dev	device handle
cmd	command control word
arg	controlled parameter
return	——
RT_EOK	function executed successfully
-RT_ENOSYS	execution faileddev is null
other error code	execution failed

The command control words available for the hardware timer device are as follows

Control word	Description
CLOCK_TIMER_CTRL_FREQ_SET	set the counting frequency
CLOCK_TIMER_CTRL_STOP	stop the timer
CLOCK_TIMER_CTRL_INFO_GET	get timer feature information
CLOCK_TIMER_CTRL_MODE_SET	set timer mode

Get the timer parameter argument, which is a pointer to the structure struct rt_clock_timer_info, to save the obtained information.
>Setting frequency is valid only when the timer hardware and included driver set the counting frequency. Generally, the default frequency of the driving setting can be used.
When setting the timer mode, the parameter argument can take the following values

```
CLOCK_TIMER_MODE_ONESHOT    /* Single timing */
CLOCK_TIMER_MODE_PERIOD     /* Periodic timing */
```

An example of using the timer count frequency and timing mode is as follows:

```
#define CLOCK_TIMER_DEV_NAME   "timer0"     /* timer name */
rt_device_t hw_dev;                     /* timer device handle */
rt_clock_timer_mode_t mode;                 /* timer mode */
rt_uint32_t freq = 10000;               /* counting frequency */

/* Timer timeout callback function */
static rt_err_t timeout_cb(rt_device_t dev, rt_size_t size)
{
    rt_kprintf("this is clock_timer timeout callback function!\n");
    rt_kprintf("tick is :%d !\n", rt_tick_get());

    return 0;
```

```
}

static int clock_timer_sample(int argc, char *argv[])
{
    /* find timer device */
    hw_dev = rt_device_find(CLOCK_TIMER_DEV_NAME);
    /* open the device in read and write mode */
    rt_device_open(hw_dev, RT_DEVICE_OFLAG_RDWR);
    /* Set the timeout callback function */
    rt_device_set_rx_indicate(hw_dev, timeout_cb);

    /* Set the counting frequency (1Mhz or the supported minimum counting frequency by default) */
    rt_device_control(hw_dev, CLOCK_TIMER_CTRL_FREQ_SET, &freq);
    /* Set the mode to periodic timer */
    mode = CLOCK_TIMER_MODE_PERIOD;
    rt_device_control(hw_dev, CLOCK_TIMER_CTRL_MODE_SET, &mode);
}
```

Details:

- CLOCK_TIMER_CTRL_FREQ_SET:
  - `arg` points to an integer frequency in Hz.
  - The driver checks against `minfreq` and `maxfreq` from rt_clock_timer_info.
  - On success, the new frequency is used for subsequent timeouts.
- CLOCK_TIMER_CTRL_STOP:
  - Stops the current timer cycle. It does not close the device.
- CLOCK_TIMER_CTRL_INFO_GET:
  - `arg` points to `struct rt_clock_timer_info`.
  - Returns hardware limits (min/max frequency, max count, count direction).
- CLOCK_TIMER_CTRL_MODE_SET:
  - `arg` points to `rt_clock_timer_mode_t`.
  - ONESHOT fires once, PERIOD repeats using the calculated reload.

Set the Timer Timeout Value

The timer timeout value can be set by the following function

```
rt_size_t rt_device_write(rt_device_t dev, rt_off_t pos, const void* buffer, rt_size_t size);
```

Parameter	Description
dev	device handle
pos	write data offset, unused now, can set 0 value
buffer	pointer to the timer timeout structure
size	timeout structure size
return	——
The actual size of the written data	
0	fail

The prototype of the timeout structure is shown below :

```
typedef struct rt_clock_timerval
{
    rt_int32_t sec;      /* second */
    rt_int32_t usec;     /* microsecond */
} rt_clock_timerval_t;
```

An example of using the timer timeout value is as follows:

```
#define CLOCK_TIMER_DEV_NAME   "timer0"     /* timer name */
rt_device_t hw_dev;                    /* timer device handle */
rt_clock_timer_mode_t mode;                /* timer mode */
```

```
rt_clock_timerval_t timeout_s;              /* Timer timeout value */

/* Timer timeout callback function */
static rt_err_t timeout_cb(rt_device_t dev, rt_size_t size)
{
    rt_kprintf("this is clock_timer timeout callback function!\n");
    rt_kprintf("tick is :%d !\n", rt_tick_get());

    return 0;
}

static int clock_timer_sample(int argc, char *argv[])
{
    /* find timer device */
    hw_dev = rt_device_find(CLOCK_TIMER_DEV_NAME);
    /* open the device in read-write mode */
    rt_device_open(hw_dev, RT_DEVICE_OFLAG_RDWR);
    /* set the timeout callback function */
    rt_device_set_rx_indicate(hw_dev, timeout_cb);
    /* set the mode as periodic timer */
    mode = CLOCK_TIMER_MODE_PERIOD;
    rt_device_control(hw_dev, CLOCK_TIMER_CTRL_MODE_SET, &mode);

    /* Set the timer timeout value to 5s and start the timer */
    timeout_s.sec = 5;      /* second */
    timeout_s.usec = 0;     /* microsecond */
    rt_device_write(hw_dev, 0, &timeout_s, sizeof(timeout_s));
}
```

Details:

- `size` must match `sizeof(rt_clock_timerval_t)`; otherwise the write fails.
- In periodic mode, the driver may split a long timeout into multiple cycles based on the hardware `maxcnt` and current frequency.
- For one-shot mode, if the timeout fits in a single cycle, the timer stops automatically after expiry.

Obtain the Current Value of the Timer

The current value of the timer can be obtained by the following function

```
rt_size_t rt_device_read(rt_device_t dev, rt_off_t pos, void* buffer, rt_size_t size);
```

Parameter	Description
dev	timer device handle
pos	write data offset, unused now , can set 0 value
buffer	output parameter, a pointer point to the timeout structure
size	timeout structure size
return	——
Timeout structure size	success
0	fail

An example of use is shown below

```
rt_clock_timerval_t timeout_s;      /* Used to save the time the timer has elapsed */
/* Read the elapsed time of the timer  */
rt_device_read(hw_dev, 0, &timeout_s, sizeof(timeout_s));
```

Details:

- The returned value represents elapsed time since the timer started.
- In periodic mode, the driver accounts for completed cycles plus the current counter value.
- In one-shot mode, the elapsed time is limited to the active cycle.

Close the Timer Device

The timer device can be closed with the following function:

```
rt_err_t rt_device_close(rt_device_t dev);
```

Parameter	Description
dev	timer device handle
return	——
RT_EOK	close device successfully
-RT_ERROR	the device has been completely shut down and cannot be closed repeatedly
other error code	fail to close the device

When a timer device has been used and is not necessary anymore, it should be closed, otherwise the device will remain in an open status.
An example of use is shown below

```
#define CLOCK_TIMER_DEV_NAME   "timer0"     /* timer name */
rt_device_t hw_dev;                    /* timer device handle */
/* find timer device */
hw_dev = rt_device_find(CLOCK_TIMER_DEV_NAME);
... ...
rt_device_close(hw_dev);
```

Details:

- Closing the device may stop the timer and deinitialize hardware resources.
- Reopen the device before using it again.

>Timing errors may occur. Assume that the counter has a maximum value of 0xFFFF, a counting frequency of 1Mhz, and a timing time of 1 second and 1 microsecond. Since the timer can only count up to 65535us at a time, the timing requirement for 1000001us can be completed 20 times at 50000us, and the calculation error will be 1us.

1.4.2.14.9.4 Driver Integration (BSP/Driver Authors)

To provide a hardware timer as a clock_timer device, implement `rt_clock_timer_ops`, fill `rt_clock_↩timer_info`, and register it with `rt_clock_timer_register()`. The ISR should call `rt_clock_↩timer_isr()` to handle overflow, callback dispatch, and (if selected) clock_time event notification.

```
static const struct rt_clock_timer_ops timer_ops =
{
    .init      = drv_timer_init,
    .start     = drv_timer_start,
    .stop      = drv_timer_stop,
    .count_get = drv_timer_count_get,
    .control   = drv_timer_control,
};

static const struct rt_clock_timer_info timer_info =
{
    .maxfreq = 1000000,
    .minfreq = 1000,
    .maxcnt  = 0xFFFFFFFF,
    .cntmode = CLOCK_TIMER_CNTMODE_UP,
};

static rt_clock_timer_t hw_timer =
{
    .ops  = &timer_ops,
    .info = &timer_info,
};

void drv_timer_register(void)
{
    rt_clock_timer_register(&hw_timer, "timer0", RT_NULL);
}

void drv_timer_isr(void)
{
    rt_clock_timer_isr(&hw_timer);
}
```

If the timer supports one-shot start, it can also serve as the default clock_time event device (used by hrtimer).
Details:

- rt_clock_timer_register():
  - Registers the device into the RT-Thread device framework as a timer class.

  - If this is the first registered timer with a valid start op, it becomes the clock_time event device automatically.
  - The default event is opened and initialized if not already active.

- rt_clock_timer_isr():
  - Maintains internal overflow/cycle accounting.
  - Triggers the user callback via `rx_indicate`.
  - If the timer is the clock_time event owner, it calls `rt_clock_time_event_isr()` to drive hrtimer.

1.4.2.14.9.5 Hardware Timer Device Usage Example

The specific use of the hardware timer device can refer to the following sample code. The main steps of the sample code are as follows:

1. First find the device handle based on the timer device name "timer0".
2. Open the device "timer0" in read-write mode.
3. Set the timer timeout callback function.
4. Set the timer mode to periodic timer and set the timeout period to 5 seconds. At this time, the timer starts.
5. Read the timer after 3500ms delay, the read value will be displayed in seconds and microseconds.

```
 /*
  * Program listing: This is a clock_timer device usage routine
  * The routine exports the clock_timer_sample command to the control terminal
  * Command call format: clock_timer_sample
  * Program function: The hardware timer timeout callback function periodically prints the current tick
      value, and the difference between the two tick values is converted to the time equivalent to the
      timing time value.
  */

#include <rtthread.h>
#include <rtdevice.h>

#define CLOCK_TIMER_DEV_NAME   "timer0"     /* timer name */

/* Timer timeout callback function */
static rt_err_t timeout_cb(rt_device_t dev, rt_size_t size)
{
    rt_kprintf("this is clock_timer timeout callback function!\n");
    rt_kprintf("tick is :%d !\n", rt_tick_get());

    return 0;
}

static int clock_timer_sample(int argc, char *argv[])
{
    rt_err_t ret = RT_EOK;
    rt_clock_timerval_t timeout_s;      /* timer timeout value */
    rt_device_t hw_dev = RT_NULL;   /* timer device value */
    rt_clock_timer_mode_t mode;         /* timer mode */

    /* find timer device */
    hw_dev = rt_device_find(CLOCK_TIMER_DEV_NAME);
    if (hw_dev == RT_NULL)
    {
        rt_kprintf("clock_timer sample run failed! can't find %s device!\n", CLOCK_TIMER_DEV_NAME);
        return -RT_ERROR;
    }

    /* Open the device in read-write mode */
    ret = rt_device_open(hw_dev, RT_DEVICE_OFLAG_RDWR);
    if (ret != RT_EOK)
    {
        rt_kprintf("open %s device failed!\n", CLOCK_TIMER_DEV_NAME);
        return ret;
    }

    /* set timeout callback function */
    rt_device_set_rx_indicate(hw_dev, timeout_cb);

    /* Setting mode is periodic timer */
    mode = CLOCK_TIMER_MODE_PERIOD;
    ret = rt_device_control(hw_dev, CLOCK_TIMER_CTRL_MODE_SET, &mode);
```

```
    if (ret != RT_EOK)
    {
        rt_kprintf("set mode failed! ret is :%d\n", ret);
        return ret;
    }

    /* Set the timer timeout value to 5s and start the timer. */
    timeout_s.sec = 5;      /* second */
    timeout_s.usec = 0;     /* microsecond */

    if (rt_device_write(hw_dev, 0, &timeout_s, sizeof(timeout_s)) != sizeof(timeout_s))
    {
        rt_kprintf("set timeout value failed\n");
        return -RT_ERROR;
    }

    /* delay 3500ms */
    rt_thread_mdelay(3500);

    /* read the current value of timer */
    rt_device_read(hw_dev, 0, &timeout_s, sizeof(timeout_s));
    rt_kprintf("Read: Sec = %d, Usec = %d\n", timeout_s.sec, timeout_s.usec);

    return ret;
}
/* Export to the msh command list */
MSH_CMD_EXPORT(clock_timer_sample, clock_timer sample);
```

1.4.2.15 HWTIMER Device

1.4.2.15.1 Introduction to the Timer

1.4.2.15.1.1 Driver / BSP quick notes

Topic	Guidance
Prescaler reload	Some timers glitch if prescaler changes while counter running—stop, reload, restart.
Cascade / 32-bit	Chaining 16-bit timers for long timeouts—handle overflow ISR race vs application read.
Power	Timer clock domain may gate in sleep—wake source setup must re-enable bus clock.

Hardware timers generally have two modes of operation, timer mode and counter mode. No matter which mode is operated, it works by counting the pulse signal counted by the internal counter module. Here are some important concepts of timers.
Timer mode: Counts the internal pulse. Timers are often used as timing clocks for timing detection, timing response, and timing control.
Counter mode: The counter can count up or down. The maximum count value of a 16-bit counter is 65535, and the maximum value of a 32-bit counter is 4 294 967 295.
Counting frequency Since the input frequency is usually fixed, the time it takes for the counter to reach its desired count number can be calculated from just the given frequency - `time = count value / count frequency`. For example, if the counting frequency is 1 MHz, the counter counts once every 1 / 1000000 seconds. That is, every 1 microsecond, the counter is incremented by one (or subtracted by one), at this time, the maximum timing capability of the 16-bit counter is 65535 microseconds, or 65.535 milliseconds.

1.4.2.15.2 Access Hardware Timer Device

The application accesses the hardware timer device through the I/O device management interface provided by RT-Thread. The related interfaces are as follows:

Function	Description
rt_device_find()	to look up the timer device
rt_device_open()	to open the timer device in read-write mode
rt_device_set_rx_indicate()	to set the timeout callback function

Function	Description
rt_device_control()	to control the timer device, you can set the timing mode (single time /↩ cycle),counting frequency, or stop the timer
rt_device_write()	to set the timeout value of the timer. The timer then starts
rt_device_read()	to get the current value of the timer
rt_device_close()	to turn off the timer device.

1.4.2.15.2.1 Find Timer Device

The application obtains the device handle based on the hardware timer device name, and thus can operate the hardware timer device. The device function is as follows:

```
rt_device_t rt_device_find(const char* name);
```

Parameter	Description
name	hardware timer device name
return	——
timer device handle	will return to the corresponding device handle if the corresponding device is found
RT_NULL	No device found

In general, the hardware timer device name registered to the system is timer0, timer1, etc. The usage examples are as follows:

```
#define HWTIMER_DEV_NAME   "timer0"     /* timer name */
rt_device_t hw_dev;                     /* timer device handle */
/* find timer device */
hw_dev = rt_device_find(HWTIMER_DEV_NAME);
```

1.4.2.15.2.2 Open Timer Device

With the device handle, the application can open the device. When the device is open, it will detect whether the device has been initialized. If it is not initialized, it will call the initialization interface to initialize the device by default. Open the device with the following function:

```
rt_err_t rt_device_open(rt_device_t dev, rt_uint16_t oflags);
```

Parameter	Description
dev	hardware timer device handle
oflags	device open mode, is generally opened in read and write mode, which is to take the value↩ RT_DEVICE_OFLAG_RDWR
return	——
RT_EOK	device opened successfully
other error code	device fail to open

An example of use is as follows:

```
#define HWTIMER_DEV_NAME   "timer0"     /* timer name */
rt_device_t hw_dev;                     /* timer device handle */
/* find timer device */
hw_dev = rt_device_find(HWTIMER_DEV_NAME);
/* to open the timer device in read-write mode */
rt_device_open(hw_dev, RT_DEVICE_OFLAG_RDWR);
```

1.4.2.15.2.3 Set the Timeout Callback Function

Set the timer timeout callback function with the following function - this is the function that will be called when the timer reaches its set count value:

```
rt_err_t rt_device_set_rx_indicate(rt_device_t dev, rt_err_t (*rx_ind)(rt_device_t dev, rt_size_t size))
```

Parameter	Description
dev	device handle
rx_ind	timeout callback function, provided by the caller
return	——
RT_EOK	success

An example of use is as follows

```
#define HWTIMER_DEV_NAME   "timer0"     /* timer name */
rt_device_t hw_dev;                     /* timer device handle */

/* timer timeout callback function */
static rt_err_t timeout_cb(rt_device_t dev, rt_size_t size)
{
    rt_kprintf("this is hwtimer timeout callback fucntion!\n");
    rt_kprintf("tick is :%d !\n", rt_tick_get());

    return 0;
}

static int hwtimer_sample(int argc, char *argv[])
{
    /* find timer device */
    hw_dev = rt_device_find(HWTIMER_DEV_NAME);
    /* open the device in read and write mode */
    rt_device_open(hw_dev, RT_DEVICE_OFLAG_RDWR);
    /* set the timeout callback function */
    rt_device_set_rx_indicate(hw_dev, timeout_cb);
}
```

1.4.2.15.2.4 Control the Timer Device

By sending control words, the application can configure the hardware timer device with the following function:

```
rt_err_t rt_device_control(rt_device_t dev, rt_uint8_t cmd, void* arg);
```

Parameter	Description
dev	device handle
cmd	command control word
arg	controlled parameter
return	——
RT_EOK	function executed successfully
-RT_ENOSYS	execution faileddev is null
other error code	execution failed

The command control words available for the hardware timer device are as follows

Control word	Description
HWTIMER_CTRL_FREQ_SET	set the counting frequency
HWTIMER_CTRL_STOP	stop the timer
HWTIMER_CTRL_INFO_GET	get timer feature information
HWTIMER_CTRL_MODE_SET	set timer mode

Get the timer parameter argument, which is a pointer to the structure struct rt_hwtimer_info, to save the obtained information.
>Setting frequency is valid only when the timer hardware and included driver set the counting frequency. Generally, the default frequency of the driving setting can be used.
When setting the timer mode, the parameter argument can take the following values

```
HWTIMER_MODE_ONESHOT     /* Single timing */
```

```
HWTIMER_MODE_PERIOD     /* Periodic timing */
```

An example of using the timer count frequency and timing mode is as follows:

```
#define HWTIMER_DEV_NAME   "timer0"     /* timer name */
rt_device_t hw_dev;                     /* timer device handle */
rt_hwtimer_mode_t mode;                 /* timer mode */
rt_uint32_t freq = 10000;               /* couting frequency */

/* Timer timeout callback function */
static rt_err_t timeout_cb(rt_device_t dev, rt_size_t size)
{
    rt_kprintf("this is hwtimer timeout callback fucntion!\n");
    rt_kprintf("tick is :%d !\n", rt_tick_get());

    return 0;
}

static int hwtimer_sample(int argc, char *argv[])
{
    /* find timer device */
    hw_dev = rt_device_find(HWTIMER_DEV_NAME);
    /* open the device in read and write mode */
    rt_device_open(hw_dev, RT_DEVICE_OFLAG_RDWR);
    /* Set the timeout callback function */
    rt_device_set_rx_indicate(hw_dev, timeout_cb);

    /* Set the counting frequency (1Mhz or the supported minimum counting frequency by default) */
    rt_device_control(hw_dev, HWTIMER_CTRL_FREQ_SET, &freq);
    /* Set the mode to periodic timer */
    mode = HWTIMER_MODE_PERIOD;
    rt_device_control(hw_dev, HWTIMER_CTRL_MODE_SET, &mode);
}
```

1.4.2.15.2.5 Set the Timer Timeout Value

The timer timeout value can be set by the following function

```
rt_size_t rt_device_write(rt_device_t dev, rt_off_t pos, const void* buffer, rt_size_t size);
```

Parameter	Description
dev	device handle
pos	write data offset, unused now, can set 0 value
buffer	pointer to the timer timeout structure
size	timeout structure size
return	——
The actual size of the written data	
0	fail

The prototype of the timeout structure is shown below :

```
typedef struct rt_hwtimerval
{
    rt_int32_t sec;      /* second */
    rt_int32_t usec;     /* microsecond */
} rt_hwtimerval_t;
```

An example of using the timer timeout value is as follows:

```
#define HWTIMER_DEV_NAME   "timer0"     /* timer name */
rt_device_t hw_dev;                     /* timer device handle */
rt_hwtimer_mode_t mode;                 /* timer mode */
rt_hwtimerval_t timeout_s;              /* Timer timeout value */

/* Timer timeout callback function */
static rt_err_t timeout_cb(rt_device_t dev, rt_size_t size)
{
    rt_kprintf("this is hwtimer timeout callback fucntion!\n");
    rt_kprintf("tick is :%d !\n", rt_tick_get());

    return 0;
}

static int hwtimer_sample(int argc, char *argv[])
{
    /* find timer device */
    hw_dev = rt_device_find(HWTIMER_DEV_NAME);
    /* open the device in read-write mode */
```

```
    rt_device_open(hw_dev, RT_DEVICE_OFLAG_RDWR);
    /* set the timeout callback function */
    rt_device_set_rx_indicate(hw_dev, timeout_cb);
    /* set the mode as periodic timer */
    mode = HWTIMER_MODE_PERIOD;
    rt_device_control(hw_dev, HWTIMER_CTRL_MODE_SET, &mode);

    /* Set the timer timeout value to 5s and start the timer */
    timeout_s.sec = 5;      /* second */
    timeout_s.usec = 0;     /* microsecond */
    rt_device_write(hw_dev, 0, &timeout_s, sizeof(timeout_s));
}
```

1.4.2.15.2.6 Obtain the Current Value of the Timer

The current value of the timer can be obtained by the following function

```
rt_size_t rt_device_read(rt_device_t dev, rt_off_t pos, void* buffer, rt_size_t size);
```

Parameter	Description
dev	timer device handle
pos	write data offset, unused now , can set 0 value
buffer	output parameter, a pointer point to the timeout structure
size	timeout structure size
return	——
Timeout structure size	success
0	fail

An example of use is shown below

```
rt_hwtimerval_t timeout_s;      /* Used to save the time the timer has elapsed */
/* Read the elapsed time of the timer  */
rt_device_read(hw_dev, 0, &timeout_s, sizeof(timeout_s));
```

1.4.2.15.2.7 Close the Timer Device

The timer device can be closed with the following function:

```
rt_err_t rt_device_close(rt_device_t dev);
```

Parameter	Description
dev	timer device handle
return	——
RT_EOK	close device successfully
-RT_ERROR	the device has been completely shut down and cannot be closed repeatedly
other error code	fail to close the device

When a timer device has been used and is not necessary anymore, it should be closed, otherwise the device will remain in an open status.
An example of use is shown below

```
#define HWTIMER_DEV_NAME   "timer0"     /* timer name */
rt_device_t hw_dev;                     /* timer device handle */
/* find timer device */
hw_dev = rt_device_find(HWTIMER_DEV_NAME);
... ...
rt_device_close(hw_dev);
```

>Timing errors may occur. Assume that the counter has a maximum value of 0xFFFF, a counting frequency of 1Mhz, and a timing time of 1 second and 1 microsecond. Since the timer can only count up to 65535us at a time, the timing requirement for 1000001us can be completed 20 times at 50000us, and the calculation error will be 1us.

1.4.2.15.3 Hardware Timer Device Usage Example

The specific use of the hardware timer device can refer to the following sample code. The main steps of the sample code are as follows:

1. First find the device handle based on the timer device name "timer0".
2. Open the device "timer0" in read-write mode.
3. Set the timer timeout callback function.
4. Set the timer mode to periodic timer and set the timeout period to 5 seconds. At this time, the timer starts.
5. Read the timer after 3500ms delay, the read value will be displayed in seconds and microseconds.

```
 /*
  * Program listing: This is an hwtimer device usage routine
  * The routine exports the hwtimer_sample command to the control terminal
  * Command call format: hwtimer_sample
  * Program function: The hardware timer timeout callback function periodically prints the current tick
      value, and the difference between the two tick values is converted to the time equivalent to the
      timing time value.
  */

#include <rtthread.h>
#include <rtdevice.h>

#define HWTIMER_DEV_NAME   "timer0"     /* timer name */

/* Timer timeout callback function */
static rt_err_t timeout_cb(rt_device_t dev, rt_size_t size)
{
    rt_kprintf("this is hwtimer timeout callback fucntion!\n");
    rt_kprintf("tick is :%d !\n", rt_tick_get());

    return 0;
}

static int hwtimer_sample(int argc, char *argv[])
{
    rt_err_t ret = RT_EOK;
    rt_hwtimerval_t timeout_s;      /* timer timeout value */
    rt_device_t hw_dev = RT_NULL;   /* timer device value */
    rt_hwtimer_mode_t mode;         /* timer mode */

    /* find timer device */
    hw_dev = rt_device_find(HWTIMER_DEV_NAME);
    if (hw_dev == RT_NULL)
    {
        rt_kprintf("hwtimer sample run failed! can't find %s device!\n", HWTIMER_DEV_NAME);
        return -RT_ERROR;
    }

    /* Open the device in read-write mode */
    ret = rt_device_open(hw_dev, RT_DEVICE_OFLAG_RDWR);
    if (ret != RT_EOK)
    {
        rt_kprintf("open %s device failed!\n", HWTIMER_DEV_NAME);
        return ret;
    }

    /* set timeout callback function */
    rt_device_set_rx_indicate(hw_dev, timeout_cb);

    /* Setting mode is periodic timer */
    mode = HWTIMER_MODE_PERIOD;
    ret = rt_device_control(hw_dev, HWTIMER_CTRL_MODE_SET, &mode);
    if (ret != RT_EOK)
    {
        rt_kprintf("set mode failed! ret is :%d\n", ret);
        return ret;
    }

    /* Set the timer timeout value to 5s and start the timer. */
    timeout_s.sec = 5;      /* second */
    timeout_s.usec = 0;     /* microsecond */

    if (rt_device_write(hw_dev, 0, &timeout_s, sizeof(timeout_s)) != sizeof(timeout_s))
    {
        rt_kprintf("set timeout value failed\n");
        return -RT_ERROR;
    }

    /* delay 3500ms */
```

```
    rt_thread_mdelay(3500);

    /* read the current value of timer */
    rt_device_read(hw_dev, 0, &timeout_s, sizeof(timeout_s));
    rt_kprintf("Read: Sec = %d, Usec = %d\n", timeout_s.sec, timeout_s.usec);

    return ret;
}
/* Export to the msh command list */
MSH_CMD_EXPORT(hwtimer_sample, hwtimer sample);
```

1.4.2.16 PTP clock subsystem

1.4.2.16.1 PTP clock subsystem (API)

Header: **components/drivers/include/drivers/ptp.h**. Core: **components/drivers/ptp/ptp.c**.

The PTP layer wraps **IEEE 1588 hardware clocks** (PHC) behind a small, Linux-style **rt_ptp_ops** interface. Hardware drivers expose **read/set time**, **frequency and phase adjustment**, **external timestamp (EXTTS)**, **periodic output (PEROUT)**, **PPS**, optional **GPIO pin routing**, and **snapshot** reads. Upper layers (PTP protocol stacks, time sync services, test tools) call the **rt_ptp_*** helpers directly; the registered **rt_device** is mainly a **named handle** for lookup (`ptp0`, `ptp1`, ...).

Requires **RT_USING_PTP** (Kconfig) and **RT_USING_DM**.

1.4.2.16.1.1 When to use this API

Use PTP when...	Prefer something else when...
The SoC or NIC provides a **dedicated PHC** with sub-microsecond discipline.	**Calendar time only** — RTC Device or POSIX `time()`.
You need **ppb frequency trim**, **phase step**, or **hardware timestamping** of external events.	**OS tick / hrtimer scheduling** — HWTIMER Device or Clock Time Core.
Firmware or DT describes **EXTTS / PEROUT / PPS pins** on a PTP block.	**Software NTP only** — no PHC driver required.

1.4.2.16.1.2 Enable in menuconfig

```
RT-Thread Components →
    Device Drivers →
        [*] Using Precision Time Protocol (PTP) IEEE 1588 clock support
```

SoC-specific options may be added under **$(SOC_DM_PTP_DIR)/Kconfig** when present.

1.4.2.16.1.3 Roles

Role	Responsibility
PHC driver	Fills **struct rt_ptp_clock**, implements **rt_ptp_ops**, calls **rt_ptp_clock_register**. Reports capabilities (`max_freq`, `alarm_nr`, `extts_nr`, `perout_nr`, `pps`, `pins_nr`). Raises **rt_ptp_clock_event** from ISR or bottom-half when hardware events occur.
Client / stack	Obtains **struct rt_ptp_clock *** (from probe, DT phandle, or **rt_device_find("ptp0")** + **rt_device_to_ptp_clock**). Uses **rt_ptp_gettime** / **rt_ptp_adjfreq** / **...** and optional **rt_ptp_clock_notifier** callbacks.

The core registers each clock as **RT_Device_Class_Char** with **RT_DEVICE_FLAG_DEACTIVATE**. There is **no** standard `read`/`write`/`control` path in **ptp.c** — all operations go through the **rt_ptp_*** API.

1.4.2.16.1.4 Data structures

Time format

```
struct rt_ptp_clock_time {
    rt_int64_t sec;
    rt_int32_t nsec;   /* 0 .. 999_999_999 when passed through rt_ptp_adjtime */
};
```

Clock object

```
struct rt_ptp_clock {
    struct rt_device parent;
    const struct rt_ptp_ops *ops;

    rt_int32_t max_freq;   /* ppb limit; required if ops->adjfreq is set */
    rt_int32_t alarm_nr;
    rt_int32_t pins_nr;
    rt_int32_t extts_nr;
    rt_int32_t perout_nr;
    rt_int32_t pps;

    rt_list_t notifier_nodes;
    struct rt_spinlock nodes_lock;

    struct rt_ptp_pin *pins;      /* pins_nr entries when pins_nr > 0 */
    struct rt_mutex pin_mutex;
};
```

Convert from a generic device pointer:

```
#define rt_device_to_ptp_clock(dev) rt_container_of(dev, struct rt_ptp_clock, parent)
```

Driver operations

```
struct rt_ptp_ops {
    rt_err_t (*adjfreq)(struct rt_ptp_clock *ptp, rt_base_t freq);
    rt_err_t (*adjphase)(struct rt_ptp_clock *ptp, rt_int32_t phase);
    rt_err_t (*getmaxphase)(struct rt_ptp_clock *ptp, rt_int32_t *out_maxphase);
    rt_err_t (*adjtime)(struct rt_ptp_clock *ptp, rt_int64_t delta);
    rt_err_t (*gettime)(struct rt_ptp_clock *ptp, struct rt_ptp_clock_time *ts);
    rt_err_t (*settime)(struct rt_ptp_clock *ptp, const struct rt_ptp_clock_time *ts);
    rt_err_t (*enable)(struct rt_ptp_clock *ptp, struct rt_ptp_clock_request *request, rt_bool_t on);
    rt_err_t (*verify)(struct rt_ptp_clock *ptp, rt_uint32_t pin, rt_uint32_t func, rt_uint32_t chan);
    rt_err_t (*getsnapshot)(struct rt_ptp_clock *ptp, struct rt_ptp_clock_time *ts);
};
```

Callback	Purpose
adjfreq	Fine frequency adjustment in **parts per billion (ppb)**. Core clamps to **±max_freq**.
adjphase	Phase shift in nanoseconds; core clamps using **getmaxphase**.
getmaxphase	Reports maximum absolute phase adjustment.
adjtime	Step the clock by **delta** nanoseconds (signed).
gettime / **settime**	Read or write absolute PHC time.
enable	Enable/disable **EXTTS**, **PEROUT**, or **PPS** (see **struct rt_ptp_clock_request**).
verify	Validate pin/function/channel routing before metadata is updated. **Required** when **pins_nr** > **0**.
getsnapshot	Read a latched / snapshot time (hardware-specific).

Unimplemented ops return **-RT_ENOSYS** from the matching **rt_ptp_*** wrapper.

1.4.2.16.1.5 Register and unregister

```
rt_err_t rt_ptp_clock_register(struct rt_ptp_clock *ptp);
rt_err_t rt_ptp_clock_unregister(struct rt_ptp_clock *ptp);
```

rt_ptp_clock_register checks:

Condition	Error
ptp or ops is NULL	**-RT_EINVAL**
pins_nr > 0 but **pins is NULL** or **verify is NULL**	**-RT_EINVAL**
extts_nr or perout_nr > 0 but **enable is NULL**	**-RT_EINVAL**
adjfreq is non-NULL but **max_freq** <= 0	**-RT_EINVAL**
IDA exhausted	**-RT_EFULL**

On success the device name is **ptp**<**N**> (for example **ptp0**).
rt_ptp_clock_unregister:

- Fails with **-RT_EINVAL** if **ref_count != 0**
- Warns if notifiers are still registered
- Disables all **EXTTS** channels, **PEROUT** channels, and **PPS**
- Detaches **pin_mutex** when **pins_nr** > 0

1.4.2.16.1.6 Time discipline API

```
rt_err_t rt_ptp_adjfreq(struct rt_ptp_clock *ptp, rt_base_t freq);
rt_err_t rt_ptp_adjphase(struct rt_ptp_clock *ptp, rt_int32_t phase);
rt_err_t rt_ptp_getmaxphase(struct rt_ptp_clock *ptp, rt_int32_t *out_maxphase);
rt_err_t rt_ptp_adjtime(struct rt_ptp_clock *ptp, struct rt_ptp_clock_time *ts);
rt_err_t rt_ptp_gettime(struct rt_ptp_clock *ptp, struct rt_ptp_clock_time *ts);
rt_err_t rt_ptp_settime(struct rt_ptp_clock *ptp, const struct rt_ptp_clock_time *ts);
rt_err_t rt_ptp_getsnapshot(struct rt_ptp_clock *ptp, struct rt_ptp_clock_time *ts);
```

rt_ptp_adjtime converts **ts->sec** / **ts->nsec** to a signed nanosecond delta and forwards it to **ops->adjtime**. **nsec** must be in **[0, 10)**.
rt_ptp_adjfreq: if **max_freq** <= 0, only **freq == 0** is accepted (no-op success).

1.4.2.16.1.7 EXTTS, PEROUT, and PPS

External timestamps (EXTTS)

```
struct rt_ptp_request_extts {
    rt_uint32_t chan;
    rt_uint32_t flags;   /* PTP_ENABLE_FEATURE, PTP_RISING_EDGE, PTP_FALLING_EDGE, PTP_STRICT_FLAGS */
};

rt_err_t rt_ptp_request_extts(struct rt_ptp_clock *ptp, struct rt_ptp_request_extts *extts);
```

- **chan** must be < **extts_nr**
- With **PTP_STRICT_FLAGS**, enabling requires at least one of **PTP_RISING_EDGE** or **PTP_FALLING_EDGE**
- Clearing **PTP_ENABLE_FEATURE** (or zero flags) disables the channel via **enable(..., RT_FALSE)**

Periodic output (PEROUT)

```
struct rt_ptp_request_perout {
    rt_uint32_t chan;
    rt_uint32_t flags;   /* PTP_PEROUT_ONE_SHOT, PTP_PEROUT_DUTY_CYCLE, PTP_PEROUT_PHASE */
    struct rt_ptp_clock_time period;       /* zero period disables output */
    struct rt_ptp_clock_time on;            /* high time when PTP_PEROUT_DUTY_CYCLE */
    struct rt_ptp_clock_time start_phase;    /* start or phase offset when PTP_PEROUT_PHASE */
};

rt_err_t rt_ptp_request_perout(struct rt_ptp_clock *ptp, struct rt_ptp_request_perout *perout);
```

- **chan** must be < **perout_nr**
- With **PTP_STRICT_FLAGS**: duty **on** must not exceed **period**; **start_phase** must be strictly less than one period when **PTP_PEROUT_PHASE** is set
- Enable is inferred: **period.sec** || **period.nsec**

PPS

```
rt_err_t rt_ptp_enable_pps(struct rt_ptp_clock *ptp, rt_bool_t on);
```

Uses **request.type = PTP_CLK_REQ_PPS** and **ops->enable**.

1.4.2.16.1.8 GPIO pin routing

When **pins_nr > 0**, the core keeps a **pins[]** table describing which physical pin is mapped to which function and channel.

```
enum {
    PTP_PIN_FUNC_NONE,
    PTP_PIN_FUNC_EXTTS,
    PTP_PIN_FUNC_PEROUT,
    PTP_PIN_FUNC_PHYSYNC,
};

rt_err_t rt_ptp_get_pin_func(struct rt_ptp_clock *ptp, rt_uint32_t pin, struct rt_ptp_pin *out_pin);
rt_err_t rt_ptp_set_pin_func(struct rt_ptp_clock *ptp, rt_uint32_t pin, rt_uint32_t func, rt_uint32_t chan);
```

Important: rt_ptp_set_pin_func only updates **routing metadata** and disables the previous function on affected pins. It does **not** start EXTTS or PEROUT hardware. After assigning a pin, call **rt_ptp_request_extts** or **rt_ptp_request_perout** separately.
rt_ptp_set_pin_func flow:

1. Validate **func** / **chan** against **extts_nr**, **perout_nr**, or PHYSYNC rules
2. Call **ops->verify**
3. Disable any other pin already using the same **func + chan**
4. Disable the old function on the target pin, then store the new mapping

Access to **pins[]** is serialized with **pin_mutex**.

1.4.2.16.1.9 Event notifiers

Hardware drivers call **rt_ptp_clock_event** when an alarm fires, an external timestamp is captured, PPS is observed, etc.

```
enum {
    PTP_CLOCK_EV_ALARM,
    PTP_CLOCK_EV_EXTTS,
    PTP_CLOCK_EV_EXTOFF,
    PTP_CLOCK_EV_PPS,
    PTP_CLOCK_EV_PPSUSR,
};

struct rt_ptp_clock_event {
    rt_uint32_t type;
    rt_uint32_t index;
    union {
        rt_uint64_t timestamp;
        rt_int64_t offset;
        struct rt_ptp_clock_time pps_times;
    };
};

struct rt_ptp_clock_notifier {
    rt_list_t list;
    rt_uint32_t event_mask;   /* RT_BIT(PTP_CLOCK_EV_*) bits */
    struct rt_ptp_clock *ptp;
    void (*callback)(struct rt_ptp_clock_notifier *notifier, struct rt_ptp_clock_event *ev);
    void *priv;
};

rt_err_t rt_ptp_clock_notifier_register(struct rt_ptp_clock *ptp, struct rt_ptp_clock_notifier *notifier);
rt_err_t rt_ptp_clock_notifier_unregister(struct rt_ptp_clock *ptp, struct rt_ptp_clock_notifier *notifier);
void rt_ptp_clock_event(struct rt_ptp_clock *ptp, struct rt_ptp_clock_event *ev);
```

event_mask must be non-zero at register time. **rt_ptp_clock_event** walks registered notifiers under **nodes_lock** and invokes matching callbacks **without holding the lock** during the callback.

1.4.2.16.1.10 Example (PHC driver sketch)

```
#include <drivers/ptp.h>

static rt_err_t my_ptp_gettime(struct rt_ptp_clock *ptp, struct rt_ptp_clock_time *ts)
{
```

```
    /* read hardware counter */
    ts->sec = 0;
    ts->nsec = 0;
    return RT_EOK;
}

static rt_err_t my_ptp_adjfreq(struct rt_ptp_clock *ptp, rt_base_t freq)
{
    /* program frequency trim in ppb */
    return RT_EOK;
}

static rt_err_t my_ptp_enable(struct rt_ptp_clock *ptp,
        struct rt_ptp_clock_request *req, rt_bool_t on)
{
    switch (req->type) {
    case PTP_CLK_REQ_EXTTS:
        /* configure edge detect on req->extts.chan */
        break;
    case PTP_CLK_REQ_PEROUT:
        /* program req->perout period/on */
        break;
    case PTP_CLK_REQ_PPS:
        /* enable/disable PPS output */
        break;
    default:
        return -RT_EINVAL;
    }
    return RT_EOK;
}

static const struct rt_ptp_ops my_ptp_ops = {
    .gettime = my_ptp_gettime,
    .settime = my_ptp_settime,
    .adjfreq = my_ptp_adjfreq,
    .adjtime = my_ptp_adjtime,
    .enable = my_ptp_enable,
    .verify = my_ptp_verify,   /* required when pins_nr > 0 */
};

static struct rt_ptp_clock my_ptp = {
    .ops = &my_ptp_ops,
    .max_freq = 500000,   /* 500 ppm if 1 ppb == 1 Hz at 1 GHz; driver-defined scale */
    .extts_nr = 2,
    .perout_nr = 1,
    .pps = 1,
};

static int my_ptp_probe(struct rt_platform_device *pdev)
{
    return rt_ptp_clock_register(&my_ptp);
}
```

Client lookup

```
struct rt_ptp_clock *ptp;
rt_device_t dev = rt_device_find("ptp0");

if (dev)
    ptp = rt_device_to_ptp_clock(dev);

if (ptp) {
    struct rt_ptp_clock_time ts;
    rt_ptp_gettime(ptp, &ts);
}
```

POSIX dynamic clock (`clock_gettime`)

When **`RT_USING_POSIX_CLOCK`**, **`RT_USING_POSIX_FS`**, and **`RT_USING_PTP`** are enabled, PHC devices follow the Linux `dynamic clock` model:

```
#include <fcntl.h>
#include <sys/time.h>
#include <unistd.h>

int fd = open("/dev/ptp0", O_RDWR);
clockid_t clkid = FD_TO_CLOCKID(fd);
struct timespec ts;

clock_gettime(clkid, &ts);
clock_settime(clkid, &ts);   /* requires ops->settime */
clock_getres(clkid, &ts);
```

Macros in **sys/time.h**: **CLOCKFD**, **FD_TO_CLOCKID**, **CLOCKID_TO_FD**. Implementation: **components/libc/compilers/common/ctime.c**.
The fd must refer to an **open** /dev/ptpN device node (devfs). **CLOCK_REALTIME** still uses RTC; dynamic **clockid_t** values are separate from fixed POSIX clock IDs.

1.4.2.16.1.11 Pitfalls

Issue	Mitigation
set_pin_func without request_extts / request_perout	Pin table updates only; explicitly enable the feature afterward.
adjfreq without max_freq	Registration fails; set **max_freq** to the hardware ppb limit.
Notifier left registered at unregister	Core logs a warning; unregister notifiers in client teardown.
Calling rt_ptp_clock_event with wrong type	**RT_ASSERT** in debug builds; keep **type** < **PTP_CLOCK_EV_MAX**.
Blocking in notifier callback	Callback runs with lock dropped but may be invoked from interrupt context depending on the driver — keep callbacks short or defer work.
Confusing PTP with RTC	RTC is wall-clock calendar time; PTP PHC is for **precision sync** and **hardware timestamping**.

1.4.2.16.1.12 See also

- RTC Device — calendar RTC
- Clock Time Core — kernel clock source / clock event devices
- HWTIMER Device — general-purpose hardware timers
- components/drivers/ptp/ptp.c
- components/drivers/include/drivers/ptp.h

1.4.2.17 TEE subsystem

1.4.2.17.1 TEE subsystem (API)

Header: **components/drivers/include/drivers/tee.h**. Core: **components/drivers/tee/tee.c**.
The TEE layer provides a **GlobalPlatform-style** client API for talking to a **Trusted Execution Environment** from the normal world. Hardware or firmware backends implement **rt_tee_ops** (version query, session management, command invocation, shared memory registration). The core serializes access with a per-device mutex, tracks registered shared-memory objects, and registers a searchable device node (tee0, tee1, ...).
An **OP-TEE** backend lives under **components/drivers/tee/optee/** and is selected when **RT_TEE_OPTEE** is enabled.
Requires **RT_USING_TEE** (Kconfig) and **RT_USING_DM**.

1.4.2.17.1.1 When to use this API

Use TEE when...	Prefer something else when...
The platform runs **OP-TEE** (or another GP-compliant TEE) and exposes **Trusted Applications (TAs)**.	Secrets can stay in normal-world software with adequate isolation — no TEE required.

Use TEE when...	Prefer something else when...
You need **crypto, secure storage, or RNG** inside a TA with a defined UUID/command model.	Single-purpose on-chip crypto — a dedicated **hwcrypto** driver may suffice.
Firmware provides a DT node such as **linaro,optee-tz**.	No secure world firmware — enable and port a TEE OS first.

1.4.2.17.1.2 Enable in menuconfig

```
RT-Thread Components →
    Device Drivers →
        [*] Using Trusted Execution Environment (TEE) support
            [*] OP-TEE                    # ARCH_ARM_CORTEX_A || ARCH_ARMV8
                [*] Examples              # FinSH command optee_example_rng
```

SoC-specific backends may be added under **$(SOC_DM_TEE_DIR)/Kconfig** when present.

1.4.2.17.1.3 Roles

Role	Responsibility
TEE backend driver	Embeds **struct rt_tee_device**, fills **rt_tee_ops**, calls **rt_tee_device_register**. Translates GP requests to SMC/HVC, mailbox, or other transport.
Client / TA user	Finds **tee0**, opens a session to a TA UUID, registers shared memory, invokes TA commands. Checks both **rt_err_t** (transport) and **arg->ret** (TEE status).

The core registers each device as **RT_Device_Class_Char** with **RT_DEVICE_FLAG_DEACTIVATE**. There is **no** generic read/write/control path in **tee.c** — use the **rt_tee_*** helpers.
All public entry points take **tdev->mutex** except registration helpers.

1.4.2.17.1.4 Data structures

Device object

```
struct rt_tee_device {
    struct rt_device parent;
    rt_list_t shm_nodes;              /* registered struct rt_tee_shm */
    const struct rt_tee_ops *ops;
    struct rt_mutex mutex;
};

#define rt_device_to_tee_device(dev) rt_container_of(dev, struct rt_tee_device, parent)
```

Version and capabilities

```
struct rt_tee_version {
    rt_uint32_t impl_id;    /* TEE_IMPL_ID_OPTEE, TEE_IMPL_ID_AMDTEE, ... */
    rt_uint32_t impl_caps;  /* e.g. TEE_OPTEE_CAP_TZ */
    rt_uint32_t gen_caps;   /* TEE_GEN_CAP_GP, TEE_GEN_CAP_REG_MEM, ... */
};
```

Parameters (open session / invoke)

Parameters follow GlobalPlatform attribute encoding:

attr	Meaning
TEE_PARAM_ATTR_TYPE_NONE	Unused slot
TEE_PARAM_ATTR_TYPE_VALUE_IN/OUT/IN_OUT	Three 64-bit values in **value.a/b/c**
TEE_PARAM_ATTR_TYPE_MEMREF_IN/OUT/IN_OUT	Shared memory reference via **memref.shm**, offset, size

Flexible arrays append parameters to the end of open/invoke argument blocks:

```
struct rt_tee_open_session_arg {
    rt_uint8_t uuid[16];
    rt_uint8_t clnt_uuid[16];
    rt_uint32_t clnt_login;
    rt_uint32_t cancel_id;
    rt_uint32_t session;       /* output: session id from TEE */
    rt_uint32_t ret;           /* output: TEE result code */
    rt_uint32_t ret_origin;    /* output: RT_TEE_ORIGIN_* */
    rt_uint32_t params_nr;
    struct rt_tee_param params[];
};

struct rt_tee_invoke_arg {
    rt_uint32_t func;
    rt_uint32_t session;
    rt_uint32_t cancel_id;
    rt_uint32_t ret;
    rt_uint32_t ret_origin;
    rt_uint32_t params_nr;
    struct rt_tee_param params[];
};
```

Shared memory

```
struct rt_tee_shm {
    rt_list_t list;
    struct rt_tee_device *tdev;
    void *vaddr;
    rt_ubase_t paddr;
    rt_size_t size;
    rt_size_t align;
    rt_uint32_t flags;   /* TEE_SHM_DRV | TEE_SHM_SYS | TEE_SHM_USER */
};
```

Flag	Usage
TEE_SHM_SYS	Core allocates backing store in **rt_tee_shm_register** via **rt_tee_shm_alloc_helper** (page allocator or aligned malloc). Caller must **rt_tee_shm_free_helper** after unregister.
TEE_SHM_USER	Caller supplies **vaddr** (and optionally **paddr**); core resolves physical address when missing.
TEE_SHM_DRV	Driver-owned buffer; registration requires **vaddr** already set.

Driver operations

```
struct rt_tee_ops {
    rt_err_t (*get_version)(struct rt_tee_device *tdev, struct rt_tee_version *out_version);
    rt_err_t (*open_session)(struct rt_tee_device *tdev, struct rt_tee_open_session_arg *arg);
    rt_err_t (*close_session)(struct rt_tee_device *tdev, rt_uint32_t session);
    rt_err_t (*invoke)(struct rt_tee_device *tdev, struct rt_tee_invoke_arg *arg);
    rt_err_t (*cancel)(struct rt_tee_device *tdev, rt_uint32_t session, rt_uint32_t cancel_id);
    rt_err_t (*shm_register)(struct rt_tee_device *tdev, struct rt_tee_shm *shm);
    rt_err_t (*shm_unregister)(struct rt_tee_device *tdev, struct rt_tee_shm *shm);
};
```

1.4.2.17.1.5 TEE result codes

Transport failures use **rt_err_t** (−RT_EINVAL, −RT_ENOMEM, −RT_ENOSYS, ...). TEE-side status is returned in **arg->ret** with origin **arg->ret_origin**:

Code	Macro
Success	**RT_TEE_SUCCESS** (0)
Bad parameters	**RT_TEE_ERROR_BAD_PARAMETERS**
Not supported	**RT_TEE_ERROR_NOT_SUPPORTED**
Out of memory	**RT_TEE_ERROR_OUT_OF_MEMORY**
Communication error	**RT_TEE_ERROR_COMMUNICATION**

Code	Macro
...	See **tee.h** for the full list

Origins: **RT_TEE_ORIGIN_COMMS**, **RT_TEE_ORIGIN_TEE**, **RT_TEE_ORIGIN_TRUSTED_APP**.
Always check **both** the function return value and **arg->ret** on open/invoke.

1.4.2.17.1.6 Register and unregister

```
rt_err_t rt_tee_device_register(struct rt_tee_device *tdev);
rt_err_t rt_tee_device_unregister(struct rt_tee_device *tdev);
```

rt_tee_device_register requires **tdev** and **tdev->ops**. Device name is **tee<N>**.
rt_tee_device_unregister:

- Fails if **ref_count != 0**
- Warns and force-unregisters SHM entries if any remain on **shm_nodes**
- Detaches **mutex** after successful device unregister

1.4.2.17.1.7 Session and command API

```
rt_err_t rt_tee_get_version(struct rt_tee_device *tdev, struct rt_tee_version *out_version);

rt_err_t rt_tee_open_session(struct rt_tee_device *tdev, struct rt_tee_open_session_arg *arg);
rt_err_t rt_tee_close_session(struct rt_tee_device *tdev, rt_uint32_t session);

rt_err_t rt_tee_invoke(struct rt_tee_device *tdev, struct rt_tee_invoke_arg *arg);
rt_err_t rt_tee_cancel(struct rt_tee_device *tdev, rt_uint32_t session, rt_uint32_t cancel_id);
```

Typical flow:

1. Allocate **open_session_arg** with room for trailing **params[]**
2. Fill **uuid** (TA) and optional client identity fields
3. **rt_tee_open_session** — on success read **arg->session**, verify **arg->ret == RT_TEE_SUCCESS**
4. Register SHM if the TA command needs buffers
5. Allocate **invoke_arg**, set **func**, **session**, parameters
6. **rt_tee_invoke** — verify **arg->ret**
7. **rt_tee_close_session**, unregister/free SHM, close device

1.4.2.17.1.8 Shared memory helpers

```
rt_err_t rt_tee_shm_alloc_helper(struct rt_tee_shm *shm);
rt_err_t rt_tee_shm_free_helper(struct rt_tee_shm *shm);
rt_err_t rt_tee_shm_register(struct rt_tee_device *tdev, struct rt_tee_shm *shm);
rt_err_t rt_tee_shm_unregister(struct rt_tee_device *tdev, struct rt_tee_shm *shm);
```

rt_tee_shm_register:

- For **TEE_SHM_SYS** (without **TEE_SHM_DRV**): allocates memory, then calls **ops->shm_register**
- On backend failure after allocation, frees the buffer automatically
- On success, links **shm** into **tdev->shm_nodes**

rt_tee_shm_unregister removes the node and calls **ops->shm_unregister**. Call **rt_tee_shm_free_helper** afterward when the core allocated the buffer.

1.4.2.17.1.9 OP-TEE backend

Files: components/drivers/tee/optee/optee.c, optee_msg.h, optee_smc.h
Platform driver: optee-smc, compatible **linaro,optee-tz**

Device tree

Property	Values	Notes
compatible	"linaro,optee-tz"	Required
method	"smc" (default) or "hvc"	Selects **arm_smccc_smc** vs **arm_smccc_hvc**

Probe requirements

At probe time the driver:

1. Selects SMC or HVC call path from **method**
2. Verifies OP-TEE UID via **OPTEE_SMC_CALLS_UID**
3. Requires **OPTEE_SMC_SEC_CAP_DYNAMIC_SHM** from capability exchange
4. Registers **struct rt_tee_device** with **optee_ops**

Normal-world supplicant (RPC)

call_with_arg handles OP-TEE **RPC** returns while a secure-world call is in flight:

RPC function	Handled behavior
OPTEE_SMC_RPC_FUNC_ALLOC / FREE	Temporary RPC buffer alloc/free
OPTEE_SMC_RPC_FUNC_CMD	**supplicant_cmd**: SHM alloc/free, optional **I2C transfer** (RT_USING_I2C)
OPTEE_SMC_RPC_FUNC_FOREIGN_INTR	No-op placeholder

Shared memory passed to the secure world uses **non-contiguous page lists** built from physically contiguous normal-world pages.

FinSH example

With **RT_TEE_OPTEE_EXAMPLES**, build OP-TEE for your platform with the **hwrng** TA (see OP-TEE QEMU documentation), then run:

```
optee_example_rng
```

The example opens **tee0**, opens a session to the hwrng TA UUID, registers a 32-byte output buffer, invokes command **0x0 (GET_ENTROPY)**, and prints the result.
Source: **components/drivers/tee/optee/optee_examples.c**

1.4.2.17.1.10 Example (client sketch)

```
#include <drivers/tee.h>

static struct rt_tee_open_session_arg *open_arg_alloc(rt_uint32_t n)
{
    return rt_calloc(1, sizeof(struct rt_tee_open_session_arg) +
                        sizeof(struct rt_tee_param) * n);
}

static void uuid_to_bytes(rt_uint8_t *d, struct rt_tee_uuid *s)
{
    d[0] = s->time_low » 24;
    d[1] = s->time_low » 16;
    d[2] = s->time_low » 8;
    d[3] = s->time_low;
    d[4] = s->time_mid » 8;
    d[5] = s->time_mid;
    d[6] = s->time_hi_and_version » 8;
    d[7] = s->time_hi_and_version;
    rt_memcpy(d + 8, s->clock_seq_and_node, 8);
}

static int ta_example(struct rt_tee_device *tdev)
```

```
{
    rt_err_t err;
    struct rt_tee_shm shm = { .size = 64, .align = RT_MM_PAGE_SIZE, .flags = TEE_SHM_SYS };
    struct rt_tee_open_session_arg *open;
    struct rt_tee_invoke_arg *invoke;
    struct rt_tee_uuid ta_uuid = { /* TA UUID fields */ };

    open = open_arg_alloc(0);
    uuid_to_bytes(open->uuid, &ta_uuid);

    if ((err = rt_tee_open_session(tdev, open)) != RT_EOK || open->ret != RT_TEE_SUCCESS)
        goto out;

    if ((err = rt_tee_shm_register(tdev, &shm)) != RT_EOK)
        goto close;

    invoke = rt_calloc(1, sizeof(*invoke) + sizeof(struct rt_tee_param));
    invoke->func = 1;
    invoke->session = open->session;
    invoke->params_nr = 1;
    invoke->params[0].attr = TEE_PARAM_ATTR_TYPE_MEMREF_OUT;
    invoke->params[0].memref.shm = &shm;
    invoke->params[0].memref.size = shm.size;

    err = rt_tee_invoke(tdev, invoke);
    if (err == RT_EOK && invoke->ret == RT_TEE_SUCCESS) {
        /* use shm.vaddr */
    }

    rt_free(invoke);
    rt_tee_shm_unregister(tdev, &shm);
    rt_tee_shm_free_helper(&shm);

close:
    rt_tee_close_session(tdev, open->session);
out:
    rt_free(open);
    return (int)err;
}
```

Device lookup

```
struct rt_device *dev = rt_device_find("tee0");
struct rt_tee_device *tdev;

if (dev && rt_device_open(dev, 0) == RT_EOK)
    tdev = rt_device_to_tee_device(dev);
```

1.4.2.17.1.11 Pitfalls

Issue	Mitigation
Ignoring `arg->ret`	**`RT_EOK`** from **`rt_tee_invoke`** only means the message reached the backend; check **`invoke->ret`**.
SHM leak	Always **`rt_tee_shm_unregister`** then **`rt_tee_shm_free_helper`** for **`TEE_SHM_SYS`** buffers.
Unregister with active refs	**`rt_tee_device_unregister`** fails when **`ref_count != 0`** — close clients first.
OP-TEE without dynamic SHM	Probe fails if **`OPTEE_SMC_SEC_CAP_DYNAMIC_SHM`** is absent — use a matching OP-TEE build.
Missing TA	**`RT_TEE_ERROR_ITEM_NOT_FOUND`** / session open failure — deploy the TA binary to OP-TEE.
I2C RPC	OP-TEE I2C supplicant calls require **`RT_USING_I2C`** and a bus named **`i2c<N>`**.
Cache coherency	OP-TEE backend flushes/invalidates caches around shared message buffers; user SHM must remain in memory visible to both worlds.

1.4.2.17.1.12 See also

- OP-TEE documentation
- components/drivers/tee/tee.c
- components/drivers/tee/optee/optee.c
- components/drivers/include/drivers/tee.h

1.4.2.18 WATCHDOG Device

1.4.2.18.1 An Introduction to WATCHDOG

The hardware watchdog timer is a timer whose timing output is connected to the reset terminal of the circuit. In a productized embedded system, in order to automatically reset the system under abnormal conditions, it generally needs a watchdog.
When the watchdog was started, the counter starts counting automatically. If it is not reset counter value before the counter overflows, the counter overflow will generate a reset signal to the CPU to restart the system. When the system is running normally, it is necessary to clear the watchdog counter within the time interval allowed by the watchdog (commonly known as "feeding the dog"), and the reset signal will not be generated. If the program can "feed the dog" on timethe system does not go wrongotherwise the system will reset.
In general, users can feed the dog in the idlehook function and key function of RT-Thread.

1.4.2.18.2 Access to the WATCHDOG Device

The application accesses the watchdog hardware through the I/O device management interface provided by RT-↩ Thread. The related interfaces are as follows:

Function	Description
rt_device_find()	Find the device handle based on the device name of the watchdog device
rt_device_init()	Initialize the watchdog device
rt_device_control()	Control the watchdog device
rt_device_close()	Close the watchdog device

1.4.2.18.2.1 Find the Watchdog Device

The application obtains the device handle based on the watchdog device's name, and then it can operate the watchdog device. The function for finding a device is as follows:

```
rt_device_t rt_device_find(const char* name);
```

Function	Description
name	the name of the watchdog device
return	——
device handle	finding the corresponding device and then return to the corresponding device handle
RT_NULL	no corresponding device object found

An usage example is as follows:

```
#define IWDG_DEVICE_NAME    "iwg"    /* the name of the watchdog device */

static rt_device_t wdg_dev;         /* device handle of the watchdog */
/* find the watchdog device based on the device's name and obtain the device handle */
wdg_dev = rt_device_find(IWDG_DEVICE_NAME);
```

1.4.2.18.2.2 Initialize the Watchdog Device

The watchdog device need to be initialized before using, which can be done by the following function:

```
rt_err_t rt_device_init(rt_device_t dev);
```

Function	Description
dev	handle of the watchdog device
return	——
RT_EOK	the device succeeded initializing
-RT_ENOSYS	initialization failed, the watchdog device driver initialization function is empty
other error code	the device failed to open

An example is as follows:

```
#define IWDG_DEVICE_NAME    "iwg"    /* the name of the watchdog device */

static rt_device_t wdg_dev;    /* handle of the watchdog device */
/* find the watchdog device based on the device's name and obtain the device handle */
wdg_dev = rt_device_find(IWDG_DEVICE_NAME);

/* initialize the device */
rt_device_init(wdg_dev);
```

1.4.2.18.2.3 Control the Watchdog Device

The application can configure the watchdog device using the command control word, which can be done by the following function:

```
rt_err_t rt_device_control(rt_device_t dev, rt_uint8_t cmd, void* arg);
```

Function	Description
dev	handle of the watchdog device
cmd	the command word
arg	controlled parameter
return	——
RT_EOK	function executed successfully
-RT_ENOSYS	execution failed, the dev is empty
other error code	execution failed

The command control word `'cmd'` can take the following macro definition values:

```
#define RT_DEVICE_CTRL_WDT_GET_TIMEOUT    (1) /* get the overflow time */
#define RT_DEVICE_CTRL_WDT_SET_TIMEOUT    (2) /* set the overflow time */
#define RT_DEVICE_CTRL_WDT_GET_TIMELEFT   (3) /* get the remaining time */
#define RT_DEVICE_CTRL_WDT_KEEPALIVE      (4) /* feed the dog */
#define RT_DEVICE_CTRL_WDT_START          (5) /* start the watchdog */
#define RT_DEVICE_CTRL_WDT_STOP           (6) /* stop the watchdog */
```

An example of setting the overflow time of the watchdog is as follows:

```
#define IWDG_DEVICE_NAME    "iwg"    /* the name of the watchdog device */

rt_uint32_t timeout = 1000;    /* the overflow time */
static rt_device_t wdg_dev;    /* handle of the watchdog device */
/* find the watchdog device based on the device's name and obtain the device handle */
wdg_dev = rt_device_find(IWDG_DEVICE_NAME);
/* initialize the device */
rt_device_init(wdg_dev);

/* set the overflow time of the watch dog */
rt_device_control(wdg_dev, RT_DEVICE_CTRL_WDT_SET_TIMEOUT, (void *)timeout);
/* set idle-hook function */
rt_thread_idle_sethook(idle_hook);
```

An example of feeding a dog in an idle thread hook function is as follows:

```
static void idle_hook(void)
{
    /* Feed the dog in the callback function of the idle thread */
    rt_device_control(wdg_dev, RT_DEVICE_CTRL_WDT_KEEPALIVE, NULL);
}
```

1.4.2.18.2.4 Close the Watchdog Device

When the application completes the operation of the watchdog, it can close the watchdog device:

```
rt_err_t rt_device_close(rt_device_t dev);
```

Function	Description
dev	handle of the watchdog device
return	——
RT_EOK	close the device successfully
-RT_ERROR	The device has been completely shut down and cannot be closed repeatedly
other error code	fail to close the device

Closing the device interface and opening the device interface need to match each other. When you open the device, you need to close the device once correspondingly, so that the device will be completely shut down, otherwise the device will remain unclosed.

1.4.2.18.3 Watchdog Device usage example

The specific use of the watchdog device can be referred to the following sample code. The main steps of the sample code are as follows:

1. First find the device handle based on the device name "iwg".
2. Set the overflow time of the watchdog after initializing the device.
3. Set the idle thread callback function.
4. This callback function will run and feed the dog when the system executes idle threads.

```
/*
 * Program list: This is an independent watchdog device usage routine
 * The routine exports the iwdg_sample command to the control terminal
 * Command call format: iwdg_sample iwg
 * Command explanation: The second parameter of the command is the name of the watchdog device to be used.
     If it is empty, you can use the default watchdog device of the routine.
 * Program function: The program finds the watchdog device through the device's name, and then initializes
     the device and sets the overflow time of the watchdog device.
 *           Then set the idle thread callback function, which will feed the dog in the idle callback
     function.
*/

#include <rtthread.h>
#include <rtdevice.h>

#define IWDG_DEVICE_NAME    "iwg"    /* the name of the watchdog device */

static rt_device_t wdg_dev;         /* handle of the watchdog device */

static void idle_hook(void)
{
    /* feed the dog in the callback function */
    rt_device_control(wdg_dev, RT_DEVICE_CTRL_WDT_KEEPALIVE, NULL);
    rt_kprintf("feed the dog!\n ");
}

static int iwdg_sample(int argc, char *argv[])
{
    rt_err_t ret = RT_EOK;
    rt_uint32_t timeout = 1000;    /* the overflow time */
    char device_name[RT_NAME_MAX];

    /* Determine if the command-line parameter is given the device name */
    if (argc == 2)
    {
        rt_strncpy(device_name, argv[1], RT_NAME_MAX);
    }
    else
    {
        rt_strncpy(device_name, IWDG_DEVICE_NAME, RT_NAME_MAX);
    }
    /* find the watchdog device based on the device's name and obtain the device handle */
    wdg_dev = rt_device_find(device_name);
```

```
    if (!wdg_dev)
    {
        rt_kprintf("find %s failed!\n", device_name);
        return -RT_ERROR;
    }
    /* initialize the device */
    ret = rt_device_init(wdg_dev);
    if (ret != RT_EOK)
    {
        rt_kprintf("initialize %s failed!\n", device_name);
        return -RT_ERROR;
    }
    /* set the overflow time of the watch dog */
    ret = rt_device_control(wdg_dev, RT_DEVICE_CTRL_WDT_SET_TIMEOUT, &timeout);
    if (ret != RT_EOK)
    {
        rt_kprintf("set %s timeout failed!\n", device_name);
        return -RT_ERROR;
    }
    /* set idle thread callback function */
    rt_thread_idle_sethook(idle_hook);

    return ret;
}
/* export to the msh command list */
MSH_CMD_EXPORT(iwdg_sample, iwdg sample);
```

1.4.2.19 WLAN Device

With the rapid development of the Internet of Things, more and more embedded devices are equipped with WIFI wireless network devices. In order to be able to manage WIFI network devices, RT-Thread introduces a WLAN device management framework. This framework has many features to control and manage WIFI, providing developers with many conveniences for using WIFI devices.

1.4.2.19.1 Introduction to the WLAN Framework

The WLAN framework is a set of middleware developed by RT-Thread for managing WIFI. Connect to the specific WIFI driver, control the WIFI connection disconnection, scan and other operations. Support different applications, provide WIFI control, events, data diversion and other operations for the application, and provide a unified WIFI control interface for the upper application. The WLAN framework consists of three main parts. The DEV driver interface layer provides a unified API for the WLAN framework. Manage layer provides users with specific functions such as WIFI scanning, connection, and disconnection. Protocol is responsible for processing the data stream generated on the WIFI. Different protocols such as LWIP can be mounted according to different usage scenarios. It has the characteristics of simple use, complete functions, convenient docking and strong compatibility.
The following figure is a hierarchical diagram of the WIFI framework:

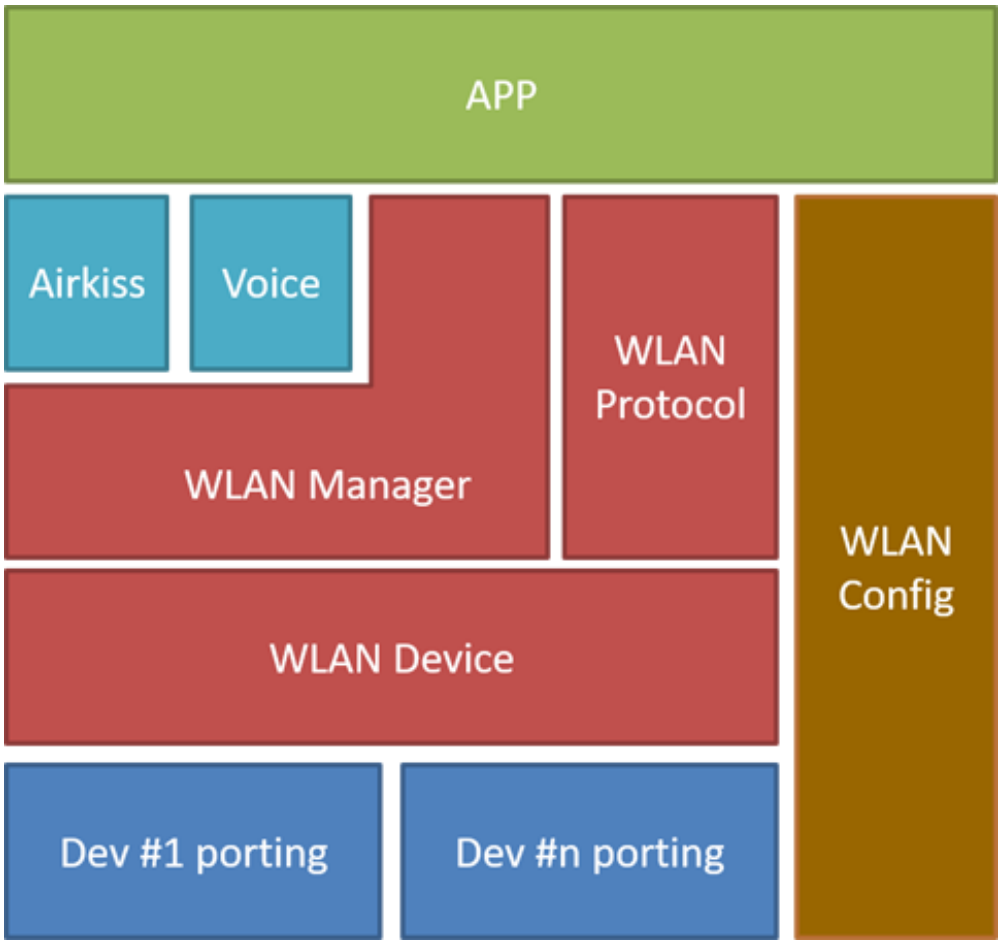


Figure 1.108 WIFI Framework

The First Part: `APP`, the application layer. It is a specific application based on the WLAN framework, such as WiFi-related shell commands.
The Second Part: `Airkiss and Voice`, the network configuration layer. Provide functions such as using wireless or sound waves to configure the network.
The Third Part: `WLAN Manager`, the WLAN management layer. Ability to control and manage WLAN devices. It has functions related to WLAN control, such as setting mode, connecting hotspots, disconnecting hotspots, enabling hotspots, scanning hotspots, etc. It also provides management functions such as reconnection after disconnection and automatic hotspot switching.
The Fourth Part: `WLAN Protocol`, the protocol layer. The data stream is submitted to a specific protocol for resolution, and the user can specify to communicate using different protocols.
The Fifth Part: `WLAN Config`, the parameter management layer. Manage hotspot information and passwords for successful connections and write them to non-volatile storage media.
The Sixth Part: `WLAN Device`, the driver interface layer. Connect to specific WLAN hardware and provide unified APIs for management.

1.4.2.19.1.1 Functions

- Automatic Connection: After using automatic connection function, as long as the WIFI is disconnected, the hotspot information of the previous successful connection will be automatically read, and the hotspot will be connected. If a hotspot connection fails, switch to the next hotspot to connect until the connection is successful. The hotspot information used by the automatic connection is sequentially tried in the order of the success of the connection, and the hotspot information of the latest connection success is preferentially used. After the connection is successful, the hotspot information is cached first, and use it first when reconnecting after the next disconnection.

- Parameter storage: Stores the WIFI parameters for successful connection. The WIFI parameter will be cached in the memory. If the external non-volatile storage interface is configured, it will be stored in the

external storage medium. Users can implement the `struct rt_wlan_cfg_ops` structure according to his actual situation and save the parameters anywhere. The cached parameters mainly provide hotspot information for automatic connections. When WIFI is unconnected, it will read the cached parameters and try to connect.

- WIFI control: Provide complete WIFI control APIs, scanning, connection, hotspot, etc. Provide WIFI related status callback events, disconnect, connection, connection failure, etc. Provide users with an easy to use WIFI management APIs.
- Shell command: You can enter the command in Msh to control WIFI to perform scanning, connecting, disconnecting and other actions. Print debugging information such as WIFI status.

1.4.2.19.1.2 Configuration

Use `menuconfig` command in ENV to enter the WLAN configuration interface by following the menu:

```
RT-Thread Components ->  Device Drivers -> Using WiFi ->
```

Configuration options are described in detail as follows:

```
[*] Using Wi-Fi framework                /* Using Wi-Fi framework */
(wlan0) The WiFi device name for station /* The default name for station */
(wlan1) The WiFi device name for ap      /* The default name for ap */
(lwip) Default transport protocol        /* Default protocol */
(10000) Set scan timeout time(ms)        /* Scan timeout time */
(10000) Set connect timeout time(ms)     /* Connect timeout time */
(32)  SSID name maximum length           /* Maximum length of SSID name */
(32)  Maximum password length            /* Maximum length of password */
[*]   Automatic sorting of scan results  /* Automatic sorting of scan results */
(3)   Maximum number of WiFi information automatically saved /* Maximum number of WiFi information
    automatically saved */
(wlan_job) WiFi work queue thread name   /* WiFi work queue thread name */
(2048) wifi work queue thread size       /* wifi work queue thread size */
(22)  WiFi work queue thread priority    /* WiFi work queue thread priority */
(2)   Maximum number of driver events    /* Maximum number of driver events in dev layer */
[ ]   Forced use of PBUF transmission    /* Forced use of PBUF transmission */
[ ]   Enable WLAN Debugging Options      /* Enable WLAN Debugging Options */
```

1.4.2.19.2 Access Wi-Fi Devices

The application accesses the WLAN device hardware through the WLAN device management interface, and the relevant interfaces are as follows:

Fuctions	Description
rt_wlan_prot_attach()	Specify the WLAN protocol attached
rt_wlan_scan_sync()	Synchronized WLAN Scan
rt_wlan_connect()	Synchronized Hotspot Connection
rt_wlan_disconnect()	Synchronized Hotspot Disconnection
rt_wlan_config_autoreconnect()	Configuration automatic reconnection mode

1.4.2.19.2.1 Specify Protocol

```
rt_err_t rt_wlan_prot_attach(const char *dev_name, const char *prot_name);
```

Parameter	**D**escription
dev_name	WLAN device name
prot_name	Protocol name, possible values: RT_WLAN_PROT_LWIP, indicates the protocol type LWIP
Return	–
-RT_ERROR	Execution failed
RT_EOK	Execution succeed

1.4.2.19.2.2 Synchronized WLAN Scan

```
struct rt_wlan_scan_result *rt_wlan_scan_sync(void);
```

Return	Description
rt_wlan_scan_result	Scan Result

The scan result is a structure as follows:

```
struct rt_wlan_scan_result
{
    rt_int32_t num;             /* info number */
    struct rt_wlan_info *info;  /* info pointer */
};
```

1.4.2.19.2.3 Synchronized Hotspot Connection

```
rt_err_t rt_wlan_connect(const char *ssid, const char *password);
```

Parameter	Description
ssid	WIFI name
password	WIFI password
Return	–
-RT_EINVAL	Parameter error
-RT_EIO	Unregistered device
-RT_ERROR	Connection failed
RT_EOK	Connection successful

1.4.2.19.2.4 Synchronized Hotspot Disconnection

```
rt_err_t rt_wlan_disconnect(void);
```

Return	Description
-RT_EIO	Unregistered device
-RT_ENOMEM	Not enough memory
-RT_ERROR	Disconnection failed
RT_EOK	Disconnection successful

1.4.2.19.2.5 Automatic Reconnection Mode Configuration

```
void rt_wlan_config_autoreconnect(rt_bool_t enable);
```

Parameter	Description
enable	enable or disable automatic reconnection

1.4.2.19.3 FinSH Command

Using shell commands can help us quickly debug WiFi-related features. The wifi related shell commands are as follows:

```
wifi                          /* Print help */
wifi help                     /* View help */
wifi join SSID [PASSWORD]     /* Connect wifi.if SSDI is empty, use configuration to connect automatically
    */
wifi ap   SSID [PASSWORD]     /* Create hotspot */
wifi scan                     /* Scan all hotspots */
wifi disc                     /* Disconnnect */
```

```
wifi ap_stop                    /* Stop hotspot */
wifi status                     /* Print wifi status sta + ap */
wifi smartconfig                /* Start to configure network function */
```

1.4.2.19.3.1 WiFi Scan

The wifi scan command is `wifi scan`. After the wifi scan command is executed, the surrounding hotspot information will be printed on the terminal. Through the printed hotspot information, you can see multiple attributes such as SSID and MAC address.
Enter the command in msh and the scan results are as follows:

```
wifi scan
SSID                             MAC            security    rssi chn Mbps
-------------------------------- -----------------  -------------- ---- --- ----
rtt_test_ssid_1                  c0:3d:46:00:3e:aa  OPEN           -14    8  300
test_ssid                        3c:f5:91:8e:4c:79  WPA2_AES_PSK   -18    6   72
rtt_test_ssid_2                  ec:88:8f:88:aa:9a  WPA2_MIXED_PSK -47    6  144
rtt_test_ssid_3                  c0:3d:46:00:41:ca  WPA2_MIXED_PSK -48    3  300
```

1.4.2.19.3.2 WiFi Connection

The wifi scan command is `wifi join`. The command needs to be followed by the hotspot name and hotspot password. If the hotspot does not have a password, you may not enter this item. After the WiFi connection command is executed, if the hotspot exists and the password is correct, the board will connect to the hotspot and obtain the IP address. After the network connection is successful, you can use `socket` sockets for network communication.
An example of using the wifi connection command is as follows. After the connection is successful, the obtained IP address will be printed on the terminal as follows:

```
wifi join ssid_test 12345678
[I/WLAN.mgnt] wifi connect success ssid:ssid_test
[I/WLAN.lwip] Got IP address : 192.168.1.110
```

1.4.2.19.3.3 WiFi Disconnection

The command to disconnect WiFi is `wifi disc`. After the WiFi disconnect command is executed, the development board will disconnect from the hotspot.
The WiFi disconnect command usage example is as follows. After the disconnection is successful, the following information will be printed on the terminal as shown below.

```
wifi disc
[I/WLAN.mgnt] disconnect success!
```

1.4.2.19.4 Example for WLAN Device Usage

1.4.2.19.4.1 WiFi Scan

The following code will show a WiFi sync scan, and then print the results on the terminal. First perform WIFI initialization, and then execute the WIFI scan function `rt_wlan_scan_sync`, this function is synchronous, the number of scans and results returned by the function. In this example, the scanned hotspot name will be printed.

```
#include <rthw.h>
#include <rtthread.h>

#include <dev_wlan_mgnt.h>
#include <dev_wlan_prot.h>
#include <dev_wlan_cfg.h>

void wifi_scan(void)
{
    struct rt_wlan_scan_result *result;
    int i = 0;

    /* Configuring WLAN device working mode */
    rt_wlan_set_mode(RT_WLAN_DEVICE_STA_NAME, RT_WLAN_STATION);
    /* WiFi scan */
    result = rt_wlan_scan_sync();
    /* Print scan results */
    rt_kprintf("scan num:%d\n", result->num);
    for (i = 0; i < result->num; i++)
    {
        rt_kprintf("ssid:%s\n", result->info[i].ssid.val);
    }
}

int scan(int argc, char *argv[])
```

```
{
    wifi_scan();
    return 0;
}
MSH_CMD_EXPORT(scan, scan test.);
```

The results are as follows:

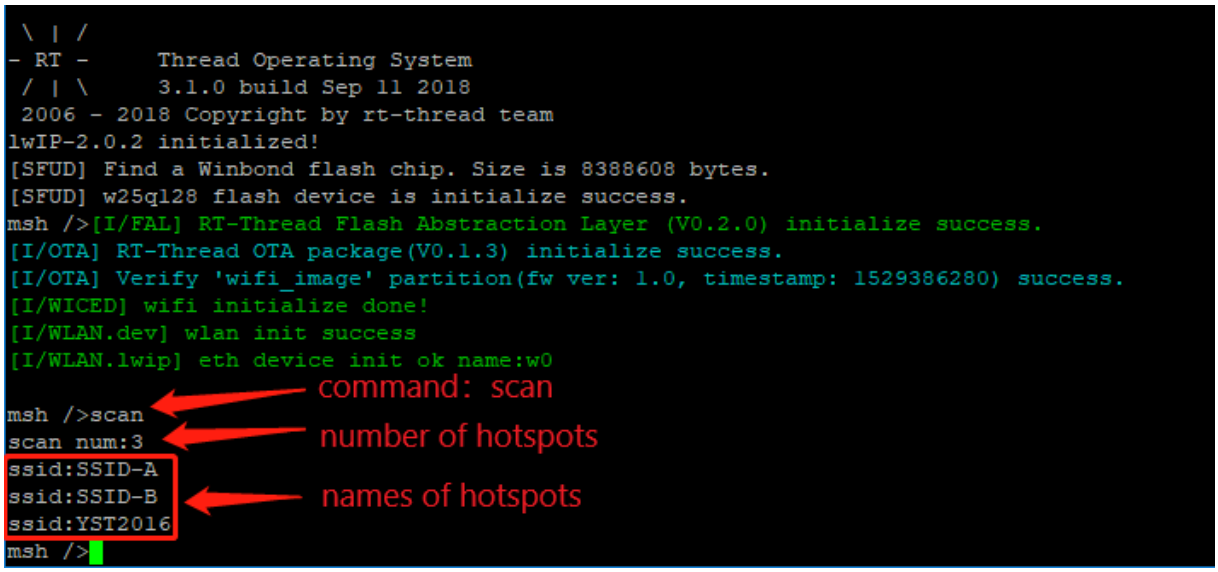


Figure 1.109 Scan

1.4.2.19.4.2 WiFi Connection and Disconnection

The code below will show a WiFi sync connection. Initialize WIFI first, and then create a semaphore for waiting for the `RT_WLAN_EVT_READY` event. Register the callback function of the event that needs attention, execute the `rt_wlan_connect` wifi connection function, and return value will indicate whether the connection has been successful. If the WiFi connection succeeds, it needs to wait for the network to get the IP address before communication. Use the semaphore created in advance to wait for the network to be ready. Once the network is ready, it will be able to communicate.

After connecting to WIFI, wait for a while and then execute `rt_wlan_disconnect` to disconnect. The disconnect operation is blocked, and the return value indicates whether the disconnection was successful.

```
#include <rthw.h>
#include <rtthread.h>

#include <dev_wlan_mgnt.h>
#include <dev_wlan_prot.h>
#include <dev_wlan_cfg.h>

#define WLAN_SSID                "SSID-A"
#define WLAN_PASSWORD            "12345678"
#define NET_READY_TIME_OUT       (rt_tick_from_millisecond(15 * 1000))

static rt_sem_t net_ready = RT_NULL;

static void
wifi_ready_callback(int event, struct rt_wlan_buff *buff, void *parameter)
{
    rt_kprintf("%s\n", __FUNCTION__);
    rt_sem_release(net_ready);
}

static void
wifi_connect_callback(int event, struct rt_wlan_buff *buff, void *parameter)
{
    rt_kprintf("%s\n", __FUNCTION__);
    if ((buff != RT_NULL) && (buff->len == sizeof(struct rt_wlan_info)))
    {
        rt_kprintf("ssid : %s \n", ((struct rt_wlan_info *)buff->data)->ssid.val);
    }
}

static void
wifi_disconnect_callback(int event, struct rt_wlan_buff *buff, void *parameter)
{
    rt_kprintf("%s\n", __FUNCTION__);
```

```
    if ((buff != RT_NULL) && (buff->len == sizeof(struct rt_wlan_info)))
    {
        rt_kprintf("ssid : %s \n", ((struct rt_wlan_info *)buff->data)->ssid.val);
    }
}

static void
wifi_connect_fail_callback(int event, struct rt_wlan_buff *buff, void *parameter)
{
    rt_kprintf("%s\n", __FUNCTION__);
    if ((buff != RT_NULL) && (buff->len == sizeof(struct rt_wlan_info)))
    {
        rt_kprintf("ssid : %s \n", ((struct rt_wlan_info *)buff->data)->ssid.val);
    }
}

rt_err_t wifi_connect(void)
{
    rt_err_t result = RT_EOK;

    /* Configuring WLAN device working mode */
    rt_wlan_set_mode(RT_WLAN_DEVICE_STA_NAME, RT_WLAN_STATION);
    /* station connect */
    rt_kprintf("start to connect ap ...\n");
    net_ready = rt_sem_create("net_ready", 0, RT_IPC_FLAG_FIFO);
    rt_wlan_register_event_handler(RT_WLAN_EVT_READY,
            wifi_ready_callback, RT_NULL);
    rt_wlan_register_event_handler(RT_WLAN_EVT_STA_CONNECTED,
            wifi_connect_callback, RT_NULL);
    rt_wlan_register_event_handler(RT_WLAN_EVT_STA_DISCONNECTED,
            wifi_disconnect_callback, RT_NULL);
    rt_wlan_register_event_handler(RT_WLAN_EVT_STA_CONNECTED_FAIL,
            wifi_connect_fail_callback, RT_NULL);

    /* connect wifi */
    result = rt_wlan_connect(WLAN_SSID, WLAN_PASSWORD);

    if (result == RT_EOK)
    {
        /* waiting for IP to be got successfully  */
        result = rt_sem_take(net_ready, NET_READY_TIME_OUT);
        if (result == RT_EOK)
        {
            rt_kprintf("networking ready!\n");
        }
        else
        {
            rt_kprintf("wait ip got timeout!\n");
        }
        rt_wlan_unregister_event_handler(RT_WLAN_EVT_READY);
        rt_sem_delete(net_ready);

        rt_thread_delay(rt_tick_from_millisecond(5 * 1000));
        rt_kprintf("wifi disconnect test!\n");
        /* disconnect */
        result = rt_wlan_disconnect();
        if (result != RT_EOK)
        {
            rt_kprintf("disconnect failed\n");
            return result;
        }
        rt_kprintf("disconnect success\n");
    }
    else
    {
        rt_kprintf("connect failed!\n");
    }
    return result;
}

int connect(int argc, char *argv[])
{
    wifi_connect();
    return 0;
}
MSH_CMD_EXPORT(connect, connect test.);
```

The results are as follows:

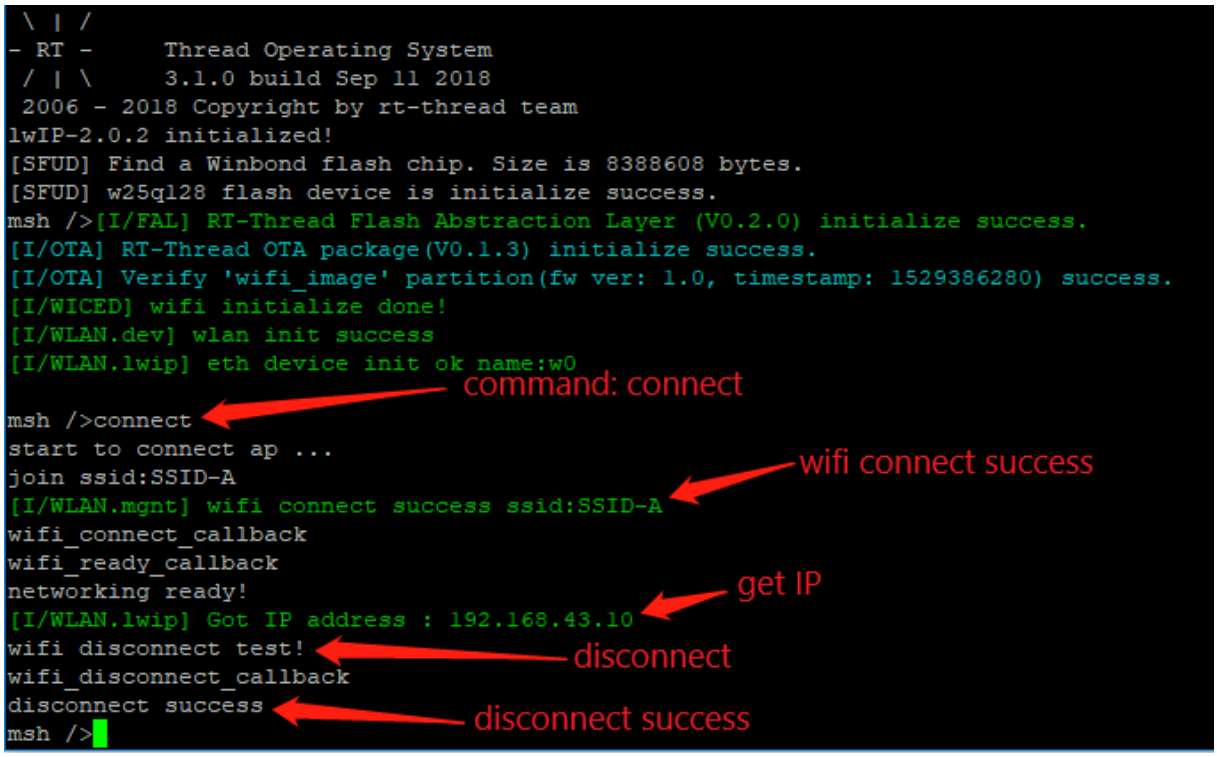


Figure 1.110 Disconnection

1.4.2.19.4.3 WiFi Auto Reconnection when Turn On

First enable the automatic reconnection function, use the command line to connect to the hotspot A, and connect another hotspot B. After waiting for a few seconds, power off hotspot B, the system will automatically retry connecting B hotspot. At this time, B hotspot connection can not be connected, and the system automatically switches hotspot A to connect. After the connection is successful, the system stops connecting.

```
#include <rthw.h>
#include <rtthread.h>

#include <dev_wlan_mgnt.h>
#include <dev_wlan_prot.h>
#include <dev_wlan_cfg.h>

static void
wifi_ready_callback(int event, struct rt_wlan_buff *buff, void *parameter)
{
    rt_kprintf("%s\n", __FUNCTION__);
}

static void
wifi_connect_callback(int event, struct rt_wlan_buff *buff, void *parameter)
{
    rt_kprintf("%s\n", __FUNCTION__);
    if ((buff != RT_NULL) && (buff->len == sizeof(struct rt_wlan_info)))
    {
        rt_kprintf("ssid : %s \n", ((struct rt_wlan_info *)buff->data)->ssid.val);
    }
}

static void
wifi_disconnect_callback(int event, struct rt_wlan_buff *buff, void *parameter)
{
    rt_kprintf("%s\n", __FUNCTION__);
    if ((buff != RT_NULL) && (buff->len == sizeof(struct rt_wlan_info)))
    {
        rt_kprintf("ssid : %s \n", ((struct rt_wlan_info *)buff->data)->ssid.val);
    }
}

static void
wifi_connect_fail_callback(int event, struct rt_wlan_buff *buff, void *parameter)
{
    rt_kprintf("%s\n", __FUNCTION__);
    if ((buff != RT_NULL) && (buff->len == sizeof(struct rt_wlan_info)))
    {
```

```
        rt_kprintf("ssid : %s \n", ((struct rt_wlan_info *)buff->data)->ssid.val);
    }
}

int wifi_autoconnect(void)
{
    /* Configuring WLAN device working mode */
    rt_wlan_set_mode(RT_WLAN_DEVICE_STA_NAME, RT_WLAN_STATION);
    /* Start automatic connection */
    rt_wlan_config_autoreconnect(RT_TRUE);
    /* register event */
    rt_wlan_register_event_handler(RT_WLAN_EVT_READY,
            wifi_ready_callback, RT_NULL);
    rt_wlan_register_event_handler(RT_WLAN_EVT_STA_CONNECTED,
            wifi_connect_callback, RT_NULL);
    rt_wlan_register_event_handler(RT_WLAN_EVT_STA_DISCONNECTED,
            wifi_disconnect_callback, RT_NULL);
    rt_wlan_register_event_handler(RT_WLAN_EVT_STA_CONNECTED_FAIL,
            wifi_connect_fail_callback, RT_NULL);
    return 0;
}

int auto_connect(int argc, char *argv[])
{
    wifi_autoconnect();
    return 0;
}
MSH_CMD_EXPORT(auto_connect, auto connect test.);
```

The results are as follows:

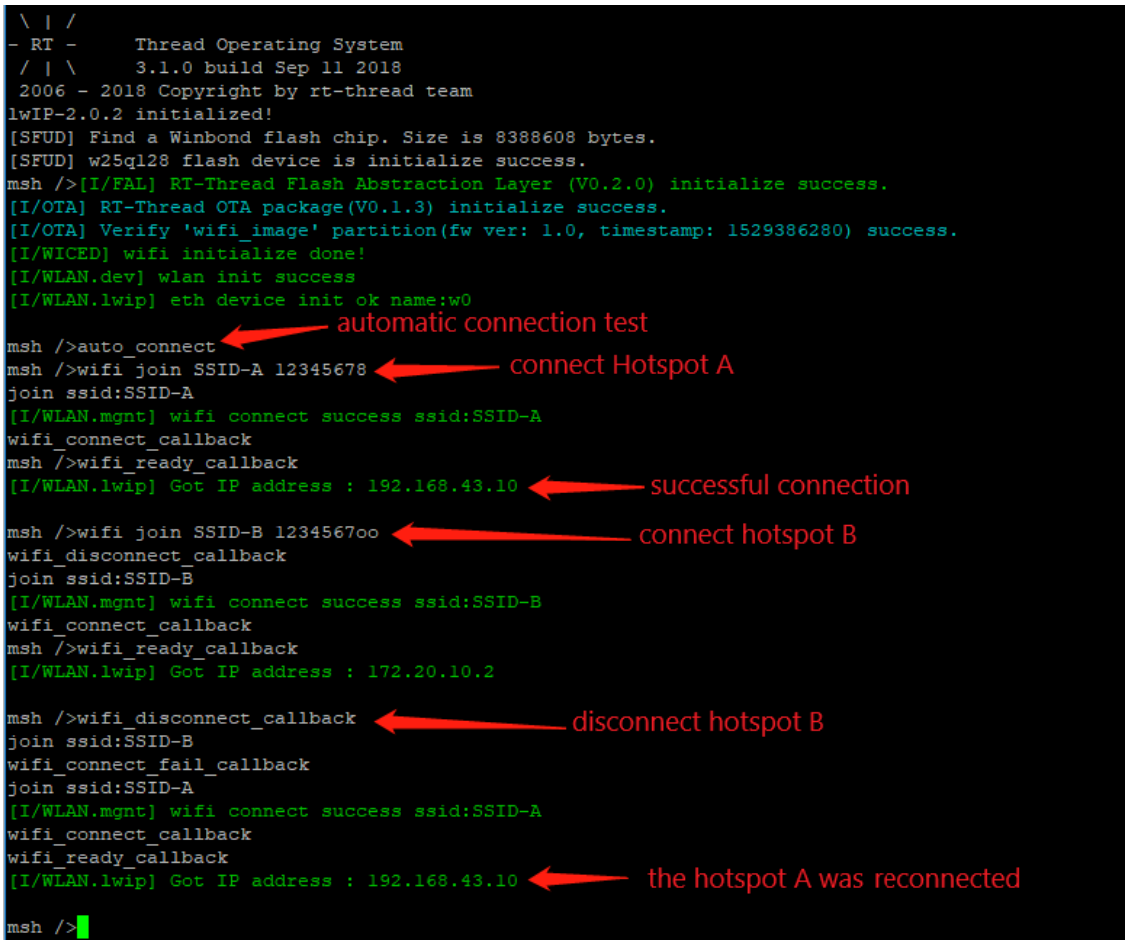


Figure 1.111 Autoconnection

1.4.2.20 Sensor Device

1.4.2.20.1 Introduction

Sensors are an important part of the Internet of Things, and sensors in an IoT system are equivalent to the eyes of humans. Without eyes, human beings can not see and interpret the world around them. The same is true for the

Internet of Things.
Nowadays, with the development of IoT, a large number of sensors are available for developers to choose, such as Accelerometers, Magnetometers, Gyroscopes, Barometers/pressure senosrs, Hygrometers/humidity meters and so on. However, these sensors, manufactured by the world's leading semiconductor manufacturers, have increased market selectivity and made application development more difficult. Because different sensor manufacturers and sensors need their own unique drivers to run, developers need to write a device driver for each sensor, which can be difficult and time-consuming. In order to reduce the difficulty of application development and increase the reusability of sensor driver, we designed a Sensor device.
The function of the Sensor device is to provide a unified operation interface for the upper layer and improve the reusability of the upper code.

1.4.2.20.1.1 Characteristics of the Sensor Device

- **Interface:** Standard device interface (open/close/read/control)
- **Work mode:** Supports polling, interrupts, three FIFO (First In, First Out) modes
- **Power mode:** support four modes: power failure, common, low power consumption and high power consumption

1.4.2.20.2 Access Sensor Device

The application accesses the sensor device through the I/O device management interface provided by RT-Thread. The related interfaces are as follows:

Functions	Description
rt_device_find()	Finding device handles based on device name of sensor device
rt_device_open()	open sensor device
rt_device_read()	read data
rt_device_control()	control sensor device
rt_device_set_rx_indicate()	setting reveive callback fuction
rt_device_close()	close sensor device

1.4.2.20.2.1 Find Sensor Device

The application obtains the device handle according to the name of the sensor device, and then can operate the sensor device. The function of finding the device is as follows:

```
rt_device_t rt_device_find(const char* name);
```

Parameter	Description
name	sensor device name
return	——
handle	Finding the corresponding device returns the corresponding device handle
RT_NULL	No corresponding device object was found

The use example is as follows:

```
#define SENSOR_DEVICE_NAME    "acce_st"    /* sensor device name */

static rt_device_t sensor_dev;         /* sensor device handle */
/* Find the sensor device according to the device name and get the device handle */
sensor_dev = rt_device_find(SENSOR_DEVICE_NAME);
```

1.4.2.20.2.2 Open Sensor Device

Through the device handle, the application can open and close the device. When the device is opened, it will check whether the device has been initialized or not. If it is not initialized, it will call the initialization interface by default.

Open the device through the following functions:

```
rt_err_t rt_device_open(rt_device_t dev, rt_uint16_t oflags);
```

Parameter	Description
dev	device handle
oflags	open mode flag
Return	——
RT_EOK	open success
-RT_EBUSY	If the RT_DEVICE_FLAG_STANDALONE parameter is included in the parameter specified at the time of device registration, the device will not be allowed to open repeatedly.
-RT_EINVAL	Unsupported open mode
other err	open failed

The oflags parameter supports the following parameters

```
#define RT_DEVICE_FLAG_RDONLY       0x001     /* Read-only mode for standard device, polling mode for
      corresponding sensors */
#define RT_DEVICE_FLAG_INT_RX       0x100     /* Interrupt Receiving Mode */
#define RT_DEVICE_FLAG_FIFO_RX      0x200     /* FIFO receiving mode */
```

There are three modes of receiving and sending sensor data: interrupt mode, polling mode and FIFO mode. When using these three modes, **only one of them can be chosen**. If the sensor's open parameter oflags does not specify the use of interrupt mode or FIFO mode, polling mode is used by default.
FIFO transmission mode needs sensor hardware support, data is stored in hardware FIFO, which reads and stores multiple data simultaneously, which allows the CPU to do other operations while gathering data. This feature is very useful in low power mode.
If the sensor uses FIFO receiving mode, the value of oflags is RT_DEVICE_FLAG_FIFO_RX.
An example of turning on sensor devices in polling mode is as follows:

```
#define SAMPLE_SENSOR_NAME       "acce_st"  /* sensor device name */
int main(void)
{
    rt_device_t dev;
    struct rt_sensor_data data;

    /* find sensor device */
    dev = rt_device_find(SAMPLE_SENSOR_NAME);
    /* Open sensor devices in read-only and polling mode */
    rt_device_open(dev, RT_DEVICE_FLAG_RDWR);

    if (rt_device_read(dev, 0, &data, 1) == 1)
    {
        rt_kprintf("acce: x:%5d, y:%5d, z:%5d, timestamp:%5d\n", data.data.acce.x, data.data.acce.y,
      data.data.acce.z, data.timestamp);
    }
    rt_device_close(dev);

    return RT_EOK;
}
```

1.4.2.20.2.3 Control Sensor Device

By command control words, the application program can configure the sensor device through the following functions:

```
rt_err_t rt_device_control(rt_device_t dev, rt_uint8_t cmd, void* arg);
```

Parameter	Description
dev	device handle
cmd	command control word, see below for more details.
arg	the parameters of command control word, see below for more details.
Return	——
RT_EOK	success
-RT_ENOSYS	faileddevice is NULL

Parameter	Description
other err	failed

cmd currently supports the following command control words:

```
#define  RT_SEN_CTRL_GET_ID             (0)  /* read device ID */
#define  RT_SEN_CTRL_GET_INFO           (1)  /* get device information */
#define  RT_SEN_CTRL_SET_RANGE          (2)  /* Setting the measuring range of the sensor */
#define  RT_SEN_CTRL_SET_ODR            (3)  /* Setting the Output Rate of Sensor Dataunit is HZ */
#define  RT_SEN_CTRL_SET_MODE           (4)  /* Setting up working mode */
#define  RT_SEN_CTRL_SET_POWER          (5)  /* Setting up power mode */
#define  RT_SEN_CTRL_SELF_TEST          (6)  /* selfcheck */
```

Get device information

```
struct rt_sensor_info info;
rt_device_control(dev, RT_SEN_CTRL_GET_INFO, &info);
LOG_I("vendor :%d", info.vendor);
LOG_I("model  :%s", info.model);
LOG_I("unit   :%d", info.unit);
LOG_I("intf_type :%d", info.intf_type);
LOG_I("period_min:%d", info.period_min);
```

Read Device ID

```
rt_uint8_t reg = 0xFF;
rt_device_control(dev, RT_SEN_CTRL_GET_ID, &reg);
LOG_I("device id: 0x%x!", reg);
```

Setting the measuring range of the sensor

The unit that sets the measuring range of the sensor is the unit that is provided when the device is registered.

```
rt_device_control(dev, RT_SEN_CTRL_SET_RANGE, (void *)1000);
```

Setting the Output Rate of Sensor Data

Set the output rate to 100Hz and call the following interface.

```
rt_device_control(dev, RT_SEN_CTRL_SET_ODR, (void *)100);
```

Setting up working mode

```
/* Set the working mode to polling mode */
rt_device_control(dev, RT_SEN_CTRL_SET_MODE, (void *)RT_SEN_MODE_POLLING);
/* Set working mode to interrupt mode */
rt_device_control(dev, RT_SEN_CTRL_SET_MODE, (void *)RT_SEN_MODE_INT);
/* Set working mode to FIFO mode */
rt_device_control(dev, RT_SEN_CTRL_SET_MODE, (void *)RT_SEN_MODE_FIFO);
```

Setting up power mode

```
/* Set power mode to power-off mode */
rt_device_control(dev, RT_SEN_CTRL_SET_POWER, (void *)RT_SEN_POWER_DOWN);
/* Set power mode to normal mode */
rt_device_control(dev, RT_SEN_CTRL_SET_POWER, (void *)RT_SEN_POWER_NORMAL);
/* Setting Power Mode to Low Power Consumption Mode */
rt_device_control(dev, RT_SEN_CTRL_SET_POWER, (void *)RT_SEN_POWER_LOW);
/* Setting Power Mode to High Performance Mode */
rt_device_control(dev, RT_SEN_CTRL_SET_POWER, (void *)RT_SEN_POWER_HIGH);
```

Device self-inspection

```
int test_res;
/* Control equipment self-check and return the results. Returning RT_EOK indicates success of self-check and
      other values indicate failure of self-check. */
rt_device_control(dev, RT_SEN_CTRL_SELF_TEST, &test_res);
```

1.4.2.20.2.4 Setting Reveive Callback Fuction

Data reception instructions can be set by following functions. When the sensor receives data, it notifies the upper application thread that data arrives:

```
rt_err_t rt_device_set_rx_indicate(rt_device_t dev, rt_err_t (*rx_ind)(rt_device_t dev,rt_size_t size));
```

Parameter	Description
dev	device handle
rx_ind	Callback function pointer
dev	device handle(parameter of callback function)
size	buffer size(parameter of callback function)
Return	——
RT_EOK	Successful setup

The callback function of the function is provided by the user. If the sensor is opened in interrupt mode, as the sensor receives data the callback function will be called. The data size of the buffer will be placed in the `size` parameter, and the sensor device handle will be placed in the `dev` parameter for users to obtain.
Generally, receiving callback function can send a semaphore or event to inform sensor data processing thread that data arrives. The use example is as follows:

```
#define SAMPLE_SENSOR_NAME       "acce_st"  /* sensor device name */
static rt_device_t dev;             /* sensoe device handle*/
static struct rt_semaphore rx_sem;    /* The semaphore used to receive messages */

/* Callback function for receiving data */
static rt_err_t sensor_input(rt_device_t dev, rt_size_t size)
{
    /* When the sensor receives the data, it generates an interrupt, calls the callback function, and sends
      the semphore . */
    rt_sem_release(&rx_sem);

    return RT_EOK;
}

static int sensor_sample(int argc, char *argv[])
{
    dev = rt_device_find(SAMPLE_SENSOR_NAME);

    /* Open Sensor Device in Interrupt Receive and Poll Send Mode */
    rt_device_open(dev, RT_DEVICE_FLAG_INT_RX);
    /* init semphore */
    rt_sem_init(&rx_sem, "rx_sem", 0, RT_IPC_FLAG_FIFO);

    /* setting reveive callback function */
    rt_device_set_rx_indicate(dev, sensor_input);
}
```

1.4.2.20.2.5 Read Data of Sensor Device

The following functions can be called to read the data received by the sensor:

```
rt_size_t rt_device_read(rt_device_t dev, rt_off_t pos, void* buffer, rt_size_t size);
```

Parameter	Description
dev	device handle
pos	Read data offset, sensor does not use this parameter
buffer	Buffer pointer, read data will be saved in the buffer
size	Size of read data
Return	——
Real size of read data	Returns the number of read data
0	The errno of the current thread needs to be read to determine the error status

The sensor uses the interrupt receiving mode and cooperates with the receiving callback function as follows:

```
static rt_device_t dev;             /* sensor device handle */
static struct rt_semaphore rx_sem;    /* The semaphore used to receive messages */

/* Threads receiving data */
static void sensor_irq_rx_entry(void *parameter)
{
    rt_device_t dev = parameter;
    struct rt_sensor_data data;
    rt_size_t res;

    while (1)
    {
        rt_sem_take(rx_sem, RT_WAITING_FOREVER);

        res = rt_device_read(dev, 0, &data, 1);
        if (res == 1)
        {
            sensor_show_data(dev, &data);
        }
    }
}
```

The sensor uses FIFO receiving mode and cooperates with receiving callback function as follows:

```
static rt_sem_t sensor_rx_sem = RT_NULL;
rt_err_t rx_cb(rt_device_t dev, rt_size_t size)
{
    rt_sem_release(sensor_rx_sem);
    return 0;
}
static void sensor_fifo_rx_entry(void *parameter)
{
    rt_device_t dev = parameter;
    struct rt_sensor_data data;
    rt_size_t res, i;

    data = rt_malloc(sizeof(struct rt_sensor_data) * 32);

    while (1)
    {
        rt_sem_take(sensor_rx_sem, RT_WAITING_FOREVER);

        res = rt_device_read(dev, 0, data, 32);
        for (i = 0; i < res; i++)
        {
            sensor_show_data(dev, &data[i]);
        }
    }
}
int main(void)
{
    static rt_thread_t tid1 = RT_NULL;
    rt_device_t dev;
    struct rt_sensor_data data;

    sensor_rx_sem = rt_sem_create("sen_rx_sem", 0, RT_IPC_FLAG_FIFO);
    tid1 = rt_thread_create("sen_rx_thread",
                            sensor_fifo_rx_entry, dev,
                            1024,
                            15, 5);
    if (tid1 != RT_NULL)
        rt_thread_startup(tid1);

    dev = rt_device_find("acce_st");
    rt_device_set_rx_indicate(dev, rx_cb);
    rt_device_open(dev, RT_SEN_FLAG_FIFO);
    return RT_EOK;
}
```

1.4.2.20.2.6 Close Sensor Device

When the application completes the sensor operation, the sensor device can be closed by the following functions:

```
rt_err_t rt_device_close(rt_device_t dev);
```

Parameter	Description
dev	device handle
Return	——
RT_EOK	The equipment was closed successfully.

Parameter	Description
-RT_ERROR	The device has been completely shut down and cannot be closed repeatedly.
other err	failed to close th device

Closing the device interface and opening the device interface should be used in pairs, opening the primary device should close the primary device, so that the device will be completely closed, otherwise the device is still in an open state.

1.4.2.20.3 Example Code for Sensor Device

The specific use of sensor devices can be referred to the following sample code, the main steps of the sample code are as follows:

1. Find the sensor device first and get the device handle.
2. Open the sensor device by polling.
3. Read the data five times in a row and print it out.
4. Close the sensor device.

This sample code is not limited to a specific BSP. According to the BSP registered sensor device, input different dev_name to run.

```
/*
 * Program List: This is a routine for sensor devices
 * The routine exports the sensor_sample command to the control terminal
 * Command Call Formatsensor_sample dev_name
 * Command Interpretation: The second parameter of the command is the name of the sensor device to be used.
 * Program function: Open the corresponding sensor, and then read the data five times in a row and print it
      out.
*/

#include "sensor.h"

static void sensor_show_data(rt_size_t num, rt_sensor_t sensor, struct rt_sensor_data *sensor_data)
{
    switch (sensor->info.type)
    {
    case RT_SENSOR_CLASS_ACCE:
        rt_kprintf("num:%3d, x:%5d, y:%5d, z:%5d, timestamp:%5d\n", num, sensor_data->data.acce.x,
      sensor_data->data.acce.y, sensor_data->data.acce.z, sensor_data->timestamp);
        break;
    case RT_SENSOR_CLASS_GYRO:
        rt_kprintf("num:%3d, x:%8d, y:%8d, z:%8d, timestamp:%5d\n", num, sensor_data->data.gyro.x,
      sensor_data->data.gyro.y, sensor_data->data.gyro.z, sensor_data->timestamp);
        break;
    case RT_SENSOR_CLASS_MAG:
        rt_kprintf("num:%3d, x:%5d, y:%5d, z:%5d, timestamp:%5d\n", num, sensor_data->data.mag.x,
      sensor_data->data.mag.y, sensor_data->data.mag.z, sensor_data->timestamp);
        break;
    case RT_SENSOR_CLASS_HUMI:
        rt_kprintf("num:%3d, humi:%3d.%d%%, timestamp:%5d\n", num, sensor_data->data.humi / 10,
      sensor_data->data.humi % 10, sensor_data->timestamp);
        break;
    case RT_SENSOR_CLASS_TEMP:
        rt_kprintf("num:%3d, temp:%3d.%dC, timestamp:%5d\n", num, sensor_data->data.temp / 10,
      sensor_data->data.temp % 10, sensor_data->timestamp);
        break;
    case RT_SENSOR_CLASS_BARO:
        rt_kprintf("num:%3d, press:%5d, timestamp:%5d\n", num, sensor_data->data.baro,
      sensor_data->timestamp);
        break;
    case RT_SENSOR_CLASS_STEP:
        rt_kprintf("num:%3d, step:%5d, timestamp:%5d\n", num, sensor_data->data.step,
      sensor_data->timestamp);
        break;
    default:
        break;
    }
}

static void sensor_sample(int argc, char **argv)
{
    rt_device_t dev = RT_NULL;
    struct rt_sensor_data data;
```

```
    rt_size_t res, i;

    /* Finding Sensor Devices in the System */
    dev = rt_device_find(argv[1]);
    if (dev == RT_NULL)
    {
        rt_kprintf("Can't find device:%s\n", argv[1]);
        return;
    }

    /* Open sensor devices in polling mode */
    if (rt_device_open(dev, RT_DEVICE_FLAG_RDWR) != RT_EOK)
    {
        rt_kprintf("open device failed!");
        return;
    }

    for (i = 0; i < 5; i++)
    {
        /* Read a data from a sensor */
        res = rt_device_read(dev, 0, &data, 1);
        if (res != 1)
        {
            rt_kprintf("read data failed!size is %d", res);
        }
        else
        {
            sensor_show_data(i, (rt_sensor_t)dev, &data);
        }
        rt_thread_mdelay(100);
    }
    /* Close the sensor device */
    rt_device_close(dev);
}
MSH_CMD_EXPORT(sensor_sample, sensor device sample);
```

1.4.2.21 Audio Device

1.4.2.21.1 Audio Introduction

The Audio device is a crucial component in embedded systems, responsible for audio data sampling and output. An Audio device typically consists of a data bus interface, control bus interface, audio codec (Codec), speaker, and microphone, as shown below:

1.4.2.21.1.1 API List

For more details, see Audio

1.4.2.21.1.2 Audio Device Features

The RT-Thread Audio device driver framework serves as the underlying layer of the Audio framework. It handles raw audio data acquisition/output, audio stream control, device management, volume adjustment, and hardware/Codec abstraction.

- **Interface**: Standard device interface (`open/close/read/control`).
- **Synchronous access mode**.
- **Supports playback and recording**.
- **Audio parameter management**.
- **Volume control**.

1.4.2.21.2 Accessing Audio Devices

1.4.2.21.2.1 Finding an Audio Device

Applications obtain a device handle using the Audio device name. The device lookup function is as follows:

```
rt_device_t rt_device_find(const char* name);
```

Parameter	Description
name	Audio device name
Return	——
Device handle	Returns the handle if found
RT_NULL	Device not found

Example usage:

```
#define SOUND_DEVICE_NAME    "sound0"    /* Audio device name */

static rt_device_t snd_dev;              /* Audio device handle */

/* Find Audio device by name and obtain handle */
snd_dev = rt_device_find(SOUND_DEVICE_NAME);
```

1.4.2.21.2.2 Opening an Audio Device

Applications can open/close devices using the device handle. To open a device:

```
rt_err_t rt_device_open(rt_device_t dev, rt_uint16_t oflags);
```

Parameter	Description
dev	Device handle
oflags	Device mode flags
Return	——
RT_EOK	Device opened successfully
-RT_EBUSY	Device cannot be reopened if registered with RT_DEVICE_FLAG_STANDALONE
-RT_EINVAL	Unsupported open flags
Other errors	Device open failure

Supported `oflags` values:

```
#define RT_DEVICE_OFLAG_WRONLY      0x002     /* Write-only mode for playback devices */
#define RT_DEVICE_FLAG_RDONLY       0x001     /* Read-only mode for recording devices */
```

Audio devices are categorized into playback (output) and recording (input). Playback devices use the write-only flag, while recording devices use read-only.

Example for opening a playback device:

```
rt_device_open(snd_dev, RT_DEVICE_OFLAG_WRONLY)
```

Example for opening a recording device:

```
rt_device_open(mic_dev, RT_DEVICE_FLAG_RDONLY)
```

1.4.2.21.2.3 Controlling an Audio Device

Applications configure Audio devices using control commands:

```
rt_err_t rt_device_control(rt_device_t dev, rt_uint8_t cmd, void* arg);
```

Parameter	Description
dev	Device handle
cmd	Control command (see below)
arg	Control parameter (see below)
Return	——
RT_EOK	Operation succeeded
-RT_ENOSYS	Failed (null device handle)
Other errors	Operation failed

Supported control commands:

```
/* AUDIO commands */
#define _AUDIO_CTL(a) (0x10 + a)

#define AUDIO_CTL_GETCAPS                   _AUDIO_CTL(1) /* Get device capabilities */
#define AUDIO_CTL_CONFIGURE                 _AUDIO_CTL(2) /* Configure device */
```

Device capability structure:

```
struct rt_audio_caps
{
    int main_type;                            /* Main command type */
    int sub_type;                             /* Sub command type */

    union
    {
        rt_uint32_t mask;
        int     value;                       /* Parameter value */
        struct rt_audio_configure config;    /* Audio configuration */
    } udata;
};
```

Setting Playback Parameters

Configure playback sample rate, channels, and bit depth:

```
struct rt_audio_caps caps;

caps.main_type               = AUDIO_TYPE_OUTPUT; /* Playback device */
caps.sub_type                = AUDIO_DSP_PARAM;   /* Set all audio parameters */
caps.udata.config.samplerate = 44100;             /* Sample rate */
caps.udata.config.channels   = 2;                 /* Channels */
caps.udata.config.samplebits = 16;                /* Bit depth */
rt_device_control(device, AUDIO_CTL_CONFIGURE, &caps);
```

Setting Playback Volume

Adjust master playback volume (0-100):

```
struct rt_audio_caps caps;

caps.main_type   = AUDIO_TYPE_MIXER;       /* Volume control type */
caps.sub_type    = AUDIO_MIXER_VOLUME;     /* Set master volume */
caps.udata.value = volume;                 /* Range: 0-100 */
rt_device_control(snd_dev, AUDIO_CTL_CONFIGURE, &caps);
```

Setting Recording Parameters

Configure recording sample rate, channels, and bit depth:

```
struct rt_audio_caps caps;

caps.main_type               = AUDIO_TYPE_INPUT;  /* Recording device */
caps.sub_type                = AUDIO_DSP_PARAM;   /* Set all audio parameters */
caps.udata.config.samplerate = 44100;             /* Sample rate */
caps.udata.config.channels   = 2;                 /* Channels */
caps.udata.config.samplebits = 16;                /* Bit depth */
rt_device_control(device, AUDIO_CTL_CONFIGURE, &caps);
```

Setting Recording Volume

Adjust microphone gain (0-100):

```
struct rt_audio_caps caps;

caps.main_type = AUDIO_TYPE_MIXER;       /* Volume control type */
caps.sub_type  = AUDIO_MIXER_MIC;        /* Set microphone gain */
caps.udata.value = volume;               /* Range: 0-100 */
rt_device_control(player->device, AUDIO_CTL_CONFIGURE, &caps);
```

1.4.2.21.2.4 Writing Audio Data

Write data to a playback device:

```
rt_size_t rt_device_write(rt_device_t dev, rt_off_t pos, const void* buffer, rt_size_t size);
```

Parameter	Description
dev	Device handle

Parameter	Description
pos	Unused (reserved for offset)
buffer	Data buffer to write
size	Data size to write
Return	——
Bytes written	Actual bytes written (synchronous operation)

This synchronous function writes data to the device's internal buffer. The call blocks when the buffer is full.

1.4.2.21.2.5 Reading Audio Data

Read data from a recording device:

```
rt_size_t rt_device_read(rt_device_t dev, rt_off_t pos, void* buffer, rt_size_t size);
```

Parameter	Description
dev	Device handle
pos	Unused (reserved for offset)
buffer	Buffer to store read data
size	Data size to read
Return	——
Bytes read	Actual bytes read (synchronous operation)
0	Check thread's errno for error status

This synchronous function reads data from the device's internal pipe. The call blocks if insufficient data is available.

1.4.2.21.2.6 Closing an Audio Device

Close the device after operations:

```
rt_err_t rt_device_close(rt_device_t dev);
```

Parameter	Description
dev	Device handle
Return	——
RT_EOK	Device closed successfully
-RT_ERROR	Device already closed
Other errors	Close failure

1.4.2.21.3 Audio Device Usage Example

Audio devices are used for playback and recording, often accompanied by audio file encoding and decoding. Below are examples of playing and recording WAV files. The complete code can be obtained from the RT-Thread WAV Player Package.

1.4.2.21.3.1 Playback

The main steps to play an audio file are as follows:

1. First, find the Audio device and obtain the device handle.
2. Open the Audio device in write-only mode.

3. Set audio parameters (sampling rate, channels, etc.).
4. Decode the audio file data.
5. Write the audio file data.
6. After playback is complete, close the device.

```
#include <rtthread.h>
#include <rtdevice.h>
#include <dfs_posix.h>

#define BUFSZ   1024
#define SOUND_DEVICE_NAME    "sound0"    /* Audio device name */
static rt_device_t snd_dev;              /* Audio device handle */

struct RIFF_HEADER_DEF
{
    char riff_id[4];     // 'R','I','F','F'
    uint32_t riff_size;
    char riff_format[4]; // 'W','A','V','E'
};

struct WAVE_FORMAT_DEF
{
    uint16_t FormatTag;
    uint16_t Channels;
    uint32_t SamplesPerSec;
    uint32_t AvgBytesPerSec;
    uint16_t BlockAlign;
    uint16_t BitsPerSample;
};

struct FMT_BLOCK_DEF
{
    char fmt_id[4];    // 'f','m','t',' '
    uint32_t fmt_size;
    struct WAVE_FORMAT_DEF wav_format;
};

struct DATA_BLOCK_DEF
{
    char data_id[4];     // 'R','I','F','F'
    uint32_t data_size;
};

struct wav_info
{
    struct RIFF_HEADER_DEF header;
    struct FMT_BLOCK_DEF   fmt_block;
    struct DATA_BLOCK_DEF  data_block;
};

int wavplay_sample(int argc, char **argv)
{
    int fd = -1;
    uint8_t *buffer = NULL;
    struct wav_info *info = NULL;
    struct rt_audio_caps caps = {0};

    if (argc != 2)
    {
        rt_kprintf("Usage:\n");
        rt_kprintf("wavplay_sample song.wav\n");
        return 0;
    }

    fd = open(argv[1], O_RDONLY);
    if (fd < 0)
    {
        rt_kprintf("open file failed!\n");
        goto __exit;
    }

    buffer = rt_malloc(BUFSZ);
    if (buffer == RT_NULL)
        goto __exit;

    info = (struct wav_info *) rt_malloc(sizeof * info);
    if (info == RT_NULL)
        goto __exit;

    if (read(fd, &(info->header), sizeof(struct RIFF_HEADER_DEF)) <= 0)
        goto __exit;
    if (read(fd, &(info->fmt_block),  sizeof(struct FMT_BLOCK_DEF)) <= 0)
```

```
        goto __exit;
    if (read(fd, &(info->data_block), sizeof(struct DATA_BLOCK_DEF)) <= 0)
        goto __exit;

    rt_kprintf("wav information:\n");
    rt_kprintf("samplerate %d\n", info->fmt_block.wav_format.SamplesPerSec);
    rt_kprintf("channel %d\n", info->fmt_block.wav_format.Channels);

    /* Find the Audio device by name and obtain the device handle */
    snd_dev = rt_device_find(SOUND_DEVICE_NAME);

    /* Open the Audio playback device in write-only mode */
    rt_device_open(snd_dev, RT_DEVICE_OFLAG_WRONLY);

    /* Set audio parameters such as sampling rate, channels, and bit depth */
    caps.main_type               = AUDIO_TYPE_OUTPUT;                          /* Output type (playback
      device) */
    caps.sub_type                = AUDIO_DSP_PARAM;                            /* Set all audio parameters
      */
    caps.udata.config.samplerate = info->fmt_block.wav_format.SamplesPerSec;   /* Sampling rate */
    caps.udata.config.channels   = info->fmt_block.wav_format.Channels;        /* Channels */
    caps.udata.config.samplebits = 16;                                         /* Bit depth */
    rt_device_control(snd_dev, AUDIO_CTL_CONFIGURE, &caps);

    while (1)
    {
        int length;

        /* Read audio data from the file system */
        length = read(fd, buffer, BUFSZ);

        if (length <= 0)
            break;

        /* Write audio data to the Audio device */
        rt_device_write(snd_dev, 0, buffer, length);
    }

    /* Close the Audio device */
    rt_device_close(snd_dev);

__exit:

    if (fd >= 0)
        close(fd);

    if (buffer)
        rt_free(buffer);

    if (info)
        rt_free(info);

    return 0;
}

MSH_CMD_EXPORT(wavplay_sample,  play wav file);
```

1.4.2.21.3.2 Recording

The main steps to record an audio file are as follows:

1. First, find the Audio device and obtain the device handle.
2. Open the Audio device in read-only mode.
3. Set audio parameters (sampling rate, channels, etc.).
4. Read data from the Audio device.
5. Process the recorded data.
6. After recording is complete, close the device.

```
#include <rtthread.h>
#include <rtdevice.h>
#include <dfs_posix.h>

#define RECORD_TIME_MS      5000
#define RECORD_SAMPLERATE   16000
#define RECORD_CHANNEL      2
#define RECORD_CHUNK_SZ     ((RECORD_SAMPLERATE * RECORD_CHANNEL * 2) * 20 / 1000)

#define SOUND_DEVICE_NAME    "mic0"      /* Audio device name */
```

```
static rt_device_t mic_dev;             /* Audio device handle */

struct wav_header
{
    char  riff_id[4];              /* "RIFF" */
    int   riff_datasize;           /* RIFF chunk data size, excluding riff_id[4] and riff_datasize, total -
      8 */
    char  riff_type[4];            /* "WAVE" */
    char  fmt_id[4];               /* "fmt " */
    int   fmt_datasize;            /* fmt chunk data size, 16 for PCM */
    short fmt_compression_code;    /* 1 for PCM */
    short fmt_channels;            /* 1(mono) or 2(stereo) */
    int   fmt_sample_rate;         /* samples per second */
    int   fmt_avg_bytes_per_sec;   /* sample_rate * channels * bit_per_sample / 8 */
    short fmt_block_align;         /* number of bytes per sample, bit_per_sample * channels / 8 */
    short fmt_bit_per_sample;      /* bits of each sample(8,16,32). */
    char  data_id[4];              /* "data" */
    int   data_datasize;           /* data chunk size, pcm_size - 44 */
};

static void wavheader_init(struct wav_header *header, int sample_rate, int channels, int datasize)
{
    memcpy(header->riff_id, "RIFF", 4);
    header->riff_datasize = datasize + 44 - 8;
    memcpy(header->riff_type, "WAVE", 4);
    memcpy(header->fmt_id, "fmt ", 4);
    header->fmt_datasize = 16;
    header->fmt_compression_code = 1;
    header->fmt_channels = channels;
    header->fmt_sample_rate = sample_rate;
    header->fmt_bit_per_sample = 16;
    header->fmt_avg_bytes_per_sec = header->fmt_sample_rate * header->fmt_channels *
      header->fmt_bit_per_sample / 8;
    header->fmt_block_align = header->fmt_bit_per_sample * header->fmt_channels / 8;
    memcpy(header->data_id, "data", 4);
    header->data_datasize = datasize;
}

int wavrecord_sample(int argc, char **argv)
{
    int fd = -1;
    uint8_t *buffer = NULL;
    struct wav_header header;
    struct rt_audio_caps caps = {0};
    int length, total_length = 0;

    if (argc != 2)
    {
        rt_kprintf("Usage:\n");
        rt_kprintf("wavrecord_sample file.wav\n");
        return -1;
    }

    fd = open(argv[1], O_WRONLY | O_CREAT);
    if (fd < 0)
    {
        rt_kprintf("open file for recording failed!\n");
        return -1;
    }
    write(fd, &header, sizeof(struct wav_header));

    buffer = rt_malloc(RECORD_CHUNK_SZ);
    if (buffer == RT_NULL)
        goto __exit;

    /* Find the Audio device by name and obtain the device handle */
    mic_dev = rt_device_find(SOUND_DEVICE_NAME);
    if (mic_dev == RT_NULL)
        goto __exit;

    /* Open the Audio recording device in read-only mode */
    rt_device_open(mic_dev, RT_DEVICE_OFLAG_RDONLY);

    /* Set audio parameters such as sampling rate, channels, and bit depth */
    caps.main_type               = AUDIO_TYPE_INPUT;                            /* Input type (recording
      device) */
    caps.sub_type                = AUDIO_DSP_PARAM;                             /* Set all audio parameters
      */
    caps.udata.config.samplerate = RECORD_SAMPLERATE;                           /* Sampling rate */
    caps.udata.config.channels   = RECORD_CHANNEL;                              /* Channels */
    caps.udata.config.samplebits = 16;                                          /* Bit depth */
    rt_device_control(mic_dev, AUDIO_CTL_CONFIGURE, &caps);

    while (1)
    {
        /* Read 20ms of audio data from the Audio device */
```

```
        length = rt_device_read(mic_dev, 0, buffer, RECORD_CHUNK_SZ);

        if (length)
        {
            /* Write the audio data to the file system */
            write(fd, buffer, length);
            total_length += length;
        }

        if ((total_length / RECORD_CHUNK_SZ) >  (RECORD_TIME_MS / 20))
            break;
    }

    /* Rewrite the WAV file header */
    wavheader_init(&header, RECORD_SAMPLERATE, RECORD_CHANNEL, total_length);
    lseek(fd, 0, SEEK_SET);
    write(fd, &header, sizeof(struct wav_header));
    close(fd);

    /* Close the Audio device */
    rt_device_close(mic_dev);

__exit:
    if (fd >= 0)
        close(fd);

    if (buffer)
        rt_free(buffer);

    return 0;
}
MSH_CMD_EXPORT(wavrecord_sample, record voice to a wav file);
```

1.4.2.22 Open Firmware (OFW)

1.4.2.22.1 Open Firmware (OFW) overview

RT-Thread **OFW** parses the boot **DTB** into **`struct rt_ofw_node`**, then exposes property I/O, addressing, interrupts, and platform device enumeration. Implementation mirrors **`components/drivers/ofw/`** source layout.

Source / header	Documentation
`base.c`, **`ofw.c`** — `ofw.h`	OFW nodes and properties
`fdt.c` — `ofw_fdt.h`	OFW boot and FDT
`io.c` — `ofw_io.h`	OFW addressing and MMIO
`irq.c` — `ofw_irq.h`	OFW interrupts
`raw.c` — `ofw_raw.h`	OFW raw FDT helpers
`core/platform_ofw.c`	OFW platform enumeration
Build **DTS** → **DTB**	Devicetree Compiler

1.4.2.22.1.1 Kconfig

Option	Role
`RT_USING_OFW`	OFW + libfdt; requires **`RT_USING_DM`**
`RT_USING_BUILTIN_FDT`	Link **`RT_BUILTIN_FDT_PATH`** (default `rtthread.dtb`)

Option	Role
RT_FDT_EARLYCON_MSG_SIZE	Early console ring (KB, default 128)
RT_USING_OFW_BUS_RANGES_NUMBER	Cached **ranges** slots (4 or 8)

1.4.2.22.1.2 Typical driver flow

```
Bootloader DTB
     → rt_fdt_prefetch / scan_memory / unflatten    (@ref page_device_ofw_boot)
     → INIT_PLATFORM_EXPORT(platform_ofw_device_probe) (@ref page_device_ofw_platform)
     → driver probe(np): reg/irq/phandles
          rt_ofw_get_address + rt_dm_dev_iomap      (@ref page_device_ofw_io)
          rt_ofw_get_irq / rt_ofw_map_irq           (@ref page_device_ofw_irq)
          rt_ofw_parse_phandle_cells + rt_platform_ofw_request (@ref page_device_ofw_base)
```

1.4.2.22.1.3 Phandle consumer pattern

Most DM subsystems share the same parse sequence:

1. **rt_ofw_parse_phandle_cells(np, "clocks", "#clock-cells", index, &args)**
2. **provider_np = args.data**; if **!rt_ofw_data(provider_np)** → **rt_platform_ofw_request(provider_np)**
3. Use subsystem API (**rt_clk_get_by_index**, **rt_mbox_request_channel**, ...)

Property	#∗-cells	See also
clocks	**#clock-cells**	Clock framework (CLK)
resets	**#reset-cells**	reset DM
mboxes	**#mbox-cells**	Mailbox device model (DM)
io-channels	**#io-channel-cells**	IIO device model (DM)
nvmem-cells	**#nvmem-cell-cells**	NVMEM device model (DM)

rt_ofw_parse_object(np, obj_name, "#clock-cells") resolves composite objects in **rt_ofw_data** when one node exports multiple controllers.

1.4.2.22.1.4 Driver quick notes

Topic	Guidance
status	Use **rt_ofw_node_is_available** before attaching hardware
reg	Use **rt_ofw_io_∗** / **rt_ofw_get_address** — never hard-code parent **#address-cells**
IRQ	Direct **interrupts** vs bridge **interrupt-map** — see OFW interrupts
Refs	Pair **rt_ofw_node_get** / **rt_ofw_node_put** when caching **np**
compatible	Most specific string first in DTS and **rt_ofw_node_id** table

1.4.2.22.1.5 Normative references

- Device Tree Specification
- **components/drivers/include/dt-bindings/** — include from DTS via cpp

1.4.2.22.1.6 See also

- components/drivers/ofw/ — implementation
- Platform bus
- Bus and driver binding (core)
- Device model helpers (dm.h)

1.4.2.23 OFW nodes and properties

1.4.2.23.1 OFW core (base.c, ofw.c)

Node tree, properties, phandles, matching, stubs, and **rt_ofw_parse_object**. Boot: OFW boot and FDT. IO: OFW addressing and MMIO. IRQ: OFW interrupts.
Headers: **drivers/ofw.h**. Sources: **components/drivers/ofw/base.c**, **ofw.c**.

1.4.2.23.1.1 Core types

Type	Role
struct rt_ofw_prop	Property list entry: **name**, **length**, **value**
struct rt_ofw_node	**full_name**, **phandle**, **props**, tree links, **ref**, **flags**, **rt_data**, optional **dev**
struct rt_ofw_cell_args	Parsed phandle + specifier (**args[]**, max **RT_OFW_MAX_CELL_ARGS**)
struct rt_ofw_node_id	**compatible** / **name** / **type** match table row
struct rt_ofw_stub	Early probe: **ids**, **handler(np, id)**

Node flags (RT_OFW_F_*)

Flag	Meaning
RT_OFW_F_SYSTEM	Meta node (/, /cpus, /chosen, ...) — platform walk skips
RT_OFW_F_READLY	Stub or driver already bound
RT_OFW_F_PLATFORM	**rt_platform_device** created
RT_OFW_F_OVERLAY	From applied DT overlay

APIs: **rt_ofw_node_set_flag**, **rt_ofw_node_test_flag**, **rt_ofw_node_clear_flag**.

1.4.2.23.1.2 Lifetime

API	Role
rt_ofw_node_get / **rt_ofw_node_put**	Reference count on **np**
rt_ofw_node_destroy	Tear down subtree when applicable
rt_ofw_data(np)	Driver private pointer in **np->rt_data**

1.4.2.23.1.3 Finding nodes

API	Role
rt_ofw_find_node_by_path	Absolute path (/soc/uart@1000)
rt_ofw_find_node_by_compatible	DFS from **from** (**RT_NULL** = whole tree)
rt_ofw_find_node_by_ids / **_r**	Match **rt_ofw_node_id** table
rt_ofw_find_node_by_phandle	Resolve numeric phandle
rt_ofw_find_node_by_name, **_by_prop**, **_by_tag**, **_by_type**	Other keys
rt_ofw_get_parent, **rt_ofw_get_next_***	Walk tree
rt_ofw_foreach_* **macros**	Prefer macros in **ofw.h** over manual sibling loops

Availability: **rt_ofw_node_is_available** (**status**), **rt_ofw_machine_is_compatible** (root).

1.4.2.23.1.4 Properties

API	Role
rt_ofw_get_prop	Find property by name
rt_ofw_prop_read_raw	Blob + length
rt_ofw_prop_read_u8/u16/u32/s32/u64/string/bool	Typed index 0
rt_ofw_prop_read_*_index, ***_array_index**	Multi-element
rt_ofw_prop_count_of_u32 (etc.)	Element count
rt_ofw_append_prop	Runtime append (rare)

1.4.2.23.1.5 Phandles

API	Role
rt_ofw_parse_phandle	Single phandle property by index
rt_ofw_parse_phandle_cells	List + **#*-cells** → **rt_ofw_cell_args**
rt_ofw_count_phandle_cells	Entry count in list
rt_ofw_parse_object	Find named object in composite **rt_ofw_data**

ofw.c registers **#clock-cells**, **#reset-cells**, **#power-domain-cells** object layouts when those subsystems are enabled.

1.4.2.23.1.6 Matching and stubs

API / macro	Role
rt_ofw_node_match, **rt_ofw_prop_match**	Match **compatible** against **rt_ofw_node_id[]**
RT_OFW_STUB_EXPORT	Link stub into **.rt_ofw_data.stub.***
rt_ofw_stub_probe_range	Run stubs on a node (class ranges via **RT_OFW_STUB_RANGE_EXPORT**)

Used by early serial, hwcache, and other non-platform bindings.

1.4.2.23.1.7 Aliases and CPU

API	Role
rt_ofw_get_alias_node, **rt_ofw_get_alias_id**	**/aliases** resolution
rt_ofw_get_cpu_node, **rt_ofw_foreach_cpu_node**	CPU nodes under **/cpus**
rt_ofw_get_cpu_id, **rt_ofw_get_cpu_hwid**	MPIDR / hardware ID
rt_ofw_map_id	Generic ∗**-map** / ∗**-map-mask** lookup

ofw_alias_scan runs during unflatten (**base.c**).

1.4.2.23.1.8 Console and debug

API	Role
rt_ofw_console_setup	Console from **/chosen**
rt_ofw_node_dump_dts	Print subtree as DTS-like text (**RT_USING_CONSOLE**)

1.4.2.23.1.9 Pitfalls

Issue	Mitigation
Leaked **np**	Always **put** after **get**
Wrong phandle index	Check **rt_ofw_count_phandle_cells** and ∗**-names**
compatible order	Most specific first in driver **ids[]**
rt_ofw_data before probe	**rt_platform_ofw_request** on provider

1.4.2.23.1.10 See also

- Open Firmware (OFW)
- OFW platform enumeration
- OFW addressing and MMIO
- OFW interrupts

1.4.2.24 OFW boot and FDT

1.4.2.24.1 FDT bring-up (fdt.c)

Raw **DTB** → **memblock** → unflattened **rt_ofw_node** tree. After unflatten, use **rt_ofw_**∗ (OFW nodes and properties), not **rt_fdt_**∗, unless still touching the blob.
Header: **drivers/ofw_fdt.h**. Source: **components/drivers/ofw/fdt.c**.

1.4.2.24.1.1 Boot sequence

```
rt_fdt_prefetch(fdt)
    → rt_fdt_scan_root()
    → rt_fdt_scan_memory()      /* memblock */
    → rt_fdt_scan_initrd()      /* optional */
    → rt_fdt_unflatten()        /* rt_ofw_node tree */
    → INIT_PLATFORM_EXPORT(platform_ofw_device_probe)
```

Global nodes (**ofw_internal.h**): **ofw_node_root**, **ofw_node_cpus**, **ofw_node_chosen**, **ofw_node_aliases**, **ofw_node_reserved_memory**.

1.4.2.24.1.2 API (blob phase)

API	Role
rt_fdt_prefetch	Store **_fdt**, validate header, **rt_fdt_scan_root**
rt_fdt_scan_root	Root **#address-cells** / **#size-cells**
rt_fdt_scan_memory	**device_type = "memory"** → **rt_memblock_add_memory**
rt_fdt_scan_initrd	Reserve initrd range
rt_fdt_unflatten	Build in-memory tree; **ofw_alias_scan**; phandle hash
rt_fdt_unflatten_single	Unflatten with explicit **fdt** pointer
rt_fdt_device_is_available	**status** on blob node (pre-unflatten)

Chosen / bootargs

API	Role
rt_fdt_scan_chosen_stdout	**stdout-path**
rt_fdt_bootargs_select	Parse **bootargs** token
rt_ofw_bootargs_select	Same on unflattened tree

Early console

API / macro	Role
RT_FDT_EARLYCON_EXPORT	Register **rt_fdt_earlycon_id** in linker section
rt_fdt_earlycon_output	Print before UART driver probe
rt_fdt_earlycon_kick	Hand off to real console

libfdt helpers

API	Role
rt_fdt_read_number, **rt_fdt_next_cell**	Cell decoding
rt_fdt_translate_address	Address on raw blob

1.4.2.24.1.3 Memory and reserved regions

Source	Effect
DTB **/memreserve**	**rt_memblock_reserve_memory**
/reserved-memory nodes	Validated **reg** (+ **ranges**)
rt_fdt_scan_initrd	Initrd carve-out

Unflatten marks system nodes with **RT_OFW_F_SYSTEM** | **RT_OFW_F_READLY** so they are not platform devices.

1.4.2.24.1.4 Builtin DTB

RT_USING_BUILTIN_FDT embeds **RT_BUILTIN_FDT_PATH**. Build flow: Devicetree Compiler.

1.4.2.24.1.5 Pitfalls

Issue	Mitigation
rt_fdt_∗ after unflatten	Prefer **rt_ofw_∗** for properties
Missing **/cpus**	**RT_ASSERT** in unflatten — BSP must supply
Memory scan skipped	Call **scan_memory** before allocator use

1.4.2.24.1.6 See also

- Open Firmware (OFW)
- OFW platform enumeration
- OFW raw FDT helpers
- Devicetree Compiler

1.4.2.25 OFW addressing and MMIO

1.4.2.25.1 Addressing and I/O (io.c)

Decode **reg**, follow **ranges** / **dma-ranges**, and **ioremap**. Header: **drivers/ofw_io.h**.

1.4.2.25.1.1 Address cells

API	Role
rt_ofw_bus_addr_cells / **rt_ofw_bus_size_cells**	Walk parents for bus **#address-cells**
rt_ofw_io_addr_cells / **rt_ofw_io_size_cells**	Parent cells used to decode this node's **reg**

rt_ofw_get_address applies **rt_ofw_translate_address** so returned CPU physical addresses include parent **ranges**.

1.4.2.25.1.2 reg decoding

API	Role
rt_ofw_get_address_count	Number of **reg** tuples
rt_ofw_get_address(np, index, &addr, &size)	One tuple (translated base + size)
rt_ofw_get_address_by_name	Match **reg-names**
rt_ofw_get_address_array	Batch decode

Example in **probe**:

```
rt_uint64_t base, size;
if (rt_ofw_get_address(np, 0, &base, &size) != RT_EOK)
    return -RT_ERROR;
pdev->iomem = rt_dm_dev_iomap(np, 0);  /* or rt_ofw_iomap(np, 0) */
```

1.4.2.25.1.3 Translation

API	Role
`rt_ofw_translate_address(np, range↩ _type, addr)`	Follow `ranges` or `dma-ranges` (`range_type` = `"ranges"` / `"dma-ranges"`, or `RT_NULL`)
`rt_ofw_reverse_address`	Child → parent bus address
`rt_ofw_translate_dma2cpu`	DMA bus → CPU physical (inline)
`rt_ofw_translate_cpu2dma`	CPU → DMA bus (inline)

`io.c` caches up to `RT_USING_OFW_BUS_RANGES_NUMBER` `ranges` tables for performance.

1.4.2.25.1.4 MMIO map

API	Role
`rt_ofw_iomap(np, index)`	`ioremap` for `reg` index
`rt_ofw_iomap_by_name`	Named `reg-names` segment

Prefer `rt_dm_dev_iomap` in DM drivers when `struct rt_device` is available (Device model helpers (dm.h)).

1.4.2.25.1.5 Pitfalls

Issue	Mitigation
Wrong `#address-cells`	Always use `rt_ofw_io_*`, not constants
`reg` without `ranges` on bus	Translation may be identity — verify DTS
Multiple `reg`	Index 0 is not always the only MMIO bank
`reg-names` mismatch	`rt_ofw_get_address_by_name` returns `-RT_EEMPTY`

1.4.2.25.1.6 See also

- Open Firmware (OFW)
- OFW nodes and properties
- OFW boot and FDT
- Platform bus

1.4.2.26 OFW interrupts

1.4.2.26.1 Interrupts (`irq.c`)

Parse `interrupts`, `interrupt-parent`, and `interrupt-map` into logical IRQ numbers for PIC overview / `rt_pic_*`.
Header: `drivers/ofw_irq.h`. Source: `components/drivers/ofw/irq.c`.

1.4.2.26.1.1 Two wiring styles

Style	DTS	Typical use
Direct	`interrupts` + optional `interrupt-parent`	SoC device on GIC
Mapped	`interrupt-map` + `interrupt-map-mask` on bridge	PCI, AMBA behind nexus

1.4.2.26.1.2 API

API	Role
rt_ofw_irq_cells(np)	**#interrupt-cells** of interrupt controller
rt_ofw_find_irq_parent	Walk **interrupt-parent**; returns parent **np** and cell count
rt_ofw_parse_irq_cells(np, index, &args)	Parse **interrupts** entry → **rt_ofw_cell_args**
rt_ofw_parse_irq_map(np, &args)	Match **interrupt-map** row for this device
rt_ofw_map_irq(&args)	Final logical IRQ number via PIC layer
rt_ofw_get_irq_count	Number of interrupt specifiers
rt_ofw_get_irq(np, index)	Convenience: parse + map in one call
rt_ofw_get_irq_by_name	**interrupt-names** lookup

1.4.2.26.1.3 Recommended probe pattern

```
struct rt_ofw_cell_args irq_args;
int irq;

if (rt_ofw_get_prop(np, "interrupt-map", RT_NULL))
{
    if (rt_ofw_parse_irq_map(np, &irq_args) != RT_EOK)
        return -RT_ERROR;
}
else
{
    if (rt_ofw_parse_irq_cells(np, 0, &irq_args) != RT_EOK)
        return -RT_ERROR;
}

irq = rt_ofw_map_irq(&irq_args);
if (irq < 0)
    return irq;

rt_pic_irq_enable(irq);  /* subsystem-specific */
```

For multiple IRQs, loop **rt_ofw_get_irq(np, i)** or use **interrupt-names**.

1.4.2.26.1.4 interrupt-map (bridge)

Each map entry contains (per spec comment in **irq.c**):

1. Child unit address (**#address-cells** of bridge)
2. Child interrupt specifier (**#interrupt-cells** of bridge)
3. **interrupt-parent** phandle
4. Parent unit address
5. Parent interrupt specifier

interrupt-map-mask masks bits before compare. PCI host bridges are the common case — see also PCI and device tree.

1.4.2.26.1.5 Pitfalls

Issue	Mitigation
Missing **interrupt-parent**	Must inherit from bus or use **interrupt-map**
Using **rt_ofw_get_irq** on PCI endpoint	Often need **parse_irq_map** first
Cell count mismatch	Verify parent **#interrupt-cells**
IRQ vs trigger type	Specifier may include flags — pass full **irq_args** to PIC driver

1.4.2.26.1.6 See also

- Open Firmware (OFW)
- OFW nodes and properties
- OFW addressing and MMIO
- PIC overview
- PCI and device tree

1.4.2.27 OFW raw FDT helpers

1.4.2.27.1 Raw FDT editing (raw.c)

Optional **in-place** libfdt helpers for firmware, loaders, or tools that modify the blob **before** **rt_fdt_unflatten**. Runtime drivers normally use OFW nodes and properties after unflatten.
Header: **drivers/ofw_raw.h**. Source: **components/drivers/ofw/raw.c**.

1.4.2.27.1.1 When to use

Use raw API	Use rt_ofw_*
Patch DTB in RAM pre-boot	Driver **probe** on live tree
Add **/chosen** properties from bootloader	Read **reg** / **interrupts**
Install initrd nodes	Platform enumeration

1.4.2.27.1.2 API highlights

API / macro	Role
fdt_add_subnode_possible	Add child if space in blob
fdt_add_mem_rsv_possible	Add memory reserve entry
fdt_setprop_cstring	Set string property
fdt_setprop_cells, **fdt_appendprop_cells**	U32 cell arrays (endian-safe)
fdt_setprop_uxx	8/16/32/64-bit property
fdt_getprop_u8 … **fdt_getprop_s32**	Typed read on blob offset
fdt_io_addr_cells, **fdt_io_size_cells**	Cells on raw node offset
fdt_install_initrd	Add **linux,initrd-*** under **chosen**

FDT_SIZE_MAX, **FDT_PADDING_SIZE** — growth limits when expanding tree.

1.4.2.27.1.3 Pitfalls

Issue	Mitigation
Blob too small	Reserve padding at build time; check return codes
Edit after unflatten	Tree and blob may diverge — rebuild or only edit pre-unflatten
Endianness	Use **fdt_setprop_cells**, not raw **fdt_setprop** for integers

1.4.2.27.1.4 See also

- OFW boot and FDT
- Devicetree Compiler
- libfdt: **components/drivers/ofw/libfdt/**

1.4.2.28 OFW platform enumeration

1.4.2.28.1 Platform bus enumeration (platform_ofw.c)

Walk the unflattened tree and register **struct rt_platform_device** for driver **probe**. Core OFW APIs: OFW nodes and properties.
Source: **components/drivers/core/platform_ofw.c**. Platform driver API: Platform bus.

1.4.2.28.1.1 Probe order

INIT_PLATFORM_EXPORT(platform_ofw_device_probe):

1. **/clocks** subtree (providers before consumers)
2. **/** root (depth-first)
3. **/firmware**
4. **/chosen/simple-framebuffer** (if present)

1.4.2.28.1.2 Child walk

For each **available** child of **parent_np**:

Skip when	Reason
np->dev set	Already instantiated
RT_OFW_F_SYSTEM or **RT_OFW_F_READLY**	Meta or bound node
No **compatible** and no real name	Anonymous container

compatible in platform_ofw_ids	Action
simple-bus, **simple-mfd**, **isa**, **arm**, **amba-bus**	Recurse **platform_ofw_device_probe_once(np)** first
Leaf	**alloc_ofw_platform_device** → **rt_platform_device_register**

ofw_device_rename: **reg**-based name (`lx.%.*s`) when decodable, else hierarchical path.

1.4.2.28.1.3 rt_platform_ofw_request

```
rt_err_t rt_platform_ofw_request(struct rt_ofw_node *np);
```

State	Behavior
No **np->dev**	Create pdev, register (runs **probe**)
dev without driver	**rt_bus_reload_driver_device**
Probed	**RT_EOK**

Call from consumers after **rt_ofw_parse_phandle_cells** when provider **rt_ofw_data** is NULL.

1.4.2.28.1.4 Other APIs

API	Role
rt_platform_ofw_device_probe_child	Register single child under non-root parent
rt_platform_ofw_free	Clear **RT_OFW_F_PLATFORM**, **rt_ofw_node_put**, free pdev

1.4.2.28.1.5 Pitfalls

Issue	Mitigation
Clock provider late	Ensure **/clocks** nodes probe first (built-in order)
No **rt_platform_ofw_request**	Provider never probes — phandle parse succeeds but **rt_ofw_data** NULL
Bus not in **platform_ofw_ids**	Children still register as leaf platform devices if **compatible** present

1.4.2.28.1.6 See also

- Open Firmware (OFW)
- OFW boot and FDT
- Platform bus
- Bus and driver binding (core)

1.4.2.29 Devicetree Compiler

1.4.2.29.1 Introduction to the DTC

Runtime driver APIs that consume the flattened device tree (**rt_ofw_**∗, **rt_fdt_**∗, MMIO and IRQ parsing) are documented under Open Firmware (OFW) and OFW boot and FDT.
Device Tree Compiler, dtc, takes as input a device-tree in a given format and outputs a device-tree in another format for booting kernels on embedded systems. Typically, the input format is "dts" (device-tree source), a human readable source format, and creates a "dtb" (device-tree binary), or binary format as output.

> If the dtc tool is not installed on your host system, the dtc module will guide you through the installation.

1.4.2.29.1.1 Generate DTS

When you have a DTB or FDT file from firmware or another runtime system, you might want to convert it into a DTS file for easier reading. You can do this in Python or your SConscript file. For example, assuming you have `dummy.dtb`:

```
import os, sys

RTT_ROOT = os.getenv('RTT_ROOT')
sys.path.append(RTT_ROOT + '/tools')

from building import *
import dtc

dtc.dtb_to_dts(RTT_ROOT, "dummy.dtb")
```

This will generate a dummy.dts in the current directory. If a file with the same name already exists, it will be replaced. To avoid overwriting, you can specify a different output name:

```
[...]

dtc.dtb_to_dts(RTT_ROOT, "dummy.dtb", "dummy-tmp.dts")
# or
dtc.dtb_to_dts(RTT_ROOT, "dummy.dtb", dts_name = "dummy-tmp.dts")
```

1.4.2.29.1.2 Generate DTB

Before generating a DTB, you may want to review the basics of DTS syntax and structure: DeviceTree Specification

Include and Macros

By default, dtc does not support C-style preprocessing (like cpp), but you can use the C preprocessor with your DTS files. Don't worry — our dtc module already includes this step.
If your DTS file uses dt-bindings headers or macros, you can write something like:

```
/*
 * Used "#include" if header file need preprocessor,
 * `components/drivers/include` and current directory path is default.
 */
#include <dt-bindings/size.h>
#include "dummy.dtsi"
/* Well, if dtsi is simple, you can use "/include/", it is supported by dtc */
/include/ "chosen.dtsi"

#define MMIO_BASE   0x10000
#define MMIO_SIZE   SIZE_GB
#define MEM_BASE    (MMIO_BASE + MMIO_SIZE)

#ifndef CPU_HARDID
#define CPU_HARDID  0
#endif

#ifndef SOC_INTC
#define SOC_INTC    intc_a
#endif

/ {
    #address-cells = <2>;
    #size-cells = <2>;
    /*
     * Macros after "&" will be replaced,
     * there will affect the interrupt controller in this SoC.
     */
    interrupt-parent = <&SOC_INTC>;

    [...]

    memory {
        /* When there is a calculation, please use "()" to include them */
        reg = <0x0 MEM_BASE 0x0 (3 * SIZE_GB)>;
        device_type = "memory";
    };

    cpus {
        #size-cells = <0>;
        #address-cells = <1>;

        /* Macros after "@" will be replaced */
        cpu0: cpu@CPU_HARDID {
            reg = <CPU_HARDID>;
            device_type = "cpu";
        };
    };

    /* Macros replace support phandle name, too */
    intc_a: intc-a {
        interrupt-controller;
```

```
    };

    intc_b: intc-b {
        interrupt-controller;
    };

    [...]
};
```

To generate the DTB:

```
import os, sys

RTT_ROOT = os.getenv('RTT_ROOT')
sys.path.append(RTT_ROOT + '/tools')

from building import *
import dtc

dtc.dts_to_dtb(RTT_ROOT, ["dummy.dts"])
```

To append more include paths, for example, SoC DM headers:

```
[...]

dtc.dts_to_dtb(RTT_ROOT, ["dummy.dts"], include_paths = ['dm/include', 'firmware'])
```

Multiple DTB

A single SoC may have different board variants. Example `dummy.dtsi` (common base):

```
/* SoC dummy */
/ {
    #address-cells = <2>;
    #size-cells = <2>;
    model = "Dummy SoC Board";

    [...]

    chosen {
        bootargs = "cma=8M coherent_pool=2M";
    };

    reserved-memory {
        #address-cells = <2>;
        #size-cells = <2>;

        isp_shm@100000 {
            reg = <0x0 0x100000 0x0 0x100000>;
        };

        dsp_shm@200000 {
            reg = <0x0 0x200000 0x0 0x100000>;
        };
    };

    dsp {
        status = "okay";
    };

    buddy {
        isp = <&{/reserved-memory/isp_shm@100000}>;
        dsp = <&{/reserved-memory/dsp_shm@200000}>;
    };

    uart0: uart {
        status = "disabled";
    };

    i2c0: i2c {
        status = "okay";
    };

    [...]
};
```

For a vendor-specific variant (Vendor A):

```
/* vendorA dummy */
#include "dummy.dtsi"

/ {
    /* No phandle name can modify in place */
    chosen {
        bootargs = "console=uart0 cma=8M coherent_pool=2M";
    };
};

/* Reference and modify direct if has phandle name */
```

```
&uart0 {
    status = "okay";
    pinctrl-0 = <&uart0_m1>;
};

&i2c0 {
    status = "disabled";
};
```

To remove nodes or properties (Vendor B):

```
/* vendorB dummy */
#include "dummy.dtsi"

/delete-node/ &dsp_shm;

/ {
    /* Delete in place if no phandle name */
    /delete-node/ dsp;

    /* Delete property */
    buddy {
        /delete-property/ dsp;
    };
};
```

To add new devices (Vendor C):

```
/* vendorC dummy */
#include "dummy.dtsi"

&i2c0 {
    rtc@0 {
        clock-frequency = <32768>;
    };
};
```

Build all DTBs together:

```
[...]

dtc.dts_to_dtb(RTT_ROOT, ["dummy-vendorA.dts", "dummy-vendorB.dts", "dummy-vendorC.dts"])
```

This will produce `dummy-vendorA.dtb`, `dummy-vendorB.dtb`, and `dummy-vendorC.dtb`

1.4.2.29.1.3 Warnings

DTC may produce warnings that are irrelevant or noisy. To suppress specific warnings:

```
[...]

dtc.dts_to_dtb(RTT_ROOT, ["dummy.dts"], ignore_warning = ["simple_bus_reg", "unit_address_vs_reg",
      "clocks_is_cell", "gpios_property"])
```

Make sure your DTS is valid!

1.4.2.29.1.4 Raw options

DTC provides additional command-line options (see dtc –help). You can pass raw options like this:

```
[...]

dtc.dtb_to_dts(RTT_ROOT, "dummy.dtb", options = "--quiet")
dtc.dts_to_dtb(RTT_ROOT, ["dummy.dts"], options = "--quiet")
```

1.4.2.30 ATA / AHCI

1.4.2.30.1 AHCI host abstraction (`ahci.h`)

This page documents **`struct rt_ahci_host`**, **`struct rt_ahci_ops`**, and per-port **`struct rt_ahci_port`** in **`drivers/ahci.h`**, as driven by **`rt_ahci_host_register`** / **`rt_ahci_host_unregister`** in `components/drivers/ata/ahci.c`.

1.4.2.30.1.1 `struct rt_ahci_host` (caller fills before `rt_ahci_host_register`)

```
 368:383:components/drivers/include/drivers/ahci.h
struct rt_ahci_host
{
    struct rt_scsi_host parent;

    int irq;
    void *regs;

    rt_size_t ports_nr;
```

```
    rt_uint32_t ports_map;
    struct rt_ahci_port ports[32];

    rt_uint32_t cap;
    rt_uint32_t max_blocks;

    const struct rt_ahci_ops *ops;
};
```

Member	Owner	Meaning
parent	**Caller**	Embedded **rt_scsi_host**; **parent.dev must be non-NULL** (used for rt_dma_alloc_coherent, etc.).
irq	**Caller**	Global host IRQ; the core installs the ISR with struct rt_ahci_host * as parameter.
regs	**Caller**	Virtual base of the HBA (AHCI 1.0) register block.
ports_nr	**Core**	Filled from **CAP.NP**; do not rely on the initial value.
ports_map	**Core (DT may override)**	Read from **PI**; if the node has **ports-implemented**, that property wins.
ports[]	**Core + ops**	See **Per-port state** below.
cap	**Core**	Read from **RT_AHCI_HBA_CAP** (masked); used with SG paths for **64-bit DMA** (**RT_AHCI_CAP_64**, etc.).
max_blocks	**Caller (may be 0)**	Max blocks per SCSI split; **0** is replaced with **0x80** at register time.
ops	**Caller**	**Required**; every callback is optional (**NULL** = core default), but failures in **host_init** / **port_init** / **port_dma_init** skip the port or log DMA errors.

1.4.2.30.1.2 struct rt_ahci_port (fields visible to ops)

```
 346:366:components/drivers/include/drivers/ahci.h
struct rt_ahci_port
{
    void *regs;

    void *dma;
    rt_ubase_t dma_handle;

    struct rt_ahci_cmd_hdr *cmd_slot;
    struct rt_ahci_sg *cmd_tbl_sg;
    void *cmd_tbl;
    rt_ubase_t cmd_tbl_dma;
    void *rx_fis;

    rt_uint32_t int_enabled;
    rt_size_t block_size;

    rt_uint16_t *ataid;

    rt_bool_t link;
    struct rt_completion done;
};
```

Member	Owner	Meaning
regs	**Core**	host->regs + 0x100 + port_index * 0x80; valid before **port_init**.
dma, dma_handle, cmd_slot, rx_fis, cmd_tbl, cmd_tbl_dma, cmd_tbl_sg	**Core**	After **link == RT_TRUE**, **RT_AHCI_DMA_SIZE** coherent memory is allocated and laid out; **CLB**/**CLBU**, **FB**/**FBU** programmed.
int_enabled	**Core**	**RT_AHCI_PORT_INTE_*** mask written to **PORT_INTE** after DMA engines start.

Member	Owner	Meaning
block_size	**Core**	**512** or **2048** from Identify / Inquiry and **SIG** / **ATAPI**.
ataid	**Core**	Identify buffer; used with **rt_ahci_ata_id**↩ _* helpers.
link	**Core**	**RT_TRUE** when link is up; ISR calls **port_isr** and **rt_completion_done** only if **link**.
done	**Core**	**rt_completion_init**; command path **rt_completion_wait(&port->done, ...)** after **PORT_CI**; **port_isr** must stay consistent with **done**.

In **port_init** / **port_dma_init** you may touch **port->regs** and other HW state not yet filled by the core. **Do not assume dma exists inside port_init** (DMA is allocated only after a successful link).

1.4.2.30.1.3 struct rt_ahci_ops (order inside rt_ahci_host_register)

```
 385:392:components/drivers/include/drivers/ahci.h
struct rt_ahci_ops
{
    rt_err_t (*host_init)(struct rt_ahci_host *host);
    rt_err_t (*port_init)(struct rt_ahci_host *host, struct rt_ahci_port *port);
    rt_err_t (*port_link_up)(struct rt_ahci_host *host, struct rt_ahci_port *port);
    rt_err_t (*port_dma_init)(struct rt_ahci_host *host, struct rt_ahci_port *port);
    rt_err_t (*port_isr)(struct rt_ahci_host *host, struct rt_ahci_port *port, rt_uint32_t isr);
};
```

Callback	When	Role
host_init	After HBA soft reset, **GHC.AHCI_EN**, **CAP** read/mask, **PI write**, before port enumeration.	Optional. HBA-level, clocks, PHY, vendor regs; non-**RT_EOK** fails **rt_ahci**↩ **_host_register**.
port_init	After **PORT_CMD** has **LIST_ON/FIS_**↩ **ON/START/FIS_RX** cleared and the port has quiesced, before **SPIN_UP**.	Optional. Port reset, **SCTL**, non-standard **SIG**; **port->regs** valid. Failure **continue**s the port.
port_link_up	After **port_init** and the core has set **SPIN_UP**.	Optional. If **NULL**, core polls **SSTS.DET** for **PHYRDY**. Must return **RT_EOK** when the link is usable.
port_dma_init	After **CLB/FB** and DMA buffers are programmed, before the core writes **PORT**↩ **_CMD** (ACTIVE, FIS_RX, POWER_ON, SPIN_UP, START).	Optional. Extra SoC DMA/port setup; errors are logged but the core still asserts **PORT_CMD**.
port_isr	Global ISR: bit set in **HBA_INTS**, **port->link**, **PORT_INTS** read into **isr**.	Optional. SoC-specific IRQ handling; core always **rt_completion_**↩ **done(&port->done)** and clears **PORT_INTS**.

Common integration pitfalls

- **port_init vs DMA**: **port->dma is not allocated yet** inside **port_init**—**only touch** **port->regs** and PHY-level setup there.
- **port_dma_init errors are logged only**: the core still enables **PORT_CMD**—**if your SoC cannot run without extra setup, fail earlier in** **port_link_up** instead.
- **port_isr**: if you override, ensure you still cooperate with **rt_completion_done** semantics expected by **ahci.c** command path, or commands hang.
- **64-bit DMA**: honor **RT_AHCI_CAP_64** when building PRDT—SG list addresses must match HBA capability.

1.4.2.30.1.4 Registration API

API	Role
`rt_ahci_host_register(host)`	Validates **`host`**, **`host->parent.dev`**, **`host->ops`**; HBA reset, ports, link, DMA, **`rt_scsi_host_register`**, IRQ.
`rt_ahci_host_unregister(host)`	Masks IRQ, frees per-port DMA, unregisters the SCSI host.

1.4.2.30.1.5 See also

- `components/drivers/include/drivers/ahci.h` (**`RT_AHCI_*`**, **`struct rt_ahci_cmd_hdr`**, **`struct rt_ahci_sg`**)
- `components/drivers/ata/ahci.c`

Same documentation pattern (fill structs + `ops` + register order): `documentation/6.components/device-driver/ufs/ufs.md`, `nvme/nvme.md`, `scsi/scsi.md` (section 1), `block/disk.md`, `dma/dma.md` (section 1).

1.4.2.31 Block layer (`blk`)

1.4.2.31.1 Block storage subsystem

The **blk** layer turns a single physical storage backend into one or more **RT-Thread block devices** that applications and **DFS** can open, read/write by **sector**, and control with standard IOCTLs. Hardware drivers implement **`struct rt_blk_disk_ops`** on an embedded **`struct rt_blk_disk`**; the core registers the disk, scans partition tables, and creates child volumes (**`struct rt_blk_device`**) such as **`sda1`** or **`mmcsd0p1`**.
Header: `components/drivers/include/drivers/blk.h`. IOCTLs and geometry: `components/drivers/include/drivers/classes/block.h`. Core: `components/drivers/block/blk.c`, `blk_dev.c`, `blk_partition.c`.

1.4.2.31.1.1 Architecture

```
[NVMe / SDIO / SCSI SD / ... driver]
              rt_blk_disk_ops (read/write/getgeome/...)

     struct rt_blk_disk   ← rt_hw_blk_disk_register()
              rt_blk_disk_probe_partition()  (inside register)

 struct rt_blk_device × N  ← disk_add_blk_dev() / blk_dev.c
              rt_device_read/write (sector-relative)

      DFS / applications / POSIX devio
```

Layer	Struct	Document
Physical disk	`struct rt_blk_disk`	Physical disk (`rt_blk_disk`)
Partition table	(on-disk metadata)	Disk partition probing
Volume (partition)	`struct rt_blk_device`	**This page** — application-facing nodes

Both disk and volume nodes use **`RT_Device_Class_Block`**. Applications normally open **volumes** (`sda1`), not the raw disk device, unless probing created a single full-disk volume with **`partno == 0`**.

1.4.2.31.1.2 Kconfig

Option	Role
`RT_USING_BLK`	Master switch for `components/drivers/block/`
`RT_BLK_PARTITION_EFI`	GPT / protective MBR parser (`partitions/efi.c`)

Option	Role
RT_BLK_PARTITION_DFS	Legacy DFS superblock layout on sector 0 (`partitions/dfs.c`, needs **RT_USING_DFS**)

rt_hw_blk_disk_register always calls **rt_blk_disk_probe_partition**; partition scan errors are **ignored** (register still succeeds).

1.4.2.31.2 Access block devices (applications)

Same usage model as other char/block devices: **find** → **open** → **read**/**write** → **control** → **close**. Sector index is **0-based within the opened device** (for a partition device, sector 0 is the start of that partition).

1.4.2.31.2.1 Find device

```
rt_device_t rt_device_find(const char *name);
```

Parameter	Description
`name`	Volume name, e.g. **sda0**, **sda1**, **mmcsd0**, **nvme0n1** (from DM naming — see Physical disk (`rt_blk_disk`))
Return	Device handle, or **RT_NULL** if not found

```
#define BLK_DEV_NAME   "sda1"
rt_device_t blk = rt_device_find(BLK_DEV_NAME);
```

1.4.2.31.2.2 Open / close

```
rt_err_t rt_device_open(rt_device_t dev, rt_uint16_t oflag);
rt_err_t rt_device_close(rt_device_t dev);
```

oflag	Notes
RT_DEVICE_OFLAG_RDONLY	Always valid
RT_DEVICE_OFLAG_WRONLY	Fails on disk/volume if backend is read-only (**ops->write == NULL** sets **read_only**)

Opening a **partition** device (`rt_blk_device`) internally **opens the parent disk** (`blk_dev_open` → `rt_device_open(&blk->disk->parent)`).

1.4.2.31.2.3 Read / write (sectors)

```
rt_ssize_t rt_device_read(rt_device_t dev, rt_off_t pos, void *buffer, rt_size_t size);
rt_ssize_t rt_device_write(rt_device_t dev, rt_off_t pos, const void *buffer, rt_size_t size);
```

For block class devices, **pos** is the **starting sector number**, **size** is **sector count** (not bytes).

Parameter	Partition volume	Whole disk
`pos`	0 … `sector_count - 1` (relative to partition)	0 … disk capacity 1
`size`	Sectors to transfer	Same
Return	Sectors actually transferred, or negative error	Same

```
#define SECTOR_SIZE   512
rt_uint8_t buf[SECTOR_SIZE];

if (rt_device_read(blk, 0, buf, 1) == 1)
{
    /* one sector at LBA 0 of this partition */
}
```

1.4.2.31.2.4 Control (IOCTL)

```
rt_err_t rt_device_control(rt_device_t dev, int cmd, void *args);
```

Standard commands (`classes/block.h`)

Command	`args` type	Partition volume	Raw disk
`RT_DEVICE_CTRL_BLK_GETGEOME`	`struct rt_device_blk_geometry *`	**`sector_count`** = partition size; **`bytes_per_sector`** from disk	Full disk geometry
`RT_DEVICE_CTRL_BLK_SYNC`	ignored	Flushes parent disk	**`ops->sync`** under lock
`RT_DEVICE_CTRL_BLK_ERASE`	ignored	Only if **`disk->partitions <= 1`**; else **`-RT_EIO`**	May remove all partitions then **`ops->erase`**
`RT_DEVICE_CTRL_BLK_AUTOREFRESH`	`rt_bool_t *` / pointer treated as enable flag	Delegates to disk if single partition	**`ops->autorefresh`**
`RT_DEVICE_CTRL_BLK_PARTITION`	`struct dfs_partition *`	Copies **`blk->partition`**	**`-RT_EINVAL`** on disk node

DFS helpers (`blk_dfs.h`)

Command	Role
`RT_DEVICE_CTRL_BLK_SSIZEGET`	Fills `*(rt_off_t *)args` with **bytes per sector**
`RT_DEVICE_CTRL_ALL_BLK_SSIZEGET`	Total byte size = sectors × bytes_per_sector (used by DFS v2 POSIX layer)

Vendor-specific IOCTLs pass through **`disk->ops->control(disk, blk, cmd, args)`** on the volume path; on the disk path **`blk` is NULL**.

Geometry structure

```
struct rt_device_blk_geometry {
    rt_uint64_t sector_count;
    rt_uint32_t bytes_per_sector;
    rt_uint32_t block_size;    /* erase block size for flash-style media */
};
```

1.4.2.31.3 Partition volumes (`rt_blk_device`)

Defined in **`blk.h`** when DFS headers are available.

Field	Meaning
`disk`	Parent **`rt_blk_disk`**
`sector_start` / `sector_count`	Slice on the physical disk (LBA range)
`partno`	Index used in device name (`spd` / `sd`)
`partition`	**`struct dfs_partition`** for DFS mount; **`partition.lock`** → **`disk->usr_lock`**
`list`	Node on **`disk->part_nodes`**

blk_dev.c forwarding

Operation	Behavior
read/write	Translates to **disk->parent** at **sector_start + pos**
GETGEOME	Disk geometry with **sector_count** replaced by partition length
SYNC	**rt_device_control(&disk->parent, SYNC)**
ERASE / AUTOREFRESH	Allowed only when **disk->partitions <= 1**

Device naming (DM)

disk_add_blk_dev (blk_dev.c):

- If disk name's last character is **'a'-'z'** (e.g. **sda**): volume name **sdap1** (spd).
- Otherwise (e.g. **mmcsd0**, **nvme0n1**): **mmcsd0p1**, **nvme0n1p2** (sd).

Each volume gets its own **rt_dm_ida_alloc(disk->ida)** slot under the same **master_id** as the disk.

1.4.2.31.4 DFS integration

When **RT_USING_DFS** and **RT_USING_POSIX_DEVIO** (and DFS v2 as implemented in-tree), **blk_dfs.c** registers file operations that:

1. On **open**: **GETGEOME** + **ALL_BLK_SSIZEGET** for file size.
2. On **read/write**: byte-oriented I/O with unaligned head/tail sector handling via a bounce sector buffer.
3. **ioctl**: forwards to **rt_device_control**.

Mount the **volume** device name (not the disk) with the appropriate filesystem driver after partition probe.

1.4.2.31.5 Drivers already using blk

These drivers embed **struct rt_blk_disk**, set **ops**, **ida**, and call **rt_hw_blk_disk_register** when media is present:

Driver	File	Typical name	Notable rt_blk_disk fields
NVMe	nvme/nvme.c	nvme0n1, ...	**parallel_io = RT_TRUE**, per-namespace disk, **max_↩ partitions = RT_BLK_↩ PARTITION_MAX**
SDIO/MMC	sdio/dev_block.c	host name / mmcsd0p*	**removable**, **max_partitions = RT_MMCSD_MAX_↩ PARTITION**, sets geometry before register
SCSI disk	scsi/scsi_sd.c	sda, sdb, ...	**parallel_io** from host, READ CAPACITY 10/16, **sd_ida**
SCSI CD-ROM	scsi/scsi_↩ cdrom.c	optical media	Read-only style backend

AHCI/SATA path: **ata/ahci.c** exposes SCSI devices; **SD card** goes through **scsi_sd** or **MMC dev_block**, then blk.

Minimal driver pattern

```
static const struct rt_blk_disk_ops my_blk_ops = {
    .read     = my_read,
```

```
    .write    = my_write,
    .getgeome = my_getgeome,
    .sync     = my_sync,
};

static struct rt_dm_ida my_ida = RT_DM_IDA_INIT(MY_DISK);

void my_disk_register(struct my_hw *hw)
{
    struct rt_blk_disk *disk = &hw->blk;

    disk->ops = &my_blk_ops;
    disk->ida = &my_ida;
    disk->parallel_io = RT_FALSE;  /* or RT_TRUE if ops are reentrant */
    rt_dm_dev_set_name(&disk->parent.parent, "sda");  /* example */

    rt_hw_blk_disk_register(disk);  /* also runs partition probe */
}
```

Details: Physical disk (`rt_blk_disk`).

1.4.2.31.6 Locks and concurrency

Mechanism	Scope
disk->usr_lock (semaphore)	Serializes disk **read**/**write**/**sync** when **parallel_io == 0**
disk->lock (spinlock)	Partition list, unregister, erase path
parallel_io == 1	Disk **read**/**write** skip **usr_lock** — backend must be safe for concurrent I/O

Partition volume I/O still goes through the disk node and follows the disk's **parallel_io** / lock rules.

1.4.2.31.7 Typical bring-up sequence

1. Allocate **struct rt_blk_disk** (often embedded in driver private data).
2. Implement **rt_blk_disk_ops**; set **ida**, name, flags (**read_only**, **removable**, **parallel_io**, **max_partitions**).
3. **rt_hw_blk_disk_register(disk)** — partition probe runs automatically.
4. Application or init code **rt_device_find("sda1")**, mount DFS, or run filesystem code.
5. **rt_hw_blk_disk_unregister(disk)** after unmount — syncs, removes all volumes, checks **ref_count**.

To **re-scan** partitions after media change, unregister, update geometry, register again (or call **rt_blk_disk_probe_partition** only if **disk->partitions == 0**).

1.4.2.31.8 Driver / BSP quick notes

Topic	Guidance
Name before register	First character of **disk->parent.parent.name** must be non-NUL
ida	Mandatory with **RT_USING_DM**; disk and each volume consume IDA slots
Probe errors	Ignored at register — check **disk->partitions** or open volumes explicitly
max_partitions	Set **RT_BLK_PARTITION_NONE** to skip table parsing
Read-only	Omit **write** in **ops** — core forces **read_only**
Unmount first	**unregister** returns **-RT_EBUSY** if **ref_count > 0**
Erase	Disk erase IOCTL removes **all** partition devices first

1.4.2.31.8.1 See also

- Physical disk (`rt_blk_disk`) — `rt_blk_disk` / `rt_blk_disk_ops` / register API
- Disk partition probing — GPT/EFI and DFS parsers
- ATA / AHCI — AHCI → SCSI → blk path
- NVMe subsystem
- SDIO / MMC core
- SCSI host and commands
- `components/drivers/block/blk.c`
- `components/drivers/block/blk_dev.c`
- `components/drivers/block/blk_partition.c`

1.4.2.32 Physical disk (`rt_blk_disk`)

1.4.2.32.1 Disk device (driver writer guide)

`struct rt_blk_disk` is the **physical backing store**: one contiguous LBA space implemented by your driver's **`rt_blk_disk_ops`**. The blk core wraps it as an **`rt_device`**, then creates **volume** children — see Block layer (`blk`) and Disk partition probing.
Implementation: **`components/drivers/block/blk.c`**. Header: **`components/drivers/include/drivers/blk.h`**.
Same documentation pattern as **`documentation/6.components/device-driver/ata/ata.md`**: fill structures before **`rt_hw_blk_disk_register`**, implement **`ops`**, then let the core attach partitions and DFS-facing devices.

1.4.2.32.1.1 `struct rt_blk_disk` (before `rt_hw_blk_disk_register`)

```
 34:57:components/drivers/include/drivers/blk.h
struct rt_blk_disk
{
    struct rt_device parent;
    const struct rt_blk_disk_ops *ops;
#ifdef RT_USING_DM
    struct rt_dm_ida *ida;
#endif
    rt_uint32_t read_only:1;
    rt_uint32_t parallel_io:1;
    rt_uint32_t removable:1;
    rt_uint32_t __magic;
    rt_uint32_t partitions;
    rt_int32_t max_partitions;
    rt_list_t part_nodes;
    struct rt_spinlock lock;
    struct rt_semaphore usr_lock;
};
```

Member	Owner	Rule / meaning
`parent`	**Caller**	Embedded **`rt_device`**. **Name must be set** before register (first char of `**parent.parent.name \0**`).
`ops`	**Caller**	**Required**; see **`struct rt_blk_disk_ops`** below.
`ida`	**Caller** (`RT_USING_DM`)	**Non-NULL**; shared with all volume children for **`rt_dm_ida_alloc`**.
`read_only`	**Caller / core**	If **`ops->write == NULL`**, core sets **`read_only = RT_TRUE`**.
`parallel_io`	**Caller**	**`RT_TRUE`**: disk read/write skip **`usr_lock`** (NVMe uses this).
`removable`	**Caller**	Hint for removable media (e.g. SDIO sets from host).

Member	Owner	Rule / meaning
max_partitions	**Caller**	Cap for parsers; **RT_BLK_PARTITION_NONE** disables probing. **RT_BLK_PARTITION_MAX** = unlimited.
partitions	**Core**	Count of registered **rt_blk_device** nodes.
part_nodes	**Core**	List of **struct rt_blk_device**.
lock	**Core**	Spinlock for list / unregister / erase.
usr_lock	**Core**	Semaphore (prio) serializing I/O when not **parallel_io**.
__magic	**Core**	Set to **RT_BLK_DISK_MAGIC** at register.

1.4.2.32.1.2 struct rt_blk_disk_ops

```
 62:73:components/drivers/include/drivers/blk.h
struct rt_blk_disk_ops
{
    rt_ssize_t (*read)(struct rt_blk_disk *disk, rt_off_t sector, void *buffer,
            rt_size_t sector_count);
    rt_ssize_t (*write)(struct rt_blk_disk *disk, rt_off_t sector, const void *buffer,
            rt_size_t sector_count);
    rt_err_t (*getgeome)(struct rt_blk_disk *disk, struct rt_device_blk_geometry *geometry);
    rt_err_t (*sync)(struct rt_blk_disk *disk);
    rt_err_t (*erase)(struct rt_blk_disk *disk);
    rt_err_t (*autorefresh)(struct rt_blk_disk *disk, rt_bool_t is_auto);
    rt_err_t (*control)(struct rt_blk_disk *disk, struct rt_blk_device *blk, int cmd, void *args);
};
```

Callback	Required?	Contract
read	**Yes** (practical)	**sector**, **sector_count** in **512-byte sectors** unless **getgeome** says otherwise. Return sectors read or < **0** error.
write	Optional	**NULL** → disk read-only. Same addressing as **read**.
getgeome	**Yes** (practical)	Fill **sector_count**, **bytes_per_sector**, **block_size**. Used by partition code and DFS.
sync	Optional	Flush caches; called on unregister and **RT_DEVICE_CTRL_BLK_SYNC**.
erase	Optional	Whole-device erase; disk IOCTL may remove partitions first.
autorefresh	Optional	**RT_DEVICE_CTRL_BLK_AUTOREFRESH** (e.g. SD auto refresh).
control	Optional	Vendor IOCTLs; **blk** is **NULL** on raw disk, non-NULL on volume forwarding.

Sector addressing expectations

- Arguments are **LBA sector indices** on the **whole disk**, not bytes.
- **getgeome()->bytes_per_sector** must match what **read**/**write** use.
- **getgeome()->sector_count** is full device capacity in sectors.
- If **parallel_io**, **read**/**write** may overlap — use driver-internal locking or queue depth as needed.

1.4.2.32.1.3 rt_hw_blk_disk_register (core sequence)

```
 228:337:components/drivers/block/blk.c
rt_err_t rt_hw_blk_disk_register(struct rt_blk_disk *disk)
```

Step	Action
1	Validate **disk**, **ops**, name, **ida** (DM).

Step	Action
2	`rt_dm_ida_alloc(disk->ida)` for the **disk** device id.
3	Init `usr_lock`, `part_nodes`, `lock`.
4	Hook `blk_ops` or `blk_parallel_ops` on `parent`.
5	If `!ops->write` → `read_only = 1`.
6	`device_set_blk_fops`, `rt_device_register` with RD/WR flags.
7	`rt_blk_disk_probe_partition(disk)` — errors ignored.

Return: register error only; partition failure does not fail register.

Disk `rt_device` ops (internal)

Op	`parallel_io == 0`	`parallel_io == 1`
read/write	Under `usr_lock`	Direct `ops->read/write`
control	`blk_control` → `ops`	Same

`RT_DEVICE_CTRL_BLK_PARTITION` on the **disk** node returns `-RT_EINVAL` (use volume devices).

1.4.2.32.1.4 `rt_hw_blk_disk_unregister`

Step	Action
1	`ref_count > 0` → `-RT_EBUSY`
2	`ops->sync` if present
3	`blk_remove_all` — unregister every `rt_blk_device`
4	`rt_dm_ida_free` for disk id, `rt_device_unregister`

Unmount DFS / close handles before unregister.

1.4.2.32.1.5 Helper APIs

API	Role
`rt_blk_disk_probe_partition(disk)`	Re-run parsers if `partitions == 0`; see Disk partition probing
`rt_blk_disk_get_capacity(disk)`	`getgeome` → `sector_count`
`rt_blk_disk_get_logical_block_size(disk)`	`getgeome` → `bytes_per_sector`

1.4.2.32.1.6 In-tree integration examples

NVMe (`nvme/nvme.c`)

Per namespace:

- `rt_dm_dev_set_name(&ndev->parent.parent, "snu", nvme->name, nsid)`
- `parallel_io = RT_TRUE`, `ops = &nvme_blk_ops`, `ida = &nvme_ida`
- `max_partitions = RT_BLK_PARTITION_MAX`

- **rt_hw_blk_disk_register(&ndev->parent)** after identify

SDIO / MMC (sdio/dev_block.c)

- Geometry from card CSD: **sector_count**, **bytes_per_sector**, **block_size**
- **mmcsd_set_blksize** before register
- **removable**, **max_partitions = RT_MMCSD_MAX_PARTITION**
- Name from **host->name** → volumes **hostp1** style

SCSI SD (scsi/scsi_sd.c)

- READ CAPACITY (10/16) → geometry
- **rt_dm_dev_set_name(..., "sdcc", letter_name(sd_id))** — names like **sda**
- **parallel_io** from SCSI host
- **ida = &scsi_sd_ida**

SCSI CD-ROM (scsi/scsi_cdrom.c)

Same blk registration pattern; typically read-only media (**write** omitted or read-only flag).

1.4.2.32.1.7 DM naming checklist

1. **RT_DM_IDA_INIT** per disk family (e.g. **scsi_sd_ida**, **nvme_ida**, **sdio_ida**).
2. Set **disk->ida** before register.
3. Disk name via **rt_dm_dev_set_name(&disk->parent.parent, ...)**.
4. Volumes are named automatically at probe — do not register **rt_blk_device** yourself unless you bypass **blk_put_partition**.

1.4.2.32.1.8 Driver cautions

Topic	Guidance
DMA / buffers	Bounce in **ops** if callers pass non-DMA-safe pointers
Capacity change	Unregister, update geometry, register again
Hotplug	Remove card → **rt_hw_blk_disk_unregister**; insert → register fresh
GPT + DFS	Enable both Kconfig parsers; EFI runs first in **partition_list[]**
Custom IOCTL	Implement **control(disk, blk, cmd, args)**; forward unknown cmds from **blk_control** default case

1.4.2.32.1.9 See also

- Block layer (`blk`) — application API and volume layer
- Disk partition probing
- components/drivers/block/blk.c
- components/drivers/include/drivers/blk.h

1.4.2.33 Disk partition probing

1.4.2.33.1 Partition tables on `rt_blk_disk`

After **`rt_hw_blk_disk_register`**, the core calls **`rt_blk_disk_probe_partition`** to read the on-disk partition layout and register one **`rt_blk_device`** per slice via **`blk_put_partition`**. Implementation: **`components/drivers/block/blk_partition.c`**; parsers in **`components/drivers/block/partitions/`**.
Volume naming and application access: Block layer (`blk`). Disk registration: Physical disk (`rt_blk_disk`).

1.4.2.33.1.1 Kconfig

Option	File	Role
`RT_BLK_PARTITION_EFI`	`partitions/efi.c`	Protective MBR + **GPT** (UEFI-style)
`RT_BLK_PARTITION_DFS`	`partitions/dfs.c`	**DFS** legacy partition table in sector 0

Both can be enabled; **`partition_list[]`** order is **EFI first**, then **DFS**. The first parser returning **0** wins.

1.4.2.33.1.2 `rt_blk_disk_probe_partition`

```
 103:154:components/drivers/block/blk_partition.c
rt_err_t rt_blk_disk_probe_partition(struct rt_blk_disk *disk)
```

Step	Behavior
1	**`disk == NULL`** → **`-RT_EINVAL`**
2	**`disk->partitions != 0`** → return **success** (already probed)
3	**`max_partitions == RT_BLK_PARTITION_NONE`** → **`-RT_EEMPTY`**
4	Loop **`partition_list[i](disk)`**
5	Parser **`-RT_ENOMEM`** → abort, return **`-RT_ENOMEM`**
6	Parser **0** → success, stop
7	No parser matched → **full-disk volume**: **`blk_put_partition(disk, NULL, 0, total_sectors, 0)`**

So a blank or unknown layout still yields **one** mountable device covering the entire disk (**`partno == 0`**).

1.4.2.33.1.3 `blk_put_partition`

```
 27:100:components/drivers/block/blk_partition.c
rt_err_t blk_put_partition(struct rt_blk_disk *disk, const char *type,
        rt_size_t start, rt_size_t count, int partno)
```

Phase	Action
Allocate	**`struct rt_blk_device`**
Init	**`blk_dev_initialize(blk)`**
Range	**`sector_start = start`**, **`sector_count = count`**, **`partno`**
DFS meta	**`partition.offset`**, **`partition.size`**, **`partition.lock = &disk->usr_lock`**
Register	**`disk_add_blk_dev(disk, blk)`** → **`rt_device_register`**, link **`part_nodes`**
Count	**`disk->partitions++`**

Console log

For non-**"dfs"** types, printk partition index, byte offset, and human-readable size (KB/MB/GB).

Failure

-RT_ENOMEM frees the partial **rt_blk_device** and logs via **LOG_E**.

1.4.2.33.1.4 EFI / GPT (partitions/efi.c)

Boot-time force_gpt

INIT_CORE_EXPORT(force_gpt_setup) reads bootarg **gpt** via **rt_ofw_bootargs_select("gpt", 0)** when **RT_USING_OFW**. When set, GPT probing is forced even if protective MBR checks are ambiguous.

Validation flow (summary)

1. Read LBA 0 — check **protective MBR** (EFI GPT type, starting LBA 1).
2. Read **primary GPT header** at LBA 1 (or from **force_gpt** path).
3. Verify signature **"EFI PART"**, header CRC (**efi_crc32**), **my_lba**, **first_usable_lba** / **last_usable_lba** vs disk capacity (**last_lba()** = **rt_blk_disk_get_capacity - 1**).
4. Load partition entry array; validate entry CRC.
5. For each valid entry: **blk_put_partition(disk, type, start, count, partno)** with type string / GUID mapping where implemented.

Sector size

All LBA math uses **rt_blk_disk_get_logical_block_size(disk)** — must stay consistent with **getgeome**.

Enable

Menu: **RT_BLK_PARTITION_EFI** (default **y** when blk partitions menu is on).

1.4.2.33.1.5 DFS partition (partitions/dfs.c)

Legacy path for RT-Thread **DFS** on-disk superblock layout:

1. Allocate one sector buffer.
2. **disk->ops->read(disk, 0, sector, 1)**.
3. Loop **dfs_filesystem_get_partition(&part, sector, i)** for **i** < **max_partitions**.
4. **blk_put_partition(disk, "dfs", part.offset, part.size, i)**.

Requires **RT_USING_DFS** and **RT_BLK_PARTITION_DFS**.

1.4.2.33.1.6 Volume naming after probe

Handled in **disk_add_blk_dev** (blk_dev.c):

Disk name ends with	Example disk	Volume examples
Letter **a–z**	**sda**	**sdap1**, **sdap2**
Digit / other	**mmcsd0**, **nvme0n1**	**mmcsd0p1**, **nvme0n1p2**

1.4.2.33.1.7 When to call probe

Scenario	Action
Normal driver	Rely on **rt_hw_blk_disk_register** (automatic)
Media just formatted	Unregister disk, register again, or call **rt_blk_disk_probe_partition** if **partitions == 0**
Disable tables	Set **max_partitions = RT_BLK_PARTITION_NONE**
RAM disk / single volume	Still get one full-disk **partno 0** unless you manage volumes manually

Call sites in tree

Location	Notes
blk.c	End of **rt_hw_blk_disk_register**
Driver after hotplug	Rare; prefer re-register

1.4.2.33.1.8 Interaction with disk IOCTLs

IOCTL	Multi-partition disk
ERASE / **AUTOREFRESH** on **volume**	**-RT_EIO** if **disk->partitions > 1**
ERASE on **disk**	Removes **all** volumes first, then **ops->erase**
GETGEOME on **volume**	**sector_count** = partition only

1.4.2.33.1.9 Driver / BSP quick notes

Topic	Guidance
Order	EFI before DFS — do not rely on DFS if GPT is present
max_partitions	SDIO passes **RT_MMCSD_MAX_PARTITION**; NVMe uses **RT_BLK_PARTITION_MAX**
Empty card	Probe still creates full-disk volume — mount may need valid FS
Re-probe	Not supported with existing volumes; unregister disk first
Custom layouts	Add a parser to **partition_list[]** or call **blk_put_partition** from driver code

1.4.2.33.1.10 See also

- Block layer (`blk`)
- Physical disk (`rt_blk_disk`)
- `components/drivers/block/blk_partition.c`
- `components/drivers/block/partitions/efi.c`
- `components/drivers/block/partitions/dfs.c`

1.4.2.34 Hardware cache (hwcache)

1.4.2.34.1 CPU cache maintenance (hwcache)

On RT-Thread, **CPU D-cache** / **I-cache** maintenance for DMA coherence, MMU table updates, and dynamic code loading is done through architecture code in **libcpu** and, when **RT_USING_HWCACHE** is enabled, the DM helper module **components/drivers/hwcache/**.
Most drivers in the tree call the BSP APIs in **include/rthw.h** (rt_hw_cpu_dcache_ops, rt_hw_↩ cpu_icache_ops). The **rt_hwcache_*** functions in **drivers/hwcache.h** are a thin DM layer that forwards to **struct rt_hwcache_ops** pointers filled from device tree stubs or SoC code.
Documentation path: **hwcache/hwcache.md** (not cache/ — the subsystem name is **hwcache**).

1.4.2.34.1.1 Kconfig

Option	Role
RT_USING_CACHE	Enables **rthw.h** cache APIs (required for real flush/invalidate)
RT_USING_HWCACHE	Builds **components/drivers/hwcache/hwcache.c**; depends on **RT↩ _USING_DM** and **RT_USING_CACHE**
SOC_DM_HWCACHE_DIR	BSP may add extra Kconfig under SoC DM hwcache drivers

If **RT_USING_CACHE** is off, **rthw.h** defines **rt_hw_cpu_*_ops** as empty macros — maintenance becomes no-ops.

1.4.2.34.1.2 Two API layers (do not mix up)

```
Application / NVMe / SDIO / DMA pool / ...

 rt_hw_cpu_dcache_ops() / rt_hw_cpu_icache_ops()   ← include/rthw.h (usual)

 libcpu/*/cache.c (per architecture)
Optional parallel path when RT_USING_HWCACHE:

 rt_hwcache_dcache_ops() / rt_hwcache_icache_ops()   ← drivers/hwcache.h

 rt_dm_cpu_dcache_ops / rt_dm_cpu_icache_ops

 struct rt_hwcache_ops (flush/invalidate from OFW stub or SoC init)
```

Layer	Header	Typical caller
Architecture (primary)	rthw.h	**nvme**, **sdio**, **ahci**, **ufs**, **dma_pool**, USB glue, **lwp**, **dlmodule**
DM hwcache	drivers/↩ hwcache.h	Code that explicitly uses **rt_hwcache_*** or SoC **RT↩ _HWCACHE_OFW_DECLARE** setup

Unless your BSP connects **rt_dm_cpu_dcache_ops** to the same implementation as **rt_hw_cpu_↩ dcache_ops**, calling only one layer may be insufficient. **Driver authors: use rt_hw_cpu_dcache_ops unless the BSP documents rt_hwcache_* as the canonical path.**

1.4.2.34.1.3 Architecture API (rthw.h)

Enabled when **RT_USING_CACHE** is set.

Opcodes

```
enum RT_HW_CACHE_OPS {
```

```
    RT_HW_CACHE_FLUSH      = 0x01,  /* clean / write-back to RAM */
    RT_HW_CACHE_INVALIDATE = 0x02,  /* discard cached copy, refetch from RAM */
};
```

RT_CPU_CACHE_LINE_SZ (default 32) is the usual alignment hint for maintenance ranges.

D-cache

Function	Role
rt_hw_cpu_dcache_enable()	Enable D-cache
rt_hw_cpu_dcache_disable()	Disable D-cache
rt_hw_cpu_dcache_status()	Non-zero if enabled
rt_hw_cpu_dcache_ops(ops, addr, size)	Flush or invalidate **[addr, addr+size)**

I-cache

Function	Role
rt_hw_cpu_icache_enable()	Enable I-cache
rt_hw_cpu_icache_disable()	Disable I-cache
rt_hw_cpu_icache_status()	Non-zero if enabled
rt_hw_cpu_icache_ops(ops, addr, size)	Typically **invalidate** after loading code

Example (DMA TX buffer)

```
/* CPU filled buffer; device DMA will read RAM */
rt_hw_cpu_dcache_ops(RT_HW_CACHE_FLUSH, buf, len);

/* After device DMA wrote RAM; CPU will read buffer */
rt_hw_cpu_dcache_ops(RT_HW_CACHE_INVALIDATE, buf, len);
```

1.4.2.34.1.4 DM hwcache API (drivers/hwcache.h)

Implementation: **components/drivers/hwcache/hwcache.c**.

struct rt_hwcache_ops

Callback	Role
enable / **disable**	Turn cache on/off
status	Query enabled state
flush(vaddr, size)	Clean D-cache (or unified) for region
invalidate(vaddr, size)	Invalidate region

Global pointers (set by platform init or OFW stub):

- **rt_dm_cpu_dcache_ops**
- **rt_dm_cpu_icache_ops**

rt_hwcache_* wrappers

Function	Maps to
rt_hwcache_dcache_enable/disable/status	**rt_dm_cpu_dcache_ops**
rt_hwcache_dcache_ops(op, addr, size)	**flush** if **op == RT_HW_CACHE_FLUSH**, **invalidate** if **RT_HW_CACHE_INVALIDATE**
rt_hwcache_icache_*	Same for **rt_dm_cpu_icache_ops**

If the corresponding **rt_dm_cpu_*_ops** is **NULL**, calls are no-ops (safe but ineffective).

Initialization

```
rt_err_t rt_hwcache_init(void);
```

With **RT_USING_OFW**, walks all DT nodes and runs **rt_ofw_stub_probe_range** for stubs in the **hwcache** linker range (**RT_OFW_STUB_RANGE_EXPORT(hwcache, ...)** in `hwcache.c`). Stub handlers typically assign **rt_dm_cpu_dcache_ops** / **rt_dm_cpu_icache_ops**.

OFW registration macro

```
RT_HWCACHE_OFW_DECLARE(name, ids, handler);
```

Expands to **RT_OFW_STUB_EXPORT(name, ids, hwcache, handler)**. **handler(struct rt_ofw_node *np, const struct rt_ofw_node_id *id)** should install **struct rt_hwcache_ops** into the global pointers. See Open Firmware (OFW) (stub sections).
SoC-specific drivers may live under **SOC_DM_HWCACHE_DIR** (per **hwcache/Kconfig**).

1.4.2.34.1.5 DMA and coherence

Path	Cache maintenance
dma-coherent DT property	**rt_dma_device_is_coherent** — map ops may skip explicit flush (platform-dependent)
Coherent DMA pool (`dma_pool.c`)	**sync_out**: **rt_hw_cpu_dcache_ops(FLUSH)**; **sync_in**: **INVALIDATE**
Non-coherent pool	Physical address only; CPU must still maintain if CPU touches the buffer

Coordinate with DMA engine and DMA memory pools — avoid **double flush** and **invalidate-before-flush** on dirty TX buffers.

1.4.2.34.1.6 Drivers that use cache maintenance (examples)

Subsystem	File	Typical pattern
NVMe	`nvme/nvme.c`	Flush PRP/list buffers before doorbell; invalidate after read
SDHCI / SDIO	`sdio/dev_sdhci.c`, `sdio-dw.c`	Flush before TX DMA; invalidate after RX
AHCI	`ata/ahci.c`	Flush/invalidate command tables and DMA buffers
UFS	`ufs/ufs.c`	Flush UTRD/UCD before run; invalidate after completion
DMA PL330	`dma/dma-pl330.c`	Flush microcode buffer

Subsystem	File	Typical pattern
USB	CherryUSB glue	Platform-specific flush/invalidate hooks
GICv3 ITS	`pic/pic-gicv3-its.c`	Flush ITS command/ITT tables
DFS pcache	`dfs_v2/dfs_pcache.c`	Flush + I-invalidate on mapped pages
Dynamic load	`libc/dlmodule.c`, `lwp/`	Flush data, invalidate I-cache for new code

These use **`rt_hw_cpu_*_ops`** from **`rthw.h`**, not **`rt_hwcache_*`**, unless your port unifies them.

1.4.2.34.1.7 Use cases

1. **Non-coherent DMA** — CPU writes → **flush** before device read; device writes → **invalidate** before CPU read.
2. **MMU** / **page tables** — flush page table memory after CPU fills entries (see **`libcpu/*/mmu.c`**).
3. **Self-modifying** / **relocated code** — flush D-cache if instructions were stored through D-cache, then **invalidate I-cache**.
4. **Framebuffer** / **uncached mapping** — prefer correct **MMU memory attributes** (Simple framebuffer (`simple-framebuffer`)); use explicit ops only when mapping is cacheable.

1.4.2.34.1.8 Driver / BSP quick notes

Topic	Guidance
Prefer `rt_hw_cpu_*`	Matches existing storage and DMA drivers in **`components/drivers`**
Range size	Cover full DMA buffer; align to cache line when port requires it
TX ordering	**Flush** before kicking device — never **invalidate** dirty lines first
RX ordering	**Invalidate** before CPU reads device-written RAM
Harvard cores	After loading code via data path: flush D + invalidate I
`rt_hwcache_init`	Call during early init if SoC uses **`RT_HWCACHE_OFW_DECLARE`** to publish ops
Performance	Scope maintenance to touched regions; do not flush entire RAM

1.4.2.34.1.9 Pitfalls

- **`rt_hwcache_*_disable()`** in **`hwcache.c`** currently invokes **`ops->enable`** in **`hwcache_disable()`** (likely a bug). Prefer **`rt_hw_cpu_dcache_disable()`** or fix SoC ops until upstream is corrected.
- **Double maintenance** with **`rt_dma_sync_*`** and manual **`rt_hw_cpu_dcache_ops`** on the same buffer.
- **`RT_USING_CACHE`** **off** — silent no-ops; DMA corruption on non-coherent SoCs.
- **Assuming `rt_hwcache_*` runs on all boards** — only effective after **`rt_dm_cpu_*_ops`** are assigned.
- **I-cache only invalidate** after external loaders without D-cache flush when stores went through D-cache.

1.4.2.34.1.10 See also

- `components/drivers/include/drivers/hwcache.h`
- `components/drivers/hwcache/hwcache.c`
- `include/rthw.h` — **`RT_HW_CACHE_*`**, **`rt_hw_cpu_dcache_ops`**

- DMA engine
- DMA memory pools
- Open Firmware (OFW) — OFW stub mechanism

1.4.2.35 DVFS

1.4.2.35.1 Dynamic Voltage and Frequency Scaling (DVFS)

Headers: **components/drivers/include/drivers/dvfs.h**. Core: **components/drivers/dvfs/dvfs.c**, governors: **components/drivers/dvfs/governor/**, shell: **components/drivers/dvfs/dvfs_cmd.c**.
DVFS selects an **OPP (Operating Performance Point)** — a frequency and supply voltage pair — according to a **governor** policy. The framework owns the OPP table, governor lifecycle, and MSH commands; **BSP or SoC drivers** implement how a given OPP is applied (clock, regulator, firmware calls).
Kconfig: **RT_USING_DVFS** (requires **RT_USING_DM**, **RT_USING_CLK**, **RT_USING_REGULATOR**). Optional: **RT_USING_DVFS_EVENT**, **RT_DVFS_SCMI_CPUFREQ**, and BSP options under **SOC_DM_DVFS_***.

1.4.2.35.1.1 Architecture

```
Device tree (operating-points-v2, supplies, clocks)
        |
        v
BSP cpufreq / devfreq driver (SOC_DM_DVFS_CPUFREQ_DIR, ...)
  fills rt_dvfs_scaling_ops (set_opp, parse_opp, ...)
        |
        v
rt_dvfs_cpufreq_register() / rt_dvfs_devfreq_register()
        |
        v
rt_dvfs_scaling  (OPP table, cur_freq, governor)
        |
        +-- governor (performance / ondemand / ...)
        +-- thermal-cool-dvfs (optional, RT_THERMAL_COOL_DVFS)
        +-- rt_dvfs_scaling_set_frequency() / apply_opp()
```

Layer	Location	Role
Framework	components/drivers/dvfs/	OPP table, governors, load stats, PM hooks, MSH
Generic SCMI	dvfs-scmi-cpufreq.c	Optional SCMI-based CPUfreq (**RT_DVFS_SCMI_CPUFREQ**)
BSP	**SOC_DM_DVFS_CPUFREQ_DIR**, **SOC_DM_DVFS_DEVFREQ_DIR**, **SOC_DM_DVFS_EVENT_DIR**	Platform probe, **set_opp**, DT parsing
Thermal	thermal-cool-dvfs.c	Map cooling levels to OPP indices — Thermal cooling devices

When **RT_USING_DVFS** is enabled, **struct rt_device::dvfs_scaling** links a device to its scaling domain for **list_dvfs** / **dvfs dump**.

1.4.2.35.1.2 Registering a scaling domain (BSP driver)

Typical CPU path:

1. Allocate **struct rt_dvfs_cpufreq** (embeds **struct rt_dvfs_scaling**).
2. Fill **scaling->ops** — at minimum **set_opp** and **parse_opp** for DT-backed OPP tables.
3. Obtain **scaling->clk** and **scaling->supply** from the CPU / cluster device node.
4. Parse OPP nodes (**operating-points-v2**) into the scaling OPP table.
5. Call **rt_dvfs_cpufreq_register()**, then **rt_dvfs_scaling_set_governor()** with the desired default governor.

Devfreq devices follow the same pattern with **struct rt_dvfs_devfreq** and optional **struct rt_dvfs_event** for load feedback (**RT_USING_DVFS_EVENT**).
If the driver does not provide **ops->set_opp**, the framework falls back to a generic sequence using **rt_clk_set_rate** and **rt_regulator_set_voltage** on **scaling->clk** / **scaling->supply**.

1.4.2.35.1.3 Governors

Name	Behavior
performance	Always **max_freq**
powersave	Always **min_freq**
freedom	No automatic changes; user sets frequency manually
ondemand	Periodic load sampling; high load → max, low load → scaled down
conservative	Step up/down by **freq_step**
schedutil	Target frequency proportional to estimated load

Dynamic governors depend on **RT_USING_IDLE_HOOK** (load estimation) and **RT_USING_SYSTEM_WORKQUEUE** (monitor timer). The default governor is chosen by the BSP driver at registration time.
Governor parameters (**up_threshold**, **sampling_rate_ms**, ...) live in **struct rt_dvfs_governor_params** and can be adjusted via **rt_dvfs_governor_set_params()**.

1.4.2.35.1.4 MSH commands

Requires **RT_USING_CONSOLE** and **RT_USING_MSH** (**dvfs_cmd.c**).

```
list_dvfs
dvfs dump <name>
dvfs set_governor <name> <governor>
dvfs set_frequency <name> <Hz>
```

set_frequency switches to **freedom** automatically when the active governor is not freedom.
Example:

```
msh />dvfs set_governor cpu0 performance
msh />dvfs dump cpu0
```

The scaling device <**name**> is assigned by the BSP driver (commonly **cpufreq0**, **dmc**, etc.).

1.4.2.35.1.5 Device tree (typical CPU)

Minimal pattern (details vary by SoC BSP):

```
cpu0: cpu@0 {
    device_type = "cpu";
    compatible = "arm,cortex-a55";
    operating-points-v2 = <&cpu0_opp_table>;
    cpu-supply = <&cpu_reg>;
    /* optional: #cooling-cells for thermal-cool-dvfs */
};

cpu0_opp_table: opp-table {
    compatible = "operating-points-v2";
    opp-1000000000 {
        opp-hz = /bits/ 64 <1000000000>;
        opp-microvolt = <900000>;
    };
    /* additional OPP nodes ... */
};
```

Binding	Role
operating-points-v2	OPP table phandle on the scaling device
opp-hz / **opp-microvolt**	Frequency (Hz) and voltage (µV) per OPP
***-supply**	Regulator phandle(s) used when scaling voltage
#cooling-cells	Enables **dvfs-cool** thermal device on the CPU node

Passive thermal **cooling-maps** can reference the DVFS cooling device to cap OPP on trip — see Thermal cooling devices.

1.4.2.35.1.6 Kconfig

Option	Role
RT_USING_DVFS	Core framework + governors
RT_USING_DVFS_EVENT	**dvfs_event.c**, devfreq load via event devices
RT_USING_DVFS_OPP_RETRY_MAX	Retries on **-RT_EBUSY** during OPP transition
RT_DVFS_SCMI_CPUFREQ	Generic **dvfs-scmi-cpufreq.c**
RT_THERMAL_COOL_DVFS	**thermal-cool-dvfs.c** (also needs **RT_USING_THERMAL**)
RT_USING_IDLE_HOOK	Required for meaningful dynamic governor load stats
SOC_DM_DVFS_CPUFREQ_DIR	BSP CPUfreq driver(s)
SOC_DM_DVFS_DEVFREQ_DIR	BSP devfreq driver(s)
SOC_DM_DVFS_EVENT_DIR	BSP devfreq event source(s)

1.4.2.35.1.7 OPP transition order

When **ops->set_opp** is provided, the BSP defines the safe ramp sequence (voltage-before-frequency on scale-up, etc.).
When the framework generic path is used (**dvfs.c**):

- **Scale up**: raise voltage (if needed) → optional **transition_latency** delay → **rt_clk_set_rate**
- **Scale down**: lower clock rate → lower voltage (if needed)

On failure, **cur_freq** and **current_opp** are not updated. **RT_USING_DVFS_OPP_RETRY_MAX** controls busy retries on regulator or clock calls.

1.4.2.35.1.8 Pitfalls

- **BSP set_opp must match hardware**: clocks, regulators, and any bus used to change voltage must remain usable across the full OPP range. Broken I2C/SPI/regulator access during a transition surfaces as stuck frequency or **-RT_EBUSY** retries.
- **Governor switch vs. pending work**: dynamic governors schedule work on the system workqueue. After **set_governor**, monitor callbacks must verify the active governor type before changing frequency (see governor sources).
- **Thermal cooling levels**: **thermal-cool-dvfs** maps cooling **level** to an OPP index; releasing cooling must restore full performance (highest OPP), not the most restrictive level. See **rt_thermal_cooling_device_kick** in **thermal.c**.
- **Verifying dynamic governors**: do not run a tight CPU loop inside the **FinSH** / **shell thread** — it usually has higher priority than the system workqueue and can prevent load sampling. Use a **separate background thread** at lower priority, or compare **performance** vs **powersave** (static governors) to confirm the scaling path works before testing **ondemand** / **schedutil**.

1.4.2.35.1.9 Related pages

- Clock framework (CLK) — **rt_clk_set_rate**, consumer clock API
- Regulator — supply rails referenced from OPP / CPU nodes
- SCMI firmware protocol — SCMI clock/regulator when firmware owns scaling resources
- Thermal cooling devices — **thermal-cool-dvfs**

1.4.2.36 CAN device

1.4.2.36.1 CAN bus device framework

RT-Thread exposes CAN controllers as **RT_Device_Class_CAN** devices. The framework in **components/drivers/can/dev_can.c** provides interrupt-driven RX/TX FIFOs, optional **hardware filters**, **CAN-FD** message layout, **blocking** and **non-blocking** send paths, and periodic **bus status** polling. Board and SoC drivers implement **struct rt_can_ops** and register with **rt_hw_can_register()**.
can_dm.c is **not** the bus core — it only supplies ISO 11898 DLC/length helpers (can_dlc2len, can_len2dlc). See CAN FD DLC helpers (`can_dm`).
Header: components/drivers/include/drivers/dev_can.h. Core: components/drivers/can/dev_can.c.

1.4.2.36.1.1 Architecture

```
Application / FinSH
       rt_device_find / open / read / write / control

struct rt_can_device  (dev_can.c)
       software RX FIFO, TX mailboxes, nb_tx ringbuffer
       rt_hw_can_isr()  ← BSP ISR

struct rt_can_ops
       configure / control / sendmsg / recvmsg / sendmsg_nonblocking

SoC CAN / CAN-FD controller (BSP or bsp/*/dm/can/)
```

Component	File	Role
Framework	dev_can.c	Device ops, FIFOs, ISR demux, rt_hw_can_register
FD DLC helpers	can_dm.c, can_dm.h	DLC byte length (FD payloads)
DM SoC example	bsp/rockchip/dm/can/canfd-rockchip.c	OFW platform driver + rt_hw_can_register
Legacy BSP	e.g. bsp/stm32/.../drv_can.c, drv_fdcan.c	HAL-based rt_can_ops

With **RT_USING_DM**, SoC-specific CAN drivers may live under **SOC_DM_CAN_DIR** (see can/Kconfig).

1.4.2.36.1.2 Kconfig

Option	Role
RT_USING_CAN	Build components/drivers/can/
RT_CAN_USING_HDR	Per-filter RX lists and filter callbacks (rt_can_hdr)
RT_CAN_USING_CANFD	64-byte payload, fd_frame / brs, bit-timing fields in can_configure
RT_CANMSG_BOX_SZ	Software RX buffer depth (messages)
RT_CANSND_BOX_NUM	Concurrent blocking TX slots (match HW mailboxes)
RT_CANSND_MSG_TIMEOUT	Blocking send completion timeout (ticks)
RT_CAN_NB_TX_FIFO_SIZE	Non-blocking TX ring buffer (bytes); size N × sizeof(struct rt_can_msg)
RT_CAN_MALLOC_NB_TX_BUFFER	Allocate NB ring at open instead of static pool in rt_can_device

Optional (not in core Kconfig): **RT_CAN_USING_BUS_HOOK** — periodic bus_hook from status timer.

1.4.2.36.2 Access CAN devices (applications)

1.4.2.36.2.1 Find and open

```
#define CAN_DEV_NAME "can1"

rt_device_t candev = rt_device_find(CAN_DEV_NAME);
rt_device_open(candev, RT_DEVICE_FLAG_INT_TX | RT_DEVICE_FLAG_INT_RX);
```

Open flag	Effect
RT_DEVICE_FLAG_INT_RX	Allocates software RX FIFO; driver enables RX IRQ via control(SET_INT, INT_RX)
RT_DEVICE_FLAG_INT_TX	Allocates TX mailbox pool + semaphore; enables TX IRQ
RT_DEVICE_FLAG_RDWR	Set at register time (rt_hw_can_register)

read only works with **INT_RX** and ref_count > 0. **write** requires the device to be opened; size must be a multiple of **sizeof(struct rt_can_msg)**.
Register **rt_device_set_rx_indicate()** to wake a thread when frames arrive (see example in dev_can.h Doxygen block).

1.4.2.36.2.2 Receive

```
struct rt_can_msg rx_msg;
rx_msg.hdr_index = -1;   /* -1: any frame; >=0: specific filter bank (RT_CAN_USING_HDR) */

rt_ssize_t n = rt_device_read(candev, 0, &rx_msg, sizeof(rx_msg));
```

Batch read: pass an array of rt_can_msg and divide returned bytes by sizeof(struct rt_can_msg).

1.4.2.36.2.3 Send — blocking vs non-blocking

Mode	How	Behavior
Blocking	Default (nonblocking == 0), thread context, INT_TX open	Waits for HW mailbox + RT_CAN_EVENT_TX_DONE / FAIL
Non-blocking	msg.nonblocking = 1, or call from ISR	Uses sendmsg_nonblocking; overflow → nb_tx_rb ring
Private mailbox	RT_CAN_CMD_SET_PRIV + msg.priv = mailbox index	_can_int_tx_priv — fixed TX slot per message

```
struct rt_can_msg tx = {0};
tx.id = 0x123;
tx.ide = RT_CAN_STDID;
tx.rtr = RT_CAN_DTR;
tx.len = 8;
tx.data[0] = 0x11;
rt_device_write(candev, 0, &tx, sizeof(tx));   /* blocking */

tx.nonblocking = 1;
rt_device_write(candev, 0, &tx, sizeof(tx));   /* accept or enqueue */
```

RT_CANSND_MSG_TIMEOUT applies to blocking waits. Non-blocking path needs **ops->sendmsg_nonblocking** implemented in the low-level driver.

1.4.2.36.2.4 Control commands (rt_device_control)

Command	Value	Argument	Role
RT_DEVICE_CTRL_CONFIG	—	struct can_configure *	Re-configure (baud, mode, box sizes)

Command	Value	Argument	Role
RT_CAN_CMD_SET_BAUD	0x14	baud (driver-defined)	Set arbitration bitrate
RT_CAN_CMD_SET_BAUD_FD	0x1B	baud	CAN-FD data phase bitrate
RT_CAN_CMD_SET_BITTIMING	0x1C	struct rt_can_bit_timing_config *	Custom prescaler/segments
RT_CAN_CMD_SET_MODE	0x15	mode	RT_CAN_MODE_NORMAL / LISTEN / LOOPBACK / LOOPBACKANLISTEN
RT_CAN_CMD_SET_PRIV	0x16	RT_CAN_MODE_PRIV / NOPRIV	Private TX mailbox mode
RT_CAN_CMD_SET_FILTER	0x13	struct rt_can_filter_config *	HW filters (needs **RT_CAN_USING_HDR**)
RT_CAN_CMD_SET_CANFD	0x1A	enable flag	Enable FD in driver
RT_CAN_CMD_START	0x1D	non-zero / zero	Start or stop controller
RT_CAN_CMD_GET_STATUS	0x17	struct rt_can_status *	Error counters, pkg stats
RT_CAN_CMD_SET_STATUS_IND	0x18	rt_can_status_ind_type_t	User callback on timer tick
RT_CAN_CMD_SET_BUS_HOOK	0x19	rt_can_bus_hook	Periodic hook (if enabled)
RT_DEVICE_CTRL_SET_INT / **CLR_INT**	—	RT_DEVICE_FLAG_INT_RX/TX, RT_DEVICE_CAN_INT_ERR	IRQ mask (used internally on open/close)

Typical bring-up sequence:

1. `rt_device_open(..., INT_TX | INT_RX)`
2. `RT_CAN_CMD_SET_BAUD` / `SET_BITTIMING` / `SET_CANFD` as needed
3. `RT_CAN_CMD_SET_FILTER` (optional)
4. `RT_CAN_CMD_SET_MODE`
5. `RT_CAN_CMD_START` with non-zero argument

On **close**, the framework calls **RT_CAN_CMD_START** with **RT_FALSE** to stop the controller.

Baud presets (enum CANBAUD)

`CAN1MBaud`, `CAN800kBaud`, `CAN500kBaud`, `CAN250kBaud`, `CAN125kBaud`, `CAN100kBaud`, `CAN50kBaud`, `CAN20kBaud`, `CAN10kBaud` — used in **can_configure.baud_rate**.

Default configuration macro

```
struct can_configure cfg = CANDEFAULTCONFIG;
cfg.baud_rate = CAN500kBaud;
cfg.msgboxsz = RT_CANMSG_BOX_SZ;
cfg.sndboxnumber = RT_CANSND_BOX_NUM;
```

1.4.2.36.2.5 Message layout (`struct rt_can_msg`)

Field	Meaning
id (29 bits)	CAN ID
ide	RT_CAN_STDID (0) or RT_CAN_EXTID (1)
rtr	RT_CAN_DTR (data) or RT_CAN_RTR (remote)
len	DLC or payload length (driver may expect bytes or DLC; FD drivers often use can_↩ len2dlc)
priv	TX mailbox index in private mode
hdr_index	RX: matched filter bank; set **-1** before read for any frame
nonblocking	1 → non-blocking send path
fd_frame / **brs**	CAN-FD (RT_CAN_USING_CANFD)
data[]	8 bytes (classic) or 64 bytes (FD)

1.4.2.36.2.6 Hardware filters (`RT_CAN_USING_HDR`)

```
struct rt_can_filter_item items[] = {
    RT_CAN_FILTER_STD_INIT(0x100, RT_NULL, RT_NULL),
    {0x555, 0, 0, 0, 0x7FF, 7},  /* fixed bank #7 */
};
struct rt_can_filter_config cfg = {
    .count = sizeof(items)/sizeof(items[0]),
    .actived = 1,
    .items = items,
};
rt_device_control(candev, RT_CAN_CMD_SET_FILTER, &cfg);
```

Filter field	Role
mode	RT_CAN_MODE_MASK (0) or RT_CAN_MODE_LIST (1)
mask	Mask mode: 1 = bit must match
hdr_bank	-1 = driver assigns; 0 = fixed bank
rxfifo	CAN_RX_FIFO0 or CAN_RX_FIFO1
ind / **args**	Optional per-filter RX callback

Framework mirrors filters into **can->hdr[]** and routes RX frames to per-bank lists when **hdr_index** matches. Helper macros: **RT_CAN_FILTER_ITEM_INIT**, **RT_CAN_FILTER_EXT_INIT**, **RT_CAN_STD_RMT_↩ FILTER_INIT**, etc. (dev_can.h).

1.4.2.36.2.7 CAN-FD

Enable **RT_CAN_USING_CANFD** in Kconfig and in driver:

- **can_configure.enable_canfd**, **baud_rate_fd**
- **use_bit_timing** + **can_timing** / **canfd_timing** (struct rt_can_bit_timing)
- **RT_CAN_CMD_SET_CANFD**, **RT_CAN_CMD_SET_BAUD_FD**, **RT_CAN_CMD_SET_BITTIMING**

Set **rt_can_msg.fd_frame** and **brs** on TX; on RX use **can_dlc2len()** if HW returns DLC nibble (CAN FD DLC helpers (`can_dm`)).

1.4.2.36.2.8 Status and errors

struct rt_can_status — REC/TEC, **errcode** (RT_CAN_BUS_*), packet/drop counters, error type histogram.

Periodic timer (**can_configure.ticks**, default from driver) calls **RT_CAN_CMD_GET_STATUS** and optional **status_indicate** callback.
enum RT_CAN_STATUS_MODE: NORMAL, ERRWARNING, ERRPASSIVE, BUSOFF (derived from **errcode** in FinSH **canstat**).
FinSH (when **RT_USING_FINSH**): **canstat can1** — exported from dev_can.c.

1.4.2.36.2.9 Driver author guide

struct rt_can_ops

Op	Required	Role
configure	Yes	Apply struct can_configure (baud, mode, FD)
control	Yes	IOCTL commands above, SET_INT/CLR_INT
sendmsg	For blocking TX	boxno = mailbox index
recvmsg	For IRQ RX	fifo = HW RX FIFO index; fill rt_can_msg
sendmsg_nonblocking	For NB TX / ISR	Return **RT_EOK** or **-RT_EBUSY**

Register

```
struct rt_can_device my_can;
struct can_configure cfg = CANDEFAULTCONFIG;
/* fill cfg, set my_can.config */
my_can.config = cfg;

rt_err_t rt_hw_can_register(struct rt_can_device *can,
                            const char *name,
                            const struct rt_can_ops *ops,
                            void *user_data);
```

Device name is typically **can0**, **can1**, ... Auto-naming: **rt_dm_dev_set_name_auto(&can->parent, "can")** in DM drivers (Rockchip).

ISR contract — rt_hw_can_isr()

BSP ISR calls **rt_hw_can_isr(can, event)**:

event & 0xff	Meaning
RT_CAN_EVENT_RX_IND (0x01)	Frame received; **event** >> **8** = RX FIFO index
RT_CAN_EVENT_RXOF_IND (0x06)	RX overflow (still tries one recv)
RT_CAN_EVENT_TX_DONE (0x02)	Mailbox **event** >> **8** completed OK
RT_CAN_EVENT_TX_FAIL (0x03)	Mailbox transmission failed

After **TX_DONE**, framework drains **nb_tx_rb** via **sendmsg_nonblocking**.
recvmsg must return byte count or **-1** if no frame. Populate **hdr_index** when hardware reports filter match.

DM platform driver pattern (Rockchip)

1. **RT_PLATFORM_DRIVER_EXPORT** with **rt_ofw_node_id** (compatible)
2. **probe**: map reg/IRQ, **rt_clk_get**, **rt_reset_control_get**, deassert reset
3. Fill **can_configure** (bit timing from DT or constants)
4. **rt_hw_interrupt_install**, **rt_hw_can_register**
5. **remove**: mask IRQ, **rt_device_unregister**

Reference: bsp/rockchip/dm/can/canfd-rockchip.c — uses **can_dm.h** for FD DLC encoding.

BSP drivers using this framework (examples)

BSP / path	Notes
STM32 `drv_can.c` / `drv_fdcan.c`	Classic + FDCAN
NXP i.MX / MCX `drv_can.c`	FlexCAN
WCH, Renesas, Phytium, Synwit, Nuvoton	Various `rt_hw_can_register`
Rockchip `canfd-rockchip.c`	DM + OFW

1.4.2.36.2.10 Engineer checklist

1. **Probe**: reset → clock → pinmux → transceiver enable/regulator → init HW → **`rt_hw_can_register`**
2. **ISR**: keep minimal — **`recvmsg`** + **`rt_hw_can_isr`**; no blocking in ISR
3. **Filters** / **baud**: configure before **`RT_CAN_CMD_START`**
4. **Mailbox count**: set **`sndboxnumber`** to match hardware TX buffers
5. **FD**: increase **`RT_CAN_NB_TX_FIFO_SIZE`** — `sizeof(struct rt_can_msg)` is much larger with 64-byte payload
6. **Termination & transceiver STB**: board-level; not handled by framework

1.4.2.36.2.11 Pitfalls

- **Open without `INT_RX`/`INT_TX`** — `read`/`write` return **`-RT_ENOSYS`**
- **Non-blocking without `sendmsg_nonblocking`** — `write` with `nonblocking=1` fails
- **RX FIFO full** — oldest frame dropped (`dropedrcvpkg`); increase **`msgboxsz`** or read faster
- **Bus-off** — check **`RT_CAN_CMD_GET_STATUS`**; recovery is driver/BSP specific
- **DLC vs length on FD** — use **`can_len2dlc`**/**`can_dlc2len`** consistently (CAN FD DLC helpers (`can_dm`))
- **Multiple `open`** — `ref_count` > 1 skips FIFO teardown on close; design single-owner stacks

1.4.2.36.2.12 See also

- CAN FD DLC helpers (`can_dm`) — FD DLC helpers
- `components/drivers/include/drivers/dev_can.h` — full API and application example
- `components/drivers/can/dev_can.c`
- `components/drivers/can/readme-zh.txt` — legacy Chinese notes for driver porting
- Open Firmware (OFW) — DT matching for DM CAN
- Clock framework (CLK), Reset controller, Regulator — probe dependencies

1.4.2.37 CAN FD DLC helpers (`can_dm`)

1.4.2.37.1 `can_dm` utilities

Header: **`components/drivers/can/can_dm.h`** (also installable as `drivers/can_dm.h` via include path). Implementation: **`components/drivers/can/can_dm.c`** — **only** ISO 11898 DLC/length conversion; **no** device register, OFW, or ISR code.
The CAN **device framework** is **`dev_can.c`** — see CAN device.

1.4.2.37.1.1 APIs

```
rt_uint8_t can_dlc2len(rt_uint8_t can_dlc);
rt_uint8_t can_len2dlc(rt_uint8_t len);
```

`can_dlc2len`

Maps **CAN FD DLC nibble** (0–15) to **payload byte length**:

DLC	0–8	9–12	13–16	17–20	21–24	25–32	33–48	49–64
Bytes	0–8	12	16	20	24	32	48	64

Only low **4 bits** of `can_dlc` are used (`can_dlc & 0x0F`).

`can_len2dlc`

Inverse mapping for lengths **0–64**; returns **`0xf`** if **`len > 64`**.

1.4.2.37.1.2 Usage in controllers

FD-capable drivers should:

1. Store/transmit hardware DLC using **`can_len2dlc(actual_payload_len)`**.
2. On RX, convert to byte count with **`can_dlc2len(rx_frame.dlc)`** before copying to upper layers.

Classic CAN (8 bytes) uses DLC 0–8 with identity mapping in both tables.

1.4.2.37.1.3 CAN ID flags (header only)

`can_dm.h` also defines Linux-style ID flags for drivers that pack ID + type into one word:

Macro	Value	Meaning
`CAN_EFF_FLAG`	`0x80000000`	Extended frame
`CAN_RTR_FLAG`	`0x40000000`	Remote frame
`CAN_ERR_FLAG`	`0x20000000`	Error frame
`CAN_SFF_MASK` / **`CAN_EFF_MASK`**	—	ID field masks

These are **not** used by `dev_can.c` (`rt_can_msg` uses separate `ide`/`rtr` bits).

1.4.2.37.1.4 Driver / BSP quick notes

Topic	Guidance
Not a DM bus	No `RT_CAN_OFW_DECLARE` in core — platform drivers (e.g. Rockchip `canfd-rockchip.c`) own OFW probe
FD vs classic	Do not use DLC > 8 on classic-only controllers
Include	`#include "can_dm.h"` or path from driver directory
`can_get_dlc` / **`canfd_get_dlc`**	Clamp DLC before `can_dlc2len`

1.4.2.37.1.5 See also

- CAN device
- `components/drivers/can/can_dm.c`

- components/drivers/can/dev_can.c

1.4.2.38 Clock framework (CLK)

1.4.2.38.1 Common Clock Framework (CCF)

RT-Thread's **CLK** subsystem (components/drivers/clk/) is the **Device Model (DM)** clock provider layer: device tree **clocks** / **#clock-cells** resolve to **struct rt_clk** handles, with **reference-counted** prepare / enable and optional **rate** / **parent** changes. It mirrors the Linux common clock model closely enough to reuse **dt-bindings** and DTS fragments.
Header: components/drivers/include/drivers/clk.h. Core: components/drivers/clk/clk.c.
Built-in providers in-tree:

Module	Role
clk-fixed-rate.c	DT **fixed-clock** — Fixed clock provider
clk-scmi.c	Clocks owned by **ARM SCMI** firmware — needs **RT_CLK_SCMI**
bsp/*/dm/clk/ (e.g. Rockchip)	SoC CCM / PLL / mux / gate — via **SOC_DM_CLK_DIR**

1.4.2.38.1.1 Kconfig

Option	Role
RT_USING_DM	Required parent — CLK is a DM feature
RT_USING_CLK	Build clk.c + selected providers; selects **RT_USING_ADT_REF**
RT_CLK_SCMI	clk-scmi.c; depends on **RT_FIRMWARE_ARM_SCMI**
SOC_DM_CLK_DIR	SoC adds Kconfig/SConscript under BSP dm/clk/

Boot argument (OFW): **clk_ignore_unused** — when set, cells with **RT_CLK_F_IGNORE_UNUSED** skip **disable** even at refcount zero (debug / bring-up).

1.4.2.38.1.2 Architecture

```
Device tree consumer node
    clocks = <&cru CLK_UART2_SCLK>;
    clock-names = "baudclk", "apb_pclk";


rt_clk_get_by_name(dev, "baudclk")  /  rt_ofw_get_clk(np, 0)


struct rt_clk  (consumer handle: dev_id, con_id, min/max rate)


struct rt_clk_cell  (one output: PLL child, gate, mux, fixed, ...)


struct rt_clk_node  (provider: registered with rt_clk_register)


rt_clk_ops  (prepare/enable/recalc_rate/set_rate/set_parent/...)
```

Object	Meaning
rt_clk_node	One **clock controller** in DT (e.g. &cru, fixed-clock, SCMI clock device)
rt_clk_cell	One **output clock** from that controller (index from #clock-cells)

Object	Meaning
rt_clk	**Consumer reference** to a cell; allocated per `get`
rt_clk_array	All clocks listed in consumer's **clocks** property

Provider registration stores the node on a global list (`RT_CLK_NODE_OBJ_NAME` = **"CLKNP"**) and attaches **rt_ofw_data(np)** so phandle lookup works.

1.4.2.38.1.3 Consumer API (platform drivers)

Typical **probe** sequence on a DM device (`struct rt_device *dev` with **dev->ofw_node**):

```
struct my_dev {
    struct rt_clk *clk;
    struct rt_clk *pclk;
};

/* By connection name (matches clock-names) */
pdev->clk = rt_clk_get_by_name(dev, "spiclk");
pdev->pclk = rt_clk_get_by_name(dev, "apb_pclk");

/* Or all clocks at once */
pdev->clk_arr = rt_clk_get_array(dev);

if (rt_clk_prepare_enable(pdev->pclk))
    return err;

if (rt_clk_prepare_enable(pdev->clk))
    goto disable_pclk;

freq = rt_clk_get_rate(pdev->clk);
```

API	Role
rt_clk_get_array(dev)	Parse **clocks** + **clock-names** → `rt_clk_array`
rt_clk_get_by_index(dev, i)	Nth entry in **clocks**
rt_clk_get_by_name(dev, name)	Match **clock-names**; fallback: scan registered cells by **cell->name**
rt_clk_prepare / **unprepare**	May sleep (rails, firmware); refcounted per **cell**
rt_clk_enable / **disable**	Fast gate; enables **parent** first; refcounted
rt_clk_prepare_enable	`prepare` then `enable` (common in probe)
rt_clk_disable_unprepare	Reverse; use in `remove` / error paths
rt_clk_get_rate / **set_rate** / **round_rate**	Rate query and change
rt_clk_set_parent / **get_parent**	Reparent mux
rt_clk_set_rate_range	Consumer min/max constraints
rt_clk_put / **rt_clk_array_put**	Free consumer handles (does not disable)

NULL clock: `rt_clk_enable(NULL)` and `rt_clk_prepare(NULL)` return **RT_EOK** — useful for optional clocks; still check **rt_clk_get_*** return before use.
Array helpers: **rt_clk_array_prepare_enable**, **rt_clk_array_disable_unprepare**, etc. — stop on first error during prepare/enable loops.

OFW-only helpers

API	Role
rt_ofw_get_clk(np, index)	Resolve phandle without `rt_device`
rt_ofw_get_clk_by_name(np, name)	By **clock-names**

API	Role
rt_ofw_get_clk_array(np)	Full **clocks** list
rt_ofw_count_of_clk(clk_np)	Provider output count
rt_ofw_clk_get_parent_name(np, index)	Debug / parent name
rt_ofw_clk_set_defaults(np)	Apply **assigned-clocks*** (see below)

If the provider is not registered yet, **ofw_get_clk** may call **rt_platform_ofw_request(clk_ofw_np)** to defer probe.

1.4.2.38.1.4 Device tree (consumers)

```
uart2: serial@ff1a0000 {
    compatible = "rockchip,rk3588-uart", "snps,dw-apb-uart";
    reg = <0x0 0xff1a0000 0x0 0x100>;
    clocks = <&cru CLK_UART2_SRC>, <&cru CLK_UART2_FRAC>, <&cru PCLK_UART2>;
    clock-names = "baudclk", "apb_pclk";
    assigned-clocks = <&cru CLK_UART2_SRC>;
    assigned-clock-rates = <24000000>;
};
```

Property	Role
clocks	Phandle + `#clock-cells` specifier per input
clock-names	Stable names for **rt_clk_get_by_name**
assigned-clocks	Clocks to configure when provider registers
assigned-clock-parents	Optional parent for each assigned clock
assigned-clock-rates	Optional Hz for each assigned clock

rt_clk_register() calls **rt_ofw_clk_set_defaults(dev->ofw_node)** when the provider has a bound OFW node — applies assigned settings **before** consumers rely on rates.
Provider node exports clocks:

```
cru: clock-controller@fd7c0000 {
    compatible = "rockchip,rk3588-cru";
    #clock-cells = <1>;
    clock-output-names = "clk_uart2_src", "...";
};
```

Custom providers with multi-cell specifiers implement **clk_np->ofw_parse(np, args)**; default parser uses **args->args[0]** as cell index. **clock-indices** remaps DT indices to internal cell order.

1.4.2.38.1.5 Provider API (clock drivers)

struct rt_clk_ops

Callback	Typical use
prepare / **unprepare**	PMIC / slow path; paired refcount
enable / **disable**	Clock gate bit
is_prepared / **is_enabled**	Query HW state (optional)
recalc_rate	Return Hz from parent rate + dividers
round_rate	Snap requested rate
set_rate	Program dividers / PLL
set_parent / **get_parent**	Mux select
set_phase / **get_phase**	DDR / SDIO phase (optional)

Leave unused ops **NULL**; core skips them.

struct rt_clk_cell flags

Flag	Effect
RT_CLK_F_SET_RATE_GATE	Gate across rate change
RT_CLK_F_SET_PARENT_GATE	Gate across reparent
RT_CLK_F_SET_RATE_PARENT	Propagate rate change to parent
RT_CLK_F_IGNORE_UNUSED	Skip disable when unused (with **clk_ignore_unused**)
RT_CLK_F_IS_CRITICAL	Never auto-disable
RT_CLK_F_GET_RATE_NOCACHE	Always call **recalc_rate**

Register a provider

```
struct my_cru {
    struct rt_clk_node clk_np;
    struct rt_clk_cell *cells[N];
    /* ... */
};

static struct rt_clk_cell *my_cru_ofw_parse(struct rt_clk_node *np,
        struct rt_ofw_cell_args *args)
{
    rt_uint32_t idx = args->args[0];
    if (idx >= np->cells_nr)
        return RT_NULL;
    return np->cells[idx];
}

/* In probe, after filling cells[] and ops */
cru->clk_np.dev = dev;
cru->clk_np.cells = cru->cells;
cru->clk_np.cells_nr = num_clocks;
cru->clk_np.ofw_parse = my_cru_ofw_parse;

err = rt_clk_register(&cru->clk_np);
```

If the provider itself has parent clocks in DT, set **clk_np->dev** before register — core calls **rt_clk_get_↩ array(dev)** into **clk_np->parents_clk**.
rt_clk_unregister() — rollback only; does not free cells.

Notifiers

rt_clk_notifier_register(clk, notifier) — callback on **RT_CLK_MSG_PRE_RATE_CHANGE**, **POST_RATE_CHANGE**, **ABORT_RATE_CHANGE**.

1.4.2.38.1.6 Reference counting behavior

- **prepare_count** / **enable_count** are per **rt_clk_cell**, shared by all consumers of that cell.
- **Enable** walks **parent** chain first; **disable** walks parent last (after local gate).
- **rt_clk_put** frees only the **consumer** `struct rt_clk` if it is not the cell's primary binding — **does not** disable the clock; call **rt_clk_disable_unprepare** before remove.

1.4.2.38.1.7 SCMI clocks

With **RT_CLK_SCMI**, firmware exposes clocks via **struct rt_scmi_device**. **clk-scmi.c** probes SCMI, enumerates clock IDs, and registers one **rt_clk_node** with many cells (`enable/disable/set_rate` via SCMI protocol).
Consumers use the same **rt_clk_get_*** APIs; DT points at the SCMI clock provider node. Details: SCMI firmware protocol and `components/drivers/clk/clk-scmi.c`.

1.4.2.38.1.8 SoC drivers (Rockchip example)

bsp/rockchip/dm/clk/ implements **rockchip,*-cru** style controllers: PLL (`clk-rk-pll.c`), mux, gate, divider, composite, CPU clock, MMC phase, etc. Kconfig: **RT_CLK_ROCKCHIP**, per-SoC **RT_CLK_↩ ROCKCHIP_RK3588**, ...

Consumer drivers (SPI, UART, ADC, PCIe, . . .) only call **rt_clk_get_by_name** + **rt_clk_prepare_enable** — they do not touch CCM registers directly.

1.4.2.38.1.9 Debug

MSH (**RT_USING_CONSOLE** + **RT_USING_MSH**):

`list_clk`

Dumps each registered **cell**: name, enable/prepare counts, rate, consumer `dev_id` / `con_id`, parent name.

1.4.2.38.1.10 When to use CLK vs direct CCM writes

Use **rt_clk_***	Direct register writes may remain
DT lists **clocks = <&provider ...>**	Early asm/ROM before DM
Multiple drivers share PLL parents	One-off bring-up with no DT
assigned-clocks / DVFS coordination	Fixed, always-on root osc (often still modeled as **fixed-clock**)

1.4.2.38.1.11 Engineer checklist

1. **Provider**: implement **rt_clk_ops**, **ofw_parse**, **rt_clk_register**; publish **clock-output-names**.
2. **Consumer**: **rt_clk_get_by_name** → **rt_clk_prepare_enable** → **rt_clk_get_rate** for divider config.
3. **Remove**: **rt_clk_disable_unprepare** in reverse order, then **rt_clk_put** / **rt_clk_array_put**.
4. **Probe order**: provider must register before consumers resolve phandles (deferred probe handles late providers).
5. **Optional clocks**: missing phandle → **NULL**; do not enable.

1.4.2.38.1.12 Pitfalls

- **rt_clk_put without disable** — leaves clocks gated on and burns power.
- **Wrong #clock-cells** — `ofw_get_clk` fails or picks wrong mux output.
- **Rate set before enable** — some IPs need **RT_CLK_F_SET_RATE_GATE** or enable-first ordering.
- **assigned-clocks on consumer vs provider** — defaults run on the node that **owns** the `assigned-clocks` property when **that** node's provider registers.
- **ISR context** — do not call **prepare** / **prepare_enable** (**RT_DEBUG_NOT_IN_INTERRUPT**).

1.4.2.38.1.13 Detailed documents

- Fixed clock provider — **fixed-clock** provider

1.4.2.38.1.14 See also

- `components/drivers/include/drivers/clk.h`
- `components/drivers/clk/clk.c`
- Open Firmware (OFW) — phandle / `clock-indices`
- SCMI firmware protocol — SCMI clock protocol
- Reset controller — often sequenced with clock enable
- Regulator, Power domain — rails with prepare

1.4.2.39 Fixed clock provider

1.4.2.39.1 `fixed-clock` platform driver

Implementation: **`components/drivers/clk/clk-fixed-rate.c`**. Driver name: **`clk-fixed-rate`**. Overview: Clock framework (CLK).
Registers a **constant-frequency**, **always-on** clock with no gate, parent, or rate programming — the leaf most SoC trees hang PLLs from.

1.4.2.39.1.1 Device tree

```
osc24m: fixed-clock {
    compatible = "fixed-clock";
    #clock-cells = <0>;
    clock-frequency = <24000000>;
    clock-accuracy = <0>;          /* optional, ppb */
    clock-output-names = "osc24m"; /* optional; becomes cell name */
};
```

Consumers:

```
uart0: serial@40000000 {
    clocks = <&osc24m>;
    /* clock-names optional when only one clock */
};
```

With **`#clock-cells = <0>`**, the specifier in **`clocks = <&osc24m>`** has **no extra integers** (implicit index 0).

1.4.2.39.1.2 Probe flow

1. **`rt_platform_driver_register`** via **`INIT_SUBSYS_EXPORT(fixed_clk_drv_register)`** — early registration so crystal nodes probe before CCM consumers.
2. **`fixed_clk_probe`**: `rt_calloc` **`struct clk_fixed`** (embeds **`rt_clk_node`** + one **`rt_clk_fixed_rate`**).
3. **`rt_dm_dev_prop_read_u32(dev, "clock-frequency", &val)`** — **required**; probe fails without it.
4. Optional **`clock-accuracy`**, **`clock-output-names`** → **`fcell.cell.name`**.
5. Wire **`cells[0]`**, **`fixed_clk_ops`** (only **`.recalc_rate`**).
6. **`rt_clk_register(&cf->parent)`** — may run **`rt_ofw_clk_set_defaults`** if the fixed-clock node has **`assigned-clocks`*** (unusual on leaf osc).

1.4.2.39.1.3 Data structures

```
struct clk_fixed {
    struct rt_clk_node parent;
    struct rt_clk_fixed_rate fcell;
    struct rt_clk_cell *cells[1];
};

struct rt_clk_fixed_rate {
    struct rt_clk_cell cell;
    rt_ubase_t fixed_rate;
    rt_ubase_t fixed_accuracy;
};
```

Member	Role
`fixed_rate`	Hz from **`clock-frequency`**
`fixed_accuracy`	Optional ppb from DT
`cell.ops->recalc_rate`	Returns **`fixed_rate`**; ignores parent

No **`enable`**/**`disable`**/**`prepare`** — enabling is a no-op at ops level; framework refcount still works for consumers.

1.4.2.39.1.4 OFW matching

```
static const struct rt_ofw_node_id fixed_clk_ofw_ids[] = {
    { .compatible = "fixed-clock" },
    { /* sentinel */ }
};
```

Linux-compatible binding; same node can be shared across BSP DTS and upstream dt-bindings.

1.4.2.39.1.5 When to use vs SoC CCM

Use `fixed-clock`	Use SoC driver (e.g. `rockchip,*-cru`)
Crystal, RC oscillator, external clock input	PLL, mux, divider, gate
Rate fixed at boot	DVFS, dynamic reparent
No MMIO (or MMIO not managed here)	Large clock controller IP

1.4.2.39.1.6 Consumer example

```
struct rt_clk *clk = rt_clk_get_by_name(dev, "uart_clk");
/* or index 0 if no clock-names */
rt_err_t err = rt_clk_prepare_enable(clk);
rt_ubase_t hz = rt_clk_get_rate(clk);
```

Fixed clocks rarely need **`rt_clk_set_rate`** — rate is fixed; **`rt_clk_get_rate`** reads **`recalc_rate`**.

1.4.2.39.1.7 Pitfalls

- **Missing `clock-frequency`** — probe fails, phandle resolution returns NULL for consumers.
- **Wrong Hz in DT** — all derived PLL math wrong; UART baud, SDIO timing, etc. break silently.
- **Using `fixed-clock` for gated peripherals** — use a gate driver cell instead.
- **Name collisions** — **`clock-output-names`** should be unique enough for **`rt_clk_get_by_name`** fallback lookup.

1.4.2.39.1.8 See also

- Clock framework (CLK)
- `components/drivers/clk/clk-fixed-rate.c`
- `components/drivers/include/drivers/clk.h`

1.4.2.40 Bus and driver binding (core)

1.4.2.40.1 Device model: bus and driver

RT-Thread **DM** binds **drivers** to **devices** through **`struct rt_bus`**: each bus keeps linked lists of devices and drivers, runs **`match`**, then **`probe`** / **`remove`** / **`shutdown`**. Most SoC drivers use the built-in **`platform`** bus; other stacks (e.g. **RPMsg**) register their own bus type on the same core.
Headers: `components/drivers/include/drivers/core/bus.h`, `driver.h`. Core: `components/drivers/core/bus.c`. Reference bus: `components/drivers/core/platform.c`.
Device-side helpers (`rt_dm_dev_iomap`, properties, IRQ): Device model helpers (dm.h).

1.4.2.40.1.1 Kconfig

Option	Role
`RT_USING_DM`	Enables **`bus.c`**, **`driver.c`**, platform bus, and DM helpers

Option	Role
RT_USING_DEV_BUS	Optional **rt_device_bus_create** — bus as **RT_Device_Class_Bus** device
RT_USING_OFW	Platform **match** uses **compatible**; devices carry **dev->ofw_node**
RT_USING_PINCTRL	Platform **probe** applies **pinctrl-*** before driver **probe**
RT_USING_CLK	Platform **probe** runs **rt_ofw_clk_set_defaults**

1.4.2.40.1.2 Architecture

```
rt_bus_register(bus)
       |
       v
struct rt_bus (platform, rpmsg, ...)
  dev_list --> rt_device
  drv_list --> rt_driver
  match / probe / remove / shutdown
```

Binding (bus.c): **rt_bus_add_device** walks **drv_list**; **rt_bus_add_driver** walks **dev_list**. **match** must succeed before **dev->drv** is set and **bus->probe(dev)** runs. Failed probe clears **dev->drv**.

Object	Role
struct rt_bus	Bus type and lifecycle hooks
struct rt_driver	Driver on a bus; **ref_count** = bound devices
struct rt_platform_device	Platform device embedding **rt_device**

1.4.2.40.1.3 struct rt_bus callbacks

Callback	Contract
match	**Pure** — no HW access; return **RT_TRUE** to bind
probe	Bus-level setup (platform: pinctrl, clocks, power domain) then driver **probe**
remove	Teardown; platform calls **pdrv->remove** and **rt_platform_ofw_free**
shutdown	Invoked from **rt_bus_shutdown()** on system off

1.4.2.40.1.4 Bus and driver API

API	Role
rt_bus_register	Register bus on global **bus_nodes**
rt_bus_add_device / **rt_bus_add_driver**	Add node and run matching
rt_bus_remove_device / **rt_bus_remove_driver**	Remove; driver busy if **ref_count** > **0**
rt_bus_for_each_dev / **rt_bus_for_each_drv**	Iterate under spinlock
rt_bus_find_by_name	Find **"platform"** etc.
rt_bus_reload_driver_device	Move device to another bus
rt_bus_shutdown	Shutdown all devices on all buses
rt_driver_register / **unregister**	Driver attach/detach

1.4.2.40.1.5 RT_DRIVER_EXPORT / platform

```
#define RT_DRIVER_EXPORT(driver, bus_name, mode)  /* INIT_DEVICE_EXPORT */
#define RT_PLATFORM_DRIVER_EXPORT(driver) \
    RT_DRIVER_EXPORT(driver, platform, BUILIN)
```

Example:

```
static const struct rt_ofw_node_id my_ids[] = {
    { .compatible = "vendor,my-ip" },
    { /* sentinel */ },
};

static struct rt_platform_driver my_drv = {
    .name = "my-ip",
    .ids  = my_ids,
    .probe  = my_probe,
    .remove = my_remove,
};
RT_PLATFORM_DRIVER_EXPORT(my_drv);
```

1.4.2.40.1.6 Platform bus

INIT_CORE_EXPORT(platform_bus_init) registers **name = "platform"**.

platform_match

1. **rt_ofw_node_match(np, pdrv->ids)** when **dev->ofw_node** is set.
2. Else compare **pdev->name** and **pdrv->name**.

platform_probe (before pdrv->probe)

1. Pinctrl default / index 0 — Pin control (pinctrl)
2. **rt_ofw_clk_set_defaults** — Clock framework (CLK)
3. **rt_dm_power_domain_attach(dev, RT_TRUE)** — **-RT_EEMPTY** OK — Power domain
4. **pdrv->probe(pdev)** — map MMIO, clocks, register **rt_device**

OFW API	Role
rt_platform_ofw_request	Instantiate from DT node
rt_platform_ofw_device_probe_child	Probe children
rt_platform_ofw_free	Free OFW state on remove

See Platform bus, Open Firmware (OFW).

1.4.2.40.1.7 Custom rt_bus

New bus when...	Use platform when...
Non-OFW enumeration (RPMsg, internal stack bus)	**/soc** MMIO + **compatible**

Implement **match/probe/remove**, **rt_bus_register** at init, then **rt_bus_add_driver** / **rt_bus_add_device**.

1.4.2.40.1.8 SDIO vs core

MMC-specific DM: SDIO device model. This page covers **rt_bus** / **rt_driver** only.

1.4.2.40.1.9 Power and probe order

Topic	Document
System shutdown	System power off and reboot (core)
Power domains	Power domain
Regulators	Regulator

Typical order: **regulator** → **power domain** → **clock/reset** → **`rt_dm_dev_iomap`**.

1.4.2.40.1.10 Engineer checklist

1. Export driver with **`RT_PLATFORM_DRIVER_EXPORT`** (or subsystem **`RT_DRIVER_EXPORT`**).
2. Request device via **`rt_platform_ofw_request`** or manual register.
3. In **`probe`**: resources → HW → register functional device; unwind on error.
4. In **`remove`**: reverse order; disable IRQ before free.

1.4.2.40.1.11 Pitfalls

- **`match` with side effects** breaks re-probe.
- **Double `rt_bus_add_device`** without remove.
- **`rt_driver_unregister` while devices bound** — **`-RT_EBUSY`**.
- **Link order is not probe order** — defer on missing phandles.

1.4.2.40.1.12 See also

- Device model helpers (dm.h)
- System power off and reboot (core)
- Platform bus
- Open Firmware (OFW), Devicetree Compiler
- SDIO device model
- `components/drivers/core/bus.c`, `platform.c`

1.4.2.41 Device model helpers (dm.h)

1.4.2.41.1 Device-side DM helpers

Header: `components/drivers/include/drivers/core/dm.h`. Implementation: `components/drivers/core/dm.c`.
These APIs sit **below** bus matching (`documentation/6.components/device-driver/core/bus.md`) and **above** subsystem drivers: a `probe` typically calls **`rt_dm_dev_iomap`**, **`rt_dm_dev_get_irq`**, and property readers, then registers the functional device (block, netdev, etc.).

1.4.2.41.1.1 Driver / BSP quick notes

Topic	Guidance
`ofw_node` required	Most helpers return **`-RT_ENOSYS`** or **NULL** when **`dev->ofw_node`** is unset—bind the node in platform probe or **`rt_dm_dev_bind_fwdata`**.

Topic	Guidance
IRQ needs PIC	**rt_dm_dev_get_irq*** is compiled only with **RT_USING_OFW** and **RT_USING_PIC**.
Property errors	Inline **rt_dm_dev_prop_read_u32** returns **RT_EOK** only when the read count is > **0**; negative counts are cast to **rt_err_t**—check return values.
IDA names	**rt_dm_ida_alloc** is per **master_id****do not mix block and other masters on the same **struct rt_dm_ida**.

1.4.2.41.1.2 ID allocation (rt_dm_ida)

Used when a subsystem needs stable numeric suffixes (e.g. **nvme0**, partition children):

API	Role
rt_dm_ida_init(ida, id)	Initialize bitmap + **MASTER_ID_*** from **master_id.h**.
rt_dm_ida_alloc(ida)	Allocate lowest free id in **0 ... 255**.
rt_dm_ida_take(ida, id)	Reserve a specific id (returns **RT_FALSE** if taken).
rt_dm_ida_free(ida, id)	Release id.
rt_dm_device_find(master_id, device_id)	Find **rt_device** by master/slave id pair.

1.4.2.41.1.3 Device naming

API	Role
rt_dm_dev_set_name(dev, fmt, ...)	Format **dev->parent.name** (truncated to **RT_NAME_MAX**).
rt_dm_dev_set_name_auto(dev, prefix)	Append auto id from internal IDA.
rt_dm_dev_get_name / rt_dm_dev_get_name_id	Query current name / numeric suffix.

1.4.2.41.1.4 Address and MMIO

API	Role
rt_dm_dev_get_address_count	Number of **reg** windows (OFW).
rt_dm_dev_get_address / _by_name	Physical base + size for index or named resource.
rt_dm_dev_get_address_array	Pack multiple regions into **out_regs**.
rt_dm_dev_iomap / _by_name	Map **reg** index or name to CPU virtual address.

Without **RT_USING_OFW**, address/IOMAP calls do not resolve hardware description.

1.4.2.41.1.5 Interrupts

API	Role
rt_dm_dev_get_irq_count	Number of interrupt specifiers.

API	Role
`rt_dm_dev_get_irq(dev, index)`	Linux-style interrupt index (**0** = first cell group).
`rt_dm_dev_get_irq_by_name`	Match **`interrupt-names`**.

Install the handler in your driver after obtaining the vector; PIC maps OFW cells to a usable IRQ number.

1.4.2.41.1.6 Firmware properties

Typed readers (arrays and index variants):

- **`rt_dm_dev_prop_read_u8/u16/u32/u64`** and **`_array_index`**
- **`rt_dm_dev_prop_read_string`** / **`_array_index`**
- **`rt_dm_dev_prop_read_bool`**
- **`rt_dm_dev_prop_count_of_`**∗, **`rt_dm_dev_prop_index_of_string`**
- **`rt_dm_dev_get_prop_fuzzy_name`** — suffix/substring match for property names

`rt_dm_dev_is_big_endian(dev)` reflects the device endianness flag for MMIO accessors.

1.4.2.41.1.7 Bind driver private data to OFW node

```
void rt_dm_dev_bind_fwdata(rt_device_t dev, void *fw_np, void *data);
void rt_dm_dev_unbind_fwdata(rt_device_t dev, void *fw_np);
```

- Sets **`dev->ofw_node`** if it was NULL and assigns **`rt_ofw_data(np) = data`** (controller **`priv`**, **`rt_mbox_controller`**, etc.).
- Consumers resolve providers via **`rt_ofw_data(provider_np)`** after **`rt_platform_ofw_request`**.

1.4.2.41.1.8 SMP

`rt_dm_secondary_cpu_init()` is the BSP hook for secondary CPUs in DM bring-up (weak/no-op if unused).

1.4.2.41.1.9 Typical `probe` sequence

1. Match on **`compatible`** (platform bus).
2. **`rt_dm_dev_bind_fwdata(dev, np, priv)`**.
3. Optional **`rt_dm_power_domain_attach(dev, RT_TRUE)`** — see **`documentation/6.components/device-driver/core/power.md`**.
4. **`regs = rt_dm_dev_iomap(dev, 0)`**, **`irq = rt_dm_dev_get_irq(dev, 0)`**.
5. Read **`clocks`** / **`resets`** / **`dmas`** via property helpers or dedicated frameworks.
6. Register **`rt_device`** / subsystem object; on failure unwind in reverse order.

1.4.2.41.1.10 Pitfalls

- **Double `bind_fwdata`** with conflicting **`data`** pointers confuses mailbox/clock clients—one owner per OFW node.
- **`iomap` without `reg`**: returns **NULL**; verify DTS and **`status = "okay"`**.
- **Assuming `prop_read_`∗ zero-fills**: on failure, output may be untouched—initialize locals first.

1.4.2.41.1.11 See also

- Bus / driver binding: `documentation/6.components/device-driver/core/bus.md`
- Platform bus: `documentation/6.components/device-driver/platform/platform.md`
- OFW runtime: `documentation/6.components/device-driver/ofw/ofw.md`
- Power off / domains: `documentation/6.components/device-driver/core/power.md`, `documentation/6.components/device-driver/power_domain/power_domain.md`
- Header: `components/drivers/include/drivers/core/dm.h`

1.4.2.42 System power off and reboot (core)

1.4.2.42.1 DM power control (`power.h`)

Header: `components/drivers/include/drivers/core/power.h`. Implementation: `components/drivers/core/power.c`.
This layer handles **system-wide shutdown and reset**, **ordered power-off callbacks**, and **reboot mode** strings—not battery/charger UI (**`documentation/6.components/device-driver/power/supply.md`**) and not per-IP **power domains** (**`documentation/6.components/device-driver/power_domain/power_domain.md`**).

1.4.2.42.1.1 Driver / BSP quick notes

Topic	Guidance
Callback context	**`rt_dm_power_off_handler`** callbacks run from **`rt_hw_cpu_shutdown`** / **`rt_hw_cpu_reset`** paths—may be IRQ or dedicated power thread; **do not sleep** unless your platform guarantees thread context.
Priority bands	Handlers run **low index first** within each **`rt_dm_power_off_priority`** level, then next priority—register reboot-mode notifiers at **`RT_DM_POWER_OFF_PRIO_HIGH`** if they must run before generic GPIO poweroff.
`-RT_EEMPTY` on domain attach	**`rt_dm_power_domain_attach`** returning **`-RT_EEMPTY`** means no **`power-domains`** in DTS—often OK; RPMsg/platform code treats it as non-fatal.

1.4.2.42.1.2 Power-off handlers

```
rt_err_t rt_dm_power_off_handler(struct rt_device *dev, int mode, int priority,
        rt_err_t (*callback)(struct rt_device *));
```

`mode`	Use
`RT_DM_POWER_OFF_MODE_SHUTDOWN`	Full power down (PMIC off, etc.).
`RT_DM_POWER_OFF_MODE_RESET`	Warm reset path before **`rt_hw_cpu_reset`**.

`priority` (increasing order)	Typical registrant
`RT_DM_POWER_OFF_PRIO_PLATFORM`	SoC-specific flush
`RT_DM_POWER_OFF_PRIO_LOW` / `DEFAULT` / `HIGH`	Drivers, storage sync
`RT_DM_POWER_OFF_PRIO_FIRMWARE`	Last-chance firmware handoff

Registration appends to a per-mode, per-priority singly linked list. On shutdown/reset the core walks **all priority levels** and invokes each callback; a non-**RT_EOK** return is logged but does not stop later handlers.
Examples: components/drivers/power/reset/gpio-poweroff.c and **gpio-restart.c** register GPIO drivers via **rt_dm_power_off_handler**.

1.4.2.42.1.3 Machine hooks

```
extern void (*rt_dm_machine_shutdown)(void);
extern void (*rt_dm_machine_reset)(void);
```

BSP or SoC code sets these to enter PMIC/PSCI/firmware sequences after DM handlers run.

1.4.2.42.1.4 Reboot mode

```
rt_err_t rt_dm_reboot_mode_register(struct rt_device *dev,
        rt_err_t (*callback)(struct rt_device *, char *cmd));
rt_err_t rt_dm_reboot_mode_unregister(struct rt_device *dev);
void rt_hw_cpu_reset_mode(char *cmd);
```

- **rt_hw_cpu_reset_mode** updates the global command string (e.g. **"recovery"**) and calls **rt_hw_cpu_reset()**.
- On reset, handlers registered at **RT_DM_POWER_OFF_MODE_RESET** run—core init registers **dm_reboot_notifiy** at **RT_DM_POWER_OFF_PRIO_HIGH** to apply reboot-mode callbacks before the CPU resets.

Use **syscon-reboot-mode** / NVMEM-backed mode drivers to persist **cmd** across boots.

1.4.2.42.1.5 Relation to power domains

rt_dm_power_domain_attach(dev, on) (in **power_domain.h**) is called from **platform probe** and some buses (e.g. RPMsg) to power **on** the device's IP block **before** driver init. That is independent of **system** shutdown:

API	Scope
rt_dm_power_domain_*	Per-device SoC power island while system is running.
rt_dm_power_off_handler	Whole-system off/reset.
rt_power_supply_*	Charger/battery reporting.

Recommended order for a consumer **probe**: **regulator enable** → **power domain on** → **clock/reset** → **rt_dm_dev_iomap**.

1.4.2.42.1.6 Pitfalls

- **Missing rt_dm_machine_shutdown**: handlers run but hardware never powers off—set machine hooks in BSP.
- **Reboot mode without storage**: **cmd** is RAM-only unless a **reboot-mode** driver writes persistent storage.
- **GPIO poweroff polarity**: wrong active level looks like "hang on shutdown"—verify DTS and **gpio-poweroff** properties.

1.4.2.42.1.7 See also

- Power domains: documentation/6.components/device-driver/power_domain/power_domain.md
- Power supply (battery/charger): documentation/6.components/device-driver/power/supply.md

- Regulator: documentation/6.components/device-driver/regulator/regulator.md
- Syscon reboot: components/drivers/power/reset/syscon-reboot.c, syscon-reboot-mode.c
- DM helpers: documentation/6.components/device-driver/core/dm.md
- Header: components/drivers/include/drivers/core/power.h

1.4.2.43 DMA engine

1.4.2.43.1 DMA subsystem

1.4.2.43.1.1 1. DMA controller (rt_dma_controller / rt_dma_controller_ops)

Same style as **ata/ata.md**, for **rt_dma_controller_register** / **rt_dma_controller_unregister** (components/drivers/dma/dma.c).

struct rt_dma_controller (before register)

Member	Rule	Meaning
dev	**Non-NULL**	Bound **rt_device**; checked at register; may **rt_dm_dev_bind_fwdata**.
ops	**Non-NULL**	**rt_dma_controller_ops**.
dir_cap	**At least one bit**	**rt_dma_controller_add_direction**; register fails if no direction is set.
addr_mask	**Caller**	**RT_DMA_ADDR_MASK(n)**, DMA-visible physical range.
channels_nodes / mutex	**Core**	Initialized in **rt_dma_controller_register**.

struct rt_dma_controller_ops

Callback	Role
request_chan	Return **rt_dma_chan** for **slave** + **fw_data** (usually parsed **dmas**/**dma-names**).
release_chan	Pair of **request_chan**.
config / **prep_*** / **start** / **pause** / **stop**	Channel programming and run control.

Slaves use **rt_dma_chan_request(dev, name)**, then **rt_dma_chan_config**, **rt_dma_prep_***, **rt_dma_chan_start**, etc. (see **drivers/dma.h**).

Buffers and dma_map_ops

rt_dma_alloc / **rt_dma_free** / **rt_dma_alloc_coherent**, **rt_dma_sync_***, **rt_dma_device_set_ops** — see **drivers/dma.h**. Coherent/CMA **pools** are documented in **documentation/6.components/device-driver/dma/pool.md**.

When DMA framework vs memcpy

Prefer rt_dma_chan_* when...	memcpy / PIO may be OK when...
SDIO / **SPI** / **UART** / **MAC** has **dmas** in DTS and high throughput.	**Bootloader-stage** tiny transfers or debug-only paths.

Prefer `rt_dma_chan_*` when...	memcpy / PIO may be OK when...
You need **scatter-gather** or **cyclic** audio/I2S.	Transfer is **< cache line** and latency matters more than CPU load.

Slave driver checklist

1. **`rt_dma_chan_request`** in `probe` with **`dma-names`** matching DTS.
2. **`rt_dma_chan_config`** with **`src_addr_width`**, **`direction`**, **`dst_maxburst`** aligned to FIFO depth—wrong burst causes underrun/overrun.
3. **`rt_dma_prep_*`** + **`rt_dma_chan_start`**: pass **physical or bus addresses** per `map_ops`—never feed unrelated virtual pointers unless your `dma_map_ops` handles them.
4. **`rt_dma_chan_release`** in `remove` even if `request` partially failed after cleanup.
5. **Completion**: ISR or **`rt_dma_chan_done`**—do not sleep in ISR; wake thread for heavy post-processing.

Controller author pitfalls

- **`dir_cap` empty** → register fails—set at least one **`RT_DMA_DIR_*`** before register.
- **`addr_mask` too narrow** → peripherals above 4G cannot be targeted—set from SoC spec.

1.4.2.43.1.2 2. API index (symbols)

Quick index into **`drivers/dma.h`** (full definitions in the header).
Enums: `enum rt_dma_slave_buswidth`, `enum rt_dma_transfer_direction`.
Macros: `RT_DMA_ADDR_MASK(n)`, `RT_DMA_F_*`, `RT_DMA_PAGE_SIZE`.
Structs: `struct rt_dma_chan`, `struct rt_dma_controller`, `struct rt_dma_controller_ops`, `struct rt_dma_map_ops`, `struct rt_dma_pool`, `struct rt_dma_slave_config`, `struct rt_dma_slave_transfer`.
Inlines: `rt_dma_alloc_coherent`, `rt_dma_free_coherent`, `rt_dma_controller_add_direction`, `rt_dma_controller_set_addr_mask`, `rt_dma_device_is_coherent`, `rt_dma_device_set_ops`.
Functions: `rt_dma_alloc` / `rt_dma_free`, `rt_dma_chan_config` / `done` / `pause` / `start` / `stop`, `rt_dma_chan_request` / `release`, `rt_dma_controller_register` / `unregister`, `rt_dma_pool_extract` / `install`, `rt_dma_prep_cyclic` / `memcpy` / `single`, `rt_dma_sync_in_data` / `out_data`.

1.4.2.43.1.3 See also

- DMA memory pools: `documentation/6.components/device-driver/dma/pool.md`
- `components/drivers/include/drivers/dma.h`
- `components/drivers/dma/dma.c`

1.4.2.44 DMA memory pools

1.4.2.44.1 DMA coherent / CMA pools

Header: `struct rt_dma_pool` and **`rt_dma_pool_*`** in `components/drivers/include/drivers/dma.h`.
Implementation: `components/drivers/dma/dma_pool.c`.
`rt_dma_alloc` / **`rt_dma_free`** (and default **`rt_dma_map_ops`**) allocate from installed pools instead of the generic heap when **`RT_USING_DMA`** and memblock reservations are configured. Controllers and slaves still use **`documentation/6.components/device-driver/dma/dma.md`** for channel programming; pools only back **buffer memory**.

1.4.2.44.1.1 Driver / BSP quick notes

Topic	Guidance
Boot reservation	Call **rt_dma_pool_extract** early (memblock **dma-pool** region) before drivers **rt_dma_alloc**.
Coherent vs CMA	**coherent_pool** is carved from the **low** end of the reserved region; **CMA** uses the remainder—**cma_size must be coherent_pool_size**.
32-bit DMA	Pools below **4 GiB** set **RT_DMA_F_32BITS** on the pool—devices with limited addressing need buffers from these pools.
dma-coherent DT	**rt_dma_device_is_coherent(dev)** reads the property; map ops may skip flush on coherent devices.

1.4.2.44.1.2 struct rt_dma_pool

Field	Meaning
region	Physical **start** / **end**, optional **name**.
map / bits	Page bitmap (**ARCH_PAGE_SIZE** granule).
flags	e.g. **RT_DMA_F_32BITS**, **RT_DMA_F_NOCACHE**.
list	Global pool list (internal).

1.4.2.44.1.3 Public API

API	Role
rt_dma_pool_install(region)	Register a physical memory region as a pool; returns **struct rt_dma_pool** * or **NULL** on error.
rt_dma_pool_extract(cma_size, coherent_pool_size)	Find memblock region named **dma-pool**, split into **coherent-pool** + **cma** sub-regions, install both pools, mark memblock as consumed. Returns **-RT_EEMPTY** if no suitable region, **-RT_EINVAL** if sizes are inconsistent.

rt_dma_pool_install is also used directly when the BSP hands a fixed **rt_region_t** without going through memblock extract.

1.4.2.44.1.4 How allocation uses pools

1. **rt_dma_alloc(dev, size, &handle, flags)** walks installed pools (under **dma_pools_lock**), picks a pool matching flags (coherent, 32-bit, device, etc.).
2. Default **rt_dma_map_ops**:
   - **Coherent**: flush on **sync_out_data**, invalidate on **sync_in_data**.
   - **Non-coherent**: physical address via **rt_kmem_v2p** without cache maintenance on sync-in.
3. With **RT_USING_OFW**, **rt_dma_device_set_ops** may install **ofw_dma_map_*** that translates CPU DMA bus addresses via **rt_ofw_translate_cpu2dma**.

Slaves should use **rt_dma_alloc_coherent** for ring buffers the CPU and DMA both touch.

1.4.2.44.1.5 Boot flow (typical)

1. Bootloader or memblock code reserves **dma-pool** in device tree / memblock.

2. Early init calls **rt_dma_pool_extract(SIZE_MB(8), SIZE_MB(2))** (example sizes).
3. Drivers **probe** and **rt_dma_alloc** for descriptor rings and payload buffers.

1.4.2.44.1.6 Debugging

With **RT_USING_CONSOLE** and **RT_USING_MSH**, **list_dma_pool** prints installed regions (name, size, base).

1.4.2.44.1.7 Pitfalls

- **Pool exhausted**: **rt_dma_alloc** returns **NULL******—size requests must fit contiguous free pages in a matching pool.**
- ****CMA smaller than coherent**: **rt_dma_pool_extract** rejects **cma_size** < **coherent_pool_size**.
- **High memory only**: if no sub-4G **dma-pool** chunk is large enough, extract may fall back with a warning—DMA to peripherals that cannot reach high memory will fail.
- **Double extract**: second call may find memblock already consumed—guard in BSP init.

1.4.2.44.1.8 See also

- DMA engine and channels: `documentation/6.components/device-driver/dma/dma.md`
- OFW DMA range: `documentation/6.components/device-driver/ofw/ofw.md`
- NUMA / affinity: `documentation/6.components/device-driver/numa/numa.md`
- Source: `components/drivers/dma/dma_pool.c`
- Header: `components/drivers/include/drivers/dma.h`

1.4.2.45 Graphics (DM)

1.4.2.45.1 Graphics subsystem (device model)

RT-Thread **graphic DM** (`components/drivers/include/drivers/graphic.h`, `components/drivers/graphic/graphic.c`) wraps display controllers as **struct rt_graphic_device**: multi-**plane** framebuffers, **EDID**, optional **backlight**, Linux-style **fb** ioctls, and legacy **rt_device_graphic_ops** via **rt_graphic_device_switch_primary**.
App-facing pixel formats and **RTGRAPHIC_CTRL_*** live in **drivers/classes/graphic.h**. DM types, planes, and registration APIs are in **graphic.h**.
Implementation split:

File	Role
graphic.c	Device register, `control`, planes, EDID parse, auto-update timer, OFW backlight
graphic_simple.c	Synthetic EDID + **rt_graphic_device_simple_register**
graphic_primary.c	**rt_graphic_device_switch_primary** → `set_pixel` / `draw_hline` on primary FB
framebuffer/fb-simple.c	DT **simple-framebuffer** — Simple framebuffer (`simple-framebuffer`)

1.4.2.45.1.1 Kconfig

Option	Role
RT_USING_DM	Required parent
RT_USING_GRAPHIC	Core graphic stack (graphic.c, graphic_primary.c, graphic_simple.c)
RT_GRAPHIC_FB	Framebuffer drivers (e.g. **simple-framebuffer**)
RT_GRAPHIC_BACKLIGHT	Backlight class + PWM/GPIO — Backlight devices
RT_GRAPHIC_LOGO	Boot logo render on register — Boot Logo

1.4.2.45.1.2 Architecture

```
Controller driver (DSI, LCD, simple-fb, ...)
      |
      |  alloc_plane + plane_ops (update / fb_remap / fb_cleanup)
      v
struct rt_graphic_device
  primary_plane  (required, one)
  cursor_plane   (optional, one)
  overlay_nodes  (0..N)
  edid, ops, backlight, update_timer
      |
      |  rt_graphic_device_register  ->  rt_device "fbN"
      v
Apps: rt_device_find + RTGRAPHIC_CTRL_* / read-write FB
      |
      |  rt_graphic_device_switch_primary(gdev)
      v
dev->user_data = rt_device_graphic_ops (graphic_primary.c)
```

Layer	Responsibility
rt_graphic_plane_ops	Hardware: allocate FB, push pixels to display (**update**), teardown (**fb_cleanup**)
rt_graphic_device_ops	Device-wide: **DPMS**, **vsync**, brightness, vendor **control**
graphic.c	Standard **rt_device** API, rect update, mode set, hotplug, periodic flush timer

1.4.2.45.1.3 Core structures

struct rt_graphic_plane

Field	Meaning
type	**PRIMARY** (one), **CURSOR** (one), **OVERLAY** (list)
mode / **modes[]**	**RTGRAPHIC_PIXEL_FORMAT_*** from classes/graphic.h
framebuffer, **line_length**, **screen_len**, **framebuffer_len**	CPU-visible buffer; may be multi-buffer if **framebuffer_len** > **screen_len**
ops	Driver callbacks (see below)

struct rt_graphic_plane_ops

Callback	When called	Contract
fb_remap	Mode or resolution change	Allocate/map FB for **mode** + **rect**; set **plane->framebuffer**
fb_cleanup	Before remap / del_plane	Release HW resources (unmap, unref GPU resource)
update	**RTGRAPHIC_CTRL_RECT_UPDATE** or auto timer	Push **rect** region to display; **width/height == 0** may mean "position only" (cursor)
fb_pan_display	**RTGRAPHIC_CTRL_PAN_DISPLAY** / open	Point scanout at offset inside **framebuffer**
prop_set	Optional	Z-order, rotate, alpha (**enum rt_graphic_plane_prop**)

struct rt_graphic_device_ops

Callback	Role
dpms_switch	**RT_GRAPHIC_DPMS_*** — often ties to **rt_graphic_device_update_auto** period
wait_vsync	**RTGRAPHIC_CTRL_WAIT_VSYNC**
set_brightness / **get_brightness**	Panel brightness; may defer to **gdev->backlight**
get_status	Optional connector status
current_plane	Override plane used for **GET_INFO** / updates (default: primary)
control	Vendor-specific commands (optional 3D / acceleration path)

EDID

struct edid in **graphic.h** mirrors VESA EDID layout. **RTGRAPHIC_CTRL_GET_EXT** copies **gdev->edid**. On register, if primary plane has no size, **graphic_edid_res()** picks mode from detailed timings.
rt_graphic_device_simple_edid(gdev, w, h, refresh_hz) (graphic_simple.c) builds a minimal EDID when hardware has no DDC (fixed panels, simple bring-up).

1.4.2.45.1.4 Registration API

API	Role
rt_graphic_device_alloc_plane(gdev, priv_size, ops, modes, modes_nr, type)	Allocate plane; **priv[0]** tail for driver data
rt_graphic_device_add_plane(gdev, plane)	Attach primary/cursor/overlay; assigns **plane_ida** id
rt_graphic_device_register(gdev)	Requires **primary_plane**; names device **fbu** via **RT_DM_IDA**; maps FB from EDID if size zero
rt_graphic_device_unregister(gdev)	DPMS off, stop timer, del all planes, **rt_device_unregister**
rt_graphic_device_simple_register(...)	Alloc primary plane + synthetic EDID + register (no full EDID path)

API	Role
rt_graphic_device_simple_↩ unregister(gdev)	Alias for **unregister**

Naming: **RT_DM_IDA_INIT(GRAPHIC_FRAMEBUFFER)** → devices **fb0**, **fb1**, ...
OFW backlight: during **register**, **graphic_ofw_init** resolves **backlight** phandle on **dev->ofw_↩ node** and opens the backlight device.

1.4.2.45.1.5 Application / framework controls

From **classes/graphic.h** (handled in **graphic.c _graphic_control**):

Command	Behavior
RTGRAPHIC_CTRL_RECT_UPDATE	**plane->ops->update(plane, rect)** on current plane
RTGRAPHIC_CTRL_SET_MODE	**fb_remap** if mode is in **plane->modes[]**
RTGRAPHIC_CTRL_GET_INFO	Fills **rt_device_graphic_info** from current plane
RTGRAPHIC_CTRL_POWERON / POWEROFF	DPMS + optional backlight power
RTGRAPHIC_CTRL_PAN_DISPLAY	Offset within framebuffer (double-buffer)
RTGRAPHIC_CTRL_WAIT_VSYNC	Driver **wait_vsync**
RT_DEVICE_CTRL_CURSOR_*	Cursor plane position / bitmap
FBIOGET_VSCREENINFO / FBIOPUT_↩ VSCREENINFO	Linux fb var screeninfo (when enabled)

Auto flush: **rt_graphic_device_update_auto(gdev, ms)** starts a periodic timer (**default idea: RT↩ _GRAPHIC_UPDATE_MS = 16**) that calls **update** on primary, overlays, and cursor. **rt_graphic_device↩ _enter** / **leave** stop/restart the timer around modeset (hotplug).
Hotplug: **rt_graphic_device_hotplug_event(gdev)** re-parses EDID size, **fb_remap** primary, then fires **event_notify**.

1.4.2.45.1.6 Primary display helpers (graphic_primary.c)

```
struct rt_device_graphic_ops *rt_graphic_device_switch_primary(struct rt_graphic_device *gdev);
```

- Sets global **primary_gdev** and returns **rt_device_graphic_ops** matching primary plane **bpp** (8/↩ 16/24/32/64).
- Legacy macro in **graphic.h**: **rt_graphix_ops(dev)** calls **switch_primary** then reads **dev->user_data**.
- Used by **finsh** / **LVGL** / **logo** code that draws via **set_pixel** / **draw_hline** without **RTGRAPHIC_↩ CTRL_RECT_UPDATE**.

After **register**, call **switch_primary** on the head you want for **rt_graphix_ops**.

1.4.2.45.1.7 Typical controller probe (SoC)

```
static const rt_uint32_t my_modes[] = {
    RTGRAPHIC_PIXEL_FORMAT_RGB565,
    RTGRAPHIC_PIXEL_FORMAT_ARGB888,
};

static const struct rt_graphic_plane_ops my_plane_ops = {
    .fb_remap   = my_fb_remap,
    .fb_cleanup = my_fb_cleanup,
    .update     = my_fb_update,
};

static rt_err_t my_probe(struct rt_platform_device *pdev)
```

```
{
    struct my_drv *priv;
    struct rt_graphic_plane *plane;
    rt_err_t err;

    priv = rt_dm_dev_iomap(pdev->parent.parent, 0);  /* or embed in pdev->priv */
    /* clocks, resets, regulator -- @ref page_device_clk, @ref page_device_reset */

    priv->gdev.ops = &my_gdev_ops;

    plane = rt_graphic_device_alloc_plane(&priv->gdev, sizeof(*priv),
            &my_plane_ops, my_modes, RT_ARRAY_SIZE(my_modes),
            RT_GRAPHIC_PLANE_TYPE_PRIMARY);
    if (!plane)
        return -RT_ENOMEM;

    err = rt_graphic_device_add_plane(&priv->gdev, plane);
    if (err)
        goto err;

    /* Fill EDID from DDC or rt_graphic_device_simple_edid() */
    err = rt_graphic_device_register(&priv->gdev);
    if (err)
        goto err;

    rt_graphic_device_switch_primary(&priv->gdev);
    rt_graphic_device_update_auto(&priv->gdev, RT_GRAPHIC_UPDATE_MS);
    return RT_EOK;
}
```

`simple-framebuffer`: maps fixed **`reg`** RAM, implements plane ops as CPU memcpy — see Simple framebuffer (`simple-framebuffer`).

1.4.2.45.1.8 PHY, clocks, backlight

Resource	Document
Pixel / link clock	Clock framework (CLK)
DSI/DP PHY	Generic PHY (Phye)
Panel backlight	Backlight devices (`backlight` DT property)
Coherent FB / cache	Hardware cache (hwcache), DMA engine

Bring-up order: regulator → clock → panel init → **`register`** → backlight **`POWERON`** → **`RTGRAPHIC_CTRL_POWERON`**.

1.4.2.45.1.9 Engineer checklist

1. Implement **`rt_graphic_plane_ops`** (at minimum **`fb_remap`**, **`update`**, **`fb_cleanup`**).
2. **`add_plane`** **PRIMARY** before **`register`**; cursor/overlay after if needed.
3. Populate **`edid`** or use **`simple_register`** / **`simple_edid`**.
4. **`register`** → **`switch_primary`** for legacy drawing APIs.
5. **`remove`**: **`unregister`** (frees planes); release clocks/reset/regulator in reverse order.
6. Hotplug: call **`hotplug_event`** after EDID/mode change under **`enter`**/**`leave`**.

1.4.2.45.1.10 Pitfalls

- **Missing `update` with auto timer off**: apps must **`RTGRAPHIC_CTRL_RECT_UPDATE`** after CPU writes or nothing reaches the panel.
- **Primary plane size 0 at register**: core tries EDID **`fb_remap`** — bogus EDID causes probe failure.
- **Two PRIMARY planes**: **`add_plane`** returns **`-RT_EINVAL`**.
- **DMA / cache**: CPU-drawn FB must use correct MMU attributes; sync cache before **`update`** if scanout is non-coherent — Hardware cache (hwcache).

- **switch_primary global**: only one **primary_gdev** — multi-head apps should use **rt_device_find("fb1")** + graphic ctrl, not **rt_graphix_ops** alone.
- **DPMS off without stopping updates**: use **rt_graphic_device_update_auto(gdev, 0)** or **enter** during suspend.

1.4.2.45.1.11 Detailed documents

- Simple framebuffer (`simple-framebuffer`) — **simple-framebuffer**
- Backlight devices — PWM/GPIO backlight
- Boot Logo — boot logo (PPM)

1.4.2.45.1.12 See also

- components/drivers/include/drivers/graphic.h
- components/drivers/include/drivers/classes/graphic.h
- components/drivers/graphic/graphic.c, graphic_simple.c, graphic_primary.c
- Device model helpers (dm.h) — rt_dm_dev_iomap, naming
- Platform bus — RT_PLATFORM_DRIVER_EXPORT

1.4.2.46 Backlight devices

1.4.2.46.1 Backlight subsystem

Header: components/drivers/include/drivers/backlight.h. Core: **components/drivers/graphic/backlight/backlight.c**. PWM and GPIO backends: **backlight-pwm.c**, **backlight-gpio.c**.
Graphic stack overview: Graphics (DM).

1.4.2.46.1.1 struct rt_backlight_device

Extends **rt_device**. Properties include **max_brightness**, **brightness**, power state. Drivers implement brightness through ops registered with the core.

Core device behavior (backlight.c)

Op	Behavior
read	ASCII decimal of current brightness
write	Parse integer string, clamp to **max_brightness**, call **rt_backlight_set_brightness**

RT_DM_IDA_INIT(GRAPHIC_BACKLIGHT) assigns stable DM names on register.

1.4.2.46.1.2 PWM backlight (pwm-backlight)

Compatible: typically **pwm-backlight** (see driver OFW table in source).

DT / property	Role
pwms	PWM channel for dimming
brightness-levels	Optional non-linear LUT
default-brightness-level	Boot brightness index

DT / property	Role
`enable-gpios`	Panel enable pin
`power-supply`	Optional **`rt_regulator`** before PWM

Probe enables regulator (if any), maps brightness levels, registers **`rt_backlight_device`**.

1.4.2.46.1.3 GPIO backlight (`gpio-backlight`)

Simple on/off or few-level backlight using a GPIO line (see `backlight-gpio.c` OFW ids).

1.4.2.46.1.4 Driver / BSP quick notes

Topic	Guidance
Enable order	Regulator → enable GPIO → PWM duty
Thermal	High brightness may interact with Thermal management policies
Userspace	Writing `"128\n"` to device node sets brightness (see core `write`)

1.4.2.46.1.5 See also

- Graphics (DM)
- Regulator
- `components/drivers/graphic/backlight/backlight.c`
- `components/drivers/graphic/backlight/backlight-pwm.c`

1.4.2.47 Simple framebuffer (`simple-framebuffer`)

1.4.2.47.1 `simple-framebuffer` platform driver

Implementation: **`components/drivers/graphic/framebuffer/fb-simple.c`**. Registers a **`struct rt_graphic_device`** from a fixed memory window described in device tree (Linux **`simple-framebuffer`** binding style).

1.4.2.47.1.1 Device tree (typical)

```
framebuffer@0 {
    compatible = "simple-framebuffer";
    reg = <0x0 0x00800000>;
    width = <1920>;
    height = <1080>;
    stride = <7680>;
    format = "a8r8g8b8";   /* see driver format table */
    clocks = <&disp_clk>;  /* optional, RT_USING_CLK */
    power-supplies = <&vdd>; /* optional, RT_USING_REGULATOR */
};
```

1.4.2.47.1.2 Probe flow

1. Parse **`width`**, **`height`**, **`stride`**, **`format`** → internal **`simplefb_format`** (bits per pixel, RT graphic mode).
2. **`ioremap`** (or use physical) **`reg`** → **`screen_base`**, compute **`screen_size`**.
3. Optional **`rt_clk_array`** prepare/enable; optional regulator **`enable`** array.
4. Fill **`rt_graphic_device`** ops (`set_pixel`, `get_pixel`, `draw_hline`, etc.) and **`rt_device_register`**.

1.4.2.47.1.3 Supported formats

Driver table maps string names (e.g. **r5g6b5**, **a8r8g8b8**, **x8r8g8b8**) to **rt_graphic_info** mode and bpp. Unknown **format** fails probe.

1.4.2.47.1.4 Remove

Disable clocks/regulators, unmap framebuffer memory, unregister graphic device.

1.4.2.47.1.5 Driver / BSP quick notes

Topic	Guidance
Cache	CPU-accessible framebuffer RAM must use correct MMU cache attributes; if cacheable, maintain with **rt_hw_cpu_dcache_ops** — see Hardware cache (hwcache)
Stride	Must match hardware line pitch (bytes), not always `width * bpp/8`
Primary display	Pair with Graphics (DM) / `graphic_primary.c` if multiple heads
No GPU	This is **linear FB** only—no command queue; use a SoC GPU driver if you need acceleration

1.4.2.47.1.6 See also

- Graphics (DM)
- `components/drivers/graphic/framebuffer/fb-simple.c`
- `components/drivers/graphic/graphic.c`

1.4.2.48 Boot Logo

1.4.2.48.1 Boot Logo (PPM)

Implementation: **components/drivers/graphic/logo/logo.c**. Build converts a **PPM** asset into **logo.inc** (RGB triplets) at compile time. At runtime **rt_graphic_logo_render()** draws the logo centered on the framebuffer when a **rt_graphic_device** registers.
Graphic stack overview: Graphics (DM).

1.4.2.48.1.1 Kconfig

Option	Role
RT_GRAPHIC_LOGO	Enable logo subsystem; selects **RT_GRAPHIC_FB**
RT_GRAPHIC_LOGO_NONE	No built-in image — use **rt_graphic_logo_change()** at runtime
RT_GRAPHIC_LOGO_RT_THREAD_CLUT224	Embed **logo-rt-thread-clut224.ppm** (default)
RT_GRAPHIC_LOGO_RT_THREAD_WHITE_CLUT224	White variant PPM
SOC_DM_GRAPHIC_LOGO_DIR	BSP adds custom logo choices via **Kconfig** / **Kconfig.path**

Custom BSP logos: define **RT_GRAPHIC_LOGO_**<**NAME**> in Kconfig and **RT_GRAPHIC_LOGO_**<**NAME**>**_PATH** (string, path to `.ppm`). **logo/SConscript** parses the file and generates **logo.inc**.

1.4.2.48.1.2 When the logo is drawn

```
rt_graphic_device_register(gdev)
        |
        v
rt_graphic_logo_render(gdev)   /* graphic.c; errors logged at DBG only */
        |
        +-- open fb device, GET_INFO, POWERON
        +-- blit PPM RGB -> framebuffer (centered)
        +-- RTGRAPHIC_CTRL_RECT_UPDATE + WAIT_VSYNC
        +-- startup_logo = NULL  (one-shot; frees built-in use)
```

- Runs when a graphic device successfully **registers** (see **`graphic.c`**).
- Requires **`RT_GRAPHIC_LOGO`** and non-empty **`startup_logo`** (built-in or **`rt_graphic_logo_change`**).
- If logo is larger than **`var.xres`** / **`var.yres`**, render fails with **`-RT_EINVAL`**.

1.4.2.48.1.3 Runtime API (`graphic.h`)

`rt_graphic_logo_change`

```
rt_err_t rt_graphic_logo_change(void *data, int width, int height, int color_max);
```

Arguments	Meaning
`data`	Pointer to **packed RGB** bytes (`width * height * 3`), same order as PPM pixmap data
`width` / **`height`**	Logo size in pixels
`color_max`	Max channel value in **`data`** (PPM **max pixel value**, e.g. **255** or **15**)

Call pattern	Effect
`data == NULL && all sizes 0`	Disable logo (**`startup_logo = NULL`**)
Valid **`data`** + **dimensions**	Replace logo used on next **`rt_graphic_logo_render`**
Otherwise	**`-RT_EINVAL`**

Use with **`RT_GRAPHIC_LOGO_NONE`** or to override the built-in clut224 asset before the first **`register`**.

`rt_graphic_logo_render`

```
rt_err_t rt_graphic_logo_render(struct rt_graphic_device *gdev);
```

Normally called from **`rt_graphic_device_register()`** — drivers rarely invoke it directly. Steps:

1. **`rt_device_open`** on the graphic **`rt_device`**
2. **`FBIOGET_VSCREENINFO`** — grayscale vs RGB bitfield layout
3. **`RTGRAPHIC_CTRL_GET_INFO`** — framebuffer base + pitch
4. **`RTGRAPHIC_CTRL_POWERON`**
5. Center logo: **`rect.x/y = (screen - logo) / 2`**
6. Per-pixel color remap into **`bits_per_pixel`**
7. **`RTGRAPHIC_CTRL_RECT_UPDATE`** + **`RTGRAPHIC_CTRL_WAIT_VSYNC`**
8. Clears **`startup_logo`** (one-shot)

Without **`RT_GRAPHIC_LOGO`**, both APIs compile to inline no-ops returning **`RT_EOK`**.

1.4.2.48.1.4 Build-time PPM → `logo.inc`

`logo/SConscript` (when a logo path is selected):

1. Read PPM header (**P1–P3** ASCII path in script).
2. Parse **`width height`**, **`max_pixel_value`**, pixel stream.
3. Emit **`logo.inc`**: rows of **`R,G,B`** byte triplets.
4. Define **`__STARTUP_LOGO_WIDTH__`**, **`__STARTUP_LOGO_HEIGHT__`**, **`__STARTUP_LOGO_COLOR_MAX__`**.

`logo.c` includes **`logo.inc`** into **`builtin_logo[]`** when all three macros are non-zero.

1.4.2.48.1.5 Supported PPM variants

Magic	Kind	Encoding	Notes
P1	Bitmap	ASCII	Build script ASCII path
P2	Graymap	ASCII	Build script ASCII path
P3	Pixmap	ASCII	Typical for clut224 logos
P4	Bitmap	Binary	Convert to P3 for embed
P5	Graymap	Binary	Convert to P3 for embed
P6	Pixmap	Binary	Use **`pnmnoraw`** → P3 for SConscript

Runtime rendering always uses **RGB triplets** in memory.

1.4.2.48.1.6 ASCII PPM layout

```
# <magic>: P1 / P2 / P3 ...
# <comment>
<width> <height>
<max_pixel_value>
<data...>
```

Example (P3, 4×4)

```
P3
# Standard 15-color test logo
4 4
15
0  0  0    0  0  0    0  0  0    15 0  15
0  0  0    0  15 7    0  0  0    0  0  0
0  0  0    0  0  0    0  15 7    0  0  0
15 0  15   0  0  0    0  0  0    0  0  0
```

	Col 0	Col 1	Col 2	Col 3
Row 0	0,0,0	0,0,0	0,0,0	15,0,15
Row 1	0,0,0	0,15,7	0,0,0	0,0,0
Row 2	0,0,0	0,0,0	0,15,7	0,0,0
Row 3	15,0,15	0,0,0	0,0,0	0,0,0

1.4.2.48.1.7 Generating a logo file

1. Convert source image to PPM:

```
magick <input>.png <output>.ppm
```

1. Reduce to **224** colors (clut224 path):

```
ppmquant 224 <input>.ppm > <tmp>_224.ppm
```

1. ASCII PPM for SConscript:

```
pnmnoraw <tmp>_224.ppm > logo-<name>-clut224.ppm
```

1. Point BSP **RT_GRAPHIC_LOGO_<NAME>_PATH** at the file, or replace in-tree **logo-rt-thread-clut224.ppm**.

1.4.2.48.1.8 Offline preview

Open **components/drivers/graphic/logo/logo.html** in a browser to preview a PPM before building.

1.4.2.48.1.9 BSP / integration quick notes

Topic	Guidance
Asset size	Large PPM bloats **logo.inc** and slows boot — keep resolution modest.
Display readiness	Logo runs at **register** — enable backlight after first frame if needed (Backlight devices).
Format	Invalid PPM fails SConscript at build time; validate in CI.
One-shot	After first render, **startup_logo** is cleared — call **rt_graphic_logo_change** again for another splash.

1.4.2.48.1.10 Pitfalls

- **Logo bigger than mode**: **-RT_EINVAL** — match PPM to panel resolution.
- **RT_GRAPHIC_LOGO_NONE without logo_change**: no logo (by design).
- **Binary P6 without conversion**: SConscript may not parse — use **pnmnoraw**.

1.4.2.48.1.11 See also

- Graphics (DM)
- Simple framebuffer (`simple-framebuffer`)
- components/drivers/graphic/logo/logo.c, logo/SConscript
- components/drivers/include/drivers/graphic.h

1.4.2.49 Hardware spinlock

1.4.2.49.1 Hardware spinlock (hwspinlock)

RT-Thread **hwspinlock** (components/drivers/hwspinlock/) models **SoC hardware mutex registers** shared across **heterogeneous CPUs** (AMP), **remote processors**, or firmware—not the same as a single-OS **rt_spinlock**. A **bank** (one controller IP) exports many **rt_hwspinlock** instances; consumers acquire through **rt_hwspin_*** or device-tree **hwlocks**.
Public API: **components/drivers/include/drivers/hwspinlock.h**. DM types: **components/drivers/hwspinlock/hwspinlock_dm.h**. Core: **components/drivers/hwspinlock/hwspinlock.c**.
SoC-specific controllers are added under **SOC_DM_HWSPINLOCK_DIR** in the BSP Kconfig.

1.4.2.49.1.1 Kconfig

Option	Role
RT_USING_DM	Required parent
RT_USING_OFW	Required — phandle parsing for **hwlocks**
RT_USING_HWSPINLOCK	Build **hwspinlock.c**; selects **RT_USING_ADT** / **RT_USING_ADT_REF**
SOC_DM_HWSPINLOCK_DIR	BSP adds platform drivers (e.g. mailbox companion lock IP)

1.4.2.49.1.2 Architecture

```
SoC hwspinlock controller (platform driver)
      |
      |  hwspinlock_bank_alloc + fill ops
      |  rt_hwspinlock_bank_register(bank)
      v
Global bank list + rt_ofw_data(provider_np) = bank
      |
      |  Consumer: hwlocks = <&hwlock0 3>; hwlock-names = "mbox";
      v
rt_ofw_get_hwspinlock_by_name(np, "mbox")
      |
      v
rt_hwspin_lock_timeout_irqsave(hwlock, ms, &flags)
      ... shared MMIO / mailbox ...
rt_hwspin_unlock_irqrestore(hwlock, &flags)
      |
      v
rt_hwspinlock_put(hwlock)   /* release allocation, not unlock */
```

Object	Role
struct rt_hwspinlock_bank	One controller: **dev**, **ops**, **base_id**, **locks[]**
struct rt_hwspinlock	One hardware lock index; per-lock software **rt_spinlock** guards metadata
struct rt_hwspinlock_ops	**trylock**, **unlock**, optional **relax** (WFE / yield in busy-wait)

Two layers of locking:

1. **Software hwlock->lock** — protects the bank's bookkeeping while calling ops (optional **irqsave** via **out_irq_level**).
2. **Hardware ops->trylock/unlock** — the actual cross-core exclusion in silicon.

rt_hw_dmb() is issued after acquire and before release for visibility ordering.

1.4.2.49.1.3 Provider: register a bank

```
struct my_hwlock_drv {
    struct rt_hwspinlock_bank *bank;
};

static rt_err_t my_hwlock_trylock(struct rt_hwspinlock *hwlock)
{
    int id = hwspinlock_find_id(hwlock);
    /* read lock register: RT_EOK or -RT_EBUSY */
}

static void my_hwlock_unlock(struct rt_hwspinlock *hwlock)
{
    int id = hwspinlock_find_id(hwlock);
    /* write release */
}

static void my_hwlock_relax(struct rt_hwspinlock *hwlock)
{
    rt_hw_cpu_relax();  /* or WFE per TRM */
```

```
}

static const struct rt_hwspinlock_ops my_hwlock_ops = {
    .trylock = my_hwlock_trylock,
    .unlock  = my_hwlock_unlock,
    .relax   = my_hwlock_relax,
};

static rt_err_t my_probe(struct rt_platform_device *pdev)
{
    struct my_hwlock_drv *priv;
    int num_locks = 32;

    priv->bank = hwspinlock_bank_alloc(priv, num_locks);
    priv->bank->dev = &pdev->parent;
    priv->bank->ops = &my_hwlock_ops;
    priv->bank->base_id = 0;   /* global ID offset for DT indices */
    priv->bank->locks_nr = num_locks;

    return rt_hwspinlock_bank_register(priv->bank);
}
```

rt_hwspinlock_bank_register requirements:

- **bank**, **bank->dev**, **bank->ops**, **locks_nr** > **0**
- Initializes each **locks[i].bank**, **used = false**, software spinlock
- Links bank on global list; **rt_dm_dev_bind_fwdata(dev, NULL, bank)** for OFW lookup

rt_hwspinlock_bank_unregister: succeeds only when **rt_ref_read(&bank->ref) == 1** (no outstanding **get** references)—else **-RT_EBUSY**.

1.4.2.49.1.4 Consumer API

Lock / unlock

API	IRQ save	Behavior
rt_hwspin_trylock_raw	optional **out_irq_level**	One try; **-RT_EBUSY** if hardware busy
rt_hwspin_lock_timeout_raw	optional	Retries until success or **timeout_ms** → **-RT_ETIMEOUT**; calls **ops->relax** while waiting
rt_hwspin_unlock_raw	restore if provided	**ops->unlock** then release software lock

Convenience inlines: **rt_hwspin_trylock**, **rt_hwspin_trylock_irqsave**, **rt_hwspin_lock_timeout**, **rt_hwspin_lock_timeout_irqsave**, matching **unlock** variants.
Rules:

- **Do not sleep** while holding the hardware lock (same as spinlocks).
- **Pair lock/unlock** on the same core that acquired the hardware lock (AMP contract).
- **unlock does not clear used** — allocation is separate from locking.

Allocate a lock handle

API	Role
rt_hwspinlock_get()	First free lock in any registered bank
rt_hwspinlock_get_by_index(dev, index)	Via consumer's OFW node

API	Role
`rt_hwspinlock_get_by_name(dev, name)`	Match `hwlock-names`
`rt_ofw_get_hwspinlock_by_index(np, index)`	Parse `hwlocks` + `#hwlock-cells`
`rt_ofw_get_hwspinlock_by_name(np, name)`	Index from `hwlock-names`
`rt_hwspinlock_put(hwlock)`	Clear `used`, `rt_ref_put` on bank — **does not unlock**

OFW parse (`hwspinlock.c`):

```
hwspinlock: hwspinlock@... {
    compatible = "vendor,hwspinlock";
    #hwlock-cells = <1>;
};

remote_ip: device@... {
    hwlocks = <&hwspinlock 5>;
    hwlock-names = "ipc";
};
```

1. `rt_ofw_parse_phandle_cells(np, "hwlocks", "#hwlock-cells", index, &args)`
2. `rt_platform_ofw_request(bank_np)` if provider not probed
3. `bank = rt_ofw_data(bank_np)`; lock id = `bank->base_id + args.args[0]` (requires `args_count == 1`)
4. Marks `hwlock->used` and `rt_ref_get(&bank->ref)`

1.4.2.49.1.5 When to use hwspinlock vs `rt_spinlock`

Use hwspinlock when...	Use `rt_spinlock` when...
Another CPU / **DSP** / **M-core** can touch the same resource without sharing RT-Thread.	Only RT-Thread SMP CPUs coordinate, all under one OS.
TRM or firmware defines **hardware lock index** before mailbox/shared SRAM access.	Exclusion inside one kernel with IRQ disable or SMP spinlock is enough.
Mailbox / **rpmsg** handshake doc lists lock IDs.	Short critical section on local driver private data.

Typical scenarios: **AMP**, **heterogeneous multicore**, **CPU remoteproc** register blocks, **shared message SRAM** between Linux and RT-Thread.

1.4.2.49.1.6 Integration with mailbox / rpmsg

Agree on a **fixed lock index map** in DTS and in remote firmware (e.g. lock **0** = mailbox header, lock **1** = virtio queue). See Mailbox subsystem and RPMsg.
Suggested **lock order** (document in BSP): acquire **hwspinlock before** ringing doorbell / sending mailbox, release **after** MMIO writes visible—avoid holding hwlock while waiting on a blocking IPC call from the other side.

1.4.2.49.1.7 Engineer checklist

1. **Provider**: `hwspinlock_bank_alloc`, implement `trylock`/`unlock`/`relax`, set `base_id` consistently with DT indices, `rt_hwspinlock_bank_register`.
2. **Consumer**: `rt_ofw_get_hwspinlock_by_name` in `probe`; `put` in `remove`.
3. **Critical section**: `lock_timeout_irqsave` → short MMIO → `unlock_irqrestore`.
4. **Teardown**: ensure all locks **unlocked** and `put` before `bank_unregister`.

1.4.2.49.1.8 Pitfalls

- **`rt_hwspinlock_put` without unlock** — next **`get`** may see hardware still owned by this or another core.
- **Wrong `#hwlock-cells` or index** — **`rt_ofw_get_`**∗ returns **`RT_ENOSYS`** or wrong slot → silent corruption.
- **Sleep while holding hwlock** — deadlock remote core or violate **`relax`** busy-wait assumptions.
- **Recursive acquire** — not supported (hardware + software locks).
- **Cross-core unlock** — undefined on most IPs; always unlock on acquirer CPU.
- **Deadlock with mailbox**: ISR holds lock A, thread waits on IPC that needs A on other core—define global lock ordering.
- **`unregister` with active refs** — **`-RT_EBUSY`**; remote side may still hold hardware lock even if ref count looks idle.

1.4.2.49.1.9 See also

- `components/drivers/include/drivers/hwspinlock.h`
- `components/drivers/hwspinlock/hwspinlock_dm.h`, `hwspinlock.c`
- Device model helpers (dm.h) — **`rt_dm_dev_bind_fwdata`**, **`rt_platform_ofw_request`**
- Open Firmware (OFW) — phandle / **`#hwlock-cells`**
- Mailbox subsystem, RPMsg
- Platform bus

1.4.2.50 Industrial I/O (IIO)

1.4.2.50.1 IIO (Industrial I/O)

Device-tree **`io-channels`** binding and DM integration: IIO device model (DM).

1.4.2.50.1.1 Overview

`drivers/iio.h` offers a small **channel discovery** API: `rt_iio_channel_get_by_index` and `rt_iio_channel_get_by_name` resolve a channel handle and index from a `rt_device`. IIO is a light abstraction for **ADC, DAC, temperature, accelerometer**, and similar sensors; data format and calibration are driver-specific.

1.4.2.50.1.2 Usage

1. Associate consumer and provider via device tree `io-channels` / `io-channel-names` or private matching.
2. Call the `get` helpers for an opaque channel pointer and logical index, then use the provider's read/write path (usually wrapped inside the driver).

1.4.2.50.1.3 DM relation

Providers are commonly `platform` devices: `probe` brings up clocks, resets, and calibration, then registers the device. Phandle-based dependencies mirror Linux IIO bindings and ease porting.

1.4.2.50.1.4 Practical limits (current API)

`drivers/iio.h` only exposes **`rt_iio_channel_get_by_index`** and **`rt_iio_channel_get_by_name`** returning an **opaque `void` ∗** channel cookie plus **`out_channel`** index. There is **no standard `put`** in this header—lifetime is **owned by the provider `rt_device`**; do not use the handle after the provider unregisters.

1.4.2.50.1.5 When IIO glue helps

Use IIO discovery when...	Skip it when...
Porting **Linux `io-channels`** consumers and you want matching property names.	You already have a **private `ioctl`** between two in-tree drivers.

1.4.2.50.1.6 Consumer flow

1. **`get_by_`*** after provider `probe` has finished (phandles resolved).
2. Pass **`out_channel`** index into **your provider's** read/write routine (implemented alongside the ADC driver).
3. On remove, drop references to the **`rt_device`** only—no separate channel free unless your provider documents one.

1.4.2.50.1.7 Pitfalls

- **Opaque pointer** is not a `struct iio_channel` like Linux—do not cast blindly across drivers.
- **Missing provider**: returns `**NULL**`—always check before dereference.

1.4.2.50.1.8 See also

- `components/drivers/include/drivers/iio.h`
- Platform: `documentation/6.components/device-driver/platform/platform.md`

1.4.2.51 IIO device model (DM)

1.4.2.51.1 Industrial I/O channel discovery under `RT_USING_DM`

RT-Thread **IIO** support is intentionally **small**: it does **not** implement a full Linux IIO subsystem (no `iio_device` bus, no buffer/trigger framework in-tree). With **`RT_USING_DM`**, **`components/drivers/iio/iio.c`** parses device-tree **`io-channels`** / **`io-channel-names`** (Linux-compatible binding names) and returns an **opaque provider handle** plus a **channel index**.
Data path (read raw ADC counts, scale to mV, trigger buffers, etc.) remains **entirely in the provider driver** (ADC, thermal, PMIC ADC, ...).
Overview of the IIO concept: Industrial I/O (IIO). This page documents **DM phandle resolution** and **provider obligations**.
Sources: **`components/drivers/iio/iio.c`**, **`components/drivers/include/drivers/iio.h`**.

1.4.2.51.1.1 Kconfig / build

Requirement	Role
`RT_USING_DM`	**`iio/SConscript`** builds **`iio.c`** only when DM is on
`RT_USING_OFW`	**`ofw_iio_channel_get_by_index`** parses phandles; without OFW, **`get_*`** **returns `NULL`**
Provider driver	Separate Kconfig (e.g. **`RT_USING_ADC`**, SoC SARADC in BSP) — not part of **`iio.c`**

`drivers/iio.h` is included from **`rtdevice.h`**; link **`iio.c`** by enabling **`RT_USING_DM`**.
There is **no** separate **`RT_USING_IIO`** menu entry — IIO glue is a DM companion module.

1.4.2.51.1.2 Architecture

```
Consumer (platform device, e.g. PMIC / headset / fuel gauge logic)
      |
      |  dev->ofw_node has io-channels / io-channel-names
      v
rt_iio_channel_get_by_name(dev, "vdd", &ch)
```

```
      |
      |  rt_dm_dev_prop_index_of_string("io-channel-names")
      |  rt_ofw_parse_phandle_cells("io-channels", "#io-channel-cells", index)
      v
Provider OFW node (e.g. saradc, tsadc)
      |
      |  rt_platform_ofw_request if not probed
      |  return rt_ofw_data(provider_np)  /* opaque void * */
      |  *out_channel = specifier args[0]
      v
Consumer casts cookie to provider private type
      + uses out_channel as logical channel index
      + calls provider-specific read API (rt_adc_*, custom fn, ...)
```

Unlike I2C device model (DM), IIO has **no `rt_bus`** and **no automatic client probe**. Only **phandle lookup** is centralized.

1.4.2.51.1.3 Public API (`iio.h`)

```
void *rt_iio_channel_get_by_index(struct rt_device *dev, int index, int *out_channel);
void *rt_iio_channel_get_by_name(struct rt_device *dev, const char *name, int *out_channel);
```

API	Behavior
`get_by_index`	Parse Nth entry in consumer's **`io-channels`** list
`get_by_name`	Resolve index via **`io-channel-names`**, then same as **`get_by_index`**
Return value	**`void *`** = **`rt_ofw_data(provider_np)`** after optional **`rt_platform_ofw_request`**
`out_channel`	First cell of provider **`#io-channel-cells`** specifier (logical channel number)

Input check	Result
`dev == NULL`, **`index < 0`**, **`name == NULL`**	**`NULL`**
Consumer has **no `dev->ofw_node`**	**`NULL`**
Missing **`io-channels`** / parse error	**`NULL`**
Provider not probed	**`rt_platform_ofw_request`** then retry **`rt_ofw_data`**

There is **no `rt_iio_channel_put`** in the header — lifetime follows the **provider `rt_device`** / platform driver **`remove`**. Do not use the cookie after the provider unregisters.

1.4.2.51.1.4 Device tree bindings

Provider (ADC / sensor front-end)

```
saradc: adc@ff100000 {
    compatible = "rockchip,rk3588-saradc";
    reg = <0x0 0xff100000 0x0 0x100>;
    interrupts = <GIC_SPI 93 IRQ_TYPE_LEVEL_HIGH>;
    clocks = <&cru CLK_SARADC>, <&cru PCLK_SARADC>;
    clock-names = "saradc", "pclk";
    vref-supply = <&vcc_1v8_s0>;

    #io-channel-cells = <1>;
    io-channel-ranges;
};
```

Property	Role
`#io-channel-cells = <1>`	One integer per phandle in consumer **`io-channels`**
`io-channel-ranges`	Optional (Linux); presence documents channel map semantics for porters
Channel index	**0 ... N-1** — meaning defined by **provider driver** (SARADC channel id, TSADC sensor input, ...)

Consumer

```
fuel-gauge {
    compatible = "vendor,fg";
    io-channels = <&saradc 4>, <&saradc 5>;
    io-channel-names = "bat-voltage", "bat-current";
};
```

Property	Role
io-channels	Phandle + one cell per channel (provider + index)
io-channel-names	Optional strings aligned with **io-channels** order

get_by_name(dev, "bat-voltage", &ch) → cookie = saradc private struct, **ch == 4**.

1.4.2.51.1.5 Provider driver contract

The provider must expose **rt_ofw_data(provider_np)** before consumers resolve channels. Typical pattern (from **adc-rockchip_saradc.c**):

```
static rt_err_t rockchip_saradc_probe(struct rt_platform_device *pdev)
{
    struct rt_device *dev = &pdev->parent;
    struct rockchip_saradc *rk_saradc = rt_calloc(1, sizeof(*rk_saradc));

    /* clocks, regulator, iomap, irq, rt_hw_adc_register(...) */

    rt_dm_dev_bind_fwdata(dev, RT_NULL, rk_saradc);
    return RT_EOK;
}

static rt_err_t rockchip_saradc_remove(struct rt_platform_device *pdev)
{
    rt_dm_dev_unbind_fwdata(dev, RT_NULL);
    /* unregister ADC, free resources */
}
```

Obligation	Detail
rt_dm_dev_bind_fwdata(dev, NULL, priv)	Sets **rt_ofw_data(dev->ofw_node) = priv** (Device model helpers (dm.h))
#io-channel-cells in DTS	Must match **args_count == 1** in **iio.c** (otherwise lookup fails)
Probe before consumer	Or rely on **rt_platform_ofw_request** during **get_***
Document channel map	What index **0, 1, 2...** mean (pin / internal mux / TSADC sensor)

Returned cookie is usually the **provider's private struct** (`struct rockchip_saradc *`, etc.) — **not** a portable `struct iio_channel`. Consumers in the same BSP cast to the known type.

1.4.2.51.1.6 Consumer driver pattern

```
#include <drivers/iio.h>

static rt_err_t my_consumer_probe(struct rt_platform_device *pdev)
{
    struct rt_device *dev = &pdev->parent;
    void *iio_priv;
    int ch;

    iio_priv = rt_iio_channel_get_by_name(dev, "bat-voltage", &ch);
    if (!iio_priv)
    {
        return -RT_EINVAL;
    }

    priv->saradc = (struct rockchip_saradc *)iio_priv;
```

```
    priv->channel = ch;

    /* Use priv->saradc + priv->channel with provider helpers
     * e.g. rt_adc_enable/read on &priv->saradc->parent, or direct HW access */
    return RT_EOK;
}
```

Step	Notes
Call **get_* in probe**	After consumer **ofw_node** is set; provider may be deferred-probed by **rt_platform_ofw_request**
Store **cookie + out_channel**	Cookie = provider priv; **ch** selects logical input
No standard read API	Pair with **rt_adc_***, **rt_sensor_***, or vendor functions
remove	Drop pointers; do not call **get_*** again

1.4.2.51.1.7 Relation to ADC / sensor frameworks

Framework	Role vs IIO
ADC Device	**rt_adc_device** is often the **provider** registered by SARADC; IIO only **discovers** which channel another driver needs
Sensor framework	Separate path (`rt_sensor_*`); may coexist — do not confuse sensor bus with IIO phandles

IIO is **glue for DT io-channels**, not a replacement for **rt_device_find("adc0")**.

1.4.2.51.1.8 Comparison with Linux IIO

Linux	RT-Thread DM IIO
`struct iio_channel` + `iio_channel_get()`	Opaque **void** * + **out_channel int
`iio_read_channel_raw()` etc.	**Not in-tree** — provider-specific
IIO core + buffers/triggers	**Not implemented**

Porting focus: reproduce **DT property names** and **channel indices**; reimplement read path against local ADC driver.

1.4.2.51.1.9 Engineer checklist

1. **Provider DTS**: **#io-channel-cells = <1>**; document channel index meanings.
2. **Provider probe**: **rt_dm_dev_bind_fwdata** with stable **priv** pointer.
3. **Consumer DTS**: **io-channels** + optional **io-channel-names**.
4. **Consumer probe**: **rt_iio_channel_get_by_***, check **NULL**, store cookie + index.
5. **Reads**: use provider APIs; honor **rt_adc_enable** / **mutex** rules of that driver.
6. **Teardown**: consumer **remove** first or ensure no use-after-free when provider exits.

1.4.2.51.1.10 Pitfalls

- **Opaque pointer is provider-specific** — do not cast to a fictional `struct iio_channel`.
- **args_count != 1 — iio.c returns failure**; multi-cell specifiers are unsupported today.

- **Missing `bind_fwdata` — `get_*` returns NULL** even if ADC device registered.
- **Consumer without `ofw_node`** — always **`NULL`** (no manual fallback in **`iio.c`**).
- **Wrong channel index** — parse succeeds but reads wrong pin / garbage data.
- **No refcount on cookie** — provider **`remove`** while consumer still holds pointer → UAF.
- **Assuming full IIO stack** — scaling, sysfs, buffer DMA are out of scope for **`iio.c`**.

1.4.2.51.1.11 See also

- Industrial I/O (IIO) — role and limits summary
- Device model helpers (dm.h) — **`rt_dm_dev_bind_fwdata`**, property helpers
- Open Firmware (OFW) — **`rt_ofw_parse_phandle_cells`**
- Platform bus — consumer/provider **`probe`**
- ADC Device — typical provider (**`rt_hw_adc_register`**)
- `components/drivers/iio/iio.c`
- `components/drivers/include/drivers/iio.h`

1.4.2.52 Input subsystem

Device-tree binding and DM registration: Input device model (DM).

Topic	Page
Userspace API	Input userspace API (UAPI)
Touch class bridge	Input touch bridge
Power / restart keys	Input power keys

1.4.2.52.1 Input subsystem (API)

Header: `drivers/input.h`. Typical stack: low-level driver → **`rt_input_event`** / **reporters** → handlers (GUI, shell) or **`/dev/inputN`** (Input userspace API (UAPI)).

Path	API
In-kernel listener	**`rt_input_add_handler`**
POSIX process (Smart)	**`open`** / **`read`** / **`ioctl`** / **`poll`** **on `/dev/inputN`** — Input userspace API (UAPI)
Legacy touch GUI	**`struct rt_touch_device`** — Input touch bridge

1.4.2.52.1.1 When to use the input core

Use `rt_input_*` when...	Consider raw **`rt_device`** read when...
You want **Linux-like** `EV_KEY` / `EV_REL` / `EV_ABS` / multitouch slot semantics and multiple listeners.	You have a **single proprietary** app talking to a vendor IOCTL-only touch stack.
You need **polling** (`**rt_input_setup_↩ polling**`) for GPIO keys without IRQ storms.	GPIO IRQ latency is fine and you do not need unified event routing.

1.4.2.52.1.2 Event contract

1. **rt_input_set_capability** before **rt_input_device_register** so users know which **type**/**code** pairs are valid.
2. For each hardware change, call **rt_input_report_**∗ then **rt_input_sync** to close the frame (**SYN_REPORT**). Missing **sync** merges events poorly for consumers.
3. **rt_input_trigger** routes through optional **idev->trigger**∗∗**—use when you wrap another input device; otherwise** ∗∗**rt_input_event** is enough.

1.4.2.52.1.3 Callback / context rules

- **rt_input_add_handler** → **callback**: invoked from **rt_input_event** path—assume **thread or workqueue** unless your driver documents IRQ delivery; keep **callback short**.
- **poller->poll**: runs on timer tick—∗∗must not block∗∗ or take mutexes that can deadlock with the same thread calling **rt_input_sync**.
- **Touch helpers**: **rt_input_setup_touch** / **rt_input_report_touch_**∗ require **RT_INPUT_TOUCHSCREEN** — Input touch bridge.

1.4.2.52.1.4 Registration lifecycle

- **rt_input_device_register**: registers underlying **rt_device**; failures leave partially init state—clean up partial setup in error paths.
- **rt_input_remove_config**: tears down polling/touch dynamic config before unregister if you toggled features at runtime.

1.4.2.52.1.5 Pitfalls

- **ABS without rt_input_set_absinfo**: consumers cannot clamp—set min/max/resolution for axes you report.
- **Key autorepeat**: not automatic unless a layer implements it—do not assume Linux repeat behavior.
- **Re-entrancy**: posting events from inside a handler for the same device can recurse—prefer queueing.

1.4.2.52.1.6 See also

- Input device model (DM)
- Input userspace API (UAPI)
- Input touch bridge
- Input power keys
- components/drivers/include/drivers/input.h
- components/drivers/include/drivers/input_uapi.h
- dt-bindings/input/event-codes.h

1.4.2.53 Input device model (DM)

1.4.2.53.1 Input subsystem under RT_USING_DM

The **input core** (components/drivers/input/input.c) is built only when **RT_USING_INPUT** is enabled. That Kconfig option **depends on RT_USING_DM** — there is **no** legacy non-DM input stack in-tree. Drivers publish Linux-style **EV_**∗ events through **struct rt_input_device**; consumers attach via **rt_input_add_handler** or POSIX **/dev/inputN** (Input userspace API (UAPI)).
Event reporting API, polling rules, and pitfalls: Input subsystem.

Topic	Page
Userspace **open**/**read**/**ioctl**	Input userspace API (UAPI)
rt_touch_device bridge	Input touch bridge
KEY_POWER / **KEY_RESTART** shutdown/reset	Input power keys

Sources:

File	Role
input.c	Global device list, **rt_dm_ida** naming (`input0...`), **rt_input_device_register**
input_power.c	Input power keys
input_touch.c	Input touch bridge
input_uapi.c	Input userspace API (UAPI)
keyboard/, **joystick/**, **touchscreen/**, **misc/**	OFW platform/SPI drivers

1.4.2.53.1.1 Kconfig / build

Option	Role
RT_USING_DM	Parent — input cannot be enabled without DM
RT_USING_INPUT	Core **input.c**; selects **RT_USING_ADT** + bitmap ADT
RT_INPUT_POWER	Input power keys (default **y**)
RT_INPUT_UAPI	Input userspace API (UAPI)
RT_INPUT_KEYBOARD	**gpio-keys**, **adc-keys** (see sub-options)
RT_INPUT_JOYSTICK	**adc-joystick**
RT_INPUT_TOUCHSCREEN	Input touch bridge
RT_INPUT_MISC	e.g. **ettus,e3x0-button**
SOC_DM_INPUT_*_DIR	BSP adds more platform drivers via **osource** in sub-Kconfig

Unlike I2C device model (DM), input has **no dedicated rt_bus**. Each hardware block is a normal **struct rt_platform_driver** (or **struct rt_spi_driver**) that embeds or owns **struct rt_input_device** and calls **rt_input_device_register** from **probe**.

1.4.2.53.1.2 Architecture

```
Device tree node (compatible = "gpio-keys", "adc-keys", ...)
      |
      |  platform bus match (rt_ofw_node_id) or SPI bus match
      v
driver probe(pdev)
      |
      |  parse OFW (gpios, io-channels, irq, ...)
      |  rt_input_set_capability(idev, EV_KEY|EV_ABS, code)
      |  optional: rt_input_setup_polling / rt_input_setup_touch
      v
rt_input_device_register(idev)
      |
      |  rt_dm_ida_alloc → name "inputN"
      |  rt_device_register (RT_Device_Class_Char, DEACTIVATE)
      |  [RT_INPUT_UAPI] input_uapi_init
      |  list on input_device_nodes
      v
[RT_INPUT_TOUCHSCREEN] input_touch_register → rt_touch_device
[RT_INPUT_POWER]       global KEY_POWER / KEY_RESTART handlers
```

```
      |
      v
Hardware IRQ / timer poll → rt_input_report_* → rt_input_sync
      |
      v
rt_input_event → handlers / UAPI queue
```

Layer	Object	Notes
Input device	**struct rt_input_device**	Embeds **struct rt_device parent**; capabilities in bitmaps
DM naming	**input_ida** in **input.c**	**rt_dm_dev_set_name(&idev->parent, "inputu", id)**
Platform binding	**RT_PLATFORM_DRIVER_EXPORT**	Probe/remove on **struct rt_platform_device**
SPI touch	**RT_SPI_DRIVER_EXPORT**	e.g. ADS7846 — see Input touch bridge

1.4.2.53.1.3 Core registration (input.c)

```
rt_err_t rt_input_device_register(struct rt_input_device *idev);
rt_err_t rt_input_device_unregister(struct rt_input_device *idev);
```

Step	Behavior
Allocate ID	**rt_dm_ida_alloc(&input_ida)** — failure → **-RT_EFULL**
Init lists / lock	Per-device **handler_nodes**, **rt_spinlock**
Register char dev	**RT_Device_Class_Char**, flag **RT_DEVICE_FLAG_DEACTIVATE**
Post-register	Start poller timer if **idev->poller**; **input_touch_register**; UAPI fops if enabled
Unregister	Refcount check; UAPI/touch teardown; **rt_dm_ida_free**

Driver pattern — embed input at a known offset:

```
struct my_input {
    struct rt_input_device parent;
    /* private */
};

static void my_report(struct my_input *mi)
{
    rt_input_report_key(&mi->parent, KEY_ENTER, 1);
    rt_input_sync(&mi->parent);
}
```

Use **rt_container_of(idev, struct my_input, parent)** in poll/IRQ callbacks.

1.4.2.53.1.4 In-tree OFW drivers

Compatible / binding	Driver file	Bus	Summary
gpio-keys	**keyboard/keys-gpio.c**	platform	Parent node; **each child** → one **inputN** per key
adc-keys	**keyboard/keys-adc.c**	platform	**io-channels** via IIO device model (DM); polling
adc-joystick	**joystick/js-adc.c**	platform	Per-child **reg**, IIO, **EV_ABS** polling
ti,ads7846 ... **ti,xpt2046**	**touchscreen/ts-ads7846.c**	SPI	Input touch bridge

Compatible / binding	Driver file	Bus	Summary
ettus,e3x0-button	**misc/button-e3x0.c**	platform	IRQ **press/release** → **KEY_POWER**

gpio-keys (platform + children)

- Parent **compatible = "gpio-keys"**; **subnodes** are individual buttons.
- Each child: **rt_ofw_get_named_pin**, property matching ***,code$** (e.g. **linux,code**), optional **debounce-interval**.
- **One rt_input_device per child** (not one device for the whole parent).

adc-keys / adc-joystick (IIO consumers)

- Resolve ADC through **rt_iio_channel_get_by_name** / **rt_iio_channel_get_by_index** (IIO device model (DM)).
- Default **poll-interval 200 ms** if property missing.

ADS7846 family (SPI)

- **RT_SPI_DRIVER_EXPORT(ads7846_driver)** ; regulator, IRQ, worker thread.
- DT axis/touch properties consumed by **input_touch_parse** — Input touch bridge.

1.4.2.53.1.5 Device tree examples

GPIO key (child node)

```
gpio-keys {
    compatible = "gpio-keys";

    power-key {
        label = "Power";
        gpios = <&gpio0 5 GPIO_ACTIVE_LOW>;
        linux,code = <116>;   /* KEY_POWER -- see page_device_input_power */
        debounce-interval = <15>;
    };
};
```

ADC resistor ladder

```
adc-keys {
    compatible = "adc-keys";
    io-channels = <&saradc 0>;
    io-channel-names = "buttons";
    poll-interval = <100>;
    keyup-threshold-microvolt = <1800000>;

    up {
        press-threshold-microvolt = <500000>;
        linux,code = <103>;   /* KEY_UP */
    };
};
```

SPI resistive touch

```
touch: touchscreen@0 {
    compatible = "ti,ads7846";
    reg = <0>;
    interrupt-parent = <&gpio1>;
    interrupts = <25 IRQ_TYPE_EDGE_FALLING>;
    ti,x-min = /bits/ 16 <0>;
    ti,x-max = /bits/ 16 <4095>;
    ti,y-min = /bits/ 16 <0>;
    ti,y-max = /bits/ 16 <4095>;
    vcc-supply = <&reg_3v3>;
};
```

(SPI **reg** / **spi-max-frequency** belong on the child under the SPI controller node per your BSP.)

1.4.2.53.1.6 New platform input driver checklist

1. Enable **RT_USING_DM**, **RT_USING_OFW**, **RT_USING_INPUT** (and subsystem menu if you add under **keyboard/** etc.).
2. **struct rt_platform_driver** with **ofw_ids[]** matching Linux binding where possible.
3. In **probe**: allocate driver private struct with **struct rt_input_device** embedded first (or pointer field).
4. **rt_input_set_capability** for every **type**/**code** you will report; for **EV_ABS**, call **rt_input_set_absinfo** (after capability allocates **absinfo**).
5. IRQ or **rt_input_setup_polling** — poller **poll** must not block (Input subsystem).
6. On each logical event frame: **rt_input_report_*** then **rt_input_sync**.
7. **rt_input_device_register** only when hardware is ready; store private data in **pdev->parent.user_data** or **rt_ofw_data(np)** for **remove**.
8. **remove**: stop IRQ/poller, **rt_input_device_unregister**, free memory.
9. **RT_PLATFORM_DRIVER_EXPORT(drv)** (or **RT_SPI_DRIVER_EXPORT** on SPI slaves).

1.4.2.53.1.7 Pitfalls

Issue	Mitigation
Missing rt_input_sync	Consumers see partial frames — always end with **rt_input_sync**.
Capability after register	Set capabilities **before rt_input_device_register**.
EV_ABS without absinfo	Call **rt_input_set_capability(EV_ABS, axis)** then **rt_input_set_absinfo**.
Unregister with open UAPI fd	Close **/dev/inputN** first — Input userspace API (UAPI)
gpio-keys child vs parent	Driver registers **per child**; do not expect one **input0** for the whole node.
IIO channel NULL	Fix DT **io-channels** — IIO device model (DM)
Power handler surprise	Input power keys

1.4.2.53.1.8 See also

- Input subsystem
- Input userspace API (UAPI)
- Input touch bridge
- Input power keys
- IIO device model (DM)
- Bus and driver binding (core)
- `components/drivers/include/drivers/input.h`
- `components/drivers/input/Kconfig`

1.4.2.54 Input userspace API (UAPI)

1.4.2.54.1 Input event UAPI (`RT_INPUT_UAPI`)

`input_uapi.c` exposes each **`rt_input_device`** as a **Linux-compatible** character device for POSIX userspace (Smart / LWP + DFS). IOCTLs and **`struct input_event`** are defined in **`drivers/input_uapi.h`** (aligned with **`include/uapi/linux/input.h`**).
DM overview and driver binding: Input device model (DM). In-kernel reporters and handlers: Input subsystem.
Sources: **`components/drivers/input/input_uapi.c`**, **`components/drivers/include/drivers/input_uapi.h`**.

1.4.2.54.1.1 Kconfig

Option	Default	Role
`RT_INPUT_UAPI`	**y** if **`RT_USING_SMART`**, else **n**	Build **`input_uapi.c`**, attach **`dfs_file_ops`** in **`input_uapi_init`**
`RT_INPUT_UAPI_EVENT_MAX`	**128**	Ring buffer slots (**`struct input_event`** per device)
`RT_UAPI_FAKE_BLOCK`	**y**	**`O_NONBLOCK`** empty **`read`** returns synthetic **`SYN_REPORT`** instead of **`-EAGAIN`**

Dependencies: **`RT_USING_INPUT`**, **`RT_USING_KTIME`**, **`RT_USING_POSIX_DEVIO`**.

1.4.2.54.1.2 Architecture

```
rt_input_report_* → rt_input_sync
      → rt_input_event (input.c)
            → input_uapi_event (if ref_count > 0)
                  ring buffer + ktime timestamp
                  SYN_REPORT → sync_count++, wake wait_queue
      → read() / poll(POLLIN) on /dev/inputN
```

Hook	When
`input_uapi_init`	**`rt_input_device_register`** — allocates **`struct input_uapi`**, sets **`idev->parent.fops`**
`input_uapi_event`	Every **`rt_input_event`** while UAPI enabled
`input_uapi_finit`	**`rt_input_device_unregister`** — free UAPI state

Registered device names are **`input0`**, **`input1`**, … (**`rt_dm_ida`** in **`input.c`**). Open via devfs:

```
int fd = open("/dev/input0", O_RDONLY);
```

`input_uapi_event` returns without queuing when **`idev->parent.ref_count == 0`** — userspace must **`open`** the node before events appear.

1.4.2.54.1.3 Event format

	Kernel handler path	UAPI queue
Struct	**`struct rt_input_event`** (`drivers/input.h`)	**`struct input_event`** (`input_uapi.h`)
Time	**`rt_tick_t tick`**	**`struct timeval`** from **`rt_ktime_boottime_get_us`**
Codes	**`dt-bindings/input/event-codes.h`**	Same

struct input_absinfo returned by **EVIOCGABS** matches **struct rt_input_absinfo** (**RT_ASSERT** on layout in **input_uapi_init**).

1.4.2.54.1.4 Ring buffer and framing

struct input_uapi holds:

- Atomic **read_idx**, **write_idx**, **sync_count**
- **events[RT_INPUT_UAPI_EVENT_MAX]**
- Optional **grabbed_file** for **EVIOCGRAB**

On each kernel event copy, **write_idx** advances. When **type == EV_SYN** and **code == SYN_REPORT**, **sync_count** increments and **rt_wqueue_wakeup** runs.
read drains events until the user buffer is full or a **SYN_REPORT** completes one frame; each consumed frame decrements **sync_count**.
Overflow: if the next write would overwrite unread data, the event is **dropped** (warning log). Raise **RT_INPUT_UAPI_EVENT_MAX** for high-rate touch streams.

1.4.2.54.1.5 read() / poll() / write()

Behavior	Detail
Min size	**count != 0** and **count < sizeof(struct input_event)** → **-EINVAL**
count == 0	Allowed (availability check after wait)
Blocking	Waits on **idev->parent.wait_queue** until **sync_count > 0**
O_NONBLOCK	Empty queue → **-EAGAIN**, or fake **SYN_REPORT** if **RT_UAPI_FAKE_BLOCK**
poll	**POLLIN** when **sync_count > 0** (**O_RDONLY** / **O_RDWR** only)
write	**-ENOSYS**

Grab: if another **dfs_file** holds **EVIOCGRAB**, **read** returns **-EAGAIN**.

1.4.2.54.1.6 ioctl() (implemented)

IOCTL	Purpose
EVIOCGVERSION	**EV_VERSION** (0x010001)
EVIOCGID	Virtual ID: bustype **0x06**, vendor **0x5354**, product **0x4556**, version **RT_VER_NUM >> 16**
EVIOCGNAME(len)	Device name (**inputN**)
EVIOCGPROP(len)	Property bitmap (direct-touch bit when buffer large enough)
EVIOCGBIT(ev, len)	Type bitmap (**ev==0**) or **EV_KEY**/**EV_REL**/**EV_ABS** maps
EVIOCGABS(abs)	Axis **struct input_absinfo**
EVIOCGRAB	Exclusive grab; non-zero **args** grabs; second grabber **-EBUSY**

EVIOCSABS, force-feedback, and other Linux input IOCTLs → **-EINVAL**.

1.4.2.54.1.7 Userspace example

```
#include <fcntl.h>
#include <unistd.h>
#include <poll.h>
#include <drivers/input_uapi.h>

void listen_input0(void)
```

```
{
    struct input_event ev[64];
    struct pollfd pfd = { .fd = open("/dev/input0", O_RDONLY), .events = POLLIN };

    for (;;)
    {
        if (poll(&pfd, 1, -1) <= 0)
            continue;

        ssize_t n = read(pfd.fd, ev, sizeof(ev));
        if (n <= 0)
            continue;

        for (int i = 0; i < n / (int)sizeof(struct input_event); ++i)
        {
            /* ev[i].type, ev[i].code, ev[i].value */
        }
    }
}
```

Use **rt_input_add_handler** in kernel; use UAPI when a POSIX process (e.g. **evtest**) reads the same event stream.

1.4.2.54.1.8 Pitfalls

Issue	Mitigation
No events before open	**ref_count** gate in **input_uapi_event**
Unregister with open fd	**rt_input_device_unregister** checks **ref_count** — close fds first
Grab conflicts	Release with **EVIOCGRAB** and zero argument
Touch bursts	Increase **RT_INPUT_UAPI_EVENT_MAX** or read faster
RT_UAPI_FAKE_BLOCK	Disable for strict **-EAGAIN** on empty non-blocking reads

1.4.2.54.1.9 See also

- Input device model (DM)
- Input subsystem
- components/drivers/include/drivers/input_uapi.h
- components/drivers/input/Kconfig

1.4.2.55 Input touch bridge

1.4.2.55.1 Touch bridge (RT_INPUT_TOUCHSCREEN)

input_touch.c bridges **struct rt_input_device** (Linux-style **EV_ABS** / multitouch slots) to **struct rt_touch_device** so GUI stacks can use the legacy **touch class** API (**rt_device_find**, **RT_TOUCH_CTRL_***, **rt_hw_touch_isr**) without duplicating panel drivers.
Input core and OFW drivers (e.g. ADS7846): Input device model (DM). Event helpers (**rt_input_setup_touch**, **rt_input_report_touch_***): Input subsystem.
Sources: **components/drivers/input/input_touch.c**, **components/drivers/input/touchscreen/**.

1.4.2.55.1.1 Kconfig

Option	Role
RT_INPUT_TOUCHSCREEN	Build **input_touch.c**; **selects RT_USING_TOUCH**
RT_INPUT_TOUCHSCREEN_ADS7846	In-tree SPI resistive driver **ts-ads7846.c**

Option	Role
SOC_DM_INPUT_TOUCHSCREEN_DIR	BSP adds more panel drivers

Requires **RT_USING_INPUT** and **RT_USING_DM**.

1.4.2.55.1.2 Architecture

```
Panel driver (e.g. ads7846)
      |
      |  rt_input_set_capability(EV_ABS, ...)
      |  rt_input_setup_touch(idev, num_slots, &touch_info)
      |  rt_input_device_register(idev)
      v
input_touch_register (after idev on global list)
      |
      |  rt_hw_touch_register → device "touchN"
      |  rt_input_add_handler → input_touch_cb
      v
Driver reports: rt_input_report_touch_* / rt_input_sync
      |
      v
input_touch_cb: EV_ABS → struct rt_touch_data
                SYN_REPORT → rt_hw_touch_isr()
      |
      v
GUI: rt_device_read / touch ISR callback
```

Object	Role
struct input_touch_properties	Stored in **idev->touch**: DT-derived min/max, invert, swap, slot count
struct input_touch_device	Embeds **struct rt_touch_device** + **rt_input_handler** + per-slot **rt_touch_data**
input_touch_ops	**touch_readpoint**, **touch_control** (range, enable/disable, status)

input_touch_register only creates the **rt_touch_device** when **rt_input_setup_touch** was called with non-NULL **struct rt_touch_info *info** (**prop->touch_dev** allocated).

1.4.2.55.1.3 Driver API (input.h)

```
rt_err_t rt_input_setup_touch(struct rt_input_device *idev,
        rt_uint32_t num_slots, struct rt_touch_info *info);
rt_err_t rt_input_parse_touch_position(struct rt_input_device *idev,
        rt_uint32_t *out_x, rt_uint32_t *out_y);
rt_bool_t rt_input_report_touch_inactive(struct rt_input_device *idev, rt_bool_t active);
void rt_input_report_touch_position(struct rt_input_device *idev,
        rt_uint32_t x, rt_uint32_t y, rt_bool_t multitouch);
```

API	Role
rt_input_setup_touch	Allocate **idev->touch**, parse DT, optional multitouch absinfo (**ABS_MT_SLOT**, **ABS_MT_TRACKING_ID**)

| **num_slots == 0** | Single-touch (**ABS_X**/**ABS_Y**); **num_slots** > **0** | Multitouch slot protocol | | **rt_input_report_touch_position** | Apply invert/swap from DT, then **rt_input_report_abs** | | **rt_input_report_touch_inactive** | Emit **ABS_MT_TRACKING_ID = -1** on release | | **rt_input_parse_touch_position** | Map raw x/y through **touchscreen-inverted-*** / **touchscreen-swapped-x-y** |
Call **rt_input_setup_touch** before **rt_input_device_register**. **input_touch_register** runs inside register after the device is linked globally.

1.4.2.55.1.4 Device tree properties (input_touch_parse)

Read from the **rt_input_device**'s **rt_device** OFW node via **rt_dm_dev_prop_***:

Property	Effect
touchscreen-min-x, **touchscreen-size-x**, **touchscreen-fuzz-x**	Override **ABS_X** or **ABS_MT_POSITION_X** absinfo
touchscreen-min-y, **touchscreen-size-y**, **touchscreen-fuzz-y**	Y axis
touchscreen-max-pressure, **touchscreen-fuzz-pressure**	Pressure axis
touchscreen-inverted-x, **touchscreen-inverted-y**	Flip coordinates in **rt_input_parse_touch_position**
touchscreen-swapped-x-y	Swap X/Y absinfo and coordinates

Missing properties fall back to values already set by **rt_input_set_absinfo** in the panel driver.

1.4.2.55.1.5 Handler translation (input_touch_cb)

When **touch_dev->enabled**:

Input event	Touch side
ABS_MT_SLOT	Select active slot index
ABS_MT_TRACKING_ID == -1	**RT_TOUCH_EVENT_UP**, **rt_hw_touch_isr**
ABS_MT_TRACKING_ID valid	DOWN / MOVE, increment **down** count
ABS_MT_POSITION_X/Y or **ABS_X/Y**	Scale raw value to **range_x**/**range_y** using absinfo min/max
EV_SYN + **SYN_REPORT**	**rt_hw_touch_isr** (frame complete)

input_touch_readpoint returns one **struct rt_touch_data** for the current slot (GUI pull model).

1.4.2.55.1.6 Example (driver probe sketch)

```
struct rt_touch_info info = {
    .type = RT_TOUCH_TYPE_CAPACITANCE,
    .vendor = RT_TOUCH_VENDOR_UNKNOWN,
    .point_num = 1,
    .range_x = 800,
    .range_y = 480,
};

rt_input_set_capability(idev, EV_ABS, ABS_X);
rt_input_set_capability(idev, EV_ABS, ABS_Y);
rt_input_set_absinfo(idev, ABS_X, 0, max_x, 0, 0);
rt_input_set_absinfo(idev, ABS_Y, 0, max_y, 0, 0);

rt_input_setup_touch(idev, 0, &info);   /* 0 slots = single touch */
rt_input_device_register(idev);

/* in IRQ / thread: */
rt_input_report_touch_position(idev, x, y, RT_FALSE);
rt_input_sync(idev);
```

For multitouch, pass **num_slots** > **0**, use **rt_input_report_touch_slot**, tracking id helpers, then **rt_input_sync** per frame.

1.4.2.55.1.7 Pitfalls

Issue	Mitigation
setup_touch after register	**input_touch_register** won't run correctly — setup before register

Issue	Mitigation
info == NULL	Properties-only mode: no **rt_touch_device** exported (handler-only path not used for HW touch register)
Missing rt_input_sync	Touch ISR won't see a full frame
ABS range zero	Scaling in **input_touch_cb** divides by **maximum - minimum** — set absinfo first
Unregister order	**input_touch_unregister** from **rt_input_device_unregister** — drop touch fd users first

1.4.2.55.1.8 See also

- Input device model (DM) — **ti,ads7846** SPI binding
- Input subsystem
- Input userspace API (UAPI) — parallel POSIX event path on same **inputN**
- components/drivers/include/drivers/dev_touch.h

1.4.2.56 Input power keys

1.4.2.56.1 Power / restart key handlers (RT_INPUT_POWER)

input_power.c installs global **rt_input_handler** instances that map **KEY_POWER** and **KEY_RESTART** release events to **rt_hw_cpu_shutdown()** and **rt_hw_cpu_reset()**. Any input device that advertises these keys in its capability bitmap can trigger system power actions without a dedicated PMIC driver hook.
DM and keyboard drivers: Input device model (DM). Handler API: Input subsystem.
Source: **components/drivers/input/input_power.c**.

1.4.2.56.1.1 Kconfig

Option	Default	Role
RT_INPUT_POWER	**y**	Build **input_power.c**, run **input_event_power_init** at env init

Depends on **RT_USING_INPUT**. Disable on boards where **KEY_POWER** must only be handled by application code.

1.4.2.56.1.2 Architecture

```
INIT_ENV_EXPORT(input_event_power_init)
      |
      +-- power_handler_install(KEY_POWER)
      +-- power_handler_install(KEY_RESTART)
              |
              |  (1) tmp_handler: identify=test, callback=power_input_callback
              |      rt_input_add_handler → scans all idevs, counts matches
              v
      (2) For each matching input device index:
              alloc rt_input_handler[N]
              identify=power_input_identify (binds to Nth device with key)
              callback=power_input_callback
              rt_input_add_handler
      |
      v
Any idev reports KEY_POWER/KEY_RESTART release (value==0)
      → rt_input_event → handler → shutdown/reset
```

Phase	Behavior
Probe pass	Temporary handler increments **cap.value** for each **rt_input_device** with **key_map[code]** set
Install pass	One persistent handler per matching device; **identify** matches the **current**-th device
Runtime	**power_input_callback**: on **value == 0** (key release), call BSP hook

Handlers return **RT_TRUE** from **callback** to stop further handler dispatch for that event.

1.4.2.56.1.3 Key actions

Linux code	Event	Action
KEY_POWER (116)	**EV_KEY**, **value == 0** (release)	**rt_hw_cpu_shutdown()**
KEY_RESTART	**EV_KEY**, **value == 0**	**rt_hw_cpu_reset()**

Press (**value == 1**) does nothing in **input_power.c** — only release triggers power ops.
BSP must implement **rt_hw_cpu_shutdown** / **rt_hw_cpu_reset** (weak symbols or SoC-specific).

1.4.2.56.1.4 Interaction with drivers

- **gpio-keys** child with **linux,code = <116>** registers an **inputN** with **KEY_POWER** in **key_map** — power handler attaches automatically when **RT_INPUT_POWER** is on.
- **ettus,e3x0-button** reports **KEY_POWER** on IRQ — same path.
- Does **not** require a dedicated compatible string; only the **advertised key code** matters.

If multiple devices expose **KEY_POWER**, each gets its own handler instance; any of them can shut down the system on release.

1.4.2.56.1.5 Disabling or overriding

Goal	Approach
No automatic shutdown	Set **RT_INPUT_POWER=n** in Kconfig
Custom policy	Disable **RT_INPUT_POWER** and use **rt_input_add_handler** with higher priority logic, or handle **KEY_POWER** only in userspace via Input userspace API (UAPI)
Hold-to-power-off	Not implemented here — implement in driver (long-press timer) before reporting release

1.4.2.56.1.6 Pitfalls

Issue	Mitigation
Unexpected shutdown	Any stray **KEY_POWER** release shuts down — verify DT **linux,code** and test harnesses
Missing capability	Driver must **rt_input_set_capability(idev, EV_KEY, KEY_POWER)** before register
Press-only testing	Release is required — press alone does not call shutdown
No matching devices	**power_handler_install** exits early if count is zero (no handlers allocated)

1.4.2.56.1.7 See also

- Input device model (DM) — **gpio-keys** DT example with **KEY_POWER**
- Input subsystem
- dt-bindings/input/linux-event-codes.h (or **event-codes.h**)

1.4.2.57 LED subsystem

Device-tree binding and platform drivers: LED device model (DM).

1.4.2.57.1 LED subsystem (API)

Header: **components/drivers/include/drivers/led.h**. Core: **components/drivers/led/led.c**.
Each **struct rt_led_device** registers as a char device **led0**, **led1**, ... under **RT_USING_DM**. Control via **rt_led_*** APIs or by **read**/**write** on the device node (shell / DFS).

1.4.2.57.1.1 When to use the LED core

Use rt_led_* when...	Raw GPIO/PWM may suffice when...
DT **gpio-leds** / **pwm-leds** should create uniform **ledN** devices.	One debug LED toggled directly in board code.
power_supply or policy code drives charging/status indicators.	LED is bootloader-only, never touched at runtime.
Userspace or MSH writes **on**/**off** or a brightness number to the device file.	No registration or naming needed.

1.4.2.57.1.2 States and operations

```
enum rt_led_state {
    RT_LED_S_OFF,
    RT_LED_S_ON,
    RT_LED_S_TOGGLE,
    RT_LED_S_BLINK,
};

rt_err_t rt_led_register(struct rt_led_device *led);
rt_err_t rt_led_unregister(struct rt_led_device *led);

rt_err_t rt_led_set_state(struct rt_led_device *led, enum rt_led_state state);
rt_err_t rt_led_get_state(struct rt_led_device *led, enum rt_led_state *out_state);
rt_err_t rt_led_set_period(struct rt_led_device *led, rt_uint32_t period_ms);
rt_err_t rt_led_set_brightness(struct rt_led_device *led, rt_uint32_t brightness);
```

API	Behavior
set_state	Calls **led->ops->set_state** under spinlock; stops software blink timer when leaving blink
RT_LED_S_BLINK	If ops returns **-RT_ENOSYS** and core allocated **blink_timer**, starts periodic timer (**heartbeat-style** timing in **led.c**)
get_state	Requires **ops->get_state** — else **-RT_ENOSYS**
set_period	Uses **ops->set_period** if present; else adjusts software blink timer period
set_brightness	Requires **ops->set_brightness** (PWM backend)

struct rt_led_ops

```
struct rt_led_ops {
```

```
    rt_err_t (*set_state)(struct rt_led_device *led, enum rt_led_state state);
    rt_err_t (*get_state)(struct rt_led_device *led, enum rt_led_state *out_state);
    rt_err_t (*set_period)(struct rt_led_device *led, rt_uint32_t period_ms);
    rt_err_t (*set_brightness)(struct rt_led_device *led, rt_uint32_t brightness);
};
```

Implement only the callbacks your hardware supports; missing hooks return **-RT_ENOSYS** from the matching **rt_led_*** wrapper.

1.4.2.57.1.3 Char device interface (led.c)

Registered with **RT_DEVICE_FLAG_RDWR**.

write

Input	Action
off, **on**, **toggle**, **blink**	String match → **rt_led_set_state**
Decimal digits	**rt_led_set_brightness** (e.g. **128** on PWM LED)

read

Returns ASCII state name: **off**, **on**, **toggle**, or **blink** (from **rt_led_get_state**).
Example (MSH / POSIX):

```
echo on > /dev/led0
echo 200 > /dev/led1    /* brightness, PWM LED */
cat /dev/led0
```

1.4.2.57.1.4 Software blink (led.c)

When **ops->set_period** is NULL but **ops->set_state** exists, **rt_led_register** creates a **blink_timer** in **led->sysdata**:

- **RT_LED_S_BLINK** with GPIO/syscon ops (no native blink) triggers the timer path in **rt_led_set_state**.
- Timer toggles off/on with **~80 ms** on and **~1000 ms** off phases (heartbeat-like pattern).
- **rt_led_set_period** can change the long off interval via **RT_TIMER_CTRL_SET_TIME**.

For **hardware** PWM blink, implement **set_period** in **rt_led_ops** instead.

1.4.2.57.1.5 Driver embed pattern

```
struct my_led {
    struct rt_led_device parent;
    /* hardware private */
};

static const struct rt_led_ops my_led_ops = {
    .set_state = my_led_set_state,
    .get_state = my_led_get_state,
};

static int my_led_init(void)
{
    struct my_led *ml = ...;
    ml->parent.ops = &my_led_ops;
    return rt_led_register(&ml->parent);
}
```

OFW platform drivers should use **rt_led_register** in **probe** and **rt_led_unregister** in **remove** — see LED device model (DM).

1.4.2.57.1.6 Pitfalls

Issue	Mitigation
ops NULL at register	**rt_led_register** returns **-RT_EINVAL**
Unregister while blinking	**rt_led_unregister** stops timer and forces off
Brightness on GPIO LED	**set_brightness** not implemented — use **set_state** only
Concurrent access	Core uses **led->spinlock** — keep **ops** callbacks short
Same pin as GPIO driver	One owner only — LED device model (DM)

1.4.2.57.1.7 See also

- LED device model (DM) — **gpio-leds**, **pwm-leds**, **register-bit-led**
- System controller (Syscon)
- Power supply — charging LEDs
- components/drivers/include/drivers/led.h

1.4.2.58 LED device model (DM)

1.4.2.58.1 LED subsystem under RT_USING_DM

RT_USING_LED (components/drivers/led/) depends on **RT_USING_DM**. The core **led.c** registers **struct rt_led_device** as char devices **led0**, **led1**, ... via **rt_dm_ida**. In-tree OFW drivers bind Linux-style **gpio-leds**, **pwm-leds**, and **register-bit-led** nodes.
Public API and char-device **read**/**write**: LED subsystem.
Sources:

File	Role
led.c	**rt_led_register**, software blink timer, sysfs-style **read**/**write**
led-gpio.c	**gpio-leds** parent + per-child GPIO LED
led-pwm.c	**pwm-leds** + **pwms** phandle, brightness
led-syscon.c	**register-bit-led** via System controller (Syscon)

1.4.2.58.1.1 Kconfig

Option	Role
RT_USING_DM	Required parent
RT_USING_LED	Core **led.c**
RT_LED_GPIO	**led-gpio.c** — needs **RT_USING_PINCTRL**, **RT_USING_OFW**
RT_LED_PWM	**led-pwm.c** — needs **RT_USING_PWM**, **RT_USING_OFW**
RT_LED_SYSCON	**led-syscon.c** — needs **RT_MFD_SYSCON**
SOC_DM_LED_DIR	BSP adds more LED drivers via **osource**

No `rt_bus`: each LED is a **`RT_PLATFORM_DRIVER_EXPORT`** that calls **`rt_led_register`** from **`probe`**.

1.4.2.58.1.2 Architecture

```
DT: gpio-leds / pwm-leds / register-bit-led
      |
      |  platform match (rt_ofw_node_id)
      v
probe: per-child or single node
      |
      |  parse gpios / pwms / syscon offset+mask
      |  fill struct rt_led_ops
      |  rt_led_register(&led->parent)  → "ledN"
      v
rt_device (RT_Device_Class_Char, RDWR)
      |
      +-- rt_led_set_state / set_brightness / set_period
      +-- optional software blink (led.c timer)
      +-- write "on"|"off"|digits  /  read state string
```

Backend	Compatible	One **rt_led_device per...
GPIO	**`gpio-leds`**	**Child subnode** (like Linux)
PWM	**`pwm-leds`**	**Child subnode**
Syscon	**`register-bit-led`**	**Platform node** (one LED per compatible node)

1.4.2.58.1.3 Core registration (`led.c`)

```
rt_err_t rt_led_register(struct rt_led_device *led);
rt_err_t rt_led_unregister(struct rt_led_device *led);
```

Requirement	Detail
`led->ops`	Must be non-NULL at register
Naming	**`rt_dm_dev_set_name(..., "ledu", id)`**
Blink fallback	If **`set_state`** exists but **`set_period`** is NULL, **`led.c`** allocates **`blink_timer`** in **`sysdata`** for **`RT_LED_S_BLINK`**
Unregister	Forces **`RT_LED_S_OFF`**, detaches blink timer

1.4.2.58.1.4 GPIO LEDs (`led-gpio.c`)

Parent node:

```
/ {
    leds {
        compatible = "gpio-leds";
        pinctrl-0 = <&led_pins>;   /* optional: apply once for all children */

        status {
            gpios = <&gpio0 10 GPIO_ACTIVE_HIGH>;
            label = "status";
            default-state = "off";
            linux,default-trigger = "heartbeat";
        };
    };
};
```

Property	Role
`gpios`	**`rt_ofw_get_named_pin`** — **`active_val`** from DT flags
`default-state`	**`"on"`** → **`RT_LED_S_ON`** at probe; else off

Property	Role
linux,default-trigger (fuzzy **default-trigger$**)	**heartbeat** or **timer** → **RT_LED_S_BLINK** (software blink in **led.c**)
pinctrl-0 on parent	Applied once for all children; else per-child **rt_pin_ctrl_confs_apply_by_name**

Ops: **set_state** (off/on/toggle), **get_state** (reads pin level). **RT_LED_S_BLINK** not implemented in GPIO ops — handled by core timer when **set_state** returns **-RT_ENOSYS**.

1.4.2.58.1.5 PWM LEDs (led-pwm.c)

```
pwm-leds {
    compatible = "pwm-leds";

    backlight {
        pwms = <&pwm0 0 1000000>;   /* channel, period (ns) */
        max-brightness = <255>;
        active-low;
        default-state = "on";
    };
};
```

Property	Role
pwms	Phandle + **#pwm-cells** — args[0] channel, args[1] **period (ns)**
max-brightness	Required; scales **rt_led_set_brightness**
active-low	Inverts duty calculation
default-state	**on**, **keep** (preserve existing PWM duty), or off
default-trigger	Same as GPIO — software blink

Ops: **set_state**, **get_state** (tracks **enabled**), **set_brightness**. Brightness maps to **rt_pwm_set** pulse width.
PWM controller must be probed (**rt_platform_ofw_request** on provider if needed); cookie **rt_ofw_data(pwm_np)** → **struct rt_device_pwm** *.

1.4.2.58.1.6 Syscon bit LEDs (led-syscon.c)

```
syscon: syscon@10000000 {
    compatible = "syscon";
    reg = <0x10000000 0x1000>;
};

status-led {
    compatible = "register-bit-led";
    offset = <0x10>;
    mask = <0x1>;
    default-state = "on";
};
```

Property	Role
offset	Byte offset into parent **syscon** register block
mask	Bit mask for **rt_syscon_update_bits**
Parent	**rt_syscon_find_by_ofw_node** on **parent OFW node**

Ops: **set_state**, **get_state** via **rt_syscon_read**. One platform device per LED node (**pdev->parent.user_data**).

1.4.2.58.1.7 New LED backend checklist

1. Enable **RT_USING_DM**, **RT_USING_LED**, and backend Kconfig.

2. Embed **struct rt_led_device parent** first in private struct.
3. Implement **struct rt_led_ops** — at minimum **set_state**; add **set_brightness** for dimming, **set_period** for hardware blink.
4. **rt_led_register** after hardware/pinctrl/PWM is ready.
5. Apply **default-state** / **default-trigger** like in-tree drivers.
6. Store **rt_ofw_data(child_np) = &led->parent** (GPIO/PWM) or **user_data** (syscon) for **remove**.
7. **remove**: **rt_led_unregister**, free private data.
8. **RT_PLATFORM_DRIVER_EXPORT**.

1.4.2.58.1.8 Pitfalls

Issue	Mitigation
Double GPIO ownership	Do not export the same pin as **RT_USING_PIN** device and LED
PWM not probed	Ensure PWM controller **compatible** probes before **pwm-leds** children
Missing max-brightness	PWM child probe fails with **-RT_EINVAL**
Syscon parent	**register-bit-led** must sit under / reference probed **syscon**
Blink on GPIO	Uses **software** timer in **led.c** — not hardware PWM blink
Char write brightness	Only works when **set_brightness** op exists (PWM path)

1.4.2.58.1.9 See also

- Input subsystem
- System controller (Syscon)
- PWM Device
- Pin control (pinctrl)
- components/drivers/include/drivers/led.h
- components/drivers/led/Kconfig

1.4.2.59 Mailbox subsystem

1.4.2.59.1 Mailbox subsystem (API)

Header: **components/drivers/include/drivers/mailbox.h**. Core: **components/drivers/mailbox/mailbox.c**.
The mailbox layer is a **lock-aware wrapper** around hardware **doorbell** / **message-slot** IPs: one side **sends** a small notification, the peer is woken by **IRQ** and handles **rx_callback**. Use for **AMP IPC**, **SCMI** doorbells, **remoteproc** handshakes, and **PIC**-backed mailboxes—not bulk DMA.
Device-tree binding and **rt-thread,pic-mailbox**: Mailbox device model (DM).

1.4.2.59.1.1 When to use this API

Use mailbox when...	Prefer something else when...
You need a **hardware channel** with **IRQ to peer** (SoC mailbox IP, PIC mailbox).	**Thread-to-thread** only: semaphore, message queue, completion.
Firmware expects **doorbell + shared memory** (SCMI, AMP).	**Large payloads** — put data in SRAM, pass a **pointer** or single word.
DT exposes **mboxes** / **#mbox-cells** / **mbox-names**.	No controller: implement **rt_mbox_controller_ops** and register first.
Named **RPMsg** endpoints and bus matching are required.	RPMsg

1.4.2.59.1.2 Roles

Role	Responsibility
Controller driver	**rt_mbox_controller_register**: **num_chans**, **ops**. Implements **send**, **release**, optional **peek**, **request**, **ofw_parse**. Calls **rt_mbox_send_done** when HW TX completes (if applicable).
Client driver	**struct rt_mbox_client** on consumer **rt_device**: **tx_prepare**, **tx_done**, **rx_callback**. **rt_mbox_request_by_index** or **by_name**.

1.4.2.59.1.3 Data structures

```
struct rt_mbox_controller {
    struct rt_device *dev;
    const struct rt_mbox_controller_ops *ops;
    rt_size_t num_chans;
    struct rt_mbox_chan *chans;
};

struct rt_mbox_controller_ops {
    rt_err_t (*request)(struct rt_mbox_chan *);
    void (*release)(struct rt_mbox_chan *);
    rt_err_t (*send)(struct rt_mbox_chan *, const void *data);
    rt_bool_t (*peek)(struct rt_mbox_chan *);
    int (*ofw_parse)(struct rt_mbox_controller *, struct rt_ofw_cell_args *);
};

struct rt_mbox_client {
    struct rt_device *dev;
    void (*rx_callback)(struct rt_mbox_client *, void *data);
    void (*tx_prepare)(struct rt_mbox_client *, const void *data);
    void (*tx_done)(struct rt_mbox_client *, const void *data, rt_err_t err);
};
```

1.4.2.59.1.4 Requesting a channel

```
struct rt_mbox_chan *rt_mbox_request_by_index(struct rt_mbox_client *client, int index);
struct rt_mbox_chan *rt_mbox_request_by_name(struct rt_mbox_client *client, char *name);
rt_err_t rt_mbox_release(struct rt_mbox_chan *chan);
```

API	Returns
request_by_index	Channel pointer or **error pointer** (rt_is_err_ptr)
request_by_name	Channel, **error pointer**, or **NULL** if name not in **mbox-names**

Prerequisites: valid **client->dev->ofw_node**; provider probed (**rt_ofw_data** = controller). Optional **ops->request** runs after channel selection; on failure core **release**s and returns error pointer.

1.4.2.59.1.5 Sending: rt_mbox_send

```
rt_err_t rt_mbox_send(struct rt_mbox_chan *chan, const void *data, rt_uint32_t timeout_ms);
void rt_mbox_send_done(struct rt_mbox_chan *chan, rt_err_t err);
```

Flow (under **chan->lock**):

1. Optional **client->tx_prepare(client, data)**
2. **ctrl->ops->send(chan, data)** — return value is **only** the ops result
3. On success: store **chan->data**, clear **complete**; if **timeout_ms != RT_WAITING_FOREVER**, start one-shot timer
4. On send failure: **complete = RT_TRUE** immediately

RT_EOK from rt_mbox_send means the controller **accepted** the submission, not that **tx_done** ran.
rt_mbox_send_done: clears **chan->data**, calls **tx_done**, sets **complete = RT_TRUE**. Controller must invoke after HW TX finish (or on error). Timer calls it with **-RT_ETIMEOUT** if still incomplete.
Implications:

- Do not free **data** (or pointed buffer) until **tx_done** or synchronous send failure
- **tx_prepare** runs **with channel spinlock** — non-blocking only
- **RT_WAITING_FOREVER**: no timer; requires **send_done** from controller for **tx_done** (see Mailbox device model (DM) for PIC caveat)

1.4.2.59.1.6 Receiving: rt_mbox_recv

```
rt_err_t rt_mbox_recv(struct rt_mbox_chan *chan, void *data);
```

Does **not** read hardware. Invokes **client->rx_callback(client, data)** if set. **Controller ISR** reads registers/FIFO, then passes payload pointer (e.g. local **uint32_t**) into **rt_mbox_recv**.

1.4.2.59.1.7 rt_mbox_peek

```
rt_bool_t rt_mbox_peek(struct rt_mbox_chan *chan);
```

Forwards to **ops->peek** if present; else **RT_FALSE**.

1.4.2.59.1.8 Controller unregister

rt_mbox_controller_unregister: removes from global list, **rt_dm_dev_unbind_fwdata**, **rt_mbox_release** on every channel (reverse order), frees **chans**. **release** must be safe even if **request** never ran for a channel.

1.4.2.59.1.9 Example (client sketch)

```
static void my_rx(struct rt_mbox_client *cl, void *data)
{
    rt_uint32_t msg = *(rt_uint32_t *)data;
    /* handle doorbell */
}

static void my_tx_done(struct rt_mbox_client *cl, const void *data, rt_err_t err)
{
    /* release buffer */
}

static struct rt_mbox_client mbox_cl = {
    .dev = &my_pdev->parent,
    .rx_callback = my_rx,
    .tx_done = my_tx_done,
};

static int my_probe(struct rt_platform_device *pdev)
{
    struct rt_mbox_chan *chan;

    mbox_cl.dev = &pdev->parent;
    chan = rt_mbox_request_by_name(&mbox_cl, "tx");
    if (!chan || rt_is_err_ptr(chan))
```

```
        return -RT_ERROR;

    rt_uint32_t doorbell = MY_MSG_ID;
    return rt_mbox_send(chan, &doorbell, 100);  /* 100 ms tx timeout */
}
```

1.4.2.59.1.10 Pitfalls

Issue	Mitigation
No `send_done` in controller	Use finite timeout or fire-and-forget (Mailbox device model (DM) — PIC)
`release` NULL	Always implement **`ops->release`**
Blocking in `tx_prepare`	Deadlock risk under spinlock
Confusing return types	Distinguish **`NULL`** (name) vs **`rt_err_ptr`** (index)

1.4.2.59.1.11 See also

- Mailbox device model (DM)
- RPMsg
- SCMI firmware protocol
- PIC overview
- `components/drivers/mailbox/mailbox.c`
- `components/drivers/mailbox/mailbox-pic.c`

1.4.2.60 Mailbox device model (DM)

1.4.2.60.1 Mailbox under `RT_USING_DM`

`RT_USING_MBOX` builds **`mailbox.c`** (controller registry, OFW **`mboxes`** resolution, **`rt_mbox_send`** / **`recv`**) and optionally **`mailbox-pic.c`** (**`rt-thread,pic-mailbox`**). The layer is a **Linux-style mailbox framework**: controllers expose channels; clients request by DT phandle and exchange **small payloads** (often one **`uint32_t`** or a pointer to shared memory).
Client/controller API semantics: Mailbox subsystem.
Sources: **`components/drivers/mailbox/mailbox.c`**, **`mailbox-pic.c`**, **`components/drivers/include/drivers/mailbox.h`**.

1.4.2.60.1.1 Kconfig

Option	Role
`RT_USING_DM`	Required
`RT_USING_OFW`	Required — **`mboxes`**, **`#mbox-cells`**, **`mbox-names`**
`RT_USING_MBOX`	Core **`mailbox.c`**
`RT_MBOX_PIC`	In-tree **`rt-thread,pic-mailbox`** (default **y** when MBOX on)
`SOC_DM_MBOX_DIR`	BSP adds SoC mailbox controllers

There is **no separate mailbox `rt_bus`** — providers register **`struct rt_mbox_controller`** and bind **`rt_ofw_data(provider_np) = ctrl`**.

1.4.2.60.1.2 Architecture

```
Mailbox controller driver (platform)
      |
      |  rt_mbox_controller_register(ctrl)
      |  rt_dm_dev_bind_fwdata(dev, NULL, ctrl)
      v
Provider OFW node  (#mbox-cells, rt_ofw_data = ctrl)
      |
      |  Consumer: mboxes = <&mbox 0>; mbox-names = "tx";
      v
rt_mbox_request_by_index / by_name(client)
      |
      |  ofw_parse → channel index
      |  ops->request(chan)  [optional]
      |  chan->client = client
      v
Client: rt_mbox_send / rt_mbox_recv path
      |
      +-- tx_prepare → ops->send → [timer] → rt_mbox_send_done → tx_done
      +-- ISR: read HW → rt_mbox_recv → rx_callback
```

Role	Object	Responsibility
Controller	**struct rt_mbox_controller**	**num_chans**, **ops**, per-chan timer (created by core)
Channel	**struct rt_mbox_chan**	Binds **client**, **priv**, send state
Client	**struct rt_mbox_client**	Lives on consumer **rt_device**; **rx_callback**, **tx_***

1.4.2.60.1.3 Controller registration

```
rt_err_t rt_mbox_controller_register(struct rt_mbox_controller *ctrl);
rt_err_t rt_mbox_controller_unregister(struct rt_mbox_controller *ctrl);
```

Check	Detail
ctrl->dev, **ops**, **num_chans** > 0	Required or **-RT_EINVAL**
ops->send, **ops->release**	**De facto mandatory** — unregister always calls **release** per channel
ops->request, **ops->peek**, **ops->ofw_parse**	Optional
Per-channel timer	One-shot; used when **rt_mbox_send(..., timeout_ms)** is not **RT_WAITING_FOREVER**

ofw_parse: custom channel index from **mboxes** cells; default parser expects **one cell** = channel index.

1.4.2.60.1.4 Consumer device tree

```
scmi: scmi {
    compatible = "arm,scmi";
    mboxes = <&pic_mbox 0>;
    mbox-names = "tx";
};

remote: remoteproc@0 {
    mboxes = <&vendor_mbox 2>, <&vendor_mbox 3>;
    mbox-names = "tx", "rx";
};
```

Property	Role
mboxes	Phandle list + **#mbox-cells** per controller
mbox-names	Optional; used with **rt_mbox_request_by_name**
#mbox-cells	On **provider** node (default **1** = channel id)

rt_mbox_request_by_index calls **rt_platform_ofw_request** on the provider if **rt_ofw_data** is empty. Returns **struct rt_mbox_chan** ∗ or **error pointer** — test with **rt_is_err_ptr**.
rt_mbox_request_by_name: returns **NULL** if the name is missing in **mbox-names** (distinct from error pointers).

1.4.2.60.1.5 In-tree: PIC mailbox (mailbox-pic.c)

Compatible: **rt-thread,pic-mailbox**

Register map (per half)

Offset	Name	Role
0x00	MAILBOX_IMASK	Channel interrupt mask
0x04	MAILBOX_ISTATE	Pending / doorbell state
0x08 + n∗4	MAILBOX_MSG(n)	32-bit message slot per channel

reg spans **local + peer** windows (each **size/2**). **position =** <**0**> (captain): iomap low half, peer = **regs + size/2**, initializes both **IMASK**/**ISTATE**. **position =** <**1**>: peer low, local high.

Property	Role
interrupts	Local mailbox IRQ
peer-interrupts	HW IRQ number on peer PIC to poke remote CPU
uid	Device name suffix **pic-mbox{N}**
#mbox-cells = <1>	Channel index **0..31** (**num_chans = 32**)

Send / receive behavior

- **send**: waits until peer **ISTATE** bit clear; writes ∗**(uint32_t** ∗**)data** to **MAILBOX_MSG(index)**; sets peer **ISTATE** bit; triggers **rt_pic_irq_set_state_raw** on **peer_hwirq**.
- **ISR**: reads pending **ISTATE**, loads **MAILBOX_MSG** from **peer** window, **rt_mbox_recv(chan, &msg)** per bit.
- **Does not call rt_mbox_send_done** — TX completion is **not** hardware-ACKed; use a **finite timeout_ms** in **rt_mbox_send** if **tx_done** matters, or treat send as fire-and-forget.

1.4.2.60.1.6 New controller driver checklist

1. Enable **RT_USING_MBOX** (+ BSP Kconfig under **SOC_DM_MBOX_DIR**).
2. Allocate **struct rt_mbox_controller** + private state; set **dev**, **num_chans**, **ops**.
3. Implement **send** (required) and **release** (required).
4. Optional **request**: unmask channel / enable clock.
5. Optional **peek**: non-destructive pending check.
6. Optional **ofw_parse**: if **#mbox-cells** is not a single channel index.
7. **rt_mbox_controller_register** after HW init; store pointer in probe (**user_data** or **rt_ofw_data**).
8. RX ISR: read hardware, then **rt_mbox_recv(chan, payload_ptr)**.
9. If hardware has TX complete IRQ: call **rt_mbox_send_done(chan, err)**.
10. **remove**: detach IRQ, **rt_mbox_controller_unregister**.

1.4.2.60.1.7 Pitfalls

Issue	Mitigation
Missing `release`	Unregister crashes — always provide **`ops->release`**
PIC + `tx_done`	PIC path never calls **`send_done`** — use timeout or do not rely on **`tx_done`**
`data` lifetime	Keep buffer valid until **`tx_done`** or synchronous send failure
`tx_prepare` under spinlock	No blocking calls
Error pointer vs NULL	**`by_index`** → err ptr; missing **`by_name`** → **`NULL`**
32-bit payload on PIC	**`send`** dereferences **`uint32_t`** — pass pointer to at least 4 bytes

---

1.4.2.60.1.8 See also

- Mailbox subsystem
- RPMsg — often layered above mailbox + shared memory
- SCMI firmware protocol — typical doorbell consumer
- PIC overview
- Open Firmware (OFW)
- `components/drivers/include/drivers/mailbox.h`

1.4.2.61 NUMA helpers

1.4.2.61.1 NUMA (non-uniform memory access)

`components/drivers/core/numa.c` provides lightweight helpers to map **CPUs**, **devices**, and **physical addresses** to NUMA nodes on multi-socket / multi-die platforms. The code is **not** a full NUMA allocator—it parses device-tree tables when enabled and exposes three query APIs in **`drivers/core/numa.h`**.
Built with **`RT_USING_DM`** (always linked from **`core/SConscript`**). Runtime tables are populated only when **`RT_USING_OFW`** and boot **`numa=on`** are active.

1.4.2.61.1.1 Enabling NUMA

Requirement	Detail
`RT_USING_DM`	**`numa.c`** in core drivers
`RT_USING_OFW`	**`INIT_CORE_EXPORT(numa_ofw_init)`** parses DT
Boot argument	**`numa=on`** in **`chosen/bootargs`** (exact string after **`numa=`**)

If **`numa=`** is missing or not **`on`**, **`numa_enabled`** stays false and APIs behave as **single-node fallback** (see below).

1.4.2.61.1.2 Initialization (`numa_ofw_init`)

```
bootargs: numa=on
      |
      v
For each CPU node: read numa-node-id → cpu_numa_map[cpuid]
      |
      v
For each memory@ node with numa-node-id:
      record [start, end) per reg entry on numa_memory_nodes list
```

DT source	Stored as
CPU nodes	**cpu_numa_map[i]** = **numa-node-id** (**i** = enumeration order, capped at **RT_CPUS_NR**)
device_type = "memory" + **numa-node-id**	Linked **struct numa_memory** ranges (**start**/**end** physical)

There is **no** hot-plug update—tables are fixed at core init.

1.4.2.61.1.3 API

```
int rt_numa_cpu_id(int cpuid);
int rt_numa_device_id(struct rt_device *dev);
rt_err_t rt_numa_memory_affinity(rt_uint64_t phy_addr, rt_bitmap_t *out_affinity);
```

rt_numa_cpu_id

Condition	Return
NUMA disabled	**-RT_ENOSYS**
cpuid >= RT_CPUS_NR	**-RT_EINVAL**
Otherwise	**cpu_numa_map[cpuid]** (node id from DT, or **-RT_ENOSYS** if never set)

rt_numa_device_id

Condition	Return
NUMA disabled	**(int)(uint32_t)-RT_ENOSYS** (legacy cast in source)
numa-node-id property present on **dev->ofw_node**	Parsed **u32** node id
Property missing / read error	Prop-read error code as **int**

Use on **rt_device** objects that carry an OFW node (PCI, platform, etc.).

rt_numa_memory_affinity

Given a **physical address**, fills **out_affinity** (**RT_BITMAP_LEN(RT_CPUS_NR)** words) with CPUs whose **rt_numa_cpu_id** matches the memory region's node.

Condition	Return
out_affinity == NULL	**-RT_EINVAL**
NUMA disabled	Sets **CPU 0** only, **RT_EOK**
Address inside a recorded range	CPUs on same node set in bitmap, **RT_EOK**
No matching range	**-RT_EEMPTY**

Typical use: pick an IRQ affinity mask or thread CPU set for DMA buffers allocated at **phy_addr**.

1.4.2.61.1.4 Device tree examples

Bootargs

```
/ {
    chosen {
```

```
        bootargs = "numa=on";
    };
};
```

CPUs and memory

```
cpu@0 {
    device_type = "cpu";
    reg = <0>;
    numa-node-id = <0>;
};

cpu@100 {
    device_type = "cpu";
    reg = <0x100>;
    numa-node-id = <1>;
};

memory@0 {
    device_type = "memory";
    reg = <0x0 0x80000000>;
    numa-node-id = <0>;
};

memory@80000000 {
    device_type = "memory";
    reg = <0x80000000 0x80000000>;
    numa-node-id = <1>;
};
```

Device node

```
nvme@1,0 {
    /* ... */
    numa-node-id = <0>;
};
```

1.4.2.61.1.5 Integration in-tree

Consumer	Usage
NVMe	**`INIT_SECONDARY_CPU_EXPORT(nvme_queue_affinify_fixup)`** sets per-queue IRQ affinity (NVMe device model (DM))
Future DMA / drivers	Call **`rt_numa_memory_affinity`** when pinning buffers or IRQs

No other in-tree callers are required for NUMA to be useful—BSPs can query node ids when placing shared memory for AMP.

1.4.2.61.1.6 When to use

Use **`rt_numa_*`** when...	Skip when...
BSP documents multiple nodes and **`numa=on`**.	Single RAM bank; **`rt_numa_cpu_id`** always maps to one node.
You pin **IRQ affinity** or **thread CPU** near PCIe/DMA memory.	All devices are local and symmetric.
Cross-node latency matters for storage/net.	Firmware hides NUMA (uniform UMA).

Treat node ids as ∗∗hints∗∗—hardware may still allow remote access.

1.4.2.61.1.7 Pitfalls

Issue	Mitigation
Forgot `numa=on`	All APIs behave as UMA (memory affinity → CPU 0 only).
CPU map vs logical id	**`cpu_numa_map`** indexed by **enumeration order**, not necessarily **`rt_hw_cpu_id()`** unless DT order matches—verify BSP.
`rt_numa_device_id` without property	Returns error from prop read—not **0**.
Address not in table	**`rt_numa_memory_affinity`** → **`-RT_EEMPTY`** — extend DT memory nodes or fall back.
No hotplug	Memory map changes are not tracked after boot.

1.4.2.61.1.8 See also

- NVMe device model (DM) — IRQ affinity fixup
- DMA engine
- Open Firmware (OFW)
- `components/drivers/include/drivers/core/numa.h`
- `components/drivers/core/numa.c`

1.4.2.62 NVMe subsystem

1.4.2.62.1 NVMe controller API

Header: **`components/drivers/include/drivers/nvme.h`**. Core: **`components/drivers/nvme/nvme.c`**.
RT-Thread implements an **NVMe host** stack: admin + I/O queues, Identify, namespace scan, and **`rt_hw_blk_disk_register`** per namespace. PCI binding: NVMe device model (DM).

1.4.2.62.1.1 Kconfig (summary)

Option	Role
`RT_USING_NVME`	Core **`nvme.c`** — needs **`RT_USING_DM`**, **`RT_USING_BLK`**, **`RT_USING_DMA`**
`RT_USING_NVME_IO_QUEUE`	I/O queue count (default scales with **`RT_THREAD_PRIORITY_`**∗)
`RT_NVME_PCI`	**`nvme-pci.c`** — needs **`RT_USING_PCI`**

1.4.2.62.1.2 Architecture

```
Transport (e.g. PCI): map BAR, setup IRQs/MSI-X
      |
      |  fill struct rt_nvme_controller + ops
      v
rt_nvme_controller_register(nvme)
      |
      |  CAP/MQES, admin queue, IO queues (DMA)
      |  Identify Controller, scan namespaces
      v
Per-NS: rt_hw_blk_disk_register → block device "nvme0n1" ...
      |
      v
blk layer: read/write/flush via nvme_submit_io_cmd (per-CPU queue)
```

1.4.2.62.1.3 `struct rt_nvme_controller` (caller fills before register)

Member	Owner	Meaning
dev	**Caller**	Used for **rt_dma_alloc**/**free** on queue buffers
regs	**Caller**	MMIO base (**CAP**, **CC**, doorbells)
ops	**Caller**	**Required**; at least **name** for logging
irqs_nr, **irqs[]**	**Caller**	One entry per queue (MSI-X preferred); see PCI glue
nvme_id, **name**	**Core**	**rt_dm_ida**, **nvmeu**
cap, **queue_depth**, **doorbell_***	**Core**	From **CAP** register
max_transfer_shift, **write_zeroes**, **sgl_mode**	**Core**	From Identify Controller
queue[], **admin_queue**, **io_queues[]**	**Core**	SQ/CQ DMA rings + completions

Do not pre-fill queue state—the core zeroes and allocates DMA in **nvme_alloc_queue**.

1.4.2.62.1.4 `struct rt_nvme_ops`

```
struct rt_nvme_ops {
    const char *name;
    rt_err_t (*setup_queue)(struct rt_nvme_queue *queue);
    rt_err_t (*cleanup_queue)(struct rt_nvme_queue *queue);
    rt_err_t (*submit_cmd)(struct rt_nvme_queue *queue, struct rt_nvme_command *cmd);
    void (*complete_cmd)(struct rt_nvme_queue *queue, struct rt_nvme_command *cmd);
};
```

Callback	When	Role
name	Logging	e.g. **"PCI"**
setup_queue	After SQ/CQ DMA alloc in **nvme_alloc_queue**	SoC-specific queue programming
cleanup_queue	**nvme_free_queue**	Undo setup
submit_cmd	After command copied to SQ slot, before doorbell	Custom doorbell / FIFO
complete_cmd	CQ ISR after phase check	Extra HW ack (shared IRQ, vendor IP)

Return polarity (important): in current **nvme.c**, **setup_queue** / **cleanup_queue** use **if (!(err = ops->...))** — a **non-zero** return is treated as **success**, **zero** as failure. Match this when porting quirks, or align **nvme.c** with upstream when fixing.
submit_cmd: uses normal **RT_EOK** — non-zero aborts submit.

1.4.2.62.1.5 Registration API

```
rt_err_t rt_nvme_controller_register(struct rt_nvme_controller *nvme);
rt_err_t rt_nvme_controller_unregister(struct rt_nvme_controller *nvme);
```

`rt_nvme_controller_register` sequence

1. Validate **nvme**, **ops**; check **CSTS** not **0xffffffff**
2. **rt_dm_ida_alloc** → **nvmeu**
3. Read **CAP**, configure **admin queue** (disable → alloc → enable)
4. Create **I/O queues** (**RT_USING_NVME_IO_QUEUE** * **RT_CPUS_NR** max)
5. **Identify Controller** — MDTS, VWC, ONCS, SGL support

6. **nvme_scan_device** — for each namespace id: Identify NS, **rt_hw_blk_disk_register**
7. Link on global **nvme_nodes** list

rt_nvme_controller_unregister

Removes namespaces, IO/admin queues, frees IDA, **shutdown + disable** controller.

1.4.2.62.1.6 Namespaces (struct rt_nvme_device)

Field	Role
Embeds **struct rt_blk_disk parent**	Block layer entry
nsid	Namespace ID (1-based in scan loop)
lba_shift	From active LBA format
id	Copy of **struct rt_nvme_id_ns**

Disk name pattern: **nvme{N}n{nsid}** (e.g. **nvme0n1**).

1.4.2.62.1.7 Command path (I/O)

- **nvme_submit_io_cmd**: picks I/O queue **rt_atomic_add(&ioqid[cpu], RT_CPUS_NR) % io_queue_max** (per-CPU striping).
- **nvme_submit_cmd**: copies **struct rt_nvme_command** to SQ tail, rings doorbell (unless **ops->submit_cmd**), **rt_completion_wait** for CQ.
- Data path: **PRP** by default; **SGL** if Identify reports SGL support.

Admin commands use **admin_queue** with a **60 s** completion timeout; I/O queues use **RT_WAITING_FOREVER**.

1.4.2.62.1.8 Pitfalls

Issue	Mitigation
setup_queue return code	Inverted vs **RT_EOK** — see ops table above
Page size	Host **ARCH_PAGE_SHIFT** must be ≥ device **MPSMIN** or register fails
IRQ count	**irqs_nr** should cover admin + IO queues used; MSI-X preferred (NVMe device model (DM))
Unregister order	Mask IRQs in transport **remove** before **rt_nvme_controller_unregister**
Admin queue blocking	Do not hold locks that block admin path from I/O context

1.4.2.62.1.9 See also

- NVMe device model (DM)
- PCI / PCIe
- NUMA helpers
- DMA engine
- https://nvmexpress.org/developers/
- components/drivers/nvme/nvme.c
- components/drivers/nvme/nvme-pci.c

1.4.2.63 NVMe device model (DM)

1.4.2.63.1 NVMe under `RT_USING_DM`

`RT_USING_NVME` builds the protocol core (**`nvme.c`**) and optionally **`nvme-pci.c`** (**`RT_NVME_PCI`**). Controllers register with **`rt_nvme_controller_register`**; each namespace becomes a **`rt_blk_disk`** for the block layer.
Controller API and queue ops: NVMe subsystem.
Sources: **`components/drivers/nvme/nvme.c`**, **`nvme-pci.c`**, **`components/drivers/include/drivers/nvme.h`**.

1.4.2.63.1.1 Kconfig / build

Option	Depends on	Role
`RT_USING_NVME`	**`RT_USING_DM`**, **`RT_USING_BLK`**, **`RT_USING_DMA`**	Core stack
`RT_USING_NVME_IO_QUEUE`	**`RT_USING_NVME`**	Number of I/O queues (2/4/8 by priority profile)
`RT_NVME_PCI`	**`RT_USING_NVME`**, **`RT_USING_PCI`**	PCI function driver (default **y**)
`SOC_DM_NVME_DIR`	BSP	Extra non-PCI controllers

`RT_USING_NVME_QUEUE` in **`nvme.h`** is **`1 + RT_USING_NVME_IO_QUEUE * RT_CPUS_NR`** (admin + per-CPU IO slots).

1.4.2.63.1.2 PCI transport (`nvme-pci.c`)

Matching

```
static const struct rt_pci_device_id pci_nvme_ids[] = {
    { RT_PCI_DEVICE_ID(PCI_VENDOR_ID_REDHAT, 0x0010) },
    { RT_PCI_DEVICE_CLASS(PCIS_STORAGE_EXPRESS, ~0) },
};
```

Any **NVMe class** PCI function binds **`nvme-pci`** driver.

Probe flow

```
pci_nvme_probe(pdev)
      |
      |  rt_pci_iomap(pdev, 0) → nvme->regs
      |  ops = quirk->ops ?: pci_nvme_std_ops ("PCI")
      |  MSI-X: rt_pci_msix_enable → nvme->irqs[i]
      |  else: single pdev->irq, rt_pci_irq_unmask
      |  rt_pci_set_master(pdev)
      v
rt_nvme_controller_register(nvme)
      |
      v
pdev->parent.user_data = pci_nvme
```

Remove / shutdown

1. **`rt_nvme_controller_unregister`**
2. MSI-X disable or INTx mask
3. **`rt_pci_clear_master`**, **`rt_iounmap(regs)`**, free **`pci_nvme_controller`**

Quirks

`pdev->id->data` may point to **`struct pci_nvme_quirk`** with alternate **`rt_nvme_ops`** (vendor MMIO). Default uses standard doorbell path in **`nvme.c`**.

1.4.2.63.1.3 IRQ and NUMA affinity

Mode	Behavior
MSI-X	Up to **RT_USING_NVME_QUEUE** vectors; linear index mapping
INTx	**irqs_nr = 1**, shared line — rely on **complete_cmd** to disambiguate if needed

INIT_SECONDARY_CPU_EXPORT(nvme_queue_affinify_fixup) (in **nvme.c**):

- Runs when secondary CPUs start
- For each controller I/O queue index **i cpuid (mod RT_CPUS_NR)**, sets IRQ affinity to that CPU via **rt_pic_irq_set_affinity**
- Pairs with NUMA helpers when **numa=on** — keep queue IRQ near memory node

Assign **nvme->irqs[]** before **rt_nvme_controller_register** — core wires ISR per queue during setup.

1.4.2.63.1.4 DMA

Queue rings allocated with **rt_dma_alloc(nvme->dev, ...)** and **nvme_queue_dma_flags()** (coherent / device attributes per **dev**).
Identify buffers use **rt_malloc_align(..., nvme->page_size)** after page size is negotiated from **CAP.MPSMIN/MAX**.

1.4.2.63.1.5 Block device exposure

After Identify NS:

```
ndev->parent.ops = &nvme_blk_ops;
ndev->parent.parallel_io = RT_TRUE;
rt_dm_dev_set_name(&ndev->parent.parent, "%sn%u", nvme->name, nsid);
rt_hw_blk_disk_register(&ndev->parent);
```

Partitions use standard **RT_BLK_PARTITION_MAX** — see disk/partitions docs if enabled in BSP.

1.4.2.63.1.6 Non-PCI controllers

1. Map **regs**, implement **struct rt_nvme_ops** (**setup_queue**/**submit_cmd**/**complete_cmd** as needed).
2. Fill **irqs[]** for admin + IO queues.
3. Set **nvme->dev** to the **rt_device** used for DMA mapping.
4. **rt_nvme_controller_register**.
5. Add Kconfig under **SOC_DM_NVME_DIR** and platform/PCIe-alt probe.

1.4.2.63.1.7 Checklist (PCI BSP)

1. Enable **RT_USING_DM**, **RT_USING_PCI**, **RT_USING_BLK**, **RT_USING_DMA**, **RT_USING_NVME**, **RT_NVME_PCI**.
2. ECAM/host bridge probes before NVMe function.
3. Ensure **rt_pci_set_master** path runs (done in **pci_nvme_probe**).
4. Prefer **MSI-X** in firmware/DT if available.
5. For multi-node servers: **numa=on** + CPU/memory **numa-node-id** in DT (NUMA helpers).
6. Verify namespace appears as **nvme0n1** (etc.) and partition mount if used.

1.4.2.63.1.8 Pitfalls

Issue	Mitigation
BAR not mapped	**`rt_pci_iomap`** failure → probe abort
MSI-X vector shortage	Falls back to INTx — may need **`complete_cmd`**
`CSTS == -1`	Usually bad BAR/endianness — register rejects early
Inverted `setup_queue` success	NVMe subsystem — match **`nvme.c`** polarity
Secondary CPU IRQ affinity	Ensure **`nvme_queue_affinify_fixup`** runs (SMP BSP)

1.4.2.63.1.9 See also

- NVMe subsystem
- PCI / PCIe
- NUMA helpers
- DMA engine
- `components/drivers/nvme/Kconfig`

1.4.2.64 NVMEM subsystem

1.4.2.64.1 Non-volatile memory (NVMEM)

Header: **`components/drivers/include/drivers/nvmem.h`**. Core: **`components/drivers/nvmem/nvmem.c`**.
NVMEM models **OTP, eFuse, EEPROM slices, PMIC trim, MAC/calibration blobs** as a **provider device** plus **cells** (named or indexed byte/bit ranges). Consumers resolve cells from device tree phandles—see NVMEM device model (DM).
There is **no in-tree OTP driver** under **`components/drivers/nvmem/`**; SoC providers live under **`SOC_DM_NVMEM_DIR`** in the BSP.

1.4.2.64.1.1 When to use NVMEM

Use `rt_nvmem_*` when...	Use a dedicated driver when...
Multiple consumers need slices of one OTP/eFuse controller.	Only one driver reads a fixed offset once at boot.
DT uses **`nvmem-cells`** / **`nvmem-cell-names`** (Linux-style).	MAC/calibration is hard-coded in BSP C sources.
You want refcounted **`rt_nvmem_cell`** handles.	Direct MMIO in a single platform driver is enough.

1.4.2.64.1.2 Data structures

```
struct rt_nvmem_device {
    struct rt_device parent;
    int cells_nr;
    rt_list_t cell_nodes;
    rt_ssize_t (*reg_read)(struct rt_nvmem_device *, int offset, void *val, rt_size_t bytes);
    rt_ssize_t (*reg_write)(struct rt_nvmem_device *, int offset, void *val, rt_size_t bytes);
    rt_ssize_t size;
    int word_size;
    int stride;
    rt_bool_t read_only;
    rt_bool_t ignore_wp;
    rt_base_t wp_pin;
    rt_uint8_t wp_pin_active;
    struct rt_ref ref;
```

```
    struct rt_spinlock spinlock;
    void *priv;
};
```

Field	Role
reg_read / **reg_write**	Byte access at **offset**
read_only	DT **read-only**, no **reg_write**, or preset flag
wp_pin	Optional **wp** GPIO for EEPROM

1.4.2.64.1.3 Provider API

```
rt_err_t rt_nvmem_device_register(struct rt_nvmem_device *ndev);
rt_err_t rt_nvmem_device_unregister(struct rt_nvmem_device *ndev);
rt_err_t rt_nvmem_device_append_cell(struct rt_nvmem_device *ndev, struct rt_nvmem_cell *cell);
```

register: optional **wp** pin, **read_only** from DT / missing **reg_write**, **rt_ofw_data(np)=ndev**.
unregister: only if **rt_ref_read(&ndev->ref)==1**.
append_cell: static cells (**free_able false**) known at probe time.

1.4.2.64.1.4 Consumer API

```
struct rt_nvmem_cell *rt_nvmem_get_cell_by_index(struct rt_device *dev, int index);
struct rt_nvmem_cell *rt_nvmem_get_cell_by_name(struct rt_device *dev, const char *id);
void rt_nvmem_put_cell(struct rt_nvmem_cell *cell);
rt_ssize_t rt_nvmem_cell_read(struct rt_nvmem_cell *cell, void *buffer, rt_size_t len);
rt_ssize_t rt_nvmem_cell_write(struct rt_nvmem_cell *cell, void *buffer, rt_size_t len);
```

- **len <= cell->bytes** and **<= nvmem->size**
- Bit fields: **bit_offset**, **nbits** — read shifts/masks; write read-modify-write
- **wp_pin** deasserted during **reg_write**
- Typed helpers: **rt_nvmem_cell_read_u8/u16/u32/u64**

1.4.2.64.1.5 Example

```
cell = rt_nvmem_get_cell_by_name(&pdev->parent, "mac-address");
if (!cell || rt_is_err_ptr(cell)) return -RT_ERROR;
rt_nvmem_cell_read(cell, mac, 6);
rt_nvmem_put_cell(cell);
```

1.4.2.64.1.6 Pitfalls

Issue	Mitigation
Forgot **put_cell**	Blocks unregister
OTP write	Omit **reg_write**
No BSP provider	**SOC_DM_NVMEM_DIR**

1.4.2.64.1.7 See also

- NVMEM device model (DM)
- IIO device model (DM)
- `components/drivers/nvmem/Kconfig`

1.4.2.65 NVMEM device model (DM)

1.4.2.65.1 NVMEM under `RT_USING_DM`

`RT_USING_NVMEM` builds **`nvmem.c`** only. **Provider** drivers register **`struct rt_nvmem_device`** and set **`rt_ofw_data(provider_np)`**. **Consumers** parse **`nvmem-cells`** phandles.
API: NVMEM subsystem. Sources: **`nvmem.c`**, **`nvmem.h`**.

1.4.2.65.1.1 Kconfig

Option	Role
`RT_USING_DM`	Required
`RT_USING_OFW`	Required
`RT_USING_PIN`	Optional **`wp`** GPIO
`RT_USING_NVMEM`	Core; selects **`RT_USING_ADT_REF`**
`SOC_DM_NVMEM_DIR`	BSP OTP/eFuse/EEPROM drivers

No **`rt_bus`** — provider **`reg_read`**/**`reg_write`** + OFW lookup only.

1.4.2.65.1.2 Architecture

```
BSP provider (efuse/otp/eeprom)
      | rt_nvmem_device_register, rt_ofw_data = ndev
      v
Consumer: nvmem-cells = <&provider N>;
      v
rt_nvmem_get_cell_by_name → rt_nvmem_cell_read/write
      v
rt_nvmem_put_cell
```

1.4.2.65.1.3 Provider DT

```
efuse: efuse@110000 {
    compatible = "vendor,soc-efuse";
    reg = <0x110000 0x1000>;
    #address-cells = <1>;
    #size-cells = <1>;
    #nvmem-cell-cells = <1>;
    read-only;

    mac@0 { reg = <0x00 0x06>; };
};
```

Property	Role
`#nvmem-cell-cells`	Usually **1** — phandle arg = cell index
`read-only`	Forces read-only at register
`wp`	Write-protect GPIO (unless **`ignore_wp`**)

1.4.2.65.1.4 Consumer DT

```
gmac: ethernet@0 {
    nvmem-cells = <&efuse 0>;
    nvmem-cell-names = "mac-address";
};
```

`ofw_nvmem_get_cell` order:

1. **`rt_ofw_data(cell_np)`** if cached
2. Match **`append_cell`** static list by index/id
3. Allocate dynamic cell from child **`reg`** (**`offset`**, **`bytes`**)

Dynamic cells are **free_able** and removed on last **put_cell**.

1.4.2.65.1.5 Provider checklist

1. Implement **reg_read**; **reg_write** only if programmable.
2. Set **size**, **parent.ofw_node**, callbacks.
3. **rt_nvmem_device_register**.
4. Optional **append_cell** for fixed index/id layouts.
5. **remove**: all **put_cell**, then **unregister** when ref is 1.
6. Add driver under **SOC_DM_NVMEM_DIR**.

1.4.2.65.1.6 Bit-packed cells

bit_offset / **nbits** on **struct rt_nvmem_cell**: core masks after byte read; write merges into existing bytes. Prefer byte-aligned **reg = <offset length>**; validate parsing in **nvmem.c** or use **append_cell**.

1.4.2.65.1.7 Pitfalls

Issue	Mitigation
Provider not probed	**rt_platform_ofw_request** first
#nvmem-cell-cells mismatch	Match provider binding
Concurrent OTP	Serialize in **reg_read**/**reg_write**
unregister EBUSY	Release all cell refs

1.4.2.65.1.8 See also

- NVMEM subsystem
- IIO device model (DM)
- Open Firmware (OFW)

1.4.2.66 PCI / PCIe

1.4.2.66.1 PCI and PCIe overview

PCI/PCIe support in RT-Thread covers config-space access, bus enumeration, BAR assignment, INTx/MSI/MSI-X, and function drivers. Implementation layout matches **components/drivers/pci/**.

Source / topic	Documentation
probe.c — scan, setup, drivers	PCI enumeration and drivers
access.c — config accessors	PCI config space access
ecam.c, **host/** — root complex	PCI host controllers
irq.c — INTx assign/mask	PCI INTx interrupts
pme.c — power management	PCI power management
ofw.c — DT ranges/IRQ	PCI and device tree
msi/	PCI MSI and MSI-X
host-bridge.c — bridge function driver	PCI bridge function driver

Source / topic	Documentation
endpoint/ — EP mode	PCI endpoint (EP) mode

Header: **drivers/pci.h**. Requires **RT_USING_DM** and **RT_USING_PIC**.

1.4.2.66.1.1 Kconfig

Option	Role
RT_USING_PCI	Core PCI stack
RT_PCI_MSI	MSI / MSI-X (default y)
RT_PCI_ENDPOINT	Endpoint framework
RT_PCI_ECAM	ECAM config access
RT_PCI_SYS_64BIT	64-bit resource alloc
RT_PCI_CACHE_LINE_SIZE	Cache line bytes written to config
RT_PCI_LOCKLESS	Skip global config spinlock

Host-specific options: **pci/host/Kconfig**, **pci/host/dw/Kconfig**.

1.4.2.66.1.2 Topology

```
CPU + Host Bridge (platform driver)
      → rt_pci_host_bridge
      → root_bus (bus 0)
      → scan: endpoints + PCI bridges
      → sub-buses
      → rt_pci_driver probe per function
```

Object	Role
struct rt_pci_host_bridge	Domain, **bus_regions**, **rt_pci_ops**, optional OFW node
struct rt_pci_bus	Bus number, device list, **ops** chain
struct rt_pci_device	BDF, BARs **resource[]**, caps, **irq**, bound driver
struct rt_pci_driver	**ids[]**, **probe**/**remove**

BDF macros: **RT_PCI_DEVFN**, **RT_PCI_DEVID**, **RT_PCI_SLOT**, **RT_PCI_FUNC**.

1.4.2.66.1.3 Bring-up order

Full call chain (host → scan → BAR → function driver): **PCI enumeration and drivers** §Full initialization flow.

1. Platform **host** driver probes — PCI host controllers
2. **rt_pci_host_bridge_init** — **ranges**, **bus-range** — PCI and device tree
3. **rt_pci_host_bridge_probe** — DFS enumeration, **rt_pci_setup_device**, **rt_pci_device_register**
4. Bus **pci_probe** → **rt_pci_assign_irq** → function **pdrv->probe** — IRQ: PCI INTx interrupts, MSI: PCI MSI and MSI-X

1.4.2.66.1.4 Function driver checklist

1. Wait until **`rt_pci_setup_device`** completed (BARs in **`pdev->resource`**).
2. **`void *base = rt_pci_iomap(pdev, bar)`** before MMIO.
3. **`rt_pci_set_master`** only when DMA is ready.
4. IRQ: **`rt_pci_alloc_vector`** or **`rt_pci_msi_enable`** — not raw INTx unless legacy.
5. **`remove`**: **`rt_pci_free_vector`**, **`rt_pci_clear_master`**, disable INTx.

1.4.2.66.1.5 When to use PCI

PCI	Platform / AMBA
Device behind root complex with config space	Fixed SoC block with only **`reg`**/**`interrupts`**

1.4.2.66.1.6 See also

- PIC overview
- OFW interrupts
- NVMe subsystem
- `components/drivers/include/drivers/pci.h`

1.4.2.67 PCI enumeration and drivers

1.4.2.67.1 Enumeration and device setup (**`probe.c`**, **`pci.c`**)

End-to-end **host bring-up** → **bus scan** → **BAR assignment** → **driver probe**. Sources: **`probe.c`** (scan/host), **`pci.c`** (regions, register, bus), **`ofw.c`** (DT windows).

1.4.2.67.1.1 Full initialization flow

Phase A — Host controller (platform driver)

Typical ECAM host (**`pci_host_common_probe`** in `host/pci-host-common.c`):

```
platform probe (e.g. pci-host-ecam-generic)
     |
     v
rt_pci_host_bridge_alloc()
     |
     v
rt_dm_dev_iomap(dev, 0)          /* ECAM config window */
     |
     v
rt_pci_host_bridge_init()
     +-- rt_pci_ofw_host_bridge_init()   [RT_USING_OFW]
     |       bus-range, pci-domain
     |       rt_pci_ofw_parse_ranges() -> bus_regions[], dma_regions[]
     |       rt_pci_region_setup()       /* allocator pools ready */
     |       irq_slot = rt_pci_irq_slot
     |       irq_map  = rt_pci_ofw_irq_parse_and_map
     |
     v
pci_ecam_create(host, ops)       /* host_bridge->ops = ECAM read/write */
conf_win->win = ECAM base
     |
     v
rt_pci_host_bridge_probe()       /* starts enumeration -- Phase B */
```

DesignWare and other hosts follow the same pattern after link/DBI setup (PCI host controllers).

Phase B — Root bus and enumeration

`rt_pci_host_bridge_probe` → `rt_pci_scan_root_bus_bridge`:

```
rt_pci_host_bridge_register(host)
     |  pci_alloc_bus(NULL) -> root_bus
     |  root_bus->number = bus_range[0]
     |  root_bus->ops = host_bridge->ops
     |
     v
rt_pci_scan_child_bus(root_bus)
     |
     v
rt_pci_scan_child_buses(bus, buses=0)
     |
     +-- for slot 0..30 (step 8 devfn):
     |       rt_pci_scan_slot(bus, devfn)
     |           for func 0..7 (ARI / multifunction):
     |               rt_pci_scan_single_device()   /* Phase C */
     |
     +-- for each bridge on bus (rt_pci_foreach_bridge):
             pci_scan_bridge_extend()
                 pcie_fixup_link() [if PCIe cap]
                 pci_alloc_bus(parent) -> child bus
                 pci_child_bus_init(bus_no, bridge pdev)
                 program PCIR_PRIBUS_1 (primary/secondary/subordinate)
                 rt_pci_scan_child_buses(child, ...)   /* recurse */
                 update subordinate bus number
```

Depth-first: all devices on bus N are scanned, then each bridge opens bus N+1 and recurses.

Phase C — Single function (`rt_pci_scan_single_device`)

```
read config PCIR_VENDOR / PCIR_DEVICE
     |  (0xffff / 0x0000 -> empty slot, skip)
     v
rt_pci_alloc_device(bus)         /* link into bus->devices_nodes */
     |
     v
rt_pci_setup_device(pdev)        /* Phase D -- must succeed */
     |
     v
pci_procfs_attach(pdev)          [optional debug]
     |
     v
rt_pci_device_register(pdev)
     -> rt_bus_add_device(pci_bus, &pdev->parent)
     -> pci bus match + pci_probe()   /* Phase E */
```

If **`rt_pci_setup_device`** fails, the **`rt_pci_device`** is freed and the slot is skipped.

Phase D — Per-device setup (`rt_pci_setup_device`)

Order inside **`probe.c`** / **`pci.c`**:

Step	Action
1	**`rt_pci_ofw_device_init`** — bind OFW node if any
2	Read **class**, **hdr_type**, clear **STATUS** errors
3	**`pci_read_irq`** — read **INTPIN** / **INTLINE** from config only (not mapped yet)
4	**`rt_pci_device_alloc_resource`** — size BARs, **`rt_pci_region_alloc`**, write BAR registers, enable bridge windows
5	**`pci_init_capabilities`** — **`rt_pci_pme_init`**, **`rt_pci_msi_init`**/**`msix_init`** (disabled), PCIe cap, **cfg_size**, ARI

Device name set to **`domain:bus:slot.func`**.

Phase E — Bus driver probe (`pci.c`)

When **`rt_pci_device_register`** adds the device to the PCI bus:

```
pci_match(pdrv, pdev)            /* vendor/device or class table */
     |
     v
pci_probe(dev)
```

```
+-- rt_pci_assign_irq(pdev)       /* host irq_map or OFW INTx */
+-- rt_pci_enable_wake(pdev, D0)
+-- pdrv->probe(pdev)             /* NVMe, NIC, ... */
```

Important: function driver **probe** runs **after** BARs are assigned and **after** **rt_pci_assign_irq**. Use **rt_↩ pci_iomap** / **rt_pci_alloc_vector** here, not during scan.

IRQ timing summary

When	What
rt_pci_setup_device	**pci_read_irq** — raw config pin/line
pci_probe (bus)	**rt_pci_assign_irq** — **irq_map** / OFW → **pdev->irq**, **intx_pic**
Function **pdrv->probe**	MSI/MSI-X enable, **rt_pic_*** register

1.4.2.67.1.2 Flow diagram



1.4.2.67.1.3 Host bridge lifecycle (API)

API	Role
rt_pci_host_bridge_alloc(priv_size)	Allocate host bridge + private tail
rt_pci_host_bridge_init	OFW: **rt_pci_ofw_host_bridge_init**
rt_pci_host_bridge_probe	**rt_pci_scan_root_bus_bridge**

API	Role
rt_pci_host_bridge_register	Create root **rt_pci_bus**
rt_pci_host_bridge_remove	Enum remove all devices

1.4.2.67.1.4 Scan API

API	Role
rt_pci_scan_child_bus(bus)	Entry: scan one bus + bridges
rt_pci_scan_child_buses	Slots 0–30, then bridge extend
rt_pci_scan_slot	All functions on one devfn (MF/ARI)
rt_pci_scan_single_device	One function if vendor valid
rt_pci_scan_bridge	Bridge secondary bus setup
rt_pci_alloc_device	Allocate **rt_pci_device**, link to bus

Bridges: **pdev->subbus** after **pci_scan_bridge_extend**. Endpoints: **subbus == NULL**.

1.4.2.67.1.5 rt_pci_setup_device detail

See **Phase D** above. Invalid header/class → error, device discarded.

1.4.2.67.1.6 Resources (pci.c)

API	Role
rt_pci_region_setup	After OFW **ranges**: log pools, **bus_start 0x1000**
rt_pci_region_alloc	Carve BAR/bus address from **bus_regions**
rt_pci_device_alloc_resource	Probe BAR sizes, assign, write config
rt_pci_find_bar	Lookup assigned BAR by flags/index

1.4.2.67.1.7 Driver model

```
static const struct rt_pci_device_id my_ids[] = {
    { RT_PCI_DEVICE_ID(0x1234, 0xabcd) },
    { RT_PCI_DEVICE_CLASS(0x01080200, 0xffffff00) },
    { /* sentinel */ },
};

static struct rt_pci_driver my_driver = {
    .name = "my_pci",
    .ids = my_ids,
    .probe = my_probe,
    .remove = my_remove,
};
RT_PCI_DRIVER_EXPORT(my_driver);
```

API	Role
rt_pci_match_ids	Match during **pci_match**
rt_pci_driver_register	Register on PCI bus (export macro at init)
rt_pci_device_register	After scan — triggers bus **probe**
rt_pci_enum_device	Walk tree callback

1.4.2.67.1.8 Helpers

API	Role
`rt_pci_domain`, `rt_pci_find_host_bridge`	Domain / host lookup
`rt_pci_is_bridge`, `rt_pci_is_pcie`	Type checks
`rt_pci_find_capability`	Capability walk

1.4.2.67.1.9 Pitfalls

Issue	Mitigation
MMIO in scan before `device_register`	Wait for function `probe`
BARs empty	Fix DT `ranges` + `rt_pci_region_setup`
`irq == 0` after probe	Check `interrupt-map` / use MSI
Bridge bus exhausted	`bus-range` too small in DTS
Scan order vs driver	`RT_PCI_DRIVER_EXPORT` drivers already registered before host probe

1.4.2.67.1.10 See also

- PCI / PCIe
- PCI host controllers
- PCI config space access
- PCI and device tree
- PCI INTx interrupts

1.4.2.68 PCI config space access

1.4.2.68.1 Config space (`access.c`, `rt_pci_ops`)

All enumeration and drivers reach hardware through `struct rt_pci_ops` on each `rt_pci_bus`.
Sources: `components/drivers/pci/access.c`, `ecam.c` (typical `ops` implementation).

1.4.2.68.1.1 `struct rt_pci_ops`

Callback	Role
`map(bus, devfn, reg)`	Optional MMIO pointer to config dword (ECAM)
`read(bus, devfn, reg, width, &value)`	Config read
`write(bus, devfn, reg, width, value)`	Config write
`add` / `remove`	Bus hook (optional)

Host bridge sets `host_bridge->ops`; child buses inherit or use `child_ops`.

1.4.2.68.1.2 Locked accessors (access.c)

API	Role
rt_pci_bus_read_config_u8/u16/u32	Bus-level read
rt_pci_bus_write_config_u8/u16/u32	Bus-level write
rt_pci_bus_read_config_uxx	Width 1/2/4; uses map if implemented
rt_pci_read_config_* (pdev, ...)	Inline wrappers using pdev->devfn

Global rt_pci_lock serializes config unless RT_PCI_LOCKLESS=y.
On read error, typed helpers may return ~0 — check rt_err_t from bus->ops->read when critical.

1.4.2.68.1.3 ECAM fast path

pci_ecam_map computes:

```
win + (bus_offset | devfn_offset | reg_offset)
```

rt_pci_bus_read_config_uxx reads via HWREG8/16/32 when map is non-NULL.
See PCI host controllers for pci_generic_ecam_ops and DesignWare bus_shift variants.

1.4.2.68.1.4 Config space size

pdev->cfg_size set in rt_pci_setup_device:

- Standard 256 bytes, or extended 4 KiB if extended capability present.

Use rt_pci_find_capability only within valid size.

1.4.2.68.1.5 MMIO from BAR (not config)

API	Role
rt_pci_iomap(pdev, bar_idx)	rt_ioremap(resource[bar].base, size)

BAR CPU addresses come from enumeration — do not confuse with ECAM config window.

1.4.2.68.1.6 Pitfalls

Issue	Mitigation
Config during IRQ	Lock held by access.c — avoid long callbacks under lock
map returns NULL	Fall back to read/write ops only
Wrong devfn on secondary bus	Always use pdev->devfn on device's bus
Byte enables on IO	Use correct width API for register layout

1.4.2.68.1.7 See also

- PCI enumeration and drivers
- PCI host controllers
- PCI / PCIe

1.4.2.69 PCI host controllers

1.4.2.69.1 Host controllers (ecam.c, host/)

SoC **root complex** drivers: map config space, parse DT ranges, call rt_pci_host_bridge_probe. Distinct from the PCI **bridge function** driver (PCI bridge function driver).

1.4.2.69.1.1 ECAM (ecam.c, ecam.h)

API	Role
`pci_ecam_create(host_bridge, ops)`	Attach `pci_generic_ecam_ops` (or variant) to host
`pci_ecam_map(bus, devfn, where)`	Config MMIO address
`pci_generic_ecam_ops`	Standard `bus_shift` ECAM

`struct pci_ecam_config_window`: `win` base, `bus_range[2]`, `bus_shift`, `ops`.

DesignWare / slot-0 quirk

`pci_dw_ecam_bus_ops`: on root bus, `devfn` `slot` $>$ `0` returns `NULL` from `map` (only device 0 on bus 0) — used by `snps,dw-pcie-ecam`, Marvell, Socionext compatibles.

1.4.2.69.1.2 Generic platform host (host/pci-host-generic.c)

`RT_PLATFORM_DRIVER_EXPORT(gen_pci_driver)`

compatible	ECAM ops
`pci-host-ecam-generic`	`pci_generic_ecam_ops`
`pci-host-cam-generic`	CAM layout, `bus_shift = 16`
`snps,dw-pcie-ecam`, `marvell,armada8k-pcie-ecam`, ...	`pci_dw_ecam_bus_ops`

`pci_host_common_probe` (host/pci-host-common.c):

1. `rt_pci_host_bridge_alloc`
2. `rt_dm_dev_iomap(dev, 0)` — ECAM `reg`
3. `rt_pci_host_bridge_init` — OFW ranges
4. `pci_ecam_create`, `conf_win->win = base`
5. `rt_pci_host_bridge_probe`

1.4.2.69.1.3 DesignWare host (host/dw/)

File	Role
`pcie-dw-host.c`	RC mode: DBI, link, `rt_pci_host_bridge_probe`
`pcie-dw-ep.c`	EP mode — PCI endpoint (EP) mode
`pcie-dw-platfrom.c`	Platform glue

Enable via `pci/host/dw/Kconfig`. DT binding typically `snps,dw-pcie` plus ECAM child node.

1.4.2.69.1.4 Host bridge callbacks (IRQ without full OFW)

`struct rt_pci_host_bridge` optional:

Field	Role
`irq_slot(pdev, &pin)`	ACPI/legacy slot for INTx
`irq_map(pdev, slot, pin)`	Return Linux IRQ number

If NULL, OFW path or no INTx (PCI INTx interrupts, PCI and device tree).

1.4.2.69.1.5 Pitfalls

Issue	Mitigation
ECAM **`reg` not iomapped**	**`pci_host_common_probe`** fails with **`-RT_EIO`**
Empty **`ranges`**	**`rt_pci_ofw_parse_ranges`** fails — no MEM/IO for BARs
Wrong **`compatible`**	Pick ops matching hardware CAM vs ECAM
Confusing with **`host-bridge.c`**	That file is PCI **function** driver for bridge devices

1.4.2.69.1.6 See also

- PCI enumeration and drivers
- PCI and device tree
- PCI config space access
- Platform bus

1.4.2.70 PCI INTx interrupts

1.4.2.70.1 Legacy INTx (`irq.c` + core)

INTx (legacy PCI pin A–D) assignment and masking. MSI/MSI-X: PCI MSI and MSI-X. DT routing: PCI and device tree.
Sources: **`components/drivers/pci/irq.c`**, INTx helpers in **`pci.c`**.

1.4.2.70.1.1 Assignment (`rt_pci_assign_irq`)

During setup, if host provides **`irq_map`**:

1. Read **`PCIR_INTPIN`** from config
2. Optional **`irq_slot`** swizzle on bridges
3. **`irq_map(pdev, slot, pin)`** → **`pdev->irq`**
4. Write **`PCIR_INTLINE`** in config

Without **`irq_map`**, logs debug and leaves IRQ 0 — use OFW or MSI.

OFW path

`rt_pci_ofw_irq_parse_and_map` sets **`pdev->intx_pic`** and IRQ via **`rt_ofw_map_irq`** — preferred on DT platforms (PCI and device tree).

1.4.2.70.1.2 Swizzling

API	Role
`rt_pci_irq_intx(pdev, pin)`	Swizzle INTPIN across P2P bridges

API	Role
`rt_pci_irq_slot(pdev, &pin)`	Slot number for `interrupt-map` key

Used when building `interrupt-map` address cells on root bridge.

1.4.2.70.1.3 Mask / unmask

API	Role
`rt_pci_intx(pdev, enable)`	Enable/disable INTx in command register
`rt_pci_check_and_mask_intx`	Mask if supported
`rt_pci_check_and_unmask_intx`	Unmask
`rt_pci_irq_mask` / `rt_pci_irq_unmask`	Device-level helpers

`pdev->broken_intx_masking`: set when hardware cannot mask INTx reliably.

1.4.2.70.1.4 Unified vector API (`pci.h`, `RT_PCI_MSI`)

`rt_pci_alloc_vector(pdev, min, max, flags, affinities)`:

Flag	Meaning
`RT_PCI_IRQ_F_LEGACY`	Allow INTx
`RT_PCI_IRQ_F_MSI`	Allow MSI
`RT_PCI_IRQ_F_MSIX`	Allow MSI-X
`RT_PCI_IRQ_F_AFFINITY`	CPU affinity

Picks best available type; `rt_pci_free_vector` on teardown.

1.4.2.70.1.5 Pitfalls

Issue	Mitigation
MSI + INTx both active	Core often masks INTx when MSI enabled — use one path
`interrupt-map` missing	`-RT_EEMPTY` from OFW — enable MSI or fix DTS
Wrong pin after bridge	Use `rt_pci_irq_intx`, not raw config pin on child
`irq == 0`	May mean unassigned — do not register handler

1.4.2.70.1.6 See also

- PCI MSI and MSI-X
- PCI and device tree
- OFW interrupts
- PIC overview

1.4.2.71 PCI power management

1.4.2.71.1 PME and D-states (`pme.c`)

PCI Power Management capability: D0/D1/D2/D3hot/D3cold and PME wake.
Source: `components/drivers/pci/pme.c`. Init from `rt_pci_setup_device` via `rt_pci_pme_init`.

1.4.2.71.1.1 API

API	Role
`rt_pci_pme_init(pdev)`	Find `PCIY_PMG` cap, fill `pme_support` bitmask
`rt_pci_pme_active(pdev, enable)`	Enable/disable PME# generation
`rt_pci_enable_wake(pdev, state, enable)`	Program wake for **D1**/**D2**/**D3hot**/**D3cold**
`rt_pci_pme_capable(pdev, state)`	Test `pme_support` bit

`enum rt_pci_power`: `RT_PCI_D0` … `RT_PCI_D3COLD`, `RT_PCI_PME_MAX`.

1.4.2.71.1.2 Bridge driver integration

`host-bridge.c` PM ops (`RT_USING_PM`) walk the bus with `rt_pci_enum_device` and call `rt_pci_enable_wake` per device on system sleep (PCI bridge function driver).

Sleep mode	PCI target
Idle	D3hot
Light / Deep	D1
Standby	D2
Shutdown	D3cold

Coordinate with `power-domain`, `clk`, `reset` from DM — PCI D-state alone may not gate SoC power.

1.4.2.71.1.3 Pitfalls

Issue	Mitigation
PME spec > 3	`rt_pci_pme_init` logs error and aborts
Wake without `pme_cap`	`rt_pci_pme_capable` returns false
D3cold without platform support	May need PMIC/SCPI outside PCI layer

1.4.2.71.1.4 See also

- PCI enumeration and drivers
- PCI bridge function driver
- PCI / PCIe

1.4.2.72 PCI MSI and MSI-X

1.4.2.72.1 PCI MSI / MSI-X (`msi/`)

Headers: `drivers/pci_msi.h`, `pci_regs.h`.

File	Role
`msi/msi.c`	Capability parse, mask/unmask, message write
`msi/irq.c`	`rt_pci_msi_setup_irqs`, PIC allocation

File	Role
`msi/device.c`	Device enable helpers

Disabled when `RT_PCI_MSI=n` — stubs in `pci.h`.

1.4.2.72.1.1 Prerequisites

- `pdev->msi_pic`: `struct rt_pic` with `irq_alloc_msi`, `irq_free_msi`, `irq_compose_msi_msg`
- `pdev->msi_cap` / `pdev->msix_cap` from `rt_pci_msi_init` / `rt_pci_msix_init` in setup

1.4.2.72.1.2 Setup flow (`irq.c`)

`rt_pci_msi_setup_irqs(pdev, nvec, type)`:

MSI

1. Read `multi_msg_use` (log2 vectors) from first `rt_pci_msi_desc`
2. `irq_alloc_msi` until `nvec` contiguous IRQs obtained
3. Per vector: `irq_compose_msi_msg`, `rt_pci_msi_write_msg`

MSI-X

- Per table entry: separate `irq_alloc_msi`
- Table must be mapped (BAR assigned before enable)

`rt_pci_msi_cleanup_irqs` frees allocations.

1.4.2.72.1.3 Driver API

API	Use
`rt_pci_msi_enable(pdev)`	Single vector
`rt_pci_msi_enable_range(pdev, min, max)`	Power-of-two count
`rt_pci_msi_enable_range_affinity(...)`	With CPU masks
`rt_pci_msix_enable(pdev, entries, count)`	MSI-X table indices
`rt_pci_msi_disable` / `rt_pci_msix_disable`	Tear down
`rt_pci_alloc_vector`	Unified legacy/MSI/MSI-X picker (PCI INTx interrupts)

1.4.2.72.1.4 Masking (`msi.c`)

- **MSI**: config `mask` register when `cap.is_masking`
- **MSI-X**: per-entry vector control in table

`rt_pci_msi_mask_irq` / `rt_pci_msi_unmask_irq` — PIC integration.

1.4.2.72.1.5 Pitfalls

Issue	Mitigation
Non-contiguous MSI vectors	Allocator retries; check return
MSI-X before BAR map	Enable only after **rt_pci_setup_device**
INTx still firing	Enable MSI only after masking INTx in core
Large MSI-X table	Default affinity storage capped — review for 2k+ entries

1.4.2.72.1.6 See also

- PCI / PCIe
- PCI INTx interrupts
- PCI and device tree
- PIC overview
- components/drivers/pci/msi/

1.4.2.73 PCI and device tree

1.4.2.73.1 PCI Open Firmware integration

Implementation: **components/drivers/pci/ofw.c**. Requires **RT_USING_OFW**.
Host controllers probe as **platform** devices; this file connects enumerated **rt_pci_device** instances to DT **ranges** and **INTx** routing.

1.4.2.73.1.1 INTx: pci_ofw_irq_parse

Resolves **legacy PCI INTx**:

1. Device node has **interrupts** → **rt_ofw_parse_irq_cells**
2. Else read **INTPIN** from config; **pin == 0** → **-RT_ENOSYS**
3. Walk bus tree:
   - Device-local **interrupt-map** if present
   - Else P2P bridge or **host bridge** node → **rt_ofw_parse_irq_map** with key:

```
map_addr[0] = (bus->number « 16) | (devfn « 8)
map_addr[3] = swizzled INTx pin   /* rt_pci_irq_intx */
```

Missing **interrupt-map** on root → **-RT_EEMPTY** (logged).

1.4.2.73.1.2 rt_pci_ofw_irq_parse_and_map

```
int rt_pci_ofw_irq_parse_and_map(struct rt_pci_device *pdev,
        rt_uint8_t slot, rt_uint8_t pin);
```

Calls **pci_ofw_irq_parse**, then **rt_ofw_map_irq**. Sets **pdev->intx_pic** from **rt_ofw_data(irq_args.data)**.
Host bridge may wire **host_bridge->irq_map** to this helper during init.

1.4.2.73.1.3 ranges parsing

pci_ofw_parse_ranges / **rt_pci_ofw_parse_ranges**:

- Decodes PCI address **space_code** (config / I/O / 32-bit mem / 64-bit mem)
- Fills **host_bridge->bus_regions**: CPU phys, PCI bus addr, size, prefetch flags
- Used before **rt_pci_region_setup**

rt_pci_ofw_host_bridge_init, **rt_pci_ofw_bus_init**, **rt_pci_ofw_device_init** — per-node OFW hooks during probe.

1.4.2.73.1.4 DT artifacts

Property	Use
Host **compatible**	Select host driver (PCI host controllers)
reg	ECAM MMIO
ranges / **dma-ranges**	MEM/IO windows
interrupt-map / **interrupt-map-mask**	INTx → PIC
Endpoint **reg** (optional)	Direct **interrupts** on slot

BAR values still come from config space after enumeration.

1.4.2.73.1.5 MSI/MSI-X

MSI message address/data from **PIC** (**pdev->msi_pic**), not **interrupts** cells. Ensure PIC/MSI parent in DT matches host setup (PCI MSI and MSI-X).

1.4.2.73.1.6 Pitfalls

Issue	Mitigation
Missing root **interrupt-map**	Use MSI or fix DTS
Wrong swizzling	Use **rt_pci_irq_intx** for map key
Empty **ranges**	BAR assignment fails

1.4.2.73.1.7 See also

- PCI / PCIe
- PCI host controllers
- PCI INTx interrupts
- PCI MSI and MSI-X
- OFW interrupts
- `components/drivers/pci/ofw.c`

1.4.2.74 PCI bridge function driver

1.4.2.74.1 PCI bridge function driver (host-bridge.c)

Not the SoC host controller (PCI host controllers). This is the **RT_PCI_DRIVER_EXPORT** driver for **PCI bridge functions** (root ports, P2P bridges, certain virtual bridges).
Source: **components/drivers/pci/host-bridge.c**.

1.4.2.74.1.1 Matched devices

```
static const struct rt_pci_device_id host_bridge_pci_ids[] = {
    { RT_PCI_DEVICE_ID(PCI_VENDOR_ID_REDHAT, 0x0008) },
    { RT_PCI_DEVICE_CLASS(PCIS_BRIDGE_PCI_NORMAL, ~0) },
    { RT_PCI_DEVICE_CLASS(PCIS_BRIDGE_PCI_SUBTRACTIVE, ~0) },
    { RT_PCI_DEVICE_CLASS(((PCIS_SYSTEM_RCEC << 8) | 0x00), ~0) },
    { /* sentinel */ }
};
```

Vendor-specific bridge drivers should register **narrower** **rt_pci_device_id** entries **before** this generic match.

1.4.2.74.1.2 Probe / remove

Callback	Behavior
`host_bridge_probe`	`rt_pci_set_master(pdev)`; optional PM register
`host_bridge_remove`	`rt_pci_clear_master`
`host_bridge_shutdown`	PM unregister

No BAR programming here — windows configured during enumeration (PCI enumeration and drivers).

1.4.2.74.1.3 Power management (`RT_USING_PM`)

`host_bridge_pm_suspend` / `resume`: `rt_pci_enum_device` on bus, `rt_pci_enable_wake` per function (PCI power management).

Sleep	PCI state
Idle	D3hot
Light / Deep	D1
Standby	D2
Shutdown	D3cold

1.4.2.74.1.4 Component map

Layer	Location	Role
Host RC	`pci/host/`, `ecam.c`	Enumeration
Bridge function	`host-bridge.c`	Bus master + PM on bridge devfn
OFW INTx	`pci/ofw.c`	`interrupt-map`

1.4.2.74.1.5 Pitfalls

Issue	Mitigation
Expect BAR setup here	Done in core PCI
Generic class match	Override with specific driver if needed

1.4.2.74.1.6 See also

- PCI / PCIe
- PCI host controllers
- PCI power management
- PCI and device tree

1.4.2.75 PCI endpoint (EP) mode

1.4.2.75.1 PCI endpoint framework (`endpoint/`)

Header: **`drivers/pci_endpoint.h`**. Core: **`endpoint/endpoint.c`**, **`endpoint/mem.c`**.
SoC as **PCIe device** under a remote host — not root complex. Kconfig: **`RT_PCI_ENDPOINT`**.

1.4.2.75.1.1 Core objects

Type	Role
`struct rt_pci_ep`	Controller: **`max_functions`**, **`ops`**, **`epf`** list
`struct rt_pci_epf`	Function image (driver)
`struct rt_pci_ep_header`	Emulated config header
`struct rt_pci_ep_bar`	BAR size/flags for programming

`rt_pci_ep_register` / **`unregister`** — global controller list under lock.

1.4.2.75.1.2 Controller ops

API	`ops`	Purpose
`rt_pci_ep_write_header`	**`write_header`**	Config header for **`func_no`**
`rt_pci_ep_set_bar` / **`clear_bar`**	**`set_bar`**	BAR programming
`rt_pci_ep_map_addr` / **`unmap_addr`**	**`map_addr`**	CPU PCI map

| **`rt_pci_ep_set_msi`** / **`get_msi`** | MSI doorbell | | **`rt_pci_ep_set_msix`** / **`get_msix`** | MSI-X table | | **`rt_pci_ep_raise_irq`** | **`raise_irq`** | INTx/MSI/MSI-X to host | | **`rt_pci_ep_start`** / **`stop`** | Link visibility |
`func_no` $<$ **`max_functions`** required; missing **`ops`** $\rightarrow$ **`-RT_ENOSYS`**.

1.4.2.75.1.3 BAR validation

`rt_pci_ep_set_bar` rejects invalid 64-bit pairs, $>$4 GiB on 32-bit BAR, bad I/O flags before calling hardware.

1.4.2.75.1.4 Memory helpers (`mem.c`)

`rt_pci_ep_mem_init`, **`rt_pci_ep_mem_array_init`** — outbound/inbound windows for host mapping.
`host/dw/pcie-dw_ep.c` — typical **`rt_pci_ep_ops`** implementation.

1.4.2.75.1.5 Host vs endpoint

Mode	APIs	Doc
Root complex	**`rt_pci_host_bridge_`**$*$, **`rt_pci_scan_`**$*$	PCI host controllers
Endpoint	**`rt_pci_ep`**$\hookleftarrow$ **`_`**$*$	this page

Do not call host enumeration on EP controllers.

1.4.2.75.1.6 Pitfalls

Issue	Mitigation
`func_no` out of range	**`-RT_EINVAL`**
IRQ before link **`start`**	Host may ignore
64-bit BAR layout	Follow PCI pair rules

1.4.2.75.1.7 See also

- PCI / PCIe
- PCI host controllers
- `components/drivers/pci/host/dw/pcie-dw_ep.c`

1.4.2.76 Generic PHY (Phye)

1.4.2.76.1 PHY abstraction (phye) overview

Phye is the DM framework for **external PHY** blocks: SerDes/USB/PCIe/SATA/MIPI PHY that sit beside the MAC/↩ host controller and are shared via device tree **`phys`** phandles.

Topic	Page
Core API, refcount, modes	Phye core API
Device tree **`phys`** / **`#phy-cells`**	Phye and device tree
Implement a PHY provider	Phye provider driver
Host / MAC consumer pattern	Phye consumer integration
Built-in **`usb-nop-xceiv`**	Generic USB PHY

Header: **`drivers/phye.h`**. Core: **`components/drivers/phye/phye.c`**.

1.4.2.76.1.1 Kconfig

Option	Role
`RT_USING_PHYE`	Framework (**`RT_USING_DM`**)
`RT_PHYE_GENERIC_USB`	**`usb-nop-xceiv`** platform driver
`SOC_DM_PHYE_DIR`	BSP adds SoC-specific PHY drivers via **`osource`**

Use phye only when the PHY is **not** fully embedded in the controller IP.

1.4.2.76.1.2 Architecture

```
PHY provider (platform driver)
     rt_phye_register()  →  rt_ofw_data(phy_node) = rt_phye *
            ^
            |  phys + #phy-cells [+ phy-names]
            |
Consumer (USB/PCIe/SATA/AHCI/... host driver)
     rt_phye_get_by_index/name → init → power_on → set_mode
     ... link training / MAC start ...
     power_off → exit → rt_phye_put
```

Providers implement **`struct rt_phye_ops`**; the core handles **reference counting** on **`init`** / **`power_on`**.

1.4.2.76.1.3 When to use phye

Use `rt_phye_*`	Skip phye
DT **phys** on host; PHY has own **compatible** node	PHY registers only touched inside one driver
Same PHY IP shared (mux) or tuning table reused	No OFW and fixed **board.c** init

1.4.2.76.1.4 Power-up order (typical)

Follow SoC TRM; common sequence:

1. Regulator / power domain
2. Reference clock (**rt_clk_prepare_enable**)
3. Deassert reset
4. **rt_phye_init** → **rt_phye_power_on**
5. **rt_phye_set_mode** (lane rate, USB HS/SS, PCIe RC/EP, . . .)
6. Host link training / MAC enable

Teardown in reverse; see Phye consumer integration.

1.4.2.76.1.5 In-tree consumers (examples)

Consumer	PHY lookup	Source
DesignWare AHCI	**rt_phye_get_by_name(dev, "sata-phy")**	**ata/ahci-dw.c**
USB host/device	**phys** → generic or SoC PHY	BSP + Generic USB PHY

PCIe host drivers may call **RT_PHYE_MODE_PCIE** before training when BSP supplies a PHY driver under **SOC_DM_PHYE_DIR**.

1.4.2.76.1.6 See also

- OFW nodes and properties — phandle pattern
- PCI host controllers
- Clock framework (CLK)
- components/drivers/include/drivers/phye.h

1.4.2.77 Phye core API

1.4.2.77.1 Core framework (phye.c)

Registration, reference-counted **init**/**power_on**, and **set_mode**. Source: **components/drivers/phye/phye.c**.

1.4.2.77.1.1 Objects

Type	Role
struct rt_phye	**dev**, **ops**, **init_count**, **power_count**, **lock**
struct rt_phye_ops	Provider callbacks (all optional except as needed)

rt_phye_register requires **phye->dev** and **phye->ops**, calls **rt_dm_dev_bind_fwdata(dev, NULL, phye)** so **rt_ofw_data(phy_np)** resolves the provider.
rt_phye_unregister fails with **-RT_EBUSY** if **dev->ref_count != 0**.

1.4.2.77.1.2 Reference counting

API	Behavior
rt_phye_init / **rt_phye_exit**	**ops->init** on first **init**; **ops->exit** when **init_count** returns to 0
rt_phye_power_on / **rt_phye_power_off**	Same pattern for **power_on**/**power_off**
rt_phye_reset	No refcount — call when driver needs pulse
rt_phye_set_mode	No refcount — configure mode each time (serialized by lock)

NULL phye pointer: APIs return **RT_EOK** (no-op) — useful for optional PHY in consumer code.

1.4.2.77.1.3 rt_phye_ops

Callback	Typical use
init	One-time calibration, load OTP/NVMEM
exit	Reverse **init**
reset	GPIO or soft reset pulse
power_on	Enable regulators/clocks, exit reset
power_off	Gate power/clocks
set_mode	USB speed, PCIe RC/EP, SATA, MIPI, ...
ofw_parse	Decode **#phy-cells** after phandle resolve

1.4.2.77.1.4 Modes (enum rt_phye_mode)

Range	Examples
< **RT_PHYE_MODE_MAX**	**RT_PHYE_MODE_USB_HOST_HS**, **RT_PHYE_MODE_PCIE**, **RT_PHYE_MODE_SATA**, **RT_PHYE_MODE_MIPI_DPHY**, ...
>= **RT_PHYE_MODE_MAX** (submode only)	**RT_PHYE_MODE_PCIE_RC**, **RT_PHYE_MODE_PCIE_EP**, **RT_PHYE_MODE_PCIE_BIFURCATION**

PCIe example (base mode + submode):

```
rt_phye_set_mode(phy, RT_PHYE_MODE_PCIE, RT_PHYE_MODE_PCIE_RC);
```

Simple mode (no submode):

```
rt_phye_set_mode_simple(phy, RT_PHYE_MODE_SATA);
```

```
/* same as set_mode(..., RT_PHYE_MODE_INVALID) */
```

rt_phye_set_mode returns **-RT_EINVAL** if **mode >= RT_PHYE_MODE_MAX** or submode is invalid.

1.4.2.77.1.5 Acquire / release (consumer)

API	Role
rt_phye_get_by_index(dev, index)	Parse **phys** on **dev->ofw_node**
rt_phye_get_by_name(dev, id)	**phy-names** → index
rt_phye_put(phye)	Decrement **dev->ref_count**

Returns **rt_err_ptr(...)** on failure — use **rt_is_err_or_null(phye)** before use.
get bumps **ref_count** under **phye->lock**.

1.4.2.77.1.6 See also

- Phye and device tree
- Phye provider driver
- Phye consumer integration

1.4.2.78 Phye and device tree

1.4.2.78.1 Device tree binding (phye.c)

Consumers reference PHY providers with standard graph properties. Requires **RT_USING_OFW**.

1.4.2.78.1.1 Properties (consumer node)

Property	Role
phys	Phandle list to PHY provider node(s)
#phy-cells	On **provider** — specifier length per entry
phy-names	Optional string ID per entry (for **rt_phye_get_by_name**)

Example (SATA host):

```
ahci: sata@... {
    phys = <&sata_phy 0>;
    phy-names = "sata-phy";
};
```

1.4.2.78.1.2 Resolution flow (ofw_phye_get_by_index)

```
rt_ofw_parse_phandle_cells(np, "phys", "#phy-cells", index, &args)
     |
     |  args.data = provider rt_ofw_node *
     v
rt_platform_ofw_request(provider_np)   /* probe provider if needed */
     |
     v
phye = rt_ofw_data(provider_np)
     |
     v
phye->ops->ofw_parse(phye, &args)     /* optional */
```

If **ofw_parse** fails, **get_by_index** returns **rt_err_ptr(err)**.
Provider platform driver must **rt_phye_register** in **probe** after **rt_dm_dev_bind_fwdata**.

1.4.2.78.1.3 Provider node

Element	Role
compatible	Selects PHY driver (SoC or generic)
#phy-cells	Usually 0 or mode/lane index
clocks / **resets** / **regulators**	Handled in provider **probe**, not by core

Cell semantics are **driver-specific** — document in BSP binding and implement in **ofw_parse**.

1.4.2.78.1.4 Pitfalls

Issue	Mitigation
Provider not probed	**rt_platform_ofw_request** before **rt_ofw_data** (core does this)
ofw_parse omitted	**#phy-cells ignored** — wrong mode if cells non-zero
Index / name mismatch	**phy-names** must align with **phys** order
Missing **phys**	**get_by_index** returns NULL — fail host **probe**

1.4.2.78.1.5 See also

- Phye core API
- OFW nodes and properties
- Phye provider driver

1.4.2.79 Phye provider driver

1.4.2.79.1 Implementing a PHY provider

Platform (or other) driver that owns PHY registers and exports **struct rt_phye**.

1.4.2.79.1.1 Minimal template

```
struct my_phy {
    struct rt_phye phye;
    void __iomem *regs;
    struct rt_clk *clk;
    /* mode, calibration, ... */
};

static rt_err_t my_phy_power_on(struct rt_phye *phye)
{
    struct my_phy *p = rt_container_of(phye, struct my_phy, phye);
    return rt_clk_prepare_enable(p->clk);
}

static rt_err_t my_phy_set_mode(struct rt_phye *phye,
        enum rt_phye_mode mode, int submode)
{
    /* program PHY for mode / submode */
    return RT_EOK;
}

static const struct rt_phye_ops my_phy_ops = {
    .power_on  = my_phy_power_on,
    .power_off = my_phy_power_off,
    .set_mode  = my_phy_set_mode,
    .ofw_parse = my_phy_ofw_parse,
};

static rt_err_t my_phy_probe(struct rt_platform_device *pdev)
{
    struct my_phy *p = rt_calloc(1, sizeof(*p));
    p->phye.dev = &pdev->parent;
    p->phye.ops = &my_phy_ops;
    return rt_phye_register(&p->phye);
}
```

1.4.2.79.1.2 Probe checklist

1. **rt_dm_dev_iomap** / clocks / resets / regulators per TRM
2. Fill **rt_phye_ops** — only implement needed ops
3. **phye->dev = &pdev->parent**
4. **rt_phye_register(&phye)** — binds **rt_ofw_data** for consumers
5. Do **not** call **set_mode** for a specific host here — consumers select mode

---

1.4.2.79.1.3 ofw_parse

```
static rt_err_t my_phy_ofw_parse(struct rt_phye *phye,
        struct rt_ofw_cell_args *args)
{
    /* args->args[0..args_count-1] from #phy-cells */
    /* store in container struct for set_mode */
    return RT_EOK;
}
```

Return error if cells are invalid — consumer **get** fails and host **probe** aborts.

1.4.2.79.1.4 Unregister

remove: **rt_phye_unregister** then free resources. Fails if consumers still hold references (**-RT_EBUSY**).

1.4.2.79.1.5 BSP placement

SoC-specific drivers often live under **SOC_DM_PHYE_DIR** (Kconfig **osource**). Enable in menuconfig alongside **RT_USING_PHYE**.

1.4.2.79.1.6 Pitfalls

Issue	Mitigation
Forgot **rt_phye_register**	**rt_ofw_data** NULL — consumer sees missing PHY
Mode in **probe**	Host may need different mode — use **set_mode** from consumer
Shared PHY	Refcount in core helps; exclusive mux still needs driver policy

1.4.2.79.1.7 See also

- Phye core API
- Phye and device tree
- Generic USB PHY

1.4.2.80 Phye consumer integration

1.4.2.80.1 Host / MAC consumer pattern

How link-layer drivers acquire and sequence PHY bring-up **before** training or DMA.

1.4.2.80.1.1 Recommended sequence

```
struct rt_phye *phy;

phy = rt_phye_get_by_name(dev, "sata-phy");
if (rt_is_err_or_null(phy))
    return rt_ptr_err(phy) ? rt_ptr_err(phy) : -RT_ERROR;

err = rt_phye_init(phy);
if (err)
    goto put;

err = rt_phye_power_on(phy);
if (err)
    goto exit;

err = rt_phye_set_mode_simple(phy, RT_PHYE_MODE_SATA);
if (err)
    goto power_off;

/* optional: rt_phye_reset(phy) */

err = host_enable_and_train();   /* MAC-specific */
if (err)
    goto power_off;

/* store phy in driver private */

power_off:
    rt_phye_power_off(phy);
exit:
    rt_phye_exit(phy);
put:
    rt_phye_put(phy);
return err;
```

On success, keep PHY powered until **`remove`** — do not **`put`** until teardown.

1.4.2.80.1.2 DesignWare AHCI example (`ata/ahci-dw.c`)

Step	Call
Get	**`rt_phye_get_by_name(dev, "sata-phy")`**
Clocks	**`rt_clk_array_prepare_enable`** (before PHY)
PHY	**`rt_phye_init`** → **`rt_phye_power_on`**
Host	**`rt_ahci_host_register`**
Fail path	**`power_off`** → **`exit`** → disable clocks → **`rt_phye_put`**
Remove	**`exit`** + **`power_off`** (note order in source)

DTS: host node needs **`phys`** / **`phy-names = "sata-phy"`** pointing at SATA PHY provider.

1.4.2.80.1.3 PCIe host (typical)

Before **`rt_pci_host_bridge_probe`** or link training:

```
phy = rt_phye_get_by_index(dev, 0);
rt_phye_init(phy);
rt_phye_power_on(phy);
rt_phye_set_mode(phy, RT_PHYE_MODE_PCIE, RT_PHYE_MODE_PCIE_RC);
/* then DWC / generic PCIe host probe */
```

Exact ordering vs **`clk`**/**`reset`** is SoC-specific — PCI host controllers.

1.4.2.80.1.4 USB

Often uses **`usb-nop-xceiv`** (Generic USB PHY) or SoC USB PHY:

```
phy = rt_phye_get_by_index(dev, 0);
rt_phye_power_on(phy);
rt_phye_set_mode_simple(phy, RT_PHYE_MODE_USB_HOST_SS);
```

Generic USB driver may only implement **`power_on`**/**`power_off`**/**`reset`** — mode set optional.

1.4.2.80.1.5 Error handling

Return	Meaning
rt_err_ptr(-RT_EINVAL)	Bad args
rt_err_ptr from ofw_parse	DT cells wrong
NULL	No **phys** / provider missing

Always **rt_phye_put** for every successful **get**.

1.4.2.80.1.6 Pitfalls

Issue	Mitigation
Train before **power_on**	Link fails — follow TRM order
set_mode while link up	Down link first
Leak **put**	Pair with **get** on all paths
Double **power_off** in remove	Match **probe** success path

1.4.2.80.1.7 See also

- Generic PHY (Phye)
- Phye and device tree
- PCI enumeration and drivers
- ATA / AHCI

1.4.2.81 Generic USB PHY

1.4.2.81.1 Generic USB PHY (phye-generic-usb.c)

Minimal PHY provider for **usb-nop-xceiv** — reset, clock, and regulators without complex **set_mode**.
Kconfig: **RT_PHYE_GENERIC_USB** (requires **RT_USING_PHYE**).

1.4.2.81.1.1 Device tree

```
usb_phy: usb-phy {
    compatible = "usb-nop-xceiv";
    clocks = <&usb_clk>;
    clock-frequency = <24000000>;
    vcc-supply = <&usb_vcc>;
    vbus-supply = <&usb_vbus>;   /* optional */
    reset-gpios = <&gpio PHY_RST GPIO_ACTIVE_LOW>;
};
```

Consumer (e.g. DWC3):

```
usb3: usb@... {
    phys = <&usb_phy>;
    phy-names = "usb";
};
```

1.4.2.81.1.2 Operations

rt_phye_ops	Behavior
power_on	**vcc** enable → **clk_prepare_enable** → **reset** pulse (15 ms assert, 20 ms deassert)
power_off	**vcc** disable, clock off
reset	GPIO reset only

No **init**, **exit**, or **set_mode** — USB mode assumed fixed after power-on.

1.4.2.81.1.3 Probe flow

1. **reset** GPIO from **reset-gpios**
2. **main_clk** via **rt_clk_get_by_name**; optional **clock-frequency**
3. **vcc**, **vbus** regulators — **vbus** enabled at probe if present
4. **rt_phye_register**

INIT_PLATFORM_EXPORT(generic_usb_phy_drv_register) — registers platform driver at init.

1.4.2.81.1.4 Consumer usage

```
phy = rt_phye_get_by_name(dev, "usb");
rt_phye_power_on(phy);   /* often sufficient for nop-xceiv */
/* start USB controller */
```

Use **rt_phye_init** only if provider adds **init** later.

1.4.2.81.1.5 Pitfalls

Issue	Mitigation
Wrong clock rate	Set **clock-frequency** in DTS
vbus always on	Probe enables **vbus** — OK for host; device role may differ
Expect **set_mode for SS**	Add SoC PHY driver or extend ops

1.4.2.81.1.6 See also

- Phye provider driver
- Phye consumer integration
- Generic PHY (Phye)

1.4.2.82 PIC overview

1.4.2.82.1 Programmable Interrupt Controller (PIC)

The **PIC** layer maps device-tree interrupts to **global virtual IRQ numbers**, dispatches ISRs, supports **cascaded** controllers, and integrates arch-specific controllers (in-tree reference: `pic-gicv2.c` / `pic-gicv3.c`) and MSI extensions.

Topic	Page
Virtual IRQ, `pirq`, global table	PIC virtual IRQ domain
Framework API (`pic.c`)	PIC core framework
Cascade & routing	PIC cascade and routing
Device tree / `rt_ofw_map_irq`	PIC and device tree
MSI / MSI-X (PCI vectors)	PIC and PCI MSI
Rockchip / Raspberry Pi examples	PIC platform examples

Header: **drivers/pic.h**. Core: **components/drivers/pic/pic.c**.

1.4.2.82.1.1 Kconfig

Option	Role
RT_USING_PIC	PIC framework (**RT_USING_DM**)
MAX_HANDLERS	Size of global virtual IRQ table (default 256)
RT_PIC_ARM_GIC	GICv1/v2 + OFW
RT_PIC_ARM_GIC_V2M	GIC MSI frame (PCI MSI)
RT_PIC_ARM_GIC_V3	GICv3
RT_PIC_ARM_GIC_V3_ITS	GICv3 ITS + LPI
RT_USING_PIC_STATISTICS	ISR timing stats
SOC_DM_PIC_DIR	BSP-specific PIC drivers

1.4.2.82.1.2 Boot sequence (typical AArch64 DM)

From **libcpu/aarch64/common/setup.c**:

```
rt_fdt_unflatten()
     |
     v
rt_pic_init()              /* scan DT interrupt-controller nodes */
     |  ofw_pic_init → RT_PIC_OFW_DECLARE stubs (e.g. GIC)
     |  rt_pic_linear_irq, rt_pic_add_traps
     v
rt_pic_irq_init()          /* per-PIC ops->irq_init (CPU IF enable) */
     |
     v
rt_components_board_init() /* platform drivers, devices */
```

Root **PIC** (e.g. SoC interrupt controller) must initialize here before devices call **rt_dm_dev_get_irq**.

1.4.2.82.1.3 Three IRQ numbers (do not confuse)

Name	Field	Used by
hwirq	**pirq->hwirq**	This PIC hardware line (GIC SPI 32 = hwirq 32)
Virtual / logical IRQ	**pirq->irq**	**rt_pic_attach_irq**, **rt_hw_interrupt_*↩**, **rt_dm_dev_get_irq**
irq_index	0 ... pic->irq_nr-1	Index into **pic->pirqs[]** for that controller

rt_pic_config_irq(pic, irq_index, hwirq) assigns **irq = pic->irq_start + irq_↩index** and stores hwirq.

1.4.2.82.1.4 End-to-end (device driver view)

```
DTS: device interrupts = <&gic GIC_SPI 77 IRQ_TYPE_LEVEL_HIGH>
     |
     v
rt_dm_dev_get_irq(dev, 0)
     → rt_ofw_get_irq → rt_ofw_map_irq
     → gic irq_parse (hwirq=77+32) + irq_map (virtual irq)
     |
     v
rt_pic_attach_irq(irq, handler, ...)   /* or rt_hw_interrupt_install */
rt_pic_irq_unmask(irq)
```

Secondary PIC (PMIC, PCIe INTx): see PIC cascade and routing.

1.4.2.82.1.5 See also

- OFW interrupts
- PCI INTx interrupts

- PCI MSI and MSI-X

1.4.2.83 PIC virtual IRQ domain

1.4.2.83.1 Virtual IRQ allocation (`pic.c`)

How RT-Thread turns many interrupt controllers into one **global IRQ space** drivers can use.

1.4.2.83.1.1 Global table `_pirq_hash[]`

- Size: **`MAX_HANDLERS`** (Kconfig, default 256)
- Each slot is a **`struct rt_pic_irq`** with **`irq`**, **`hwirq`**, **`pic`**, ISR list, optional **`parent`**

Allocation pointer **`_pirq_hash_idx`**:

Region	Indices	Purpose
IPI	`0 ... RT_MAX_IPI-1`	SMP inter-processor interrupts
PIC domains	`_pirq_hash_idx` upward	Each **`rt_pic_linear_irq`** takes a contiguous block

`rt_pic_config_ipi` uses fixed low indices. Peripheral IRQs come from **`rt_pic_config_irq`**.

1.4.2.83.1.2 Per-controller window

`rt_pic_linear_irq(pic, irq_nr)` (under global lock):

```
pic->irq_start = _pirq_hash_idx;   /* first virtual IRQ for this PIC */
pic->irq_nr    = irq_nr;
pic->pirqs     = &_pirq_hash[pic->irq_start];
_pirq_hash_idx += irq_nr;
```

Example (GICv2 with 988 SPIs):

- **`irq_nr = max_irq + 1 - 32`** (SGI/PPI handled separately)
- Virtual IRQs might be **`[RT_MAX_IPI, RT_MAX_IPI + irq_nr)`**
- **`irq_index = hwirq - 32`** for SPI hwirq 32 → index 0

1.4.2.83.1.3 Mapping API

```
int rt_pic_config_irq(struct rt_pic *pic, int irq_index, int hwirq);
int rt_pic_config_ipi(struct rt_pic *pic, int ipi_index, int hwirq);
```

Input	Meaning
`irq_index`	Offset within this PIC's window
`hwirq`	Hardware ID in that controller
Return value	**Global virtual IRQ** = **`pic->irq_start + irq_index`**

Lookup:

API	Returns
`rt_pic_find_irq(pic, irq_index)`	**`&pic->pirqs[irq_index]`**
`rt_pic_find_pirq(pic, virtual_irq)`	**`pirq`** if **`virtual_irq`** in range
`irq2pirq(irq)`	Global lookup for **`rt_pic_irq_*`** wrappers

1.4.2.83.1.4 `struct rt_pic_irq` fields

Field	Role
`irq`	Virtual IRQ (global); **`-1`** = slot unused
`hwirq`	Controller-specific line
`mode`	Edge/level (`RT_IRQ_MODE_*`)
`pic`	Owning **`struct rt_pic *`**
`parent`	Cascade parent **`pirq`**
`children_nodes`	Routed children if **`RT_PIC_F_IRQ_ROUTING`**
`isr`	Handler chain

1.4.2.83.1.5 IPI vs device IRQ

	IPI	Device IRQ
Config	**`rt_pic_config_ipi`**	**`rt_pic_config_irq`** or **`irq_map`**
Affinity	All CPUs set in **`config_ipi`**	Per **`irq_set_affinity`**
Dispatch	GIC trap reads hwirq $< 16 \rightarrow$ **`rt_pic_find_ipi`**	hwirq $32 \rightarrow$ **`rt_pic_find_irq`**

1.4.2.83.1.6 Debug

With MSH: **`list_irq`** / **`list_irq all`** (**`pic.c`**) dumps virtual IRQ, PIC name, hwirq, mode, handlers. **`rt_pic_linear_irq`** comment: intended for production PIC registration; **`rt_pic_cancel_irq`** only for failed init teardown.

1.4.2.83.1.7 Pitfalls

Issue	Mitigation
`MAX_HANDLERS` too small	Sum all PIC **`irq_nr`** + IPI + headroom
Use hwirq as **`rt_pic_attach_irq` argument**	Always use **virtual `irq`** from **`rt_dm_dev_get_irq`** / **`irq_map`**
Re-`**config_irq**` on busy slot	**`config_pirq`** asserts empty ISR lists if PIC changes

1.4.2.83.1.8 See also

- PIC core framework
- PIC and device tree

1.4.2.84 PIC core framework

1.4.2.84.1 Core framework (`pic.c`, `pic_rthw.c`)

Registration, ISR attach, trap dispatch, and global IRQ control wrappers.

1.4.2.84.1.1 `struct rt_pic` / `struct rt_pic_ops`

`rt_pic_ops`	Role
`irq_init` / `irq_finit`	Controller-wide setup (called from `rt_pic_irq_init`)
`irq_enable` / `irq_disable`	Optional gate
`irq_mask` / `irq_unmask`	Mask line at hardware
`irq_ack` / `irq_eoi`	Acknowledge / end of interrupt
`irq_map`	**Allocate virtual IRQ** from hwirq + mode (required for OFW)
`irq_parse`	Decode `#interrupt-cells` into `out_pirq` (required for OFW)
`irq_set_priority` / `irq_set_affinity` / `irq_set_triger_mode`	Configuration
`irq_send_ipi`	SMP IPI
MSI: `irq_alloc_msi`, `irq_compose_msi_msg`, ...	PIC and PCI MSI
`flags`	`RT_PIC_F_IRQ_ROUTING` — parent dispatches children in `rt_pic_handle_isr`

1.4.2.84.1.2 Trap path (root PIC)

Root PIC (CPU-facing) registers a **trap handler**:

```
rt_pic_add_traps(my_root_handler, pic);
```

CPU exception → `rt_pic_do_traps()` → first handler returning `RT_TRUE`:

1. Read hwirq from hardware (e.g. GIC IAR in `pic-gicv2.c`)
2. Map to `pirq` (IPI vs SPI)
3. `irq_ack` → `rt_pic_handle_isr(pirq)` → `irq_eoi`

Child PICs without traps are reached via **cascade** or **dedicated parent ISR** (PIC cascade and routing).

1.4.2.84.1.3 `rt_pic_handle_isr`

1. If `children_nodes` non-empty: for each child, `irq_ack`, recursive `rt_pic_handle_isr(child)`, `irq_eoi`
2. Call primary `pirq->isr.action.handler(irq, param)`
3. Call chained ISRs on `pirq->isr.list`

Used by root trap and by software demux (e.g. Rockchip PCIe legacy ISR).

1.4.2.84.1.4 Attach / detach

```
rt_err_t rt_pic_attach_irq(int irq, rt_isr_handler_t handler, void *uid,
        const char *name, int flags);
rt_err_t rt_pic_detach_irq(int irq, void *uid);
```

- `irq`: **virtual** global number
- First handler uses embedded `pirq->isr`; additional handlers malloc `struct rt_pic_isr` nodes
- `rt_hw_interrupt_install` in `pic_rthw.c` wraps `rt_pic_attach_irq`

1.4.2.84.1.5 Global IRQ API

Thin wrappers: **`irq2pirq(irq)`** → **`pirq->pic->ops->...`**

```
void rt_pic_irq_mask(int irq);
void rt_pic_irq_unmask(int irq);
void rt_pic_irq_eoi(int irq);
/* ... priority, affinity, trigger mode, IPI ... */
```

`rt_hw_interrupt_mask` / **`umask`** in arch code call these.

1.4.2.84.1.6 Init refcount (phye-style pattern in phye.c — here init/power on PIC is per-op at device level)

For **`rt_phye`**-like refcount see phye; PIC uses **per-`pirq` ISR** and **init_count** only inside individual drivers. PIC core **`rt_phye_init`** N/A — use **`irq_map`** once per DT line.

1.4.2.84.1.7 `rt_pic_register` pattern (provider)

There is no single **`rt_pic_register`** — providers:

1. Embed **`struct rt_pic`** in driver private data
2. Set **`ops`**, **`priv_data`**
3. **`rt_pic_linear_irq`**
4. **`rt_ofw_data(np) = &pic`** (or **`rt_dm_dev_bind_fwdata`**)
5. Optional **`rt_pic_user_extends`**, **`rt_pic_add_traps`**

1.4.2.84.1.8 See also

- PIC virtual IRQ domain
- PIC cascade and routing
- PIC and device tree

1.4.2.85 PIC cascade and routing

1.4.2.85.1 Cascaded interrupt controllers

When a **secondary PIC** (PMIC, GPIO aggregator, PCIe INTx block, GICv2m MSI frame) sits **below** a **parent** line on another PIC.

1.4.2.85.1.1 Concepts

```
Device hwirq  →  Child PIC  →  parent virtual IRQ  →  Root PIC (GIC)  →  CPU
     |              |                  |
 irq_parse      irq_map            cascade link
```

Link	Set by
Child `pirq->hwirq`	Child **`irq_parse`**
Child virtual `irq`	Child **`irq_map`** → **`rt_pic_config_irq`**
Parent line	**`rt_pic_cascade(child_pirq, parent_virtual_irq)`**

1.4.2.85.1.2 `rt_pic_cascade` / `rt_pic_uncascade`

```
rt_err_t rt_pic_cascade(struct rt_pic_irq *pirq, int parent_irq);
```

Step	Effect
`pirq->parent`	Parent **`pirq`** from **`irq2pirq(parent_irq)`**

Step	Effect
Copy	**priority**, **affinity** from parent
RT_PIC_F_IRQ_ROUTING	Insert **pirq** on **parent->children_nodes**

rt_pic_uncascade clears parent link and removes from children list.
parent_irq must be a **virtual IRQ** on the **parent PIC** (often from parent **irq_map**).

1.4.2.85.1.3 Dispatch modes

A — Hardware routing (RT_PIC_F_IRQ_ROUTING)

Parent **rt_pic_handle_isr** walks **children_nodes**, ack/ISR/eoi each child.
Used when parent trap fires for a **shared** line and children are registered in software tree.

B — Dedicated parent ISR (no routing flag)

Parent IRQ has its own handler that **polls** child status and calls **rt_pic_handle_isr** per child.
Rockchip PCIe INTx (`pcie-dw-rockchip.c`):

- GIC **legacy** IRQ → **rockchip_pcie_legacy_isr**
- Reads **PCIE_CLIENT_INTR_STATUS_LEGACY**, per pin **rt_pic_find_irq(&intx_pic, pin)** → **rt_pic_handle_isr**

Child **intx_pic** has **irq_parse**/**irq_map** for PCI **interrupt-map**; cascade to GIC is via **DT** mapping the legacy line, not **rt_pic_cascade**.

C — MSI / MSI-X child PIC

MSI provider allocates a line on the **child** PIC, then wires it to the **parent**:

```
parent_irq = parent_pic->ops->irq_map(parent_pic, parent_hwirq, mode);
irq = rt_pic_config_irq(msi_pic, index, hwirq);
rt_pic_cascade(pirq, parent_irq);
```

PCI sees the **child** virtual IRQ from **irq_alloc_msi**. Hardware delivery is parent-specific — see PIC and PCI MSI.

1.4.2.85.1.4 Parent helpers

After cascade, child drivers may call:

```
rt_pic_irq_parent_mask(pirq);
rt_pic_irq_parent_unmask(pirq);
rt_pic_irq_parent_eoi(pirq);
/* ... */
```

1.4.2.85.1.5 Rockchip RK8xx PMIC IRQ chip

bsp/rockchip/dm/mfd/rk8xx.c — reference cascade:

1. PMIC **rk8xx->irq** (GPIO/IRQ from SoC) → **rk8xx_pmic_isr** wakes thread
2. Child **rk8xx_irqchip**: **rt_pic_linear_irq**, **irq_map** does:
   ```
   irq = rt_pic_config_irq(pic, hwirq, hwirq);
   rt_pic_cascade(pirq, rk8xx_ic->rk8xx->irq);  /* PMIC pin → PMIC sub-IRQ */
   ```
3. **irq_parse**: one cell = sub-hwirq index
4. Thread scans status registers → **rt_pic_handle_isr** per pending bit

DT: **interrupts** on RK8xx children point at **interrupt-controller** sub-node.

1.4.2.85.1.6 Design checklist

Step	Action
1	Parent PIC up (**rt_pic_init**)
2	Child **rt_pic_linear_irq**
3	Child **irq_map** → virtual IRQ
4	**rt_pic_cascade(child_pirq, parent_irq)**
5	**rt_ofw_data(child_ic_np) = child_pic**
6	Parent ISR or **RT_PIC_F_IRQ_ROUTING**

1.4.2.85.1.7 Pitfalls

Issue	Mitigation
Cascade before parent mapped	**irq_map** parent first
Wrong **parent_irq**	Must be virtual IRQ on parent PIC
Level IRQ EOI order	Child clear source before parent **EOI**
Missing routing + no demux ISR	Children never run

1.4.2.85.1.8 See also

- PIC platform examples
- PIC and PCI MSI

1.4.2.86 PIC and device tree

1.4.2.86.1 Device tree integration

How **interrupts** properties become virtual IRQs via **irq_parse** + **irq_map**.
Sources: **pic.c** (ofw_pic_init), **ofw/irq.c** (rt_ofw_map_irq).

1.4.2.86.1.1 Boot: discovering controllers

rt_pic_init() → **ofw_pic_init()**:

```
rt_ofw_foreach_node_by_prop(ic_np, "interrupt-controller")
    rt_ofw_stub_probe_range(ic_np, &_pic_ofw_start, &_pic_ofw_end);
```

RT_PIC_OFW_DECLARE(name, ids, handler) expands to **RT_OFW_STUB_EXPORT(..., pic, handler)** — same mechanism as other OFW stubs, linker section **.rt_ofw_data.stub.***.
Handler (e.g. **gicv2_ofw_init**) must:

1. Map MMIO
2. **rt_pic_linear_irq**
3. **rt_pic_add_traps** (root only)
4. **rt_ofw_data(np) = &pic**

1.4.2.86.1.2 Device IRQ resolution

rt_dm_dev_get_irq(dev, index) → **ofw_api_call(get_irq)** → **rt_ofw_get_irq** → **rt_ofw_map_irq**.

`rt_ofw_map_irq` (core logic)

```
pic = rt_pic_dynamic_cast(rt_ofw_data(ic_np));
pic->ops->irq_parse(pic, irq_args, &pirq);   /* RT_EOK == 0 */
irq = pic->ops->irq_map(pic, pirq.hwirq, pirq.mode);
```

Step	Responsibility
`rt_ofw_parse_irq_cells`	Fill **`rt_ofw_cell_args`** from **`interrupts`**
`irq_parse`	Set **`out_pirq->hwirq`**, **`mode`** (GIC: type + number)
`irq_map`	**`rt_pic_config_irq`**, return **virtual IRQ**

`rt_pic_dynamic_cast`: finds **`struct rt_pic`** embedded after **`rt_device`** or **`rt_object`** (name **`"PIC"`**).

1.4.2.86.1.3 Example: GIC-style `#interrupt-cells` (3 cells)

From **`pic-gicv2.c`** / **`pic-gicv3.c`**:

Cell 0	Cell 1	Cell 2
0 = SPI, **1** = PPI, ...	Interrupt number	Flags → **`RT_IRQ_MODE_*`**

SPI DT number **0** → **hwirq 32** (`args[1] + 32`).
DTS example (Raspberry Pi 4, **`bcm2711-rpi-4-b.dts`**):

```
interrupts = <GIC_SPI 77 IRQ_TYPE_LEVEL_HIGH>;
```

1.4.2.86.1.4 Secondary `interrupt-controller`

Child node must expose **`interrupt-controller`** + **`#interrupt-cells`** and set **`rt_ofw_data`** to its **`struct rt_pic`**.
Consumers use **`interrupt-parent = <&child_ic>`** or inherited parent.
PCI `interrupt-map`: resolves to parent GIC cells via PCI and device tree — still ends in **`rt_ofw_map_irq`**.

1.4.2.86.1.5 Provider requirements for OFW

ops	Required for DT devices
`irq_parse`	Yes
`irq_map`	Yes
`irq_mask` / **`irq_unmask`**	Strongly recommended

Without **`irq_map`**, **`rt_ofw_map_irq`** logs error and fails.

1.4.2.86.1.6 `RT_PIC_OFW_DECLARE` template

```
static rt_err_t my_ic_ofw_init(struct rt_ofw_node *np, const struct rt_ofw_node_id *id)
{
    /* map MMIO, fill my_pic.ops */
    rt_pic_linear_irq(&my_pic, NUM_LINES);
    rt_ofw_data(np) = &my_pic;
    return RT_EOK;
}

static const struct rt_ofw_node_id my_ic_ids[] = {
    { .compatible = "vendor,my-ic" },
    { /* sentinel */ },
};
RT_PIC_OFW_DECLARE(my_ic, my_ic_ids, my_ic_ofw_init);
```

Non-root: no **`rt_pic_add_traps`**; use parent ISR or cascade.

1.4.2.86.1.7 Pitfalls

Issue	Mitigation
`rt_ofw_data` NULL	Provider not probed — **`rt_platform_ofw_request`** before cast
`irq_parse` returns error	**`rt_ofw_map_irq`** fails — check cell count
Wrong **`interrupt-parent`**	Walk **`rt_ofw_find_irq_parent`** in OFW IRQ code
Mix hwirq / virtual in driver	Only use **`rt_dm_dev_get_irq`** return value

1.4.2.86.1.8 See also

- PIC virtual IRQ domain
- OFW interrupts

1.4.2.87 PIC and PCI MSI

1.4.2.87.1 MSI / MSI-X at the PIC layer

PCI **message-signalled interrupts** need an interrupt controller that can **allocate a vector**, **build the MSI message** (address + data), and **deliver** it into the same virtual IRQ space as wired IRQs.
PCI enable path: PCI MSI and MSI-X. Wiring to PIC: this page.

1.4.2.87.1.1 What the MSI-capable PIC must do

Responsibility	Typical `rt_pic_ops`
Hand out a **virtual IRQ** per MSI/MSI-X vector	**`irq_alloc_msi`** / **`irq_free_msi`**
Tell PCI what to write to the device	**`irq_compose_msi_msg`**, **`irq_write_msi_msg`**
Mask / unmask at hardware	Often **`irq_mask`**/**`irq_unmask`** + **`rt_pci_msi_`**∗ helpers
Reach the CPU interrupt architecture	Usually **cascade** to a parent PIC line (PIC cascade and routing) or a **dedicated status ISR** that demuxes bits

The PIC driver does **not** replace PCI config-space MSI capability setup — that stays in **`pci/msi/`**. The PIC side owns **vector backing** and **message programming**.

1.4.2.87.1.2 Call flow (PCI → PIC)

```
pci/msi: rt_pci_msi_setup_irqs(pdev, nvec, type)
     |
     v
pdev->msi_pic->ops->irq_alloc_msi(pic, msi_desc)
     |  reserve hw line / table entry
     |  rt_pic_config_irq → virtual irq
     |  optional rt_pic_cascade(child_pirq, parent_irq)
     |
     v
irq_compose_msi_msg(pirq, &msg)  →  rt_pci_msi_write_msg
     |
     v
Driver: rt_pic_attach_irq(virtual_irq, handler, ...)
```

`struct rt_pci_device::msi_pic` must point at the **`struct rt_pic`** that implements these ops (from DT **`msi-parent`**, **`msi-map`**, or board setup).

1.4.2.87.1.3 Implementing `irq_alloc_msi`

Typical steps inside the provider:

1. Pick a free **hwirq** / table index (bitmap or hardware allocator).

2. **rt_pic_config_irq(pic, index, hwirq)** — obtain **global virtual IRQ** for **rt_pic_attach_irq**.
3. If this PIC is not CPU-root: **parent_irq = parent_pic->ops->irq_map(...)** then **rt_pic_cascade(pirq, parent_irq)** so the parent line fires when MSI is triggered.
4. Store **msi_desc** on **pirq** if the PCI layer needs it for mask/unmask.
5. Return **virtual IRQ** (not hwirq) to **pci/msi/irq.c**.

irq_free_msi: **rt_pic_uncascade**, release index, clear bitmap.

1.4.2.87.1.4 irq_compose_msi_msg

Fill **struct rt_pci_msi_msg**:

Field	Meaning
address_lo / **address_hi**	Doorbell or MSI frame write target
data	Vector data written by device

SoC-specific: address may be a dedicated MSI window or a parent SPI doorbell — implement per TRM, not in the generic framework.

1.4.2.87.1.5 MSI vs MSI-X

	MSI	MSI-X
Vectors	Often power-of-two block, may need **contiguous** virtual IRQs	Per-table-entry allocation common
Table	Config space	Often **BAR-mapped** table — map BAR before unmask
PIC	Same ops; PCI layer passes count / type	Same ops; one **irq_alloc_msi** per entry typical

1.4.2.87.1.6 DT (optional)

Platforms may describe:

- **msi-parent** — phandle to MSI-capable interrupt controller
- **msi-map** / **msi-map-mask** — map PCI requester ID to controller + specifier

Parsing is SoC-specific; result must be **pdev->msi_pic** before **rt_pci_msi_enable**.

1.4.2.87.1.7 Pitfalls

Issue	Mitigation
RT_PCI_MSI disabled	Ops stubbed; PCI MSI APIs no-op
msi_pic NULL	Set in host/PCI probe before enable
Return **hwirq** to PCI	Return **virtual IRQ** from **irq_alloc_msi**
No parent link	MSI never reaches CPU — cascade or demux ISR
MSI-X table unmapped	Enable only after BAR iomap in PCI setup

1.4.2.87.1.8 See also

- PCI MSI and MSI-X
- PIC cascade and routing
- PIC virtual IRQ domain
- PIC core framework

1.4.2.88 PIC platform examples

1.4.2.88.1 Rockchip & Raspberry Pi DM patterns

Concrete BSP references (paths under **rt-thread/bsp/**).

1.4.2.88.1.1 Raspberry Pi 4 — GIC root (dm/dts/bcm2711-rpi-4-b.dts)

```
gicv2: interrupt-controller@40041000 {
    compatible = "arm,gic-400";
    interrupt-controller;
    #interrupt-cells = <3>;
    reg = <0x40041000 0x1000>, <0x40042000 0x2000>;
};
```

- Probed by in-tree **pic-gicv2.c** during **rt_pic_init**
- All devices: **interrupt-parent = <&gicv2>** + **GIC_SPI n type**
- Drivers: **rt_dm_dev_get_irq** → virtual IRQ → **rt_pic_attach_irq**

PCIe **interrupt-map** on host points at **&gicv2 GIC_SPI 143...**.

1.4.2.88.1.2 Raspberry Pi — BCM2835 mailbox (dm/mailbox/mailbox-bcm2835.c)

```
bcm_mbox->irq = rt_dm_dev_get_irq(dev, 0);
rt_hw_interrupt_install(bcm_mbox->irq, bcm2835_mbox_isr, ...);
rt_hw_interrupt_umask(bcm_mbox->irq);
```

Single interrupt line from GIC — no cascade. Shows standard DM consumer pattern.

1.4.2.88.1.3 Rockchip — RK8xx PMIC IRQ (dm/mfd/rk8xx.c)

Piece	Detail
Root	PMIC **rk8xx->irq** from SoC GPIO
Child PIC	**rk8xx_irqchip**, **rt_pic_linear_irq**
Cascade	**rt_pic_cascade(pirq, rk8xx->irq)** in **rk8xx_irq_map**
Demux	Thread reads **status_base** regs → **rt_pic_handle_isr**
DT	**interrupts = <&rk8xx_intc 0>** on sub-devices

Template for **GPIO/PMIC expander** IRQ chips.

1.4.2.88.1.4 Rockchip — PCIe DW INTx (dm/pci/pcie-dw-rockchip.c)

Piece	Detail
Child node	**legacy-interrupt-controller** (OFW child)
Child PIC	**rockchip_intx_ops**, 4 lines (**RT_PCI_INTX_PIN_MAX**)
rt_ofw_data(intx_np) = &intx_pic	PCI **interrupt-map** targets this

Piece	Detail
Parent ISR	**rockchip_pcie_legacy_isr** on **legacy** GIC IRQ
No RT_PIC_F_IRQ_ROUTING	Software poll of legacy status register

PCI function drivers use **rt_pci_ofw_irq_parse_and_map** → maps through **intx_pic** → GIC.

1.4.2.88.1.5 Rockchip DM consumers (simple)

Many drivers only use allocated virtual IRQ:

```
i2c->irq = rt_dm_dev_get_irq(dev, 0);
rt_pic_attach_irq(i2c->irq, handler, i2c, name, flags);
rt_pic_irq_unmask(i2c->irq);
```

Examples: **i2c-rk3x.c**, **spi-rockchip.c**, **mailbox-rockchip.c**.

1.4.2.88.1.6 Choosing a pattern

Hardware	Pattern	Example
Single GIC line per device	**get_irq + attach**	UART, I2C
Aggregated status register	Child PIC + parent ISR poll	RK PCIe INTx
Many lines behind one pin	Child PIC + **cascade**	RK8xx PMIC
Message-signalled	MSI PIC + PCI	See PIC and PCI MSI

1.4.2.88.1.7 See also

- PIC cascade and routing
- PCI and device tree

1.4.2.89 Platform bus

1.4.2.89.1 Platform bus

Most **MMIO** / **on-SoC** peripherals under **RT_USING_DM** bind through the **platform bus**: a **rt_platform_driver** matches a **rt_platform_device**, then **probe** runs after the bus has applied common setup (pinctrl, clocks, power domain, IOMMU).

- Header: **components/drivers/include/drivers/platform.h**
- Bus logic: **components/drivers/core/platform.c**
- OFW enumeration: **components/drivers/core/platform_ofw.c**

1.4.2.89.1.1 Data structures

```
struct rt_platform_device {
    struct rt_device parent;   /* parent.ofw_node, parent.bus, parent.drv */
    int dev_id;
    const char *name;        /* non-OFW matching only */
    const struct rt_ofw_node_id *id;  /* filled by match; use id->data */
    void *priv;              /* optional driver private pointer */
};

struct rt_platform_driver {
    struct rt_driver parent;
    const char *name;
    const struct rt_ofw_node_id *ids;   /* compatible table, sentinel last */
    rt_err_t (*probe)(struct rt_platform_device *pdev);
    rt_err_t (*remove)(struct rt_platform_device *pdev);
    rt_err_t (*shutdown)(struct rt_platform_device *pdev);
};
```

In **probe**, almost all DM helpers take **struct rt_device *dev = &pdev->parent**.

1.4.2.89.1.2 Boot and init order

Init levels (linker sort key, see **rtdef.h**):

Level	Typical use
INIT_CORE_EXPORT("1.0")	**platform_bus_init** — **rt_bus_register(&platform_bus)**
INIT_SUBSYS_EXPORT("1.1")	Early drivers: manual **rt_platform_driver_register()** (e.g. **pin-pl061**, **pinctrl-single**)
INIT_PLATFORM_EXPORT("1.2")	**platform_ofw_device_probe** — walk DT, **rt_platform_device_register**
INIT_DEVICE_EXPORT("3")	**RT_PLATFORM_DRIVER_EXPORT** — registers driver, then probes **all** devices already on the bus

```
INIT_CORE          → platform bus exists
INIT_SUBSYS        → some drivers registered (optional)
INIT_PLATFORM      → DT nodes → platform devices on bus → match + probe if driver already present
INIT_DEVICE        → RT_PLATFORM_DRIVER_EXPORT → register driver → probe every unmatched device
```

Takeaway: DT devices appear at **INIT_PLATFORM**. Drivers using **RT_PLATFORM_DRIVER_EXPORT** register later and then bind via **rt_bus_add_driver** -> **rt_bus_for_each_dev**. Drivers that must run before the bulk DT scan use **INIT_SUBSYS_EXPORT** + manual **rt_platform_driver_register**.

1.4.2.89.1.3 Registering a driver (important)

Method A — RT_PLATFORM_DRIVER_EXPORT (most common)

```
static const struct rt_ofw_node_id my_ofw_ids[] = {
    { .compatible = "vendor,my-device" },
    { /* sentinel */ }
};

static rt_err_t my_probe(struct rt_platform_device *pdev)
{
    struct rt_device *dev = &pdev->parent;
    /* iomap, clk, init hardware, register char/block device */
    return RT_EOK;
}

static rt_err_t my_remove(struct rt_platform_device *pdev)
{
    /* reverse probe: unregister, free */
    return RT_EOK;
}

static struct rt_platform_driver my_driver = {
    .name = "my-device",
    .ids = my_ofw_ids,
    .probe = my_probe,
    .remove = my_remove,
};

RT_PLATFORM_DRIVER_EXPORT(my_driver);
```

Macro expansion (**driver.h**):

```
#define RT_PLATFORM_DRIVER_EXPORT(driver)  RT_DRIVER_EXPORT(driver, platform, BUILIN)

/* → static init calls rt_platform_driver_register(&driver) at INIT_DEVICE_EXPORT */
```

Requirements:

- **name**: driver name string — used for **name-based match** (see below); keep unique.
- **ids**: optional — **compatible** table for DT match; omit or set **RT_NULL** when using **name-only** binding.
- **probe**: return **RT_EOK** on success, **negative rt_err_t** on failure.
- **remove**: optional but recommended if **probe** allocates resources.

Matching: ids (compatible) vs name

platform_match tries **both** (in order):

Step	Condition	Match when
1	**dev->ofw_node** set and **pdrv->ids** non-↩ NULL	**compatible** matches an entry in **ids** → **pdev->id** points at that entry
2	**pdev->name** and **pdrv->name** both set	Strings equal (same pointer or **rt_strcmp == 0**)

Implications:

- **DT devices** (from **INIT_PLATFORM**): normally matched by **ids** / **compatible**. This is what most drivers use.
- **No ids**: set **.ids = RT_NULL** (or omit the field) and match by **name** — typical when the **device** is created in board code with **rt_platform_device_alloc("same-string")** and **pdev->name** equals **pdrv->name**.
- **Both** can be filled: compatible is tried first; if it fails, **name** is still tried (useful only if **pdev->name** is a non-empty string).

Name-only driver + device:

```
static rt_err_t legacy_probe(struct rt_platform_device *pdev) { ... return RT_EOK; }

static struct rt_platform_driver legacy_driver = {
    .name = "board-uart",    /* must equal pdev->name */
    .ids  = RT_NULL,         /* no compatible table */
    .probe = legacy_probe,
};

RT_PLATFORM_DRIVER_EXPORT(legacy_driver);

/* In board init, after bus is up: */
void board_uart_init(void)
{
    struct rt_platform_device *pdev = rt_platform_device_alloc("board-uart");
    pdev->name = "board-uart";
    rt_platform_device_register(pdev);
}
```

Method B — manual rt_platform_driver_register

Used when the driver must register **before INIT_PLATFORM** (dependency ordering) or the file predates the export macro:

```
static int my_drv_register(void)
{
    return rt_platform_driver_register(&my_driver);
}
INIT_SUBSYS_EXPORT(my_drv_register);
```

Same **struct rt_platform_driver** layout as method A.

What rt_platform_driver_register does

1. Sets **pdrv->parent.bus = &platform_bus**
2. Copies **pdrv->name** into **pdrv->parent.parent.name**
3. **rt_driver_register** → **rt_bus_add_driver**
4. **rt_bus_for_each_dev** — for each device without **dev->drv**, **platform_match**; on match, **platform_probe**

Registering the driver **never** creates devices; it only enables binding.

1.4.2.89.1.4 How devices get on the bus

Automatic — device tree (INIT_PLATFORM)

platform_ofw_device_probe walks the tree (root, **/clocks**, **/firmware**, chosen framebuffer, ...). For each **available** child with **compatible** (or a valid node name):

1. **alloc_ofw_platform_device(np)** — **rt_platform_device_alloc**, **pdev->parent.ofw_node = np**, **np->dev = &pdev->parent**
2. **rt_platform_device_register(pdev)** → **rt_bus_add_device** → try all drivers

Bus nodes (`simple-bus`, `simple-mfd`, `arm,amba-bus`, ...): the walker **recurses into children first** without instantiating a device for the bus node itself (unless something else requests it).
Device naming: **ofw_device_rename** — e.g. **fe300000.serial:uart0** from **reg** + node name + alias.

On demand — rt_platform_ofw_request(np)

Called when another driver needs a **provider** (clock, gpio, pinctrl, regulator, phye, ...) before or without global probe:

np->dev	Action
NULL	Allocate platform device, register on bus → match + probe
Set, **dev->drv NULL**	**rt_bus_reload_driver_device** — retry match
Set, **dev->drv set**	**RT_EOK** (already probed)

Manual — non-OFW / board code

```
struct rt_platform_device *pdev = rt_platform_device_alloc("my-uart");
pdev->name = "my-uart";   /* must match pdrv->name for name-based match */
rt_platform_device_register(pdev);
```

Rare on DM + DT boards; used for legacy or test bring-up.

Child node only — rt_platform_ofw_device_probe_child(np)

Registers **one** node as a platform device when it is not under **/** and not already **RT_OFW_F_PLATFORM**. Used by umbrella drivers that expose children explicitly.

1.4.2.89.1.5 Match rules (platform_match)

See **Matching: ids vs name** above. Summary:

1. **OFW** / **DT**: **rt_ofw_node_match(np, pdrv->ids)** when **ids** is set — **pdev->id** → use **pdev->id->data** for SoC variants.
2. **Name**: **pdev->name** vs **pdrv->name** — works without **ids**; primary path for **manually registered** devices.

DT enumeration sets **pdev->name** from **rt_platform_device_alloc("")** (empty); binding those nodes still relies on **compatible** unless board code assigns a non-empty **pdev->name** that matches the driver.
Only **one** driver binds per device. No match → device stays on **dev_list** until a driver registers later.

1.4.2.89.1.6 platform_probe wrapper (before your probe)

The bus runs **before pdrv->probe(pdev)** :

Step	Function	Notes
1	**rt_pin_ctrl_confs_apply_by_name(dev, NULL)** / **pinctrl-0**	Pin control (pinctrl)
2	**rt_ofw_clk_set_defaults(dev->ofw_node)**	Assigned clocks
3	**rt_dm_power_domain_attach(dev, RT_TRUE)**	**-RT_EEMPTY** OK
4	**rt_iommu_attach(dev)**	**-RT_EEMPTY** OK if no IOMMU
5	**pdrv->probe(pdev)**	Your driver

On **probe failure**: IOMMU detach, power domain detach, **dev->drv cleared**.
On **success**: **RT_OFW_F_READLY** set on the OFW node.
Your **probe** should: parse resources (**rt_dm_dev_iomap**, **rt_dm_dev_get_irq**, **rt_clk_get_by_name**, ...) → enable hardware → register the functional **rt_device** (UART, pin, LED, ...). Return negative **rt_err_t** on hard errors and free what you already allocated.

1.4.2.89.1.7 Typical probe / remove pattern

```
static rt_err_t my_probe(struct rt_platform_device *pdev)
{
    rt_err_t err;
    struct rt_device *dev = &pdev->parent;
    struct my_dev *priv = rt_calloc(1, sizeof(*priv));

    if (!priv)
        return -RT_ENOMEM;

    priv->regs = rt_dm_dev_iomap(dev, 0);
    if (!priv->regs) {
        err = -RT_EIO;
        goto err_free;
    }

    priv->clk = rt_clk_get_by_index(dev, 0);
    if (rt_is_err(priv->clk)) {
        err = rt_ptr_err(priv->clk);
        goto err_unmap;
    }

    if ((err = rt_clk_prepare_enable(priv->clk)))
        goto err_clk_put;

    /* register rt_device, set rt_ofw_data for providers */
    rt_dm_dev_bind_fwdata(dev, RT_NULL, priv);

    return RT_EOK;

err_clk_put:
    rt_clk_put(priv->clk);
err_unmap:
    rt_iounmap(priv->regs);
err_free:
    rt_free(priv);
    return err;
}

static rt_err_t my_remove(struct rt_platform_device *pdev)
{
    struct my_dev *priv = rt_dm_dev_get_fwdata(&pdev->parent);
    /* unregister child device, disable clk, iounmap, rt_free(priv) */
    return RT_EOK;
}
```

remove / **shutdown**: bus calls **pdrv->remove**, then **rt_iommu_detach**, **rt_dm_power_domain_detach**, **rt_platform_ofw_free(pdev)** (drops OFW node ref, **rt_free(pdev)**).

1.4.2.89.1.8 pdev fields in practice

Field	Use
&pdev->parent	Pass to **rt_dm_dev_***, **rt_pin_get_named_pin**, **rt_ofw_***
pdev->id	After match: SoC variant via **pdev->id->data** (see **pinctrl-single pcs_ofw_ids**)
pdev->priv	Optional; core does not allocate it — prefer embedding **rt_platform_device** in your private struct or **rt_dm_dev_bind_fwdata**
pdev->parent.ofw_node	DT node for this instance

Common pattern: embed **struct rt_platform_device** as **not** used — instead embed nothing and use **struct rt_device *dev = &pdev->parent** only; provider drivers embed **struct rt_device_pin parent** etc.

1.4.2.89.1.9 Examples

A — DT + `ids` (typical)

Device tree:

```
mydev: device@40010000 {
    compatible = "vendor,my-device";
    reg = <0x40010000 0x100>;
    interrupts = <GIC_SPI 42 IRQ_TYPE_LEVEL_HIGH>;
};
```

Driver:

```
static rt_err_t my_probe(struct rt_platform_device *pdev) { ... }

static const struct rt_ofw_node_id my_ids[] = {
    { .compatible = "vendor,my-device" },
    { },
};

static struct rt_platform_driver my_driver = {
    .name = "vendor-my-device",
    .ids = my_ids,              /* match by compatible */
    .probe = my_probe,
};
RT_PLATFORM_DRIVER_EXPORT(my_driver);
```

`INIT_PLATFORM` creates the device from DT; **`INIT_DEVICE`** registers the driver and **`probe`** runs. No **`rt_platform_device_register`** in board code.

B — name only, no `ids`

No **`compatible`** binding — driver and device share the same **`name`** string:

```
static struct rt_platform_driver board_uart_driver = {
    .name  = "uart0",
    .ids   = RT_NULL,
    .probe = board_uart_probe,
};
RT_PLATFORM_DRIVER_EXPORT(board_uart_driver);

/* board.c */
    struct rt_platform_device *pdev = rt_platform_device_alloc("uart0");
    pdev->name = "uart0";
    rt_platform_device_register(pdev);
```

Use for legacy BSP devices not described in DT, or tests. On DM + full DT boards, prefer **example A**.

1.4.2.89.1.10 When to use platform vs other buses

Use platform	Use another bus
SoC DT node with **`compatible`**, **`reg`**, **`interrupts`**	**PCI** endpoint → **`rt_pci_driver`**
GPIO LED, UART, GPIO controller, syscon, firmware node	**I2C** sensor → **`rt_i2c_driver`** + client
Provider requested via **`rt_platform_ofw_request`**	**SPI** slave → **`rt_spi_driver`**

Platform is the default for **on-chip** devices described in the board DTS.

1.4.2.89.1.11 Pitfalls

Issue	Mitigation
`compatible` typo	No match on DT path; device never probes — check logs
Name mismatch	**`pdev->name`** must equal **`pdrv->name`**; empty **`pdev->name`** does not match
Expect name match on DT node	DT devices need **`ids`** unless you set **`pdev->name`** explicitly
Driver after device is OK	**`RT_PLATFORM_DRIVER_EXPORT`** relies on **`rt_bus_add_driver`** re-walk
Driver too late for early deps	Use **`INIT_SUBSYS_EXPORT`** or split provider driver earlier

Issue	Mitigation
probe success but no driver	Returning **RT_EOK** without work — still marks node **READLY**
Missing **remove**	OFW node freed by bus; leaks if you allocated IRQ/DMA without teardown
Provider not probed	**rt_platform_ofw_request(np)** from consumer before using phandle
pdev->priv unset	Do not assume bus sets it; use **rt_calloc** / **bind_fwdata**
Console not ready in early **probe**	**INIT_DEVICE** is late; use **LOG_D** or defer prints
Re-probe	**rt_bus_reload_driver_device** when driver registers after deferred provider

1.4.2.89.1.12 See also

- Pin control (pinctrl) — applied in **platform_probe** before driver **probe**
- Open Firmware (OFW) — DT nodes, phandles
- Device model helpers (dm.h) — **rt_dm_dev_iomap**, clocks, IRQ
- `documentation/6.components/device-driver/core/bus.md` — generic **rt_bus** / **RT_DRIVER_EXPORT**
- `components/drivers/core/platform.c`, `platform_ofw.c`

1.4.2.90 Power subsystem overview

Power supply Charger (gpio-charger) Board poweroff and restart

1.4.2.90.1 Power (overview)

RT-Thread splits **power** into several layers. They are related but **not interchangeable**:

Layer	Page	Scope
Regulator	Regulator	Shared rails (LDO/DCDC): enable, voltage, refcount
Power domain	Power domain	SoC power islands while system is running
Power supply	Power supply	Battery/charger reporting (`rt_power_supply`)
Board off/restart	Board poweroff and restart	GPIO/syscon **shutdown** and **reset**
System DM power	System power off and reboot (core)	Ordered **rt_dm_power_off_handler** + machine hooks

Reset controller (IP block reset lines) is a different subsystem: Reset controller — not under `components/drivers/power/reset/`.
Sources under **components/drivers/power/**:

Path	Role
supply/	`rt_power_supply` core, daemon, **gpio-charger**, emulator

| ** eset/** | **gpio-poweroff**, **gpio-restart**, **syscon-*** reboot/poweroff |
Power domain providers: **components/drivers/pmdomain/** (see Power domain).

1.4.2.90.1.1 Typical consumer `probe` order

When a platform device comes up (Platform bus):

```
pinctrl (bus)
    → clock defaults (bus)
    → power domain attach (bus)      [optional, -RT_EEMPTY OK]
    → IOMMU attach (bus)             [optional]
    → driver probe:
          regulator get/enable       [your driver]
          reset deassert / clk enable
          rt_dm_dev_iomap
          register functional device
```

For **battery/charger UI** or **PM policy**, drivers register **`rt_power_supply`** (see Power supply).
For **whole-system power off / reboot**, drivers register **`rt_dm_power_off_handler`** (see Board poweroff and restart, System power off and reboot (core)).

1.4.2.90.1.2 Kconfig (`components/drivers/power/`)

Option	Role
`RT_USING_POWER_SUPPLY`	Power supply class + workqueue
`RT_POWER_SUPPLY_DAEMON`	Poll/notify policy thread
`RT_POWER_SUPPLY_CHARGER_GPIO`	**`gpio-charger`**
`RT_POWER_SUPPLY_EMU`	Test battery/charger
`RT_USING_POWER_RESET`	Board poweroff/restart drivers
`RT_POWER_RESET_GPIO_*`, **`RT_POWER_RESET_SYSCON_*`**	In-tree reset drivers

Regulator and power domain have separate Kconfig trees under **`components/drivers/regulator/`** and **`core/power_domain.c`**.

1.4.2.90.1.3 Quick map: what to implement

Goal	Register / API
Report SOC, charging, USB online	**`rt_power_supply_register`** + **`RT_PLATFORM_DRIVER_EXPORT`**
Shutdown PMIC / assert poweroff GPIO	**`rt_dm_power_off_handler(..., SHUTDOWN, ...)`**
Warm reset / recovery mode	**`rt_dm_power_off_handler(..., RESET, ...)`** + **`rt_dm_reboot_mode_register`**
Keep an IP block powered	**`rt_dm_power_domain_attach`** / **`rt_regulator_get`**
Share 3.3 V rail	**`rt_regulator_enable`** / **`put`**

1.4.2.90.1.4 See also

- Platform bus — device/driver binding
- PIN Device — GPIO for chargers and gpio-poweroff
- `components/drivers/power/Kconfig`

1.4.2.90.2 Power supply

1.4.2.90.2.1 Power supply (`rt_power_supply`)

Header: **`components/drivers/include/drivers/power_supply.h`**. Core: **`components/drivers/power/supply/supply.c`**.

Models **batteries, USB ports, mains**, etc. as property objects (Linux power_supply–like). Chargers use the **same class** — Charger (gpio-charger).
Kconfig: **RT_USING_POWER_SUPPLY**, optional **RT_POWER_SUPPLY_DAEMON**.

Architecture

```
Platform driver probe
        |
        v
  Fill struct rt_power_supply (type, properties, ops, dev)
        |
        v
  rt_power_supply_register(psy)
        |
        +-- global list + rt_ofw_data(np) = psy
        +-- optional thermal zone (PROP_TEMP)
        +-- changed_work for notifiers / LED update
        |
        v
  Consumers: rt_power_supply_get_property / notifier / daemon
```

There is **no separate platform bus** for power supplies: the **platform driver** owns **struct rt_platform_device**, and the **rt_power_supply** hangs off **psy->dev = &pdev->parent**.

Registering a power supply (important)

1. Platform driver (RT_PLATFORM_DRIVER_EXPORT)

Bind to DT with **compatible**, then register the supply in **probe**:

```
struct my_psy_priv {
    struct rt_power_supply psy;
    /* hardware state */
};

static enum rt_power_supply_property my_props[] = {
    RT_POWER_SUPPLY_PROP_STATUS,
    RT_POWER_SUPPLY_PROP_CAPACITY,
};

static rt_err_t my_get_property(struct rt_power_supply *psy,
        enum rt_power_supply_property prop, union rt_power_supply_property_val *val)
{
    /* read hardware, fill val->intval or val->strval */
    return RT_EOK;
}

static const struct rt_power_supply_ops my_ops = {
    .get_property = my_get_property,
};

static rt_err_t my_probe(struct rt_platform_device *pdev)
{
    struct rt_device *dev = &pdev->parent;
    struct my_psy_priv *priv = rt_calloc(1, sizeof(*priv));

    priv->psy.dev = dev;
    priv->psy.type = RT_POWER_SUPPLY_TYPE_BATTERY;
    priv->psy.properties = my_props;
    priv->psy.properties_nr = RT_ARRAY_SIZE(my_props);
    priv->psy.ops = &my_ops;

    return rt_power_supply_register(&priv->psy);
}

static rt_err_t my_remove(struct rt_platform_device *pdev)
{
    struct my_psy_priv *priv = rt_dm_dev_get_fwdata(&pdev->parent);
    rt_power_supply_unregister(&priv->psy);
    rt_free(priv);
    return RT_EOK;
}
```

Reference: **gpio-charger.c** — **RT_PLATFORM_DRIVER_EXPORT(gpio_charger_driver)**.

2. `rt_power_supply_register` requirements

Field	Required
psy->dev	Yes — usually **&pdev->parent**
psy->ops + **properties** + **properties_nr**	Yes, unless **battery_info** is set (static battery descriptor path)
psy->type	Set before register (USB type, battery, mains, ...)
psy->battery_info	Alternative to dynamic properties for some battery profiles

On success:

- Inserts **psy** into global list
- **rt_dm_dev_bind_fwdata(psy->dev, NULL, psy)** when **dev->ofw_node** is set
- Initializes **changed_work** for **rt_power_supply_changed()**

rt_power_supply_unregister: removes from list; fails with **-RT_EBUSY** if **rt_power_supply_get** refcount > 1.

3. Optional: `set_property`

Implement **ops->set_property** for writable limits (e.g. charge current). Only properties listed in **properties[]** are exposed via **rt_power_supply_set_property**.

4. Notify userspace / daemon / LED

After hardware state changes (GPIO IRQ, PMIC interrupt):

```
rt_power_supply_changed(psy);
```

This queues **power_supply_changed_work**, which:

- Updates bound **led_dev** if **RT_USING_LED**
- Calls all **rt_power_supply_notifier** callbacks

Register a notifier:

```
static rt_err_t my_notify(struct rt_power_supply_notifier *n,
        struct rt_power_supply *psy) { ... return RT_EOK; }

static struct rt_power_supply_notifier n = {
    .callback = my_notify,
};
rt_power_supply_notifier_register(&n);
```

RT_POWER_SUPPLY_DAEMON: built-in notifier drives low-battery / PM hints when **RT_USING_PM**.

Property API

```
rt_err_t rt_power_supply_get_property(psy, prop, &val);
rt_err_t rt_power_supply_set_property(psy, prop, &val);
```

get_property only succeeds if **prop** is in **psy->properties[]** (or **battery_info** path for supported static props).
Common properties: **STATUS**, **CAPACITY**, **ONLINE**, **VOLTAGE_NOW**, **CURRENT_NOW**, **TEMP**, **CONSTANT_CHARGE_CURRENT_MAX**, ... — see enum in **power_supply.h**.

Lookup from another device

```
struct rt_power_supply *psy = rt_power_supply_get(consumer_dev, "battery");
if (rt_is_err(psy))
    ...
rt_power_supply_get_property(psy, RT_POWER_SUPPLY_PROP_CAPACITY, &val);
rt_power_supply_put(psy);
```

id is a **DT phandle property name** on **consumer_dev** (e.g. **power-supply = <&bat> "power-supply"**). Resolves **rt_ofw_data** on the supply's OFW node after register.

DT consumer example

```
thermal-zone {
    polling-delay = <1000>;
    thermal-sensors = <&bat>;
};

bat: battery {
    compatible = "vendor,fuel-gauge";
    monitored-battery;
};
```

Fuel-gauge driver **probe** registers **rt_power_supply** on node **bat**. Thermal references **&bat** via **rt_power_supply_thermal_register** when **PROP_TEMP** exists.

Integration

Subsystem	Hook
LED	Assign **psy->led_dev** before register; **changed** updates indicators
Thermal	**power_supply_thermal_register** if **PROP_TEMP**
Regulator	Charger **set_property** may adjust rails (see **gpio-charger**)
PM	Daemon may request sleep modes on low SOC

Pitfalls

Issue	Mitigation
Register before **psy->dev** set	**register** returns **-RT_EINVAL**
Empty **properties**	Must list every prop **get_property** handles
Forget **rt_power_supply_changed**	Notifiers/LED/PM stay stale
unregister with active get	**put** all references first
get_property from ISR	Defer to workqueue; gauge I2C may block
Stale SOC cache	Re-read hardware in **get_property**

See also

- Power subsystem overview — stack overview
- Charger (gpio-charger) — **gpio-charger**
- Platform bus — driver export
- `components/drivers/power/supply/supply.c`

1.4.2.90.3 Charger (gpio-charger)

1.4.2.90.3.1 Charger as rt_power_supply

Chargers are **not** a separate class. A charger is a **struct rt_power_supply** with USB/mains **type** and properties such as **ONLINE**, **STATUS**, **CONSTANT_CHARGE_CURRENT_MAX**.
Core registration flow: Power supply.
Kconfig: **RT_POWER_SUPPLY_CHARGER_GPIO** → **components/drivers/power/supply/gpio-charger.c**.

Registration flow (`gpio-charger`)

```
RT_PLATFORM_DRIVER_EXPORT(gpio_charger_driver)
        |
        v
gpio_charger_probe(pdev)
        |
        +-- rt_pin_get_named_pin (VBUS / status GPIOs)
        +-- optional charge-current-limit GPIOs + mapping table
        +-- parse charger-type → psy->type
        +-- build gpio_charger_properties[] (only supported props)
        +-- psy->dev = &pdev->parent; psy->ops = &gpio_charger_ops
        +-- rt_power_supply_register(&gpioc->parent)
        +-- GPIO IRQ → gpio_charger_isr → rt_power_supply_changed()
```

Device tree (typical)

```
charger {
    compatible = "gpio-charger";
    charger-type = "usb-dcp";
    gpios = <&gpio0 12 GPIO_ACTIVE_HIGH>;          /* online */
    charge-status-gpios = <&gpio0 13 GPIO_ACTIVE_LOW>;
    charge-current-limit-gpios = <&gpio0 14>, <&gpio0 15>;
    charge-current-limit-mapping = <100000 0>, <500000 1>, <1500000 3>;
};
```

Properties actually exposed depend on which GPIOs/mapping are present — the driver builds **`properties[]`** dynamically in **`probe`**.

`get_property` / `set_property`

Property	Behavior
`ONLINE`	Read VBUS GPIO
`STATUS`	Charging vs not from **`charge-status`** GPIO
`CONSTANT_CHARGE_CURRENT_MAX`	Mapping table + limit GPIOs; **`set_property`** selects limit

Emulator

`emu-power.c` (**`RT_POWER_SUPPLY_EMU`**): registers test battery + charger; MSH **`emu_charger`** toggles state for thermal/PM tests.

See also

- Power supply
- Regulator
- PIN Device
- `components/drivers/power/supply/gpio-charger.c`

1.4.2.90.4 Board poweroff and restart

1.4.2.90.4.1 Board-level shutdown and restart

`components/drivers/power/reset/` — drivers that hook **system-wide** power off or CPU reset, not per-IP reset lines (Reset controller).
Kconfig: **`RT_USING_POWER_RESET`**, plus **`RT_POWER_RESET_GPIO_`**∗ or **`RT_POWER_RESET_SYSCON`**↩ **`_`**∗.
API header: **`components/drivers/include/drivers/core/power.h`** — details in System power off and reboot (core).

Registration model

These drivers are **platform drivers** whose **probe** registers a **callback** with the DM power core. They do **not** register a char device for normal I/O.

```
RT_PLATFORM_DRIVER_EXPORT(gpio_poweroff_driver)
        |
        v
gpio_poweroff_probe(pdev)
        |
        +-- parse GPIO (rt_pin_get_named_pin) or syscon reg
        +-- rt_dm_power_off_handler(dev, MODE, PRIORITY, callback)
        |
        v
On rt_hw_cpu_shutdown() / rt_hw_cpu_reset():
        walk handlers by priority → callback → rt_dm_machine_shutdown/reset
```

rt_dm_power_off_handler

```
rt_err_t rt_dm_power_off_handler(struct rt_device *dev, int mode, int priority,
        rt_err_t (*callback)(struct rt_device *));
```

mode	When invoked
RT_DM_POWER_OFF_MODE_SHUTDOWN	System power off (**rt_hw_cpu_shutdown**)
RT_DM_POWER_OFF_MODE_RESET	Warm reset path (**rt_hw_cpu_reset**)

priority (low → high)	Typical use
RT_DM_POWER_OFF_PRIO_PLATFORM	SoC flush
RT_DM_POWER_OFF_PRIO_LOW / **DEFAULT** / **HIGH**	Drivers, filesystem sync
RT_DM_POWER_OFF_PRIO_FIRMWARE	Last handoff

Callbacks run in order; non-**RT_EOK** is logged but later handlers still run.
dev: passed to callback as **dev->user_data** context holder (GPIO drivers store private struct in **user_data**).

In-tree drivers

Driver	Compatible / binding	Registration
gpio-poweroff	**gpio-poweroff**	**RT_DM_POWER_OFF_MODE_SHUTDOWN**, **rt_pin_get_named_pin**
gpio-restart	**gpio-restart**	**RT_DM_POWER_OFF_MODE_RESET**
syscon-poweroff	syscon reg	MMIO write in callback
syscon-reboot	syscon reg	reset trigger
syscon-reboot-mode	NVMEM + syscon	**rt_dm_reboot_mode_register**

All use **RT_PLATFORM_DRIVER_EXPORT** except some syscon drivers use **INIT_SUBSYS_EXPORT** + **rt_platform_driver_register** (same binding, different init level — see Platform bus).

GPIO poweroff example (flow)

```
static rt_err_t gpio_poweroff_do_poweroff(struct rt_device *dev)
{
    struct gpio_poweroff *gp = dev->user_data;
    /* assert GPIO sequence, mdelay per DT timeout-ms */
    return RT_EOK;
}

static rt_err_t gpio_poweroff_probe(struct rt_platform_device *pdev)
{
```

```
    ...
    dev->user_data = gp;
    return rt_dm_power_off_handler(dev, RT_DM_POWER_OFF_MODE_SHUTDOWN,
            RT_DM_POWER_OFF_PRIO_DEFAULT, gpio_poweroff_do_poweroff);
}
RT_PLATFORM_DRIVER_EXPORT(gpio_poweroff_driver);
```

DT:

```
poweroff {
    compatible = "gpio-poweroff";
    gpios = <&gpio0 99 GPIO_ACTIVE_HIGH>;
    timeout-ms = <3000>;
    active-delay-ms = <100>;
    inactive-delay-ms = <100>;
};
```

Reboot mode

```
rt_dm_reboot_mode_register(dev, reboot_mode_callback);
rt_hw_cpu_reset_mode("recovery");  /* sets cmd + rt_hw_cpu_reset() */
```

syscon-reboot-mode persists mode to NVMEM/syscon so the next boot loader can read it. See **reboot-mode.h**, System power off and reboot (core).

Machine hooks

BSP sets after handlers:

```
rt_dm_machine_shutdown = board_shutdown;  /* PMIC off */
rt_dm_machine_reset = board_reset;        /* PSCI / watchdog */
```

Without these, callbacks may run but SoC never powers off.

Pitfalls

Issue	Mitigation
Wrong GPIO polarity	Match **gpios** flags and **active_value** from **rt_pin_get_named_pin**
RT_USING_PINCTRL off for gpio-poweroff	Kconfig depends on pinctrl for pad mux
Confuse with **rt_reset_control**	IP reset lines system poweroff
Handler sleeps in IRQ context	GPIO driver uses **rt_thread_mdelay** — ensure thread context
No **rt_dm_machine_shutdown**	Set BSP hook

See also

- System power off and reboot (core) — handler lists, priorities
- Power subsystem overview — overview
- Platform bus
- Reset controller — reset **controller** API (different)
- components/drivers/power/reset/

1.4.2.90.5 Power supply

1.4.2.90.5.1 Power supply (rt_power_supply)

Header: **components/drivers/include/drivers/power_supply.h**. Core: **components/drivers/power/supply/supply.c**.
Models **batteries, USB ports, mains**, etc. as property objects (Linux power_supply–like). Chargers use the **same class** — Charger (gpio-charger).
Kconfig: **RT_USING_POWER_SUPPLY**, optional **RT_POWER_SUPPLY_DAEMON**.

Architecture

```
Platform driver probe
        |
        v
  Fill struct rt_power_supply (type, properties, ops, dev)
        |
        v
  rt_power_supply_register(psy)
        |
        +-- global list + rt_ofw_data(np) = psy
        +-- optional thermal zone (PROP_TEMP)
        +-- changed_work for notifiers / LED update
        |
        v
  Consumers: rt_power_supply_get_property / notifier / daemon
```

There is **no separate platform bus** for power supplies: the **platform driver** owns **`struct rt_platform_device`**, and the **`rt_power_supply`** hangs off **`psy->dev = &pdev->parent`**.

Registering a power supply (important)

1. Platform driver (`RT_PLATFORM_DRIVER_EXPORT`)

Bind to DT with **`compatible`**, then register the supply in **`probe`**:

```
struct my_psy_priv {
    struct rt_power_supply psy;
    /* hardware state */
};

static enum rt_power_supply_property my_props[] = {
    RT_POWER_SUPPLY_PROP_STATUS,
    RT_POWER_SUPPLY_PROP_CAPACITY,
};

static rt_err_t my_get_property(struct rt_power_supply *psy,
        enum rt_power_supply_property prop, union rt_power_supply_property_val *val)
{
    /* read hardware, fill val->intval or val->strval */
    return RT_EOK;
}

static const struct rt_power_supply_ops my_ops = {
    .get_property = my_get_property,
};

static rt_err_t my_probe(struct rt_platform_device *pdev)
{
    struct rt_device *dev = &pdev->parent;
    struct my_psy_priv *priv = rt_calloc(1, sizeof(*priv));

    priv->psy.dev = dev;
    priv->psy.type = RT_POWER_SUPPLY_TYPE_BATTERY;
    priv->psy.properties = my_props;
    priv->psy.properties_nr = RT_ARRAY_SIZE(my_props);
    priv->psy.ops = &my_ops;

    return rt_power_supply_register(&priv->psy);
}

static rt_err_t my_remove(struct rt_platform_device *pdev)
{
    struct my_psy_priv *priv = rt_dm_dev_get_fwdata(&pdev->parent);
    rt_power_supply_unregister(&priv->psy);
    rt_free(priv);
    return RT_EOK;
}
```

Reference: **`gpio-charger.c`** — **`RT_PLATFORM_DRIVER_EXPORT(gpio_charger_driver)`**.

2. `rt_power_supply_register` requirements

Field	Required
`psy->dev`	Yes — usually **`&pdev->parent`**
`psy->ops` + **`properties`** + **`properties_nr`**	Yes, unless **`battery_info`** is set (static battery descriptor path)

Field	Required
psy->type	Set before register (USB type, battery, mains, ...)
psy->battery_info	Alternative to dynamic properties for some battery profiles

On success:

- Inserts **psy** into global list
- **rt_dm_dev_bind_fwdata(psy->dev, NULL, psy)** when **dev->ofw_node** is set
- Initializes **changed_work** for **rt_power_supply_changed()**

rt_power_supply_unregister: removes from list; fails with **-RT_EBUSY** if **rt_power_supply_get** refcount > 1.

3. Optional: set_property

Implement **ops->set_property** for writable limits (e.g. charge current). Only properties listed in **properties[]** are exposed via **rt_power_supply_set_property**.

4. Notify userspace / daemon / LED

After hardware state changes (GPIO IRQ, PMIC interrupt):

```
rt_power_supply_changed(psy);
```

This queues **power_supply_changed_work**, which:

- Updates bound **led_dev** if **RT_USING_LED**
- Calls all **rt_power_supply_notifier** callbacks

Register a notifier:

```
static rt_err_t my_notify(struct rt_power_supply_notifier *n,
        struct rt_power_supply *psy) { ... return RT_EOK; }

static struct rt_power_supply_notifier n = {
    .callback = my_notify,
};
rt_power_supply_notifier_register(&n);
```

RT_POWER_SUPPLY_DAEMON: built-in notifier drives low-battery / PM hints when **RT_USING_PM**.

Property API

```
rt_err_t rt_power_supply_get_property(psy, prop, &val);
rt_err_t rt_power_supply_set_property(psy, prop, &val);
```

get_property only succeeds if **prop** is in **psy->properties[]** (or **battery_info** path for supported static props).
Common properties: **STATUS**, **CAPACITY**, **ONLINE**, **VOLTAGE_NOW**, **CURRENT_NOW**, **TEMP**, **CONSTANT_CHARGE_CURRENT_MAX**, ... — see enum in **power_supply.h**.

Lookup from another device

```
struct rt_power_supply *psy = rt_power_supply_get(consumer_dev, "battery");
if (rt_is_err(psy))
    ...
rt_power_supply_get_property(psy, RT_POWER_SUPPLY_PROP_CAPACITY, &val);
rt_power_supply_put(psy);
```

id is a **DT phandle property name** on **consumer_dev** (e.g. **power-supply = <&bat> → "power-supply"**). Resolves **rt_ofw_data** on the supply's OFW node after register.

DT consumer example

```
thermal-zone {
    polling-delay = <1000>;
    thermal-sensors = <&bat>;
};

bat: battery {
    compatible = "vendor,fuel-gauge";
    monitored-battery;
};
```

Fuel-gauge driver **probe** registers **rt_power_supply** on node **bat**. Thermal references **&bat** via **rt_power_supply_thermal_register** when **PROP_TEMP** exists.

Integration

Subsystem	Hook
LED	Assign **psy->led_dev** before register; **changed** updates indicators
Thermal	**power_supply_thermal_register** if **PROP_TEMP**
Regulator	Charger **set_property** may adjust rails (see **gpio-charger**)
PM	Daemon may request sleep modes on low SOC

Pitfalls

Issue	Mitigation
Register before **psy->dev** set	**register** returns **-RT_EINVAL**
Empty **properties**	Must list every prop **get_property** handles
Forget **rt_power_supply_changed**	Notifiers/LED/PM stay stale
unregister with active get	**put** all references first
get_property from ISR	Defer to workqueue; gauge I2C may block
Stale SOC cache	Re-read hardware in **get_property**

See also

- Power subsystem overview — stack overview
- Charger (gpio-charger) — **gpio-charger**
- Platform bus — driver export
- `components/drivers/power/supply/supply.c`

1.4.2.90.6 Charger (gpio-charger)

1.4.2.90.6.1 Charger as rt_power_supply

Chargers are **not** a separate class. A charger is a **struct rt_power_supply** with USB/mains **type** and properties such as **ONLINE**, **STATUS**, **CONSTANT_CHARGE_CURRENT_MAX**.
Core registration flow: Power supply.
Kconfig: **RT_POWER_SUPPLY_CHARGER_GPIO** → **components/drivers/power/supply/gpio-charger.c**.

Registration flow (`gpio-charger`)

```
RT_PLATFORM_DRIVER_EXPORT(gpio_charger_driver)
        |
        v
gpio_charger_probe(pdev)
        |
        +-- rt_pin_get_named_pin (VBUS / status GPIOs)
        +-- optional charge-current-limit GPIOs + mapping table
        +-- parse charger-type → psy->type
        +-- build gpio_charger_properties[] (only supported props)
        +-- psy->dev = &pdev->parent; psy->ops = &gpio_charger_ops
        +-- rt_power_supply_register(&gpioc->parent)
        +-- GPIO IRQ → gpio_charger_isr → rt_power_supply_changed()
```

Device tree (typical)

```
charger {
    compatible = "gpio-charger";
    charger-type = "usb-dcp";
    gpios = <&gpio0 12 GPIO_ACTIVE_HIGH>;          /* online */
    charge-status-gpios = <&gpio0 13 GPIO_ACTIVE_LOW>;
    charge-current-limit-gpios = <&gpio0 14>, <&gpio0 15>;
    charge-current-limit-mapping = <100000 0>, <500000 1>, <1500000 3>;
};
```

Properties actually exposed depend on which GPIOs/mapping are present — the driver builds **`properties[]`** dynamically in **`probe`**.

`get_property` / `set_property`

Property	Behavior
`ONLINE`	Read VBUS GPIO
`STATUS`	Charging vs not from **`charge-status`** GPIO
`CONSTANT_CHARGE_CURRENT_MAX`	Mapping table + limit GPIOs; **`set_property`** selects limit

Emulator

`emu-power.c` (**`RT_POWER_SUPPLY_EMU`**): registers test battery + charger; MSH **`emu_charger`** toggles state for thermal/PM tests.

See also

- Power supply
- Regulator
- PIN Device
- `components/drivers/power/supply/gpio-charger.c`

1.4.2.90.7 Board poweroff and restart

1.4.2.90.7.1 Board-level shutdown and restart

`components/drivers/power/reset/` — drivers that hook **system-wide** power off or CPU reset, not per-IP reset lines (Reset controller).
Kconfig: **`RT_USING_POWER_RESET`**, plus **`RT_POWER_RESET_GPIO_`**∗ or **`RT_POWER_RESET_SYSCON`**↩ **`_`**∗.
API header: **`components/drivers/include/drivers/core/power.h`** — details in System power off and reboot (core).

Registration model

These drivers are **platform drivers** whose **probe** registers a **callback** with the DM power core. They do **not** register a char device for normal I/O.

```
RT_PLATFORM_DRIVER_EXPORT(gpio_poweroff_driver)
        |
        v
gpio_poweroff_probe(pdev)
        |
        +-- parse GPIO (rt_pin_get_named_pin) or syscon reg
        +-- rt_dm_power_off_handler(dev, MODE, PRIORITY, callback)
        |
        v
On rt_hw_cpu_shutdown() / rt_hw_cpu_reset():
        walk handlers by priority → callback → rt_dm_machine_shutdown/reset
```

rt_dm_power_off_handler

```
rt_err_t rt_dm_power_off_handler(struct rt_device *dev, int mode, int priority,
        rt_err_t (*callback)(struct rt_device *));
```

mode	When invoked
RT_DM_POWER_OFF_MODE_SHUTDOWN	System power off (**rt_hw_cpu_shutdown**)
RT_DM_POWER_OFF_MODE_RESET	Warm reset path (**rt_hw_cpu_reset**)

priority (low → high)	Typical use
RT_DM_POWER_OFF_PRIO_PLATFORM	SoC flush
RT_DM_POWER_OFF_PRIO_LOW / **DEFAULT** / **HIGH**	Drivers, filesystem sync
RT_DM_POWER_OFF_PRIO_FIRMWARE	Last handoff

Callbacks run in order; non-**RT_EOK** is logged but later handlers still run.
dev: passed to callback as **dev->user_data** context holder (GPIO drivers store private struct in **user_data**).

In-tree drivers

Driver	Compatible / binding	Registration
gpio-poweroff	**gpio-poweroff**	**RT_DM_POWER_OFF_MODE_SHUTDOWN**, **rt_pin_get_named_pin**
gpio-restart	**gpio-restart**	**RT_DM_POWER_OFF_MODE_RESET**
syscon-poweroff	syscon reg	MMIO write in callback
syscon-reboot	syscon reg	reset trigger
syscon-reboot-mode	NVMEM + syscon	**rt_dm_reboot_mode_register**

All use **RT_PLATFORM_DRIVER_EXPORT** except some syscon drivers use **INIT_SUBSYS_EXPORT** + **rt_platform_driver_register** (same binding, different init level — see Platform bus).

GPIO poweroff example (flow)

```
static rt_err_t gpio_poweroff_do_poweroff(struct rt_device *dev)
{
    struct gpio_poweroff *gp = dev->user_data;
    /* assert GPIO sequence, mdelay per DT timeout-ms */
    return RT_EOK;
}

static rt_err_t gpio_poweroff_probe(struct rt_platform_device *pdev)
{
```

```
    ...
    dev->user_data = gp;
    return rt_dm_power_off_handler(dev, RT_DM_POWER_OFF_MODE_SHUTDOWN,
            RT_DM_POWER_OFF_PRIO_DEFAULT, gpio_poweroff_do_poweroff);
}
RT_PLATFORM_DRIVER_EXPORT(gpio_poweroff_driver);
```

DT:

```
poweroff {
    compatible = "gpio-poweroff";
    gpios = <&gpio0 99 GPIO_ACTIVE_HIGH>;
    timeout-ms = <3000>;
    active-delay-ms = <100>;
    inactive-delay-ms = <100>;
};
```

Reboot mode

```
rt_dm_reboot_mode_register(dev, reboot_mode_callback);
rt_hw_cpu_reset_mode("recovery");  /* sets cmd + rt_hw_cpu_reset() */
```

syscon-reboot-mode persists mode to NVMEM/syscon so the next boot loader can read it. See **reboot-mode.h**, System power off and reboot (core).

Machine hooks

BSP sets after handlers:

```
rt_dm_machine_shutdown = board_shutdown;  /* PMIC off */
rt_dm_machine_reset = board_reset;        /* PSCI / watchdog */
```

Without these, callbacks may run but SoC never powers off.

Pitfalls

Issue	Mitigation
Wrong GPIO polarity	Match **gpios** flags and **active_value** from **rt_pin_get_named_pin**
RT_USING_PINCTRL off for gpio-poweroff	Kconfig depends on pinctrl for pad mux
Confuse with **rt_reset_control**	IP reset lines system poweroff
Handler sleeps in IRQ context	GPIO driver uses **rt_thread_mdelay** — ensure thread context
No **rt_dm_machine_shutdown**	Set BSP hook

See also

- System power off and reboot (core) — handler lists, priorities
- Power subsystem overview — overview
- Platform bus
- Reset controller — reset **controller** API (different)
- components/drivers/power/reset/

1.4.2.91 Power domain

1.4.2.91.1 Power domain (rt_dm_power_domain)

Header: **components/drivers/include/drivers/core/power_domain.h**. Core: **components/drivers/core/power_domain.c**.
A **power domain** models a switchable SoC power island (GPU block, USB PHY, display cluster, . . .). Consumers reference domains in DT via **power-domains**; the **provider** driver registers **struct rt_dm_power_domain** (or a **proxy**) and implements **power_on** / **power_off**.

This is **not** system shutdown (System power off and reboot (core)) and **not** a regulator rail (Regulator).
In-tree provider example: **components/drivers/pmdomain/pm-domain-scmi.c** (SCMI **genpd**).

1.4.2.91.1.1 End-to-end flow

Consumer (device with power-domains in DT)

```
platform_probe (platform.c)
        |
        v
rt_dm_power_domain_attach(dev, RT_TRUE)
        |
        +-- Parse first phandle in "power-domains" (index 0 only in attach)
        +-- Resolve provider rt_ofw_data → domain or proxy->ofw_parse()
        +-- domain->attach_dev(domain, dev) [optional]
        +-- if on: rt_dm_power_domain_power_on(domain)
        |
        v
pdrv->probe(pdev)    /* MMIO safe after domain on */
```

On **remove**/**shutdown** / failed probe:

```
rt_dm_power_domain_detach(dev, RT_TRUE)   /* optional power_off */
```

-RT_EEMPTY: no **power-domains** property — Platform bus treats this as OK.

Provider (controller / firmware)

```
Driver probe (platform / SCMI)
        |
        v
Per domain: fill power_on, power_off [, attach_dev, detach_dev]
        |
        v
rt_dm_power_domain_register(domain)
        |
        +-- For DT phandle resolution: rt_ofw_data(provider_np) = domain
        |     OR proxy + rt_dm_power_domain_proxy_ofw_bind(np)
        |
        v
Consumer attach / get_by_index → power_on when ref transitions 0→1 usage
```

1.4.2.91.1.2 Device tree

Consumer:

```
gpu@0x10000000 {
    compatible = "vendor,gpu";
    reg = <0x10000000 0x10000>;
    power-domains = <&pd_gpu>;
    /* optional: */
    power-domain-names = "core";
};
```

Property	Role
power-domains	Phandle(s) to provider; cells per **#power-domain-cells**
power-domain-names	Maps names to index for **rt_dm_power_domain_get_by_name**

Provider node must expose **#power-domain-cells** and bind **rt_ofw_data** to either:

rt_ofw_data object name	Type
PMD (RT_POWER_DOMAIN_OBJ_NAME)	Direct **struct rt_dm_power_domain**
PMP (RT_POWER_DOMAIN_PROXY_OBJ_NAME)	**struct rt_dm_power_domain_proxy** with **ofw_parse**

OFW core registers these in **ofw.c** object table.

1.4.2.91.1.3 Registering a provider (important)

Direct domain

```
struct my_pd {
    struct rt_dm_power_domain domain;
    void __iomem *regs;
};

static rt_err_t my_pd_power_on(struct rt_dm_power_domain *domain)
{
    struct my_pd *pd = rt_container_of(domain, struct my_pd, domain);
    /* enable island -- may sleep (I2C/SCMI/MMIO) */
    return RT_EOK;
}

static rt_err_t my_pd_power_off(struct rt_dm_power_domain *domain)
{
    ...
    return RT_EOK;
}

static rt_err_t my_pd_probe(struct rt_platform_device *pdev)
{
    struct rt_device *dev = &pdev->parent;
    struct my_pd *pd = rt_calloc(1, sizeof(*pd));

    pd->domain.dev = dev;
    pd->domain.power_on = my_pd_power_on;
    pd->domain.power_off = my_pd_power_off;

    rt_dm_power_domain_register(&pd->domain);
    rt_ofw_data(dev->ofw_node) = &pd->domain;   /* object name becomes "PMD" */

    return RT_EOK;
}
```

`rt_dm_power_domain_register`: initializes lists, **`ref`**, spinlock — does **not** touch hardware.

Proxy (multiple domains per node)

When one DT node represents many domains (SCMI, generic genpd):

```
static struct rt_dm_power_domain *my_proxy_parse(
        struct rt_dm_power_domain_proxy *proxy, struct rt_ofw_cell_args *args)
{
    return &domains[args->args[0]];   /* cell index selects domain */
}

/* probe: register each domain, then */
proxy->ofw_parse = my_proxy_parse;
rt_dm_power_domain_proxy_ofw_bind(proxy, provider_ofw_node);
```

Reference: **`scmi_pm_domain_probe`** — registers **`num_domains`** entries, binds proxy on SCMI device node.

Child domains

```
rt_dm_power_domain_register_child(parent, child);
```

Sets **`child->parent_domain`**. **`power_on(parent)`** recursively powers on children listed in **`parent->child_nodes`** (BSP must link children into that list if hierarchy is used). **`unregister_child`** removes the child when the parent is idle.

1.4.2.91.1.4 Consumer API

API	Role
`rt_dm_power_domain_attach(dev, on)`	Link **`dev`** to **first** **`power-domains`** entry; **`on==RT_TRUE`** → **`power_on`**
`rt_dm_power_domain_detach(dev, off)`	Unlink; **`off==RT_TRUE`** → **`power_off`**
`rt_dm_power_domain_get_by_index(dev, index)`	Resolve domain pointer (no refcount change in **`put`** today)
`rt_dm_power_domain_get_by_name(dev, name)`	Uses **`power-domain-names`**

API	Role
rt_dm_power_domain_put(domain)	Currently no-op success — hold domain via attach lifecycle
rt_dm_power_domain_power_on/↩ off(domain)	Explicit on/off with **ref** counting

Reference counting

- **rt_ref_init**: domain starts at refcount **1** (idle / registered).
- **power_on**: if **ref == 1**, calls hardware **power_on**, then **ref_get**.
- **power_off**: **ref_put**; when **ref returns to 1**, calls hardware **power_off**.

Multiple consumers should share one domain through **attach** + refcount, not duplicate **power_on** without coordination.

Manual use in driver

Usually **unnecessary** — platform bus already calls **attach(dev, RT_TRUE)**.
Extra domains (index > 0):

```
struct rt_dm_power_domain *pd = rt_dm_power_domain_get_by_index(dev, 1);
if (pd)
    rt_dm_power_domain_power_on(pd);
```

attach only handles index 0 (see comment in **power_domain.c**).

1.4.2.91.1.5 Optional callbacks

Callback	Role
attach_dev / **detach_dev**	Provider notified when a consumer links (clock/regulator coordination)
pirv	Provider private pointer (upstream spelling)

1.4.2.91.1.6 Platform bus integration

From **platform_probe**:

```
err = rt_dm_power_domain_attach(dev, RT_TRUE);
if (err && err != -RT_EEMPTY)
    return err;
```

Failed driver **probe** → **rt_dm_power_domain_detach(dev, RT_TRUE)**.
RPMsg and **PCI PME** also call **attach** where DT requires it.

1.4.2.91.1.7 Recommended sequencing

With regulator and reset (Power subsystem overview, Clock framework (CLK)):

```
regulator enable (if needed)
    → power domain on (attach or power_on)
    → clock / reset release
    → rt_dm_dev_iomap + driver init
```

Power **off** for suspend (if implemented): reverse order per SoC manual - often **idle clocks -> reset assert -> power_off**.

1.4.2.91.1.8 Kconfig

Option	Role
RT_PMDOMAIN_SCMI	SCMI power protocol → **pm-domain-scmi.c**

Option	Role
SOC_DM_PMDOMAIN_DIR	BSP-specific genpd drivers

Requires **RT_USING_DM** and **RT_USING_OFW** for DT phandle parsing.

1.4.2.91.1.9 Pitfalls

Issue	Mitigation
No **rt_ofw_data** on provider	**ofw_find_power_domain** fails — bind domain or proxy at provider probe
attach only index 0	Use **get_by_index(1, ...)** + **power_on** for more domains
-RT_EEMPTY vs error	Missing property is normal; log real failures only
power_on in IRQ	Callbacks may sleep — use thread context
put is stub	Rely on **detach** / **power_off** for lifecycle
Provider not probed before consumer	Ensure provider driver probes first or defer consumer
Confuse with **rt_dm_power_off_handler**	Domain = IP block; handler = whole system off

1.4.2.91.1.10 See also

- Power subsystem overview — stack overview
- Platform bus — auto **attach** in probe
- Regulator
- Open Firmware (OFW) — phandles, **#power-domain-cells**
- SCMI agent transports — SCMI provider
- `components/drivers/core/power_domain.c`
- `components/drivers/pmdomain/pm-domain-scmi.c`

1.4.2.92 Regulator

1.4.2.92.1 Regulator subsystem

Headers: **components/drivers/include/drivers/regulator.h**, **components/drivers/regulator/regulator_dm.h**. Core: **components/drivers/regulator/regulator.c**, DT parse: **regulator_dm.c**.
Models **LDO/DCDC/load switches** as **struct rt_regulator_node** (provider) and opaque **struct rt_regulator** (consumer handle). **Misuse can damage hardware** — use only the public API.
Kconfig: **RT_USING_REGULATOR** (requires **RT_USING_DM**). In-tree: **regulator-fixed**, **regulator-gpio**, **regulator-pwm**, **regulator-fan53555**, **regulator-scmi**, plus **SOC_DM_REGULATOR_DIR**.

1.4.2.92.1.1 End-to-end flow

Provider (rail definition)

```
Platform/PMIC driver probe
        |
        v
regulator_ofw_parse(np, &param)     /* DT limits, boot-on, always-on */
        |
        v
Fill rt_regulator_ops (enable, set_voltage, ...)
        |
```

```
        v
rt_regulator_register(reg_np)
        |
        +-- rt_ofw_data(regulator_np) = reg_np
        +-- optional: boot_on / always_on → enable at register
```

Consumer (device that needs a rail)

```
Consumer probe (after regulator provider probed)
        |
        v
reg = rt_regulator_get(dev, "vcc")   /* DT: vcc-supply = <&reg_xyz> */
        |
        v
rt_regulator_set_voltage(reg, min_uv, max_uv)   /* before enable on some PMICs */
rt_regulator_enable(reg)
        |
        v
... use hardware ...
        |
        v
rt_regulator_disable(reg)
rt_regulator_put(reg)
```

Not automatic: unlike power domains, **`platform_probe` does not enable regulators** — the consumer driver must **`get`** / **`enable`** in **`probe`** and **`disable`** / **`put`** in **`remove`**.
Recommended order with other power (Power subsystem overview): **regulator** → **power domain** → **clock/reset** → **iomap**.

1.4.2.92.1.2 Registering a regulator (important)

Requirements for `rt_regulator_register`

Field	Required
`reg_np->dev`	Owning **`rt_device`** (usually **`&pdev->parent`** of provider)
`reg_np->param`	**`struct rt_regulator_param`** (name, voltage/current limits, delays, flags)
`reg_np->ops`	At least **`enable`**/**`disable`** (and **`set_voltage`**/**`get_voltage`** if adjustable)
`reg_np->supply_name`	Rail name (often **`param->name`** from **`regulator-name`**)

```
struct my_regulator {
    struct rt_regulator_node node;
    struct rt_regulator_param param;
};

static const struct rt_regulator_ops my_ops = {
    .enable = my_enable,
    .disable = my_disable,
    .set_voltage = my_set_voltage,
    .get_voltage = my_get_voltage,
};

static rt_err_t my_probe(struct rt_platform_device *pdev)
{
    struct rt_device *dev = &pdev->parent;
    struct my_regulator *r = rt_calloc(1, sizeof(*r));

    regulator_ofw_parse(dev->ofw_node, &r->param);

    r->node.dev = dev;
    r->node.param = &r->param;
    r->node.supply_name = r->param.name;
    r->node.ops = &my_ops;

    return rt_regulator_register(&r->node);
}
```

On success with **`RT_USING_OFW`**: **`rt_ofw_data(dev->ofw_node) = reg_np`** so consumers can resolve phandles.
`boot_on` / **`always_on`** in **`param`**: **`register`** calls **`regulator_enable`** immediately.

Platform driver export

regulator-fixed pattern:

```
RT_PLATFORM_DRIVER_EXPORT(regulator_fixed_driver);
/* or INIT_SUBSYS_EXPORT + rt_platform_driver_register */
```

Provider must probe **before** consumers that call **rt_regulator_get** (or **get** will **rt_platform_ofw_request** the regulator node).

1.4.2.92.1.3 Device tree

Regulator node (provider):

```
vcc_3v3: regulator-3v3 {
    compatible = "regulator-fixed";
    regulator-name = "vcc";
    regulator-min-microvolt = <3300000>;
    regulator-max-microvolt = <3300000>;
    regulator-boot-on;
    gpio = <&gpio0 10 GPIO_ACTIVE_HIGH>;
};
```

Consumer:

```
mmc@ffe00000 {
    compatible = "vendor,mmc";
    vmmc-supply = <&vcc_3v3>;
    vqmmc-supply = <&vcc_1v8>;
};
```

Consumer property	Lookup
vcc-supply	**rt_regulator_get(dev, "vcc")**
vqmmc-supply	**rt_regulator_get(dev, "vqmmc")**

Parent rail: regulator node may have **vin-supply**; **rt_regulator_get** links **parent** via **regulator_check_parent** for enable/disable/voltage propagation up the tree.

1.4.2.92.1.4 Consumer API

API	Role
rt_regulator_get(dev, id)	Resolve **{id}-supply** phandle → allocate **struct rt_regulator**, **ref_get** on node
rt_regulator_put(reg)	**ref_put**; may **unregister** node when last reference
rt_regulator_enable(reg)	Enable rail (parent first); applies **enable_delay**
rt_regulator_disable(reg)	Consumer refcount then hardware off when last user
rt_regulator_set_voltage(reg, min, max)	Notifiers + **set_voltage**; parent voltage updated too
rt_regulator_get_voltage(reg)	Read back µV
rt_regulator_is_supported_voltage(reg, min, max)	Check against **param** range
rt_regulator_set_mode / **get_mode**	Optional PMIC modes

Enable / disable sharing

Multiple consumers on one rail:

- First **enable**: runs hardware **ops->enable**, bumps internal **enabled_count**.
- Further **enable**: if already on, returns **RT_EOK** immediately.
- **disable**: decrements consumer **enabled_count** until zero, then hardware **disable**.

Each consumer should still call **enable**/**disable**/**put** symmetrically.

Notifiers

```
static rt_err_t my_notifier(struct rt_regulator_notifier *n,
        rt_ubase_t msg, void *data)
{
    /* RT_REGULATOR_MSG_ENABLE, DISABLE, VOLTAGE_CHANGE, ... */
    return RT_EOK;
}

struct rt_regulator_notifier notifier = { .callback = my_notifier };
rt_regulator_notifier_register(reg, &notifier);
```

Keep callbacks short — they may run under regulator lock.

1.4.2.92.1.5 `struct rt_regulator_param` (DT via `regulator_ofw_parse`)

DT property	`param` field
`regulator-name`	`name`
`regulator-min/max-microvolt`	`min_uvolt` / `max_uvolt`
`regulator-min/max-microamp`	`min_uamp` / `max_uamp`
`regulator-boot-on`	`boot_on`
`regulator-always-on`	`always_on`
`enable-active-high`	`enable_active_high`
`regulator-ramp-delay`	`ramp_delay` (or `ramp_disable`)
`regulator-enable-ramp-delay`	`enable_delay`

1.4.2.92.1.6 In-tree provider drivers (reference)

Driver	Compatible / binding	Registration
fixed	`regulator-fixed`	GPIO enable, `regulator_ofw_parse`, `rt_regulator_register`
gpio	GPIO-controlled rail	Same pattern
pwm	PWM duty → voltage	`RT_REGULATOR_PWM`
fan53555	I2C PMIC	`RT_REGULATOR_FAN53555`
scmi	SCMI voltage protocol	`RT_REGULATOR_SCMI`

1.4.2.92.1.7 Example: consumer in `probe`

```
static rt_err_t my_drv_probe(struct rt_platform_device *pdev)
{
    struct rt_device *dev = &pdev->parent;
    struct rt_regulator *vcc;
    rt_err_t err;

    vcc = rt_regulator_get(dev, "vcc");
    if (rt_is_err(vcc))
        return rt_ptr_err(vcc);

    if (!rt_regulator_is_supported_voltage(vcc, 3300000, 3300000))
    {
        err = -RT_EINVAL;
        goto err_put;
    }

    err = rt_regulator_set_voltage(vcc, 3300000, 3300000);
    if (err)
        goto err_put;

    err = rt_regulator_enable(vcc);
```

```
    if (err)
        goto err_put;

    /* iomap, clocks, ... */
    pdev->priv = vcc;
    return RT_EOK;

err_put:
    rt_regulator_put(vcc);
    return err;
}

static rt_err_t my_drv_remove(struct rt_platform_device *pdev)
{
    struct rt_regulator *vcc = pdev->priv;

    rt_regulator_disable(vcc);
    rt_regulator_put(vcc);
    return RT_EOK;
}
```

1.4.2.92.1.8 Pitfalls

Issue	Mitigation
Provider not registered	**get** fails — ensure regulator **probe** runs first or defer consumer
Wrong property name	**id** must match **{id}-supply** in DT
Enable before voltage	Many PMICs need **set_voltage** then **enable**
Skip put	Leaks **struct rt_regulator** and node **ref**
set_voltage in IRQ	May sleep (I2C PMIC)
Shared rail	One consumer lowering voltage affects all — coordinate in DT
Bypass framework	Do not write PMIC registers from random drivers
always_on rail	**disable** may no-op in ops — still call **put**

1.4.2.92.1.9 See also

- Power subsystem overview — probe ordering
- Power domain
- Platform bus — provider **RT_PLATFORM_DRIVER_EXPORT**
- PIN Device — GPIO enables on **regulator-fixed**
- `components/drivers/regulator/regulator.c`
- `components/drivers/regulator/regulator-fixed.c`

1.4.2.93 Reset controller

1.4.2.93.1 Reset controller (rt_reset_control)

Header: **components/drivers/include/drivers/reset.h**. Core: **components/drivers/reset/reset.c**.
Resolves device-tree **resets** / **#reset-cells** / **reset-names** into **struct rt_reset_control** handles, then drives **assert** / **deassert** / **reset** on the owning **struct rt_reset_controller**.
This is **per-IP reset lines** (SoC reset manager), **not** whole-system power off — see Board poweroff and restart and System power off and reboot (core).
Kconfig: **RT_USING_RESET** (requires **RT_USING_DM** + **RT_USING_OFW**). In-tree: **reset-simple**, **reset-scmi**, **SOC_DM_RESET_DIR**.

1.4.2.93.1.1 End-to-end flow

Controller (provider)

```
Platform / SCMI driver probe
        |
        v
Fill rt_reset_control_ops (ofw_parse, assert, deassert [, reset, status])
        |
        v
rt_reset_controller_register(rstcer)
        |
        +-- rstcer->ofw_node set
        +-- rt_ofw_data(np) = rstcer   (object name "RSTC")
```

Consumer (device using a reset line)

```
Consumer probe (reset controller must be registered first)
        |
        v
rstc = rt_reset_control_get_by_name(dev, "reset")
     or rt_reset_control_get_by_index(dev, 0)
        |
        v
rt_reset_control_deassert(rstc)    /* release IP from reset -- typical */
        |
        v
... clocks enabled, iomap, init ...
        |
        v
remove: rt_reset_control_assert(rstc)   /* optional */
        rt_reset_control_put(rstc)
```

Not automatic: **`platform_probe`** does not deassert resets — the driver must **`get`** and **`deassert`** explicitly. Typical bring-up order (Power subsystem overview): **regulator** → **power domain** → **clock** → **reset deassert** → **MMIO**.

1.4.2.93.1.2 Registering a reset controller (important)

`rt_reset_controller_register`

Field	Role
`rstcer->ofw_node`	Controller DT node — **`rt_ofw_data`** stored here
`rstcer->ops`	**`struct rt_reset_control_ops`**
`rstcer->priv`	Driver private state (MMIO base, SCMI device, . . .)

```
struct my_rstc {
    struct rt_reset_controller controller;
    void __iomem *base;
};

static rt_err_t my_assert(struct rt_reset_control *rstc)
{
    /* rstc->id from #reset-cells -- put line into reset */
    return RT_EOK;
}

static rt_err_t my_deassert(struct rt_reset_control *rstc)
{
    /* take line out of reset */
    return RT_EOK;
}

static const struct rt_reset_control_ops my_rst_ops = {
    .assert = my_assert,
    .deassert = my_deassert,
};

static rt_err_t my_rst_probe(struct rt_platform_device *pdev)
{
    struct rt_device *dev = &pdev->parent;
    struct my_rstc *r = rt_calloc(1, sizeof(*r));

    r->base = rt_dm_dev_iomap(dev, 0);
    r->controller.ofw_node = dev->ofw_node;
```

```
    r->controller.ops = &my_rst_ops;
    r->controller.priv = r;

    return rt_reset_controller_register(&r->controller);
}
```

Reference: **reset-simple.c** — MMIO bit per line, **INIT_SUBSYS_EXPORT** + **rt_platform_driver_register**.

rt_reset_control_ops

Callback	Role
ofw_parse(rstc, args)	Optional: validate cells, store extra data in **rstc->priv**
assert	Active **in reset** (hold block)
deassert	Active **out of reset** (run block)
reset	Optional pulse: assert → delay → deassert
status	Optional: report whether line is asserted

Default after parse: **rstc->id = reset_args.args[0]** (first **#reset-cells** argument).
Polarity (**active-low** vs **high**) is **inside** your **assert**/**deassert** — read the SoC TRM, not just DT naming.

Unregister

rt_reset_controller_unregister: **-RT_EBUSY** if any consumer handle remains on **rstc_nodes** — **put** all **rt_reset_control** first.

1.4.2.93.1.3 Device tree

Controller:

```
rst: reset-controller@1000 {
    compatible = "vendor,reset";
    reg = <0x1000 0x100>;
    #reset-cells = <1>;
};
```

Consumer:

```
uart@2000 {
    compatible = "vendor,uart";
    reg = <0x2000 0x100>;
    resets = <&rst 5>;
    reset-names = "uart";
};
```

Property	Role
resets	Phandle + cell(s) per **#reset-cells**
reset-names	Maps name → index for **get_by_name**

Multiple lines:

```
dma@3000 {
    resets = <&rst 10>, <&rst 11>;
    reset-names = "dma", "dma-ocp";
};
```

1.4.2.93.1.4 Consumer API

API	Role
rt_reset_control_get_by_index(dev, index)	Parse **resets** entry **index**
rt_reset_control_get_by_name(dev, name)	Match **reset-names**

API	Role
rt_reset_control_get_array(dev)	Synthetic array control (see below)
rt_reset_control_put(rstc)	Remove from controller list, **rt_free**
rt_reset_control_deassert(rstc)	Release from reset (usual **probe** step)
rt_reset_control_assert(rstc)	Hold in reset (**remove** / suspend)
rt_reset_control_reset(rstc)	Pulse if **ops->reset** exists
rt_reset_control_status(rstc)	**-RT_ENOSYS** if no **status** op

OFW variants on raw node: **rt_ofw_get_reset_control_by_index**, **_by_name**, **_array**.

Error pointers

get_* may return **rt_err_ptr(-RT_ENOMEM)** etc. Always test:

```
rstc = rt_reset_control_get_by_name(dev, "reset");
if (rt_is_err(rstc))
    return rt_ptr_err(rstc);
if (!rstc)
    return -RT_EEMPTY;   /* no resets property */
```

NULL handle

assert / **deassert** / **reset** on **NULL** return **RT_EOK** (no-op). Do not use that to detect "no reset in DT" — check **get** return.

Reset arrays

rt_reset_control_get_array: builds a captain **rstc** with **is_array**; **assert/deassert/reset** recurse into child controls with **rollback** on partial failure. Use when one logical block needs several **resets** entries applied together.

1.4.2.93.1.5 Consumer probe / remove pattern

```
static rt_err_t my_probe(struct rt_platform_device *pdev)
{
    struct rt_device *dev = &pdev->parent;
    struct rt_reset_control *rstc;
    rt_err_t err;

    rstc = rt_reset_control_get_by_name(dev, "reset");
    if (rt_is_err(rstc))
        return rt_ptr_err(rstc);

    if (rstc && (err = rt_reset_control_deassert(rstc)))
        goto err_put;

    /* clk_prepare_enable, rt_dm_dev_iomap, ... */
    pdev->priv = rstc;
    return RT_EOK;

err_put:
    if (rstc)
        rt_reset_control_put(rstc);
    return err;
}

static rt_err_t my_remove(struct rt_platform_device *pdev)
{
    struct rt_reset_control *rstc = pdev->priv;

    if (rstc) {
        rt_reset_control_assert(rstc);
        rt_reset_control_put(rstc);
    }
    return RT_EOK;
}
```

Example in-tree: **dma-pl330.c** (**dma**, **dma-ocp**), **sdio-dw.c**, **8250-ofw.c**, **pinctrl-single.c** (**aib_rst**).

1.4.2.93.1.6 In-tree providers

Driver	Binding	Registration
reset-simple	**snps,dw-low-reset**, **st,stm32-rcc**, Allwinner, Aspeed, ...	Platform **probe** → **rt_reset_controller_register**
reset-scmi	SCMI protocol **reset**	SCMI **probe** → register on agent node

1.4.2.93.1.7 Semantics cheat sheet

Term in API	Typical meaning
assert	Line active → block held in reset
deassert	Line inactive → block can run
reset	Short assert then deassert (pulse)

Exact bit polarity is **driver-specific** (**reset-simple** uses **active_low** in platform data).

1.4.2.93.1.8 Pitfalls

Issue	Mitigation
Controller not probed	**get** fails — **rt_platform_ofw_request** on provider or fix init order
Clock vs reset order	Enable clock only after **deassert** (or follow TRM)
Skip put	**unregister** busy; memory leak
Shared reset line	One driver **assert** while another runs → crash
Wrong id cell	Match **#reset-cells** layout in BSP doc
Confuse with **gpio-poweroff**	System off IP **resets**
Array partial failure	Core rolls back earlier entries — ops must be reversible

1.4.2.93.1.9 See also

- Power subsystem overview — ordering with regulator / domain / clk
- Board poweroff and restart — system shutdown GPIO (not this API)
- Platform bus — provider driver binding
- Clock framework (CLK)
- Open Firmware (OFW) — phandles, **#reset-cells**
- components/drivers/reset/reset.c, reset-simple.c

1.4.2.94 RPMsg

RPMsg character device RPMsg name service

1.4.2.94.1 RPMsg subsystem

Header: **components/drivers/include/drivers/rpmsg.h**. Core bus: **components/drivers/rpmsg/rpmsg.c**.

RPMsg is an **endpoint-oriented IPC bus** on top of a **transport** (VirtIO + shared memory + mailbox in the in-tree example). The core matches **`struct rt_rpmsg_device`** to **`struct rt_rpmsg_driver`** by **`id.name`**, then manages **endpoints** (name + src/dst addresses) and **`send`** via **`struct rt_rpmsg_ops`**.

Layer	File	Role
RPMsg bus	**`rpmsg/rpmsg.c`**	Endpoints, match **`rpmsg-char`** / **`rpmsg-name-service`**
VirtIO RPMsg device	**`virtio/virtio-rpmsg.c`**	**`rt_rpmsg_ops`**, virtqueues, NS announce
AMP transport	**`rpmsg/rpmsg-rt-thread-virtio.c`**	Shared mem, mailbox doorbell, VirtIO transport
Class drivers	**`rpmsg_char.c`**, **`rpmsg_ns.c`**	Char devices + name service

Kconfig: **`RT_USING_RPMSG`**, **`RT_RPMSG_RT_THREAD_VIRTIO`** (default y), **`RT_RPMSG_CHAR_`**∗.

1.4.2.94.1.1 Complete stack (VirtIO example)

This is the **reference integration** in-tree: **`rpmsg-rt-thread-virtio.c`** + **`virtio-rpmsg.c`**.

```
DT: rt-thread,virtio-rpmsg (master or slave)
      |
      v
rpmsg_virtio_probe (platform)
      |-- mbox: rt_mbox_request_by_index
      |-- shmem: ioremap from "shmem" phandle
      |-- slab DMA pool in shared RAM
      |-- rt_virtio_device_register (device_id = VIRTIO_DEVICE_ID_RPMSG)
      |
      v
virtio_rpmsg_probe (virtio driver, RT_VIRTIO_DRIVER_EXPORT)
      |-- virtio_rpmsg_vq_init: rx + tx virtqueues
      |-- rt_rpmsg_device_register(id.name = "rpmsg-char")
      |-- optional: second device "rpmsg-name-service" if VIRTIO_RPMSG_F_NS
      |
      v
rpmsg bus: match rpmsg_char_driver / rpmsg_ns_driver
      |-- rpmsg_char_probe: control dev rpmsg_charN
      |-- rpmsg_ns_probe: endpoint name-service @ 0x35
      |
      v
Application: RT_DEVICE_CTRL_RPMSG_CREATE_EPT or NS_CREATE from remote
      |
      v
rt_rpmsg_send -> virtio_rpmsg_send -> virtqueue kick -> mbox notify peer
```

Physical pieces

Piece	DT / code	Role
Shared memory	**`shmem = <&soc_sram>`**	Descriptor rings + virtio buffers + link registers at base
Mailbox	**`mboxes = <&mbox N>`**	**`RPMSG_VIRTIO_EVENT_LINK`** (exchange queue pointers) and **`EVENT_QUEUE`** (kick)
`queue-max`	Both sides **must match**	Virtqueue depth
`master` / **`slave`**	Property on node	Master publishes queue offsets; slave reads peer TX ring for RX

Boot role: **`rpmsg.mode=master`** or **`slave`** in **`/chosen bootargs`** (`INIT_CORE_EXPORT rpmsg_mode_setup` in **`rpmsg.c`**). Must match firmware on the other core. **`virtio_rpmsg_probe`** swaps RX/TX queue indices for slave vs master.

1.4.2.94.1.2 Device tree example (master + slave)

```
/ {
```

```
    chosen {
        bootargs = "rpmsg.mode=master";   /* or slave on the other OS */
    };

    soc_sram: memory@60000000 {
        reg = <0x60000000 0x100000>;
    };

    soc_mailbox: mailbox@ ... { /* provider */ };

    rpmsg_virtio0: rpmsg-virtio@0 {
        compatible = "rt-thread,virtio-rpmsg";
        shmem = <&soc_sram>;
        mboxes = <&soc_mailbox 0>;   /* master uses index 0 */
        queue-max = <64>;
        master;
    };
};
```

Remote side (conceptually):

```
rpmsg_virtio1: rpmsg-virtio@0 {
    compatible = "rt-thread,virtio-rpmsg";
    shmem = <&soc_sram>;           /* same region */
    mboxes = <&soc_mailbox 1>;   /* slave uses different mbox index */
    queue-max = <64>;
    slave;
};
```

After both sides probe: master writes **MASTER_QUEUE_*** offsets; **EVENT_LINK** makes slave map **peer_tx_virtq** and process inbound RPMsg frames.

1.4.2.94.1.3 Layer 1 — Register transport (rpmsg-rt-thread-virtio)

RT_PLATFORM_DRIVER_EXPORT(rpmsg_virtio_driver) — does **not** register **rt_rpmsg_device** itself.
rpmsg_virtio_probe checklist:

1. **rt_mbox_request_by_index(&mbox_client, 0)** — **rx_callback = rpmsg_virtio_rx_callback**
2. Parse **shmem**, **rt_ioremap_cached**
3. **master** property → **is_master**, feature **VIRTIO_RPMSG_F_NS** only on master
4. **rt_slab_init** on half of shmem (DMA buffers for virtqueues)
5. **rt_dma_device_set_ops** — copy to/from shmem for non-coherent AMP
6. Fill **rt_virtio_device**: **VIRTIO_DEVICE_ID_RPMSG**, **rpmsg_virtio_trans**
7. **rt_virtio_device_register(vdev)** → triggers **virtio_rpmsg_probe**

Mailbox ISR (rpmsg_virtio_rx_callback):

- **EVENT_LINK**: peer published descriptor ring addresses — wire **peer_tx_virtq**
- **EVENT_QUEUE**: **rt_virtqueue_isr** for local virtqueues
- Peer TX avail ring progress → DMA tag fixup → **used** index update (feeds virtio RX path)

1.4.2.94.1.4 Layer 2 — Register RPMsg device (virtio-rpmsg.c)

RT_VIRTIO_DRIVER_EXPORT(virtio_rpmsg_driver)
virtio_rpmsg_probe:

```
vrpmsg->parent.id.name = "rpmsg-char";
vrpmsg->parent.ops = &virtio_rpmsg_ops;   /* create / destroy / send */
rt_rpmsg_device_register(&vrpmsg->parent);

if (VIRTIO_RPMSG_F_NS) {
    vrpmsg->ns->id.name = "rpmsg-name-service";
    rt_rpmsg_device_register(vrpmsg->ns);
}
```

`struct rt_rpmsg_ops` (backend contract)

Op	`virtio_rpmsg` behavior
`create_endpoint`	Local: optional **NS CREATE** message to `RT_RPMSG_NS_ADDR`. Remote-created: `src == RT_RPMSG_ADDR_ANY`
`destroy_endpoint`	Optional **NS DESTROY**
`send`	Build `struct virtio_rpmsg_hdr` (src, dst, len), `rt_virtqueue_add_outbuf`, `kick`

Wire header (`virtio-rpmsg.h`):

```
struct virtio_rpmsg_hdr {
    le32 src, dst, reserved;
    le16 len, flags;
    u8 data[];
};   /* max payload VIRTIO_RPMSG_MAX_BUF_SIZE - sizeof(hdr) */
```

RX path (`virtio_rpmsg_rx_done`)

1. `rt_virtqueue_read_buf` on **RX** queue
2. Find endpoint: `find_endpoint(rdev, dst=hdr->dst, src=ANY)`
   If `dst == RT_RPMSG_NS_ADDR` → use `vrpmsg->ns` device
3. `ept->rx_callback(rdev, hdr->src, data, len)`
4. Re-queue buffer with `rt_virtqueue_add_inbuf`

1.4.2.94.1.5 Layer 3 — RPMsg bus (`rpmsg.c`)

Register a class driver (char / NS / custom)

```
static const struct rt_rpmsg_device_id my_ids[] = {
    { .name = "rpmsg-char" },   /* must match rdev->id.name from backend */
    { },
};

static struct rt_rpmsg_driver my_driver = {
    .ids = my_ids,
    .probe = my_probe,
    .remove = my_remove,
    .rx_callback = my_default_rx,   /* fallback if create_endpoint(..., NULL) */
};
RT_RPMSG_DRIVER_EXPORT(my_driver);
```

`RT_RPMSG_DRIVER_EXPORT` → `INIT_DEVICE_EXPORT` → `rt_rpmsg_driver_register` → probes all `rt_rpmsg_device` on bus with matching `id.name`.

Register an `rt_rpmsg_device` (custom backend)

If you implement a non-VirtIO link:

```
rdev->id.name = "rpmsg-char";   /* or your id, but char driver won't bind */
rdev->ops = &my_rpmsg_ops;
rt_rpmsg_device_register(rdev);
```

Bus `probe`: `rt_dm_power_domain_attach` then `rdrv->probe(rdev)`.

1.4.2.94.1.6 Endpoints and addressing

```
struct rt_rpmsg_endpoint_info info;

rt_strncpy(info.name, "tty", RT_RPMSG_NAME_SIZE);
info.src = 0x400;              /* local address (assigned by you or NS) */
info.dst = 0x401;              /* remote peer */

ept = rt_rpmsg_create_endpoint(rdev, &info, my_rx_cb);
if (rt_is_err(ept))
    return rt_ptr_err(ept);

rt_rpmsg_send(ept, buf, len);
```

API	Role
rt_rpmsg_create_endpoint	**ops->create_endpoint**, insert on **ept_nodes**
rt_rpmsg_destroy_endpoint	Backend destroy + **rt_free**
rt_rpmsg_find_endpoint	Match src/dst/name wildcards (**RT_RPMSG_ADDR_ANY**)
rt_rpmsg_send / **sendto** / **send_wait**	**ops->send** under **ept->lock**

RX callback: per-endpoint **rx_cb**, or driver-wide **struct rt_rpmsg_driver::rx_callback** if **rx_cb == NULL**.

1.4.2.94.1.7 Name service (summary)

Well-known address **RT_RPMSG_NS_ADDR (0x35)**. Remote **CREATE** can auto-create endpoints (driver default RX logs only). Local **CREATE** from **virtio_rpmsg_create_endpoint** announces name+addr to peer when **VIRTIO_RPMSG_F_NS**.
Details: RPMsg name service.

1.4.2.94.1.8 Userspace / driver channel (summary)

1. Open control device **rpmsg_charN** (from **rpmsg_char probe**)
2. **RT_DEVICE_CTRL_RPMSG_CREATE_EPT** with **struct rt_rpmsg_endpoint_info**
3. Open **rpmsg_<src>x<dst>**, **read**/**write**

Details: RPMsg character device.

1.4.2.94.1.9 Bring-up checklist

Step	Check
1	**RT_USING_RPMSG**, **RT_RPMSG_RT_THREAD_VIRTIO**, mailbox + shmem drivers
2	Same **queue-max**, same **shmem** region, correct **mboxes** index per role
3	**rpmsg.mode=** bootarg matches remote
4	Master has **master;** property; slave has **slave;**
5	**virtio_rpmsg_probe** succeeds → **rpmsg_char** / **rpmsg-name-service** on bus
6	Create endpoint or wait for NS CREATE; verify **rt_rpmsg_send** returns OK
7	Hardware spinlock / Mailbox subsystem ordering if shared with other AMP IPC

1.4.2.94.1.10 Custom backend (non-VirtIO)

Implement **struct rt_rpmsg_ops** and register **rt_rpmsg_device** with an **id.name** your class driver matches (or reuse **rpmsg-char**):

- **create_endpoint**: book-keep addresses / shared memory slots
- **send**: push frame with src/dst header to remote
- On RX interrupt: parse header → **rt_rpmsg_find_endpoint** → call **ept->rx_callback**

Do **not** call **rt_rpmsg_*** from ISR if your **send** sleeps on virtqueue completion — use workqueue.

1.4.2.94.1.11 Pitfalls

Issue	Mitigation
`id.name` mismatch	Backend **`rpmsg-char`** vs driver table **`rpmsg-char`** / **`rpmsg-raw`**
Master/slave queue swap	Wrong **`rpmsg.mode`** → no RX
`queue-max` mismatch	Corrupt rings
NS without feature	Master must advertise **`VIRTIO_RPMSG_F_NS`**
Endpoint without RX cb	**`create_endpoint(..., NULL)`** needs driver **`rx_callback`**
Buffer size	Payload **`VIRTIO_RPMSG_MAX_BUF_SIZE - sizeof(hdr)`** (512 default)
NULL `send` addr	**`RT_RPMSG_ADDR_ANY`** rejected in **`virtio_rpmsg_send`**
Power domain	Bus probe attaches domain; **`-RT_EEMPTY`** OK

1.4.2.94.1.12 See also

- RPMsg character device
- RPMsg name service
- Mailbox subsystem
- Hardware spinlock
- Platform bus
- `components/drivers/rpmsg/rpmsg-rt-thread-virtio.c`
- `components/drivers/virtio/virtio-rpmsg.c`
- `components/drivers/virtio/virtio_config/virtio-rpmsg.h`

1.4.2.94.2 RPMsg character device

1.4.2.94.2.1 RPMsg char driver (`rpmsg_char.c`)

`RT_RPMSG_DRIVER_EXPORT(rpmsg_char_driver)` — matches **`rt_rpmsg_device`** with **`id.name`** **`rpmsg-char`** or **`rpmsg-raw`** (set by **`virtio-rpmsg.c`**).
Prerequisite: transport + **`rt_rpmsg_device_register`** — see RPMsg (VirtIO stack).

Topology

Device	Name pattern	Ops
Control	**`rpmsg_charN`**	**`control`** only — create/destroy endpoints
Per endpoint	**`rpmsg_<src>x<dst>`**	**`read`** / **`write`** / **`control`**

`probe`: allocates **`rpmsg_char_ctrl`**, stores in **`rdev->parent.user_data`**, registers control **`rt_device`**.

Create endpoint

On control device:

```
struct rt_rpmsg_endpoint_info info;

rt_strncpy(info.name, "my-channel", RT_RPMSG_NAME_SIZE);
info.src = local_addr;
info.dst = remote_addr;

rt_device_control(ctrl, RT_DEVICE_CTRL_RPMSG_CREATE_EPT, &info);
```

Flow:

1. Default name **rpmsg-raw** if **info.name** empty
2. **rt_rpmsg_create_endpoint(rdev, &info, rpmsg_char_rx_callback)**
3. Ring buffer pool: **RT_RPMSG_CHAR_MSG_MAX** × **RT_RPMSG_CHAR_MSG_SIZE_MAX** (Kconfig)
4. Register char device **rpmsg_uxu**

Data path

Direction	Path
TX	**write** → **rt_rpmsg_send**
RX	VirtIO/backend → endpoint cb → **rt_ringbuffer_put** → **read**
Overwrite	**RT_DEVICE_CTRL_RPMSG_DATA_OVERWRITE** → **put_force** when full

RX matching uses **find_endpoint** with **dst = incoming src**, **src = ANY** — addresses must match how the endpoint was created.

Destroy

RT_DEVICE_CTRL_RPMSG_DESTROY_EPT: deferred via **rt_work** until **ref_count == 1** and device closed.

See also

- RPMsg
- components/drivers/rpmsg/rpmsg_char.c

1.4.2.94.3 RPMsg name service

1.4.2.94.3.1 RPMsg name service (rpmsg_ns.c)

RT_RPMSG_DRIVER_EXPORT(rpmsg_ns_driver) — matches **id.name = "rpmsg-name-service"** (second **rt_rpmsg_device** from **virtio-rpmsg.c** when **VIRTIO_RPMSG_F_NS**).
Stack context: RPMsg.

Fixed endpoint

rpmsg_ns_probe creates:

Field	Value
info.name	"name-service"
src / **dst**	**RT_RPMSG_NS_ADDR (0x35)**
RX	**rpmsg_ns_rx_callback**

Wire message

```
struct rt_rpmsg_ns_msg {
    char name[32];
    le32 addr;
    le32 flags;    /* RT_RPMSG_NS_CREATE or RT_RPMSG_NS_DESTROY */
};
```

rpmsg_ns_rx_callback:

Flag	Action
CREATE	**rt_rpmsg_create_endpoint(rdev, &info, NULL)** → driver **rx_callback** fallback
DESTROY	**find_endpoint** + **destroy_endpoint**

virtio_rpmsg routes packets with **dst == 0x35** to **vrpmsg->ns** device, not the char device.

Local CREATE side

When **virtio_rpmsg_create_endpoint** runs on a **local** endpoint and **supp_ns**:

- Sends **NS CREATE** to **RT_RPMSG_NS_ADDR** with **addr = ept->info.src** (little-endian on wire)

Remote can then open the announced channel.

Pitfalls

Issue	Mitigation
Endianness	**addr** is **le32** on wire
CREATE without consumer	Orphan endpoints until DESTROY
Only master has NS feature	**rpmsg_virtio_get_features** sets **VIRTIO_RPMSG_F_NS** on master

See also

- RPMsg
- RPMsg character device
- components/drivers/rpmsg/rpmsg_ns.c

1.4.2.94.4 RPMsg character device

1.4.2.94.4.1 RPMsg char driver (rpmsg_char.c)

RT_RPMSG_DRIVER_EXPORT(rpmsg_char_driver) — matches **rt_rpmsg_device** with **id.name** **rpmsg-char** or **rpmsg-raw** (set by **virtio-rpmsg.c**).
Prerequisite: transport + **rt_rpmsg_device_register** — see RPMsg (VirtIO stack).

Topology

Device	Name pattern	Ops
Control	**rpmsg_charN**	**control** only — create/destroy endpoints
Per endpoint	**rpmsg_<src>x<dst>**	**read** / **write** / **control**

probe: allocates **rpmsg_char_ctrl**, stores in **rdev->parent.user_data**, registers control **rt_device**.

Create endpoint

On control device:

```
struct rt_rpmsg_endpoint_info info;

rt_strncpy(info.name, "my-channel", RT_RPMSG_NAME_SIZE);
```

```
info.src = local_addr;
info.dst = remote_addr;

rt_device_control(ctrl, RT_DEVICE_CTRL_RPMSG_CREATE_EPT, &info);
```

Flow:

1. Default name **rpmsg-raw** if **info.name** empty
2. **rt_rpmsg_create_endpoint(rdev, &info, rpmsg_char_rx_callback)**
3. Ring buffer pool: **RT_RPMSG_CHAR_MSG_MAX** × **RT_RPMSG_CHAR_MSG_SIZE_MAX** (Kconfig)
4. Register char device **rpmsg_uxu**

Data path

Direction	Path
TX	**write** → **rt_rpmsg_send**
RX	VirtIO/backend → endpoint cb → **rt_ringbuffer_put** → **read**
Overwrite	**RT_DEVICE_CTRL_RPMSG_DATA_OVERWRITE** → **put_force** when full

RX matching uses **find_endpoint** with **dst = incoming src**, **src = ANY** — addresses must match how the endpoint was created.

Destroy

RT_DEVICE_CTRL_RPMSG_DESTROY_EPT: deferred via **rt_work** until **ref_count == 1** and device closed.

See also

- RPMsg
- components/drivers/rpmsg/rpmsg_char.c

1.4.2.94.5 RPMsg name service

1.4.2.94.5.1 RPMsg name service (rpmsg_ns.c)

RT_RPMSG_DRIVER_EXPORT(rpmsg_ns_driver) — matches **id.name = "rpmsg-name-service"** (second **rt_rpmsg_device** from **virtio-rpmsg.c** when **VIRTIO_RPMSG_F_NS**).
Stack context: RPMsg.

Fixed endpoint

rpmsg_ns_probe creates:

Field	Value
info.name	"name-service"
src / **dst**	**RT_RPMSG_NS_ADDR** (0x35)
RX	**rpmsg_ns_rx_callback**

Wire message

```
struct rt_rpmsg_ns_msg {
    char name[32];
    le32 addr;
    le32 flags;   /* RT_RPMSG_NS_CREATE or RT_RPMSG_NS_DESTROY */
};
```

rpmsg_ns_rx_callback:

Flag	Action
CREATE	**rt_rpmsg_create_endpoint(rdev, &info, NULL)** → driver **rx_callback** fallback
DESTROY	**find_endpoint** + **destroy_endpoint**

virtio_rpmsg routes packets with **dst == 0x35** to **vrpmsg->ns** device, not the char device.

Local CREATE side

When **virtio_rpmsg_create_endpoint** runs on a **local** endpoint and **supp_ns**:

- Sends **NS CREATE** to **RT_RPMSG_NS_ADDR** with **addr = ept->info.src** (little-endian on wire)

Remote can then open the announced channel.

Pitfalls

Issue	Mitigation
Endianness	**addr** is **le32** on wire
CREATE without consumer	Orphan endpoints until DESTROY
Only master has NS feature	**rpmsg_virtio_get_features** sets **VIRTIO_RPMSG_F_NS** on master

---

See also

- RPMsg
- RPMsg character device
- components/drivers/rpmsg/rpmsg_ns.c

1.4.2.95 SCMI firmware protocol

1.4.2.95.1 SCMI (System Control and Management Interface)

On AArch64 platforms, **secure firmware** (TF-A, SCP, vendor EL3) often owns clocks, regulators, resets, and power domains. The OS talks to that firmware through **SCMI** instead of touching the same hardware directly.
RT-Thread splits the stack into:

Layer	Location	Role
Message / protocol IDs	components/drivers/include/drivers/scmi.h	Payload structs, SCMI_PROTOCOL_ID_*, RT_SCMI_MSG_* helpers
SCMI bus	components/drivers/firmware/arm_scmi/bus.c	Match rt_scmi_driver rt_scmi_device by protocol (and optional name)

Layer	Location	Role
Agent + transport	components/drivers/↩ firmware/arm_scmi/	Platform driver arm-scmi, shared memory, mailbox / SMC / virtio
Protocol providers	*-scmi.c under clk, regulator, reset, ...	probe on SCMI bus → register normal DM consumers (rt_clk_*, rt↩ _regulator_*, ...)

Transport details: SCMI agent transports.
Kconfig: RT_FIRMWARE_ARM_SCMI (needs RT_USING_FIRMWARE, RT_USING_OFW, Cortex-A / ARMv8). Enable mailbox and/or SMC transport options as your DT uses.

1.4.2.95.1.1 End-to-end flow

```
DT: compatible "arm,scmi" (+ mboxes/shmem or smc-id)
        ↓
INIT_SUBSYS: platform driver "arm-scmi" registered (agent.c)
        ↓
INIT_PLATFORM: platform device probe → scmi_probe()
         agent_ops->setup()     bind transport
         scmi_channels_setup()  foreach child "reg" → rt_scmi_device_register()
         register BASE protocol device (0x10)
        ↓
INIT_DEVICE (or SUBSYS for clk): RT_SCMI_DRIVER_EXPORT / rt_scmi_driver_register()
        ↓
scmi bus match(protocol_id [, name]) → driver->probe(sdev)
        ↓
Provider registers clk / regulator / reset / genpd / ...
        ↓
Other drivers consume via normal OFW (clocks = <&...>, resets = <&...>, ...)
```

Application and most drivers **never call SCMI directly**; they use the same APIs as MMIO backends (rt_clk_get_by_index, rt_regulator_get, rt_reset_control_get, ...).

1.4.2.95.1.2 Data structures (scmi.h)

```
struct rt_scmi_device {
    struct rt_device parent;    /* parent.ofw_node, parent.bus, parent.drv */
    const char *name;           /* optional; used in bus match */
    rt_uint8_t protocol_id;     /* from child node property "reg" */
    struct scmi_agent *agent;
};

struct rt_scmi_device_id {
    rt_uint8_t protocol_id;
    const char *name;           /* NULL = match any name for this protocol */
};

struct rt_scmi_driver {
    struct rt_driver parent;
    const char *name;
    const struct rt_scmi_device_id *ids;   /* sentinel: protocol_id == 0 */
    rt_err_t (*probe)(struct rt_scmi_device *sdev);
    rt_err_t (*remove)(struct rt_scmi_device *sdev);
    rt_err_t (*shutdown)(struct rt_scmi_device *sdev);
};
```

Bus match (bus.c): for each ids[] entry, id->protocol_id == device->protocol_id, and if id->name is set it must equal device->name (from OFW protocol-name or similar on the channel node).

1.4.2.95.1.3 Boot and init order

Export / register	Level	What runs
scmi_bus_init	INIT_CORE_EXPORT	rt_bus_register(&scmi_↩ bus)
scmi_drv_register (arm-scmi platform driver)	INIT_SUBSYS_EXPORT	Agent ready before bulk DT probe
platform_ofw_device_↩ probe	INIT_PLATFORM_EXPORT	Creates arm,scmi platform device → scmi_probe

Export / register	Level	What runs
RT_SCMI_DRIVER_↩ EXPORT(...)	INIT_DEVICE_EXPORT	Protocol driver registers → probes pending rt_scmi_devices
scmi_clk_drv_register	INIT_SUBSYS_EXPORT	**Exception**: clk-scmi.↩ c calls rt_scmi_driver_↩ register() manually (earlier than RT_SCMI_DRIVER_EXPORT)

If a protocol driver registers **after** its rt_scmi_device already exists, rt_bus_add_driver walks devices and runs probe (same pattern as platform bus).

1.4.2.95.1.4 Sending a message

All transports funnel through:

```
rt_err_t rt_scmi_process_msg(struct rt_scmi_device *sdev, struct rt_scmi_msg *msg);
```

Build payloads with typed structs in scmi.h and:

```
struct rt_scmi_msg msg = RT_SCMI_MSG_IN_OUT(SCMI_VOLTAGE_DOMAIN_LEVEL_SET, &in, &out);
err = rt_scmi_process_msg(sdev, &msg);
/* check err, then out.status (firmware SCMI status, not always -errno) */
```

Helper	Use
RT_SCMI_MSG_IN_OUT(id, in, out)	Fixed-size in + out
RT_SCMI_MSG_IN(id, in)	Command with input only
RT_SCMI_MSG_OUT(id, out)	Command with output only
RT_SCMI_MSG_RAW(...)	Variable buffer sizes

Firmware returns **SCMI_SUCCESS** (0) or negative **SCMI_ERR_*** in message status fields; rt_scmi_↩ strerror() maps those codes to short names (agent.c).
Threading: process_msg is blocking for mailbox/SMC paths; do not call from ISR unless your transport is explicitly IRQ-safe.

1.4.2.95.1.5 Device tree (controller)

Example mailbox transport (simplified):

```
scmi: scmi {
    compatible = "arm,scmi";
    mboxes = <&mbox 0>;
    shmem = <&scmi_shmem>;

    /* Optional per-protocol channel nodes */
    protocol@14 {
        reg = <0x14>;              /* SCMI_PROTOCOL_ID_CLOCK */
        protocol-name = "clocks";  /* must match driver ids[].name if set */
    };
    protocol@17 {
        reg = <0x17>;
        protocol-name = "regulator";
    };
};

scmi_shmem: sram@... {
    compatible = "arm,scmi-shmem";
    reg = <...>;
};
```

compatible	Transport (agent.c scmi_ofw_ids)
arm,scmi	Mailbox (agent-mailbox.c)
arm,scmi-smc, arm,scmi-smc-param, qcom,scmi-smc	SMC (agent-smc.c)

compatible	Transport (agent.c scmi_ofw_ids)
arm,scmi-virtio	Virtio (virtio-scmi.c, if RT_FIRMWARE_ARM↩ _SCMI_TRANSPORT_VIRTIO)

Child **reg** is the protocol ID (same values as SCMI_PROTOCOL_ID_* in scmi.h). The agent registers one **rt_scmi_device** per child; protocol drivers bind to that node's parent.ofw_node for provider-specific subnodes (e.g. regulators under voltage protocol).

1.4.2.95.1.6 Built-in protocol drivers

Protocol ID	ids[].name	Source	Registers
0x14 CLOCK	"clocks"	clk/clk-scmi.c	rt_clk provider
0x17 VOLTAGE	"regulator"	regulator/↩ regulator-scmi.c	rt_regulator_register
0x16 RESET	(see driver)	reset/reset-scmi.c	reset controller
0x11 POWER	"genpd"	pmdomain/↩ pm-domain-scmi.c	power domain proxy
0x19 PINCTRL	(see driver)	pinctrl/pinctrl-scmi.c	pin config via SCMI
SENSOR / thermal	(see driver)	thermal/thermal-scmi.c	thermal zone

Consumers in DT stay unchanged: e.g. clocks = <&scmi_clk 42>;, resets = <&scmi_reset 3>;, power-domains = <&scmi_pd 1>; — see Clock framework (CLK), Regulator, Reset controller, Power domain, Pin control (pinctrl).

1.4.2.95.1.7 Adding a protocol driver

1. Implement probe(struct rt_scmi_device *sdev) — use sdev->parent.ofw_node and sdev->agent is already set.
2. Issue commands with rt_scmi_process_msg(sdev, &msg).
3. Register the high-level provider (clock, regulator, ...) your consumers expect.
4. Publish the driver:

```
static const struct rt_scmi_device_id my_ids[] = {
    { SCMI_PROTOCOL_ID_xxx, "my-protocol-name" },  /* name must match DT if used */
    { /* sentinel */ },
};

static struct rt_scmi_driver my_scmi_driver = {
    .name = "my-scmi",
    .ids = my_ids,
    .probe = my_probe,
};
RT_SCMI_DRIVER_EXPORT(my_scmi_driver);
```

Match **protocol-name** on the channel node to ids[].name when the table entry sets name non-NULL.

1.4.2.95.1.8 Debugging

- Confirm **arm-scmi probe** and child reg values in logs (DBG_TAG scmi.agent, scmi.bus).
- Mailbox path: mailbox TX/RX, shared memory **channel free** bit (shmem.c), 30-tick timeout on rt_mbox↩ _send.
- SMC path: arm,smc-id, IRQ completion, page/offset for arm,scmi-smc-param.
- Version skew: use BASE protocol SCMI_COM_MSG_VERSION if commands return SCMI_ERR_↩ PROTOCOL.

1.4.2.95.1.9 See also

- SCMI agent transports — mailbox, SMC, shmem layout
- Mailbox subsystem — doorbell for `arm,scmi`
- `components/drivers/include/drivers/scmi.h`
- `components/drivers/firmware/arm_scmi/agent.c`, `bus.c`

1.4.2.96 SCMI agent transports

1.4.2.96.1 SCMI agent layer

Overview and protocol drivers: SCMI firmware protocol.
Implementation directory: **`components/drivers/firmware/arm_scmi/`**.
The **agent** is the only piece that knows how to move a `struct rt_scmi_msg` across the SoC boundary. Every `rt_scmi_device` on the SCMI bus shares the same **`struct scmi_agent`** instance created during platform **`scmi_probe`**.

1.4.2.96.1.1 Agent object (`agent.h`)

```
struct scmi_agent_ops {
    const char *name;
    rt_err_t (*setup)(struct scmi_agent *agent, struct rt_device *dev);
    rt_err_t (*process_msg)(struct scmi_agent *agent, struct rt_scmi_msg *msg);
};

struct scmi_agent {
    const struct scmi_agent_ops *ops;
    void *priv;   /* transport state (mailbox, SMC, virtio) */
};
```

Callback	When it runs
`setup`	Once per SCMI controller: parse DT, map shmem, request mailbox channel / program SMC
`process_msg`	Every `rt_scmi_process_msg()` — serialize request, wait for reply, fill `out_msg`

`rt_scmi_process_msg()` (`agent.c`) sets `msg->sdev` and calls `agent->ops->process_msg`.

1.4.2.96.1.2 Platform driver registration (`agent.c`)

```
static const struct rt_ofw_node_id scmi_ofw_ids[] = {
#ifdef RT_FIRMWARE_ARM_SCMI_TRANSPORT_MAILBOX
    { .compatible = "arm,scmi", .data = &scmi_agent_mailbox_ops },
#endif
#ifdef RT_FIRMWARE_ARM_SCMI_TRANSPORT_SMC
    { .compatible = "arm,scmi-smc", .data = &scmi_agent_smc_ops },
    /* arm,scmi-smc-param, qcom,scmi-smc */
#endif
#ifdef RT_FIRMWARE_ARM_SCMI_TRANSPORT_VIRTIO
    { .compatible = "arm,scmi-virtio", .data = &scmi_agent_virtio_ops },
#endif
    { /* sentinel */ },
};

static struct rt_platform_driver scmi_driver = {
    .name = "arm-scmi",
    .ids = scmi_ofw_ids,
    .probe = scmi_probe,
};
INIT_SUBSYS_EXPORT(scmi_drv_register);
```

`scmi_probe` sequence:

1. `agent_ops = pdev->id->data` — transport selected by `compatible`.
2. `agent_ops->setup(agent, &pdev->parent)` — transport private state in `agent->priv`.
3. **`scmi_channels_setup`**: for each available child of the controller node, read **`reg`** → `protocol_id`, allocate `rt_scmi_device`, set `sdev->agent`, `rt_scmi_device_register(sdev)`.

4. Register an extra device for **SCMI_PROTOCOL_ID_BASE** (common protocol) on the same agent.

Protocol drivers attach later via the SCMI bus (see main SCMI page).

1.4.2.96.1.3 Shared memory (shmem.c / shmem.h)

Mailbox and SMC transports place the SCMI frame in **struct scmi_shared_mem**:

- channel_status — **FREE** / **ERROR** bits; writer clears FREE before sending.
- msg_header — built with scmi_header(message_id, type, protocol_id, token) from scmi.h.
- msg_payload[] — command input; reply read back into msg->out_msg.

API	Role
scmi_shmem_msg_write(shmem, msg)	Pack header + in_msg, mark channel busy
scmi_shmem_msg_read(shmem, msg)	Copy reply after firmware marks channel free
scmi_shmem_clear_channel(shmem)	Reset channel after TX done (mailbox)

DT **shmem** phandle must point at a node with **compatible = "arm,scmi-shmem"**; address is **rt_ioremap**'d in agent setup.
Ensure the mapping matches firmware expectations (often **device** / **non-cacheable**). If the region is cacheable, flush or invalidate around access per your BSP / TRM.

1.4.2.96.1.4 Mailbox transport (agent-mailbox.c)

Depends on **RT_USING_MBOX** and RT_FIRMWARE_ARM_SCMI_TRANSPORT_MAILBOX.
Setup

- Count **mboxes** and **shmem** phandles; pick channel index when multiple mailboxes exist (e.g. high vs low priority).
- rt_mbox_request_by_index() with client callbacks:
  - **tx_prepare** → scmi_shmem_msg_write()
  - **tx_done** → scmi_shmem_clear_channel() on success
  - **rx_callback** → optional msg->rx_callback for async use

process_msg

- Spinlock around rt_mbox_send(chan, msg, timeout) (30 ticks in tree).
- Mailbox driver completes the round trip and fills output via shmem read path.

Typical DT:

```
scmi {
    compatible = "arm,scmi";
    mboxes = <&firmware_mbox 0>;
    shmem = <&scmi_shmem>;
};
```

See Mailbox subsystem and Mailbox device model (DM).

1.4.2.96.1.5 SMC transport (agent-smc.c)

Depends on **RT_FIRMWARE_ARM_SCMI_TRANSPORT_SMC**.
Setup

- **arm,smc-id** — SMC function number for SCMI.
- Map **shmem** like mailbox; optional **interrupts** — ISR sets done flag.

- Variants with **arm,scmi-smc-param** use page/offset fields in `struct scmi_agent_smc` for shared memory location passed in registers.

process_msg

- Writes message to shmem, issues **smccc** call, waits for IRQ or polling `done`, reads reply from shmem.

Use when firmware exposes SCMI only through EL3 traps (no separate mailbox IP).

1.4.2.96.1.6 Virtio transport (virtio-scmi.c)

Enabled when **RT_VIRTIO_SCMI** / `RT_FIRMWARE_ARM_SCMI_TRANSPORT_VIRTIO` is on. **compatible = "arm,scmi-virtio"** selects `scmi_agent_virtio_ops` — SCMI over a virtio queue instead of SoC mailbox/SMC. Typical in guest / hypervisor scenarios.

1.4.2.96.1.7 Error handling

Return	Meaning
`-RT_*` from `setup` / `process_msg`	RT-Thread transport failure (ENOMEM, EIO, mbox timeout, ...)
`out.status` / payload status field	Firmware **SCMI_ERR_*** — use `rt_scmi_strerror(status)` for logs

Do not assume firmware status equals POSIX `-errno` unless your wrapper converts it (protocol drivers often map `SCMI_SUCCESS` to `RT_EOK` explicitly).

1.4.2.96.1.8 See also

- SCMI firmware protocol
- `components/drivers/firmware/arm_scmi/agent.c`
- `components/drivers/firmware/arm_scmi/agent-mailbox.c`
- `components/drivers/firmware/arm_scmi/agent-smc.c`
- `components/drivers/virtio/virtio-scmi.c`

1.4.2.97 SCSI host and commands

1.4.2.97.1 SCSI subsystem

RT-Thread provides a **small SCSI core** for storage backends: a **host** (`rt_scsi_host`) scans targets, issues commands through **rt_scsi_ops::transfer**, and attaches **type-specific upper drivers** (disk `sd*`, CD-ROM, ...) that register **rt_blk_disk** devices.
The core is **not** a device-model bus (see comment in `scsi.c`) — there is no `rt_scmi`-style match/probe bus. Host adapters call **rt_scsi_host_register()** after filling **rt_scsi_host**.

Piece	Path
API / CDB layouts	`components/drivers/include/drivers/scsi.h`
Core	`components/drivers/scsi/scsi.c`
Block disk (type 0)	`components/drivers/scsi/scsi_sd.c`
CD-ROM (type 5)	`components/drivers/scsi/scsi_cdrom.c`

Kconfig: RT_USING_SCSI (needs RT_USING_DM). Options RT_SCSI_SD / RT_SCSI_CDROM (need RT_USING_BLK).

1.4.2.97.1.1 End-to-end flow

```
Host adapter (AHCI / UFS / VirtIO-SCSI / ...)
    fills rt_scsi_host { dev, ops, max_id, max_lun, parallel_io }
    calls rt_scsi_host_register()
        ↓
For each (id, lun): INQUIRY → devtype / removable
    unknown type or inquiry fail → skip LUN
    devtype >= SCSI_DEVICE_TYPE_MAX → shrink scan, stop
        ↓
scsi_device_setup(sdev): optional reset → TEST UNIT READY (5 s) → driver_table[devtype].probe
        ↓
scsi_sd_probe / scsi_cdrom_probe: READ CAPACITY → rt_hw_blk_disk_register ("sd*a", ...)
        ↓
Applications use blk API (@ref page_device_blk, @ref page_device_disk)
```

Unregister: rt_scsi_host_unregister() walks **lun_nodes**, optional **ops->reset**, type **remove**, frees each **rt_scsi_device**.

1.4.2.97.1.2 Data structures (scsi.h)

Host and device

```
struct rt_scsi_host {
    struct rt_device *dev;       /* required; DMA / naming */
    const struct rt_scsi_ops *ops;
    rt_size_t max_id;            /* both must be non-zero */
    rt_size_t max_lun;
    rt_bool_t parallel_io;       /* hint for blk layer */
    rt_list_t lun_nodes;         /* filled by register */
};

struct rt_scsi_device {
    struct rt_scsi_host *host;
    rt_list_t list;
    rt_size_t id;
    rt_size_t lun;
    rt_uint32_t devtype;         /* from INQUIRY */
    rt_uint32_t removable;
    rt_size_t last_block;        /* filled by READ CAPACITY helpers */
    rt_size_t block_size;
    void *priv;                  /* scsi_sd / scsi_cdrom */
};

struct rt_scsi_ops {
    rt_err_t (*reset)(struct rt_scsi_device *sdev);   /* optional */
    rt_err_t (*transfer)(struct rt_scsi_device *sdev, struct rt_scsi_cmd *cmd);  /* required */
};
```

Command envelope

```
struct rt_scsi_cmd {
    union { /* CDB: inquiry, read10, write16, ... */ } op;
    rt_size_t op_size;

    union {
        struct { /* fixed response structs */ } ;
        struct { void *ptr; rt_size_t size; } data;  /* DMA buffer for read/write */
    } data;
};
```

Build CDBs with packed structs in **scsi.h** (rt_scsi_inquiry, rt_scsi_read10, ...) or fill **cmd.op** manually. Set **op_size** to the CDB length your transport accepts.
High-level helpers in **scsi.c** (rt_scsi_inquiry, rt_scsi_read10, rt_scsi_write16, rt_scsi_read_capacity10, ...) assemble **rt_scsi_cmd** and call **host->ops->transfer**. On failure they often issue **rt_scsi_request_sense**.
rt_scsi_cmd_is_write(cmd) — used by VirtIO-SCSI to pick IN vs OUT virtqueue buffers.

1.4.2.97.1.3 `rt_scsi_host_register` / `unregister`

Before register (caller)

Field	Rule
`dev`	Non-NULL
`ops`	Non-NULL; **`transfer`** required
`max_id` / `max_lun`	Both non-zero; define scan grid
`parallel_io`	Set if adapter can pipeline multiple LUN commands
`lun_nodes`	Do not pre-fill; core initializes the list

Scan behavior (`scsi.c`)

1. Nested loop **`id [0, max_id)`**, **`lun [0, max_lun)`**.
2. **`rt_scsi_inquiry(&tmp_sdev, NULL)`** — failure skips the LUN.
3. If **`devtype >= SCSI_DEVICE_TYPE_MAX`**, scan **stops** and **`max_id`** / **`max_lun`** are clipped to the current index (simple bus assumption).
4. Allocated **`rt_scsi_device`** is copied from the temp device (id, lun, devtype, removable).
5. **`scsi_device_setup`**: optional **`reset`** → wait **TEST UNIT READY** (up to ~5 s) → **`driver_table[devtype].probe`**.

Returns **`RT_EOK`** if at least one LUN probed; **`-RT_EEMPTY`** if none; **`-RT_EINVAL`** / **`-RT_ENOMEM`** on bad args or OOM (partial LUNs may remain on ENOMEM mid-scan).

`transfer` contract

- Fill or consume **`cmd->data`** (`ptr` + `size` for READ/WRITE; embedded structs for INQUIRY / capacity).
- Return **`RT_EOK`** on command completion; RT error codes on transport failure.
- Sense: adapters may copy sense into **`cmd->data.request_sense`**; helpers already call **REQUEST SENSE** after some failures.

1.4.2.97.1.4 Upper-layer drivers (`driver_table`)

devtype (`SCSI_DEVICE_TYPE_*`)	**Kconfig**	**Probe**	**Block device**
`0x00` DIRECT	`RT_SCSI_SD`	`scsi_sd_probe`	`sd*a` — read/write/sync/erase
`0x05` CDROM	`RT_SCSI_CDROM`	`scsi_cdrom_probe`	`sr*a` — read-only

Other inquiry types log **not supported** and are skipped.
`scsi_sd_probe`: READ CAPACITY 10; if last LBA is `0xffffffff`, READ CAPACITY 16 and use READ/WRITE **16** ops; sets geometry from **`sdev->block_size`** / **`last_block`**; **`rt_hw_blk_disk_register`**. Honors **`host->parallel_io`** on the disk.
`scsi_cdrom_probe`: capacity + read-only **`rt_blk_disk`** (uses READ 10/12/16 by LBA width).

1.4.2.97.1.5 Built-in SCSI hosts (examples)

These embed **`struct rt_scsi_host parent`** and call **`rt_scsi_host_register`** after hardware init:

Backend	Source	Notes
AHCI	`ata/ahci.c`	SATA/ATAPI → `ahci_scsi_ops.transfer`; see ATA / AHCI
UFS	`ufs/ufs.c`	UTP carries SCSI CDBs; `ufs_host_ops`, often `parallel_io = RT_TRUE`
VirtIO SCSI	`virtio/↩ virtio-scsi.c`	`RT_VIRTIO_DRIVER_EXPORT`; 3 queues; hotplug via `config_changed` + work queue re-register

Typical host setup (pattern):

```
struct my_hba {
    struct rt_scsi_host parent;
    /* private */
};

static rt_err_t my_scsi_transfer(struct rt_scsi_device *sdev, struct rt_scsi_cmd *cmd)
{
    /* map id/lun, push CDB + data to hardware */
    return RT_EOK;
}

static const struct rt_scsi_ops my_scsi_ops = {
    .reset = my_scsi_reset,      /* optional */
    .transfer = my_scsi_transfer,
};

/* after controller ready */
hba->parent.dev = &controller_dev;
hba->parent.ops = &my_scsi_ops;
hba->parent.max_id = 1;
hba->parent.max_lun = 8;
hba->parent.parallel_io = RT_FALSE;
rt_scsi_host_register(&hba->parent);
```

Teardown: **`rt_scsi_host_unregister(&hba->parent)`** before freeing the controller.

1.4.2.97.1.6 Command reference (`scsi.h`)

Opcodes and structs are defined for common storage commands:

Opcode macro	Struct	Helper
`RT_SCSI_CMD_INQUIRY`	`rt_scsi_inquiry`	`rt_scsi_inquiry()`
`RT_SCSI_CMD_TEST_UNIT↩ _READY`	`rt_scsi_test_unit_ready`	`rt_scsi_test_unit_↩ ready()`
`RT_SCSI_CMD_READ_↩ CAPACITY10/16`	capacity structs	`rt_scsi_read_↩ capacity10/16()`
`RT_SCSI_CMD_READ10/↩ 12/16`	read structs	`rt_scsi_read10/12/16()`
`RT_SCSI_CMD_WRITE10/↩ 12/16`	write structs	`rt_scsi_write10/12/16()`
`RT_SCSI_CMD_↩ SYNCHRONIZE_CACHE10/16`	sync cache	`rt_scsi_synchronize_↩ cache10/16()`
`RT_SCSI_CMD_WRITE_↩ SAME10/16`	write same	`rt_scsi_write_same10/↩ 16()` (erase path in `scsi_sd`)
Mode select/sense	mode structs	`rt_scsi_mode_*` (auto-refresh in `scsi_sd`)

Use these when implementing **`transfer`** for a new HBA or when extending **`driver_table`** for another device type.

1.4.2.97.1.7 Debugging

Symptom	Checks
`-RT_EEMPTY`	No LUN answered INQUIRY — wiring, power, `max_id`/`max_lun`, firmware
`-RT_ETIMEOUT` in setup	Media not ready — TEST UNIT READY loop
`-RT_ENOSYS`	Unsupported `devtype` — enable `RT_SCSI_SD` / `RT_SCSI_CDROM` or add `driver_table` entry
READ/WRITE fail	Sense via `rt_scsi_request_sense`; verify `cmd->data.size` is in **sectors** for READ/WRITE helpers
VirtIO hotplug	`virtio_scsi_config_changed` → rescan; log `virtio.dev.scsi`

1.4.2.97.1.8 See also

- ATA / AHCI — AHCI port → SCSI → `sd*`
- Block layer (`blk`) — block layer API
- Physical disk (`rt_blk_disk`) — `rt_blk_disk` registration
- `components/drivers/include/drivers/scsi.h`
- `components/drivers/scsi/scsi.c`
- `components/drivers/virtio/virtio-scsi.c`

1.4.2.98 SDIO / MMC core

1.4.2.98.1 SDIO / MMC core (API)

`RT_USING_SDIO` enables the shared MMC/SD/SDIO core (`dev_mmcsd_core.c`, `dev_sd.c`, `dev_sdio.c`, ...). It does **not** require **`RT_USING_DM`**.

Topic	Page	Needs DM?
OFW parse, **`sdu`** naming	SDIO device model	Yes
`vmmc` / **`vqmmc`**	SDIO supplies (regulator)	Yes (+ **`RT_USING_REGULATOR`**)
SDHCI controller	SDHCI host controller	Yes (+ **`RT_USING_SDHCI`**)

Headers: `drivers/dev_mmcsd_core.h`, `drivers/mmcsd_host.h`, `drivers/mmcsd_card.h`, `drivers/mmcsd_cmd.h`. Symbols below are in **lexicographic order** (only representative macros where there are many; see headers for full lists).

1.4.2.98.1.1 `mmcsd_*` functions (A–Z)

Symbol	Description
`mmcsd_alloc_host`	Allocate `rt_mmcsd_host`.
`mmcsd_change`	Host/card state change entry (after identification and negotiation).
`mmcsd_detect`	Card-detect worker / callback entry.
`mmcsd_deselect_cards`	Deselect cards.
`mmcsd_excute_tuning`	Run tuning (spelling matches header).

Symbol	Description
mmcsd_free_host	Free host.
mmcsd_get_cid / mmcsd_get_csd	Read CID/CSD.
mmcsd_go_idle	GO_IDLE_STATE.
mmcsd_host_init	Initialize host software state.
mmcsd_host_lock / mmcsd_host_unlock	Host bus lock.
mmcsd_host_set_uhs_voltage	Host-side UHS voltage switch.
mmcsd_req_complete	Called by low level when one rt_mmcsd_req completes.
mmcsd_select_card	Select card.
mmcsd_select_voltage	Choose operating voltage from OCR.
mmcsd_send_abort_tuning	Abort tuning sequence.
mmcsd_send_cmd	Send one command (with retries).
mmcsd_send_request	Submit full rt_mmcsd_req.
mmcsd_send_tuning	Send tuning-related command.
mmcsd_set_bus_mode / mmcsd_set_bus_↩ width / mmcsd_set_clock / mmcsd_set_↩ timing	Update host io_cfg and usually call set_iocfg.
mmcsd_set_chip_select	SPI chip select mode.
mmcsd_set_data_timeout	Set data timeout from card capability.
mmcsd_set_initial_signal_voltage	Initial signal voltage.
mmcsd_set_signal_voltage	Switch signal voltage.
mmcsd_set_uhs_voltage	UHS voltage path.
mmcsd_spi_read_ocr / mmcsd_spi_use_↩ crc	SPI OCR/CRC.
mmcsd_wait_cd_changed	Wait for card-detect change.

1.4.2.98.1.2 rt_mmcsd_* functions (A–Z)

Symbol	Description
rt_mmcsd_blk_probe / rt_mmcsd_blk_remove	Bind/unbind block device.
rt_mmcsd_core_init	Initialize MMC core.

1.4.2.98.1.3 struct rt_mmcsd_host_ops callbacks (A–Z)

Symbol	Description
card_busy	Query card busy.
enable_sdio_irq	SDIO IRQ enable/disable.
execute_tuning	Sampling tuning.
get_card_status	Read card status.
request	Submit rt_mmcsd_req.
set_iocfg	Apply rt_mmcsd_io_cfg (clock, width, timing, voltage, ...).
signal_voltage_switch	Signal voltage switch.

1.4.2.98.1.4 Main structs (A–Z)

Symbol	Notes
`struct rt_mmcsd_cmd`	`cmd_code`, `arg`, `resp`, `flags`, `data`, `retries`, `err`, …
`struct rt_mmcsd_data`	`blksize`, `blks`, `buf`, `flags`, timeout, scatter-gather fields.
`struct rt_mmcsd_host`	`ops`, `io_cfg`, `freq_*`, `flags`, `card`, mutex/semaphore; with **`RT_USING_REGULATOR`**: optional `supply`; with OFW: `ofw_node`.
`struct rt_mmcsd_io_cfg`	`clock`, `vdd`, `bus_mode`, `bus_width`, `timing`, `signal_voltage`, …
`struct rt_mmcsd_req`	`cmd`, `data`, `stop`, `sbc`, `cap_cmd_during_tfr`.

1.4.2.98.1.5 See also

- SDIO device model — OFW / host naming (DM)
- SDIO supplies (regulator) — supplies (DM + regulator)
- SDHCI host controller — SDHCI (DM + SDHCI)
- `components/drivers/sdio/SConscript`
- `components/drivers/include/drivers/dev_mmcsd_core.h`
- `components/drivers/include/drivers/mmcsd_host.h`
- `components/drivers/include/drivers/mmcsd_card.h`
- `components/drivers/include/drivers/mmcsd_cmd.h`

1.4.2.99 SDIO device model

1.4.2.99.1 SDIO / MMC device model (DM)

The **MMC/SD core** (`dev_mmcsd_core.c`, …) builds with **`RT_USING_SDIO`** alone. **DM helpers** are an extra layer: only when **`RT_USING_DM`** is on does **`SConscript`** add **`dev_sdio_dm.c`** (see **`components/drivers/sdio/SConscript`**). APIs live in **`dev_sdio_dm.h`**.

Feature	Kconfig (typical)	Source (only with `RT_USING_DM`)
Host naming, OFW parse	**`RT_USING_DM`** + **`RT_USING_OFW`** for parse	`dev_sdio_dm.c`
SDHCI stack	**`RT_USING_DM`** + **`RT_USING_SDIO`** + **`RT_USING_SDHCI`**	`dev_sdhci.c`, `dev_sdhci_dm.c`, `dev_sdhci_host.c`
`vmmc` / **`vqmmc`**	**`RT_USING_DM`** + **`RT_USING_REGULATOR`**	`dev_regulator.c`
SoC SDHCI / DW MMC hosts	**`RT_USING_DM`** + **`RT_USING_SDIO`**	`sdio/host/*` (see **`host/Kconfig`**)

Without **`RT_USING_DM`**, implement **`struct rt_mmcsd_host_ops`** yourself and use SDIO / MMC core; there is **no** in-tree **`dev_sdhci_*`** or **`sdio_regulator_*`** build.

1.4.2.99.1.1 Driver / BSP quick notes

Topic	Guidance
Parse before card activity	Call **sdio_ofw_parse** (when **RT_USING_OFW**) while filling **struct rt_mmcsd_host** and **before** the core starts enumeration—invalid **bus-width** yields **-RT_EIO**; handle in **probe** instead of at first transaction.
Stable host names	Prefer DTS **aliases { mmcN = &...; }** so **sdio_host_set_name** matches board wiring and userland scripts; purely software hosts still rely on **rt_ofw_get_alias_last_id("mmc")** to avoid colliding with aliased indices.
Supplies	**vmmc** / **vqmmc** need DM + regulator—see SDIO supplies (regulator) and Regulator.
SDHCI (DM only)	**RT_USING_SDHCI** is implemented only with DM. Platform drivers use **dev_sdhci_dm.c** (**rt_dm_dev_iomap**, **rt_dm_dev_get_irq**, **rt_sdhci_get_property**)—see SDHCI host controller.

1.4.2.99.1.2 Host naming: sdio_host_set_name

sdio_host_set_name(host, out_devname) fills **host->name** with a stable **sdu** string.

- If **RT_USING_OFW** and **host->ofw_node** are set, it first tries **rt_ofw_get_alias_id(host->ofw_node, "mmc")** so names align with **aliases { mmc0 = &...; }** in DTS.
- Otherwise it allocates a monotonic numeric id (atomically, seeded from **rt_ofw_get_alias_last_id("mmc")** when OFW is enabled so software-only hosts do not collide with aliased indices).

Optional **out_devname** receives a copy of the same string (up to **RT_NAME_MAX**).

1.4.2.99.1.3 Open Firmware: sdio_ofw_parse

When **RT_USING_OFW** is enabled, **sdio_ofw_parse(dev_np, host)** parses common MMC controller properties from **dev_np** into **struct rt_mmcsd_host**:

Property	Effect
bus-width	**1** / **4** / **8** → sets **MMCSD_BUSWIDTH_*** in **host->flags**; invalid values return **-RT_EIO**.
max-frequency	**host->freq_max**.
non-removable	**MMCSD_SUP_NONREMOVABLE**.
cap-sdio-irq	**MMCSD_SUP_SDIO_IRQ**.
cap-sd-highspeed / **cap-mmc-highspeed**	**MMCSD_SUP_HIGHSPEED**.
mmc-ddr-*, **mmc-hs200-***, **mmc-hs400-***	DDR / HS200 / HS400 capability flags.
sd-uhs-*	UHS SDR/DDR capability flags.

It also assigns **host->ofw_node** if it was not already set (**host->ofw_node ? : dev_np**), and forces **MMCSD_MUTBLKWRITE** in **host->flags**.
Without **RT_USING_OFW**, **sdio_ofw_parse** is a stub that returns **RT_EOK** and does nothing.

1.4.2.99.1.4 Regulator hooks (DM + regulator)

dev_regulator.c is compiled only when **RT_USING_DM** and **RT_USING_REGULATOR** are both set (regulator framework itself depends on DM). See SDIO supplies (regulator) for **vmmc** / **vqmmc**.

1.4.2.99.1.5 SDHCI (DM + SDHCI)

RT_USING_SDHCI pulls in **dev_sdhci.c** only under **RT_USING_DM**. Platform probe uses **dev_sdhci_dm.c** (**rt_sdhci_pltfm_init**, **rt_dm_dev_iomap**, **rt_dm_dev_get_irq**, **rt_sdhci_get_property**). Details: SDHCI host controller.

1.4.2.99.1.6 See also

- components/drivers/sdio/dev_sdio_dm.c, dev_sdio_dm.h
- components/drivers/sdio/SConscript
- Core bus / driver binding: documentation/6.components/device-driver/core/bus.md
- OFW runtime API: documentation/6.components/device-driver/ofw/ofw.md
- SDIO supplies (regulator)
- SDHCI host controller

1.4.2.100 SDIO supplies (regulator)

1.4.2.100.1 SDIO / MMC and regulators

Requires device model. **dev_regulator.c** is added in **components/drivers/sdio/SConscript** only when **RT_USING_DM** and **RT_USING_REGULATOR** are both enabled (the regulator subsystem also depends on **RT_USING_DM**). You still need **RT_USING_SDIO** for the MMC stack.
This file links **struct rt_mmcsd_host** to optional **vmmc** (card VDD) and **vqmmc** (I/O / VCCQ) through **struct rt_mmcsd_supply** in **drivers/mmcsd_host.h**. Public helpers are declared in **dev_sdio_dm.h** (under **#ifdef RT_USING_REGULATOR**).
Without **RT_USING_DM**, there is no **sdio_regulator_*** implementation in-tree—handle power in your host driver or board init.

1.4.2.100.1.1 Supply handles

Field	Role
host->supply.vmmc	Card power rail from **rt_regulator_get(dev, "vmmc")**.
host->supply.vqmmc	I/O rail from **rt_regulator_get(dev, "vqmmc")**.

Missing supplies are allowed (**RT_NULL**); hard errors from **rt_regulator_get** are propagated and any acquired handles are **rt_regulator_put** on failure paths in **sdio_regulator_get_supply**.

1.4.2.100.1.2 API summary

Function	Role
sdio_regulator_get_supply(dev, host)	Resolve **vmmc** / **vqmmc** for the DM **rt_device** backing the host. Call once from probe after **host** exists.
sdio_regulator_set_ocr(host, supply, vdd_bit)	Drives **vmmc** from the card OCR **vdd** bit: **rt_regulator_set_voltage** after **ocrbitnum_to_vdd**; enable/disable paths use **host->supply.regulator_enabled** (see **components/drivers/sdio/dev_regulator.c**). **supply** may be **RT_NULL** (no-op).

Function	Role
sdio_regulator_set_vqmmc(host, ios)	Adjusts **vqmmc** voltage for **ios->signal_voltage** (**MMCSD_SIGNAL_VOLTAGE_330** / **_180** / **_120**), using **rt_regulator_is_supported_voltage**, **rt_regulator_get_voltage**, and **rt_regulator_set_voltage_triplet**. For **3.3 V**, the target may be derived from **host->io_cfg.vdd**. Returns **-RT_EINVAL** if **vqmmc** is missing.
sdio_regulator_enable_vqmmc / **sdio_regulator_disable_vqmmc**	Enable or disable **vqmmc** and track **vqmmc_enabled**.

host->supply.regulator_enabled tracks whether **vmmc** was enabled through the OCR path; **vqmmc_enabled** tracks the I/O rail.
SDHCI and DesignWare MMC hosts in-tree call these helpers during power and UHS voltage switch when **RT_USING_REGULATOR** is enabled (those host drivers also require DM).

1.4.2.100.1.3 Device tree

Use the same supply names your **rt_regulator_get** path expects (**vmmc**, **vqmmc**) so phandles resolve to the correct **struct rt_regulator**. The generic regulator subsystem is described in Regulator.

1.4.2.100.1.4 See also

- components/drivers/sdio/dev_regulator.c, dev_sdio_dm.h
- components/drivers/sdio/SConscript
- components/drivers/include/drivers/mmcsd_host.h (struct rt_mmcsd_supply, **RT_USING_REGULATOR**)
- Regulator
- SDIO device model
- SDHCI host controller

1.4.2.101 SDHCI host controller

1.4.2.101.1 SDHCI (SD Host Controller Interface)

Requires device model. In **components/drivers/sdio/SConscript**, **dev_sdhci.c**, **dev_sdhci_dm.c**, and **dev_sdhci_host.c** are built only when **RT_USING_DM**, **RT_USING_SDIO**, and **RT_USING_SDHCI** are all enabled. Kconfig may list **RT_USING_SDHCI** under SDIO alone, but the SDHCI sources in this tree **do not compile without DM**.
dev_sdhci.c implements **SDHCI 2.x/3.x/4.x** on the shared MMC/SD stack. Types and register offsets: **drivers/dev_sdhci.h**; **struct rt_mmc_host** bridge: **drivers/dev_sdhci_host.h**.

Path	DM needed?	Notes
Platform **rt_sdhci_pltfm_***	Yes	**dev_sdhci_dm.c**: **rt_dm_dev_iomap**, **rt_dm_dev_get_irq**, DT quirks
PCI **sdhci-pci.c**	Yes	**host/Kconfig** is under **RT_USING_DM && RT_USING_SDIO**
Custom host without SDHCI	No (core only)	**rt_mmcsd_host_ops** per SDIO / MMC core

vmmc / **vqmmc**: power/UHS paths call **sdio_regulator_*** when **RT_USING_REGULATOR** is on—see SDIO supplies (regulator).

1.4.2.101.1.1 Driver / BSP quick notes

Topic	Guidance
quirks / quirks2	Prefer DT-driven bits from **rt_sdhci_get_property** / **rt_sdhci_get_of_property**, then add SoC-only flags—wrong **ADMA** or **DMA width** quirks often show up only under load as CMD / data errors.
Order of init	With DM, **rt_dm_dev_iomap** / **rt_dm_dev_get_irq** (or PCI BAR/IRQ) must be valid **before** **rt_sdhci_setup_host** reads capabilities and selects SDMA/ADMA/PIO.
Probe failure / remove	Mirror **rt_sdhci_init_host** with **rt_sdhci_uninit_host** and **rt_sdhci_free_host** on error and on **remove**—leaving a registered IRQ or workqueue causes use-after-free on unplug.
UHS / 1.8 V	**set_uhs_signaling**, **voltage_switch**, and tuning paths are board-sensitive; with regulator support, **sdio_regulator_set_vqmmc** must succeed.
Card detect	DT **broken-cd**, **non-removable**, and **cap-*** must match the socket—wrong CD polarity or missing **non-removable** generates spurious detect storms or no card at all.
OFW	Property names and capability flags: SDIO device model.

1.4.2.101.1.2 Layering

Piece	Role
struct rt_sdhci_host	Hardware state: **ioaddr**, **irq**, **quirks**/**quirks2**, **ops**, DMA flags, linked **struct rt_mmc_host *mmc**.
struct rt_mmc_host	Core MMC/SD/SDIO view; embedded **rthost** (**struct rt_mmcsd_host**) is what **mmcsd_core** uses.
struct rt_sdhci_ops	Optional MMIO overrides and SoC hooks (clock, bus width, reset, UHS, tuning, ...).

Default MMIO access uses **rt_sdhci_readl** / **writel** (and 8/16-bit variants) when **ops->read_*/write_*** are not set.

1.4.2.101.1.3 Bring-up sequence (driver writer)

Typical order matches **rt_sdhci_set_and_add_host** in **dev_sdhci.c**:

1. **rt_sdhci_alloc_host(dev, priv_size)** — allocates **rt_sdhci_host** plus tail **private** bytes; wires **mmc** and default **mmc_host_ops**.
2. Fill **host->ioaddr**, **host->irq**, **host->hw_name**, optional **host->ops**, **quirks**, **dma_mask**, etc. (platform: via **rt_sdhci_pltfm_init**).
3. **rt_sdhci_setup_host** — read HCI capabilities, OCR masks, choose SDMA/ADMA/PIO, host limits.
4. **rt_sdhci_init_host** — timers, workqueues, IRQ **rt_hw_interrupt_install**, **rt_mmc_add_host**, card-detect enable. **rt_sdhci_set_and_add_host** wraps **setup_host + init_host**.

Teardown: **rt_sdhci_uninit_host**, **rt_sdhci_free_host**.

1.4.2.101.1.4 `struct rt_sdhci_ops` (selected callbacks)

Callback	Typical use
`set_clock` / **`set_bus_width`** / **`reset`** / **`set_uhs_signaling`**	Platform defaults in **`dev_sdhci_dm.c`** delegate to **`rt_sdhci_set_clock`**, **`rt_sdhci_set_bus_width`**, **`rt_sdhci_reset`**, **`rt_sdhci_set_uhs`**.
`read_*` / **`write_*`**	Non-memory-mapped HCI (sparse IO) or endian fix-ups.
`irq`	Vendor-specific interrupt demux before generic SDHCI handling.
`set_power`, **`voltage_switch`**, **`hw_reset`**, **`card_event`**	Power / card-detect / eMMC HW reset integration.
`platform_execute_tuning`	SoC-specific tuning when the generic CMD19 path is not enough.

Full list: **`drivers/dev_sdhci.h`**.

1.4.2.101.1.5 Quirks

`RT_SDHCI_QUIRK_*` and **`RT_SDHCI_QUIRK2_*`** in **`dev_sdhci.h`** document controller-specific workarounds (DMA width, ADMA, card-detect, HS200/DDR50, CMD23, preset values, ...). Set them from **`struct rt_sdhci_pltfm_data`** or after **`rt_sdhci_get_property`** merges DT-driven bits.

1.4.2.101.1.6 Platform DM (`dev_sdhci_dm.c` / `dev_sdhci_dm.h`)

Only with **`RT_USING_DM`**. **`rt_platform_device`** probe should use these helpers instead of duplicating MMIO/IRQ mapping.

API	Role
`rt_sdhci_pltfm_init(pdev, pdata, priv_size)`	**`rt_dm_dev_iomap(dev, 0)`**, **`rt_dm_dev_get_irq(dev, 0)`**, **`rt_sdhci_alloc_host`**, assign **`sdhci_pltfm_ops`** or **`pdata->ops`**, copy **`pdata->quirks`**.
`rt_sdhci_get_property` / **`rt_sdhci_get_of_property`**	Reads **`sdhci,auto-cmd12`**, **`sdhci,1-bit-only`**, **`bus-width`**, **`broken-cd`**, **`no-1-8-v`**, **`clock-frequency`**, **`keep-power-in-suspend`**, **`wakeup-source`** / **`enable-sdio-wakeup`**, and sets **`quirks`**, **`quirks2`**, **`pm_caps`**.
`rt_sdhci_pltfm_init_and_add_host`	**`pltfm_init`** → **`get_property`** → **`rt_sdhci_init_host`**.
`rt_sdhci_pltfm_remove` / **`rt_sdhci_pltfm_free`**	**`rt_sdhci_uninit_host`** and **`rt_sdhci_free_host`**.
`rt_sdhci_pltfm_clk_get_max_clock`	**`rt_clk_get_rate`** on the platform **`clk`**.

`struct rt_sdhci_pltfm_data` carries **`ops`**, **`quirks`**, **`quirks2`**. **`struct rt_sdhci_pltfm_host`** holds **`clk`**, **`clock`**, **`xfer_mode_shadow`**, and trailing private data (**`rt_sdhci_pltfm_priv`**).

1.4.2.101.1.7 PCI glue

`components/drivers/sdio/host/sdhci-pci.c` (Kconfig: **`RT_SDIO_SDHCI_PCI`**, requires **`RT_USING_DM`** and **`RT_USING_SDIO`**) allocates the host with **`rt_sdhci_alloc_host`**, maps BAR0 via **`rt_`**

pci_iomap, uses **pdev->irq**, **rt_pci_irq_unmask**, **rt_pci_set_master**, then **rt_sdhci_set_and_add_host**.

1.4.2.101.1.8 Registers and spec

drivers/dev_sdhci.h defines **RT_SDHCI_*** MMIO offsets and capability bits aligned with the **SDHCI** and **SD Physical Layer** documents. For behaviour beyond this tree, refer to the **SD Host Controller Simplified Specification** (SD Association).

1.4.2.101.1.9 See also

- components/drivers/sdio/dev_sdhci.c, dev_sdhci_host.c, dev_sdhci_dm.c, host/sdhci-pci.c
- components/drivers/sdio/SConscript
- components/drivers/include/drivers/dev_sdhci.h, dev_sdhci_host.h, sdio/dev_sdhci_dm.h
- SDIO / MMC core
- SDIO device model
- SDIO supplies (regulator)
- PCI / PCIe

1.4.2.102 System controller (Syscon)

1.4.2.102.1 Syscon (system control registers)

Syscon is a small DM helper for a **shared MMIO register block** that several drivers touch (reboot magic, boot mode, LED bits, strap latches). One **struct rt_syscon** maps the region once and serializes **32-bit** accesses with a per-instance spinlock.

Piece	Path
API	components/drivers/include/drivers/syscon.h
Provider	components/drivers/mfd/mfd-syscon.c
Kconfig	**RT_USING_MFD** → **RT_MFD_SYSCON** (needs **RT_USING_DM**, **RT_USING_OFW**)

1.4.2.102.1.1 End-to-end flow

```
DT: compatible "syscon" (+ reg)
        ↓
INIT_SUBSYS: platform driver "mfd-syscon" registered
        ↓
INIT_PLATFORM: platform device probe → syscon_probe()
         rt_ofw_get_address → rt_ioremap
         insert on global _syscon_nodes list
         rt_ofw_data(np) = syscon
        ↓
Consumer (reboot, LED, ...) in its probe:
        rt_syscon_find_by_ofw_phandle(np, "regmap")
        or rt_syscon_find_by_ofw_node(parent_np)
        ↓
rt_syscon_read / write / update_bits (spinlock, HWREG32)
```

Lazy probe: If a node was not probed at init but is **compatible = "syscon"** or **"simple-mfd"**, **rt_syscon_find_by_ofw_node** can run **syscon_probe** on a stack **rt_platform_device** and then return **rt_ofw_data(np)**. Prefer normal DT probe for the syscon node so **iomem_base** exists before consumers start.

1.4.2.102.1.2 struct rt_syscon

```
struct rt_syscon {
    rt_list_t list;
    struct rt_ofw_node *np;
    void *iomem_base;
    rt_size_t iomem_size;      /* byte size of mapped region */
    struct rt_spinlock rw_lock;
};
```

Field	Set by
np	Syscon DT node
iomem_base / iomem_size	First **reg** region in **syscon_probe**
list	Global registry **_syscon_nodes** (lookup under **_syscon_nodes_lock**)

Published to OFW: **rt_ofw_data(syscon_np) = syscon** after probe.

1.4.2.102.1.3 Provider driver (mfd-syscon.c)

```
static const struct rt_ofw_node_id syscon_ofw_ids[] = {
    { .compatible = "syscon" },
    { /* sentinel */ },
};

static struct rt_platform_driver syscon_driver = {
    .name = "mfd-syscon",
    .ids = syscon_ofw_ids,
    .probe = syscon_probe,
    .remove = syscon_remove,
};
INIT_SUBSYS_EXPORT(syscon_drv_register);
```

Step (syscon_probe)	Action
Map	**rt_ofw_get_address(np, 0, &addr, &size)** → **rt_ioremap**
Register	Insert into **_syscon_nodes**, init **rw_lock**
Publish	**rt_ofw_data(np) = syscon**, **pdev->parent.user_data = syscon**

compatible = "simple-mfd" is **not** in **syscon_ofw_ids**; those nodes are only handled via **rt_syscon_find_by_ofw_node** lazy probe (or as parent of children that call **find** on the parent).

1.4.2.102.1.4 Access API

All three functions use **byte offset** into **iomem_base** (must satisfy **offset < iomem_size**). Each access is one **32-bit HWREG32** — offsets are **not** auto-scaled to word index.

```
rt_err_t rt_syscon_read(struct rt_syscon *syscon, rt_off_t offset, rt_uint32_t *out_val);
rt_err_t rt_syscon_write(struct rt_syscon *syscon, rt_off_t offset, rt_uint32_t val);
rt_err_t rt_syscon_update_bits(struct rt_syscon *syscon, rt_off_t offset,
        rt_uint32_t mask, rt_uint32_t val);
```

rt_syscon_update_bits (under spinlock):

```
new = (old & ~mask) | (val & mask)
```

Only bits set in **mask** are changed; **val** bits outside **mask** are ignored. Safe for typical RMW glue; **W1C** / hardware that ignores read data may need a dedicated driver instead of blind **update_bits**.
Callable from thread or ISR context (spinlock-based); keep critical sections short.

1.4.2.102.1.5 Lookup helpers

API	Use
rt_syscon_find_by_ofw_node(np)	Direct syscon (or **simple-mfd**) node; walks global list, else lazy **syscon_probe**
rt_syscon_find_by_ofw_phandle(np, "regmap")	Consumer node property → target syscon (Linux-style **regmap** phandle)
rt_syscon_find_by_ofw_compatible("vendor,foo-syscon")	First matching compatible in DT

Returns **RT_NULL** if map fails or compatible is not **syscon** / **simple-mfd** (for lazy path).

1.4.2.102.1.6 Device tree

Provider node

```
syscon: syscon@10000000 {
    compatible = "syscon";
    reg = <0x10000000 0x1000>;
};
```

Consumer: reboot (syscon-reboot)

```
reboot {
    compatible = "syscon-reboot";
    regmap = <&syscon>;
    offset = <0x200>;
    mask = <0x1>;
    value = <0x1>;
};
```

Kconfig: **RT_POWER_RESET_SYSCON_REBOOT**. Installs **rt_dm_machine_reset** hook that calls **rt_syscon_update_bits**.

Consumer: poweroff (syscon-poweroff)

Same pattern with **compatible = "syscon-poweroff"** → **rt_dm_machine_poweroff**.

Consumer: reboot mode (syscon-reboot-mode)

Child under syscon (or parent resolved via **rt_ofw_get_parent**); writes boot mode magic via **reboot-mode** helper.

Consumer: LED (register-bit-led)

```
syscon: syscon@10000000 {
    compatible = "syscon";
    reg = <0x10000000 0x100>;

    led0 {
        compatible = "register-bit-led";
        offset = <0x4>;
        mask = <0x10>;
        default-state = "off";
    };
};
```

led-syscon.c: **rt_syscon_find_by_ofw_node(parent)**, **rt_syscon_read** / **update_bits** for on/off. See LED device model (DM).

1.4.2.102.1.7 Kconfig and related drivers

Option	Role
RT_USING_MFD	MFD menu (requires **RT_USING_DM**)

Option	Role
RT_MFD_SYSCON	Build **mfd-syscon.c**
RT_POWER_RESET_SYSCON_REBOOT	power/reset/↩ syscon-reboot.c
RT_POWER_RESET_SYSCON_POWEROFF	power/reset/↩ syscon-poweroff.c
RT_POWER_RESET_SYSCON_REBOOT_MODE	power/reset/↩ syscon-reboot-mode.c
RT_LED_SYSCON	led/↩ led-syscon.c

1.4.2.102.1.8 Writing a consumer driver

1. In **probe**, obtain **struct rt_syscon** * via phandle or parent node — do not **rt_ioremap** the same **reg** again.
2. Parse **offset**, **mask**, **value** (or chip-specific properties) from your DT node.
3. Use **rt_syscon_read** for status; **rt_syscon_update_bits** for controlled bit changes.
4. Coordinate with Board poweroff and restart if you hook **rt_dm_machine_reset** / **rt_dm_machine**↩ **_poweroff** (only one handler).

1.4.2.102.1.9 Pitfalls

- **Offset vs register index**: API uses **byte** offset; register *N* is often **N** * **4** depending on TRM.
- **Region size**: **offset >= iomem_size** → **-RT_EINVAL** — extend **reg** in DTS if layout needs more space.
- **64-bit** / **16-bit only registers**: not supported by this API — use a dedicated MMIO driver.
- **Lazy vs platform probe**: consumers that **find** before syscon **INIT_PLATFORM** probe may rely on lazy probe; ordering bugs are easier if the syscon node is a normal platform device probed first.
- **Mask/value**: **update_bits(..., mask, val)** clears all **mask** bits then sets **val & mask** — do not use for status bits that clear on read (W1C).
- **Duplicate mapping**: multiple drivers **ioremap** the same physical address without syscon → races; centralize on one **rt_syscon**.

1.4.2.102.1.10 See also

- Platform bus — **RT_PLATFORM_DRIVER_EXPORT**, probe order
- Board poweroff and restart — machine reset / poweroff hooks
- LED device model (DM) — **register-bit-led**
- Open Firmware (OFW) — phandles, **rt_ofw_data**
- components/drivers/include/drivers/syscon.h
- components/drivers/mfd/mfd-syscon.c

1.4.2.103 Thermal management

1.4.2.103.1 Thermal subsystem

RT-Thread thermal management ties **temperature sensors** (**thermal zones**) to **cooling devices** (fan, DVFS, . . .) using **trip points** and **governors**. Temperatures are in **millidegrees Celsius (m℃)**.

Piece	Path
API	components/drivers/include/drivers/↩ thermal.h
Core + OFW	components/drivers/thermal/thermal.c, thermal_dm.c
Kconfig	**RT_USING_THERMAL** (requires **RT_USING_DM**, **RT_USING_OFW** for DT maps)

Cooling device details: Thermal cooling devices.

1.4.2.103.1.1 Architecture

```
Sensor driver (I2C, on-chip, SCMI, ...)
    embeds struct rt_thermal_zone_device
    fills rt_thermal_zone_ops (get_temp required)
    sets parent.ofw_node, zone_id
        ↓
rt_thermal_zone_device_register(zdev)
     thermal_ofw_setup()  ← walks /thermal-zones, trips, cooling-maps
     work queue: thermal_zone_poll → rt_thermal_zone_device_update()
     global thermal_zone_device_nodes list

Cooling driver (pwm-fan, dvfs, gpio-fan, ...)
    rt_thermal_cooling_device_register(cdev)
     global thermal_cooling_device_nodes list
        ↓
OFW cooling-map links trip → cdev via phandle
    thermal_bind() in thermal_ofw_setup
        ↓
Trip exceeded → rt_thermal_cooling_device_kick()
    governor tuning → cdev->ops->set_cur_level()
```

Governor: Built-in **dumb** governor is registered at **INIT_CORE_EXPORT**. **rt_thermal_cooling_↩ device_register** attaches **dumb** unless **rt_thermal_cooling_device_change_governor** selects another.

1.4.2.103.1.2 Kconfig

Option	Role
RT_USING_THERMAL	Core **thermal.c** + **thermal_dm.c**
RT_THERMAL_SCMI	Sensor zones via SCMI (thermal-scmi.c, needs SCMI firmware protocol)
RT_THERMAL_COOL_PWM_FAN	thermal-cool-pwm-fan.c
RT_THERMAL_COOL_GPIO_FAN	thermal-cool-gpio-fan.c
RT_THERMAL_COOL_DVFS	thermal-cool-dvfs.c (needs **RT_USING_DVFS**)
SOC_DM_THERMAL_DIR	BSP sensor drivers
SOC_DM_THERMAL_COOL_DIR	BSP cooling drivers

1.4.2.103.1.3 Thermal zone

```
struct rt_thermal_zone_device {
    struct rt_device parent;
    int zone_id;
    const struct rt_thermal_zone_ops *ops;
    struct rt_thermal_trip *trips;
    rt_size_t trips_nr;
```

```
    struct rt_thermal_cooling_map *cooling_maps;
    rt_tick_t polling_delay, passive_delay;
    /* temperature, cooling, poller work, mutex, ... */
};

struct rt_thermal_zone_ops {
    rt_err_t (*get_temp)(struct rt_thermal_zone_device *zdev, int *out_temp);  /* required */
    rt_err_t (*set_trips)(...);      /* optional HW trip window */
    rt_err_t (*set_trip_temp)(...);
    rt_err_t (*set_trip_hyst)(...);
    void (*hot)(...);
    void (*critical)(...);
};
```

Registering a zone

```
static rt_err_t my_get_temp(struct rt_thermal_zone_device *zdev, int *out_temp)
{
    *out_temp = read_sensor_mc();  /* m°C, or RT_THERMAL_TEMP_INVALID */
    return RT_EOK;
}

static const struct rt_thermal_zone_ops my_tz_ops = {
    .get_temp = my_get_temp,
};

/* probe */
zdev->ops = &my_tz_ops;
zdev->zone_id = 0;
zdev->parent.ofw_node = sensor_np;
rt_thermal_zone_device_register(zdev);
```

Requirement	Detail
`get_temp`	Mandatory; called from **`rt_thermal_zone_device_update`**
`parent.ofw_node`	Sensor node referenced from **`/thermal-zones/.../thermal-sensors`**
Trips	From DT under zone, or pre-filled **`zdev->trips`** before register
Polling	**`rt_work`** resubmits every **`polling_delay`** / **`passive_delay`** (from DT, ms → ticks)

1.4.2.103.1.4 Device tree (`thermal_ofw_setup`)

Linux-style layout under **`/thermal-zones`**:

```
thermal-zones {
    cpu-thermal {
        polling-delay-passive = <250>;
        polling-delay = <1000>;
        thermal-sensors = <&cpu_temp 0>;

        trips {
            passive: trip-passive {
                temperature = <85000>;   /* 85 °C in m°C */
                hysteresis = <2000>;
                type = "passive";
            };
            critical: trip-critical {
                temperature = <105000>;
                hysteresis = <2000>;
                type = "critical";
            };
        };

        cooling-maps {
            map0 {
                trip = <&passive>;
                cooling-device = <&fan0 0 3>, <&cpu_dvfs 0 5>;
                contribution = <1024>;
            };
        };
    };
};

cpu_temp: cpu-temp@... { /* sensor OFW node */ };

fan0: fan {
```

```
    compatible = "pwm-fan";
    pwms = <&pwm0 0 40000>;
    cooling-levels = <0 50 100 150 200 255>;
    /* optional: regulator consumer supply id "fan" (see regulator DT) */
};
```

Zone property	Effect
thermal-sensors	Phandle + optional index → matches sensor **ofw_node** and **zone_id**
polling-delay / **polling-delay-passive**	Work reschedule interval
sustainable-power, **coefficients**	**rt_thermal_zone_params**
Child trips	**temperature**, **hysteresis**, **type** (active / passive / hot / critical)
cooling-maps	**trip**, **cooling-device** (#cooling-cells: min/max level), **contribution**

cooling-device phandle resolves to a registered **rt_thermal_cooling_device** by matching **cdev->parent.ofw_node**. Register **cooling devices before** the zone that references them (or ensure cooling nodes are probed first).

1.4.2.103.1.5 Cooling device

```
struct rt_thermal_cooling_device {
    struct rt_device parent;
    const struct rt_thermal_cooling_device_ops *ops;
    rt_ubase_t max_level;
    struct rt_thermal_cooling_governor *gov;
    /* ... */
};

struct rt_thermal_cooling_device_ops {
    rt_err_t (*bind)(struct rt_thermal_cooling_device *cdev,
            struct rt_thermal_zone_device *zdev);
    rt_err_t (*unbind)(...);
    rt_err_t (*get_max_level)(...);
    rt_err_t (*get_cur_level)(...);
    rt_err_t (*set_cur_level)(struct rt_thermal_cooling_device *cdev, rt_ubase_t level);
};
```

rt_thermal_cooling_device_register requires **get_max_level**, **get_cur_level**, **set_cur_level**. **bind/unbind** optional (forwarded by **thermal_bind** in thermal_dm.c).

1.4.2.103.1.6 Update loop and trips

rt_thermal_zone_device_update(zdev, msg):

1. Reads temperature via **get_temp**.
2. For each trip with **temperature** < **current**, handles type:
   - **PASSIVE** / default → **cooling = true**, **rt_thermal_cooling_device_kick**
   - **CRITICAL** → **ops->critical** or cool; may **rt_hw_cpu_reset** if still over limit
   - **HOT** → **ops->hot** if present
3. Governor **tuning** adjusts levels per **cooling-map** cell; **set_cur_level** applied.
4. Notifiers receive **msg** bitmask (RT_THERMAL_MSG_TRIP_VIOLATED, RT_THERMAL_MSG_EVENT_TEMP_SAMPLE, ...).

Call **update** from the zone poller (automatic after register) or manually for testing.

1.4.2.103.1.7 Built-in drivers

Driver	Binding	Role
thermal-scmi.c	**RT_SCMI_DRIVER_EXPORT**, protocol sensor "thermal"	One zone per SCMI temperature sensor
thermal-cool-pwm-fan.c	**compatible = "pwm-fan"**	PWM + optional regulator — Thermal cooling devices
thermal-cool-gpio-fan.c	GPIO + speed table	Discrete fan speeds
thermal-cool-dvfs.c	DVFS OPP levels as cooling steps	Throttle CPU/GPU via DVFS

BSP sensors (e.g. board thermal IC) typically **rt_thermal_zone_device_register** in platform **probe** after filling trips in code or relying entirely on **thermal_ofw_setup**.

1.4.2.103.1.8 Key APIs

API	Role
rt_thermal_zone_device_register / unregister	Zone lifecycle + OFW parse + start poller
rt_thermal_zone_device_update	Sample temp, trips, cooling, notifiers
rt_thermal_zone_set_trip / get_trip	Runtime trip change
rt_thermal_zone_notifier_register	Event callback
rt_thermal_cooling_device_register / unregister	Cooling device
rt_thermal_cooling_device_kick	Re-run governor for a zone
rt_thermal_cooling_governor_register	Custom governor
rt_thermal_cooling_device_change_governor	Per-device governor selection

Constants: **RT_THERMAL_TEMP_INVALID** (274000), **RT_THERMAL_NO_LIMIT**.

1.4.2.103.1.9 Pitfalls

- **Units**: always **m℃** in trips and **get_temp** (85 ℃ → 85000).
- **get_temp cost**: slow buses (I2C) in every poll — cache or shorten **polling-delay** thoughtfully.
- **Cooling registration order**: maps reference **cdev** by OFW node; probe cooling before zone register.
- **Missing maps**: trips fire but no **cooling-maps** → log/events only, no fan/DVFS change.
- **Hysteresis** / **noise**: tight trips without **hysteresis** cause level oscillation.
- **Mutex**: transfer APIs in other subsystems aside — zone **update** uses **zdev->mutex** when not in ISR.
- **Unregister**: detach notifiers and stop polling before destroying hardware.

1.4.2.103.1.10 See also

- Thermal cooling devices — PWM fan cooling device
- DVFS — DVFS framework and Rockchip cpufreq
- SCMI firmware protocol — SCMI sensor protocol
- PWM Device

- Regulator
- components/drivers/include/drivers/thermal.h
- components/drivers/thermal/thermal.c

1.4.2.104 Thermal cooling devices

1.4.2.104.1 Cooling devices (in-tree)

Cooling drivers implement **struct rt_thermal_cooling_device_ops** and register with **rt_thermal_cooling_device_register**. Zones reference them from **cooling-maps** (see Thermal management).

Driver	Kconfig	compatible (typical)
PWM fan	**RT_THERMAL_COOL_PWM_FAN**	**pwm-fan**
GPIO fan	**RT_THERMAL_COOL_GPIO_FAN**	(see thermal-cool-gpio-fan.c)
DVFS	**RT_THERMAL_COOL_DVFS**	(see thermal-cool-dvfs.c)

Sources: components/drivers/thermal/thermal-cool-*.c.

1.4.2.104.1.1 PWM fan (thermal-cool-pwm-fan.c)

Role

Maps thermal **cooling level** (0 . . . **max_level**) to **PWM duty** via optional **cooling-levels** LUT. Optional **fan** regulator supply (property resolves as supply name **fan** → **rt_regulator_get(dev, "fan")**).

Device tree

```
fan0: fan {
    compatible = "pwm-fan";
    pwms = <&pwm0 0 40000>;          /* phandle, channel, period (ns) */
    cooling-levels = <0 64 128 192 255>;
    /* optional: bind regulator supply id "fan" on this node */
};
```

Referenced from zone:

```
cooling-device = <&fan0 0 4>;   /* min level 0, max level 4 */
```

Property	Role
pwms	**#pwm-cells**: channel, period
cooling-levels	PWM scale values per level; array length 1 = **max_level**
Regulator	**rt_regulator_get(dev, "fan")** — wire **fan** supply in regulator/DT as on your BSP

Ops behavior

Op	Behavior
get_max_level	**levels_nr - 1**
get_cur_level	Current level (spinlock)
set_cur_level	LUT → **rt_pwm_set**; level 0 powers off PWM/regulator

Power sequence: on — **rt_pwm_enable** then **rt_regulator_enable**; off — reverse with rollback on error.

Probe flow

1. Parse **pwms**, ensure PWM controller probed (**rt_platform_ofw_request** if needed).

2. Optional **rt_regulator_get(dev, "fan")**.
3. Read **cooling-levels** array.
4. **rt_thermal_cooling_device_register** — attaches default **dumb** governor.
5. Initial **set_cur_level(0)** (fan off).

Platform driver: **RT_PLATFORM_DRIVER_EXPORT(pwm_fan_cool_driver)**.

1.4.2.104.1.2 GPIO fan (thermal-cool-gpio-fan.c)

Kconfig: **RT_THERMAL_COOL_GPIO_FAN** (**RT_USING_PIN**, **RT_USING_OFW**).
Controls fan speed via **GPIO pins** and a **speed table** (RPM / control value). Suitable for multi-wire fan tach/control GPIOs rather than PWM.
Inspect **gpio_fan_cool_probe** in the source for exact property names on your BSP.

1.4.2.104.1.3 DVFS cooling (thermal-cool-dvfs.c)

Kconfig: **RT_THERMAL_COOL_DVFS** + **RT_USING_DVFS**.
Exposes **OPP table indices** as cooling levels — **set_cur_level** selects a lower performance point. Use when passive trip should throttle CPU/GPU frequency instead of spinning a fan.

1.4.2.104.1.4 Integration checklist

1. **Register cooling device** in platform **probe** before dependent zone (or rely on init order).
2. Set **parent.ofw_node** on **rt_thermal_cooling_device.parent** so **thermal_ofw_setup** can match **cooling-device** phandles.
3. Define **cooling-levels** (PWM) or max level in ops consistently with DT **cooling-device** min/max cells.
4. Zone **cooling-maps trip** must point at a trip node parsed under the same thermal zone.
5. Test with **rt_thermal_zone_device_update** or wait for poller — verify **set_cur_level** at each trip.

1.4.2.104.1.5 Pitfalls

- **Level 0**: PWM driver turns off output; confirm acoustic/thermal spec allows full stop.
- **Regulator + PWM order**: supply must match driver sequence (enable PWM before rail or as coded).
- **set_cur_level in ISR**: uses spinlock — do not call from ISR.
- **Governor**: **dumb** only nudges level on temperature delta vs **hysteresis** — custom governor may be needed for fine control.

1.4.2.104.1.6 See also

- Thermal management
- PWM Device
- Regulator
- PIN Device
- components/drivers/thermal/thermal-cool-pwm-fan.c

1.4.2.105 UFS host

1.4.2.105.1 UFS host controller

Header: **components/drivers/include/drivers/ufs.h**. Core: **components/drivers/ufs/ufs.c**, link/PA PM: **ufs_pm.c**.
RT-Thread implements a **UFSHCI host**: UIC for UniPro link (DME), UTP for SCSI-over-UPIU, then **rt_scsi_host_register** so the existing SCSI core scans LUNs and registers **sd*** disks (SCSI host and commands, **RT_SCSI_SD**).
There is **no separate UFS device-model bus** in-tree—only the **host adapter** API plus optional **PCI** glue (`ufs-pci.c`). SoC controllers use **struct rt_ufs_ops::init** / **exit** (and BSP code under **SOC_DM_UFS_DIR** when enabled in Kconfig) for clocks, reset, PHY, and supplies.

1.4.2.105.1.1 Kconfig

Option	Depends on	Role
RT_USING_UFS	**RT_USING_DM**, **RT_USING_DMA**, **RT_SCSI_SD**	**ufs.c**, **ufs_pm.c**
RT_UFS_PCI	**RT_USING_UFS**, **RT_USING_PCI**	**ufs-pci.c** (RT_PCI_DRIVER_EXPORT)
SOC_DM_UFS_DIR	**RT_USING_UFS**	BSP Kconfig / SoC UFS platform driver (out of tree here)

Enable **RT_USING_SCSI** and **RT_SCSI_SD** before UFS; the stack ends at **rt_blk_disk** / **sd*** (Block layer (`blk`), Physical disk (`rt_blk_disk`)).

1.4.2.105.1.2 End-to-end flow

```
Platform / PCI: MMIO (regs), IRQ, parent.dev for DMA
      |
      |  static struct my_ufs { struct rt_ufs_host parent; ... };
      |  fill regs, irq, ops; parent.dev = &controller->parent;
      v
rt_ufs_host_register(ufs)
      |  CAP → nutrs; alloc UTRL/UCD/bounce (coherent via parent.dev)
      |  ops->init (optional): clk, reset, regulator, PHY
      |  HCE, program UTRLBA/UTMRLBA, UTRLRSR/UTMRLRSR
      |  DME_LINKSTARTUP if HCS not ready (+ link_startup_notify PRE/POST)
      |  rt_ufs_pm_post_linkup (IRQ aggregation, Auto-H8 AHIT)
      |  IRQ install; parent.ops = ufs_host_ops; parallel_io = RT_TRUE
      v
rt_scsi_host_register(&ufs->parent)
      |  INQUIRY per (id, lun) → scsi_sd_probe → sd*
      v
Applications use blk API
```

Unregister: **rt_ufs_host_unregister** → **rt_scsi_host_unregister** → **ops->exit**, free DMA, mask IRQ.
Runtime I/O: SCSI **transfer** builds UPIU + UTRD, rings doorbell, waits on **ufs->done** (ISR sets completion on **UTRCS** / fatal bits). Small buffers use an internal **bounce** region (4 KiB) so inquiry/sense/capacity do not require caller DMA mapping.

1.4.2.105.1.3 Host driver responsibilities (before register)

Field	Caller	Notes
parent.dev	**Required**	Every **rt_dma_alloc_coherent** uses this device; set **ofw_node** consistently for DM naming.
regs	**Required**	UFSHCI MMIO base.
irq	**Required**	Installed at end of register; name **ufs-<devname>**.
ops	**Required**	Struct pointer; individual hooks optional.

Field	Caller	Notes
ucd_size	Optional	**0** → **RT_UFS_UCD_SIZE**.
ahit	Optional	Raw **AHIT** register; **0** → default in **rt_ufs_pm_post_linkup** when **CAP_AUTOH8**.
parent.max_id / **max_lun**	Optional	Raised to at least **1** if zero; tune scan range for your topology.
UTRL/UCD/bounce/cap/nutrs/...	**Core**	Do not pre-allocate.

After success, **ufs->parent.ops** points to internal **ufs_host_ops** (SCSI path). Vendor hooks stay in **struct rt_ufs_ops** only.

1.4.2.105.1.4 struct rt_ufs_ops

Callback	When	Role
init	After DMA alloc, before **HCE** and list base programming	Clocks, reset, PHY, vendor regs; failure rolls back buffers.
link_startup_notify	Before/after **DME_LINKSTARTUP** when lists not ready	**RT_UFS_NOTIFY_CHANGE_STATUS_PRE** / **POST** for extra DME traffic—keep short.
reset	SCSI host **reset** path	Controller/link recovery (optional).
exit	After **rt_scsi_host_unregister**	Undo **init**; errors logged only.

Minimal PCI reference uses empty **pci_ufs_std_ops**—**BAR map + **rt_ufs_host_register** only (ufs-pci.c). SoC hosts should implement **init**/**exit** for Regulator, Clock framework (CLK), Reset controller, Power domain.

1.4.2.105.1.5 rt_ufs_host_register sequence (core)

1. Validate **ufs**, **ops**, **regs**; read **CAP**, **nutrs** (cap 32 slots).
2. Allocate **UTRL** (1 KiB aligned), **UCD**, **bounce** via **parent.dev**.
3. **ops->init** if present.
4. **HCE** pulse; program **UTRLBA**/**U**, **UTMRLBA**/**U**; set **UTRLRSR**, **UTMRLRSR**.
5. If **HCS** lacks **UTRLRDY**/**UTMRLRDY**/**UCRDY**: optional notify → **rt_ufs_uic_cmd_send(DME_LINKSTARTUP)** → notify.
6. **rt_ufs_pm_post_linkup** (see below).
7. IRQ + **rt_scsi_host_register** with **ufs_host_ops**.

UIC helpers (**rt_ufs_uic_cmd_send**, **rt_ufs_dme_set**/**get**) are for bring-up and PM; normal storage traffic stays on the SCSI path.

1.4.2.105.1.6 Link and HCI power management

Implementation: **ufs_pm.c**. This layer is **UniPro PA** / **Hibernate8** / **UFSHCI idle features**, not system suspend. Rails and clocks belong in **init**/**exit**.

Boot defaults (rt_ufs_pm_post_linkup)

Called from register after link-up (safe to call again after custom training):

Step	API	Role
IRQ aggregation	**rt_ufs_intr_aggr_configure(ufs, RT_TRUE, cnt, 2)**	**UTRIACR** — batch completion IRQs (cnt **nutrs - 1**, max 31)
Auto-Hibernate8	**rt_ufs_auto_hibern8_set**	If **cap & RT_UFS_REG_CAP_AUTOH8**, programs **AHIT**

ahit: **0** uses **RT_UFS_AHIT_DEFAULT** (∼150 ms idle); non-zero is written as-is; disable later with **rt_ufs_auto_hibern8_set(ufs, 0)**. Encode timers with **rt_ufs_ahit_encode(timer, scale)**.

PA power mode (optional, not auto at register)

```
struct rt_ufs_pa_layer_attr attr = { /* gear, lanes, pwr_rx/tx, hs_rate */ };
rt_ufs_pa_power_mode_set(ufs, &attr, force);
```

Skips when attributes match cached **pwr_active** unless **force**. Use after link is stable for performance vs power; **ufs-pci.c** does not set PA mode today.

Hibernate8

API	Use
rt_ufs_hibern8_enter / **exit**	Explicit **DME_HIBERNATE_ENTER/EXIT** when **CAP_AUTOH8** is absent or for long idle policy

Finish outstanding UTP transfers before enter; exit before new SCSI commands.

Suggested bring-up order

1. **ops->init**: SoC clock / reset / regulator / PHY.
2. **rt_ufs_host_register** → link startup → **rt_ufs_pm_post_linkup**.
3. Optional **rt_ufs_pa_power_mode_set** for target gear/mode.
4. Runtime: Auto-H8 and/or manual Hibernate8.
5. **rt_ufs_host_unregister** → **ops->exit**.

1.4.2.105.1.7 PCI binding (RT_UFS_PCI)

ufs-pci.c: **pci_ufs_probe** iomaps BAR0, takes **pdev->irq**, **rt_pci_set_master**, **rt_ufs_host_register**. IDs include Red Hat **0x0013** and Samsung **0xc00c**. **remove**/**shutdown** call **rt_ufs_host_unregister**, unmap, free host struct.
No regulator or PA tuning in PCI glue—add a **pci_ufs_quirk** with custom **rt_ufs_ops** if a device needs it.

1.4.2.105.1.8 Caches and debugging

UFS DMA paths use **rt_hw_cpu_dcache_ops** around UTRD/UCD (see Hardware cache (hwcache)). Log tags: **rtdm.ufs**, **rtdm.ufs.pm**.

1.4.2.105.1.9 See also

Topic	Page / path
SCSI scan, **sd**∗, CDB flow	SCSI host and commands
AHCI as another SCSI host	ATA / AHCI

Topic	Page / path
Sources	components/drivers/ufs/ufs.c, ufs_pm.c, ufs-pci.c

1.4.3 FinSH Console

In the early days of computer development, before the advent of graphics systems, there was no mouse or even a keyboard. How did people interact with computers at the time? The earliest computers used a punched note to enter commands into the computer and write the program. Later, with the continuous development of computers, monitors and keyboards became the standard configuration of computers, but the operating system at this time did not support the graphical interface. Computer pioneers developed a software that accepts commands entered by the user, and after interpretation, passes it to The operating system and return the results of the operating system execution to the user. This program wraps around the operating system like a layer of shell, so it's called a shell.
Embedded devices usually need to connect the development board to the PC for communication. Common connections include: serial port, USB, Ethernet, Wi-Fi, etc. A flexible shell should also support working on multiple connection methods. With the shell, the developer can easily get the system running and control the operation of the system through commands. Especially in the debugging phase, with the shell, in addition to being able to locate the problem more quickly, the developer can also use the shell to call the test function, change the parameters of the test function, reduce the number of times the code is downloaded, and shorten the development time of the project.
FinSH is the command line component (shell) of RT-Thread. It is based on the above considerations. FinSH is pronounced [fn]. After reading this chapter, we will have a deeper understanding of how FinSH works and how to export your own commands to FinSH.

1.4.3.1 Introduction of FinSH

FinSH is the command line component of RT-Thread. It provides a set of operation interfaces for users to call from the command line. It is mainly used to debug or view system information. It can communicate with a PC using serial/Ethernet/USB, etc. The hardware topology is shown below:

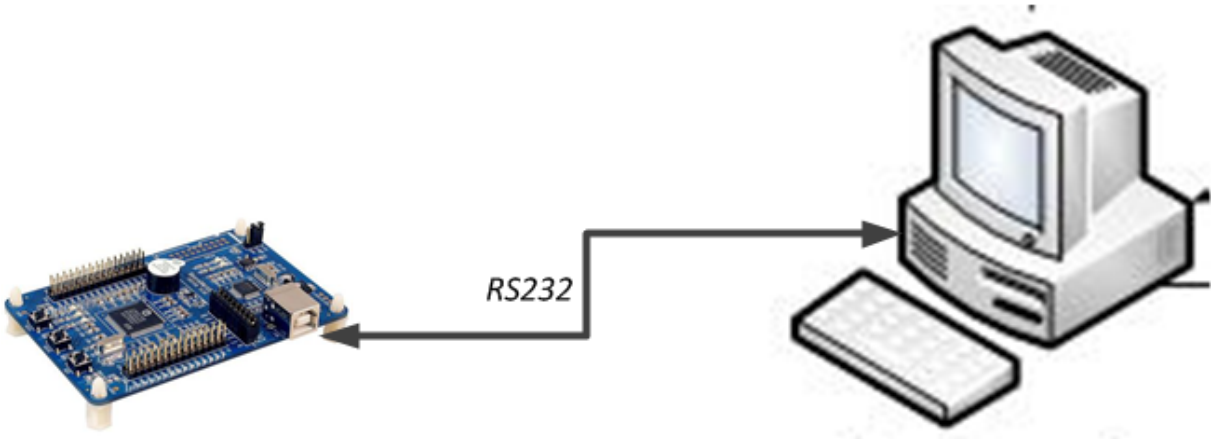


Figure 1.112 FinSH Hardware connection diagram

The user inputs a command in the control terminal, and the control terminal transmits the command to the FinSH in the device through the serial port, USB, network, etc., FinSH will read the device input command, parse and automatically scan the internal function table, find the corresponding function name, and execute the function. The response is output, the response is returned through the original path, and the result is displayed on the control terminal.
When using a serial port to connect a device to a control terminal, the execution flow of the FinSH command is as follows:

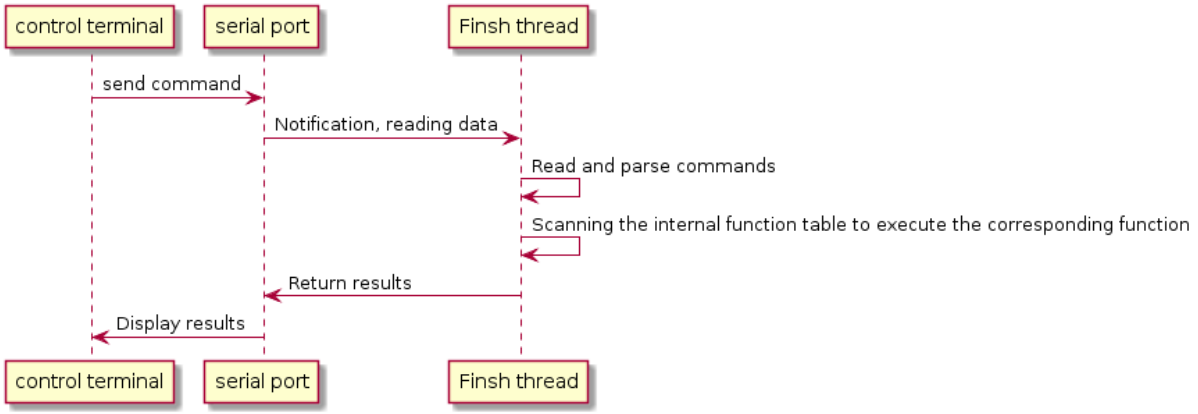


Figure 1.113 FinSH FinSH Command execution flow chart

FinSH supports the rights verification function. After the system is started, the system will perform the rights verification. Only when the rights verification is passed, the FinSH function will be enabled. This improves the security of system input.
FinSH supports auto-completion, and viewing history commands, etc. These functions can be easily accessed through the keys on the keyboard. The keys supported by FinSH are shown in the following table:

Keys	Functional Description
Tab key	Pressing the Tab key when no characters are entered will print all commands supported by the current system. If you press the Tab key when you have entered some characters, it will find the matching command, and will also complete the file name according to the file system's current directory, and you can continue to input, multiple completions.
↑↓ key	Scroll up and down the recently entered history command
Backspace key	Delete character
←→ key	Move the cursor left or right

FinSH uses the traditional command line mode, also known as MSH (module shell).

1.4.3.1.1 MSH Command-Line Mode

In MSH mode, FinSH works like a traditional shell such as DOS or Bash. For example, you can switch to the root directory with the `cd /` command.
MSH can parse commands into parameters and parameters separated by spaces. Its command execution format is as follows:

```
command [arg1] [arg2] [...]
```

The command can be either a built-in command in RT-Thread or an executable file.

1.4.3.2 FinSH Built-in Commands

Some FinSH commands are built in by default in RT-Thread. You can print all commands supported by the current system by entering help in FinSH and pressing Enter or directly pressing Tab.
In msh mode, you can list all currently supported commands by pressing the Tab key. The number of default commands is not fixed, and the various components of RT-Thread will output some commands to FinSH. For example, when the DFS component is opened, commands such as `ls`, `cp`, and `cd` are added to FinSH for developers to debug.
The following are all currently supported commands that display RT-Thread kernel status information after pressing the Tab key. The command name is on the left and the description of the command on the right:

```
RT-Thread shell commands:
version          - show RT-Thread version information
list_thread      - list thread
list_sem         - list semaphore in system
list_event       - list event in system
list_mutex       - list mutex in system
```

```
list_mailbox     - list mail box in system
list_msgqueue    - list message queue in system
list_timer       - list timer in system
list_device      - list device in system
exit             - return to RT-Thread shell mode.
help             - RT-Thread shell help.
ps               - List threads in the system.
time             - Execute command with time.
free             - Show the memory usage in the system.
```

Here lists the field information returned after entering the common commands, so that the developer can understand the content of the returned information.

1.4.3.2.1 Display Thread Status

Use the `ps` or `list_thread` command to list all thread information in the system, including thread priority, state, maximum stack usage, and more.

```
msh />list_thread
thread   pri  status      sp     stack size max used left tick  error
-------- ---  ------- ---------- ----------  ------  ---------- ---
tshell    20  ready   0x00000118 0x00001000    29%   0x00000009 000
tidle     31  ready   0x0000005c 0x00000200    28%   0x00000005 000
timer      4  suspend 0x00000078 0x00000400    11%   0x00000009 000
```

list_thread Return field description:

Field	Description
thread	Thread name
pri	Thread priority
status	The current state of the thread
sp	The current stack position of the thread
stack size	Thread stack size
max used	The maximum stack position used in thread history
left tick	The number of remaining ticks of the thread
error	Thread error code

1.4.3.2.2 Display Semaphore Status

Use the `list_sem` command to display all semaphore information in the system, including the name of the semaphore, the value of the semaphore, and the number of threads waiting for this semaphore.

```
msh />list_sem
semaphore v   suspend thread
--------- --- --------------
shrx      000 0
e0        000 0
```

list_sem Return field description:

Field	Description
semaphore	Semaphore name
v	The current value of semaphore
suspend thread	The number of threads waiting for this semaphore

1.4.3.2.3 Display Event Status

Use the `list_event` command to display all event information in the system, including the event name, the value of the event, and the number of threads waiting for this event.

```
msh />list_event
event      set    suspend thread
-----  ---------- --------------
```

list_event Return field description:

Field	Description
event	Event set name
set	The current event in the event set
suspend thread	The number of threads waiting for an event in this event set

1.4.3.2.4 Display Mutex Status

Use the `list_mutex` command to display all mutex information in the system, including the mutex name, the owner of the mutex, and the number of nestings the owner holds on the mutex.

```
msh />list_mutex
mutex    owner    hold suspend thread
-------- -------- ---- --------------
fat0     (NULL)   0000 0
sal_lock (NULL)   0000 0
```

list_mutex Return field description:

Field	Description
mutxe	Mutex name
owner	The thread currently holding the mutex
hold	The number of times the holder is nested on this mutex
suspend thread	The number of threads waiting for this mutex

1.4.3.2.5 Display Mailbox Status

Use the `list_mailbox` command to display all mailbox information in the system, including the mailbox name, the number of messages in the mailbox, and the maximum number of messages the mailbox can hold.

```
msh />list_mailbox
mailbox  entry size suspend thread
-------- ----  ---- --------------
etxmb    0000  0008 1:etx
erxmb    0000  0008 1:erx
```

list_mailbox Return field description:

Field	Description
mailbox	Mailbox name
entry	The number of messages included in the mailbox
size	The maximum number of messages a mailbox can hold
suspend thread	The number of threads waiting for this mailbox

1.4.3.2.6 Display Message Queue Status

Use the `list_msgqueue` command to display all message queue information in the system, including the name of the message queue, the number of messages it contains, and the number of threads waiting for this message queue.

```
msh />list_msgqueue
msgqueue entry suspend thread
-------- ----  --------------
```

list_msgqueue Return field description:

Field	Description
msgqueue	Message queue name
entry	The number of messages currently included in the message queue
suspend thread	Number of threads waiting for this message queue

1.4.3.2.7 Display Memory Pool Status

Use the `list_mempool` command to display all the memory pool information in the system, including the name of the memory pool, the size of the memory pool, and the maximum memory size used.

```
msh />list_mempool
mempool block total free suspend thread
------- ----  ----  ---- --------------
signal  0012  0032  0032 0
```

list_mempool Return field description:

Field	Description
mempool	Memory pool name
block	Memory block size
total	Total memory block
free	Free memory block
suspend thread	The number of threads waiting for this memory pool

1.4.3.2.8 Display Timer Status

Use the `list_timer` command to display all the timer information in the system, including the name of the timer, whether it is the periodic timer, and the number of beats of the timer timeout.

```
msh />list_timer
timer     periodic   timeout       flag
-------- ---------- ---------- -----------
tshell   0x00000000 0x00000000 deactivated
tidle    0x00000000 0x00000000 deactivated
timer    0x00000000 0x00000000 deactivated
```

list_timer Return field description:

Field	Description
timer	Timer name
periodic	Whether the timer is periodic
timeout	The number of beats when the timer expires
flag	The state of the timer, activated indicates active, and deactivated indicates inactive

1.4.3.2.9 Display Device Status

Use the `list_device` command to display all device information in the system, including the device name, device type, and the number of times the device was opened.

```
msh />list_device
device         type      ref count
------ ----------------- ----------
e0     Network Interface 0
uart0  Character Device  2
```

list_device Return field description:

Field	Description
device	Device name
type	Device type
ref count	The number of times the device was opened

1.4.3.2.10 Display Dynamic Memory Status

Use the `free` command to display all memory information in the system.

```
msh />free
total memory: 7669836
used memory : 15240
```

```
maximum allocated memory: 18520
```

free Return field description:

Field	Description
total memory	Total memory size
used memory	Used memory size
maximum allocated memory	Maximum allocated memory

1.4.3.3 Custom FinSH Command

In addition to the commands that come with FinSH, FinSH also provides multiple macro interfaces to export custom commands. The exported commands can be executed directly in FinSH.

1.4.3.3.1 Custom msh Command

The custom msh command can be run in msh mode. To export a command to msh mode, you can use the following macro interface

```
MSH_CMD_EXPORT(name, desc);
```

Parameter	Description
name	The command to export
desc	Description of the export command

This command can export commands with parameters, or export commands without parameters. When exporting a parameterless command, the input parameter of the function is void. The example is as follows

```
void hello(void)
{
    rt_kprintf("hello RT-Thread!\n");
}

MSH_CMD_EXPORT(hello , say hello to RT-Thread);
```

When exporting a command with parameters, the function's input parameters are `int argc` and `char**argv`. Argc represents the number of arguments, and argv represents a pointer to a command-line argument string pointer array. An example of exporting a parameter command is as follows:

```
static void atcmd(int argc, char**argv)
{
    ......
}

MSH_CMD_EXPORT(atcmd, atcmd sample: atcmd <server|client>);
```

1.4.3.3.2 Access Variables from an MSH Command

The C-Style interpreter and `FINSH_VAR_EXPORT` were removed from RT-Thread. There is no direct replacement for exporting a variable to the shell. To inspect or change a variable at runtime, expose a small MSH command that reads or updates it.

The following example exports a `dummy` command. Run `dummy` to read the value or `dummy <value>` to change it:

```
#include <stdlib.h>
#include <rtthread.h>
#include <finsh.h>

static int dummy = 0;

static void dummy_cmd(int argc, char **argv)
{
    if (argc == 1)
    {
        rt_kprintf("dummy = %d\n", dummy);
        return;
    }

    if (argc == 2)
```

```
    {
        dummy = atoi(argv[1]);
        rt_kprintf("dummy = %d\n", dummy);
        return;
    }

    rt_kprintf("Usage: dummy [value]\n");
}

MSH_CMD_EXPORT_ALIAS(dummy_cmd, dummy, show or set the dummy value);
```

1.4.3.3.3 Custom MSH Command Name

Use MSH_CMD_EXPORT_ALIAS to expose a function under a different command name:

```
MSH_CMD_EXPORT_ALIAS(name, alias, desc);
```

Parameter	Description
name	The function to export
alias	The command name displayed in MSH
desc	Description of the exported command

The following example exports the `hello` function under the command name `ho`:

```
void hello(void)
{
    rt_kprintf("hello RT-Thread!\n");
}

MSH_CMD_EXPORT_ALIAS(hello, ho, say hello to RT-Thread);
```

1.4.3.4 FinSH Function Configuration

The FinSH function can be cropped, and the macro configuration options are defined in the rtconfig.h file. The specific configuration items are shown in the following table.

Macro Definition	Value Type	Description	Default
#define RT_USING_MSH	None	Enable the MSH command shell	on
#define RT_USING_FINSH	None	Enable FinSH support (selected by RT_USING_MSH)	on
#define FINSH_THREAD_NAME	String	FinSH thread name	"tshell"
#define FINSH_USING_HISTORY	None	Turn on historical traceback	on
#define FINSH_HISTORY_LINES	Integer type	Number of historical command lines that can be traced back	5
#define FINSH_USING_SYMTAB	None	Symbol table can be used in FinSH	on
#define FINSH_USING_DESCRIPTION	None	Add a description to each FinSH symbol	on
#define FINSH_USING_MSH	None	Enable the MSH implementation (selected by RT_USING_MSH)	on
#define FINSH_ARG_MAX	Integer type	Maximum number of input parameters	10
#define FINSH_USING_AUTH	None	Enable permission verification	off
#define FINSH_DEFAULT_PASSWORD	String	Authority verification password	off

The reference configuration example in rtconfig.h is as follows, and can be configured according to actual functional requirements.

```
/* Enable the MSH command shell */
```

```
#define RT_USING_MSH

/* Symbols selected by RT_USING_MSH */
#define RT_USING_FINSH
#define FINSH_USING_MSH

/* Define the thread name as tshell */
#define FINSH_THREAD_NAME "tshell"

/* Open history command */
#define FINSH_USING_HISTORY
/* Record 5 lines of history commands */
#define FINSH_HISTORY_LINES 5

/* Enable the use of symbol table */
#define FINSH_USING_SYMTAB
/* Turn on description */
#define FINSH_USING_DESCRIPTION

/* Define FinSH thread priority to 20 */
#define FINSH_THREAD_PRIORITY 20
/* Define the stack size of the FinSH thread to be 4KB */
#define FINSH_THREAD_STACK_SIZE 4096
/* Define the command character length to 80 bytes */
#define FINSH_CMD_SIZE 80

/* The maximum number of input parameters is 10 */
#define FINSH_ARG_MAX 10
```

1.4.3.5 FinSH Application Examples

1.4.3.5.1 Examples of msh Command without Arguments

This section demonstrates how to export a custom command to msh. The sample code is as follows, the hello function is created in the code, and the `hello` function can be exported to the FinSH command list via the `MSH_CMD_EXPORT` command.

```
#include <rtthread.h>

void hello(void)
{
    rt_kprintf("hello RT-Thread!\n");
}

MSH_CMD_EXPORT(hello , say hello to RT-Thread);
```

Once the system is up and running, press the tab key in the FinSH console to see the exported command:

```
msh />
RT-Thread shell commands:
hello             - say hello to RT-Thread
version           - show RT-Thread version information
list_thread       - list thread
......
```

Run the `hello` command and the results are as follows:

```
msh />hello
hello RT_Thread!
msh />
```

1.4.3.5.2 Example of msh Command with Parameters

This section demonstrates how to export a custom command with parameters to FinSH. The sample code is as follows, the `atcmd()` function is created in the code, and the `atcmd()` function can be exported to the msh command list via the MSH_CMD_EXPORT command.

```
#include <rtthread.h>

static void atcmd(int argc, char**argv)
{
    if (argc < 2)
    {
        rt_kprintf("Please input'atcmd <server|client>'\n");
        return;
    }

    if (!rt_strcmp(argv[1], "server"))
    {
        rt_kprintf("AT server!\n");
    }
    else if (!rt_strcmp(argv[1], "client"))
    {
        rt_kprintf("AT client!\n");
```

```
    }
    else
    {
        rt_kprintf("Please input'atcmd <server|client>'\n");
    }
}

MSH_CMD_EXPORT(atcmd, atcmd sample: atcmd <server|client>);
```

Once the system is running, press the Tab key in the FinSH console to see the exported command:

```
msh />
RT-Thread shell commands:
hello             - say hello to RT-Thread
atcmd             - atcmd sample: atcmd <server|client>
version           - show RT-Thread version information
list_thread       - list thread
......
```

Run the `atcmd` command and the result is as follows:

```
msh />atcmd
Please input 'atcmd <server|client>'
msh />
```

Run the `atcmd server` command and the result is as follows:

```
msh />atcmd server
AT server!
msh />
```

Run the `atcmd client` command and the result is as follows:

```
msh />atcmd client
AT client!
msh />
```

1.4.3.6 FinSH Porting

FinSH is written entirely in ANSI C and has excellent portability; it has a small memory footprint, and FinSH will not dynamically request memory if you do not use the functions described in the previous section to dynamically add symbols to FinSH. The FinSH source is located in the `components/finsh` directory. Porting FinSH requires attention to the following aspects:

- FinSH thread

Each command execution is done in the context of a FinSH thread (that is, a tshell thread). When the RT_USING_FINSH macro is defined, the FinSH thread can be initialized by calling `finsh_system_init()` in the initialization thread. In RT-Thread 1.2.0 and later, you don't have to use the `finsh_set_device(const char* device_name)` function to explicitly specify the device to be used. Instead, the `rt_console_get_device()` function is called automatically to use the console device (The `finsh_set_device(const char* device_name)` must be used in 1.1.x and below to specify the device used by FinSH. The FinSH thread is created in the function `finsh_system_init()` function, which will wait for the rx_sem semaphore.

- FinSH output

The output of FinSH depends on the output of the system and relies on the `rt_kprintf()` output in RT-Thread. In the startup function `rt_hw_board_init()`, the `rt_console_set_device(const char* name)` function sets the FinSH printout device.

- FinSH input

After the rin_sem semaphore is obtained, the FinSH thread calls the `rt_device_read()` function to obtain a character from the device (select serial device) and then process it. So the migration of FinSH requires the implementation of the `rt_device_read()` function. The release of the rx_sem semaphore completes the input notification to the FinSH thread by calling the `rx_indicate()` function. The usual process is that when the serial port receive interrupt occurs (that is, the serial port has input character), the interrupt service routine calls the `rx_indicate()` function to notify the FinSH thread that there is input, and then the FinSH thread obtains the serial port input and finally performs the corresponding command processing.

1.4.4 utest Framework

1.4.4.1 utest Introduction

utest (unit test) is a unit testing framework developed by RT-Thread. The original intention of designing utest is to make it easier for RT-Thread developers to write test programs using a unified framework interface for unit testing,

coverage testing, and integration testing.

1.4.4.1.1 Test Case Definition

A test case (tc) is a single test performed to achieve a specific test objective. It is a specification that includes test input, execution conditions, test procedures, and expected results. It is a infinite loop with clear end conditions and test results.
The utest (unit test) framework defines user-written test programs as **test cases**, and a test case contains only one *testcase* function (similar to the main function), which can contain multiple *test unit* functions.
The test code for a function, specifically through the API provided by the utest framework, is a test case.

1.4.4.1.2 Test Unit Definition

The test unit is a test point subdivided by the function to be tested. Each test point can be the smallest measurable unit of the function to be tested. Of course, different classification methods will subdivide different test units.

1.4.4.1.3 utest Application Block Diagram

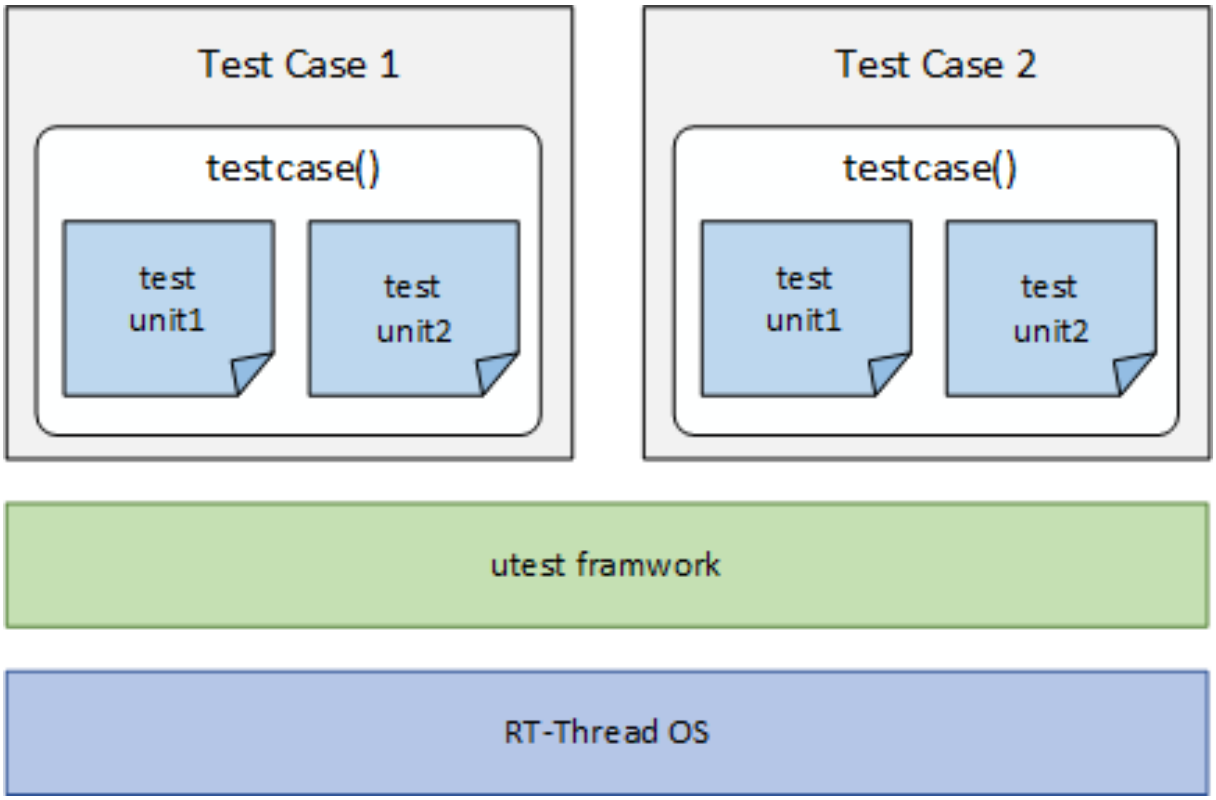


Figure 1.114 utest Application Block Diagram

As shown in the figure above, the test case is designed based on the service interface provided by the test framework utest, which supports compiling multiple test cases together for testing. In addition, as you can see from the figure, a test case corresponds to a unique *testcase* function, and multiple test units are included in *testcase*.

1.4.4.2 utest API

To enable uniform test case code, the test framework utest provides a common API interface for test case writing.

1.4.4.2.1 Macros of assertion

> NOTE: Here assert only records the number of passes and failures, it does not generate assertions or terminates program execution. Its function is not equivalent to RT_ASSERT.

assert Macro	Description
uassert_true(value)	If the value is true then the test passes, otherwise the test fails.
uassert_false(value)	If the value is false then the test passes, otherwise the test fails.
uassert_null(value)	If the Value is null then the test passes, otherwise the test fails
uassert_not_null(value)	If the value is a non-null value, the test passes, otherwise the test fails.
uassert_int_equal(a, b)	If the values of a and b are equal, the test passes, otherwise the test fails.
uassert_int_not_equal(a, b)	If the values of a and b are not equal, the test passes, otherwise the test fails.
uassert_str_equal(a, b)	If the string a and the string b are the same, the test passes, otherwise the test fails.
uassert_str_not_equal(a, b)	If the string a and the string b are not the same, the test passes, otherwise the test fails.
uassert_in_range(value, min, max)	If the value is in the range of min and max, the test passes, otherwise the test fails.
uassert_not_in_range(value, min, max)	If the value is not in the range of min and max, the test passes, otherwise the test fails.

1.4.4.2.2 Macros for Running Test Units

```
UTEST_UNIT_RUN(test_unit_func)
```

In the test case, the specified test unit function `test_unit_func` is executed using the `UTEST_UNIT_RUN` macro. The test unit must be executed using the `UTEST_UNIT_RUN` macro.

1.4.4.2.3 Macros for Exporting Test Cases

```
UTEST_TC_EXPORT(testcase, name, init, cleanup, timeout)
```

Parameters	Description
testcase	Test case main-bearing function (**specifies** using a function called *static void testcase(void)*
name	Test case name (uniqueness). Detailed requirements see below **Test case naming requirements:**
init	the initialization function before Test case startup
cleanup	Cleanup function after the end of the test case
timeout	Test case expected test time (in seconds)

Test case naming requirements:
To ensure unique unit-testcase names throughout the RT-Thread source code repository, the full name consists of two parts:
Module-Prefix.Test-Function

- **Module-Prefix**: Use the path of the *module's utest directory* relative to the source code repository root (excluding utest), connected by a dot ("."). For more information on *module's utest directory*, see section **How-to add utest cases into RT-Thread for your module**.
- **Test-Function**: Define your own name, ensuring it's unique within the same **Module-Prefix** (it doesn't have to match the test case file name).

Test case naming example:
Assuming that there is a *module's utest directory*: `components/dfs/utest`, which contains utest source files for module "DFS". We can define the testcase in file `components/dfs/utest/tc_dfs_api.c` as below:

```
static void testcase(void)
{
```

```
    /* Skip filesystem mount test for now due to mutex issues */
    UTEST_UNIT_RUN(test_mkfs);
    // ......
}

UTEST_TC_EXPORT(testcase, "components.dfs.fs_dfs_api_tc", utest_tc_init, utest_tc_cleanup, 10);
```

Here, the global unique unit-testcase name is "components.dfs.fs_dfs_api_tc". Of this name, **Module-Prefix** is "components.dfs", which corresponds to the path name `components/dfs`; **Test-Function** is `fs_dfs_api_tc`, which uniquely identifies a suite of cases which will be run in the function `testcase()`, and note that the `fs_dfs_api_tc` need not be the same as the file name of `tc_dfs_api.c`.
Particularly, for modules in the `src` directory, since the name `src` is ambiguous, it is recommended to replace it with `core` to contrast it with `components`. This indicates that the code in `src` is the kernel's *core* code module relative to `components`.
For example: The utest case in `src/klibc/utest/TC_rt_memcmp.c` can be written as:

```
UTEST_TC_EXPORT(utest_do_tc, "core.klibc.rt_memcmp", RT_NULL, RT_NULL, 1000);
```

1.4.4.2.4 Test Case LOG Output Interface

The utest framework relies on the *ulog log module* for log output and the log output level in the utest framework. So just add `#include "utest.h"` to the test case to use all level interfaces (LOG_D/LOG_I/LOG_E) of the ulog log module.
In addition, the utest framework adds an additional log control interface as follows:

```
#define UTEST_LOG_ALL    (1u)
#define UTEST_LOG_ASSERT (2u)

void utest_log_lv_set(rt_uint8_t lv);
```

Users can use the `utest_log_lv_set` interface to control the log output level in test cases. The `UTEST_LOG_ALL` configuration outputs all logs, and the `UTEST_LOG_ASSERT` configuration only outputs logs after the failure of uassert.

1.4.4.3 Configuration Enable

Using the utest framework requires the following configuration in the ENV tool using menuconfig:

```
RT-Thread Kernel  --->
    Kernel Device Object  --->
        (256) the buffer size for console log printf /* The minimum buffer required by the utest log */
RT-Thread Components  --->
    Utilities  --->
        -*- Enable utest (RT-Thread test framework) /* Enable utest framework */
        (4096) The utest thread stack size          /* Set the utest thread stack (required for -thread
      mode) */
        (20)   The utest thread priority            /* Set utest thread priority (required for -thread mode)
      */
```

1.4.4.4 Application Paradigm

The utest framework and related APIs were introduced earlier. The basic test case code structure is described here. The code blocks necessary for the test case file are as follows:

```
/*
 * Copyright (c) 2006-2019, RT-Thread Development Team
 *
 * SPDX-License-Identifier: Apache-2.0
 *
 * Change Logs:
 * Date           Author       Notes
 * 2019-01-16     MurphyZhao   the first version
 */

#include <rtthread.h>
#include "utest.h"

static void test_xxx(void)
{
    uassert_true(1);
}

static rt_err_t utest_tc_init(void)
{
    return RT_EOK;
}

static rt_err_t utest_tc_cleanup(void)
```

```
{
    return RT_EOK;
}

static void testcase(void)
{
    UTEST_UNIT_RUN(test_xxx);
}
UTEST_TC_EXPORT(testcase, "components.utilities.utest.sample.sample_tc", utest_tc_init, utest_tc_cleanup,
      10);
```

A basic test case must contain the following:

- File comment header (Copyright)

  The test case file must contain a file comment header containing `Copyright`, time, author, and description information.

- utest_tc_init(void)

  The initialization function before the test run is generally used to initialize the environment required for the test.

- utest_tc_cleanup(void)

  The cleanup function after the test is used to clean up the resources (such as memory, threads, semaphores, etc.) applied during the test.

- testcase(void)

  The mainly function of testcase, a test case implementation can only contain one testcase function (similar to the main function). Usually this function is only used to run the test unit execution function `UTEST_UNIT_RUN`.

  A testcase can contain multiple test units, each of which is executed by `UTEST_UNIT_RUN`.

- UTEST_UNIT_RUN

  Test unit execution function.

- test_xxx(void)

  Test implementation of each functional unit. The user determines the function name and function implementation based on the requirements.

- uassert_true

  The assertion macro used to determine the test result (this assertion macro does not terminate the program run). Test cases must use the `uassert_xxx` macro to determine the test results, otherwise the test framework does not know if the test passed.

  After all the `uassert_xxx` macros have been passed, the entire test case is passed.

- UTEST_TC_EXPORT

  Export the test case testcase function to the test framework.

1.4.4.5 Requirements for running test cases

The test framework utest exports all test cases to the `UtestTcTab` code segment. The `UtestTcTab` section is not required to be defined in the link script in the IAR and MDK compilers, but it needs to be explicitly set in the link script when GCC is compiled.
Therefore, in order for test cases to be compiled and run under GCC, the `UtestTcTab` code segment must be defined in the *link script* of GCC.
In the `.text` of the GCC link script, add the definition of the `UtestTcTab` section in the following format:

```
/* section information for utest */
. = ALIGN(4);
__rt_utest_tc_tab_start = .;
KEEP(*(UtestTcTab))
__rt_utest_tc_tab_end = .;
```

1.4.4.6 Running Test Cases

The test framework provides the following commands to make it easy for users to run test cases on the RT-Thread MSH command line. The commands are as follows:
***utest_list* command**
Lists the test cases supported by the current system, including the name of the test case and the time required for the test. This command has no parameters.
***utest_run* command**
Test case execution command, the format of the command is as follows:

```
utest_run [-thread or -help] [testcase name] [loop num]
```

utest_run Command Parameters	Description
-thread	Run the test framework in threaded mode
-help	Print help information
testcase name	Specify the test case name. Using the wildcard * is supported, specifying front byte of the test case name is supported.
loop num	the number of iterations of test cases

Example of test command usage:

```
msh />utest_list
[14875] I/utest: Commands list :
[14879] I/utest: [testcase name]:components.filesystem.dfs.dfs_api_tc; [run timeout]:30
[14889] I/utest: [testcase name]:components.filesystem.posix.posix_api_tc; [run timeout]:30
[14899] I/utest: [testcase name]:packages.iot.netutils.iperf.iperf_tc; [run timeout]:30
msh />
msh />utest_run components.filesystem.dfs.dfs_api_tc
[83706] I/utest: [==========] [ utest    ] started
[83712] I/utest: [----------] [ testcase ] (components.filesystem.dfs.dfs_api_tc) started
[83721] I/testcase: in testcase func...
[84615] D/utest: [    OK    ] [ unit     ] (test_mkfs:26) is passed
[84624] D/testcase: dfs mount rst: 0
[84628] D/utest: [    OK    ] [ unit     ] (test_dfs_mount:35) is passed
[84639] D/utest: [    OK    ] [ unit     ] (test_dfs_open:40) is passed
[84762] D/utest: [    OK    ] [ unit     ] (test_dfs_write:74) is passed
[84770] D/utest: [    OK    ] [ unit     ] (test_dfs_read:113) is passed
[85116] D/utest: [    OK    ] [ unit     ] (test_dfs_close:118) is passed
[85123] I/utest: [  PASSED  ] [ result   ] testcase (components.filesystem.dfs.dfs_api_tc)
[85133] I/utest: [----------] [ testcase ] (components.filesystem.dfs.dfs_api_tc) finished
[85143] I/utest: [==========] [ utest    ] finished
msh />
```

1.4.4.6.1 Test result analysis

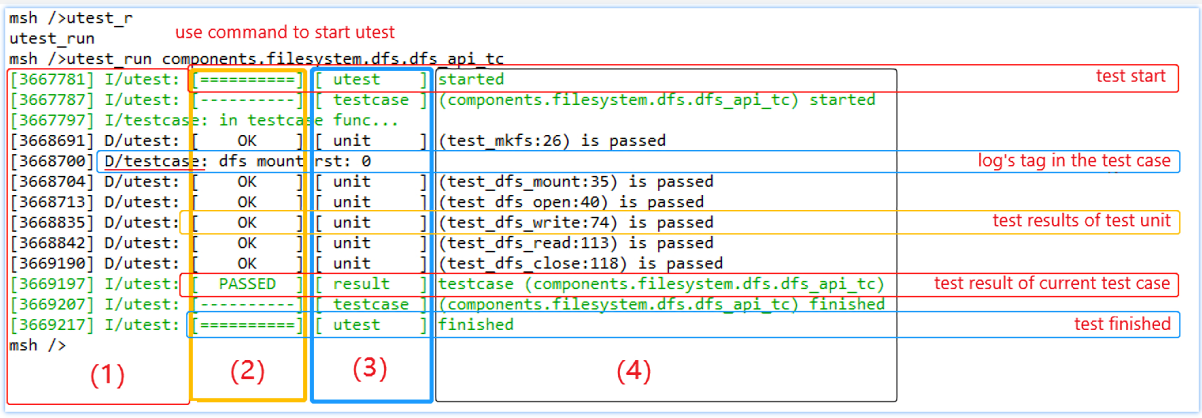


Figure 1.115 utest log display

As shown in the figure above, the log of the test case run is divided into four columns from left to right, which are `(1) log header information`, `(2) result bar`, `(3) property bar`, and `(4) detail information display bar`. The test case test result (PASSED or FAILED) is identified in the log using

the `result` attribute.

1.4.4.7 Test Case Run Process

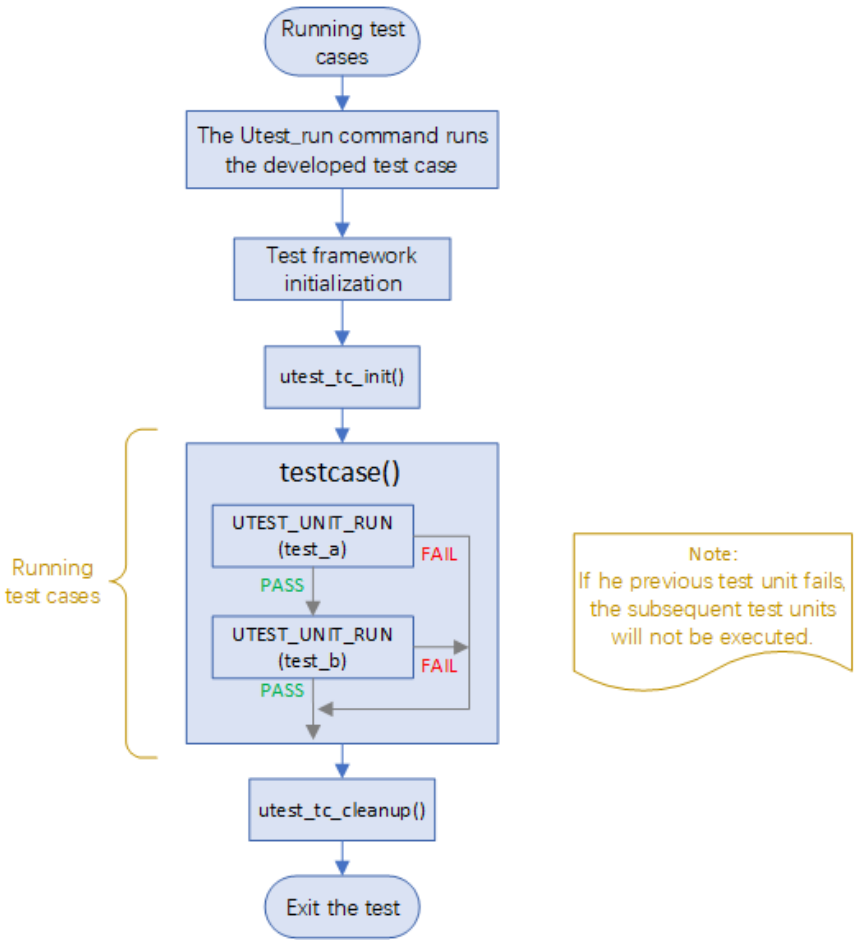


Figure 1.116 Test Case Run Process

From the above flow chart you can get the following:

- The utest framework is a sequential execution of all **test units** in the *testcase* function
- Assert of the previous UTEST_UNIT_RUN macro has occurred, and all subsequent UTEST_UNIT_RUN will skip executio

1.4.4.7.1 utest_run auto-completion Function

The utest_run tool now supports dynamic auto-completion for test case names. After entering the utest_run command in the terminal, users can press the TAB key to activate the auto-completion feature. The system will then dynamically match and suggest available test case names in real time.

```
msh />utest_run
dev
components.utili - (null)
testcases.kernel - (null)
testcases.kernel - (null)
testcases.kernel - (null)
testcases.kernel - (null)

msh />utest_run c
components.utilities.utest.uassert

msh />utest_run components.utilities.utest.uassert
[I/utest] [==========] [ utest    ] loop 1/1
[I/utest] [==========] [ utest    ] started
[I/utest] [----------] [ testcase ] (components.utilities.utest.uassert) started
[I/utest] [==========] utest unit name: (TC_uassert_true_false)
[I/utest] [==========] utest unit name: (TC_uassert_null_not_null)
[I/utest] [==========] utest unit name: (TC_uassert_int_op)
[I/utest] [==========] utest unit name: (TC_uassert_float_op)
[I/utest] [==========] utest unit name: (TC_uassert_ptr_op)
[I/utest] [==========] utest unit name: (TC_uassert_str_op)
[I/utest] [==========] utest unit name: (TC_uassert_in_range)
[I/utest] [  PASSED  ] [ result   ] testcase (components.utilities.utest.uassert)
[I/utest] [----------] [ testcase ] (components.utilities.utest.uassert) finished
[I/utest] [==========] [ utest    ] 1 tests from 5 testcase ran.
[I/utest] [  PASSED  ] [ result   ] 1 tests.
[I/utest] [==========] [ utest    ] finished
```

Figure 1.117 utest_run_auto_option

1.4.4.8 NOTE

- Determine whether the link script has the `UtestTcTab` section added before compiling with GCC.
- Make sure `RT-Thread Kernel -> Kernel Device Object -> (256) the buffer size for console log printf` is at least 256 bytes before compiling.
- The resources (threads, semaphores, timers, memory, etc.) created in the test case need to be released before the test ends.
- A test case implementation can only export a test body function (testcase function) using `UTEST_TC_EXPORT`
- Write a `README.md` document for the your test case to guide the user through configuring the test environment.

1.4.4.9 How-to add utest cases into RT-Thread for your module

The source code of utest cases is recommended to be placed in each module for maintenance, but the entry of Kconfig should all be placed(rsourced) in `components/utilities/utest/Kconfig` for unified maintenance. In this way, when executing menuconfig, people can enter and configure from one place, avoiding searching for utest configuration switches scattering in the menuconfig interface.
For each module, you can maintain unit testcases in a unified manner in the following way:

- Create a subdirectory named 'utest' in the directory where your module is located.
- Store the following files in the 'utest' subdirectory:
  - Unit-testcase program source code files for this module.
  - `Kconfig` file, which defining configuration options for the unit test cases of this module, the recommended option is named `RT_UTEST_XXXX`, XXXX is the global unique module name of this module.
  - `SConscript` file, when adding src files, you need to rely on `RT_UTEST_XXXX`.

After completing the above steps, rsource the path of the Kconfig file of utest of this module to the file `components/utilities/utest/Kconfig`.
You can find a good example in `"klibc"`.

1.4.5 Dynamic Module: dlmodule

In traditional desktop operating systems, user space and kernel space are separate. The application runs in user space, and the kernel and kernel modules run in kernel space. The kernel module can be dynamically loaded and deleted to extend the kernel functionality. `dlmodule` is a software component of the dynamic module loading mechanism provided in kernel space of RT-Thread. In versions of RT-Thread v3.1.0, this was also called the `Application Module`. After RT-Thread v3.1.0 and later, it returned to the tradition and was named after the `dynamic module`.
`dlmodule` is more of an ELF format loader. The code segment and data segment of a separately compiled elf file are loaded into memory, and the symbols are parsed and bound to the API address exported by the kernel. The elf files are primarily placed on file systems under RT-Thread.

1.4.5.1 Introduction

The dynamic module provides a mechanism for dynamically loading program modules for RT-Thread. Because it is also compiled independently of the kernel, it is more flexible to use. In terms of implementation, this is a mechanism to separate the kernel from the dynamic modules. Through this mechanism, the kernel and dynamic modules can be compiled separately, and at runtime, the compiled dynamic modules are loaded into the kernel through the module loader in the kernel.
In the dynamic module of RT-Thread, two formats are currently supported:

- `.mo` is an executable dynamic module that is suffixed with `.mo` when compiled; it can be loaded, and a main thread is automatically created in the system to execute `main()` function in this dynamic module; at the same time, `main(int argc, char**argv)` can also accept arguments on the command line.
- `.so` is a dynamic library compiled with `.so` as a suffix; it can be loaded and resided in memory, and some set of functions is used by other programs (code or dynamic modules in the kernel).

The current RT-Thread architecture supporting dynamic modules mainly includes ARM architecture and x86 architecture, and will be extended to MIPS and RISC-V architecture in the future. The RT-Thread kernel firmware section can use a variety of compiler toolchains, such as GCC, ARMCC, IAR and other toolchains; however, dynamic module partial compilation currently only supports GNU GCC toolchain compilation. Therefore, compiling the RT-Thread module requires downloading GCC tools, such as CodeSourcery's arm-none-eabi toolchain. In general, it's best to use kernel and dynamic modules to compile with the same toolchain (so that it doesn't produce inconsistent behavior in *libc*). In addition, dynamic modules can only be loaded into RAM and used for symbol resolution binding to the API address exported by the kernel. Instead of running directly in XIP mode based on Flash (because Flash can't modify the code segment again).

1.4.5.2 Using Dynamic Module

When you want to use the dynamic modules in your system, you need to compile a firmware that supports dynamic modules, as well as dynamic modules that need to be run. The following two parts are compiling firmware and compiling dynamic modules.

1.4.5.2.1 Compile Firmware

When you want to use the dynamic module, you need to open the corresponding option in the firmware configuration, use menuconfig to open the following configuration:

```
RT-Thread Components  --->
    POSIX layer and C standard library  --->
        [*] Enable dynamic module with dlopen/dlsym/dlclose feature
```

Also open the configuration options of the file system:

```
RT-Thread Components  --->
     Device virtual file system  --->
           [*] Using device virtual file system
```

The configuration parameters required for dynamic module compilation are set in *rtconfig.py* file corresponding to bsp:

```
M_CFLAGS = CFLAGS + '-mlong-calls -fPIC'
M_CXXFLAGS = CXXFLAGS + '-mlong-calls -fPIC'
M_LFLAGS = DEVICE + CXXFLAGS + '-Wl,--gc-sections,-z,max-page-size=0x4' +\
                               '-shared -fPIC -nostartfiles -nostdlib -static-libgcc'
M_POST_ACTION = STRIP + '-R .hash $TARGET\n' + SIZE + '$TARGET \n'
M_BIN_PATH = r'E:\qemu-dev310\fatdisk\root'
```

The relevant explanation is as follows:

- **M_CFLAGS**: C code compilation parameters used in dynamic module compilation, generally compiled here in PIC mode (that is, the code address supports floating mode execution);
- **M_CXXFLAGS**: C++ code compilation parameters used in dynamic module compilation, parameters similar to M_CFLAGS above;
- **M_LFLAGS**: Parameters when the dynamic module is linked. The same is the PIC method, and is linked in a shared library (partial link);
- **M_POST_ACTION**: the action to be performed after the dynamic module is compiled. Here, the elf file is stripped to reduce the size of the elf file.
- **M_BIN_PATH**: When the dynamic module is successfully compiled, whether the corresponding dynamic module file needs to be copied to a unified place;

Basically, these compilation configuration parameters for the ARM9, Cortex-A, and Cortex-M series are the same.
The kernel firmware also exports some function APIs to the dynamic module via `RTM(function)`. All exported symbols information in firmware can be listed by command A under MSH:

```
list_symbols
```

The `dlmodule` loader also parses the symbols that need to be parsed in the dynamic module according to the symbol table exported here to complete the final binding action.
This symbol table will be placed in a special section named `RTMSymTab`, so the corresponding firmware link script also needs to retain this area, and will not be removed by the linker optimization. You can add the corresponding information in the link script:

```
/* section information for modules */
. = ALIGN(4);
__rtmsymtab_start = .;
KEEP(*(RTMSymTab))
__rtmsymtab_end = .;
```

Then execute the `scons` under the BSP project directory and generate the firmware without errors. Execute the command in the BSP project directory:

```
scons --target=ua -s
```

to generate the kernel header file search path and global macro definitions that need to be included when compiling the dynamic module.

1.4.5.2.2 Compile Dynamic Module

There is a separate repository on github: `rtthread-apps` , which contains some examples of dynamic modules and dynamic libraries.
Its directory structure is as follows:

Directory	Description
cxx	Demonstrates how to program in C++ using dynamic modules
hello	A "hello world" example
lib	Example of dynamic library
md5	Generate md5 code for a file
tools	*Python/SConscript* scripts required for dynamic module compilation
ymodem	Download a file to the file system through the serial port using the YModem protocol

You can clone this repository locally and compile it with the scons from the command line. For Windows platforms, the ENV tool is recommended.
After entering the console command line, enter the directory where the `rtthread-apps` repo is located (the same, please ensure that the full path of this directory does not contain spaces, Chinese characters, etc.), and set two variables:

- RTT_ROOT: points to the root directory of the RT-Thread code;
- BSP_ROOT: points to the project directory of the BSP;

Use follow commands on Windows in Env tool(assuming the BSP used is qemu-vexpress-a9):

```
set RTT_ROOT=d:\your_rtthread
set BSP_ROOT=d:\your_rtthread\bsp\qemu-vexpress-a9
```

to set the corresponding environment variable. Then use the following command to compile the dynamic module, such as the hello example:

```
scons --app=hello
```

After compiling successfully, it will generate a `hello.mo` file in the `rtthread-apps/hello` directory.
You can also compile dynamic libraries, such as the lib example:

```
scons --lib=lib
```

After compiling successfully, it will generate the `lib.so` file in the `rtthread-apps/lib` directory.
We can put these `mo` and `so` files under the RT-Thread file system. Under msh, you can simply execute the `hello.mo` dynamic module as a `hello` command:

```
msh />ls
Directory /:
hello.mo            1368
lib.so              1376
msh />hello
msh />Hello, world
```

After calling hello, the main function in `hello.mo` will be executed, and the corresponding dynamic module will be exited after execution. The code for `hello/main.c` is as follows:

```
#include <stdio.h>

int main(int argc, char *argv[])
{
    printf("Hello, world\n");

    return 0;
}
```

1.4.5.3 APIs of Dynamic Module

In addition to dynamically loading and executing dynamic modules via msh, dynamic modules can be loaded or unloaded using the dynamic module API provided by RT-Thread in the main program.

1.4.5.3.1 Load Dynamic Module

```
struct rt_dlmodule *dlmodule_load(const char* pgname);
```

Parameter	Description
pgname	Dynamic module path
Return	——
Dynamic module pointer	Successfully loaded
RT_NULL	Failed

This function loads the dynamic module from the file system into memory, and if it is loaded correctly, returns a pointer to the module. This function does not create a thread to execute this dynamic module, just load the module into memory and parse the symbolic address.

1.4.5.3.2 Execute Dynamic Module

```
struct rt_dlmodule *dlmodule_exec(const char* pgname, const char* cmd, int cmd_size);
```

Parameter	Description
pgname	Dynamic module path
cmd	Command line string including the dynamic module command itself
cmd_size	Command line string size
Return	——
Dynamic module pointer	Run successfully

Parameter	Description
RT_NULL	Failed

This function loads the dynamic module according to the `pgname` path and starts a thread to execute `main` of the dynamic module. At the same time, `cmd` is passed as the command line Parameter to `main` entry of the dynamic module.

1.4.5.3.3 Exit Dynamic Module

```
void dlmodule_exit(int ret_code);
```

Parameter	Description
ret_code	Module's return parameter

This function is called by the module runtime, it can set the return value of the module exit `ret_code`, and then exit from the module.

1.4.5.3.4 Find Dynamic Modules

```
struct rt_dlmodule *dlmodule_find(const char *name);
```

Parameter	Description
name	Module name
Return	——
Dynamic module pointer	Successful
RT_NULL	Failed

This function uses `name` to find out if there is already a dynamic module loaded in the system.

1.4.5.3.5 Return Dynamic Module

```
struct rt_dlmodule *dlmodule_self(void);
```

Return	Description
Dynamic module pointer	Successful
RT_NULL	Failed

This function returns a pointer of the dynamic module in the calling context.

1.4.5.3.6 Find Symbol

```
rt_uint32_t dlmodule_symbol_find(const char *sym_str);
```

Parameter	Description
sym_str	Symbol name
Return	——
Symbol address	Successful
0	Failed

This function returns the symbol address based on the symbol name.

1.4.5.4 Libdl API of POSIX Standard

The POSIX standard libdl API is also supported in RT-Thread dlmodule. It is similar to loading a dynamic library into memory (and parsing some of the symbol information). This dynamic library provides the corresponding set of function operations. The libdl API needs to include the header files: `#include <dlfcn.h>`

1.4.5.4.1 Open Dynamic Library

```
void * dlopen (const char * pathname, int mode);
```

Parameter	Description
pathname	Dynamic library path name
mode	The mode when opening a dynamic library, not used in RT-Thread
Return	——
Dynamic library handle (`struct dlmodule` structure pointer)	Successful
NULL	Failed

This function is similar to the `dlmodule_load` , which loads the dynamic library from the file system and returns the handle pointer of the dynamic library.

1.4.5.4.2 Find Symbol

```
void* dlsym(void *handle, const char *symbol);
```

Parameter	Description
handle	Dynamic library handle, return value of `dlopen`
symbol	The symbol address to return
Return	——
symbol address	Successful
NULL	Failed

This function looks in the dynamic library `handle` for the presence of the symbol of `symbol` , if there is an address that returns it.

1.4.5.4.3 Close Dynamic Library

```
int dlclose (void *handle);
```

Parameter	Description
handle	Dynamic library handle
Return	——
0	Successful
Negative number	Failed

This function closes the dynamic library pointed to by `handle` and unloads it from memory. It should be noted that when the dynamic library is closed, the symbolic address originally returned by `dlsym` will no longer be available. If you still try to access it, it may cause a fault error.

1.4.5.5 FAQs

Please refer to Env for issues related to the Env tool.

1.4.5.5.1 Q: Dynamic modules cannot be run successfully according to the documentation.

A: Please update the RT-Thread source code to version 3.1.0 and above.

1.4.5.5.2 Q: Compile the project with the scons command, prompting "undefined reference to __rtmsymtab_start".

A: Please refer to the qemu-vexpress-a9 BSP GCC link script file *link.lds* to add the following to the TEXT section of the project's GCC link script.

```
/* section information for modules */
. = ALIGN(4);
__rtmsymtab_start = .;
KEEP(*(RTMSymTab))
__rtmsymtab_end = .;
```

1.4.6 Socket Abstraction Layer: SAL

1.4.6.1 SAL Introduction

In order to adapt to more network protocol stack types and avoid the system's dependence on a single network protocol stack, the RT-Thread system provides a SAL (Socket Abstraction Layer) components that implement different network protocol stacks or network implementations. The abstraction of the interface provides a set of standard BSD Socket APIs to the upper layer, so that developers only need to care about and use the network interface provided by the network application layer, without concern for the underlying specific network protocol stack type and implementation, which greatly improves the system's compatibility makes it easy for developers to complete protocol stack adaptation and network-related development. Main features of the SAL component are as follows:

- Abstract and unified multiple network protocol stack interfaces;
- Provide Socket-level TLS encrypted transport feature;
- Support standard BSD Socket APIs
- Unified FD management for easy operation of network functions using read/write poll/select

1.4.6.1.1 SAL Network Framework

The SAL network framework of RT-Thread is mainly shown in the following structure:

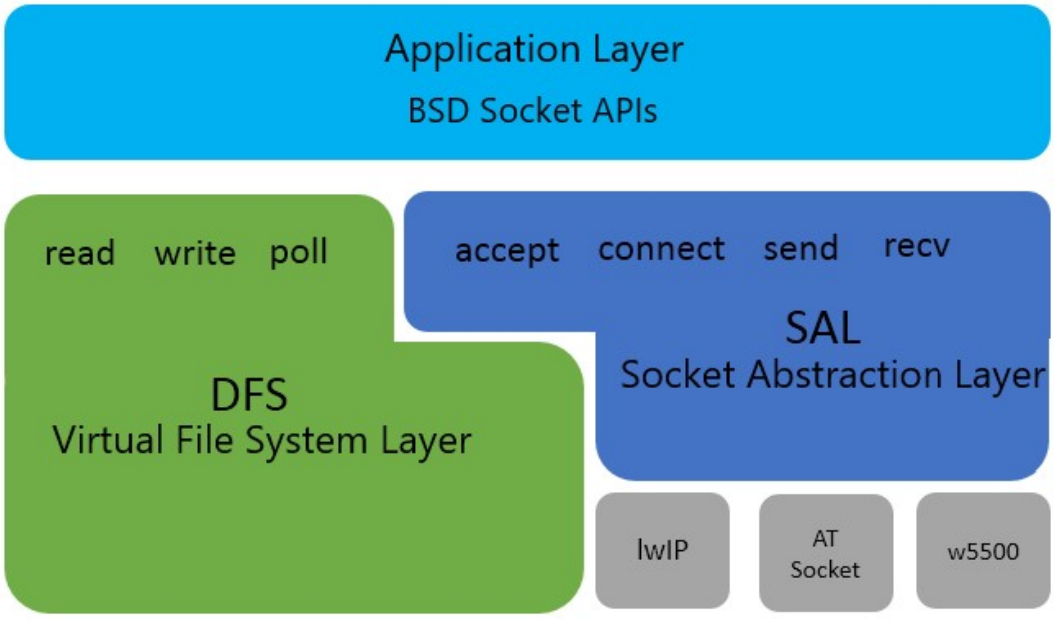


Figure 1.118 SAL

The top layer is the network's application layer, provides a set of standard BSD Socket APIs, such as `socket`, `connect` and other functions, for most of the system network development applications.
The second part is the virtual file system layer. In the RT-Thread system, the DFS file system program can implement different file system operations using standard interface functions.The network socket interface also supports the file system structure. The network socket descriptor created when using the network socket interface is uniformly managed by the file system. Therefore, the network socket descriptor can also use the standard file operation interface.The interfaces provided for the upper application layer are: `read`, `write`, `close`, `poll/select`, and so on.
The third part is the socket abstraction layer, through which RT-thread system can adapt to different network protocol stacks in the lower layer and provide unified network programming interface in the upper layer to facilitate access of different protocol stacks. The socket abstraction layer provides interfaces for the upper application layer, such as `accept`, `connect`, `send`, `recv`, etc.
The fourth part is the protocol stack layer, which includes several commonly used TCP/IP protocol stacks, such as lwIP, a light TCP/IP protocol stack commonly used in embedded development, and AT Socket network function implementation developed by rt-thread. These protocol stacks or network functions directly contact the hardware to complete the transformation of data from the network layer to the transmission layer.
The network application layer of RT-thread provides interfaces mainly based on the standard BSD Socket API, which ensures that programs can be written on PC, debugged, and then ported to the RT-thread operating system.

1.4.6.1.2 Working Principles

The working principle of SAL component is mainly divided into the following three parts:

- Unified abstract functions of multi-protocol stack access and interface functions
- SAL TLS encryption transmission function

1.4.6.1.2.1 Multi-Protocol Stack Access and Unified Abstract Function Of Interface Function

For different protocol stacks or network function implementations, the names of network interfaces may be different. Take the `connect` connection function as an example. The interface name in the lwIP protocol stack is `lwip_connect`, and the interface name in the AT Socket network implementation is `at_connect`. The SAL component

provides abstraction and unification of the interface of different protocol stacks or networks. When the socket is created, the component **judges the protocol stack or network function used by judging the incoming protocol domain type**, and completes the RT-Thread.
Currently, the protocol stack or network implementation types supported by the SAL component are: **lwIP protocol stack**, **AT Socket protocol stack**, **WIZnet hardware TCP/IP protocol stack**.

```
int socket(int domain, int type, int protocol);
```

The above is the definition of the socket creation function in the standard BSD Socket API. The `domain` indicates that the protocol domain is also called the protocol domain. It is used to determine which protocol stack or network implementation to use. The domain type used by the AT Socket protocol stack is **AF_AT**, lwIP protocol stack uses protocol domain type **AF_INET**, etc., WIZnet protocol stack uses protocol domain type **AF_WIZ**.
For different software packages, the protocol domain type passed to the socket may be fixed and will not change depending on how the SAL component is accessed. **In order to dynamically adapt access to different protocol stacks or network implementations**, the SAL component provides two protocol domain type matching methods for each protocol stack or network implementation: **Primary protocol domain type and secondary protocol domain type**. When socket is created, it first determines whether the incoming protocol domain type has the supported primary protocol type. If it is, it uses the corresponding protocol stack or network implementation, if not, determine whether the subprotocol cluster type supports. The current system support protocol domain types are as follows:

1. lwIP Protocol stack family = AF_INETsec_family = AF_INET
2. AT Socket Protocol stack family = AF_ATsec_family = AF_INET
3. WIZnet hardware TCP/IP Protocol stack family = AF_WIZsec_family = AF_INET

The main function of the SAL component is to unify the underlying BSD Socket API interface. The following takes the `connect` function call flow as an example to illustrate the SAL component function call method:

- `connect`: The abstract BSD Socket API provided by the SAL component for unified FD management
- `sal_connect`: The `connect` implementation function in the SAL component that is used to call the `operation` function registered by the underlying stack.
- `lwip_connect`: The layer `connect` connection function provided by the underlying protocol stack is registered in the SAL component when the NIC is initialized, and the final call operation function

```
/* SAL component provides BSD Socket APIs for the application layer  */
int connect(int s, const struct sockaddr *name, socklen_t namelen)
{
    /* Get the SAL socket descriptor */
    int socket = dfs_net_getsocket(s);

    /* Execute the sal_connect function with a SAL socket descriptor*/
    return sal_connect(socket, name, namelen);
}

/* SAL component abstract function interface implementation */
int sal_connect(int socket, const struct sockaddr *name, socklen_t namelen)
{
    struct sal_socket *sock;
    struct sal_proto_family *pf;
    int ret;

    /* Check if the SAL socket structure is normal */
    SAL_SOCKET_OBJ_GET(sock, socket);

    /* Check if the current socket network connection status is normal. */
    SAL_NETDEV_IS_COMMONICABLE(sock->netdev);
    /* Check if the underlying operation function corresponding to the current socket is normal.  */
    SAL_NETDEV_SOCKETOPS_VALID(sock->netdev, pf, connect);

    /* The connect operation function that performs the underlying registration */
    ret = pf->skt_ops->connect((int) sock->user_data, name, namelen);
#ifdef SAL_USING_TLS
    if (ret >= 0 && SAL_SOCKOPS_PROTO_TLS_VALID(sock, connect))
    {
        if (proto_tls->ops->connect(sock->user_data_tls) < 0)
        {
            return -1;
        }
        return ret;
    }
#endif
    return ret;
```

```
}

/* The underlying connect function of the lwIP protocol stack function */
int lwip_connect(int socket, const struct sockaddr *name, socklen_t namelen)
{
    ...
}
```

1.4.6.1.2.2 SAL TLS Encrypted Transmission Function

1. SAL TLS Feature

In the transmission of protocol data such as TCP and UDP, since the data packet is plaintext, it is likely to be intercepted and parsed by others, which has a great impact on the secure transmission of information. In order to solve such problems, users generally need to add the SSL/TLS protocol between the application layer and the transport layer.

TLS (Transport Layer Security) is a protocol based on the transport layer TCP protocol. Its predecessor is SSL (Secure Socket Layer). Its main function is to encrypt the application layer message asymmetrically and then transmit it by TCP protocol, so as to achieve secure data encryption interaction.

Currently used TLS methods: **MbedTLS, OpenSSL, s2n**, etc., but for different encryption methods, you need to use their specified encryption interface and process for encryption, the migration of some application layer protocols is more complicated. Therefore, the SAL TLS function is generated, the main function is to **provide TLS-encrypted transmission characteristics at the Socket level, abstract multiple TLS processing methods, and provide a unified interface for completing TLS data interaction**.

2. How to use the SAL TLS feature

The process is as follows:

- Configure to enable any network protocol stack support (such as lwIP protocol stack)
- Configure to enable the MbedTLS package (currently only supports MbedTLS type encryption)
- Configure to enable SAL_TLS support (as shown in the configuration options section below)

After the configuration is complete, as long as the `protocol` type passed in the socket creation uses **PROTOCOL_TLS** or **PROTOCOL_DTLS **, the standard BSD Socket API interface can be used to complete the establishment of the TLS connection and the data transmission and reception. The sample code is as follows:

```
#include <stdio.h>
#include <string.h>

#include <rtthread.h>
#include <sys/socket.h>
#include <netdb.h>

/* RT-Thread offical websitesupporting TLS function */
#define SAL_TLS_HOST    "www.rt-thread.org"
#define SAL_TLS_PORT    443
#define SAL_TLS_BUFSZ   1024

static const char *send_data = "GET /download/rt-thread.txt HTTP/1.1\r\n"
    "Host: www.rt-thread.org\r\n"
    "User-Agent: rtthread/4.0.1 rtt\r\n\r\n";

void sal_tls_test(void)
{
    int ret, i;
    char *recv_data;
    struct hostent *host;
    int sock = -1, bytes_received;
    struct sockaddr_in server_addr;

    /* Get the host address through the function entry parameter url (if it is a domain name, it will do
      domain name resolution) */
    host = gethostbyname(SAL_TLS_HOST);

    recv_data = rt_calloc(1, SAL_TLS_BUFSZ);
    if (recv_data == RT_NULL)
    {
        rt_kprintf("No memory\n");
        return;
    }

    /* Create a socket of type SOCKET_STREAM, TCP protocol, TLS type */
    if ((sock = socket(AF_INET, SOCK_STREAM, PROTOCOL_TLS)) < 0)
    {
```

```
        rt_kprintf("Socket error\n");
        goto __exit;
    }

    /* Initialize the server address */
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(SAL_TLS_PORT);
    server_addr.sin_addr = *((struct in_addr *)host->h_addr);
    rt_memset(&(server_addr.sin_zero), 0, sizeof(server_addr.sin_zero));

    if (connect(sock, (struct sockaddr *)&server_addr, sizeof(struct sockaddr)) < 0)
    {
        rt_kprintf("Connect fail!\n");
        goto __exit;
    }

    /* Send data to the socket connection */
    ret = send(sock, send_data, strlen(send_data), 0);
    if (ret <= 0)
    {
        rt_kprintf("send error,close the socket.\n");
        goto __exit;
    }

    /* Receive and print the response data, using encrypted data transmission */
    bytes_received = recv(sock, recv_data, SAL_TLS_BUFSZ  - 1, 0);
    if (bytes_received <= 0)
    {
        rt_kprintf("received error,close the socket.\n");
        goto __exit;
    }

    rt_kprintf("recv data:\n");
    for (i = 0; i < bytes_received; i++)
    {
        rt_kprintf("%c", recv_data[i]);
    }

__exit:
    if (recv_data)
        rt_free(recv_data);

    if (sock >= 0)
        closesocket(sock);
}

#ifdef FINSH_USING_MSH
#include <finsh.h>
MSH_CMD_EXPORT(sal_tls_test, SAL TLS function test);
#endif /* FINSH_USING_MSH */
```

1.4.6.1.3 Configuration Options

When we use the SAL component we need to define the following macro definition in rtconfig.h

Macro definition	Description
RT_USING_SAL	Enable the SAL function
SAL_USING_LWIP	Enable lwIP stack support
SAL_USING_AT	Enable AT Socket protocol stack support
SAL_USING_TLS	Enable SAL TLS feature support
SAL_USING_POSIX	Enable POSIX file system related function support, such as read, write, select/poll, etc.

Currently, the SAL abstraction layer supports the lwIP protocol stack, the AT Socket protocol stack, and the WIZnet hardware TCP/IP protocol stack. To enable SAL in the system, at least one protocol stack support is required. The above configuration options can be added directly to the `rtconfig.h` file or added by the component package management tool Env configuration option. The specific configuration path in the Env tool is as follows:

```
RT-Thread Components  --->
    Network  --->
        Socket abstraction layer  --->
        [*] Enable socket abstraction layer
            protocol stack implement --->
            [ ] Support lwIP stack
            [ ] Support AT Commands stack
            [ ] Support MbedTLS protocol
```

```
    [*]    Enable BSD socket operated by file system API
```

After the configuration is complete, you can use the `scons` command to regenerate the function and complete the addition of the SAL component.

1.4.6.2 Initialization

SAL SAL

```
int sal_init(void);
```

The initialization function is mainly for initializing the SAL component, supporting the component to repeatedly initialize the judgment, and completing the initialization of the resource such as the mutex used in the component. There is no new thread created in the SAL component, which means that the SAL component resource is very small. Currently, the **SAL component resource is occupied by ROM 2.8K and RAM 0.6K**.

1.4.6.3 BSD Socket API Introduction

The SAL component abstracts the standard BSD Socket API interface. The following is an introduction to common network interfaces:

1.4.6.3.1 Create a Socket (socket)

```
int socket(int domain, int type, int protocol);
```

Parameter	Description
domain	protocol domain type
type	protocol type
protocol	Transport layer protocol for actual use
back	–
>=0	Success, an integer representing the socket descriptor will be returned
-1	Fail

This function is used to assign a socket descriptor and the resources it USES based on the specified address family, data type, and protocol.
domain (protocol domain type)

- AF_INET IPv4
- AF_INET6 IPv6

type (protocol type)

- SOCK_STREAMStream socket
- SOCK_DGRAM Datagram socket
- SOCK_RAW Raw socket

1.4.6.3.2 Bind Socket (bind)

```
int bind(int s, const struct sockaddr *name, socklen_t namelen);
```

Parameter	Description
s	socket descriptor
name	a pointer to the sockaddr structure representing the address to bind to
namelen	length of sockaddr structure
back	–

Parameter	Description
0	success
-1	fail

This function is used to bind the port number and IP address to the specified socket
SAL components depend on the `netdev` components, when using `bind ()` function, can get IP address information through the netdev nic name, is used to create Socket bound to the specified object of network interface card. The following example completes the process of binding the IP address of the network interface card and connecting to the server through the name of the incoming network interface card

```
#include <rtthread.h>
#include <arpa/inet.h>
#include <netdev.h>

#define SERVER_HOST    "192.168.1.123"
#define SERVER_PORT    1234

static int bing_test(int argc, char **argv)
{
    struct sockaddr_in client_addr;
    struct sockaddr_in server_addr;
    struct netdev *netdev = RT_NULL;
    int sockfd = -1;

    if (argc != 2)
    {
        rt_kprintf("bind_test [netdev_name]  --bind network interface device by name.\n");
        return -RT_ERROR;
    }

    /* Get the netdev network interface card object by name */
    netdev = netdev_get_by_name(argv[1]);
    if (netdev == RT_NULL)
    {
        rt_kprintf("get network interface device(%s) failed.\n", argv[1]);
        return -RT_ERROR;
    }

    if ((sockfd = socket(AF_INET, SOCK_STREAM, 0)) < 0)
    {
        rt_kprintf("Socket create failed.\n");
        return -RT_ERROR;
    }

    /* Initializes the client address to bind to */
    client_addr.sin_family = AF_INET;
    client_addr.sin_port = htons(8080);
    /* Gets the IP address information in the network interface card object */
    client_addr.sin_addr.s_addr = netdev->ip_addr.addr;
    rt_memset(&(client_addr.sin_zero), 0, sizeof(client_addr.sin_zero));

    if (bind(sockfd, (struct sockaddr *)&client_addr, sizeof(struct sockaddr)) < 0)
    {
        rt_kprintf("socket bind failed.\n");
        closesocket(sockfd);
        return -RT_ERROR;
    }
    rt_kprintf("socket bind network interface device(%s) success!\n", netdev->name);

    /* Initializes the server address for the pre-connection */
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(SERVER_PORT);
    server_addr.sin_addr.s_addr = inet_addr(SERVER_HOST);
    rt_memset(&(server_addr.sin_zero), 0, sizeof(server_addr.sin_zero));

    /* Connect to the server */
    if (connect(sockfd, (struct sockaddr *)&server_addr, sizeof(struct sockaddr)) < 0)
    {
        rt_kprintf("socket connect failed!\n");
        closesocket(sockfd);
        return -RT_ERROR;
    }
    else
    {
        rt_kprintf("socket connect success!\n");
    }

    /* Close the connection */
    closesocket(sockfd);
    return RT_EOK;
```

```
}

#ifdef FINSH_USING_MSH
#include <finsh.h>
MSH_CMD_EXPORT(bing_test, bind network interface device test);
#endif /* FINSH_USING_MSH */
```

1.4.6.3.3 Listen Socket (listen)

```
int listen(int s, int backlog);
```

Parameter	Description
s	socket descriptor
backlog	represents the maximum number of connections that can wait at a time
back	–
0	success
-1	fail

This function is used by the TCP server to listen for a specified socket connection

1.4.6.3.4 Accept Connection (accept)

```
int accept(int s, struct sockaddr *addr, socklen_t *addrlen);
```

Parameter	Descrption
s	socket descriptor
addr	represents the maximum number of connections that can wait at a time
addrlen	length of client device address structure
back	–
0	success, return the newly created socket descriptor
-1	fail

When the application listens for connections from other hosts, the connection is initialized with the `accept()` function, and `accept()` creates a new socket for each connection and removes the connection from the listen queue.

1.4.6.3.5 Establish Connection (connect)

```
int connect(int s, const struct sockaddr *name, socklen_t namelen);
```

Parameter	Description
s	socket descriptor
name	server address information
namelen	server address structure length
back	–
0	successful, return the newly created socket descriptor
-1	fail

This function is used to establish a connection to the specified socket.

1.4.6.3.6 Send TCP Data (send)

```
int send(int s, const void *dataptr, size_t size, int flags);
```

Parameter	Description
s	socket descriptor
dataptr	sent data pointer
size	length of sent data
flags	flags, usually 0
back	–
>0	success, the length of the sent data is returned
<=0	Fail

This function is commonly used to send data over a TCP connection

1.4.6.3.7 Receive TCP Data (recv)

```
int recv(int s, void *mem, size_t len, int flags);
```

Parameter	Description
s	socket descriptor
mem	received data pointer
len	length of received data
flags	flags, usually 0
back	–
>0	success, the length of the received data will be returned
=0	the destination address has been transmitted and the connection is closed
<0	fail

This function is used to receive data over a TCP connection

1.4.6.3.8 Send UDP Data (sendto)

```
int sendto(int s, const void *dataptr, size_t size, int flags, const struct sockaddr *to, socklen_t tolen);
```

Parameter	Description
s	socket descriptor
dataptr	sent data pointer
size	length of sent data
flags	flags, usually 0
to	target address structure pointer
tolen	length of the target address structure
return	–
>0	success, the length of the received data will be returned
<=0	fail

This function is used for UDP connections to send data

1.4.6.3.9 Receive UDP Data (recvfrom)

```
int recvfrom(int s, void *mem, size_t len, int flags, struct sockaddr *from, socklen_t *fromlen);
```

Parameter	Description
s	socket descriptor
mem	received data pointer
len	length of data received
flags	flags, usually 0
from	received address structure pointer
fromlen	length of received address structure
back	–
>0	success, return the length of the received data
=0	the receiving address has been transferred and the connection is closed
<0	fail

This function is used to receive data on a UDP connection

1.4.6.3.10 Close Socket (closesocket

```
int closesocket(int s);
```

Parameter	Description
s	Socket descriptor
back	–
0	success
-1	fail

This function is used to close the connection and release the resource.

1.4.6.3.11 Shutdown The Socket By Settingshutdown

```
int shutdown(int s, int how);
```

Parameter	Descrption
s	socket descriptor
how	socket control
back	–
0	fail
-1	success

This function provides more permissions to control the closing process of the socket.
How (socket control)

- 0 Stop receiving current data and reject future data reception
- 1 Stop sending data and discard unsent data
- 2 Stop receiving and sending data

1.4.6.3.12 Set Socket Optionssetsockopt

```
int setsockopt(int s, int level, int optname, const void *optval, socklen_t optlen);
```

Parameter	Description
s	socket descriptor
level	protocol stack configuration options
optname	option name to be set
optval	set the buffer address of the option value
optlen	set the buffer length of the option value
back	–
=0	success
<0	fail

This function is used to set the socket mode and modify the socket configuration options.
level (protocol stack configuration options)

- SOL_SOCKETSocket layer
- IPPROTO_TCPTCP layer
- IPPROTO_IPIP layer

optname (Option name to be set):

- SO_KEEPALIVESet keep alive options
- SO_RCVTIMEOSet socket data reception timeout
- SO_SNDTIMEOSet socket data sending timeout

1.4.6.3.13 Get Socket Optionsgetsockopt

```
int getsockopt(int s, int level, int optname, void *optval, socklen_t *optlen);
```

Parameter	Description
s	socket descriptor
level	protocol stack configuration options
optname	option name to be set
optval	get the buffer address of the option value
optlen	get the buffer length address of the option value
back	–
=0	success
<0	fail

This function is used to get the socket configuration options

1.4.6.3.14 Get Remote Address Information (getpeername)

```
int getpeername(int s, struct sockaddr *name, socklen_t *namelen);
```

Parameter	Description
s	socket descriptor
name	received address structure pointer
namelen	length of received address structure
back	–
=0	success
<0	fail

This function is used to get the remote address information associated with the socket

1.4.6.3.15 Get Local Address Information (getsockname)

```
int getsockname(int s, struct sockaddr *name, socklen_t *namelen);
```

Parameter	Description
s	Socket descriptor
name	received address structure pointer
namelen	length of received address structure
back	–
=0	success
<0	fail

This function is used to get local socket address information

1.4.6.3.16 Configure Socket Parameters (ioctlsocket)

```
int ioctlsocket(int s, long cmd, void *arg);
```

Parameter	Description
s	socket descriptor
cmd	socket operation command
arg	operation command's parameters
back	–
=0	success
<0	fail

This function sets the socket control mode
The CMD supports following commands

- FIONBIO: Turns on or off the socket's non-blocking mode. Arg parameter 1 is open non-blocking and 0 is closed non-blocking.

1.4.6.4 Network Protocol Stack Access

Access to the network protocol stack or network function implementation is mainly to initialize and register the protocol cluster structure, and add it to the protocol cluster list in SAL component. The protocol cluster structure is defined as follows

```
/* network interface socket opreations */
struct sal_socket_ops
{
    int (*socket)     (int domain, int type, int protocol);
```

```
    int (*closesocket)(int s);
    int (*bind)       (int s, const struct sockaddr *name, socklen_t namelen);
    int (*listen)     (int s, int backlog);
    int (*connect)    (int s, const struct sockaddr *name, socklen_t namelen);
    int (*accept)     (int s, struct sockaddr *addr, socklen_t *addrlen);
    int (*sendto)     (int s, const void *data, size_t size, int flags, const struct sockaddr *to, socklen_t
      tolen);
    int (*recvfrom)   (int s, void *mem, size_t len, int flags, struct sockaddr *from, socklen_t *fromlen);
    int (*getsockopt) (int s, int level, int optname, void *optval, socklen_t *optlen);
    int (*setsockopt) (int s, int level, int optname, const void *optval, socklen_t optlen);
    int (*shutdown)   (int s, int how);
    int (*getpeername)(int s, struct sockaddr *name, socklen_t *namelen);
    int (*getsockname)(int s, struct sockaddr *name, socklen_t *namelen);
    int (*ioctlsocket)(int s, long cmd, void *arg);
#ifdef SAL_USING_POSIX
    int (*poll)       (struct dfs_file *file, struct rt_pollreq *req);
#endif
};

/* sal network database name resolving */
struct sal_netdb_ops
{
    struct hostent* (*gethostbyname)  (const char *name);
    int             (*gethostbyname_r)(const char *name, struct hostent *ret, char *buf, size_t buflen,
      struct hostent **result, int *h_errnop);
    int             (*getaddrinfo)    (const char *nodename, const char *servname, const struct addrinfo
      *hints, struct addrinfo **res);
    void            (*freeaddrinfo)   (struct addrinfo *ai);
};

/* Protocol domain structure definition */
struct sal_proto_family
{
    int family;                                 /* primary protocol families type */
    int sec_family;                             /* secondary protocol families type */
    const struct sal_socket_ops *skt_ops;       /* socket opreations */
    const struct sal_netdb_ops *netdb_ops;      /* network database opreations */
};
```

- `family` Each protocol stack supports the main protocol cluster types, such as AF_INET for lwIP, AF_AT Socket, and AF_WIZ for WIZnet
- `sec_family`The type of sub-protocol domain supported by each protocol stack, used to support a single protocol stack or network implementation, that matches other types of protocol cluster types in the package
- `skt_ops` Define socket related functions, such as connect, send, recv, etc., each protocol cluster has a different set of implementation
- `netdb_ops`Define non-socket-related execution functions, such as `gethostbyname`, `getaddrinfo`, `freeaddrinfo`, etc. Each protocol cluster has a different set of implementations

The following is the access registration process implemented by AT Socket network. Developers can refer to other protocol stacks or network implementations for access:

```
#include <rtthread.h>
#include <netdb.h>
#include <sal_low_lvl.h>            /* SAL component structure holds the header file */
#include <at_socket.h>      /* AT Socket related header file */
#include <af_inet.h>

#include <netdev.h>         /*network interface card function related header file */

#ifdef SAL_USING_POSIX
#include <dfs_poll.h>       /* Poll function related header file */
#endif

#ifdef SAL_USING_AT

/* A custom poll execution function that handles the events received in the poll */
static int at_poll(struct dfs_file *file, struct rt_pollreq *req)
{
    int mask = 0;
    struct at_socket *sock;
    struct socket *sal_sock;

    sal_sock = sal_get_socket((int) file->data);
    if(!sal_sock)
    {
        return -1;
    }

    sock = at_get_socket((int)sal_sock->user_data);
    if (sock != NULL)
```

```
    {
        rt_base_t level;

        rt_poll_add(&sock->wait_head, req);

        level = rt_hw_interrupt_disable();
        if (sock->rcvevent)
        {
            mask |= POLLIN;
        }
        if (sock->sendevent)
        {
            mask |= POLLOUT;
        }
        if (sock->errevent)
        {
            mask |= POLLERR;
        }
        rt_hw_interrupt_enable(level);
    }

    return mask;
}
#endif

/* Define and assign socket execution functions, and the SAL component calls the underlying function of the
      registration when it executes relevant functions */
static const struct proto_ops at_inet_stream_ops =
{
    at_socket,
    at_closesocket,
    at_bind,
    NULL,
    at_connect,
    NULL,
    at_sendto,
    at_recvfrom,
    at_getsockopt,
    at_setsockopt,
    at_shutdown,
    NULL,
    NULL,
    NULL,

#ifdef SAL_USING_POSIX
    at_poll,
#else
    NULL,
#endif /* SAL_USING_POSIX */
};

static const struct sal_netdb_ops at_netdb_ops =
{
    at_gethostbyname,
    NULL,
    at_getaddrinfo,
    at_freeaddrinfo,
};

/* define and assign AT Socket protocol domain structure */
static const struct sal_proto_family at_inet_family =
{
    AF_AT,
    AF_INET,
    &at_socket_ops,
    &at_netdb_ops,
};

/* Used to set the protocol domain information in the network interface card device */
int sal_at_netdev_set_pf_info(struct netdev *netdev)
{
    RT_ASSERT(netdev);

    netdev->sal_user_data = (void *) &at_inet_family;
    return 0;
}

#endif /* SAL_USING_AT */
```

1.4.7 AT Commands

1.4.7.1 Introduction to AT Commands

The AT command set was originally a control protocol invented by Dennis Hayes, initially used to control dial-up modems. Later, with the upgrade of network bandwidth, low-speed dial-up modems with very low speed essentially exited the general market, but the AT command set was retained. However, the AT command set lived on when major mobile phone manufacturers jointly developed a set of commands to control the GSM modules of mobile phones. The AT command set evolved on this basis and added the GSM 07.05 standard and the later GSM 07.07 standard to achieve more robust standardization.
After seeing use in GPRS control, 3G modules and similar devices, AT commands gradually become the de facto standard in product development. Nowadays, AT commands are also widely used in embedded development. AT commands are the protocol interfaces of the main chip and communication module. The hardware interface is usually the serial port, so the main control device can complete various operations through simple commands and hardware design.
AT commands are a way of facilitating device connections and data communication between an AT Server and an AT Client. The basic structure is shown below:

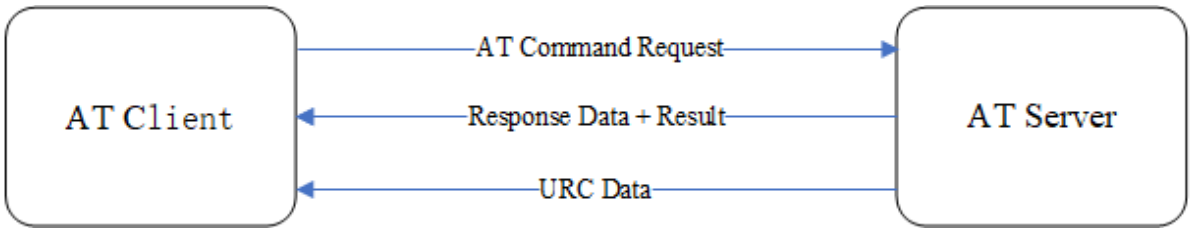


Figure 1.119 AT Command Set

1. The AT command consists of three parts: prefix, body, and terminator. The prefix consists of the character AT; the body consists of commands, parameters, and possibly used data; the terminator typically is <CR><LF> ("\r\n").
2. The implementation of AT functionality requires the AT Server and the AT Client to work together.
3. An AT Server mainly receives commands sent by the AT Client, interprets the received commands and parameter formats, and delivers corresponding response data or actively sends data.
4. An AT Client is mainly used to send commands, wait for the AT Server to respond, and parse the AT Server response data or the actively sent data to obtain related information.
5. A variety of data communication methods (UART, SPI, etc.) are supported between an AT Server and an AT Client. Currently, the most commonly used communications protocol is UART.
6. The data that the AT Server sends to the AT Client is divided into two types: response data and URC data.

- Response Data: The AT Server response status and information received by the AT Client after sending the command.
- URC Data: Data that the AT Server actively sends to the AT Client generally appears only in some special cases, such as when a WiFi connection is disconnected, while it's receiving TCP data, etc. These situations often require special user handling.

With the popularization of AT commands, more and more embedded products use AT commands. The AT commands are used as the protocol interfaces of the main chip and the communication module. The hardware interface is generally a serial port, so that the master device can performs a variety of operations using simple commands and hardware design.
Although the AT command set has standardization to a certain degree, the AT commands supported by different chips are not completely unified, which directly increases the complexity to use. There is no uniform way to handle the sending and receiving of AT commands and the parsing of data.
In order to facilitate the user to use AT commands which can be easily adapted to different AT modules, RT-Thread provides AT components for AT Device connectivity and data communication. The implementation of the AT components consists of both a client and a server.

1.4.7.2 Introduction to AT Components

The AT component is based on the implementation of the `AT Server` and `AT Client` of the RT-Thread system. The component completes the AT command transmission, command format and parameter parsing, command response, response data reception, response data parsing, URC data processing, etc..

Command data interaction process.

Through the AT component, the device can use the serial port to connect to other devices to send and receive parsed data. It can be used as an AT Server to allow other devices or even the computer to connect to complete the response of sending data. It can also start the CLI mode in the local shell to enable the device to support AT Server and AT Client at the same time. The usage of both modes simultaneously is mostly used for device development and debugging.

AT component resource usage:

- AT Client: 4.6K ROM and 2.0K RAM
- AT Server: 4.0K ROM and 2.5K RAM
- AT CLI: 1.5K ROM and almost no RAM is used.

Overall, the AT component resources are extremely small, making them ideal for use in embedded devices with limited resources. The AT component code is primarily located in `rt-thread/components/net/at/`. What follows are the main function of both AT Servers and AT Clients.

Main Functions of AT Server:

- Basic commands: A variety of common basic commands are implemented (ATE, ATZ, etc.);
- Command compatibility: Commands are case-insensitive, which increases command compatibility;
- Command detection: Commands support custom parameter expressions and implement self-detection of received command parameters;
- Command registration: User-defined commands can be easily added, in a way, similar to how the `finsh/msh` command is added;
- Debug mode: An AT Server CLI command line interaction mode is provided, which is mainly used for device debugging.

Main Functions of AT Client:

- URC data processing: The complete URC data processing method;
- Data analysis: Supports the analysis of custom response data, and facilitates the acquisition of relevant information in the response data;
- Debug mode: Provides AT Client CLI command line interaction mode, mainly used for device debugging;
- AT Socket: An extension of AT Client function, it uses the AT command set to send and receive as the basis. This is implemented through the standard BSD Socket API, which completes the data sending and receiving function, and enables users to implement complete device networking and data communication through AT commands.
- Multi-client support: The AT component supports multiple clients running simultaneously.

1.4.7.3 AT Server

1.4.7.3.1 AT Server Configuration

When we use the AT Server feature in the AT component, we need to define the following configuration in rtconfig.h:

Macro Definition	Description
RT_USING_AT	Enable AT component
AT_USING_SERVER	Enable AT Server function

Macro Definition	Description
AT_SERVER_DEVICE	Define the serial communication device name used by AT Server on the device to ensure that it is not used and the device name is unique, such as `uart3` device.
AT_SERVER_RECV_BUFF_LEN	The maximum length of data received by the AT Server device
AT_CMD_END_MARK_CRLF	Determine the line terminator of the received command
AT_USING_CLI	Enable server-command-line interaction mode
AT_DEBUG	Enable AT component DEBUG mode to display more debug log information
AT_PRINT_RAW_CMD	Enable real-time display AT command communication data mode for easy debugging

For different AT devices, there are several formats of the line terminator for the sending commands: `"\r\n""\r""\n"` - the user needs to select the corresponding line terminator in accordance to the device type connected to the AT Server and then determine the end of transmission command, defined as follows:

Macro Definition	Terminator
AT_CMD_END_MARK_CRLF	`"\r\n"`
AT_CMD_END_MARK_CR	`"\r"`
AT_CMD_END_MARK_LF	`"\n"`

The above configuration options can be added by Env tool. The specific path in Env is as follows:

```
RT-Thread Components  --->
    Network  --->
        AT commands  --->
            [*] Enable AT commands
            [*]   Enable debug log output
            [*] Enable AT commands server
            (uart3) Server device name
            (256)   The maximum length of server data accepted
                    The commands new line sign (\r\n)  --->
            [ ]   Enable AT commands client
            [*]   Enable command-line interface for AT commands
            [ ]   Enable print RAW format AT command communication data
```

After the add configuration is complete, you can use the command line to rebuild the project, or use `scons` to compile.

1.4.7.3.2 AT Server Initialization

After enabling the AT Server in Env, you need to initialize it at startup to enable the AT Server function. If the component has been initialized automatically, no additional initialization is required. Otherwise, you need to call the following function in the initialization task:

```
int at_server_init(void);
```

The AT Server initialization function, which belongs to the application layer function, needs to be called before using the AT Server function or using the AT Server CLI function. `at_server_init()` function completes initialization of resources stored by AT commands, such as data segment initialization, AT Server device initialization, and semaphore usage by the AT Server. It also creates an at_server thread for parsing the received data in the AT Server.

After the AT Server is successfully initialized, the device can be used as an AT Server to connect to an AT Client's serial device for data communication, or use a serial-to-USB conversion tool to connect to a PC, so that the PC-side serial debugging assistant can communicate with the AT Client.

1.4.7.3.3 Add custom AT commands

At present, the format of the AT command set used by AT devices of different manufacturers does not have a completely uniform standard, so the AT Server in the AT component only supports some basic general AT commands, such as ATE, AT+RST, etc. These commands can only be used to meet the basic operation of the device. If users want to use more functions, they need to implement custom AT Server commands for different AT devices. The AT

component provides an AT command addition method similar to the FinSH/msh command addition method, which is convenient for users to implement the required commands.
The basic commands currently supported by AT Server are as follows:

- AT: AT test command;
- ATZ: The device is restored to factory settings;
- AT+RST: Reboot device;
- ATE: ATE1 turns on echo, ATE0 turns off echo;
- AT&L: List all commands;
- AT+UART: Set the serial port information.

AT commands can implement different functions depending on the format of the incoming parameters. For each AT command, there are up to four functions, as described below:

- Test Function: `AT+<x>=?`, used to query the command's parameter, format and value range;
- Query Function: `AT+<x>?`, used to return the current value of the command parameter;
- Setting Function: `AT+<x>=...`, used for user-defined parameter values;
- Execution Function: `AT+<x>`, used to perform related operations.

The four functions of each command do not need to be fully implemented. When you add an AT Server command, you can implement one or several of the above functions according to your needs. Unimplemented functions can be represented by `NULL`. Then the custom function can be added to the list of basic commands. The addition method is similar to the way the `finsh/msh` commands are added. The function for adding commands is as follows:

```
AT_CMD_EXPORT(_name_, _args_expr_, _test_, _query_, _setup_, _exec_);
```

Parameter	Description
name	AT command name
_args_expr_	AT command parameter expression; (NULL means no parameter, <> means mandatory parameter and [] means optional parameter
test	AT test function name; (NULL means no parameter)
query	AT query function name; (ibid.)
setup	AT setup function name; (ibid.)
exec	AT performs the function name; (ibid.)

The AT command registration example is as follows. The `AT+TEST` command has two parameters. The first parameter is a mandatory parameter, and the second parameter is an optional parameter. The command implements the query function and the execution function:

```
static at_result_t at_test_exec(void)
{
    at_server_printfln("AT test commands execute!");

    return 0;
}
static at_result_t at_test_query(void)
{
    at_server_printfln("AT+TEST=1,2");

    return 0;
}

AT_CMD_EXPORT("AT+TEST", =<value1>[,<value2>], NULL, at_test_query, NULL, at_test_exec);
```

1.4.7.3.4 AT Server APIs

1.4.7.3.4.1 Send Data to the Client (no newline)

```
void at_server_printf(const char *format, ...);
```

This function is used by the AT Server to send fixed-format data to the corresponding AT Client serial device through the serial device. The data ends without a line break. Used to customize the function functions of AT commands in AT Server.

Parameter	**D**escription
format	Customize the expression of the input data
...	Input data list, variable parameters

1.4.7.3.4.2 Send Data to the Client (newline)

```
void at_server_printfln(const char *format, ...);
```

This function is used by the AT Server to send fixed-format data to the corresponding AT Client serial device through the serial device, with a newline at the end of the data. Used to customize the function functions of AT commands in AT Server.

Parameter	Description
format	Customize the expression of the input data
...	Input data list, variable parameters

1.4.7.3.4.3 Send Command Execution Results to the Client

```
void at_server_print_result(at_result_t result);
```

This function is used by the AT Server to send command execution results to the corresponding AT Client serial device through the serial device. The AT component provides a variety of fixed command execution result types. When you customize a command, you can use the function to return the result directly;

Parameter	Description
result	Command execution result type

The command execution result type in the AT component is given in the enumerated type, as shown in the following table:

Types of Command Execution Result	Description
AT_RESULT_OK	Command Execution Succeeded
AT_RESULT_FAILE	Command Execution Failed
AT_RESULT_NULL	Command No Result
AT_RESULT_CMD_ERR	Command Input Error
AT_RESULT_CHECK_FAILE	Parameter Expression Matching Error
AT_RESULT_PARSE_FAILE	Parameter Parsing Error

See the following code to learn how to use the `at_server_print_result` function:

```
static at_result_t at_test_setup(const char *args)
{
    if(!args)
    {
        /* If the parameter error after incoming orders, returns expression match error results */
        at_server_print_result(AT_RESULT_CHECK_FAILE);
    }

    /* Return to successful execution under normal conditions */
```

```
    at_server_print_result(AT_RESULT_OK);
    return 0;
}
static at_result_t at_test_exec(void)
{
    // execute some functions of the AT command.

    /* This command does not need to return results */
    at_server_print_result(AT_RESULT_NULL);
    return 0;
}
AT_CMD_EXPORT("AT+TEST", =<value1>,<value2>, NULL, NULL, at_test_setup, at_test_exec);
```

1.4.7.3.4.4 Parsing Input Command Parameters

```
int at_req_parse_args(const char *req_args, const char *req_expr, ...);
```

Among the four functions of an AT command, only the setting function has an input parameter, and the input parameter is to remove the rest of the AT command. For example, for a given command input `"AT+TEST=1,2,3,4"`, set the input parameter of the function to the parameter string `"=1,2,3,4"`.
The command parsing function is mainly used in the AT function setting function, which is used to parse the incoming string parameter and obtain corresponding multiple input parameters for performing the following operations. The standard `sscanf` parsing grammar used in parsing grammar here will also be described in detail later in the AT Client parameter parsing function.

Parameter	Description
req_args	The incoming parameter string of the request command
req_expr	Custom parameter parsing expression for parsing the above incoming parameter data
...	Output parsing parameter list, which is a variable parameter
Return	–
>0	Successful, returns the number of variable parameters matching the parameter expression
=0	Failed, no parameters matching the parameter expression
-1	Failed, parameter parsing error

See the following code to learn how to use the at_server_print_result function:

```
static at_result_t at_test_setup(const char *args)
{
    int value1,value2;

    /* The input standard format of args should be "=1, 2", "=%d, %d" is a custom parameter parsing
      expression, and the result is parsed and stored in the value1 and value2 variables. */
    if (at_req_parse_args(args, "=%d,%d", &value1, &value2) > 0)
    {
        /* Data analysis succeeds, echoing data to AT Server serial device */
        at_server_printfln("value1 : %d, value2 : %d", value1, value2);

        /* The data is parsed successfully. The number of parsing parameters is greater than zero. The
      execution is successful. */
        at_server_print_result(AT_RESULT_OK);
    }
    else
    {
        /* Data parsing failed, the number of parsing parameters is not greater than zero, and the parsing
      failure result type is returned. */
        at_server_print_result(AT_RESULT_PARSE_FAILE);
    }
    return 0;
}
/* Add the "AT+TEST" command to the AT command list. The command parameters are formatted as two mandatory
      parameters <value1> and <value2>. */
AT_CMD_EXPORT("AT+TEST", =<value1>,<value2>, NULL, NULL, at_test_setup, NULL);
```

1.4.7.3.4.5 Porting-related interfaces

The AT Server supports a variety of basic commands (ATE, ATZ, etc.) by default. The function implementation of some commands is related to hardware or platform and requires user-defined implementation. The AT component source code `src/at_server.c` file gives the weak function definition of the migration file. The user can create a new migration file in the project to implement the following function to complete the migration interface, or modify

the weak function to complete the migration interface directly in the file.

1. Device restart function: `void at_port_reset(void);`. This function completes the device soft restart function and is used to implement the basic command AT+RST in AT Server.
2. The device restores the factory settings function: `void at_port_factory_reset(void);`. This function completes the device factory reset function and is used to implement the basic command ATZ in AT Server.
3. Add a command table in the link script (add only in gcc, no need to add in Keil and IAR).

If you use the gcc toolchain in your project, you need to add the *section* corresponding to the AT server command table in the link script. Refer to the following link script:

```
/* Constant data goes into FLASH */
.rodata :
{
    ...

    /* section information for RT-thread AT package */
    . = ALIGN(4);
    __rtatcmdtab_start = .;
    KEEP(*(RtAtCmdTab))
    __rtatcmdtab_end = .;
    . = ALIGN(4);
} > CODE
```

1.4.7.4 AT Client

1.4.7.4.1 AT Client Configuration

When using the AT Client feature in the AT component, the following configuration in rtconfig.h needs to be defined:

```
#define RT_USING_AT
#define AT_USING_CLIENT
#define AT_CLIENT_NUM_MAX 1
#define AT_USING_SOCKET
#define AT_USING_CLI
#define AT_PRINT_RAW_CMD
```

- `RT_USING_AT`: Used to enable or disable the AT component;
- `AT_USING_CLIENT`: Used to enable the AT Client function;
- `AT_CLIENT_NUM_MAX`: Maximum number of AT clients supported at the same time.
- `AT_USING_SOCKET`: Used by the AT client to support the standard BSD Socket API and enable the AT Socket function.
- `AT_USING_CLI`: Used to enable or disable the client command line interaction mode.
- `AT_PRINT_RAW_CMD`: Used to enable the real-time display mode of AT command communication data for easy debugging.

The above configuration options can be added directly to the `rtconfig.h` file or added by the Env. The specific path in Env is as follows:

```
RT-Thread Components  --->
    Network  --->
        AT commands  --->
            [*] Enable AT commands
            [ ]   Enable debug log output
            [ ]   Enable AT commands server
            [*]   Enable AT commands client
            (1)   The maximum number of supported clients
            [*]   Enable BSD Socket API support by AT commnads
            [*]   Enable command-line interface for AT commands
            [ ]   Enable print RAW format AT command communication data
```

After the configuration is complete, you can use the command line to rebuild the project, or use `scons` to compile.

1.4.7.4.2 AT Client Initialization

After configuring the AT Client, you need to initialize it at startup to enable the AT Client function. If the component has been initialized automatically, no additional initialization is required. Otherwise, you need to call the following function in the initialization task:

```
int at_client_init(const char *dev_name,  rt_size_t recv_bufsz);
```

The AT Client initialization function, which belongs to the application layer function, needs to be called before using the AT Client function or using the AT Client CLI function. The `at_client_init()` function completes the initialization of the AT Client device, the initialization of the AT Client porting function, the semaphore and mutex used by the AT Client, and other resources, and creates the `at_client` thread for parsing the data received in the AT Client and for processing the URC data.

1.4.7.4.3 AT Client data receiving and sending

The main function of the AT Client is to send AT commands, receive data, and parse data. The following is an introduction to the processes and APIs related to AT Client data reception and transmission.
Related structure definition:

```
struct at_response
{
    /* response buffer */
    char *buf;
    /* the maximum response buffer size */
    rt_size_t buf_size;
    /* the number of setting response lines
     * == 0: the response data will auto return when received 'OK' or 'ERROR'
     * != 0: the response data will return when received setting lines number data */
    rt_size_t line_num;
    /* the count of received response lines */
    rt_size_t line_counts;
    /* the maximum response time */
    rt_int32_t timeout;
};
typedef struct at_response *at_response_t;
```

In the AT component, this structure is used to define a control block for AT command response data, which is used to store or limit the data format of the AT command response data.

- `buf` is used to store the received response data. Note that the data stored in the buf is not the original response data, but the data of the original response data removal terminator (`"\r\n"`). Each row of data in the buf is split by '\0' to make it easy to get data by row.
- `buf_size` is a user-defined length of the received data that is most supported by this response. The length of the return value is defined by the user according to his own command.
- `line_num` is the number of rows that the user-defined response data needs to receive. **If there is no response line limit requirement, it can be set to 0.**
- `line_counts` is used to record the total number of rows of this response data.
- `timeout` is the user-defined maximum response time for this response data.

`buf_size`, `line_num`, `timeout` parameters in the structure are restricted conditions, which are set when the structure is created, and other parameters are used to store data parameters for later data analysis.
Introduction to related API interfaces:

1.4.7.4.3.1 Create a Response Structure

```
at_response_t at_create_resp(rt_size_t buf_size, rt_size_t line_num, rt_int32_t timeout);
```

Parameter	Description
buf_size	Maximum length of received data supported by this response
line_num	This response requires the number of rows of data to be returned. The number of rows is divided by a standard terminator (such as "\r\n"). If it is 0, it will end the response reception after receiving the "OK" or "ERROR" data; if it is greater than 0, it will return successfully after receiving the data of the current set line number.

Parameter	Description
timeout	The maximum response time of the response data, receiving data timeout will return an error
Return	–
!= NULL	Successful, return a pointer to the response structure
= NULL	Failed, insufficient memory

This function is used to create a custom response data receiving structure for later receiving and parsing the send command response data.

1.4.7.4.3.2 Delete a Response Structure

```
void at_delete_resp(at_response_t resp);
```

Parameter	Description
resp	The response structure pointer to be deleted

This function is used to delete the created response structure object, which is generally paired with the **at_create_resp** creation function.

1.4.7.4.3.3 Set the Parameters of Response Structure

```
at_response_t at_resp_set_info(at_response_t resp, rt_size_t buf_size, rt_size_t line_num, rt_int32_t
      timeout);
```

Parameter	Description
resp	Response structure pointer that has been created
buf_size	Maximum length of received data supported by this response
line_num	This response requires the number of rows of data to be returned. The number of lines is divided by the standard terminator. If it is 0, the response is received after receiving the "OK" or "ERROR" data. If it is greater than 0, the data is successfully returned after receiving the data of the currently set line number.
timeout	The maximum response time of the response data, receiving data timeout will return an error.
Return	–
!= NULL	Successful, return a pointer to the response structure
= NULL	Failed, insufficient memory

This function is used to set the response structure information that has been created. It mainly sets the restriction information on the response data. It is generally used after creating the structure and before sending the AT command. This function is mainly used to send commands when the device is initialized, which can reduce the number of times the response structure is created and reduce the code resource occupation.

1.4.7.4.3.4 Send a Command and Receive a Response

```
rt_err_t at_exec_cmd(at_response_t resp, const char *cmd_expr, ...);
```

Parameter	Description
resp	Response structure body pointer created
cmd_expr	Customize the expression of the input command
...	Enter the command data list, a variable parameter

Parameter	Description
Return	–
>=0	Successful
-1	Failed
-2	Failed, receive response timed out

This function is used by the AT Client to send commands to the AT Server and wait for a response. `resp` is a pointer to the response structure that has been created. The AT command uses the variable argument input of the match expression. **You do not need to add a command terminator at the end of the input command.**
Refer to the following code to learn how to use the above AT commands to send and receive related functions:

```
/*
 * Program listing: AT Client sends commands and receives response routines
 */

#include <rtthread.h>
#include <at.h>   /* AT component header file */

int at_client_send(int argc, char**argv)
{
    at_response_t resp = RT_NULL;

    if (argc != 2)
    {
        LOG_E("at_cli_send [command]  - AT client send commands to AT server.");
        return -RT_ERROR;
    }

    /* Create a response structure, set the maximum support response data length to 512 bytes, the number of
      response data lines is unlimited, and the timeout period is 5 seconds. */
    resp = at_create_resp(512, 0, rt_tick_from_millisecond(5000));
    if (!resp)
    {
        LOG_E("No memory for response structure!");
        return -RT_ENOMEM;
    }

    /* Send AT commands and receive AT Server response data, data and information stored in the resp
      structure */
    if (at_exec_cmd(resp, argv[1]) != RT_EOK)
    {
        LOG_E("AT client send commands failed, response error or timeout !");
        return -ET_ERROR;
    }

    /* Command sent successful */
    LOG_D("AT Client send commands to AT Server success!");

    /* Delete response structure */
    at_delete_resp(resp);

    return RT_EOK;
}
#ifdef FINSH_USING_MSH
#include <finsh.h>
/* Output at_Client_send to msh */
MSH_CMD_EXPORT(at_Client_send, AT Client send commands to AT Server and get response data);
#endif
```

The implementation principle of sending and receiving data is relatively simple. It mainly reads and writes the serial port device bound by the AT client, and sets the relevant number of rows and timeout to limit the response data. It is worth noting that the `res` response needs to be created first. The structure passed `in_exec_cmd` function is for data reception. When the `at_exec_cmd` function's parameter `resp` is NULL, it means that the data sent this time **does not consider processing the response data and directly returns the result**.

1.4.7.4.4 AT Client Data Parsing Method

After the data is normally acquired, the response data needs to be parsed, which is one of the important functions of the AT Client. Parsing of data in the AT Client provides a parsed form of a custom parsing expression whose parsing syntax uses the standard `sscanf` parsing syntax. Developers can use the custom data parsing expression to respond to useful information in the data, provided that the developer needs to review the relevant manual in advance to understand the basic format of the AT Server device response data that the AT Client connects to. The following is a simple AT Client data parsing method through several functions and routines.

1.4.7.4.4.1 Get Response Data for the Specified Line Number

```
const char *at_resp_get_line(at_response_t resp, rt_size_t resp_line);
```

Parameter	Description
resp	Response structure pointer
resp_line	Required line number for obtaining data
Return	–
!= NULL	Successful, return a pointer to the corresponding line number data
= NULL	Failed, input line number error

This function is used to get a row of data with the specified line number in the AT Server response data. The line number is judged by the standard data terminator. The above send and receive functions `at_exec_cmd` have recorded and processed the data and line numbers of the response data in the `resp` response structure, where the data information of the corresponding line number can be directly obtained.

1.4.7.4.4.2 Get Response Data by the Specified Keyword

```
const char *at_resp_get_line_by_kw(at_response_t resp, const char *keyword);
```

Parameter	Description
resp	Response structure pointer
keyword	Keyword information
Return	–
!= NULL	Successful, return a pointer to the corresponding line number data
= NULL	Failed, no keyword information found

This function is used to get a corresponding row of data by keyword in the AT Server response data.

1.4.7.4.4.3 Parse Response Data for the Specified Line Number

```
int at_resp_parse_line_args(at_response_t resp, rt_size_t resp_line, const char *resp_expr, ...);
```

Parameter	Description
resp	Response structure pointer
resp_line	Parsed data line number required, **from the start line number count 1**
resp_expr	Custom parameter parsing expression
...	Parsing the parameter list as a variable parameter
Return	–
>0	Successful, return the number of parameters successfully parsed
=0	Failed, no parameters matching the parsing expression
-1	Failed, parameter parsing error

This function is used to get a row of data with the specified line number in the AT Server response data, and parse the parameters in the row data.

1.4.7.4.4.4 Parse Response Data for a Row with Specified Keyword

```
int at_resp_parse_line_args_by_kw(at_response_t resp, const char *keyword, const char *resp_expr, ...);
```

Parameter	Description
resp	Response structure pointer
keyword	Keyword information
resp_expr	Custom parameter parsing expression
...	Parsing the parameter list as a variable parameter
Return	–
>0	Successful, return the number of parameters successfully parsed
=0	Failed, no parameters matching the parsing expression
-1	Failed, parameter parsing error

This function is used to get a row of data containing a keyword in the AT Server response data and parse the parameters in the row data.
The data parsing syntax uses the standard `sscanf` syntax, the content of the syntax is more, developers can search their parsing syntax, here two procedures are used to introduce the simple use method.

1.4.7.4.4.5 Serial Port Configuration Information Analysis Example

The data sent by the client:

```
AT+UART?
```

The response data obtained by the client:

```
UART=115200,8,1,0,0\r\n
OK\r\n
```

The pseudo code is parsed as follows:

```
/* Create a server response structure, the maximum length of user-defined receive data is 64 */
resp = at_create_resp(64, 0, rt_tick_from_millisecond(5000));

/* Send data to the server and receive response data in the resp structure */
at_exec_cmd(resp, "AT+UART?");

/* Analyze the serial port configuration information, 1 means parsing the first line of response data,
      '%*[^=]' means ignoring the data before the equal sign */
at_resp_parse_line_args(resp, 1,"%*[^=]=%d,%d,%d,%d,%d", &baudrate, &databits, &stopbits, &parity,
      &control);
printf("baudrate=%d, databits=%d, stopbits=%d, parity=%d, control=%d\n",
        baudrate, databits, stopbits, parity, control);

/* Delete server response structure */
at_delete_resp(resp);
```

1.4.7.4.4.6 IP and MAC Address Resolution Example

The data sent by the client:

```
AT+IPMAC?
```

The response data obtained by the server:

```
IP=192.168.1.10\r\n
MAC=12:34:56:78:9a:bc\r\n
OK\r\n
```

The pseudo code is parsed as follows:

```
/* Create a server response structure, the maximum length of user-defined receive data is 128 */
resp = at_create_resp(128, 0, rt_tick_from_millisecond(5000));

at_exec_cmd(resp, "AT+IPMAC?");

/* Customize the parsing expression to parse the information in the current line number data */
at_resp_parse_line_args(resp, 1,"IP=%s", ip);
at_resp_parse_line_args(resp, 2,"MAC=%s", mac);
printf("IP=%s, MAC=%s\n", ip, mac);

at_delete_resp(resp);
```

The key to parsing data is to correctly define the expression. Because the response data of the different device manufacturers is not unique to the response data of the AT device, only the form of the custom parsing expression can be obtained to obtain the required information. The design of the `at_resp_parse_line_args` parsing parameter function is based on the `sscanf` data parsing method. Before using it, the developer needs to understand the basic parsing syntax and then design the appropriate parsing syntax in combination with the response

data. If the developer does not need to parse the specific parameters, you can use the `at_resp_get_line` function to get the specific data of a row.

1.4.7.4.5 AT Client URC Data Processing

The processing of URC data is another important feature of AT Client. URC data is the data that is actively sent by the server. It cannot be received by the above data sending and receiving functions. The URC data format and function are different for different devices. Therefore, the URC data processing mode needs to be customized. The AT component provides a list management method for the processing of URC data. Users can customize the addition of URC data and its execution functions to the management list, so the processing of URC data is also the main porting work of AT Client.
Related structure:

```
struct at_urc
{
    const char *cmd_prefix;             // URC data prefix
    const char *cmd_suffix;             // URC data suffix
    void (*func)(const char *data, rt_size_t size);     // URC data execution function
};
typedef struct at_urc *at_urc_t;
```

Each URC data has a structure control block that defines and determines the prefix and suffix of the URC data, as well as the execution function of the URC data. A piece of data can be defined as URC data only if it matches the prefix and suffix of the URC exactly. The URC data execution function is executed immediately after the matching URC data is obtained. So developers adding a URC data requires a custom matching prefix, suffix, and execution function.

1.4.7.4.5.1 URC Data List Initialization

```
void at_set_urc_table(const struct at_urc *table, rt_size_t size);
```

Parameter	Description
table	URC data structure array pointer
size	Number of URC data

This function is used to initialize the developer-defined URC data list, mainly used in the AT Client porting function. The example of AT Client migration is given below. This example mainly shows the specific processing of URC data in the `at_client_port_init()` porting function. The developer can directly apply it to his own porting file, or customize the implementation function to complete the AT Client porting.

```
static void urc_conn_func(const char *data, rt_size_t size)
{
    /* WIFI connection success information */
    LOG_D("AT Server device WIFI connect success!");
}

static void urc_recv_func(const char *data, rt_size_t size)
{
    /* Received data from the server */
    LOG_D("AT Client receive AT Server data!");
}

static void urc_func(const char *data, rt_size_t size)
{
    /* Device startup information */
    LOG_D("AT Server device startup!");
}

static struct at_urc urc_table[] = {
    {"WIFI CONNECTED",    "\r\n",     urc_conn_func},
    {"+RECV",             ":",          urc_recv_func},
    {"RDY",               "\r\n",     urc_func},
};

int at_client_port_init(void)
{
    /* Add multiple URC data to the URC list and execute the URC function when receiving data that matches
      both the URC prefix and the suffix */
    at_set_urc_table(urc_table, sizeof(urc_table) / sizeof(urc_table[0]));
    return RT_EOK;
}
```

1.4.7.4.6 AT Client Other APIs Introduction

1.4.7.4.6.1 Send Specified Length Data

```
rt_size_t at_client_send(const char *buf, rt_size_t size);
```

Parameter	Description
buf	Pointer to send data
size	Length of data sent
Return	–
>0	Successful, return the length of the data sent
<=0	Failed

This function is used to send the specified length data to the AT Server device through the AT Client device, which is mostly used for the AT Socket function.

1.4.7.4.6.2 Receive Specified Length Data

```
rt_size_t at_client_recv(char *buf, rt_size_t size,rt_int32_t timeout);
```

Parameter	Description
buf	Pointer to receive data
size	Maximum length for receiving data
timeout	Receive data timeout (tick)
Return	–
>0	Successful, return the length of the data received successfully
<=0	Failed, receiving data error or timeout

This function is used to receive data of a specified length through the AT Client device, and is mostly used for the AT Socket function. This function can only be used in URC callback handlers.

1.4.7.4.6.3 Set the line terminator for receiving data

```
void at_set_end_sign(char ch);
```

Parameter	
ch	Line terminator
Return	
-	-

This function is used to set the line terminator, which is used to judge the end of a row of data received by the client, and is mostly used for the AT Socket function.

1.4.7.4.6.4 Waiting for module initialization to complete

```
int at_client_wait_connect(rt_uint32_t timeout);
```

Parameter	Description
timeout	Waiting timeout

Parameter	Description
Return	**Description**
0	Successful
<0	Failed, no data returned during the timeout period

This function is used to cyclically send AT commands when the AT module starts, until the module responds to the data, indicating that the module is successfully started.

1.4.7.4.7 AT Client Multi-Client Support

In general, the device as the AT Client only connects to one AT module (the AT module acts as the AT Server) and can directly use the above functions of data transmission and reception and command parsing. In a few cases, the device needs to connect multiple AT modules as the AT Client. In this case, the multi-client support function of the device is required.
The AT component provides support for multi-client connections and provides two different sets of function interfaces: **Single-Client Mode Functions** and **Multi-Client Mode Functions**.

- Single-Client Mode Function: This type of function interface is mainly used when the device is connected to only one AT module, or when the device is connected to multiple AT modules, it is used in the **first AT client**.
- Multi-Client Mode Function: This type of function interface mainly uses devices to connect multiple AT modules.

The advantages and disadvantages of the two different mode functions and in different application scenarios are as follows:

Modes / Application Scenarios	Single-Client Mode Functions	Multi-Client Mode Functions
Single Module	Can be used directly	**Pros**: Support single module scenarios to improve compatibility **Cons**: Need to get the client object first, the use process is more complicated than that of the single client mode
Multiple Modules	**Pros**: Simplified function operation **Cons**: In the case of multiple modules, this function mode can only be used for **the first initialized** module.	Need to get the client object first, then use it directly

Figure 1.120 at client modes comparison

The single client mode function definition is mainly different from the single connection mode function. The definition of the incoming client object is different. The single client mode function uses the first initialized AT client object by default, and the multi-client mode function can pass in the user-defined custom client object. The function to get the client object is as follows:

```
at_client_t at_client_get(const char *dev_name);
```

This function obtains the AT client object created by the device through the incoming device name, which is used to distinguish different clients when connecting multiple clients.
The single client mode and multi-client mode function interface definitions differ from the following functions:

Single-Client Mode Functions	Multi-Client Mode Functions
at_exec_cmd(...)	at_obj_exec_cmd(client, ...)
at_set_end_sign(...)	at_obj_set_end_sign(client, ...)
at_set_urc_table(...)	at_obj_set_urc_table(client, ...)
at_client_wait_connect(...)	at_client_obj_wait_connect(client, ...)
at_client_send(...)	at_client_obj_send(client, ...)
at_client_recv(...)	at_client_obj_recv(client, ...)

The two modes of client data transmission and parsing are basically the same, and the function usage process is different, as shown below:

```
/* Single client mode function usage */

at_response_t resp = RT_NULL;

at_client_init("uart2", 512);

resp = at_create_resp(256, 0, 5000);

/* Send commands using a single client mode function */
at_exec_cmd(resp, "AT+CIFSR");

at_delete_resp(resp);
/* Multi-client mode functions usage */

at_response_t resp = RT_NULL;
at_client_t client = RT_NULL;

/* Initialize two AT clients */
at_client_init("uart2", 512);
at_client_init("uart3", 512);

/* Get the corresponding AT client object by name */
client = at_client_get("uart3");

resp = at_create_resp(256, 0, 5000);

/* Send commands using multi-client mode functions */
at_obj_exec_cmd(client, resp, "AT+CIFSR");

at_delete_resp(resp);
```

The process differences used by other functions are similar to the above `at_obj_exec_cmd()` function. The main function is to obtain the client object through the `at_client_get()` function, and then determine which client is the client through the incoming object to achieve multi-client support.

1.4.7.5 FAQs

1.4.7.5.1 Q: What should I do if the log on the shell shows an error when enabling the AT command to send and receive data real-time printing function. ?

A: Increase the baudrate of the serial port device corresponding to the shell to 921600, improve the serial port printing speed, and prevent the printing error when the data is too large.

1.4.7.5.2 Q: When the AT Socket function is started, the compile prompt "The AT socket device is not selected, please select it through the env menuconfig".

A: After the AT Socket function is enabled, the corresponding device model is enabled in the at device package by default. Enter the at device package, configure the device as an ESP8266 device, configure WIFI information, re-generate the project, compile and download.

1.4.7.5.3 Q: AT Socket function data reception timeout or data reception is not complete.

A: The error may be that the receive data buffer in the serial device used by the AT is too small (RT_SERIAL_RB↩ _BUFSZ default is 64 bytes), and the data is overwritten after the data is not received in time. The buffer size of the serial port receiving data (such as 256 bytes) is appropriately increased.

1.4.8 POSIX Interface

1.4.8.1 Introduction to Pthreads

POSIX Threads is abbreviated as Pthreads. POSIX is the abbreviation of "Portable Operating System Interface". POSIX is a set of standards established by IEEE Computer Society to improve the compatibility of different operating systems and the portability of applications. Pthreads is a threaded POSIX standard defined in the POSIX.1c, Threads extensions (IEEE Std1003.1c-1995) standard, which defines a set of C programming language types, functions, and constants. Defined in the `pthread.h` header file and a thread library, there are about 100 APIs, all of which have a "`pthread_`" prefix and can be divided into 4 categories:

- **Thread management**: Thread creating, detaching, joining, and setting and querying thread attributes.
- **Mutex**: Abbreviation for "mutual exclusion", which restricts thread access to shared data and protects the integrity of shared data. This includes creating, destroying, locking, and unlocking mutex and some functions for setting or modifying mutex properties.
- **Condition variable**: Communication between threads used to share a mutex. It includes functions such as creation, destruction, waiting condition variables, and sending signal .
- **Read/write locks and barriers**: including the creation, destruction, wait, and related property settings of read-write locks and barriers.

POSIX semaphores are used with Pthreads, but are not part of the Pthreads standard definition and are defined in POSIX.1b, Real-time extensions (IEEE Std1003.1b-1993). Therefore the prefix of the semaphore correlation function is "`sem_`" instead of "`pthread_`".
Message queues, like semaphores, are used with Pthreads and are not part of the Pthreads standard definition and are defined in the IEEE Std 1003.1-2001 standard. The prefix of the message queue related function is "`mq_`".

Function Prefix	Function Group
`pthread_`	Thread itself and various related functions
`pthread_attr_`	Thread attribute object
`Pthread_mutex_`	Mutex
`pthread_mutexattr_`	Mutex attribute object
`pthread_cond_`	Conditional variable
`pthread_condattr_`	Condition variable attribute object
`pthread_rwlock_`	Read-write lock
`pthread_rwlockattr_`	Read-write lock attribute object
`pthread_spin_`	Spin lock
`pthread_barrier_`	Barrier
`pthread_barrierattr_`	Barrier attribute object
`sem_`	Semaphore
`mq_`	Message queue

Most Pthreads functions return a value of 0 if they succeed, and an error code contained in the `errno.h` header file if unsuccessful. Many operating systems support Pthreads, such as Linux, MacOSX, Android, and Solaris, so applications written using Pthreads functions are very portable and can be compiled and run directly on many platforms that support Pthreads.

1.4.8.1.1 Use POSIX in RT-Thread

Using the POSIX API interface in RT-Thread includes several parts: libc (for example, newlib), filesystem, pthread, and so on. Need to open the relevant options in rtconfig.h:

```
#define RT_USING_LIBC
#define RT_USING_DFS
#define RT_USING_DFS_DEVFS
```

```
#define RT_USING_PTHREADS
```

RT-Thread implements most of the functions and constants of Pthreads, defined in the pthread.h, mqueue.h, semaphore.h, and sched.h header files according to the POSIX standard. Pthreads is a sublibrary of libc, and Pthreads in RT-Thread are based on the encapsulation of RT-Thread kernel functions, making them POSIX compliant. The Pthreads functions and related functions implemented in RT-Thread are described in detail in the following sections.

1.4.8.2 Thread

1.4.8.2.1 Thread Handle

```
typedef rt_thread_t pthread_t;
```

`Pthread_t` is a redefinition of the `rt_thread_t` type, defined in the `pthread.h` header file. rt_thread_t is the thread handle (or thread identifier) of the RT-Thread and is a pointer to the thread control block. You need to define a variable of type pthread_t before creating a thread. Each thread corresponds to its own thread control block, which is a data structure used by the operating system to control threads. It stores some information about the thread, such as priority, thread name, and thread stack address. Thread control blocks and thread specific information are described in detail in the Thread Management chapter.

1.4.8.2.2 Create Thread

```
int pthread_create (pthread_t *tid,
                    const pthread_attr_t *attr,
                    void *(*start) (void *), void *arg);
```

Parameter	Description
tid	Pointer to thread handle (thread identifier), cannot be NULL
attr	Pointer to the thread property, if NULL is used, the default thread property is used
start	Thread entry function address
arg	The argument passed to the thread entry function
return	——
0	succeeded
EINVAL	Invalid parameter
ENOMEM	Dynamic allocation of memory failed

This function creates a pthread thread. This function dynamically allocates the POSIX thread data block and the RT-Thread thread control block, and saves the start address (thread ID) of the thread control block in the memory pointed to by the parameter tid, which can be used to operate in other threads. This thread; and the thread attribute pointed to by attr, the thread entry function pointed to by start, and the entry function parameter arg are stored in the thread data block and the thread control block. If the thread is created successfully, the thread immediately enters the ready state and participates in the scheduling of the system. If the thread creation fails, the resources occupied by the thread are released.
Thread properties and related functions are described in detail in the *Thread Advanced Programming* chapter. In general, the default properties can be used.

> After the pthread thread is created, if the thread needs to be created repeatedly, you need to set the pthread thread to detach mode, or use pthread_join to wait for the created pthread thread to finish.

1.4.8.2.2.1 Example Code for Creating Thread

The following program initializes two threads, which have a common entry function, but their entry parameters are not the same. Others, they have the same priority and are scheduled for rotation in time slices.

```
#include <pthread.h>
#include <unistd.h>
#include <stdio.h>

/* Thread control block */
static pthread_t tid1;
```

```
static pthread_t tid2;

/* Function return value check */
static void check_result(char* str,int result)
{
    if (0 == result)
    {
        printf("%s successfully!\n",str);
    }
    else
    {
        printf("%s failed! error code is %d\n",str,result);
    }
}

/* Thread entry function */
static void* thread_entry(void* parameter)
{
    int count = 0;
    int no = (int) parameter; /* Obtain the thread's entry parameters */

    while (1)
    {
        /* Printout thread count value */
        printf("thread%d count: %d\n", no, count ++);

        sleep(2);    /* Sleep for 2 seconds */
    }
}

/* User application portal */
int rt_application_init()
{
    int result;

    /* Create thread 1, the property is the default value, the entry function is thread_entry, and the entry
      function parameter is 1 */
    result = pthread_create(&tid1,NULL,thread_entry,(void*)1);
    check_result("thread1 created", result);

    /* Create thread 2, the property is the default value, the entry function is thread_entry, and the entry
      function parameter is 2 */
    result = pthread_create(&tid2,NULL,thread_entry,(void*)2);
    check_result("thread2 created", result);

    return 0;
}
```

1.4.8.2.3 Detach Thread

```
int pthread_detach (pthread_t thread);
```

Parameter	Description
thread	Thread handle (thread identifier)
return	——
0	succeeded

Call this function, If the pthread does not finish running, set the detach state of the thread thread property to detached; when the thread thread has finished, the system will reclaim the resources occupied by the pthread thread.
Usage: The child thread calls `pthread_detach(pthread_self())` (*pthread_self()* returns the thread handle of the currently calling thread), or another thread calls `pthread_detach(thread_id)`. The separation state of the thread attributes will be described in detail later.

> Once the detach state of the thread property is set to detached, the thread cannot be waited by the pthread_join() function or re-set to detached.

1.4.8.2.3.1 Example Code for Detaching Thread

The following program initializes 2 threads, which have the same priority and are scheduled according to the time slice. Both threads will be set to the detached state. The 2 threads will automatically exit after printing 3 times of information. After exiting, the system will automatically reclaim its resources.

```
#include <pthread.h>
#include <unistd.h>
#include <stdio.h>

/* Thread control block */
static pthread_t tid1;
static pthread_t tid2;

/* Function return value check */
static void check_result(char* str,int result)
{
    if (0 == result)
    {
        printf("%s successfully!\n",str);
    }
    else
    {
        printf("%s failed! error code is %d\n",str,result);
    }
}

/* Thread 1 entry function */
static void* thread1_entry(void* parameter)
{
    int i;

    printf("i'm thread1 and i will detach myself!\n");
    pthread_detach(pthread_self());        /* Thread 1 detach itself */

    for (i = 0;i < 3;i++)    /* Cycle print 3 times */
    {
        printf("thread1 run count: %d\n",i);
        sleep(2);    /* Sleep 2 seconds */
    }

    printf("thread1 exited!\n");
    return NULL;
}

/* Thread 2 entry function */
static void* thread2_entry(void* parameter)
{
    int i;

    for (i = 0;i < 3;i++)    /* Cycle print 3 times */
    {
        printf("thread2 run count: %d\n",i);
        sleep(2);    /* Sleep 2 seconds */
    }

    printf("thread2 exited!\n");
    return NULL;
}
/* User application portal */
int rt_application_init()
{
    int result;

    /* Create thread 1, property is the default value, separation state is the default value joinable,
     * The entry function is thread1_entry and the entry function parameter is NULL */
    result = pthread_create(&tid1,NULL,thread1_entry,NULL);
    check_result("thread1 created",result);

    /* Create thread 2, the property is the default value, the separation state is the default value
      joinable,
     * The entry function is thread2_entry and the entry function parameter is NULL */
    result = pthread_create(&tid2,NULL,thread2_entry,NULL);
    check_result("thread2 created",result);

    pthread_detach(tid2);    /* detach thread 2 */

    return 0;
}
```

1.4.8.2.4 Waiting for Thread to End

```
int pthread_join (pthread_t thread, void**value_ptr);
```

Parameter	Description
thread	Thread handle (thread identifier)

Parameter	Description
value_ptr	User-defined pointer to store the return value of the waiting thread, which can be obtained by the function pthread_join()
Return	——
0	succeeded
EDEADLK	Thread join itself
EINVAL	Join a thread with a detached state
ESRCH	Could not find the thread

The thread calling this function blocks and waits for the thread with the joinable property to finish running and gets the return value of the thread. The address of the returned value is stored in `value_ptr` and frees the resources held by thread.
The pthread_join() and pthread_detach() functions are similar in that they are used to reclaim the resources occupied by threads after the thread running ends. A thread cannot wait for itself to end. The detached state of the thread thread must be `joinable`, and one thread only corresponds to the `pthread_join()` call. A thread with a split state of joinable will only release the resources it occupies when other threads execute `pthread_join()` on it. So in order to avoid memory leaks, all threads that will end up running, either detached or set to detached, or use pthread_join() to reclaim the resources they consume.

1.4.8.2.4.1 Example Code for Waiting for the Thread to End

The following program code initializes 2 threads, they have the same priority, and the threads of the same priority are scheduled according to the time slice. The separation status of the 2 thread attributes is the default value joinable, and thread 1 starts running first, and ends after printing 3 times of information. Thread 2 calls pthread_join() to block waiting for thread 1 to end, and reclaims the resources occupied by thread 1, and thread 2 prints the message every 2 seconds.

```
#include <pthread.h>
#include <unistd.h>
#include <stdio.h>

/* Thread control block */
static pthread_t tid1;
static pthread_t tid2;

/* Function return value check */
static void check_result(char* str,int result)
{
    if (0 == result)
    {
        printf("%s successfully!\n",str);
    }
    else
    {
        printf("%s failed! error code is %d\n",str,result);
    }
}

/* Thread 1 entry function */
static void* thread1_entry(void* parameter)
{
    int i;

    for (int i = 0;i < 3;i++)    /* Cycle print 3 times */
    {
        printf("thread1 run count: %d\n",i);
        sleep(2);    /* Sleep 2 seconds */
    }

    printf("thread1 exited!\n");
    return NULL;
}

/* Thread 2 entry function */
static void* thread2_entry(void* parameter)
{
    int count = 0;
    void* thread1_return_value;

    /* Blocking waiting thread 1 running end */
```

```
    pthread_join(tid1, NULL);

    /* Thread 2 print information to start output */
    while(1)
    {
        /* Print thread count value output */
        printf("thread2 run count: %d\n",count ++);
        sleep(2);    /* Sleep 2 seconds */
    }

    return NULL;
}

/* User application portal */
int rt_application_init()
{
    int result;
    /* Create thread 1, property is the default value, separation state is the default value joinable,
     * The entry function is thread1_entry and the entry function parameter is NULL */
    result = pthread_create(&tid1,NULL,thread1_entry,NULL);
    check_result("thread1 created",result);

    /* Create thread 2, the property is the default value, the separation state is the default value
      joinable,
     * The entry function is thread2_entry and the entry function parameter is NULL */
    result = pthread_create(&tid2,NULL,thread2_entry,NULL);
    check_result("thread2 created",result);

    return 0;
}
```

1.4.8.2.5 Exit Thread

```
void pthread_exit(void *value_ptr);
```

Parameter	Description
value_ptr	User-defined pointer to store the return value of the waiting thread, which can be obtained by the function pthread_join()

Calling this function by the pthread thread terminates execution, just as the process calls the exit() function and returns a pointer to the value returned by the thread. The thread exit is initiated by the thread itself.

> If the split state of the thread is joinable, the resources occupied by the thread will not be released after the thread exits. The pthread_join() function must be called to release the resources occupied by the thread.

1.4.8.2.5.1 Example Code for Exiting Thread

This program initializes 2 threads, they have the same priority, and the threads of the same priority are scheduled according to the time slice. The separation state of the two thread attributes is the default value joinable, and thread 1 starts running first, sleeps for 2 seconds after printing the information once, and then prints the exit information and then ends the operation. Thread 2 calls pthread_join() to block waiting for thread 1 to end, and reclaims the resources occupied by thread 1, and thread 2 prints the message every 2 seconds.

```
#include <pthread.h>
#include <unistd.h>
#include <stdio.h>

/* Thread control block */
static pthread_t tid1;
static pthread_t tid2;

/* Function return value check function */
static void check_result(char* str,int result)
{
    if (0 == result)
    {
        printf("%s successfully!\n",str);
    }
    else
    {
        printf("%s failed! error code is %d\n",str,result);
    }
}

/* Thread 1 entry function */
```

```
static void* thread1_entry(void* parameter)
{
    int count = 0;
    while(1)
    {
        /* Print thread count value output */
        printf("thread1 run count: %d\n",count ++);
        sleep(2);    /* Sleep 2 seconds */
        printf("thread1 will exit!\n");

        pthread_exit(0);    /* Thread 1 voluntarily quits */
    }
}

/* Thread 2 entry function */
static void* thread2_entry(void* parameter)
{
    int count = 0;

    /* The block waits for thread 1 to finish running */
    pthread_join(tid1,NULL);
    /* Thread 2 starts outputting print information */
    while(1)
    {
        /* Print thread count value output */
        printf("thread2 run count: %d\n",count ++);
        sleep(2);    /* Sleep 2 seconds */
    }
}

/* User application portal */
int rt_application_init()
{
    int result;

    /* Create thread 1, property is the default value, separation state is the default value joinable,
     * The entry function is thread1_entry and the entry function parameter is NULL */
    result = pthread_create(&tid1,NULL,thread1_entry,NULL);
    check_result("thread1 created",result);

    /* Create thread 2, the property is the default value, the separation state is the default value
      joinable,
     * The entry function is thread2_entry and the entry function parameter is NULL */
    result = pthread_create(&tid2,NULL,thread2_entry,NULL);
    check_result("thread2 created",result);

    return 0;
}
```

1.4.8.3 Mutex

Mutexes, also known as mutually exclusive semaphores, are a special binary semaphore. Mutexes are used to ensure the integrity of shared resources. Only one thread can access the shared resource at any time. To access shared resources, the thread must first obtain the mutex. After the access is complete, the mutex must be released. Embedded shared resources include memory, IO, SCI, SPI, etc. If two threads access shared resources at the same time, there may be problems because one thread may use the resource while another thread modifies the shared resource and consider sharing.

There are only two kinds of operations of mutex, locking or unlocking, and only one thread holds a mutex at a time. When a thread holds it, the mutex is latched and its ownership is obtained by this thread. Conversely, when this thread releases it, it unlocks the mutex and loses its ownership. When a thread holds a mutex, other threads will not be able to unlock it or hold it.

The main APIs of the mutex include: calling `pthread_mutex_init()` to initialize a mutex, `pthread_mutex_destroy()` to destroy the mutex, pthread_mutex_lock() to lock the mutex, and `pthread_mutex_unlock()` to unlock the mutex.

The rt-thread operating system implements a priority inheritance algorithm to prevent priority inversion.Priority inheritance is the practice of raising the priority of a low-priority thread that occupies a resource to the same level as the highest-priority thread of all the threads waiting for the resource, then executing, and then returning to the initial setting when the low-priority thread releases the resource.Thus, threads that inherit priority prevent system resources from being preempted by any intermediate priority thread.

For a detailed introduction to priority reversal, please refer to the Inter-thread Synchronization Mutex section.

1.4.8.3.1 Mutex Lock Control Block

Each mutex corresponds to a mutex control block that contains some information about the control of the mutex. Before creating a mutex, you must first define a variable of type `pthread_mutex_t`. pthread_mutex_t is a redefinition of pthread_mutex. The pthread_mutex data structure is defined in the pthread.h header file. The data structure is as follows:

```
struct pthread_mutex
{
    pthread_mutexattr_t attr;     /* Mutex attribute */
    struct rt_mutex lock;         /* RT-Thread Mutex lock control block */
};
typedef struct pthread_mutex pthread_mutex_t;

//rt_mutex is a data structure defined in the RT-Thread kernel, defined in the rtdef.h header file. The data
      structure is as follows:

struct rt_mutex
{
    struct rt_ipc_object parent;                /* Inherited from the ipc_object class */
    rt_uint16_t          value;                 /* Mutex value */
    rt_uint8_t           original_priority;     /* thread's original priority */
    rt_uint8_t           hold;                  /* Mutex lock holding count  */
    struct rt_thread    *owner;                 /* Thread that currently has a mutex */
};
typedef struct rt_mutex* rt_mutex_t;            /* Rt_mutext_t is a pointer to the mutex structure */
```

1.4.8.3.2 Initialize the Mutex

```
int pthread_mutex_init(pthread_mutex_t *mutex, const pthread_mutexattr_t *attr);
```

Parameter	Description
mutex	Mutex lock handle, cannot be NULL
attr	Pointer to the mutex attribute, if the pointer is NULL, the default attribute is used.
return	——
0	succeeded
EINVAL	Invalid parameter

This function initializes the mutex `mutex` and sets the mutex property according to the mutex attribute object pointed to by `attr`. After successful initialization, the mutex is unlocked and the thread can obtain it. This function encapsulates the rt_mutex_init() function.
In addition to calling the pthread_mutex_init() function to create a mutex, you can also statically initialize the mutex with the macro PTHREAD_MUTEX_INITIALIZER by: `pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER` (structure constant), which is equivalent to specifying attr to NULL when calling pthread_mutex_init().
The mutex lock properties and related functions are described in detail in the *thread advanced programming* chapter. In general, the default properties can be used.

1.4.8.3.3 Destroy Mutex

```
int pthread_mutex_destroy(pthread_mutex_t *mutex);
```

Parameter	Description
mutex	Mutex lock handle, cannot be NULL
return	——
0	Succeeded
EINVAL	Mutex is empty or mutex has been destroyed
EBUSY	Mutex is being used

This function destroys the mutex `mutex`. Mutex is mutable in an uninitialized state after destruction. After destroying

the mutex's properties and control block parameters will not be valid, but you can call pthread_mutex_init() to reinitialize the destroyed mutex. However, there is no need to destroy the mutex that is statically initialized with the macro PTHREAD_MUTEX_INITIALIZER.
The mutex can be destroyed when it is determined that the mutex is not locked and no thread is blocked on the mutex.

1.4.8.3.4 Blocking Mode Locks the Mutex

```
int pthread_mutex_lock(pthread_mutex_t *mutex);
```

Parameter	Description
mutex	Mutex lock handle, cannot be NULL
return	——
0	Succeeded
EINVAL	Invalid parameter
EDEADLK	Mutexes mutex do not call this function repeatedly for a thread with a nested lock

This function locks the mutex `mutex`, which is a wrapper of the rt_mutex_take() function. If the mutex has not been locked yet, the thread applying for the mutex will successfully lock the mutex. If the mutex has been locked by the current thread and the mutex type is a nested lock, the mutex's holding count is incremented by one, and the current thread will not suspend waiting (deadlock), but the thread must corresponds to the same number of unlocks. If the mutex is held by another thread, the current thread will be blocked until the other thread unlocks the mutex, and the thread waiting for the mutex will acquire the mutex according to the *first in first out* principle. .

1.4.8.3.5 Non-blocking Mode Locks the Mutex

```
int pthread_mutex_trylock(pthread_mutex_t *mutex);
```

Parameter	Description
mutex	Mutex lock handle, cannot be NULL
return	——
0	succeeded
EINVAL	Invalid parameter
EDEADLK	Mutexes are not nested locks, but threads call this function repeatedly
EBUSY	Mutexes mutex has been locked by other threads

This function is a non-blocking version of the pthread_mutex_lock() function. The difference is that if the mutex has been locked, the thread will not be blocked, but the error code will be returned immediately.

1.4.8.3.6 Unlock the Mutex

```
int pthread_mutex_unlock(pthread_mutex_t *mutex);
```

Parameter	Description
mutex	Mutex lock handle, cannot be NULL
return	——
0	Succeeded
EINVAL	Invalid parameter
EPERM	This function is called repeatedly by a thread when the mutex is not a nested lock

Parameter	Description
EBUSY	Unlock the mutex held by other threads with the type of error detection lock

Calling this function to unlock the mutex. This function is a wrapper of the rt_mutex_release() function. When the thread completes the access of the shared resource, it should release the possessed mutex as soon as possible, so that other threads can acquire the mutex in time. Only a thread that already has a mutex can release it, and its holding count is decremented by one each time the mutex is released. When the mutex's holding count is zero (ie, the holding thread has released all holding operations), the mutex becomes available, and the thread waiting on the mutex is placed in a first-in-first-out manner. If the thread's run priority is promoted by the mutex lock, then when the mutex is released, the thread reverts to the priority before holding the mutex.

1.4.8.3.7 Example Code for Mutex Lock

This program will initialize 2 threads, they have the same priority, 2 threads will call the same printer() function to output their own string, the printer() function will output only one character at a time, then sleep for 1 second, call printer The thread of the () function also sleeps. If you do not use a mutex, thread 1 prints a character, and after hibernation, thread 2 is executed, and thread 2 prints a character, so that the thread 1 and thread 2 strings cannot be completely printed, and the printed string is confusing. If a mutex is used to protect the print function printer() shared by 2 threads, thread 1 takes the mutex and executes the printer() print function to print a character, then sleeps for 1 second, which is switched to thread 2 because The nick lock has been locked by thread 1, and thread 2 will block until thread 1 of thread 1 is fully released and the thread 2 is woken up after the mutex is actively released.

```
#include <pthread.h>
#include <unistd.h>
#include <stdio.h>

/* Thread control block */
static pthread_t tid1;
static pthread_t tid2;
/* Mutex lock control block */
static pthread_mutex_t mutex;
/* Thread-sharing print function */
static void printer(char* str)
{
    while(*str != 0)
    {
        putchar(*str);    /* Output one character */
        str++;
        sleep(1);    /* Sleep 1 second */
    }
    printf("\n");
}
/* Function return value check */
static void check_result(char* str,int result)
{
    if (0 == result)
    {
        printf("%s successfully!\n",str);
    }
    else
    {
        printf("%s failed! error code is %d\n",str,result);
    }
}
/* Thread entry */
static void* thread1_entry(void* parameter)
{
    char* str = "thread1 hello RT-Thread";
    while (1)
    {
        pthread_mutex_lock(&mutex);      /* Mutex lock */

        printer(str);  /* Access shared print function */

        pthread_mutex_unlock(&mutex);  /* Unlock after access is complete */

        sleep(2);    /* Sleep 2 seconds */
    }
}
static void* thread2_entry(void* parameter)
{
    char* str = "thread2 hi world";
    while (1)
    {
```

```
        pthread_mutex_lock(&mutex);  /* The mutex locks */

        printer(str);  /* Access shared print function */

        pthread_mutex_unlock(&mutex);  /* Unlock after access is complete */

        sleep(2);    /* Sleep 2 seconds */
    }
}
/* User application portal */
int rt_application_init()
{
    int result;
    /* Initialize a mutex */
    pthread_mutex_init(&mutex,NULL);

    /* Create thread 1, the thread entry is thread1_entry, the attribute parameter is NULL, the default
      value is selected, and the entry parameter is NULL.*/
    result = pthread_create(&tid1,NULL,thread1_entry,NULL);
    check_result("thread1 created",result);

    /* Create thread 2, thread entry is thread2_entry, property parameter is NULL select default value,
      entry parameter is NULL*/
    result = pthread_create(&tid2,NULL,thread2_entry,NULL);
    check_result("thread2 created",result);

    return 0;
}
```

1.4.8.4 Conditional Variable

A condition variable is actually a semaphore used for synchronization between threads. A condition variable is used to block a thread. When a condition is met, a condition is sent to the blocked thread. The blocking thread is woken up. The condition variable needs to be used with the mutex. The mutex is used to protect the shared data.
Condition variables can be used to inform shared data status. For example, if a thread that processes a shared resource queue finds that the queue is empty, then the thread can only wait until one node is added to the queue. After adding, a conditional variable signal is sent to activate the waiting thread.
The main operations of the condition variable include: calling `pthread_cond_init()` to initialize the condition variable, calling `pthread_cond_destroy()` to destroy a condition variable, calling `pthread_cond_wait()` to wait for a condition variable, and calling `pthread_cond_signal()` to send a condition variable.

1.4.8.4.1 Condition Variable Control Block

Each condition variable corresponds to a condition variable control block, including some information about the operation of the condition variable. A `pthread_cond_t` condition variable control block needs to be defined before initializing a condition variable. `pthread_cond_t` is a redefinition of the `pthread_cond` structure type, defined in the pthread.h header file.

```
struct pthread_cond
{
    pthread_condattr_t attr;        /* Condition variable attribute */
    struct rt_semaphore sem;        /* RT-Thread semaphore control block */
};
typedef struct pthread_cond pthread_cond_t;

Rt_semaphore is a data structure defined in the RT-Thread kernel. It is a semaphore control block defined in
      the rtdef.h header file.

struct rt_semaphore
{
   struct rt_ipc_object parent; /* Inherited from the ipc_object class */
   rt_uint16_t value;           /* Semaphore value  */
};
```

1.4.8.4.2 Initialization Condition Variable

```
int pthread_cond_init(pthread_cond_t *cond, const pthread_condattr_t *attr);
```

Parameter	Description
cond	Conditional variable handle, cannot be NULL
attr	Pointer to the condition variable property, if NULL then use the default property value

Parameter	Description
return	——
0	succeeded
EINVAL	Invalid parameter

This function initializes the `cond` condition variable and sets its properties according to the condition variable property pointed to by `attr` , which is a wrapper of the `rt_sem_init()` function, based on semaphore implementation. The condition variable is not available after successful initialization.
You can also statically initialize a condition variable with the macro PTHREAD_COND_INITIALIZER by: `pthread_cond_t cond = PTHREAD_COND_INITIALIZER` (structural constant), which is equivalent to specifying NULL when calling `pthread_cond_init()`.
Attr General setting NULL use the default value, as described in the thread advanced programming chapter.

1.4.8.4.3 Destroy Condition Variables

```
int pthread_cond_destroy(pthread_cond_t *cond);
```

Parameter	Description
cond	Conditional variable handle, cannot be NULL
return	——
0	Succeeded
EINVAL	Invalid parameter
EPERM	Mutexes are not nested locks, but threads call this function repeatedly
EBUSY	Condition variables are being used

This function destroys the `cond` condition variable, and the `cond` is uninitialized after destruction. The attribute and control block parameters of the condition variable will not be valid after destruction, but can be reinitialized by calling `pthread_cond_init()` or statically.
Before destroying a condition variable, you need to make sure that no threads are blocked on the condition variable and will not wait to acquire, signal, or broadcast.

1.4.8.4.4 Blocking Mode to Obtain Condition Variables

```
int pthread_cond_wait(pthread_cond_t *cond, pthread_mutex_t *mutex);
```

Parameter	Description
cond	Conditional variable handle, cannot be NULL
mutex	Pointer to the mutex control block, cannot be NULL
return	——
0	Succeeded
EINVAL	Invalid parameter

This function gets the `cond` condition variable in blocking mode. The thread needs to lock the mutex before waiting for the condition variable. This function first determines whether the condition variable is available. If it is not available, initializes a condition variable, then unlocks the mutex and then tries to acquire a semaphore when the semaphore's value is greater than zero, it indicates that the semaphore is available, the thread will get the semaphore, and the condition variable will be obtained, and the corresponding semaphore value will be decremented by 1. If the value of the semaphore is equal to zero, indicating that the semaphore is not available, the thread will block until the semaphore is available, after which the mutex will be locked again.

1.4.8.4.5 Specify Blocking Time to Obtain Condition Variables

```
int pthread_cond_timedwait(pthread_cond_t *cond,
                           pthread_mutex_t *mutex,
                           const struct timespec *abstime);
```

Parameter	Description
cond	Conditional variable handle, cannot be NULL
mutex	Pointer to the mutex control block, cannot be NULL
abstime	The specified wait time in operating system clock tick (OS Tick)
return	——
0	Succeeded
EINVAL	Invalid parameter
EPERM	Mutexes are not nested locks, but threads call this function repeatedly
ETIMEDOUT	time out

The only difference between this function and the `pthread_cond_wait()` function is that if the condition variable is not available, the thread will be blocked for the `abstime` duration. After the timeout, the function will directly return the ETIMEDOUT error code and the thread will be woken up to the ready state.

1.4.8.4.6 Send a Conditional Semaphore

```
int pthread_cond_signal(pthread_cond_t *cond);
```

Parameter	Description
cond	Conditional variable handle, cannot be NULL
return	——
0	Succeeded

This function sends a signal and wakes up only one thread waiting for the `cond` condition variable, which encapsulates the rt_sem_release() function, which is to send a semaphore. When the value of the semaphore is equal to zero, and a thread waits for this semaphore, it will wake up the first thread waiting in the queue of the semaphore to get the semaphore. Otherwise the value of the semaphore will be increased by 1.

1.4.8.4.7 Broadcast

```
int pthread_cond_broadcast(pthread_cond_t *cond);
```

Parameter	Description
cond	Conditional variable handle, cannot be NULL
return	——
0	Succeeded
EINVAL	Invalid parameter

Calling this function will wake up all threads waiting for the `cond` condition variable.

1.4.8.4.8 Example Code for Condition Variable

This example is a producer consumer model with a producer thread and a consumer thread that have the same priority. The producer will produce a number every 2 seconds, put it in the list pointed to by the `head`, and then call

pthread_cond_signal() to send signal to the consumer thread to inform the consumer that there is data in the thread list. The consumer thread calls pthread_cond_wait() to wait for the producer thread to send a signal.

```
#include <pthread.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>

/* Statically initialize a mutex and a condition variable */
static pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
static pthread_cond_t cond = PTHREAD_COND_INITIALIZER;

/* Pointer to the thread control block */
static pthread_t tid1;
static pthread_t tid2;

/* Function return value check */
static void check_result(char* str,int result)
{
    if (0 == result)
    {
        printf("%s successfully!\n",str);
    }
    else
    {
        printf("%s failed! error code is %d\n",str,result);
    }
}

/* The structure data produced by the producer is stored in the linked list. */
struct node
{
    int n_number;
    struct node* n_next;
};
struct node* head = NULL; /* Link header, is a shared resource */

/* Consumer thread entry function */
static void* consumer(void* parameter)
{
    struct node* p_node = NULL;

    pthread_mutex_lock(&mutex);    /* Lock on mutex */

    while (1)
    {
        while (head == NULL)    /* Determine if there are elements in the list */
        {
            pthread_cond_wait(&cond,&mutex); /* Try to get a condition variable */
        }
        /*
        Pthread_cond_wait() will unlock the mutex first, then block in the wait queue until the fetch
      condition variable is awakened. After being woken up, the thread will lock the mutex again and
      successfully enter the critical section.
        */

        p_node = head;    /* Obtain resources */
        head = head->n_next;    /* Header pointing to the next resource */
        /* Printout */
        printf("consume %d\n",p_node->n_number);

        free(p_node);    /* Release the memory occupied by the node after obtaining the resource */
    }
    pthread_mutex_unlock(&mutex);    /* Release the mutex */
    return 0;
}
/* Producer thread entry function */
static void* product(void* patameter)
{
    int count = 0;
    struct node *p_node;

    while(1)
    {
        /* Dynamically allocate a block of structured memory */
        p_node = (struct node*)malloc(sizeof(struct node));
        if (p_node != NULL)
        {
            p_node->n_number = count++;
            pthread_mutex_lock(&mutex);    /* To operate on the critical resource head, lock it first */

            p_node->n_next = head;
            head = p_node;    /* Insert data into the list header */

            pthread_mutex_unlock(&mutex);    /* Unlock */
```

```
            printf("produce %d\n",p_node->n_number);

            pthread_cond_signal(&cond);    /* send a Signal to wake up  a thread */

            sleep(2);    /* Sleep 2 seconds */
        }
        else
        {
            printf("product malloc node failed!\n");
            break;
        }
    }
}

int rt_application_init()
{
    int result;

    /* Create a producer thread, the property is the default value, the entry function is product, and the
      entry function parameter is NULL*/
    result = pthread_create(&tid1,NULL,product,NULL);
    check_result("product thread created",result);

    /* Create a consumer thread, the property is the default value, the entry function is consumer, and the
      entry function parameter is NULL */
    result = pthread_create(&tid2,NULL,consumer,NULL);
    check_result("consumer thread created",result);

    return 0;
}
```

1.4.8.5 Read-write Lock

Read-write locks are also known as multi-reader single-writer locks. The read-write lock divides the visitors of the shared resource into readers and writers. The reader only reads and accesses the shared resources, and the writer needs to write the shared resources. Only one thread can occupy the read-write lock of the write mode at the same time, but there can be multiple threads simultaneously occupying the read-write lock of the read mode. Read-write locks are suitable for reading data structures much more often than writes because read patterns can be shared when locked, and write mode locks are exclusive.
Read-write locks are usually implemented based on mutex locks and condition variables. A thread can lock a read-write lock several times, and it must also have the corresponding number of unlocks.
The main operations of the read-write lock include: calling `pthread_rwlock_init()` to initialize a read-write lock, the write thread calling `pthread_rwlock_wrlock()` to lock the read-write lock, and the read thread calling `pthread_rwlock_rdlock()` to lock the read-write lock , when this read-write lock is not required, calling `pthread_rwlock_destroy()` to destroys the read-write lock.

1.4.8.5.1 Read-write Lock Control Block

Each read-write lock corresponds to a read-write lock control block, including some information about the operation of the read-write lock. `pthread_rwlock_t` is a redefinition of the `pthread_rwlock` data structure, defined in the `pthread.h` header file. Before creating a read-write lock, you need to define a data structure of type `pthread_rwlock_t`.

```
struct pthread_rwlock
{
    pthread_rwlockattr_t attr;        /* Read-write lock attribute */
    pthread_mutex_t rw_mutex;         /* Mutex lock */
    pthread_cond_t rw_condreaders;    /* Conditional variables for the reader thread to use */
    pthread_cond_t rw_condwriters;    /* Conditional variable for the writer thread to use */
    int rw_nwaitreaders;    /* Reader thread waiting to count */
    int rw_nwaitwriters;    /* Writer thread waiting to count */
    /* Read-write lock value, value 0: unlocked, value -1: is locked by the writer thread, greater than 0
      value: locked by the reader thread */
    int rw_refcount;
};
typedef struct pthread_rwlock pthread_rwlock_t;     /* Type redefinition */
```

1.4.8.5.2 Initialize Read-write Lock

```
int pthread_rwlock_init (pthread_rwlock_t *rwlock,
                         const pthread_rwlockattr_t *attr);
```

Parameter	Description
rwlock	Read-write lock handle, cannot be NULL
attr	Pointer to the read-write lock property, RT-Thread does not use this variable
return	——
0	Succeeded
EINVAL	Invalid parameter

This function initializes an `rwlock` read-write lock. This function initializes the semaphore and condition variables of the read-write lock control block with default values, and the associated count parameter is initially 0. The read-write lock after initialization is in an unlocked state.
You can also use the macro PTHREAD_RWLOCK_INITIALIZER to statically initialize the read-write lock by: `pthread_rwlock_t mutex = PTHREAD_RWLOCK_INITIALIZER` (structural constant), which is equivalent to specifying `attr` a NULL value when calling pthread_rwlock_init().
`attr` generally sets NULL to the default value, as described in the chapter on advanced threading.

1.4.8.5.3 Destroy Read-write Lock

```
int pthread_rwlock_destroy (pthread_rwlock_t *rwlock);
```

Parameter	Description
rwlock	Read-write lock handle, cannot be NULL
return	——
0	Succeeded
EINVAL	Invalid parameter
EBUSY	The read-write lock is currently being used or has a thread waiting for the read-write lock
EDEADLK	Deadlock

This function destroys a `rwlock` read-write lock, which destroys the mutex and condition variables in the read-write lock. After the destruction, the properties of the read-write lock and the control block parameters will not be valid, but you can call pthread_rwlock_init() or re-initialize the read-write lock in static mode.

1.4.8.5.4 Read-Lock of Read-Write Lock

1.4.8.5.4.1 Blocking mode Read-lock the read-write locks

```
int pthread_rwlock_rdlock (pthread_rwlock_t *rwlock);
```

Parameter	Description
rwlock	Read-write lock handle, cannot be NULL
return	——
0	Succeeded
EINVAL	Invalid parameter
EDEADLK	Deadlock

The reader thread can call this function to read-lock the `rwlock` read-write lock. If the read-write lock is not write-locked and no writer thread is blocked on the read-write lock, the read-write thread will successfully acquire the read-write lock. If the read-write lock has been write-locked, the reader thread will block until the thread that executes the write-lock unlocks the read-write lock.

1.4.8.5.4.2 Non-blocking Mode Read-lock Read-write Locks

```
int pthread_rwlock_tryrdlock (pthread_rwlock_t *rwlock);
```

Parameter	Description
rwlock	Read-write lock handle, cannot be NULL
return	——
0	Succeeded
EINVAL	Invalid parameter
EBUSY	The read-write lock is currently being used or has a thread waiting for the read-write lock
EDEADLK	Deadlock

This function differs from the pthread_rwlock_rdlock() function in that if the read-write lock is already write-locked, the reader thread is not blocked, but instead returns an error code EBUSY.

1.4.8.5.4.3 Specify Blocking Time for the Read-write Lock to be Read-Locked

```
int pthread_rwlock_timedrdlock (pthread_rwlock_t *rwlock,
                                const struct timespec *abstime);
```

Parameter	Description
rwlock	Read-write lock handle, cannot be NULL
abstime	The specified wait time in operating system clock tick (OS Tick)
return	——
0	Succeeded
EINVAL	Invalid parameter
ETIMEDOUT	Time out
EDEADLK	Deadlock

The difference between this function and the pthread_rwlock_rdlock() function is that if the read-write lock has been write-locked, the reader thread will block the specified abstime duration. After the timeout, the function will return the error code ETIMEDOUT and the thread will be woken up to the ready state.

1.4.8.5.5 Write-Lock of Read-Write Lock

1.4.8.5.5.1 Blocking Mode Write-Locks a Read-write Lock

```
int pthread_rwlock_wrlock (pthread_rwlock_t *rwlock);
```

Parameter	Description
rwlock	Read-write lock handle, cannot be NULL
return	——
0	Succeeded
EINVAL	Invalid parameter
EDEADLK	Deadlock

The writer thread calls this function to write-lock the `rwlock` read-write lock. A write-lock read-write lock is similar to a mutex, and only one thread can write-lock a read-write lock at a time. If no thread locks the read-write lock, that is, the read-write lock value is 0, the writer thread that calls this function will write-lock the read-write lock, and other threads cannot acquire the read-write lock at this time. If there is already a thread locked the read-write lock, ie the

read/write lock value is not 0, then the writer thread will be blocked until the read-write lock is unlocked.

1.4.8.5.5.2 Non-blocking Mode Write-Lock a Read-write Lock

```
int pthread_rwlock_trywrlock (pthread_rwlock_t *rwlock);
```

Parameter	Description
rwlock	Read-write lock handle, cannot be NULL
return	——
0	Succeeded
EINVAL	Invalid parameter
EBUSY	The read-write lock is currently Write-Locked or there are reader threads blocked on the read-write lock
EDEADLK	Deadlock

The only difference between this function and the pthread_rwlock_wrlock() function is that if a thread has locked the read-write lock, ie the read-write lock value is not 0, the writer thread that called the function will directly return an error code, and the thread will not be Blocked.

1.4.8.5.5.3 Specify Blocking Time for the Read-write Lock to be Write-Lock

```
int pthread_rwlock_timedwrlock (pthread_rwlock_t *rwlock,
                                const struct timespec *abstime);
```

Parameter	Description
rwlock abstime	Read-write lock handle, cannot specify the wait time for NULL, the unit is the operating system clock beat (OS Tick)
return	——
0	Succeeded
EINVAL	Invalid parameter
ETIMEDOUT	Time out
EDEADLK	Deadlock

The only difference between this function and the pthread_rwlock_wrlock() function is that if a thread locks the read-write lock, that is, the read-write lock value is not 0, the calling thread blocks the specified `abstime` duration. After the timeout, the function returns the error code ETIMEDOUT, and the thread will be woken up to the ready state.

1.4.8.5.6 Unlock the Read-write Lock

```
int pthread_rwlock_unlock (pthread_rwlock_t *rwlock);
```

Parameter	Description
rwlock	Read-write lock handle, cannot be NULL
return	——
0	Succeeded
EINVAL	Invalid parameter
EDEADLK	Deadlock

This function unlocks the `rwlock` read-write lock. A thread locks the same read-write lock multiple times and must have the same number of unlocks. If multiple threads wait for the read-write lock to lock after unlocking, the system

will activate the waiting thread according to the first-in-first-out rule.

1.4.8.5.7 Example Code for Read-write Lock

This program has two reader threads, one reader thread. The two reader threads read-lock the read-write lock first, then sleep for 2 seconds. This time the other reader threads can read-lock the read-write lock, and then read the shared data.

```
#include <pthread.h>
#include <sched.h>
#include <stdio.h>

/* Thread control block */
static pthread_t reader1;
static pthread_t reader2;
static pthread_t writer1;
/* Shared data book */
static int book = 0;
/* Read-write lock */
static pthread_rwlock_t rwlock;
/* Function result check */
static void check_result(char* str,int result)
{
    if (0 == result)
    {
        printf("%s successfully!\n",str);
    }
    else
    {
        printf("%s failed! error code is %d\n",str,result);
    }
}
/* Thread entry */
static void* reader1_entry(void* parameter)
{
    while (1)
    {

        pthread_rwlock_rdlock(&rwlock);  /* Try to read-lock the read-write lock */

        printf("reader1 read book value is %d\n",book);
        sleep(2);  /* The thread sleeps for 2 seconds, switching to another thread to run */

        pthread_rwlock_unlock(&rwlock);  /* Unlock the read-write lock after the thread runs */
    }
}
static void* reader2_entry(void* parameter)
{
    while (1)
    {
        pthread_rwlock_rdlock(&rwlock);  /* Try to read-lock the read-write lock */

        printf("reader2 read book value is %d\n",book);
        sleep(2);  /* The thread sleeps for 2 seconds, switching to another thread to run */

        pthread_rwlock_unlock(&rwlock);  /* Unlock the read-write lock after the thread runs */
    }
}
static void* writer1_entry(void* parameter)
{
    while (1)
    {
        pthread_rwlock_wrlock(&rwlock);  /* Try to write-lock the read-write lock */

        book++;
        printf("writer1 write book value is %d\n",book);

        pthread_rwlock_unlock(&rwlock);  /* Unlock the read-write lock */

        sleep(2);  /* The thread sleeps for 2 seconds, switching to another thread to run */
    }
}
/* User application portal */
int rt_application_init()
{
    int result;
    /* Default property initializes read-write lock */
    pthread_rwlock_init(&rwlock,NULL);

    /* Create a reader1 thread, the thread entry is reader1_entry, the thread attribute is the default
      value, and the entry parameter is NULL*/
    result = pthread_create(&reader1,NULL,reader1_entry,NULL);
    check_result("reader1 created",result);

    /* Create a reader2 thread, the thread entry is reader2_entry, the thread attribute is the default
```

```
      value, and the entry parameter is NULL*/
    result = pthread_create(&reader2,NULL,reader2_entry,NULL);
    check_result("reader2 created",result);

    /* Create a writer1 thread, the thread entry is writer1_entry, the thread attribute is, and the entry
      parameter is NULL*/
    result = pthread_create(&writer1,NULL,writer1_entry,NULL);
    check_result("writer1 created",result);

    return 0;
}
```

1.4.8.6 Barrier

Barriers are a way to synchronize multithreading. Barrier means a barrier or railing that blocks multiple threads that arrive in the same railing until all threads arrived, then remove the railings and let them go at the same time. The thread that arrives first will block, and when all the threads that call the pthread_barrier_wait() function (the number equal to the count specified by the barrier initialization) arrive, the threads will enter the ready state from the blocked state and participate in the system scheduling again.
Barriers are implemented based on condition variables and mutex locks. The main operations include: calling `pthread_barrier_init()` to initialize a barrier, and other threads calling `pthread_barrier_wait()`. After all threads arrived, the thread wakes up to the ready state. Destroy a barrier by calling pthread_barrier_↩ destroy() when the barrier will not be used.

1.4.8.6.1 Barrier Control Block

Before creating a barrier, you need to define a `pthread_barrier_t` barrier control block. `pthread_↩ barrier_t` is a redefinition of the `pthread_barrier` structure type, defined in the pthread.h header file.

```
struct pthread_barrier
{
    int count;    /* The number of waiting threads specified */
    pthread_cond_t cond;      /* Conditional variable */
    pthread_mutex_t mutex;    /* Mutex lock */
};
typedef struct pthread_barrier pthread_barrier_t;
```

1.4.8.6.2 Create a Barrier

```
int pthread_barrier_init(pthread_barrier_t *barrier,
                         const pthread_barrierattr_t *attr,
                         unsigned count);
```

Parameter	Description
attr	Pointer to the barrier property, if passing NULL, use the default value. PTHREAD_PROCESS_↩ PRIVATE must be used as a non-NULL value.
barrier	Barrier handle
count	The number of waiting threads specified
return	——
0	Succeeded
EINVAL	Invalid parameter

This function creates a `barrier` barrier and initializes the conditional variables and mutex locks of the barrier control block according to the default parameters. The number of waiting threads specified after initialization is `count`, and pthread_barrier_wait() must be called for `count` threads.
attr generally sets NULL to the default value, as described in the chapter on *thread advanced programming*.

1.4.8.6.3 Destruction of Barrier

```
int pthread_barrier_destroy(pthread_barrier_t *barrier);
```

Parameter	Description
barrier	Barrier handle
return	——
0	Succeeded
EINVAL	Invalid parameter

This function destroys a barrier. The barrier's properties and control block parameters will not be valid after destruction, but can be reinitialized by calling pthread_barrier_init().

1.4.8.6.4 Wait for Barrier

```
int pthread_barrier_wait(pthread_barrier_t *barrier);
```

Parameter	Description
barrier	Barrier handle
return	——
0	Succeeded
EINVAL	Invalid parameter

This function synchronizes the threads waiting in front of the barrier and called by each thread. If the number of queue waiting threads is not 0, count will be decremented by 1. If the count is 0, indicating that all threads have reached the railing. All arriving threads will be woken up and re-entered into the ready state to participate in system scheduling. If count is not 0 after the decrease, it indicates that there is still threads that do not reach the barrier, and the calling thread will block until all threads reach the barrier.

1.4.8.6.5 Example Code for Barrier

This program will create 3 threads, initialize a barrier, and the barrier waits for 3 threads. 3 threads will call pthread_barrier_wait() to wait in front of the barrier. When all 3 threads are arrived, 3 threads will enter the ready state. The output count information is printed every 2 seconds.

```
#include <pthread.h>
#include <unistd.h>
#include <stdio.h>

/* Thread control block */
static pthread_t tid1;
static pthread_t tid2;
static pthread_t tid3;
/* Barrier control block */
static pthread_barrier_t barrier;
/* Function return value check function */
static void check_result(char* str,int result)
{
    if (0 == result)
    {
            printf("%s successfully!\n",str);
    }
    else
    {
            printf("%s failed! error code is %d\n",str,result);
    }
}
/* Thread 1 entry function */
static void* thread1_entry(void* parameter)
{
    int count = 0;

    printf("thread1 have arrived the barrier!\n");
    pthread_barrier_wait(&barrier);    /* Reach the barrier and wait for other threads to arrive */

    while (1)
    {
        /* Print thread count value output */
        printf("thread1 count: %d\n",count ++);

        /* Sleep 2 seconds */
```

```
        sleep(2);
    }
}
/* Thread 2 entry function */
static void* thread2_entry(void* parameter)
{
    int count = 0;

    printf("thread2 have arrived the barrier!\n");
    pthread_barrier_wait(&barrier);

    while (1)
    {
        /* Print thread count value */
        printf("thread2 count: %d\n",count ++);

        /* Sleep 2 seconds */
        sleep(2);
    }
}
/* Thread 3 entry function */
static void* thread3_entry(void* parameter)
{
    int count = 0;

    printf("thread3 have arrived the barrier!\n");
    pthread_barrier_wait(&barrier);

    while (1)
    {
        /* Print thread count value */
        printf("thread3 count: %d\n",count ++);

        /* Sleep 2 seconds */
        sleep(2);
    }
}
/* User application portal */
int rt_application_init()
{
    int result;
    pthread_barrier_init(&barrier,NULL,3);

    /* Create thread 1, thread entry is thread1_entry, property parameter is set to NULL, select default
      value, entry parameter is NULL*/
    result = pthread_create(&tid1,NULL,thread1_entry,NULL);
    check_result("thread1 created",result);

    /* Create thread 2, thread entry is thread2_entry, property parameter is set to NULL, select default
      value, entry parameter is NULL*/
    result = pthread_create(&tid2,NULL,thread2_entry,NULL);
    check_result("thread2 created",result);

    /* Create thread 3, thread entry is thread3_entry, property parameter is set to NULL Select default
      value, entry parameter is NULL*/
    result = pthread_create(&tid3,NULL,thread3_entry,NULL);
    check_result("thread3 created",result);

}
```

1.4.8.7 Semaphore

Semaphores can be used for communication between processes and processes, or between in-process threads. Each semaphore has a semaphore value that is not less than 0, corresponding to the available amount of semaphore. Call sem_init() or sem_open() to assign an initial value to the semaphore . Call sem_post() to increment the semaphore value by 1. Call sem_wait() to decrement the semaphore value by 1. If the current semaphore is 0, call sem_wait(), the thread will suspended on the wait queue for this semaphore until the semaphore value is greater than 0 and is available.
Depending on the value of the semaphore (representing the number of available resources), POSIX semaphores can be divided into:

- **Binary semaphore**: The value of the semaphore is only 0 and 1, and the initial value is specified as 1. This is the same as a mutex. If the resource is locked, the semaphore value is 0. If the resource is available, the semaphore value is 1. Equivalent to only one key, after the thread gets the key, after completing the access to the shared resource, you need to unlock it, put the key back, and use it for other threads that need this key. The method is the same as the mutex lock. The wait semaphore function must be used in pairs with the send semaphore function. It cannot be used alone.
- **Count semaphore**: The value of the semaphore ranges from 0 to a limit greater than 1 (POSIX indicates that

the system's maximum limit is at least 32767). This count indicates the number of available semaphores. At this point, the send semaphore function can be called separately to send the semaphore, which is equivalent to having more than one key, the thread takes a key and consumes one, and the used key does not have to be put back.

POSIX semaphores are also divided into named semaphores and unnamed semaphores:

- A named semaphore: its value is stored in a file and is generally used for inter-process synchronization or mutual exclusion.
- Unnamed semaphore: Its value is stored in memory and is generally used for inter-thread synchronization or mutual exclusion.

The POSIX semaphore of the RT-Thread operating system is mainly based on a package of RT-Thread kernel semaphores, mainly used for communication between threads in the system. It is used in the same way as the semaphore of the RT-Thread kernel.

1.4.8.7.1 Semaphore Control Block

Each semaphore corresponds to a semaphore control block. Before creating a semaphore, you need to define a sem_t semaphore control block. Sem_t is a redefinition of the posix_sem structure type, defined in the semaphore.h header file.

```
struct posix_sem
{
    rt_uint16_t refcount;
    rt_uint8_t unlinked;
    rt_uint8_t unamed;
    rt_sem_t sem;    /* RT-Thread semaphore */
    struct posix_sem* next;     /* Point to the next semaphore control block */
};
typedef struct posix_sem sem_t;

Rt_sem_t is the RT-Thread semaphore control block, defined in the rtdef.h header file.

struct rt_semaphore
{
   struct rt_ipc_object parent;/* Inherited from the ipc_object class */
   rt_uint16_t value;   /* Semaphore value  */
};
/* rt_sem_t is a pointer type to the semaphore structure */
typedef struct rt_semaphore* rt_sem_t;
```

1.4.8.7.2 Unnamed semaphore

The value of an unnamed semaphore is stored in memory and is generally used for inter-thread synchronization or mutual exclusion. Before using it, you must first call sem_init() to initialize it.

1.4.8.7.2.1 Initialize the unnamed semaphore

```
int sem_init(sem_t *sem, int pshared, unsigned int value);
```

Parameter	Description
sem	Semaphore handle
value	The initial value of the semaphore, indicating the available amount of semaphore resources
pshared	RT-Thread unimplemented parameters
return	——
0	Succeeded
-1	Failed

This function initializes an unnamed semaphore sem, initializes the semaphore related data structure according to the given or default parameters, and puts the semaphore into the semaphore list. The semaphore value after initialization is the given initial value. This function is a wrapper of the rt_sem_create() function.

1.4.8.7.2.2 Destroy Unnamed Semaphore

```
int sem_destroy(sem_t *sem);
```

Parameter	Description
sem	Semaphore handle
return	——
0	Succeeded
-1	Failed

This function destroys an unnamed semaphore sem and releases the resources occupied by the semaphore.

1.4.8.7.3 Named Semaphore

A named semaphore whose value is stored in a file and is generally used for inter-process synchronization or mutual exclusion. Two processes can operate on named semaphores of the same name. The well-known semaphore implementation in the RT-Thread operating system is similar to the unnamed semaphore. It is designed for communication between threads and is similar in usage.

1.4.8.7.3.1 Create or Open a Named Semaphore

```
sem_t *sem_open(const char *name, int oflag, ...);
```

Parameter	Description
name	Semaphore name
oflag	The way the semaphore is opened
return	——
Semaphore handle	Succeeded
NULL	Failed

This function creates a new semaphore based on the semaphore name or opens an existing semaphore. The optional values for Oflag are `0`, `O_CREAT` or `O_CREAT|O_EXCL`. If Oflag is set to `O_CREAT` , a new semaphore is created. If Oflag sets to `O_CREAT|O_EXCL`, it returns NULL if the semaphore already exists, and creates a new semaphore if it does not exist. If Oflag is set to 0, a semaphore does not exist and NULL is returned.

1.4.8.7.3.2 Detach the Named Semaphore

```
int sem_unlink(const char *name);
```

Parameter	Description
name	Semaphore name
return	——
0	Succeeded
-1	Failed, semaphore does not exist

This function looks up the semaphore based on the semaphore name, and marks the semaphore as a detached state if the semaphore is present. Then check the reference count. If the value is 0, the semaphore is deleted immediately. If the value is not 0, it will not be deleted until all threads holding the semaphore close the semaphore.

1.4.8.7.3.3 Close the Named Semaphore

```
int sem_close(sem_t *sem);
```

Parameter	Description
sem	Semaphore handle
return	——
0	Succeeded
-1	Failed

When a thread terminates,it closes the semaphore it occupies. Whether the thread terminates voluntarily or involuntarily, this closing operation is performed. This is equivalent to a reduction of 1 in the number of semaphores held. If the holding count is zero after subtracting 1 and the semaphore is in separated state, the `sem` semaphore will be deleted and the resources it occupies will be released.

1.4.8.7.4 Obtain Semaphore Value

```
int sem_getvalue(sem_t *sem, int *sval);
```

Parameter	Description
sem	Semaphore handle, cannot be NULL
sval	Save the obtained semaphore value address, cannot be NULL
return	——
0	Succeeded
-1	Failed

This function obtains the value of the semaphore and saves it in the memory pointed to by `sval` to know the amount of semaphore resources.

1.4.8.7.5 Blocking Mode to Wait Semaphore

```
int sem_wait(sem_t *sem);
```

Parameter	Description
sem	Semaphore handle, cannot be NULL
return	——
0	Succeeded
-1	Failed

The thread calls this function to get the semaphore, which is a wrapper of the `rt_sem_take(sem,RT_WAITING_FOREVER)` function. If the semaphore value is greater than zero, the semaphore is available, the thread gets the semaphore, and the semaphore value is decremented by one. If the semaphore value is equal to 0, indicating that the semaphore is not available, the thread is blocked and entering the suspended state and queued in a first-in, first-out manner until the semaphore is available.

1.4.8.7.6 Non-blocking Mode to Wait Semaphore

```
int sem_trywait(sem_t *sem);
```

Parameter	Description
sem	Semaphore handle, cannot be NULL
return	——
0	Succeeded

Parameter	Description
-1	Failed

This function is a non-blocking version of the sem_wait() function and is a wrapper of the `rt_sem_take(sem,0)` function. When the semaphore is not available, the thread does not block, but returns directly.

1.4.8.7.7 Specify the Blocking Time Waiting for the Semaphore

```
int sem_timedwait(sem_t *sem, const struct timespec *abs_timeout);
```

Parameter	Description
sem	Semaphore handle, cannot be NULL
abs_timeout	The specified wait time in operating system clock tick (OS Tick)
return	——
0	Succeeded
-1	Failed

The difference between this function and `the sem_wait()` function is that if the semaphore is not available, the thread will block the duration of `abs_timeout`. After the timeout, the function returns -1, and the thread will be awakened from the blocking state to the ready state.

1.4.8.7.8 Send Semaphore

```
int sem_post(sem_t *sem);
```

Parameter	Description
sem	Semaphore handle, cannot be NULL
return	——
0	Succeeded
-1	Failed

This function will release a sem semaphore, which is a wrapper of the rt_sem_release() function. If the thread queue waiting for the semaphore is not empty, indicating that there are threads waiting for the semaphore, the first thread waiting for the semaphore will switch from the suspended state to the ready state, waiting for system scheduling. If no thread is waiting for the semaphore, the semaphore value will be incremented by one.

1.4.8.7.9 Example Code for Unnamed Semaphore

A typical case of semaphore usage is the producer consumer model. A producer thread and a consumer thread operate on the same block of memory, the producer fills the shared memory, and the consumer reads the data from the shared memory.
This program creates 2 threads, 2 semaphores, one semaphore indicates that the shared data is empty, one semaphore indicates that the shared data is not empty, and a mutex is used to protect the shared resource. After the producer thread produces the data, it will send a `full_sem` semaphore to the consumer, informing the consumer that the thread has data available, and waiting for the `empty_sem` semaphore sent by the consumer thread after 2 seconds of sleep. The consumer thread processes the shared data after the `full_sem` sent by the producer, and sends an `empty_sem` semaphore to the producer thread after processing. The program will continue to loop like this.

```
#include <pthread.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
```

```c
#include <semaphore.h>

/* Statically initialize a mutex to protect shared resources */
static pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
/* 2 semaphore control blocks, one for resource empty signals and one for resource full signals */
static sem_t empty_sem,full_sem;

/* Pointer to the thread control block */
static pthread_t tid1;
static pthread_t tid2;

/* Function return value check */
static void check_result(char* str,int result)
{
    if (0 == result)
    {
        printf("%s successfully!\n",str);
    }
    else
    {
        printf("%s failed! error code is %d\n",str,result);
    }
}

/* The structure data produced by the producer is stored in the linked list. */
struct node
{
    int n_number;
    struct node* n_next;
};
struct node* head = NULL; /* Link header, a shared resource */

/* Consumer thread entry function */
static void* consumer(void* parameter)
{
    struct node* p_node = NULL;

    while (1)
    {
        sem_wait(&full_sem);
        pthread_mutex_lock(&mutex);    /* Lock mutex  */

        while (head != NULL)    /* Determine if there are elements in the list */
        {
            p_node = head;    /* Obtain resources */
            head = head->n_next;    /* Header pointing to the next resource */
            /* Print */
            printf("consume %d\n",p_node->n_number);

            free(p_node);    /* Release the memory occupied by the node after getting the resource */
        }

        pthread_mutex_unlock(&mutex);    /* The critical section data operation is completed, and the mutex
      is released. */

        sem_post(&empty_sem);  /* Send a null semaphore to the producer */
    }
}
/* Producer thread entry function */
static void* product(void* patameter)
{
    int count = 0;
    struct node *p_node;

    while(1)
    {
        /* Dynamically allocate a block of structured memory */
        p_node = (struct node*)malloc(sizeof(struct node));
        if (p_node != NULL)
        {
            p_node->n_number = count++;
            pthread_mutex_lock(&mutex);    /* To operate on the critical resource head, lock it first */

            p_node->n_next = head;
            head = p_node;    /* Insert data into the list header */

            pthread_mutex_unlock(&mutex);    /* Unlock */
            printf("produce %d\n",p_node->n_number);

            sem_post(&full_sem);  /* Send a full semaphore to the consumer */
        }
        else
        {
            printf("product malloc node failed!\n");
            break;
        }
        sleep(2);    /* Sleep 2 seconds */
```

```
        sem_wait(&empty_sem);  /* Wait for consumers to send empty semapho */
    }
}

int rt_application_init()
{
    int result;

    sem_init(&empty_sem,NULL,0);
    sem_init(&full_sem,NULL,0);
    /* Create a producer thread, the property is the default value, the entry function is product, and the
      entry function parameter is NULL*/
    result = pthread_create(&tid1,NULL,product,NULL);
    check_result("product thread created",result);

    /* Create a consumer thread, the property is the default value, the entry function is consumer, and the
      entry function parameter is NULL */
    result = pthread_create(&tid2,NULL,consumer,NULL);
    check_result("consumer thread created",result);

    return 0;
}
```

1.4.8.8 Message Queue

Message Queuing is another commonly used inter-thread communication method that accepts messages of unfixed length from threads or interrupt service routines and caches the messages in their own memory space. Other threads can also read the corresponding message from the message queue, and when the message queue is empty, the reader thread can be suspended. When a new message arrives, the suspended thread will be woken up to receive and process the message.
The main operations of the message queue include: creating or opening by the function `mq_open()`, calling `mq_send()` to send a message to the message queue, calling `mq_receive()` to get a message from the message queue, and when the message queue is not in use, you can call `mq_unlink()` to delete message queue.
POSIX message queue is mainly used for inter-process communication. The POSIX message queue of RT-Thread operating system is mainly based on a package of RT-Thread kernel message queue, mainly used for communication between threads in the system. It is used in the same way as the message queue of the RT-Thread kernel.

1.4.8.8.1 Message Queue Control Block

Each message queue corresponds to a message queue control block. Before creating a message queue, you need to define a message queue control block. The message queue control block is defined in the mqueue.h header file.

```
struct mqdes
{
    rt_uint16_t refcount;  /* Reference count */
    rt_uint16_t unlinked;  /* Separation status of the message queue, a value of 1 indicates that the
      message queue has been separated */
    rt_mq_t mq;            /* RT-Thread message queue control block */
    struct mqdes* next;    /* Point to the next message queue control block */
};
typedef struct mqdes* mqd_t;  /* Message queue control block pointer type redefinition */
```

1.4.8.8.2 Create or Open a Message Queue

```
mqd_t mq_open(const char *name, int oflag, ...);
```

Parameter	Description
name	Message queue name
oflag	Message queue open mode
return	——
Message queue handle	Succeeded
NULL	Failed

This function creates a new message queue or opens an existing message queue based on the name of the message queue. The optional values for Oflag are `0`, `O_CREAT` or `O_CREAT\|O_EXCL`. If Oflag is set to `O_CREAT` then a new message queue is created. If Oflag sets `O_CREAT\|O_EXCL`, it returns NULL if the message

queue already exists, and creates a new message queue if it does not exist. If Oflag is set to `0`, the message queue does not exist and returns NULL.

1.4.8.8.3 Detach Message Queue

```
int mq_unlink(const char *name);
```

Parameter	Description
name	Message queue name
return	——
0	Succeeded
-1	Failed

This function finds the message queue based on the message queue name name. If found, it sets the message queue to a detached state. If the hold count is 0, the message queue is deleted and the resources occupied by the message queue are released.

1.4.8.8.4 Close the Message Queue

```
int mq_close(mqd_t mqdes);
```

Parameter	Description
mqdes	Message queue handle
return	——
0	Succeeded
-1	Failed

When a thread terminates,it closes the message queue it occupies. Whether the thread terminates voluntarily or involuntarily, this closure is performed. This is equivalent to the message queue holding count minus 1. If the holding count is 0 after the minus 1 and the message queue is in the separated state, the `mqdes` message queue will be deleted and released the resources it occupies.

1.4.8.8.5 Block Mode to Send a Message

```
int mq_send(mqd_t mqdes,
            const char *msg_ptr,
            size_t msg_len,
            unsigned msg_prio);
```

Parameter	Description
mqdes	Message queue handle, cannot be NULL
sg_ptr	Pointer to the message to be sent, cannot be NULL
msg_len	The length of the message sent
msg_prio	RT-Thread unimplemented this parameter
return	——
0	Succeeded
-1	Failed

This function is used to send a message to the `mqdes` message queue, which is a wrapper of the rt_mq_send() function. This function adds the message pointed to by `msg_ptr` to the `mqdes` message queue, and the length

of the message sent `msg_len` must be less than or equal to the maximum message length set when the message queue is created.
If the message queue is full, that is, the number of messages in the message queue is equal to the maximum number of messages, the thread that sent the message or the interrupt program will receive an error code (-RT_EFULL).

1.4.8.8.6 Specify Blocking Time to Send a Message

```
int mq_timedsend(mqd_t mqdes,
                 const char *msg_ptr,
                 size_t msg_len,
                 unsigned msg_prio,
                 const struct timespec *abs_timeout);
```

Parameter	Description
mqdes	Message queue handle, cannot be NULL
msg_ptr	Pointer to the message to be sent, cannot be NULL
msg_len	The length of the message sent
msg_prio	RT-Thread unimplemented parameters
abs_timeout	The specified wait time in operating system clock tick (OS Tick)
Parameter	——
0	Succeeded
-1	Failed

Currently RT-Thread does not support sending messages with the specified blocking time, but the function interface has been implemented, which is equivalent to calling mq_send().

1.4.8.8.7 Blocking Mode to Receive Message

```
ssize_t mq_receive(mqd_t mqdes,
                   char *msg_ptr,
                   size_t msg_len,
                   unsigned *msg_prio);
```

Parameter	Description
mqdes	Message queue handle, cannot be NULL
msg_ptr	Pointer to the message to be sent, cannot be NULL
msg_len	The length of the message sent
msg_prio	RT-Thread unimplemented parameters
return	——
Message length	Succeeded
-1	Failed

This function removes the oldest message from the `mqdes` message queue and puts the message in the memory pointed to by `msg_ptr`. If the message queue is empty, the thread that called the mq_receive() function will block until the message in the message queue is available.

1.4.8.8.8 Specify Blocking Time to Receive Message

```
ssize_t mq_timedreceive(mqd_t mqdes,
                        char *msg_ptr,
                        size_t msg_len,
                        unsigned *msg_prio,
                        const struct timespec *abs_timeout);
```

Parameter	Description
mqdes	Message queue handle, cannot be NULL
msg_ptr	Pointer to the message to be sent, cannot be NULL
msg_len	The length of the message sent
msg_prio	RT-Thread unimplemented parameters
abs_timeout	The specified wait time in operating system clock tick (OS Tick)
return	——
Message length	Succeeded
-1	Failed

The difference between this function and the mq_receive() function is that if the message queue is empty, the thread will block the `abs_timeout` duration. After the timeout, the function will return `-1`, and the thread will be awakened from the blocking state to the ready state.

1.4.8.8.9 Example Code for Message Queue

This program creates 3 threads, thread2 accepts messages from the message queue, and thread2 and thread3 send messages to the message queue.

```
#include <mqueue.h>
#include <stdio.h>

/* Thread control block */
static pthread_t tid1;
static pthread_t tid2;
static pthread_t tid3;
/* Message queue handle */
static mqd_t mqueue;

/* Function return value check function */
static void check_result(char* str,int result)
{
    if (0 == result)
    {
            printf("%s successfully!\n",str);
    }
    else
    {
            printf("%s failed! error code is %d\n",str,result);
    }
}
/* Thread 1 entry function */
static void* thread1_entry(void* parameter)
{
    char buf[128];
    int result;

    while (1)
    {
        /* Receive messages from the message queue */
        result = mq_receive(mqueue, &buf[0], sizeof(buf), 0);
        if (result != -1)
        {
            /* Output content */
            printf("thread1 recv [%s]\n", buf);
        }

        /* Sleep 1 second */
       // sleep(1);
    }
}
/* Thread 2 entry function */
static void* thread2_entry(void* parameter)
{
    int i, result;
    char buf[] = "message2 No.x";

    while (1)
    {
       for (i = 0; i < 10; i++)
        {
            buf[sizeof(buf) - 2] = '0' + i;

            printf("thread2 send [%s]\n", buf);
```

```
            /* Send a message to the message queue */
            result = mq_send(mqueue, &buf[0], sizeof(buf), 0);
            if (result == -1)
            {
                /* Message queue full, delayed 1s */
                printf("thread2:message queue is full, delay 1s\n");
                sleep(1);
            }
        }

        /* Sleep 2 seconds */
        sleep(2);
    }
}
/* Thread 3 entry function */
static void* thread3_entry(void* parameter)
{
    int i, result;
    char buf[] = "message3 No.x";

    while (1)
    {
       for (i = 0; i < 10; i++)
        {
            buf[sizeof(buf) - 2] = '0' + i;

            printf("thread3 send [%s]\n", buf);
            /* Send messages to the message queue */
            result = mq_send(mqueue, &buf[0], sizeof(buf), 0);
            if (result == -1)
            {
                /* Message queue full, delayed 1s */
                printf("thread3:message queue is full, delay 1s\n");
                sleep(1);
            }
        }

        /* Sleep 2 seconds */
        sleep(2);
    }
}
/* User application portal */
int rt_application_init()
{
    int result;
    struct mq_attr mqstat;
    int oflag = O_CREAT|O_RDWR;
#define MSG_SIZE    128
#define MAX_MSG        128
    memset(&mqstat, 0, sizeof(mqstat));
    mqstat.mq_maxmsg = MAX_MSG;
    mqstat.mq_msgsize = MSG_SIZE;
    mqstat.mq_flags = 0;
    mqueue = mq_open("mqueue1",O_CREAT,0777,&mqstat);

    /* Create thread 1, thread entry is thread1_entry, property parameter is set to NULL, select default
      value, entry parameter is NULL*/
    result = pthread_create(&tid1,NULL,thread1_entry,NULL);
    check_result("thread1 created",result);

    /* Create thread 2, thread entry is thread2_entry, property parameter is set to NULL, select default
      value, entry parameter is NULL*/
    result = pthread_create(&tid2,NULL,thread2_entry,NULL);
    check_result("thread2 created",result);

    /* Create thread 3, thread entry is thread3_entry, property parameter is set to NULL Select default
      value, entry parameter is NULL*/
    result = pthread_create(&tid3,NULL,thread3_entry,NULL);
    check_result("thread3 created",result);


    return 0;
}
```

1.4.8.9 Thread Advanced Programming

This section provides a detailed introduction to some of the rarely used property objects and related functions. The thread attributes implemented by RT-Thread include thread stack size, thread priority, thread separation status, and thread scheduling policy. `pthread_create()` must initialize the property object before using the property object. APIs such as setting thread properties should be called before the thread is created. Changes of thread attributes do not affect the threads that have been created.

The thread attribute structure `pthread_attr_t` is defined in the pthread.h header file. The thread attribute

structure is as follows:

```
/* pthread_attr_t Type redefinition */
typedef struct pthread_attr pthread_attr_t;
/* Thread attribute structure */
struct pthread_attr
{
    void*      stack_base;      /* Thread stack address */
    rt_uint32_t stack_size;     /* Thread stack size */
    rt_uint8_t priority;        /* Thread priority */
    rt_uint8_t detachstate;     /* Thread detached state */
    rt_uint8_t policy;          /* Thread scheduling policy */
    rt_uint8_t inheritsched;    /* Thread inheritance */
};
```

1.4.8.9.1 Thread Property Initialization and Deinitialization

The thread property initialization and deinitialization functions are as follows:

```
int pthread_attr_init(pthread_attr_t *attr);
int pthread_attr_destroy(pthread_attr_t *attr);
```

Parameter	Description
attr	Pointer to the thread property
return	——
0	Succeeded

Using the pthread_attr_init() function initializes the thread attribute structure `attr` with the default value, which is equivalent to setting the parameter to NULL when calling the thread initialization function. You need to define a `pthread_attr_t` attribute object before use. This function must be called before the pthread_create() function. The pthread_attr_destroy() function deinitializes the property pointed to by `attr` and can then reinitialize this property object by calling the pthread_attr_init() function again.

1.4.8.9.2 Thread Detached State

Setting or getting the separation state of a thread is as follows. By default, the thread is non-separated.

```
int pthread_attr_setdetachstate(pthread_attr_t *attr, int state);
int pthread_attr_getdetachstate(pthread_attr_t const *attr, int *state);
```

Parameter	Description
attr	Pointer to the thread property
state	Thread detached state
return	——
0	Succeeded

The thread separation state property value state can be `PTHREAD_CREATE_JOINABL` (non-detached) and `THREAD_CREATE_DETACHED` (detached).
The detached state of a thread determines how a thread reclaims the resources it occupies after the end of its run. There are two types of thread separation: joinable or detached. When the thread is created, you should call pthread_join() or pthread_detach() to reclaim the resources occupied by the thread after it finishes running. If the thread's detached state is joinable, other threads can call the pthread_join() function to wait for the thread to finish and get the thread return value, and then reclaim the resources occupied by the thread. A thread with a detached state cannot be joined by another thread. Immediately after the end of its operation, the system resources are released.

1.4.8.9.3 Thread Scheduling Policy

Setting \ Obtaining thread scheduling policy function is as follows:

```
int pthread_attr_setschedpolicy(pthread_attr_t *attr, int policy);
int pthread_attr_getschedpolicy(pthread_attr_t const *attr, int *policy);
```

Only the function interface is implemented. The default different priorities are based on priority scheduling, and the same priority time slice polling scheduling

1.4.8.9.4 Thread Scheduling Parameter

Set / Obtain the thread's priority function as follows:

```
int pthread_attr_setschedparam(pthread_attr_t *attr,
                               struct sched_param const *param)
int pthread_attr_getschedparam(pthread_attr_t const *attr,
                               struct sched_param *param)
```

Parameter	Description
attr	Pointer to the thread property
param	Pointer to the dispatch parameter
return	——
0	Succeeded

The `pthread_attr_setschedparam()` function sets the priority of the thread. Use `param` to set the thread priority.
Parameter : The `struct sched_param` is defined in sched.h and has the following structure:

```
struct sched_param
{
    int sched_priority;    /* Thread priority */
};
```

The member `sched_paraority` of the `sched_param` controls the priority value of the thread.

1.4.8.9.5 Thread Stack Size

Set / Obtain the stack size of a thread is as follows:

```
int pthread_attr_setstacksize(pthread_attr_t *attr, size_t stack_size);
int pthread_attr_getstacksize(pthread_attr_t const *attr, size_t *stack_size);
```

Parameter	Description
attr	Pointer to the thread property
stack_size	Thread stack size
return	——
0	Succeeded

The `pthread_attr_setstacksize()` function sets the stack size in bytes. Stack space address alignment is required on most systems (for example, the ARM architecture needs to be aligned to a 4-byte address).

1.4.8.9.6 Thread Stack Size and Address

Set / Obtain the stack address and stack size of a thread is as follows:

```
int pthread_attr_setstack(pthread_attr_t *attr,
                          void *stack_base,
                          size_t stack_size);
int pthread_attr_getstack(pthread_attr_t const *attr,
                          void**stack_base,
                          size_t *stack_size);
```

Parameter	Description
attr	Pointer to the thread property
stack_size	Thread stack size
stack_base	Thread stack address

Parameter	Description
return	——
0	Succeeded

1.4.8.9.7 Thread Attribute Related Function

The function that sets / obtains the scope of the thread is as follows:

```
int pthread_attr_setscope(pthread_attr_t *attr, int scope)
int pthread_attr_getscope(pthread_attr_t const *attr);
```

Parameter	Description
attr	Pointer to the thread property
scope	Thread scope
return	——
0	scope is PTHREAD_SCOPE_SYSTEM
EOPNOTSUPP	scope is PTHREAD_SCOPE_PROCESS
EINVAL	scope is PTHREAD_SCOPE_SYSTEM

1.4.8.9.8 Example Code for Thread Property

This program will initialize 2 threads, they have a common entry function, but their entry parameters are not the same. The first thread created will use the provided `attr` thread attribute, and the other thread will use the system default attribute. Thread priority is a very important parameter, so this program will modify the first created thread to have a priority of 8, and the system default priority of 24.

```
#include <pthread.h>
#include <unistd.h>
#include <stdio.h>
#include <sched.h>

/* Thread control block */
static pthread_t tid1;
static pthread_t tid2;

/* Function return value check */
static void check_result(char* str,int result)
{
    if (0 == result)
    {
        printf("%s successfully!\n",str);
    }
    else
    {
        printf("%s failed! error code is %d\n",str,result);
    }
}
/* Thread entry function */
static void* thread_entry(void* parameter)
{
    int count = 0;
    int no = (int) parameter; /* Obtain the thread's entry parameters */

    while (1)
    {
        /* Printout thread count value */
        printf("thread%d count: %d\n", no, count ++);

        sleep(2);    /* Sleep 2 seconds */
    }
}

/* User application portal */
int rt_application_init()
{
    int result;
    pthread_attr_t attr;      /* Thread attribute */
    struct sched_param prio;  /* Thread priority */

    prio.sched_priority = 8;  /* Priority is set to 8 */
    pthread_attr_init(&attr);  /* Initialize the property with default values first */
```

```
    pthread_attr_setschedparam(&attr,&prio);  /* Modify the priority corresponding to the attribute */

    /* Create thread 1, attribute is attr, entry function is thread_entry, and the entry function parameter
      is 1 */
    result = pthread_create(&tid1,&attr,thread_entry,(void*)1);
    check_result("thread1 created",result);

    /* Create thread 2, the property is the default value, the entry function is thread_entry, and the entry
      function parameter is 2 */
    result = pthread_create(&tid2,NULL,thread_entry,(void*)2);
    check_result("thread2 created",result);

    return 0;
}
```

1.4.8.9.9 Thread Cancellation

Cancellation is a mechanism that allows one thread to end other threads. A thread can send a cancel request to another thread. Depending on the settings, the target thread may ignore it and may end immediately or postpone it until the next cancellation point.

1.4.8.9.9.1 Send Cancellation Request

The cancellation request can be sent using the following function:

```
int pthread_cancel(pthread_t thread);
```

Parameter	Description
thread	Thread handle
return	——
0	Succeeded

This function sends a cancel request to the `thread` thread. Whether the thread will respond to the cancellation request and when it responds depends on the state and type of thread cancellation.

1.4.8.9.9.2 Set Cancel Status

The cancellation request can be set using the following function:

```
int pthread_setcancelstate(int state, int *oldstate);
```

Parameter	Description
state	There are two values: `PTHREAD_CANCEL_ENABLE`: Cancel enable. `PTHREAD_CANCEL_DISABLE`: Cancel disabled (default value when thread is created).
oldstate	Save the original cancellation status
return	——
0	Succeeded
EINVAL	state is not PTHREAD_CANCEL_ENABLE or PTHREAD_CANCEL_DISABLE

This function sets the cancel state and is called by the thread itself. Canceling the enabled thread will react to the cancel request, and canceling the disabled thread will not react to the cancel request.

1.4.8.9.9.3 Set Cancellation Type

You can use the following function to set the cancellation type, which is called by the thread itself:

```
int pthread_setcanceltype(int type, int *oldtype);
```

Parameter	Description
type	There are 2 values: PTHREAD_CANCEL_DEFFERED: After the thread receives the cancellation request, it will continue to run to the next cancellation point and then end. (Default value when thread is created) . PTHREAD_CANCEL_ASYNCHRONOUS: The thread ends immediately.
oldtype	Save the original cancellation type
return	——
0	Succeeded
EINVAL	state is neither PTHREAD_CANCEL_DEFFERED nor PTHREAD_CANCEL_ASYNCHRONOUS

1.4.8.9.9.4 Set Cancellation Point

The cancellation point can be set using the following function:

```
void pthread_testcancel(void);
```

This function creates a cancellation point where the thread is called. Called primarily by a thread that does not contain a cancellation point, it can respond to a cancellation request. This function does not work if pthread_↩ testcancel() is called while the cancel state is disabled.

1.4.8.9.9.5 Cancellation Point

The cancellation point is where the thread ends when it accepts the cancellation request. According to the POSIX standard, system calls that cause blocking, such as pthread_join(), pthread_testcancel(), pthread_cond_wait(), pthread_cond_timedwait(), and sem_wait(), are cancellation points.
All cancellation points included in RT-Thread are as follows:

- mq_receive()
- mq_send()
- mq_timedreceive()
- mq_timedsend()
- msgrcv()
- msgsnd()
- msync()
- pthread_cond_timedwait()
- pthread_cond_wait()
- pthread_join()
- pthread_testcancel()
- sem_timedwait()
- sem_wait()
- pthread_rwlock_rdlock()
- pthread_rwlock_timedrdlock()
- pthread_rwlock_timedwrlock()
- pthread_rwlock_wrlock()

1.4.8.9.9.6 Example Code for Thread Cancel

This program creates 2 threads. After thread2 starts running, it sleeps for 8 seconds. Thread1 sets its own cancel state and type, and then prints the run count information in an infinite loop. After thread2 wakes up, it sends a cancel request to thread1, and thread1 ends the run immediately after receiving the cancel request.

```
#include <pthread.h>
#include <unistd.h>
#include <stdio.h>

/* Thread control block */
static pthread_t tid1;
static pthread_t tid2;

/* Function return value check */
static void check_result(char* str,int result)
{
    if (0 == result)
    {
        printf("%s successfully!\n",str);
    }
    else
    {
        printf("%s failed! error code is %d\n",str,result);
    }
}
/* Thread 1 entry function */
static void* thread1_entry(void* parameter)
{
    int count = 0;
    /* Set the cancel state of thread 1 to be enabled. The cancel type is terminated immediately after the
      thread receives the cancel point. */
    pthread_setcancelstate(PTHREAD_CANCEL_ENABLE, NULL);
    pthread_setcanceltype(PTHREAD_CANCEL_ASYNCHRONOUS, NULL);

    while(1)
    {
        /* Print thread count value output */
        printf("thread1 run count: %d\n",count ++);
        sleep(2);    /* Sleep 2 seconds */
    }
}
/* Thread 2 entry function */
static void* thread2_entry(void* parameter)
{
    int count = 0;
    sleep(8);
    /* Send a cancel request to thread 1 */
    pthread_cancel(tid1);
    /* Waiting for thread 1 to finish in blocking mode */
    pthread_join(tid1,NULL);
    printf("thread1 exited!\n");
    /* Thread 2 print information to start output */
    while(1)
    {
        /* Print thread count value output */
        printf("thread2 run count: %d\n",count ++);
        sleep(2);    /* Sleep 2 seconds */
    }
}
/* User application portal */
int rt_application_init()
{
    int result;
    /* Create thread 1, the property is the default value, the separation state is the default value
      joinable, the entry function is thread1_entry, and the entry function parameter is NULL */
    result = pthread_create(&tid1,NULL,thread1_entry,NULL);
    check_result("thread1 created",result);

    /* Create thread 2, the property is the default value, the separation state is the default value
      joinable, the entry function is thread2_entry, and the entry function parameter is NULL */
    result = pthread_create(&tid2,NULL,thread2_entry,NULL);
    check_result("thread2 created",result);

    return 0;
}
```

1.4.8.9.10 One-time Initialization

It can be initialized once using the following function:

```
int pthread_once(pthread_once_t * once_control, void (*init_routine) (void));
```

Parameter	Description
once_control	Control variable
init_routine	Execute function
return	——
0	Succeeded

Sometimes we need to initialize some variables only once. If we do multiple initialization procedures, it will get an error. In traditional sequential programming, one-time initialization is often managed by using Boolean variables. The control variable is statically initialized to 0, and any code that relies on initialization can test the variable. If the variable value is still 0, it can be initialized and then set the variable to 1. Codes that are checked later will skip initialization.

1.4.8.9.11 Clean up after the Thread Ends

The thread cleanup function interface:

```
void pthread_cleanup_pop(int execute);
void pthread_cleanup_push(void (*routine)(void*), void *arg);
```

Parameter	Description
execute	0 or 1, determin whether to execute the cleanup function
routine	Pointer to the cleanup function
arg	The parameter passed to the cleanup function

pthread_cleanup_push() puts the specified cleanup `routine` into the thread's cleanup function list. pthread_↩ cleanup_pop() takes the first function from the header of the cleanup function list. If `execute` is a non-zero value, then this function is executed.

1.4.8.9.12 Other Thread Related Functions

1.4.8.9.12.1 Determine if two Threads are Equal

```
int pthread_equal (pthread_t t1, pthread_t t2);
```

Parameter	Description
pthread↩ _t	Thread handle
return	——
0	Not equal
1	Equal

1.4.8.9.12.2 Obtain Thread Handle

```
pthread_t pthread_self (void);
```

pthread_self() returns the handle of the calling thread.

1.4.8.9.12.3 Get the Maximum and Minimum Priority

```
int sched_get_priority_min(int policy);
int sched_get_priority_max(int policy);
```

Parameter	Description
policy	2 values are optional: SCHED_FIFO, SCHED_RR

sched_get_priority_min() returns a value of 0, with the highest priority in RT-Thread and sched_get_priority_max() with the lowest priority.

1.4.8.9.13 Mutex Attribute

The mutex properties implemented by RT-Thread include the mutex type and the mutex scope.

1.4.8.9.13.1 Mutex Lock Attribute Initialization and Deinitialization

```
int pthread_mutexattr_init(pthread_mutexattr_t *attr);
int pthread_mutexattr_destroy(pthread_mutexattr_t *attr);
```

Parameter	Description
attr	Pointer to the mutex attribute object
return	——
0	Succeeded
EINVAL	Invalid parameter

The pthread_mutexattr_init() function initializes the property object pointed to by `attr` with the default value, which is equivalent to setting the property parameter to NULL when calling the pthread_mutex_init() function.
The pthread_mutexattr_destroy() function will initialize the property object pointed to by `attr` and can be reinitialized by calling the pthread_mutexattr_init() function.

1.4.8.9.13.2 Mutex Lock Scope

```
int pthread_mutexattr_setpshared(pthread_mutexattr_t *attr, int  pshared);
int pthread_mutexattr_getpshared(pthread_mutexattr_t *attr, int *pshared);
```

Parameter	Description
type	Mutex type
pshared	There are 2 optional values: `PTHREAD_PROCESS_PRIVATE`: The default value, used to synchronize only threads in the process. `PTHREAD_PROCESS_SHARED`: Used to synchronize threads in this process and other processes.
return	——
0	Succeeded
EINVAL	Invalid parameter

1.4.8.9.13.3 Mutex Type

```
int pthread_mutexattr_settype(pthread_mutexattr_t *attr, int type);
int pthread_mutexattr_gettype(const pthread_mutexattr_t *attr, int *type);
```

Parameter	Description
type	Mutex type
attr	Pointer to the mutex attribute object
return	——
0	Succeeded
EINVAL	Invalid parameter

The type of mutex determines how a thread behaves when it acquires a mutex. RT-Thread implements three mutex types:

- **PTHREAD_MUTEX_NORMAL**: Normal lock. When a thread is locked, the remaining threads requesting the lock will form a wait queue, and after unlocking, the lock will be acquired in the first-in first-out manner. If a thread attempts to regain the mutex without first releasing the mutex, it does not generate a deadlock, but instead returns an error code, just like the error checking lock.
- **PTHREAD_MUTEX_RECURSIVE**: Nested locks that allow a thread to successfully acquire the same lock multiple times, requiring the same number of unlocks to release the mutex.
- **PTHREAD_MUTEX_ERRORCHECK**: Error checking lock, if a thread tries to regain the mutex without first releasing the mutex, an error is returned. This ensures that deadlocks do not occur when multiple locks are not allowed.

1.4.8.9.14 Condition Variable Attribute

Use the default value PTHREAD_PROCESS_PRIVATE to initialize the condition variable attribute attr to use the following function:

```
int pthread_condattr_init(pthread_condattr_t *attr);
```

Parameter	Description
attr	Pointer to a condition variable property object
return	——
0	Succeeded
EINVAL	Invalid parameter

1.4.8.9.14.1 Obtain Condition Variable Scope

```
int pthread_mutexattr_getpshared(pthread_mutexattr_t *attr, int *pshared);
```

Parameter	Description
attr	Pointer to a condition variable property object
return	——
0	Succeeded
EINVAL	Invalid parameter

1.4.8.9.15 Read-write Lock Attribute

1.4.8.9.15.1 Initialize Property

```
int pthread_rwlockattr_init (pthread_rwlockattr_t *attr);
```

Parameter	Description
attr	Pointer to the read-write lock property
return	——
0	Succeeded
-1	Invalid parameter

This function initializes the read-write lock attribute `attr` with the default value PTHREAD_PROCESS_PRIVATE.

1.4.8.9.15.2 Obtain Scope

```
int pthread_rwlockattr_getpshared (const pthread_rwlockattr_t *attr, int *pshared);
```

Parameter	Description
attr	Pointer to the read-write lock property
pshared	Pointer to the scope of the read-write lock
return	——
0	Succeeded
-1	Invalid parameter

The memory pointed to by pshared is saved as PTHREAD_PROCESS_PRIVATE.

1.4.8.9.16 Barrier Attribute

1.4.8.9.16.1 Initialize Property

```
int pthread_barrierattr_init(pthread_barrierattr_t *attr);
```

Parameter	Description
attr	Pointer to the barrier property
return	——
0	Succeeded
-1	Invalid parameter

The modified function initializes the barrier attribute `attr` with the default value PTHREAD_PROCESS_PRIVATE.

1.4.8.9.16.2 Obtain Scope

```
int pthread_barrierattr_getpshared(const pthread_barrierattr_t *attr, int *pshared);
```

Parameter	Description
attr	Pointer to the barrier property
pshared	Pointer to save barrier scope data
return	——
0	Succeeded
-1	Invalid parameter

1.4.8.9.17 Message Queue Property

The message queue attribute control block is as follows:

```
struct mq_attr
{
    long mq_flags;      /* Message queue flag to indicate whether to block */
    long mq_maxmsg;     /* Message queue maximum number of messages */
    long mq_msgsize;    /* The maximum number of bytes per message in the message queue */
    long mq_curmsgs;    /* Message queue current message number */
};
```

1.4.8.9.17.1 Obtain Attribute

```
int mq_getattr(mqd_t mqdes, struct mq_attr *mqstat);
```

Parameter	Description
mqdes	Pointer to the message queue control block
mqstat	Pointer to save the get data

Parameter	Description
return	——
0	Succeeded
-1	Invalid parameter

1.4.9 Ulog Log

1.4.9.1 Ulog Introduction

∗∗Log definition∗∗The log is to output the status, process and other information of the software to different media (for example: file, console, display, etc.), display and save. Provide reference for software traceability, performance analysis, system monitoring, fault warning and other functions during software debugging and maintenance. It can be said that the use of logs consumes at least 80% of the software life cycle.
∗∗The importance of the log∗∗For the operating system, because the complexity of the software is very large, single-step debugging is not suitable in some scenarios, the log component is almost standard part on the operating system. A sophisticated logging system can also make the debugging of the operating system more effective.
The origin of ulog: RT-Thread has always lacked a small, useful log component, and the birth of ulog complements this short board. It will be open sourced as a basic component of RT-Thread, allowing our developers to use a simple and easy-to-use logging system to improve development efficiency.
Ulog is a very simple and easy to use C/C++ log component. The first letter u stands for , which means micro. It can achieve the lowest **ROM<1K, RAM<0.2K** resource usage. Ulog is not only small in size, but also has very comprehensive functions. Its design concept refers to another C/C++ open source log library: EasyLogger (referred to as elog), and has made many improvements in terms of functions and performance. The main features are as follows:

- The backend of the log output is diversified and can support, for example, serial port, network, file, flash memory and other backend forms.
- The log output is designed to be thread-safe and supports asynchronous output mode.
- The logging system is highly reliable and is still available in complex environments such as interrupted ISRs and Hardfault.
- The log supports runtime/compilation time to set the output level.
- The log content supports global filtering by keyword and label.
- The APIs and log formats are compatible with linux syslog.
- Support for dumping debug data to the log in hex format.
- Compatible with `rtdbg` (RTT's early log header file) and EasyLogger's log output API.

1.4.9.1.1 Ulog Architecture

The following figure shows the ulog log component architecture diagram:

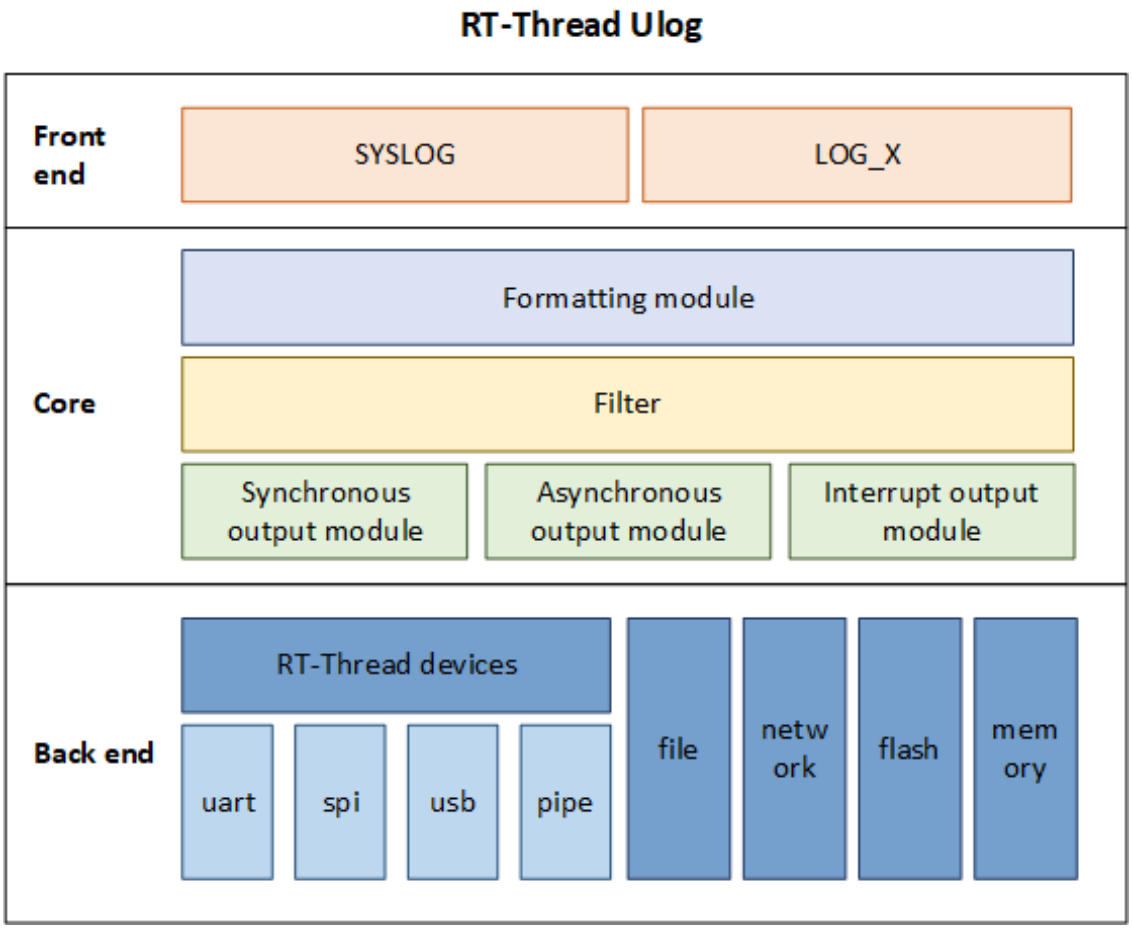


Figure 1.121 ulog architecture

- **Front end**This layer is the closest layer to the application, and provides users with two types of API interfaces, `syslog` and `LOG_X`, which are convenient for users to use in different scenarios.
- **Core**The main work of the middle core layer is to format and filter the logs passed by the upper layer, and then generate log frames, and finally output them to the lowest-end back-end devices through different output modules.
- **Back end**After receiving the log frames sent from the core layer, the logs are output to the registered log backend devices, such as files, consoles, log servers, and so on.

1.4.9.1.2 Configuration Options

The path to configure ulog using menuconfig in the ENV tool is as follows:

```
RT-Thread Components → Utilities → Enable ulog
```

The ulog configuration options are described below. In general, the default configuration is used:

```
[*] Enable ulog                  /* Enable ulog */
      The static output log level./* Select a static log output level. After the selection is completed, the
      log level lower than the set level (here specifically the log using the LOG_X API) will not be
      compiled into the ROM. */
[ ]   Enable ISR log.            /* Enable interrupted ISR log, ie log output API can also be used in ISR
      */
[*]   Enable assert check.       /* Enable assertion checks. If fter disabled, the asserted log will not be
      compiled into ROM */
(128) The log's max width.       /* The maximum length of the log. Since ulog's logging API is in units of
      rows, this length also represents the maximum length of a row of logs. */
[ ]   Enable async output mode.  /* Enable asynchronous log output mode. When this mode is turned on, the
      log will not be output to the backend immediately, but will be cached first, and then handed to the
      log output thread (for example: idle thread) to output. */
      log format  --->           /* Configure the format of the log, such as time information, color
      information, thread information, whether to support floating point, etc. */
[*]   Enable console backend.    /* Enable the console as a backend. After enabling, the log can be output
      to the console serial port. It is recommended to keep it on. */
```

```
[ ]   Enable runtime log filter.  /* Enable the runtime log filter, which is dynamic filtering. After
      enabling, the log will support dynamic filtering when the system is running, by means of tags,
      keywords, and so on. */
```

The configuration log format option description is as follows:

```
[ ] Enable float number support. It will using more thread stack.   /* Supporting floating-point variables
      (traditional rtdbg/rt_kprintf does not support floating-point logs) */
    [*] Enable color log.                   /* Colored log */
    [*] Enable time information.            /* Time information */
    [ ]   Enable timestamp format for time. /* Including timestamp */
    [*] Enable level information.           /* Level information */
    [*] Enable tag information.             /* Label Information */
    [ ] Enable thread information.          /* Thread information */
```

1.4.9.1.3 Log Level

The log level represents the importance of the log, from high to low in ulog, with the following log levels:

Level	Name	Description
LOG_LVL_ASSERT	assertion	Unhandled and fatal errors occurred, so that the system could not continue to run. These are assertion logs.
LOG_LVL_ERROR	error	The log that is output when a serious, **unrepairable** error occurs is an error level log.
LOG_LVL_WARNING	warning	These warning logs are output when there are some less important errors with **repairability**.
LOG_LVL_INFO	information	A log of important prompt information that is viewed by the upper-level user of the module, for example, initialization success, current working status, and so on. This level of log is generally **retained** during mass production.
LOG_LVL_DBG	debug	The debug log that is viewed by the developer of this module. This level of log is generally **closed** during mass production.

The log level in ulog also has the following classification:

- **Static and dynamic levels**∗∗**Classify according to whether the log can be modified during the run phase. The dynamic level that can be modified during the run phase can only be called static level in the** ∗∗**compilation phase**. Logs that are lower than the static level (here specifically the logs using the `LOG_X` API) will not be compiled into the ROM and will not be output or displayed. The dynamic level can control logs that their level are higher than or equal to the static level. Logs that are lower than the dynamic level are filtered out when ulog is running.
- ∗∗Global level and module level∗∗Classification by scope. Each file (module) can also be set to a separate log level in ulog. The global level scope is larger than the module level, that is, the module level can only control module logs higher than or equal to the global level.

As can be seen from the above classification, the output level of the log can be set in the following four aspects of ulog:

- ∗∗Global static ∗∗log levelConfigured in menuconfig, corresponding to the `ULOG_OUTPUT_LVL` macro.
- **Global Dynamics** log levelUse the `void ulog_global_filter_lvl_set(rt_uint32_t level)` function to set it.
- **Module static** log levelThe `LOG_LVL` macro is defined in the module (file), similar to the way the log tag macro `LOG_TAG` is defined.
- **Module dynamics** log levelUse the `int ulog_tag_lvl_filter_set(const char *tag, rt_uint32_t level)` function to set it.

Their scope of action is∗∗Global Static∗∗>**Global Dynamics**>**Module Static**>**Module Dynamic**.

1.4.9.1.4 Log Label

Due to the increasing log output, in order to avoid the log being outputted indiscriminately, it is necessary to use a tag to classify each log. The definition of the label is in the form of **modular**, for example: Wi-Fi components include device driver (wifi_driver), device management (wifi_mgnt) and other modules, Wi-Fi component internal module can use `wifi.driver`, `wifi.mgnt` is used as a label to perform classified output of logs.
The tag attribute of each log can also be output and displayed. At the same time, ulog can also set the output level of each tag (module) corresponding to the log. The log of the current unimportant module can be selectively closed, which not only reduces ROM resources, but also helps developers filter irrelevant logs.
See the `rt-thread\examples\ulog_example.c` ulog routine file with the `LOG_TAG` macro defined at the top of the file:

```
#define LOG_TAG     "example"     // The label corresponding to this module. When not defined, default:
      NO_TAG
#define LOG_LVL     LOG_LVL_DBG   // The log output level corresponding to this module. When not defined,
      default: debug level
#include <ulog.h>                 // this header file Must be under LOG_TAG and LOG_LVL
```

Note that the definition log tag must be above `#include <ulog.h>`, otherwise the default `NO_TAG` will be used (not recommended to define these macros in the header file).
The scope of the log tag is the current source file, and the project source code will usually be classified according to the module. Therefore, when defining a label, you can specify the module name and sub-module name as the label name. This is not only clear and intuitive when the log output is displayed, but also facilitates subsequent dynamic adjustment of the level or filtering by label.

1.4.9.2 Log Initialization

1.4.9.2.1 Initialization

```
int ulog_init(void)
```

Return	Description
>=0	Succeeded
-5	Failed, insufficient memory

This function must be called to complete ulog initialization before using ulog. This function will also be called automatically if component auto-initialization is turned on.

1.4.9.2.2 Deinitialization

```
void ulog_deinit(void)
```

This deinit release resource can be executed when ulog is no longer used.

1.4.9.3 Log Output API

Ulog mainly has two log output macro APIs, which are defined in the source code as follows:

```
#define LOG_E(...)                          ulog_e(LOG_TAG, __VA_ARGS__)
#define LOG_W(...)                          ulog_w(LOG_TAG, __VA_ARGS__)
#define LOG_I(...)                          ulog_i(LOG_TAG, __VA_ARGS__)
#define LOG_D(...)                          ulog_d(LOG_TAG, __VA_ARGS__)
#define LOG_RAW(...)                        ulog_raw(__VA_ARGS__)
#define LOG_HEX(name, width, buf, size)     ulog_hex(name, width, buf, size)
```

- The macro `LOG_X(...)`:`X` corresponds to the first letter of the different levels. The parameter `...` is the log content, and the format is the same as printf. This method is preferred because on the one hand, because its API format is simple, only one log information is entered, and the static log level filtering by module is also supported.
- The macro `ulog_x(LOG_TAG, __VA_ARGS__)`: `x` corresponds to a different level of shorthand. The parameter `LOG_TAG` is the log label, the parameter `...` is the log content, and the format is the same as printf. This API is useful when you use different tag output logs in one file.

API	Description
LOG_E(...)	Error level log
LOG_W(...)	Error level log
LOG_I(...)	Prompt level log
LOG_D(...)	Debug level log
LOG_RAW(...)	Output raw log
LOG_HEX(name, width, buf, size)	Output hexadecimal format data to the log

API such as `LOG_X` and `ulog_x` , the output are formatted logs. When you need to output logs without any format, you can use `LOG_RAW` or `ulog_raw()`. E.g:

```
LOG_RAW("\r");
ulog_raw("\033[2A");
```

You can use `LOG_HEX()` or `ulog_hex` to dump data into the log in hexadecimal hex format. The function parameters and descriptions are as follows:

Parameter	Description
tag	Log label
width	The width (number) of a line of hex content
buf	Data content to be output
size	Data size

The `hexdump` log is DEBUG level, supports runtime level filtering. The tag corresponding to the hexdump log supports tag filtering during runtime.
Ulog also provides the assertion API: `ASSERT(expression)`. When the assertion is triggered, the system will stop running, and `ulog_flush()` will be executed internally, and all log backends will execute flush. If asynchronous mode is turned on, all logs in the buffer will also be flushed. An example of the use of assertions is as follows:

```
void show_string(const char *str)
{
    ASSERT(str);
    ...
}
```

1.4.9.4 ULog Usage Example

1.4.9.4.1 Example

The following is a description of the ulog routine. Open `rt-thread\examples\ulog_example.c` and you can see that there are labels and static priorities defined at the top.

```
#define LOG_TAG              "example"
#define LOG_LVL              LOG_LVL_DBG
#include <ulog.h>
```

The `LOG_X` API is used in the `void ulog_example(void)` function, which is roughly as follows:

```
/* output different level log by LOG_X API */
LOG_D("LOG_D(%d): RT-Thread is an open source IoT operating system from China.", count);
LOG_I("LOG_I(%d): RT-Thread is an open source IoT operating system from China.", count);
LOG_W("LOG_W(%d): RT-Thread is an open source IoT operating system from China.", count);
LOG_E("LOG_E(%d): RT-Thread is an open source IoT operating system from China.", count);
```

These log output APIs support the printf format and will automatically wrap lines at the end of the log.
The following will show the effect of the ulog routine on qemu:

- Copy `rt-thread\examples\ulog_example.c` to the `rt-thread\bsp\qemu-vexpress-a9\applications` folder.
- Go to the `rt-thread\bsp\qemu-vexpress-a9` directory in Env
- After determining that the configuration of ulog has been executed before, execute the `scons` command and wait for the compilation to complete.

- Run `qemu.bat` to open RT-Thread's qemu simulator
- Enter the `ulog_example` command to see the results of the ulog routine. The effect is as follows.

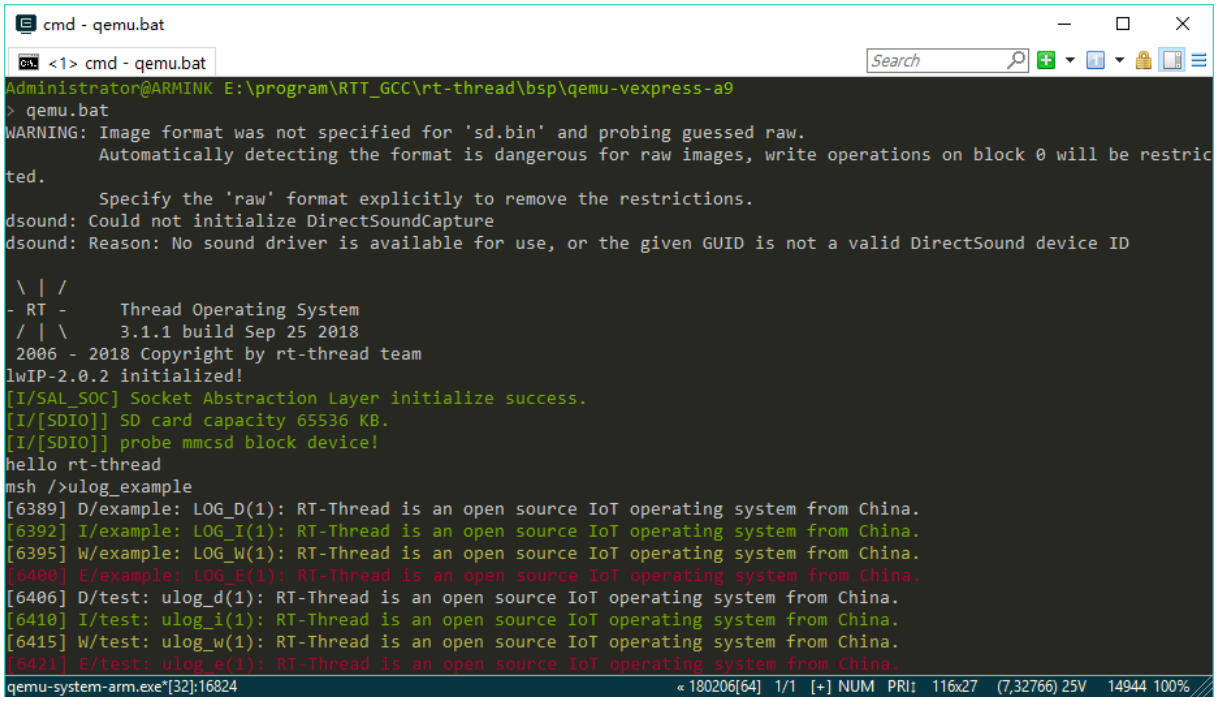


Figure 1.122 ulog routine

You can see that each log is displayed in rows, and different levels of logs have different colors. At the top of the log is the tick of the current system, with the log level and label displayed in the middle, and the specific log content at the end. These log formats and configuration instructions are also highlighted later in this article.

1.4.9.4.2 Used in Interrupt ISR

Many times you need to output a log in the interrupt ISR, but the ISR may interrupt the thread that is doing the log output. To ensure that the interrupt log and the thread log do not interfere with each other, special handling must be performed for the interrupt condition.
Ulog has integrated interrupt log function, but it is not enabled by default. Open the `Enable ISR log` option when using it. The API of the log is the same as that used in the thread, for example:

```
#define LOG_TAG              "driver.timer"
#define LOG_LVL              LOG_LVL_DBG
#include <ulog.h>

void Timer2_Handler(void)
{
    /* enter interrupt */
    rt_interrupt_enter();

    LOG_D("I'm in timer2 ISR");

    /* leave interrupt */
    rt_interrupt_leave();
}
```

Here are the different strategies for interrupt logging in ulog in synchronous mode and asynchronous mode:
In synchronous mode If the thread is interrupted when the log is being output at this time, and there is a log to be output in the interrupt, it will be directly output to the console, and output to other backends is not supported;
In asynchronous mode If the above situation occurs, the log in the interrupt will be put into the buffer first, and finally sent to the log output thread for processing together with the thread log.

1.4.9.4.3 Set the Log Format

The log format supported by ulog can be configured in menuconfig, located in `RT-Thread Components` → `Utilities` → `ulog` → `log format`. The specific configuration is as follows:

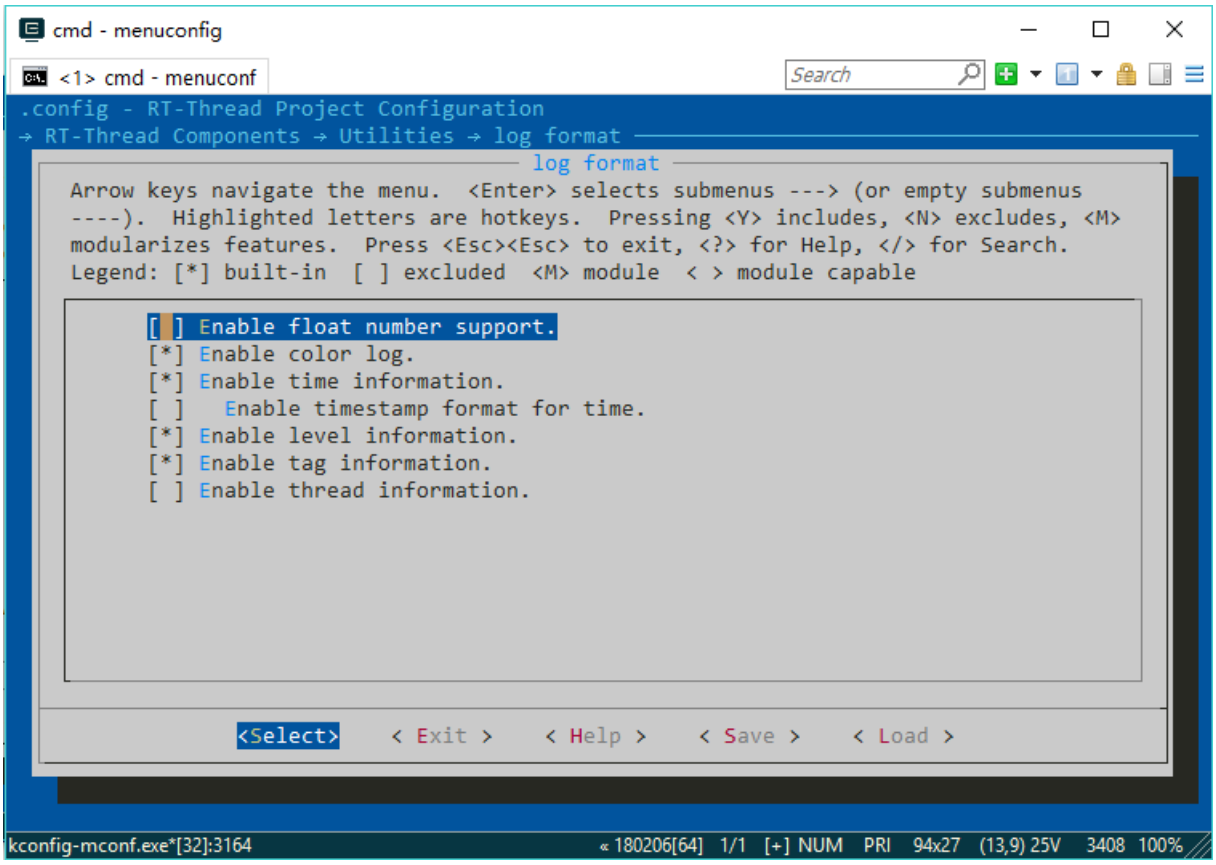


Figure 1.123 ulog format configuration

They can be configured separately: floating-point number support (traditional rtdbg/rt_kprintf does not support floating-point logs), colored logs, time information (including timestamps), level information, tag information, thread information. Below we will **select all of these options**, save and recompile and run the ulog routine again in qemu to see the actual effect:

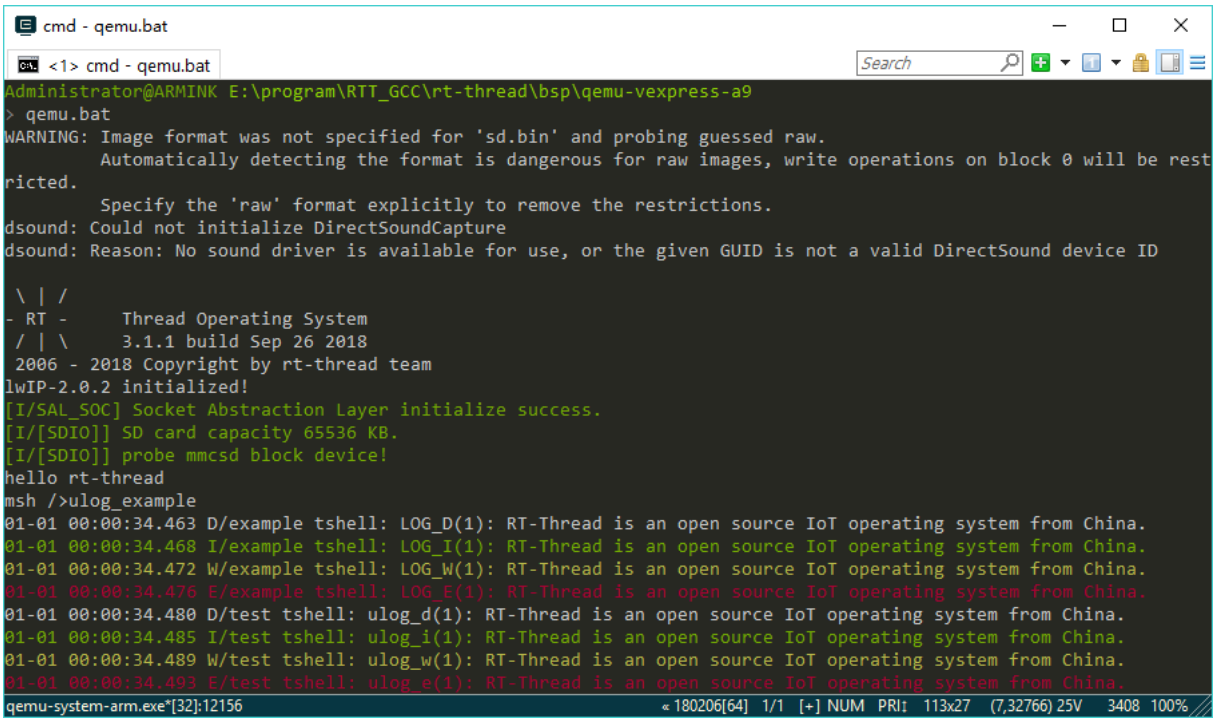


Figure 1.124 ulog routine (all formats)

It can be seen that the time information has been changed from the tick value of the system to the timestamp information compared to the first run routine, and the thread information has also been output.

1.4.9.4.4 Hexdump Output Using

Hexdump is also a more common function when logging output. hexdump can output a piece of data in hex format. The corresponding API is: `void ulog_hexdump(const char *tag, rt_size_t width, rt_uint8_t *buf, rt_size_t size)` , see below the specific use method and operation effect:

```
/* Define an array of 128 bytes in length */
uint8_t i, buf[128];
/* Fill the array with numbers */
for (i = 0; i < sizeof(buf); i++)
{
    buf[i] = i;
}
/* Dumps the data in the array in hex format with a width of 16 */
ulog_hexdump("buf_dump_test", 16, buf, sizeof(buf));
```

You can copy the above code into the ulog routine, and then look at the actual running results:

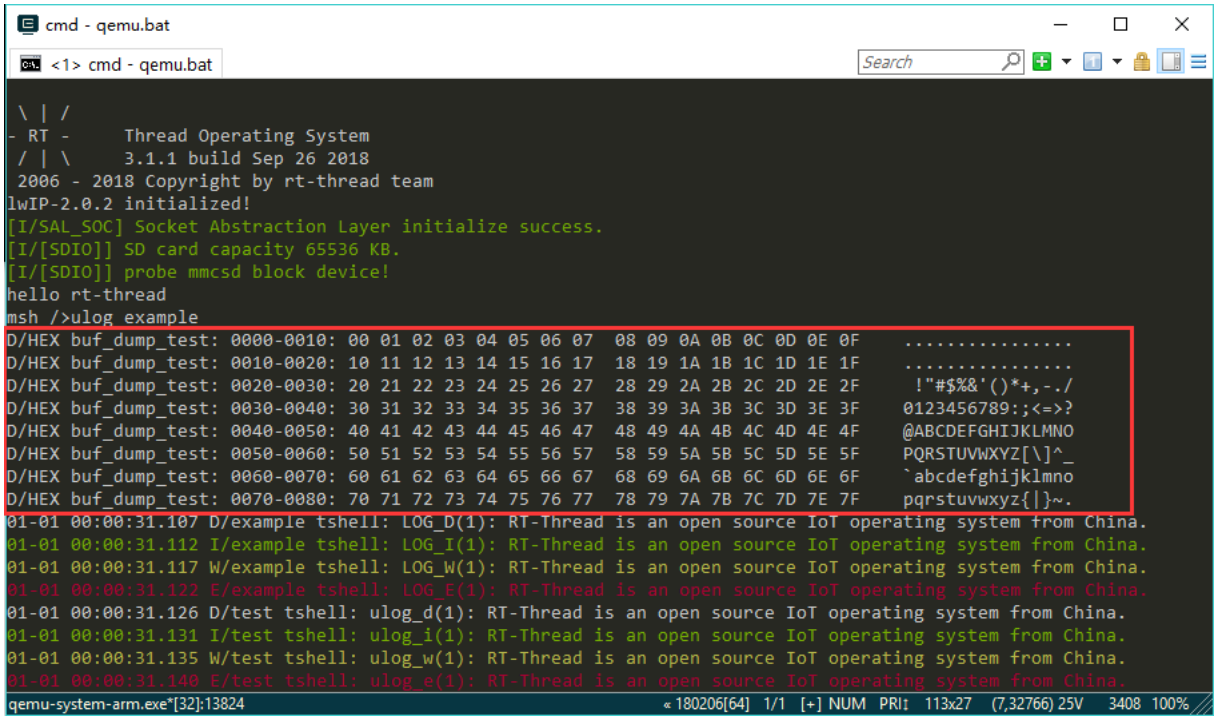


Figure 1.125 ulog routine (hexdump)

It can be seen that the middle is the hexadecimal information of the buf data, and the rightmost is the character information corresponding to each data.

1.4.9.5 Log Advanced Features

After understanding the introduction of the log in the previous section, the basic functions of ulog can be mastered. In order to let everyone better use ulog, this application note will focus on the advanced features of ulog and some experience and skills in log debugging. After learning these advanced uses, developers can also greatly improve the efficiency of log debugging.
It also introduces the advanced mode of ulog: syslog mode, which is fully compatible with the Linux syslog from the front-end API to the log format, greatly facilitating the migration of software from Linux.

1.4.9.5.1 Log Backend

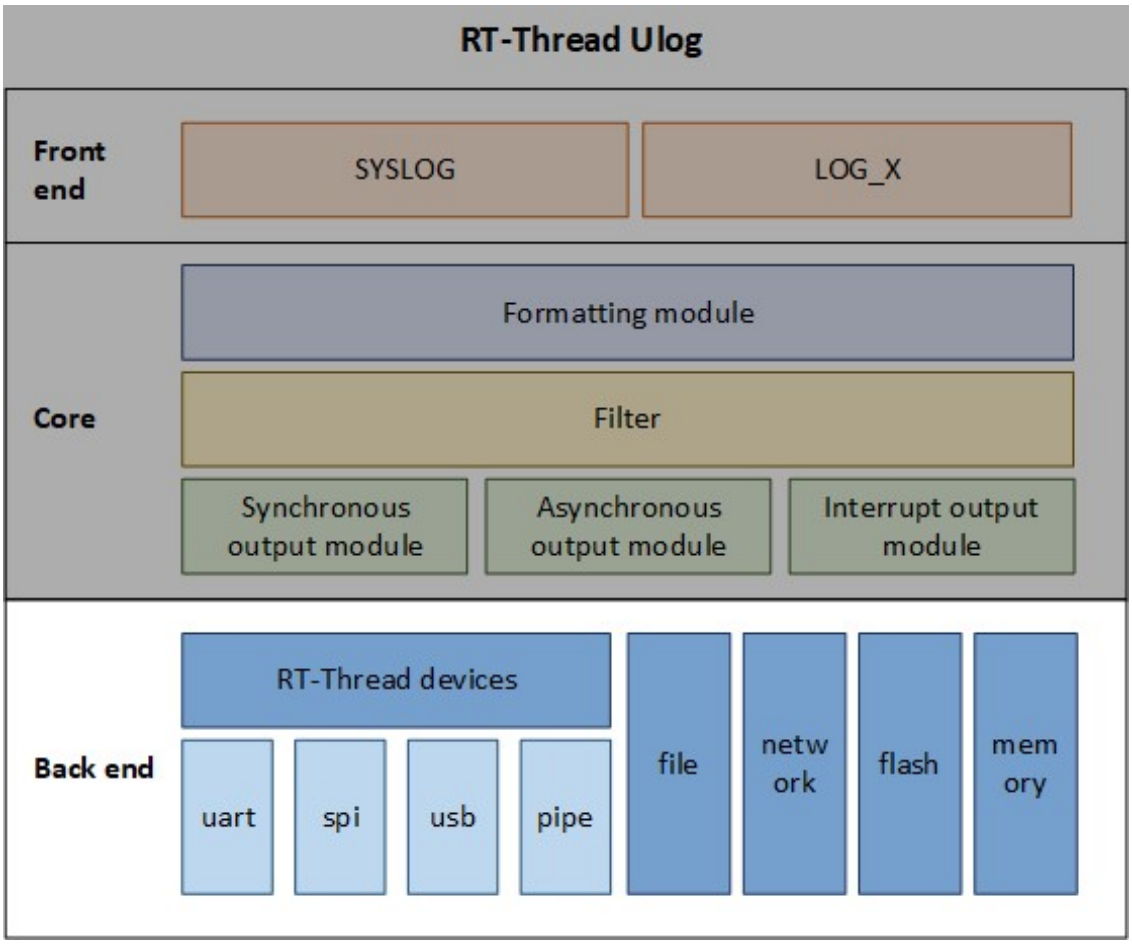


Figure 1.126 Ulog framework

Speaking of the backend, let's review the ulog's framework. As can be seen from the above figure, ulog is a design with front and back ends separated, and there is no dependence on the front and back ends. And the backends that are supported are diversified, no matter what kind of backends, as long as they are implemented, they can be registered.
Currently ulog has integrated the console backend, the traditional device that outputs `rt_kprintf` print logs. Ulog also supports the Flash backend, which seamlessly integrates with EasyFlash. See its package for details.`Click to view`Later ulog will also increase the implementation of backends such as file backends and network backends. Of course, if there are special needs, users can also implement the backend themselves.

1.4.9.5.1.1 Register Backend Device

```
rt_err_t ulog_backend_register(ulog_backend_t backend, const char *name, rt_bool_t support_color)
```

Parameter	Description
backend	Backend device handle
name	Backend device name
support_color	Whether it supports color logs
return	–
>=0	Succeeded

This function is used to register the backend device into the ulog, ensuring that the function members in the backend

device structure are set before registration.

1.4.9.5.1.2 Logout Backend Device

```
rt_err_t ulog_backend_unregister(ulog_backend_t backend);
```

Parameter	Description
backend	Backend device handle
return	–
>=0	Succeeded

This function is used to unregister a backend device that has already been registered.

1.4.9.5.1.3 Backend Implementation and Registration Examples

The console backend is taken as an example to briefly introduce the implementation method and registration method of the backend.
Open the `rt-thread/components/utilities/ulog/backend/console_be.c` file and you can see the following:

```
#include <rthw.h>
#include <ulog.h>

/* Defining console backend devices */
static struct ulog_backend console;
/* Console backend output function */
void ulog_console_backend_output(struct ulog_backend *backend, rt_uint32_t level, const char *tag, rt_bool_t
      is_raw, const char *log, size_t len)
{
    ...
    /* Output log to the console */
    ...
}
/* Console backend initialization */
int ulog_console_backend_init(void)
{
    /* Set output function */
    console.output = ulog_console_backend_output;
    /* Registration backend */
    ulog_backend_register(&console, "console", RT_TRUE);

    return 0;
}
INIT_COMPONENT_EXPORT(ulog_console_backend_init);
```

Through the above code, it can be seen that the implementation of the console backend is very simple. Here, the `output` function of the backend device is implemented, and the backend is registered in the ulog, and then the log of ulog is output to the console.
If you want to implement a more complex back-end device, you need to understand the back-end device structure, as follows:

```
struct ulog_backend
{
    char name[RT_NAME_MAX];
    rt_bool_t support_color;
    void (*init)  (struct ulog_backend *backend);
    void (*output)(struct ulog_backend *backend, rt_uint32_t level, const char *tag, rt_bool_t is_raw, const
      char *log, size_t len);
    void (*flush) (struct ulog_backend *backend);
    void (*deinit)(struct ulog_backend *backend);
    rt_slist_t list;
};
```

From the perspective of this structure, the requirements for implementing the backend device are as follows:

- `The name` and `support_color` properties can be passed in through the `ulog_backend_register()` function.
- `output` is the back-end specific output function, and all backends must implement the interface.
- `init/deinit` is optional, `init` is called at `register`, and `deinit` is called at `ulog_deinit`.

- `flush` is also optional, and some internal output cached backends need to implement this interface. For example, some file systems with RAM cache. The flush of the backend is usually called by `ulog_flush` in the case of an exception such as assertion or hardfault.

1.4.9.5.2 Asynchronous Log

In ulog, the default output mode is synchronous mode, and in many scenarios users may also need asynchronous mode. When the user calls the log output API, the log is cached in the buffer, and the thread dedicated to the log output takes out the log and outputs it to the back end.
Asynchronous mode and synchronous mode are the same for the user, there is no difference in the use of the log API, because ulog will distinguish between the underlying processing. The difference between the two works is as follows:

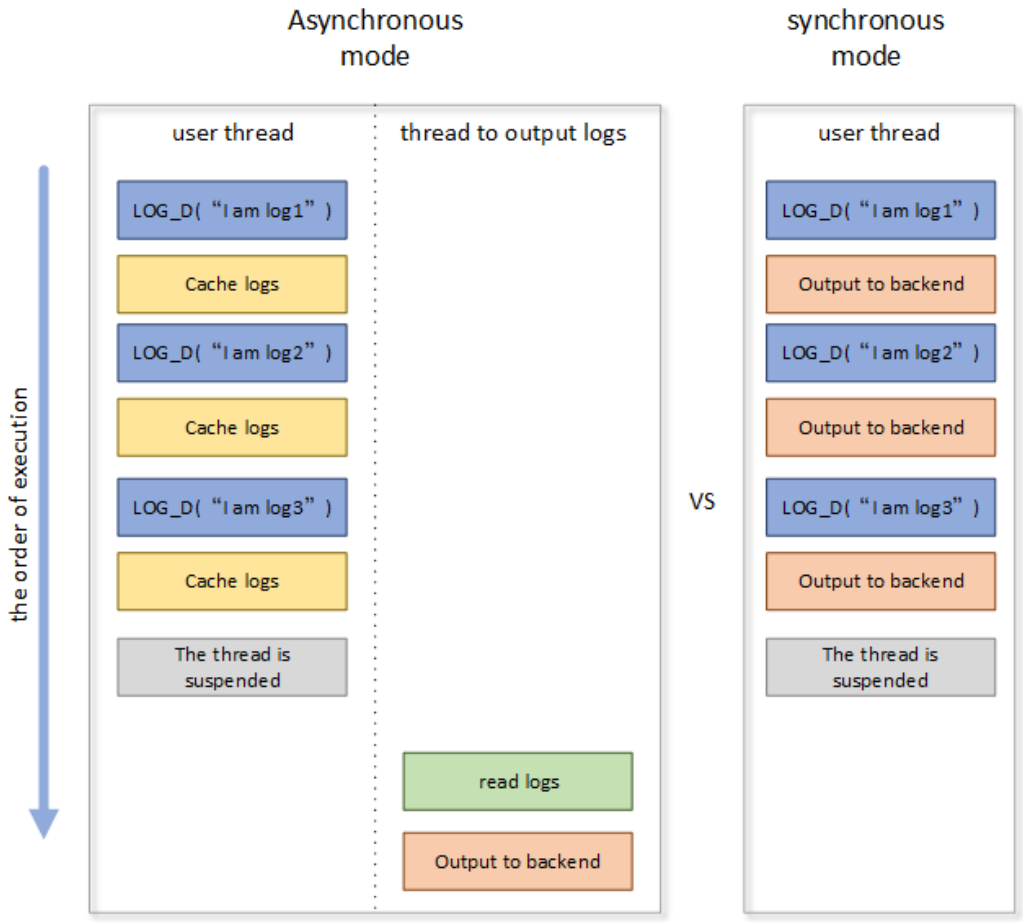


Figure 1.127 ulog asynchronous VS synchronization

The advantages and disadvantages of asynchronous mode are as follows:
∗∗Advantage∗∗

- First, the log output will not block the current thread, and some backend output rates are low, so using the synchronous output mode may affect the timing of the current thread. The asynchronous mode does not have this problem.
- Secondly, since each thread that uses the log omits the action of the backend output, the stack overhead of these threads may also be reduced, and from this perspective, the resource consumption of the entire system can also be reduced.
- Interrupt logs in synchronous mode can only be output to the console backend, while in asynchronous mode interrupt logs can be output to all backends.

∗∗Disadvantage∗∗First, the asynchronous mode requires a log buffer. Furthermore, the output of the asynchronous log needs to be completed by a special thread, such as an idle thread or a user-defined thread, which is slightly

more complicated to use. The overall sense of asynchronous mode resource occupancy will be higher than the synchronous mode.

1.4.9.5.2.1 Configuration Option

Use menuconfig in the Env tool to enter the ulog configuration options:

```
RT-Thread Components → Utilities → Enable ulog
```

The asynchronous mode related configuration options are described as follows:

```
[*]   Enable async output mode.                 /* Enable asynchronous mode */
(2048)  The async output buffer size.           /* Asynchronous buffer size, default is 2048*/
[*]     Enable async output by thread.          /* Whether to enable the asynchronous log output thread in
      ulog, the thread will wait for log notification when it runs, and then output the log to all backends.
      This option is turned on by default, and can be turned off if you want to modify it to another thread,
      such as an idle thread. */
(1024)    The async output thread stack size.   /* Asynchronous output thread stack size, default is 1024 */
(30)      The async output thread stack priority./* The priority of the asynchronous output thread, the
      default is 30*/
```

When using the idle thread output, the implementation is simple, just call `rt_thread_idle_sethook(ulog_async_output)` at the application layer, but there are some limitations.

- The idle thread stack size needs to be adjusted based on actual backend usage.
- Because thread suspend operations are not allowed inside idle threads, backends such as Flash and networking may not be available based on idle threads.

1.4.9.5.2.2 Use Example

Save the asynchronous output option configuration and copy `rt-thread\examples\ulog_example.c` to the `rt-thread\bsp\qemu-vexpress-a9\applications` folder.
Execute the `scons` command and wait for the compilation to complete. Run `qemu.bat` to open the qemu emulator for RT-Thread. Enter the `ulog_example` command to see the results of the ulog routine. The approximate effect is as follows:

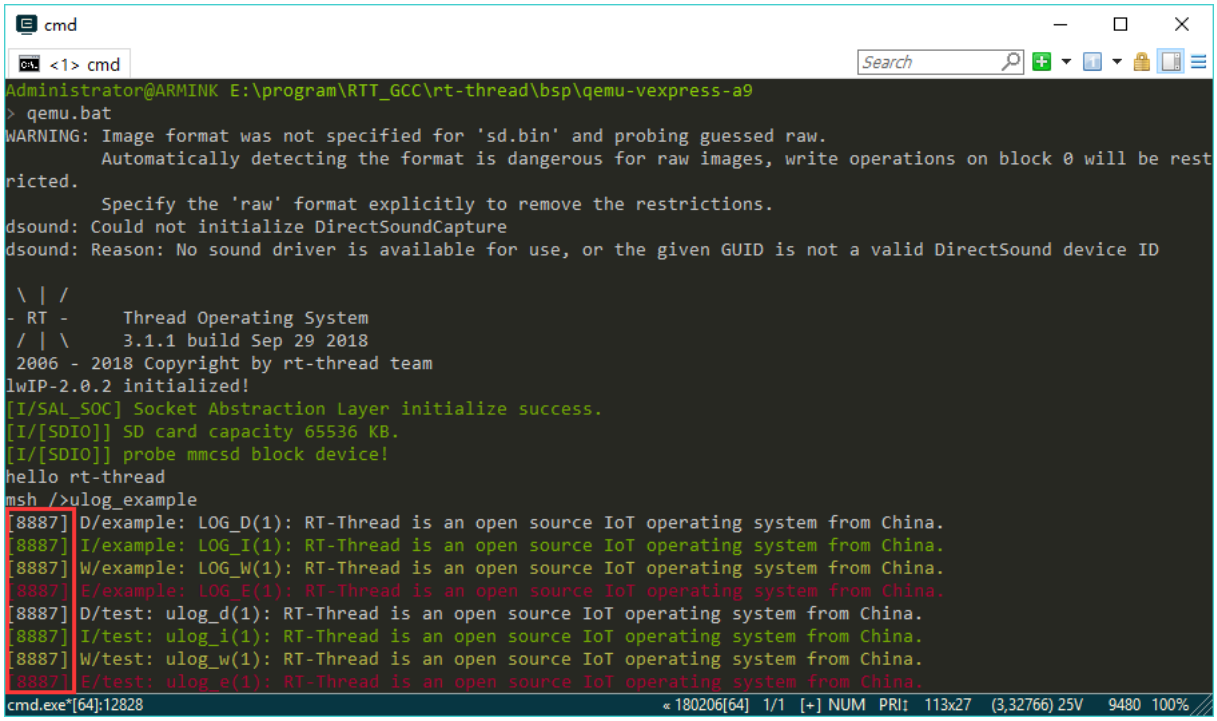


Figure 1.128 ulog asynchronous routine

If you look carefully, you can see that after the asynchronous mode is turned on, the time information of these logs that are very close in code is almost the same. However, in synchronous mode, the log is output using the user thread. Since the log output takes a certain amount of time, there is a certain interval between each log. It also fully shows that the asynchronous log output is very efficient, and it takes almost no time for the caller.

1.4.9.5.3 Log Dynamic Filter

In the previous section, some static filtering functions have been introduced. Static filtering has its advantages such as saving resources, but in many cases, users need to dynamically adjust the filtering mode of the log while the software is running. This allows the dynamic filter function of ulog to be used. To use the dynamic filter feature, turn on the `Enable runtime log filter.` option in menuconfig, which is **turned off by default**.
There are four types of dynamic filtering supported by ulog, and there are corresponding API functions and Finsh/MSH commands, which will be introduced one by one.

1.4.9.5.3.1 Filter by Module Level

```
int ulog_tag_lvl_filter_set(const char *tag, rt_uint32_t level)
```

Parameter	Description
tag	Log label
level	Set log level
return	–
>=0	Succeeded
-5	Failed, not enough memory

- Command format `ulog_tag_lvl <tag> <level>`

The **module** referred to here represents a class of log code with the same tag attributes. Sometimes it is necessary to dynamically modify the log output level of a module at runtime.
The parameter log `level` can take the following values:

Level	Name
LOG_LVL_ASSERT	Assertion
LOG_LVL_ERROR	Error
LOG_LVL_WARNING	Warning
LOG_LVL_INFO	Information
LOG_LVL_DBG	Debug
LOG_FILTER_LVL_SILENT	Silent (stop output)
LOG_FILTER_LVL_ALL	All

An example of a function call and command is as follows:

Function	Function Call	Execute an order
Close all logs of `wifi` module	`ulog_tag_lvl_filter_set("wifi", LOG_FILTER_LVL_SILENT);`	`ulog_tag_lvl wifi 0`
Open all logs of `wifi` module	`ulog_tag_lvl_filter_set("wifi", LOG_FILTER_LVL_ALL);`	`ulog_tag_lvl wifi 7`
Set the `wifi` module log level to warning	`ulog_tag_lvl_filter_set("wifi", LOG_LVL_WARNING);`	`ulog_tag_lvl wifi 4`

1.4.9.5.3.2 Global Filtering by Label

```
void ulog_global_filter_tag_set(const char *tag)
```

Parameter	Description
tag	Set filter label

- The command format: `ulog_tag [tag]`, when the tag is empty, the label filtering is canceled.

This filtering method can perform label filtering on all logs, and only the **log containing the label information** is allowed to output.
For example: there are 3 kinds of tags for `wifi.driver`, `wifi.mgnt`, `audio.driver`. When setting the filter tag to `wifi`, only the tags are `wifi.driver` and `wifi.mgnt`. `The log of` will be output. Similarly, when the filter tag is set to `driver`, only logs with tags `wifi.driver` and `audio.driver` will be output. Examples of function calls and commands for common functions are as follows:

Function	Function Call	Execute an Order
Set the filter tag to `wifi`	`ulog_global_filter_tag_set("wifi");`	`ulog_tag wifi`
Set the filter tag to `driver`	`ulog_global_filter_tag_set("driver");`	`ulog_tag driver`
Cancel label filtering	`ulog_global_filter_tag_set("");`	`ulog_tag`

1.4.9.5.3.3 Global Filtering by Level

```
void ulog_global_filter_lvl_set(rt_uint32_t level)
```

Parameter	Description
level	Set log level

- Command format: `ulog_lvl <level>` , level Refer to the following table:

Value	Description
0	assertion
3	error
4	warning
6	information
7	debug

After setting the global filter level by function or command, the log below **setting level** will stop output. Examples of function calls and commands for common functions are as follows:

Function	Function Call	Execute an order
Close all logs	`ulog_global_filter_lvl_set(LOG_FILTER_LVL_SILENT);`	`ulog_lvl 0`
Open all logs	`ulog_global_filter_lvl_set(LOG_FILTER_LVL_ALL);`	`ulog_lvl 7`
Set the log level to warning	`ulog_global_filter_lvl_set(LOG_LVL_WARNING);`	`ulog_lvl 4`

1.4.9.5.3.4 Global Filtering by Keyword

```
void ulog_global_filter_kw_set(const char *keyword)
```

Parameter	Description
keyword	Set filter keywords

- The command format: `ulog_kw [keyword]`, when the keyword is empty, the keyword filtering is canceled.

This filtering method can filter all the logs by keyword, and the log **containing the keyword information** is allowed to output. Examples of function calls and commands for common functions are as follows:

Function	Function Call	Execute an order
Set the filter keyword to `wifi`	`ulog_global_filter_kw_set("wifi");`	`ulog_kw wifi`
Clear filter keywords	`ulog_global_filter_kw_set("");`	`ulog_kw`

1.4.9.5.3.5 View Filter Information

After setting the filter parameters, if you want to view the current filter information, you can enter the `ulog_filter` command. The approximate effect is as follows:

```
msh />ulog_filter
---------------------------------------
ulog global filter:
level   : Debug
tag     : NULL
keyword : NULL
---------------------------------------
ulog tag's level filter:
wifi                  : Warning
audio.driver          : Error
msh />
```

> Filter parameters are also supported storing in Flash and also support boot autoload configuration. If you need this feature, please see the instructions for the **ulog_easyflash** package.Click to check

1.4.9.5.3.6 Use Example

Still executing in qemu BSP, first open dynamic filter in menuconfig, then save the configuration and compile and run the routine. After the log output is about **20** times, the corresponding filter code in ulog_example.c will be executed:

```
if (count == 20)
{
    /* Set the global filer level is INFO. All of DEBUG log will stop output */
    ulog_global_filter_lvl_set(LOG_LVL_INFO);
    /* Set the test tag's level filter's level is ERROR. The DEBUG, INFO, WARNING log will stop output. */
    ulog_tag_lvl_filter_set("test", LOG_LVL_ERROR);
}
...
```

At this point, the global filter level is set to the INFO level, so you can no longer see logs lower than the INFO level. At the same time, the log output level of the `test` tag is set to ERROR, and the log lower than ERROR in the `test` tag is also stopped. In each log, there is a count value of the current log output count. The effect of the comparison is as follows:

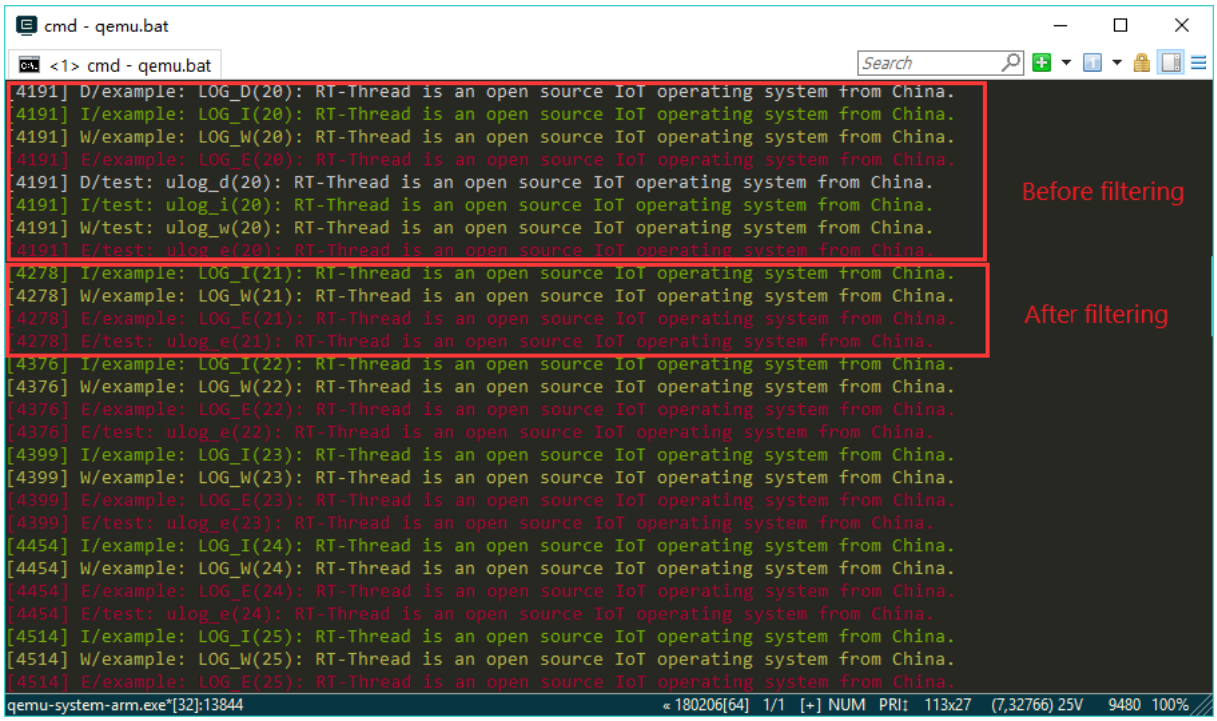


Figure 1.129 ulog filter routine 20

After the log output is about **30** times, the following filter code corresponding to ulog_example.c is executed:

```
...
else if (count == 30)
{
    /* Set the example tag's level filter's level is LOG_FILTER_LVL_SILENT, the log enter silent mode. */
    ulog_tag_lvl_filter_set("example", LOG_FILTER_LVL_SILENT);
    /* Set the test tag's level filter's level is WARNING. The DEBUG, INFO log will stop output. */
    ulog_tag_lvl_filter_set("test", LOG_LVL_WARNING);
}
...
```

At this point, the filter of the `example` module has been added, and all the logs of this module are stopped, so the module log will not be visible next. At the same time, reduce the log output level of the `test` tag to WARING. At this point, you can only see the WARING and ERROR level logs of the `test` tag. The effect is as follows:

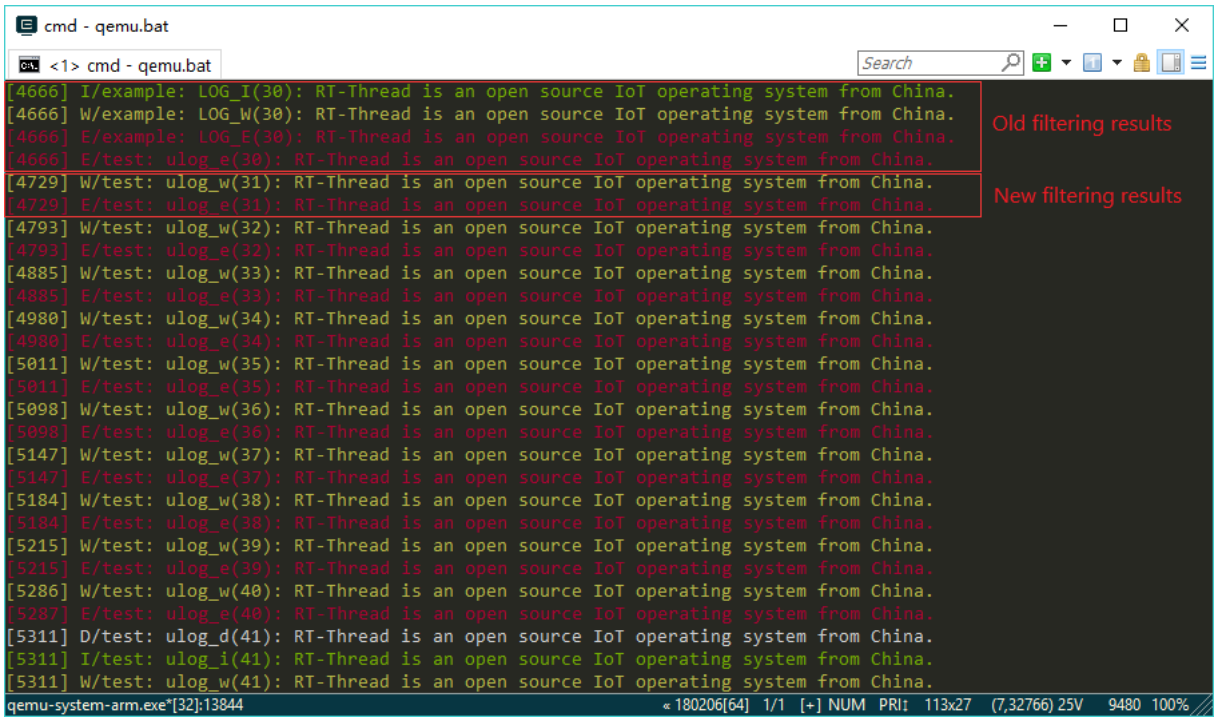


Figure 1.130 ulog filter routine 30

After the log output is about **40** times, the following filter code corresponding to ulog_example.c is executed:

```
...
else if (count == 40)
{
    /* Set the test tag's level filter's level is LOG_FILTER_LVL_ALL. All level log will resume output. */
    ulog_tag_lvl_filter_set("test", LOG_FILTER_LVL_ALL);
    /* Set the global filer level is LOG_FILTER_LVL_ALL. All level log will resume output */
    ulog_global_filter_lvl_set(LOG_FILTER_LVL_ALL);
}
```

At this time, the log output level of the `test` module is adjusted to `LOG_FILTER_LVL_ALL`, that is, the log of any level of the module is no longer filtered. At the same time, the global filter level is set to `LOG_FILTER_LVL_ALL`, so all the logs of the `test` module will resume output. The effect is as follows:

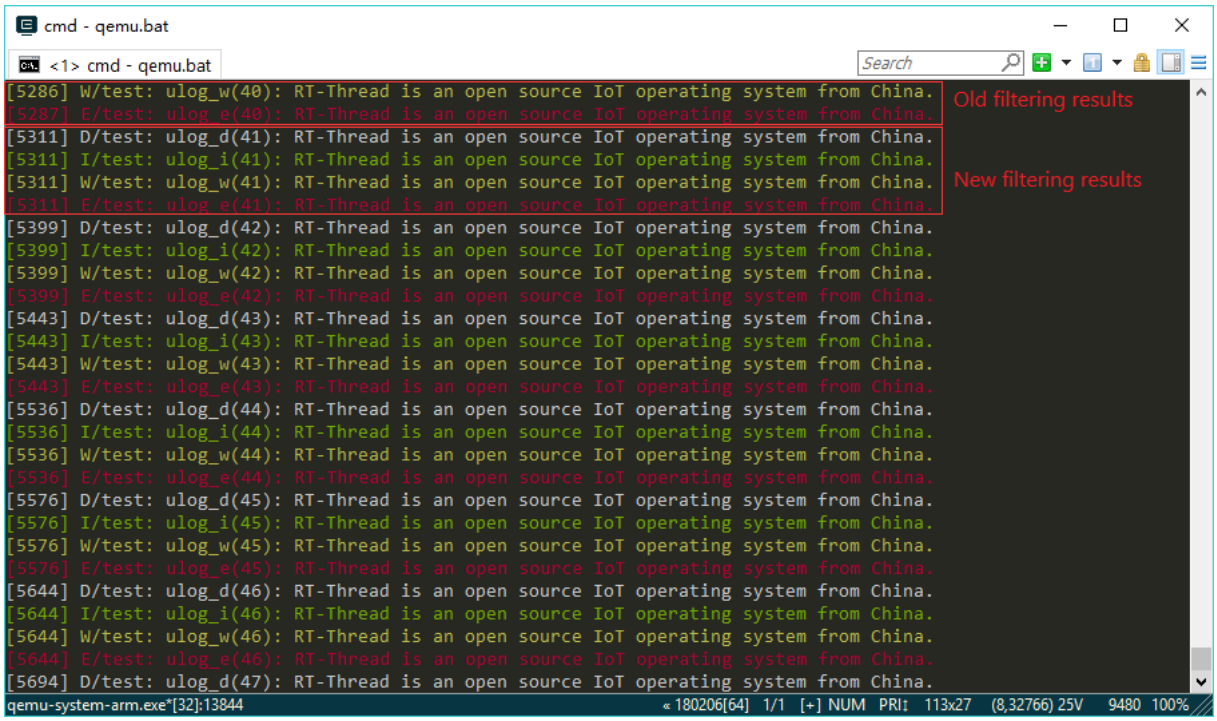


Figure 1.131 ulog filter routine 40

1.4.9.5.4 Usage when the System is Abnormal

Since the asynchronous mode of ulog has a caching mechanism, the registered backend may also have a cache inside. If there are error conditions such as hardfault and assertion in the system, but there are still logs in the cache that are not output, which may cause the log to be lost. It is impossible to find the cause of the exception.
For this scenario, ulog provides a unified log flush function: `void ulog_flush(void)`. When an exception occurs, when the exception information log is output, the function is called at the same time to ensure that the remaining logs in the cache can also be output to the back end.
The following is an example of RT-Thread assertion and CmBacktrace:

1.4.9.5.4.1 Assertion

RT-Thread assertions support assertion callbacks. We define an assertion hook function similar to the following, and then set it to the system via the `rt_assert_set_hook(rtt_user_assert_hook);` function.

```
static void rtt_user_assert_hook(const char* ex, const char* func, rt_size_t line)
{
    rt_enter_critical();

    ulog_output(LOG_LVL_ASSERT, "rtt", RT_TRUE, "(%s) has assert failed at %s:%ld.", ex, func, line);
    /* flush all log */
    ulog_flush();
    while(1);
}
```

1.4.9.5.4.2 CmBacktrace

CmBacktrace is a fault diagnosis library for ARM Cortex-M series MCUs. It also has a corresponding RT-Thread package, and the latest version of the package has been adapted for ulog. The adaptation code is located in `cmb_cfg.h`:

```
...
/* print line, must config by user */
#include <rtthread.h>
#ifndef RT_USING_ULOG
#define cmb_println(...)                rt_kprintf(__VA_ARGS__);rt_kprintf("\r\n")
#else
#include <ulog.h>
#define cmb_println(...)                ulog_e("cmb", __VA_ARGS__);ulog_flush()
#endif /* RT_USING_ULOG */
...
```

It can be seen that when ulog is enabled, each log output of CmBacktrace will use the error level, and `ulog_flush` will be executed at the same time, and the user does not need to make any modifications.

1.4.9.5.5 Syslog Mode

On Unix-like operating systems, syslog is widely used in system logs. The common backends of syslog are files and networks. The syslog logs can be recorded in local files or sent over the network to the server that receives the syslog.
Ulog provides support for the syslog mode, not only the front-end API is exactly the same as the syslog API, but the log format is also RFC compliant. However, it should be noted that after the syslog mode is enabled, the log output format of the entire ulog will be in syslog format regardless of which log output API is used.
To use the syslog configuration you need to enable the `Enable syslog format log and API.` option.

1.4.9.5.5.1 Log Format

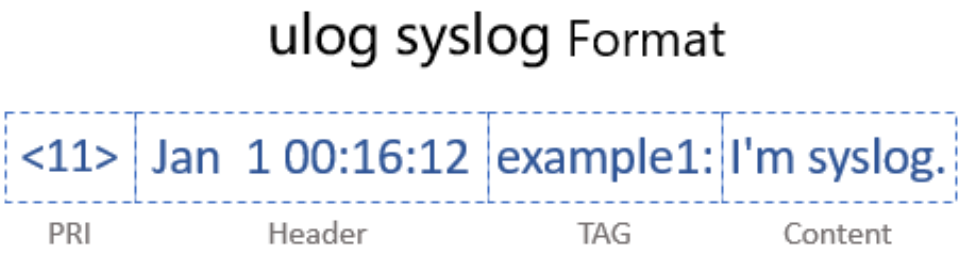


Figure 1.132 ulog syslog format

As shown in the figure above, the ulog syslog log format is divided into the following four parts:

Format	Description
PRI	The PRI part consists of a number enclosed in angle brackets. This number contains the Facility and Severity information, which is made by Facility multiplying 8 and then adding Severity. Facility and Severity are passed in by the syslog function. For details, see syslog.h.
Header	The Header part is mainly a timestamp indicating the time of the current log;
TAG	The current log label can be passed in via the `openlog` function. If not specified, `rtt` will be used as the default label.
Content	The specific content of the log

1.4.9.5.5.2 Instruction

The syslog option needs to be enabled in menuconfig before use. The main commonly used APIs are:

- Open syslog`void openlog(const char *ident, int option, int facility)`
- Output syslog log`void syslog(int priority, const char *format, ...)`

  Hint: Calling `openlog` is optional. If you do not call `openlog`, the openlog is automatically called when syslog is called for the first time.

syslog() is very simple to use. The input format is the same as the printf function. There are also syslog routines in ulog_example.c that work as follows in qemu:

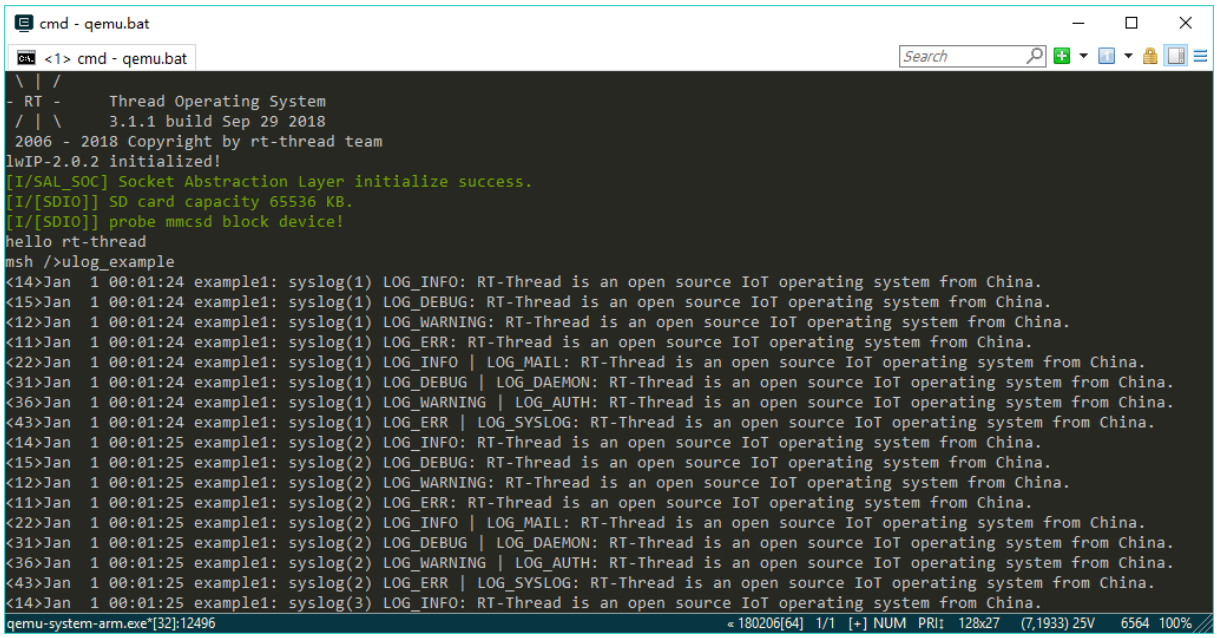


Figure 1.133 ulog syslog routine

1.4.9.5.6 Migrate from *rt_dbg.h* or elog to ulog

If the two types of log components were used in the project before, when ulog is to be used, it will involve how to make the previous code also support ulog. The following will focus on the migration process.

1.4.9.5.6.1 Migrate from rt_dbg.h

Currently rtdbg has completed **seamless docking** ulog. After ulog is turned on, the code of rtdbg in the old project can be used to complete the log output without any modification.

1.4.9.5.6.2 Migrate from Elog(EasyLogger)

If you are not sure that a source code file is running on a target platform that will use ulog, then it is recommended to add the following changes to the file:

```
#ifdef RT_USING_ULOG
#include <ulog.h>
#else
#include <elog.h>
#endif /* RT_USING_ULOG */
```

If you explicitly only use the ulog component, then simply change the header file reference from `elog.h` to `ulog.h`, and no other code needs to be changed.

1.4.9.5.7 Log Usage Tip

With the logging tool, if it is used improperly, it will also cause the log to be abused, and the log information cannot be highlighted. Here we will focus on sharing some tips on the use of the log component to make the log information more intuitive. The main concerns are:

1.4.9.5.7.1 Rational Use of Label Classification

Reasonably use the label function. For each module code before the use of the log, first clear the module, sub-module name. This also allows the logs to be categorized at the very beginning and ready for later log filtering.

1.4.9.5.7.2 Rational Use of Log Levels

When you first use the log library, you will often encounter warnings and errors. The logs cannot be distinguished. The information cannot be distinguished from the debug logs, which makes the log level selection inappropriate.

Some important logs may not be visible, and unimportant logs are full of problems. Therefore, be sure to read the log level section carefully before using it. For each level, there are clear standards.

1.4.9.5.7.3 Avoid Repetitive and Redundant Logs

In some cases, repeated or cyclic execution of code occurs, and the same, similar log problems are output multiple times. Such a log not only takes up a lot of system resources, but also affects the developer's positioning of the problem. Therefore, in this case, it is recommended to add special processing for repetitive logs, such as: let the upper layer output some business-related logs, the bottom layer only returns the specific result status; the same log at the same time point, can increase to deal with the re-processing, only output once and so on when the error state has not changed.

1.4.9.5.7.4 Open More Log Formats

The timestamp and thread information are not turned on in the default log format of ulog. These two log messages are useful on RTOS. They can help developers to understand the running time and time difference of each log, and clearly see which thread is executing the current code. So if conditions permit, it is still recommended to open.

1.4.9.5.7.5 Close Unimportant Logs

Ulog provides log switch and filtering functions in various dimensions, which can completely control the refinement. Therefore, if you debug a function module, you can properly close the log output of other unrelated modules, so that you can focus on the current debugging on the module.

1.4.9.6 Common Problems

1.4.9.6.1 Q: The log code has been executed, but there is no output.

A: Refer to the Log Levels section for the log level classification and check the log filtering parameters. There is also the possibility of accidentally closing the console backend and re-opening `Enable console backend`.

1.4.9.6.2 Q: After ulog is turned on, the system crashes, for example: thread stack overflow.

A: Ulog will occupy more part of the thread stack space than the previous rtdbg or `rt_kprintf` printout function. If floating-point printing support is enabled, it is recommended because it uses the internal memory of libc with a large amount of `vsnprintf`. Reserve more than 250 bytes. If the timestamp function is turned on, the stack recommends a reserve of 100 bytes.

1.4.9.6.3 Q: The end of the log content is missing.

A: This is because the log content exceeds the maximum width of the set log. Check the `The log's max width` option and increase it to the appropriate size.

1.4.9.6.4 Q: After turning on the timestamp, why can't I see the millisecond time?

A: This is because ulog currently only supports millisecond timestamps when the software emulation RTC state is turned on. To display, just turn on the RT-Thread software to simulate the RTC function.

1.4.9.6.5 Q: Define LOG_TAG and LOG_LVL before each include ulog header file, can it be simplified?

A: If `LOG_TAG` is not defined, the `NO_TAG` tag will be used by default, so the output log will be easily misunderstood. Therefore the tag macro is not recommended to be omitted.
If `LOG_LVL` is not defined, the debug level is used by default. If the module is in the development stage, this process can be omitted, but if the module code is stable, it is recommended to define the macro and modify the level to information level.

1.4.9.6.6 Q: Warning when runningWarning: There is no enough buffer for saving async log, please increase the ULOG_ASYNC_OUTPUT_BUF_SIZE option

A: When this prompt is encountered, it indicates that the buffer in the asynchronous mode has overflowed, which will cause some of the log to be lost. Increasing the ULOG_ASYNC_OUTPUT_BUF_SIZE option can solve the problem.

1.4.9.6.7 Q: Compile time promptThe idle thread stack size must more than 384 when using async output by idle (ULOG_ASYNC_OUTPUT_BY_IDLE)

A: When using an idle thread as the output thread, the stack size of the idle thread needs to be increased, depending on the specific backend device. For example, when the console being a backend, the idle thread must be at least 384 bytes.

1.4.10 Power Management: PM

The purpose of low power management of embedded system is to reduce system energy consumption as much as possible to prolong the standby time of equipment on the premise of satisfying users'performance requirements. The contradiction between high performance and limited battery energy is most prominent in embedded systems. The combination of hardware low power design and software low power management has become an effective means to solve the contradiction. Nowadays, all kinds of MCUs provide management interfaces in low power consumption more or less. For example, adjusting the frequency of the main control clock, changing the working voltage, adjusting or even closing the bus frequency, closing the working clock of peripheral equipment, etc. With the support of hardware, reasonable software design becomes the key to energy saving. Generally, low power management can be divided into three categories:

- Processor Power Management
  - The main ways of realization are dynamic management of CPU frequency and adjustment of working mode when the system is idle.
- Power management of equipment
  - The main way to achieve this is to shut down individual idle devices
- Power Management of System Platform
  - The main way to realize it is to specific customization of infrequent devices on specific system platforms.

With the rise of the Internet of Things (IoT), the demand for power consumption of products becomes more and more intense. Sensor nodes as data acquisition usually need to work long-term when the battery is powered, and SOC connected to the network also needs fast response function and low power consumption.
In the initial stage of product development, the first consideration is to complete the development of product functions as soon as possible. After the function of the product is gradually improved, it is necessary to add the power management (PM) function. To meet this need of IoT, RT-Thread provides power management components. The idea of power management components is to be as transparent as possible, making it easier for products to add low power functions.

1.4.10.1 Introduction of PM Components

RT-Thread's PM components adopt a layered design idea, separating architecture and chip-related parts, and extracting common parts as the core. While providing a common interface to the upper layer, it also makes it easier for the bottom driver to adapt components.

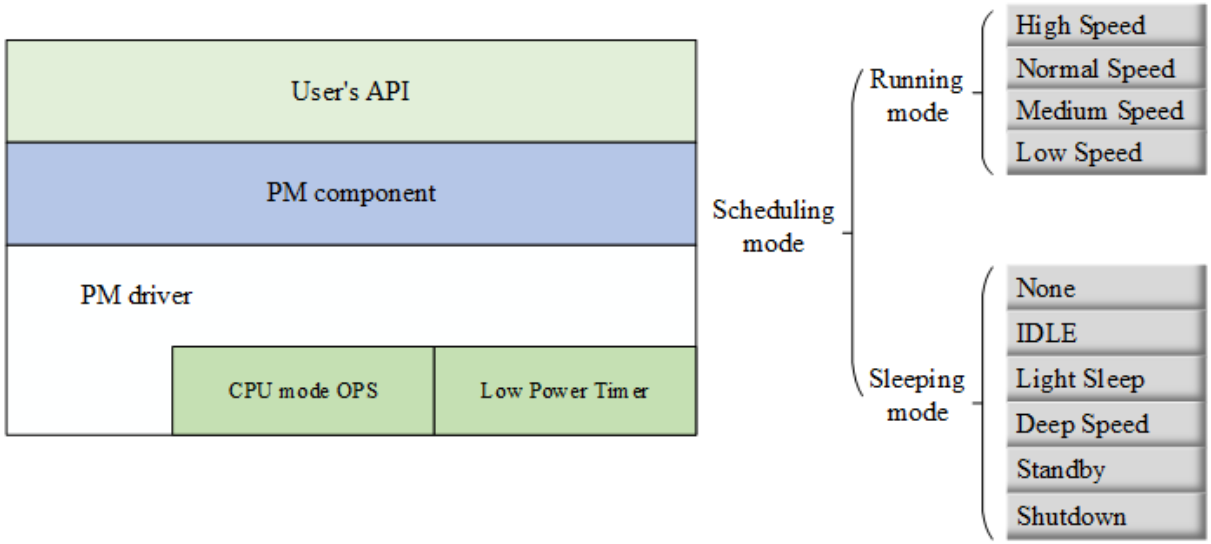


Figure 1.134 PM Component Overview

1.4.10.1.1 Main Features

The main features of RT-Thread PM components are as follows:

- It manages power consumption based on mode, dynamically adjusts working mode in idle time, and supports multiple levels of sleeping.
- Transparent to applications, components automatically complete power management at the bottom layer.
- It supports dynamic frequency conversion in running mode and updates the frequency configuration of equipment automatically according to the mode to ensure normal operation in different running modes.
- Support equipment power management, automatically manage device suspending and resuming according to the mode, ensure correct suspending and resuming in different sleep mode.
- Optional sleep time compensation is supported to make OS Tick dependent applications transparent.
- Provide the device interface to the upper layer. If the devfs component is opened, it can also be accessed through the file system interface.

1.4.10.1.2 Working Principle

The essence of low power consumption is that when the system is idle, the CPU stops working, interrupts or resumes working after the event wakes up. In RTOS, there is usually an IDLE task, which has the lowest priority and remains ready. When the high priority task is not ready, the OS executes the IDLE task. Generally, the CPU executes empty instructions in IDLE tasks without low power processing. The power management component of RT-Thread can effectively reduce the power consumption of the system by managing CPU, clock and equipment in IDLE tasks.

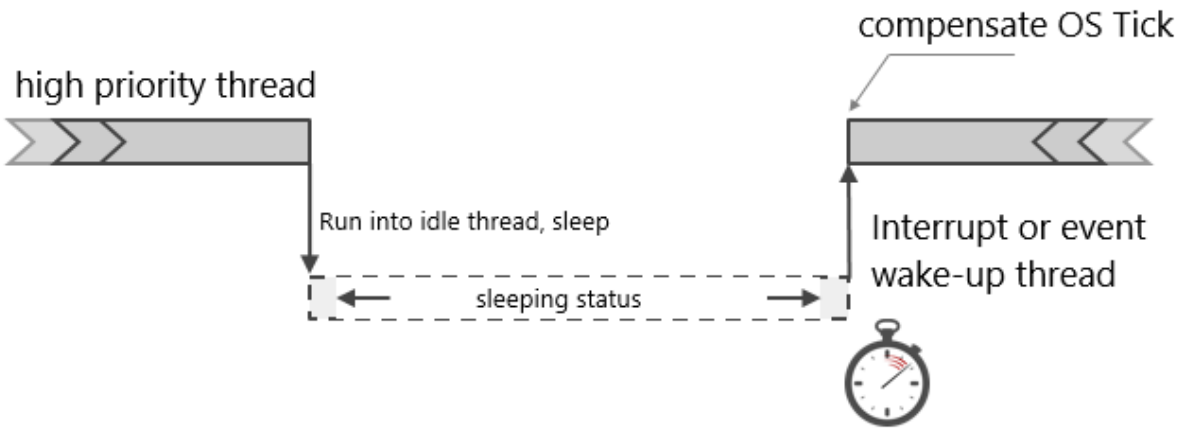


Figure 1.135 PM

As shown in the figure above, when the high priority thread runs out or is suspended, the system enters the IDLE thread . After the IDLE thread is executed, it will determine whether the system can go to sleep (to save power). If the system goes to sleep, Some hardware modules will be shut down depending on the chip condition, and OS Tick is also very likely to enter a pause state. At this time, the power management framework will calculate the next timeout point according to the system timer situation, and set a low-power timer, so that the device can wake up at that point, and carry out follow-up work. When the system is awakened (low power timer interrupt or other wake-up interrupt source), the system also needs to know how long it sleeps, and compensate for OS Tick, so that the OS tick value of the system is adjusted to a correct value.

1.4.10.2 PM Framework

In RT-Thrad PM components, peripherals or applications vote on the required power consumption mode by voting mechanism. When the system is idle, the appropriate power consumption mode is decided according to the voting number, and the abstract interface is called to control the chip to enter a low power consumption state, so as to reduce the power consumption of the system. When no vote is taken, it is entered in the default mode (usually idle mode). Unlike applications, some peripherals may perform specific operations when they enter a low-power state and take measures to recover when they exit a low-power state, which can be achieved by registering PM devices. By registering PM devices, `suspend` callbacks of registered devices will be triggered before entering a low power state. Developers can perform their own operations in the callbacks. Similarly, `resume` callbacks will be triggered when exiting from a low power state.

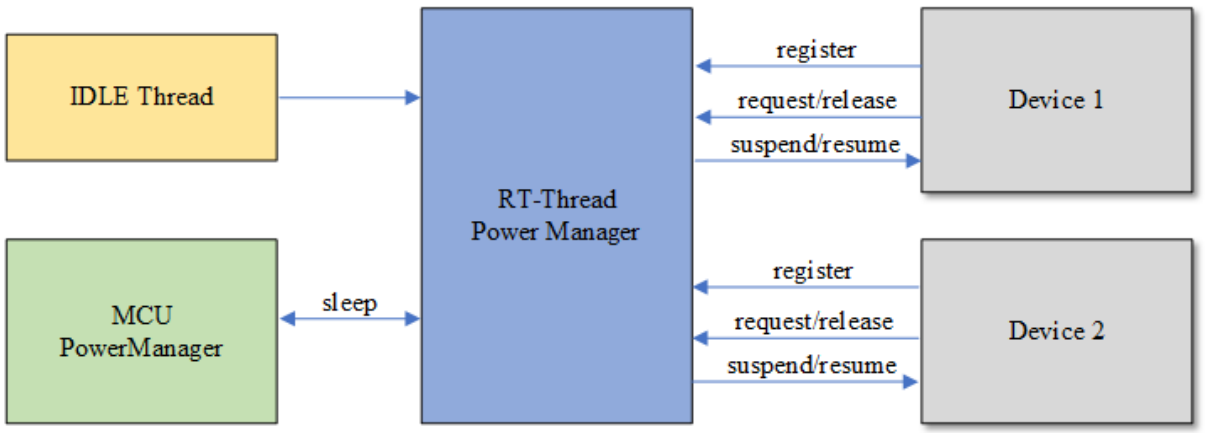


Figure 1.136 PM Framework

1.4.10.3 Low Power State and Mode

The RT-Thread PM component divides the system into two states: RUN(running state) and Sleep(sleeping state). Running state controls the frequency of CPU, which is suitable for frequency conversion scenarios. Sleeping state realizes sleeping CPU according to SOC characteristics to reduce power consumption. The two states are controlled independently using different API interfaces.

- **Sleeping state**

Sleeping state is also a low power state in the usual sense. By closing the peripherals and executing the SOC power management interface, the system power consumption is reduced.
The state of sleeping is divided into six patterns, which take the form of pyramids. As the mode increases, the power consumption decreases step by step. The following is the definition of the mode in the sleeping state. Developers can implement the corresponding mode according to the specific SOC, but they need to follow the characteristics of power consumption decreasing step by step.

Patterns	Level	Description
PM_SLEEP_MODE_NONE	0	The system is active without any power reduction.
PM_SLEEP_MODE_IDLE	1	The idle mode, which stops CPU and part of the clock when the system is idle. Any event or interrupt can wake up system.
PM_SLEEP_MODE_LIGHT	2	Light sleep modes, CPU stops, most clocks and peripherals stop, and time compensation is required after wake-up.
PM_SLEEP_MODE_DEEP	3	Deep sleep mode, CPU stops, only a few low power peripheral work, can be awakened by special interrupts
PM_SLEEP_MODE_STANDBY	4	Standby mode, CPU stops, device context loss (can be saved to special peripherals), usually reset after wake-up
PM_SLEEP_MODE_SHUTDOWN	5	Shutdown mode, lower power consumption than Standby mode, context is usually irrecoverable, reset after wake-up

>Note: The implementation of power management varies from chip to chip. The above description only gives some recommended scenarios, not all of which need to be implemented. Developers can choose several of them to implement according to their own situation, but they need to follow the principle of higher level and lower power consumption.

- **Running state**

Running state is usually used to change the running frequency of CPU, independent of sleep mode. Current operation status is divided into four levels: high-speed, normal-speed, medium-speed and low-speed, as follows:

Patterns	Description
PM_RUN_MODE_HIGH_SPEED	High-speed mode, suitable for some over-frequency scenarios
PM_RUN_MODE_NORMAL_SPEED	Normal-speed mode, which is the default running state
PM_RUN_MODE_MEDIUM_SPEED	Medium-speed mode, reduce CPU running speed, thereby reducing power consumption
PM_RUN_MODE_LOW_SPEED	Low-speed mode, CPU frequency further reduced

1.4.10.3.1 Request and release of patterns

In PM components, upper applications can actively participate in power management by requesting and releasing sleep modes. Applications can request different sleep modes according to scenarios and release them after processing. As long as any application or device requests higher-level power mode, it will not switch to a lower mode. Therefore, the requests and releases of sleep mode usually occur in pairs and can be used to protect a certain stage, such as the peripheral DMA transmission process.

1.4.10.3.2 Device Sensitive to Mode Changes

In PM components, switching to a new mode of operation may lead to changes in CPU frequency. If peripherals and CPUs share a part of the clock, the peripheral clock will be affected. When entering the new sleep mode, most clock sources will be stopped. If the peripheral does not support the freezing function of sleep, then the peripheral clock needs to be reconfigured when waking up from sleep. So PM components support PM mode sensitive PM devices. It enables the device to work normally when switching to a new operation mode or a new sleep mode. This function requires the underlying driver to implement the relevant interface and register as a device sensitive to mode changes.

1.4.10.4 The calling process

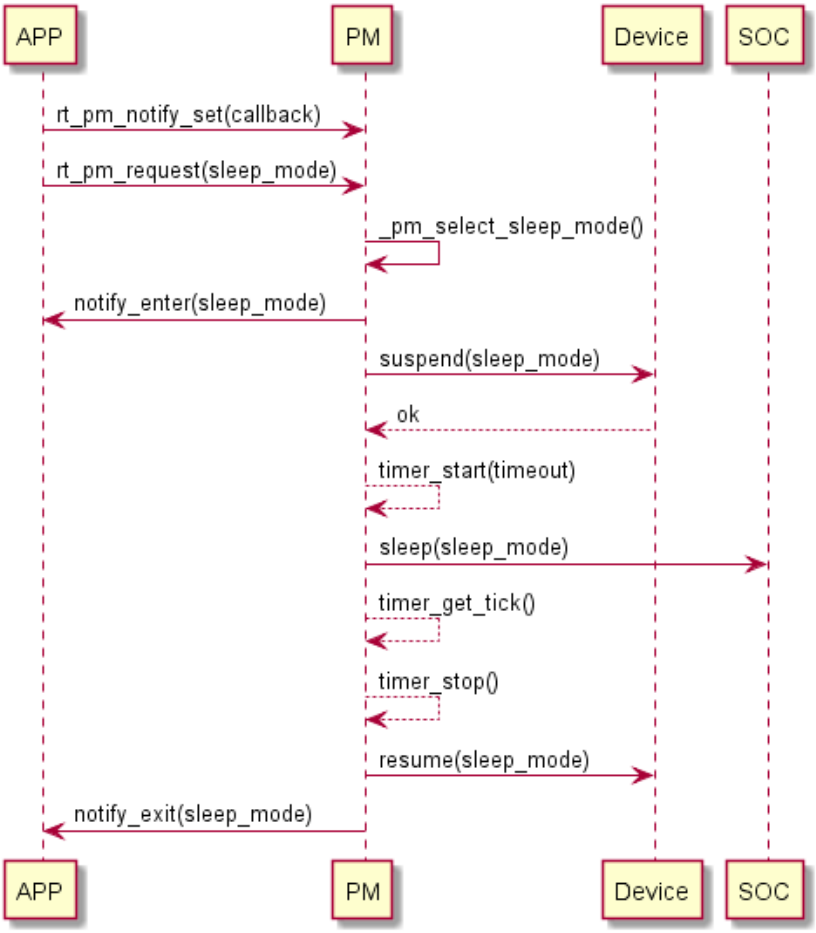


Figure 1.137 PM Sequence

Firstly, the application layer sets the callback function of entering and exiting the dormancy state, and then calls `rt_pm_request` to request the sleeping mode to trigger the sleeping operation. The PM component checks the number of sleeping modes when the system is idle, and gives the recommended mode according to the number of votes. Then the PM component calls `notfiy` to inform the application that it is going to enter the sleep mode, and then suspends the registered PM device and executes the sleep mode implemented by SOC after returning to OK. The system enters the sleep state (if the enabling time is compensated, the low-power timer will be started before the sleep). At this point, the CPU stops working and waits for an event or interrupt to wake up. When the system is awakened, because the global interruption is closed, the system continues to execute from there, gets the sleep time to compensate the OS tick of the system, wakes up the device in turn, and notifies the application to exit from the sleep mode. Such a cycle is completed, exits, and waits for the system to be idle next time.

1.4.10.5 Introduction to APIs

1.4.10.5.1 Request Sleep Mode

```
void rt_pm_request(uint8_t sleep_mode);
```

Parameter	Mode
sleep_mode	Request Sleep mode level

Sleep_mode takes the following enumeration values:

```
enum
{
    /* sleep modes */
    PM_SLEEP_MODE_NONE = 0,     /* active state */
    PM_SLEEP_MODE_IDLE,         /* idle mode (default) */
    PM_SLEEP_MODE_LIGHT,        /* Light Sleep Mode */
    PM_SLEEP_MODE_DEEP,         /* Deep Sleep Mode */
    PM_SLEEP_MODE_STANDBY,      /* Standby Mode */
    PM_SLEEP_MODE_SHUTDOWN,     /* Shutdowm Mode */
    PM_SLEEP_MODE_MAX,
};
```

Calling this function adds the corresponding pattern count to 1 and locks the pattern. At this point, if a lower level of power mode is requested, it will not be accessible. Only after releasing (unlocking) the previously requested mode, the system can enter a lower level of power mode; requests to higher power mode are not affected by this. This function needs to be used in conjunction with `rt_pm_release` to protect a certain stage or process.

1.4.10.5.2 Release Sleep Mode

```
void rt_pm_release(uint8_t sleep_mode);
```

Parameter	Mode
sleep_mode	Sleep mode level

Calling this function decreases the corresponding pattern count by 1, and releases the previously requested pattern in conjunction with `rt_pm_request`.

1.4.10.5.3 Setting up Running Mode

```
int rt_pm_run_enter(uint8_t run_mode);
```

Parameter	Mode
run_mode	Running mode level

`run_mode` can take the following enumeration values:

```
enum
{
    /* run modes*/
    PM_RUN_MODE_HIGH_SPEED = 0,     /* high speed */
    PM_RUN_MODE_NORMAL_SPEED,       /* Normal speed(default) */
    PM_RUN_MODE_MEDIUM_SPEED,       /* Medium speed */
    PM_RUN_MODE_LOW_SPEED,          /* low speed */
    PM_RUN_MODE_MAX,
};
```

Calling this function changes the CPU's running frequency, thereby reducing the power consumption at runtime. This function only provides levels, and the specific CPU frequency should depend on the actual situation during the migration phase.

1.4.10.5.4 Setting up callback notifications for entering/exiting sleep mode

```
void rt_pm_notify_set(void (*notify)(uint8_t event, uint8_t mode, void *data), void *data);
```

Parameter	Mode
notify	callback function
data	Private data

Evet is the following two enumeration values, identifying entry/exit sleep mode respectively.

```
enum
{
    RT_PM_ENTER_SLEEP = 0,     /* entry sleep mode */
    RT_PM_EXIT_SLEEP,          /* exit sleep mode */
};
```

1.4.10.6 Instruction for Use

1.4.10.6.1 Setting Low Power Level

If the system needs to enter a specified level of low power consumption, it can be achieved by calling rt_pm_request. For example, into deep sleep mode:

```
/* Request for Deep Sleep Mode */
rt_pm_request(PM_SLEEP_MODE_DEEP);
```

> Note: If higher power consumption modes, such as Light Mode or Idle Mode, are requested elsewhere in the program, then the corresponding mode needs to be released before the deep sleep mode can be entered.

1.4.10.6.2 Protect a stage or process

In special cases, for example, the system is not allowed to enter a lower power mode at a certain stage, which can be protected by rt_pm_request and rt_pm_release. For example, deep sleep mode (which may cause peripherals to stop working) is not allowed during I2C reading data, so the following processing can be done:

```
/* Request light sleep mode (I2C peripheral works normally in this mode) */
rt_pm_request(PM_SLEEP_MODE_LIGHT);

/* Reading Data Procedure */

/* Release this model */
rt_pm_release(PM_SLEEP_MODE_LIGHT);
```

1.4.10.6.3 Changing CPU Running Frequency

Reducing the running frequency can effectively reduce the power consumption of the system, and the running frequency of the CPU can be changed through the `rt_pm_run_enter` interface. Generally speaking, frequency reduction means that CPU performance decreases and processing speed decreases, which may lead to the increase of task execution time and need to be weighed reasonably.

```
/* Enter Medium Speed Mode */
rt_pm_run_enter(PM_RUN_MODE_MEDIUM_SPEED);
```

1.4.10.7 Migration instructions

Low power management is a very meticulous task. Developers need not only to fully understand the power management of the chip itself, but also to be familiar with the peripheral circuit of the board and deal with it one by one when they enter the low power state, so as to avoid leakage of the peripheral circuit and pull up the overall power consumption.
RT-Thread PM components Abstract each part and provide different OPS interfaces for developers to adapt. The following points need to be paid attention to when they migration:

```
struct rt_pm_ops
{
    /* Sleep interface for adapting chip-related low power characteristics */
    void (*sleep)(struct rt_pm *pm, uint8_t mode);
    /* Run Interface for Frequency Conversion and Voltage Conversion in Running Mode */
    void (*run)(struct rt_pm *pm, uint8_t mode);
    /* The following three interfaces are used to start a low-power timer after cardiac arrest to compensate
      for the OS tick */
    void (*timer_start)(struct rt_pm *pm, rt_uint32_t timeout);
    void (*timer_stop)(struct rt_pm *pm);
    rt_tick_t (*timer_get_tick)(struct rt_pm *pm);
```

```
};

/* The OPS is used to manage the power consumption of peripherals */
struct rt_device_pm_ops
{
    /* Suspending the peripheral before entering sleeping mode and return to non-zero to indicate that the
      peripheral is not ready and cannot enter */
    int (*suspend)(const struct rt_device *device, uint8_t mode);
    /* Resume peripherals after exiting from sleep mode */
    void (*resume)(const struct rt_device *device, uint8_t mode);
    /* When the mode changes, notify peripheral processing in the running state */
    int (*frequency_change)(const struct rt_device *device, uint8_t mode);
};

/* Register a PM device */
void rt_pm_device_register(struct rt_device *device, const struct rt_device_pm_ops *ops);
```

1.4.10.7.1 Power Consumption Characteristics of Chips

```
void (*sleep)(struct rt_pm *pm, uint8_t mode);
```

Each chip has different definitions and management of low power mode. PM component abstracts chip-related characteristics into sleep interface. The interface adapts to low power management related to chips. When entering different `sleep` modes, some hardware-related configurations, storage and other related processing are needed.

1.4.10.7.2 Time Compensation for Sleep Mode

```
void (*timer_start)(struct rt_pm *pm, rt_uint32_t timeout);
void (*timer_stop)(struct rt_pm *pm);
rt_tick_t (*timer_get_tick)(struct rt_pm *pm);
```

In some sleep modes (Light Sleep or Deep Sleep), the OS Tick timer in the kernel may be stopped. At this time, it is necessary to start a timer to measure the time of sleep and compensate for the OS Tick after waking up. Time compensated timer must still be able to work properly in this mode and wake up the system, otherwise it is meaningless!

- **timer_start**: Start a low-power timer, and the input parameter is the latest next task readiness time.
- **timer_get_tick**: Get the sleep time when the system is awakened;
- **timer_stop**: Used to stop low-power timers after system wake-up.

Note: Time compensation for sleep mode needs to be turned on at the initialization stage by setting the bit control of the corresponding mode of timer_mask. For example, if time compensation in Deep Sleep mode needs to be turned on, then after implementing the timer-related OPS interface, the corresponding bit is set at initialization:

```
rt_uint8_t timer_mask = 0;

/* Set the bit corresponding to Deep Sleep mode to enable sleep time compensation */
timer_mask = 1UL « PM_SLEEP_MODE_DEEP;

/* initialize system pm module */
rt_system_pm_init(&_ops, timer_mask, RT_NULL);
```

1.4.10.7.3 Frequency Conversion in Running Mode

```
void (*run)(struct rt_pm *pm, uint8_t mode);
```

The frequency conversion of operation mode is realized by adapting the `run` interface in `rt_pm_ops`, and the appropriate frequency is selected according to the use scenario.

1.4.10.7.4 Power management of peripherals

Power processing of peripherals is an important part of low power management system. When entering some level of sleep mode, it is usually necessary to process some peripherals, such as emptying DMA, closing clock or setting IO to reset state, and recover after quitting sleep.
In this case, PM devices can be registered through rt_pm_device_register interface, suspend and resume callbacks of registered devices will be executed when entering/exiting Sleeping mode, and frequency_change callbacks of devices will also be triggered by frequency changes in Running mode.
A more detailed migration case can be referred to stm32l476-nucleo BSP in the RT-Thread repository.

1.4.10.8 MSH Commands

1.4.10.8.1 Request Sleep Mode

The pm_request command can be used to request related patterns, using an example as follows:

```
msh />pm_request 0
msh />
```

The range of parameter values is 0-5, corresponding to the following enumeration values:

```
enum
{
    /* sleep modes */
    PM_SLEEP_MODE_NONE = 0,     /* active state */
    PM_SLEEP_MODE_IDLE,         /* idle mode (default) */
    PM_SLEEP_MODE_LIGHT,        /* Light Sleep Mode */
    PM_SLEEP_MODE_DEEP,         /* Deep Sleep Mode */
    PM_SLEEP_MODE_STANDBY,      /* Standby Mode */
    PM_SLEEP_MODE_SHUTDOWN,     /* Shutdowm Mode */
    PM_SLEEP_MODE_MAX,
};
```

1.4.10.8.2 Release Sleep Mode

You can use the pm_release command to release the sleep mode. The range of parameters is 0-5, and the examples are as follows:

```
msh />pm_release 0
msh />
```

1.4.10.8.3 Setting up Running Mode

You can use the pm_run command to switch the mode of operation with parameters ranging from 0 to 3, as shown in the following example

```
msh />pm_run 2
msh />
```

The range of parameters is 0 to 3:

```
enum
{
    /* run modes*/
    PM_RUN_MODE_HIGH_SPEED = 0,
    PM_RUN_MODE_NORMAL_SPEED,
    PM_RUN_MODE_MEDIUM_SPEED,
    PM_RUN_MODE_LOW_SPEED,
    PM_RUN_MODE_MAX,
};
```

1.4.10.8.4 View mode status

You can use the pm_dump command to view the mode state of the PM component, as shown in the following example

```
msh >
msh >pm_dump
| Power Management Mode | Counter | Timer |
+-----------------------+---------+-------+
|              None Mode |       0 |     0 |
|              Idle Mode |       0 |     0 |
|       LightSleep Mode |       1 |     0 |
|        DeepSleep Mode |       0 |     1 |
|          Standby Mode |       0 |     0 |
|         Shutdown Mode |       0 |     0 |
+-----------------------+---------+-------+
pm current sleep mode: LightSleep Mode
pm current run mode:   Normal Speed
msh >
```

In the pattern list of pm_dump, the priority of sleep mode is arranged from high to low.
The Counter column identifies the count of requests. The graph shows that the Light Sleep mode is requested once, so the current work is in a slight sleep state.
The Timer column identifies whether to turn on sleep time compensation. In the figure, only Deep Sleep mode is used for time compensation.
The bottom part identifies the current sleep mode and running mode level respectively.

1.4.10.9 Common problems and debugging methods

- When the system enters the low power mode, the power consumption is too high.

According to the peripheral circuit, check whether the equipment is in a reasonable state to avoid leakage of peripheral equipment.
According to the product's own situation, turn off the peripherals and clocks that are not used during the corresponding sleep mode.

- Unable to enter lower levels of power consumption

Check whether the higher-level power consumption mode is not released. RT-Thread's PM component uses `rt_pm_request` to request dormancy mode. If it is not released after requesting high-power consumption mode, the system will not be able to switch to lower-level power consumption.
For example, after requesting Light Sleep mode, then requesting Deep Sleep mode, the system is still in Light Sleep mode. By calling the interface `rt_pm_request` to release the Light Sleep mode, the system will automatically switch to the Deep Sleep mode.

1.4.11 Network Framework

With the popularity of the Internet, people's lives are increasingly dependent on the application of the network. More and more products need to connect to the Internet, and device networking has become a trend. To achieve the connection between the device and the network, you need to follow the TCP/IP protocol, you can run the network protocol stack on the device to connect to the network, or you can use devices (chips with hardware network protocol stack interfaces) to connect to the Internet.
When the device is connected to the network, it is like plugging in the wings. You can use the network to upload data in real time. The user can see the current running status and collected data of the device in a hundred thousand miles, and remotely control the device to complete specific tasks. You can also play online music, make online calls, and act as a LAN storage server through your device.
This chapter will explain the related content of the RT-Thread network framework, and introduce you to the concept, function and usage of the network framework. After reading this chapter, you will be familiar with the concept and implementation principle of the RT-Thread network framework and familiar with network programming using Socket API.

1.4.11.1 TCP/IP Introduction to Network Protocols

TCP/IP is short for Transmission Control Protocol/Internet Protocol. It is not a single protocol, but a general term for a protocol family. It includes IP protocol, ICMP protocol, TCP protocol, and http and ftp, pop3, https protocol, etc., which define how electronic devices connect to the Internet and the standards by which data is transferred between them.

1.4.11.1.1 OSI Reference Model

OSI (Open System Interconnect), which is an open system interconnection. Generally referred to as the OSI reference model, it is a network interconnection model studied by the ISO (International Organization for Standardization) in 1985. The architecture standard defines a seven-layer framework for the network interconnection (physical layer, data link layer, network layer, transport layer, session layer, presentation layer, and application layer), that is, the ISO open system interconnection reference model. The first to third layers belong to the lower three layers of the OSI Reference Model and are responsible for creating links for network communication connections; the fourth to seventh layers are the upper four layers of the OSI reference model and is responsible for end-to-end data communication. The capabilities of each layer are further detailed in this framework to achieve interconnectivity, interoperability, and application portability in an open system environment.

1.4.11.1.2 TCP/IP Reference Model

The TCP/IP communication protocol uses a four-layer hierarchical structure, and each layer calls the network provided by its next layer to fulfill its own needs. The four layers are:

- **Application layer**: Different types of network applications have different communication rules, so the application layer protocols are various, such as Simple Mail Transfer Protocol (SMTP), File Transfer Protocol (FTP), and network remote access protocol (Telnet).

- **Transport layer**: In this layer, it provides data transfer services between nodes, such as Transmission Control Protocol (TCP), User Datagram Protocol (UDP), etc. TCP and UDP add data to the data packet and transmit it to the next layer, this layer is responsible for transmitting data and determining that the data has been delivered and received.
- **Network layer**: responsible for providing basic data packet transfer functions, so that each packet can reach the destination host (but not check whether it is received correctly), such as Internet Protocol (IP).
- **Network interface layer**: Management of actual network media, defining how to use actual networks (such as Ethernet, Serial Line, etc.) to transmit data.

1.4.11.1.3 Difference between TCP/IP Reference Model and OSI Reference Model

The following figure shows the TCP/IP reference model and the OSI reference model diagram:

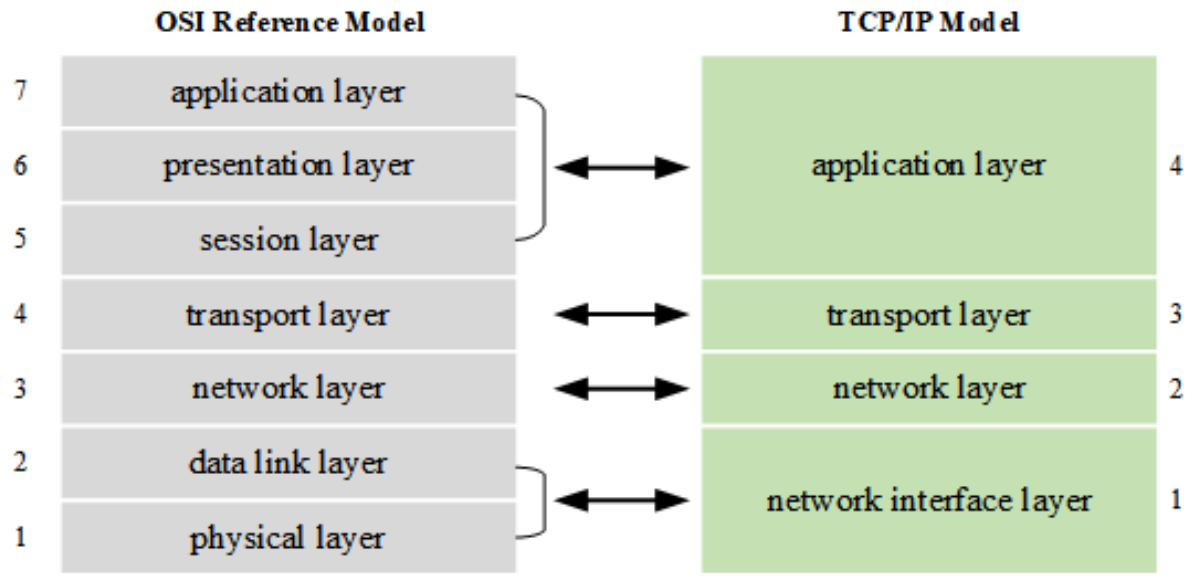


Figure 1.138 TCP/IP Reference Model and OSI Reference Model

Both the OSI reference model and the TCP/IP reference model are hierarchical, based on the concept of a separate protocol stack. The OSI reference model has 7 layers, while the TCP/IP reference model has only 4 layers, that is, the TCP/IP reference model has no presentation layer and session layer, and the data link layer and physical layer are merged into a network interface layer. However, there is a certain correspondence between the two layers. Due to the complexity of the OSI system and the design prior to implementation, many designs are too ideal and not very convenient for software implementation. Therefore, there are not many systems that fully implement the OSI reference model, and the scope of application is limited. The TCP/IP reference model was first implemented in a computer system. It has a stable implementation on UNIX and Windows platforms, and provides a simple and convenient programming interface (API) on which a wide range of applications are developed. The TCP/IP reference model has become the international standard and industry standard for Internet connectivity.

1.4.11.1.4 IP Address

The IP address refers to the Internet Protocol Address (also translated as the Internet Protocol Address) and is a uniform address format that assigns a logical address to each network and each host on the Internet to mask physical address differences provided by Internet Protocol. The common LAN IP address is 192.168.X.X.

1.4.11.1.5 Subnet Mask

Subnet mask (also called netmask, address mask), which is used to indicate which bits of an IP address identify the subnet where the host is located, and which bits are identified as the bit mask of the host. The subnet mask cannot exist alone, it must be used in conjunction with an IP address. Subnet mask has only one effect, which is to divide an IP address into two parts: network address and host address. The subnet mask is the bit of 1, the IP address is the network address, the subnet mask is the bit of 0, and the IP address is the host address. Taking the

IP address 192.168.1.10 and the subnet mask 255.255.255.0 as an example, the first 24 bits of the subnet mask (converting decimal to binary) is 1, so the first 24 bits of the IP address 192.168.1 represent the network address. The remaining 0 is the host address.

1.4.11.1.6 MAC Address

MAC (figures Access Control or Medium Access Control) address, which is translated as media access control, or physical address, hardware address, used to define the location of network devices. In OSI model, the third layer network Layer is responsible for IP address, the second layer data link layer is responsible for the MAC address. A host will have at least one MAC address.

1.4.11.2 Introduction to the Network Framework of RT-Thread

In order to support various network protocol stacks, RT-Thread has developed a **SAL** component, the full name of the **Socket abstraction layer**. RT-Thread can seamlessly access various protocol stacks, including several commonly used TCP/IP protocol stack, such as the LwIP protocol stack commonly used in embedded development and the AT Socket protocol stack component developed by RT-Thread, which complete the conversion of data from the network layer to the transport layer.
The main features of the RT-Thread network framework are as follows:

- Support for standard network sockets BSD Socket API, support for poll/select
- Abstract, unified multiple network protocol stack interfaces
- Support various physical network cards, network communication module hardware
- The resource occupancy of SAL socket abstraction layer component is small: ROM 2.8K and RAM 0.6K.

RT-Thread's network framework adopts a layered design with four layers, each layer has different responsibilities. The following figure shows the RT-Thread network framework structure:

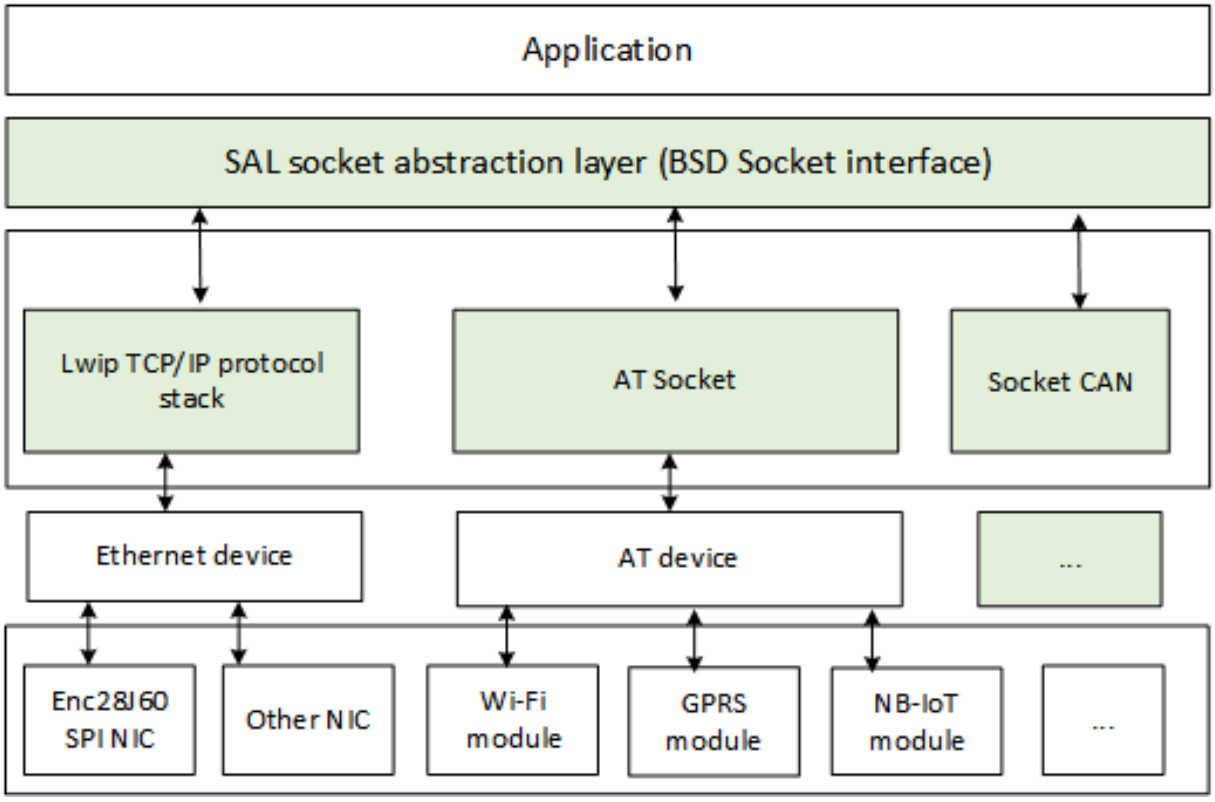


Figure 1.139 RT-Thread network framework structure

The network framework provides a standard BSD Socket interface to user applications. Developers use the BSD Socket interface to operate without worrying about how the underlying network is implemented, and no need to care

which network protocol stack the network data passes through. The socket abstraction layer provides the upper application layer. The interfaces are: `accept`, `connect`, `send`, `recv`, etc.
Below the SAL layer is the protocol stack layer. The main protocol stacks supported in the current network framework are as follows:

- **LwIP** is an open source TCP/IP protocol stack implementation that reduces RAM usage while maintaining the main functionality of the TCP/IP protocol, making the LwIP protocol stack ideal for use in embedded systems.
- **AT Socket** is a component for modules that support AT instructions. The AT command uses a standard serial port for data transmission and reception, and converts complex device communication methods into simple serial port programming, which greatly simplifies the hardware design and software development costs of the product, which makes almost all network modules such as GPRS, 3G/4G, NB-IoT, Bluetooth, WiFi, GPS and other modules are very convenient to access the RT-Thread network framework, develop network applications through the standard BSD Socket method, greatly simplifying the development of upper-layer applications.
- **Socket CAN** is a way of programming CAN, it is easy to use and easy to program. By accessing the SAL layer, developers can implement Socket CAN programming on RT-Thread.

Below the protocol stack layer is an abstract device layer that is connected to various network protocol stacks by abstracting hardware devices into Ethernet devices or AT devices.
The bottom layer is a variety of network chips or modules (for example: Ethernet chips with built-in protocol stack such as W5500/CH395, WiFi module with AT command, GPRS module, NB-IoT module, etc.). These hardware modules are the carrier that truly performs the network communication function and is responsible for communicating with various physical networks.
In general, the RT-Thread network framework allows developers to only care about and use the standard BSD Socket network interface for network application development, without concern for the underlying specific network protocol stack type and implementation, greatly improving system compatibility and convenience. Developers have completed the development of network-related applications, and have greatly improved the compatibility of RT-↩ Thread in different areas of the Internet of Things.
In addition, based on the network framework, RT-Thread provides a large number of network software packages, which are various network applications based on the SAL layer, such as **Paho MQTT**, **WebClient**, **cJSON**, **netutils**, etc., which can be obtained from the online package management center. These software packages are web application tools. Using them can greatly simplify the development of network applications and shorten the network application development cycle. At present, there are more than a dozen network software packages. The following table lists some of the network software packages currently supported by RT-Thread, and the number of software packages is still increasing.

Package Name	Description
Paho MQTT	Based on Eclipse open source Paho MQTT, it has done a lot of functions and performance optimization, such as: increased automatic reconnection after disconnection, pipe model, support for non-blocking API, support for TLS encrypted transmission, etc.
WebClient	Easy-to-use HTTP client with support for HTTP GET/POST and other common request functions, support for HTTPS, breakpoint retransmission, etc.
mongoose	Embedded Web server network library, similar to Nginx in the embedded world. Licensing is not friendly enough, business needs to be charged
WebTerminal	Access Finsh/MSH Shell packages in the browser or on the mobile
cJSON	Ultra-lightweight JSON parsing library
ljson	Json to struct parsing, output library
ezXML	XML file parsing library, currently does not support parsing XML data
nanopb	Protocol Buffers format data parsing library, Protocol Buffers format takes up less resources than JSON, XML format resources
GAgent	Software package for accessing Gizwits Cloud Platform
Marvell WiFi	Marvell WiFi driver
Wiced WiFi	WiFi driver for Wiced interface

Package Name	Description
CoAP	Porting libcoap's CoAP communication package
nopoll	Ported open source WebSocket communication package
netutils	A collection of useful network debugging gadgets, including: ping, TFTP, iperf, NetIO, NTP, Telnet, etc.
OneNet	Software for accessing China Mobile OneNet Cloud

1.4.11.3 Network Framework Workflow

Using the RT-Thread network framework, you first need to initialize the SAL, then register various network protocol clusters to ensure that the application can communicate using the socket network socket interface. This section mainly uses LwIP as an example.

1.4.11.3.1 Register the Network Protocol Cluster

First use the `sal_init()` interface to initialize resources such as mutex locks used in the component. The interface looks like this:

```
int sal_init(void);
```

After the SAL is initialized, then use the the `sal_proto_family_register()` interface to register network protocol cluster, for example, the LwIP network protocol cluster is registered to the SAL. The sample code is as follows:

```
static const struct proto_family LwIP_inet_family_ops = {
    "LwIP",
    AF_INET,
    AF_INET,
    inet_create,
    LwIP_gethostbyname,
    LwIP_gethostbyname_r,
    LwIP_freeaddrinfo,
    LwIP_getaddrinfo,
};

int LwIP_inet_init(void)
{
    sal_proto_family_register(&LwIP_inet_family_ops);

    return 0;
}
```

`AF_INET` stands for IPv4 address, for example 127.0.0.1; `AF` is short for "Address Family" and `INET` is short for "Internet".

The `sal_proto_family_register()` interface is defined as follows:

```
int sal_proto_family_register(const struct proto_family *pf)
```

Parameters	Description
pf	Protocol cluster structure pointer
Return	——
0	registration success
-1	registration failed

1.4.11.3.2 Network Data Receiving Process

After the LwIP is registered to the SAL, the application can send and receive network data through the network socket interface. In LwIP, several main threads are created, and they are `tcpip` thread, `erx` receiving thread and `etx` sending thread. The network data receiving process is as shown in the following picture. The application receives data by calling the standard socket interface `recv()` with blocking mode. When the Ethernet hardware device receives the network data packet, it stores the packet in the receiving buffer, and then sends an email to notify the `erx` thread that the data arrives through the Ethernet interrupt program. The `erx` thread applies for the `pbuf` memory block according to the received data length and put the data into the pbuf's `payload` data, then send the `pbuf` memory block to the `tcpip` thread via mailbox, and the `tcpip` thread returns the data to the application that is blocking the receiving data.

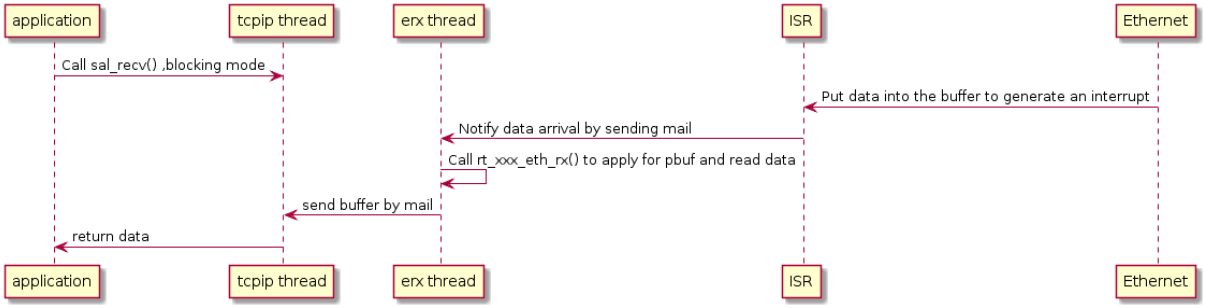


Figure 1.140 Data receiving function call flow chart

1.4.11.3.3 Network Data Sending Process

The network data sending process is shown in the figure below. When there is data to send, the application calls the standard network socket interface `send()` to hand the data to the `tcpip` thread. The `tcpip` thread sends a message to wake up the `etx` thread. The `etx` thread first determines if the Ethernet is sending data. If data is not being sent, it will put the data to be sent into the send buffer, and then send the data through the Ethernet device. If data is being sent, the `etx` thread suspends itself until the Ethernet device is idle before sending the data out.

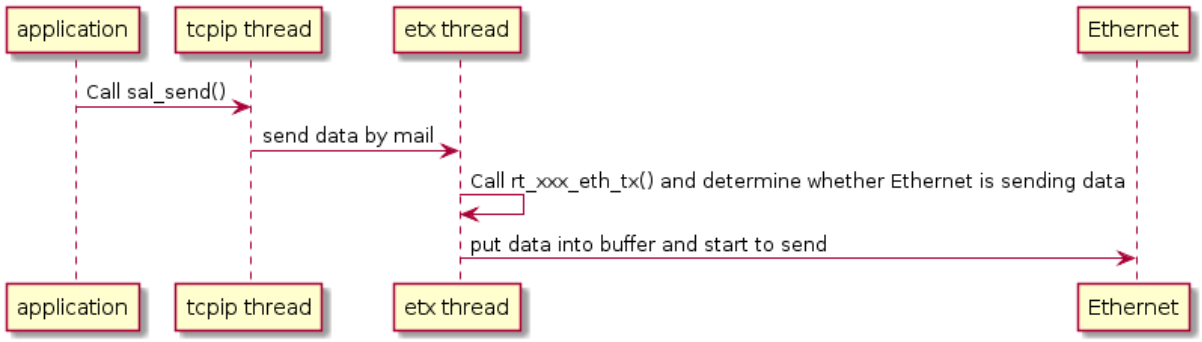


Figure 1.141 Data sending function call flow chart

1.4.11.4 Network Socket Programming

The application uses Socket (network socket) interface programming to implement network communication functions. Socket is a set of application program interface (API), which shields the communication details of each protocol, so that the application does not need to pay attention to the protocol itself, directly using the interfaces provide by socket to communicate between different hosts interconnected.

1.4.11.4.1 TCP socket Communication Process

TCP(Tranfer Control Protocol) is a connection-oriented protocol to ensure reliable data transmission. Through the TCP protocol transmission, a sequential error-free data stream is obtained. The TCP-based socket programming flow diagram is shown in the following figure. A connection must be established between the sender and the receiver's two sockets in order to communicate on the basis of the TCP protocol. When a socket (usually a server socket) waits for a connection to be established. Another socket can request a connection. Once the two sockets are connected, they can perform two-way data transmission, and both sides can send or receive data. A TCP connection is a reliable connection that guarantees that packets arrive in order, and if a packet loss occurs, the packet is automatically resent.
For example, TCP is equivalent to calling in life. When you call the other party, you must wait for the other party to answer. Only when the other party answers your call can he/she establish a connection with you. The two parties can talk and pass information to each other. Of course, the information passed at this time is reliable, because the other party can't hear what you said and can ask you to repeat the content again. When either party on the call wants to end the call, they will bid farewell to the other party and wait until the other party bids farewell to them before they hang up and end the call.

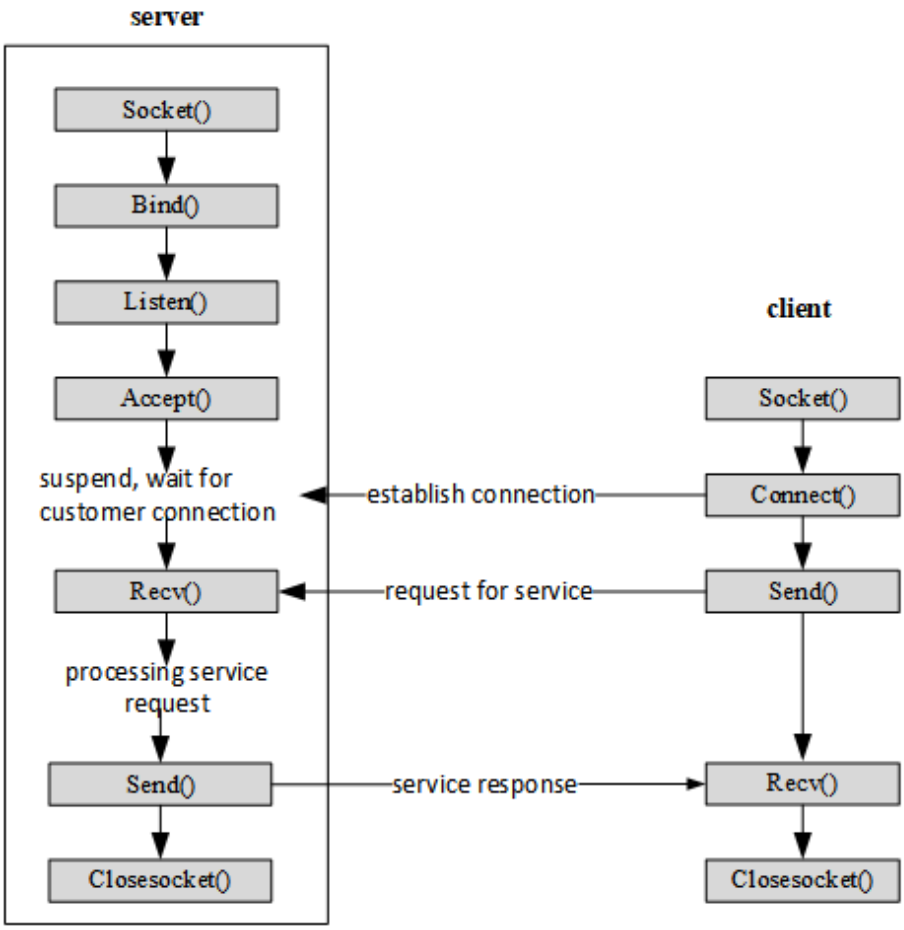


Figure 1.142 TCP-based socket programming flow chart

1.4.11.4.2 UDP socket Communication Process

UDP is short for User Datagram Protocol. It is a connectionless protocol. Each datagram is a separate information, including the complete source address and destination address. It is transmitted to the destination on the network in any possible path. Therefore, whether the destination can be reached, the time to reach the destination, and the correctness of the content cannot be guaranteed. The UDP-based socket programming flow is shown in the following figure.

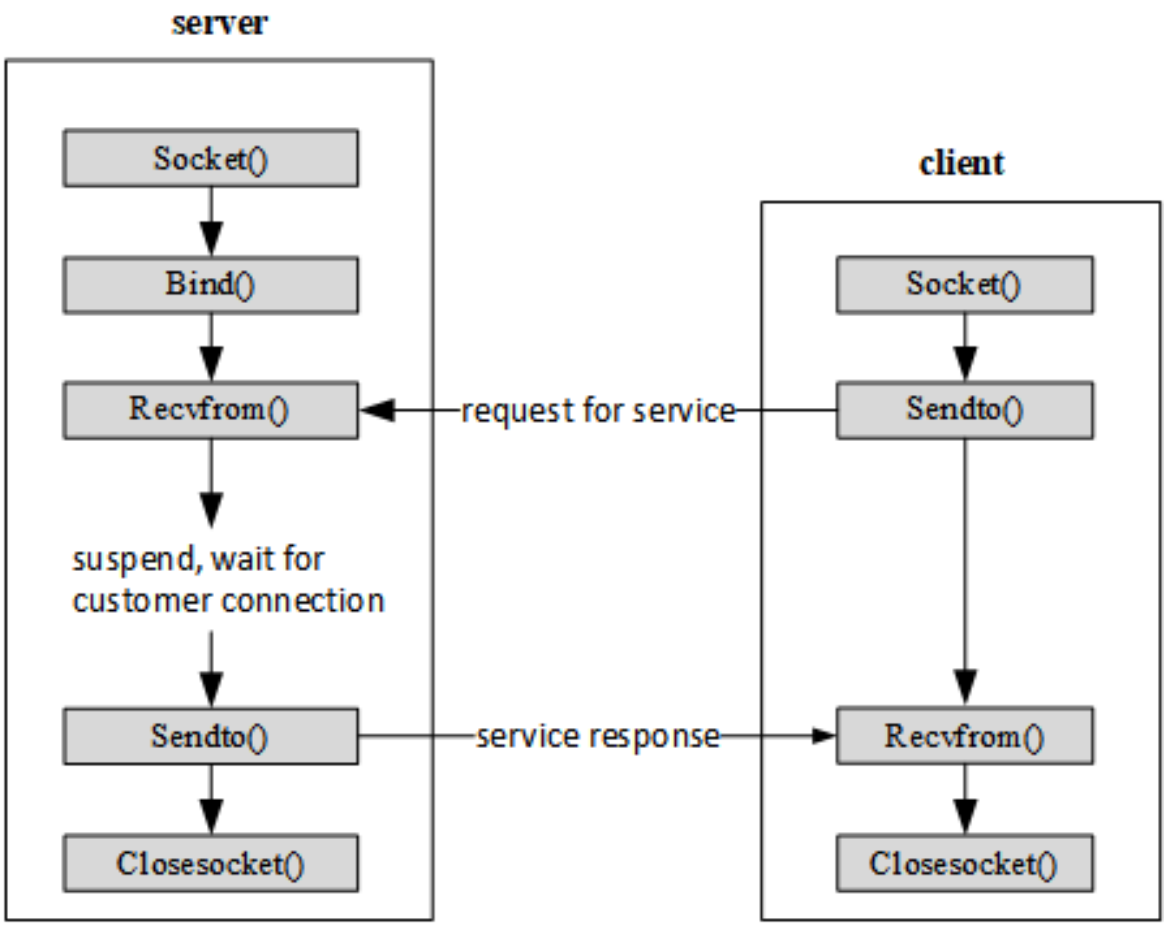


Figure 1.143 UDP-based socket programming flow

For example, UDP is equivalent to the walkie-talkie communication in life. After you set up the channel, you can directly say the information you want to express. The data is sent out by the walkie-talkie, but you don't know if your message has been received by others. By the way, unless someone else responds to you with a walkie-talkie. So this method is not reliable.

1.4.11.4.3 Create a Socket

Before communicating, the communicating parties first use the `socket()` interface to create a socket, assigning a socket descriptor and its resources based on the specified address family, data type, and protocol. The interface is as follows:

```
int socket(int domain, int type, int protocol);
```

Parameters	Description
domain	Protocol family
type	Specify the communication type, including values SOCK_STREAM and SOCK_DGRAM.
protocol	Protocol allows to specify a protocol for the socket, which is set to 0 by default.
Return	——
>=0	Successful, returns an integer representing the socket descriptor
-1	Fail

Communication types include SOCK_STREAM and SOCK_DGRAM.
SOCK_STREAM indicates connection-oriented TCP data transfer. Data can arrive at another computer without any errors. If it is damaged or lost, it can be resent, but it is relatively slow.

SOCK_DGRAM indicates the connectionless UDP data transfer method. The computer only transmits data and does not perform data verification. If the data is damaged during transmission or does not reach another computer, there is no way to remedy it. In other words, if the data is wrong, it is wrong and cannot be retransmitted. Because SOCK_DGRAM does less validation work, it is more efficient than SOCK_STREAM.
The sample code for creating a TCP type socket is as follows:

```
/* Create a socket, type is SOCKET_STREAMTCP type */
   if ((sock = socket(AF_INET, SOCK_STREAM, 0)) == -1)
   {
       /* failed to create socket*/
       rt_kprintf("Socket error\n");

       return;
   }
```

1.4.11.4.4 Binding Socket

A binding socket is used to bind a port number and an IP address to a specified socket. When using socket() to create a socket, only the protocol family is given, and no address is assigned. Before the socket receives a connection from another host, it must bind it with an address and port number using bind(). The interface is as follows:

```
int bind(int s, const struct sockaddr *name, socklen_t namelen);
```

Parameters	Description
S	Socket descriptor
name	Pointer to the sockaddr structure representing the address to be bound
namelen	Length of sockaddr structure
Return	——
0	Successful
-1	Fail

1.4.11.4.5 Establishing a TCP Connection

For server-side programs, after using `bind()` to bind the socket, you also need to use the `listen()` function to make the socket enter the passive listening state, and then call the `accept()` function to respond to the client at any time.

1.4.11.4.5.1 Listening Socket

The listening socket is used by the TCP server to listen for the specified socket connection. The interface is as follows:

```
int listen(int s, int backlog);
```

Parameters	Description
s	Socket descriptor
backlog	Indicates the maximum number of connections that can be waited at a time
Return	——
0	Successful
-1	Fail

1.4.11.4.5.2 Accept the Connection

When the application listens for connections from other clients, the connection must be initialized with the `accept()` function, which creates a new socket for each connection and removes the connection from the listen queue. The interface is as follows:

```
int accept(int s, struct sockaddr *addr, socklen_t *addrlen);
```

Parameters	Description
s	Socket descriptor
addr	Client device address information
addrlen	Client device address structure length
Return	——
>=0	Successful, return the newly created socket descriptor
-1	Fail

1.4.11.4.5.3 Establish Connection

Used by the client to establish a connection with the specified server. The interface is as follows:

```
int connect(int s, const struct sockaddr *name, socklen_t namelen);
```

Parameters	Description
s	Socket descriptor
name	Server address information
namelen	Server address structure length
Return	——
0	Successful
-1	Fail

When the client connects to the server, first set the server address and then use the `connect()` function to connect. The sample code is as follows:

```
struct sockaddr_in server_addr;
/* Initialize the pre-connected server address */
server_addr.sin_family = AF_INET;
server_addr.sin_port = htons(port);
server_addr.sin_addr = *((struct in_addr *)host->h_addr);
rt_memset(&(server_addr.sin_zero), 0, sizeof(server_addr.sin_zero));

/* Connect to the server */
if (connect(sock, (struct sockaddr *)&server_addr, sizeof(struct sockaddr)) == -1)
{
    /* Connection failed */
    closesocket(sock);

    return;
}
```

1.4.11.4.6 Data Transmission

TCP and UDP have different data transmission methods. TCP needs to establish a connection before data transmission, use `send()` function for data transmission, use `recv()` function for data reception, and UDP does not need to establish connection. It uses `sendto()` function sends data and receives data using the `recvfrom()` function.

1.4.11.4.6.1 TCP Data Transmission

After the TCP connection is established, the data is sent using the `send()` function. The interface is as follows:

```
int send(int s, const void *dataptr, size_t size, int flags);
```

Parameters	Description
s	Socket descriptor
dataptr	The data pointer to send
size	Length of data sent

Parameters	Description
flags	Flag, generally 0
Return	——
>0	Successful, return the length of the sent data
<=0	Failed

1.4.11.4.6.2 TCP Data Reception

After the TCP connection is established, use `recv()` to receive the data. The interface is as follows:

```
int recv(int s, void *mem, size_t len, int flags);
```

Parameters	Description
s	Socket descriptor
mem	Received data pointer
len	Received data length
flags	Flag, generally 0
Return	**Description**
>0	Successful, return the length of the received data
=0	The destination address has been transferred and the connection is closed
<0	Fail

1.4.11.4.6.3 UDP Data transmission

In the case where a connection is not established, you can use the `sendto()` function to send UDP data to the specified destination address, as shown below:

```
int sendto(int s, const void *dataptr, size_t size, int flags,
           const struct sockaddr *to, socklen_t tolen);
```

Parameters	Description
s	Socket descriptor
dataptr	Data pointer sent
size	Length of data sent
flags	Flag, generally 0
to	Target address structure pointer
tolen	Target address structure length
Return	——
>0	Successful, return the length of the sent data
<=0	Fail

1.4.11.4.6.4 UDP Data Reception

To receive UDP data, use the `recvfrom()` function, and the interface is:

```
int recvfrom(int s, void *mem, size_t len, int flags,
             struct sockaddr *from, socklen_t *fromlen);
```

Parameters	Description
s	Socket descriptor
mem	Received data pointer

Parameters	Description
len	Received data length
flags	Flag, generally 0
from	Receive address structure pointer
fromlen	Receive address structure length
Return	——
>0	Successful, return the length of the received data
0	The receiving address has been transferred and the connection is closed
<0	Fail

1.4.11.4.7 Close Network Connection

After the network communication is over, you need to close the network connection. There are two ways to use `closesocket()` and `shutdown()`.
The `closesocket()` interface is used to close an existing socket connection, release the socket resource, clear the socket descriptor from memory, and then the socket could not be used again. The connection and cache associated with the socket are also lost the meaning, the TCP protocol will automatically close the connection. The interface is as follows:

```
int closesocket(int s);
```

Parameters	Description
s	Socket descriptor
Return	——
0	Successful
-1	Fail

Network connections can also be turned off using the `shutdown()` function. The TCP connection is full-duplex. You can use the `shutdown()` function to implement a half-close. It can close the read or write operation of the connection, or both ends, but it does not release the socket resource. The interface is as follows:

```
int shutdown(int s, int how);
```

Parameters	Description
s	Socket descriptor
how	SHUT_RD closes the receiving end of the connection and no longer receives data. SHUT_WR closes the connected sender and no longer sends data. SHUT_RDWR is closed at both ends.
Return	——
0	Successful
-1	Fail

1.4.11.5 Network Function Configuration

The main functional configuration options of the network framework are shown in the following table, which can be configured according to different functional requirements:
SAL component configuration options:

Macro Definition	Value Type	Description
RT_USING_SAL	Boolean	Enable SAL

Macro Definition	Value Type	Description
SAL_USING_LWIP	Boolean	Enable LwIP component
SAL_USING_AT	Boolean	Enable the AT component
SAL_USING_POSIX	Boolean	Enable POSIX interface
SAL_PROTO_FAMILIES_NUM	Integer	the maximum number of protocol families supported

LwIP Configuration options:

Macro Definition	Value Type	Description
RT_USING_LWIP	Boolean	Enable LwIP protocol
RT_USING_LWIP_IPV6	Boolean	Enable IPV6 protocol
RT_LWIP_IGMP	Boolean	Enable the IGMP protocol
RT_LWIP_ICMP	Boolean	Enable the ICMP protocol
RT_LWIP_SNMP	Boolean	Enable the SNMP protocol
RT_LWIP_DNS	Boolean	Enable DHCP function
RT_LWIP_DHCP	Boolean	Enable DHCP function
IP_SOF_BROADCAST	Integer	filtering roadcasting Packets Sended by IP
IP_SOF_BROADCAST_RECV	Integer	filtering roadcasting Packets received by IP
RT_LWIP_IPADDR	String	IP address
RT_LWIP_GWADDR	String	Gateway address
RT_LWIP_MSKADDR	String	Subnet mask
RT_LWIP_UDP	Boolean	Enable UDP protocol
RT_LWIP_TCP	Boolean	Enable TCP protocol
RT_LWIP_RAW	Boolean	Enable RAW API
RT_MEMP_NUM_NETCONN	Integer	Support Numbers of network links
RT_LWIP_PBUF_NUM	Integer	pbuf number of memory blocks
RT_LWIP_RAW_PCB_NUM	Integer	Maximum number of connections for RAW
RT_LWIP_UDP_PCB_NUM	Integer	Maximum number of connections for UDP
RT_LWIP_TCP_PCB_NUM	Integer	Maximum number of connections for TCP
RT_LWIP_TCP_SND_BUF	Integer	TCP send buffer size
RT_LWIP_TCP_WND	Integer	TCP sliding window size
RT_LWIP_TCPTHREAD_PRIORITY	Integer	TCP thread priority
RT_LWIP_TCPTHREAD_MBOX_SIZE	Integer	TCP thread mailbox size
RT_LWIP_TCPTHREAD_STACKSIZE	Integer	TCP thread stack size
RT_LWIP_ETHTHREAD_PRIORITY	Integer	Receive/send thread's priority
RT_LWIP_ETHTHREAD_STACKSIZE	Integer	Receive/send thread's stack size
RT_LwIP_ETHTHREAD_MBOX_SIZE	Integer	Receive/send thread's mailbox size

1.4.11.6 Network Application Example

1.4.11.6.1 View IP Address

In the console, you can use the ifconfig command to check the network status. The IP address is 192.168.12.26, and the FLAGS status is LINK_UP, indicating that the network is configured:

```
msh >ifconfig
network interface: e0 (Default)
```

```
MTU: 1500
MAC: 00 04 a3 12 34 56
FLAGS: UP LINK_UP ETHARP BROADCAST IGMP
ip address: 192.168.12.26
gw address: 192.168.10.1
net mask  : 255.255.0.0·
dns server #0: 192.168.10.1
dns server #1: 223.5.5.5
```

1.4.11.6.2 Ping Network Test

Use the ping command for network testing:

```
msh />ping rt-thread.org
60 bytes from 116.62.244.242 icmp_seq=0 ttl=49 time=11 ticks
60 bytes from 116.62.244.242 icmp_seq=1 ttl=49 time=10 ticks
60 bytes from 116.62.244.242 icmp_seq=2 ttl=49 time=12 ticks
60 bytes from 116.62.244.242 icmp_seq=3 ttl=49 time=10 ticks
msh />ping 192.168.10.12
60 bytes from 192.168.10.12 icmp_seq=0 ttl=64 time=5 ticks
60 bytes from 192.168.10.12 icmp_seq=1 ttl=64 time=1 ticks
60 bytes from 192.168.10.12 icmp_seq=2 ttl=64 time=2 ticks
60 bytes from 192.168.10.12 icmp_seq=3 ttl=64 time=3 ticks
msh />
```

Getting the above output indicates that the connection network is successful!

1.4.11.6.3 TCP Client Example

After the network is successfully connected, you can run the network example, first run the TCP client example. This example will open a TCP server on the PC, open a TCP client on the IoT Board, and both parties will communicate on the network.

In the example project, there is already a TCP client program `tcpclient_sample.c`. The function is to implement a TCP client that can receive and display the information sent from the server. If it receives the information starting with 'q' or 'Q', then exit the program directly and close the TCP client. The program exports the tcpclient command to the FinSH console. The command format is `tcpclient URL PORT`, where URL is the server address and PORT is the port number. The sample code is as follows:

```
/*
 * Program list: tcp client
 *
 * This is a tcp client routine
 * Export the tcpclient command to MSH
 * Command call format: tcpclient URL PORT
 * URL: server address PORT:: port number
 * Program function: Receive and display the information sent from the server, and receive the information
      that starts with 'q' or 'Q' to exit the program.
*/

#include <rtthread.h>
#include <sys/socket.h> /* To use BSD socket, you need to include the socket.h header file. */
#include <netdb.h>
#include <string.h>
#include <finsh.h>

#define BUFSZ   1024

static const char send_data[] = "This is TCP Client from RT-Thread."; /* Sending used data */
void tcpclient(int argc, char**argv)
{
    int ret;
    char *recv_data;
    struct hostent *host;
    int sock, bytes_received;
    struct sockaddr_in server_addr;
    const char *url;
    int port;

    /* Received less than 3 parameters */
    if (argc < 3)
    {
        rt_kprintf("Usage: tcpclient URL PORT\n");
        rt_kprintf("Like: tcpclient 192.168.12.44 5000\n");
        return ;
    }

    url = argv[1];
    port = strtoul(argv[2], 0, 10);

    /* Get the host address through the function entry parameter url (if it is a domain name, it will do
      domain name resolution) */
```

```
    host = gethostbyname(url);

    /* Allocate buffers for storing received data */
    recv_data = rt_malloc(BUFSZ);
    if (recv_data == RT_NULL)
    {
        rt_kprintf("No memory\n");
        return;
    }

    /* Create a socket of type SOCKET_STREAM, TCP type */
    if ((sock = socket(AF_INET, SOCK_STREAM, 0)) == -1)
    {
        /* Failed to create socket */
        rt_kprintf("Socket error\n");

        /* Release receive buffer */
        rt_free(recv_data);
        return;
    }

    /* Initialize the pre-connected server address */
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(port);
    server_addr.sin_addr = *((struct in_addr *)host->h_addr);
    rt_memset(&(server_addr.sin_zero), 0, sizeof(server_addr.sin_zero));

    /* Connect to the server */
    if (connect(sock, (struct sockaddr *)&server_addr, sizeof(struct sockaddr)) == -1)
    {
        /* Connection failed */
        rt_kprintf("Connect fail!\n");
        closesocket(sock);

        /* Release receive buffer */
        rt_free(recv_data);
        return;
    }

    while (1)
    {
        /* Receive maximum BUFSZ-1 byte data from a sock connection */
        bytes_received = recv(sock, recv_data, BUFSZ - 1, 0);
        if (bytes_received < 0)
        {
            /* Receive failed, close this connection */
            closesocket(sock);
            rt_kprintf("\nreceived error,close the socket.\r\n");

            /* Release receive buffer */
            rt_free(recv_data);
            break;
        }
        else if (bytes_received == 0)
        {
            /* Print the recv function returns a warning message with a value of 0 */
            rt_kprintf("\nReceived warning,recv function returns 0.\r\n");

            continue;
        }

        /* Received data, clear the end */
        recv_data[bytes_received] = '\0';

        if (strncmp(recv_data, "q", 1) == 0 || strncmp(recv_data, "Q", 1) == 0)
        {
            /* If the initial letter is q or Q, close this connection */
            closesocket(sock);
            rt_kprintf("\n got a'q'or'Q',close the socket.\r\n");

            /* Release receive buffer */
            rt_free(recv_data);
            break;
        }
        else
        {
            /* Display the received data at the control terminal */
            rt_kprintf("\nReceived data = %s", recv_data);
        }

        /* Send data to sock connection */
        ret = send(sock, send_data, strlen(send_data), 0);
        if (ret < 0)
        {
            /* Receive failed, close this connection */
            closesocket(sock);
```

```
            rt_kprintf("\nsend error,close the socket.\r\n");

            rt_free(recv_data);
            break;
        }
        else if (ret == 0)
        {
            /* Print the send function returns a warning message with a value of 0 */
            rt_kprintf("\n Send warning,send function returns 0.\r\n");
        }
    }
    return;
}
MSH_CMD_EXPORT(tcpclient, a tcp client sample);
```

When running the example, first open a network debugging assistant on your computer and open a TCP server. Select the protocol type as TCP Server, fill in the local IP address and port 5000, as shown below.

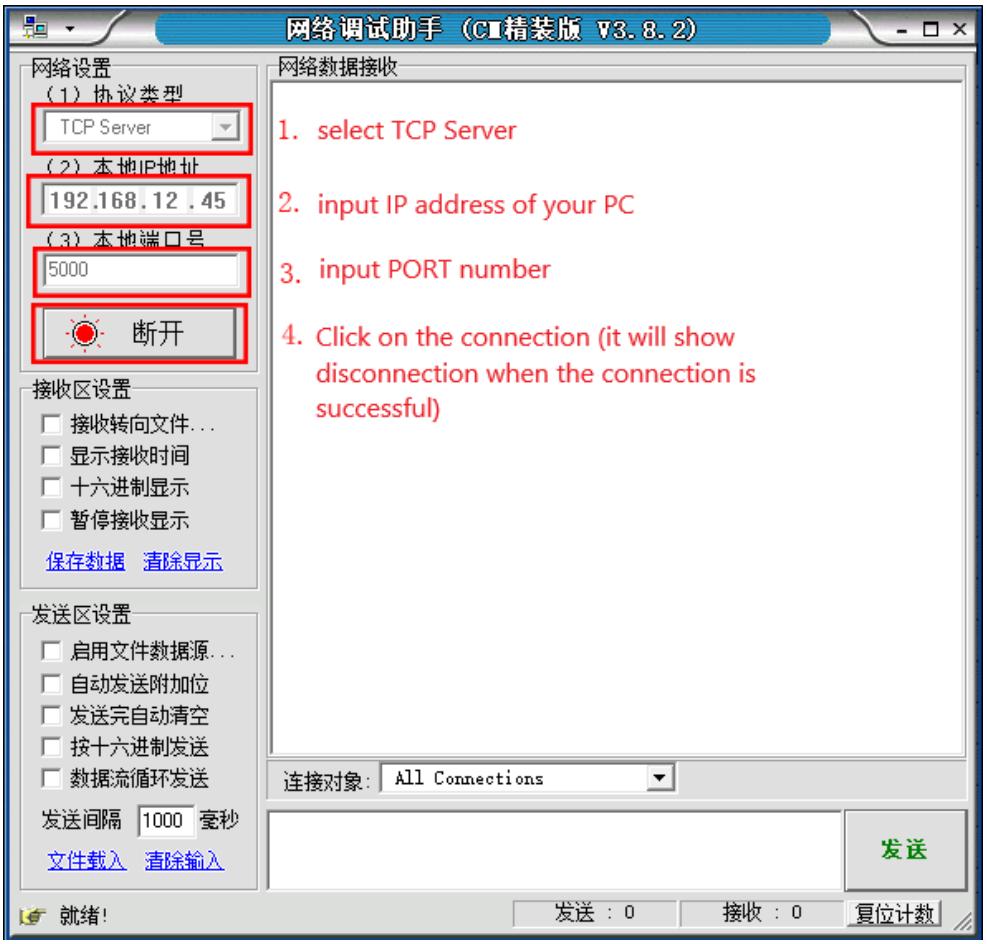


Figure 1.144 Network debugging tool interface

Then start the TCP client to connect to the TCP server by entering the following command in the FinSH console:

```
msh />tcpclient 192.168.12.45 5000  // Input according to actual situation
Connect successful
```

When the console outputs the log message "Connect successful", it indicates that the TCP connection was successfully established. Next, you can perform data communication. In the network debugging tool window, send Hello RT-Thread!, which means that a data is sent from the TCP server to the TCP client, as shown in the following figure:

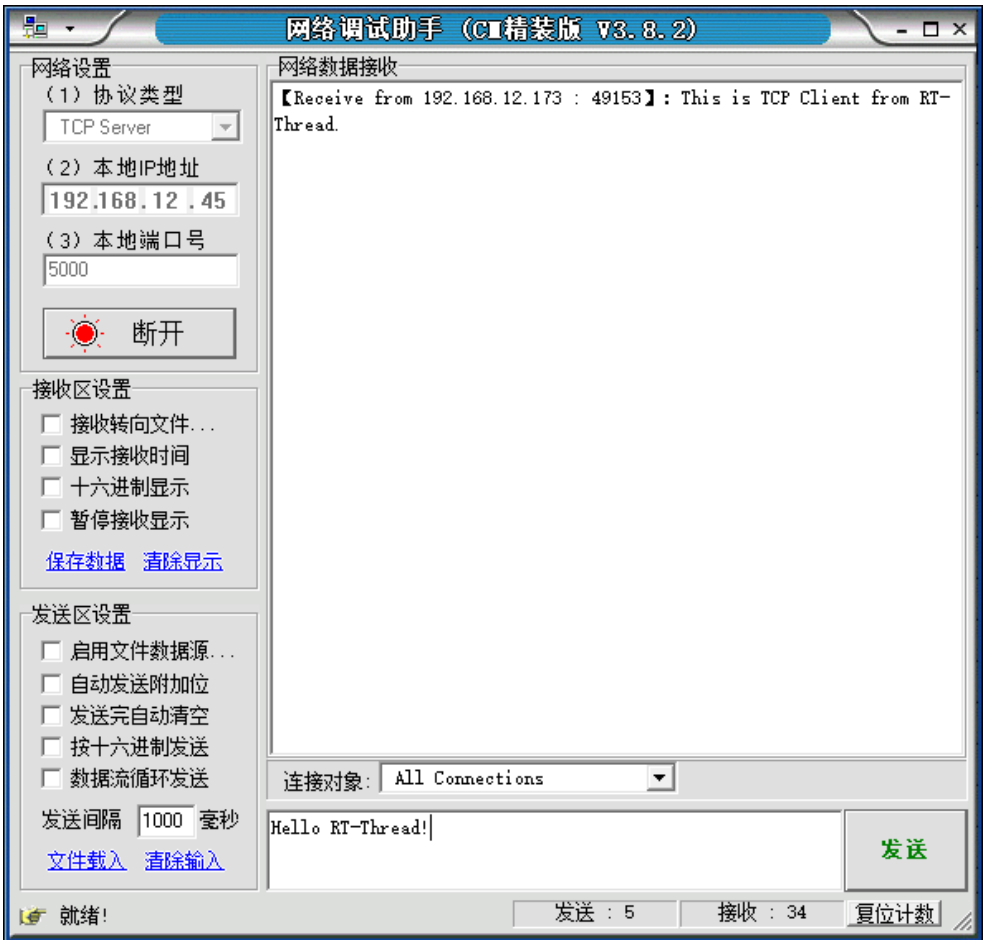


Figure 1.145

After receiving the data on the FinSH console, the corresponding log information will be output, you can see:

```
msh >tcpclient 192.168.12.130 5000
Connect successful
Received data = hello world
Received data = hello world
Received data = hello world
Received data = hello world
Received data = hello world
 got a 'q' or 'Q',close the socket.
msh >
```

The above information indicates that the TCP client received 5 "hello world" data sent from the server. Finally, the exit command 'q' was received from the TCP server, and the TCP client program exited the operation and returned to the FinSH console.

1.4.11.6.4 UDP Client Example

This is an example of a UDP client. This example will open a UDP server on the PC and open a UDP client on the IoT Board for network communication. A UDP client program has been implemented in the sample project. The function is to send data to the server. The sample code is as follows:

```
/*
 * Program list: udp client
 *
 * This is a udp client routine
 * Export the udpclient command to the msh
 * Command call format: udpclient URL PORT [COUNT = 10]
 * URL: Server Address PORT: Port Number COUNT: Optional Parameter Default is 10
 * Program function: send COUNT datas to the remote end of the service
*/

#include <rtthread.h>
#include <sys/socket.h> /* To use BSD socket, you need to include the sockets.h header file. */
#include <netdb.h>
#include <string.h>
```

```
#include <finsh.h>

const char send_data[] = "This is UDP Client from RT-Thread.\n"; /* data */

void udpclient(int argc, char**argv)
{
    int sock, port, count;
    struct hostent *host;
    struct sockaddr_in server_addr;
    const char *url;

    /* Received less than 3 parameters */
    if (argc < 3)
    {
        rt_kprintf("Usage: udpclient URL PORT [COUNT = 10]\n");
        rt_kprintf("Like: udpclient 192.168.12.44 5000\n");
        return ;
    }

    url = argv[1];
    port = strtoul(argv[2], 0, 10);

    if (argc> 3)
        count = strtoul(argv[3], 0, 10);
    else
        count = 10;

    /* Get the host address through the function entry parameter url (if it is a domain name, it will do
      domain name resolution) */
    host = (struct hostent *) gethostbyname(url);

    /* Create a socket of type SOCK_DGRAM, UDP type */
    if ((sock = socket(AF_INET, SOCK_DGRAM, 0)) == -1)
    {
        rt_kprintf("Socket error\n");
        return;
    }

    /* Initialize the pre-connected server address */
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(port);
    server_addr.sin_addr = *((struct in_addr *)host->h_addr);
    rt_memset(&(server_addr.sin_zero), 0, sizeof(server_addr.sin_zero));

    /* Send count data in total */
    while (count)
    {
        /* Send data to the remote end of the service */
        sendto(sock, send_data, strlen(send_data), 0,
               (struct sockaddr *)&server_addr, sizeof(struct sockaddr));

        /* Thread sleep for a while */
        rt_thread_delay(50);

        /* Count value minus one */
        count --;
    }

    /* Turn off this socket */
    closesocket(sock);
}
```

When running the example, first open a network debugging assistant on your computer and open a UDP server. Select the protocol type as UDP and fill in the local IP address and port 5000, as shown in the figure below.

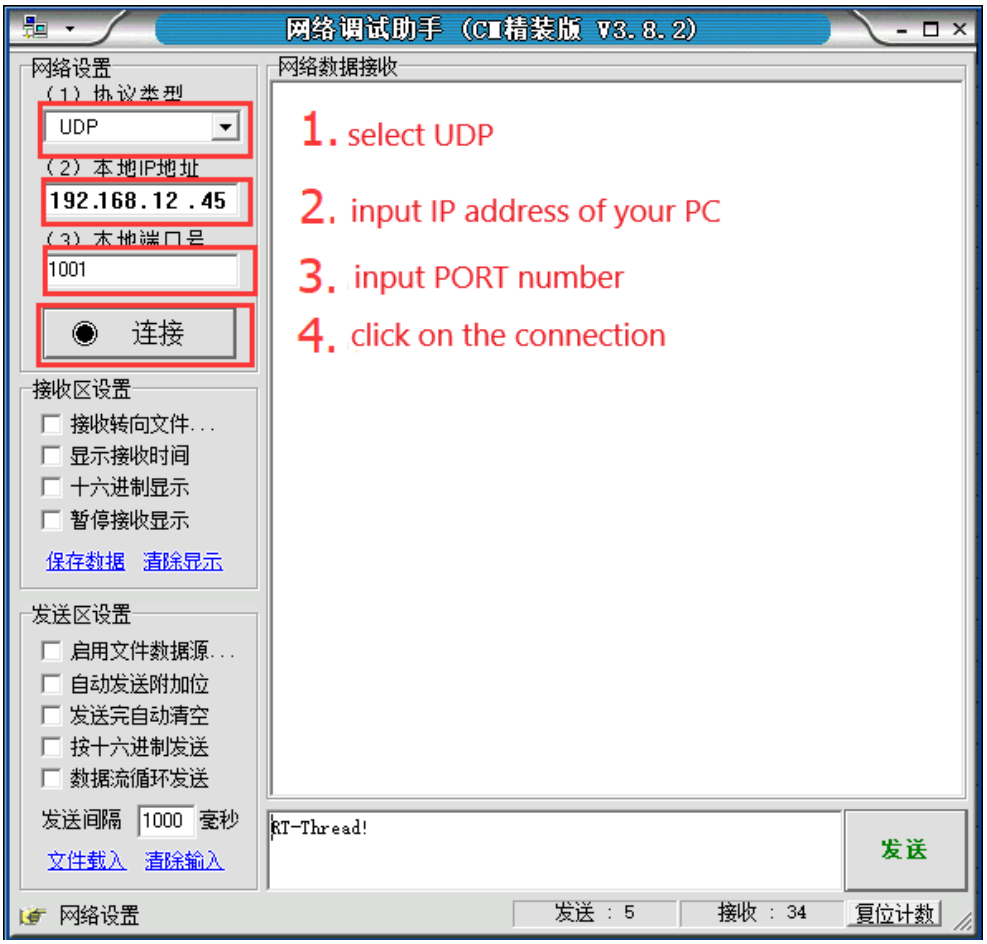


Figure 1.146

Then you can enter the following command in the FinSH console to send data to the UDP server.

```
msh />udpclient 192.168.12.45 1001          // Need to enter according to the real situation
```

The server will receive 10 messages from This is UDP Client from RT-Thread., as shown below:

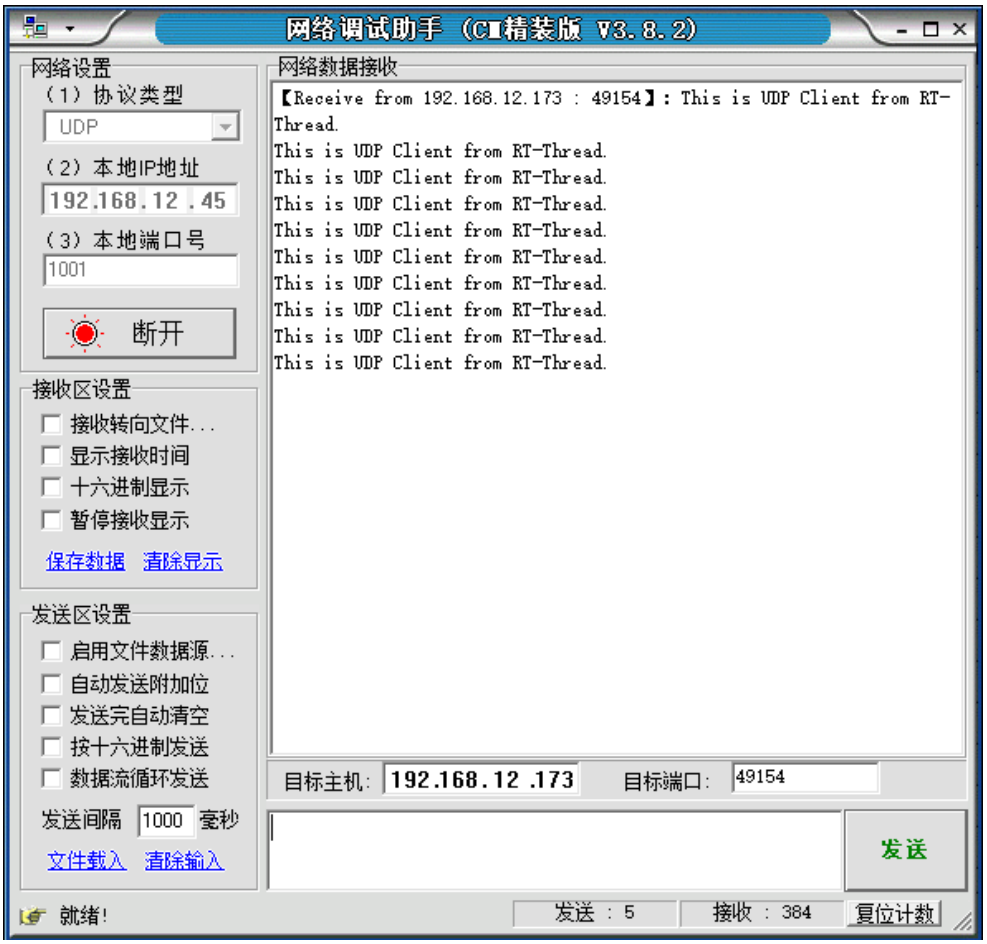


Figure 1.147

1.5 Tool

- Env
- SCons

1.5.1 Env

Env is a handy utility tool developed by RT-Thread team to build environment, graphical system configuration, and packages management for software projects that intend to run on RT-Thread operating system. Env tool come with source code builder, compilation environment and package management system.
It is a wrapper tool for build-in menuconfig, which is an open source GUI tool which is designed to tailor for ease of use for developers. It can also be used to configure the kernel configuration parameters, components and software packages so that developers can construct the system like lego blocks.

1.5.1.1 Main Features

- menuconfig provides graphical interface to interact with operational logic and configuration parameters
- Each configuration option come with help session by default.
- It automates dependencies installation process.
- Automatically generate rtconfig.h without manual modification.
- It uses scons tool to streamline build project and compliation environment.
- Modular software packages and decoupling design make it easier to maintain.

- It also featured with point and click to download additional software packages and dependencies directly from Internet.

1.5.1.2 User Guide

Lastest user guide see https://github.com/RT-Thread/env/blob/master/README.md.

1.5.1.3 Archive Information

NOTE: The following information may be outdated, please read carefully.

1.5.1.3.1 User Guide of Env

1.5.1.3.1.1 Open the control Console

The rt-thread software package environment is mainly based on the command line console so that the rt-thread development environment can be set up by minimizing the modification of configuration files.
There are two ways to open the console:

1. click the Env directory executable file

To enter the Env directory, you can run `env.exe` in this directory. If it fails to open, you can try to use `env.bat`.

2. open the Env console from the right-click menu in the folder

Add Env to the right-click menu:



Figure 1.148 Add_Env_To_Right-click_Menu-1

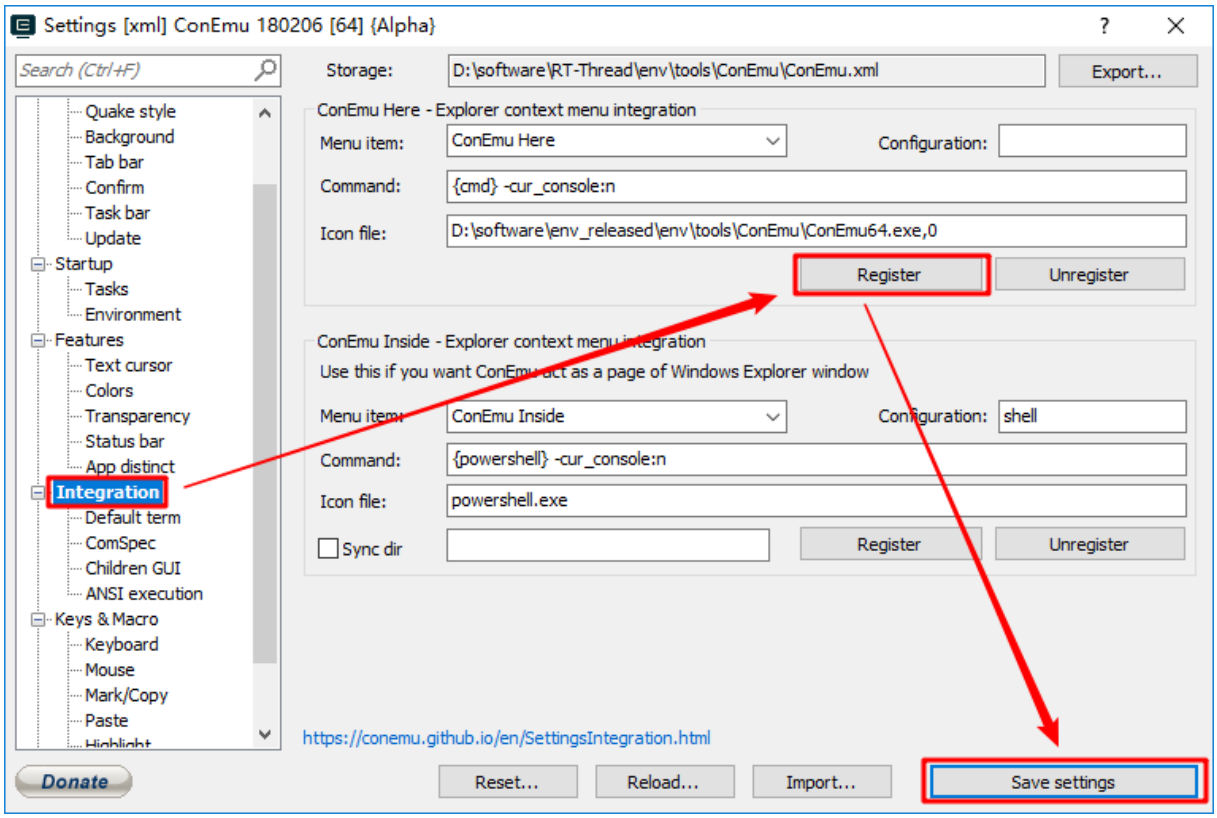


Figure 1.149 Add_Env_To_Right-click_Menu-2

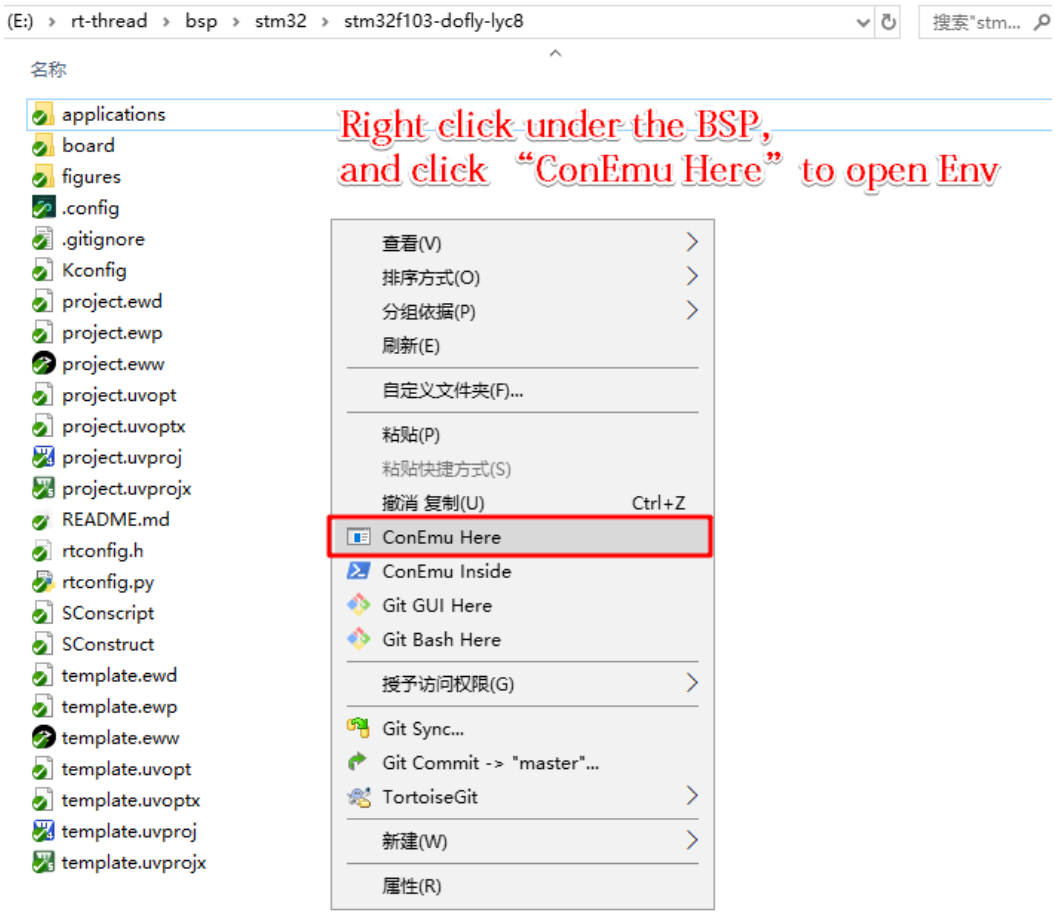


Figure 1.150 Add_Env_To_Right-click_Menu-3

Follow the steps from the image to launch the Env console from the right-click menu in any folder. Result are as follows:

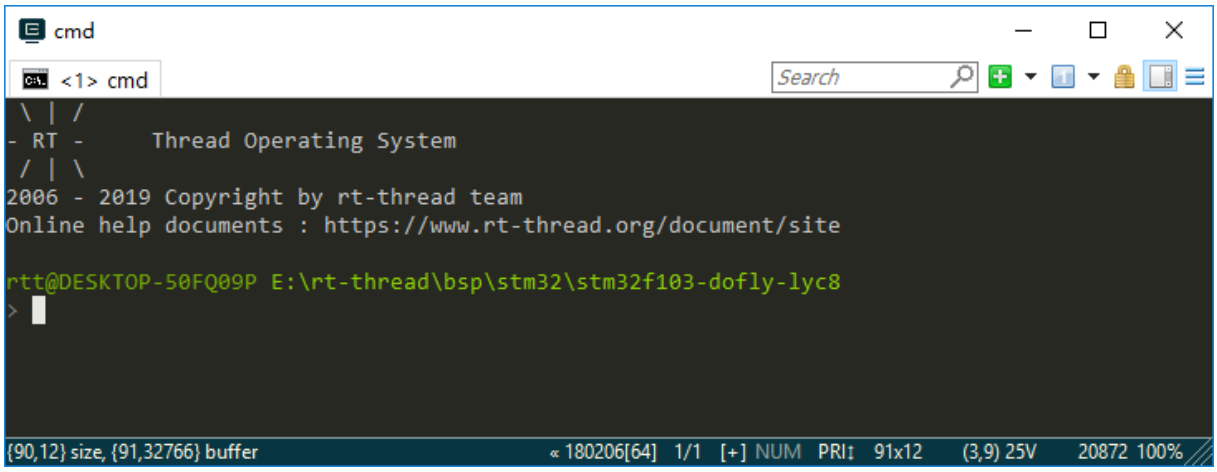


Figure 1.151 Right-click menu to launch the Env console

Due to the need for the environment variables of the Env process to be set, the anti-virus software may misreport at the first startup. If the anti-virus software misreport is encountered, allow Env related programs to run, and then add related programs to the white list.

1.5.1.3.1.2 Compile

Scons is a compile building tool used by RT-Thread to compile RT-Threads using the scons related commands.

Step One: Switch to the BSP root directory

If you use Method 1 to open Env console, you need to switch to the target BSP using the `cd` command.
For example, the target BSP is `rt-thread\bsp\stm32\stm32f103-dofly-lyc8`:

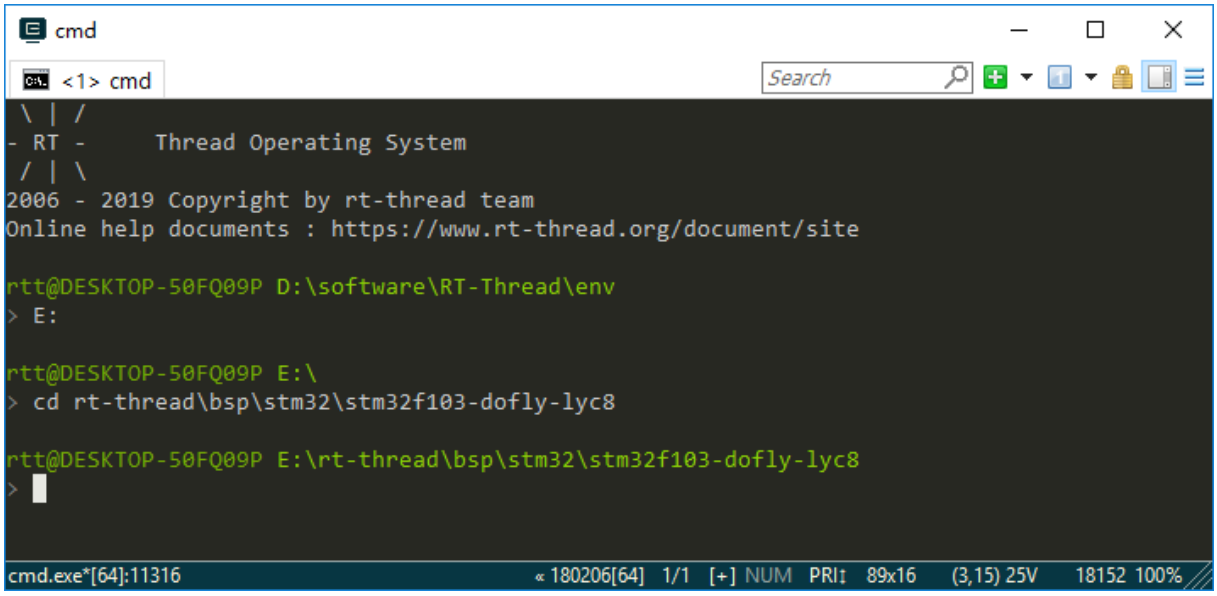


Figure 1.152 stm32f429-apollo project directory

Step Two: Compile the BSP

Env for Windows carries Python2.7 & Scons . To compile BSP, just use the default ARM_GCC toolchain by running `scons` or `scons -j12` (12 CPU cores compiling) command in the target BSP directory.
Env for Windows carries ARM GCC toolchain by default. When users compile ARM BSPs, users can directly type `scons` or `scons -j12` command to compile the BSP. However, if the BSP is other architecture, e.g. RISC-V, users need to use `scons --exec-path='C:\\xxx\\sdk-toolchain-RISC-V-GCC-WCH-1.0.0\\bin' -j12` command to specify the toolchain when compiling.

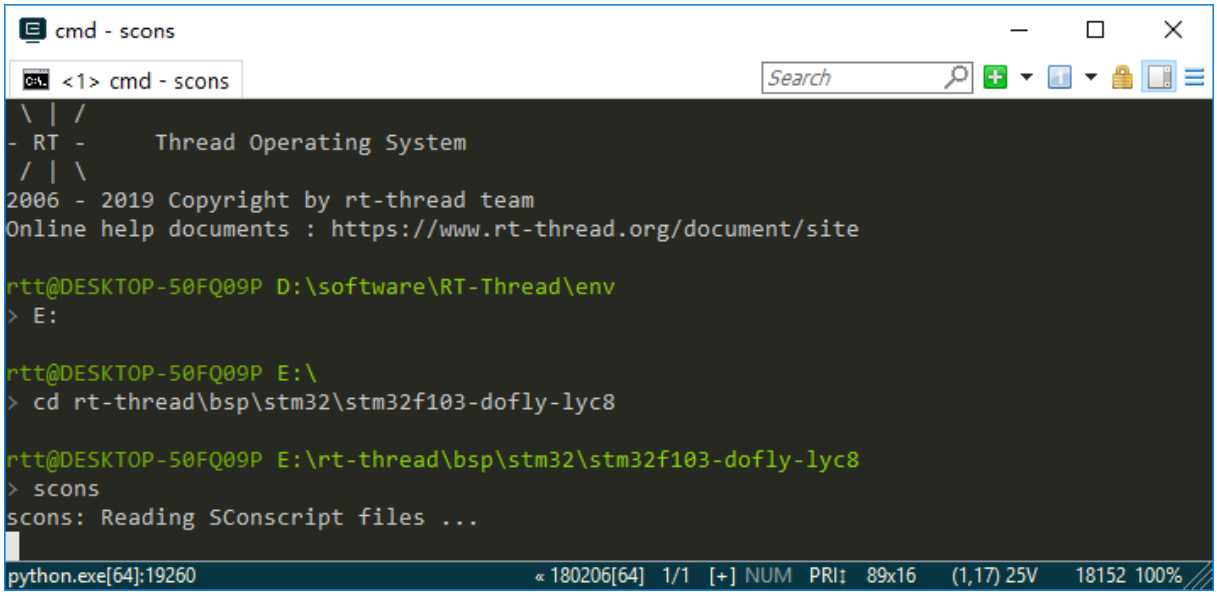


Figure 1.153 compilation project using scons command

Compiled successfully:

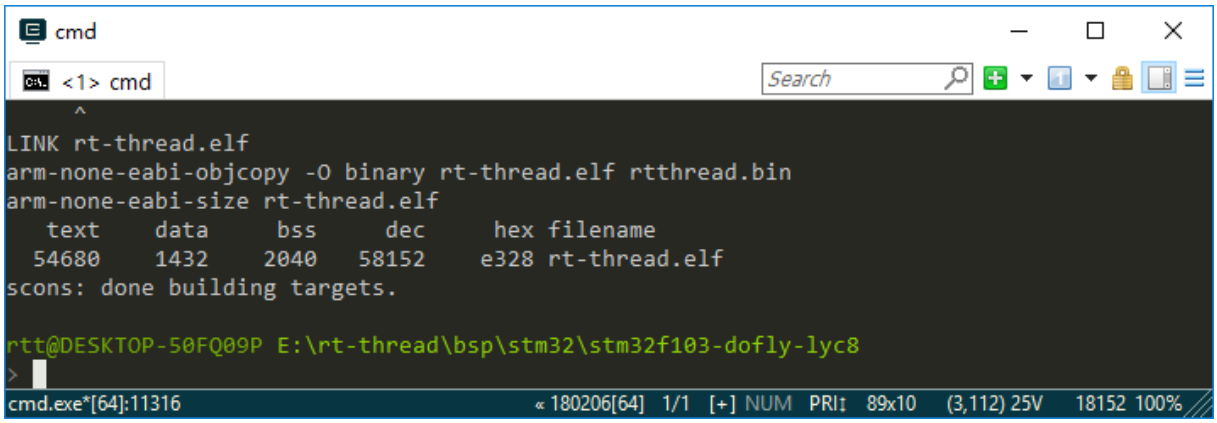


Figure 1.154 complied successfully

If you use mdk/iar for project development, you can use the project file in the BSP directly or use one of the following commands to regenerate the project and compile and download it.

```
scons --target=vsc
scons --target=iar
scons --target=mdk4
scons --target=mdk5
```

For more scons tutorials, please refer to SCons.

1.5.1.3.1.3 BSP configuration: menuconfig

Menuconfig is a graphical configuration tool that RT-Thread uses to configure and tailor the entire system.

Instruction for Shortcuts

Go to the BSP root directory and open the interface by entering `menuconfig`. The menuconfig common shortcuts are as shown:

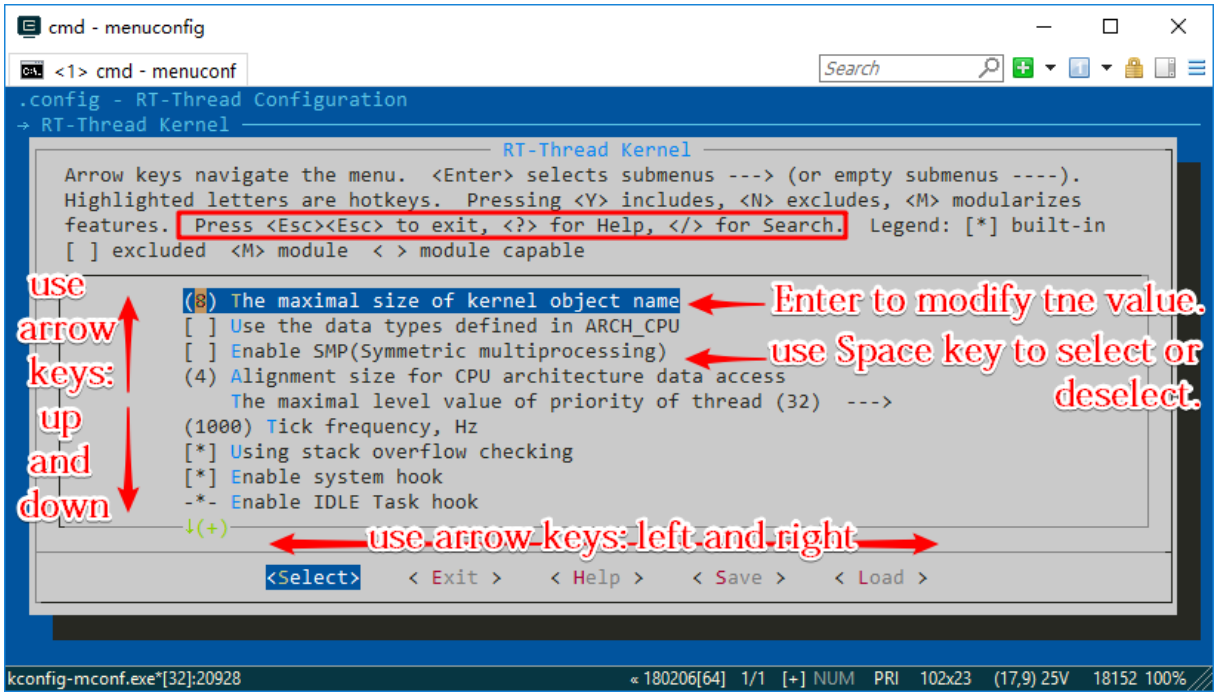


Figure 1.155 Commonly-used Shortcuts for menuconfig

Modify Settings

There are many types of configuration items in menuconfig, and the modification methods are different. The common types are:

- On/Off Type: Use the space bar to select or close
- Value, string type: After pressing the Enter key, a dialog box will appear, and the configuration items will be modified in the dialog box.

Save Settings

After selecting the configuration item, press `ESC` to exit, and select `Save` to automatically generate the `rtconfig.h` file. At this point, using the `scons` command again will recompile the project according to the new rtconfig.h file.

1.5.1.3.1.4 Package Management

RT-Thread provides a package management platform where the officially available or developer-supplied packages are stored. The platform provides developers with a choice of reusable software packages that are an important part of RT-Thread.
`Click here` to view the official RT-Thread package, most of which have detailed documentation and usage examples.
As a part of Env, the `package` tool provides developers with management functions such as downloading, updating, and deleting packages.
Enter the `pkgs` command on the Env command line to see an introduction to the command:

```
> pkgs
usage: env.py package [-h] [--update] [--update-force] [--list] [--wizard]
                      [--upgrade] [--upgrade-force] [--upgrade-script-force]
                      [--upgrade-modules] [--printenv]

optional arguments:
  -h, --help            show this help message and exit
  --update              update packages, install or remove the packages by
                        your settings in menuconfig
  --update-force, --force-update
                        forcely update and clean packages, install or remove
                        packages by settings in menuconfig
  --list                list target packages
  --wizard              create a new package with wizard
  --upgrade             upgrade local packages index from git repository
  --upgrade-force, --force-upgrade
                        forcely upgrade local packages index from git
                        repository
  --upgrade-script-force
                        forcely upgrade local packages index and Env script
                        from git repository
  --upgrade-modules     upgrade python modules, e.g. requests module
  --printenv            print environmental variables to check
```

Download, update, and delete packages

Before downloading and updating the software package, you need to **open** the target package in `menuconfig`. These packages locates in `RT-Thread online packages` , Once you enter the menu, you can see the following package categories:

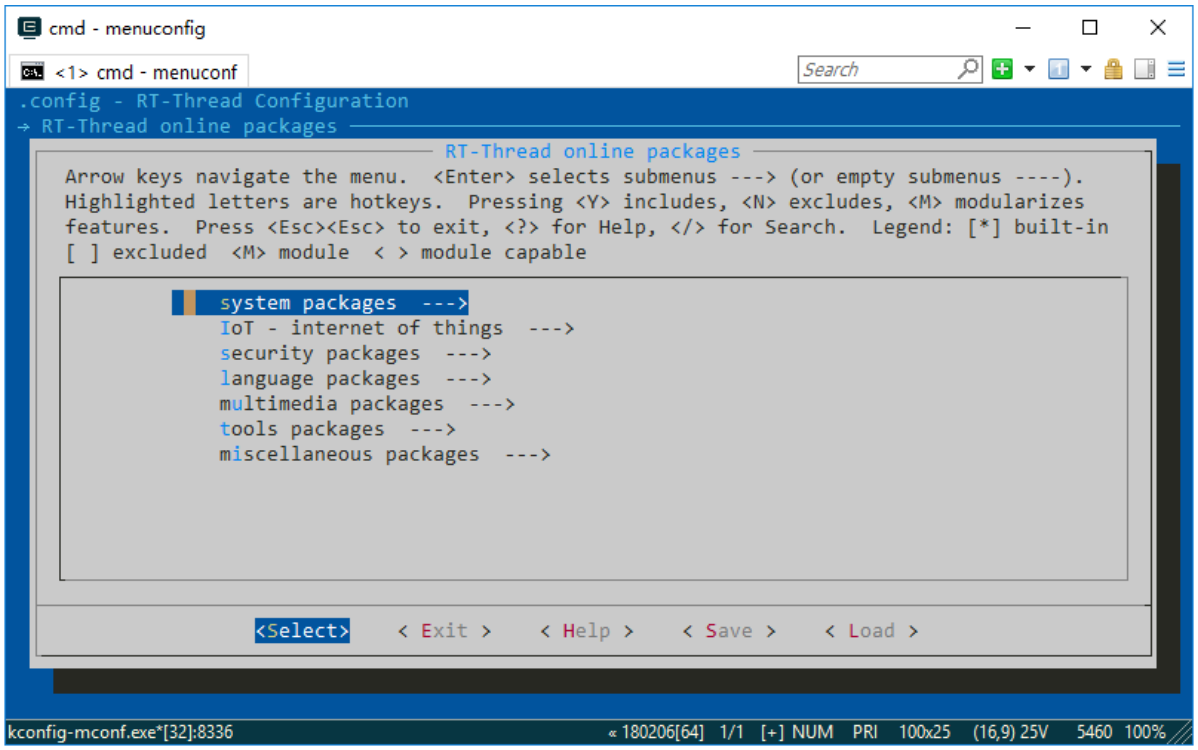


Figure 1.156 Package Categories

Find the package you need and open, then save and exit menuconfig. The package will be marked, but has not been downloaded locally, so it is still unavailable.

- **download**: if the software package is selected but not downloaded, enter: `pkgs --update`, then the software package will be downloaded automatically;
- **update**: if the selected package has a latest update on the server and the version is selected **latest**, then enter `pkgs --update` , the package will be updated in local;
- **delete**: if a software package is not needed, deselect it in menuconfig and then use `pkgs --update` command. Then locally downloaded but unselected packages will be deleted.

Update local package information and index

As the package system grows, more and more packages will be added, so the list of packages in menuconfig may be **unsynchronized** with the server. This can be fixed by using `pkgs --upgrade` or `pkgs --upgrade-force` command (recommended), which not only synchronizes updates to local software packages `information and index`, which are recommended for regular use. If there is a upgrade failure or merge failure, please use `pkgs --upgrade-force` command to force upgrade the local software packages index (recommended to use).
If wants to upgrade the `Env script` as well, please use `pkgs --upgrade-script-force` command. This command not only can upgrade the software packages index, but also can upgrade the Env script.

1.5.1.3.1.5 Env Tool Configuration

- The new version of the Env tool includes an automatic update package and an option to automatically generate mdk/iar projects. The default is not enabled. It can be configured using `menuconfig -s/--setting` .
- Use `menuconfig -s` command to enter the Env configuration interface

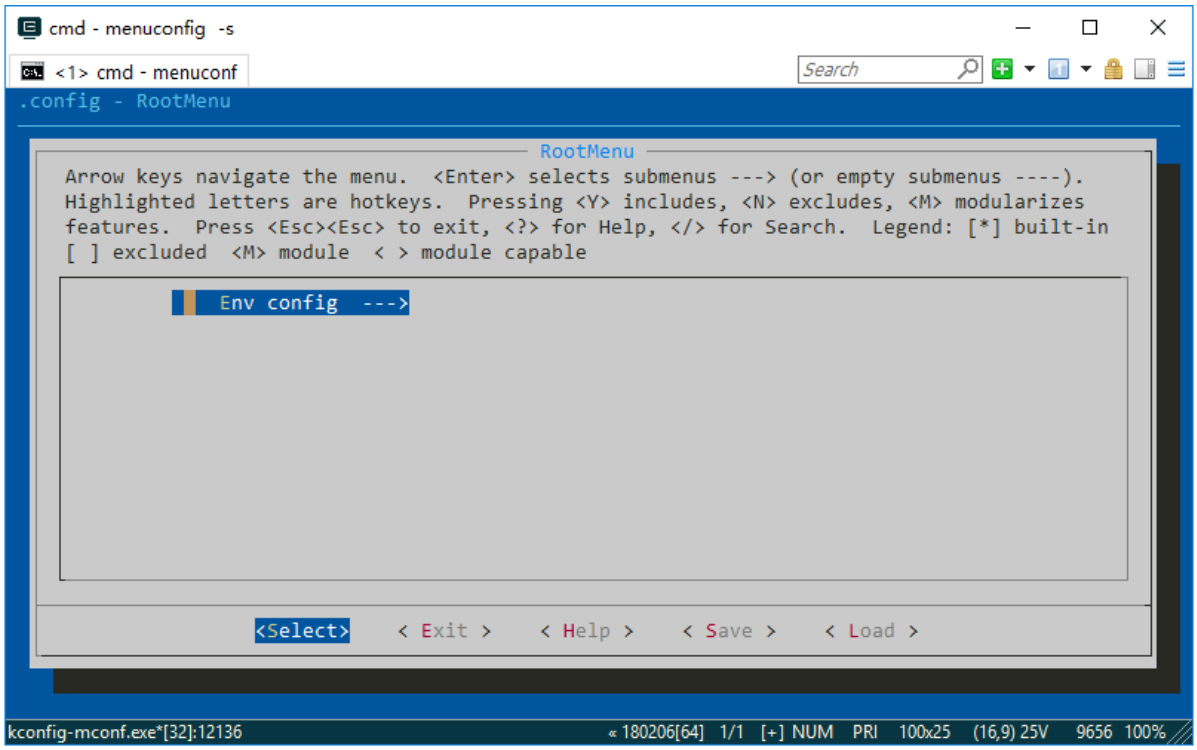


Figure 1.157 Env Configuration Interface

Press Enter to enter the configuration menu with 3 configuration options:

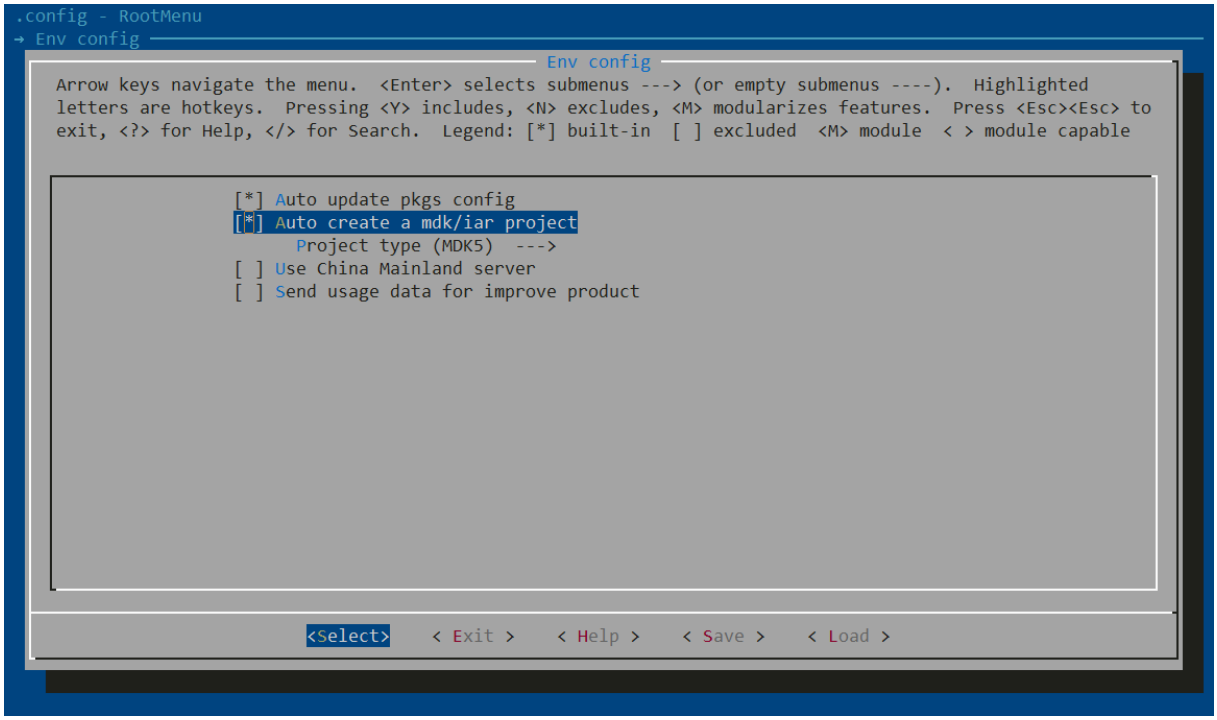


Figure 1.158 configuration options

The three options are:

- **Auto update pkgs config**Automatic package update function: After exiting the menuconfig function, `pkgs --update` command is automatically used to download and install the package and delete the old package. This feature is used when downloading online packages.**

- **Auto create a MDK/IAR project**: After modifying the menuconfig configuration, you must re-generate the project by typing `scons --target=xxx`. Turning on this feature will automatically regenerate the project when you exit menuconfig, without having to manually enter the scons command to regenerate the project.
- **Send usage data for improve product**: For user number statistic.

1.5.1.3.2 Use Env in Your Project

1.5.1.3.2.1 How to Modify Options in Menuconfig

If you want to add a macro definition in the configuration item of menuconfig, you can modify the `Kconfig` file under BSP. The modification method can search Kconfig syntax on the Internet for detailed documentation, or refer to the Kconfig file in RT-Thread or The Kconfig file in the BSP that supports menuconfig.

1.5.1.3.2.2 To Add menuconfig function to New Project

New project here refers to a newly developed project that has not yet generated `.config` and `rtconfig.h`. Because these two files are only created when menuconfig is first saved. The specific process is as follows:

1. Copy the kconfig file from the BSP that already supports the menuconfig function to the new project root directory.
2. Note that modifying the RTT_ROOT value in Kconfig is the directory where RT-Thread is located, otherwise RTT_ROOT may be not found.
3. Start the configuration with the menuconfig command.

1.5.1.3.2.3 To Add menuconfig function to old Project

Old project here refers to the development that has been going on for a while, and there is a modified `rtconfig.h` file in the project, but there is no project configured with menuconfig. The specific process is as follows:

1. First back up the rtconfig.h file in the old project.
2. Use `scons --genconfig` command to generate a `.config` file from the existing `rtconfig.h` file. The `.config` file generated here saves the configuration parameters of the `rtconfig.h` file in the old project.
3. Copy the `kconfig` file from the BSP that already supports the menuconfig function to the root directory of the project you want to modify.
4. Note that modifying the RTT_ROOT value in Kconfig is the directory where RT-Thread is located, otherwise RTT_ROOT may be not found.
5. Use the menuconfig command to configure the old project we want to modify. Menuconfig will read the `.config` file generated in the second step and generate a new `.config` file and rtconfig.h file based on the configuration parameters of the old project.
6. Check the old and new rtconfig.h files. If there are any inconsistencies, you can use the menuconfig command to adjust the configuration items.

1.5.1.3.3 Notes for Using Env

- For the first time, Env is recommended to go to the official website to download the latest version of the Env tool. The new version of Env will have better compatibility and also support automatic update commands.
- You can use the Env built-in command `pkgs --upgrade` or `pkgs --upgrade-force` (recommended to use) to update the package list and Env's function code to minimize the problems you have fixed.
- Do not have Chinese or spaces in the routes of Env.
- Do not have Chinese or spaces in the routes where the BSP project is located.

1.5.1.3.4 FAQs

1.5.1.3.4.1 Q: It prompts that the git command cannot be found.

'git' is not recognized as an internal or external command, possible program or batch file.
A: Git is not installed. You need to install git and add environment variables.

1.5.1.3.4.2 Q: It prompts that the CMD command cannot be found.

A: Right-click–>> Property—>> Advanced System Settings—>> Environment Variable, Add `C:\Windows\System32` to system environment variables.

1.5.1.3.4.3 Q: Prompt "no module named site" when running python.

A: Computer right button –>> Properties—>> Advanced System Settings—>> Environment Variable, in the user variable of the administrator, create a new variable named PYTHONHOME, and the variable value is: `F:\git_repositories\env\tools\Python27` the installation route of Python in Env, do not add ";" afterwards, otherwise it will be invalid. If add PYTHONHOME can not solve theproblem, try to add PYTHONPATH in the same way.

1.5.1.3.4.4 Q: What types of projects can I generate under Env?

A:

1. Currently, you can use the scons tool to generate mdk/iar projects under Env. There is no automatic generation of eclipse projects.
2. Generally, using gcc's toolchain, using an editor such as source insight or VS Code to edit the code and compile with `scons` command.

1.5.1.3.4.5 Q: How can my own BSP support menuconfig?

A: You can refer to this chapter **Use Env in Your Project**.

1.5.1.3.4.6 Q: What is the difference between the pkgs –upgrade/–upgrade-force command and the pkgs –update command?

A:

1. The `pkgs --upgrade/--upgrade-force` command is used to upgrade the Env script itself and the list of packages. You cannot select a recently updated package without the latest package list.
2. The `pkgs --update` command is used to update the package itself. For example, if you selected json and mqtt packages in menuconfig, you did not download them when you exit menuconfig. You need to use the `pkgs --update` command, at which point Env will download the package you selected and add it to your project.
3. The new version of Env supports the `menuconfig -s/--setting` command. If you don't want to use the `pkgs --update` command after replacing the package, configure Env after using the `menuconfig -s/--setting` command. Select each time you use menuconfig. After the package is automatically updated.

1.5.1.3.4.7 Q: Prompt "can't find file Kconfig" while using menuconfig.

A: The Kconfig file is missing from the current working BSP directory. Please refer *To Add menuconfig function to New Project* and *To Add menuconfig function to Old Project*.

1.5.1.3.4.8 Q: There's unintelligible texts appear in Env.

A: First check if there is a Chinese route.
Check if the `chcp` command has been added to the system environment variable and try to change the character format to English using the `chcp 437` command. If the prompt does not have a `chcp` command, it is considered not to be added to the environment variable.
The directory where the `chcp` command is located may be added to the environment variable in the `system32` directory.

1.5.2 SCons

1.5.2.1 Introduction to SCons

SCons is an open source build system written in the Python language, similar to GNU Make. It uses a different approach than the usual Makefile, but instead uses SConstruct and SConscript files instead. These files are also Python scripts that can be written using standard Python syntax, so the Python standard library can be called in SConstruct, SConscript files for a variety of complex processing, not limited to the rules set by the Makefile.
A detailed `SCons user manual` can be found on the SCons website. This section describes the basic usage of SCons and how to use the SCons tool in RT-Thread.

1.5.2.1.1 What is Construction Tool

A software construction tool is a piece of software that compiles source code into an executable binary program according to certain rules or instructions. This is the most basic and important feature of building tools. In fact, these are not the only functions of construction tools. Usually these rules have a certain syntax and are organized into files. These files are used to control the behavior of the build tool, and you can do other things besides software building.
The most popular build tool today is GNU Make. Many well-known open source software, such as the Linux kernel, are built using Make. Make detects the organization and dependencies of the file by reading the Makefile and completes the commands specified in the Makefile.
Due to historical reasons, the syntax of the Makefile is confusing, which is not conducive to beginners. In addition, it is not convenient to use Make on the Windows platform, you need to install the Cygwin environment. To overcome the shortcomings of Make, other build tools have been developed, such as CMake and SCons.

1.5.2.1.2 RT-Thread Construciton Tool

RT-Thread was built using Make/Makefile in the earlier stage. Starting from 0.3.x, the RT-Thread development team gradually introduced the SCons build system. The only goal of introducing SCons is to get everyone out of the complex Makefile configuration, IDE configuration, and focus on RT-Thread function development.
Some may doubt the difference between the build tools described here and the IDE. The IDE completes the build through the operation of the graphical interface. Most IDEs generate script files like Makefile or SConscript based on the source code added by the user, and call the tools like Make or SCons to build the source code.

1.5.2.1.3 Install SCons

It needs to be installed on the PC host before using the SCons system because it is written in the Python language, so you need to install the Python runtime environment before using SCons.
The Env configuration tool provided by RT-Thread comes with SCons and Python, so using SCons on Windows platforms does not require the installation of these two software.
In Linux and BSD environments, Python should already be installed by default, which is also the Python environment of the 2.x version. At this time, you only need to install SCons. For example, in Ubuntu you can install SCons with the following command:

```
sudo apt-get install scons
```

1.5.2.2 Basic Functions of SCons

The RT-Thread build system supports multiple compilers, including ARM GCC, MDK, IAR, VisualStudio, and Visual DSP. The mainstream ARM Cortex M0, M3, M4 platforms, basically all support ARM GCC, MDK, IAR. Some BSPs may only support one compiler, and readers can read the currently supported compiler by reading the CROSS_↩ TOOL option in rtconfig.py under the BSP directory.

If it is a chip on the ARM platform, you can use the Env tool and enter the scons command to compile the BSP directly. At this time, the ARM GCC compiler is used by default because the Env tool comes with the ARM GCC compiler. Compile a BSP using the scons command as shown below, and the SCons will be based on this BSP.

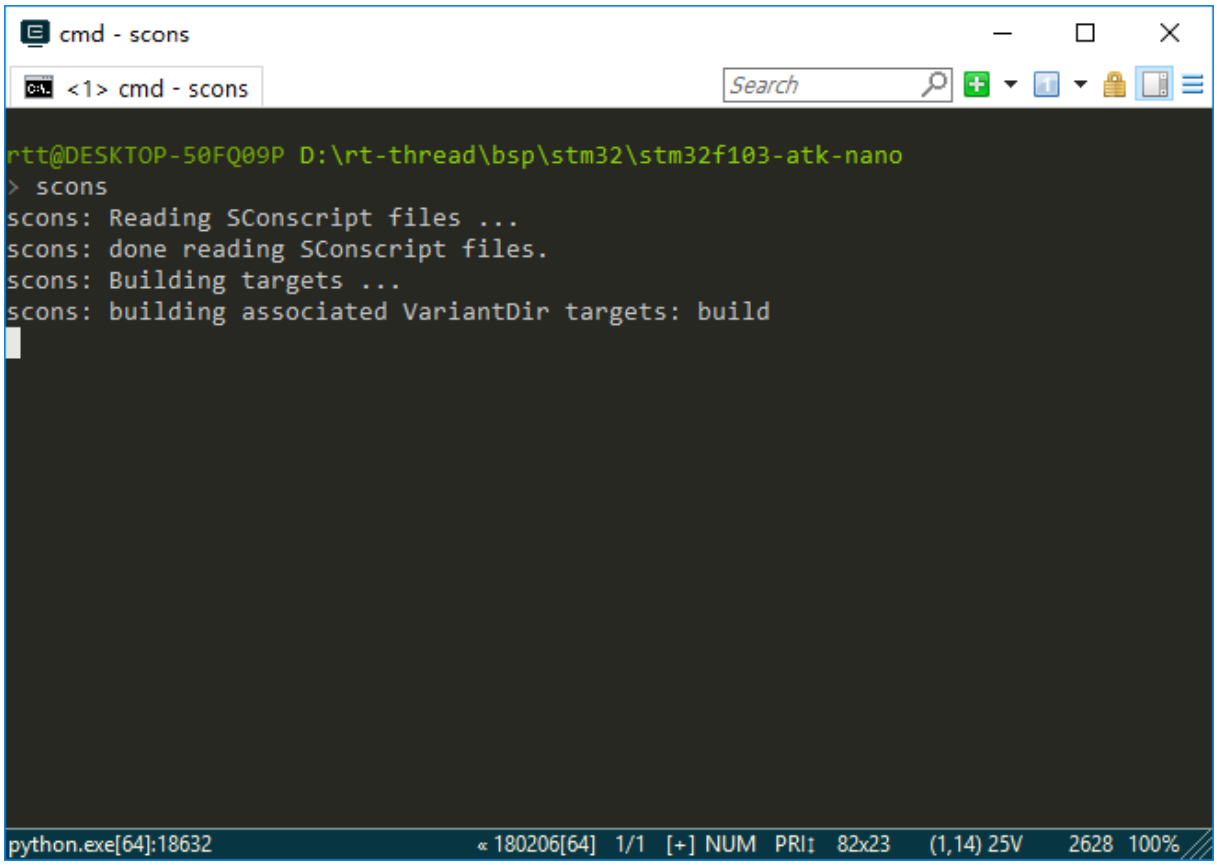


Figure 1.159 Compile BSP using scons

If the user wants to use another compiler that the BSP already supports to compile the project, or if the BSP is a non-ARM platform chip, then you can't compile the project directly with the scons command. You need to install the corresponding compiler yourself and specify the compiler path to use. Before compiling the project, you can use the following two commands in the Env command line interface to specify the compiler path for the MDK and the compiler path to MDK.

```
set RTT_CC=keil
set RTT_EXEC_PATH=C:/Keilv5
```

1.5.2.2.1 Commonly Used SCons Commands

This section describes the SCons commands that are commonly used in RT-Thread. SCons not only completes basic compilation, but also generates MDK/IAR/VS projects.

1.5.2.2.1.1 scons

Go to the BSP project directory to be compiled in the Env command line window, and then use this command to compile the project directly. If some source files are modified after executing the `scons` command, and the scons command is executed again, SCons will incrementally compile and compile only the modified source files and link them.
`scons` can also be followed by a `-s` parameter, the command `scons -s`, which differs from the `scons` command in that it does not print specific internal commands.

1.5.2.2.1.2 scons -c

Clear the compilation target. This command clears the temporary and target files generated when `scons` is executed.

1.5.2.2.1.3 scons –target=XXX

If you use mdk/iar for project development, when you open or close some components, you need to use one of the following commands to regenerate the corresponding customized project, then compile and download in mdk/iar.

```
scons --target=iar
scons --target=mdk4
scons --target=mdk5
```

In the Env command line window, enter the BSP project directory to be compiled. After using the `scons --target=mdk5` command, a new MDK project file named project.uvprojx will be generated in the BSP directory. Double-click it to open and you can use MDK to compile and debug. Using the `scons --target=iar` command will generate a new IAR project file named project.eww. Users who are not used to SCons can use this method. If project.uvproj fails to open, please delete project.uvopt and rebuild the project.
Under `bsp/simulator` directory , you can use the following command to generate a project for vs2012 or a project for vs2005.

```
scons --target=vs2012
Scons --target=vs2005
```

If you provide template files for other IDE projects in the BSP directory, you can also use this command to generate corresponding new projects, such as ua, vs, cb, cdk.
This command can also be followed by a `-s` parameter, such as the command `scons -target=mdk5 -s`, which does not print specific internal commands when executing this command.

> To generate a MDK or IAR project file, the prerequisite is that there is a project template file in the BSP directory, and then the scons will add relevant source code, header file search path, compilation parameters, link parameters, etc. according to the template file. As for which chip this project is for, it is directly specified by this engineering template file. So in most cases, this template file is an empty project file that is used to assist SCons in generating project.uvprojx or project.eww.

1.5.2.2.1.4 scons -jN

Multi-threaded compilation target, you can use this command to speed up compilation on multi-core computers. In general, a cpu core can support 2 threads. Use the `scons -j4` command on a dual-core machine.

> If you just want to look at compilation errors or warnings, it's best not to use the -j parameter so that the error message won't be mixed with multiple files in parallel.

1.5.2.2.1.5 scons –dist

Build a project framework. Using this command will generate the `dist` directory in the BSP directory, this is the directory structure of the development project, including RT-Thread source code and BSP related projects, irrelevant BSP folder and libcpu will be removed, and you can freely copy this work to any directory.

1.5.2.2.1.6 scons –verbose

By default, output compiled with the scons command does not display compilation parameters as follows:

```
D:\repository\rt-thread\bsp\stm32f10x>scons
scons: Reading SConscript files ...
scons: done reading SConscript files.
scons: Building targets ...
scons: building associated VariantDir targets: build
CC build\applications\application.o
CC build\applications\startup.o
CC build\components\drivers\serial\serial.o
...
```

The effect of using the scons –verbose command is as follows:

```
armcc -o build\src\mempool.o -c --device DARMSTM --apcs=interwork -ID:/Keil/ARM/
RV31/INC -g -O0 -DUSE_STDPERIPH_DRIVER -DSTM32F10X_HD -Iapplications -IF:\Projec
t\git\rt-thread\applications -I. -IF:\Project\git\rt-thread -Idrivers -IF:\Proje
ct\git\rt-thread\drivers -ILibraries\STM32F10x_StdPeriph_Driver\inc -IF:\Project
\git\rt-thread\Libraries\STM32F10x_StdPeriph_Driver\inc -ILibraries\STM32_USB-FS
-Device_Driver\inc -IF:\Project\git\rt-thread\Libraries\STM32_USB-FS-Device_Driv
er\inc -ILibraries\CMSIS\CM3\DeviceSupport\ST\STM32F10x -IF:\Project\git\rt-thre
...
```

1.5.2.3 SCons Advanced

SCons uses SConscript and SConstruct files to organize the source structure. Usually a project has only one SConstruct, but there will be multiple SConscripts. In general, an SConscript will be placed in each subdirectory

where the source code is stored.
In order to make RT-Thread better support multiple compilers and to easily adjust compilation parameters, RT-Thread creates a separate file for each BSP called `rtconfig.py`. So the following three files exist in every RT-Thread BSP directory: `rtconfig.py`, `SConstruct`, and `SConscript`, which control the compilation of the BSP. There is only one SConstruct file in a BSP, but there are multiple SConscript files. It can be said that the SConscript file is the main force of the organization source code.
RT-Thread SConscript files are also present in most source folders. These files are "found" by the SConscript file in the BSP directory to add the source code corresponding to the macro defined in rtconfig.h to the compiler. The following article will take stm32f10x-HAL BSP as an example to explain how SCons builds the project.

1.5.2.3.1 SCons Build-In Functions

If you want to add some of your own source code to the SCons build environment, you can usually create or modify an existing SConscript file. The SConscript file can control the addition of source files and can specify the group of files (similar to the concept of Groups in IDEs such as MDK/IAR).
SCons provides a lot of built-in functions to help us quickly add source code, and with these simple Python statements we can add or remove source code to our project. The following is a brief introduction to some common functions.

1.5.2.3.1.1 GetCurrentDir()

Get current directory.

1.5.2.3.1.2 Glob('*.c')

Get all C files in the current directory. Modify the value of the parameter to match the suffix to match all files of the current directory.

1.5.2.3.1.3 GetDepend(macro)

This function is defined in the script file in the `tools` directory. It reads the configuration information from the rtconfig.h file with the macro name in rtconfig.h. This method (function) returns true if rtconfig.h has a macro turned on, otherwise it returns false.

1.5.2.3.1.4 Split(str)

Split the string str into a list list.

1.5.2.3.1.5 DefineGroup(name src depend**parameters)

This is a method (function) of RT-Thread based on the SCons extension. DefineGroup is used to define a component. A component can be a directory (under a file or subdirectory) and a Group or folder in some subsequent IDE project files.
Parameter description of `DefineGroup()` :

Parameter	**D**escription
name	name of Group
src	The files contained in the Group generally refer to C/C++ source files. For convenience, you can also use the Glob function to list the matching files in the directory where the SConscript file is located by using a wildcard.
depend	The options that the Group depends on when compiling (for example, the FinSH component depends on the RT_USING_FINSH macro definition). The compile option generally refers to the RT_USING_xxx macro defined in rtconfig.h. When the corresponding macro is defined in the rtconfig.h configuration file, then this group will be added to the build environment for compilation. If the dependent macro is not defined in rtconfig.h, then this Group will not be added to compile. Similarly, when using scons to generate as an IDE project file, if the dependent macros are not defined, the corresponding Group will not appear in the project file.

Parameter	**D**escription
parameters	Configure other parameters, the values can be found in the table below. You do not need to configure all parameters in actual use.

parameters that could be added

Parameter	Description
CCFLAGS	source file compilation parameters
CPPPATH	head file path
CPPDEFINES	Link parameter
LIBRARY	Include this parameter, the object file generated by the component will be packaged into a library file

1.5.2.3.1.6 SConscript(dirsvariant_dirduplicate)

Read the new SConscript file, and the parameter description of the SConscript() function is as follows:

Parameter	Description
dirs	SConscript file path
variant_dir	Specify the path to store the generated target file
duiplicate	Set whether to copy or link the source file to variant_dir

1.5.2.4 SConscript Examples

Below we will use a few SConscript as an example to explain how to use the scons tool.

1.5.2.4.1 SConscript Example 1

Let's start with the SConcript file in the stm32f10x-HAL BSP directory. This file manages all the other SConscript files under the BSP, as shown below.

```
import os
cwd = str(Dir('#'))
objs = []
list = os.listdir(cwd)
for d in list:
    path = os.path.join(cwd d)
       if os.path.isfile(os.path.join(path 'SConscript')):
        objs = objs + SConscript(os.path.join(d 'SConscript'))
Return('objs')
```

- `import os`: Importing the Python system programming os module, you can call the functions provided by the os module to process files and directories.
- `cwd = str(Dir('#'))` Get the top-level directory of the project and assign it to the string variable cwd, which is the directory where the project's SConstruct is located, where it has the same effect as `cwd = GetCurrentDir()`.
- `objs = []`: An empty list variable objs is defined.
- `list = os.listdir(cwd)` Get all the subdirectories under the current directory and save them to the variable list.
- This is followed by a python for loop that walks through all the subdirectories of the BSP and runs the SConscript files for those subdirectories. The specific operation is to take a subdirectory of the current directory, use `os.path.join(cwdd)` to splicing into a complete path, and then determine whether there is a file named SConscript in this subdirectory. If it exists, execute `objs = objs + SConscript(os.path.join(d'SConscript'))`. This sentence uses a built-in function `SConscript()` provided by SCons, which can read in a new SConscript file and add the source code specified in the SConscript file to the source compilation list objs.

With this SConscript file, the source code required by the BSP project is added to the compilation list.

1.5.2.4.2 SConscript Example 2

So what about stm32f10x-HAL BSP other SConcript files? Let's take a look at the SConcript file in the drivers directory, which will manage the source code under the drivers directory. The drivers directory is used to store the underlying driver code implemented by the driver framework provided by RT-Thread.

```
Import('rtconfig')
from building import *

cwd = GetCurrentDir()

# add the general drivers.
src = Split("""
board.c
stm32f1xx_it.c
""")

if GetDepend(['RT_USING_PIN']):
    src += ['drv_gpio.c']
if GetDepend(['RT_USING_SERIAL']):
    src += ['drv_usart.c']
if GetDepend(['RT_USING_SPI']):
    src += ['drv_spi.c']
if GetDepend(['RT_USING_USB_DEVICE']):
    src += ['drv_usb.c']
if GetDepend(['RT_USING_SDCARD']):
    src += ['drv_sdcard.c']

if rtconfig.PLATFORM in ['gcc']:
    src += ['gcc_startup.s']

CPPPATH = [cwd]

group = DefineGroup('Drivers', src, depend = [''], CPPPATH = CPPPATH)

Return('group')
```

- `Import('rtconfig')` Import the rtconfig object, and the rtconfig.CROSS_TOOL used later is defined in this rtconfig module.
- `from building import *` All the contents of the building module are imported into the current module, and the DefineGroup used later is defined in this module.
- `cwd = GetCurrentDir()` Get the current path and save it to the string variable cwd.

The next line uses the `Split()` function to split a file string into a list, the effect of which is equivalent to `src = ['board.c''stm32f1xx_it.c']`
Later, `if` judgement and `GetDepend ()` are used to check whether a macro in `rtconfig.h` is open or not, and if so, `src += [src_name]` is used to append the source code file to the list variable src.

- `CPPPATH = [cwd]`: Save the current path to a list variable CPPPATH.

The last line uses DefineGroup to create a group called Drivers, which corresponds to the grouping in the MDK or IAR. The source code file for this group is the file specified by src, and the dependency is empty to indicate that the group does not depend on any macros of rtconfig.h.
`CPPPATH =CPPPATH` means to add the current path to the system's header file path. The CPPPATH on the left is a built-in parameter in the DefineGroup that represents the header file path. The CPPPATH on the right is defined in the previous line of this document. This way we can reference the header files in the drivers directory in other source code.

1.5.2.4.3 SConscript Example 3

Let's take a look at the SConcript file in the applications directory, which will manage the source code under the applications directory for the user's own application code.

```
from building import *

cwd = GetCurrentDir()
src = Glob('*.c')
CPPPATH = [cwd, str(Dir('#'))]

group = DefineGroup('Applications', src, depend = [''], CPPPATH = CPPPATH)

Return('group')
```

`src = Glob('*.c')` Get all the C files in the current directory.
`CPPPATH = [cwd, str(Dir('#'))]` Save the current path and the path of the project's SConstruct to the list variable CPPPATH.
The last line uses DefineGroup to create a group called `Applications`. The source code file for this group is the file specified by src. If the dependency is empty, the group does not depend on any rtconfig.h macros, and the path saved by CPPPATH is added to the system header search path. Such application directories and header files in the stm32f10x-HAL BSP directory can be referenced elsewhere in the source code.
To sum up, this source program will add all c programs in the current directory to the group `Applications`, so if you add or delete files in this directory, you can add files to the project or delete them from the project. It is suitable for adding source files in batches.

1.5.2.4.4 SConscript Example 4

Below is the contents of the RT-Thread source code `component/finsh/SConscript` file, which will manage the source code under the finsh directory.

```
Import('rtconfig')
from building import *

cwd     = GetCurrentDir()
src     = Split('''
shell.c
symbol.c
cmd.c
''')

fsh_src = Split('''
finsh_compiler.c
finsh_error.c
finsh_heap.c
finsh_init.c
finsh_node.c
finsh_ops.c
finsh_parser.c
finsh_var.c
finsh_vm.c
finsh_token.c
''')

msh_src = Split('''
msh.c
msh_cmd.c
msh_file.c
''')

CPPPATH = [cwd]
if rtconfig.PLATFORM in ['armcc', 'armclang']:
    LINKFLAGS = '--keep *.o(FSymTab)'

    if not GetDepend('FINSH_USING_MSH_ONLY'):
        LINKFLAGS = LINKFLAGS + '--keep *.o(VSymTab)'
else:
    LINKFLAGS = ''

if GetDepend('FINSH_USING_MSH'):
    src = src + msh_src
if not GetDepend('FINSH_USING_MSH_ONLY'):
    src = src + fsh_src

group = DefineGroup('finsh', src, depend = ['RT_USING_FINSH'], CPPPATH = CPPPATH, LINKFLAGS = LINKFLAGS)

Return('group')
```

Let's take a look at the contents of the first Python conditional statement in the file. If the compilation tool is keil, the variable `LINKFLAGS = '--keep *.o(FSymTab)'` left blank.
DefinGroup also creates the file specified by src in the finsh directory as a finsh group. `depend = ['RT_↩ USING_FINSH']` indicates that this group depends on the macro RT_USING_FINSH in `rtconfig.h`. When the macro RT_USING_FINSH is opened in rtconfig.h, the source code in the finsh group will be compiled, otherwise SCons will not compile.
Then add the finsh directory to the system header directory so that we can reference the header files in the finsh directory in other source code.
`LINKFLAGS = LINKFLAGS` has the same meaning as `CPPPATH = CPPPATH` . The LINKFLAGS on the left represents the link parameter, and the LINKFLAGS on the right is the value defined by the previous if else statement. That is, specify the link parameters for the project.

1.5.2.5 Manage Projects with SCons

The previous section on the SConscript related to the RT-Thread source code is explained in detail, you should also know some common ways to write SConscript files, this section will guide you how to use SCons to manage your own projects.

1.5.2.5.1 Add App Code

As mentioned earlier, the Applications folder under BSP is used to store the user's own application code. Currently there is only one main.c file. If the user's application code is not a lot, it is recommended that the relevant source files be placed under this folder. Two simple files hello.c and hello.h have been added under the Applications folder, as shown below.

```
/* file: hello.h */

#ifndef _HELLO_H_
#define _HELLO_H_

int hello_world(void);

#endif /* _HELLO_H_ */

/* file: hello.c */
#include <stdio.h>
#include <finsh.h>
#include <rtthread.h>

int hello_world(void)
{
    rt_kprintf("Hello, world!\n");

    return 0;
}

MSH_CMD_EXPORT(hello_world, Hello world!)
```

The SConcript file in the applications directory will add all source files in the current directory to the project. You need to use the `scons --target=xxx` command to add the 2 new files to your project. Note that the project will be regenerated each time a new file is added.

1.5.2.5.2 Add Module

As mentioned above, in the case that there are not many source code files, it is recommended that all source code files be placed in the applications folder. If the user has a lot of source code and wants to create your own project module, or need to use other modules that you have obtained, what would be appropriate?
Take also hello.c and hello.h mentioned above for example, these two files will be managed in a separate folder, and have their own grouping in the MDK project file, and can be selected through menuconfig whether to use This module. Add a hello folder under BSP.

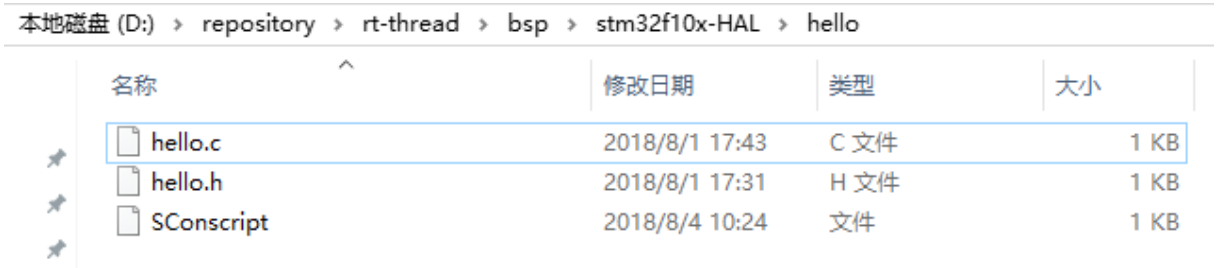


Figure 1.160 New added hello folder

Noticed that there is an additional SConscript file in the folder. If you want to add some of your own source code to the SCons build environment, you can usually create or modify an existing SConscript file. Refer to the above analysis of the SConscript file for the RT-Thread source code. The contents of this new hello module SConscript file are as follows:

```
from building import *

cwd          = GetCurrentDir()
include_path = [cwd]
```

```
src          = []

if GetDepend(['RT_USING_HELLO']):
    src += ['hello.c']

group = DefineGroup('hello', src, depend = [''], CPPPATH = include_path)

Return('group')
```

Through the above simple lines of code, a new group hello is created, and the source file to be added to the group can be controlled by the macro definition, and the directory where the group is located is added to the system header file path. So how is the custom macro RT_USING_HELLO defined? Here is a new file called Kconfig. Kconfig is used to configure the kernel, and the configuration interface generated by the menuconfig command used when configuring the system with Env relies on the Kconfig file. The menuconfig command generates a configuration interface for the user to configure the kernel by reading the various Kconfig files of the project. Finally, all configuration-related macro definitions are automatically saved to the rtconfig.h file in the BSP directory. Each BSP has an rtconfig.h file. That is the configuration information of this BSP.

There is already a Kconfig file for this BSP in the stm32f10x-HAL BSP directory, and we can add the configuration options we need based on this file. The following configuration options have been added to the hello module. The # sign is followed by a comment.

```
menu "hello module"                             # create a "hello module" menu

    config RT_USING_HELLO                       # RT_USING_HELLO configuration options
        bool "Enable hello module"              # RT_USING_HELLO is a bool variable and display as "Enable
      hello module"
        default y                               # RT_USING_HELLO can take values y and n, default y
        help                                    # If use help, it would display "this hello module only used
      for test"
        this hello module only used for test

    config RT_HELLO_NAME                        # RT_HELLO_NAME configuration options
        string "hello name"                     # RT_HELLO_NAME is a string variable and the menu show as
      "hello name"
        default "hello"                         # default name is "hello"

    config RT_HELLO_VALUE                       # RT_HELLO_VALUE configuration options
        int "hello value"                       # RT_HELLO_VALUE is an int variable and the menu show as
      "hello value"
        default 8                               # the default value is 8

endmenu                                         # the hello menu is end
```

After entering the stm32f10x-HAL BSP directory using the Env tool, you can use the menuconfig command to see the configuration menu of the new hello module at the bottom of the main page. After entering the menu, the following figure is displayed.

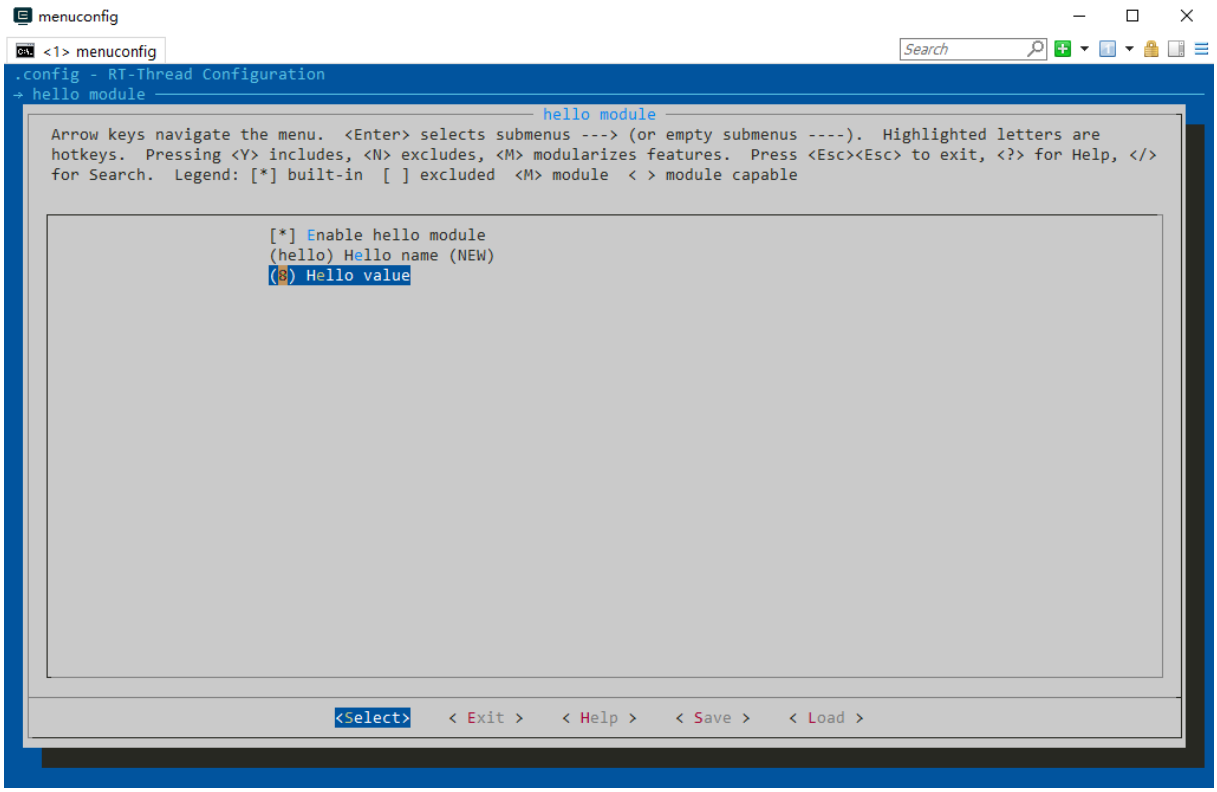


Figure 1.161 hello Module Configuration Menu

You can also modify the value of the hello value.

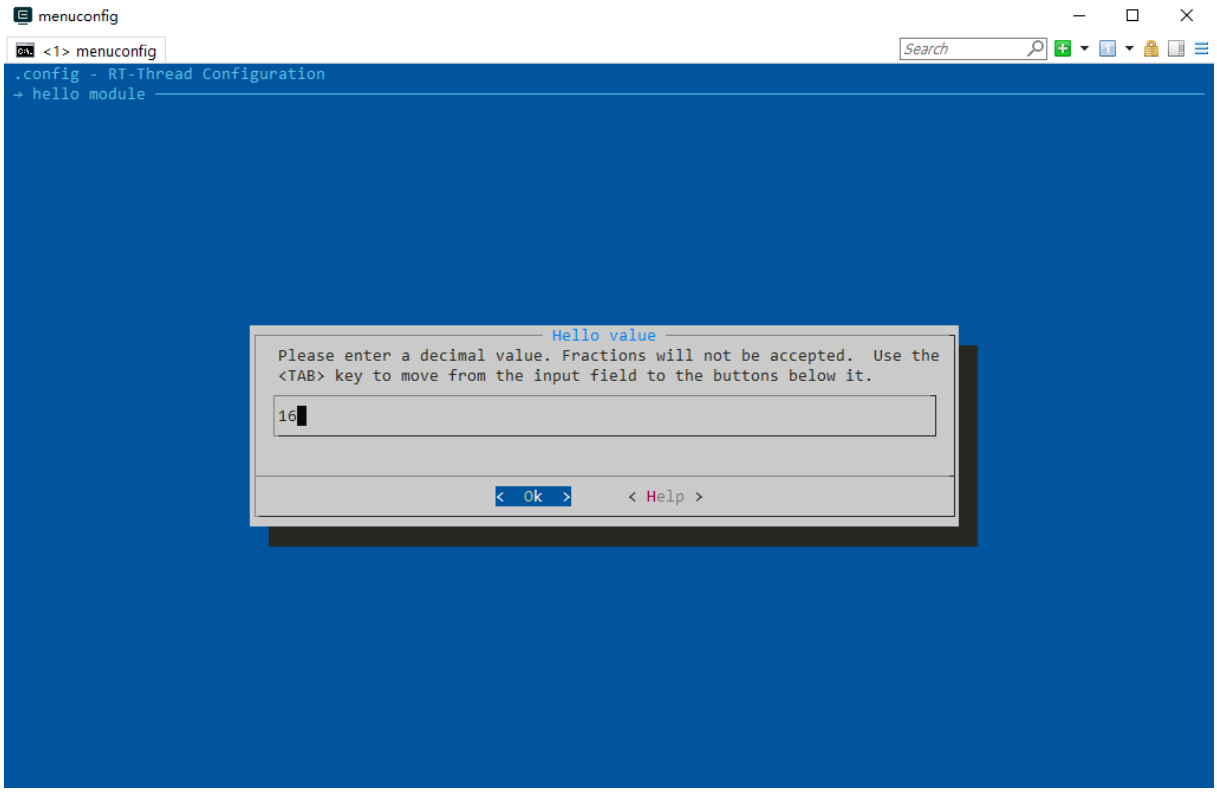


Figure 1.162 Modify the Value of Hello Value

After saving the configuration, exit the configuration interface and open the rtconfig.h file in the stm32f10x-HAL BSP directory. You can see that the configuration information of the hello module is available.

rtconfig.h

```
148 #define RT_USING_UART1
149 #define RT_USING_UART2
150
151 /* hello module */
152
153 #define RT_USING_HELLO
154 #define RT_HELLO_NAME "hello"
155 #define RT_HELLO_VALUE 16
156
157 #endif
158
```

Figure 1.163 hello Module Related Macro Definition

Note: Use the scons –target=XXX command to generate a new project each time menuconfig is configured. Because the RT_USING_HELLO macro has been defined in rtconfig.h, the source file for hello.c is added to the new project when the project is newly built.
The above simply enumerates the configuration options for adding your own modules to the Kconfig file. Users can also refer to Env, which also explains how to modify and add configuration options. They can also view the Kconfig documentation in your own Baidu to implement other more complex configuration options.

1.5.2.5.3 Add Library

If you want to add an extra library to your project, you need to pay attention to the naming of the binary library by different toolchains. For example, the GCC toolchain, which recognizes library names such as `libabc.a`, specifies `abc` instead of `libabc` when specifying a library. So you need to pay special attention to the SConscript file when linking additional libraries. In addition, when specifying additional libraries, it is also a good idea to specify the corresponding library search path. Here is an example:

```
Import('rtconfig')
from building import *

cwd = GetCurrentDir()
src = Split("'
"')

LIBPATH = [cwd + '/libs']
LIBS = ['abc']

group = DefineGroup('ABC' src depend = [''] LIBS = LIBS LIBPATH=LIBPATH)
```

LIBPATH specifies the path to the library, and LIBS specifies the name of the library. If the toolchain is GCC, the name of the library should be libabc.a; if the toolchain is armcc, the name of the library should be abc.lib.`LIBPATH = [cwd + '/libs']` indicates that the search path for the library is the 'libs' directory in the current directory.

1.5.2.5.4 Compiler Options

`rtconfig.py` is a RT-Thread standard compiler configuration file that controls most of the compilation options. It is a script file written in Python and is used to do the following:

- Specify the compiler (choose one of the multiple compilers you support).
- Specify compiler parameters such as compile options, link options, and more.

When we compile the project using the scons command, the project is compiled according to the compiler configuration options of `rtconfig.py`. The following code is part of the rtconfig.py code in the stm32f10x-HAL BSP

directory.

```
import os

# toolchains options
ARCH='arm'
CPU='cortex-m3'
CROSS_TOOL='gcc'

if os.getenv('RTT_CC'):
    CROSS_TOOL = os.getenv('RTT_CC')

# cross_tool provides the cross compiler
# EXEC_PATH is the compiler execute path, for example, CodeSourcery, Keil MDK, IAR

if  CROSS_TOOL == 'gcc':
    PLATFORM    = 'gcc'
    EXEC_PATH   = '/usr/local/gcc-arm-none-eabi-5_4-2016q3/bin/'
elif CROSS_TOOL == 'keil':
    PLATFORM    = 'armcc'
    EXEC_PATH   = 'C:/Keilv5'
elif CROSS_TOOL == 'iar':
    PLATFORM    = 'iccarm'
    EXEC_PATH   = 'C:/Program Files/IAR Systems/Embedded Workbench 6.0 Evaluation'

if os.getenv('RTT_EXEC_PATH'):
    EXEC_PATH = os.getenv('RTT_EXEC_PATH')

BUILD = 'debug'

if PLATFORM == 'gcc':
    # toolchains
    PREFIX = 'arm-none-eabi-'
    CC = PREFIX + 'gcc'
    AS = PREFIX + 'gcc'
    AR = PREFIX + 'ar'
    LINK = PREFIX + 'gcc'
    TARGET_EXT = 'elf'
    SIZE = PREFIX + 'size'
    OBJDUMP = PREFIX + 'objdump'
    OBJCPY = PREFIX + 'objcopy'

    DEVICE = '-mcpu=cortex-m3 -mthumb -ffunction-sections -fdata-sections'
    CFLAGS = DEVICE
    AFLAGS = '-c' + DEVICE + '-x assembler-with-cpp'
    LFLAGS = DEVICE + '-Wl,--gc-sections,-Map=rtthread-stm32.map,-cref,-u,Reset_Handler -T stm32_rom.ld'
```

Where CFLAGS is the compile option for C files, AFLAGS is the compile option for assembly files, and LFLAGS is the link option. The BUILD variable controls the level of code optimization. The default BUILD variable takes the value 'debug', which is compiled using debug mode, with an optimization level of 0. If you modify this variable to something else, it will compile with optimization level 2. The following are all possible ways to write (in short, no 'debug').

```
BUILD = ''
BUILD = 'release'
BUILD = 'hello world'
```

It is recommended to use the debug mode to compile during the development phase, without optimization, and then consider optimization after the product is stable.

The specific meaning of these options needs to refer to the compiler manual, such as `armcc` used above is the underlying compiler of MDK. The meaning of its compile options is detailed in MDK help.

As mentioned earlier, if the user wants to compile the project with another compiler when executing the scons command, you can use the relevant commands to specify the compiler and compiler paths on the command line side of Env. However, this modification is only valid for the current Env process. When you open it again, you need to re-use the command settings. We can directly modify the `rtconfig.py` file to achieve the purpose of permanently configuring the compiler. In general, we only need to modify the CROSS_TOOL and EXEC_PATH options below.

- CROSS_TOOLSpecify the compiler. The optional values are keil, gcc, iar. Browse `rtconfig.py` to see the compilers supported by the current BSP. If MDK is installed on your machine, you can modify CROSS_TOOL to keil and use MDK to compile the project.
- EXEC_PATH: The installation path of the compiler. There are two points to note here

(1) When installing the compiler (such as MDK, GNU GCC, IAR, etc.), do not install it in a path with Chinese or spaces. Otherwise, some errors will occur when parsing the path. Some programs are installed by default into the `C:\Program Files` directory with spaces in between. It is recommended to choose other paths during installation to develop good development habits.

(2) When modifying EXEC_PATH, you need to pay attention to the format of the path. On Windows platforms, the default path splitter is the backslash "\", which is used for escape characters in both C and Python. So when modifying the path, you can change "\" to "/"or add r (python-specific syntax for raw data).
Suppose a compiler is installed under `D:\Dir1\Dir2`. The following are the correct ways to write:

- EXEC_PATH = `r'D:\\Dir1\\Dir2'` Note that with the string `r` in front of the string, "\"can be used normally.
- EXEC_PATH = `'D:/Dir1/Dir2'` Note that instead of "/", there is no `r` in front.
- EXEC_PATH = `'D:\\Dir1\\Dir2'` Note that the escapement of "\" s used here to escape "\" itself.
- This is the wrong way to write: EXEC_PATH = `'D:\\Dir1\\Dir2'`

If the rtconfig.py file has the following code, comment out the following code when configuring your own compiler.

```
if os.getenv('RTT_CC'):
    CROSS_TOOL = os.getenv('RTT_CC')
... ...
if os.getenv('RTT_EXEC_PATH'):
    EXEC_PATH = os.getenv('RTT_EXEC_PATH')
```

The above 2 `if` judgments will set CROSS_TOOL and EXEC_PATH to the default value of Env.
After the compiler is configured, we can use SCons to compile the BSP of RT-Thread. Open a command line window in the BSP directory and execute the `scons` command to start the compilation process.

1.5.2.5.5 RT-Thread Auxiliary Compilation Script

In the tools directory of the RT-Thread source code, there are some auxiliary compiled scripts defined by RT-Thread, such as the project files for automatically generating RT-Thread for some IDE. The most important of these is the building.py script.

1.5.2.5.6 SCons Further Usage

For a complex, large-scale system, it is obviously more than just a few files in a directory. It is probably a combination of several folders at the first level.
In SCons, you can write SConscript script files to compile files in these relatively independent directories, and you can also use the Export and Import functions in SCons to share data between SConstruct and SConscript files (that is, an object data in Python). For more information on how to use SCons, please refer to the `SCons Official Documentation`.

1.6 Contribution

1.6.1 Contribution Guide

We sincerely thank you for your contribution, and welcome to submit the code through GitHub's fork and Pull Request processes. RT-Thread 3.1.0 version and its earlier versions follow the GPL V2 open source license agreement. Versions from the 3.1.0 version onwards follow the Apache License 2.0 open source license agreement.
All the real-time operating system kernel and open source components can be used free of charge for commercial products, there is no potential commercial risk and you will not being request to publish application source.
RT-Thread Coding Style
BSP Contribution Guide
How to write doxygen documentation for RT-Thread
RT-Thread Versioning Policy and Implementation Rules
RT-Thread Release Process

1.6.2 RT-Thread Coding Style

This is an developing instruction for RT-Thread developers. As open source software, RT-Thread is created by the cooperation of different people. This document is a guide for RT-Thread developers. Please obey it as you develop. RT-Thread users should also get to know some conventions in the code through it and thus easier to understand the implementations of RT-Thread.

1.6.2.1 1. Directory Naming

In normal conditions, please name directories in lowercase. Directories should have descriptive names. For example, the port of a chip should be composed of the name of the chip and the category of the chip. Directories under components/ should name what the component does.

1.6.2.2 2. File Naming

In normal conditions, please name files in lowercase. If the file is referencing other places, it can have the original name. To avoid naming collision, do not use general names or the names that are frequently used.

1.6.2.3 3. Header Files

To avoid include the same header file for multiple times, you need to define a symbol like this:

```
#ifndef __FILE_H__
#define __FILE_H__
/* header file content */
#endif
```

The symbol should begin and end with "__" to avoid naming collision. The words of the file name should be connected by "_". (This convention is called "snake case".)

1.6.2.4 4. Header File Comments

In every header file, there should be copyright information and Change Log record like this:

```
/*
* Copyright (c) 2006-2020, RT-Thread Development Team
*
* SPDX-License-Identifier: Apache-2.0
*
* Change Logs:
* Date           Author       Notes
* 2006-03-18     Bernard      the first version
* 2006-04-26     Bernard      add semaphore APIs
*/
```

1.6.2.5 5. Structure Defines

Please name structures in lowercase and connect words with "_". For example:

```
struct rt_list_node
{
    struct rt_list_node *next;
    struct rt_list_node *prev;
};
```

Braces should have their own lines and the members should be indented.
The names of type defines such as structure types should be the structure name plus "_t". For example:

```
typedef struct rt_list_node rt_list_t;
```

In order to be easily referenced, the types of objects in the kernel are pointers. For example:

```
typedef struct rt_timer* rt_timer_t;
```

1.6.2.6 6. Macros

In RT-Thread, please use uppercase names for macro definitions. Words are connected by "_". Like:

```
#define RT_TRUE                         1
```

1.6.2.7 7. Function Naming and Declaration

Please name functions in lowercase. Separate words with "_". The API provided to upper application should be declared in header files. If the function don't have parameters, it should be declared as void:

```
rt_thread_t rt_thread_self(void);
```

1.6.2.8 8. Commenting

Please use English to comment. There shouldn't be many comments as the comments should describe what the code does. It should describe complicated algorithms, for example. Comments for statements should be placed before the statements or to the right of them. Any other locations are invalid.

Macro comments

Like statement comments, a macro can be documented either to the right of the definition or above it. What decides it is how much explanation the macro needs, and only then the line length.

Use a trailing `/**<` for simple constant macros, so that a group of values can be read in one pass:

```
#define RT_EOK                          0               /**< There is no error */
#define RT_ERROR                        255             /**< A generic/unknown error happens */
```

Such lines are column aligned and often exceed 80 columns once the comment is added (the `RT_ERROR` line above is 95 columns). That is accepted: for a group of constant macros, keeping the columns aligned matters more than the line length.

When the macro needs more than one line of explanation - for example a function-like macro whose arguments, expansion or restrictions have to be described - move the comment above the definition, use a `/**` block and mark the macro with `@def`:

```
/**
 * @ingroup group_basic_definition
 *
 * @def RT_ALIGN(size, align)
 * Return the most contiguous size aligned at specified width. RT_ALIGN(13, 4)
 * would return 16.
 * @note align Must be an integer power of 2 or the result will be incorrect
 */
#define RT_ALIGN(size, align)           (((size) + (align) - 1) & ~((align) - 1))
```

In short: if a single line of comment says it, keep it trailing; if it takes a paragraph, put it above.

Doxygen accepts both forms, but there is one difference worth knowing: with `MACRO_EXPANSION` enabled, a trailing `/**<` is attached to the expansion site, so a macro expanded in several places can carry its comment somewhere unexpected. A block comment above the definition always stays with the definition, so prefer it for macros that get expanded frequently.

1.6.2.9 9. Indent

Please use TAB or 4 spaces to indent. It's preferred to use 4 spaces. If no other special meanings, the indent should begin right after "{":

```
if (condition)
{
    /* others */
}
```

The only one exception is switch. In case statements for switches, "case" should be aligned with "switch":

```
switch (value)
{
case value1:
    break;
}
```

"case" is aligned with "switch". The following code block should be indented.

1.6.2.10 10. Braces and Spaces

For ease of reading, it is advised that braces should occupy the whole line instead of following other statements. Like:

```
if (condition)
{
    /* others */
}
```

When matching braces have their own lines, the reader identifies the code blocks easily.
There should be a space before parentheses when it's not a function call. For example:

```
if (x <= y)
{
    /* others */
}

for (index = 0; index < MAX_NUMBER; index ++)
{
    /* others */
}
```

In expressions, there should be a space between most binary and ternary operators and the strings. There should be no spaces around(inside) parentheses, like:

```
if ( x <= y )
{
    /* other */
}
```

This is a bad practice.

1.6.2.11 11. trace, log Information

In RT-Thread, rt_kprintf is a commonly used logging routine. In RT-Thread rt_kprintf is implemented as a polling, non-interrupting string output. It is suitable in "instant" situations such as interrupt context. The polling method would have influence to the timing sequence of the log output.
It is not recommended to use rt_kprintf frequently. Please be aware that frequen use will make your code run slower. Logging should be off by default and can be turned on by a switch (e.g. a variable or a macro). When logging, it should be easy to understand and easy to determine where the problem is.

1.6.2.12 12. Functions

Functions in kernel should be K.I.S.S. ("Keep it simple, stupid.") If the function is too long, you should split it into smaller ones, with each of them simplified to be easy to understand.

1.6.2.13 13. Objects

The kernel of RT-Thread uses object-oriented techniques in C. The naming convention is: structure names are the object names, object names + verb phrases are the method names of objects:

```
struct rt_timer
{
    struct rt_object parent;
    /* other fields */
};
typedef struct rt_timer* rt_timer_t;
```

The definition of structure rt_timer stands for the object definition of timer object.

```
rt_timer_t rt_timer_create(const char* name,
    void (*timeout)(void* parameter), void* parameter,
    rt_tick_t time, rt_uint8_t flag);
rt_err_t rt_timer_delete(rt_timer_t timer);
rt_err_t rt_timer_start(rt_timer_t timer);
rt_err_t rt_timer_stop(rt_timer_t timer);
```

rt_timer + verb phrase stands for the method that could be used on a timer object.
When creating a new object, think twice on memory allocations: whether a static object could be created or it could only created dynamically on the heap. Allocations can be slower, but may be necessary in dynamic environments.

1.6.2.14 14. Format Code

RT-Thread now uses clang-format as the unified code formatting tool. Before submitting C/C++ source files, header files, and related source files, format them with the `.clang-format` configuration in the repository root. Do not use manual formatting or old astyle parameters that conflict with this configuration.
Use clang-format **22.1.8**, pinned in `tools/ci/requirements-format.txt`. Local development, format checks, and the manual formatting workflow use the same version to avoid formatting differences. Install it from the repository root and verify the version:

```
python -m pip install -r tools/ci/requirements-format.txt
clang-format --version
```

Configure your editor to use the same installed clang-format executable.
Common commands:

```
clang-format -style=file -i path/to/file.c
clang-format -style=file -i path/to/file.h
```

When formatting files in batches, limit the command to files related to the current change and avoid large unrelated formatting changes. If a subdirectory has its own `.clang-format`, use the nearest configuration for files under that directory.
In addition to clang-format, keep source files encoded as UTF-8, use `\n` line endings, remove trailing whitespace, and avoid committing unrelated formatting changes produced by editors.

1.6.3 BSP Contribution Guide

1.6.3.1 BSP Contribution Guide

This document describes the directory hierarchy requirements for adding or updating BSPs, so board-level, chip-level, and architecture-level code stay in clear layers.

1.6.3.1.1 BSP Directory Hierarchy

BSP directories usually fall into two hierarchy styles depending on where shared code is kept: vendor-level sharing, and vendor + series-level sharing. In either case, follow these principles:

- `board/` contains board initialization, pin settings, clock settings, linker scripts, and other files that only belong to the current board.
- The upper or sibling shared `libraries/` directory contains chip libraries, HAL, CMSIS, SDK, drivers, and driver adaptation code shared by BSPs from the same vendor or series. Driver directories should also be placed under `libraries/`.
- The sibling `tools/` directory contains helper scripts shared by BSPs from the same vendor or series.

If multiple BSPs share the same package or SDK, do not maintain duplicate copies under each BSP. Put the shared source in the common layer or use the package mechanism, then reference it from the BSP build scripts. When adding a BSP, do not create extra directory levels without a clear need, and do not place BSP code under non-BSP directories such as `components/`, `src/`, or `include/`. Abstract driver frameworks belong in `components/drivers/`, and shared chip or CPU architecture porting code belongs in `libcpu/`.

1.6.3.1.1.1 Vendor-Level Shared Layout (STM32 Example)

STM32 BSPs are usually placed directly under `bsp/stm32/<board>`. Driver adaptation code, templates, tools, and package selections shared by STM32 BSPs are kept under `bsp/stm32/libraries/` and `bsp/stm32/tools/`:

```
bsp/stm32/
 libraries/
    HAL_Drivers/        # Driver adaptation shared by STM32 BSPs
    drivers/            # Drivers shared by multiple STM32 BSPs, if any
    templates/          # Reference templates for new BSPs
    Kconfig             # Vendor HAL/CMSIS package selections
 tools/                  # Vendor-level helper scripts
 stm32f407-atk-explorer/
    board/              # Board-only initialization, pins, clocks, linker scripts
```

```
    Kconfig
    SConscript
    SConstruct
 stm32f429-atk-apollo/
     board/
     Kconfig
     SConscript
     SConstruct
```

1.6.3.1.1.2 Vendor + Series-Level Shared Layout (NXP Example)

NXP-style BSPs are usually split first by vendor, product line, or chip series. Driver adaptation code, templates, tools, and package selections shared by a series are kept under that series directory. For example, i.MX RT uses `bsp/nxp/imx/imxrt/libraries/`, LPC55S uses `bsp/nxp/lpc/lpc55sxx/libraries/`, and MCX keeps separate `libraries/` directories under series such as `mcxa`, `mcxc`, `mcxe`, and `mcxn`.
The NXP tree also contains historical BSP layouts, such as single-BSP LPC directories like `lpc176x`, and a few older directories named `Libraries/`. Do not extend those legacy layouts when adding or restructuring BSPs. If drivers, SDK code, or templates are reused by multiple BSPs, move or place them under the corresponding series-level `libraries/` and `tools/` layer.

```
bsp/nxp/
 imx/
    imxrt/
        libraries/
           drivers/        # Drivers and driver adaptation shared by i.MX RT BSPs
           templates/      # Reference templates for new BSPs
           Kconfig         # NXP i.MX RT SDK package selections
        tools/              # Helper scripts shared by this series
        imxrt1052-nxp-evk/
            board/          # Board-only initialization, pins, clocks, linker scripts
            Kconfig
            SConscript
            SConstruct
 lpc/
    lpc55sxx/
        libraries/          # Driver adaptation, templates, and package selections shared by LPC55S BSPs
        tools/
        lpc55s69_nxp_evk/
            board/
            Kconfig
            SConscript
            SConstruct
 mcx/
     tools/                  # Helper tools shared by MCX series
     mcxn/
         libraries/          # CMSIS, series drivers, and package selections shared by MCXN BSPs
         frdm-mcxn947/
             board/
             Kconfig
             SConscript
             SConstruct
```

1.6.3.1.1.3 Vendor HAL/SDK Packages

If the vendor HAL, CMSIS, or SDK is already provided as an RT-Thread package, the BSP should not submit or maintain another full copy of the same source code. The BSP should select the corresponding `PKG_USING_*` package in `Kconfig` and reference package contents from `SConscript` according to the enabled configuration. `pkgs --update` is responsible for downloading package sources.
In this case, `libraries/` mainly keeps driver adaptation code, reusable drivers, templates, helper scripts, and package selection logic. Do not commit package directories downloaded by `pkgs --update` as BSP source, and do not let `scons --dist` include packages or libraries that are not enabled by the current BSP.

1.6.3.1.2 Submission Requirements

- Keep the BSP hierarchy clear and only submit board-level files required by the current board.
- Libraries, driver adaptation code, reusable drivers, templates, and scripts shared by BSPs from the same vendor or series should be placed in the shared `libraries/` or `tools/` layer to avoid duplicate copies across BSPs.
- Driver adaptation code and reusable drivers should be placed under `libraries/`, not under a single board directory.

- If the vendor HAL, CMSIS, or SDK has been packaged, prefer the package mechanism and do not submit duplicate package source code under the BSP.
- Build scripts should reference common code from the shared layer, and `scons --dist` should not include packages or libraries that are not required by the current BSP.

1.6.4 How to write doxygen documentation for RT-Thread

RT-Thread Online Documentation is created based on doxygen and is available at: https://rt-thread.github.io/rt-thread/. It is consisted of two parts.
One part is the detailed introduction of Kernel, which is written in markdown format and converted into HTML page by doxygen. It is displayed under "RT-Thread User Guide" in Treeview on the left side of the browser. Each sub-chapter is organized into a hierarchical structure by using doxygen's `subpage mechanism`. To define a page, we use `@page` command. The page name should be prefixed with a "page_", and the page name should be unique. Besides this, there are no special requirements for writing this part, so I will not go into details in this article.
The other part is the description of API. RT-Thread uses `doxygen` to automate the generation of documentation from source code comments, parsing information about classes, functions, and variables to produce output in format of HTML. It is displayed under "Modules" in Treeview on the left side of the browser. Each sub-chapter is organized into a hierarchical structure by using doxygen's `topics mechanism`. The main content of this article is to describe how to write API with doxygen.

1.6.4.1 General Rules on writing API documentation

Note

> The code comments and HTML content generated by doxygen in this guide, for the **structures**, **constants(macro definition)**, **enumeration values**, **union values**, **global functions**, **global variables** and other objects involved are all within the scope of the **RT-Thread kernel API**. The code of internal functions, variables (such as static functions etc.) are not belong to the API scope, how to write comment for these elements are not considered in this guide.

By default, API documentation is written in header files, but there are exceptions, such as for **functions**.
There are several ways to mark a comment block as a detailed description. We prefer JavaDoc-style (C-style) comment block with some additional markings to document the code, like this:

```
/**
 * ... text ...
 */
```

When you want to put documentation after members, we prefer a Qt style, like this:

```
int var; /**< Detailed description after the member */
```

When writing comments based on doxygen, several commands defined by doxygen are used. See https://www.doxygen.nl/manual/commands.html for more details about doxygen commands.
More details refer to `Doxygen Docs: Documenting the code`

1.6.4.2 Detailed Rules on writing API documentation

This article provide an `example`.
Click Doxygen Example for the corresponding HTML documentation that is generated by Doxygen.
The contents of Example are described in the following chapters:

- How to use groups in doxygen documentation.
- How to write doxygen documentation for macro.
- How to write doxygen documentation for structure.
- How to write doxygen documentation for union.
- How to write doxygen documentation for enumeration.
- How to write doxygen documentation for typedef.
- How to write doxygen documentation for function.

1.6.4.3 Build & Run

1.6.4.3.1 How to build & run doxygen html on Ubuntu

The following steps are verified on Ubuntu 22.04

```
$ lsb_release -a
No LSB modules are available.
Distributor ID: Ubuntu
Description:    Ubuntu 22.04.5 LTS
Release:    22.04
Codename:   jammy
```

The following packages (and dependents) need to be installed:

```
$ sudo apt update
$ sudo apt install doxygen
$ sudo apt install graphviz
```

Assume that the path of RT-Thead code tree is $RTT, execute the following command to build html.

```
$ cd $RTT/documentation
$ rm -rf html
$ doxygen
```

A new html directory will be created and all the html files will be placed in this directory.
If you want to quickly browse HTML locally (in Ubuntu environment), you can enter the html directory and start a local HTML server through Python.

```
$ cd html
$ python3 -m http.server
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
```

A bash script `run.sh` is provided to automatic upon operations.

```
$ cd $RTT/documentation
$ ./run.sh
```

Then open the browser and enter `http://<IP>:8000/index.html` to access the created html web pages. If it is a local access, then `<IP>` should be replaced by `localhost`. If it is a remote access, then `<IP>` should be replaced by the actual accessible IP address of the machine where HTML is located.

1.6.4.3.2 How to build & run doxygen html with Doxywizard

1. download from https://doxygen.nl/index.html
2. open `Doxywizard`
3. `File` -> `Open`
4. Open the file ./Doxyfile
5. To tab `Run` , Click `Run doxygen`

1.6.4.3.3 How to contribute doxygen comments on vscode more comfortably

There is a script can help you write comments more easily.You can copy the code to your setting.json inner .vscode folder.

```
"doxdocgen.c.triggerSequence": "/**",
"doxdocgen.c.firstLine": "/**",
"doxdocgen.c.commentPrefix": " * ",
"doxdocgen.c.lastLine": " */",
"doxdocgen.generic.briefTemplate": "@brief ",

//You can set param to @param[in] or @param[out] for your preference
"doxdocgen.generic.paramTemplate": "@param[] {param} \n ",

//You can comment out returnTemplate to auto addof return value type after @return
"doxdocgen.generic.returnTemplate": "@return ",

//You can comment out customTags to unconfig @note line, meanwhile comment out "empty" item before custom
      item will make comment more compliant
"doxdocgen.generic.customTags":[
    "@note "
],

"doxdocgen.generic.order":[
    "brief",
    "empty",
    "param",
    "return",
    "empty",
    "custom",
```

],
If you perfer to write mutiple lines comments at behind one doxygen command, Auto Comment Blocks extension can auto complete "*" at the start of line when you start a new line.

1.6.4.4 How to use groups in doxygen documentation.

Doxygen has three mechanisms to group things together. For RT-Thread API documentation, we use 'topics' to create new page for each group. On HTML browser, the topics pages are shown under the "Modules" in the treeview on the left.
To define a group, we use @defgroup command. The group name should be prefixed with a "group_", and the group name should be unique. We can define a group anywhere, not necessarily in the same file as the members of the group. Generally, we define a group in a header file.
To make an entity (structure, function etc. or another group) a member of a speicific group, we can use @ingroup command and put the @ingroup command inside its documentation block.
To avoid putting @ingroup commands in the documentation for each member you can use @addtogroup and group members together by the open marker @{ before the group and the closing marker @} after the group.
See documentation/0.doxygen/example/include/groups.h for code example.
See Doxygen Example of Groups for html output.
More information can be found in the Doxygen manual: Grouping.

1.6.4.5 How to write doxygen documentation for macro.

There are two typical types of macro definitions.

- One is to define constant macros. For this type of macro, we recommend putting documentation after members. See DOXYGEN_EXAMPLE_CONST_A and DOXYGEN_EXAMPLE_CONST_B in documentation/0.doxygen/example/include/macro.h for code exmaple.
- The other is to define macros with parameters. For this type of macro, we recommend using a method similar to documenting for functions, that is, writing comment block before the macro definition. More details please see How to write doxygen documentation for function. See DOXYGEN_EXAMPLE_ABS in documentation/0.doxygen/example/include/macro.h for code example.

We often categorize macros in our code. Similarly, when writing doxygen comments for these categorized macros, we can also group them. See DOXYGEN_EXAMPLE_GROUP_A_X/DOXYGEN_EXAMPLE_GROUP_A_Y and DOXYGEN_EXAMPLE_GROUP_B_X/DOXYGEN_EXAMPLE_GROUP_B_Y in documentation/0.↩doxygen/example/include/macro.h for code example.
See Doxygen Example of Macro for html output.

1.6.4.6 How to write doxygen documentation for structure.

We recommend the following approach:
A comment block before the structure definition is recommended to describe the general information of the structure type. In the comment block, a @brief is required, other commands (such as @note) are optional.
If you feel that the description of @brief is not enough, you can add a detailed description part, which is also optional.
Put member comments inside the structure definition and after every member. See struct dogygen_example_struct for example.
If the structure member is too long (this often happens when the structure member is a callback function), when the comment is written after the member, it will make the code line too long. In this case, we can also put the comment before the member. See struct dogygen_example_struct_another for example.
Going a step further, for this kind of structure whose members are callback functions, we can describe the members in style for functions, it is more complicated and contains more content. It is useful when we want to describe the parameters and return values of the callback function in more detail. See struct dogygen_example_struct_another2 for example.
See documentation/0.doxygen/example/include/struct.h for code example.
See Doxygen Example of Structure for html output.

1.6.4.7 How to write doxygen documentation for union.

A comment block before the union definition is recommended to describe the general information of the union type. In the comment block, a `@brief` is required, other commands (such as `@note`) are optional.
To describe the values of the union, document is recommended to be put after each value.
See `documentation/0.doxygen/example/include/union.h` for code example.
See Doxygen Example of Union for html output.

1.6.4.8 How to write doxygen documentation for enumeration.

A comment block before the enumeration definition is recommended to describe the general information of the enumeration type. In the comment block, a `@brief` is required, other commands (such as `@note`) are optional.
To describe the values of the enumeration, document is recommended to be put after each value.
See `documentation/0.doxygen/example/include/enum.h` for code example.
See Doxygen Example of Enumeration for html output.

1.6.4.9 How to write doxygen documentation for typedef.

It is recommended to use separate typedef statements rather than a combination. That is:
Recommended:

```
struct S { ... };
typedef struct S S_t;
```

Not recommended:

```
typedef struct S { ... } S_t;
```

The reason is we found that the former is more readable, and when we write comment block with `@typedef`, the latter may cause unexpceted behaviour for doxygen (as of version 1.9.1).
See `documentation/0.doxygen/example/include/typedef.h` for code example.
See Doxygen Example of Typedef for html output.

1.6.4.10 How to write doxygen documentation for function.

Function comments can be placed in the header file (before the function declaration) OR in the source file (before the function definition).
The advantage of placing it in the header file is that we generally think that the header file is the place to declare the API, but the problem is that when a module has many API extern functions, if the comments are placed in the header file, the header file will be very long. You can imagine that for a module with many APIs and many comments, the header file will be full of large green comments, and the function declaration part is mixed in the middle and difficult to distinguish. And if you want to fully understand which extern functions this module exports, you need to scroll a long file for a long time to get a general idea. Especially for RTT, see `include/rtthread.h` as an example.
Putting the comment in the source file can avoid the above problems. For developers, it is also convenient to read the comments together with the code implementation. The disadvantage is that it would be different from the function side, from other types, such as structures, i.e. the API comments of functions need to be read in the source file instead of directly in the header file.
Comprehensive consideration can be combined with the actual situation. For example, if there are too many API functions in a header file, it is recommended to put the function comments in the source file.
So, it is **strongly recommended** to put comments in the source file when writing new functions or annotating functions.
To documenting for functions, a comment block before the function declaraion/definition is recommended to describe the general information of the function. In the comment block, a `@brief` is required. A `@return` is also required if the function is intended to return a value, otherwise if the function is implemented with a void return type, `@return` is not required. `@param` is required if any, and if it is provided, direction [in]/[out]/[in,out] should be provide together. Other commands (such as `@note`) are optional.
If you feel that the description of `@brief` is not enough, you can add a detailed description part, which is also optional.
See `documentation/0.doxygen/example/src/function.c` for code example.
See Doxygen Example of Function for html output.

Note

documentation/0.doxygen/example/src/function.h is just an example of the header file where we declare the API without doxygen documentation.

1.6.5 RT-Thread Versioning Policy and Implementation Rules

1.6.5.1 RT-Thread Versioning Policy and Implementation Rules

| Release Process

Item	Details
Document version	1.0
Date	2026-10-27
Scope	RT-Thread mainline release planning, release publication, and stable branch maintenance
Companion document	Release Process

Release cadence: In principle, a Z release is published every month. A Y release is published every 3–6 months, depending on accumulated features and maintenance resources. X releases have no fixed schedule.

1.6.5.1.1 1. Version Number Format

Official releases use the following format:
vX.Y.Z
Examples: v6.0.0, v6.1.0, and v6.1.1.
The version number must be consistent across source version macros, the Git tag, the release, and deliverable names:

Item	Example
Git tag	v6.↩ 1.0
RT_VERSION_MAJOR	6
RT_VERSION_MINOR	1
RT_VERSION_PATCH	0

Routine commits do not increment the official version number. Development snapshots must be identified by the target version, full Git SHA, and build records, and must not be labeled as published official releases.

1.6.5.1.2 2. Version Components

Component	Purpose	Typical RT-Thread changes
X: major	Major features or an overhaul of the overall technical architecture	A new kernel, reworked core kernel mechanisms, a new device or driver framework, system architecture changes, coordinated upgrades across core subsystems, or major API/ABI or configuration changes requiring user migration

Component	Purpose	Typical RT-Thread changes
Y: minor	Feature extensions to the existing kernel and frameworks; released every 3–6 months, depending on accumulated features and resources	New kernel or component APIs, driver framework capabilities, file system or networking capabilities, architecture or chip family support, or a set of BSP or component features that forms a complete deliverable
Z: patch	Focused fixes, support, and local improvements; in principle, released every month	Bug and security fixes, local feature improvements, performance or resource optimizations, individual BSP adaptations, build fixes, and documentation fixes

1.6.5.1.2.1 2.1 X: Major Version

An increment of X requires a comprehensive review and decision for changes such as:

- Introducing a new kernel or replacing core kernel mechanisms.
- Introducing a new device model, driver framework, or system-level software framework.
- Coordinating upgrades across multiple core subsystems that change how the system is used as a whole.
- Removing, renaming, or changing public APIs, the ABI, structure layouts, or system call conventions.
- Changing Kconfig, configuration formats, startup sequences, or data formats so that existing projects require migration.
- Delivering a major upgrade in overall capabilities and architecture, even if some APIs remain compatible.

Related supporting features must be included in the same major release plan.

1.6.5.1.2.2 2.2 Y: Minor Version

Increment Y for changes that meet the following conditions:

- Adding kernel APIs, component APIs, or reusable public capabilities while preserving compatibility with existing interfaces.
- Adding capabilities to driver frameworks, file systems, network stacks, or package frameworks.
- Adding support for an architecture, chip family, or group of BSPs that forms an independently deliverable platform capability.
- Completing several related small features that meet the release goals and form a complete, verifiable, releasable feature set.

An isolated feature, local parameter extension, or optimization of a single implementation does not automatically trigger a Y increment. Features must meet the delivery scope defined by the release milestone and have corresponding test evidence. The Y release cycle is 3–6 months; once selected, the specific cycle must be stated in the release goals. Features that cannot complete validation within that cycle must be deferred to the next release.

1.6.5.1.2.3 2.3 Z: Patch Version

Increment Z for changes that meet the following conditions:

- Fixing deadlocks, out-of-bounds access, memory leaks, boot failures, security vulnerabilities, build failures, or performance regressions.
- Improving local functionality, error handling, logging, performance, or resource usage.
- Adding an adaptation for an individual BSP, chip, or platform.
- Fixing documentation, tests, build scripts, or release materials.

A Z release must not introduce new public APIs, complete components, driver frameworks, architectural frameworks, or planned incompatible changes. If an individual BSP introduces a new driver framework, device model, or public API, the change requires a Y or X increment.

1.6.5.1.3 3. Version Classification Priority

When a release contains multiple types of changes, use the highest impact level to determine its version number:
`X: major > Y: minor > Z: patch`

Combination of changes	Classification
Major kernel/framework changes + new components + bug fixes	`X`
New component/API/platform capabilities + local fixes	`Y`
Bug fixes + individual BSP adaptations + performance optimizations	`Z`
Documentation, tests, CI, or release materials only	No official version increment

Do not determine version numbers by commit count, PR count, lines of code, or development cycle length. Use feature boundaries, user impact, architectural impact, and completeness of delivery.

1.6.5.1.4 4. Implementation Rules by Version Component

Basis for release: periodic evaluation + accumulated features.
Developers are advised to use `X/Y` releases. `Z` releases serve as intermediate stabilization points for traceability.

1.6.5.1.4.1 4.1 `X` Releases

To be discussed.

1.6.5.1.4.2 4.2 `Y` Releases

Item	Requirement
Maintenance approach	Manual work + CI, with staffing focused on BSP and driver maintenance
Main purpose	Stabilize and release major driver framework and kernel API changes, as well as support for new architectures and frameworks
Release frequency	A 3–6 month cycle, with core maintainers reviewing and deciding whether to release

Release strategy:

1. CI provides a test checklist and firmware for the corresponding test cases.
2. Testers perform manual validation and upload the results.
3. AI uses prompts or skills to consolidate the results into a release test report.
4. Humans review the report, and core maintainers decide whether to release based on accumulated features and validation results.

1.6.5.1.4.3 4.3 `Z` Releases

Prerequisite: the mainline has an initial, reasonably complete automated workflow in place.

Item	Requirement
Maintenance approach	Primarily QEMU + CI automation
Test scope	The kernel, components, and drivers supported by simulation
Main purpose	Stabilize the mainline periodically through small releases that support traceability
Release frequency	Monthly evaluation, with a human decision on whether to trigger a release

Release strategy:

1. CI runs tests automatically and collects test coverage and pass rates.
2. CI generates a release test report covering coverage, pass rates, functionality introduced by commits since the previous release, and known issues.
3. A release proposal discussion is created automatically to notify core maintainers and invite their input.
4. Core maintainers decide whether to release. A release may be skipped if only a few fixes or optimizations have accumulated during the period.

The overall pass rate does not have to reach 100%, but known issues must be listed so maintainers can assess them before making the release decision.

1.6.5.1.5 5. Version Planning Process

See the Release Process for release stages and responsibilities.

```
A requirement or PR proposes a version classification
    -> Module maintainer makes an initial assessment
    -> Core maintainer group reviews high-risk changes
    -> Changes enter the release milestone
    -> X.Y is confirmed before feature freeze
    -> X.Y.Z is confirmed before release
    -> Release owner creates the tag and release
```

1.6.5.1.5.1 5.1 Release Preparation

- Collect unresolved issues from the previous release, community requirements, and new features.
- Assign an initial `X`, `Y`, or `Z` classification based on feature impact.
- Create a release milestone that distinguishes mandatory, deferrable, and excluded items.
- Assign the release owner, module maintainers, target platforms, and test owners.
- Plan `Z` releases backward from a one-month cycle. For `Y` releases, choose a 3–6 month cycle according to feature scope and available resources. Confirm test capabilities and maintenance staffing before defining the feature scope.

1.6.5.1.5.2 5.2 Release Development

- PRs adding new features must state the proposed version level and the reason for that classification when they are submitted for merging.
- Module maintainers confirm feature boundaries, compatibility, and test scope.
- If an `X`-level change is identified, reassess the release goals, migration cost, and release date.
- If a new feature cannot form a complete deliverable, keep it on the mainline and defer it to the next release. Do not conceal scope changes by changing the version number.
- Maintain only one `Y` release target per release cycle. Do not accept untested large features at the last minute to meet a release date. Defer features that have not completed validation to the next release.

1.6.5.1.5.3 5.3 Feature Freeze

- Begin stabilizing fixes for a `Z` release approximately one month before release. Begin stabilizing features for a `Y` release approximately one month before release, and confirm `X.Y` before the freeze.
- After the freeze, do not add features, public APIs, components, or framework capabilities.
- After the freeze, allow only fixes for release blockers, necessary security fixes, and corrections to release materials.
- Remove `X`- or `Y`-level changes discovered after the freeze from the current release. Replan them for a major release or the next minor release on a 3–6 month cycle, respectively.

1.6.5.1.5.4 5.4 Official Release

- Set the release version to `X.Y.0`, or to `X.Y.Z` on a stable branch.
- The release owner checks consistency across version macros, the Git tag, the release, the changelog, and deliverables.
- If an issue is found after publication, publish a new `X.Y.Z+1` release. Do not modify the original tag, release contents, or deliverables.

1.6.5.1.5.5 5.5 Release Cadence

Cycle	Main activity
1 month	Publish a `Z` release focused on defects, security issues, local optimizations, and BSP adaptations
3–6 months	Publish a `Y` release when its feature set, test scope, and maintenance resources meet the requirements
`X` releases	Plan separately according to readiness for kernel, framework, or system-level upgrades, without a fixed cycle

No release type requires a fixed publication date. A release may be deferred if blockers remain within its guaranteed support scope. Do not force a release by reducing promised testing, narrowing the support scope, or skipping reviews of high-risk changes. Once the cycle for a `Y` release is selected, do not shorten it to accommodate last-minute features.

1.6.5.1.6 6. Compatibility and Migration Requirements

The following changes require a compatibility review:

- Public interfaces and behavior of thread scheduling, IPC, timers, memory management, interrupts, or SMP.
- Device driver interfaces, BSP initialization, Kconfig, default configurations, and component configuration formats.
- File systems, network stacks, package APIs, error codes, blocking semantics, and interrupt context restrictions.
- Structure layouts, the ABI, toolchain requirements, default stack sizes, heap layouts, and critical resource usage.

Compatibility changes must record affected users or components, migration methods, representative BSPs, and known limitations. `X` releases must provide migration instructions. `Y` and `Z` releases must also explicitly document changes to default behavior or resource usage in the release notes.

1.6.5.1.7 7. Release Checklist

- ☐ The release milestone defines mandatory scope and release goals.
- ☐ All included changes have an `X/Y/Z` classification.
- ☐ The core maintainer group has approved all `X`-level changes.
- ☐ `Y`-level features are independently deliverable and have completed testing.
- ☐ `Z`-level changes introduce no new public APIs, complete components, or framework capabilities.
- ☐ No unapproved feature changes have occurred after the freeze.
- ☐ `RT_VERSION_MAJOR/MINOR/PATCH` matches the Git tag.

1.6.6 RT-Thread Release Process

1.6.6.1 RT-Thread Release Process

| Versioning Policy and Implementation Rules

This document describes the six stages of an RT-Thread mainline release, from requirements preparation to publication. See the Versioning Policy and Implementation Rules for version classification, compatibility requirements, and the release checklist.

1.6.6.1.1 Stage 1: Release Preparation

1.6.6.1.1.1 1. Primary Goal

- Finalize the development tasks for this release and create a view of the release development tasks.

1.6.6.1.1.2 2. Main Activities

Key Activities

- Prepare the release draft: review the previous release completion report and development draft to create an initial plan for this release.
- Organize GitHub issues: review unresolved issues from earlier releases, collect issues for new features, and mark them with release labels.

Other Activities

- Collect requirements from other departments and consolidate functional requirements into a requirements document.
- Evaluate emerging technologies for possible integration into the mainline, recording proposals as issues or discussions.
- Compare the performance of similar products and identify useful features to introduce, recording findings as issues or discussions.



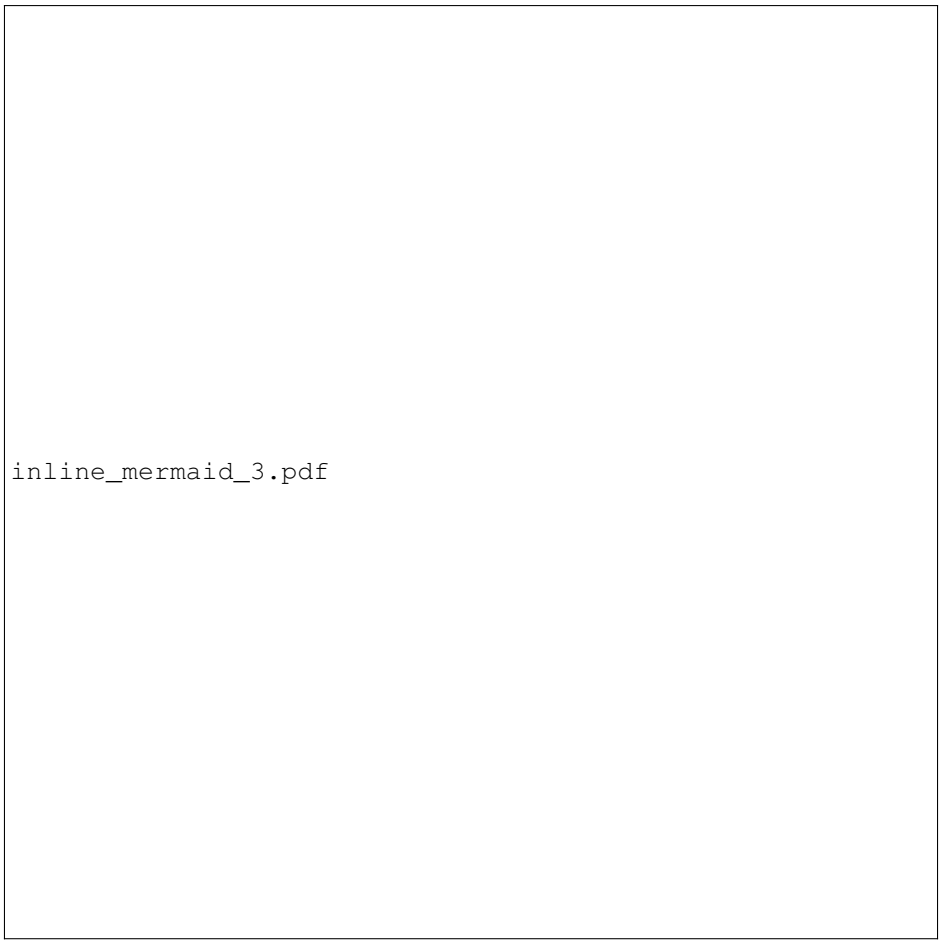
1.6.6.1.2 Stage 2: Define Initial Goals for the Next Release

1.6.6.1.2.1 1. Primary Goal

- Define the initial tasks for the next release.

1.6.6.1.2.2 2. Main Activities

1. Collect community requirements and feedback as issues, starting after the previous release is published.
2. Group the issues into milestones.
3. Define the initial goals for the next release.



1.6.6.1.3 Stage 3: Development Preparation

1.6.6.1.3.1 1. Primary Goal

- Assign development tasks using the release development task view.

1.6.6.1.3.2 2. Main Activities

- Assign developers to the development tasks for this release.
- Assign reviewers to PRs for different features.



1.6.6.1.4 Stage 4: Release Development

1.6.6.1.4.1 1. Primary Goal

- Track progress and announce the feature freeze in advance.

1.6.6.1.4.2 2. Main Activities

- Complete the release development goals and record progress in the release plan.
- For core PRs, announce the feature freeze two weeks before release.
- Mark new feature PRs from community developers with GitHub labels.
- Process routine PRs throughout release development.



1.6.6.1.5 Stage 5: Pre-release

1.6.6.1.5.1 1. Primary Goal

- Stabilize features and complete bug regression testing.

1.6.6.1.5.2 2. Main Activities

Core Activities

- Stabilize the feature set by following up with authors to complete the merging of new feature PRs.
- Complete bug regression testing and collect release issues reported by the community, covering both development and published versions.

Testing

- Use the release test case template to test different architectures, BSPs, kernels, components, peripherals, file systems, and other areas, and record issues. See the previous release test case document.

Reporting

- Combine release development task progress with a summary of new feature PRs from community developers to produce a release task completion report.



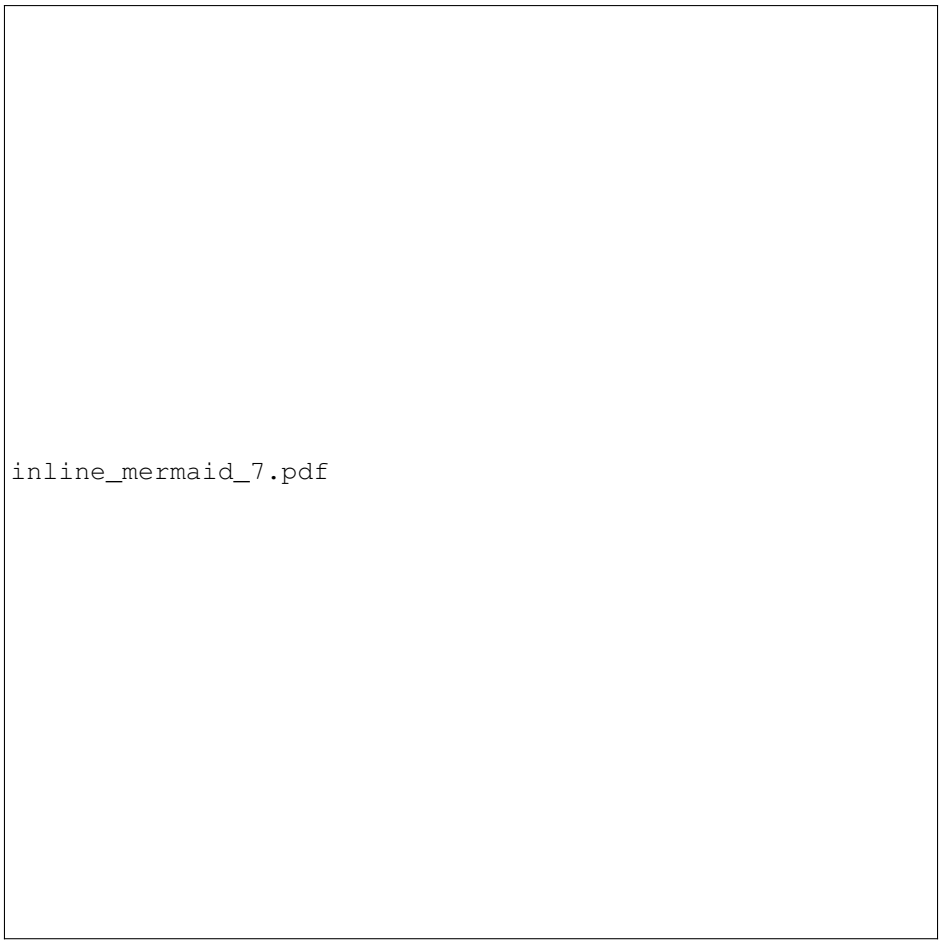
1.6.6.1.6 Stage 6: Release Publication

1.6.6.1.6.1 1. Key Activities

- Publish the version tag and release on GitHub.
- Publish the version tag and release on Gitee.
- Write documentation describing the new features in the release.
- Write the English changelog for the GitHub release. See the version comparison example.
- Publish the release announcement on the community's WeChat official account.
- Hold a release launch event.

1.6.6.1.6.2 2. Other Activities

- Publish a community thread inviting developers to discuss the release and provide feedback.
- Update RT-Thread Studio project files and publish projects for the new version.
- Update chip support information on the commercial website, focusing on newly supported architectures. See the chip support page.
- Update documentation and links for newly supported features in the documentation center.
- Update the cloud download link for RT-Thread source code on the official website. See the source code download page.



1.6.6.1.7 Release Process Overview

The diagrams above detail the activities within each stage. The overall sequence is:



1.6.6.1.8 References

- Tencent Docs: Release Process Diagram

Chapter 2

libcpu Kconfig ARM

2.1

libcpu/Kconfig CPU ARM ARM libcpu/arm/Kconfig
ARM BSP

2.2

libcpu/Kconfig	CPU
libcpu/arm/Kconfig	ARM ARM
libcpu/aarch64/Kconfig	AArch64
libcpu/risc-v/Kconfig	RISC-V

Kconfig

2.3

- ARM ARCH_ARM ARCH_ARM_CORTEX_*ARCH_ARM_ARM9ARCH_ARM_ARM11 ARCH_ARMV8
- ARM ARCH_ARM_BOOTWITH_FLUSH_CACHEARCH_ARM_MMUARCH_ARM_MPU ARCH_ARM_↩ SECURE_MODE
- Cortex-A RT_SMP_AUTO_BOOTRT_USING_GIC_V2RT_USING_GIC_V3 RT_NO_USING_GIC
- Cortex-A RT_IOREMAP_LATE RT_BACKTRACE_FUNCTION_NAME

ARM

2.4

libcpu/Kconfig ARCH_ARM_BOOTWITH_FLUSH_CACHE ARM
rsource "arm/Kconfig"
rsource Kconfig BSP libcpu/Kconfig
if ARCH_ARM ARCH_ARM select

- CPU FFS
- ARCH_MM_MMUARCH_MM_MPUARCH_REMAP_KERNELARCH_USING_ASID

- KERNEL_VADDR_START ARMARMv8 RISC-V if RT_USING_SMART

RT_IOREMAP_LATE if RT_USING_SMART Cortex-AMMU GIC if ARCH_ARM_CORTEX_A
ARCH_ARMV8 ARM ARCH_ARMV8 && ARCH_CPU_64BIT AArch64
ARCH_ARM_CORTEX_R ARCH_CPU_STACK_GROWS_UPWARD

2.5

Kconfiglib 14.1.0 BSP .config

BSP		
bsp/frdm-k64f	Cortex-M4	
bsp/qemu-vexpress-a9	Cortex-A9MMUGIC	
bsp/renesas/rzt2m_rsk	Cortex-R52	
bsp/qemu-virt64-aarch64	ARMv864 AArch64	
bsp/qemu-virt64-riscv	RISC-V	

select imply
Cortex-A9AArch64 RISC-V BSP RT_USING_SMART

- Cortex-A9 KERNEL_VADDR_START 0xc0000000 RT_IOREMAP_LATE
- AArch64 KERNEL_VADDR_START 0xffff000000000000
- RISC-V KERNEL_VADDR_START 0xffffffc000000000

BSP .config rtconfig.h

2.6

Kconfiglib Python BSP

```
python3 - «'PY'
import kconfiglib

config = kconfiglib.Kconfig("Kconfig")
config.load_config(".config")
for name in ("ARCH_ARM", "ARCH_ARMV8", "ARCH_CPU_64BIT", "ARCH_RISCV"):
    print("{}={}".format(name, config.syms[name].str_value))
PY
```

BSP

Chapter 3

Topic Index

3.1 RT-Thread API Guide

This manual describes the API usage of RT-Thread written in the C programming language.

- Basic Definitions . . . 745
- Kernel . . . 746
  - Object Management . . . 747
  - Thread Management . . . 755
  - Clock and Timer Management . . . 774
  - Inter-Thread Communication . . . 781
    - Signal . . . 789
    - Semaphore . . . 791
    - Mutex . . . 795
    - Event . . . 800
    - MailBox . . . 804
    - Message Queue . . . 809
  - Memory Management . . . 814
  - Runtime Trace and Record . . . 821
  - Other useful kernel service . . . 822
  - Error Code . . . 835
  - System Initialization . . . 835
- Components . . . 836
  - Device Virtual File System . . . 836
    - File Descriptor . . . 843
    - File System API . . . 844
    - File API . . . 844
    - File POSIX API . . . 844
  - Device Driver . . . 854
    - ADC . . . 860
    - Clock . . . 862
    - DAC . . . 875
    - Audio . . . 877
      - AUDIO_CTL . . . 879
      - AUDIO_TYPE . . . 880
      - AUDIO_SAMP_RATES . . . 881
      - AUDIO_BIT_RATES . . . 882
      - AUDIO_DSP . . . 883
      - AUDIO_MIXER . . . 884
      - CODEC_CMD . . . 885
    - CAN Driver . . . 886
    - I2C . . . 891
    - Pin . . . 896
    - PWM . . . 902

RTC . . . 906
Serial . . . 909
Serial v2 . . . 912
SPI . . . 915
Touch . . . 927
blk . . . 929
Finsh Shell . . . 931
Application Module . . . 933
Hardware Related Package . . . 933
Doxygen Example . . . 935
Doxygen Example of Enumeration . . . 935
Doxygen Example of Groups . . . 936
Doxygen Example of Macro . . . 937
Group A of Macros . . . 937
Group B of Macros . . . 938
Doxygen Example of Structure . . . 938
Doxygen Example of Typedef . . . 938
Doxygen Example of Union . . . 939
Doxygen Example of Function . . . 939

Chapter 4

Data Structure Index

4.1 Data Structures

Here are the data structures with brief descriptions:

can_configure
CAN device configuration structure . . . 941
dfs_seq_file . . . 942
dfs_seq_ops . . . 942
dogygen_example_struct
Brief description this structure . . . 942
dogygen_example_struct_another
Brief description this structure . . . 943
dogygen_example_struct_another2
Brief description this structure . . . 943
doxygen_example_union
Brief description of this union . . . 944
rt_adc_core_ops
ADC core driver operation table . . . 945
rt_adc_device
Adc device . . . 946
rt_adc_ops
Configure the adc device . . . 947
rt_adc_sequence_cfg
ADC synchronous one-frame sequence request . . . 947
rt_adc_sequence_ops
ADC finite-sequence operation table . . . 948
rt_adc_session_ctrl
ADC active session runtime control block . . . 949
rt_audio_buf_info
Audio buffer info . . . 950
rt_audio_caps
Audio capabilities . . . 950
rt_audio_configure
Audio configuration . . . 951
rt_audio_device
Audio device . . . 951
rt_audio_ops
Aduio device operators . . . 952
rt_audio_pipe
Audio buffer info . . . 953
rt_audio_portal_device
Portal device . . . 954
rt_audio_record
Audio record, the audio device pipe container of ringbuffer . . . 954

rt_audio_replay
Audio replay . . . 954
rt_blk_device
Logical blk device, if you don't used DFS it will be defined by default . . . 955
rt_blk_disk
Physical blk device . . . 955
rt_blk_disk_ops
Configure the blk device . . . 956
rt_can_bit_timing
CAN bit-timing parameters structure . . . 956
rt_can_bit_timing_config
CAN bit timing configuration list structure . . . 956
rt_can_device
The core CAN device structure . . . 957
rt_can_filter_config
CAN hardware filter configuration structure. This structure is passed to the driver via rt_↩ device_control with the RT_CAN_CMD_SET_FILTER command . . . 958
rt_can_filter_item
CAN filter item structure . . . 959
rt_can_msg
The CAN message structure . . . 960
rt_can_ops
The CAN device driver operations structure . . . 961
rt_can_status
CAN device status and error statistics structure . . . 963
rt_can_status_ind_type
Structure to hold the status indication callback and its arguments . . . 965
rt_clk
Clock handle - represents a consumer reference to a clock . . . 965
rt_clk_array
Clock array container . . . 966
rt_clk_cell
Clock cell - represents a single hardware clock element . . . 966
rt_clk_fixed_rate
Fixed-rate clock descriptor . . . 966
rt_clk_node
Clock provider node - represents a hardware clock controller . . . 966
rt_clk_notifier
Clock notifier descriptor . . . 967
rt_cpu_usage_stats . . . 967
rt_dac_device
DAC device structure . . . 967
rt_dac_ops
Configuration of DAC device . . . 967
rt_device . . . 968
rt_device_blk_geometry . . . 969
rt_device_blk_sectors . . . 969
rt_device_graphic_info . . . 970
rt_device_graphic_ops . . . 971
rt_device_notify . . . 971
rt_device_pin
Pin device structure . . . 971
rt_device_pin_mode
Pin mode structure . . . 971
rt_device_pin_value
Pin value structure . . . 971
rt_device_pwm
PWM device . . . 971

rt_device_rect_info . . . 971
rt_dma_chan
DMA channel descriptor . . . 972
rt_dma_controller
DMA controller descriptor . . . 973
rt_dma_controller_ops
DMA controller operations vtable . . . 974
rt_dma_map_ops
DMA memory mapping operations vtable . . . 978
rt_dma_pool
DMA memory pool descriptor . . . 979
rt_dma_slave_config
DMA slave channel configuration . . . 980
rt_dma_slave_transfer
DMA slave transfer descriptor . . . 981
rt_hw_backtrace_frame . . . 982
rt_i2c_bus_device
I2C Bus Device . . . 982
rt_i2c_bus_device_ops
I2C Bus Device Operations . . . 983
rt_i2c_client
I2C Client . . . 983
rt_i2c_configuration . . . 983
rt_i2c_msg
I2C Message . . . 983
rt_i2c_priv_data
I2C Private Data . . . 983
rt_inputcapture_ops . . . 983
rt_interrupt_context . . . 984
rt_ipc_object . . . 984
rt_list_node . . . 984
rt_lockless_slist_node . . . 985
rt_mailbox . . . 985
rt_mempool . . . 986
rt_messagequeue . . . 987
rt_mutex . . . 988
rt_object . . . 989
rt_object_information . . . 990
rt_pin_irq_hdr
Pin irq structure . . . 990
rt_pin_ops
Pin device operations . . . 991
rt_pm . . . 991
rt_pm_ops . . . 991
rt_pwm_configuration
PWM configuration . . . 991
rt_pwm_ops
PWM operations . . . 991
rt_qspi_configuration
QSPI configuration structure . . . 991
rt_qspi_device
QSPI operators . . . 991
rt_qspi_message
QSPI message structure . . . 992
rt_rtc_device
RTC device structure . . . 992
rt_rtc_ops
RTC operations . . . 992

rt_rtc_wkalarm
RTC alarm structure . . . 992
rt_sched_thread_ctx . . . 992
rt_sched_thread_priv . . . 993
rt_semaphore . . . 994
rt_serial_device
Serial device structure . . . 994
rt_serial_rx_fifo
Serial Receive FIFO mode . . . 994
rt_serial_tx_fifo
Serial Transmit FIFO mode . . . 994
rt_slist_node . . . 995
rt_spi_bus
SPI bus structure . . . 995
rt_spi_configuration
SPI configuration structure . . . 995
rt_spi_device
SPI Virtual BUS, one device must connected to a virtual BUS . . . 995
rt_spi_message
SPI message structure . . . 995
rt_spi_ops
SPI operators . . . 996
rt_spinlock . . . 996
rt_thread
Thread Control Block . . . 996
rt_timer . . . 997
rt_touch_config
Touch configuration . . . 998
rt_touch_data
Touch data . . . 998
rt_touch_device
Touch device . . . 998
rt_touch_info
Touch information . . . 999
rt_touch_ops
Touch device operations . . . 999
rt_uart_ops
Configure the serial device . . . 999
rt_wqueue . . . 999
scmi_clk_attributes . . . 999
scmi_clk_describe_attributes_in . . . 999
scmi_clk_describe_attributes_out . . . 1000
scmi_clk_describe_rates_in . . . 1000
scmi_clk_describe_rates_out . . . 1000
scmi_clk_name_in . . . 1000
scmi_clk_name_out . . . 1000
scmi_clk_rate_get_in . . . 1000
scmi_clk_rate_get_out . . . 1000
scmi_clk_rate_set_in . . . 1001
scmi_clk_rate_set_out . . . 1001
scmi_clk_state_in . . . 1001
scmi_clk_state_out . . . 1001
scmi_pinctrl_attributes_in . . . 1001
scmi_pinctrl_attributes_out . . . 1001
scmi_pinctrl_list_assoc_in . . . 1002
scmi_pinctrl_list_assoc_out . . . 1002
scmi_pinctrl_name_get_in . . . 1002
scmi_pinctrl_name_get_out . . . 1002

scmi_pinctrl_protocol_attributes . . . 1002
scmi_pinctrl_request_in . . . 1002
scmi_pinctrl_request_out . . . 1002
scmi_pinctrl_settings_conf_in . . . 1003
scmi_pinctrl_settings_conf_out . . . 1003
scmi_pinctrl_settings_get_in . . . 1003
scmi_pinctrl_settings_get_out . . . 1003
scmi_power_attributes . . . 1003
scmi_power_state_set_in . . . 1003
scmi_power_state_set_out . . . 1004
scmi_reset_attr_in . . . 1004
scmi_reset_attr_out . . . 1004
scmi_reset_in . . . 1004
scmi_reset_out . . . 1004
scmi_scmi_sensor_axis_description_get_in . . . 1004
scmi_scmi_sensor_axis_description_get_out . . . 1004
scmi_sensor_attributes . . . 1005
scmi_sensor_axis_name_in . . . 1005
scmi_sensor_axis_name_out . . . 1005
scmi_sensor_config_get_in . . . 1005
scmi_sensor_config_get_out . . . 1005
scmi_sensor_config_set_in . . . 1005
scmi_sensor_config_set_out . . . 1006
scmi_sensor_continuous_update_notify_in . . . 1006
scmi_sensor_continuous_update_notify_out . . . 1006
scmi_sensor_description_get_in . . . 1006
scmi_sensor_description_get_out . . . 1006
scmi_sensor_list_update_intervals_in . . . 1006
scmi_sensor_list_update_intervals_out . . . 1006
scmi_sensor_name_in . . . 1007
scmi_sensor_name_out . . . 1007
scmi_sensor_reading_in . . . 1007
scmi_sensor_reading_out . . . 1007
scmi_sensor_trip_point_config_in . . . 1007
scmi_sensor_trip_point_config_out . . . 1007
scmi_sensor_trip_point_notify_in . . . 1008
scmi_sensor_trip_point_notify_out . . . 1008
scmi_voltage_attr_in . . . 1008
scmi_voltage_attr_out . . . 1008
scmi_voltage_attributes . . . 1008
scmi_voltage_config_get_in . . . 1008
scmi_voltage_config_get_out . . . 1008
scmi_voltage_config_set_in . . . 1009
scmi_voltage_config_set_out . . . 1009
scmi_voltage_level_get_in . . . 1009
scmi_voltage_level_get_out . . . 1009
scmi_voltage_level_set_in . . . 1009
scmi_voltage_level_set_out . . . 1009

Chapter 5

File Index

5.1 File List

Here is a list of all documented files with brief descriptions:

C:/Renesas/rt-thread/components/drivers/include/drivers/adc_v2.h
Standalone ADC V2 framework declarations 1011
C:/Renesas/rt-thread/components/drivers/include/drivers/adc_v2_trigger.h
ADC V2 trigger framework declarations 1016
C:/Renesas/rt-thread/components/drivers/include/drivers/dma.h
DMA (Direct Memory Access) controller framework public API 1016

Chapter 6

Topic Documentation

6.1 Basic Definitions

Macros

- #define RT_VERSION_MAJOR 5
- #define RT_VERSION_MINOR 4
- #define RT_VERSION_PATCH 0
- #define RT_IS_ALIGN(addr, align)
- #define RT_ALIGN(size, align)
- #define RT_ALIGN_DOWN(size, align)

6.1.1 Detailed Description

Basic data type in RT-Thread RTOS.
These are the basic definitions which used in RT-Thread RTOS. In general, RT-Thread kernel uses its own definition of the basic data types, such as rt_uint32_t, rt_uint8_t, etc., which does not depend on the compiler or architecture.

6.1.2 Macro Definition Documentation

6.1.2.1 RT_VERSION_MAJOR

```
#define RT_VERSION_MAJOR 5
```

Major version number (X.x.x)

6.1.2.2 RT_VERSION_MINOR

```
#define RT_VERSION_MINOR 4
```

Minor version number (x.X.x)

6.1.2.3 RT_VERSION_PATCH

```
#define RT_VERSION_PATCH 0
```

Patch version number (x.x.X)

6.1.2.4 RT_IS_ALIGN

```
#define RT_IS_ALIGN(
              addr,
              align)
```

Value:

```
((!(addr & (align - 1))) && (addr != RT_NULL))
```

Return true(1) or false(0). RT_IS_ALIGN(128, 4) is judging whether 128 aligns with 4. The result is 1, which means 128 aligns with 4.

Note

If the address is NULL, false(0) will be returned

6.1.2.5 RT_ALIGN

```
#define RT_ALIGN(
                size,
                align)
```

Value:

```
(((size) + (align) - 1) & ~((align) - 1))
```

Return the most contiguous size aligned at specified width. RT_ALIGN(13, 4) would return 16.

Note

align Must be an integer power of 2 or the result will be incorrect

6.1.2.6 RT_ALIGN_DOWN

```
#define RT_ALIGN_DOWN(
                size,
                align)
```

Value:

```
((size) & ~((align) - 1))
```

Return the down number of aligned at specified width. RT_ALIGN_DOWN(13, 4) would return 12.

Note

align Must be an integer power of 2 or the result will be incorrect

6.2 Kernel

Collaboration diagram for Kernel:

Topics

- Object Management
- Thread Management
- Clock and Timer Management
- Inter-Thread Communication
- Memory Management
- Runtime Trace and Record
- Other useful kernel service
- Error Code
- System Initialization

Functions

- rt_weak void rt_interrupt_enter (void)
- rt_weak void rt_interrupt_leave (void)
- rt_weak rt_uint8_t rt_interrupt_get_nest (void)

6.2.1 Detailed Description

Core of RT-Thread, see Kernel for more details.

6.2.2 Function Documentation

6.2.2.1 rt_interrupt_enter()

```
rt_weak void rt_interrupt_enter (
            void )
```

This function will be invoked by BSP, when enter interrupt service routine.

Note

Please don't invoke this routine in application

See also

rt_interrupt_leave

6.2.2.2 rt_interrupt_leave()

```
rt_weak void rt_interrupt_leave (
            void )
```

This function will be invoked by BSP, when leave interrupt service routine.

Note

Please don't invoke this routine in application

See also

rt_interrupt_enter

6.2.2.3 rt_interrupt_get_nest()

```
rt_weak rt_uint8_t rt_interrupt_get_nest (
            void )
```

This function will return the nest of interrupt.
User application can invoke this function to get whether current context is interrupt context.

Returns

the number of nested interrupts.

6.2.3 Object Management

Collaboration diagram for Object Management:

Data Structures

- struct rt_object
- struct rt_object_information

Macros

- #define RT_OBJECT_FLAG_MODULE 0x80
- #define _RT_OBJECT_HOOK_CALL(func, argv)

Typedefs

- typedef struct rt_object * rt_object_t
- typedef rt_err_t(*) rt_object_iter_t(rt_object_t object, void *data)

Enumerations

- enum rt_object_class_type {
  RT_Object_Class_Null = 0x00 , RT_Object_Class_Thread = 0x01 , RT_Object_Class_Semaphore = 0x02 ,
  RT_Object_Class_Mutex = 0x03 ,
  RT_Object_Class_Event = 0x04 , RT_Object_Class_MailBox = 0x05 , RT_Object_Class_MessageQueue = 0x06 , RT_Object_Class_MemHeap = 0x07 ,
  RT_Object_Class_MemPool = 0x08 , RT_Object_Class_Device = 0x09 , RT_Object_Class_Timer = 0x0a ,
  RT_Object_Class_Module = 0x0b ,
  RT_Object_Class_Memory = 0x0c , RT_Object_Class_Channel = 0x0d , RT_Object_Class_ProcessGroup = 0x0e , RT_Object_Class_Session = 0x0f ,
  RT_Object_Class_Custom = 0x10 , RT_Object_Class_Unknown = 0x11 , RT_Object_Class_Static = 0x80 }

Functions

- struct rt_object_information * rt_object_get_information (enum rt_object_class_type type)
- int rt_object_get_length (enum rt_object_class_type type)
- int rt_object_get_pointers (enum rt_object_class_type type, rt_object_t *pointers, int maxlen)
- void rt_object_init (struct rt_object *object, enum rt_object_class_type type, const char *name)
- void rt_object_detach (rt_object_t object)
- rt_object_t rt_object_allocate (enum rt_object_class_type type, const char *name)
- void rt_object_delete (rt_object_t object)
- rt_bool_t rt_object_is_systemobject (rt_object_t object)
- rt_uint8_t rt_object_get_type (rt_object_t object)
- rt_err_t rt_object_for_each (rt_uint8_t type, rt_object_iter_t iter, void *data)
- rt_object_t rt_object_find (const char *name, rt_uint8_t type)
- rt_err_t rt_object_get_name (rt_object_t object, char *name, rt_uint8_t name_size)
- rt_object_t rt_custom_object_create (const char *name, void *data, rt_err_t(*data_destroy)(void *))
- rt_err_t rt_custom_object_destroy (rt_object_t obj)

6.2.3.1 Detailed Description

See Object Management.
The Kernel object system can access and manage all of the kernel objects.
Kernel objects include most of the facilities in the kernel:

- thread
- semaphore and mutex
- event/fast event, mailbox, messagequeue
- memory pool
- timer

Kernel objects can be static objects, whose memory is allocated in compiling. It can be dynamic objects as well, whose memory is allocated from system heaps in runtime.

6.2.3.2 Macro Definition Documentation

6.2.3.2.1 RT_OBJECT_FLAG_MODULE

```
#define RT_OBJECT_FLAG_MODULE 0x80
```

is module object.

6.2.3.2.2 _RT_OBJECT_HOOK_CALL

```
#define _RT_OBJECT_HOOK_CALL(
              func,
              argv)
```

Value:

```
__ON_HOOK_ARGS(func, argv)
```

Add hook point in the routines.
The hook function call macro

Note

Usage: void foo() { do_something();

RT_OBJECT_HOOK_CALL(foo);
do_other_things(); }

6.2.3.3 Typedef Documentation

6.2.3.3.1 rt_object_t

```
typedef struct rt_object* rt_object_t
```

Type for kernel objects.

6.2.3.3.2 rt_object_iter_t

```
typedef rt_err_t(*) rt_object_iter_t(rt_object_t object, void *data)
```

iterator of rt_object_for_each()
data is the data passing in to rt_object_for_each(). iterator can return RT_EOK to continue the iteration; or any positive value to break the loop successfully; or any negative errno to break the loop on failure.

6.2.3.4 Enumeration Type Documentation

6.2.3.4.1 rt_object_class_type

```
enum rt_object_class_type
```

The object type can be one of the follows with specific macros enabled:

- Thread
- Semaphore
- Mutex
- Event
- MailBox
- MessageQueue
- MemHeap
- MemPool
- Device
- Timer
- Module
- Unknown
- Static

Enumerator

RT_Object_Class_Null	The object is not used.
RT_Object_Class_Thread	The object is a thread.
RT_Object_Class_Semaphore	The object is a semaphore.
RT_Object_Class_Mutex	The object is a mutex.
RT_Object_Class_Event	The object is a event.
RT_Object_Class_MailBox	The object is a mail box.
RT_Object_Class_MessageQueue	The object is a message queue.
RT_Object_Class_MemHeap	The object is a memory heap.
RT_Object_Class_MemPool	The object is a memory pool.
RT_Object_Class_Device	The object is a device.
RT_Object_Class_Timer	The object is a timer.
RT_Object_Class_Module	The object is a module.
RT_Object_Class_Memory	The object is a memory.
RT_Object_Class_Channel	The object is a channel
RT_Object_Class_ProcessGroup	The object is a process group
RT_Object_Class_Session	The object is a session
RT_Object_Class_Custom	The object is a custom object
RT_Object_Class_Unknown	The object is unknown.
RT_Object_Class_Static	The object is a static object.

6.2.3.5 Function Documentation

6.2.3.5.1 rt_object_get_information()

```
struct rt_object_information * rt_object_get_information (
            enum rt_object_class_type type)
```

This function will return the specified type of object information.

Parameters

type	is the type of object, which can be RT_Object_Class_Thread/Semaphore/Mutex... etc

Returns

the object type information or RT_NULL

6.2.3.5.2 rt_object_get_length()

```
int rt_object_get_length (
            enum rt_object_class_type type)
```

This function will return the length of object list in object container.

Parameters

type	is the type of object, which can be RT_Object_Class_Thread/Semaphore/Mutex... etc

Returns

the length of object list

6.2.3.5.3 rt_object_get_pointers()

```
int rt_object_get_pointers (
            enum rt_object_class_type type,
            rt_object_t * pointers,
            int maxlen)
```

This function will copy the object pointer of the specified type, with the maximum size specified by maxlen.

Parameters

type	is the type of object, which can be RT_Object_Class_Thread/Semaphore/Mutex... etc
pointers	is the pointer will be saved to.
maxlen	is the maximum number of pointers can be saved.

Returns

the copied number of object pointers.

6.2.3.5.4 rt_object_init()

```
void rt_object_init (
            struct rt_object * object,
            enum rt_object_class_type type,
            const char * name)
```

This function will initialize an object and add it to object system management.

Parameters

object	The specified object to be initialized. The object pointer that needs to be initialized must point to a specific object memory block, not a null pointer or a wild pointer.
type	The object type. The type of the object must be a enumeration type listed in rt_object_class_type, RT↩ _Object_Class_Static excluded. (For static objects, or objects initialized with the rt_object_init interface, the system identifies it as an RT_Object_Class_Static type)
name	Name of the object. In system, the object's name must be unique. Each object can be set to a name, and the maximum length for the name is specified by RT_NAME_MAX. The system does not care if it uses '\0' as a terminal symbol.

6.2.3.5.5 rt_object_detach()

```
void rt_object_detach (
            rt_object_t object)
```

This function will detach a static object from object system, and the memory of static object is not freed.

Parameters

object	the specified object to be detached.

6.2.3.5.6 rt_object_allocate()

```
rt_object_t rt_object_allocate (
            enum rt_object_class_type type,
            const char * name)
```

This function will allocate an object from object system.

Parameters

type	Type of object. The type of the allocated object can only be of type rt_object_class_type other than RT_Object_Class_Static. In addition, the type of object allocated through this interface is dynamic, not static.
name	Name of the object. In system, the object's name must be unique. Each object can be set to a name, and the maximum length for the name is specified by RT_NAME_MAX. The system does not care if it uses '\0' as a terminal symbol.

Returns

object handle allocated successfully, or RT_NULL if no memory can be allocated.

6.2.3.5.7 rt_object_delete()

```
void rt_object_delete (
            rt_object_t object)
```

This function will delete an object and release object memory.

Parameters

object	The specified object to be deleted.

6.2.3.5.8 rt_object_is_systemobject()

```
rt_bool_t rt_object_is_systemobject (
            rt_object_t object)
```

This function will judge the object is system object or not.

Note

Normally, the system object is a static object and the type of object set to RT_Object_Class_Static.

Parameters

object	The specified object to be judged.

Returns

RT_TRUE if a system object, RT_FALSE for others.

6.2.3.5.9 rt_object_get_type()

```
rt_uint8_t rt_object_get_type (
            rt_object_t object)
```

This function will return the type of object without RT_Object_Class_Static flag.

Parameters

object	is the specified object to be get type.

Returns

the type of object.

6.2.3.5.10 rt_object_for_each()

```
rt_err_t rt_object_for_each (
             rt_uint8_t type,
             rt_object_iter_t iter,
             void * data)
```

This function will iterate through each object from object container.

Parameters

type	is the type of object
iter	is the iterator
data	is the specified data passed to iterator

Returns

RT_EOK on succeed, otherwise the error from `iter`

Note

this function shall not be invoked in interrupt status.

6.2.3.5.11 rt_object_find()

```
rt_object_t rt_object_find (
             const char * name,
             rt_uint8_t type)
```

This function will find specified name object from object container.

Parameters

name	is the specified name of object.
type	is the type of object

Returns

the found object or RT_NULL if there is no this object in object container.

Note

this function shall not be invoked in interrupt status.

6.2.3.5.12 rt_object_get_name()

```
rt_err_t rt_object_get_name (
             rt_object_t object,
             char * name,
             rt_uint8_t name_size)
```

This function will return the name of the specified object container.

Parameters

object	the specified object to be get name
name	buffer to store the object name string
name_size	maximum size of the buffer to store object name

Returns

-RT_EINVAL if any parameter is invalid or RT_EOK if the operation is successfully executed

Note

this function shall not be invoked in interrupt status

6.2.3.5.13 rt_custom_object_create()

```
rt_object_t rt_custom_object_create (
             const char * name,
             void * data,
             rt_err_t(* data_destroy )(void *))
```

This function will create a custom object container.

Parameters

name	the specified name of object.
data	the custom data
data_destroy	the custom object destroy callback

Returns

the found object or RT_NULL if there is no this object in object container.

Note

this function shall not be invoked in interrupt status.

6.2.3.5.14 rt_custom_object_destroy()

```
rt_err_t rt_custom_object_destroy (
             rt_object_t obj)
```

This function will destroy a custom object container.

Parameters

obj	the specified name of object.

Note

this function shall not be invoked in interrupt status.

6.2.4 Thread Management

Collaboration diagram for Thread Management:

Data Structures

- struct rt_cpu_usage_stats
- struct rt_interrupt_context
- struct rt_thread

Macros

- #define RT_THREAD_INIT 0x00
- #define RT_THREAD_CLOSE 0x01
- #define RT_THREAD_READY 0x02
- #define RT_THREAD_RUNNING 0x03
- #define RT_THREAD_SUSPEND_INTERRUPTIBLE (RT_THREAD_SUSPEND_MASK)
- #define RT_THREAD_SUSPEND_KILLABLE (RT_THREAD_SUSPEND_MASK | RT_SIGNAL_COMMON↩ _WAKEUP_MASK)
- #define RT_THREAD_SUSPEND_UNINTERRUPTIBLE (RT_THREAD_SUSPEND_MASK | RT_SIGNAL↩ _COMMON_WAKEUP_MASK | RT_SIGNAL_KILL_WAKEUP_MASK)
- #define RT_THREAD_STAT_YIELD 0x08
- #define RT_THREAD_STAT_SIGNAL 0x10
- #define RT_THREAD_STAT_SIGNAL_WAIT 0x20
- #define RT_THREAD_STAT_SIGNAL_PENDING 0x40
- #define RT_THREAD_CTRL_STARTUP 0x00
- #define RT_THREAD_CTRL_CLOSE 0x01
- #define RT_THREAD_CTRL_CHANGE_PRIORITY 0x02
- #define RT_THREAD_CTRL_INFO 0x03
- #define RT_THREAD_CTRL_BIND_CPU 0x04
- #define RT_THREAD_CTRL_RESET_PRIORITY 0x05

Typedefs

- typedef struct rt_interrupt_context * rt_interrupt_context_t
- typedef void(*) rt_thread_inited_hookproto_t(rt_thread_t thread)

Functions

- rt_err_t rt_thread_idle_delhook (void(*hook)(void))
- void rt_thread_idle_init (void)
- rt_thread_t rt_thread_idle_gethandler (void)
- rt_bool_t rt_thread_is_idle_thread (rt_thread_t thread)
- void rt_system_scheduler_init (void)
- void rt_system_scheduler_start (void)
- void rt_scheduler_ipi_handler (int vector, void *param)
- rt_err_t rt_sched_lock (rt_sched_lock_level_t *plvl)
- rt_err_t rt_sched_unlock (rt_sched_lock_level_t level)
- rt_bool_t rt_sched_is_locked (void)
- rt_err_t rt_sched_unlock_n_resched (rt_sched_lock_level_t level)
- void rt_schedule (void)
- void rt_scheduler_do_irq_switch (void *context)
- void rt_sched_insert_thread (struct rt_thread *thread)
- void rt_sched_remove_thread (struct rt_thread *thread)
- void rt_sched_thread_init_priv (struct rt_thread *thread, rt_uint32_t tick, rt_uint8_t priority)
- void rt_sched_thread_startup (struct rt_thread *thread)
- void rt_sched_post_ctx_switch (struct rt_thread *thread)

- void rt_exit_critical_safe (rt_base_t critical_level)
- rt_base_t rt_enter_critical (void)
- void rt_exit_critical (void)
- rt_uint16_t rt_critical_level (void)
- rt_err_t rt_sched_thread_bind_cpu (struct rt_thread *thread, int cpu)
- int rt_thread_kill (rt_thread_t tid, int sig)
- rt_err_t rt_thread_init (struct rt_thread *thread, const char *name, void(*entry)(void *parameter), void *parameter, void *stack_start, rt_uint32_t stack_size, rt_uint8_t priority, rt_uint32_t tick)
- rt_thread_t rt_thread_self (void)
- rt_err_t rt_thread_startup (rt_thread_t thread)
- rt_err_t rt_thread_close (rt_thread_t thread)
- rt_err_t rt_thread_detach (rt_thread_t thread)
- rt_thread_t rt_thread_create (const char *name, void(*entry)(void *parameter), void *parameter, rt_uint32_t stack_size, rt_uint8_t priority, rt_uint32_t tick)
- rt_err_t rt_thread_delete (rt_thread_t thread)
- rt_err_t rt_thread_yield (void)
- rt_err_t rt_thread_delay (rt_tick_t tick)
- rt_err_t rt_thread_delay_until (rt_tick_t *tick, rt_tick_t inc_tick)
- rt_err_t rt_thread_mdelay (rt_int32_t ms)
- rt_err_t rt_thread_control (rt_thread_t thread, int cmd, void *arg)
- rt_err_t rt_thread_suspend_to_list (rt_thread_t thread, rt_list_t *susp_list, int ipc_flags, int suspend_flag)
- rt_err_t rt_thread_suspend_with_flag (rt_thread_t thread, int suspend_flag)
- rt_err_t rt_thread_suspend (rt_thread_t thread)
- rt_err_t rt_thread_resume (rt_thread_t thread)
- rt_thread_t rt_thread_find (char *name)
- rt_err_t rt_thread_get_name (rt_thread_t thread, char *name, rt_uint8_t name_size)
- rt_err_t rt_thread_idle_sethook (void(*hook)(void))

6.2.4.1 Detailed Description

See Thread Management.

6.2.4.2 Macro Definition Documentation

6.2.4.2.1 RT_THREAD_INIT

```
#define RT_THREAD_INIT 0x00
```

Initialized status

6.2.4.2.2 RT_THREAD_CLOSE

```
#define RT_THREAD_CLOSE 0x01
```

Closed status

6.2.4.2.3 RT_THREAD_READY

```
#define RT_THREAD_READY 0x02
```

Ready status

6.2.4.2.4 RT_THREAD_RUNNING

```
#define RT_THREAD_RUNNING 0x03
```

Running status

6.2.4.2.5 RT_THREAD_SUSPEND_INTERRUPTIBLE

```
#define RT_THREAD_SUSPEND_INTERRUPTIBLE (RT_THREAD_SUSPEND_MASK)
```

Suspend interruptable 0x4

6.2.4.2.6 RT_THREAD_SUSPEND_KILLABLE

```
#define RT_THREAD_SUSPEND_KILLABLE (RT_THREAD_SUSPEND_MASK | RT_SIGNAL_COMMON_WAKEUP_MASK)
```

Suspend with killable 0x6

6.2.4.2.7 RT_THREAD_SUSPEND_UNINTERRUPTIBLE

```
#define RT_THREAD_SUSPEND_UNINTERRUPTIBLE (RT_THREAD_SUSPEND_MASK | RT_SIGNAL_COMMON_WAKEUP_↩
MASK | RT_SIGNAL_KILL_WAKEUP_MASK)
```

Suspend with uninterruptable 0x7

6.2.4.2.8 RT_THREAD_STAT_YIELD

```
#define RT_THREAD_STAT_YIELD 0x08
```

indicate whether remaining_tick has been reloaded since last schedule

6.2.4.2.9 RT_THREAD_STAT_SIGNAL

```
#define RT_THREAD_STAT_SIGNAL 0x10
```

task hold signals

6.2.4.2.10 RT_THREAD_STAT_SIGNAL_WAIT

```
#define RT_THREAD_STAT_SIGNAL_WAIT 0x20
```

task is waiting for signals

6.2.4.2.11 RT_THREAD_STAT_SIGNAL_PENDING

```
#define RT_THREAD_STAT_SIGNAL_PENDING 0x40
```

signals is held and it has not been procressed

6.2.4.2.12 RT_THREAD_CTRL_STARTUP

```
#define RT_THREAD_CTRL_STARTUP 0x00
```

thread control command definitions Startup thread.

6.2.4.2.13 RT_THREAD_CTRL_CLOSE

```
#define RT_THREAD_CTRL_CLOSE 0x01
```

Close thread.

6.2.4.2.14 RT_THREAD_CTRL_CHANGE_PRIORITY

```
#define RT_THREAD_CTRL_CHANGE_PRIORITY 0x02
```

Change thread priority.

6.2.4.2.15 RT_THREAD_CTRL_INFO

```
#define RT_THREAD_CTRL_INFO 0x03
```

Get thread information.

6.2.4.2.16 RT_THREAD_CTRL_BIND_CPU

```
#define RT_THREAD_CTRL_BIND_CPU 0x04
```

Set thread bind cpu.

6.2.4.2.17 RT_THREAD_CTRL_RESET_PRIORITY

```
#define RT_THREAD_CTRL_RESET_PRIORITY 0x05
```

Reset thread priority.

6.2.4.3 Typedef Documentation

6.2.4.3.1 rt_interrupt_context_t

```
typedef struct rt_interrupt_context * rt_interrupt_context_t
```

interrupt/exception frame handling

6.2.4.3.2 rt_thread_inited_hookproto_t

```
typedef void(*) rt_thread_inited_hookproto_t(rt_thread_t thread)
```

Sets a hook function when a thread is initialized.

Parameters

thread	is the target thread that initializing

6.2.4.4 Function Documentation

6.2.4.4.1 rt_thread_idle_delhook()

```
rt_err_t rt_thread_idle_delhook (
            void(* hook )(void))
```

delete the idle hook on hook list.

Parameters

hook	the specified hook function.

Returns

RT_EOK: delete OK. -RT_ENOSYS: hook was not found.

6.2.4.4.2 rt_thread_idle_init()

```
void rt_thread_idle_init (
            void )
```

This function will initialize idle thread, then start it.

Note

this function must be invoked when system init.

6.2.4.4.3 rt_thread_idle_gethandler()

```
rt_thread_t rt_thread_idle_gethandler (
            void )
```

This function will get the handler of the idle thread.

6.2.4.4.4 rt_thread_is_idle_thread()

```
rt_bool_t rt_thread_is_idle_thread (
            rt_thread_t thread)
```

Check whether the specified thread is one of the system idle threads.
RT-Thread creates an idle thread for each CPU. These idle threads are special scheduler-owned threads that act as the fallback runnable threads when no other ready thread exists.
This helper is mainly used for defensive checks in code paths that may block or suspend a thread, because an idle thread must never enter a blocking or suspended state. Suspending an idle thread may leave the system with no ready thread and break scheduling.

Parameters

thread	The thread to test.

Returns

RT_TRUE if `thread` is an idle thread of any CPU; otherwise RT_FALSE.

Note

- In SMP configurations, there is one idle thread per CPU, so this function checks against all idle thread objects.
- Passing RT_NULL returns RT_FALSE.

6.2.4.4.5 rt_system_scheduler_init()

```
void rt_system_scheduler_init (
            void )
```

Initialize the system scheduler.

Note

This function must be called before any thread scheduling can occur. It prepares the scheduler data structures for multi-core operation.

This function has both MP version and UP version.

6.2.4.4.6 rt_system_scheduler_start()

```
void rt_system_scheduler_start (
            void )
```

Start the system scheduler and switch to the highest priority thread.

Note

This function will not return after successful execution. It performs the initial thread switch during system startup. The scheduler must be initialized before calling this function.

This function has both MP version and UP version.

6.2.4.4.7 rt_scheduler_ipi_handler()

```
void rt_scheduler_ipi_handler (
            int vector,
            void * param)
```

This function will handle IPI interrupt and do a scheduling in system.
This function should be invoke or register as ISR in BSP.

Parameters

vector	The number of IPI interrupt for system scheduling.
param	Not used, and can be set to RT_NULL.

Note

This function only has MP version.

6.2.4.4.8 rt_sched_lock()

```
rt_err_t rt_sched_lock (
            rt_sched_lock_level_t * plvl)
```

Lock the system scheduler.

Parameters

plvl	Pointer to the object where lock level stores to

Returns

rt_err_t

- RT_EOK on success
- -RT_EINVAL if plvl is NULL

Note

This function has both MP version and UP version.

6.2.4.4.9 rt_sched_unlock()

```
rt_err_t rt_sched_unlock (
            rt_sched_lock_level_t level)
```

Unlock the scheduler and restore the interrupt level.

Parameters

level	The interrupt level to restore (previously saved by rt_sched_lock)

Returns

rt_err_t Always returns RT_EOK

Note

This function has both MP version and UP version.

6.2.4.4.10 rt_sched_is_locked()

```
rt_bool_t rt_sched_is_locked (
            void )
```

Check if the scheduler is currently locked.
This function checks the scheduler lock status in a thread-safe manner by temporarily disabling interrupts to get consistent state.

Returns

rt_bool_t Returns RT_TRUE if scheduler is locked, RT_FALSE otherwise

Note

This function only has MP version.

6.2.4.4.11 rt_sched_unlock_n_resched()

```
rt_err_t rt_sched_unlock_n_resched (
            rt_sched_lock_level_t level)
```

Unlock scheduler and perform rescheduling if needed.

Parameters

level	The scheduler lock level obtained from rt_sched_lock()

Returns

rt_err_t

- RT_EOK: Success
- -RT_EBUSY: Scheduler not available
- -RT_ESCHEDISR: Called in interrupt context
- -RT_ESCHEDLOCKED: Scheduler still locked by others

This function:

- Releases scheduler lock at specified level
- Checks if rescheduling is needed
- If needed, finds highest priority thread and switches to it
- Processes pending signals for current thread
- Handles various error conditions

Note

Must be called in pair with rt_sched_lock() May trigger context switch if conditions met

This function has both MP version and UP version.

6.2.4.4.12 rt_schedule()

```
void rt_schedule (
            void )
```

This function will perform one scheduling. It will select one thread with the highest priority level in global ready queue or local ready queue, then switch to it.

Note

This function has both MP version and UP version.

6.2.4.4.13 rt_scheduler_do_irq_switch()

```
void rt_scheduler_do_irq_switch (
            void * context)
```

Perform thread scheduling after an interrupt context switch.

Parameters

context	The interrupt context pointer

This function handles scheduling when returning from interrupt context:

- Checks if scheduling is needed (irq_switch_flag set)

- If needed, finds highest priority ready thread
- Performs context switch to new thread if available
- Handles cases where scheduler is locked or still in interrupt context
- Processes pending signals before scheduling

Note

This function only has MP version.

6.2.4.4.14 rt_sched_insert_thread()

```
void rt_sched_insert_thread (
            struct rt_thread * thread)
```

This function will insert a thread to the system ready queue. The state of thread will be set as READY and the thread will be removed from suspend queue.

Parameters

thread	The thread to be inserted.

Note

Please do not invoke this function in user application. Caller must hold the scheduler lock

This function has both MP version and UP version.

6.2.4.4.15 rt_sched_remove_thread()

```
void rt_sched_remove_thread (
            struct rt_thread * thread)
```

This function will remove a thread from system ready queue.

Parameters

thread	The thread to be removed.

Note

Please do not invoke this function in user application.

This function has both MP version and UP version.

6.2.4.4.16 rt_sched_thread_init_priv()

```
void rt_sched_thread_init_priv (
            struct rt_thread * thread,
            rt_uint32_t tick,
            rt_uint8_t priority)
```

Initialize thread's scheduling private data.

Parameters

thread	The thread to be initialized

tick	Initial time slice value for the thread
priority	Initial priority of the thread

This function performs the following initialization:

- Initializes thread's ready list node
- Sets initial and current priority (must be < RT_THREAD_PRIORITY_MAX)
- Initializes priority bitmasks (handles >32 priorities if needed)
- Sets initial time slice values
- For SMP systems, initializes critical section nesting counter

Note

This function has both MP version and UP version.

6.2.4.4.17 rt_sched_thread_startup()

```
void rt_sched_thread_startup (
             struct rt_thread * thread)
```

Initialize thread scheduling attributes for startup.

Parameters

thread	The thread to be initialized

This function:

- For systems with >32 priorities:
  - Sets priority number (5 bits)
  - Initializes number_mask and high_mask (3 bits)
- For systems with <=32 priorities:
  - Initializes number_mask
- Sets thread state to SUSPEND

Note

This is a lockless operation as it's called during thread creation when no concurrent access is possible

This function has both MP version and UP version.

6.2.4.4.18 rt_sched_post_ctx_switch()

```
void rt_sched_post_ctx_switch (
             struct rt_thread * thread)
```

Update thread scheduling status after context switch.

Parameters

thread	The thread that will be running after the context switch

This function performs critical post-context-switch operations:

- Verifies interrupts are disabled (RT_ASSERT)
- Releases scheduler lock from previous thread if exists
- Updates current thread pointer for the CPU
- Must be called with interrupts disabled

Note

this operation is taken as an atomic operation of the update of SP. Since the local irq is disabled, it's okay to assume that the stack will not be modified meanwhile.

This function only has MP version.

6.2.4.4.19 rt_exit_critical_safe()

```
void rt_exit_critical_safe (
            rt_base_t critical_level)
```

Safely exit critical section (non-debug version).

Parameters

critical_level	The expected critical level (unused in non-debug build)

This is the non-debug version that simply calls rt_exit_critical(). The critical_level parameter is ignored in this implementation.

Note

This function has both MP version and UP version.

6.2.4.4.20 rt_enter_critical()

```
rt_base_t rt_enter_critical (
            void )
```

Enter a critical section and lock the scheduler.

Returns

The critical level after entering (nesting count) Returns -RT_EINVAL if scheduler is unavailable

This function:

- Disables interrupts to ensure atomic operation
- Gets current thread context
- Increments critical section nesting counter
- Returns the new nesting level
- Handles both hardware-assisted and software thread context cases

Note

Must be paired with rt_exit_critical() Can be nested, each call must have matching exit call

This function has both MP version and UP version.

6.2.4.4.21 rt_exit_critical()

```
void rt_exit_critical (
            void )
```

Exit a critical section and unlock the scheduler.
This function performs the following operations:

- Gets current thread context (using hardware-assisted or software method)
- Decrements critical section nesting counter
- If nesting level reaches 0:
  - Checks if rescheduling is needed
  - Clears critical switch flag
  - Performs rescheduling if needed
- Verifies critical section nesting is valid (debug assertion)
- Restores interrupt state

Note

Must be called in pair with rt_enter_critical() Handles both hardware-assisted and software thread context cases May trigger rescheduling if conditions met

This function has both MP version and UP version.

6.2.4.4.22 rt_critical_level()

```
rt_uint16_t rt_critical_level (
            void )
```

Get the scheduler lock level.

Returns

The level of the scheduler lock. 0 means unlocked.

Note

This function has both MP version and UP version.

6.2.4.4.23 rt_sched_thread_bind_cpu()

```
rt_err_t rt_sched_thread_bind_cpu (
            struct rt_thread * thread,
            int cpu)
```

Bind a thread to a specific CPU core.

Parameters

thread	The thread to be bound
cpu	The target CPU core index (RT_CPUS_NR for no binding)

Returns

rt_err_t

- RT_EOK: Operation successful

This function handles thread-CPU binding with the following logic:

- If thread is READY:
  - Removes from current ready queue
  - Updates bind CPU information
  - Inserts to new ready queue
  - Triggers rescheduling if needed
- If thread is RUNNING:
  - Updates bind CPU information
  - Sends IPI to target CPU if binding changed
  - Triggers rescheduling if needed
- For other states, just updates bind CPU info

Note

Caller must ensure scheduler is not locked before calling This function will acquire scheduler lock internally

This function has both MP version and UP version, but UP version does nothing and always returns -RT_↩ EINVAL.

6.2.4.4.24 rt_thread_kill()

```
int rt_thread_kill (
            rt_thread_t tid,
            int sig)
```

This function can be used to send any signal to any thread.

Parameters

tid	is a pointer to the thread that receives the signal.
sig	is a specific signal value (range: 0 ∼ RT_SIG_MAX).

Returns

Return the operation status. When the return value is RT_EOK, the operation is successful. If the return value is any other values, it means that the signal send failed.

6.2.4.4.25 rt_thread_init()

```
rt_err_t rt_thread_init (
            struct rt_thread * thread,
            const char * name,
            void(* entry )(void *parameter),
            void * parameter,
            void * stack_start,
            rt_uint32_t stack_size,
            rt_uint8_t priority,
            rt_uint32_t tick)
```

This function will initialize a thread. It's used to initialize a static thread object.

Parameters

thread	Thread handle. Thread handle is provided by the user and points to the corresponding thread control block memory address.
name	Name of the thread (shall be unique); the maximum length of the thread name is specified by the RT_NAME_MAX macro defined in rtconfig.h, and the extra part is automatically truncated.
entry	Entry function of thread.
parameter	Parameter of thread entry function.
stack_start	Start address of thread stack.
stack_size	Size of thread stack in bytes. Stack space address alignment is required in most systems (for example, alignment to 4-byte addresses in the ARM architecture).
priority	Priority of thread. The priority range is based on the system configuration (macro definition RT_↩ THREAD_PRIORITY_MAX in rtconfig.h). If 256 levels of priority are supported, the range is from 0 to 255. The smaller the value, the higher the priority, and 0 is the highest priority.
tick	Time slice if there are same priority thread. The unit of the time slice (tick) is the tick of the operating system. When there are threads with the same priority in the system, this parameter specifies the maximum length of time of a thread for one schedule. At the end of this time slice run, the scheduler automatically selects the next ready state of the same priority thread to run.

Returns

Return the operation status. If the return value is RT_EOK, the function is successfully executed. If the return value is any other values, it means this operation failed.

6.2.4.4.26 rt_thread_self()

```
rt_thread_t rt_thread_self (
            void )
```

This function will return self thread object.

Returns

The self thread object. If returns RT_NULL, it means that the scheduler has not started yet.

6.2.4.4.27 rt_thread_startup()

```
rt_err_t rt_thread_startup (
            rt_thread_t thread)
```

This function will start a thread and put it to system ready queue.

Parameters

thread	Handle of the thread to be started.

Returns

Return the operation status. If the return value is RT_EOK, the function is successfully executed. If the return value is any other values, it means this operation failed.

6.2.4.4.28 rt_thread_close()

```
rt_err_t rt_thread_close (
            rt_thread_t thread)
```

This function will close a thread. The thread object will be removed from thread queue and detached/deleted from the system object management. It's different from rt_thread_delete or rt_thread_detach that this will not enqueue the closing thread to cleanup queue.

Parameters

thread	is the thread to be closed.

Returns

Return the operation status. If the return value is RT_EOK, the function is successfully executed. If the return value is any other values, it means this operation failed.

6.2.4.4.29 rt_thread_detach()

```
rt_err_t rt_thread_detach (
            rt_thread_t thread)
```

This function will detach a thread. The thread object will be removed from thread queue and detached/deleted from the system object management.

Parameters

thread	Handle of the thread to be deleted. The thread must be initialized by `rt_thread_init()`.

Returns

Return the operation status. If the return value is `RT_EOK`, the function is successfully executed. If the return value is any other values, it means this operation failed.

6.2.4.4.30 rt_thread_create()

```
rt_thread_t rt_thread_create (
            const char * name,
            void(* entry )(void *parameter),
            void * parameter,
            rt_uint32_t stack_size,
            rt_uint8_t priority,
            rt_uint32_t tick)
```

This function will create a thread object and allocate thread object memory. and stack.

Parameters

name	The name of the thread (shall be unique.); the maximum length of the thread name is specified by macro `RT_NAME_MAX` in `rtconfig.h`, and the extra part is automatically truncated.
entry	Entry function of thread.
parameter	Parameter of thread entry function.
stack_size	Size of thread stack in bytes.
priority	Priority of thread. The priority range is based on the system configuration (macro definition `RT_THREAD_PRIORITY_MAX` in rtconfig.h). If 256-level priority is supported, then the range is from 0 to 255. The smaller the value, the higher the priority, and 0 is the highest priority.
tick	Time slice if there are same priority thread. The unit of the time slice (tick) is the tick of the operating system. When there are threads with the same priority in the system, this parameter specifies the maximum length of time of a thread for one schedule. At the end of this time slice run, the scheduler automatically selects the next ready state of the same priority thread to run.

Returns

If the return value is a rt_thread structure pointer, the function is successfully executed. If the return value is RT_NULL, it means this operation failed.

6.2.4.4.31 rt_thread_delete()

```
rt_err_t rt_thread_delete (
            rt_thread_t thread)
```

This function will delete a thread. The thread object will be removed from thread queue and deleted from system object management in the idle thread.

Parameters

thread	Handle of the thread to be deleted.

Returns

Return the operation status. If the return value is RT_EOK, the function is successfully executed. If the return value is any other values, it means this operation failed.

6.2.4.4.32 rt_thread_yield()

```
rt_err_t rt_thread_yield (
            void )
```

This function will let current thread yield processor, and scheduler will choose the highest thread to run. After yield processor, the current thread is still in READY state.

Returns

Return the operation status. If the return value is RT_EOK, the function is successfully executed. If the return value is any other values, it means this operation failed.

6.2.4.4.33 rt_thread_delay()

```
rt_err_t rt_thread_delay (
            rt_tick_t tick)
```

This function will let current thread delay for some ticks.

Parameters

tick	The delay ticks, in units of 1 OS Tick.

Returns

Return the operation status. If the return value is RT_EOK, the function is successfully executed. If the return value is any other values, it means this operation failed.

6.2.4.4.34 rt_thread_delay_until()

```
rt_err_t rt_thread_delay_until (
            rt_tick_t * tick,
            rt_tick_t inc_tick)
```

This function will let current thread delay until (*tick + inc_tick).

Parameters

tick	is the tick of last wakeup.
inc_tick	is the increment tick.

Returns

Return the operation status. If the return value is RT_EOK, the function is successfully executed. If the return value is any other values, it means this operation failed.

6.2.4.4.35 rt_thread_mdelay()

```
rt_err_t rt_thread_mdelay (
            rt_int32_t ms)
```

This function will let current thread delay for some milliseconds.

Parameters

ms	The delay time in units of 1ms.

Returns

Return the operation status. If the return value is RT_EOK, the function is successfully executed. If the return value is any other values, it means this operation failed.

6.2.4.4.36 rt_thread_control()

```
rt_err_t rt_thread_control (
            rt_thread_t thread,
            int cmd,
            void * arg)
```

This function will control thread behaviors according to control command.

Parameters

thread	Handle of the thread to be controlled.
cmd	Control command, which includes. • RT_THREAD_CTRL_CHANGE_PRIORITY for changing priority level of thread. • RT_THREAD_CTRL_STARTUP for starting a thread, equivalent to the rt_thread_startup() function call. • RT_THREAD_CTRL_CLOSE for closing a thread, equivalent to the rt_thread_delete() function call. • RT_THREAD_CTRL_BIND_CPU for bind the thread to a CPU. • RT_THREAD_CTRL_RESET_PRIORITY for reset priority level of thread.
arg	Argument of control command.

Returns

Return the operation status. If the return value is RT_EOK, the function is successfully executed. If the return value is any other values, it means this operation failed.

6.2.4.4.37 rt_thread_suspend_to_list()

```
rt_err_t rt_thread_suspend_to_list (
            rt_thread_t thread,
            rt_list_t * susp_list,
            int ipc_flags,
            int suspend_flag)
```

This function will suspend the specified thread and change it to suspend state.

Note

This function ONLY can suspend current thread itself. rt_thread_suspend(rt_thread_self());

Do not use the rt_thread_suspend to suspend other threads. You have no way of knowing what code a thread is executing when you suspend it. If you suspend a thread while sharing a resouce with other threads and occupying this resouce, starvation can occur very easily.

Parameters

thread	the thread to be suspended.
susp_list	the list thread enqueued to. RT_NULL if no list.
ipc_flags	is a flag for the thread object to be suspended. It determines how the thread is suspended. The flag can be ONE of the following values: RT_IPC_FLAG_PRIO The pending threads will queue in order of priority. RT_IPC_FLAG_FIFO The pending threads will queue in the first-in-first-out method (also known as first-come-first-served (FCFS) scheduling strategy). NOTE: RT_IPC_↩ FLAG_FIFO is a non-real-time scheduling mode. It is strongly recommended to use RT_IPC↩ _FLAG_PRIO to ensure the thread is real-time UNLESS your applications concern about the first-in-first-out principle, and you clearly understand that all threads involved in this semaphore will become non-real-time threads.
suspend_flag	status flag of the thread to be suspended.

Returns

Return the operation status. If the return value is RT_EOK, the function is successfully executed. If the return value is any other values, it means this operation failed.

enqueue thread on the push list before leaving critical region of scheduler, so we won't miss notification of async events.

6.2.4.4.38 rt_thread_suspend_with_flag()

```
rt_err_t rt_thread_suspend_with_flag (
            rt_thread_t thread,
            int suspend_flag)
```

This function will suspend the specified thread and change it to suspend state.

Note

This function ONLY can suspend current thread itself. rt_thread_suspend(rt_thread_self());

Do not use the rt_thread_suspend to suspend other threads. You have no way of knowing what code a thread is executing when you suspend it. If you suspend a thread while sharing a resouce with other threads and occupying this resouce, starvation can occur very easily.

Parameters

thread	the thread to be suspended.
suspend_flag	status flag of the thread to be suspended.

Returns

Return the operation status. If the return value is RT_EOK, the function is successfully executed. If the return value is any other values, it means this operation failed.

6.2.4.4.39 rt_thread_suspend()

```
rt_err_t rt_thread_suspend (
             rt_thread_t thread)
```

This function will suspend the specified thread and change it to suspend state.

Note

This function can suspend both the current thread itself and other threads. Please use this API with extreme caution when suspending other threads.

When suspending the current thread: rt_thread_suspend(rt_thread_self()); This is generally safe as the thread voluntarily suspends itself.
When suspending other threads: You have no way of knowing what code another thread is executing when you suspend it. If you suspend a thread while it is sharing a resource with other threads and occupying this resource (such as holding a mutex, semaphore, or other synchronization objects), deadlock or resource starvation can occur very easily.

Warning

Suspending other threads arbitrarily can lead to:

- Deadlock situations
- Resource starvation
- System instability
- Unpredictable behavior

Only suspend other threads when you have full control over the system state and understand the implications.

Parameters

thread	Handle of the thread to be suspended. Can be the current thread (rt_thread_self()) or any other thread.

Returns

Return the operation status. If the return value is RT_EOK, the function is successfully executed. If the return value is any other values, it means this operation failed.

6.2.4.4.40 rt_thread_resume()

```
rt_err_t rt_thread_resume (
             rt_thread_t thread)
```

This function will resume a thread and put it to system ready queue.

Parameters

thread	Handle of the thread to be resumed.

Returns

Return the operation status. If the return value is RT_EOK, the function is successfully executed. If the return value is any other values, it means this operation failed.

RT_ESCHEDLOCKED indicates that the current thread is in a critical section, rather than 'thread' can't be resumed. Therefore, we can ignore this error.

6.2.4.4.41 rt_thread_find()

```
rt_thread_t rt_thread_find (
            char * name)
```

This function will find the specified thread.

Note

Please don't invoke this function in interrupt status.

Parameters

name	is the name of thread finding.

Returns

If the return value is a rt_thread structure pointer, the function is successfully executed. If the return value is RT_NULL, it means this operation failed.

6.2.4.4.42 rt_thread_get_name()

```
rt_err_t rt_thread_get_name (
             rt_thread_t thread,
             char * name,
             rt_uint8_t name_size)
```

This function will return the name of the specified thread.

Note

Please don't invoke this function in interrupt status

Parameters

thread	the thread to retrieve thread name
name	buffer to store the thread name string
name_size	maximum size of the buffer to store the thread name

Returns

If the return value is RT_EOK, the function is successfully executed If the return value is -RT_EINVAL, it means this operation failed

6.2.4.4.43 rt_thread_idle_sethook()

```
rt_err_t rt_thread_idle_sethook (
             void(* hook )(void))
```

This function sets a hook function to idle thread loop. When the system performs idle loop, this hook function should be invoked.

Parameters

hook	the specified hook function.

Returns

RT_EOK: set OK. -RT_EFULL: hook list is full.

Note

the hook function must be simple and never be blocked or suspend.

6.2.5 Clock and Timer Management

Collaboration diagram for Clock and Timer Management:

Data Structures

- struct rt_timer

Macros

- #define RT_TIMER_FLAG_DEACTIVATED 0x0
- #define RT_TIMER_FLAG_ACTIVATED 0x1
- #define RT_TIMER_FLAG_ONE_SHOT 0x0
- #define RT_TIMER_FLAG_PERIODIC 0x2
- #define RT_TIMER_FLAG_HARD_TIMER 0x0
- #define RT_TIMER_FLAG_SOFT_TIMER 0x4
- #define RT_TIMER_FLAG_THREAD_TIMER (0x8 | RT_TIMER_FLAG_HARD_TIMER)
- #define RT_TIMER_CTRL_SET_TIME 0x0
- #define RT_TIMER_CTRL_GET_TIME 0x1
- #define RT_TIMER_CTRL_SET_ONESHOT 0x2
- #define RT_TIMER_CTRL_SET_PERIODIC 0x3
- #define RT_TIMER_CTRL_GET_STATE 0x4
- #define RT_TIMER_CTRL_GET_REMAIN_TIME 0x5
- #define RT_TIMER_CTRL_GET_FUNC 0x6
- #define RT_TIMER_CTRL_SET_FUNC 0x7
- #define RT_TIMER_CTRL_GET_PARM 0x8
- #define RT_TIMER_CTRL_SET_PARM 0x9
- #define RT_TIMER_SKIP_LIST_MASK 0x3

Typedefs

- typedef void(∗) rt_timer_func_t(void ∗parameter)

Functions

- rt_tick_t rt_tick_get (void)
- rt_tick_t rt_tick_get_delta (rt_tick_t base)
- void rt_tick_set (rt_tick_t tick)
- void rt_tick_increase (void)
- void rt_tick_increase_tick (rt_tick_t tick)
- rt_tick_t rt_tick_from_millisecond (rt_int32_t ms)
- rt_weak rt_tick_t rt_tick_get_millisecond (void)
- void rt_timer_init (rt_timer_t timer, const char ∗name, void(∗timeout)(void ∗parameter), void ∗parameter, rt↩_tick_t time, rt_uint8_t flag)
- rt_err_t rt_timer_detach (rt_timer_t timer)
- rt_timer_t rt_timer_create (const char ∗name, void(∗timeout)(void ∗parameter), void ∗parameter, rt_tick_t time, rt_uint8_t flag)
- rt_err_t rt_timer_delete (rt_timer_t timer)
- rt_err_t rt_timer_start (rt_timer_t timer)
- rt_err_t rt_timer_stop (rt_timer_t timer)

- rt_err_t rt_timer_control (rt_timer_t timer, int cmd, void *arg)
- void rt_timer_check (void)
- rt_tick_t rt_timer_next_timeout_tick (void)
- void rt_system_timer_init (void)
- void rt_system_timer_thread_init (void)

6.2.5.1 Detailed Description

See Clock & Timer Management.

6.2.5.2 Macro Definition Documentation

6.2.5.2.1 RT_TIMER_FLAG_DEACTIVATED

```
#define RT_TIMER_FLAG_DEACTIVATED 0x0
```

clock & timer macros timer is deactive

6.2.5.2.2 RT_TIMER_FLAG_ACTIVATED

```
#define RT_TIMER_FLAG_ACTIVATED 0x1
```

timer is active

6.2.5.2.3 RT_TIMER_FLAG_ONE_SHOT

```
#define RT_TIMER_FLAG_ONE_SHOT 0x0
```

one shot timer

6.2.5.2.4 RT_TIMER_FLAG_PERIODIC

```
#define RT_TIMER_FLAG_PERIODIC 0x2
```

periodic timer

6.2.5.2.5 RT_TIMER_FLAG_HARD_TIMER

```
#define RT_TIMER_FLAG_HARD_TIMER 0x0
```

hard timer,the timer's callback function will be called in tick isr.

6.2.5.2.6 RT_TIMER_FLAG_SOFT_TIMER

```
#define RT_TIMER_FLAG_SOFT_TIMER 0x4
```

soft timer,the timer's callback function will be called in timer thread.

6.2.5.2.7 RT_TIMER_FLAG_THREAD_TIMER

```
#define RT_TIMER_FLAG_THREAD_TIMER (0x8 | RT_TIMER_FLAG_HARD_TIMER)
```

thread timer that cooperates with scheduler directly

6.2.5.2.8 RT_TIMER_CTRL_SET_TIME

```
#define RT_TIMER_CTRL_SET_TIME 0x0
```

set timer control command

6.2.5.2.9 RT_TIMER_CTRL_GET_TIME

```
#define RT_TIMER_CTRL_GET_TIME 0x1
```

get timer control command

6.2.5.2.10 RT_TIMER_CTRL_SET_ONESHOT

```
#define RT_TIMER_CTRL_SET_ONESHOT 0x2
```

change timer to one shot

6.2.5.2.11 RT_TIMER_CTRL_SET_PERIODIC

```
#define RT_TIMER_CTRL_SET_PERIODIC 0x3
```
change timer to periodic

6.2.5.2.12 RT_TIMER_CTRL_GET_STATE

```
#define RT_TIMER_CTRL_GET_STATE 0x4
```
get timer run state active or deactive

6.2.5.2.13 RT_TIMER_CTRL_GET_REMAIN_TIME

```
#define RT_TIMER_CTRL_GET_REMAIN_TIME 0x5
```
get the remaining hang time

6.2.5.2.14 RT_TIMER_CTRL_GET_FUNC

```
#define RT_TIMER_CTRL_GET_FUNC 0x6
```
get timer timeout func

6.2.5.2.15 RT_TIMER_CTRL_SET_FUNC

```
#define RT_TIMER_CTRL_SET_FUNC 0x7
```
set timer timeout func

6.2.5.2.16 RT_TIMER_CTRL_GET_PARM

```
#define RT_TIMER_CTRL_GET_PARM 0x8
```
get timer parameter

6.2.5.2.17 RT_TIMER_CTRL_SET_PARM

```
#define RT_TIMER_CTRL_SET_PARM 0x9
```
set timer parameter

6.2.5.2.18 RT_TIMER_SKIP_LIST_MASK

```
#define RT_TIMER_SKIP_LIST_MASK 0x3
```
Timer skips the list mask

6.2.5.3 Typedef Documentation

6.2.5.3.1 rt_timer_func_t

```
typedef void(*) rt_timer_func_t(void *parameter)
```
timeout handler of rt_timer

6.2.5.4 Function Documentation

6.2.5.4.1 rt_tick_get()

```
rt_tick_t rt_tick_get (
            void )
```
This function will return current tick from operating system startup.

Returns

Return current tick.

6.2.5.4.2 rt_tick_get_delta()

```
rt_tick_t rt_tick_get_delta (
            rt_tick_t base)
```

This function will return delta tick from base.

Parameters

base	to consider

Returns

Return delta tick.

6.2.5.4.3 rt_tick_set()

```
void rt_tick_set (
            rt_tick_t tick)
```

This function will set current tick.

Parameters

tick	is the value that you will set.

6.2.5.4.4 rt_tick_increase()

```
void rt_tick_increase (
            void )
```

This function will notify kernel there is one tick passed. Normally, this function is invoked by clock ISR.

6.2.5.4.5 rt_tick_increase_tick()

```
void rt_tick_increase_tick (
            rt_tick_t tick)
```

This function will notify kernel there is n tick passed. Normally, this function is invoked by clock ISR.

6.2.5.4.6 rt_tick_from_millisecond()

```
rt_tick_t rt_tick_from_millisecond (
            rt_int32_t ms)
```

This function will calculate the tick from millisecond.

Parameters

ms	is the specified millisecond. • Negative Number wait forever • Zero not wait • Max 0x7fffffff

Returns

Return the calculated tick.

6.2.5.4.7 rt_tick_get_millisecond()

```
rt_tick_t rt_tick_get_millisecond (
            void )
```

This function will return the passed millisecond from boot.

Note

if the value of RT_TICK_PER_SECOND is lower than 1000 or is not an integral multiple of 1000, this function will not provide the correct 1ms-based tick.

Returns

Return passed millisecond from boot.

6.2.5.4.8 rt_timer_init()

```
void rt_timer_init (
            rt_timer_t timer,
            const char * name,
            void(* timeout )(void *parameter),
            void * parameter,
            rt_tick_t time,
            rt_uint8_t flag)
```

This function will initialize a timer normally this function is used to initialize a static timer object.

Parameters

timer	is the point of timer
name	is a pointer to the name of the timer
timeout	is the callback of timer
parameter	is the param of the callback
time	is timeout ticks of timer `NOTE: The max timeout tick should be no more than (RT_TICK_MAX/2 - 1).`
flag	is the flag of timer

6.2.5.4.9 rt_timer_detach()

```
rt_err_t rt_timer_detach (
            rt_timer_t timer)
```

This function will detach a timer from timer management.

Parameters

timer	is the timer to be detached

Returns

the status of detach

6.2.5.4.10 rt_timer_create()

```
rt_timer_t rt_timer_create (
            const char * name,
```

```
            void(* timeout )(void *parameter),
            void * parameter,
            rt_tick_t time,
            rt_uint8_t flag)
```

This function will create a timer.

Parameters

name	is the name of timer
timeout	is the timeout function
parameter	is the parameter of timeout function
time	is timeout ticks of the timer NOTE: The max timeout tick should be no more than (RT_TICK_MAX/2 - 1).
flag	is the flag of timer. Timer will invoke the timeout function according to the selected values of flag, if one or more of the following flags is set. RT_TIMER_FLAG_ONE_SHOT One shot timing RT_TIMER_FLAG_PERIODIC Periodic timing RT_TIMER_FLAG_HARD_TIMER Hardware timer RT_TIMER_FLAG_SOFT_TIMER Software timer RT_TIMER_FLAG_THREAD_TIMER Thread timer NOTE: You can use multiple values with "|" logical operator. By default, system will use the RT↩ _TIME_FLAG_HARD_TIMER.

Returns

the created timer object

6.2.5.4.11 rt_timer_delete()

```
rt_err_t rt_timer_delete (
            rt_timer_t timer)
```

This function will delete a timer and release timer memory.

Parameters

timer	the timer to be deleted

Returns

the operation status, RT_EOK on OK; -RT_ERROR on error

6.2.5.4.12 rt_timer_start()

```
rt_err_t rt_timer_start (
            rt_timer_t timer)
```

This function will start the timer.

Parameters

timer	the timer to be started

Returns

the operation status, RT_EOK on OK, -RT_ERROR on error

6.2.5.4.13 rt_timer_stop()

```
rt_err_t rt_timer_stop (
            rt_timer_t timer)
```

This function will stop the timer.

Parameters

timer	the timer to be stopped

Returns

the operation status, RT_EOK on OK, -RT_ERROR on error

6.2.5.4.14 rt_timer_control()

```
rt_err_t rt_timer_control (
            rt_timer_t timer,
            int cmd,
            void * arg)
```

This function will get or set some options of the timer.

Parameters

timer	the timer to be get or set
cmd	the control command
arg	the argument

Returns

the statu of control

6.2.5.4.15 rt_timer_check()

```
void rt_timer_check (
            void )
```

This function will check timer list, if a timeout event happens, the corresponding timeout function will be invoked.

Note

This function shall be invoked in operating system timer interrupt.

6.2.5.4.16 rt_timer_next_timeout_tick()

```
rt_tick_t rt_timer_next_timeout_tick (
            void )
```

This function will return the next timeout tick in the system.

Returns

the next timeout tick in the system

6.2.5.4.17 rt_system_timer_init()

```
void rt_system_timer_init (
            void )
```

This function will initialize system timer.

6.2.5.4.18 rt_system_timer_thread_init()

```
void rt_system_timer_thread_init (
            void )
```

This function will initialize system timer thread.

6.2.6 Inter-Thread Communication

Collaboration diagram for Inter-Thread Communication:

Topics

- Signal
- Semaphore
- Mutex
- Event
- MailBox
- Message Queue

Data Structures

- struct rt_ipc_object

Macros

- #define RT_IPC_FLAG_FIFO 0x00
- #define RT_IPC_FLAG_PRIO 0x01
- #define RT_IPC_CMD_UNKNOWN 0x00
- #define RT_IPC_CMD_RESET 0x01
- #define RT_IPC_CMD_GET_STATE 0x02
- #define RT_IPC_CMD_SET_VLIMIT 0x03
- #define RT_WAITING_FOREVER -1
- #define RT_WAITING_NO 0

Functions

- void rt_spin_lock_init (struct rt_spinlock *lock)
- void rt_spin_lock (struct rt_spinlock *lock)
- void rt_spin_unlock (struct rt_spinlock *lock)
- rt_base_t rt_spin_lock_irqsave (struct rt_spinlock *lock)
- void rt_spin_unlock_irqrestore (struct rt_spinlock *lock, rt_base_t level)
- struct rt_cpu * rt_cpu_self (void)
- struct rt_cpu * rt_cpu_index (int index)
- rt_base_t rt_cpus_lock (void)
- void rt_cpus_unlock (rt_base_t level)
- void rt_cpus_lock_status_restore (struct rt_thread *thread)
- rt_base_t rt_cpu_get_id (void)
- rt_inline rt_err_t _ipc_object_init (struct rt_ipc_object *ipc)

- struct rt_thread * rt_susp_list_dequeue (rt_list_t *susp_list, rt_err_t thread_error)
- rt_err_t rt_susp_list_resume_all (rt_list_t *susp_list, rt_err_t thread_error)
- rt_err_t rt_susp_list_resume_all_irq (rt_list_t *susp_list, rt_err_t thread_error, struct rt_spinlock *lock)
- rt_err_t rt_susp_list_enqueue (rt_list_t *susp_list, rt_thread_t thread, int ipc_flags)
- void rt_susp_list_print (rt_list_t *list)
- rt_err_t rt_thread_suspend_to_list (rt_thread_t thread, rt_list_t *susp_list, int ipc_flags, int suspend_flag)
- void rt_thread_defunct_enqueue (rt_thread_t thread)
- rt_thread_t rt_thread_defunct_dequeue (void)
- void rt_defunct_execute (void)

6.2.6.1 Detailed Description

See Inter-thread Communication.

6.2.6.2 Macro Definition Documentation

6.2.6.2.1 RT_IPC_FLAG_FIFO

```
#define RT_IPC_FLAG_FIFO 0x00
```

IPC flags and control command definitions FIFOed IPC. Inter-Thread Communication.

6.2.6.2.2 RT_IPC_FLAG_PRIO

```
#define RT_IPC_FLAG_PRIO 0x01
```

PRIOed IPC. Inter-Thread Communication.

6.2.6.2.3 RT_IPC_CMD_UNKNOWN

```
#define RT_IPC_CMD_UNKNOWN 0x00
```

unknown IPC command

6.2.6.2.4 RT_IPC_CMD_RESET

```
#define RT_IPC_CMD_RESET 0x01
```

reset IPC object

6.2.6.2.5 RT_IPC_CMD_GET_STATE

```
#define RT_IPC_CMD_GET_STATE 0x02
```

get the state of IPC object

6.2.6.2.6 RT_IPC_CMD_SET_VLIMIT

```
#define RT_IPC_CMD_SET_VLIMIT 0x03
```

set max limit value of IPC value

6.2.6.2.7 RT_WAITING_FOREVER

```
#define RT_WAITING_FOREVER -1
```

Block forever until get resource.

6.2.6.2.8 RT_WAITING_NO

```
#define RT_WAITING_NO 0
```

Non-block.

6.2.6.3 Function Documentation

6.2.6.3.1 rt_spin_lock_init()

```
void rt_spin_lock_init (
            struct rt_spinlock * lock)
```

Initialize a static spinlock object.

Parameters

lock	is a pointer to the spinlock to initialize.

Note

This function has UP version and MP version.

6.2.6.3.2 rt_spin_lock()

```
void rt_spin_lock (
            struct rt_spinlock * lock)
```

This function will lock the spinlock, will lock the thread scheduler.
If the spinlock is locked, the current CPU will keep polling the spinlock state until the spinlock is unlocked.

Parameters

lock	is a pointer to the spinlock.

Note

This function has UP version and MP version.

6.2.6.3.3 rt_spin_unlock()

```
void rt_spin_unlock (
            struct rt_spinlock * lock)
```

This function will unlock the spinlock, will unlock the thread scheduler.
If the scheduling function is called before unlocking, it will be scheduled in this function.

Parameters

lock	is a pointer to the spinlock.

Note

This function has UP version and MP version.

6.2.6.3.4 rt_spin_lock_irqsave()

```
rt_base_t rt_spin_lock_irqsave (
            struct rt_spinlock * lock)
```

This function will disable the local interrupt and then lock the spinlock, will lock the thread scheduler.
If the spinlock is locked, the current CPU will keep polling the spinlock state until the spinlock is unlocked.

Parameters

lock	is a pointer to the spinlock.

Returns

Return current cpu interrupt status.

Note

This function has UP version and MP version.

6.2.6.3.5 rt_spin_unlock_irqrestore()

```
void rt_spin_unlock_irqrestore (
            struct rt_spinlock * lock,
            rt_base_t level)
```

This function will unlock the spinlock and then restore current cpu interrupt status, will unlock the thread scheduler. If the scheduling function is called before unlocking, it will be scheduled in this function.

Parameters

lock	is a pointer to the spinlock.
level	is interrupt status returned by rt_spin_lock_irqsave().

Note

This function has UP version and MP version.

6.2.6.3.6 rt_cpu_self()

```
struct rt_cpu * rt_cpu_self (
            void )
```

This fucntion will return current cpu object.

Returns

Return a pointer to the current cpu object.

Note

This function has UP version and MP version.

CPU object

6.2.6.3.7 rt_cpu_index()

```
struct rt_cpu * rt_cpu_index (
            int index)
```

This fucntion will return the cpu object corresponding to index.

Parameters

index	is the index of target cpu object.

Returns

Return a pointer to the cpu object corresponding to index.

Note

This function has UP version and MP version.

6.2.6.3.8 rt_cpus_lock()

```
rt_base_t rt_cpus_lock (
              void )
```

This function will lock all cpus's scheduler and disable local irq.

Returns

Return current cpu interrupt status.

Note

This function only has MP version.

6.2.6.3.9 rt_cpus_unlock()

```
void rt_cpus_unlock (
              rt_base_t level)
```

This function will restore all cpus's scheduler and restore local irq.

Parameters

level	is interrupt status returned by rt_cpus_lock().

Note

This function only has MP version.

6.2.6.3.10 rt_cpus_lock_status_restore()

```
void rt_cpus_lock_status_restore (
              struct rt_thread * thread)
```

This function is invoked by scheduler. It will restore the lock state to whatever the thread's counter expects. If target thread not locked the cpus then unlock the cpus lock.

Parameters

thread	is a pointer to the target thread.

Note

This function only has MP version.

6.2.6.3.11 rt_cpu_get_id()

```
rt_base_t rt_cpu_get_id (
              void )
```

Get logical CPU ID.

Returns

logical CPU ID

Note

This function only has MP version.

6.2.6.3.12 _ipc_object_init()

```
rt_inline rt_err_t _ipc_object_init (
              struct rt_ipc_object * ipc)
```

This function will initialize an IPC object, such as semaphore, mutex, messagequeue and mailbox.

Note

Executing this function will complete an initialization of the suspend thread list of the ipc object.

Parameters

ipc	is a pointer to the IPC object.

Returns

Return the operation status. When the return value is RT_EOK, the initialization is successful. When the return value is any other values, it means the initialization failed.

Warning

This function can be called from all IPC initialization and creation.

6.2.6.3.13 rt_susp_list_dequeue()

```
struct rt_thread * rt_susp_list_dequeue (
              rt_list_t * susp_list,
              rt_err_t thread_error)
```

Dequeue a thread from suspended list and set it to ready. The 2 are taken as an atomic operation, so if a thread is returned, it's resumed by us, not any other threads or async events. This is useful if a consumer may be resumed by timeout, signals... besides its producer.

Parameters

susp_list	the list thread dequeued from. RT_NULL if no list.
thread_error	thread error number of the resuming thread. A negative value in this set will be discarded, and thread error will not be changed.

Returns

struct rt_thread * RT_NULL if failed, otherwise the thread resumed

6.2.6.3.14 rt_susp_list_resume_all()

```
rt_err_t rt_susp_list_resume_all (
              rt_list_t * susp_list,
              rt_err_t thread_error)
```

This function will resume all suspended threads in the IPC object list, including the suspended list of IPC object, and private list of mailbox etc.

Note

This function will resume all threads in the IPC object list. By contrast, the rt_ipc_list_resume() function will resume a suspended thread in the list of a IPC object.

Parameters

susp_list	is a pointer to a suspended thread list of the IPC object.
thread_error	thread error number of the resuming thread. A negative value in this set will be discarded, and thread error will not be changed.

Returns

Return the operation status. When the return value is RT_EOK, the function is successfully executed. When the return value is any other values, it means this operation failed.

6.2.6.3.15 rt_susp_list_resume_all_irq()

```
rt_err_t rt_susp_list_resume_all_irq (
             rt_list_t * susp_list,
             rt_err_t thread_error,
             struct rt_spinlock * lock)
```

This function will resume all suspended threads in the IPC object list, including the suspended list of IPC object, and private list of mailbox etc. A lock is passing and hold while operating.

Note

This function will resume all threads in the IPC object list. By contrast, the rt_ipc_list_resume() function will resume a suspended thread in the list of a IPC object.

Parameters

susp_list	is a pointer to a suspended thread list of the IPC object.
thread_error	thread error number of the resuming thread. A negative value in this set will be discarded, and thread error will not be changed.
lock	the lock to be held while operating susp_list

Returns

Return the operation status. When the return value is RT_EOK, the function is successfully executed. When the return value is any other values, it means this operation failed.

6.2.6.3.16 rt_susp_list_enqueue()

```
rt_err_t rt_susp_list_enqueue (
             rt_list_t * susp_list,
             rt_thread_t thread,
             int ipc_flags)
```

Add a thread to the suspend list.

Note

Caller must hold the scheduler lock

Parameters

susp_list	the list thread enqueued to
thread	the suspended thread
ipc_flags	the pattern of suspend list

Returns

RT_EOK on succeed, otherwise a failure

6.2.6.3.17 rt_susp_list_print()

```
void rt_susp_list_print (
            rt_list_t * list)
```

Print thread on suspend list to system console.
Suspend list - A basic building block for IPC primitives which interacts with scheduler directly. Its API is similar to a FIFO list.
Note: don't use in application codes directly

6.2.6.3.18 rt_thread_suspend_to_list()

```
rt_err_t rt_thread_suspend_to_list (
            rt_thread_t thread,
            rt_list_t * susp_list,
            int ipc_flags,
            int suspend_flag)
```

This function will suspend the specified thread and change it to suspend state.

Note

This function ONLY can suspend current thread itself. rt_thread_suspend(rt_thread_self());

Do not use the rt_thread_suspend to suspend other threads. You have no way of knowing what code a thread is executing when you suspend it. If you suspend a thread while sharing a resouce with other threads and occupying this resouce, starvation can occur very easily.

Parameters

thread	the thread to be suspended.
susp_list	the list thread enqueued to. RT_NULL if no list.
ipc_flags	is a flag for the thread object to be suspended. It determines how the thread is suspended. The flag can be ONE of the following values: RT_IPC_FLAG_PRIO The pending threads will queue in order of priority. RT_IPC_FLAG_FIFO The pending threads will queue in the first-in-first-out method (also known as first-come-first-served (FCFS) scheduling strategy). NOTE: RT_IPC_↩ FLAG_FIFO is a non-real-time scheduling mode. It is strongly recommended to use RT_IPC↩ _FLAG_PRIO to ensure the thread is real-time UNLESS your applications concern about the first-in-first-out principle, and you clearly understand that all threads involved in this semaphore will become non-real-time threads.
suspend_flag	status flag of the thread to be suspended.

Returns

Return the operation status. If the return value is RT_EOK, the function is successfully executed. If the return value is any other values, it means this operation failed.

enqueue thread on the push list before leaving critical region of scheduler, so we won't miss notification of async events.

6.2.6.3.19 rt_thread_defunct_enqueue()

```
void rt_thread_defunct_enqueue (
            rt_thread_t thread)
```

Enqueue a thread to defunct queue.

Parameters

thread	the thread to be enqueued.

Note

It must be called between rt_hw_interrupt_disable and rt_hw_interrupt_enable

6.2.6.3.20 rt_thread_defunct_dequeue()

```
rt_thread_t rt_thread_defunct_dequeue (
            void )
```

Dequeue a thread from defunct queue.

6.2.6.3.21 rt_defunct_execute()

```
void rt_defunct_execute (
            void )
```

This function will perform system background job when system idle.

6.2.6.4 Signal

Collaboration diagram for Signal:

Functions

- void rt_signal_mask (int signo)
- void rt_signal_unmask (int signo)
- rt_sighandler_t rt_signal_install (int signo, rt_sighandler_t handler)
- int rt_signal_wait (const rt_sigset_t ∗set, rt_siginfo_t ∗si, rt_int32_t timeout)

6.2.6.4.1 Detailed Description

signal is used for thread kill etc.
A signal (also known as a soft interrupt signal), from a software perspective, is a simulation of interrupt mechanism. When it comes to its principle, thread receiving a signal is similar to processor receiving an interrupt request.

6.2.6.4.2 Function Documentation

6.2.6.4.2.1 rt_signal_mask()

```
void rt_signal_mask (
            int signo)
```

This function will block the specified signal.

Note

This function will block the specified signal, even if the rt_thread_kill() function is called to send this signal to the current thread, it will no longer take effect.

See also

rt_thread_kill()

Parameters

signo	is a specific signal value (range: 0 ~ RT_SIG_MAX).

6.2.6.4.2.2 rt_signal_unmask()

```
void rt_signal_unmask (
            int signo)
```

This function will unblock the specified signal.

Note

This function will unblock the specified signal. After calling the rt_thread_kill() function to send this signal to the current thread, it will take effect.

See also

rt_thread_kill()

Parameters

signo	is a specific signal value (range: 0 ~ RT_SIG_MAX).

6.2.6.4.2.3 rt_signal_install()

```
rt_sighandler_t rt_signal_install (
            int signo,
            rt_sighandler_t handler)
```

This function will install a processing function to a specific signal and return the old processing function of this signal.

Note

This function needs to be used in conjunction with the rt_signal_unmask() function to make the signal effective.

See also

rt_signal_unmask()

Parameters

signo	is a specific signal value (range: 0 ~ RT_SIG_MAX).
handler	is sets the processing of signal value.

Returns

Return the old processing function of this signal. ONLY When the return value is SIG_ERR, the operation is failed.

6.2.6.4.2.4 rt_signal_wait()

```
int rt_signal_wait (
            const rt_sigset_t * set,
            rt_siginfo_t * si,
            rt_int32_t timeout)
```

This function will wait for the arrival of the set signal. If it does not wait for this signal, the thread will be suspended until it waits for this signal or the waiting time exceeds the specified timeout: timeout.

Parameters

set	is the set of signal values to be waited for. Use the function sigaddset() to add the signal.
si	is a pointer to the received signal info. If you don't care about this value, you can use RT_NULL to set.
timeout	is a timeout period (unit: an OS tick).

Returns

Return the operation status. When the return value is RT_EOK, the operation is successful. If the return value is any other values, it means that the signal wait failed.

6.2.6.5 Semaphore

Collaboration diagram for Semaphore:

Data Structures

- struct rt_semaphore

Functions

- rt_err_t rt_sem_init (rt_sem_t sem, const char ∗name, rt_uint32_t value, rt_uint8_t flag)
- rt_err_t rt_sem_detach (rt_sem_t sem)
- rt_sem_t rt_sem_create (const char ∗name, rt_uint32_t value, rt_uint8_t flag)
- rt_err_t rt_sem_delete (rt_sem_t sem)
- rt_err_t rt_sem_trytake (rt_sem_t sem)
- rt_err_t rt_sem_release (rt_sem_t sem)
- rt_err_t rt_sem_control (rt_sem_t sem, int cmd, void ∗arg)

6.2.6.5.1 Detailed Description

6.2.6.5.2 Function Documentation

6.2.6.5.2.1 rt_sem_init()

```
rt_err_t rt_sem_init (
            rt_sem_t sem,
            const char * name,
            rt_uint32_t value,
            rt_uint8_t flag)
```

This function will initialize a static semaphore object.

Note

For the static semaphore object, its memory space is allocated by the compiler during compiling, and shall placed on the read-write data segment or on the uninitialized data segment. By contrast, the rt_sem_create() function will allocate memory space automatically and initialize the semaphore.

See also

rt_sem_create()

Parameters

sem	is a pointer to the semaphore to initialize. It is assumed that storage for the semaphore will be allocated in your application.
name	is a pointer to the name you would like to give the semaphore.
value	is the initial value for the semaphore. If used to share resources, you should initialize the value as the number of available resources. If used to signal the occurrence of an event, you should initialize the value as 0.
flag	is the semaphore flag, which determines the queuing way of how multiple threads wait when the semaphore is not available. The semaphore flag can be ONE of the following values: RT_IPC_FLAG_PRIO The pending threads will queue in order of priority. RT_IPC_FLAG_FIFO The pending threads will queue in the first-in-first-out method (also known as first-come-first-served (FCFS) scheduling strategy). NOTE: RT_IPC_FLAG_FIFO is a non-real-time scheduling mode. It is strongly recommended to use RT_IPC_FLAG_PRIO to ensure the thread is real-time UNLESS your applications concern about the first-in-first-out principle, and you clearly understand that all threads involved in this semaphore will become non-real-time threads.

Returns

Return the operation status. When the return value is RT_EOK, the initialization is successful. If the return value is any other values, it represents the initialization failed.

Warning

This function can ONLY be called from threads.

6.2.6.5.2.2 rt_sem_detach()

```
rt_err_t rt_sem_detach (
             rt_sem_t sem)
```

This function will detach a static semaphore object.

Note

This function is used to detach a static semaphore object which is initialized by rt_sem_init() function. By contrast, the rt_sem_delete() function will delete a semaphore object. When the semaphore is successfully detached, it will resume all suspended threads in the semaphore list.

See also

rt_sem_delete()

Parameters

sem	is a pointer to a semaphore object to be detached.

Returns

Return the operation status. When the return value is RT_EOK, the initialization is successful. If the return value is any other values, it means that the semaphore detach failed.

Warning

This function can ONLY detach a static semaphore initialized by the rt_sem_init() function. If the semaphore is created by the rt_sem_create() function, you MUST NOT USE this function to detach it, ONLY USE the rt_sem_delete() function to complete the deletion.

6.2.6.5.2.3 rt_sem_create()

```
rt_sem_t rt_sem_create (
            const char * name,
            rt_uint32_t value,
            rt_uint8_t flag)
```

Creating a semaphore object.

Note

For the semaphore object, its memory space is allocated automatically. By contrast, the rt_sem_init() function will initialize a static semaphore object.

See also

rt_sem_init()

Parameters

name	is a pointer to the name you would like to give the semaphore.
value	is the initial value for the semaphore. If used to share resources, you should initialize the value as the number of available resources. If used to signal the occurrence of an event, you should initialize the value as 0.
flag	is the semaphore flag, which determines the queuing way of how multiple threads wait when the semaphore is not available. The semaphore flag can be ONE of the following values: RT_IPC_FLAG_PRIO The pending threads will queue in order of priority. RT_IPC_FLAG_FIFO The pending threads will queue in the first-in-first-out method (also known as first-come-first-served (FCFS) scheduling strategy). NOTE: RT_IPC_FLAG_FIFO is a non-real-time scheduling mode. It is strongly recommended to use RT_IPC_FLAG_PRIO to ensure the thread is real-time UNLESS your applications concern about the first-in-first-out principle, and you clearly understand that all threads involved in this semaphore will become non-real-time threads.

Returns

Return a pointer to the semaphore object. When the return value is RT_NULL, it means the creation failed.

Warning

This function can NOT be called in interrupt context. You can use macor RT_DEBUG_NOT_IN_INTERRUPT to check it.

6.2.6.5.2.4 rt_sem_delete()

```
rt_err_t rt_sem_delete (
            rt_sem_t sem)
```

This function will delete a semaphore object and release the memory space.

Note

This function is used to delete a semaphore object which is created by the rt_sem_create() function. By contrast, the rt_sem_detach() function will detach a static semaphore object. When the semaphore is successfully deleted, it will resume all suspended threads in the semaphore list.

See also

rt_sem_detach()

Parameters

sem	is a pointer to a semaphore object to be deleted.

Returns

Return the operation status. When the return value is RT_EOK, the operation is successful. If the return value is any other values, it means that the semaphore detach failed.

Warning

This function can ONLY delete a semaphore initialized by the rt_sem_create() function. If the semaphore is initialized by the rt_sem_init() function, you MUST NOT USE this function to delete it, ONLY USE the rt_sem_detach() function to complete the detachment.

6.2.6.5.2.5 rt_sem_trytake()

```
rt_err_t rt_sem_trytake (
            rt_sem_t sem)
```

This function will try to take a semaphore, if the semaphore is unavailable, the thread returns immediately.

Note

This function is very similar to the rt_sem_take() function, when the semaphore is not available, the rt_sem_trytake() function will return immediately without waiting for a timeout. In other words, rt_sem_↩ trytake(sem) has the same effect as rt_sem_take(sem, 0).

See also

rt_sem_take()

Parameters

sem	is a pointer to a semaphore object.

Returns

Return the operation status. ONLY When the return value is RT_EOK, the operation is successful. If the return value is any other values, it means that the semaphore take failed.

6.2.6.5.2.6 rt_sem_release()

```
rt_err_t rt_sem_release (
            rt_sem_t sem)
```

This function will release a semaphore. If there is thread suspended on the semaphore, it will get resumed.

Note

If there are threads suspended on this semaphore, the first thread in the list of this semaphore object will be resumed, and a thread scheduling (rt_schedule) will be executed. If no threads are suspended on this semaphore, the count value sem->value of this semaphore will increase by 1.

Parameters

sem	is a pointer to a semaphore object.

Returns

Return the operation status. When the return value is RT_EOK, the operation is successful. If the return value is any other values, it means that the semaphore release failed.

6.2.6.5.2.7 rt_sem_control()

```
rt_err_t rt_sem_control (
            rt_sem_t sem,
            int cmd,
            void * arg)
```

This function will set some extra attributions of a semaphore object.

Note

Currently this function only supports the RT_IPC_CMD_RESET command to reset the semaphore.

Parameters

sem	is a pointer to a semaphore object.
cmd	is a command word used to configure some attributions of the semaphore.
arg	is the argument of the function to execute the command.

Returns

Return the operation status. When the return value is RT_EOK, the operation is successful. If the return value is any other values, it means that this function failed to execute.

6.2.6.6 Mutex

Collaboration diagram for Mutex:

Data Structures

- struct rt_mutex

Functions

- rt_err_t rt_mutex_init (rt_mutex_t mutex, const char *name, rt_uint8_t flag)
- rt_err_t rt_mutex_detach (rt_mutex_t mutex)
- void rt_mutex_drop_thread (rt_mutex_t mutex, rt_thread_t thread)

- rt_uint8_t rt_mutex_setprioceiling (rt_mutex_t mutex, rt_uint8_t priority)
- rt_uint8_t rt_mutex_getprioceiling (rt_mutex_t mutex)
- rt_mutex_t rt_mutex_create (const char *name, rt_uint8_t flag)
- rt_err_t rt_mutex_delete (rt_mutex_t mutex)
- rt_err_t rt_mutex_trytake (rt_mutex_t mutex)
- rt_err_t rt_mutex_release (rt_mutex_t mutex)
- rt_err_t rt_mutex_control (rt_mutex_t mutex, int cmd, void *arg)

6.2.6.6.1 Detailed Description

6.2.6.6.2 Function Documentation

6.2.6.6.2.1 rt_mutex_init()

```
rt_err_t rt_mutex_init (
            rt_mutex_t mutex,
            const char * name,
            rt_uint8_t flag)
```

Initialize a static mutex object.

Note

For the static mutex object, its memory space is allocated by the compiler during compiling, and shall placed on the read-write data segment or on the uninitialized data segment. By contrast, the rt_mutex_create() function will automatically allocate memory space and initialize the mutex.

See also

rt_mutex_create()

Parameters

mutex	is a pointer to the mutex to initialize. It is assumed that storage for the mutex will be allocated in your application.
name	is a pointer to the name that given to the mutex.
flag	is the mutex flag, which determines the queuing way of how multiple threads wait when the mutex is not available. NOTE: This parameter has been obsoleted. It can be RT_IPC_FLAG_PRIO, RT_IPC_↩ FLAG_FIFO or RT_NULL.

Returns

Return the operation status. When the return value is RT_EOK, the initialization is successful. If the return value is any other values, it represents the initialization failed.

Warning

This function can ONLY be called from threads.

6.2.6.6.2.2 rt_mutex_detach()

```
rt_err_t rt_mutex_detach (
            rt_mutex_t mutex)
```

This function will detach a static mutex object.

Note

This function is used to detach a static mutex object which is initialized by rt_mutex_init() function. By contrast, the rt_mutex_delete() function will delete a mutex object. When the mutex is successfully detached, it will resume all suspended threads in the mutex list.

See also

rt_mutex_delete()

Parameters

mutex	is a pointer to a mutex object to be detached.

Returns

Return the operation status. When the return value is RT_EOK, the initialization is successful. If the return value is any other values, it means that the mutex detach failed.

Warning

This function can ONLY detach a static mutex initialized by the rt_mutex_init() function. If the mutex is created by the rt_mutex_create() function, you MUST NOT USE this function to detach it, ONLY USE the rt_mutex_delete() function to complete the deletion.

6.2.6.6.2.3 rt_mutex_drop_thread()

```
void rt_mutex_drop_thread (
             rt_mutex_t mutex,
             rt_thread_t thread)
```

drop a thread from the suspend list of mutex

Parameters

mutex	is a pointer to a mutex object.
thread	is the thread should be dropped from mutex.

Should change the priority of mutex owner thread Note: After current thread is detached from mutex pending list, there is a chance that the mutex owner has been released the mutex. Which means mutex->owner can be NULL at this point. If that happened, it had already reset its priority. So it's okay to skip

6.2.6.6.2.4 rt_mutex_setprioceiling()

```
rt_uint8_t rt_mutex_setprioceiling (
             rt_mutex_t mutex,
             rt_uint8_t priority)
```

set the prioceiling attribute of the mutex.

Parameters

mutex	is a pointer to a mutex object.
priority	is the priority should be set to mutex.

Returns

return the old priority ceiling

6.2.6.6.2.5 rt_mutex_getprioceiling()

```
rt_uint8_t rt_mutex_getprioceiling (
             rt_mutex_t mutex)
```

set the prioceiling attribute of the mutex.

Parameters

mutex	is a pointer to a mutex object.

Returns

return the current priority ceiling of the mutex.

6.2.6.6.2.6 rt_mutex_create()

```
rt_mutex_t rt_mutex_create (
            const char * name,
            rt_uint8_t flag)
```

This function will create a mutex object.

Note

For the mutex object, its memory space is automatically allocated. By contrast, the rt_mutex_init() function will initialize a static mutex object.

See also

rt_mutex_init()

Parameters

name	is a pointer to the name that given to the mutex.
flag	is the mutex flag, which determines the queuing way of how multiple threads wait when the mutex is not available. NOTE: This parameter has been obsoleted. It can be RT_IPC_FLAG_PRIO, RT_IPC_↩ FLAG_FIFO or RT_NULL.

Returns

Return a pointer to the mutex object. When the return value is RT_NULL, it means the creation failed.

Warning

This function can ONLY be called from threads.

6.2.6.6.2.7 rt_mutex_delete()

```
rt_err_t rt_mutex_delete (
            rt_mutex_t mutex)
```

This function will delete a mutex object and release this memory space.

Note

This function is used to delete a mutex object which is created by the rt_mutex_create() function. By contrast, the rt_mutex_detach() function will detach a static mutex object. When the mutex is successfully deleted, it will resume all suspended threads in the mutex list.

See also

rt_mutex_detach()

Parameters

mutex	is a pointer to a mutex object to be deleted.

Returns

Return the operation status. When the return value is RT_EOK, the operation is successful. If the return value is any other values, it means that the mutex detach failed.

Warning

This function can ONLY delete a mutex initialized by the rt_mutex_create() function. If the mutex is initialized by the rt_mutex_init() function, you MUST NOT USE this function to delete it, ONLY USE the rt_mutex_detach() function to complete the detachment.

6.2.6.6.2.8 rt_mutex_trytake()

```
rt_err_t rt_mutex_trytake (
            rt_mutex_t mutex)
```

This function will try to take a mutex, if the mutex is unavailable, the thread returns immediately.

Note

This function is very similar to the rt_mutex_take() function, when the mutex is not available, except that rt_mutex_trytake() will return immediately without waiting for a timeout when the mutex is not available. In other words, rt_mutex_trytake(mutex) has the same effect as rt_mutex_take(mutex, 0).

See also

rt_mutex_take()

Parameters

mutex	is a pointer to a mutex object.

Returns

Return the operation status. ONLY When the return value is RT_EOK, the operation is successful. If the return value is any other values, it means that the mutex take failed.

6.2.6.6.2.9 rt_mutex_release()

```
rt_err_t rt_mutex_release (
            rt_mutex_t mutex)
```

This function will release a mutex. If there is thread suspended on the mutex, the thread will be resumed.

Note

If there are threads suspended on this mutex, the first thread in the list of this mutex object will be resumed, and a thread scheduling (rt_schedule) will be executed. If no threads are suspended on this mutex, the count value mutex->value of this mutex will increase by 1.

Parameters

mutex	is a pointer to a mutex object.

Returns

Return the operation status. When the return value is RT_EOK, the operation is successful. If the return value is any other values, it means that the mutex release failed.

a timeout timer had triggered while we try. So we skip this thread and try again.

6.2.6.6.2.10 rt_mutex_control()

```
rt_err_t rt_mutex_control (
             rt_mutex_t mutex,
             int cmd,
             void * arg)
```

This function will set some extra attributions of a mutex object.

Note

Currently this function does not implement the control function.

Parameters

mutex	is a pointer to a mutex object.
cmd	is a command word used to configure some attributions of the mutex.
arg	is the argument of the function to execute the command.

Returns

Return the operation status. When the return value is RT_EOK, the operation is successful. If the return value is any other values, it means that this function failed to execute.

6.2.6.7 Event

Collaboration diagram for Event:

Macros

- #define RT_EVENT_FLAG_AND 0x01
- #define RT_EVENT_FLAG_OR 0x02
- #define RT_EVENT_FLAG_CLEAR 0x04

Functions

- rt_err_t rt_event_init (rt_event_t event, const char ∗name, rt_uint8_t flag)
- rt_err_t rt_event_detach (rt_event_t event)
- rt_event_t rt_event_create (const char ∗name, rt_uint8_t flag)
- rt_err_t rt_event_delete (rt_event_t event)
- rt_err_t rt_event_send (rt_event_t event, rt_uint32_t set)
- rt_err_t rt_event_control (rt_event_t event, int cmd, void ∗arg)

6.2.6.7.1 Detailed Description

6.2.6.7.2 Macro Definition Documentation

6.2.6.7.2.1 RT_EVENT_FLAG_AND

```
#define RT_EVENT_FLAG_AND 0x01
```

flag definitions in event logic and

6.2.6.7.2.2 RT_EVENT_FLAG_OR

```
#define RT_EVENT_FLAG_OR 0x02
```
logic or

6.2.6.7.2.3 RT_EVENT_FLAG_CLEAR

```
#define RT_EVENT_FLAG_CLEAR 0x04
```
clear flag

6.2.6.7.3 Function Documentation

6.2.6.7.3.1 rt_event_init()

```
rt_err_t rt_event_init (
            rt_event_t event,
            const char * name,
            rt_uint8_t flag)
```
The function will initialize a static event object.

Note

For the static event object, its memory space is allocated by the compiler during compiling, and shall placed on the read-write data segment or on the uninitialized data segment. By contrast, the rt_event_create() function will allocate memory space automatically and initialize the event.

See also

rt_event_create()

Parameters

event	is a pointer to the event to initialize. It is assumed that storage for the event will be allocated in your application.
name	is a pointer to the name that given to the event.
flag	is the event flag, which determines the queuing way of how multiple threads wait when the event is not available. The event flag can be ONE of the following values: RT_IPC_FLAG_PRIO The pending threads will queue in order of priority. RT_IPC_FLAG_FIFO The pending threads will queue in the first-in-first-out method (also known as first-come-first-served (FCFS) scheduling strategy). NOTE: RT_IPC_FLAG_FIFO is a non-real-time scheduling mode. It is strongly recommended to use RT_IPC_FLAG_PRIO to ensure the thread is real-time UNLESS your applications concern about the first-in-first-out principle, and you clearly understand that all threads involved in this event will become non-real-time threads.

Returns

Return the operation status. When the return value is RT_EOK, the initialization is successful. If the return value is any other values, it represents the initialization failed.

Warning

This function can ONLY be called from threads.

6.2.6.7.3.2 rt_event_detach()

```
rt_err_t rt_event_detach (
            rt_event_t event)
```

This function will detach a static event object.

Note

This function is used to detach a static event object which is initialized by rt_event_init() function. By contrast, the rt_event_delete() function will delete an event object. When the event is successfully detached, it will resume all suspended threads in the event list.

See also

rt_event_delete()

Parameters

event	is a pointer to an event object to be detached.

Returns

Return the operation status. When the return value is RT_EOK, the initialization is successful. If the return value is any other values, it means that the event detach failed.

Warning

This function can ONLY detach a static event initialized by the rt_event_init() function. If the event is created by the rt_event_create() function, you MUST NOT USE this function to detach it, ONLY USE the rt_event_delete() function to complete the deletion.

6.2.6.7.3.3 rt_event_create()

```
rt_event_t rt_event_create (
            const char * name,
            rt_uint8_t flag)
```

Creating an event object.

Note

For the event object, its memory space is allocated automatically. By contrast, the rt_event_init() function will initialize a static event object.

See also

rt_event_init()

Parameters

name	is a pointer to the name that given to the event.

flag	is the event flag, which determines the queuing way of how multiple threads wait when the event is not available. The event flag can be ONE of the following values:
	`RT_IPC_FLAG_PRIO The pending threads will queue in order of priority.`
	
	`RT_IPC_FLAG_FIFO The pending threads will queue in the first-in-first-out method (also known as first-come-first-served (FCFS) scheduling strategy).`
	
	`NOTE: RT_IPC_FLAG_FIFO is a non-real-time scheduling mode. It is strongly recommended to use RT_IPC_FLAG_PRIO to ensure the thread is real-time UNLESS your applications concern about the first-in-first-out principle, and you clearly understand that all threads involved in this event will become non-real-time threads.`

Returns

Return a pointer to the event object. When the return value is RT_NULL, it means the creation failed.

Warning

This function can ONLY be called from threads.

6.2.6.7.3.4 rt_event_delete()

```
rt_err_t rt_event_delete (
             rt_event_t event)
```

This function will delete an event object and release the memory space.

Note

This function is used to delete an event object which is created by the rt_event_create() function. By contrast, the rt_event_detach() function will detach a static event object. When the event is successfully deleted, it will resume all suspended threads in the event list.

See also

rt_event_detach()

Parameters

event	is a pointer to an event object to be deleted.

Returns

Return the operation status. When the return value is RT_EOK, the operation is successful. If the return value is any other values, it means that the event detach failed.

Warning

This function can ONLY delete an event initialized by the rt_event_create() function. If the event is initialized by the rt_event_init() function, you MUST NOT USE this function to delete it, ONLY USE the rt_event_detach() function to complete the detachment.

6.2.6.7.3.5 rt_event_send()

```
rt_err_t rt_event_send (
            rt_event_t event,
            rt_uint32_t set)
```

This function will send an event to the event object. If there is a thread suspended on the event, the thread will be resumed.

Note

When using this function, you need to use the parameter (set) to specify the event flag of the event object, then the function will traverse the list of suspended threads waiting on the event object. If there is a thread suspended on the event, and the thread's event_info and the event flag of the current event object matches, the thread will be resumed.

Parameters

event	is a pointer to the event object to be sent.
set	is a flag that you will set for this event's flag. You can set an event flag, or you can set multiple flags through OR logic operation.

Returns

Return the operation status. When the return value is RT_EOK, the operation is successful. If the return value is any other values, it means that the event detach failed.

6.2.6.7.3.6 rt_event_control()

```
rt_err_t rt_event_control (
            rt_event_t event,
            int cmd,
            void * arg)
```

This function will set some extra attributions of an event object.

Note

Currently this function only supports the RT_IPC_CMD_RESET command to reset the event.

Parameters

event	is a pointer to an event object.
cmd	is a command word used to configure some attributions of the event.
arg	is the argument of the function to execute the command.

Returns

Return the operation status. When the return value is RT_EOK, the operation is successful. If the return value is any other values, it means that this function failed to execute.

6.2.6.8 MailBox

Collaboration diagram for MailBox:

Data Structures

- struct rt_mailbox

Functions

- rt_err_t rt_mb_init (rt_mailbox_t mb, const char *name, void *msgpool, rt_size_t size, rt_uint8_t flag)
- rt_err_t rt_mb_detach (rt_mailbox_t mb)
- rt_mailbox_t rt_mb_create (const char *name, rt_size_t size, rt_uint8_t flag)
- rt_err_t rt_mb_delete (rt_mailbox_t mb)
- rt_err_t rt_mb_send (rt_mailbox_t mb, rt_ubase_t value)
- rt_err_t rt_mb_urgent (rt_mailbox_t mb, rt_ubase_t value)
- rt_err_t rt_mb_control (rt_mailbox_t mb, int cmd, void *arg)

6.2.6.8.1 Detailed Description

6.2.6.8.2 Function Documentation

6.2.6.8.2.1 rt_mb_init()

```
rt_err_t rt_mb_init (
            rt_mailbox_t mb,
            const char * name,
            void * msgpool,
            rt_size_t size,
            rt_uint8_t flag)
```

Initialize a static mailbox object.

Note

For the static mailbox object, its memory space is allocated by the compiler during compiling, and shall placed on the read-write data segment or on the uninitialized data segment. By contrast, the rt_mb_create() function will allocate memory space automatically and initialize the mailbox.

See also

rt_mb_create()

Parameters

mb	is a pointer to the mailbox to initialize. It is assumed that storage for the mailbox will be allocated in your application.
name	is a pointer to the name that given to the mailbox.
msgpool	the begin address of buffer to save received mail.
size	is the maximum number of mails in the mailbox. For example, when the mailbox buffer capacity is N, size is N/4.
flag	is the mailbox flag, which determines the queuing way of how multiple threads wait when the mailbox is not available. The mailbox flag can be ONE of the following values: RT_IPC_FLAG_PRIO The pending threads will queue in order of priority. RT_IPC_FLAG_FIFO The pending threads will queue in the first-in-first-out method (also known as first-come-first-served (FCFS) scheduling strategy). NOTE: RT_IPC_FLAG_FIFO is a non-real-time scheduling mode. It is strongly recommended to use RT_IPC_FLAG_PRIO to ensure the thread is real-time UNLESS your applications concern about the first-in-first-out principle, and you clearly understand that all threads involved in this mailbox will become non-real-time threads.

Returns

Return the operation status. When the return value is RT_EOK, the initialization is successful. If the return value is any other values, it represents the initialization failed.

Warning

This function can ONLY be called from threads.

6.2.6.8.2.2 rt_mb_detach()

```
rt_err_t rt_mb_detach (
             rt_mailbox_t mb)
```

This function will detach a static mailbox object.

Note

This function is used to detach a static mailbox object which is initialized by rt_mb_init() function. By contrast, the rt_mb_delete() function will delete a mailbox object. When the mailbox is successfully detached, it will resume all suspended threads in the mailbox list.

See also

rt_mb_delete()

Parameters

mb	is a pointer to a mailbox object to be detached.

Returns

Return the operation status. When the return value is RT_EOK, the initialization is successful. If the return value is any other values, it means that the mailbox detach failed.

Warning

This function can ONLY detach a static mailbox initialized by the rt_mb_init() function. If the mailbox is created by the rt_mb_create() function, you MUST NOT USE this function to detach it, ONLY USE the rt_mb_delete() function to complete the deletion.

6.2.6.8.2.3 rt_mb_create()

```
rt_mailbox_t rt_mb_create (
             const char * name,
             rt_size_t size,
             rt_uint8_t flag)
```

Creating a mailbox object.

Note

For the mailbox object, its memory space is allocated automatically. By contrast, the rt_mb_init() function will initialize a static mailbox object.

See also

rt_mb_init()

Parameters

name	is a pointer that given to the mailbox.
size	is the maximum number of mails in the mailbox. For example, when mailbox buffer capacity is N, size is N/4.

flag	is the mailbox flag, which determines the queuing way of how multiple threads wait when the mailbox is not available. The mailbox flag can be ONE of the following values:
	`RT_IPC_FLAG_PRIO The pending threads will queue in order of priority.`
	
	`RT_IPC_FLAG_FIFO The pending threads will queue in the first-in-first-out method (also known as first-come-first-served (FCFS) scheduling strategy).`
	
	`NOTE: RT_IPC_FLAG_FIFO is a non-real-time scheduling mode. It is strongly recommended to use RT_IPC_FLAG_PRIO to ensure the thread is real-time UNLESS your applications concern about the first-in-first-out principle, and you clearly understand that all threads involved in this mailbox will become non-real-time threads.`

Returns

Return a pointer to the mailbox object. When the return value is RT_NULL, it means the creation failed.

Warning

This function can ONLY be called from threads.

6.2.6.8.2.4 rt_mb_delete()

```
rt_err_t rt_mb_delete (
            rt_mailbox_t mb)
```

This function will delete a mailbox object and release the memory space.

Note

This function is used to delete a mailbox object which is created by the rt_mb_create() function. By contrast, the rt_mb_detach() function will detach a static mailbox object. When the mailbox is successfully deleted, it will resume all suspended threads in the mailbox list.

See also

rt_mb_detach()

Parameters

mb	is a pointer to a mailbox object to be deleted.

Returns

Return the operation status. When the return value is RT_EOK, the operation is successful. If the return value is any other values, it means that the mailbox detach failed.

Warning

This function can only delete mailbox created by the rt_mb_create() function. If the mailbox is initialized by the rt_mb_init() function, you MUST NOT USE this function to delete it, ONLY USE the rt_mb_detach() function to complete the detachment.

6.2.6.8.2.5 rt_mb_send()

```
rt_err_t rt_mb_send (
            rt_mailbox_t mb,
            rt_ubase_t value)
```

This function will send an mail to the mailbox object. If there is a thread suspended on the mailbox, the thread will be resumed.

Note

When using this function to send a mail, if the mailbox is fully used, this function will return an error code immediately without waiting time. By contrast, the rt_mb_send_wait() function is set a timeout to wait for the mail to be sent.

See also

rt_mb_send_wait()

Parameters

mb	is a pointer to the mailbox object to be sent.
value	is a value to the content of the mail you want to send.

Returns

Return the operation status. When the return value is RT_EOK, the operation is successful. If the return value is any other values, it means that the mailbox detach failed.

6.2.6.8.2.6 rt_mb_urgent()

```
rt_err_t rt_mb_urgent (
            rt_mailbox_t mb,
            rt_ubase_t value)
```

This function will send an urgent mail to the mailbox object.

Note

This function is almost the same as the rt_mb_send() function. The only difference is that when sending an urgent mail, the mail will be placed at the head of the mail queue so that the recipient can receive the urgent mail first.

See also

rt_mb_send()

Parameters

mb	is a pointer to the mailbox object to be sent.
value	is the content of the mail you want to send.

Returns

Return the operation status. When the return value is RT_EOK, the operation is successful. If the return value is any other values, it means that the mailbox detach failed.

6.2.6.8.2.7 rt_mb_control()

```
rt_err_t rt_mb_control (
            rt_mailbox_t mb,
            int cmd,
            void * arg)
```

This function will set some extra attributions of a mailbox object.

Note

Currently this function only supports the RT_IPC_CMD_RESET command to reset the mailbox.

Parameters

mb	is a pointer to a mailbox object.
cmd	is a command used to configure some attributions of the mailbox.
arg	is the argument of the function to execute the command.

Returns

Return the operation status. When the return value is RT_EOK, the operation is successful. If the return value is any other values, it means that this function failed to execute.

6.2.6.9 Message Queue

Collaboration diagram for Message Queue:

Data Structures

- struct rt_messagequeue

Functions

- rt_err_t rt_mq_init (rt_mq_t mq, const char ∗name, void ∗msgpool, rt_size_t msg_size, rt_size_t pool_size, rt_uint8_t flag)
- rt_err_t rt_mq_detach (rt_mq_t mq)
- rt_mq_t rt_mq_create (const char ∗name, rt_size_t msg_size, rt_size_t max_msgs, rt_uint8_t flag)
- rt_err_t rt_mq_delete (rt_mq_t mq)
- rt_err_t rt_mq_send (rt_mq_t mq, const void ∗buffer, rt_size_t size)
- rt_err_t rt_mq_urgent (rt_mq_t mq, const void ∗buffer, rt_size_t size)
- rt_err_t rt_mq_control (rt_mq_t mq, int cmd, void ∗arg)

6.2.6.9.1 Detailed Description

6.2.6.9.2 Function Documentation

6.2.6.9.2.1 rt_mq_init()

```
rt_err_t rt_mq_init (
            rt_mq_t mq,
            const char * name,
            void * msgpool,
            rt_size_t msg_size,
            rt_size_t pool_size,
            rt_uint8_t flag)
```

Initialize a static messagequeue object.

Note

For the static messagequeue object, its memory space is allocated by the compiler during compiling, and shall placed on the read-write data segment or on the uninitialized data segment. By contrast, the rt_mq_create() function will allocate memory space automatically and initialize the messagequeue.

See also

rt_mq_create()

Parameters

mq	is a pointer to the messagequeue to initialize. It is assumed that storage for the messagequeue will be allocated in your application.
name	is a pointer to the name that given to the messagequeue.
msgpool	is a pointer to the starting address of the memory space you allocated for the messagequeue in advance. In other words, msgpool is a pointer to the messagequeue buffer of the starting address.
msg_size	is the maximum length of a message in the messagequeue (Unit: Byte).
pool_size	is the size of the memory space allocated for the messagequeue in advance.

flag	is the messagequeue flag, which determines the queuing way of how multiple threads wait when the messagequeue is not available. The messagequeue flag can be ONE of the following values:
	`RT_IPC_FLAG_PRIO The pending threads will queue in order of priority.`
	
	`RT_IPC_FLAG_FIFO The pending threads will queue in the first-in-first-out method (also known as first-come-first-served (FCFS) scheduling strategy).`
	
	`NOTE: RT_IPC_FLAG_FIFO is a non-real-time scheduling mode. It is strongly recommended to use RT_IPC_FLAG_PRIO to ensure the thread is real-time UNLESS your applications concern about the first-in-first-out principle, and you clearly understand that all threads involved in this messagequeue will become non-real-time threads.`

Returns

Return the operation status. When the return value is RT_EOK, the initialization is successful. If the return value is any other values, it represents the initialization failed.

Warning

This function can ONLY be called from threads.

6.2.6.9.2.2 rt_mq_detach()

```
rt_err_t rt_mq_detach (
              rt_mq_t mq)
```

This function will detach a static messagequeue object.

Note

This function is used to detach a static messagequeue object which is initialized by rt_mq_init() function. By contrast, the rt_mq_delete() function will delete a messagequeue object. When the messagequeue is successfully detached, it will resume all suspended threads in the messagequeue list.

See also

rt_mq_delete()

Parameters

mq	is a pointer to a messagequeue object to be detached.

Returns

Return the operation status. When the return value is RT_EOK, the initialization is successful. If the return value is any other values, it means that the messagequeue detach failed.

Warning

This function can ONLY detach a static messagequeue initialized by the rt_mq_init() function. If the messagequeue is created by the rt_mq_create() function, you MUST NOT USE this function to detach it, and ONLY USE the rt_mq_delete() function to complete the deletion.

6.2.6.9.2.3 rt_mq_create()

```
rt_mq_t rt_mq_create (
            const char * name,
            rt_size_t msg_size,
            rt_size_t max_msgs,
            rt_uint8_t flag)
```

Creating a messagequeue object.

Note

For the messagequeue object, its memory space is allocated automatically. By contrast, the rt_mq_init() function will initialize a static messagequeue object.

See also

rt_mq_init()

Parameters

name	is a pointer that given to the messagequeue.
msg_size	is the maximum length of a message in the messagequeue (Unit: Byte).
max_msgs	is the maximum number of messages in the messagequeue.
flag	is the messagequeue flag, which determines the queuing way of how multiple threads wait when the messagequeue is not available. The messagequeue flag can be ONE of the following values: RT_IPC_FLAG_PRIO The pending threads will queue in order of priority. RT_IPC_FLAG_FIFO The pending threads will queue in the first-in-first-out method (also known as first-come-first-served (FCFS) scheduling strategy). NOTE: RT_IPC_FLAG_FIFO is a non-real-time scheduling mode. It is strongly recommended to use RT_IPC_FLAG_PRIO to ensure the thread is real-time UNLESS your applications concern about the first-in-first-out principle, and you clearly understand that all threads involved in this messagequeue will become non-real-time threads.

Returns

Return a pointer to the messagequeue object. When the return value is RT_NULL, it means the creation failed.

Warning

This function can NOT be called in interrupt context. You can use macor RT_DEBUG_NOT_IN_INTERRUPT to check it.

6.2.6.9.2.4 rt_mq_delete()

```
rt_err_t rt_mq_delete (
            rt_mq_t mq)
```

This function will delete a messagequeue object and release the memory.

Note

This function is used to delete a messagequeue object which is created by the rt_mq_create() function. By contrast, the rt_mq_detach() function will detach a static messagequeue object. When the messagequeue is successfully deleted, it will resume all suspended threads in the messagequeue list.

See also

rt_mq_detach()

Parameters

mq	is a pointer to a messagequeue object to be deleted.

Returns

Return the operation status. When the return value is RT_EOK, the operation is successful. If the return value is any other values, it means that the messagequeue detach failed.

Warning

This function can ONLY delete a messagequeue initialized by the rt_mq_create() function. If the messagequeue is initialized by the rt_mq_init() function, you MUST NOT USE this function to delete it, ONLY USE the rt_mq_detach() function to complete the detachment. for example,the rt_mq_create() function, it cannot be called in interrupt context.

6.2.6.9.2.5 rt_mq_send()

```
rt_err_t rt_mq_send (
            rt_mq_t mq,
            const void * buffer,
            rt_size_t size)
```

This function will send a message to the messagequeue object. If there is a thread suspended on the messagequeue, the thread will be resumed.

Note

When using this function to send a message, if the messagequeue is fully used, the current thread will wait for a timeout. By contrast, when the messagequeue is fully used, the rt_mq_send_wait() function will return an error code immediately without waiting.

See also

rt_mq_send_wait()

Parameters

mq	is a pointer to the messagequeue object to be sent.
buffer	is the content of the message.
size	is the length of the message(Unit: Byte).

Returns

Return the operation status. When the return value is RT_EOK, the operation is successful. If the return value is any other values, it means that the messagequeue detach failed.

Warning

This function can be called in interrupt context and thread context.

6.2.6.9.2.6 rt_mq_urgent()

```
rt_err_t rt_mq_urgent (
             rt_mq_t mq,
             const void * buffer,
             rt_size_t size)
```

This function will send an urgent message to the messagequeue object.

Note

This function is almost the same as the rt_mq_send() function. The only difference is that when sending an urgent message, the message is placed at the head of the messagequeue so that the recipient can receive the urgent message first.

See also

rt_mq_send()

Parameters

mq	is a pointer to the messagequeue object to be sent.
buffer	is the content of the message.
size	is the length of the message(Unit: Byte).

Returns

Return the operation status. When the return value is RT_EOK, the operation is successful. If the return value is any other values, it means that the mailbox detach failed.

6.2.6.9.2.7 rt_mq_control()

```
rt_err_t rt_mq_control (
             rt_mq_t mq,
             int cmd,
             void * arg)
```

This function will set some extra attributions of a messagequeue object.

Note

Currently this function only supports the RT_IPC_CMD_RESET command to reset the messagequeue.

Parameters

mq	is a pointer to a messagequeue object.
cmd	is a command used to configure some attributions of the messagequeue.
arg	is the argument of the function to execute the command.

Returns

Return the operation status. When the return value is RT_EOK, the operation is successful. If the return value is any other values, it means that this function failed to execute.

6.2.7 Memory Management

Collaboration diagram for Memory Management:

Data Structures

- struct rt_mempool

Functions

- void * rt_smem_alloc (rt_smem_t m, rt_size_t size)
- void * rt_smem_realloc (rt_smem_t m, void *rmem, rt_size_t newsize)
- void rt_smem_free (void *rmem)
- rt_err_t rt_mp_init (struct rt_mempool *mp, const char *name, void *start, rt_size_t size, rt_size_t block_size)
- rt_err_t rt_mp_detach (struct rt_mempool *mp)
- rt_mp_t rt_mp_create (const char *name, rt_size_t block_count, rt_size_t block_size)
- rt_err_t rt_mp_delete (rt_mp_t mp)
- void * rt_mp_alloc (rt_mp_t mp, rt_int32_t time)
- void rt_mp_free (void *block)
- void rt_system_heap_init (void *begin_addr, void *end_addr)
- void rt_system_heap_init_generic (void *begin_addr, void *end_addr)
- void * rt_malloc (rt_size_t size)
- void rt_free (void *ptr)
- void * rt_realloc (void *ptr, rt_size_t newsize)
- void * rt_calloc (rt_size_t count, rt_size_t size)
- void * rt_malloc_align (rt_size_t size, rt_size_t align)
- void rt_free_align (void *ptr)
- void rt_memory_info (rt_size_t *total, rt_size_t *used, rt_size_t *max_used)
- rt_smem_t rt_smem_init (const char *name, void *begin_addr, rt_size_t size)
- rt_err_t rt_smem_detach (rt_smem_t m)

- void rt_realloc_set_entry_hook (void(*hook)(void **ptr, rt_size_t size))
- void rt_realloc_set_exit_hook (void(*hook)(void **ptr, rt_size_t size))
- void rt_free_sethook (void(*hook)(void **ptr))

6.2.7.1 Detailed Description

memory management for memory pool and heap memory
RT-Thread operating system supports two types memory management:

- Static memory pool management
- Dynamic memory heap management.

The time to allocate a memory block from the memory pool is determinant. When the memory pool is empty, the allocated thread can be blocked (or immediately return, or waiting for sometime to return, which are determined by a timeout parameter). When other thread releases memory blocks to this memory pool, the blocked thread is wake up.
There are two methods in dynamic memory heap management, one is used for small memory, such as less than 1MB. Another is a SLAB like memory management, which is suitable for large memory system. All of them has no real-time character.

6.2.7.2 Function Documentation

6.2.7.2.1 rt_smem_alloc()

```
void * rt_smem_alloc (
            rt_smem_t m,
            rt_size_t size)
```

Allocate a block of memory with a minimum of 'size' bytes.

Parameters

m	the small memory management object.
size	is the minimum size of the requested block in bytes.

Returns

the pointer to allocated memory or NULL if no free memory was found.

6.2.7.2.2 rt_smem_realloc()

```
void * rt_smem_realloc (
            rt_smem_t m,
            void * rmem,
            rt_size_t newsize)
```

This function will change the size of previously allocated memory block.

Parameters

m	the small memory management object.
rmem	is the pointer to memory allocated by rt_mem_alloc.
newsize	is the required new size.

Returns

the changed memory block address.

6.2.7.2.3 rt_smem_free()

```
void rt_smem_free (
            void * rmem)
```

This function will release the previously allocated memory block by rt_mem_alloc. The released memory block is taken back to system heap.

Parameters

rmem	the address of memory which will be released.

6.2.7.2.4 rt_mp_init()

```
rt_err_t rt_mp_init (
             struct rt_mempool * mp,
             const char * name,
             void * start,
             rt_size_t size,
             rt_size_t block_size)
```

This function will initialize a memory pool object, normally which is used for static object.

Parameters

mp	is the memory pool object.
name	is the name of the memory pool.
start	is the start address of the memory pool.
size	is the total size of the memory pool.
block_size	is the size for each block..

Returns

RT_EOK

6.2.7.2.5 rt_mp_detach()

```
rt_err_t rt_mp_detach (
             struct rt_mempool * mp)
```

This function will detach a memory pool from system object management.

Parameters

mp	is the memory pool object.

Returns

RT_EOK

6.2.7.2.6 rt_mp_create()

```
rt_mp_t rt_mp_create (
             const char * name,
             rt_size_t block_count,
             rt_size_t block_size)
```

This function will create a mempool object and allocate the memory pool from heap.

Parameters

name	is the name of memory pool.
block_count	is the count of blocks in memory pool.
block_size	is the size for each block.

Returns

the created mempool object

6.2.7.2.7 rt_mp_delete()

```
rt_err_t rt_mp_delete (
            rt_mp_t mp)
```

This function will delete a memory pool and release the object memory.

Parameters

mp	is the memory pool object.

Returns

RT_EOK

6.2.7.2.8 rt_mp_alloc()

```
void * rt_mp_alloc (
            rt_mp_t mp,
            rt_int32_t time)
```

This function will allocate a block from memory pool.

Parameters

mp	is the memory pool object.
time	is the maximum waiting time for allocating memory. • 0 for not waiting, allocating memory immediately.

Returns

the allocated memory block or RT_NULL on allocated failed.

6.2.7.2.9 rt_mp_free()

```
void rt_mp_free (
            void * block)
```

This function will release a memory block.

Parameters

block	the address of memory block to be released.

6.2.7.2.10 rt_system_heap_init()

```
void rt_system_heap_init (
            void * begin_addr,
            void * end_addr)
```

This function will init system heap. User can override this API to complete other works, like heap sanitizer initialization.

Parameters

begin_addr	the beginning address of system page.
end_addr	the end address of system page.

6.2.7.2.11 rt_system_heap_init_generic()

```
void rt_system_heap_init_generic (
            void * begin_addr,
            void * end_addr)
```

This function will do the generic system heap initialization.

Parameters

begin_addr	the beginning address of system page.
end_addr	the end address of system page.

6.2.7.2.12 rt_malloc()

```
void * rt_malloc (
            rt_size_t size)
```

Allocate a block of memory with a minimum of 'size' bytes.

Parameters

size	is the minimum size of the requested block in bytes.

Returns

the pointer to allocated memory or NULL if no free memory was found.

6.2.7.2.13 rt_free()

```
void rt_free (
            void * ptr)
```

This function will release the previously allocated memory block by rt_malloc. The released memory block is taken back to system heap.

Parameters

ptr	the address of memory which will be released.

6.2.7.2.14 rt_realloc()

```
void * rt_realloc (
```

```
            void * ptr,
            rt_size_t newsize)
```

This function will change the size of previously allocated memory block.

Parameters

ptr	is the pointer to memory allocated by rt_malloc.
newsize	is the required new size.

Returns

the changed memory block address.

6.2.7.2.15 rt_calloc()

```
void * rt_calloc (
            rt_size_t count,
            rt_size_t size)
```

This function will contiguously allocate enough space for count objects that are size bytes of memory each and returns a pointer to the allocated memory.

Note

The allocated memory is filled with bytes of value zero.

Parameters

count	is the number of objects to allocate.
size	is the size of one object to allocate.

Returns

pointer to allocated memory / NULL pointer if there is an error.

6.2.7.2.16 rt_malloc_align()

```
void * rt_malloc_align (
            rt_size_t size,
            rt_size_t align)
```

This function allocates a memory block, which address is aligned to the specified alignment size.

Parameters

size	is the allocated memory block size.
align	is the alignment size.

Returns

The memory block address was returned successfully, otherwise it was returned empty RT_NULL.

6.2.7.2.17 rt_free_align()

```
void rt_free_align (
            void * ptr)
```

This function release the memory block, which is allocated by rt_malloc_align function and address is aligned.

Parameters

ptr	is the memory block pointer.

6.2.7.2.18 rt_memory_info()

```
void rt_memory_info (
            rt_size_t * total,
            rt_size_t * used,
            rt_size_t * max_used)
```

This function will caculate the total memory, the used memory, and the max used memory.

Parameters

total	is a pointer to get the total size of the memory.
used	is a pointer to get the size of memory used.
max_used	is a pointer to get the maximum memory used.

6.2.7.2.19 rt_realloc_set_entry_hook()

```
void rt_realloc_set_entry_hook (
            void(* hook )(void **ptr, rt_size_t size))
```

This function will set a hook function, which will be invoked when a memory block is allocated from heap memory.

Parameters

hook	the hook function.

6.2.7.2.20 rt_realloc_set_exit_hook()

```
void rt_realloc_set_exit_hook (
            void(* hook )(void **ptr, rt_size_t size))
```

This function will set a hook function, which will be invoked when a memory block is allocated from heap memory.

Parameters

hook	the hook function.

6.2.7.2.21 rt_free_sethook()

```
void rt_free_sethook (
            void(* hook )(void **ptr))
```

This function will set a hook function, which will be invoked when a memory block is released to heap memory.

Parameters

hook	the hook function

6.2.7.2.22 rt_smem_init()

```
rt_smem_t rt_smem_init (
            const char * name,
            void * begin_addr,
            rt_size_t size)
```

This function will initialize small memory management algorithm.
small memory object interface

Parameters

name	is the name of the small memory management object.
begin_addr	the beginning address of memory.
size	is the size of the memory.

Returns

Return a pointer to the memory object. When the return value is RT_NULL, it means the init failed.

6.2.7.2.23 rt_smem_detach()

```
rt_err_t rt_smem_detach (
            rt_smem_t m)
```

This function will remove a small mem from the system.

Parameters

m	the small memory management object.

Returns

RT_EOK

6.2.8 Runtime Trace and Record

Collaboration diagram for Runtime Trace and Record:

- void rt_malloc_sethook (void(∗hook)(void ∗∗ptr, rt_size_t size))
- void rt_realloc_set_entry_hook (void(∗hook)(void ∗∗ptr, rt_size_t size))
- void rt_realloc_set_exit_hook (void(∗hook)(void ∗∗ptr, rt_size_t size))
- void rt_free_sethook (void(∗hook)(void ∗∗ptr))

6.2.8.1 Detailed Description

the hook function set in runtime
In order to trace and record RT-Thread activity in runtime, a hook mechanism is introduced.
The hooks are a series of routines, which are invoked in some special checkpoints. The hook routines include:

- object hook, invoked at object created, deleted, taken and put etc.
- scheduler hook, invoked at thread switch and idle thread loop.
- memory hook, invoked when allocate or free memory block.
- timer hook, invoked when timer is timeout.

6.2.8.2 Function Documentation

6.2.8.2.1 rt_malloc_sethook()

```
void rt_malloc_sethook (
            void(* hook )(void **ptr, rt_size_t size))
```

This function will set a hook function, which will be invoked when a memory block is allocated from heap memory.

Parameters

hook	the hook function.

6.2.8.2.2 rt_realloc_set_entry_hook()

```
void rt_realloc_set_entry_hook (
            void(* hook )(void **ptr, rt_size_t size))
```

This function will set a hook function, which will be invoked when a memory block is allocated from heap memory.

Parameters

hook	the hook function.

6.2.8.2.3 rt_realloc_set_exit_hook()

```
void rt_realloc_set_exit_hook (
            void(* hook )(void **ptr, rt_size_t size))
```

This function will set a hook function, which will be invoked when a memory block is allocated from heap memory.

Parameters

hook	the hook function.

6.2.8.2.4 rt_free_sethook()

```
void rt_free_sethook (
            void(* hook )(void **ptr))
```

This function will set a hook function, which will be invoked when a memory block is released to heap memory.

Parameters

hook	the hook function

6.2.9 Other useful kernel service

Collaboration diagram for Other useful kernel service:

Macros

- #define RT_HW_BACKTRACE_FRAME_GET_SELF(frame)
- #define rt_container_of(ptr, type, member)
- #define RT_LIST_OBJECT_INIT(object)
- #define rt_list_entry(node, type, member)
- #define rt_list_for_each(pos, head)
- #define rt_list_for_each_safe(pos, n, head)
- #define rt_list_for_each_entry(pos, head, member)

- #define rt_list_for_each_entry_safe(pos, n, head, member)
- #define rt_list_first_entry(ptr, type, member)
- #define rt_slist_entry(node, type, member)
- #define rt_slist_for_each(pos, head)
- #define rt_slist_for_each_entry(pos, head, member)
- #define rt_slist_first_entry(ptr, type, member)
- #define rt_slist_tail_entry(ptr, type, member)

Functions

- rt_weak rt_err_t rt_hw_backtrace_frame_get (rt_thread_t thread, struct rt_hw_backtrace_frame *frame)
- rt_weak rt_err_t rt_hw_backtrace_frame_unwind (rt_thread_t thread, struct rt_hw_backtrace_frame *frame)
- void rt_show_version (void)
- rt_weak rt_err_t rt_backtrace (void)
- rt_weak rt_err_t rt_backtrace_frame (rt_thread_t thread, struct rt_hw_backtrace_frame *frame)
- rt_weak rt_err_t rt_backtrace_formatted_print (rt_ubase_t *buffer, long buflen)
- rt_weak rt_err_t rt_backtrace_to_buffer (rt_thread_t thread, struct rt_hw_backtrace_frame *frame, long skip, rt_ubase_t *buffer, long buflen)
- rt_err_t rt_backtrace_thread (rt_thread_t thread)
- void rt_system_heap_init_generic (void *begin_addr, void *end_addr)
- rt_weak void rt_system_heap_init (void *begin_addr, void *end_addr)
- rt_weak void * rt_malloc (rt_size_t size)
- rt_weak void * rt_realloc (void *ptr, rt_size_t newsize)
- rt_weak void * rt_calloc (rt_size_t count, rt_size_t size)
- rt_weak void rt_free (void *ptr)
- rt_weak void rt_memory_info (rt_size_t *total, rt_size_t *used, rt_size_t *max_used)
- rt_weak void * rt_malloc_align (rt_size_t size, rt_size_t align)
- rt_weak void rt_free_align (void *ptr)
- int __rt_fls (int val)
- int __rt_ffs (int value)
- rt_inline void rt_list_init (rt_list_t *l)
- rt_inline void rt_list_insert_after (rt_list_t *l, rt_list_t *n)
- rt_inline void rt_list_insert_before (rt_list_t *l, rt_list_t *n)
- rt_inline void rt_list_remove (rt_list_t *n)
- rt_inline int rt_list_isempty (const rt_list_t *l)
- rt_inline unsigned int rt_list_len (const rt_list_t *l)
- rt_inline void rt_slist_init (rt_slist_t *l)

- void rt_realloc_set_entry_hook (void(*hook)(void **ptr, rt_size_t size))
- void rt_realloc_set_exit_hook (void(*hook)(void **ptr, rt_size_t size))
- void rt_free_sethook (void(*hook)(void **ptr))

6.2.9.1 Detailed Description

other useful service in the kernel

6.2.9.2 Macro Definition Documentation

6.2.9.2.1 RT_HW_BACKTRACE_FRAME_GET_SELF

```
#define RT_HW_BACKTRACE_FRAME_GET_SELF(
                frame)
```

Value:

```
do                                      \
{                                       \
    (frame)->fp = 0;                    \
    (frame)->pc = 0;                    \
} while (0)
```

Note

can be overridden by cpuport.h which is defined by a specific arch

6.2.9.2.2 rt_container_of

```
#define rt_container_of(
              ptr,
              type,
              member)
```

Value:

```
((type *)((char *)(ptr) - (unsigned long)(&((type *)0)->member)))
```

rt_container_of - return the start address of struct type, while ptr is the member of struct type.

6.2.9.2.3 RT_LIST_OBJECT_INIT

```
#define RT_LIST_OBJECT_INIT(
              object)
```

Value:

```
{ &(object), &(object) }
```

initialize a list object

6.2.9.2.4 rt_list_entry

```
#define rt_list_entry(
              node,
              type,
              member)
```

Value:

```
rt_container_of(node, type, member)
```

get the struct for this entry

Parameters

node	the entry point
type	the type of structure
member	the name of list in structure

6.2.9.2.5 rt_list_for_each

```
#define rt_list_for_each(
              pos,
              head)
```

Value:

```
for (pos = (head)->next; pos != (head); pos = pos->next)
```

rt_list_for_each - iterate over a list

Parameters

pos	the rt_list_t * to use as a loop cursor.
head	the head for your list.

6.2.9.2.6 rt_list_for_each_safe

```
#define rt_list_for_each_safe(
              pos,
```

```
                n,
                head)
```

Value:

```
for (pos = (head)->next, n = pos->next; pos != (head); \
    pos = n, n = pos->next)
```

rt_list_for_each_safe - iterate over a list safe against removal of list entry

Parameters

pos	the rt_list_t * to use as a loop cursor.
n	another rt_list_t * to use as temporary storage
head	the head for your list.

6.2.9.2.7 rt_list_for_each_entry

```
#define rt_list_for_each_entry(
                pos,
                head,
                member)
```

Value:

```
for (pos = rt_list_entry((head)->next, rt_typeof(*pos), member); \
     &pos->member != (head); \
     pos = rt_list_entry(pos->member.next, rt_typeof(*pos), member))
```

rt_list_for_each_entry - iterate over list of given type

Parameters

pos	the type * to use as a loop cursor.
head	the head for your list.
member	the name of the list_struct within the struct.

6.2.9.2.8 rt_list_for_each_entry_safe

```
#define rt_list_for_each_entry_safe(
                pos,
                n,
                head,
                member)
```

Value:

```
for (pos = rt_list_entry((head)->next, rt_typeof(*pos), member), \
     n = rt_list_entry(pos->member.next, rt_typeof(*pos), member); \
     &pos->member != (head); \
     pos = n, n = rt_list_entry(n->member.next, rt_typeof(*n), member))
```

rt_list_for_each_entry_safe - iterate over list of given type safe against removal of list entry

Parameters

pos	the type * to use as a loop cursor.
n	another type * to use as temporary storage
head	the head for your list.
member	the name of the list_struct within the struct.

6.2.9.2.9 rt_list_first_entry

```
#define rt_list_first_entry(
                ptr,
                type,
                member)
```

Value:

```
rt_list_entry((ptr)->next, type, member)
```

rt_list_first_entry - get the first element from a list

Parameters

ptr	the list head to take the element from.
type	the type of the struct this is embedded in.
member	the name of the list_struct within the struct.

Note, that list is expected to be not empty.

6.2.9.2.10 rt_slist_entry

```
#define rt_slist_entry(
                node,
                type,
                member)
```

Value:

```
rt_container_of(node, type, member)
```

get the struct for this single list node

Parameters

node	the entry point
type	the type of structure
member	the name of list in structure

6.2.9.2.11 rt_slist_for_each

```
#define rt_slist_for_each(
                pos,
                head)
```

Value:

```
for (pos = (head)->next; pos != RT_NULL; pos = pos->next)
```

rt_slist_for_each - iterate over a single list

Parameters

pos	the rt_slist_t * to use as a loop cursor.
head	the head for your single list.

6.2.9.2.12 rt_slist_for_each_entry

```
#define rt_slist_for_each_entry(
                pos,
                head,
                member)
```

Value:

```
for (pos = ((head)->next == (RT_NULL) ? (RT_NULL) : rt_slist_entry((head)->next, rt_typeof(*pos), member));
      \
     pos != (RT_NULL) && &pos->member != (RT_NULL); \
     pos = (pos->member.next == (RT_NULL) ? (RT_NULL) : rt_slist_entry(pos->member.next, rt_typeof(*pos),
      member)))
```

rt_slist_for_each_entry - iterate over single list of given type

Parameters

pos	the type * to use as a loop cursor.
head	the head for your single list.
member	the name of the list_struct within the struct.

6.2.9.2.13 rt_slist_first_entry

```
#define rt_slist_first_entry(
              ptr,
              type,
              member)
```

Value:

```
rt_slist_entry((ptr)->next, type, member)
```

rt_slist_first_entry - get the first element from a slist

Parameters

ptr	the slist head to take the element from.
type	the type of the struct this is embedded in.
member	the name of the slist_struct within the struct.

Note, that slist is expected to be not empty.

6.2.9.2.14 rt_slist_tail_entry

```
#define rt_slist_tail_entry(
              ptr,
              type,
              member)
```

Value:

```
rt_slist_entry(rt_slist_tail(ptr), type, member)
```

rt_slist_tail_entry - get the tail element from a slist

Parameters

ptr	the slist head to take the element from.
type	the type of the struct this is embedded in.
member	the name of the slist_struct within the struct.

Note, that slist is expected to be not empty.

6.2.9.3 Function Documentation

6.2.9.3.1 rt_hw_backtrace_frame_get()

```
rt_weak rt_err_t rt_hw_backtrace_frame_get (
             rt_thread_t thread,
```

```
            struct rt_hw_backtrace_frame * frame)
```

Get the inner most frame of target thread.

Parameters

thread	the thread which frame belongs to
frame	the specified frame to be unwound

Returns

rt_err_t 0 is succeed, otherwise a failure

6.2.9.3.2 rt_hw_backtrace_frame_unwind()

```
rt_weak rt_err_t rt_hw_backtrace_frame_unwind (
            rt_thread_t thread,
            struct rt_hw_backtrace_frame * frame)
```

Unwind the target frame.

Parameters

thread	the thread which frame belongs to
frame	the specified frame to be unwound

Returns

rt_err_t 0 is succeed, otherwise a failure

6.2.9.3.3 rt_show_version()

```
void rt_show_version (
            void )
```

This function will show the version of rt-thread rtos.

6.2.9.3.4 rt_backtrace()

```
rt_err_t rt_backtrace (
            void )
```

Print backtrace of current thread to system console device.

Returns

rt_err_t 0 is success, otherwise a failure

6.2.9.3.5 rt_backtrace_frame()

```
rt_err_t rt_backtrace_frame (
            rt_thread_t thread,
            struct rt_hw_backtrace_frame * frame)
```

Print backtrace from frame to system console device.

Parameters

thread	the thread which frame belongs to

frame	where backtrace starts from

Returns

rt_err_t 0 is success, otherwise a failure

6.2.9.3.6 rt_backtrace_formatted_print()

```
rt_err_t rt_backtrace_formatted_print (
            rt_ubase_t * buffer,
            long buflen)
```

Print backtrace from buffer to system console.

Parameters

buffer	where traced frames saved
buflen	number of items in buffer

Returns

rt_err_t 0 is success, otherwise a failure

6.2.9.3.7 rt_backtrace_to_buffer()

```
rt_err_t rt_backtrace_to_buffer (
            rt_thread_t thread,
            struct rt_hw_backtrace_frame * frame,
            long skip,
            rt_ubase_t * buffer,
            long buflen)
```

Print backtrace from frame to the given buffer.

Parameters

thread	the thread which frame belongs to
frame	where backtrace starts from. NULL if it's the current one
skip	the number of frames to discarded counted from calling function. Noted that the inner most frame is always discarded and not counted, which is obviously reasonable since that's this function itself.
buffer	where traced frames saved
buflen	max number of items can be saved in buffer. If there are no more than buflen items to be saved, there will be a NULL after the last saved item in the buffer.

Returns

rt_err_t 0 is success, otherwise a failure

6.2.9.3.8 rt_backtrace_thread()

```
rt_err_t rt_backtrace_thread (
            rt_thread_t thread)
```

Print backtrace of a thread to system console device.

Parameters

thread	which call stack is traced

Returns

rt_err_t 0 is success, otherwise a failure

6.2.9.3.9 rt_realloc_set_entry_hook()

```
void rt_realloc_set_entry_hook (
            void(* hook )(void **ptr, rt_size_t size))
```

This function will set a hook function, which will be invoked when a memory block is allocated from heap memory.

Parameters

hook	the hook function.

6.2.9.3.10 rt_realloc_set_exit_hook()

```
void rt_realloc_set_exit_hook (
            void(* hook )(void **ptr, rt_size_t size))
```

This function will set a hook function, which will be invoked when a memory block is allocated from heap memory.

Parameters

hook	the hook function.

6.2.9.3.11 rt_free_sethook()

```
void rt_free_sethook (
            void(* hook )(void **ptr))
```

This function will set a hook function, which will be invoked when a memory block is released to heap memory.

Parameters

hook	the hook function

6.2.9.3.12 rt_system_heap_init_generic()

```
void rt_system_heap_init_generic (
            void * begin_addr,
            void * end_addr)
```

This function will do the generic system heap initialization.

Parameters

begin_addr	the beginning address of system page.
end_addr	the end address of system page.

6.2.9.3.13 rt_system_heap_init()

```
rt_weak void rt_system_heap_init (
            void * begin_addr,
            void * end_addr)
```

This function will init system heap. User can override this API to complete other works, like heap sanitizer initialization.

Parameters

begin_addr	the beginning address of system page.
end_addr	the end address of system page.

6.2.9.3.14 rt_malloc()

```
rt_weak void * rt_malloc (
            rt_size_t size)
```

Allocate a block of memory with a minimum of 'size' bytes.

Parameters

size	is the minimum size of the requested block in bytes.

Returns

the pointer to allocated memory or NULL if no free memory was found.

6.2.9.3.15 rt_realloc()

```
rt_weak void * rt_realloc (
            void * ptr,
            rt_size_t newsize)
```

This function will change the size of previously allocated memory block.

Parameters

ptr	is the pointer to memory allocated by rt_malloc.
newsize	is the required new size.

Returns

the changed memory block address.

6.2.9.3.16 rt_calloc()

```
rt_weak void * rt_calloc (
            rt_size_t count,
            rt_size_t size)
```

This function will contiguously allocate enough space for count objects that are size bytes of memory each and returns a pointer to the allocated memory.

Note

The allocated memory is filled with bytes of value zero.

Parameters

count	is the number of objects to allocate.
size	is the size of one object to allocate.

Returns

pointer to allocated memory / NULL pointer if there is an error.

6.2.9.3.17 rt_free()

```
rt_weak void rt_free (
             void * ptr)
```

This function will release the previously allocated memory block by rt_malloc. The released memory block is taken back to system heap.

Parameters

ptr	the address of memory which will be released.

6.2.9.3.18 rt_memory_info()

```
rt_weak void rt_memory_info (
             rt_size_t * total,
             rt_size_t * used,
             rt_size_t * max_used)
```

This function will caculate the total memory, the used memory, and the max used memory.

Parameters

total	is a pointer to get the total size of the memory.
used	is a pointer to get the size of memory used.
max_used	is a pointer to get the maximum memory used.

6.2.9.3.19 rt_malloc_align()

```
rt_weak void * rt_malloc_align (
             rt_size_t size,
             rt_size_t align)
```

This function allocates a memory block, which address is aligned to the specified alignment size.

Parameters

size	is the allocated memory block size.
align	is the alignment size.

Returns

The memory block address was returned successfully, otherwise it was returned empty RT_NULL.

6.2.9.3.20 rt_free_align()

```
rt_weak void rt_free_align (
             void * ptr)
```

This function release the memory block, which is allocated by rt_malloc_align function and address is aligned.

Parameters

ptr	is the memory block pointer.

6.2.9.3.21 __rt_fls()

```
int __rt_fls (
             int val)
```

Find the index of the most significant set bit in a 32-bit integer.
The result is the position of the highest bit set to 1, counting from 1 for the least significant bit. If the input value is 0, the function returns 0.
Examples:

- fls(0) = 0
- fls(1) = 1
- fls(0x80000000) = 32

Parameters

val	32-bit integer value to examine.

Returns

Position of the most significant set bit (1–32), or 0 if `val` is 0.

6.2.9.3.22 __rt_ffs()

```
int __rt_ffs (
             int value)
```

This function finds the first bit set (beginning with the least significant bit) in value and return the index of that bit. Bits are numbered starting at 1 (the least significant bit). A return value of zero from any of these functions means that the argument was zero.

Parameters

value	is the value to find the first bit set in.

Returns

Return the index of the first bit set. If value is 0, then this function shall return 0.

6.2.9.3.23 rt_list_init()

```
rt_inline void rt_list_init (
             rt_list_t * l)
```

initialize a list

Parameters

l	list to be initialized

6.2.9.3.24 rt_list_insert_after()

```
rt_inline void rt_list_insert_after (
             rt_list_t * l,
             rt_list_t * n)
```

insert a node after a list

Parameters

l	list to insert it
n	new node to be inserted

6.2.9.3.25 rt_list_insert_before()

```
rt_inline void rt_list_insert_before (
             rt_list_t * l,
             rt_list_t * n)
```

insert a node before a list

Parameters

n	new node to be inserted
l	list to insert it

6.2.9.3.26 rt_list_remove()

```
rt_inline void rt_list_remove (
             rt_list_t * n)
```

remove node from list.

Parameters

n	the node to remove from the list.

6.2.9.3.27 rt_list_isempty()

```
rt_inline int rt_list_isempty (
             const rt_list_t * l)
```

tests whether a list is empty

Parameters

l	the list to test.

6.2.9.3.28 rt_list_len()

```
rt_inline unsigned int rt_list_len (
             const rt_list_t * l)
```

get the list length

Parameters

l	the list to get.

6.2.9.3.29 rt_slist_init()

```
rt_inline void rt_slist_init (
              rt_slist_t * l)
```

initialize a single list

Parameters

l	the single list to be initialized

6.2.10 Error Code

Collaboration diagram for Error Code: error code
The error code is defined to identify which kind of error occurs. When some bad things happen, the current thread's errno will be set.

6.2.11 System Initialization

Collaboration diagram for System Initialization:

Functions

- void rt_application_init ()
- void rt_system_heap_init (void ∗begin_addr, void ∗end_addr)
- void rt_system_timer_init (void)
- void rt_system_timer_thread_init (void)

6.2.11.1 Detailed Description

System initialization procedure.
When RT-Thread operating system starts up, the basic operating system facility initialization routines must be invoked.
The suggested initialization sequence is:

- initialize device hardware rt_hw_board_init();

User can put the low level hardware initialization in this function, such as DDR memory setting, pinmux setting, console device setting etc.

- show version rt_show_version();
- initialize timer system rt_system_timer_init();
- initialize system heap memory rt_system_heap_init(__bss_end, __end_of_memory);
- initialize module system rt_system_module_init();
- initialize scheduler system rt_system_scheduler_init();
- initialize application rt_application_init();
- initialize system timer thread rt_system_timer_thread_init();
- initialize idle thread rt_thread_idle_init();
- start scheduler rt_system_scheduler_start();

6.2.11.2 Function Documentation

6.2.11.2.1 rt_application_init()

```
void rt_application_init ()
```

This function will initialize user application.
This function will be invoked when system initialization and system scheduler has not started. User can allocate memory, create thread, semaphore etc. However, user shall not suspend 'current' thread.

6.2.11.2.2 rt_system_heap_init()

```
void rt_system_heap_init (
            void * begin_addr,
            void * end_addr)
```

This function will init system heap. User can override this API to complete other works, like heap sanitizer initialization.

Parameters

begin_addr	the beginning address of system page.
end_addr	the end address of system page.

6.2.11.2.3 rt_system_timer_init()

```
void rt_system_timer_init (
            void )
```

This function will initialize system timer.

6.2.11.2.4 rt_system_timer_thread_init()

```
void rt_system_timer_thread_init (
            void )
```

This function will initialize system timer thread.

6.3 Components

Collaboration diagram for Components:

Topics

- Device Virtual File System
- Device Driver
- Finsh Shell
- Application Module

6.3.1 Detailed Description

Components of RT-Thread, see Components for more details.

6.3.2 Device Virtual File System

Collaboration diagram for Device Virtual File System:

Topics

- File Descriptor
- File System API
- File API
- File POSIX API

Functions

- rt_err_t dfs_file_lock (void)
- void dfs_file_unlock (void)
- int dfs_init (void)
- struct dfs_file * dfs_file_create (void)
- void dfs_file_destroy (struct dfs_file *file)
- void fdt_fd_release (struct dfs_fdtable *fdt, int fd)
- int fdt_fd_associate_file (struct dfs_fdtable *fdt, int fd, struct dfs_file *file)
- int fd_new (void)
- struct dfs_file * fd_get (int fd)
- struct dfs_fdtable * dfs_fdtable_get (void)
- struct dfs_fdtable * dfs_fdtable_get_global (void)
- int dfs_fdtable_dup (struct dfs_fdtable *fdt_dst, struct dfs_fdtable *fdt_src, int fd_src)
- int dfs_fdtable_drop_fd (struct dfs_fdtable *fdt, int fd)
- int dfs_dup (int oldfd, int startfd)
- int dfs_dup_to (int oldfd, struct dfs_fdtable *fdtab)
- int dfs_dup_from (int oldfd, struct dfs_fdtable *fdtab)
- int sys_dup (int oldfd)
- rt_err_t sys_dup2 (int oldfd, int newfd)
- const char * dfs_subdir (const char *directory, const char *filename)
- char * dfs_normalize_path (const char *directory, const char *filename)
- int list_fd (void)
- int dfs_fd_dump (int argc, char **argv)

6.3.2.1 Detailed Description

DFS is a virtual file system in RT-Thread RTOS.
The DFS (Device Virtual File System) is a vfs file system of RT-Thread RTOS, which is focused on embedded device. VFS is an abstraction layer on top of a more concrete file system. The purpose of a VFS is to allow client applications to access different types of concrete file systems in a uniform way.
The DFS specifies an interface between the kernel and a concrete file system. Therefore, it is easy to add support for new file system types to the kernel simply by fulfilling the interface.

6.3.2.2 Function Documentation

6.3.2.2.1 dfs_file_lock()

```
rt_err_t dfs_file_lock (
            void )
```

Lock the file descriptor table mutex.

Returns

rt_err_t RT_EOK if lock acquired successfully, -RT_ENOSYS if filesystem not initialized, -RT_EBUSY if mutex is already locked (retries until acquired)

Note

This function will block indefinitely until the lock is acquired. Should not be called from interrupt service routines.

6.3.2.2.2 dfs_file_unlock()

```
void dfs_file_unlock (
            void )
```

Unlock the file descriptor table mutex.

Note

This function releases the lock acquired by dfs_file_lock(). Should be called in the same context as the corresponding lock. Should not be called from interrupt service routines.

6.3.2.2.3 dfs_init()

```
int dfs_init (
            void )
```

this function will initialize device file system.

6.3.2.2.4 dfs_file_create()

```
struct dfs_file * dfs_file_create (
            void )
```

Create and initialize a new file descriptor structure.

Returns

struct dfs_file* Pointer to the newly created file descriptor structure, NULL if memory allocation failed

6.3.2.2.5 dfs_file_destroy()

```
void dfs_file_destroy (
            struct dfs_file * file)
```

Destroy and free a file descriptor structure.

Parameters

in	*file*	Pointer to the file descriptor structure to be destroyed

6.3.2.2.6 fdt_fd_release()

```
void fdt_fd_release (
            struct dfs_fdtable * fdt,
            int fd)
```

Release a file descriptor from the file descriptor table.

Parameters

in,out	*fdt*	Pointer to the file descriptor table
in	*fd*	The file descriptor to be released

6.3.2.2.7 fdt_fd_associate_file()

```
int fdt_fd_associate_file (
            struct dfs_fdtable * fdt,
            int fd,
            struct dfs_file * file)
```

Associate a file structure with a file descriptor in the file descriptor table.

Parameters

in,out	*fdt*	Pointer to the file descriptor table
in	*fd*	The file descriptor to associate with
in	*file*	Pointer to the file structure to associate

Returns

int The file descriptor if successful (>= 0), -1 if association failed (invalid parameters or fd already in use), -RT_ENOSYS if filesystem lock failed

6.3.2.2.8 fd_new()

```
int fd_new (
            void )
```

Allocate a new file descriptor in current process's file descriptor table.

Returns

int The allocated file descriptor index if successful (>= 0), -1 if allocation failed

6.3.2.2.9 fd_get()

```
struct dfs_file * fd_get (
            int fd)
```

Get file structure by file descriptor from current process's fd table.

Parameters

in	*fd*	The file descriptor to lookup

Returns

struct dfs_file* Pointer to the file structure if found, NULL if invalid fd or file not found

6.3.2.2.10 dfs_fdtable_get()

```
struct dfs_fdtable * dfs_fdtable_get (
            void )
```

This function will get the file descriptor table of current process.

6.3.2.2.11 dfs_fdtable_get_global()

```
struct dfs_fdtable * dfs_fdtable_get_global (
            void )
```

Get the global file descriptor table.

Returns

struct dfs_fdtable* Pointer to the global file descriptor table

6.3.2.2.12 dfs_fdtable_dup()

```
int dfs_fdtable_dup (
            struct dfs_fdtable * fdt_dst,
```

```
            struct dfs_fdtable * fdt_src,
            int fd_src)
```

Dup the specified fd_src from fdt_src to fdt_dst.

Parameters

out	*fdt_dst*	is the fd table for destination, if empty, use global (_fdtab).
in	*fdt_src*	is the fd table for source, if empty, use global (_fdtab).
in	*fd_src*	is the fd in the designate fdt_src table.

Returns

-1 on failed or the allocated file descriptor.

6.3.2.2.13 dfs_fdtable_drop_fd()

```
int dfs_fdtable_drop_fd (
            struct dfs_fdtable * fdt,
            int fd)
```

drop fd from the fd table.

Parameters

fdt	is the fd table, if empty, use global (_fdtab).
fd	is the fd in the designate fd table.

Returns

-1 on failed the drop file descriptor.

6.3.2.2.14 dfs_dup()

```
int dfs_dup (
            int oldfd,
            int startfd)
```

Duplicate a file descriptor in the current process's file descriptor table.

Parameters

in	*oldfd*	The file descriptor to duplicate
in	*startfd*	The starting index to search for an available file descriptor slot

Returns

int The new file descriptor if successful (>=0), -1 if failed (invalid fd or allocation failed), -RT_ENOSYS if filesystem lock failed

6.3.2.2.15 dfs_dup_to()

```
int dfs_dup_to (
            int oldfd,
            struct dfs_fdtable * fdtab)
```

Duplicate a file descriptor from current process to target file descriptor table.

Parameters

in	*oldfd*	is the fd in current process.
in,out	*fdtab*	is the fd table to dup, if empty, use global (_fdtab).

Returns

-1 on failed or the allocated file descriptor.

6.3.2.2.16 dfs_dup_from()

```
int dfs_dup_from (
            int oldfd,
            struct dfs_fdtable * fdtab)
```

Duplicate a file descriptor from source table to current process.

Parameters

in	*oldfd*	is the fd in the designate fd table.
in,out	*fdtab*	is the fd table for oldfd, if empty, use global (_fdtab).

Returns

-1 on failed or the allocated file descriptor.

6.3.2.2.17 sys_dup()

```
int sys_dup (
            int oldfd)
```

System call to duplicate a file descriptor.

Parameters

in	*oldfd*	The file descriptor to duplicate

Returns

sysret_t/int The new file descriptor if successful (>=0), negative error code if failed

6.3.2.2.18 sys_dup2()

```
rt_err_t sys_dup2 (
            int oldfd,
            int newfd)
```

System call to duplicate a file descriptor to a specific descriptor number.

Parameters

in	*oldfd*	The file descriptor to duplicate
in	*newfd*	The desired file descriptor number

Returns

rt_err_t The new file descriptor number if successful (>=0), -RT_ENOSYS if filesystem lock failed, -1 if operation failed (invalid descriptors or allocation failed)

6.3.2.2.19 dfs_subdir()

```
const char * dfs_subdir (
            const char * directory,
            const char * filename)
```

Get the subdirectory path relative to a parent directory.

Parameters

in	*directory*	The parent directory path
in	*filename*	The full path including parent directory and subpath

Returns

const char* Pointer to the subdirectory portion of filename, NULL if paths are identical or invalid

6.3.2.2.20 dfs_normalize_path()

```
char * dfs_normalize_path (
            const char * directory,
            const char * filename)
```

Normalize a path by combining directory and filename into an absolute path.

Parameters

in	*directory*	The parent directory path (NULL means use working directory)
in	*filename*	The filename or relative path to be normalized

Returns

char* The normalized absolute path (must be freed by caller), NULL if path is invalid or memory allocation fails

Note

This function will:

- Handle working directory when directory is NULL
- Join directory and filename with proper separators
- Resolve . and .. in paths
- Remove redundant slashes
- Ensure path starts with /
- Allocate memory for the returned path string

6.3.2.2.21 list_fd()

```
int list_fd (
            void )
```

List all open file descriptors in the current process.

Returns

int 0 on success, -1 if failed to get file descriptor table

6.3.2.2.22 dfs_fd_dump()

```
int dfs_fd_dump (
            int argc,
            char ** argv)
```

Dump all file descriptors information in the global file descriptor table.

Parameters

in	*argc*	Number of command line arguments (unused)
in	*argv*	Array of command line arguments (unused)

Returns

int 0 on success, -RT_ENOSYS if failed to acquire file system lock

6.3.2.3 File Descriptor

Collaboration diagram for File Descriptor:

Functions

- int fdt_fd_new (struct dfs_fdtable ∗fdt)
- struct dfs_file ∗ fdt_get_file (struct dfs_fdtable ∗fdt, int fd)
- void fd_release (int fd)

6.3.2.3.1 Detailed Description

6.3.2.3.2 Function Documentation

6.3.2.3.2.1 fdt_fd_new()

```
int fdt_fd_new (
            struct dfs_fdtable * fdt)
```

Allocate a new file descriptor in the file descriptor table.

Parameters

in,out	*fdt*	Pointer to the file descriptor table to allocate from

Returns

int The allocated file descriptor index if successful (>= 0), -RT_ENOSYS if filesystem lock failed, -1 if allocation failed (no empty slots or memory allocation failed)

6.3.2.3.2.2 fdt_get_file()

```
struct dfs_file * fdt_get_file (
            struct dfs_fdtable * fdt,
            int fd)
```

This function will return a file descriptor structure according to file descriptor.

Returns

NULL on on this file descriptor or the file descriptor structure pointer.

6.3.2.3.2.3 fd_release()

```
void fd_release (
            int fd)
```

This function will put the file descriptor.

6.3.2.4 File System API

Collaboration diagram for File System API:

6.3.2.5 File API

Collaboration diagram for File API:

6.3.2.6 File POSIX API

Collaboration diagram for File POSIX API:

Functions

- int open (const char *file, int flags,...)
- int openat (int dirfd, const char *path, int flag,...)
- int creat (const char *path, mode_t mode)
- int close (int fd)
- ssize_t read (int fd, void *buf, size_t len)
- ssize_t write (int fd, const void *buf, size_t len)
- off_t lseek (int fd, off_t offset, int whence)
- int rename (const char *old_file, const char *new_file)
- int unlink (const char *pathname)
- int stat (const char *file, struct stat *buf)
- int fstat (int fildes, struct stat *buf)
- int fsync (int fildes)
- int fcntl (int fildes, int cmd,...)
- int ioctl (int fildes, int cmd,...)
- int ftruncate (int fd, off_t length)
- int statfs (const char *path, struct statfs *buf)
- int fstatfs (int fildes, struct statfs *buf)
- int mkdir (const char *path, mode_t mode)
- int rmdir (const char *pathname)
- DIR * opendir (const char *name)
- struct dirent * readdir (DIR *d)
- long telldir (DIR *d)
- void seekdir (DIR *d, long offset)
- void rewinddir (DIR *d)
- int closedir (DIR *d)
- int access (const char *path, int amode)
- void setcwd (char *buf)
- char * getcwd (char *buf, size_t size)
- ssize_t pread (int fd, void *buf, size_t len, off_t offset)
- ssize_t pwrite (int fd, const void *buf, size_t len, off_t offset)

6.3.2.6.1 Detailed Description

6.3.2.6.2 Function Documentation

6.3.2.6.2.1 open()

```
int open (
             const char * file,
             int flags,
              ...)
```

this function is a POSIX compliant version, which will open a file and return a file descriptor according specified flags.

Parameters

file	the path name of file.
flags	the file open flags. Common values include: • Access modes (mutually exclusive): – `O_RDONLY`: Open for read-only access. – `O_WRONLY`: Open for write-only access. – `O_RDWR`: Open for both reading and writing. • File status flags (can be combined with bitwise OR |): – `O_CREAT`: Create the file if it does not exist. Requires a `mode` argument. – `O_TRUNC`: Truncate the file to zero length if it already exists. – `O_APPEND`: Append writes to the end of the file. – `O_EXCL`: Ensure that `O_CREAT` creates the file exclusively. – Other platform-specific flags

Returns

the non-negative integer on successful open, others for failed.

6.3.2.6.2.2 openat()

```
int openat (
             int dirfd,
             const char * path,
             int flag,
              ...)
```

Opens a file relative to a directory file descriptor.

Parameters

dirfd	The file descriptor of the directory to base the relative path on.
path	The path to the file to be opened, relative to the directory specified by `dirfd`. Can be an absolute path (in which case `dirfd` is ignored).
flag	File access and status flags (e.g., `O_RDONLY`, `O_WRONLY`, `O_CREAT`).

Returns

On success, returns a new file descriptor for the opened file. On failure, returns `-1` and sets `errno` to indicate the error.

Note

When using relative paths, ensure `dirfd` is a valid directory descriptor. When `pathname` is absolute, the `dirfd` argument is ignored.

6.3.2.6.2.3 creat()

```
int creat (
            const char * path,
            mode_t mode)
```

this function is a POSIX compliant version, which will create a new file or rewrite an existing one

Parameters

path	the path name of file.
mode	the file permission bits to be used in creating the file (not used, can be 0)

Returns

the non-negative integer on successful open, others for failed.

6.3.2.6.2.4 close()

```
int close (
            int fd)
```

this function is a POSIX compliant version, which will close the open file descriptor.

Parameters

fd	the file descriptor.

Returns

0 on successful, -1 on failed.

6.3.2.6.2.5 read()

```
ssize_t read (
            int fd,
            void * buf,
            size_t len)
```

this function is a POSIX compliant version, which will read specified data buffer length for an open file descriptor.

Parameters

fd	the file descriptor.
buf	the buffer to save the read data.
len	the maximal length of data buffer

Returns

the actual read data buffer length. If the returned value is 0, it may be reach the end of file, please check errno.

6.3.2.6.2.6 write()

```
ssize_t write (
            int fd,
            const void * buf,
            size_t len)
```

this function is a POSIX compliant version, which will write specified data buffer length for an open file descriptor.

Parameters

fd	the file descriptor
buf	the data buffer to be written.
len	the data buffer length.

Returns

the actual written data buffer length.

6.3.2.6.2.7 lseek()

```
off_t lseek (
            int fd,
            off_t offset,
            int whence)
```

this function is a POSIX compliant version, which will Reposition the file offset for an open file descriptor.
The `lseek` function sets the file offset for the file descriptor `fd` to a new value, determined by the `offset` and `whence` parameters. It can be used to seek to specific positions in a file for reading or writing.

Parameters

fd	the file descriptor.
offset	The offset, in bytes, to set the file position. The meaning of `offset` depends on the value of `whence`.
whence	the directive of seek. It can be one of: • `SEEK_SET`: Set the offset to `offset` bytes from the beginning of the file. • `SEEK_CUR`: Set the offset to its current location plus `offset` bytes. • `SEEK_END`: Set the offset to the size of the file plus `offset` bytes.

Returns

the resulting read/write position in the file, or -1 on failed.

6.3.2.6.2.8 rename()

```
int rename (
            const char * old_file,
            const char * new_file)
```

this function is a POSIX compliant version, which will rename old file name to new file name.

Parameters

old_file	the old file name.
new_file	the new file name.

Returns

0 on successful, -1 on failed.

note: the old and new file name must be belong to a same file system.

6.3.2.6.2.9 unlink()

```
int unlink (
            const char * pathname)
```

this function is a POSIX compliant version, which will unlink (remove) a specified path file from file system.

Parameters

pathname	the specified path name to be unlinked.

Returns

0 on successful, -1 on failed.

6.3.2.6.2.10 stat()

```
int stat (
            const char * file,
            struct stat * buf)
```

this function is a POSIX compliant version, which will get file information.

Parameters

file	the file name
buf	the data buffer to save stat description.

Returns

0 on successful, -1 on failed.

6.3.2.6.2.11 fstat()

```
int fstat (
            int fildes,
            struct stat * buf)
```

this function is a POSIX compliant version, which will get file status.

Parameters

fildes	the file description
buf	the data buffer to save stat description.

Returns

0 on successful, -1 on failed.

6.3.2.6.2.12 fsync()

```
int fsync (
            int fildes)
```

this function is a POSIX compliant version, which shall request that all data for the open file descriptor named by fildes is to be transferred to the storage device associated with the file described by fildes.

Parameters

fildes	the file description

Returns

0 on successful completion. Otherwise, -1 shall be returned and errno set to indicate the error.

6.3.2.6.2.13 fcntl()

```
int fcntl (
            int fildes,
            int cmd,
             ...)
```

this function is a POSIX compliant version, which shall perform a variety of control functions on devices.

Parameters

fildes	the file description
cmd	the specified command, Common values include: • `F_DUPFD`: Duplicate a file descriptor. • `F_GETFD`: Get the file descriptor flags. • `F_SETFD`: Set the file descriptor flags. • `F_GETFL`: Get the file status flags. • `F_SETFL`: Set the file status flags.
...	represents the additional information that is needed by this specific device to perform the requested function. For example: • When `cmd` is `F_SETFL`, an additional integer argument specifies the new status flags.

Returns

0 on successful completion. Otherwise, -1 shall be returned and errno set to indicate the error.

6.3.2.6.2.14 ioctl()

```
int ioctl (
            int fildes,
            int cmd,
             ...)
```

this function is a POSIX compliant version, which shall perform a variety of control functions on devices.

Parameters

fildes	the file description
cmd	the specified command
...	represents the additional information that is needed by this specific device to perform the requested function.

Returns

0 on successful completion. Otherwise, -1 shall be returned and errno set to indicate the error.

6.3.2.6.2.15 ftruncate()

```
int ftruncate (
            int fd,
            off_t length)
```

this function is a POSIX compliant version, which cause the regular file referenced by fd to be truncated to a size of precisely length bytes.

Parameters

fd	the file descriptor.
length	the length to be truncated.

Returns

Upon successful completion, ftruncate() shall return 0; otherwise, -1 shall be returned and errno set to indicate the error.

6.3.2.6.2.16 statfs()

```
int statfs (
            const char * path,
            struct statfs * buf)
```

this function is a POSIX compliant version, which will return the information about a mounted file system.

Parameters

path	the path which mounted file system.
buf	the buffer to save the returned information.

Returns

0 on successful, others on failed.

6.3.2.6.2.17 fstatfs()

```
int fstatfs (
            int fildes,
            struct statfs * buf)
```

this function is a POSIX compliant version, which will return the information about a mounted file system.

Parameters

fildes	the file description.
buf	the buffer to save the returned information.

Returns

0 on successful, others on failed.

6.3.2.6.2.18 mkdir()

```
int mkdir (
              const char * path,
              mode_t mode)
```

this function is a POSIX compliant version, which will make a directory

Parameters

path	the directory path to be made.
mode	The permission mode for the new directory (unused here, can be set to 0).

Returns

0 on successful, others on failed.

6.3.2.6.2.19 rmdir()

```
int rmdir (
              const char * pathname)
```

this function is a POSIX compliant version, which will remove a directory.

Parameters

pathname	the path name to be removed.

Returns

0 on successful, others on failed.

6.3.2.6.2.20 opendir()

```
DIR * opendir (
              const char * name)
```

this function is a POSIX compliant version, which will open a directory.

Parameters

name	the path name to be open.

Returns

the DIR pointer of directory, NULL on open directory failed.

6.3.2.6.2.21 readdir()

```
struct dirent * readdir (
            DIR * d)
```

this function is a POSIX compliant version, which will return a pointer to a dirent structure representing the next directory entry in the directory stream.

Parameters

d	the directory stream pointer.

Returns

the next directory entry, NULL on the end of directory or failed.

6.3.2.6.2.22 telldir()

```
long telldir (
            DIR * d)
```

this function is a POSIX compliant version, which will return current location in directory stream.

Parameters

d	the directory stream pointer.

Returns

the current location in directory stream.

6.3.2.6.2.23 seekdir()

```
void seekdir (
            DIR * d,
            long offset)
```

this function is a POSIX compliant version, which will set position of next directory structure in the directory stream.

Parameters

d	the directory stream.
offset	the offset in directory stream.

6.3.2.6.2.24 rewinddir()

```
void rewinddir (
            DIR * d)
```

this function is a POSIX compliant version, which will reset directory stream.

Parameters

d	the directory stream.

6.3.2.6.2.25 closedir()

```
int closedir (
            DIR * d)
```

this function is a POSIX compliant version, which will close a directory stream.

Parameters

d	the directory stream.

Returns

0 on successful, -1 on failed.

6.3.2.6.2.26 access()

```
int access (
            const char * path,
            int amode)
```

this function is a POSIX compliant version, which shall check the file named by the pathname pointed to by the path argument for accessibility according to the bit pattern contained in amode.

Parameters

path	the specified file/dir path.
amode	the value is either the bitwise-inclusive OR of the access permissions to be checked (R_OK, W_OK, X_OK) or the existence test (F_OK).

6.3.2.6.2.27 setcwd()

```
void setcwd (
            char * buf)
```

this function is a POSIX compliant version, which will set current working directory.

Parameters

buf	the current directory.

6.3.2.6.2.28 getcwd()

```
char * getcwd (
            char * buf,
            size_t size)
```

this function is a POSIX compliant version, which will return current working directory.

Parameters

buf	the returned current directory.
size	the buffer size.

Returns

the returned current directory.

6.3.2.6.2.29 pread()

```
ssize_t pread (
            int fd,
            void * buf,
            size_t len,
            off_t offset)
```

this function is a POSIX compliant version, which will read specified data buffer length for an open file descriptor.

Parameters

fd	the file descriptor.
buf	the buffer to save the read data.
len	the maximal length of data buffer
offset	the file pos

Returns

the actual read data buffer length. If the returned value is 0, it may be reach the end of file, please check errno.

6.3.2.6.2.30 pwrite()

```
ssize_t pwrite (
            int fd,
            const void * buf,
            size_t len,
            off_t offset)
```

this function is a POSIX compliant version, which will write specified data buffer length for an open file descriptor.

Parameters

fd	the file descriptor
buf	the data buffer to be written.
len	the data buffer length.
offset	the file pos

Returns

the actual written data buffer length.

6.3.3 Device Driver

Collaboration diagram for Device Driver:

Topics

- ADC
- Clock
- DAC
- Audio
- CAN Driver
- I2C
- Pin
- PWM

- RTC
- Serial
- Serial v2
- SPI
- Touch
- blk

Data Structures

- struct rt_wqueue
- struct rt_device
- struct rt_device_notify

Macros

- #define RT_DEVICE_FLAG_DEACTIVATE 0x000
- #define RT_DEVICE_FLAG_RDONLY 0x001
- #define RT_DEVICE_FLAG_WRONLY 0x002
- #define RT_DEVICE_FLAG_RDWR 0x003
- #define RT_DEVICE_FLAG_REMOVABLE 0x004
- #define RT_DEVICE_FLAG_STANDALONE 0x008
- #define RT_DEVICE_FLAG_ACTIVATED 0x010
- #define RT_DEVICE_FLAG_SUSPENDED 0x020
- #define RT_DEVICE_FLAG_STREAM 0x040
- #define RT_DEVICE_FLAG_DYNAMIC 0x080
- #define RT_DEVICE_FLAG_INT_RX 0x100
- #define RT_DEVICE_FLAG_DMA_RX 0x200
- #define RT_DEVICE_FLAG_INT_TX 0x400
- #define RT_DEVICE_FLAG_DMA_TX 0x800
- #define RT_DEVICE_OFLAG_CLOSE 0x000
- #define RT_DEVICE_OFLAG_RDONLY 0x001
- #define RT_DEVICE_OFLAG_WRONLY 0x002
- #define RT_DEVICE_OFLAG_RDWR 0x003
- #define RT_DEVICE_OFLAG_OPEN 0x008
- #define RT_DEVICE_OFLAG_MASK 0xf0f
- #define RT_DEVICE_CTRL_RESUME 0x01
- #define RT_DEVICE_CTRL_SUSPEND 0x02
- #define RT_DEVICE_CTRL_CONFIG 0x03
- #define RT_DEVICE_CTRL_CLOSE 0x04
- #define RT_DEVICE_CTRL_NOTIFY_SET 0x05
- #define RT_DEVICE_CTRL_SET_INT 0x06
- #define RT_DEVICE_CTRL_CLR_INT 0x07
- #define RT_DEVICE_CTRL_GET_INT 0x08
- #define RT_DEVICE_CTRL_CONSOLE_OFLAG 0x09
- #define RT_DEVICE_CTRL_MASK 0x1f
- #define RT_DEVICE_CTRL_BASE(Type)

Enumerations

- enum rt_device_class_type {
  RT_Device_Class_Char = 0 , RT_Device_Class_Block , RT_Device_Class_NetIf , RT_Device_Class_MTD ,
  RT_Device_Class_CAN , RT_Device_Class_RTC , RT_Device_Class_Sound , RT_Device_Class_Graphic ,
  RT_Device_Class_I2CBUS , RT_Device_Class_USBDevice , RT_Device_Class_USBHost , RT_Device_Class_USBOTG
  ,
  RT_Device_Class_SPIBUS , RT_Device_Class_SPIDevice , RT_Device_Class_SDIO , RT_Device_Class_PM
  ,
  RT_Device_Class_Pipe , RT_Device_Class_Portal , RT_Device_Class_Timer , RT_Device_Class_Miscellaneous
  ,
  RT_Device_Class_Sensor , RT_Device_Class_Touch , RT_Device_Class_PHY , RT_Device_Class_Security
  ,
  RT_Device_Class_WLAN , RT_Device_Class_Pin , RT_Device_Class_ADC , RT_Device_Class_DAC ,
  RT_Device_Class_WDT , RT_Device_Class_PWM , RT_Device_Class_Bus , RT_Device_Class_Unknown }

6.3.3.1 Detailed Description

device I/O subsystem
The Device System is designed as simple and minimum layer to help communication between applications and drivers.
The Device System provide five interfaces to driver:

- open, open a device
- close, close a device
- read, read some data from a device
- write, write some data to a device
- control, send some control command to a device

6.3.3.2 Macro Definition Documentation

6.3.3.2.1 RT_DEVICE_FLAG_DEACTIVATE

```
#define RT_DEVICE_FLAG_DEACTIVATE 0x000
```

device flags definitions device is not not initialized

6.3.3.2.2 RT_DEVICE_FLAG_RDONLY

```
#define RT_DEVICE_FLAG_RDONLY 0x001
```

read only

6.3.3.2.3 RT_DEVICE_FLAG_WRONLY

```
#define RT_DEVICE_FLAG_WRONLY 0x002
```

write only

6.3.3.2.4 RT_DEVICE_FLAG_RDWR

```
#define RT_DEVICE_FLAG_RDWR 0x003
```

read and write

6.3.3.2.5 RT_DEVICE_FLAG_REMOVABLE

```
#define RT_DEVICE_FLAG_REMOVABLE 0x004
```

removable device

6.3.3.2.6 RT_DEVICE_FLAG_STANDALONE

```
#define RT_DEVICE_FLAG_STANDALONE 0x008
```
standalone device

6.3.3.2.7 RT_DEVICE_FLAG_ACTIVATED

```
#define RT_DEVICE_FLAG_ACTIVATED 0x010
```
device is activated

6.3.3.2.8 RT_DEVICE_FLAG_SUSPENDED

```
#define RT_DEVICE_FLAG_SUSPENDED 0x020
```
device is suspended

6.3.3.2.9 RT_DEVICE_FLAG_STREAM

```
#define RT_DEVICE_FLAG_STREAM 0x040
```
stream mode

6.3.3.2.10 RT_DEVICE_FLAG_DYNAMIC

```
#define RT_DEVICE_FLAG_DYNAMIC 0x080
```
device is determined when open()

6.3.3.2.11 RT_DEVICE_FLAG_INT_RX

```
#define RT_DEVICE_FLAG_INT_RX 0x100
```
INT mode on Rx

6.3.3.2.12 RT_DEVICE_FLAG_DMA_RX

```
#define RT_DEVICE_FLAG_DMA_RX 0x200
```
DMA mode on Rx

6.3.3.2.13 RT_DEVICE_FLAG_INT_TX

```
#define RT_DEVICE_FLAG_INT_TX 0x400
```
INT mode on Tx

6.3.3.2.14 RT_DEVICE_FLAG_DMA_TX

```
#define RT_DEVICE_FLAG_DMA_TX 0x800
```
DMA mode on Tx

6.3.3.2.15 RT_DEVICE_OFLAG_CLOSE

```
#define RT_DEVICE_OFLAG_CLOSE 0x000
```
device is closed

6.3.3.2.16 RT_DEVICE_OFLAG_RDONLY

```
#define RT_DEVICE_OFLAG_RDONLY 0x001
```
read only access

6.3.3.2.17 RT_DEVICE_OFLAG_WRONLY

```
#define RT_DEVICE_OFLAG_WRONLY 0x002
```
write only access

6.3.3.2.18 RT_DEVICE_OFLAG_RDWR

```
#define RT_DEVICE_OFLAG_RDWR 0x003
```
read and write

6.3.3.2.19 RT_DEVICE_OFLAG_OPEN

```
#define RT_DEVICE_OFLAG_OPEN 0x008
```
device is opened

6.3.3.2.20 RT_DEVICE_OFLAG_MASK

```
#define RT_DEVICE_OFLAG_MASK 0xf0f
```
mask of open flag

6.3.3.2.21 RT_DEVICE_CTRL_RESUME

```
#define RT_DEVICE_CTRL_RESUME 0x01
```
general device commands 0x01 - 0x1F general device control commands 0x20 - 0x3F udevice control commands 0x40 - special device control commands resume device

6.3.3.2.22 RT_DEVICE_CTRL_SUSPEND

```
#define RT_DEVICE_CTRL_SUSPEND 0x02
```
suspend device

6.3.3.2.23 RT_DEVICE_CTRL_CONFIG

```
#define RT_DEVICE_CTRL_CONFIG 0x03
```
configure device

6.3.3.2.24 RT_DEVICE_CTRL_CLOSE

```
#define RT_DEVICE_CTRL_CLOSE 0x04
```
close device

6.3.3.2.25 RT_DEVICE_CTRL_NOTIFY_SET

```
#define RT_DEVICE_CTRL_NOTIFY_SET 0x05
```
set notify func

6.3.3.2.26 RT_DEVICE_CTRL_SET_INT

```
#define RT_DEVICE_CTRL_SET_INT 0x06
```
set interrupt

6.3.3.2.27 RT_DEVICE_CTRL_CLR_INT

```
#define RT_DEVICE_CTRL_CLR_INT 0x07
```
clear interrupt

6.3.3.2.28 RT_DEVICE_CTRL_GET_INT

```
#define RT_DEVICE_CTRL_GET_INT 0x08
```
get interrupt status

6.3.3.2.29 RT_DEVICE_CTRL_CONSOLE_OFLAG

```
#define RT_DEVICE_CTRL_CONSOLE_OFLAG 0x09
```
get console open flag

6.3.3.2.30 RT_DEVICE_CTRL_MASK

```
#define RT_DEVICE_CTRL_MASK 0x1f
```

mask for contrl commands

6.3.3.2.31 RT_DEVICE_CTRL_BASE

```
#define RT_DEVICE_CTRL_BASE(
              Type)
```

Value:

```
((RT_Device_Class_##Type + 1) * 0x100)
```

device control

6.3.3.3 Enumeration Type Documentation

6.3.3.3.1 rt_device_class_type

```
enum rt_device_class_type
```

device (I/O) class type

Enumerator

RT_Device_Class_Char	character device
RT_Device_Class_Block	block device
RT_Device_Class_NetIf	net interface
RT_Device_Class_MTD	memory device
RT_Device_Class_CAN	CAN device
RT_Device_Class_RTC	RTC device
RT_Device_Class_Sound	Sound device
RT_Device_Class_Graphic	Graphic device
RT_Device_Class_I2CBUS	I2C bus device
RT_Device_Class_USBDevice	USB slave device
RT_Device_Class_USBHost	USB host bus
RT_Device_Class_USBOTG	USB OTG bus
RT_Device_Class_SPIBUS	SPI bus device
RT_Device_Class_SPIDevice	SPI device
RT_Device_Class_SDIO	SDIO bus device
RT_Device_Class_PM	PM pseudo device
RT_Device_Class_Pipe	Pipe device
RT_Device_Class_Portal	Portal device
RT_Device_Class_Timer	Timer device
RT_Device_Class_Miscellaneous	Miscellaneous device
RT_Device_Class_Sensor	Sensor device
RT_Device_Class_Touch	Touch device
RT_Device_Class_PHY	PHY device
RT_Device_Class_Security	Security device
RT_Device_Class_WLAN	WLAN device
RT_Device_Class_Pin	Pin device

RT_Device_Class_ADC	ADC device
RT_Device_Class_DAC	DAC device
RT_Device_Class_WDT	WDT device
RT_Device_Class_PWM	PWM device
RT_Device_Class_Bus	Bus device
RT_Device_Class_Unknown	unknown device

6.3.3.4 ADC

Collaboration diagram for ADC:



Data Structures

- struct rt_adc_ops
- struct rt_adc_device

Functions

- rt_err_t rt_hw_adc_register (rt_adc_device_t adc, const char *name, const struct rt_adc_ops *ops, const void *user_data)
- rt_uint32_t rt_adc_read (rt_adc_device_t dev, rt_int8_t channel)
- rt_err_t rt_adc_enable (rt_adc_device_t dev, rt_int8_t channel)
- rt_err_t rt_adc_disable (rt_adc_device_t dev, rt_int8_t channel)
- rt_int16_t rt_adc_voltage (rt_adc_device_t dev, rt_int8_t channel)

6.3.3.4.1 Detailed Description

ADC driver api.

Example

```
#define ADC_DEV_NAME        "adc1"
#define ADC_DEV_CHANNEL     5
#define REFER_VOLTAGE       330
#define CONVERT_BITS        (1 << 12)

static int adc_vol_sample(int argc, char *argv[])
{
    rt_adc_device_t adc_dev;
    rt_uint32_t value, vol;

    rt_err_t ret = RT_EOK;

    adc_dev = (rt_adc_device_t)rt_device_find(ADC_DEV_NAME);
    if (adc_dev == RT_NULL)
    {
        rt_kprintf("adc sample run failed! can't find %s device!\n", ADC_DEV_NAME);
        return -RT_ERROR;
    }

    ret = rt_adc_enable(adc_dev, ADC_DEV_CHANNEL);

    value = rt_adc_read(adc_dev, ADC_DEV_CHANNEL);
    rt_kprintf("the value is :%d \n", value);

    vol = value * REFER_VOLTAGE / CONVERT_BITS;
    rt_kprintf("the voltage is :%d.%02d \n", vol / 100, vol % 100);

    ret = rt_adc_disable(adc_dev, ADC_DEV_CHANNEL);

    return ret;
}
MSH_CMD_EXPORT(adc_vol_sample, adc voltage convert sample);
```

6.3.3.4.2 Function Documentation

6.3.3.4.2.1 rt_hw_adc_register()

```
rt_err_t rt_hw_adc_register (
            rt_adc_device_t adc,
```

```
            const char * name,
            const struct rt_adc_ops * ops,
            const void * user_data)
```

register the adc device

Parameters

adc	adc device
name	device name
ops	device ops
user_data	device private data

Returns

rt_err_t error code

6.3.3.4.2.2 rt_adc_read()

```
rt_uint32_t rt_adc_read (
            rt_adc_device_t dev,
            rt_int8_t channel)
```

read the adc value

Parameters

dev	adc device
channel	adc channel

Returns

rt_uint32_t adc value

6.3.3.4.2.3 rt_adc_enable()

```
rt_err_t rt_adc_enable (
            rt_adc_device_t dev,
            rt_int8_t channel)
```

enable the adc channel

Parameters

dev	adc device
channel	adc channel

Returns

rt_err_t error code

6.3.3.4.2.4 rt_adc_disable()

```
rt_err_t rt_adc_disable (
            rt_adc_device_t dev,
            rt_int8_t channel)
```

disable the adc channel

Parameters

dev	adc device
channel	adc channel

Returns

rt_err_t error code

6.3.3.4.2.5 rt_adc_voltage()

```
rt_int16_t rt_adc_voltage (
            rt_adc_device_t dev,
            rt_int8_t channel)
```

get the converted voltage value from an ADC channel

Parameters

dev	adc device
channel	adc channel

Returns

rt_int16_t voltage value (implementation-defined unit, typically mV)

6.3.3.5 Clock

Collaboration diagram for Clock:

Data Structures

- struct rt_clk_node
- struct rt_clk_cell
- struct rt_clk_notifier
- struct rt_clk_array
- struct rt_clk
- struct rt_clk_fixed_rate

Macros

- #define RT_CLK_F_SET_RATE_GATE RT_BIT(0)
- #define RT_CLK_F_SET_PARENT_GATE RT_BIT(1)
- #define RT_CLK_F_SET_RATE_PARENT RT_BIT(2)
- #define RT_CLK_F_IGNORE_UNUSED RT_BIT(3)
- #define RT_CLK_F_SET_RATE_UNGATE RT_BIT(4)
- #define RT_CLK_F_IS_CRITICAL RT_BIT(5)
- #define RT_CLK_F_GET_RATE_NOCACHE RT_BIT(6)
- #define RT_CLK_F_SET_RATE_NO_REPARENT RT_BIT(7)

Functions

- rt_err_t rt_clk_register (struct rt_clk_node *clk_np)
- rt_err_t rt_clk_unregister (struct rt_clk_node *clk_np)
- rt_err_t rt_clk_notifier_register (struct rt_clk *clk, struct rt_clk_notifier *notifier)
- rt_err_t rt_clk_notifier_unregister (struct rt_clk *clk, struct rt_clk_notifier *notifier)
- rt_err_t rt_clk_prepare (struct rt_clk *clk)
- void rt_clk_unprepare (struct rt_clk *clk)
- rt_err_t rt_clk_enable (struct rt_clk *clk)
- void rt_clk_disable (struct rt_clk *clk)
- rt_err_t rt_clk_prepare_enable (struct rt_clk *clk)
- void rt_clk_disable_unprepare (struct rt_clk *clk)
- rt_err_t rt_clk_array_prepare (struct rt_clk_array *clk_arr)
- void rt_clk_array_unprepare (struct rt_clk_array *clk_arr)
- rt_err_t rt_clk_array_enable (struct rt_clk_array *clk_arr)
- void rt_clk_array_disable (struct rt_clk_array *clk_arr)
- rt_err_t rt_clk_array_prepare_enable (struct rt_clk_array *clk_arr)
- void rt_clk_array_disable_unprepare (struct rt_clk_array *clk_arr)
- rt_err_t rt_clk_set_rate_range (struct rt_clk *clk, rt_ubase_t min, rt_ubase_t max)
- rt_err_t rt_clk_set_min_rate (struct rt_clk *clk, rt_ubase_t rate)
- rt_err_t rt_clk_set_max_rate (struct rt_clk *clk, rt_ubase_t rate)
- rt_err_t rt_clk_set_rate (struct rt_clk *clk, rt_ubase_t rate)
- rt_ubase_t rt_clk_get_rate (struct rt_clk *clk)
- rt_base_t rt_clk_round_rate (struct rt_clk *clk, rt_ubase_t rate)
- rt_err_t rt_clk_set_parent (struct rt_clk *clk, struct rt_clk *clk_parent)
- struct rt_clk * rt_clk_get_parent (struct rt_clk *clk)
- rt_err_t rt_clk_set_phase (struct rt_clk *clk, int degrees)
- rt_base_t rt_clk_get_phase (struct rt_clk *clk)
- struct rt_clk * rt_clk_cell_get_clk (const struct rt_clk_cell *cell, const char *con_id)
- rt_bool_t rt_clk_cell_is_prepared (const struct rt_clk_cell *cell)
- rt_bool_t rt_clk_cell_is_enabled (const struct rt_clk_cell *cell)
- rt_ubase_t rt_clk_cell_get_rate (const struct rt_clk_cell *cell)
- rt_ubase_t rt_clk_cell_round_rate (struct rt_clk_cell *cell, rt_ubase_t rate)
- struct rt_clk_cell * rt_clk_cell_get_parent (const struct rt_clk_cell *cell)
- struct rt_clk_cell * rt_clk_cell_get_parent_by_index (const struct rt_clk_cell *cell, rt_uint8_t idx)
- rt_uint8_t rt_clk_cell_get_parent_index (struct rt_clk_cell *cell)
- rt_err_t rt_clk_cell_set_parent (struct rt_clk_cell *cell, struct rt_clk_cell *parent)
- struct rt_clk_array * rt_clk_get_array (struct rt_device *dev)
- struct rt_clk * rt_clk_get_by_index (struct rt_device *dev, int index)
- struct rt_clk * rt_clk_get_by_name (struct rt_device *dev, const char *name)
- void rt_clk_array_put (struct rt_clk_array *clk_arr)
- void rt_clk_put (struct rt_clk *clk)

6.3.3.5.1 Detailed Description

Clock driver API.

6.3.3.5.2 Macro Definition Documentation

6.3.3.5.2.1 RT_CLK_F_SET_RATE_GATE

```
#define RT_CLK_F_SET_RATE_GATE RT_BIT(0)
```

Must be gated across rate change

6.3.3.5.2.2 RT_CLK_F_SET_PARENT_GATE

```
#define RT_CLK_F_SET_PARENT_GATE RT_BIT(1)
```

Must be gated across re-parent

6.3.3.5.2.3 RT_CLK_F_SET_RATE_PARENT

```
#define RT_CLK_F_SET_RATE_PARENT RT_BIT(2)
```

Propagate rate change up one level

6.3.3.5.2.4 RT_CLK_F_IGNORE_UNUSED

```
#define RT_CLK_F_IGNORE_UNUSED RT_BIT(3)
```

Do not gate even if unused

6.3.3.5.2.5 RT_CLK_F_SET_RATE_UNGATE

```
#define RT_CLK_F_SET_RATE_UNGATE RT_BIT(4)
```

Clock needs to run to set rate

6.3.3.5.2.6 RT_CLK_F_IS_CRITICAL

```
#define RT_CLK_F_IS_CRITICAL RT_BIT(5)
```

Do not gate, ever

6.3.3.5.2.7 RT_CLK_F_GET_RATE_NOCACHE

```
#define RT_CLK_F_GET_RATE_NOCACHE RT_BIT(6)
```

Do not get rate by cache

6.3.3.5.2.8 RT_CLK_F_SET_RATE_NO_REPARENT

```
#define RT_CLK_F_SET_RATE_NO_REPARENT RT_BIT(7)
```

Keep the selected parent across rate changes

6.3.3.5.3 Function Documentation

6.3.3.5.3.1 rt_clk_register()

```
rt_err_t rt_clk_register (
            struct rt_clk_node * clk_np)
```

Register a clock node into the global clock framework.

This function initializes an rt_clk_node structure and inserts it into the global clock node list. Each node may contain multiple rt_clk_cell instances, which represent the individual output clocks provided by the node.

If the node is associated with a device (clk_np->dev is not NULL), the framework will automatically try to obtain its parent clocks by calling rt_clk_get_array(clk_np->dev). Otherwise, the node will be treated as a root-level provider (e.g., fixed clock).

The caller must ensure that all rt_clk_cell entries are allocated and linked to the node before calling this function.

Parameters

clk_np	Pointer to the clock node to be registered.

Return values

RT_EOK	Successfully registered.
-RT_EINVAL	Invalid argument or missing cell list.
-RT_ENOMEM	Failed to allocate parent clock array.

6.3.3.5.3.2 rt_clk_unregister()

```
rt_err_t rt_clk_unregister (
            struct rt_clk_node * clk_np)
```

Unregister a clock node from global clock list.
This API is intended for rollback use only, typically called when a clock provider fails after registration but before any consumer starts using its clocks.
It removes the specified clock node from the global list and releases its parent clock array if present. The function does not free memory of rt_clk_cell instances or the node itself.

Parameters

clk_np	Pointer to the clock node to be unregistered.

Return values

RT_EOK	Successfully unregistered.
-RT_EINVAL	Invalid parameter.
-RT_EBUSY	One or more cells are active and cannot be removed.

6.3.3.5.3.3 rt_clk_notifier_register()

```
rt_err_t rt_clk_notifier_register (
            struct rt_clk * clk,
            struct rt_clk_notifier * notifier)
```

Register clock notifier into notifier list.

Parameters

clk	point to clock
notifier	point to notifier for register

Return values

RT_EOK	
-RT_EINVAL	

6.3.3.5.3.4 rt_clk_notifier_unregister()

```
rt_err_t rt_clk_notifier_unregister (
            struct rt_clk * clk,
            struct rt_clk_notifier * notifier)
```

Unregister clock notifier into notifier list.

Parameters

clk	point to clock
notifier	point to notifier for unregister

Return values

RT_EOK	
-RT_EINVAL	

6.3.3.5.3.5 rt_clk_prepare()

```
rt_err_t rt_clk_prepare (
            struct rt_clk * clk)
```

Prepare clock.

Parameters

clk	

Returns

rt_err_t RT_EOK on prepare clock sucessfully, and other value is failed.

6.3.3.5.3.6 rt_clk_unprepare()

```
void rt_clk_unprepare (
            struct rt_clk * clk)
```

Unprepare clock.

Parameters

clk	Ponit to clock that will be unprepared

6.3.3.5.3.7 rt_clk_enable()

```
rt_err_t rt_clk_enable (
            struct rt_clk * clk)
```

Enable clock.

Parameters

clk	point to clock

Returns

rt_err_t RT_EOK on enable clock sucessfully, and other value is failed.

6.3.3.5.3.8 rt_clk_disable()

```
void rt_clk_disable (
            struct rt_clk * clk)
```

Disable clock.

Parameters

clk	point to clock

6.3.3.5.3.9 rt_clk_prepare_enable()

```
rt_err_t rt_clk_prepare_enable (
            struct rt_clk * clk)
```

Prepare and enable clock.

Parameters

clk	point to clock

Returns

rt_err_t RT_EOK on prepare and enable clock sucessfully, and other value is failed.

6.3.3.5.3.10 rt_clk_disable_unprepare()

```
void rt_clk_disable_unprepare (
            struct rt_clk * clk)
```

Disable and unprepare clock.

Parameters

clk	point to clock

6.3.3.5.3.11 rt_clk_array_prepare()

```
rt_err_t rt_clk_array_prepare (
            struct rt_clk_array * clk_arr)
```

Prepare clock array for mutipule out clock.

Parameters

clk_arr	point to clock array

Returns

rt_err_t RT_EOK on prepare clock array sucessfully, and other value is failed.

6.3.3.5.3.12 rt_clk_array_unprepare()

```
void rt_clk_array_unprepare (
            struct rt_clk_array * clk_arr)
```

Unprepare clock array for mutipule out clock.

Parameters

clk_arr	point to clock array

6.3.3.5.3.13 rt_clk_array_enable()

```
rt_err_t rt_clk_array_enable (
            struct rt_clk_array * clk_arr)
```

Enable clock array for mutipule out clock.

Parameters

clk_arr	point to clock array

Returns

rt_err_t RT_EOK on Enable clock array sucessfully, and other value is failed.

6.3.3.5.3.14 rt_clk_array_disable()

```
void rt_clk_array_disable (
            struct rt_clk_array * clk_arr)
```

Enable clock array for mutipule out clock.

Parameters

clk_arr	point to clock array

6.3.3.5.3.15 rt_clk_array_prepare_enable()

```
rt_err_t rt_clk_array_prepare_enable (
            struct rt_clk_array * clk_arr)
```

Prepare and enable clock array.

Parameters

clk_arr	point to clock array

Returns

rt_err_t RT_EOK on prepare and enable clock array sucessfully, and other value is failed.

6.3.3.5.3.16 rt_clk_array_disable_unprepare()

```
void rt_clk_array_disable_unprepare (
            struct rt_clk_array * clk_arr)
```

Disable and unprepare clock array.

Parameters

clk_arr	point to clock array

6.3.3.5.3.17 rt_clk_set_rate_range()

```
rt_err_t rt_clk_set_rate_range (
            struct rt_clk * clk,
            rt_ubase_t min,
            rt_ubase_t max)
```

Set clock rate range.

Parameters

clk	point to clock

min	minimum clock rate
max	minimum clock rate

Returns

rt_err_t RT_EOK on set clock rate range sucessfully, and other value is failed.

6.3.3.5.3.18 rt_clk_set_min_rate()

```
rt_err_t rt_clk_set_min_rate (
            struct rt_clk * clk,
            rt_ubase_t rate)
```

Set minimum clock rate.

Parameters

clk	point to clock
rate	miminum clock rate

Returns

rt_err_t RT_EOK on set minimum clock rate sucessfully, and other value is failed.

6.3.3.5.3.19 rt_clk_set_max_rate()

```
rt_err_t rt_clk_set_max_rate (
            struct rt_clk * clk,
            rt_ubase_t rate)
```

Set maximum clock rate.

Parameters

clk	point to clock
rate	maximum clock rate

Returns

rt_err_t RT_EOK on set maximum clock rate sucessfully, and other value is failed.

6.3.3.5.3.20 rt_clk_set_rate()

```
rt_err_t rt_clk_set_rate (
            struct rt_clk * clk,
            rt_ubase_t rate)
```

Set clock rate.

Parameters

clk	point to clock
rate	target rate

Returns

rt_err_t RT_EOK on set clock rate sucessfully, and other value is failed.

6.3.3.5.3.21 rt_clk_get_rate()

```
rt_ubase_t rt_clk_get_rate (
            struct rt_clk * clk)
```

Get clock rate.

Parameters

clk	point to clock

Returns

rt_ubase_t clock rate or error code

6.3.3.5.3.22 rt_clk_round_rate()

```
rt_base_t rt_clk_round_rate (
            struct rt_clk * clk,
            rt_ubase_t rate)
```

Check if clock rate is in the minimum to maximun and get it.

Parameters

clk	point to clock
rate	rate will be checked

Returns

rt_base_t get the correct rate

Note

if parameter rate less than the minimum or more than maximum, the retrun rate will be set to minimum ormaximum value

6.3.3.5.3.23 rt_clk_set_parent()

```
rt_err_t rt_clk_set_parent (
            struct rt_clk * clk,
            struct rt_clk * clk_parent)
```

Set clock parent object.

Parameters

clk	point to clock
clk_parent	point to parent clock

Returns

rt_err_t RT_EOK on set clock parent sucessfully, and other value is failed.

6.3.3.5.3.24 rt_clk_get_parent()

```
struct rt_clk * rt_clk_get_parent (
            struct rt_clk * clk)
```

Get parent clock pointer.

Parameters

clk	child clock

Returns

struct rt_clk* parent clock object pointer will be return, unless child clock node havn't parent node instead return RT_NULL

6.3.3.5.3.25 rt_clk_set_phase()

```
rt_err_t rt_clk_set_phase (
            struct rt_clk * clk,
            int degrees)
```

Set clock phase.

Parameters

clk	point to clock
degrees	target phase and the unit of phase is degree

Returns

rt_err_t RT_EOK on set clock phase sucessfully, and other value is failed.

6.3.3.5.3.26 rt_clk_get_phase()

```
rt_base_t rt_clk_get_phase (
            struct rt_clk * clk)
```

Get clock phase.

Parameters

clk	point to clock

Returns

rt_base_t clock phase or error code

6.3.3.5.3.27 rt_clk_cell_get_clk()

```
struct rt_clk * rt_clk_cell_get_clk (
            const struct rt_clk_cell * cell,
            const char * con_id)
```

Get or create clock handle for a clock cell.

Parameters

cell	Pointer to clock cell
con↩ _id	Connection identifier for the clock cell

Returns

Pointer to clock handle, or RT_NULL on failure

Note

If the clock handle does not exist, it will be created automatically.

6.3.3.5.3.28 rt_clk_cell_is_prepared()

```
rt_bool_t rt_clk_cell_is_prepared (
            const struct rt_clk_cell * cell)
```

Check if the clock cell is prepared.

Parameters

cell	Pointer to clock cell

Returns

RT_TRUE if prepared, otherwise RT_FALSE

6.3.3.5.3.29 rt_clk_cell_is_enabled()

```
rt_bool_t rt_clk_cell_is_enabled (
            const struct rt_clk_cell * cell)
```

Check if the clock cell is enabled.

Parameters

cell	Pointer to clock cell

Returns

RT_TRUE if enabled, otherwise RT_FALSE

6.3.3.5.3.30 rt_clk_cell_get_rate()

```
rt_ubase_t rt_clk_cell_get_rate (
            const struct rt_clk_cell * cell)
```

Get current rate of the clock cell.

Parameters

cell	Pointer to clock cell

Returns

Current rate in Hz

6.3.3.5.3.31 rt_clk_cell_round_rate()

```
rt_ubase_t rt_clk_cell_round_rate (
            struct rt_clk_cell * cell,
            rt_ubase_t rate)
```

Round a desired rate to the nearest supported rate.

Parameters

cell	Pointer to clock cell
rate	Desired frequency in Hz

Returns

Closest supported frequency in Hz

6.3.3.5.3.32 rt_clk_cell_get_parent()

```
struct rt_clk_cell * rt_clk_cell_get_parent (
            const struct rt_clk_cell * cell)
```

Get parent clock cell.

Parameters

cell	Pointer to clock cell

Returns

Pointer to parent clock cell, or RT_NULL if none

6.3.3.5.3.33 rt_clk_cell_get_parent_by_index()

```
struct rt_clk_cell * rt_clk_cell_get_parent_by_index (
            const struct rt_clk_cell * cell,
            rt_uint8_t idx)
```

Get parent clock cell by index.

Parameters

cell	Pointer to clock cell
idx	Parent index

Returns

Pointer to parent clock cell, or RT_NULL if not found

6.3.3.5.3.34 rt_clk_cell_get_parent_index()

```
rt_uint8_t rt_clk_cell_get_parent_index (
            struct rt_clk_cell * cell)
```

Get current parent index.

Parameters

cell	Pointer to clock cell

Returns

Parent index on success, negative error code on failure

6.3.3.5.3.35 rt_clk_cell_set_parent()

```
rt_err_t rt_clk_cell_set_parent (
            struct rt_clk_cell * cell,
            struct rt_clk_cell * parent)
```

Set new parent clock cell.

Parameters

cell	Pointer to clock cell
parent	Pointer to new parent clock cell

Returns

RT_EOK on success, or error code on failure

6.3.3.5.3.36 rt_clk_get_array()

```
struct rt_clk_array * rt_clk_get_array (
            struct rt_device * dev)
```

Get clock array pointer from ofw device node.

Parameters

dev	point to dev

Returns

struct rt_clk_array* if use ofw and under normal circumstance, it will return clock array pointer and other value is RT_NULL

6.3.3.5.3.37 rt_clk_get_by_index()

```
struct rt_clk * rt_clk_get_by_index (
            struct rt_device * dev,
            int index)
```

Get clock pointer from ofw device node by index.

Parameters

dev	point to dev
index	index of clock object

Returns

struct rt_clk* if use ofw and under normal circumstance, it will return clock pointer and other value is RT_NULL

6.3.3.5.3.38 rt_clk_get_by_name()

```
struct rt_clk * rt_clk_get_by_name (
            struct rt_device * dev,
            const char * name)
```

Get clock pointer from ofw device node by name.

Parameters

dev	point to dev
name	name of clock object

Returns

struct rt_clk* if use ofw and under normal circumstance, it will return clock pointer and other value is RT_NULL

6.3.3.5.3.39 rt_clk_array_put()

```
void rt_clk_array_put (
            struct rt_clk_array * clk_arr)
```

Put reference count of all colock in the clock array.

Parameters

clk_arr	point to clock array

6.3.3.5.3.40 rt_clk_put()

```
void rt_clk_put (
            struct rt_clk * clk)
```

Put reference count of clock.

Parameters

clk	point to clock

6.3.3.6 DAC

Collaboration diagram for DAC:

Data Structures

- struct rt_dac_ops
- struct rt_dac_device

Functions

- rt_err_t rt_hw_dac_register (rt_dac_device_t dac, const char *name, const struct rt_dac_ops *ops, const void *user_data)
- rt_err_t rt_dac_write (rt_dac_device_t dev, rt_uint32_t channel, rt_uint32_t value)
- rt_err_t rt_dac_enable (rt_dac_device_t dev, rt_uint32_t channel)
- rt_err_t rt_dac_disable (rt_dac_device_t dev, rt_uint32_t channel)

6.3.3.6.1 Detailed Description

DAC driver api.

Example

```
#include <rtthread.h>
#include <rtdevice.h>
#include <stdlib.h>
#define DAC_DEV_NAME        "dac1"
#define DAC_DEV_CHANNEL     1
#define REFER_VOLTAGE       330
#define CONVERT_BITS        (1 << 12)

static int dac_vol_sample(int argc, char *argv[])
{
    rt_dac_device_t dac_dev;
    rt_uint32_t value, vol;
    rt_err_t ret = RT_EOK;

    dac_dev = (rt_dac_device_t)rt_device_find(DAC_DEV_NAME);
    if (dac_dev == RT_NULL)
    {
        rt_kprintf("dac sample run failed! can't find %s device!\n", DAC_DEV_NAME);
        return -RT_ERROR;
    }

    ret = rt_dac_enable(dac_dev, DAC_DEV_CHANNEL);

    value = atoi(argv[1]);
    rt_dac_write(dac_dev, DAC_DEV_CHANNEL, value);
    rt_kprintf("the value is :%d \n", value);

    vol = value * REFER_VOLTAGE / CONVERT_BITS;
    rt_kprintf("the voltage is :%d.%02d \n", vol / 100, vol % 100);

    rt_thread_mdelay(500);

    ret = rt_dac_disable(dac_dev, DAC_DEV_CHANNEL);

    return ret;
}
MSH_CMD_EXPORT(dac_vol_sample, dac voltage convert sample);
```

6.3.3.6.2 Function Documentation

6.3.3.6.2.1 rt_hw_dac_register()

```
rt_err_t rt_hw_dac_register (
              rt_dac_device_t dac,
              const char * name,
              const struct rt_dac_ops * ops,
              const void * user_data)
```

Register a DAC device.

Parameters

dac	DAC device
name	DAC name
ops	the operations of DAC device
user_data	device private data

Returns

rt_err_t error code

6.3.3.6.2.2 rt_dac_write()

```
rt_err_t rt_dac_write (
              rt_dac_device_t dev,
```

```
            rt_uint32_t channel,
            rt_uint32_t value)
```

set the value of DAC

Parameters

dev	DAC device
channel	DAC channel
value	the value of DAC

Returns

rt_err_t error code

6.3.3.6.2.3 rt_dac_enable()

```
rt_err_t rt_dac_enable (
            rt_dac_device_t dev,
            rt_uint32_t channel)
```

enable the DAC channel

Parameters

dev	DAC device
channel	DAC channel

Returns

rt_err_t error code

6.3.3.6.2.4 rt_dac_disable()

```
rt_err_t rt_dac_disable (
            rt_dac_device_t dev,
            rt_uint32_t channel)
```

disable the DAC channel

Parameters

dev	DAC device
channel	DAC channel

Returns

rt_err_t error code

6.3.3.7 Audio

Collaboration diagram for Audio:

Topics

- AUDIO_CTL
- AUDIO_TYPE

- AUDIO_SAMP_RATES
- AUDIO_BIT_RATES
- AUDIO_DSP
- AUDIO_MIXER
- CODEC_CMD

Data Structures

- struct rt_audio_buf_info
- struct rt_audio_ops
- struct rt_audio_configure
- struct rt_audio_caps
- struct rt_audio_device
- struct rt_audio_record
- struct rt_audio_replay

Enumerations

- enum audio_stream

Functions

- rt_err_t rt_audio_register (struct rt_audio_device *audio, const char *name, rt_uint32_t flag, void *data)
- void rt_audio_tx_complete (struct rt_audio_device *audio)
- void rt_audio_rx_done (struct rt_audio_device *audio, rt_uint8_t *pbuf, rt_size_t len)
- int rt_audio_samplerate_to_speed (rt_uint32_t bitValue)

6.3.3.7.1 Detailed Description

Audio driver API.

6.3.3.7.2 Enumeration Type Documentation

6.3.3.7.2.1 audio_stream

```
enum audio_stream
```

Audio stream control command.

6.3.3.7.3 Function Documentation

6.3.3.7.3.1 rt_audio_register()

```
rt_err_t rt_audio_register (
            struct rt_audio_device * audio,
            const char * name,
            rt_uint32_t flag,
            void * data)
```

Register and initialize audio device.

Parameters

in	*audio*	pointer to audio deive
in	*name*	device name
in	*flag*	device flags
in	*data*	user data

Returns

error code, RT_EOK is successful otherwise means failure

6.3.3.7.3.2 rt_audio_tx_complete()

```
void rt_audio_tx_complete (
            struct rt_audio_device * audio)
```

Send a replay frame to the audio hardware device.
See _audio_send_replay_frame for details

Parameters

in	*audio*	pointer to audio device

6.3.3.7.3.3 rt_audio_rx_done()

```
void rt_audio_rx_done (
            struct rt_audio_device * audio,
            rt_uint8_t * pbuf,
            rt_size_t len)
```

Receive recording from audio device.

Parameters

in	*audio*	pointer to audio device
in	*pbuf*	pointer ro data to be received
in	*len*	buffer size

6.3.3.7.3.4 rt_audio_samplerate_to_speed()

```
int rt_audio_samplerate_to_speed (
            rt_uint32_t bitValue)
```

Set audio sample rate.

Parameters

in	*bitValue*	audio sample rate, it can be one of value in AUDIO_SAMP_RATES

Returns

speed has been set

6.3.3.7.4 AUDIO_CTL

Collaboration diagram for AUDIO_CTL:

Macros

- #define _AUDIO_CTL(a)
- #define AUDIO_CTL_GETCAPS _AUDIO_CTL(1)
- #define AUDIO_CTL_CONFIGURE _AUDIO_CTL(2)
- #define AUDIO_CTL_START _AUDIO_CTL(3)
- #define AUDIO_CTL_STOP _AUDIO_CTL(4)
- #define AUDIO_CTL_GETBUFFERINFO _AUDIO_CTL(5)

6.3.3.7.4.1 Detailed Description

Control audio device.

6.3.3.7.4.2 Macro Definition Documentation

_AUDIO_CTL

```
#define _AUDIO_CTL(
                a)
```

Value:

```
(RT_DEVICE_CTRL_BASE(Sound) + a)
```

Generate audio command code with *a*.

Parameters

in	*a*	offset of command.

Returns

audio device control command code.

AUDIO_CTL_GETCAPS

```
#define AUDIO_CTL_GETCAPS _AUDIO_CTL(1)
```

Get audio device capabilities

AUDIO_CTL_CONFIGURE

```
#define AUDIO_CTL_CONFIGURE _AUDIO_CTL(2)
```

Get audio device configuration

AUDIO_CTL_START

```
#define AUDIO_CTL_START _AUDIO_CTL(3)
```

Start audio device

AUDIO_CTL_STOP

```
#define AUDIO_CTL_STOP _AUDIO_CTL(4)
```

Stop audio device

AUDIO_CTL_GETBUFFERINFO

```
#define AUDIO_CTL_GETBUFFERINFO _AUDIO_CTL(5)
```

Get audio device buffer information

6.3.3.7.5 AUDIO_TYPE

Collaboration diagram for AUDIO_TYPE:

Macros

- #define AUDIO_TYPE_QUERY 0x00
- #define AUDIO_TYPE_INPUT 0x01
- #define AUDIO_TYPE_OUTPUT 0x02
- #define AUDIO_TYPE_MIXER 0x04

6.3.3.7.5.1 Detailed Description

Audio Device Types.

6.3.3.7.5.2 Macro Definition Documentation

AUDIO_TYPE_QUERY

```
#define AUDIO_TYPE_QUERY 0x00
```

Query audio device type

AUDIO_TYPE_INPUT

```
#define AUDIO_TYPE_INPUT 0x01
```

Set audio device type to input type

AUDIO_TYPE_OUTPUT

```
#define AUDIO_TYPE_OUTPUT 0x02
```

Set audio device type to output type

AUDIO_TYPE_MIXER

```
#define AUDIO_TYPE_MIXER 0x04
```

Set audio device type to mixer type

6.3.3.7.6 AUDIO_SAMP_RATES

Collaboration diagram for AUDIO_SAMP_RATES:

Macros

- #define AUDIO_SAMP_RATE_8K 0x0001
- #define AUDIO_SAMP_RATE_11K 0x0002
- #define AUDIO_SAMP_RATE_16K 0x0004
- #define AUDIO_SAMP_RATE_22K 0x0008
- #define AUDIO_SAMP_RATE_32K 0x0010
- #define AUDIO_SAMP_RATE_44K 0x0020
- #define AUDIO_SAMP_RATE_48K 0x0040
- #define AUDIO_SAMP_RATE_96K 0x0080
- #define AUDIO_SAMP_RATE_128K 0x0100
- #define AUDIO_SAMP_RATE_160K 0x0200
- #define AUDIO_SAMP_RATE_172K 0x0400
- #define AUDIO_SAMP_RATE_192K 0x0800

6.3.3.7.6.1 Detailed Description

Supported audio sample rates for the audio device.

6.3.3.7.6.2 Macro Definition Documentation

AUDIO_SAMP_RATE_8K

```
#define AUDIO_SAMP_RATE_8K 0x0001
```

Set audio device sample rate to 8K

AUDIO_SAMP_RATE_11K

```
#define AUDIO_SAMP_RATE_11K 0x0002
```

Set audio device sample rate to 11K

AUDIO_SAMP_RATE_16K

```
#define AUDIO_SAMP_RATE_16K 0x0004
```
Set audio device sample rate to 16K

AUDIO_SAMP_RATE_22K

```
#define AUDIO_SAMP_RATE_22K 0x0008
```
Set audio device sample rate to 22K

AUDIO_SAMP_RATE_32K

```
#define AUDIO_SAMP_RATE_32K 0x0010
```
Set audio device sample rate to 32K

AUDIO_SAMP_RATE_44K

```
#define AUDIO_SAMP_RATE_44K 0x0020
```
Set audio device sample rate to 44K

AUDIO_SAMP_RATE_48K

```
#define AUDIO_SAMP_RATE_48K 0x0040
```
Set audio device sample rate to 48K

AUDIO_SAMP_RATE_96K

```
#define AUDIO_SAMP_RATE_96K 0x0080
```
Set audio device sample rate to 96K

AUDIO_SAMP_RATE_128K

```
#define AUDIO_SAMP_RATE_128K 0x0100
```
Set audio device sample rate to 128K

AUDIO_SAMP_RATE_160K

```
#define AUDIO_SAMP_RATE_160K 0x0200
```
Set audio device sample rate to 160K

AUDIO_SAMP_RATE_172K

```
#define AUDIO_SAMP_RATE_172K 0x0400
```
Set audio device sample rate to 172K

AUDIO_SAMP_RATE_192K

```
#define AUDIO_SAMP_RATE_192K 0x0800
```
Set audio device sample rate to 192K

6.3.3.7.7 AUDIO_BIT_RATES

Collaboration diagram for AUDIO_BIT_RATES:

Macros

- #define AUDIO_BIT_RATE_22K 0x01
- #define AUDIO_BIT_RATE_44K 0x02
- #define AUDIO_BIT_RATE_48K 0x04
- #define AUDIO_BIT_RATE_96K 0x08

- #define AUDIO_BIT_RATE_128K 0x10
- #define AUDIO_BIT_RATE_160K 0x20
- #define AUDIO_BIT_RATE_172K 0x40
- #define AUDIO_BIT_RATE_192K 0x80

6.3.3.7.7.1 Detailed Description

Supported bit rates for the audio device.

6.3.3.7.7.2 Macro Definition Documentation

AUDIO_BIT_RATE_22K

```
#define AUDIO_BIT_RATE_22K 0x01
```

Set audio device bit rates to 22K

AUDIO_BIT_RATE_44K

```
#define AUDIO_BIT_RATE_44K 0x02
```

Set audio device bit rates to 44K

AUDIO_BIT_RATE_48K

```
#define AUDIO_BIT_RATE_48K 0x04
```

Set audio device bit rates to 48K

AUDIO_BIT_RATE_96K

```
#define AUDIO_BIT_RATE_96K 0x08
```

Set audio device bit rates to 96K

AUDIO_BIT_RATE_128K

```
#define AUDIO_BIT_RATE_128K 0x10
```

Set audio device bit rates to 128K

AUDIO_BIT_RATE_160K

```
#define AUDIO_BIT_RATE_160K 0x20
```

Set audio device bit rates to 160K

AUDIO_BIT_RATE_172K

```
#define AUDIO_BIT_RATE_172K 0x40
```

Set audio device bit rates to 172K

AUDIO_BIT_RATE_192K

```
#define AUDIO_BIT_RATE_192K 0x80
```

Set audio device bit rates to 192K

6.3.3.7.8 AUDIO_DSP

Collaboration diagram for AUDIO_DSP:

Macros

- #define AUDIO_DSP_PARAM 0
- #define AUDIO_DSP_SAMPLERATE 1
- #define AUDIO_DSP_CHANNELS 2
- #define AUDIO_DSP_SAMPLEBITS 3

6.3.3.7.8.1 Detailed Description

Support Dsp(input/output) Units controls. The macro group from application level, can set audio mixer parameters including samplerate, channels etc.

6.3.3.7.8.2 Macro Definition Documentation

AUDIO_DSP_PARAM

```
#define AUDIO_DSP_PARAM 0
```
get/set all params

AUDIO_DSP_SAMPLERATE

```
#define AUDIO_DSP_SAMPLERATE 1
```
samplerate

AUDIO_DSP_CHANNELS

```
#define AUDIO_DSP_CHANNELS 2
```
channels

AUDIO_DSP_SAMPLEBITS

```
#define AUDIO_DSP_SAMPLEBITS 3
```
sample bits width

6.3.3.7.9 AUDIO_MIXER

Collaboration diagram for AUDIO_MIXER:

Macros

- #define AUDIO_MIXER_QUERY 0x0000
- #define AUDIO_MIXER_MUTE 0x0001
- #define AUDIO_MIXER_VOLUME 0x0002
- #define AUDIO_MIXER_BASS 0x0004
- #define AUDIO_MIXER_MID 0x0008
- #define AUDIO_MIXER_TREBLE 0x0010
- #define AUDIO_MIXER_EQUALIZER 0x0020
- #define AUDIO_MIXER_LINE 0x0040
- #define AUDIO_MIXER_DIGITAL 0x0080
- #define AUDIO_MIXER_MIC 0x0100
- #define AUDIO_MIXER_VITURAL 0x0200
- #define AUDIO_MIXER_EXTEND 0x8000

6.3.3.7.9.1 Detailed Description

Supported Mixer Units controls. The macro group from driver level, can set audio mixer parameters including volume, frequence db, microphone etc.

6.3.3.7.9.2 Macro Definition Documentation

AUDIO_MIXER_QUERY

```
#define AUDIO_MIXER_QUERY 0x0000
```
Query mixer capabilities

AUDIO_MIXER_MUTE

```
#define AUDIO_MIXER_MUTE 0x0001
```
Mute audio device

AUDIO_MIXER_VOLUME

```
#define AUDIO_MIXER_VOLUME 0x0002
```
Set mixer volume

AUDIO_MIXER_BASS

```
#define AUDIO_MIXER_BASS 0x0004
```
Set the low-frequency section of the mixer

AUDIO_MIXER_MID

```
#define AUDIO_MIXER_MID 0x0008
```
Set the mid-frequency section of the mixer

AUDIO_MIXER_TREBLE

```
#define AUDIO_MIXER_TREBLE 0x0010
```
Set the high-frequency section of the mixer

AUDIO_MIXER_EQUALIZER

```
#define AUDIO_MIXER_EQUALIZER 0x0020
```
Set equalizer option

AUDIO_MIXER_LINE

```
#define AUDIO_MIXER_LINE 0x0040
```
Set line control option

AUDIO_MIXER_DIGITAL

```
#define AUDIO_MIXER_DIGITAL 0x0080
```
Set digital source

AUDIO_MIXER_MIC

```
#define AUDIO_MIXER_MIC 0x0100
```
Set microphone option

AUDIO_MIXER_VITURAL

```
#define AUDIO_MIXER_VITURAL 0x0200
```
Set virtual audio option

AUDIO_MIXER_EXTEND

```
#define AUDIO_MIXER_EXTEND 0x8000
```
Extend mixer command

6.3.3.7.10 CODEC_CMD

Collaboration diagram for CODEC_CMD:

Macros

- #define CODEC_CMD_RESET 0
- #define CODEC_CMD_SET_VOLUME 1
- #define CODEC_CMD_GET_VOLUME 2
- #define CODEC_CMD_SAMPLERATE 3
- #define CODEC_CMD_EQ 4
- #define CODEC_CMD_3D 5

6.3.3.7.10.1 Detailed Description

Device Control Commands. The macro group from hardware level, can set codec parametes including volume, EQ and 3D etc.

6.3.3.7.10.2 Macro Definition Documentation

CODEC_CMD_RESET

```
#define CODEC_CMD_RESET 0
```

Reset audio device by codec

CODEC_CMD_SET_VOLUME

```
#define CODEC_CMD_SET_VOLUME 1
```

Set volume by codec

CODEC_CMD_GET_VOLUME

```
#define CODEC_CMD_GET_VOLUME 2
```

Get volume by codec

CODEC_CMD_SAMPLERATE

```
#define CODEC_CMD_SAMPLERATE 3
```

Set sample rate by codec

CODEC_CMD_EQ

```
#define CODEC_CMD_EQ 4
```

Set equalizer by codec

CODEC_CMD_3D

```
#define CODEC_CMD_3D 5
```

Set 3D effect by codec

6.3.3.8 CAN Driver

Collaboration diagram for CAN Driver:

Data Structures

- struct rt_can_filter_item
- struct rt_can_filter_config
- struct rt_can_bit_timing
- struct rt_can_bit_timing_config
- struct can_configure
- struct rt_can_status
- struct rt_can_status_ind_type
- struct rt_can_msg

- struct rt_can_device
- struct rt_can_ops

Macros

- #define CAN_RX_FIFO0 (0x00000000U)
- #define CAN_RX_FIFO1 (0x00000001U)
- #define RT_CAN_FILTER_ITEM_INIT(id, ide, rtr, mode, mask)

Typedefs

- typedef rt_err_t(∗) rt_canstatus_ind(struct rt_can_device ∗can, void ∗args)
- typedef struct rt_can_status_ind_type ∗ rt_can_status_ind_type_t
- typedef void(∗) rt_can_bus_hook(struct rt_can_device ∗can)

Functions

- rt_err_t rt_hw_can_register (struct rt_can_device ∗can, const char ∗name, const struct rt_can_ops ∗ops, void ∗data)
- void rt_hw_can_isr (struct rt_can_device ∗can, int event)

6.3.3.8.1 Detailed Description

CAN driver api.

Example: Demonstrating CAN RX, Filters, and Blocking/Non-Blocking TX

```
#include <rtthread.h>
#include <rtdevice.h>

#define CAN_DEV_NAME       "can1"      // The name of the CAN device

static rt_device_t can_dev;            // CAN device handle
static struct rt_semaphore rx_sem;     // Semaphore for message reception

// Callback function for CAN reception
static rt_err_t can_rx_callback(rt_device_t dev, rt_size_t size)
{
    // The CAN interrupt calls this callback when data is received.
    // Release the semaphore to notify the receiving thread.
    rt_sem_release(&rx_sem);
    return RT_EOK;
}

static void can_rx_thread(void *parameter)
{
    rt_err_t res;
    struct rt_can_msg rx_msg = {0};

    // Set the receive callback function
    rt_device_set_rx_indicate(can_dev, can_rx_callback);

#ifdef RT_CAN_USING_HDR
    // Example of configuring multiple hardware filters
    struct rt_can_filter_item items[] =
    {
        // Filter 1: Match standard frames with IDs from 0x100 to 0x1FF. hdr_index will be -1.
        RT_CAN_FILTER_ITEM_INIT(0x100, 0, 0, 0, 0x700, RT_NULL, RT_NULL),
        // Filter 2: Match standard frames with IDs from 0x300 to 0x3FF. hdr_index will be -1.
        RT_CAN_FILTER_ITEM_INIT(0x300, 0, 0, 0, 0x700, RT_NULL, RT_NULL),
        // Filter 3: Exactly match standard frame with ID 0x211. hdr_index will be -1.
        RT_CAN_FILTER_ITEM_INIT(0x211, 0, 0, 0, 0x7FF, RT_NULL, RT_NULL),
        // Filter 4: Exactly match standard frame with ID 0x486 using a helper macro. hdr_index will be -1.
        RT_CAN_FILTER_STD_INIT(0x486, RT_NULL, RT_NULL),
        // Filter 5: Exactly match standard frame with ID 0x555 and explicitly assign it to filter bank #7.
        // This uses direct struct initialization: {id, ide, rtr, mode, mask, hdr_bank}.
        {0x555, 0, 0, 0, 0x7FF, 7}
    };
    // Create the filter configuration structure with 5 active filters.
    struct rt_can_filter_config cfg = {sizeof(items)/sizeof(items[0]), 1, items};
    // Set the hardware filters for the CAN device.
    res = rt_device_control(can_dev, RT_CAN_CMD_SET_FILTER, &cfg);
    RT_ASSERT(res == RT_EOK);
#endif

    // Some drivers might require an explicit start command.
    // This is driver-specific.
```

```
    rt_uint32_t cmd_arg = 1; // Argument to enable the controller
    res = rt_device_control(can_dev, RT_CAN_CMD_START, &cmd_arg);
    RT_ASSERT(res == RT_EOK);

    #define CAN_RX_BATCH_SIZE 8

    while (1)
    {
        // Block and wait for the semaphore, which is released by the receive callback.
        rt_sem_take(&rx_sem, RT_WAITING_FOREVER);

        // Drain all pending frames in batches.
        struct rt_can_msg rx_buf[CAN_RX_BATCH_SIZE];
        rt_ssize_t read_size;
        rt_size_t count;
        do
        {
            for (rt_size_t i = 0; i < CAN_RX_BATCH_SIZE; i++)
            {
                rx_buf[i].hdr_index = -1;
            }

            read_size = rt_device_read(can_dev, 0, rx_buf, sizeof(rx_buf));
            if (read_size <= 0)
            {
                break;
            }
            count = (rt_size_t)(read_size / sizeof(rx_buf[0]));
            for (rt_size_t i = 0; i < count; i++)
            {
                // Print the received message's ID and data.
                rt_kprintf("Received a message. ID: 0x%x, Data: ", rx_buf[i].id);
                for (int j = 0; j < rx_buf[i].len; j++)
                {
                    rt_kprintf("%02x ", rx_buf[i].data[j]);
                }
                rt_kprintf("\n");
            }
        }
        while(count == CAN_RX_BATCH_SIZE);
    }
}

int can_sample(int argc, char *argv[])
{
    rt_err_t res;
    rt_thread_t thread;
    char can_name[RT_NAME_MAX];

    // Allow specifying the CAN device name from the command line, e.g., "can_sample can2"
    if (argc == 2)
    {
        rt_strncpy(can_name, argv[1], RT_NAME_MAX);
    }
    else
    {
        rt_strncpy(can_name, CAN_DEV_NAME, RT_NAME_MAX);
    }

    // Find the CAN device by name
    can_dev = rt_device_find(can_name);
    if (!can_dev)
    {
        rt_kprintf("find device %s failed!\n", can_name);
        return -RT_ERROR;
    }

    // Initialize the receive semaphore
    rt_sem_init(&rx_sem, "rx_sem", 0, RT_IPC_FLAG_FIFO);

    // Open the CAN device in interrupt-driven TX/RX mode
    res = rt_device_open(can_dev, RT_DEVICE_FLAG_INT_TX | RT_DEVICE_FLAG_INT_RX);
    RT_ASSERT(res == RT_EOK);

    // Create and start the data receiving thread
    thread = rt_thread_create("can_rx", can_rx_thread, RT_NULL, 1024, 25, 10);
    if (thread != RT_NULL)
    {
        rt_thread_startup(thread);
    }
    else
    {
        rt_kprintf("create can_rx thread failed!\n");
        return -RT_ERROR;
```

```
    }

    rt_kprintf("CAN device %s opened successfully.\n", can_name);

    // --- Demonstrate Blocking Send ---
    struct rt_can_msg blocking_msg = {0};
    blocking_msg.id = 0x78;
    blocking_msg.ide = RT_CAN_STDID;
    blocking_msg.rtr = RT_CAN_DTR;
    blocking_msg.len = 8;
    // The `nonblocking` flag is 0 by default for blocking mode.
    for(int i = 0; i < 8; i++) blocking_msg.data[i] = i;

    rt_kprintf("Attempting to send a message in BLOCKING mode...\n");
    if (rt_device_write(can_dev, 0, &blocking_msg, sizeof(blocking_msg)) == sizeof(blocking_msg))
    {
        rt_kprintf("Blocking message sent successfully.\n");
    }
    else
    {
        rt_kprintf("Blocking message send failed.\n");
    }

    rt_thread_mdelay(100); // Wait a moment for clarity in the log

    // --- Demonstrate Non-Blocking Send ---
    struct rt_can_msg nonblocking_msg = {0};
    nonblocking_msg.id = 0x79;
    nonblocking_msg.ide = RT_CAN_STDID;
    nonblocking_msg.rtr = RT_CAN_DTR;
    nonblocking_msg.len = 4;
    nonblocking_msg.data[0] = 0xDE;
    nonblocking_msg.data[1] = 0xAD;
    nonblocking_msg.data[2] = 0xBE;
    nonblocking_msg.data[3] = 0xEF;
    nonblocking_msg.nonblocking = 1; // <-- Key: Set the non-blocking flag

    rt_kprintf("Attempting to send a message in NON-BLOCKING mode...\n");
    if (rt_device_write(can_dev, 0, &nonblocking_msg, sizeof(nonblocking_msg)) == sizeof(nonblocking_msg))
    {
        rt_kprintf("Non-blocking message was accepted (sent or enqueued).\n");
    }
    else
    {
        rt_kprintf("Non-blocking send failed (buffer was full).\n");
    }

    return res;
}
// Export the function to the MSH command line
MSH_CMD_EXPORT(can_sample, can device usage example);
```

6.3.3.8.2 Macro Definition Documentation

6.3.3.8.2.1 CAN_RX_FIFO0

```
#define CAN_RX_FIFO0 (0x00000000U)
```

CAN receive FIFO 0

6.3.3.8.2.2 CAN_RX_FIFO1

```
#define CAN_RX_FIFO1 (0x00000001U)
```

CAN receive FIFO 1

6.3.3.8.2.3 RT_CAN_FILTER_ITEM_INIT

```
#define RT_CAN_FILTER_ITEM_INIT(
                id,
                ide,
                rtr,
                mode,
                mask)
```

Value:

```
{(id), (ide), (rtr), (mode), (mask), -1, CAN_RX_FIFO0 }
```

A helper macro to initialize a rt_can_filter_item structure for Mask Mode.

Parameters

in	*id*	The CAN ID for the filter.
in	*ide*	Identifier type (0 for Standard, 1 for Extended).
in	*rtr*	Frame type (0 for Data, 1 for Remote).
in	*mode*	Filter mode (0 for Mask, 1 for List).
in	*mask*	The mask to be applied.

6.3.3.8.3 Typedef Documentation

6.3.3.8.3.1 rt_canstatus_ind

```
typedef rt_err_t(*) rt_canstatus_ind(struct rt_can_device *can, void *args)
```

Typedef for the CAN status indication callback function.

Parameters

in	*can*	A pointer to the CAN device.
in	*args*	User-provided arguments.

Returns

The operation status.

6.3.3.8.3.2 rt_can_status_ind_type_t

```
typedef struct rt_can_status_ind_type * rt_can_status_ind_type_t
```

Structure to hold the status indication callback and its arguments.

6.3.3.8.3.3 rt_can_bus_hook

```
typedef void(*) rt_can_bus_hook(struct rt_can_device *can)
```

Typedef for the periodic bus hook function.

Parameters

in	*can*	A pointer to the CAN device.

Returns

void

6.3.3.8.4 Function Documentation

6.3.3.8.4.1 rt_hw_can_register()

```
rt_err_t rt_hw_can_register (
            struct rt_can_device * can,
            const char * name,
            const struct rt_can_ops * ops,
            void * data)
```

This function registers a CAN device with the device framework.

Parameters

in	*can*	A pointer to the CAN device object to be registered.
in	*name*	The name that the device will be registered with.
in	*ops*	A pointer to the structure containing the low-level CAN driver operations.
in	*data*	A pointer to a user-defined data structure, which can be accessed via `can->parent.user_data`.

Returns

`RT_EOK` on successful registration, or a negative error code on failure.

6.3.3.8.4.2 rt_hw_can_isr()

```
void rt_hw_can_isr (
            struct rt_can_device * can,
            int event)
```

The framework-level ISR handler for CAN devices.
This function is called by the low-level BSP ISR and acts as the central dispatcher for all CAN-related interrupt events. It handles both receive events and transmission-complete events.

Parameters

in	*can*	A pointer to the CAN device structure.
in	*event*	The interrupt event mask, indicating the cause of the interrupt.

6.3.3.9 I2C

Collaboration diagram for I2C:

Data Structures

- struct rt_i2c_priv_data
- struct rt_i2c_msg
- struct rt_i2c_bus_device_ops
- struct rt_i2c_configuration
- struct rt_i2c_bus_device
- struct rt_i2c_client

Macros

- #define RT_I2C_WR 0x0000
- #define RT_I2C_RD (1u << 0)
- #define RT_I2C_ADDR_10BIT (1u << 2)
- #define RT_I2C_NO_START (1u << 4)
- #define RT_I2C_IGNORE_NACK (1u << 5)
- #define RT_I2C_NO_STOP (1u << 7)

Functions

- rt_err_t rt_i2c_bus_device_device_init (struct rt_i2c_bus_device *bus, const char *name)
- rt_err_t rt_i2c_bus_device_register (struct rt_i2c_bus_device *bus, const char *bus_name)
- struct rt_i2c_bus_device * rt_i2c_bus_device_find (const char *bus_name)
- rt_ssize_t rt_i2c_transfer (struct rt_i2c_bus_device *bus, struct rt_i2c_msg msgs[], rt_uint32_t num)
- rt_err_t rt_i2c_control (struct rt_i2c_bus_device *bus, int cmd, void *args)

- rt_ssize_t rt_i2c_master_send (struct rt_i2c_bus_device *bus, rt_uint16_t addr, rt_uint16_t flags, const rt_uint8_t *buf, rt_uint32_t count)
- rt_ssize_t rt_i2c_master_recv (struct rt_i2c_bus_device *bus, rt_uint16_t addr, rt_uint16_t flags, rt_uint8_t *buf, rt_uint32_t count)

6.3.3.9.1 Detailed Description

I2C driver api.

Example

```
#include <rtthread.h>
#include <rtdevice.h>

#define AHT10_I2C_BUS_NAME          "i2c1"  // I2C
#define AHT10_ADDR                  0x38    //
#define AHT10_CALIBRATION_CMD       0xE1    //
#define AHT10_NORMAL_CMD            0xA8    //
#define AHT10_GET_DATA              0xAC    //

static struct rt_i2c_bus_device *i2c_bus = RT_NULL;     // I2C
static rt_bool_t initialized = RT_FALSE;                //

//
static rt_err_t write_reg(struct rt_i2c_bus_device *bus, rt_uint8_t reg, rt_uint8_t *data)
{
    rt_uint8_t buf[3];
    struct rt_i2c_msg msgs;
    rt_uint32_t buf_size = 1;

    buf[0] = reg; //cmd
    if (data != RT_NULL)
    {
        buf[1] = data[0];
        buf[2] = data[1];
        buf_size = 3;
    }

    msgs.addr = AHT10_ADDR;
    msgs.flags = RT_I2C_WR;
    msgs.buf = buf;
    msgs.len = buf_size;

    // I2C
    if (rt_i2c_transfer(bus, &msgs, 1) == 1)
    {
        return RT_EOK;
    }
    else
    {
        return -RT_ERROR;
    }
}

//
static rt_err_t read_regs(struct rt_i2c_bus_device *bus, rt_uint8_t len, rt_uint8_t *buf)
{
    struct rt_i2c_msg msgs;

    msgs.addr = AHT10_ADDR;
    msgs.flags = RT_I2C_RD;
    msgs.buf = buf;
    msgs.len = len;

    // I2C
    if (rt_i2c_transfer(bus, &msgs, 1) == 1)
    {
        return RT_EOK;
    }
    else
    {
        return -RT_ERROR;
    }
}

static void read_temp_humi(float *cur_temp, float *cur_humi)
{
    rt_uint8_t temp[6];

    write_reg(i2c_bus, AHT10_GET_DATA, RT_NULL);      //
    rt_thread_mdelay(400);
    read_regs(i2c_bus, 6, temp);                //

    //
    *cur_humi = (temp[1] << 12 | temp[2] << 4 | (temp[3] & 0xf0) >> 4) * 100.0 / (1 << 20);
```

```
    //
    *cur_temp = ((temp[3] & 0xf) « 16 | temp[4] « 8 | temp[5]) * 200.0 / (1 « 20) - 50;
}

static void aht10_init(const char *name)
{
    rt_uint8_t temp[2] = {0, 0};

    // I2CI2C
    i2c_bus = (struct rt_i2c_bus_device *)rt_device_find(name);

    if (i2c_bus == RT_NULL)
    {
        rt_kprintf("can't find %s device!\n", name);
    }
    else
    {
        write_reg(i2c_bus, AHT10_NORMAL_CMD, temp);
        rt_thread_mdelay(400);

        temp[0] = 0x08;
        temp[1] = 0x00;
        write_reg(i2c_bus, AHT10_CALIBRATION_CMD, temp);
        rt_thread_mdelay(400);
        initialized = RT_TRUE;
    }
}

static void i2c_aht10_sample(int argc, char *argv[])
{
    float humidity, temperature;
    char name[RT_NAME_MAX];

    humidity = 0.0;
    temperature = 0.0;

    if (argc == 2)
    {
        rt_strncpy(name, argv[1], RT_NAME_MAX);
    }
    else
    {
        rt_strncpy(name, AHT10_I2C_BUS_NAME, RT_NAME_MAX);
    }

    if (!initialized)
    {
        //
        aht10_init(name);
    }
    if (initialized)
    {
        //
        read_temp_humi(&temperature, &humidity);

        rt_kprintf("read aht10 sensor humidity   : %d.%d %%\n", (int)humidity, (int)(humidity * 10) % 10);
        if( temperature >= 0 )
        {
            rt_kprintf("read aht10 sensor temperature: %d.%d°C\n", (int)temperature, (int)(temperature * 10)
      % 10);
        }
        else
        {
            rt_kprintf("read aht10 sensor temperature: %d.%d°C\n", (int)temperature, (int)(-temperature *
      10) % 10);
        }
    }
    else
    {
        rt_kprintf("initialize sensor failed!\n");
    }
}
//  msh
MSH_CMD_EXPORT(i2c_aht10_sample, i2c aht10 sample);
```

6.3.3.9.2 Macro Definition Documentation

6.3.3.9.2.1 RT_I2C_WR

```
#define RT_I2C_WR 0x0000
```

i2c wirte flag

6.3.3.9.2.2 RT_I2C_RD

```
#define RT_I2C_RD (1u << 0)
```

i2c read flag

6.3.3.9.2.3 RT_I2C_ADDR_10BIT

```
#define RT_I2C_ADDR_10BIT (1u << 2)
```

this is a ten bit chip address

6.3.3.9.2.4 RT_I2C_NO_START

```
#define RT_I2C_NO_START (1u << 4)
```

do not generate START condition

6.3.3.9.2.5 RT_I2C_IGNORE_NACK

```
#define RT_I2C_IGNORE_NACK (1u << 5)
```

ignore NACK from slave

6.3.3.9.2.6 RT_I2C_NO_STOP

```
#define RT_I2C_NO_STOP (1u << 7)
```

do not generate STOP condition

6.3.3.9.3 Function Documentation

6.3.3.9.3.1 rt_i2c_bus_device_device_init()

```
rt_err_t rt_i2c_bus_device_device_init (
            struct rt_i2c_bus_device * bus,
            const char * name)
```

I2C Bus Device Initialization.

Parameters

bus	the I2C bus device
name	the name of I2C bus device

Returns

rt_err_t error code

6.3.3.9.3.2 rt_i2c_bus_device_register()

```
rt_err_t rt_i2c_bus_device_register (
            struct rt_i2c_bus_device * bus,
            const char * bus_name)
```

I2C Bus Device Register.

Parameters

bus	the I2C bus device
bus_name	the name of I2C bus device

Returns

rt_err_t error code

6.3.3.9.3.3 rt_i2c_bus_device_find()

```
struct rt_i2c_bus_device * rt_i2c_bus_device_find (
            const char * bus_name)
```

I2C Bus Device Find.

Parameters

bus_name	the name of I2C bus device

Returns

rt_i2c_bus_device the I2C bus device

6.3.3.9.3.4 rt_i2c_transfer()

```
rt_ssize_t rt_i2c_transfer (
            struct rt_i2c_bus_device * bus,
            struct rt_i2c_msg msgs[],
            rt_uint32_t num)
```

I2C data transmission.

Parameters

bus	the I2C bus device
msgs	the I2C message list
num	the number of I2C message

Returns

rt_ssize_t the actual length of transmitted

6.3.3.9.3.5 rt_i2c_control()

```
rt_err_t rt_i2c_control (
            struct rt_i2c_bus_device * bus,
            int cmd,
            void * args)
```

I2C Control.

Parameters

bus	the I2C bus device
cmd	the I2C control command
args	the I2C control arguments

Returns

rt_err_t error code

6.3.3.9.3.6 rt_i2c_master_send()

```
rt_ssize_t rt_i2c_master_send (
             struct rt_i2c_bus_device * bus,
             rt_uint16_t addr,
             rt_uint16_t flags,
             const rt_uint8_t * buf,
             rt_uint32_t count)
```

I2C Master Send.

Parameters

bus	the I2C bus device
addr	the I2C slave address
flags	the I2C flags
buf	the I2C send buffer
count	the I2C send buffer length

Returns

rt_ssize_t the actual length of transmitted

6.3.3.9.3.7 rt_i2c_master_recv()

```
rt_ssize_t rt_i2c_master_recv (
             struct rt_i2c_bus_device * bus,
             rt_uint16_t addr,
             rt_uint16_t flags,
             rt_uint8_t * buf,
             rt_uint32_t count)
```

I2C Master Receive.

Parameters

bus	the I2C bus device
addr	the I2C slave address
flags	the I2C flags
buf	the I2C receive buffer
count	the I2C receive buffer length

Returns

rt_ssize_t the actual length of received

6.3.3.10 Pin

Collaboration diagram for Pin:

Data Structures

- struct rt_device_pin
- struct rt_device_pin_mode
- struct rt_device_pin_value
- struct rt_pin_irq_hdr
- struct rt_pin_ops

Macros

- #define PIN_LOW 0x00
- #define PIN_HIGH 0x01
- #define PIN_MODE_OUTPUT 0x00
- #define PIN_MODE_INPUT 0x01
- #define PIN_MODE_INPUT_PULLUP 0x02
- #define PIN_MODE_INPUT_PULLDOWN 0x03
- #define PIN_MODE_OUTPUT_OD 0x04
- #define PIN_IRQ_MODE_RISING 0x00
- #define PIN_IRQ_MODE_FALLING 0x01
- #define PIN_IRQ_MODE_RISING_FALLING 0x02
- #define PIN_IRQ_MODE_HIGH_LEVEL 0x03
- #define PIN_IRQ_MODE_LOW_LEVEL 0x04
- #define PIN_IRQ_DISABLE 0x00
- #define PIN_IRQ_ENABLE 0x01
- #define PIN_IRQ_PIN_NONE PIN_NONE

Functions

- int rt_device_pin_register (const char *name, const struct rt_pin_ops *ops, void *user_data)
- void rt_pin_mode (rt_base_t pin, rt_uint8_t mode)
- void rt_pin_write (rt_base_t pin, rt_ssize_t value)
- rt_ssize_t rt_pin_read (rt_base_t pin)
- rt_base_t rt_pin_get (const char *name)
- rt_err_t rt_pin_attach_irq (rt_base_t pin, rt_uint8_t mode, void(*hdr)(void *args), void *args)
- rt_err_t rt_pin_detach_irq (rt_base_t pin)
- rt_err_t rt_pin_irq_enable (rt_base_t pin, rt_uint8_t enabled)
- rt_err_t rt_pin_debounce (rt_base_t pin, rt_uint32_t debounce)

6.3.3.10.1 Detailed Description

Pin driver api.

Example

```
#include <rtthread.h>
#include <rtdevice.h>


#ifndef BEEP_PIN_NUM
    #define BEEP_PIN_NUM            35  // PB0
#endif
#ifndef KEY0_PIN_NUM
    #define KEY0_PIN_NUM            55  // PD8
#endif
#ifndef KEY1_PIN_NUM
    #define KEY1_PIN_NUM            56  // PD9
#endif

void beep_on(void *args)
{
    rt_kprintf("turn on beep!\n");

    rt_pin_write(BEEP_PIN_NUM, PIN_HIGH);
}

void beep_off(void *args)
{
    rt_kprintf("turn off beep!\n");

    rt_pin_write(BEEP_PIN_NUM, PIN_LOW);
}

static void pin_beep_sample(void)
{
    rt_pin_mode(BEEP_PIN_NUM, PIN_MODE_OUTPUT);
    rt_pin_write(BEEP_PIN_NUM, PIN_LOW);

    rt_pin_mode(KEY0_PIN_NUM, PIN_MODE_INPUT_PULLUP);
    rt_pin_attach_irq(KEY0_PIN_NUM, PIN_IRQ_MODE_FALLING, beep_on, RT_NULL);
```

```
    rt_pin_irq_enable(KEY0_PIN_NUM, PIN_IRQ_ENABLE);


    rt_pin_mode(KEY1_PIN_NUM, PIN_MODE_INPUT_PULLUP);
    rt_pin_attach_irq(KEY1_PIN_NUM, PIN_IRQ_MODE_FALLING, beep_off, RT_NULL);
    rt_pin_irq_enable(KEY1_PIN_NUM, PIN_IRQ_ENABLE);
}

MSH_CMD_EXPORT(pin_beep_sample, pin beep sample);
```

6.3.3.10.2 Macro Definition Documentation

6.3.3.10.2.1 PIN_LOW

```
#define PIN_LOW 0x00
```

low level

6.3.3.10.2.2 PIN_HIGH

```
#define PIN_HIGH 0x01
```

high level

6.3.3.10.2.3 PIN_MODE_OUTPUT

```
#define PIN_MODE_OUTPUT 0x00
```

output mode

6.3.3.10.2.4 PIN_MODE_INPUT

```
#define PIN_MODE_INPUT 0x01
```

input mode

6.3.3.10.2.5 PIN_MODE_INPUT_PULLUP

```
#define PIN_MODE_INPUT_PULLUP 0x02
```

input mode with pull-up

6.3.3.10.2.6 PIN_MODE_INPUT_PULLDOWN

```
#define PIN_MODE_INPUT_PULLDOWN 0x03
```

input mode with pull-down

6.3.3.10.2.7 PIN_MODE_OUTPUT_OD

```
#define PIN_MODE_OUTPUT_OD 0x04
```

output mode with open-drain

6.3.3.10.2.8 PIN_IRQ_MODE_RISING

```
#define PIN_IRQ_MODE_RISING 0x00
```

rising edge trigger

6.3.3.10.2.9 PIN_IRQ_MODE_FALLING

```
#define PIN_IRQ_MODE_FALLING 0x01
```

falling edge trigger

6.3.3.10.2.10 PIN_IRQ_MODE_RISING_FALLING

```
#define PIN_IRQ_MODE_RISING_FALLING 0x02
```

rising and falling edge trigger

6.3.3.10.2.11 PIN_IRQ_MODE_HIGH_LEVEL

```
#define PIN_IRQ_MODE_HIGH_LEVEL 0x03
```

high level trigger

6.3.3.10.2.12 PIN_IRQ_MODE_LOW_LEVEL

```
#define PIN_IRQ_MODE_LOW_LEVEL 0x04
```

low level trigger

6.3.3.10.2.13 PIN_IRQ_DISABLE

```
#define PIN_IRQ_DISABLE 0x00
```

disable irq

6.3.3.10.2.14 PIN_IRQ_ENABLE

```
#define PIN_IRQ_ENABLE 0x01
```

enable irq

6.3.3.10.2.15 PIN_IRQ_PIN_NONE

```
#define PIN_IRQ_PIN_NONE PIN_NONE
```

no pin irq

6.3.3.10.3 Function Documentation

6.3.3.10.3.1 rt_device_pin_register()

```
int rt_device_pin_register (
            const char * name,
            const struct rt_pin_ops * ops,
            void * user_data)
```

register a pin device

Parameters

name	the name of pin device
ops	the operations of pin device
user_data	the user data of pin device

Returns

int error code

6.3.3.10.3.2 rt_pin_mode()

```
void rt_pin_mode (
            rt_base_t pin,
            rt_uint8_t mode)
```

set pin mode

Parameters

pin	the pin number
mode	the pin mode

6.3.3.10.3.3 rt_pin_write()

```
void rt_pin_write (
            rt_base_t pin,
            rt_ssize_t value)
```

write pin value

Parameters

pin	the pin number
value	the pin value

6.3.3.10.3.4 rt_pin_read()

```
rt_ssize_t rt_pin_read (
            rt_base_t pin)
```

read pin value

Parameters

pin	the pin number

Returns

rt_ssize_t the pin value

6.3.3.10.3.5 rt_pin_get()

```
rt_base_t rt_pin_get (
            const char * name)
```

get pin number by name

Parameters

name	the pin name

Returns

rt_base_t the pin number

6.3.3.10.3.6 rt_pin_attach_irq()

```
rt_err_t rt_pin_attach_irq (
            rt_base_t pin,
            rt_uint8_t mode,
            void(* hdr )(void *args),
            void * args)
```

bind the pin interrupt callback function

Parameters

pin	the pin number
mode	the irq mode

hdr	the irq callback function
args	the argument of the callback function

Returns

rt_err_t error code

6.3.3.10.3.7 rt_pin_detach_irq()

```
rt_err_t rt_pin_detach_irq (
            rt_base_t pin)
```

detach the pin interrupt callback function

Parameters

pin	the pin number

Returns

rt_err_t error code

6.3.3.10.3.8 rt_pin_irq_enable()

```
rt_err_t rt_pin_irq_enable (
            rt_base_t pin,
            rt_uint8_t enabled)
```

enable or disable the pin interrupt

Parameters

pin	the pin number
enabled	PIN_IRQ_ENABLE or PIN_IRQ_DISABLE

Returns

rt_err_t error code

6.3.3.10.3.9 rt_pin_debounce()

```
rt_err_t rt_pin_debounce (
            rt_base_t pin,
            rt_uint32_t debounce)
```

set the pin's debounce time

Parameters

pin	the pin number
debounce	time

Returns

rt_err_t error code

6.3.3.11 PWM

Collaboration diagram for PWM:

Data Structures

- struct rt_pwm_configuration
- struct rt_pwm_ops
- struct rt_device_pwm

Functions

- rt_err_t rt_device_pwm_register (struct rt_device_pwm *device, const char *name, const struct rt_pwm_ops *ops, const void *user_data)
- rt_err_t rt_pwm_enable (struct rt_device_pwm *device, int channel)
- rt_err_t rt_pwm_disable (struct rt_device_pwm *device, int channel)
- rt_err_t rt_pwm_set (struct rt_device_pwm *device, int channel, rt_uint32_t period, rt_uint32_t pulse)
- rt_err_t rt_pwm_set_period (struct rt_device_pwm *device, int channel, rt_uint32_t period)
- rt_err_t rt_pwm_set_pulse (struct rt_device_pwm *device, int channel, rt_uint32_t pulse)
- rt_err_t rt_pwm_set_dead_time (struct rt_device_pwm *device, int channel, rt_uint32_t dead_time)
- rt_err_t rt_pwm_set_phase (struct rt_device_pwm *device, int channel, rt_uint32_t phase)
- rt_err_t rt_pwm_get (struct rt_device_pwm *device, struct rt_pwm_configuration *cfg)

6.3.3.11.1 Detailed Description

PWM driver api.

Example

```
#include <rtthread.h>
#include <rtdevice.h>

#define PWM_DEV_NAME        "pwm3"  // PWM
#define PWM_DEV_CHANNEL     4       // PWM

struct rt_device_pwm *pwm_dev;      // PWM

static int pwm_led_sample(int argc, char *argv[])
{
    rt_uint32_t period, pulse, dir;

    period = 500000;    // 0.5msns
    dir = 1;            // PWM
    pulse = 0;          // PWMns

    //
    pwm_dev = (struct rt_device_pwm *)rt_device_find(PWM_DEV_NAME);
    if (pwm_dev == RT_NULL)
    {
        rt_kprintf("pwm sample run failed! can't find %s device!\n", PWM_DEV_NAME);
        return -RT_ERROR;
    }

    // PWM
    rt_pwm_set(pwm_dev, PWM_DEV_CHANNEL, period, pulse);
    //
    rt_pwm_enable(pwm_dev, PWM_DEV_CHANNEL);

    while (1)
    {
        rt_thread_mdelay(50);
        if (dir)
        {
            pulse += 5000;      // 05000ns
        }
        else
        {
            pulse -= 5000;      // 5000ns
        }
        if (pulse >= period)
        {
            dir = 0;
        }
        if (0 == pulse)
        {
            dir = 1;
```

```
        }

        // PWM
        rt_pwm_set(pwm_dev, PWM_DEV_CHANNEL, period, pulse);
    }
}

MSH_CMD_EXPORT(pwm_led_sample, pwm sample);
```

6.3.3.11.2 Function Documentation

6.3.3.11.2.1 rt_device_pwm_register()

```
rt_err_t rt_device_pwm_register (
             struct rt_device_pwm * device,
             const char * name,
             const struct rt_pwm_ops * ops,
             const void * user_data)
```

register a PWM device

Parameters

device	the PWM device
name	the name of PWM device
ops	the operations of PWM device
user_data	the user data of PWM device

Returns

rt_err_t error code

6.3.3.11.2.2 rt_pwm_enable()

```
rt_err_t rt_pwm_enable (
             struct rt_device_pwm * device,
             int channel)
```

enable the PWM channel

Parameters

device	the PWM device
channel	the channel of PWM

Returns

rt_err_t error code

6.3.3.11.2.3 rt_pwm_disable()

```
rt_err_t rt_pwm_disable (
             struct rt_device_pwm * device,
             int channel)
```

disable the PWM channel

Parameters

device	the PWM device

channel	the channel of PWM

Returns

rt_err_t error code

6.3.3.11.2.4 rt_pwm_set()

```
rt_err_t rt_pwm_set (
            struct rt_device_pwm * device,
            int channel,
            rt_uint32_t period,
            rt_uint32_t pulse)
```

set the PWM channel

Parameters

device	the PWM device
channel	the channel of PWM
period	the period of PWM
pulse	the pulse of PWM

Returns

rt_err_t error code

6.3.3.11.2.5 rt_pwm_set_period()

```
rt_err_t rt_pwm_set_period (
            struct rt_device_pwm * device,
            int channel,
            rt_uint32_t period)
```

set the PWM channel period

Parameters

device	the PWM device
channel	the channel of PWM
period	the period of PWM

Returns

rt_err_t error code

6.3.3.11.2.6 rt_pwm_set_pulse()

```
rt_err_t rt_pwm_set_pulse (
            struct rt_device_pwm * device,
            int channel,
            rt_uint32_t pulse)
```

set the PWM channel pulse

Parameters

device	the PWM device
channel	the channel of PWM
pulse	the period of PWM

Returns

rt_err_t error code

6.3.3.11.2.7 rt_pwm_set_dead_time()

```
rt_err_t rt_pwm_set_dead_time (
            struct rt_device_pwm * device,
            int channel,
            rt_uint32_t dead_time)
```

set the dead zone time of PWM

Parameters

device	the PWM device
channel	the channel of PWM
dead_time	dead zone time

Returns

rt_err_t error code

6.3.3.11.2.8 rt_pwm_set_phase()

```
rt_err_t rt_pwm_set_phase (
            struct rt_device_pwm * device,
            int channel,
            rt_uint32_t phase)
```

set the phase of PWM

Parameters

device	the PWM device
channel	the channel of PWM
phase	phase

Returns

rt_err_t error code

6.3.3.11.2.9 rt_pwm_get()

```
rt_err_t rt_pwm_get (
            struct rt_device_pwm * device,
            struct rt_pwm_configuration * cfg)
```

get the PWM configuration

Parameters

device	the PWM device
cfg	the configuration of PWM

Returns

rt_err_t error code

6.3.3.12 RTC

Collaboration diagram for RTC:

Data Structures

- struct rt_rtc_wkalarm
- struct rt_rtc_ops
- struct rt_rtc_device

Macros

- #define RT_DEVICE_CTRL_RTC_GET_TIME (RT_DEVICE_CTRL_BASE(RTC) + 0x01)
- #define RT_DEVICE_CTRL_RTC_SET_TIME (RT_DEVICE_CTRL_BASE(RTC) + 0x02)
- #define RT_DEVICE_CTRL_RTC_GET_TIMEVAL (RT_DEVICE_CTRL_BASE(RTC) + 0x03)
- #define RT_DEVICE_CTRL_RTC_SET_TIMEVAL (RT_DEVICE_CTRL_BASE(RTC) + 0x04)
- #define RT_DEVICE_CTRL_RTC_GET_ALARM (RT_DEVICE_CTRL_BASE(RTC) + 0x05)
- #define RT_DEVICE_CTRL_RTC_SET_ALARM (RT_DEVICE_CTRL_BASE(RTC) + 0x06)
- #define RT_DEVICE_CTRL_RTC_GET_TIMESPEC (RT_DEVICE_CTRL_BASE(RTC) + 0x07)
- #define RT_DEVICE_CTRL_RTC_SET_TIMESPEC (RT_DEVICE_CTRL_BASE(RTC) + 0x08)
- #define RT_DEVICE_CTRL_RTC_GET_TIMERES (RT_DEVICE_CTRL_BASE(RTC) + 0x09)

Typedefs

- typedef struct rt_rtc_device rt_rtc_dev_t

Functions

- rt_err_t rt_hw_rtc_register (rt_rtc_dev_t *rtc, const char *name, rt_uint32_t flag, void *data)
- rt_err_t set_date (rt_uint32_t year, rt_uint32_t month, rt_uint32_t day)
- rt_err_t set_time (rt_uint32_t hour, rt_uint32_t minute, rt_uint32_t second)
- rt_err_t set_timestamp (time_t timestamp)
- rt_err_t get_timestamp (time_t *timestamp)

6.3.3.12.1 Detailed Description

RTC driver api.

Example

```
#include <rtthread.h>
#include <rtdevice.h>

#define RTC_NAME       "rtc"

static int rtc_sample(int argc, char *argv[])
{
    rt_err_t ret = RT_EOK;
    time_t now;
    rt_device_t device = RT_NULL;

    device = rt_device_find(RTC_NAME);
    if (!device)
    {
      LOG_E("find %s failed!", RTC_NAME);
      return -RT_ERROR;
```

```
    }

    if(rt_device_open(device, 0) != RT_EOK)
    {
      LOG_E("open %s failed!", RTC_NAME);
      return -RT_ERROR;
    }

    ret = set_date(2018, 12, 3);
    if (ret != RT_EOK)
    {
        rt_kprintf("set RTC date failed\n");
        return ret;
    }

    ret = set_time(11, 15, 50);
    if (ret != RT_EOK)
    {
        rt_kprintf("set RTC time failed\n");
        return ret;
    }

    rt_thread_mdelay(3000);

    now = time(RT_NULL);
    rt_kprintf("%s\n", ctime(&now));

    return ret;
}
MSH_CMD_EXPORT(rtc_sample, rtc sample);
```

6.3.3.12.2 Macro Definition Documentation

6.3.3.12.2.1 RT_DEVICE_CTRL_RTC_GET_TIME

```
#define RT_DEVICE_CTRL_RTC_GET_TIME (RT_DEVICE_CTRL_BASE(RTC) + 0x01)
```

get second time

6.3.3.12.2.2 RT_DEVICE_CTRL_RTC_SET_TIME

```
#define RT_DEVICE_CTRL_RTC_SET_TIME (RT_DEVICE_CTRL_BASE(RTC) + 0x02)
```

set second time

6.3.3.12.2.3 RT_DEVICE_CTRL_RTC_GET_TIMEVAL

```
#define RT_DEVICE_CTRL_RTC_GET_TIMEVAL (RT_DEVICE_CTRL_BASE(RTC) + 0x03)
```

get timeval for gettimeofday

6.3.3.12.2.4 RT_DEVICE_CTRL_RTC_SET_TIMEVAL

```
#define RT_DEVICE_CTRL_RTC_SET_TIMEVAL (RT_DEVICE_CTRL_BASE(RTC) + 0x04)
```

set timeval for gettimeofday

6.3.3.12.2.5 RT_DEVICE_CTRL_RTC_GET_ALARM

```
#define RT_DEVICE_CTRL_RTC_GET_ALARM (RT_DEVICE_CTRL_BASE(RTC) + 0x05)
```

get alarm

6.3.3.12.2.6 RT_DEVICE_CTRL_RTC_SET_ALARM

```
#define RT_DEVICE_CTRL_RTC_SET_ALARM (RT_DEVICE_CTRL_BASE(RTC) + 0x06)
```

set alarm

6.3.3.12.2.7 RT_DEVICE_CTRL_RTC_GET_TIMESPEC

```
#define RT_DEVICE_CTRL_RTC_GET_TIMESPEC (RT_DEVICE_CTRL_BASE(RTC) + 0x07)
```

get timespec for clock_gettime

6.3.3.12.2.8 RT_DEVICE_CTRL_RTC_SET_TIMESPEC

```
#define RT_DEVICE_CTRL_RTC_SET_TIMESPEC (RT_DEVICE_CTRL_BASE(RTC) + 0x08)
```

set timespec for clock_settime

6.3.3.12.2.9 RT_DEVICE_CTRL_RTC_GET_TIMERES

```
#define RT_DEVICE_CTRL_RTC_GET_TIMERES (RT_DEVICE_CTRL_BASE(RTC) + 0x09)
```

get resolution for clock_getres

6.3.3.12.3 Typedef Documentation

6.3.3.12.3.1 rt_rtc_dev_t

```
typedef struct rt_rtc_device rt_rtc_dev_t
```

RTC device structure.

6.3.3.12.4 Function Documentation

6.3.3.12.4.1 rt_hw_rtc_register()

```
rt_err_t rt_hw_rtc_register (
            rt_rtc_dev_t * rtc,
            const char * name,
            rt_uint32_t flag,
            void * data)
```

Register a RTC device.

Parameters

rtc	RTC device
name	RTC device name
flag	RTC device flag
data	RTC device data

Returns

rt_err_t error code

6.3.3.12.4.2 set_date()

```
rt_err_t set_date (
            rt_uint32_t year,
            rt_uint32_t month,
            rt_uint32_t day)
```

set date

Parameters

year	year
month	month
day	day

Returns

rt_err_t error code

6.3.3.12.4.3 set_time()

```
rt_err_t set_time (
            rt_uint32_t hour,
            rt_uint32_t minute,
            rt_uint32_t second)
```

set time

Parameters

hour	hour
minute	minute
second	second

Returns

rt_err_t error code

6.3.3.12.4.4 set_timestamp()

```
rt_err_t set_timestamp (
            time_t timestamp)
```

set timestamp

Parameters

timestamp	A pointer to time

Returns

rt_err_t error code

6.3.3.12.4.5 get_timestamp()

```
rt_err_t get_timestamp (
            time_t * timestamp)
```

get timestamp

Parameters

timestamp	A secondary pointer to time

Returns

rt_err_t error code

6.3.3.13 Serial

Collaboration diagram for Serial:

Data Structures

- struct rt_serial_rx_fifo
- struct rt_serial_tx_fifo
- struct rt_serial_device
- struct rt_uart_ops

Typedefs

- typedef void(∗) rt_hw_serial_rxind_hookproto_t(rt_device_t dev, rt_size_t size)

Functions

- void rt_hw_serial_isr (struct rt_serial_device ∗serial, int event)
- rt_err_t rt_hw_serial_register (struct rt_serial_device ∗serial, const char ∗name, rt_uint32_t flag, void ∗data)
- rt_err_t rt_hw_serial_register_tty (struct rt_serial_device ∗serial)

6.3.3.13.1 Detailed Description

Serial driver api.

Example

```
#include <rtthread.h>

#define SAMPLE_UART_NAME       "uart2"
static struct rt_semaphore rx_sem;
static rt_device_t serial;

static rt_err_t uart_input(rt_device_t dev, rt_size_t size)
{

    rt_sem_release(&rx_sem);

    return RT_EOK;
}

static void serial_thread_entry(void *parameter)
{
    char ch;

    while (1)
    {

        while (rt_device_read(serial, -1, &ch, 1) != 1)
        {

            rt_sem_take(&rx_sem, RT_WAITING_FOREVER);
        }

        ch = ch + 1;
        rt_device_write(serial, 0, &ch, 1);
    }
}

static int uart_sample(int argc, char *argv[])
{
    rt_err_t ret = RT_EOK;
    char uart_name[RT_NAME_MAX];
    char str[] = "hello RT-Thread!\r\n";

    if (argc == 2)
    {
        rt_strncpy(uart_name, argv[1], RT_NAME_MAX);
    }
    else
    {
        rt_strncpy(uart_name, SAMPLE_UART_NAME, RT_NAME_MAX);
    }


    serial = rt_device_find(uart_name);
    if (!serial)
    {
        rt_kprintf("find %s failed!\n", uart_name);
        return -RT_ERROR;
```

```
    }

    rt_sem_init(&rx_sem, "rx_sem", 0, RT_IPC_FLAG_FIFO);

    rt_device_open(serial, RT_DEVICE_FLAG_INT_RX);

    rt_device_set_rx_indicate(serial, uart_input);

    rt_device_write(serial, 0, str, (sizeof(str) - 1));

    rt_thread_t thread = rt_thread_create("serial", serial_thread_entry, RT_NULL, 1024, 25, 10);

    if (thread != RT_NULL)
    {
        rt_thread_startup(thread);
    }
    else
    {
        ret = -RT_ERROR;
    }

    return ret;
}

MSH_CMD_EXPORT(uart_sample, uart device sample);
```

6.3.3.13.2 Typedef Documentation

6.3.3.13.2.1 rt_hw_serial_rxind_hookproto_t

```
typedef void(*) rt_hw_serial_rxind_hookproto_t(rt_device_t dev, rt_size_t size)
```

Sets a hook function when RX indicate is called.

6.3.3.13.3 Function Documentation

6.3.3.13.3.1 rt_hw_serial_isr()

```
void rt_hw_serial_isr (
            struct rt_serial_device * serial,
            int event)
```

Serial interrupt service routine.

Parameters

serial	serial device
event	event mask

6.3.3.13.3.2 rt_hw_serial_register()

```
rt_err_t rt_hw_serial_register (
            struct rt_serial_device * serial,
            const char * name,
            rt_uint32_t flag,
            void * data)
```

Register a serial device to device list.

Parameters

serial	serial device
name	device name
flag	device flag
data	device private data

Returns

rt_err_t error code

Note

This function will register a serial device to system device list, and add a device object to system object list.

6.3.3.13.3.3 rt_hw_serial_register_tty()

```
rt_err_t rt_hw_serial_register_tty (
            struct rt_serial_device * serial)
```

register a serial device to system device list and add a device object to system object list

Parameters

serial	serial device

Returns

rt_err_t error code

6.3.3.14 Serial v2

Collaboration diagram for Serial v2:

Data Structures

- struct rt_serial_rx_fifo
- struct rt_serial_tx_fifo
- struct rt_serial_device
- struct rt_uart_ops

Macros

- #define RT_DEVICE_FLAG_RX_BLOCKING 0x1000
- #define RT_DEVICE_CHECK_OPTMODE 0x20
- #define RT_HW_SERIAL_CTRL_GETC 0x01
- #define RT_SERIAL_EVENT_RX_IND 0x01
- #define RT_SERIAL_CTRL_SET_RX_TIMEOUT 0x41

Typedefs

- typedef void(∗) rt_hw_serial_rxind_hookproto_t(rt_device_t dev, rt_size_t size)

Functions

- void rt_hw_serial_isr (struct rt_serial_device ∗serial, int event)
- rt_err_t rt_hw_serial_register (struct rt_serial_device ∗serial, const char ∗name, rt_uint32_t flag, void ∗data)
- rt_err_t rt_hw_serial_register_tty (struct rt_serial_device ∗serial)

6.3.3.14.1 Detailed Description

Serial v2 driver api.

Example

```
#include <rtthread.h>
#include <rtdevice.h>

#define SAMPLE_UART_NAME       "uart1"

struct rx_msg
```

```
{
    rt_device_t dev;
    rt_size_t size;
};
static rt_device_t serial;
static struct rt_messagequeue rx_mq;

static rt_err_t uart_input(rt_device_t dev, rt_size_t size)
{
    struct rx_msg msg;
    rt_err_t result;
    msg.dev = dev;
    msg.size = size;

    result = rt_mq_send(&rx_mq, &msg, sizeof(msg));
    if (result == -RT_EFULL)
    {
        rt_kprintf("message queue full\n");
    }
    return result;
}

static void serial_thread_entry(void *parameter)
{
    struct rx_msg msg;
    rt_err_t result;
    rt_uint32_t rx_length;
    static char rx_buffer[BSP_UART1_RX_BUFSIZE + 1];

    while (1)
    {
        rt_memset(&msg, 0, sizeof(msg));
        result = rt_mq_recv(&rx_mq, &msg, sizeof(msg), RT_WAITING_FOREVER);
        if (result > 0)
        {
            rx_length = rt_device_read(msg.dev, 0, rx_buffer, msg.size);
            rx_buffer[rx_length] = '\0';
            rt_device_write(serial, 0, rx_buffer, rx_length);
            rt_kprintf("%s\n",rx_buffer);
        }
    }
}

static int uart_dma_sample(int argc, char *argv[])
{
    rt_err_t ret = RT_EOK;
    char uart_name[RT_NAME_MAX];
    static char msg_pool[256];
    char str[] = "hello RT-Thread!\r\n";

    if (argc == 2)
    {
        rt_strncpy(uart_name, argv[1], RT_NAME_MAX);
    }
    else
    {
        rt_strncpy(uart_name, SAMPLE_UART_NAME, RT_NAME_MAX);
    }

    serial = rt_device_find(uart_name);
    if (!serial)
    {
        rt_kprintf("find %s failed!\n", uart_name);
        return -RT_ERROR;
    }

    rt_mq_init(&rx_mq, "rx_mq",
               msg_pool,
               sizeof(struct rx_msg),
               sizeof(msg_pool),
               RT_IPC_FLAG_FIFO);

    rt_device_open(serial, RT_DEVICE_FLAG_RX_NON_BLOCKING | RT_DEVICE_FLAG_TX_BLOCKING);
    rt_device_set_rx_indicate(serial, uart_input);
    rt_device_write(serial, 0, str, (sizeof(str) - 1));

    rt_thread_t thread = rt_thread_create("serial", serial_thread_entry, RT_NULL, 1024, 25, 10);
    if (thread != RT_NULL)
    {
        rt_thread_startup(thread);
    }
    else
    {
        ret = RT_ERROR;
    }

    return ret;
```

```
}
MSH_CMD_EXPORT(uart_dma_sample, uart device dma sample);
```

6.3.3.14.2 Macro Definition Documentation

6.3.3.14.2.1 RT_DEVICE_FLAG_RX_BLOCKING

```
#define RT_DEVICE_FLAG_RX_BLOCKING 0x1000
```

device flag

6.3.3.14.2.2 RT_DEVICE_CHECK_OPTMODE

```
#define RT_DEVICE_CHECK_OPTMODE 0x20
```

hw device control commands

6.3.3.14.2.3 RT_HW_SERIAL_CTRL_GETC

```
#define RT_HW_SERIAL_CTRL_GETC 0x01
```

hw serial control commands

6.3.3.14.2.4 RT_SERIAL_EVENT_RX_IND

```
#define RT_SERIAL_EVENT_RX_IND 0x01
```

hw isr event

6.3.3.14.2.5 RT_SERIAL_CTRL_SET_RX_TIMEOUT

```
#define RT_SERIAL_CTRL_SET_RX_TIMEOUT 0x41
```

device commands 0x40 - special device control commands

6.3.3.14.3 Typedef Documentation

6.3.3.14.3.1 rt_hw_serial_rxind_hookproto_t

```
typedef void(*) rt_hw_serial_rxind_hookproto_t(rt_device_t dev, rt_size_t size)
```

Serial receive indicate hook function type.

6.3.3.14.4 Function Documentation

6.3.3.14.4.1 rt_hw_serial_isr()

```
void rt_hw_serial_isr (
            struct rt_serial_device * serial,
            int event)
```

Serial interrupt service routine.

Parameters

serial	serial device
event	event mask

6.3.3.14.4.2 rt_hw_serial_register()

```
rt_err_t rt_hw_serial_register (
            struct rt_serial_device * serial,
            const char * name,
            rt_uint32_t flag,
            void * data)
```

Register a serial device to device list.

Parameters

serial	serial device
name	device name
flag	device flag
data	device private data

Returns

rt_err_t error code

Note

This function will register a serial device to system device list, and add a device object to system object list.

6.3.3.14.4.3 rt_hw_serial_register_tty()

```
rt_err_t rt_hw_serial_register_tty (
            struct rt_serial_device * serial)
```

register a serial device to system device list and add a device object to system object list

Parameters

serial	serial device

Returns

rt_err_t error code

6.3.3.15 SPI

Collaboration diagram for SPI:

Data Structures

- struct rt_spi_message
- struct rt_spi_configuration
- struct rt_spi_bus
- struct rt_spi_ops
- struct rt_spi_device
- struct rt_qspi_message
- struct rt_qspi_device
- struct rt_qspi_configuration

Macros

- #define RT_SPI_CPHA (1<<0)
- #define RT_SPI_CPOL (1<<1)
- #define RT_SPI_LSB (0<<2)
- #define RT_SPI_MSB (1<<2)
- #define RT_SPI_MASTER (0<<3)
- #define RT_SPI_SLAVE (1<<3)
- #define RT_SPI_CS_HIGH (1<<4)
- #define RT_SPI_NO_CS (1<<5)
- #define RT_SPI_3WIRE (1<<6)

- #define RT_SPI_READY (1<<7)
- #define RT_SPI_MODE_0 (0 | 0)
- #define RT_SPI_MODE_1 (0 | RT_SPI_CPHA)
- #define RT_SPI_MODE_2 (RT_SPI_CPOL | 0)
- #define RT_SPI_MODE_3 (RT_SPI_CPOL | RT_SPI_CPHA)

Functions

- rt_err_t rt_spi_bus_register (struct rt_spi_bus *bus, const char *name, const struct rt_spi_ops *ops)
- rt_err_t rt_spi_bus_attach_device (struct rt_spi_device *device, const char *name, const char *bus_name, void *user_data)
- rt_err_t rt_spi_bus_detach_device (struct rt_spi_device *device)
- rt_err_t rt_spi_bus_attach_device_cspin (struct rt_spi_device *device, const char *name, const char *bus_name, rt_base_t cs_pin, void *user_data)
- rt_err_t rt_spi_bus_detach_device_cspin (struct rt_spi_device *device)
- rt_err_t rt_spi_bus_configure (struct rt_spi_device *device)
- rt_err_t rt_spi_take_bus (struct rt_spi_device *device)
- rt_err_t rt_spi_release_bus (struct rt_spi_device *device)
- rt_err_t rt_spi_take (struct rt_spi_device *device)
- rt_err_t rt_spi_release (struct rt_spi_device *device)
- rt_err_t rt_spi_configure (struct rt_spi_device *device, struct rt_spi_configuration *cfg)
- rt_err_t rt_spi_send_then_recv (struct rt_spi_device *device, const void *send_buf, rt_size_t send_length, void *recv_buf, rt_size_t recv_length)
- rt_err_t rt_spi_send_then_send (struct rt_spi_device *device, const void *send_buf1, rt_size_t send_length1, const void *send_buf2, rt_size_t send_length2)
- rt_ssize_t rt_spi_transfer (struct rt_spi_device *device, const void *send_buf, void *recv_buf, rt_size_t length)
- rt_err_t rt_spi_sendrecv8 (struct rt_spi_device *device, rt_uint8_t senddata, rt_uint8_t *recvdata)
- rt_err_t rt_spi_sendrecv16 (struct rt_spi_device *device, rt_uint16_t senddata, rt_uint16_t *recvdata)
- struct rt_spi_message * rt_spi_transfer_message (struct rt_spi_device *device, struct rt_spi_message *message)
- rt_inline rt_size_t rt_spi_recv (struct rt_spi_device *device, void *recv_buf, rt_size_t length)
- rt_inline rt_size_t rt_spi_send (struct rt_spi_device *device, const void *send_buf, rt_size_t length)
- rt_inline void rt_spi_message_append (struct rt_spi_message *list, struct rt_spi_message *message)
- rt_err_t rt_qspi_configure (struct rt_qspi_device *device, struct rt_qspi_configuration *cfg)
- rt_err_t rt_qspi_bus_register (struct rt_spi_bus *bus, const char *name, const struct rt_spi_ops *ops)
- rt_ssize_t rt_qspi_transfer_message (struct rt_qspi_device *device, struct rt_qspi_message *message)
- rt_ssize_t rt_qspi_send_then_recv (struct rt_qspi_device *device, const void *send_buf, rt_size_t send_length, void *recv_buf, rt_size_t recv_length)
- rt_ssize_t rt_qspi_send (struct rt_qspi_device *device, const void *send_buf, rt_size_t length)

6.3.3.15.1 Detailed Description

SPI driver api.

Example

```
#include <rtthread.h>
#include <rtdevice.h>

#define W25Q_SPI_DEVICE_NAME     "qspi10"

static void spi_w25q_sample(int argc, char *argv[])
{
    struct rt_spi_device *spi_dev_w25q;
    char name[RT_NAME_MAX];
    rt_uint8_t w25x_read_id = 0x90;
    rt_uint8_t id[5] = {0};

    if (argc == 2)
    {
        rt_strncpy(name, argv[1], RT_NAME_MAX);
    }
    else
```

```
    {
        rt_strncpy(name, W25Q_SPI_DEVICE_NAME, RT_NAME_MAX);
    }

    //  spi
    spi_dev_w25q = (struct rt_spi_device *)rt_device_find(name);
    if (!spi_dev_w25q)
    {
        rt_kprintf("spi sample run failed! can't find %s device!\n", name);
    }
    else
    {
        // 1 rt_spi_send_then_recv()ID
        rt_spi_send_then_recv(spi_dev_w25q, &w25x_read_id, 1, id, 5);
        rt_kprintf("use rt_spi_send_then_recv() read w25q ID is:%x%x\n", id[3], id[4]);

        // 2 rt_spi_transfer_message()ID
        struct rt_spi_message msg1, msg2;

        msg1.send_buf   = &w25x_read_id;
        msg1.recv_buf   = RT_NULL;
        msg1.length     = 1;
        msg1.cs_take    = 1;
        msg1.cs_release = 0;
        msg1.next       = &msg2;

        msg2.send_buf   = RT_NULL;
        msg2.recv_buf   = id;
        msg2.length     = 5;
        msg2.cs_take    = 0;
        msg2.cs_release = 1;
        msg2.next       = RT_NULL;

        rt_spi_transfer_message(spi_dev_w25q, &msg1);
        rt_kprintf("use rt_spi_transfer_message() read w25q ID is:%x%x\n", id[3], id[4]);

    }
}
//  msh
MSH_CMD_EXPORT(spi_w25q_sample, spi w25q sample);
```

6.3.3.15.2 Macro Definition Documentation

6.3.3.15.2.1 RT_SPI_CPHA

```
#define RT_SPI_CPHA (1<<0)
```

At CPOL=0 the base value of the clock is zero

- For CPHA=0, data are captured on the clock's rising edge (low->high transition) and data are propagated on a falling edge (high->low clock transition).
- For CPHA=1, data are captured on the clock's falling edge and data are propagated on a rising edge. At CPOL=1 the base value of the clock is one (inversion of CPOL=0)
- For CPHA=0, data are captured on clock's falling edge and data are propagated on a rising edge.
- For CPHA=1, data are captured on clock's rising edge and data are propagated on a falling edge. bit[0]:CPHA, clock phase

6.3.3.15.2.2 RT_SPI_CPOL

```
#define RT_SPI_CPOL (1<<1)
```

bit[1]:CPOL, clock polarity

6.3.3.15.2.3 RT_SPI_LSB

```
#define RT_SPI_LSB (0<<2)
```

bit[2]: 0-LSB

6.3.3.15.2.4 RT_SPI_MSB

```
#define RT_SPI_MSB (1<<2)
```

bit[2]: 1-MSB

6.3.3.15.2.5 RT_SPI_MASTER

```
#define RT_SPI_MASTER (0<<3)
```
SPI master device

6.3.3.15.2.6 RT_SPI_SLAVE

```
#define RT_SPI_SLAVE (1<<3)
```
SPI slave device

6.3.3.15.2.7 RT_SPI_CS_HIGH

```
#define RT_SPI_CS_HIGH (1<<4)
```
Chipselect active high

6.3.3.15.2.8 RT_SPI_NO_CS

```
#define RT_SPI_NO_CS (1<<5)
```
No chipselect

6.3.3.15.2.9 RT_SPI_3WIRE

```
#define RT_SPI_3WIRE (1<<6)
```
SI/SO pin shared

6.3.3.15.2.10 RT_SPI_READY

```
#define RT_SPI_READY (1<<7)
```
Slave pulls low to pause

6.3.3.15.2.11 RT_SPI_MODE_0

```
#define RT_SPI_MODE_0 (0 | 0)
```
CPOL = 0, CPHA = 0

6.3.3.15.2.12 RT_SPI_MODE_1

```
#define RT_SPI_MODE_1 (0 | RT_SPI_CPHA)
```
CPOL = 0, CPHA = 1

6.3.3.15.2.13 RT_SPI_MODE_2

```
#define RT_SPI_MODE_2 (RT_SPI_CPOL | 0)
```
CPOL = 1, CPHA = 0

6.3.3.15.2.14 RT_SPI_MODE_3

```
#define RT_SPI_MODE_3 (RT_SPI_CPOL | RT_SPI_CPHA)
```
CPOL = 1, CPHA = 1

6.3.3.15.3 Function Documentation

6.3.3.15.3.1 rt_spi_bus_register()

```
rt_err_t rt_spi_bus_register (
            struct rt_spi_bus * bus,
            const char * name,
            const struct rt_spi_ops * ops)
```
register a SPI bus

Parameters

bus	the SPI bus
name	the name of SPI bus
ops	the operations of SPI bus

Returns

rt_err_t error code

6.3.3.15.3.2 rt_spi_bus_attach_device()

```
rt_err_t rt_spi_bus_attach_device (
             struct rt_spi_device * device,
             const char * name,
             const char * bus_name,
             void * user_data)
```

attach a device on SPI bus

Parameters

device	the SPI device
name	the name of SPI device
bus_name	the name of SPI bus
user_data	the user data of SPI device

Returns

rt_err_t error code

6.3.3.15.3.3 rt_spi_bus_detach_device()

```
rt_err_t rt_spi_bus_detach_device (
             struct rt_spi_device * device)
```

Detach a device from the SPI bus.

This function serves as the high-level API to detach a SPI device from its bus. It unregisters the device from the device framework and ensures all associated resources, such as the chip select pin, are properly released by calling the underlying implementation.

Parameters

device	The SPI device to be detached.

Returns

rt_err_t The result of the operation. RT_EOK on success, otherwise an error code.

6.3.3.15.3.4 rt_spi_bus_attach_device_cspin()

```
rt_err_t rt_spi_bus_attach_device_cspin (
             struct rt_spi_device * device,
             const char * name,
             const char * bus_name,
```

```
            rt_base_t cs_pin,
            void * user_data)
```

attach a device on SPI bus with CS pin

Parameters

device	the SPI device
name	the name of SPI device
bus_name	the name of SPI bus
cs_pin	the CS pin of SPI device
user_data	the user data of SPI device

Returns

rt_err_t error code

6.3.3.15.3.5 rt_spi_bus_detach_device_cspin()

```
rt_err_t rt_spi_bus_detach_device_cspin (
            struct rt_spi_device * device)
```

Detach a device from the SPI bus and release its CS pin.
This function provides the low-level implementation for detaching a device from the SPI bus. It specifically handles the operations for the chip select (CS) pin, resetting it to input mode to release it. This function is typically called by the higher-level rt_spi_bus_detach_device() and should not be called directly by the user application.

Parameters

device	The SPI device to be detached.

Returns

rt_err_t The result of the operation. RT_EOK on success, otherwise an error code.

6.3.3.15.3.6 rt_spi_bus_configure()

```
rt_err_t rt_spi_bus_configure (
            struct rt_spi_device * device)
```

Reconfigure the SPI bus for the specified device.

Parameters

device	Pointer to the SPI device attached to the SPI bus.

Return values

RT_EOK	if the SPI device was successfully released and the bus was configured. RT_EBUSY if the SPI bus is currently in use; the new configuration will take effect once the device releases the bus. Other return values indicate failure to configure the SPI bus due to various reasons.

Note

If the configuration of the SPI device has been updated and requires bus re-initialization, call this function directly. This function will reconfigure the SPI bus for the specified device. If this is the first time to initialize the SPI device, please call rt_spi_configure or rt_qspi_configure. This function is used to reconfigure the SPI bus when the SPI device is already in use. For further details, refer to: https://github.com/RT-Thread/rt-thread/pull/8528

6.3.3.15.3.7 rt_spi_take_bus()

```
rt_err_t rt_spi_take_bus (
            struct rt_spi_device * device)
```

This function takes SPI bus.

Parameters

device	the SPI device attached to SPI bus

Returns

RT_EOK on taken SPI bus successfully. others on taken SPI bus failed.

6.3.3.15.3.8 rt_spi_release_bus()

```
rt_err_t rt_spi_release_bus (
            struct rt_spi_device * device)
```

This function releases SPI bus.

Parameters

device	the SPI device attached to SPI bus

Returns

RT_EOK on release SPI bus successfully.

6.3.3.15.3.9 rt_spi_take()

```
rt_err_t rt_spi_take (
            struct rt_spi_device * device)
```

This function take SPI device (takes CS of SPI device).

Parameters

device	the SPI device attached to SPI bus

Returns

RT_EOK on release SPI bus successfully. others on taken SPI bus failed.

6.3.3.15.3.10 rt_spi_release()

```
rt_err_t rt_spi_release (
            struct rt_spi_device * device)
```

This function releases SPI device (releases CS of SPI device).

Parameters

device	the SPI device attached to SPI bus

Returns

RT_EOK on release SPI device successfully.

6.3.3.15.3.11 rt_spi_configure()

```
rt_err_t rt_spi_configure (
            struct rt_spi_device * device,
            struct rt_spi_configuration * cfg)
```

This function can set configuration on SPI device.

Parameters

device	the SPI device attached to SPI bus
cfg	the configuration pointer.

Return values

RT_EOK	on release SPI device successfully. RT_EBUSY is not an error condition and the configuration will take effect once the device has the bus others on taken SPI bus failed.

6.3.3.15.3.12 rt_spi_send_then_recv()

```
rt_err_t rt_spi_send_then_recv (
            struct rt_spi_device * device,
            const void * send_buf,
            rt_size_t send_length,
            void * recv_buf,
            rt_size_t recv_length)
```

This function can send data then receive data from SPI device.

Parameters

device	the SPI device attached to SPI bus
send_buf	the buffer to be transmitted to SPI device.
send_length	the number of data to be transmitted.
recv_buf	the buffer to be recivied from SPI device.
recv_length	the data to be recivied.

Returns

rt_err_t error code

6.3.3.15.3.13 rt_spi_send_then_send()

```
rt_err_t rt_spi_send_then_send (
            struct rt_spi_device * device,
```

```
            const void * send_buf1,
            rt_size_t send_length1,
            const void * send_buf2,
            rt_size_t send_length2)
```

This function can send data then send data from SPI device.

Parameters

device	the SPI device attached to SPI bus
send_buf1	the buffer to be transmitted to SPI device.
send_length1	the number of data to be transmitted.
send_buf2	the buffer to be transmitted to SPI device.
send_length2	the number of data to be transmitted.

Returns

the status of transmit.

6.3.3.15.3.14 rt_spi_transfer()

```
rt_ssize_t rt_spi_transfer (
            struct rt_spi_device * device,
            const void * send_buf,
            void * recv_buf,
            rt_size_t length)
```

This function transmits data to SPI device.

Parameters

device	the SPI device attached to SPI bus
send_buf	the buffer to be transmitted to SPI device.
recv_buf	the buffer to save received data from SPI device.
length	the length of transmitted data.

Returns

the actual length of transmitted.

6.3.3.15.3.15 rt_spi_sendrecv8()

```
rt_err_t rt_spi_sendrecv8 (
            struct rt_spi_device * device,
            rt_uint8_t senddata,
            rt_uint8_t * recvdata)
```

The SPI device transmits 8 bytes of data.

Parameters

device	the SPI device attached to SPI bus
senddata	send data buffer
recvdata	receive data buffer

Returns

rt_err_t error code

6.3.3.15.3.16 rt_spi_sendrecv16()

```
rt_err_t rt_spi_sendrecv16 (
            struct rt_spi_device * device,
            rt_uint16_t senddata,
            rt_uint16_t * recvdata)
```

The SPI device transmits 16 bytes of data.

Parameters

device	the SPI device attached to SPI bus
senddata	send data buffer
recvdata	receive data buffer

Returns

rt_err_t error code

6.3.3.15.3.17 rt_spi_transfer_message()

```
struct rt_spi_message * rt_spi_transfer_message (
            struct rt_spi_device * device,
            struct rt_spi_message * message)
```

This function transfers a message list to the SPI device.

Parameters

device	the SPI device attached to SPI bus
message	the message list to be transmitted to SPI device

Returns

RT_NULL if transmits message list successfully, SPI message which be transmitted failed.

6.3.3.15.3.18 rt_spi_recv()

```
rt_inline rt_size_t rt_spi_recv (
            struct rt_spi_device * device,
            void * recv_buf,
            rt_size_t length)
```

This function receives data from SPI device.

Parameters

device	the SPI device attached to SPI bus
recv_buf	the buffer to be recivied from SPI device.
length	the data to be recivied.

Returns

the actual length of received.

6.3.3.15.3.19 rt_spi_send()

```
rt_inline rt_size_t rt_spi_send (
            struct rt_spi_device * device,
            const void * send_buf,
            rt_size_t length)
```

This function sends data to SPI device.

Parameters

device	the SPI device attached to SPI bus
send_buf	the buffer to be transmitted to SPI device.
length	the number of data to be transmitted.

Returns

the actual length of send.

6.3.3.15.3.20 rt_spi_message_append()

```
rt_inline void rt_spi_message_append (
            struct rt_spi_message * list,
            struct rt_spi_message * message)
```

This function appends a message to the SPI message list.

Parameters

list	the SPI message list header.
message	the message pointer to be appended to the message list.

6.3.3.15.3.21 rt_qspi_configure()

```
rt_err_t rt_qspi_configure (
            struct rt_qspi_device * device,
            struct rt_qspi_configuration * cfg)
```

This function can set configuration on QSPI device.

Parameters

device	the QSPI device attached to QSPI bus.
cfg	the configuration pointer.

Returns

the actual length of transmitted.

6.3.3.15.3.22 rt_qspi_bus_register()

```
rt_err_t rt_qspi_bus_register (
            struct rt_spi_bus * bus,
```

```
            const char * name,
            const struct rt_spi_ops * ops)
```

This function can register a SPI bus for QSPI mode.

Parameters

bus	the SPI bus for QSPI mode.
name	The name of the spi bus.
ops	the SPI bus instance to be registered.

Returns

the actual length of transmitted.

6.3.3.15.3.23 rt_qspi_transfer_message()

```
rt_ssize_t rt_qspi_transfer_message (
            struct rt_qspi_device * device,
            struct rt_qspi_message * message)
```

This function transmits data to QSPI device.

Parameters

device	the QSPI device attached to QSPI bus.
message	the message pointer.

Returns

the actual length of transmitted.

6.3.3.15.3.24 rt_qspi_send_then_recv()

```
rt_ssize_t rt_qspi_send_then_recv (
            struct rt_qspi_device * device,
            const void * send_buf,
            rt_size_t send_length,
            void * recv_buf,
            rt_size_t recv_length)
```

This function can send data then receive data from QSPI device.

Parameters

device	the QSPI device attached to QSPI bus.
send_buf	the buffer to be transmitted to QSPI device.
send_length	the number of data to be transmitted.
recv_buf	the buffer to be recivied from QSPI device.
recv_length	the data to be recivied.

Returns

the status of transmit.

6.3.3.15.3.25 rt_qspi_send()

```
rt_ssize_t rt_qspi_send (
              struct rt_qspi_device * device,
              const void * send_buf,
              rt_size_t length)
```

This function can send data to QSPI device.

Parameters

device	the QSPI device attached to QSPI bus.
send_buf	the buffer to be transmitted to QSPI device.
length	the number of data to be transmitted.

Returns

the status of transmit.

6.3.3.16 Touch

Collaboration diagram for Touch:

Data Structures

- struct rt_touch_info
- struct rt_touch_config
- struct rt_touch_device
- struct rt_touch_data
- struct rt_touch_ops

Functions

- int rt_hw_touch_register (rt_touch_t touch, const char *name, rt_uint32_t flag, void *data)
- void rt_hw_touch_isr (rt_touch_t touch)

6.3.3.16.1 Detailed Description

Touch driver api.

Example

```
#include <rtthread.h>
#include "rtdevice.h"

static rt_thread_t  gt9147_thread = RT_NULL;
static rt_sem_t     gt9147_sem = RT_NULL;
static rt_device_t  dev = RT_NULL;
static struct       rt_touch_data *read_data;

//
static void gt9147_entry(void *parameter)
{
    struct rt_touch_data *read_data;
    read_data = (struct rt_touch_data *)rt_malloc(sizeof(struct rt_touch_data) * 5);

    while (1)
    {
        //
        rt_sem_take(gt9147_sem, RT_WAITING_FOREVER);
        //
        if (rt_device_read(dev, 0, read_data, 5) == 5)
        {
            for (rt_uint8_t i = 0; i < 5; i++)
            {
                if (read_data[i].event == RT_TOUCH_EVENT_DOWN || read_data[i].event == RT_TOUCH_EVENT_MOVE)
                {
                    rt_kprintf("%d %d %d %d %d\n",
                               read_data[i].track_id,
                               read_data[i].x_coordinate,
```

```
                                read_data[i].y_coordinate,
                                read_data[i].timestamp,
                                read_data[i].width);
                }
            }
        }
        //
        rt_device_control(dev, RT_TOUCH_CTRL_ENABLE_INT, RT_NULL);
    }
}

//
static rt_err_t rx_callback(rt_device_t dev, rt_size_t size)
{
    //
    rt_device_control(dev, RT_TOUCH_CTRL_DISABLE_INT, RT_NULL);
    //
    rt_sem_release(gt9147_sem);
    return 0;
}

static int gt9147_sample(void)
{
    //  Touch
    dev = rt_device_find("touch");

    if (dev == RT_NULL)
    {
        rt_kprintf("can't find device:%s\n", "touch");
        return -1;
    }
    //
    if (rt_device_open(dev, RT_DEVICE_FLAG_INT_RX) != RT_EOK)
    {
        rt_kprintf("open device failed!");
        return -1;
    }
    //
    rt_device_set_rx_indicate(dev, rx_callback);
    //
    gt9147_sem = rt_sem_create("dsem", 0, RT_IPC_FLAG_PRIO);

    if (gt9147_sem == RT_NULL)
    {
        rt_kprintf("create dynamic semaphore failed.\n");
        return -1;
    }
    //
    gt9147_thread = rt_thread_create("thread1",
                                     gt9147_entry,
                                     RT_NULL,
                                     THREAD_STACK_SIZE,
                                     THREAD_PRIORITY,
                                     THREAD_TIMESLICE);
    //
    if (gt9147_thread != RT_NULL)
        rt_thread_startup(gt9147_thread);

    return 0;
}
MSH_CMD_EXPORT(gt9147_sample, gt9147 sample);
```

6.3.3.16.2 Function Documentation

6.3.3.16.2.1 rt_hw_touch_register()

```
int rt_hw_touch_register (
            rt_touch_t touch,
            const char * name,
            rt_uint32_t flag,
            void * data)
```

register a touch device

Parameters

touch	the touch device
name	the name of touch device
flag	the flag of touch device

data	the user data of touch device

Returns

rt_err_t error code

6.3.3.16.2.2 rt_hw_touch_isr()

```
void rt_hw_touch_isr (
            rt_touch_t touch)
```

Touch irq handle.

Parameters

touch	the touch device

Note

If you doesn't use pin device. you must call this function in your touch irq callback

6.3.3.17 blk

Collaboration diagram for blk:

Data Structures

- struct rt_blk_disk
- struct rt_blk_disk_ops
- struct rt_blk_device

Functions

- rt_err_t rt_hw_blk_disk_register (struct rt_blk_disk ∗disk)
- rt_err_t rt_hw_blk_disk_unregister (struct rt_blk_disk ∗disk)
- rt_err_t rt_blk_disk_probe_partition (struct rt_blk_disk ∗disk)
- rt_ssize_t rt_blk_disk_get_capacity (struct rt_blk_disk ∗disk)
- rt_ssize_t rt_blk_disk_get_logical_block_size (struct rt_blk_disk ∗disk)

6.3.3.17.1 Detailed Description

blk driver api

6.3.3.17.2 Function Documentation

6.3.3.17.2.1 rt_hw_blk_disk_register()

```
rt_err_t rt_hw_blk_disk_register (
            struct rt_blk_disk * disk)
```

Register the blk disk device.

Parameters

disk	Point to blk disk

Returns

rt_err_t error code

6.3.3.17.2.2 rt_hw_blk_disk_unregister()

```
rt_err_t rt_hw_blk_disk_unregister (
            struct rt_blk_disk * disk)
```

Unregister the blk disk device.

Parameters

disk	Point to blk disk

Returns

rt_err_t error code

6.3.3.17.2.3 rt_blk_disk_probe_partition()

```
rt_err_t rt_blk_disk_probe_partition (
            struct rt_blk_disk * disk)
```

Probe and register the blk disk partition.

Parameters

disk	Point to blk disk

Returns

rt_err_t error code

6.3.3.17.2.4 rt_blk_disk_get_capacity()

```
rt_ssize_t rt_blk_disk_get_capacity (
            struct rt_blk_disk * disk)
```

Get the blk disk capacity.

Parameters

disk	Point to blk disk

Returns

rt_ssize_t sector count or error code

6.3.3.17.2.5 rt_blk_disk_get_logical_block_size()

```
rt_ssize_t rt_blk_disk_get_logical_block_size (
            struct rt_blk_disk * disk)
```

Get the sector size.

Parameters

disk	Point to blk disk

Returns

rt_ssize_t bytes per sector or error code

6.3.4 Finsh Shell

Collaboration diagram for Finsh Shell:

Macros

- #define FINSH_FUNCTION_EXPORT(name, desc)
- #define MSH_CMD_EXPORT(...)
- #define MSH_CMD_EXPORT_ALIAS(...)

Functions

- rt_uint32_t finsh_get_prompt_mode (void)
- void finsh_set_prompt_mode (rt_uint32_t prompt_mode)
- void finsh_set_device (const char *device_name)
- const char * finsh_get_device ()
- void finsh_set_echo (rt_uint32_t echo)
- rt_uint32_t finsh_get_echo ()
- void finsh_thread_entry_sethook (void(*hook)(void))

6.3.4.1 Detailed Description

finsh shell is a user command shell in RT-Thread RTOS.
finsh shell is a user command shell in RT-Thread RTOS, which is a shell can accept C-expression like syntax in command. From finsh shell, user can access system area, such as memory, variables and function by input C-expression in command.
There is a shell thread, which named as "tshell", in the finsh shell, it read user command from console device, and then invokes system function or access system variable to output result (by rt_kprintf).

6.3.4.2 Macro Definition Documentation

6.3.4.2.1 FINSH_FUNCTION_EXPORT

```
#define FINSH_FUNCTION_EXPORT(
                name,
                desc)
```

This macro exports a system function to finsh shell.

Parameters

in	*name*	Name of function.
in	*desc*	Description of function, which will show in help.

6.3.4.2.2 MSH_CMD_EXPORT

```
#define MSH_CMD_EXPORT(
                ...)
```

Value:

```
__MSH_GET_MACRO(__VA_ARGS__, _MSH_FUNCTION_CMD2_OPT,    \
    _MSH_FUNCTION_CMD2)(__VA_ARGS__)
```

Exports a command to module shell.

Parameters

[in] **command** Name of the command.
[in] **desc** Description of the command, which will show in help list.
[in] **opt** This is an option, enter any content to enable option completion

Note

This macro can be used in two ways:

```
MSH_CMD_EXPORT(command, desc)
```

or

```
MSH_CMD_EXPORT(command, desc, opt)
```

6.3.4.2.3 MSH_CMD_EXPORT_ALIAS

```
#define MSH_CMD_EXPORT_ALIAS(
                ...)
```

Value:

```
__MSH_GET_EXPORT_MACRO(__VA_ARGS__, _MSH_FUNCTION_EXPORT_CMD3_OPT,      \
        _MSH_FUNCTION_EXPORT_CMD3)(__VA_ARGS__)
```

Exports a command with alias to module shell.

Parameters

[in] **command** Name of the command.
[in] **alias** Alias of the command.
[in] **desc** Description of the command, which will show in help list.
[in] **opt** An option, enter any content to enable option completion.

Note

This macro can be used in two ways:

```
#define MSH_CMD_EXPORT_ALIAS(command, alias, desc)
```

or

```
#define MSH_CMD_EXPORT_ALIAS(command, alias, desc, opt)
```

6.3.4.3 Function Documentation

6.3.4.3.1 finsh_get_prompt_mode()

```
rt_uint32_t finsh_get_prompt_mode (
              void )
```

This function get the prompt mode of finsh shell.

Returns

prompt the prompt mode, 0 disable prompt mode, other values enable prompt mode.

6.3.4.3.2 finsh_set_prompt_mode()

```
void finsh_set_prompt_mode (
              rt_uint32_t prompt_mode)
```

This function set the prompt mode of finsh shell.
The parameter 0 disable prompt mode, other values enable prompt mode.

Parameters

prompt_mode	the prompt mode

6.3.4.3.3 finsh_set_device()

```
void finsh_set_device (
              const char * device_name)
```

This function sets the input device of finsh shell.

Parameters

device_name	the name of new input device.

6.3.4.3.4 finsh_get_device()

```
const char * finsh_get_device (
             void )
```

This function returns current finsh shell input device.

Returns

the finsh shell input device name is returned.

6.3.4.3.5 finsh_set_echo()

```
void finsh_set_echo (
             rt_uint32_t echo)
```

This function set the echo mode of finsh shell.
FINSH_OPTION_ECHO=0x01 is echo mode, other values are none-echo mode.

Parameters

echo	the echo mode

6.3.4.3.6 finsh_get_echo()

```
rt_uint32_t finsh_get_echo (
             void )
```

This function gets the echo mode of finsh shell.

Returns

the echo mode

6.3.4.3.7 finsh_thread_entry_sethook()

```
void finsh_thread_entry_sethook (
             void(* hook )(void))
```

This function set a hook function at the entry of finsh thread.

Parameters

hook	the function point to be called

6.3.5 Application Module

Collaboration diagram for Application Module: Application Module is a feature let user to execute application in RT-Thread RTOS.
Application Module is implemented as dynamic object loader, but it can handle the dependences relationship between application and dynamic library, moreover, it also can handle the kernel object destroy and memory release issue when application (abnormally) exit.

6.4 Hardware Related Package

Functions

- rt_base_t rt_hw_interrupt_disable (void)

- void rt_hw_interrupt_enable (rt_base_t level)
- void rt_hw_interrupt_init (void)
- void rt_hw_interrupt_mask (int vector)
- void rt_hw_interrupt_umask (int vector)
- void rt_hw_interrupt_install (int vector, rt_isr_handler_t new_handler, rt_isr_handler_t *old_handler)

6.4.1 Detailed Description

Hardware Related Package includes board support package(BSP) and CSP(Chip Support Package).
Board Support Package(BSP) is the hardware related wrapper, for example, peripherals in board, the pinmux setting etc. In RT-Thread RTOS, the bsp is placed under bsp directory.
Chip Support Package(CSP) is a software set that contains chip specific software. A CSP usually includes operating system porting and peripheral device drivers inside chip. In RT-Thread RTOS, the csp is placed under libcpu directory.

6.4.2 Function Documentation

6.4.2.1 rt_hw_interrupt_disable()

```
rt_base_t rt_hw_interrupt_disable (
            void )
```

This function will return current system interrupt status and disable system interrupt.

Returns

the current system interrupt status.

6.4.2.2 rt_hw_interrupt_enable()

```
void rt_hw_interrupt_enable (
            rt_base_t level)
```

This function will set the specified interrupt status, which shall saved by rt_hw_intterrupt_disable function. If the saved interrupt status is interrupt opened, this function will open system interrupt status.

Parameters

level	the interrupt status to be set.

6.4.2.3 rt_hw_interrupt_init()

```
void rt_hw_interrupt_init (
            void )
```

This function initializes interrupt.

6.4.2.4 rt_hw_interrupt_mask()

```
void rt_hw_interrupt_mask (
            int vector)
```

This function masks the specified interrupt.

Parameters

vector	the interrupt number to be masked.

Note

not all of platform provide this function.

6.4.2.5 rt_hw_interrupt_umask()

```
void rt_hw_interrupt_umask (
            int vector)
```

This function umasks the specified interrupt.

Parameters

vector	the interrupt number to be unmasked.

Note

not all of platform provide this function.

6.4.2.6 rt_hw_interrupt_install()

```
void rt_hw_interrupt_install (
            int vector,
            rt_isr_handler_t new_handler,
            rt_isr_handler_t * old_handler)
```

This function will install specified interrupt handler.

Parameters

vector	the interrupt number to be installed.
new_handler	the new interrupt handler.
old_handler	the old interrupt handler. This parameter can be RT_NULL.

Note

not all of platform provide this function.

6.5 Doxygen Example

Collaboration diagram for Doxygen Example:

Topics

- Doxygen Example of Enumeration
- Doxygen Example of Groups
- Doxygen Example of Macro
- Doxygen Example of Structure
- Doxygen Example of Typedef
- Doxygen Example of Union
- Doxygen Example of Function

6.5.1 Detailed Description

Introduce how to write doxygen documentation.
See How to write doxygen documentation for RT-Thread for more details.

6.5.2 Doxygen Example of Enumeration

Collaboration diagram for Doxygen Example of Enumeration:

Enumerations

- enum doxygen_example_enum { V1 , V2 }

6.5.2.1 Detailed Description

Doxygen Example of Enumeration.

6.5.2.2 Enumeration Type Documentation

6.5.2.2.1 doxygen_example_enum

```
enum doxygen_example_enum
```

Brief description of this enumeration.

Enumerator

V1	description for value 1
V2	description for value 2

6.5.3 Doxygen Example of Groups

Collaboration diagram for Doxygen Example of Groups:

Macros

- #define DOXYGEN_EXAMPLE_CONST_C 100
- #define DOXYGEN_EXAMPLE_CONST_D 200

Enumerations

- enum doxygen_example_enum_2 { V1 , V2 }

6.5.3.1 Detailed Description

Define a sub group of Doxygen Example.
All members of this group will be displayed in one HTML page.

6.5.3.2 Macro Definition Documentation

6.5.3.2.1 DOXYGEN_EXAMPLE_CONST_C

```
#define DOXYGEN_EXAMPLE_CONST_C 100
```

Description of macro const C

6.5.3.2.2 DOXYGEN_EXAMPLE_CONST_D

```
#define DOXYGEN_EXAMPLE_CONST_D 200
```

Description of macro const D

6.5.3.3 Enumeration Type Documentation

6.5.3.3.1 doxygen_example_enum_2

```
enum doxygen_example_enum_2
```

Brief description of this enumeration.
We use `@ingroup` to add this enum to the group_doxygen_example_sub separately.

Enumerator

V1	description for value 1
V2	description for value 2

6.5.4 Doxygen Example of Macro

Collaboration diagram for Doxygen Example of Macro:

Topics

- Group A of Macros
- Group B of Macros

Macros

- #define DOXYGEN_EXAMPLE_CONST_A 100
- #define DOXYGEN_EXAMPLE_CONST_B 200
- #define DOXYGEN_EXAMPLE_ABS(x)

6.5.4.1 Detailed Description

Doxygen Example of Macro.

6.5.4.2 Macro Definition Documentation

6.5.4.2.1 DOXYGEN_EXAMPLE_CONST_A

```
#define DOXYGEN_EXAMPLE_CONST_A 100
```

Description of macro const A

6.5.4.2.2 DOXYGEN_EXAMPLE_CONST_B

```
#define DOXYGEN_EXAMPLE_CONST_B 200
```

Description of macro const B

6.5.4.2.3 DOXYGEN_EXAMPLE_ABS

```
#define DOXYGEN_EXAMPLE_ABS(
            x)
```

Value:

```
(((x)>0)?(x):-(x))
```

Computes the absolute value of its argument *x*.

Parameters

in	*x*	input value.

Returns

absolute value of *x*.

6.5.4.3 Group A of Macros

Collaboration diagram for Group A of Macros:

Macros

- #define DOXYGEN_EXAMPLE_GROUP_A_X 0x0000
- #define DOXYGEN_EXAMPLE_GROUP_A_Y 0x0001

6.5.4.3.1 Detailed Description

Doxygen Example of Macros grouped in A.

6.5.4.3.2 Macro Definition Documentation

6.5.4.3.2.1 DOXYGEN_EXAMPLE_GROUP_A_X

```
#define DOXYGEN_EXAMPLE_GROUP_A_X 0x0000
```

Description of X in group A

6.5.4.3.2.2 DOXYGEN_EXAMPLE_GROUP_A_Y

```
#define DOXYGEN_EXAMPLE_GROUP_A_Y 0x0001
```

Description of Y in group A

6.5.4.4 Group B of Macros

Collaboration diagram for Group B of Macros:

Macros

- #define DOXYGEN_EXAMPLE_GROUP_B_X 0x0000
- #define DOXYGEN_EXAMPLE_GROUP_B_Y 0x0001

6.5.4.4.1 Detailed Description

Doxygen Example of Macros grouped in B.

6.5.4.4.2 Macro Definition Documentation

6.5.4.4.2.1 DOXYGEN_EXAMPLE_GROUP_B_X

```
#define DOXYGEN_EXAMPLE_GROUP_B_X 0x0000
```

Description of X in group B

6.5.4.4.2.2 DOXYGEN_EXAMPLE_GROUP_B_Y

```
#define DOXYGEN_EXAMPLE_GROUP_B_Y 0x0001
```

Description of Y in group B

6.5.5 Doxygen Example of Structure

Collaboration diagram for Doxygen Example of Structure:

Data Structures

- struct dogygen_example_struct
- struct dogygen_example_struct_another
- struct dogygen_example_struct_another2

6.5.5.1 Detailed Description

Doxygen Example of Structure.

6.5.6 Doxygen Example of Typedef

Collaboration diagram for Doxygen Example of Typedef:

Typedefs

- typedef struct dogygen_example_struct dogygen_example_struct_t
- typedef struct dogygen_example_struct_another dogygen_example_struct_another_t
- typedef enum doxygen_example_enum doxygen_example_enum_t

6.5.6.1 Detailed Description

Doxygen Example of Typedef.

6.5.6.2 Typedef Documentation

6.5.6.2.1 dogygen_example_struct_t

```
typedef struct dogygen_example_struct dogygen_example_struct_t
```

Alias of `struct dogygen_example_struct`.

6.5.6.2.2 dogygen_example_struct_another_t

```
typedef struct dogygen_example_struct_another dogygen_example_struct_another_t
```

Alias of `struct dogygen_example_struct_another`.

6.5.6.2.3 doxygen_example_enum_t

```
typedef enum doxygen_example_enum doxygen_example_enum_t
```

Alias of `enum doxygen_example_enum`.

6.5.7 Doxygen Example of Union

Collaboration diagram for Doxygen Example of Union:

Data Structures

- union doxygen_example_union

6.5.7.1 Detailed Description

Doxygen Example of Union.

6.5.8 Doxygen Example of Function

Collaboration diagram for Doxygen Example of Function:

Functions

- void doxygen_example_func_foo (int a, int b)
- int doxygen_example_func_bar (int a, int *b)

6.5.8.1 Detailed Description

Doxygen Example of Function.

6.5.8.2 Function Documentation

6.5.8.2.1 doxygen_example_func_foo()

```
void doxygen_example_func_foo (
            int a,
            int b)
```

Brief description for the function.
Detailed description starts here, one line or multiple lines. Blah blah blah...

Parameters

in	*a*	Description of param a
in	*b*	Description of param b

Note

This is a note for this structure, blah blah blah...

6.5.8.2.2 doxygen_example_func_bar()

```
int doxygen_example_func_bar (
            int a,
            int * b)
```

Brief description for the function.
Detailed description starts here, one line or multiple lines. Blah blah blah...

Parameters

in	*a*	Description of param a
out	*b*	Description of param b

Returns

the return value, 0 for success, -1 for failure

Note

This is a note for this structure, blah blah blah...

Chapter 7

Data Structure Documentation

7.1 can_configure Struct Reference

`#include <dev_can.h>`

Data Fields

- rt_uint32_t baud_rate
- rt_uint32_t msgboxsz
- rt_uint32_t sndboxnumber
- rt_uint32_t mode: 8
- rt_uint32_t privmode: 8
- rt_uint32_t reserved: 16
- rt_uint32_t ticks

7.1.1 Detailed Description

CAN device configuration structure.

7.1.2 Field Documentation

7.1.2.1 baud_rate

`rt_uint32_t can_configure::baud_rate`
The baud rate for the arbitration phase (e.g., CAN500kBaud).

7.1.2.2 msgboxsz

`rt_uint32_t can_configure::msgboxsz`
The size of the software receive buffer (in number of messages).

7.1.2.3 sndboxnumber

`rt_uint32_t can_configure::sndboxnumber`
The number of hardware mailboxes used for blocking send operations.

7.1.2.4 mode

`rt_uint32_t can_configure::mode`
The CAN operation mode (e.g., RT_CAN_MODE_NORMAL, RT_CAN_MODE_LOOPBACK).

7.1.2.5 privmode

`rt_uint32_t can_configure::privmode`
Private mode flag. If set, the `priv` field of `rt_can_msg` specifies the hardware mailbox.

7.1.2.6 reserved

`rt_uint32_t can_configure::reserved`
Reserved for future use.

7.1.2.7 ticks

`rt_uint32_t can_configure::ticks`
The period in OS ticks for the status-checking timer.

7.2 dfs_seq_file Struct Reference

`#include <dfs_seq_file.h>`

7.2.1 Detailed Description

Sequence file control structure

7.3 dfs_seq_ops Struct Reference

`#include <dfs_seq_file.h>`

7.3.1 Detailed Description

Sequence file operations structure

7.4 dogygen_example_struct Struct Reference

`#include <struct.h>`

Data Fields

- int m1
- int m2
- int m3
- int m4
- int m5

7.4.1 Detailed Description

Brief description this structure.
Detailed description starts here, one line or multiple lines. Blah blah blah...

Note

This is a note for this structure, blah blah blah...

7.4.2 Field Documentation

7.4.2.1 m1

`int dogygen_example_struct::m1`
Some documentation for member 'm1'... Multiple lines ... Note the "multi-line" here refers to the code. Whether it is displayed in multiple lines on the specific HTML page depends on the browser rendering.

Note

We can also embed note for member 'm1'...

7.4.2.2 m2

```
int dogygen_example_struct::m2
```
Some documentation for member 'm2'...

7.4.2.3 m3

```
int dogygen_example_struct::m3
```
Some documentation for member 'm3'...

7.4.2.4 m4

```
int dogygen_example_struct::m4
```
Some documentation for member 'm4'...

7.4.2.5 m5

```
int dogygen_example_struct::m5
```
Some documentation for member 'm5'...

7.5 dogygen_example_struct_another Struct Reference

```
#include <struct.h>
```

Data Fields

- int(∗) m1 (int ∗param1, int param2, int param3, int param4)
- int(∗) m2 (int ∗param1, int param2, int param3, int param4)

7.5.1 Detailed Description

Brief description this structure.
Detailed description starts here, one line or multiple lines. Blah blah blah...

Note

This is a note for this structure, blah blah blah...

7.5.2 Field Documentation

7.5.2.1 m1

```
int(*) dogygen_example_struct_another::m1(int *param1, int param2, int param3, int param4)
```
Some documentation for member 'm1'...

7.5.2.2 m2

```
int(*) dogygen_example_struct_another::m2(int *param1, int param2, int param3, int param4)
```
Some documentation for member 'm2'...

7.6 dogygen_example_struct_another2 Struct Reference

```
#include <struct.h>
```

Data Fields

- int(∗) m1 (int ∗param1, int param2, int param3, int param4)
- int(∗) m2 (int ∗param1, int param2, int param3, int param4)

7.6.1 Detailed Description

Brief description this structure.
Detailed description starts here, one line or multiple lines. Blah blah blah...

Note

This is a note for this structure, blah blah blah...

7.6.2 Field Documentation

7.6.2.1 m1

```
int(*) dogygen_example_struct_another2::m1(int *param1, int param2, int param3, int param4)
```

Brief description of m1.

Parameters

param1	The first param.
param2	The second param.
param3	The third param.
param4	The fourth param.

Returns

the operation status, 0 on successful

7.6.2.2 m2

```
int(*) dogygen_example_struct_another2::m2(int *param1, int param2, int param3, int param4)
```

Brief description of m2.

Parameters

param1	The first param.
param2	The second param.
param3	The third param.
param4	The fourth param.

Returns

the operation status, 0 on successful

7.7 doxygen_example_union Union Reference

```
#include <union.h>
```

Data Fields

- int v1
- double v2

7.7.1 Detailed Description

Brief description of this union.

7.7.2 Field Documentation

7.7.2.1 v1

```
int doxygen_example_union::v1
```
description for v1

7.7.2.2 v2

```
double doxygen_example_union::v2
```
description for v2

7.8 rt_adc_core_ops Struct Reference

```
#include <adc_v2.h>
```

Data Fields

- rt_err_t(∗) open (struct rt_adc_device ∗device)
- rt_err_t(∗) close (struct rt_adc_device ∗device)
- rt_err_t(∗) session_config (struct rt_adc_device ∗device, rt_uint32_t channels)
- rt_err_t(∗) control (struct rt_adc_device ∗device, int cmd, void ∗args)

7.8.1 Detailed Description

ADC core driver operation table.

7.8.2 Field Documentation

7.8.2.1 open

```
rt_err_t(*) rt_adc_core_ops::open(struct rt_adc_device *device)
```
Open and initialize the ADC hardware.

Parameters

device	Pointer to the ADC device object.

Returns

Operation status.

7.8.2.2 close

```
rt_err_t(*) rt_adc_core_ops::close(struct rt_adc_device *device)
```
Close and deinitialize the ADC hardware.

Parameters

device	Pointer to the ADC device object.

Returns

Operation status.

7.8.2.3 session_config

`rt_err_t(*) rt_adc_core_ops::session_config(struct rt_adc_device *device, rt_uint32_t channels)`
Configure one ADC conversion session before data transfer.

Parameters

device	Pointer to the ADC device object.
channels	ADC channel selection mask.

Returns

Operation status.

7.8.2.4 control

`rt_err_t(*) rt_adc_core_ops::control(struct rt_adc_device *device, int cmd, void *args)`
Handle ADC control commands.

Parameters

device	Pointer to the ADC device object.
cmd	Control command.
args	Pointer to the control argument buffer.

Returns

Operation status.

7.9 rt_adc_device Struct Reference

`#include <adc_v2.h>`

Data Fields

- struct rt_device parent
- const struct rt_adc_ops * ops
- rt_atomic_t state
- struct rt_spinlock spinlock
- rt_uint32_t default_vref_mv
- struct rt_adc_session_ctrl session_ctrl

7.9.1 Detailed Description

adc device
ADC device object.

7.9.2 Field Documentation

7.9.2.1 parent

`struct rt_device rt_adc_device::parent`
RT-Thread device object.

7.9.2.2 ops

`const struct rt_adc_ops * rt_adc_device::ops`
ADC driver operation table.

7.9.2.3 state

`rt_atomic_t rt_adc_device::state`
Atomic ADC runtime state.

7.9.2.4 spinlock

`struct rt_spinlock rt_adc_device::spinlock`
Protects ADC stream FIFO shared by thread context and ISR.

7.9.2.5 default_vref_mv

`rt_uint32_t rt_adc_device::default_vref_mv`
Default reference voltage in millivolts.

7.9.2.6 session_ctrl

`struct rt_adc_session_ctrl rt_adc_device::session_ctrl`
ADC active session control block.

7.10 rt_adc_ops Struct Reference

`#include <adc_v2.h>`

Data Fields

- const struct rt_adc_core_ops * core
- const struct rt_adc_sequence_ops * sequence

7.10.1 Detailed Description

Configure the adc device.
ADC driver operation table.

7.10.2 Field Documentation

7.10.2.1 core

`const struct rt_adc_core_ops* rt_adc_ops::core`
Core operation table.

7.10.2.2 sequence

`const struct rt_adc_sequence_ops* rt_adc_ops::sequence`
Sequence-session operations.

7.11 rt_adc_sequence_cfg Struct Reference

`#include <adc_v2.h>`

Data Fields

- rt_uint32_t * buffer
- rt_size_t buffer_length
- rt_int32_t timeout_ms

7.11.1 Detailed Description

ADC synchronous one-frame sequence request.

7.11.2 Field Documentation

7.11.2.1 buffer

`rt_uint32_t* rt_adc_sequence_cfg::buffer`
Destination sample buffer.

7.11.2.2 buffer_length

`rt_size_t rt_adc_sequence_cfg::buffer_length`
Destination buffer length in samples.

7.11.2.3 timeout_ms

`rt_int32_t rt_adc_sequence_cfg::timeout_ms`
Per-sample timeout; negative means wait forever.

7.12 rt_adc_sequence_ops Struct Reference

`#include <adc_v2.h>`

Data Fields

- rt_err_t(*) start (struct rt_adc_device *device, rt_uint32_t channels, const struct rt_adc_sequence_cfg *cfg)
- rt_err_t(*) read (struct rt_adc_device *device, rt_uint32_t *value, rt_int32_t timeout_ms)
- rt_err_t(*) stop (struct rt_adc_device *device)

7.12.1 Detailed Description

ADC finite-sequence operation table.

7.12.2 Field Documentation

7.12.2.1 start

`rt_err_t(*) rt_adc_sequence_ops::start(struct rt_adc_device *device, rt_uint32_t channels, const struct rt_adc_sequence_cfg *cfg)`
Start one active finite transfer session.

Parameters

device	Pointer to the ADC device object.
channels	ADC channel selection mask.
cfg	Pointer to the sequence request object.

Returns

Operation status.

7.12.2.2 read

`rt_err_t(*) rt_adc_sequence_ops::read(struct rt_adc_device *device, rt_uint32_t *value, rt_int32_t timeout_ms)`
Read one converted sample from an active finite transfer session.

Parameters

device	Pointer to the ADC device object.
value	Pointer to the output sample value.
timeout_ms	Read timeout in milliseconds; negative means wait forever.

Returns

Operation status.

7.12.2.3 stop

`rt_err_t(*) rt_adc_sequence_ops::stop(struct rt_adc_device *device)`
Stop one active finite transfer session.

Parameters

device	Pointer to the ADC device object.

Returns

Operation status.

7.13 rt_adc_session_ctrl Struct Reference

`#include <adc_v2.h>`

Data Fields

- rt_uint32_t channels
- rt_size_t channel_count
- rt_bool_t configured

7.13.1 Detailed Description

ADC active session runtime control block.

7.13.2 Field Documentation

7.13.2.1 channels

`rt_uint32_t rt_adc_session_ctrl::channels`
Cached active ADC channel selection mask.

7.13.2.2 channel_count

`rt_size_t rt_adc_session_ctrl::channel_count`
Cached active session channel count.

7.13.2.3 configured

`rt_bool_t rt_adc_session_ctrl::configured`
Whether the cached session configuration is valid.

7.14 rt_audio_buf_info Struct Reference

`#include <dev_audio.h>`

Data Fields

- rt_uint8_t * buffer
- rt_uint16_t block_size
- rt_uint16_t block_count
- rt_uint32_t total_size

7.14.1 Detailed Description

Audio buffer info.
The preferred number and size of audio pipeline buffer for the audio device, it will be used in rt_audio_replay struct.

7.14.2 Field Documentation

7.14.2.1 buffer

`rt_uint8_t* rt_audio_buf_info::buffer`
Audio buffer information

7.14.2.2 block_size

`rt_uint16_t rt_audio_buf_info::block_size`
Audio block_size information for replay function

7.14.2.3 block_count

`rt_uint16_t rt_audio_buf_info::block_count`
Audio block_count information for replay function

7.14.2.4 total_size

`rt_uint32_t rt_audio_buf_info::total_size`
Audio total_size which is equal to block_size multiplying block_count information for replay function

7.15 rt_audio_caps Struct Reference

`#include <dev_audio.h>`

Data Fields

- int main_type
- int sub_type
- union udata

7.15.1 Detailed Description

Audio capabilities.

7.15.2 Field Documentation

7.15.2.1 main_type

`int rt_audio_caps::main_type`
Audio main type, one value of AUDIO_TYPE

7.15.2.2 sub_type

`int rt_audio_caps::sub_type`
Audio sub type, one value of AUDIO_DSP or AUDIO_MIXER

7.15.2.3 udata

`union rt_audio_caps::[union].udata rt_audio_caps::udata`
User data

7.16 rt_audio_configure Struct Reference

`#include <dev_audio.h>`

Data Fields

- rt_uint32_t samplerate
- rt_uint16_t channels
- rt_uint16_t samplebits

7.16.1 Detailed Description

Audio configuration.
The preferred number and size of audio pipeline buffer for the audio device, it will be used in rt_audio_caps struct.

7.16.2 Field Documentation

7.16.2.1 samplerate

`rt_uint32_t rt_audio_configure::samplerate`
Audio samplerate information

7.16.2.2 channels

`rt_uint16_t rt_audio_configure::channels`
Audio channels information

7.16.2.3 samplebits

`rt_uint16_t rt_audio_configure::samplebits`
Audio samplebits information

7.17 rt_audio_device Struct Reference

`#include <dev_audio.h>`

Data Fields

- struct rt_device parent
- struct rt_audio_ops * ops
- struct rt_audio_replay * replay
- struct rt_audio_record * record

7.17.1 Detailed Description

Audio device.

7.17.2 Field Documentation

7.17.2.1 parent

`struct rt_device rt_audio_device::parent`
Audio device parents

7.17.2.2 ops

`struct rt_audio_ops* rt_audio_device::ops`
Audio device operator

7.17.2.3 replay

`struct rt_audio_replay* rt_audio_device::replay`
Pointer to audio replay structure

7.17.2.4 record

`struct rt_audio_record* rt_audio_device::record`
Pointer to audio record structure

7.18 rt_audio_ops Struct Reference

`#include <dev_audio.h>`

Data Fields

- rt_err_t(*) getcaps (struct rt_audio_device *audio, struct rt_audio_caps *caps)
- rt_err_t(*) configure (struct rt_audio_device *audio, struct rt_audio_caps *caps)
- rt_err_t(*) init (struct rt_audio_device *audio)
- rt_err_t(*) start (struct rt_audio_device *audio, int stream)
- rt_err_t(*) stop (struct rt_audio_device *audio, int stream)
- rt_ssize_t(*) transmit (struct rt_audio_device *audio, const void *writeBuf, void *readBuf, rt_size_t size)
- void(*) buffer_info (struct rt_audio_device *audio, struct rt_audio_buf_info *info)

7.18.1 Detailed Description

Aduio device operators.

7.18.2 Field Documentation

7.18.2.1 getcaps

`rt_err_t(*) rt_audio_ops::getcaps(struct rt_audio_device *audio, struct rt_audio_caps *caps)`
Get audio capabilities information

7.18.2.2 configure

```
rt_err_t(*) rt_audio_ops::configure(struct rt_audio_device *audio, struct rt_audio_caps *caps)
```
Configure audio devices

7.18.2.3 init

```
rt_err_t(*) rt_audio_ops::init(struct rt_audio_device *audio)
```
Initialize audio device

7.18.2.4 start

```
rt_err_t(*) rt_audio_ops::start(struct rt_audio_device *audio, int stream)
```
Turn on the audio device

7.18.2.5 stop

```
rt_err_t(*) rt_audio_ops::stop(struct rt_audio_device *audio, int stream)
```
Turn off the audio device

7.18.2.6 transmit

```
rt_ssize_t(*) rt_audio_ops::transmit(struct rt_audio_device *audio, const void *writeBuf, void
*readBuf, rt_size_t size)
```
Transmit data between application and device

7.18.2.7 buffer_info

```
void(*) rt_audio_ops::buffer_info(struct rt_audio_device *audio, struct rt_audio_buf_info
*info)
```
Get page size of codec or private buffer's info

7.19 rt_audio_pipe Struct Reference

```
#include <dev_audio_pipe.h>
```

Data Fields

- struct rt_ringbuffer ringbuffer
- rt_list_t suspended_read_list
- rt_list_t suspended_write_list

7.19.1 Detailed Description

Audio buffer info.
The preferred number and size of audio pipeline buffer for the audio device, it will be used in rt_audio_replay struct.

7.19.2 Field Documentation

7.19.2.1 ringbuffer

```
struct rt_ringbuffer rt_audio_pipe::ringbuffer
```
ring buffer in pipe device

7.19.2.2 suspended_read_list

```
rt_list_t rt_audio_pipe::suspended_read_list
```
suspended thread list for reading

7.19.2.3 suspended_write_list

`rt_list_t rt_audio_pipe::suspended_write_list`
suspended thread list for writing

7.20 rt_audio_portal_device Struct Reference

`#include <dev_audio_pipe.h>`

7.20.1 Detailed Description

Portal device.

7.21 rt_audio_record Struct Reference

`#include <dev_audio.h>`

7.21.1 Detailed Description

Audio record, the audio device pipe container of ringbuffer.

7.22 rt_audio_replay Struct Reference

`#include <dev_audio.h>`

Data Fields

- struct rt_mempool * mp
- struct rt_data_queue queue
- struct rt_mutex lock
- struct rt_completion cmp
- struct rt_audio_buf_info buf_info
- rt_uint8_t * write_data
- rt_uint16_t write_index
- rt_uint16_t read_index
- rt_uint32_t pos
- rt_uint8_t event
- rt_bool_t activated

7.22.1 Detailed Description

Audio replay.

7.22.2 Field Documentation

7.22.2.1 mp

`struct rt_mempool* rt_audio_replay::mp`
Memory pool for audio replay

7.22.2.2 queue

`struct rt_data_queue rt_audio_replay::queue`
Replay data queue

7.22.2.3 lock

`struct rt_mutex rt_audio_replay::lock`
Replay mutex lock

7.22.2.4 cmp

`struct rt_completion rt_audio_replay::cmp`
Replay completion, it will be

7.22.2.5 buf_info

`struct rt_audio_buf_info rt_audio_replay::buf_info`
Replay buffer information

7.22.2.6 write_data

`rt_uint8_t* rt_audio_replay::write_data`
Pointer to the data to be written into data queue

7.22.2.7 write_index

`rt_uint16_t rt_audio_replay::write_index`
Index of pointer write_data.It records how much data has been written in currently being played block

7.22.2.8 read_index

`rt_uint16_t rt_audio_replay::read_index`
Index of replaying data for audio device, it indicates index of replay in the blocks which is currently being played

7.22.2.9 pos

`rt_uint32_t rt_audio_replay::pos`
Global position of audio replay

7.22.2.10 event

`rt_uint8_t rt_audio_replay::event`
Event flag

7.22.2.11 activated

`rt_bool_t rt_audio_replay::activated`
Activaty flag

7.23 rt_blk_device Struct Reference

`#include <blk.h>`

7.23.1 Detailed Description

Logical blk device, if you don't used DFS it will be defined by default.

7.24 rt_blk_disk Struct Reference

`#include <blk.h>`

7.24.1 Detailed Description

Physical blk device.

7.25 rt_blk_disk_ops Struct Reference

`#include <blk.h>`

7.25.1 Detailed Description

Configure the blk device.

7.26 rt_can_bit_timing Struct Reference

`#include <dev_can.h>`

Data Fields

- rt_uint16_t prescaler
- rt_uint16_t num_seg1
- rt_uint16_t num_seg2
- rt_uint8_t num_sjw
- rt_uint8_t num_sspoff

7.26.1 Detailed Description

CAN bit-timing parameters structure.

7.26.2 Field Documentation

7.26.2.1 prescaler

`rt_uint16_t rt_can_bit_timing::prescaler`
Baud rate prescaler.

7.26.2.2 num_seg1

`rt_uint16_t rt_can_bit_timing::num_seg1`
Bit Timing Segment 1, in terms of Time Quanta (Tq).

7.26.2.3 num_seg2

`rt_uint16_t rt_can_bit_timing::num_seg2`
Bit Timing Segment 2, in terms of Time Quanta (Tq).

7.26.2.4 num_sjw

`rt_uint8_t rt_can_bit_timing::num_sjw`
Synchronization Jump Width, in terms of Time Quanta (Tq).

7.26.2.5 num_sspoff

`rt_uint8_t rt_can_bit_timing::num_sspoff`
Secondary Sample Point Offset, in terms of Time Quanta (Tq) for CAN-FD.

7.27 rt_can_bit_timing_config Struct Reference

`#include <dev_can.h>`

Data Fields

- rt_uint32_t count
- struct rt_can_bit_timing * items

7.27.1 Detailed Description

CAN bit timing configuration list structure.

Note

items[0] is for CAN 2.0 / CAN-FD Arbitration Phase. items[1] is for CAN-FD Data Phase (if applicable).

7.27.2 Field Documentation

7.27.2.1 count

```
rt_uint32_t rt_can_bit_timing_config::count
```

The number of bit-timing configurations (typically 1 for CAN, 2 for CAN-FD).

7.27.2.2 items

```
struct rt_can_bit_timing* rt_can_bit_timing_config::items
```

A pointer to an array of bit-timing structures.

7.28 rt_can_device Struct Reference

```
#include <dev_can.h>
```

Data Fields

- struct rt_device parent
- const struct rt_can_ops * ops
- struct can_configure config
- struct rt_can_status status
- rt_uint32_t timerinitflag
- struct rt_timer timer
- struct rt_can_status_ind_type status_indicate
- struct rt_mutex lock
- void * can_rx
- void * can_tx
- struct rt_ringbuffer nb_tx_rb
- rt_uint8_t nb_tx_rb_pool [RT_CAN_NB_TX_FIFO_SIZE]

7.28.1 Detailed Description

The core CAN device structure.

7.28.2 Field Documentation

7.28.2.1 parent

```
struct rt_device rt_can_device::parent
```

Inherits from the base RT-Thread device structure.

7.28.2.2 ops

```
const struct rt_can_ops* rt_can_device::ops
```

A pointer to the low-level driver operations.

7.28.2.3 config

`struct can_configure rt_can_device::config`
The current configuration of the CAN device.

7.28.2.4 status

`struct rt_can_status rt_can_device::status`
The current status and error statistics of the CAN device.

7.28.2.5 timerinitflag

`rt_uint32_t rt_can_device::timerinitflag`
A flag to indicate if the status timer has been initialized.

7.28.2.6 timer

`struct rt_timer rt_can_device::timer`
A timer for periodically checking the CAN bus status.

7.28.2.7 status_indicate

`struct rt_can_status_ind_type rt_can_device::status_indicate`
The user-registered status indication callback.

7.28.2.8 lock

`struct rt_mutex rt_can_device::lock`
A mutex for thread-safe access to the device.

7.28.2.9 can_rx

`void* rt_can_device::can_rx`
A pointer to the software receive FIFO structure (`rt_can_rx_fifo`).

7.28.2.10 can_tx

`void* rt_can_device::can_tx`
A pointer to the software transmit FIFO structure (`rt_can_tx_fifo`).

7.28.2.11 nb_tx_rb

`struct rt_ringbuffer rt_can_device::nb_tx_rb`
The ring buffer for non-blocking transmissions.

7.28.2.12 nb_tx_rb_pool

`rt_uint8_t rt_can_device::nb_tx_rb_pool[RT_CAN_NB_TX_FIFO_SIZE]`
The statically allocated pool for the non-blocking TX ring buffer.

7.29 rt_can_filter_config Struct Reference

`#include <dev_can.h>`

Data Fields

- rt_uint32_t count
- rt_uint32_t actived
- struct rt_can_filter_item * items

7.29.1 Detailed Description

CAN hardware filter configuration structure. This structure is passed to the driver via `rt_device_control` with the `RT_CAN_CMD_SET_FILTER` command.

7.29.2 Field Documentation

7.29.2.1 count

`rt_uint32_t rt_can_filter_config::count`
The number of filter items in the `items` array.

7.29.2.2 actived

`rt_uint32_t rt_can_filter_config::actived`
A flag to activate (1) or deactivate (0) the filters.

7.29.2.3 items

`struct rt_can_filter_item* rt_can_filter_config::items`
A pointer to an array of filter items.

7.30 rt_can_filter_item Struct Reference

`#include <dev_can.h>`

Data Fields

- rt_uint32_t id: 29
- rt_uint32_t ide: 1
- rt_uint32_t rtr: 1
- rt_uint32_t mode: 1
- rt_uint32_t mask
- rt_int32_t hdr_bank
- rt_uint32_t rxfifo

7.30.1 Detailed Description

CAN filter item structure.

7.30.2 Field Documentation

7.30.2.1 id

`rt_uint32_t rt_can_filter_item::id`
The CAN ID to be filtered.

7.30.2.2 ide

`rt_uint32_t rt_can_filter_item::ide`
Identifier type. 0 for Standard ID, 1 for Extended ID.

7.30.2.3 rtr

`rt_uint32_t rt_can_filter_item::rtr`
Frame type. 0 for Data Frame, 1 for Remote Frame.

7.30.2.4 mode

`rt_uint32_t rt_can_filter_item::mode`
Filter mode. 0 for Mask Mode, 1 for List Mode.

7.30.2.5 mask

`rt_uint32_t rt_can_filter_item::mask`
The filter mask. In Mask Mode, a '1' bit means the corresponding ID bit must match.

7.30.2.6 hdr_bank

`rt_int32_t rt_can_filter_item::hdr_bank`
The specific hardware filter bank index to use. A value of -1 allows the driver to auto-assign.

7.30.2.7 rxfifo

`rt_uint32_t rt_can_filter_item::rxfifo`
The target RX FIFO for matched messages (CAN_RX_FIFO0 or CAN_RX_FIFO1).

7.31 rt_can_msg Struct Reference

`#include <dev_can.h>`

Data Fields

- rt_uint32_t id: 29
- rt_uint32_t ide: 1
- rt_uint32_t rtr: 1
- rt_uint32_t rsv: 1
- rt_uint32_t len: 8
- rt_uint32_t priv: 8
- rt_int32_t hdr_index: 8
- rt_uint32_t rxfifo: 2
- rt_uint32_t nonblocking: 1
- rt_uint8_t data [8]

7.31.1 Detailed Description

The CAN message structure.

7.31.2 Field Documentation

7.31.2.1 id

`rt_uint32_t rt_can_msg::id`
CAN ID (Standard or Extended).

7.31.2.2 ide

`rt_uint32_t rt_can_msg::ide`
Identifier type: 0=Standard ID, 1=Extended ID.

7.31.2.3 rtr

`rt_uint32_t rt_can_msg::rtr`
Frame type: 0=Data Frame, 1=Remote Frame.

7.31.2.4 rsv

`rt_uint32_t rt_can_msg::rsv`
Reserved bit.

7.31.2.5 len

`rt_uint32_t rt_can_msg::len`
Data Length Code (DLC) from 0 to 8.

7.31.2.6 priv

`rt_uint32_t rt_can_msg::priv`
Private data, used to specify the hardware mailbox in private mode.

7.31.2.7 hdr_index

`rt_int32_t rt_can_msg::hdr_index`
For received messages, the index of the hardware filter that matched the message.

7.31.2.8 rxfifo

`rt_uint32_t rt_can_msg::rxfifo`
The RX FIFO where the message was received.

7.31.2.9 nonblocking

`rt_uint32_t rt_can_msg::nonblocking`
Send mode: 0=Blocking (default), 1=Non-blocking.

7.31.2.10 data

`rt_uint8_t rt_can_msg::data[8]`
CAN message payload (up to 8 bytes).

7.32 rt_can_ops Struct Reference

`#include <dev_can.h>`

Data Fields

- rt_err_t(∗) configure (struct rt_can_device ∗can, struct can_configure ∗cfg)
- rt_err_t(∗) control (struct rt_can_device ∗can, int cmd, void ∗arg)
- rt_ssize_t(∗) sendmsg (struct rt_can_device ∗can, const void ∗buf, rt_uint32_t boxno)
- rt_ssize_t(∗) recvmsg (struct rt_can_device ∗can, void ∗buf, rt_uint32_t fifo)
- rt_ssize_t(∗) sendmsg_nonblocking (struct rt_can_device ∗can, const void ∗buf)

7.32.1 Detailed Description

The CAN device driver operations structure.
This structure contains pointers to the low-level functions that implement the CAN functionality. It must be provided when registering a CAN device.

7.32.2 Field Documentation

7.32.2.1 configure

`rt_err_t(*) rt_can_ops::configure(struct rt_can_device *can, struct can_configure *cfg)`
Configures the CAN controller with the specified settings.

Parameters

in	*can*	A pointer to the CAN device structure.
in	*cfg*	A pointer to the configuration structure.

Returns

`RT_EOK` on success, or a negative error code on failure.

7.32.2.2 control

`rt_err_t(*) rt_can_ops::control(struct rt_can_device *can, int cmd, void *arg)`
Sends control commands to the CAN device.

Parameters

in	*can*	A pointer to the CAN device structure.
in	*cmd*	The control command (e.g., RT_CAN_CMD_SET_FILTER).
in	*arg*	A pointer to the arguments for the command.

Returns

`RT_EOK` on success, or a negative error code on failure.

7.32.2.3 sendmsg

`rt_ssize_t(*) rt_can_ops::sendmsg(struct rt_can_device *can, const void *buf, rt_uint32_t boxno)`
Low-level blocking function to send a CAN message.

Parameters

in	*can*	A pointer to the CAN device structure.
in	*buf*	A pointer to the `rt_can_msg` to be sent.
in	*boxno*	The hardware mailbox number to use for transmission.

Returns

The number of bytes sent on success, or a negative error code on failure.

7.32.2.4 recvmsg

`rt_ssize_t(*) rt_can_ops::recvmsg(struct rt_can_device *can, void *buf, rt_uint32_t fifo)`
Low-level function to receive a CAN message.

Parameters

in,out	*can*	A pointer to the CAN device structure.
out	*buf*	A pointer to the buffer to store the received rt_can_msg.
in	*fifo*	The hardware FIFO number to read from.

Returns

The number of bytes received on success, or a negative error code on failure.

7.32.2.5 sendmsg_nonblocking

```
rt_ssize_t(*) rt_can_ops::sendmsg_nonblocking(struct rt_can_device *can, const void *buf)
```

Low-level, hardware-specific function to send a CAN message non-blockingly.
This function attempts to place a message into a hardware transmission mailbox and returns immediately without waiting for the transmission to complete.

Parameters

in	*can*	A pointer to the CAN device structure.
in	*buf*	A pointer to the rt_can_msg to be sent.

Returns

- RT_EOK if the message was successfully accepted by the hardware.
- -RT_EBUSY if all hardware mailboxes are currently full.
- Other negative error codes for different failures.

7.33 rt_can_status Struct Reference

```
#include <dev_can.h>
```

Data Fields

- rt_uint32_t rcverrcnt
- rt_uint32_t snderrcnt
- rt_uint32_t errcode
- rt_uint32_t rcvpkg
- rt_uint32_t dropedrcvpkg
- rt_uint32_t sndpkg
- rt_uint32_t dropedsndpkg
- rt_uint32_t bitpaderrcnt
- rt_uint32_t formaterrcnt
- rt_uint32_t ackerrcnt
- rt_uint32_t biterrcnt
- rt_uint32_t crcerrcnt
- rt_uint32_t rcvchange
- rt_uint32_t sndchange
- rt_uint32_t lasterrtype

7.33.1 Detailed Description

CAN device status and error statistics structure.

7.33.2 Field Documentation

7.33.2.1 rcverrcnt

`rt_uint32_t rt_can_status::rcverrcnt`
Receive Error Counter (REC).

7.33.2.2 snderrcnt

`rt_uint32_t rt_can_status::snderrcnt`
Transmit Error Counter (TEC).

7.33.2.3 errcode

`rt_uint32_t rt_can_status::errcode`
The current bus error code (see `enum RT_CAN_BUS_ERR`).

7.33.2.4 rcvpkg

`rt_uint32_t rt_can_status::rcvpkg`
Total number of successfully received packages.

7.33.2.5 dropedrcvpkg

`rt_uint32_t rt_can_status::dropedrcvpkg`
Number of received packages dropped due to full buffers.

7.33.2.6 sndpkg

`rt_uint32_t rt_can_status::sndpkg`
Total number of successfully sent packages.

7.33.2.7 dropedsndpkg

`rt_uint32_t rt_can_status::dropedsndpkg`
Number of sent packages dropped due to full buffers or errors.

7.33.2.8 bitpaderrcnt

`rt_uint32_t rt_can_status::bitpaderrcnt`
Bit stuffing error count.

7.33.2.9 formaterrcnt

`rt_uint32_t rt_can_status::formaterrcnt`
Format error count.

7.33.2.10 ackerrcnt

`rt_uint32_t rt_can_status::ackerrcnt`
Acknowledgment error count.

7.33.2.11 biterrcnt

`rt_uint32_t rt_can_status::biterrcnt`
Bit error (recessive/dominant) count.

7.33.2.12 crcerrcnt

`rt_uint32_t rt_can_status::crcerrcnt`
CRC error count.

7.33.2.13 rcvchange

`rt_uint32_t rt_can_status::rcvchange`
A flag indicating that the RX buffer status has changed.

7.33.2.14 sndchange

`rt_uint32_t rt_can_status::sndchange`
A bitmask indicating which TX mailboxes have changed status.

7.33.2.15 lasterrtype

`rt_uint32_t rt_can_status::lasterrtype`
The type of the last error that occurred.

7.34 rt_can_status_ind_type Struct Reference

`#include <dev_can.h>`

Data Fields

- rt_canstatus_ind ind
- void * args

7.34.1 Detailed Description

Structure to hold the status indication callback and its arguments.

7.34.2 Field Documentation

7.34.2.1 ind

`rt_canstatus_ind rt_can_status_ind_type::ind`
Pointer to the status indication callback function.

7.34.2.2 args

`void* rt_can_status_ind_type::args`
Pointer to user arguments for the callback.

7.35 rt_clk Struct Reference

`#include <clk.h>`

Data Fields

- const char * dev_id
- const char * con_id

7.35.1 Detailed Description

Clock handle - represents a consumer reference to a clock.
Each consumer obtains an rt_clk instance bound to a specific clk_cell. The handle stores consumer-specific constraints such as min/max rate.

7.35.2 Field Documentation

7.35.2.1 dev_id

```
const char* rt_clk::dev_id
```
Device identifier using this clock

7.35.2.2 con_id

```
const char* rt_clk::con_id
```
Connection identifier (name)

7.36 rt_clk_array Struct Reference

```
#include <clk.h>
```

7.36.1 Detailed Description

Clock array container.
Represents a group of rt_clk handles, typically used for devices that require multiple clock inputs.

7.37 rt_clk_cell Struct Reference

```
#include <clk.h>
```

Data Fields

- rt_ubase_t rate

7.37.1 Detailed Description

Clock cell - represents a single hardware clock element.
A clk_cell is the fundamental unit of a clock tree, such as a PLL, divider, mux, or gate. It maintains its relationship to parent clocks using pointers, not lists, for lightweight hierarchy management.
The 'ops' field defines hardware-specific callbacks. The framework invokes these during enable, disable, and rate changes.

7.37.2 Field Documentation

7.37.2.1 rate

```
rt_ubase_t rt_clk_cell::rate
```
Cached or fixed rate (not always accurate)

7.38 rt_clk_fixed_rate Struct Reference

```
#include <clk.h>
```

7.38.1 Detailed Description

Fixed-rate clock descriptor.
Used for constant-frequency clocks without configurable parents or dividers.

7.39 rt_clk_node Struct Reference

```
#include <clk.h>
```

7.39.1 Detailed Description

Clock provider node - represents a hardware clock controller.
A rt_clk_node corresponds to one hardware clock provider in the system, such as a PLL controller, a clock multiplexer, or a composite clock block defined in the device tree.
Each clock node may contain multiple hardware clock outputs, described as rt_clk_cell structures, which represent individual leaf clocks.
Members:

- `parent` — embedded rt_object header for RT-Thread object system.
- `dev` — back-reference to the hardware device providing this clock domain.
- `parents_clk` — optional array of parent clock handles.
- `multi_clk` — number of clock outputs exported by this provider.
- `cells` — list of rt_clk_cell pointers representing each output.
- `ofw_parse` — callback used to parse clock arguments from device tree (`#clock-cells`) and select the corresponding rt_clk_cell.
- `priv` — implementation-specific private data.

Typical usage:

1. Define a rt_clk_node describing the hardware clock controller.
2. Implement `ofw_parse()` to resolve device tree `phandle` arguments.
3. Register the node using rt_clk_register().

7.40 rt_clk_notifier Struct Reference

`#include <clk.h>`

7.40.1 Detailed Description

Clock notifier descriptor.
Used to register callbacks for clock events (rate change, abort, etc). Each notifier is linked to a specific clock and triggered on rate changes.

7.41 rt_cpu_usage_stats Struct Reference

`#include <rtdef.h>`

7.41.1 Detailed Description

CPU usage statistics data

7.42 rt_dac_device Struct Reference

`#include <dac.h>`

7.42.1 Detailed Description

DAC device structure.

7.43 rt_dac_ops Struct Reference

`#include <dac.h>`

7.43.1 Detailed Description

Configuration of DAC device.

7.44 rt_device Struct Reference

`#include <rtdef.h>`

Data Fields

- struct rt_object parent
- enum rt_device_class_type type
- rt_uint16_t flag
- rt_uint16_t open_flag
- rt_uint8_t ref_count
- rt_uint8_t device_id
- rt_err_t(∗) readlink (rt_device_t dev, char ∗buf, int len)
- void ∗ user_data

7.44.1 Detailed Description

Device structure

7.44.2 Field Documentation

7.44.2.1 parent

`struct rt_object rt_device::parent`
inherit from rt_object

7.44.2.2 type

`enum rt_device_class_type rt_device::type`
device type

7.44.2.3 flag

`rt_uint16_t rt_device::flag`
device flag

7.44.2.4 open_flag

`rt_uint16_t rt_device::open_flag`
device open flag

7.44.2.5 ref_count

`rt_uint8_t rt_device::ref_count`
reference count

7.44.2.6 device_id

`rt_uint8_t rt_device::device_id`
0 - 255

7.44.2.7 readlink

`rt_err_t(*) rt_device::readlink(rt_device_t dev, char *buf, int len)`
for dynamic device

7.44.2.8 user_data

`void* rt_device::user_data`
device private data

7.45 rt_device_blk_geometry Struct Reference

`#include <block.h>`

Data Fields

- rt_uint64_t sector_count
- rt_uint32_t bytes_per_sector
- rt_uint32_t block_size

7.45.1 Detailed Description

block device geometry structure

7.45.2 Field Documentation

7.45.2.1 sector_count

`rt_uint64_t rt_device_blk_geometry::sector_count`
count of sectors

7.45.2.2 bytes_per_sector

`rt_uint32_t rt_device_blk_geometry::bytes_per_sector`
number of bytes per sector

7.45.2.3 block_size

`rt_uint32_t rt_device_blk_geometry::block_size`
number of bytes to erase one block

7.46 rt_device_blk_sectors Struct Reference

`#include <block.h>`

Data Fields

- rt_uint64_t sector_begin
- rt_uint64_t sector_end

7.46.1 Detailed Description

sector arrange struct on block device

7.46.2 Field Documentation

7.46.2.1 sector_begin

`rt_uint64_t rt_device_blk_sectors::sector_begin`
begin sector

7.46.2.2 sector_end

`rt_uint64_t rt_device_blk_sectors::sector_end`
end sector

7.47 rt_device_graphic_info Struct Reference

`#include <graphic.h>`

Data Fields

- rt_uint8_t pixel_format
- rt_uint8_t bits_per_pixel
- rt_uint16_t pitch
- rt_uint16_t width
- rt_uint16_t height
- rt_uint8_t * framebuffer
- rt_uint32_t smem_len

7.47.1 Detailed Description

graphic device information structure

7.47.2 Field Documentation

7.47.2.1 pixel_format

`rt_uint8_t rt_device_graphic_info::pixel_format`
graphic format

7.47.2.2 bits_per_pixel

`rt_uint8_t rt_device_graphic_info::bits_per_pixel`
bits per pixel

7.47.2.3 pitch

`rt_uint16_t rt_device_graphic_info::pitch`
bytes per line

7.47.2.4 width

`rt_uint16_t rt_device_graphic_info::width`
width of graphic device

7.47.2.5 height

`rt_uint16_t rt_device_graphic_info::height`
height of graphic device

7.47.2.6 framebuffer

`rt_uint8_t* rt_device_graphic_info::framebuffer`
frame buffer

7.47.2.7 smem_len

`rt_uint32_t rt_device_graphic_info::smem_len`
allocated frame buffer size

7.48 rt_device_graphic_ops Struct Reference

`#include <graphic.h>`

7.48.1 Detailed Description

graphic operations

7.49 rt_device_notify Struct Reference

`#include <rtdef.h>`

7.49.1 Detailed Description

Notify structure

7.50 rt_device_pin Struct Reference

`#include <dev_pin.h>`

7.50.1 Detailed Description

pin device structure

7.51 rt_device_pin_mode Struct Reference

`#include <dev_pin.h>`

7.51.1 Detailed Description

pin mode structure

7.52 rt_device_pin_value Struct Reference

`#include <dev_pin.h>`

7.52.1 Detailed Description

pin value structure

7.53 rt_device_pwm Struct Reference

`#include <dev_pwm.h>`

7.53.1 Detailed Description

PWM device.

7.54 rt_device_rect_info Struct Reference

`#include <graphic.h>`

Data Fields

- rt_uint16_t x
- rt_uint16_t y
- rt_uint16_t width
- rt_uint16_t height

7.54.1 Detailed Description

rectangle information structure

7.54.2 Field Documentation

7.54.2.1 x

`rt_uint16_t rt_device_rect_info::x`
x coordinate

7.54.2.2 y

`rt_uint16_t rt_device_rect_info::y`
y coordinate

7.54.2.3 width

`rt_uint16_t rt_device_rect_info::width`
width

7.54.2.4 height

`rt_uint16_t rt_device_rect_info::height`
height

7.55 rt_dma_chan Struct Reference

`#include <dma.h>`

Data Fields

- const char * name
- struct rt_dma_controller * ctrl
- struct rt_device * slave
- rt_list_t list
- rt_err_t conf_err
- rt_err_t prep_err
- struct rt_dma_slave_config conf
- struct rt_dma_slave_transfer transfer
- void(*) callback (struct rt_dma_chan *chan, rt_size_t size)
- void * priv

7.55.1 Detailed Description

DMA channel descriptor.
Represents an allocated DMA channel bound to a specific slave device. Tracks configuration and preparation errors to prevent operations on improperly set up channels.

7.55.2 Field Documentation

7.55.2.1 name

`const char* rt_dma_chan::name`
Channel name (from dma-names in device tree)

7.55.2.2 ctrl

`struct rt_dma_controller* rt_dma_chan::ctrl`
Parent DMA controller

7.55.2.3 slave

`struct rt_device* rt_dma_chan::slave`
Slave device that requested this channel

7.55.2.4 list

`rt_list_t rt_dma_chan::list`
Node in the controller's channels list

7.55.2.5 conf_err

`rt_err_t rt_dma_chan::conf_err`
Last configuration error (-RT_ERROR = not configured)

7.55.2.6 prep_err

`rt_err_t rt_dma_chan::prep_err`
Last preparation error (-RT_ERROR = not prepared)

7.55.2.7 conf

`struct rt_dma_slave_config rt_dma_chan::conf`
Saved slave configuration

7.55.2.8 transfer

`struct rt_dma_slave_transfer rt_dma_chan::transfer`
Saved transfer descriptor

7.55.2.9 callback

`void(*) rt_dma_chan::callback(struct rt_dma_chan *chan, rt_size_t size)`
Transfer completion callback

7.55.2.10 priv

`void* rt_dma_chan::priv`
Controller driver private data

7.56 rt_dma_controller Struct Reference

`#include <dma.h>`

Public Member Functions

- RT_BITMAP_DECLARE (dir_cap, RT_DMA_DIR_MAX)

Data Fields

- rt_list_t list
- struct rt_device * dev
- rt_uint64_t addr_mask
- const struct rt_dma_controller_ops * ops
- rt_list_t channels_nodes
- struct rt_mutex mutex

7.56.1 Detailed Description

DMA controller descriptor.
Represents a single DMA controller instance. Each controller has a set of operations (ops), supported transfer directions (dir_cap), an address mask for validating transfer addresses, and a list of currently allocated channels.

7.56.2 Member Function Documentation

7.56.2.1 RT_BITMAP_DECLARE()

```
rt_dma_controller::RT_BITMAP_DECLARE (
            dir_cap ,
            RT_DMA_DIR_MAX )
```

Bitmap of supported transfer directions

7.56.3 Field Documentation

7.56.3.1 list

```
rt_list_t rt_dma_controller::list
```

Node in the global controller list

7.56.3.2 dev

```
struct rt_device* rt_dma_controller::dev
```

Owning platform device

7.56.3.3 addr_mask

```
rt_uint64_t rt_dma_controller::addr_mask
```

Compute DMA address mask from bit width (e.g., RT_DMA_ADDR_MASK(32) = 0xFFFFFFFF).
Address mask: bits outside this mask are invalid for DMA

7.56.3.4 ops

```
const struct rt_dma_controller_ops* rt_dma_controller::ops
```

Controller-specific operations

7.56.3.5 channels_nodes

```
rt_list_t rt_dma_controller::channels_nodes
```

List of currently allocated channels

7.56.3.6 mutex

```
struct rt_mutex rt_dma_controller::mutex
```

Mutex for controller-level operations

7.57 rt_dma_controller_ops Struct Reference

```
#include <dma.h>
```

Data Fields

- struct rt_dma_chan *(*) request_chan (struct rt_dma_controller *ctrl, struct rt_device *slave, void *fw_data)
- rt_err_t(*) release_chan (struct rt_dma_chan *chan)
- rt_err_t(*) start (struct rt_dma_chan *chan)
- rt_err_t(*) pause (struct rt_dma_chan *chan)
- rt_err_t(*) stop (struct rt_dma_chan *chan)
- rt_err_t(*) config (struct rt_dma_chan *chan, struct rt_dma_slave_config *conf)
- rt_err_t(*) prep_memcpy (struct rt_dma_chan *chan, rt_ubase_t dma_addr_src, rt_ubase_t dma_addr_dst, rt_size_t len)
- rt_err_t(*) prep_cyclic (struct rt_dma_chan *chan, rt_ubase_t dma_buf_addr, rt_size_t buf_len, rt_size_t period_len, enum rt_dma_transfer_direction dir)
- rt_err_t(*) prep_single (struct rt_dma_chan *chan, rt_ubase_t dma_buf_addr, rt_size_t buf_len, enum rt_dma_transfer_direction dir)

7.57.1 Detailed Description

DMA controller operations vtable.
Each DMA controller driver provides implementations of these operations. start, stop, and config are mandatory; the rest are optional.

7.57.2 Field Documentation

7.57.2.1 request_chan

```
struct rt_dma_chan *(*) rt_dma_controller_ops::request_chan(struct rt_dma_controller *ctrl,
struct rt_device *slave, void *fw_data)
```

Request a DMA channel (optional).

Parameters

in	*ctrl*	DMA controller
in	*slave*	Device requesting the channel
in	*fw_data*	Device tree cell args (NULL for mem-to-mem)

Returns

Channel pointer, RT_NULL, or error pointer

7.57.2.2 release_chan

```
rt_err_t(*) rt_dma_controller_ops::release_chan(struct rt_dma_chan *chan)
```

Release a DMA channel (optional).

Parameters

in	*chan*	Channel to release

Returns

RT_EOK on success

7.57.2.3 start

`rt_err_t(*) rt_dma_controller_ops::start(struct rt_dma_chan *chan)`
Start a prepared transfer (mandatory).

Parameters

in	*chan*	Channel with a successfully prepared transfer

Returns

RT_EOK on success

7.57.2.4 pause

`rt_err_t(*) rt_dma_controller_ops::pause(struct rt_dma_chan *chan)`
Pause a running transfer (optional).
If not implemented, the framework falls back to stop().

Parameters

in	*chan*	Channel to pause

Returns

RT_EOK on success

7.57.2.5 stop

`rt_err_t(*) rt_dma_controller_ops::stop(struct rt_dma_chan *chan)`
Stop a running transfer (mandatory).

Parameters

in	*chan*	Channel to stop

Returns

RT_EOK on success

7.57.2.6 config

`rt_err_t(*) rt_dma_controller_ops::config(struct rt_dma_chan *chan, struct rt_dma_slave_config *conf)`
Configure a DMA channel (mandatory).

Parameters

in	*chan*	Channel to configure
in	*conf*	Slave configuration

Returns

RT_EOK on success

7.57.2.7 prep_memcpy

```
rt_err_t(*) rt_dma_controller_ops::prep_memcpy(struct rt_dma_chan *chan, rt_ubase_t dma_addr_↩
src, rt_ubase_t dma_addr_dst, rt_size_t len)
```

Prepare a memory-to-memory transfer (optional).

Parameters

in	*chan*	DMA channel
in	*dma_addr_src*	Source physical address
in	*dma_addr_dst*	Destination physical address
in	*len*	Transfer length in bytes

Returns

RT_EOK on success

7.57.2.8 prep_cyclic

```
rt_err_t(*) rt_dma_controller_ops::prep_cyclic(struct rt_dma_chan *chan, rt_ubase_t dma_buf_↩
addr, rt_size_t buf_len, rt_size_t period_len, enum rt_dma_transfer_direction dir)
```

Prepare a cyclic (repeating) transfer (optional).

Parameters

in	*chan*	DMA channel
in	*dma_buf_addr*	Buffer physical address
in	*buf_len*	Total buffer length in bytes
in	*period_len*	Bytes per period (callback interval)
in	*dir*	Transfer direction

Returns

RT_EOK on success

7.57.2.9 prep_single

```
rt_err_t(*) rt_dma_controller_ops::prep_single(struct rt_dma_chan *chan, rt_ubase_t dma_buf_↩
addr, rt_size_t buf_len, enum rt_dma_transfer_direction dir)
```

Prepare a single (one-shot) transfer (optional).

Parameters

in	*chan*	DMA channel
in	*dma_buf_addr*	Buffer physical address
in	*buf_len*	Transfer length in bytes
in	*dir*	Transfer direction

Returns

RT_EOK on success

7.58 rt_dma_map_ops Struct Reference

`#include <dma.h>`

Data Fields

- void *(*) alloc (struct rt_device *dev, rt_size_t size, rt_ubase_t *dma_handle, rt_ubase_t flags)
- void(*) free (struct rt_device *dev, rt_size_t size, void *cpu_addr, rt_ubase_t dma_handle, rt_ubase_t flags)
- rt_err_t(*) sync_out_data (struct rt_device *dev, void *data, rt_size_t size, rt_ubase_t *dma_handle, rt_↩ ubase_t flags)
- rt_err_t(*) sync_in_data (struct rt_device *dev, void *out_data, rt_size_t size, rt_ubase_t dma_handle, rt_↩ ubase_t flags)

7.58.1 Detailed Description

DMA memory mapping operations vtable.
Abstracts device-specific DMA memory management: allocation, freeing, and cache synchronization for coherent and non-coherent devices. Devices may override these via dev->dma_ops or the device tree.

7.58.2 Field Documentation

7.58.2.1 alloc

`void *(*) rt_dma_map_ops::alloc(struct rt_device *dev, rt_size_t size, rt_ubase_t *dma_handle, rt_ubase_t flags)`
Allocate a DMA buffer (optional, may fall back to default).

Parameters

in	*dev*	DMA-capable device
in	*size*	Requested size in bytes
out	*dma_handle*	Physical DMA address
in	*flags*	Allocation flags (RT_DMA_F_*)

Returns

Kernel virtual address, or RT_NULL on failure

7.58.2.2 free

`void(*) rt_dma_map_ops::free(struct rt_device *dev, rt_size_t size, void *cpu_addr, rt_ubase_t dma_handle, rt_ubase_t flags)`
Free a DMA buffer (optional).

Parameters

in	*dev*	DMA-capable device
in	*size*	Buffer size in bytes
in	*cpu_addr*	Kernel virtual address
in	*dma_handle*	Physical DMA address
in	*flags*	Allocation flags

7.58.2.3 sync_out_data

```
rt_err_t(*) rt_dma_map_ops::sync_out_data(struct rt_device *dev, void *data, rt_size_t size,
rt_ubase_t *dma_handle, rt_ubase_t flags)
```

Synchronize data for DMA output (CPU → device).
Performs any necessary cache flush and returns the DMA bus address.

Parameters

in	*dev*	DMA-capable device
in	*data*	Virtual address of data buffer
in	*size*	Buffer size in bytes
out	*dma_handle*	DMA bus address (may be NULL)
in	*flags*	Sync flags

Returns

RT_EOK on success

7.58.2.4 sync_in_data

```
rt_err_t(*) rt_dma_map_ops::sync_in_data(struct rt_device *dev, void *out_data, rt_size_t size,
rt_ubase_t dma_handle, rt_ubase_t flags)
```

Synchronize data for DMA input (device → CPU).
Performs any necessary cache invalidation.

Parameters

in	*dev*	DMA-capable device
in	*out_data*	Virtual address of data buffer
in	*size*	Buffer size in bytes
in	*dma_handle*	DMA bus address
in	*flags*	Sync flags

Returns

RT_EOK on success

7.59 rt_dma_pool Struct Reference

```
#include <dma.h>
```

Data Fields

- rt_region_t region
- rt_list_t list
- rt_ubase_t flags
- rt_bitmap_t * map
- rt_size_t bits
- rt_ubase_t start
- struct rt_device * dev

7.59.1 Detailed Description

DMA memory pool descriptor.
Manages a contiguous physical memory region for DMA allocations. Uses a bitmap to track free/used pages. Pools are organized in a global linked list.

7.59.2 Field Documentation

7.59.2.1 region

`rt_region_t rt_dma_pool::region`
Memory region (name, start, end)

7.59.2.2 list

`rt_list_t rt_dma_pool::list`
Node in the global pool list

7.59.2.3 flags

`rt_ubase_t rt_dma_pool::flags`
Pool flags (RT_DMA_F_LINEAR, RT_DMA_F_32BITS, etc.)

7.59.2.4 map

`rt_bitmap_t* rt_dma_pool::map`
Page allocation bitmap (1 bit per page)

7.59.2.5 bits

`rt_size_t rt_dma_pool::bits`
Total number of pages in this pool

7.59.2.6 start

`rt_ubase_t rt_dma_pool::start`
Page-aligned start address

7.59.2.7 dev

`struct rt_device* rt_dma_pool::dev`
Device that owns this pool (for RT_DMA_F_DEVICE)

7.60 rt_dma_slave_config Struct Reference

`#include <dma.h>`

Data Fields

- enum rt_dma_transfer_direction direction
- enum rt_dma_slave_buswidth src_addr_width
- enum rt_dma_slave_buswidth dst_addr_width
- rt_ubase_t src_addr
- rt_ubase_t dst_addr
- rt_uint32_t src_maxburst
- rt_uint32_t dst_maxburst
- rt_uint32_t src_port_window_size
- rt_uint32_t dst_port_window_size

7.60.1 Detailed Description

DMA slave channel configuration.
Passed to rt_dma_chan_config() to set up a channel before starting transfers. Defines transfer direction, address widths, fixed source/ destination addresses, and burst sizes.

7.60.2 Field Documentation

7.60.2.1 direction

`enum rt_dma_transfer_direction rt_dma_slave_config::direction`
Transfer direction (MEM_TO_DEV, DEV_TO_MEM, etc.)

7.60.2.2 src_addr_width

`enum rt_dma_slave_buswidth rt_dma_slave_config::src_addr_width`
Source bus width in bytes

7.60.2.3 dst_addr_width

`enum rt_dma_slave_buswidth rt_dma_slave_config::dst_addr_width`
Destination bus width in bytes

7.60.2.4 src_addr

`rt_ubase_t rt_dma_slave_config::src_addr`
Fixed source address (peripheral FIFO register)

7.60.2.5 dst_addr

`rt_ubase_t rt_dma_slave_config::dst_addr`
Fixed destination address (peripheral FIFO register)

7.60.2.6 src_maxburst

`rt_uint32_t rt_dma_slave_config::src_maxburst`
Maximum source burst size in transfer-width units

7.60.2.7 dst_maxburst

`rt_uint32_t rt_dma_slave_config::dst_maxburst`
Maximum destination burst size in transfer-width units

7.60.2.8 src_port_window_size

`rt_uint32_t rt_dma_slave_config::src_port_window_size`
Source port window size (for scatter-gather)

7.60.2.9 dst_port_window_size

`rt_uint32_t rt_dma_slave_config::dst_port_window_size`
Destination port window size (for scatter-gather)

7.61 rt_dma_slave_transfer Struct Reference

`#include <dma.h>`

Data Fields

- rt_ubase_t src_addr
- rt_ubase_t dst_addr
- void * buffer
- rt_ubase_t dma_handle
- rt_size_t buffer_len
- rt_size_t period_len

7.61.1 Detailed Description

DMA slave transfer descriptor.
Describes a single DMA transfer: source/destination buffer addresses, total buffer length, and period length for cyclic transfers. Passed to rt_dma_prep_memcpy(), rt_dma_prep_cyclic(), or rt_dma_prep_single().

7.61.2 Field Documentation

7.61.2.1 src_addr

```
rt_ubase_t rt_dma_slave_transfer::src_addr
```
Source buffer physical address

7.61.2.2 dst_addr

```
rt_ubase_t rt_dma_slave_transfer::dst_addr
```
Destination buffer physical address

7.61.2.3 buffer

```
void* rt_dma_slave_transfer::buffer
```
Virtual address of the DMA buffer (if mapped)

7.61.2.4 dma_handle

```
rt_ubase_t rt_dma_slave_transfer::dma_handle
```
DMA bus address of the buffer

7.61.2.5 buffer_len

```
rt_size_t rt_dma_slave_transfer::buffer_len
```
Total buffer length in bytes

7.61.2.6 period_len

```
rt_size_t rt_dma_slave_transfer::period_len
```
Period length in bytes (for cyclic transfers only)

7.62 rt_hw_backtrace_frame Struct Reference

```
#include <rthw.h>
```

7.62.1 Detailed Description

Hardware Layer Backtrace Service

7.63 rt_i2c_bus_device Struct Reference

```
#include <dev_i2c.h>
```

7.63.1 Detailed Description

I2C Bus Device.

7.64 rt_i2c_bus_device_ops Struct Reference

`#include <dev_i2c.h>`

7.64.1 Detailed Description

I2C Bus Device Operations.

7.65 rt_i2c_client Struct Reference

`#include <dev_i2c.h>`

7.65.1 Detailed Description

I2C Client.

7.66 rt_i2c_configuration Struct Reference

`#include <dev_i2c.h>`

7.66.1 Detailed Description

I2C configuration structure. mode : master: 0x00; slave: 0x01; max_hz: Maximum limit baud rate. usage_freq: Actual usage baud rate.

7.67 rt_i2c_msg Struct Reference

`#include <dev_i2c.h>`

7.67.1 Detailed Description

I2C Message.

7.68 rt_i2c_priv_data Struct Reference

`#include <dev_i2c.h>`

7.68.1 Detailed Description

I2C Private Data.

7.69 rt_inputcapture_ops Struct Reference

`#include <rt_inputcapture.h>`

7.69.1 Detailed Description

capture operators

7.70 rt_interrupt_context Struct Reference

`#include <rtdef.h>`

Data Fields

- void * context
- rt_slist_t node

7.70.1 Detailed Description

interrupt/exception frame handling

7.70.2 Field Documentation

7.70.2.1 context

`void* rt_interrupt_context::context`
arch specific context

7.70.2.2 node

`rt_slist_t rt_interrupt_context::node`
node for nested interrupt

7.71 rt_ipc_object Struct Reference

`#include <rtdef.h>`

Data Fields

- struct rt_object parent
- rt_list_t suspend_thread

7.71.1 Detailed Description

Base structure of IPC object

7.71.2 Field Documentation

7.71.2.1 parent

`struct rt_object rt_ipc_object::parent`
inherit from rt_object

7.71.2.2 suspend_thread

`rt_list_t rt_ipc_object::suspend_thread`
threads pended on this resource

7.72 rt_list_node Struct Reference

`#include <rttypes.h>`

Data Fields

- struct rt_list_node * next
- struct rt_list_node * prev

7.72.1 Detailed Description

Double List structure

7.72.2 Field Documentation

7.72.2.1 next

`struct rt_list_node* rt_list_node::next`
point to next node.

7.72.2.2 prev

`struct rt_list_node* rt_list_node::prev`
point to prev node.

7.73 rt_lockless_slist_node Struct Reference

`#include <rttypes.h>`

Data Fields

- rt_atomic_t next

7.73.1 Detailed Description

Lock-less Single List structure

7.73.2 Field Documentation

7.73.2.1 next

`rt_atomic_t rt_lockless_slist_node::next`
point to next node.

7.74 rt_mailbox Struct Reference

`#include <rtdef.h>`

Data Fields

- struct rt_ipc_object parent
- rt_ubase_t * msg_pool
- rt_uint16_t size
- rt_uint16_t entry
- rt_uint16_t in_offset
- rt_uint16_t out_offset
- rt_list_t suspend_sender_thread

7.74.1 Detailed Description

mailbox structure

7.74.2 Field Documentation

7.74.2.1 parent

`struct rt_ipc_object rt_mailbox::parent`
inherit from ipc_object

7.74.2.2 msg_pool

`rt_ubase_t* rt_mailbox::msg_pool`
start address of message buffer

7.74.2.3 size

`rt_uint16_t rt_mailbox::size`
size of message pool

7.74.2.4 entry

`rt_uint16_t rt_mailbox::entry`
index of messages in msg_pool

7.74.2.5 in_offset

`rt_uint16_t rt_mailbox::in_offset`
input offset of the message buffer

7.74.2.6 out_offset

`rt_uint16_t rt_mailbox::out_offset`
output offset of the message buffer

7.74.2.7 suspend_sender_thread

`rt_list_t rt_mailbox::suspend_sender_thread`
sender thread suspended on this mailbox

7.75 rt_mempool Struct Reference

`#include <rtdef.h>`

Data Fields

- struct rt_object parent
- void * start_address
- rt_size_t size
- rt_size_t block_size
- rt_uint8_t * block_list
- rt_size_t block_total_count
- rt_size_t block_free_count
- rt_list_t suspend_thread

7.75.1 Detailed Description

Base structure of Memory pool object

7.75.2 Field Documentation

7.75.2.1 parent

`struct rt_object rt_mempool::parent`
inherit from rt_object

7.75.2.2 start_address

`void* rt_mempool::start_address`
memory pool start

7.75.2.3 size

`rt_size_t rt_mempool::size`
size of memory pool

7.75.2.4 block_size

`rt_size_t rt_mempool::block_size`
size of memory blocks

7.75.2.5 block_list

`rt_uint8_t* rt_mempool::block_list`
memory blocks list

7.75.2.6 block_total_count

`rt_size_t rt_mempool::block_total_count`
numbers of memory block

7.75.2.7 block_free_count

`rt_size_t rt_mempool::block_free_count`
numbers of free memory block

7.75.2.8 suspend_thread

`rt_list_t rt_mempool::suspend_thread`
threads pended on this resource

7.76 rt_messagequeue Struct Reference

`#include <rtdef.h>`

Data Fields

- struct rt_ipc_object parent
- void * msg_pool
- rt_uint16_t msg_size
- rt_uint16_t max_msgs
- rt_uint16_t entry
- void * msg_queue_head
- void * msg_queue_tail
- void * msg_queue_free
- rt_list_t suspend_sender_thread

7.76.1 Detailed Description

message queue structure

7.76.2 Field Documentation

7.76.2.1 parent

`struct rt_ipc_object rt_messagequeue::parent`
inherit from ipc_object

7.76.2.2 msg_pool

`void* rt_messagequeue::msg_pool`
start address of message queue

7.76.2.3 msg_size

`rt_uint16_t rt_messagequeue::msg_size`
message size of each message

7.76.2.4 max_msgs

`rt_uint16_t rt_messagequeue::max_msgs`
max number of messages

7.76.2.5 entry

`rt_uint16_t rt_messagequeue::entry`
index of messages in the queue

7.76.2.6 msg_queue_head

`void* rt_messagequeue::msg_queue_head`
list head

7.76.2.7 msg_queue_tail

`void* rt_messagequeue::msg_queue_tail`
list tail

7.76.2.8 msg_queue_free

`void* rt_messagequeue::msg_queue_free`
pointer indicated the free node of queue

7.76.2.9 suspend_sender_thread

`rt_list_t rt_messagequeue::suspend_sender_thread`
sender thread suspended on this message queue

7.77 rt_mutex Struct Reference

`#include <rtdef.h>`

Data Fields

- struct rt_ipc_object parent
- rt_uint8_t ceiling_priority
- rt_uint8_t priority
- rt_uint8_t hold
- rt_uint8_t reserved
- struct rt_thread * owner
- rt_list_t taken_list

7.77.1 Detailed Description

Mutual exclusion (mutex) structure

7.77.2 Field Documentation

7.77.2.1 parent

`struct rt_ipc_object rt_mutex::parent`
inherit from ipc_object

7.77.2.2 ceiling_priority

`rt_uint8_t rt_mutex::ceiling_priority`
the priority ceiling of mutexe

7.77.2.3 priority

`rt_uint8_t rt_mutex::priority`
the maximal priority for pending thread

7.77.2.4 hold

`rt_uint8_t rt_mutex::hold`
numbers of thread hold the mutex

7.77.2.5 reserved

`rt_uint8_t rt_mutex::reserved`
reserved field

7.77.2.6 owner

`struct rt_thread* rt_mutex::owner`
current owner of mutex

7.77.2.7 taken_list

`rt_list_t rt_mutex::taken_list`
the object list taken by thread

7.78 rt_object Struct Reference

`#include <rtdef.h>`

Data Fields

- const char * name
- rt_uint8_t type
- rt_uint8_t flag
- void * module_id
- rt_list_t list

7.78.1 Detailed Description

Base structure of Kernel object

7.78.2 Field Documentation

7.78.2.1 name

`const char* rt_object::name`
static name of kernel object

7.78.2.2 type

`rt_uint8_t rt_object::type`
type of kernel object

7.78.2.3 flag

`rt_uint8_t rt_object::flag`
flag of kernel object

7.78.2.4 module_id

`void* rt_object::module_id`
id of application module

7.78.2.5 list

`rt_list_t rt_object::list`
list node of kernel object

7.79 rt_object_information Struct Reference

`#include <rtdef.h>`

Data Fields

- enum rt_object_class_type type
- rt_list_t object_list
- rt_size_t object_size

7.79.1 Detailed Description

The information of the kernel object

7.79.2 Field Documentation

7.79.2.1 type

`enum rt_object_class_type rt_object_information::type`
object class type

7.79.2.2 object_list

`rt_list_t rt_object_information::object_list`
object list

7.79.2.3 object_size

`rt_size_t rt_object_information::object_size`
object size

7.80 rt_pin_irq_hdr Struct Reference

`#include <dev_pin.h>`

7.80.1 Detailed Description

pin irq structure

7.81 rt_pin_ops Struct Reference

`#include <dev_pin.h>`

7.81.1 Detailed Description

pin device operations

7.82 rt_pm Struct Reference

`#include <pm.h>`

7.82.1 Detailed Description

power management

7.83 rt_pm_ops Struct Reference

`#include <pm.h>`

7.83.1 Detailed Description

low power mode operations

7.84 rt_pwm_configuration Struct Reference

`#include <dev_pwm.h>`

7.84.1 Detailed Description

PWM configuration.

7.85 rt_pwm_ops Struct Reference

`#include <dev_pwm.h>`

7.85.1 Detailed Description

PWM operations.

7.86 rt_qspi_configuration Struct Reference

`#include <dev_spi.h>`

7.86.1 Detailed Description

QSPI configuration structure.

7.87 rt_qspi_device Struct Reference

`#include <dev_spi.h>`

7.87.1 Detailed Description

QSPI operators.

7.88 rt_qspi_message Struct Reference

`#include <dev_spi.h>`

7.88.1 Detailed Description

QSPI message structure.

7.89 rt_rtc_device Struct Reference

`#include <dev_rtc.h>`

7.89.1 Detailed Description

RTC device structure.

7.90 rt_rtc_ops Struct Reference

`#include <dev_rtc.h>`

7.90.1 Detailed Description

RTC operations.

7.91 rt_rtc_wkalarm Struct Reference

`#include <dev_rtc.h>`

7.91.1 Detailed Description

RTC alarm structure.

7.92 rt_sched_thread_ctx Struct Reference

`#include <rtsched.h>`

Data Fields

- rt_list_t thread_list_node
- rt_uint8_t stat
- rt_uint8_t sched_flag_locked:1
- rt_uint8_t sched_flag_ttmr_set:1
- struct rt_sched_thread_priv sched_thread_priv

7.92.1 Detailed Description

Scheduler public status binding on thread. Caller must hold the scheduler lock before access any one of its member.

7.92.2 Field Documentation

7.92.2.1 thread_list_node

`rt_list_t rt_sched_thread_ctx::thread_list_node`
node in thread list

7.92.2.2 stat

`rt_uint8_t rt_sched_thread_ctx::stat`
thread status

7.92.2.3 sched_flag_locked

`rt_uint8_t rt_sched_thread_ctx::sched_flag_locked`
calling thread have the scheduler locked

7.92.2.4 sched_flag_ttmr_set

`rt_uint8_t rt_sched_thread_ctx::sched_flag_ttmr_set`
thread timer is start

7.92.2.5 sched_thread_priv

`struct rt_sched_thread_priv rt_sched_thread_ctx::sched_thread_priv`
private context of scheduler

7.93 rt_sched_thread_priv Struct Reference

`#include <rtsched.h>`

Data Fields

- rt_tick_t init_tick
- rt_tick_t remaining_tick
- rt_uint8_t current_priority
- rt_uint8_t init_priority
- rt_uint32_t number_mask

7.93.1 Detailed Description

Scheduler private status binding on thread. Caller should never accessing these members.

7.93.2 Field Documentation

7.93.2.1 init_tick

`rt_tick_t rt_sched_thread_priv::init_tick`
thread's initialized tick

7.93.2.2 remaining_tick

`rt_tick_t rt_sched_thread_priv::remaining_tick`
remaining tick

7.93.2.3 current_priority

`rt_uint8_t rt_sched_thread_priv::current_priority`
current priority

7.93.2.4 init_priority

`rt_uint8_t rt_sched_thread_priv::init_priority`
initialized priority

7.93.2.5 number_mask

`rt_uint32_t rt_sched_thread_priv::number_mask`
priority number mask

7.94 rt_semaphore Struct Reference

`#include <rtdef.h>`

Data Fields

- struct rt_ipc_object parent
- rt_uint16_t value

7.94.1 Detailed Description

Semaphore structure

7.94.2 Field Documentation

7.94.2.1 parent

`struct rt_ipc_object rt_semaphore::parent`
inherit from ipc_object

7.94.2.2 value

`rt_uint16_t rt_semaphore::value`
value of semaphore.

7.95 rt_serial_device Struct Reference

`#include <dev_serial_v2.h>`

7.95.1 Detailed Description

serial device structure

7.96 rt_serial_rx_fifo Struct Reference

`#include <dev_serial_v2.h>`

7.96.1 Detailed Description

Serial Receive FIFO mode.

7.97 rt_serial_tx_fifo Struct Reference

`#include <dev_serial_v2.h>`

7.97.1 Detailed Description

Serial Transmit FIFO mode.

7.98 rt_slist_node Struct Reference

`#include <rttypes.h>`

Data Fields

- struct rt_slist_node * next

7.98.1 Detailed Description

Single List structure

7.98.2 Field Documentation

7.98.2.1 next

`struct rt_slist_node* rt_slist_node::next`
point to next node.

7.99 rt_spi_bus Struct Reference

`#include <dev_spi.h>`

7.99.1 Detailed Description

SPI bus structure.

7.100 rt_spi_configuration Struct Reference

`#include <dev_spi.h>`

7.100.1 Detailed Description

SPI configuration structure.

7.101 rt_spi_device Struct Reference

`#include <dev_spi.h>`

7.101.1 Detailed Description

SPI Virtual BUS, one device must connected to a virtual BUS.

7.102 rt_spi_message Struct Reference

`#include <dev_spi.h>`

7.102.1 Detailed Description

SPI message structure.

7.103 rt_spi_ops Struct Reference

`#include <dev_spi.h>`

7.103.1 Detailed Description

SPI operators.

7.104 rt_spinlock Struct Reference

`#include <rttypes.h>`

7.104.1 Detailed Description

Spinlock

7.105 rt_thread Struct Reference

`#include <rtdef.h>`

Data Fields

- void * sp
- void * entry
- void * parameter
- void * stack_addr
- rt_uint32_t stack_size
- rt_err_t error
- RT_SCHED_THREAD_CTX struct rt_timer thread_timer
- rt_thread_cleanup_t cleanup
- rt_sigset_t sig_pending
- rt_sigset_t sig_mask
- void * sig_ret
- rt_sighandler_t * sig_vectors
- void * si_list
- rt_ubase_t user_data

7.105.1 Detailed Description

Thread Control Block.

7.105.2 Field Documentation

7.105.2.1 sp

`void* rt_thread::sp`
stack point

7.105.2.2 entry

`void* rt_thread::entry`
entry

7.105.2.3 parameter

`void* rt_thread::parameter`
parameter

7.105.2.4 stack_addr

```
void* rt_thread::stack_addr
```
stack address

7.105.2.5 stack_size

```
rt_uint32_t rt_thread::stack_size
```
stack size

7.105.2.6 error

```
rt_err_t rt_thread::error
```
error code

7.105.2.7 thread_timer

```
RT_SCHED_THREAD_CTX struct rt_timer rt_thread::thread_timer
```
built-in thread timer

7.105.2.8 cleanup

```
rt_thread_cleanup_t rt_thread::cleanup
```
cleanup function when thread exit

7.105.2.9 sig_pending

```
rt_sigset_t rt_thread::sig_pending
```
the pending signals

7.105.2.10 sig_mask

```
rt_sigset_t rt_thread::sig_mask
```
the mask bits of signal

7.105.2.11 sig_ret

```
void* rt_thread::sig_ret
```
the return stack pointer from signal

7.105.2.12 sig_vectors

```
rt_sighandler_t* rt_thread::sig_vectors
```
vectors of signal handler

7.105.2.13 si_list

```
void* rt_thread::si_list
```
the signal infor list

7.105.2.14 user_data

```
rt_ubase_t rt_thread::user_data
```
private user data beyond this thread

7.106 rt_timer Struct Reference

```
#include <rtdef.h>
```

Data Fields

- struct rt_object parent
- rt_timer_func_t timeout_func
- void * parameter
- rt_tick_t init_tick
- rt_tick_t timeout_tick

7.106.1 Detailed Description

timer structure

7.106.2 Field Documentation

7.106.2.1 parent

`struct rt_object rt_timer::parent`
inherit from rt_object

7.106.2.2 timeout_func

`rt_timer_func_t rt_timer::timeout_func`
timeout function

7.106.2.3 parameter

`void* rt_timer::parameter`
timeout function's parameter

7.106.2.4 init_tick

`rt_tick_t rt_timer::init_tick`
timer timeout tick

7.106.2.5 timeout_tick

`rt_tick_t rt_timer::timeout_tick`
timeout tick

7.107 rt_touch_config Struct Reference

`#include <dev_touch.h>`

7.107.1 Detailed Description

Touch configuration.

7.108 rt_touch_data Struct Reference

`#include <dev_touch.h>`

7.108.1 Detailed Description

Touch data.

7.109 rt_touch_device Struct Reference

`#include <dev_touch.h>`

7.109.1 Detailed Description

Touch device.

7.110 rt_touch_info Struct Reference

`#include <dev_touch.h>`

7.110.1 Detailed Description

Touch information.

7.111 rt_touch_ops Struct Reference

`#include <dev_touch.h>`

7.111.1 Detailed Description

Touch device operations.

7.112 rt_uart_ops Struct Reference

`#include <dev_serial_v2.h>`

7.112.1 Detailed Description

Configure the serial device.
uart device operations

7.113 rt_wqueue Struct Reference

`#include <rtdef.h>`

7.113.1 Detailed Description

WaitQueue structure

7.114 scmi_clk_attributes Struct Reference

`#include <scmi.h>`

7.114.1 Detailed Description

struct scmi_clk_attributes - Response payload for SCMI_COM_MSG_ATTRIBUTES command

7.115 scmi_clk_describe_attributes_in Struct Reference

`#include <scmi.h>`

7.115.1 Detailed Description

struct scmi_clk_describe_attributes_in - Message payload for SCMI_CLOCK_ATTRIBUTES command

7.116 scmi_clk_describe_attributes_out Struct Reference

`#include <scmi.h>`

7.116.1 Detailed Description

struct scmi_clk_describe_attributes_out - Response payload for SCMI_CLOCK_ATTRIBUTES command clock in response to a clock enable request from an agent

7.117 scmi_clk_describe_rates_in Struct Reference

`#include <scmi.h>`

7.117.1 Detailed Description

struct scmi_clk_describe_rates_in - Message payload for SCMI_CLOCK_DESCRIBE_RATES command

7.118 scmi_clk_describe_rates_out Struct Reference

`#include <scmi.h>`

7.118.1 Detailed Description

struct scmi_clk_describe_rates_out - Response payload for SCMI_CLOCK_DESCRIBE_RATES command

7.119 scmi_clk_name_in Struct Reference

`#include <scmi.h>`

7.119.1 Detailed Description

struct scmi_clk_name_in - Message payload for SCMI_CLOCK_NAME_GET command

7.120 scmi_clk_name_out Struct Reference

`#include <scmi.h>`

7.120.1 Detailed Description

struct scmi_clk_name_out - Response payload for SCMI_CLOCK_NAME_GET command

7.121 scmi_clk_rate_get_in Struct Reference

`#include <scmi.h>`

7.121.1 Detailed Description

struct scmi_clk_state_in - Message payload for SCMI_CLOCK_RATE_GET command

7.122 scmi_clk_rate_get_out Struct Reference

`#include <scmi.h>`

7.122.1 Detailed Description

struct scmi_clk_rate_get_out - Response payload for SCMI_CLOCK_RATE_GET command

7.123 scmi_clk_rate_set_in Struct Reference

`#include <scmi.h>`

7.123.1 Detailed Description

struct scmi_clk_state_in - Message payload for SCMI_CLOCK_RATE_SET command

7.124 scmi_clk_rate_set_out Struct Reference

`#include <scmi.h>`

7.124.1 Detailed Description

struct scmi_clk_rate_set_out - Response payload for SCMI_CLOCK_RATE_SET command

7.125 scmi_clk_state_in Struct Reference

`#include <scmi.h>`

7.125.1 Detailed Description

struct scmi_clk_state_in - Message payload for SCMI_CLOCK_CONFIG_SET command

7.126 scmi_clk_state_out Struct Reference

`#include <scmi.h>`

7.126.1 Detailed Description

struct scmi_clk_state_out - Response payload for SCMI_CLOCK_CONFIG_SET command

7.127 scmi_pinctrl_attributes_in Struct Reference

`#include <scmi.h>`

7.127.1 Detailed Description

struct scmi_pinctrl_attributes_in - Message payload for SCMI_PINCTRL_ATTRIBUTES command

7.128 scmi_pinctrl_attributes_out Struct Reference

`#include <scmi.h>`

7.128.1 Detailed Description

struct scmi_pinctrl_attributes_out - Response payload for SCMI_PINCTRL_ATTRIBUTES command

7.129 scmi_pinctrl_list_assoc_in Struct Reference

`#include <scmi.h>`

7.129.1 Detailed Description

struct scmi_pinctrl_list_assoc_in - Message payload for SCMI_PINCTRL_LIST_ASSOCIATIONS command

7.130 scmi_pinctrl_list_assoc_out Struct Reference

`#include <scmi.h>`

7.130.1 Detailed Description

struct scmi_pinctrl_list_assoc_out - Response payload for SCMI_PINCTRL_LIST_ASSOCIATIONS command

7.131 scmi_pinctrl_name_get_in Struct Reference

`#include <scmi.h>`

7.131.1 Detailed Description

struct scmi_pinctrl_request_in - Message payload for SCMI_PINCTRL_NAME_GET command

7.132 scmi_pinctrl_name_get_out Struct Reference

`#include <scmi.h>`

7.132.1 Detailed Description

struct scmi_pinctrl_name_get_out - Response payload for SCMI_PINCTRL_NAME_GET command

7.133 scmi_pinctrl_protocol_attributes Struct Reference

`#include <scmi.h>`

7.133.1 Detailed Description

struct scmi_pinctrl_protocol_attributes - Response payload for SCMI_COM_MSG_ATTRIBUTES command

7.134 scmi_pinctrl_request_in Struct Reference

`#include <scmi.h>`

7.134.1 Detailed Description

struct scmi_pinctrl_request_in - Message payload for SCMI_PINCTRL_REQUEST command

7.135 scmi_pinctrl_request_out Struct Reference

`#include <scmi.h>`

7.135.1 Detailed Description

struct scmi_pinctrl_request_out - Response payload for SCMI_PINCTRL_REQUEST command

7.136 scmi_pinctrl_settings_conf_in Struct Reference

`#include <scmi.h>`

7.136.1 Detailed Description

struct scmi_pinctrl_settings_conf_in - Message payload for SCMI_PINCTRL_SETTINGS_CONFIGURE command

7.137 scmi_pinctrl_settings_conf_out Struct Reference

`#include <scmi.h>`

7.137.1 Detailed Description

struct scmi_pinctrl_settings_conf_out - Response payload for SCMI_PINCTRL_SETTINGS_CONFIGURE command

7.138 scmi_pinctrl_settings_get_in Struct Reference

`#include <scmi.h>`

7.138.1 Detailed Description

struct scmi_pinctrl_settings_get_in - Message payload for SCMI_PINCTRL_SETTINGS_GET command

7.139 scmi_pinctrl_settings_get_out Struct Reference

`#include <scmi.h>`

7.139.1 Detailed Description

struct scmi_pinctrl_settings_get_out - Response payload for SCMI_PINCTRL_SETTINGS_GET command

7.140 scmi_power_attributes Struct Reference

`#include <scmi.h>`

7.140.1 Detailed Description

struct scmi_power_attributes - Response payload for SCMI_COM_MSG_ATTRIBUTES command

7.141 scmi_power_state_set_in Struct Reference

`#include <scmi.h>`

7.141.1 Detailed Description

struct scmi_power_state_set_in - Message payload for SCMI_POWER_STATE_SET command

7.142 scmi_power_state_set_out Struct Reference

`#include <scmi.h>`

7.142.1 Detailed Description

struct scmi_power_state_set_out - Response payload for SCMI_POWER_STATE_SET command

7.143 scmi_reset_attr_in Struct Reference

`#include <scmi.h>`

7.143.1 Detailed Description

struct scmi_reset_attr_in - Payload for SCMI_RESET_DOMAIN_ATTRIBUTES message

7.144 scmi_reset_attr_out Struct Reference

`#include <scmi.h>`

7.144.1 Detailed Description

struct scmi_reset_attr_out - Payload for SCMI_RESET_DOMAIN_ATTRIBUTES response

7.145 scmi_reset_in Struct Reference

`#include <scmi.h>`

7.145.1 Detailed Description

struct scmi_reset_in - Message payload for SCMI_RESET_RESET command

7.146 scmi_reset_out Struct Reference

`#include <scmi.h>`

7.146.1 Detailed Description

struct scmi_reset_out - Response payload for SCMI_RESET_RESET command

7.147 scmi_scmi_sensor_axis_description_get_in Struct Reference

`#include <scmi.h>`

7.147.1 Detailed Description

struct scmi_scmi_sensor_axis_description_get_in - Payload for SCMI_SENSOR_AXIS_DESCRIPTION_GET command

7.148 scmi_scmi_sensor_axis_description_get_out Struct Reference

`#include <scmi.h>`

7.148.1 Detailed Description

struct scmi_scmi_sensor_axis_description_get_out - Response payload for SCMI_SENSOR_AXIS_DESCRIPTION↩
_GET command

7.149 scmi_sensor_attributes Struct Reference

`#include <scmi.h>`

7.149.1 Detailed Description

struct scmi_sensor_attributes - Response payload for SCMI_COM_MSG_ATTRIBUTES command

7.150 scmi_sensor_axis_name_in Struct Reference

`#include <scmi.h>`

7.150.1 Detailed Description

struct scmi_sensor_axis_name_in - Payload for SCMI_SENSOR_AXIS_NAME_GET command

7.151 scmi_sensor_axis_name_out Struct Reference

`#include <scmi.h>`

7.151.1 Detailed Description

struct scmi_sensor_axis_name_out - Response payload for SCMI_SENSOR_AXIS_NAME_GET command

7.152 scmi_sensor_config_get_in Struct Reference

`#include <scmi.h>`

7.152.1 Detailed Description

struct scmi_sensor_config_get_in - Payload for SCMI_SENSOR_CONFIG_GET command

7.153 scmi_sensor_config_get_out Struct Reference

`#include <scmi.h>`

7.153.1 Detailed Description

struct scmi_sensor_config_get_out - Response payload for SCMI_SENSOR_CONFIG_GET command

7.154 scmi_sensor_config_set_in Struct Reference

`#include <scmi.h>`

7.154.1 Detailed Description

struct scmi_sensor_config_set_in - Payload for SCMI_SENSOR_CONFIG_SET command

7.155 scmi_sensor_config_set_out Struct Reference

`#include <scmi.h>`

7.155.1 Detailed Description

struct scmi_sensor_config_set_out - Response payload for SCMI_SENSOR_CONFIG_SET command

7.156 scmi_sensor_continuous_update_notify_in Struct Reference

`#include <scmi.h>`

7.156.1 Detailed Description

struct scmi_sensor_continuous_update_notify_in - Payload for SCMI_SENSOR_CONTINUOUS_UPDATE_↩ NOTIFY command

7.157 scmi_sensor_continuous_update_notify_out Struct Reference

`#include <scmi.h>`

7.157.1 Detailed Description

struct scmi_sensor_continuous_update_notify_out - Response payload for SCMI_SENSOR_CONTINUOUS_↩ UPDATE_NOTIFY command

7.158 scmi_sensor_description_get_in Struct Reference

`#include <scmi.h>`

7.158.1 Detailed Description

struct scmi_sensor_description_get_in - Payload for SCMI_SENSOR_DESCRIPTION_GET command

7.159 scmi_sensor_description_get_out Struct Reference

`#include <scmi.h>`

7.159.1 Detailed Description

struct scmi_sensor_description_get_out - Response payload for SCMI_SENSOR_DESCRIPTION_GET command

7.160 scmi_sensor_list_update_intervals_in Struct Reference

`#include <scmi.h>`

7.160.1 Detailed Description

struct scmi_sensor_list_update_intervals_in - Payload for SCMI_SENSOR_LIST_UPDATE_INTERVALS command

7.161 scmi_sensor_list_update_intervals_out Struct Reference

`#include <scmi.h>`

7.161.1 Detailed Description

struct scmi_sensor_list_update_intervals_out - Response payload for SCMI_SENSOR_LIST_UPDATE_↩ INTERVALS command

7.162 scmi_sensor_name_in Struct Reference

`#include <scmi.h>`

7.162.1 Detailed Description

struct scmi_sensor_name_in - Payload for SCMI_SENSOR_NAME_GET command

7.163 scmi_sensor_name_out Struct Reference

`#include <scmi.h>`

7.163.1 Detailed Description

struct scmi_sensor_name_out - Response payload for SCMI_SENSOR_NAME_GET command

7.164 scmi_sensor_reading_in Struct Reference

`#include <scmi.h>`

7.164.1 Detailed Description

struct scmi_sensor_reading_in - Payload for SCMI_SENSOR_READING_GET command

7.165 scmi_sensor_reading_out Struct Reference

`#include <scmi.h>`

7.165.1 Detailed Description

struct scmi_sensor_reading_out - Response payload for SCMI_SENSOR_READING_GET command

7.166 scmi_sensor_trip_point_config_in Struct Reference

`#include <scmi.h>`

7.166.1 Detailed Description

struct scmi_sensor_trip_point_config_in - Payload for SCMI_SENSOR_TRIP_POINT_CONFIG command

7.167 scmi_sensor_trip_point_config_out Struct Reference

`#include <scmi.h>`

7.167.1 Detailed Description

struct scmi_sensor_trip_point_config_out - Response payload for SCMI_SENSOR_TRIP_POINT_CONFIG command

7.168 scmi_sensor_trip_point_notify_in Struct Reference

`#include <scmi.h>`

7.168.1 Detailed Description

struct scmi_sensor_trip_point_notify_in - Payload for SCMI_SENSOR_TRIP_POINT_NOTIFY command

7.169 scmi_sensor_trip_point_notify_out Struct Reference

`#include <scmi.h>`

7.169.1 Detailed Description

struct scmi_sensor_trip_point_notify_out - Response payload for SCMI_SENSOR_TRIP_POINT_NOTIFY command

7.170 scmi_voltage_attr_in Struct Reference

`#include <scmi.h>`

7.170.1 Detailed Description

struct scmi_voltage_attr_in - Payload for SCMI_VOLTAGE_DOMAIN_ATTRIBUTES message

7.171 scmi_voltage_attr_out Struct Reference

`#include <scmi.h>`

7.171.1 Detailed Description

struct scmi_voltage_attr_out - Payload for SCMI_VOLTAGE_DOMAIN_ATTRIBUTES response

7.172 scmi_voltage_attributes Struct Reference

`#include <scmi.h>`

7.172.1 Detailed Description

struct scmi_voltage_attributes - Response payload for SCMI_COM_MSG_ATTRIBUTES command

7.173 scmi_voltage_config_get_in Struct Reference

`#include <scmi.h>`

7.173.1 Detailed Description

struct scmi_voltage_config_get_in - Message payload for SCMI_VOLTAGE_CONFIG_GET cmd

7.174 scmi_voltage_config_get_out Struct Reference

`#include <scmi.h>`

7.174.1 Detailed Description

struct scmi_voltage_config_get_out - Response for SCMI_VOLTAGE_CONFIG_GET command

7.175 scmi_voltage_config_set_in Struct Reference

`#include <scmi.h>`

7.175.1 Detailed Description

struct scmi_voltage_config_set_in - Message payload for SCMI_VOLTAGE_DOMAIN_CONFIG_SET cmd

7.176 scmi_voltage_config_set_out Struct Reference

`#include <scmi.h>`

7.176.1 Detailed Description

struct scmi_voltage_config_set_out - Response for SCMI_VOLTAGE_DOMAIN_CONFIG_SET command

7.177 scmi_voltage_level_get_in Struct Reference

`#include <scmi.h>`

7.177.1 Detailed Description

struct scmi_voltage_level_get_in - Message payload for SCMI_VOLTAGE_DOMAIN_LEVEL_GET cmd

7.178 scmi_voltage_level_get_out Struct Reference

`#include <scmi.h>`

7.178.1 Detailed Description

struct scmi_voltage_level_get_out - Response for SCMI_VOLTAGE_DOMAIN_LEVEL_GET command

7.179 scmi_voltage_level_set_in Struct Reference

`#include <scmi.h>`

7.179.1 Detailed Description

struct scmi_voltage_level_set_in - Message payload for SCMI_VOLTAGE_DOMAIN_LEVEL_SET cmd

7.180 scmi_voltage_level_set_out Struct Reference

`#include <scmi.h>`

7.180.1 Detailed Description

struct scmi_voltage_level_set_out - Response for SCMI_VOLTAGE_DOMAIN_LEVEL_SET command

Chapter 8

File Documentation

8.1 C:/Renesas/rt-thread/components/drivers/include/drivers/adc_v2.h File Reference

```
#include <rtconfig.h>
#include "misc.h"
#include <rtatomic.h>
#include <rtdevice.h>
#include <rtthread.h>
```

Include dependency graph for adc_v2.h:

Data Structures

- struct rt_adc_core_ops
- struct rt_adc_sequence_ops
- struct rt_adc_ops
- struct rt_adc_sequence_cfg
- struct rt_adc_session_ctrl
- struct rt_adc_device

Macros

- #define RT_ADC_CHANNEL_MASK(_channel)

Enumerations

- enum rt_adc_control_cmd { RT_ADC_CMD_GET_RESOLUTION = RT_DEVICE_CTRL_BASE(ADC) + 1 , RT_ADC_CMD_CALIBRATE , RT_ADC_CMD_SET_VREF , RT_ADC_CMD_CALC_VREF , RT_ADC_CMD_GET_VREF_CHANNEL , RT_ADC_CMD_SET_SESSION , RT_ADC_CMD_SET_CONFIG , RT_ADC_CMD_GET_CONFIG , RT_ADC_CMD_CLEAR_CHANNEL_CONFIG }
- enum rt_adc_input_mode { RT_ADC_INPUT_MODE_SINGLE_ENDED = 0x01U , RT_ADC_INPUT_MODE_DIFFERENTIAL = 0x02U }
- enum rt_adc_state { RT_ADC_STATE_IDLE = 0 , RT_ADC_STATE_LOCKED , RT_ADC_STATE_STREAM }

Functions

- rt_inline rt_size_t rt_adc_channel_mask_count (rt_uint32_t channels)
- rt_inline rt_uint8_t rt_adc_channel_mask_take_lsb (rt_uint32_t *channels)
- rt_err_t rt_hw_adc_register (rt_adc_device_t device, const char *name, const struct rt_adc_ops *ops, const void *user_data)
- rt_err_t rt_adc_session_channel_index (rt_uint32_t session_channels, rt_uint32_t channel, rt_size_t *index)

- rt_err_t rt_adc_read_sequence (rt_adc_device_t device, rt_uint32_t channels, const struct rt_adc_sequence_cfg *cfg, rt_size_t *read_count)
- rt_err_t rt_adc_raw_to_voltage_mv (rt_uint32_t raw_value, rt_uint32_t vref_mv, rt_uint8_t resolution_bits, rt_uint32_t *voltage_mv)
- rt_err_t rt_adc_raw_to_scaled_voltage_mv (rt_uint32_t raw_value, rt_uint32_t vref_mv, rt_uint8_t resolution_bits, rt_uint32_t scale_num, rt_uint32_t scale_den, rt_uint32_t *voltage_mv)
- rt_err_t rt_adc_voltage (rt_adc_device_t device, rt_uint32_t *voltages_mv, rt_size_t size, rt_int32_t timeout_ms)

8.1.1 Detailed Description

Standalone ADC V2 framework declarations.

8.1.2 Macro Definition Documentation

8.1.2.1 RT_ADC_CHANNEL_MASK

```
#define RT_ADC_CHANNEL_MASK(
            _channel)
```

Value:

```
(1UL << (rt_uint32_t)(_channel))
```

Build an ADC channel mask from one logical channel index.

Parameters

_channel	Logical ADC channel index.

8.1.3 Enumeration Type Documentation

8.1.3.1 rt_adc_control_cmd

```
enum rt_adc_control_cmd
```

ADC control command identifiers.

Enumerator

RT_ADC_CMD_GET_RESOLUTION	Get resolution in bits.
RT_ADC_CMD_CALIBRATE	Run ADC calibration.
RT_ADC_CMD_SET_VREF	Set fallback VREF from rt_uint32_t mV.
RT_ADC_CMD_CALC_VREF	Calculate VDDA from a raw VREF sample.
RT_ADC_CMD_GET_VREF_CHANNEL	Get the backend VREF sample channel as rt_uint8_t.
RT_ADC_CMD_SET_SESSION	Configure and cache conversion channels from rt_uint32_t mask.
RT_ADC_CMD_SET_CONFIG	Set backend private device configuration.
RT_ADC_CMD_GET_CONFIG	Get backend private device configuration.
RT_ADC_CMD_CLEAR_CHANNEL_CONFIG	Clear backend channel/rank configuration explicitly.

8.1.3.2 rt_adc_input_mode

```
enum rt_adc_input_mode
```

ADC input mode request flags.

Enumerator

RT_ADC_INPUT_MODE_SINGLE_ENDED	Single-ended ADC input mode.
RT_ADC_INPUT_MODE_DIFFERENTIAL	Differential ADC input mode.

8.1.3.3 rt_adc_state

```
enum rt_adc_state
```

ADC framework runtime state.

Enumerator

RT_ADC_STATE_IDLE	The ADC device is idle.
RT_ADC_STATE_LOCKED	The ADC device is temporarily locked by a framework operation.
RT_ADC_STATE_STREAM	The ADC device is owned by an active stream session.

8.1.4 Function Documentation

8.1.4.1 rt_adc_channel_mask_count()

```
rt_inline rt_size_t rt_adc_channel_mask_count (
            rt_uint32_t channels)
```

Count selected ADC channels in one channel mask.

Parameters

channels	ADC channel selection mask.

Returns

Number of selected channels.

8.1.4.2 rt_adc_channel_mask_take_lsb()

```
rt_inline rt_uint8_t rt_adc_channel_mask_take_lsb (
            rt_uint32_t * channels)
```

Take the lowest selected ADC channel from one channel mask.

Parameters

channels	Pointer to the ADC channel selection mask.

Returns

Selected channel index, or 0xff if the mask is empty.

8.1.4.3 rt_hw_adc_register()

```
rt_err_t rt_hw_adc_register (
            rt_adc_device_t device,
            const char * name,
            const struct rt_adc_ops * ops,
            const void * user_data)
```

Register an ADC device with the RT-Thread device framework.

Parameters

device	Pointer to the ADC device object.
name	Pointer to the ADC device name string.
ops	Pointer to the ADC driver operation table.
user_data	Pointer to driver private data.

Returns

Operation status.

8.1.4.4 rt_adc_session_channel_index()

```
rt_err_t rt_adc_session_channel_index (
             rt_uint32_t session_channels,
             rt_uint32_t channel,
             rt_size_t * index)
```

Get the raw sample index of one channel in an ADC session mask.

Parameters

session_channels	Configured ADC session channel mask.
channel	Single-channel mask to locate.
index	Pointer to the output raw sample index.

Returns

Operation status.

8.1.4.5 rt_adc_read_sequence()

```
rt_err_t rt_adc_read_sequence (
             rt_adc_device_t device,
             rt_uint32_t channels,
             const struct rt_adc_sequence_cfg * cfg,
             rt_size_t * read_count)
```

Read one ADC conversion sequence frame.

Parameters

device	Pointer to the ADC device object.
channels	ADC channel selection mask.
cfg	Pointer to the ADC sequence request object.
read_count	Pointer to the stored sample count.

Returns

Operation status.

8.1.4.6 rt_adc_raw_to_voltage_mv()

```
rt_err_t rt_adc_raw_to_voltage_mv (
             rt_uint32_t raw_value,
             rt_uint32_t vref_mv,
             rt_uint8_t resolution_bits,
             rt_uint32_t * voltage_mv)
```

Convert one raw ADC sample to millivolts.

Parameters

raw_value	Raw ADC sample value.
vref_mv	ADC reference voltage in millivolts.
resolution_bits	ADC resolution in bits.
voltage_mv	Pointer to the output voltage in millivolts.

Returns

Operation status.

8.1.4.7 rt_adc_raw_to_scaled_voltage_mv()

```
rt_err_t rt_adc_raw_to_scaled_voltage_mv (
             rt_uint32_t raw_value,
             rt_uint32_t vref_mv,
             rt_uint8_t resolution_bits,
             rt_uint32_t scale_num,
             rt_uint32_t scale_den,
             rt_uint32_t * voltage_mv)
```

Convert one ADC raw sample to a scaled voltage in millivolts.

Parameters

raw_value	Raw ADC sample code.
vref_mv	ADC reference voltage in millivolts.
resolution_bits	ADC resolution in bits.
scale_num	Voltage scaling numerator.
scale_den	Voltage scaling denominator.
voltage_mv	Pointer to the output scaled voltage in millivolts.

Returns

Operation status.

The scaled voltage is calculated as:

```
raw_to_voltage_mv(raw_value) * scale_num / scale_den
```

Note

`scale_num` and `scale_den` may describe any positive rational scaling ratio that does not overflow the internal calculation. The final scaled millivolt value must fit in rt_uint32_t.

Return values

RT_EOK	The conversion completed successfully.
-RT_EINVAL	A pointer is null, a scale argument is zero, the ADC conversion arguments are invalid, or the scaled result overflows rt_uint32_t.

8.1.4.8 rt_adc_voltage()

```
rt_err_t rt_adc_voltage (
            rt_adc_device_t device,
            rt_uint32_t * voltages_mv,
            rt_size_t size,
            rt_int32_t timeout_ms)
```

Read ADC samples and convert them to millivolts.

Parameters

device	Pointer to the ADC device object.
voltages_mv	Pointer to the output voltage buffer in millivolts.
size	Output voltage buffer length.
timeout_ms	Per-sample timeout in milliseconds; negative means wait forever.

Returns

Operation status.

Note

This API reads the currently configured ADC session and does not modify the active channel mask. If default↩ _vref_mv is zero, callers that need backend-calculated VDDA must include the backend VREF channel in the configured session. The VREF sample is used only as the reference for conversion and is not auto-inserted by this API.

8.2 C:/Renesas/rt-thread/components/drivers/include/drivers/adc_v2_↩ trigger.h File Reference

```
#include <rtconfig.h>
#include <rtthread.h>
```

Include dependency graph for adc_v2_trigger.h:

8.2.1 Detailed Description

ADC V2 trigger framework declarations.

8.3 C:/Renesas/rt-thread/components/drivers/include/drivers/dma.h File Reference

```
#include <rtthread.h>
#include <drivers/ofw.h>
```

```
#include <drivers/misc.h>
#include <drivers/core/dm.h>
#include <mmu.h>
#include <mm_page.h>
#include <bitmap.h>
```

Include dependency graph for dma.h:

Data Structures

- struct rt_dma_slave_config
- struct rt_dma_slave_transfer
- struct rt_dma_controller
- struct rt_dma_controller_ops
- struct rt_dma_chan
- struct rt_dma_pool
- struct rt_dma_map_ops

Macros

- #define RT_DMA_F_LINEAR RT_BIT(0)
- #define RT_DMA_F_32BITS RT_BIT(1)
- #define RT_DMA_F_NOCACHE RT_BIT(2)
- #define RT_DMA_F_DEVICE RT_BIT(3)
- #define RT_DMA_F_NOMAP RT_BIT(4)
- #define RT_DMA_F_WT RT_BIT(5)
- #define RT_DMA_PAGE_SIZE ARCH_PAGE_SIZE

Enumerations

- enum rt_dma_transfer_direction {
  RT_DMA_MEM_TO_MEM , RT_DMA_MEM_TO_DEV , RT_DMA_DEV_TO_MEM , RT_DMA_DEV_TO_DEV
  ,
  RT_DMA_DIR_MAX }
- enum rt_dma_slave_buswidth {
  RT_DMA_SLAVE_BUSWIDTH_UNDEFINED = 0 , RT_DMA_SLAVE_BUSWIDTH_1_BYTE = 1 ,
  RT_DMA_SLAVE_BUSWIDTH_2_BYTES = 2 , RT_DMA_SLAVE_BUSWIDTH_3_BYTES = 3 ,
  RT_DMA_SLAVE_BUSWIDTH_4_BYTES = 4 , RT_DMA_SLAVE_BUSWIDTH_8_BYTES = 8 , RT_DMA_SLAVE_BUSWIDTH_
  = 16 , RT_DMA_SLAVE_BUSWIDTH_32_BYTES = 32 ,
  RT_DMA_SLAVE_BUSWIDTH_64_BYTES = 64 , RT_DMA_SLAVE_BUSWIDTH_128_BYTES = 128 ,
  RT_DMA_SLAVE_BUSWIDTH_BYTES_MAX }

Functions

- rt_inline void rt_dma_controller_add_direction (struct rt_dma_controller *ctrl, enum rt_dma_transfer_direction dir)
- rt_inline void rt_dma_controller_set_addr_mask (struct rt_dma_controller *ctrl, rt_uint64_t mask)
- rt_err_t rt_dma_controller_register (struct rt_dma_controller *ctrl)
- rt_err_t rt_dma_controller_unregister (struct rt_dma_controller *ctrl)
- rt_err_t rt_dma_chan_start (struct rt_dma_chan *chan)
- rt_err_t rt_dma_chan_pause (struct rt_dma_chan *chan)
- rt_err_t rt_dma_chan_stop (struct rt_dma_chan *chan)
- rt_err_t rt_dma_chan_config (struct rt_dma_chan *chan, struct rt_dma_slave_config *conf)
- rt_err_t rt_dma_chan_done (struct rt_dma_chan *chan, rt_size_t size)
- rt_err_t rt_dma_prep_memcpy (struct rt_dma_chan *chan, struct rt_dma_slave_transfer *transfer)
- rt_err_t rt_dma_prep_cyclic (struct rt_dma_chan *chan, struct rt_dma_slave_transfer *transfer)
- rt_err_t rt_dma_prep_single (struct rt_dma_chan *chan, struct rt_dma_slave_transfer *transfer)
- struct rt_dma_chan * rt_dma_chan_request (struct rt_device *dev, const char *name)

- rt_err_t rt_dma_chan_release (struct rt_dma_chan *chan)
- void * rt_dma_alloc (struct rt_device *dev, rt_size_t size, rt_ubase_t *dma_handle, rt_ubase_t flags)
- void rt_dma_free (struct rt_device *dev, rt_size_t size, void *cpu_addr, rt_ubase_t dma_handle, rt_ubase_t flags)
- rt_inline void * rt_dma_alloc_coherent (struct rt_device *dev, rt_size_t size, rt_ubase_t *dma_handle)
- rt_inline void rt_dma_free_coherent (struct rt_device *dev, rt_size_t size, void *cpu_addr, rt_ubase_t dma↩ _handle)
- rt_err_t rt_dma_sync_out_data (struct rt_device *dev, void *data, rt_size_t size, rt_ubase_t *dma_handle, rt_ubase_t flags)
- rt_err_t rt_dma_sync_in_data (struct rt_device *dev, void *out_data, rt_size_t size, rt_ubase_t dma_handle, rt_ubase_t flags)
- rt_inline rt_bool_t rt_dma_device_is_coherent (struct rt_device *dev)
- rt_inline void rt_dma_device_set_ops (struct rt_device *dev, const struct rt_dma_map_ops *ops)
- struct rt_dma_pool * rt_dma_pool_install (rt_region_t *region)
- rt_err_t rt_dma_pool_extract (rt_size_t cma_size, rt_size_t coherent_pool_size)

8.3.1 Detailed Description

DMA (Direct Memory Access) controller framework public API.
Defines the DMA controller, channel, memory pool, and mapping operations interfaces. Provides APIs for controller registration, channel lifecycle management, transfer preparation (memcpy, cyclic, single), DMA buffer allocation with cache coherence management, and device tree integration.

8.3.2 Macro Definition Documentation

8.3.2.1 RT_DMA_F_LINEAR

```
#define RT_DMA_F_LINEAR RT_BIT(0)
```

DMA pool: linear address space (all DMA pools have this flag).

8.3.2.2 RT_DMA_F_32BITS

```
#define RT_DMA_F_32BITS RT_BIT(1)
```

DMA pool: limited to 32-bit addressable memory (below 4GB).

8.3.2.3 RT_DMA_F_NOCACHE

```
#define RT_DMA_F_NOCACHE RT_BIT(2)
```

DMA allocation: use non-cacheable memory mapping.

8.3.2.4 RT_DMA_F_DEVICE

```
#define RT_DMA_F_DEVICE RT_BIT(3)
```

DMA pool: device-private pool (matches pool->dev).

8.3.2.5 RT_DMA_F_NOMAP

```
#define RT_DMA_F_NOMAP RT_BIT(4)
```

DMA allocation: do not map virtual address, return physical.

8.3.2.6 RT_DMA_F_WT

```
#define RT_DMA_F_WT RT_BIT(5)
```

DMA allocation: use write-through cache mapping.

8.3.2.7 RT_DMA_PAGE_SIZE

```
#define RT_DMA_PAGE_SIZE ARCH_PAGE_SIZE
```

DMA pool page size matches architecture page size.

8.3.3 Enumeration Type Documentation

8.3.3.1 rt_dma_transfer_direction

```
enum rt_dma_transfer_direction
```

DMA transfer direction.

Defines the data flow direction for a DMA transfer. The controller's dir_cap bitmap indicates which directions are supported.

Enumerator

RT_DMA_MEM_TO_MEM	Memory-to-memory copy (no peripheral involved)
RT_DMA_MEM_TO_DEV	Memory to peripheral (TX / playback)
RT_DMA_DEV_TO_MEM	Peripheral to memory (RX / capture)
RT_DMA_DEV_TO_DEV	Device-to-device (rare, may not be supported)
RT_DMA_DIR_MAX	Sentinel: total number of direction types

8.3.3.2 rt_dma_slave_buswidth

```
enum rt_dma_slave_buswidth
```

DMA slave bus width options.

Specifies the transfer width for source and destination on the DMA bus. Must match the peripheral's FIFO width for correct operation.

Enumerator

RT_DMA_SLAVE_BUSWIDTH_UNDEFINED	Undefined / not configured
RT_DMA_SLAVE_BUSWIDTH_1_BYTE	8-bit transfers
RT_DMA_SLAVE_BUSWIDTH_2_BYTES	16-bit transfers
RT_DMA_SLAVE_BUSWIDTH_3_BYTES	24-bit transfers
RT_DMA_SLAVE_BUSWIDTH_4_BYTES	32-bit transfers
RT_DMA_SLAVE_BUSWIDTH_8_BYTES	64-bit transfers
RT_DMA_SLAVE_BUSWIDTH_16_BYTES	128-bit transfers
RT_DMA_SLAVE_BUSWIDTH_32_BYTES	256-bit transfers
RT_DMA_SLAVE_BUSWIDTH_64_BYTES	512-bit transfers
RT_DMA_SLAVE_BUSWIDTH_128_BYTES	1024-bit transfers
RT_DMA_SLAVE_BUSWIDTH_BYTES_MAX	Sentinel: maximum bus width enum value

8.3.4 Function Documentation

8.3.4.1 rt_dma_controller_add_direction()

```
rt_inline void rt_dma_controller_add_direction (
            struct rt_dma_controller * ctrl,
            enum rt_dma_transfer_direction dir)
```

Add a supported transfer direction to a DMA controller.

Controllers call this during probe to declare which directions they support.

Parameters

in	*ctrl*	DMA controller

in	*dir*	Transfer direction to add

8.3.4.2 rt_dma_controller_set_addr_mask()

```
rt_inline void rt_dma_controller_set_addr_mask (
            struct rt_dma_controller * ctrl,
            rt_uint64_t mask)
```

Set the address mask for a DMA controller.
The mask defines the valid address range for DMA transfers. For example, RT_DMA_ADDR_MASK(32) limits transfers to the lower 4GB.

Parameters

in	*ctrl*	DMA controller
in	*mask*	Address bit mask

8.3.4.3 rt_dma_controller_register()

```
rt_err_t rt_dma_controller_register (
            struct rt_dma_controller * ctrl)
```

Register a DMA controller with the framework.

8.3.4.4 rt_dma_controller_unregister()

```
rt_err_t rt_dma_controller_unregister (
            struct rt_dma_controller * ctrl)
```

Unregister a DMA controller (fails if channels are in use).

8.3.4.5 rt_dma_chan_start()

```
rt_err_t rt_dma_chan_start (
            struct rt_dma_chan * chan)
```

Start a prepared DMA transfer on a channel.

8.3.4.6 rt_dma_chan_pause()

```
rt_err_t rt_dma_chan_pause (
            struct rt_dma_chan * chan)
```

Pause a running DMA transfer (falls back to stop if not supported).

8.3.4.7 rt_dma_chan_stop()

```
rt_err_t rt_dma_chan_stop (
            struct rt_dma_chan * chan)
```

Stop a running DMA transfer.

8.3.4.8 rt_dma_chan_config()

```
rt_err_t rt_dma_chan_config (
            struct rt_dma_chan * chan,
            struct rt_dma_slave_config * conf)
```

Configure a DMA channel with slave-specific parameters.

8.3.4.9 rt_dma_chan_done()

```
rt_err_t rt_dma_chan_done (
            struct rt_dma_chan * chan,
            rt_size_t size)
```

Signal transfer completion (called by controller ISR).

8.3.4.10 rt_dma_prep_memcpy()

```
rt_err_t rt_dma_prep_memcpy (
            struct rt_dma_chan * chan,
            struct rt_dma_slave_transfer * transfer)
```

Prepare a memory-to-memory DMA transfer.

8.3.4.11 rt_dma_prep_cyclic()

```
rt_err_t rt_dma_prep_cyclic (
            struct rt_dma_chan * chan,
            struct rt_dma_slave_transfer * transfer)
```

Prepare a cyclic (repeating) DMA transfer.

8.3.4.12 rt_dma_prep_single()

```
rt_err_t rt_dma_prep_single (
            struct rt_dma_chan * chan,
            struct rt_dma_slave_transfer * transfer)
```

Prepare a single (one-shot) DMA transfer.

8.3.4.13 rt_dma_chan_request()

```
struct rt_dma_chan * rt_dma_chan_request (
            struct rt_device * dev,
            const char * name)
```

Request a DMA channel for a slave device (by DT name or auto).

8.3.4.14 rt_dma_chan_release()

```
rt_err_t rt_dma_chan_release (
            struct rt_dma_chan * chan)
```

Release a DMA channel back to the controller.

8.3.4.15 rt_dma_alloc()

```
void * rt_dma_alloc (
            struct rt_device * dev,
            rt_size_t size,
            rt_ubase_t * dma_handle,
            rt_ubase_t flags)
```

Allocate a DMA-capable memory buffer.

8.3.4.16 rt_dma_free()

```
void rt_dma_free (
            struct rt_device * dev,
            rt_size_t size,
            void * cpu_addr,
            rt_ubase_t dma_handle,
            rt_ubase_t flags)
```

Free a DMA buffer previously allocated with rt_dma_alloc().

8.3.4.17 rt_dma_alloc_coherent()

```
rt_inline void * rt_dma_alloc_coherent (
            struct rt_device * dev,
            rt_size_t size,
            rt_ubase_t * dma_handle)
```

Allocate a coherent (non-cacheable) DMA buffer.
Convenience wrapper around rt_dma_alloc() with RT_DMA_F_NOCACHE | RT_DMA_F_LINEAR.

Parameters

in	*dev*	DMA-capable device
in	*size*	Requested size in bytes
out	*dma_handle*	Physical DMA address

Returns

Kernel virtual address, or RT_NULL on failure

8.3.4.18 rt_dma_free_coherent()

```
rt_inline void rt_dma_free_coherent (
            struct rt_device * dev,
            rt_size_t size,
            void * cpu_addr,
            rt_ubase_t dma_handle)
```

Free a coherent DMA buffer.

Parameters

in	*dev*	DMA-capable device
in	*size*	Buffer size in bytes
in	*cpu_addr*	Kernel virtual address
in	*dma_handle*	Physical DMA address

8.3.4.19 rt_dma_sync_out_data()

```
rt_err_t rt_dma_sync_out_data (
            struct rt_device * dev,
            void * data,
            rt_size_t size,
            rt_ubase_t * dma_handle,
            rt_ubase_t flags)
```

Synchronize CPU cache for DMA output (CPU → device).

8.3.4.20 rt_dma_sync_in_data()

```
rt_err_t rt_dma_sync_in_data (
            struct rt_device * dev,
            void * out_data,
            rt_size_t size,
            rt_ubase_t dma_handle,
            rt_ubase_t flags)
```

Synchronize CPU cache for DMA input (device → CPU).

8.3.4.21 rt_dma_device_is_coherent()

```
rt_inline rt_bool_t rt_dma_device_is_coherent (
            struct rt_device * dev)
```

Check if a device is DMA-coherent.
Reads the "dma-coherent" boolean property from the device's device tree node (if present).

Parameters

in	*dev*	Device to check

Returns

RT_TRUE if the device is DMA-coherent

8.3.4.22 rt_dma_device_set_ops()

```
rt_inline void rt_dma_device_set_ops (
            struct rt_device * dev,
            const struct rt_dma_map_ops * ops)
```

Override the DMA mapping operations for a device.

Parameters

in	*dev*	Device to configure
in	*ops*	Custom DMA map operations (or NULL to reset)

8.3.4.23 rt_dma_pool_install()

```
struct rt_dma_pool * rt_dma_pool_install (
            rt_region_t * region)
```

Install a DMA memory pool from a region descriptor.

8.3.4.24 rt_dma_pool_extract()

```
rt_err_t rt_dma_pool_extract (
            rt_size_t cma_size,
            rt_size_t coherent_pool_size)
```

Extract coherent pool and CMA from memblock reserved memory.